

从 C++ 到 计算系统：第二册

硬件原理、多核并行、高性能计算、同步、无锁结构与
分布式计算

CPU Performance Study

本册接续第一册的程序执行基础，系统讲解现代 CPU 硬件、多核心并发、高性能计算、锁与无锁数据结构、并行算法、运行时和分布式计算。AI 模型、算子开发和推理引擎放入第三册。

前言

第一册已经把一个 C++ 程序放到单机上讲清楚：源码怎样变成二进制，进程怎样获得虚拟地址空间，函数调用怎样落到 ABI 和栈帧，汇编指令怎样改变寄存器和内存，Linux 工具怎样帮助我们确认程序确实按某种方式执行。第二册从这里继续向下、向外、向复杂处推进。它不再重复第一册的基础，而是追问同一个程序进入现代计算环境后会发生什么：一个核心为什么有时很快、有时被分支和依赖拖住；一组核心为什么会因为 cache line、TLB、NUMA 和一致性协议互相影响；一个线程池为什么可能让 ready 的任务无人执行；一次写文件为什么不能简单等同于结果已经生效；一个跨机器请求为什么在超时后仍然可能已经执行。

本册的任务不是把所有 C++ 工程代码塞进正文，而是建立读懂、设计和审查计算系统所需的因果模型。正文使用伪代码、状态表、实验矩阵和关键代码片段解释机制；完整实现、构建脚本、测试、性能报告和故障注入由配套实践材料承接。

因此，本册不会把高性能计算写成技巧清单，也不会把并发写成 API 目录。流水线、乱序执行、分支预测、cache、TLB、NUMA、锁、原子、无锁结构、任务运行时、异步 I/O 和分布式协议，都必须从失败中长出来。朴素程序在小输入上正确，输入变大后热路径可能被字节搬运限制；线程数增加后，全局计数器可能把所有核心拖到同一条 cache line 上；无界队列在短 benchmark 中看起来吞吐很好，长时间运行却会把内存吃光；worker 写出临时文件后崩溃，目录里有文件并不能证明结果已经提交；请求超时后重试，旧响应迟到时不能再次生效。只有把这些反例讲清楚，机制才不是名词。

核心思想

第二册的核心问题是：在真实硬件、操作系统和网络边界下，怎样组织数据、执行、同步、任务、I/O 和通信，使计算结果正确、过程可观察、性能可解释、失败可恢复。

AI 模型、张量、算子开发和本地推理引擎属于第三册。第二册仍然会讨论矩阵、stencil、图遍历和向量化内核，因为这些是现代 CPU 计算的基本形状；但它不会把重心提前转向 AI 推理框架。

完成本册后，面对新的系统问题，应当能够重新建立模型：先定义正确性和 reference，再画出数据路径和地址路径，然后判断硬件等待、同步等待、I/O 等待和网络等待分别在哪里，最后用实验、trace、源码入口和工程报告证明自己的判断。真正的掌握不是背熟所有名词，而是在新机器、新输入和新故障下仍然能推导、测量和取舍。

怎样阅读本册

本册默认已经具备第一册的最低基础：能编译 C++ 程序，能确认二进制产物，能读基础 x86-64 汇编，能解释调用约定和栈帧，能写隔离热路径的 benchmark，能用 Linux 工具观察进程、地址空间和基础性能计数。第二册不会在每个章节重新讲这些背景；它会把这些能力当作工具，用来解释多核硬件、同步、运行时、I/O 和分布式系统中的新成本。

本册最好按顺序阅读。第一部分建立硬件执行模型：核心内部怎样供给、调度和提交工作，cache、TLB、page cache、NUMA 和一致性协议怎样改变同一段程序的成本。第二部分进入单机高性能计算：怎样把“程序慢”拆成可测假设，怎样从数据布局、局部性、SIMD、指令级并行和内核形状推导优化方向。第三部分讲共享内存中的正确性：线程、锁、条件变量、原子操作、内存序和无锁结构都围绕对象生命周期、可见性、线性化点和等待谓词展开。第四部分把多个线程组织成并行算法、任务图、运行时和异步 I/O。第五部分把同一套身份、提交、恢复和背压原则扩展到消息、分片、复制、共识和分布式计算。

本册是原理卷，配套代码实践卷单独成册。原理卷使用三种代码形式。第一种是伪代码，用来表达状态机、协议和算法步骤，不绑定某个工程目录；第二种是关键片段，用来展示 C++ 语义边界，例如一个发布标志、一个等待谓词或一个内存布局；第三种是实验口径，用输入矩阵、指标和预期现象说明验证方式。完整 C++ 工程实现、单元测试、benchmark harness、报告格式、故障注入和逐章项目任务，放在代码实践卷中展开。

贯穿系统统一称为 **Compute Systems Engine**（计算系统引擎）。原理卷只保留它的概念演化：第一阶段定义日志统计的输入合同、顺序 reference 和错误策略；第二阶段观察热循环、地址路径和工作集；第三阶段加入 batch、冷热字段和典型计算内核；第四阶段加入线程、队列、同步和原子发布；第五阶段加入任务图、异步 I/O、manifest 和恢复；第六阶段加入消息、attempt、epoch、分片、复制和分布式提交。代码实践卷会把这些阶段落成完整实现。

和 MiniLLM-CPU 的连接

第二册的主案例仍然是 **Compute Systems Engine**，但它也要为第三册的 MiniLLM-CPU 铺机器和运行时能力。同一个 **linear** 或 **attention** 热路径，到了第二册就不再只是“能运行的函数”，而是一组现代计算系统问题：地址流是否连续，工作集落在哪级 cache，TLB 是否抖动，SIMD 是否吃满，线程是否产生 false sharing，NUMA 是否远端访问，队列是否制造尾延迟，取消和背压是否释放资源。

第二册机制	MiniLLM-CPU 中的对应问题	必须留下的证据
cache/TLB/NUMA	hidden、weight、KV cache 的地址流	字节数、工作集、页数、miss 或替代证据
SIMD/ILP	GEMV、RMSNorm、top-k 热循环	scalar/SIMD 对照、反汇编、吞吐曲线
线程和同步	output channel 切分、decode session 调度	扩展曲线、等待点、false sharing 检查
运行时队列	prefill/decode/stream event 队列	queue wait、p50/p95、取消回收时间
I/O 和提交	模型加载、报告写出、trace 持久化	manifest、原始 CSV、失败复现命令

因此第二册不是第三册的“背景阅读”，而是第三册性能和服务化章节的直接工具箱。读者学完第二册，应能拿一个张量 kernel 或一个请求调度器，先定义 reference，再估算字节数和算术强度，画地址路径和等待路径，设计实验矩阵，最后用 PMU、trace、CSV 或故障注入报告证明自己的判断。

心智模型

阅读每一章时始终追问五件事：结果怎样定义，数据怎样移动，谁拥有状态，等待在哪里发生，失败后谁是权威事实。

每章都要按“问题、朴素方案、失败、机制、证据、边界”阅读。若一节讲 cache，就不要只记 cache line 这个词，而要问哪个地址序列导致它生效或失效；若一节讲条件变量，就要问等待谓词是什么，谁改变谓词，谁负责唤醒，关闭路径怎样收束；若一节讲 manifest，就要问文件存在和业务生效为什么不是同一件事。真正学会一个概念，标志不是能复述定义，而是能解释它修复了哪个失败、引入了什么成本，以及在什么输入下不值得使用。

常见误区

不要把高性能计算理解成“把线程数开大”。线程数只是资源请求；性能来自数据复用、负载均衡、同步控制、I/O 边界和测量证据。没有这些证据，加线程常常只是更快地制造争用。

本册的习题和实验题不是附属装饰。实验至少保留三类笔记：第一，语义笔记，写清 reference、输入合同、错误策略和提交事实；第二，成本笔记，写清字节数、地址路径、工作集、线程数、队列水位和等待点；第三，证据笔记，保存命令、输入规模、平台、测量结果和无法验证的边界。代码实践卷中的完整实现应服务这些笔记，而不是替代它们。

学完第二册应能做什么

验收与评分标准

- 给定一个热循环，能区分它更可能受前端、后端、内存带宽、cache/TLB、分支、同步还是调度限制。
- 能为一个 kernel 写 roofline 风格账本：FLOPs、读写字节、算术强度、理论上限、实测差距和下一步证据。
- 能写并发正确性报告：共享状态、等待谓词、happens-before、线性化点、关闭路径和资源生命周期。
- 能设计线程池、队列、背压、取消和异步 I/O 的最小可测试版本，并用 p50/p95 与故障注入验证。
- 能把一次性能优化写成可复现报告，而不是只给“快了多少”的截图。

常见缩写和术语先导

第二册把程序放进现代硬件、并发运行时和分布式系统里解释。缩写和工程词很多，但它们都服务同一件事：把“慢、卡、错、丢、重复”变成可观察事实。

CPU/GPU/OS/ISA/API/ABI CPU 是中央处理器；GPU 是图形处理器或通用并行加速器；OS 是操作系统；ISA 是指令集架构；API 是源码层接口；ABI 是二进制接口。第二册会把同一段语义落到不同机器和调用边界上解释。

AMD Advanced Micro Devices，常见 CPU/GPU 厂商名。本书提到 AMD 时，多数是在说明平台、微架构或工具事件不能和 Intel/Arm 混为一谈。

C++20/MSVC C++20 是本书并发和同步示例使用的标准版本；MSVC 是 Microsoft Visual C++ 工具链。不同标准库和工具链的实现细节会影响源码阅读和实验结果。

CFG/STL/HPC CFG 是控制流图；STL 是 C++ 标准库常见统称；HPC 是高性能计算。第二册会把这些词放进机器证据和工程实现里。

ASCII/UTF-8 ASCII 是早期单字节字符集合；UTF-8 是 Unicode 的常见变长字节编码。第二册解析器和 SIMD 字节分类会把“字节合同”和“文本语义”分开。

SIMD/ILP SIMD 是一条指令处理多个数据；ILP 是 Instruction-Level Parallelism，指令级并行，让互不依赖的指令重叠执行。

AVX/AVX2/AVX-512 x86 常见向量扩展。AVX/AVX2 常见宽度是 256 位；AVX-512 提供更宽寄存器和 mask 机制，但要验证平台覆盖、频率和短输入成本。

VNNI/AMX/FMA VNNI 更适合低精度点积；AMX 提供 tile 级矩阵能力；FMA 是 fused multiply-add，一条指令完成乘加。

ARM/POWER/SSE/LTO/PGO/JIT ARM 和 POWER 是常见处理器架构生态；SSE 是较早的 x86 SIMD 扩展；LTO 是链接期优化；PGO 是 Profile-Guided Optimization，基于运行画像优化；JIT 是运行时即时编译或生成代码。

LLVM/GCC/SLP LLVM 和 GCC 是常见编译器生态；SLP 是 Superword-Level Parallelism，编译器把多条相似标量语句合成向量操作的一类向量化方式。

perf/objdump/strace/numactl 第二册常用实验工具。perf 采样和读取 PMU 计数器；objdump 看机器码和反汇编；strace 观察系统调用；numactl 控制或观察 NUMA 节点绑定。

PC/ALU/AGU/ROB/SIB/LEA PC 是 program counter，程序计数器；ALU 是算术逻辑单元；AGU 是地址生成单元；ROB 是重排序缓冲区；SIB 是 x86 地址编码字段；LEA 是地址计算指令。它们分别解释取指位置、算术、地址和乱序窗口资源。

MMU/TLB/ITLB/DTLB/PTE/LLC MMU 是内存管理单元；TLB 是地址翻译缓存；ITLB 缓存指令地址翻译，DTLB 缓存数据地址翻译；PTE 是页表项；LLC 是最后一级缓存。它们都是解释访存成本的核心证据来源。

L1/L2/L3/L1D 缓存层级名。L1 通常最小最快，L2 居中，L3 常作为 LLC；L1D 特指一级数据缓存。

BTB/RAS BTB 是 Branch Target Buffer，分支目标缓存；RAS 是 Return Address Stack，返回地址栈。它们帮助 CPU 猜下一条指令来自哪里。

CPI/p50/p95/p99 CPI 是每条指令平均周期数；p50、p95、p99 是分位数指标。并发性能不能只看平均值，p95/p99 往往暴露排队、迁核、缺页、锁竞争和 I/O 抖动。

MESI/MESIF/MOESI/HITM/LL/SC MESI、MESIF、MOESI 是缓存一致性状态

模型变体；HITM 表示命中被其他核心修改过的缓存行；LL/SC 是 load-linked/store-conditional 原子更新风格。

SRAM/RFO/THP/ULP SRAM 是常用于 cache 的静态随机访问存储器；RFO 是 Read For Ownership，写入前获取缓存行所有权；THP 是透明大页；ULP 是浮点末位单位。

DRAM/VM/VMA/VFS/ASLR DRAM 是主存；VM 是 virtual memory，虚拟内存；VMA 是进程虚拟内存区域；VFS 是 Linux 虚拟文件系统层；ASLR 是地址空间布局随机化。它们分别解释内存硬件、地址空间和文件系统抽象。

NUMA Non-Uniform Memory Access，非统一内存访问。线程和内存不在同一节点时，远端访问会增加延迟和带宽压力。

PMU/TMA/IPC/CSV/GFLOP PMU 是硬件性能计数器来源；TMA 是 Top-down Microarchitecture Analysis，一类把瓶颈分到前端、后端、错误推测和退休的分析方法；IPC 是每周指令数；CSV 是实验表格文本；GFLOP/s 是每秒十亿次浮点运算。

SMT/COW/RMW SMT 是同时多线程；COW 是写时复制；RMW 是读改写原子序列。并发性能经常卡在共享资源、共享页面或共享缓存行上。

TSO/TSAN/RAII TSO 是 x86 常见的 Total Store Order 内存模型；TSAN 是 ThreadSanitizer，数据竞争检测工具；RAII 是 C++ 用对象生命周期管理资源的惯用法。

ASan/UBSan/sanitizer ASan 查越界和 use-after-free；UBSan 查未定义行为；sanitizer 泛指这类插桩诊断工具。它们服务正确性，不代表发布性能。

CV/CFS/IRIW CV 是 condition variable，条件变量；CFS 是 Linux Completely Fair Scheduler；IRIW 是 Independent Reads of Independent Writes，内存模型经典 litmus test。

ISR Interrupt Service Routine，中断服务例程。它属于内核/硬件事件处理路径，实时系统和 I/O 讨论中会出现。

CAS Compare-And-Swap，比较并交换。它是很多无锁结构的基础原子操作。

ABA 无锁算法中的经典问题：一个位置从 A 变到 B 又变回 A，比较值相同但对象历史已经改变。

SPSC/MPSC/SPMC/MPMC SPSC 是单生产者单消费者；MPSC 是多生产者单消费者；SPMC 是单生产者多消费者；MPMC 是多生产者多消费者。队列边界不同，原子变量和进度证明也不同。

EOF/EAGAIN/EINTR/ETIMEDOUT EOF 表示输入结束；EAGAIN 表示稍后重试；EINTR 表示系统调用被信号中断；ETIMEDOUT 表示等待超时。并发和 I/O 状态机必须分别处理它们。

POSIX/epoll/SIGBUS POSIX 是 Unix-like 系统接口规范；epoll 是 Linux 常见 I/O 事件通知机制；SIGBUS 是总线错误信号，mmap 文件被截断或映射生命周期错误时常见。

EFD/EPOLLIN/EPOLLOUT EFD 常指出现在 eventfd flag 中的前缀；EPOLLIN 表示可读，EPOLLOUT 表示可写。它们都属于 Linux 事件驱动 I/O 的状态机输入。

EPOLLONESHOT/EPOLLRDHUP epoll 事件标志。EPOLLONESHOT 表示一次事件后需要重新 armed；EPOLLRDHUP 表示对端半关闭，连接状态机必须处理。

SQE/CQE/SQ/CQ io_uring 的队列项和队列。SQE 是 submission queue entry，提交请求；CQE 是 completion queue entry，完成结果；SQ/CQ 是提交队列和完成队列。

EPOLLET/EPOLLERR/EPOLLHUP/SO_ERROR EPOLLET 是边沿触发模式；EPOLLERR 表示错误事件；EPOLLHUP 表示挂起或关闭；SO_ERROR 用来读取 socket 上保存的异步错误。非阻塞 I/O 代码不能只处理“可读/可写”两个分支。

io_uring Linux 异步 I/O 接口。用户态把 SQE 放入提交队列，内核完成后写 CQE；

要分清“提交成功”“I/O 完成”和“业务提交”三件事。

C++ 并发对象 `shared_ptr` 是引用计数共享所有权；`lock_guard` 是作用域锁；`co_await` 是协程等待点。第二册看到它们时，要把语法还原成并发事实：谁拥有对象、谁持锁、谁可能暂停、谁负责恢复。

C 文件对象 `FILE` 是 C 标准库文件流类型。异步 I/O 章节出现 `FILE` 时，通常是为了和 `file descriptor`、`socket`、`mmap` 或 `io_uring` 的完成语义对比；它不是文件名。

FIFO/LIFO/PID/TID FIFO 是先进先出；LIFO 是后进先出；PID 是进程编号；TID 是线程编号。运行时和系统章节会用它们定位调度、线程和 I/O 对象。

RCU Read-Copy-Update。读路径尽量无锁，更新路径复制新版本并延迟回收旧版本。

SLAB/SLUB Linux 内核小对象分配器族，用对象缓存降低频繁分配释放成本。SLUB 是现代 Linux 常见实现。

TLS Thread-Local Storage，线程局部存储。它能减少共享，但也会引入初始化、析构和汇总成本。

DMA/RDMA/UPI DMA 让设备直接访问内存；RDMA 让远端机器的网卡直接读写内存区域；UPI 是 Intel 多 socket 互连之一，用于连接 CPU socket。

RPC Remote Procedure Call，远程过程调用。看起来像本地函数调用，本质上跨机器、跨网络和失败边界。

RTT Round-Trip Time，往返时间。分布式章节用它估算请求、复制和超时成本。

TCP/WAL/NFS TCP 是可靠字节流传输协议；WAL 是 Write-Ahead Log，预写日志；NFS 是网络文件系统。分布式章节用它们解释网络边界、存储语义和崩溃恢复边界。

DAG Directed Acyclic Graph，有向无环图。任务运行时常用 DAG 表达依赖关系。

BFS Breadth-First Search，广度优先搜索。第二册提到 BFS 时，多用于图计算 kernel 的 frontier 扩展，而不是面试模板。

CSR/GEMM/GEMV CSR 是压缩稀疏行格式；GEMM 是矩阵乘矩阵；GEMV 是矩阵乘向量。它们代表三类典型数据移动和计算形态。

CAP Consistency、Availability、Partition tolerance 的分布式系统权衡框架。它不是设计清单，而是提醒网络分区下不能同时保证所有语义。

Raft/Paxos/quorum Raft 和 Paxos 是共识协议族；quorum 是多数或法定票数集合。第二册用它们解释控制日志、leader、commit 和恢复边界。

leader term/fencing leader term 是领导者任期；fencing 是用 term、epoch 或 lease version 阻止旧持有者继续写外部事实。

SLO Service Level Objective，服务等级目标，例如 p95 延迟必须低于某个阈值。

P95/P99/RSS/GC/HDFS P95/P99 是 95/99 分位延迟；RSS 是常驻内存大小；GC 是垃圾回收；HDFS 是 Hadoop Distributed File System。它们常出现在服务和分布式系统指标里。

GB/GFLOP/FLOP GB 是十亿字节量级；FLOP 是浮点运算次数；GFLOP/s 是每秒十亿次浮点运算。Roofline 报告必须同时写字节和 FLOP 口径。

DWORD double word，常见于 x86/Intel 风格汇编文本，通常表示 32 位内存操作数。

KV Key/Value。第二册偶尔用 KV state 或 KV store 表示键值状态；第三册会把 KV cache 作为 Transformer 推理核心结构展开。

OOM Out Of Memory，内存不足。并发和运行时章节遇到 OOM 时，要追问谁拥有内存、谁释放、失败能否恢复。

MR/NR/LSM/BLIS/XNNPACK/LMAX MR/NR 是 GEMM microkernel 常见寄存器 tile 维度名；LSM 是日志结构合并树；BLIS 是高性能 BLAS 风格线性代数库；XNNPACK 是移动/边缘推理常见 CPU kernel 库；LMAX Disruptor 是序号环队列设计案例。本书提到它们时，是把工业项目当作源码阅读样本。

oneTBB Intel oneAPI Threading Building Blocks, 一套 C++ 并行运行时/算法库。第二册提到它时, 重点是把任务、work stealing、arena 和调度证据和自写运行时对照。

CFS/EEVDF Linux 调度器概念。CFS 是 Completely Fair Scheduler; EEVDF 是 newer Linux 中用于选择任务的虚拟截止期思想之一。读调度章节时, 先把它们理解成“内核如何决定下一个线程运行”的策略。

FUTEX/FUTEX_WAIT/FUTEX_WAKE futex 是 Linux 用户态锁和内核等待队列配合的基础接口。WAIT 负责睡眠等待某个内存值变化, WAKE 负责唤醒等待者; 高层 mutex/condition variable 常会落到这类机制。

实验常量 `SUM_LANE_COUNT`、`CACHE_LINE_BYTES`、`EVENT_BUCKET_COUNT`、`BUFFER_SLOT_CAPACITY` 这类名字是实验或数据结构参数。它们把向量 lane 数、缓存行大小、事件桶数和环槽容量写清, 便于复现实验。

CSV/JSON CSV 是表格文本格式, 适合保存实验数据; JSON 是结构化文本格式, 适合保存配置、manifest 或报告。

worker/task/future/continuation worker 是执行任务的线程或执行单元; task 是被调度的工作; future 是稍后可取得的结果; continuation 是前一步完成后继续执行的后续动作。

manifest/checkpoint/attempt/epoch manifest 是机器可读清单; checkpoint 是可恢复状态; attempt 是同一个逻辑任务的一次物理尝试; epoch 是版本或轮次。它们共同解决“重试以后到底提交了什么”的问题。

owner/lease/fencing/snapshot owner 是当前拥有资源或写权限的一方; lease 是有期限的租约; fencing 用版本 token 拦住旧 owner; snapshot 是某一时刻的状态副本。

partition/shard/replica/quorum partition 是把数据或任务切分; shard 是切分后的分片; replica 是副本; quorum 是足以提交事实的法定票数集合。

batch/chunk/bucket/frontier batch 是成批执行的一组输入; chunk 是连续切片; bucket 是按范围或哈希归类的桶; frontier 是图遍历当前要扩展的一层边界。

ring/head/tail/mask ring 是环形缓冲; head/tail 是读写边界; mask 常用于把递增序号映射到环大小内的位置。不要把这些词当变量名, 要看它们维护的队列不变量。

trace/profile/benchmark/baseline trace 是按时间记录事件; profile 是运行画像; benchmark 是受控性能测量; baseline 是后续优化要对比的基线。

kernel/backend/runtime/scheduler kernel 是热路径核心实现; backend 是落到具体硬件或系统接口的实现层; runtime 是负责调度、内存和同步的运行时; scheduler 是决定谁先执行、在哪里执行的策略。

layout/padding/stride/tile/lane layout 是内存排列; padding 是填充; stride 是地址步幅; tile 是块化计算单元; lane 是 SIMD 向量中的元素槽。

backpressure/latency/throughput/bandwidth backpressure 是下游处理不过来时让上游减速或失败; latency 是延迟; throughput 是吞吐; bandwidth 是单位时间数据搬运量。

block/stage/frame/pipeline block 是一块数据、任务或代码基本块, 具体含义看上下文; stage 是流水线阶段; frame 是栈帧、消息帧或数据帧; pipeline 是多个阶段串成的处理链。

reference/oracle/checksum/hash reference 是定义正确结果的清晰实现; oracle 是判定输出是否正确的比较器; checksum 是结果摘要; hash 是把对象映射成摘要或桶位置的函数结果。

generation/watermark/metadata/schema/config generation 是版本号; watermark 是“已经安全推进到哪里”的水位; metadata 是描述数据的数据; schema 是结构规则; config 是配置。

register/stack/frame/exception/syscall register 是寄存器；stack 是调用栈或栈式数据结构；frame 是一次调用或一帧数据；exception 是异常；syscall 是用户态进入内核的系统调用。

affinity/arena/span/fallback/handler affinity 是线程、内存或中断绑定到特定 CPU/节点的亲和性；arena 是一块预规划内存；span 是非拥有连续视图；fallback 是保底路径；handler 是处理函数。

frontend/diagnostic/descriptor/target/pass frontend 是输入导入和校验层；diagnostic 是结构化错误事实；descriptor 是描述对象；target 是构建目标或优化目标；pass 是分析或改写步骤。

IOMMU/MLP IOMMU 是设备 DMA 访问内存时的地址翻译和隔离单元；MLP 在硬件性能语境常指 Memory-Level Parallelism，内存级并行，不是第三册 Transformer 里的多层感知机。

stream/prefill/tensor/replication/prefetch stream 是异步事件流或执行队列；prefill 是模型服务先处理输入上下文的阶段；tensor 是带形状和元素类型的数据视图；replication 是复制副本；prefetch 是提前取数据以隐藏延迟。

coherence/shape/dtype/heap coherence 是多核缓存一致性；shape 是数据形状或张量维度；dtype 是元素类型；heap 是动态分配区域。第二册会把这些词和 cache line、allocator、runtime state 放在一起看。

contract/callback/lowering/Raft contract 是输入输出和失败边界；callback 是事件完成后的回调；lowering 是把高层描述降到更接近执行后端的表示；Raft 是共识协议族，用 leader、term、log 和 commit index 组织复制状态。

protocol version/magic/max frame 分布式协议常见字段。version 表示协议版本，magic 用固定字节快速识别帧格式，max frame 限制单条消息大小，避免旧协议误读或恶意输入拖垮解析器。

trace 代号 **T2/A1**、**T1/A0** 这类短写通常是教学 trace 中的 task/attempt 代号。它们不是协议名，只是为了在日志表里压缩展示“第几个任务、第几次尝试”。

roofline Roofline 是用算术强度、内存带宽和计算峰值估计性能上界的模型。它不是跑分图装饰，而是帮助判断一个 kernel 更受计算限制还是带宽限制。

数据布局缩写 AoS 是 array of structs，把每条记录的字段放在一起；SoA 是 struct of arrays，把同一字段集中成数组；AoSoA 是二者折中，按小块把字段重排。它们影响 cache line、SIMD 载入和预取效果，不是代码风格偏好。

C++ 原子接口 **fetch_add** 是原子读改写加法；**memory_order_relaxed** 只保证原子性，不建立跨线程顺序；**memory_order_acquire/release** 常用于建立“发布数据、随后读取数据”的同步边。读原子代码必须先画出谁写、谁读、靠哪个顺序保证可见。

线程等待接口 **condition_variable** 是条件变量；**notify_one/notify_all** 唤醒等待者；**try_push**、**not_full**、**not_empty** 这类名字通常来自队列合同，分别表示尝试入队、队列未满、队列非空。它们的关键是不变量和唤醒条件，而不是函数名本身。

运行时对象名 **TaskState**、**FutureState**、**MessageBus**、**BatchView**、**RunReport**、**QueueContract** 这类 CamelCase 名字是本书实验框架对象。**TaskState** 保存任务状态，**FutureState** 保存异步结果，**MessageBus** 传递消息，**BatchView** 描述一批输入视图，**RunReport** 保存一次运行报告，**QueueContract** 描述队列语义。

分布式字段名 **task_id**、**worker_id**、**request_id**、**record_id**、**leader_term**、**commit_index** 这类字段不是缩写谜语，而是身份和版本边界。id 定位对象，term 区分领导者任期，commit index 表示日志已经提交到哪里。

开源系统名 DuckDB 是嵌入式分析数据库；ClickHouse 是列式分析数据库；OpenMP

是用编译指令表达并行的编程模型。附录把它们作为源码阅读对象，不要求先掌握全部内部实现。

实验短码 `T0/T1/T2`、`V0/V1`、`B0/B1`、`S0/S1` 这类短码通常是 `trace`、版本、批次或阶段编号。它们服务局部表格和日志压缩，不是新协议；读的时候看同一小节给它们绑定了哪个任务、版本或缓冲槽。

目录

怎样阅读本册	iii
常见缩写和术语先导	v
第一部分 硬件执行与单机性能	
第 1 章 从一个计算任务开始	2
1.1 为什么从小任务开始	2
1.2 输入合同：先规定什么算一条记录	2
1.3 Reference：先有可信答案	3
1.4 记录身份：切分输入之前先知道每条记录是谁	4
1.5 数据路径：从字节到热字段	5
1.6 硬件第一层：同一行日志怎样穿过 CPU	6
1.7 从五行到一亿行：规模怎样改变问题形状	11
1.8 从 C++ 语句到执行账本	13
1.9 执行与等待：从一个循环到多个工作者	17
1.10 提交事实：文件存在不等于结果生效	17
1.11 第一层最小账本	18
1.12 版本阶梯：同一个系统怎样被迫长大	18
1.13 怎样把底层账本用于后续机制	22
1.14 从机器最小事实推导系统边界	23
1.15 从五行输入推到资源预算	25
1.16 一条记录的完整剖面	28
1.17 实验	29
1.18 小结	29
第 2 章 取指前端、流水线、乱序执行与分支预测	31
2.1 从现象到假设	32
2.2 取指供给的物理矛盾	33
2.3 从 C++ 到指令字节	34
2.4 I-cache：指令字节为什么要按块缓存	35
2.5 I-cache 的组相联结构	37
2.6 ITLB：PC 为什么还要经过地址翻译	39
2.7 预测下一条 PC：BTB、RAS 和方向历史	40
2.8 译码器：从机器码字节到微操作	42
2.9 Micro-op cache：为什么重复译码可以被缓存	43
2.10 完整账本：一条热循环迭代怎样变成可退休结果	45
2.11 把前端问题落到热循环	47
2.12 前端瓶颈的实验判别	48
2.13 完整案例：把逐字符解析器改造成两阶段热路径	48
2.14 流水线不是源码顺序	52
2.15 乱序执行和依赖链	54
2.16 Load/Store Queue、别名和发布可见性	58
2.17 分支预测来自输入分布	59
2.18 前端设计：热路径要短、直、可批量	61
2.19 工业界的诊断和内核组织	61
2.20 测量闭环：源码、二进制、计数器	63
2.21 项目交付：HotKernelReport	66
2.22 流水线实验门禁和习题	68
第 3 章 缓存、TLB 与虚拟内存的地址路径	69
3.1 地址首先是硬件选择问题	70
3.2 取指路径和取数路径必须同时看	72
3.3 数据 cache 的组相联结构	73
3.4 先画地址序列，再谈缓存层级	77
3.5 Cache line：搬运单位也是争用单位	78
3.6 工作集、预取和分块	79
3.7 TLB：页级覆盖也是性能资源	81
3.8 完整案例：同一批记录的四条地址路径	86
3.9 首次触页、NUMA 与测量边界	89
3.10 mmap、page cache 和持久化边界	90
3.11 案例追查：哈希聚合怎样从地址问题变成系统问题	91
3.12 MemoryShape 和 AccessTrace	94
3.13 源码阅读：从 Linux 页表和页缓存看内存成本	95
3.14 实验矩阵：把 cache、TLB 和 page fault 分开	97

3.15 内存实验门禁和习题	101
第 4 章 NUMA、互连与缓存一致性	103
4.1 从一个 atomic 事故推导共享写成本	103
4.2 缓存一致性从哪里来	106
4.3 互连：所有权消息走在哪里	109
4.4 NUMA：内存不是无位置的池	111
4.5 把拓扑写进算法：分层 partial 和本地优先	113
4.6 从对照判读区分一致性、带宽和 NUMA	115
4.7 完整案例：一条 cache line 从本地写到跨 socket 争用	117
4.8 C++ 接口怎样表达位置	120
4.9 从共享计数走到分层归约	122
4.10 Linux 观察入口和源码阅读	123
4.11 PMU 事件怎样映射到硬件结构	123
4.12 NUMA 的定量模型：远端不是慢一点这么简单	124
4.13 案例：日志统计的拓扑化改造	125
4.14 展开：MESI 状态机和 false sharing 的证据边界	126
4.15 状态机和事故：MESI 直觉如何变成生产诊断	128
4.16 常见误区和边界	129
4.17 习题	130
第 5 章 可信测量、Roofline 与性能计数器	131
5.1 先固定语义，再比较速度	131
5.2 计时边界：端到端、阶段和热循环	133
5.3 输入矩阵和统计方法	134
5.4 Roofline：用字节账本约束优化方向	137
5.5 计数器、Profile 和 Trace	141
5.6 反汇编、计数器和输入矩阵的三角校验	145
5.7 反证判读：不要只支持单一解释	147
5.8 测量审计：从错误样本追到机器证据	148
5.9 BenchmarkCase：把测量变成项目基础设施	151
5.10 完整案例：三倍加速为什么不能直接相信	152
5.11 工业工具和源码阅读	154
5.12 实现边界：测量对象	155
5.13 PMU 指标的因果表：从数字回到结构	155
5.14 测量报告的反面案例	161
5.15 状态机：一次性能结论怎样从假设走到验收	161
5.16 性能报告门禁和习题	163
第 6 章 PMU、Top-Down 与 SIMD 资源账本	165
6.1 从耗时数字走向硬件归因	165
6.2 PMU 事件到瓶颈类型的映射	165
6.3 完整 PMU 报告模板	166
6.4 SIMD 资源账本：宽寄存器只是起点	167
6.5 dot product 的四级资源账本	167
6.6 反汇编证据：证明 fast path 真的执行	168
6.7 端口压力和 uop 思维	169
6.8 实验矩阵	169
第 7 章 数据布局、局部性与预取	171
7.1 先从 cache line 和地址序列推导布局	171
7.2 AoS、SoA 和 AoSoA 从哪里来	174
7.3 预取：让未来地址可预测	177
7.4 生命周期、分配器和容器形状	178
7.5 图、哈希表和稀疏结构	180
7.6 并发写布局：先标出写者	181
7.7 压缩、编码和计算成本交换	182
7.8 迁移旧代码：保护接口和证据	182
7.9 布局判读	183
7.10 案例：LogRecord 到 BatchView	183
7.11 完整改造账本：从 LogRecord 到可审计热路径	184
7.12 完整迁移案例：从对象接口到 BatchView 内核	188
7.13 排障案例：改成列式以后为什么仍然慢	191
7.14 习题	194
第 8 章 SIMD、指令级并行与向量化	195
8.1 从一条标量循环看机器瓶颈	195
8.2 批量语义先于向量指令	196
8.3 lane、mask 和 tail 从哪里来	197
8.4 端口、lane 利用率和 reduction 成本	200
8.5 指令级并行：不用 SIMD 也能变快的原因	202
8.6 内存路径决定向量质量	203

8.7 自动向量化是编译器在做证明	203
8.8 三个案例	204
8.9 数值语义、溢出和特殊指令	205
8.10 平台分派和回退	205
8.11 从标量循环推到向量执行	206
8.12 源码阅读路线	207
8.13 实现边界	208
8.14 完整执行账本：ASCII 分类从标量到 SIMD	208
8.15 排障案例：向量版本为什么没有变快	211
8.16 ISA 现实：宽向量不是单调收益	215
8.17 默认路径：向量内核怎样进入系统	217
8.18 展开：SIMD 的指令级资源账本	218
8.19 状态机：从标量循环到 SIMD kernel	224
8.20 习题	226

第 9 章 典型计算内核模式 227

9.1 先写内核合同，而不是先写并行代码	228
9.2 完整案例：从内核合同推导机器形状和恢复边界	231
9.3 一对一和多对一：Map 与 Reduce	233
9.4 输出位置未知：Scan、Filter 与 Partition	235
9.5 共享写和数据倾斜：Histogram 与 Top-k	238
9.6 数据形状改变策略：Sort、Join、Stencil 与 Graph Frontier	240
9.7 把内核组合成可分析流水线	242
9.8 工业界设计参照	244
9.9 实现路径：KernelPatternSuite	246
9.10 内核模式实验门禁	246
9.11 接口延伸	247
9.12 习题	247

第二部分 并发、同步与运行时

第 10 章 线程、调度与亲和性 250

10.1 从执行预算推导 task、thread 和 worker	250
10.2 事故复盘：CPU 空闲，任务却不前进	257
10.3 操作系统调度器看到的不是业务优先级	259
10.4 任务粒度：队列成本、局部性和负载均衡的折中	261
10.5 阻塞隔离：把等待预算和计算预算分开	261
10.6 内核调度器看到的是 thread	263
10.7 亲和性：把线程、数据和拓扑放在一起	264
10.8 线程迁移的硬件后果：从 run queue 到 cache 热度	265
10.9 可观测性：WorkerSnapshot 和 ThreadTimeline	267
10.10 完整排障路径：从 ready 队列增长到调度结论	269
10.11 工业设计参照	271
10.12 Compute Systems Engine 实现边界	272
10.13 完整案例：一个任务从提交到真正运行	273
10.14 习题	277

第 11 章 Linux 调度、内存管理与系统瓶颈边界 279

11.1 为什么性能书必须讲内核边界	279
11.2 调度器看到的是 thread，不是业务 task	279
11.3 CFS/EEVDF、runqueue 和 cgroup 的最小模型	279
11.4 上下文切换和 off-CPU 时间	280
11.5 虚拟内存：VMA、PTE 和物理页	280
11.6 buddy、SLUB、reclaim、swap 和 OOM 的工程直觉	281
11.7 NUMA migration 和 first-touch	281
11.8 page cache、direct I/O 和可靠写	281
11.9 源码阅读路径	282
11.10 实验与验收	282

第 12 章 同步原语：锁、条件变量、信号量、阶段和结果 283

12.1 同步原语的统一坐标系	283
12.2 从自旋到睡眠：为什么需要条件变量	285
12.3 互斥锁保护的是不变量	286
12.4 futex 路径：用户态原子怎样变成内核等待	290
12.5 条件变量等待的是谓词	299
12.6 等待协议的三件事：状态、谓词、通知	301
12.7 有界队列：同步语义的最小系统	303
12.8 信号量：资源额度，不是对象不变量	307
12.9 一次性完成、阶段同步和一次性结果	315
12.10 同步原语选择族谱：把坐标系统一回工程报告	329
12.11 锁顺序、性能路径和工业现实	337
12.12 完整执行账本：从 wait 到 futex 再到关闭	337

12.13 读写锁、RCU 和原子的边界	340
12.14 QueueContract: 项目中的同步合同	340
12.15 完整案例: 容量为一的队列如何暴露所有同步错误	342
12.16 接口延伸	344
12.17 习题	344
第 13 章 原子操作与内存序	346
13.1 从锁版 Reference 到原子协议	346
13.2 内存模型的四个边界	352
13.3 用反例推导内存序	353
13.4 证明工具: happens-before 图和线性化点	356
13.5 从锁到原子的迁移步骤	361
13.6 Relaxed: 只关心自身数值	362
13.7 Release/Acquire: 发布普通数据	363
13.8 CAS: 线性化点和责任转移	366
13.9 生命周期: 可见不等于活着	369
13.10 Seq-cst、Fence 和硬件差异	370
13.11 工业设计参照	372
13.12 实现边界: AtomicProtocolNote	372
13.13 完整案例: TaskCompletion 的三次重构	374
13.14 接口延伸	378
13.15 习题	378
第 14 章 无锁数据结构	380
14.1 先问是否需要无锁	380
14.2 SPSC Ring: 最合适的起点	381
14.3 SPSC、MPSC、MPMC 不能混讲	384
14.4 线性化、回收和进展: 无锁结构的三份证明	385
14.5 从 CAS 到对象生命周期: 无锁结构真正难在哪里	386
14.6 ABA 和内存回收: CAS 看不见历史	389
14.7 退避、让出和背压	392
14.8 Treiber 栈: 短代码背后的生命周期	393
14.9 回收协议: Hazard、Epoch、RCU	395
14.10 完整执行账本: Treiber pop、ABA 和回收	398
14.11 无锁和等待、关闭、异常	400
14.12 工业设计参照	401
14.13 实现边界: LockFreeLifecycle	402
14.14 完整案例: SPSC ring 为什么比 Treiber 栈更适合作为起点	404
14.15 展开: linearizability、progress 和 reclamation 的证明模板	407
14.16 状态机: SPSC、MPMC 和关闭协议	409
14.17 接口延伸	411
14.18 习题	411
第 15 章 无锁正确性证明、回收与实现边界	413
15.1 为什么原理之后还要证明	413
15.2 Linearizability: 每个操作在哪一瞬间生效	413
15.3 内存序证明: 发布的是普通数据, 不是心情	414
15.4 ABA: 地址相同不代表对象相同	414
15.5 Hazard pointer: 保护窗口必须重新读取	414
15.6 Epoch reclamation: 快, 但怕暂停线程	415
15.7 RCU: 读多写少的另一种选择	415
15.8 进展证明要包含 allocator 和等待策略	415
15.9 测试矩阵: 无锁需要压力和反例	416
15.10 验收标准	416
第 16 章 归约、扫描与分区	417
16.1 从串行语义推导并行结构	417
16.2 Reduce: 把共享状态变成 partial	419
16.3 Scan: 把未知输出数量变成确定位置	421
16.4 Partition: 多目的地的 scan	424
16.5 工业界设计参照	426
16.6 确定性、审计和恢复	427
16.7 深度案例: 把共享 push 改造成可恢复 stage	427
16.8 从账本到伪代码: 完整执行路径	428
16.9 阶段账本: 让 reduce、scan、partition 可恢复	435
16.10 实现路径	439
16.11 并行算法实验门禁	440
16.12 任务运行时接口	441
16.13 习题	441

第 17 章 任务运行时与工作窃取	443
17.1 从线程池事故到运行时语义	443
17.2 任务图：把等待变成状态	456
17.3 从 worker loop 到任务图调度：完整执行路径	458
17.4 运行时证据链：ready、running、blocked 和 helping	464
17.5 Work Stealing：本地性和负载均衡的折中	468
17.6 取消、异常、关闭和可见性	472
17.7 工业界设计案例	474
17.8 深度案例：stable filter 如何进入任务运行时	475
17.9 实现路径	476
17.10 运行时实验门禁	478
17.11 习题	479
第 18 章 I/O、异步与背压	480
18.1 从阻塞 I/O 到完成事件	480
18.2 从系统调用到设备完成：底层路径账本	495
18.3 完整案例：从阻塞 write 改造成有背压的完成流水线	504
18.4 所有权、Buffer 池和取消	506
18.5 背压：容量合同和传播链	508
18.6 可靠写入：从临时文件到 Manifest	510
18.7 工业界设计案例	511
18.8 深度案例：异步写出 Shuffle Block	513
18.9 实现流程：把 I/O 代码拆成可分析状态	514
18.10 完整案例：一次 ShuffleBlock 写出怎样变成提交事实	515
18.11 稳态判断：吞吐、债务和恢复窗口	521
18.12 I/O 可靠性门禁	523
18.13 习题	523
第三部分 分布式计算与工程边界	
第 19 章 分布式计算模型	526
19.1 从响应丢失开始	527
19.2 消息身份和协议边界	528
19.3 协议版本、兼容和滚动升级	532
19.4 Deadline、重试预算和重试风暴	534
19.5 状态表和响应处理	536
19.6 重放日志和确定性调试	539
19.7 幂等提交和 Manifest	540
19.8 Deadline、重试预算和背压	541
19.9 Epoch、恢复和旧事件隔离	542
19.10 单进程 MessageBus：可控故障先于真实网络	542
19.11 从单进程 MessageBus 到多进程	543
19.12 工业设计参照	545
19.13 实现路径	546
19.14 完整案例：一次成功响应丢失后的正确恢复	548
19.15 消息故障门禁	550
19.16 习题	550
第 20 章 分片、复制与共识	552
20.1 旧 Owner 事故：状态归属先于算法	552
20.2 分片：集群尺度的数据布局	554
20.3 路由 Epoch 和再分片状态机	555
20.4 复制：写成功到底证明什么	557
20.5 Quorum 和读一致性	558
20.6 副本放置、成员变更和删除	560
20.7 Leader、Term 和共识边界	560
20.8 控制日志模拟器：把共识边界落到项目	564
20.9 工业设计参照	569
20.10 贯穿审查：只跟踪一个 Key 的一生	570
20.11 完整执行账本：热点 key 从发现到提交	571
20.12 实现路径	574
20.13 完整案例：旧 leader 为什么不能再写 manifest	576
20.14 分片复制门禁	579
20.15 习题	579
第 21 章 分布式计算工程实践	581
21.1 Reference 先行	581
21.2 端到端架构：控制面和数据面分开	582
21.3 Map、Combine、Shuffle 和 Reduce	583
21.4 网络、序列化和 shuffle 现实成本	584
21.5 端到端执行账本：三 split、两 worker、一个 Driver	588
21.6 Attempt、提交事实和恢复权威	590

21.7 Driver 状态机：消息怎样变成提交事实	591
21.8 可靠 I/O、Checkpoint 和 Backpressure	597
21.9 失败路径判读：不要让 happy path 定义系统	597
21.10 最终验收：不是跑完，而是可证明	600
21.11 里程碑：每一步都能失败、解释和回退	602
21.12 一次失败排查日记	602
21.13 工业设计参照	603
21.14 端到端反例：一个小优化怎样破坏整条链	604
21.15 面向后续推理系统的接口	605
21.16 端到端交付门禁	605
21.17 从 C++ 提交代码走到协议状态	606
21.18 模块边界：代码可以换，语义不能换	607
21.19 习题	609
附录 A 实验路线	611
A.1 统一交付格式	611
A.2 工程目录和交付闭环	611
A.3 统一实验协议	613
A.4 章节到实验的证据矩阵	616
A.5 已落地的同步原语实验	617
A.6 统一对象账本	618
A.7 阶段一：测量基础、热循环报告和顺序 reference	619
A.8 阶段二：硬件成本和数据布局	619
A.9 阶段三：单机高性能内核	619
A.10 阶段四：SIMD、ILP 和 Roofline	620
A.11 阶段五：线程、同步和内存模型	620
A.12 阶段六：并行算法和任务运行时	620
A.13 阶段七：I/O、异步和背压	621
A.14 阶段八：单节点分布式计算模拟	621
A.15 毕业项目	621
附录 B 源码阅读索引	623
B.1 阅读方法	623
B.2 工业案例报告模板	623
B.3 Linux 内核	624
B.4 C/C++ 运行时和标准库实现	624
B.5 编译器和运行时	624
B.6 并发库和无锁结构	625
B.7 HPC 和 AI 计算库	625
B.8 数据库、存储和分布式系统	625
B.9 第十六至十八章的专项阅读	626
B.10 版本和可复现性	626
附录 C 实践卷：并发、运行时和系统合同	627
C.1 零、贯穿主线工程	627
C.2 一、工程入口	627
C.3 二、阶段 0：并行归约先对照串行 reference	628
C.4 三、阶段 1：计数器和同步原语	628
C.5 四、阶段 2：有限队列和 backpressure	628
C.6 五、阶段 3：permit、barrier、future 和 atomic wait	629
C.7 六、阶段 4：wait channel 和 eventfd/epoll	629
C.8 七、阶段 5：任务运行时和同池等待事故	629
C.9 八、阶段 6：futex 快路径和慢路径	629
C.10 九、和代码实践卷衔接	630
C.11 十、每章代码任务矩阵	630
C.12 十一、递进大作业：Compute Systems Runtime	631
附录 D 完整源码清单：Compute Systems 并发实验	633
D.1 一、CMakeLists.txt	633
D.2 二、README	635
D.3 三、逐章模型头文件	638
D.4 四、并行归约头文件	643
D.5 五、同步原语头文件	645
D.6 六、等待通道头文件	646
D.7 七、任务运行时头文件	646
D.8 八、Futex 头文件	648
D.9 九、逐章模型实现	649
D.10 十、并行归约实现	659
D.11 十一、同步原语实现	664
D.12 十二、等待通道实现	668
D.13 十三、任务运行时实现	673
D.14 十四、Futex 实现	679

D.15	十五、主程序	687
D.16	十六、逐章模型 probe	688
D.17	十七、wait channel probe	692
D.18	十八、TaskRuntime probe	693
D.19	十九、futex probe	694
D.20	二十、完整测试	695
	第二册能力清单	709

第零部分

硬件执行与单机性能

第 1 章

从一个小计算任务开始

1.1 为什么从小任务开始

现代计算系统最容易被讲成一串名词：流水线、乱序执行、缓存、TLB、NUMA、锁、原子、无锁队列、线程池、异步 I/O、分布式协议。名词当然重要，但名词本身不会解释它为什么存在，也不会说明什么时候不该使用它。更稳的入口是一个可以手工检查的小任务：先让任务在规模、并发、I/O 和失败中逐步暴露问题，再从问题中推出机制。

本章使用的任务叫 **Compute Systems Engine**（计算系统引擎）。这里先把它压缩成一道极小的日志统计题：读入若干行日志，解析事件名，统计每种事件出现次数，输出稳定结果。这个任务足够小，能手工检查；也足够真实，后面可以自然暴露硬件、并发、运行时、I/O 和分布式边界。

最小输入如下：

Listing 1.1 五行日志输入

```
10:00 u001 login 1
10:01 u002 view 1
10:02 u001 click 1
10:03 u003 logout 1
10:04 u002 view 1
```

暂时只规定四列：时间戳、用户、事件名和值。第一版任务只统计第三列 `event_name`。这看起来像一个几十行程序能解决的问题；正因为它小，每一步才有手工验证的可能。若一个系统在五行输入上都无法说清正确结果，它在一亿行输入上的吞吐数字就没有意义。

核心思想

可靠系统从边界开始：先定义结果，再定义错误；先写 `reference`，再谈优化；先固定记录身份，再切分输入；先解释数据怎样流动，再解释硬件和线程为什么影响它；先区分临时输出和提交事实，再进入恢复和分布式。

本章先固定六个基础边界：输入合同、`reference`、记录身份、数据路径、执行与等待、提交事实。它们不是目录提示，而是系统分析的最小坐标系：热循环解释执行成本，地址路径解释缓存和 TLB，执行实体解释线程、锁和原子，输出边界解释 `completion`、可靠提交、重试和恢复。

1.2 输入合同：先规定什么算一条记录

很多系统错误不是从复杂算法开始的，而是从输入合同含糊开始的。日志统计的第一版合同可以写得非常窄：

- 每条记录占一行，以换行符结束；最后一行可以没有换行符。
- 每行恰好有四个以空白分隔的字段：`timestamp`、`user_id`、`event_name`、`value`。
- `event_name` 是非空字符串；第一版只把它当作 `key`，不解释业务含义。
- `value` 在第一版中不参与统计，但必须能被解析为整数，避免坏数据悄悄通过。
- 坏行不进入计数，必须进入诊断结果。

这个合同故意简单，但它已经比“读取日志然后统计”严格得多。若第三列缺失，程序不能随便把第四列当事件名；若某行有五列，程序不能悄悄忽略多余字段；若 value 不是整数，程序不能一边报错一边仍把事件计入结果。合同越早写清，后续优化越不容易偷偷改变语义。

从正常输入到失败输入 正常输入只能证明程序在顺路情况下工作；失败输入才能迫使边界清楚。把第五行改成下面这样：

Listing 1.2 坏行输入

```
10:04 u002
```

reference 应当报告 accepted records 为四，rejected records 为一，并且计数中不再有第二个 view。若程序只是打印一条错误日志却仍然继续计数，结果就错了；若程序终止整个作业，也许适合某些严格批处理，但那是另一种错误策略，必须在合同中声明。第一版采用“坏行进入诊断，合法行继续统计”的策略，因为它同时保留诊断身份和局部错误处理。

合同不是注释 输入合同必须能转成测试和报告字段。测试检查 accepted、rejected、每个 key 的计数和错误记录身份；报告写出输入版本、解析规则、错误策略和校验摘要。若合同只写在注释里，后续 SIMD parser、多线程 split 或分布式重试很容易无意破坏它。

1.3 Reference：先有可信答案

reference（参考实现）是用来定义正确结果的实现。它可以慢，但必须清楚、确定、容易检查。Reference 不是“第一版随便写的低性能代码”；它是后续所有优化版本的裁判。只要输入合同没有变，优化版本、并行版本、异步版本和分布式版本都必须与 reference 在可观察结果上一致。

对五行输入，手工结果是：

Listing 1.3 手工期望结果

```
accepted = 5
rejected = 0
counts:
  click  = 1
  login  = 1
  logout = 1
  view   = 2
```

对含坏行的输入，期望结果是：

Listing 1.4 含坏行的期望结果

```
accepted = 4
rejected = 1
counts:
  click  = 1
  login  = 1
  logout = 1
  view   = 1
diagnostics:
  record 4: missing fields
```

这里的 `record4` 还只是一个行号，后面会升级为稳定的记录身份。现在先注意一点：`reference` 同时定义正常结果和错误结果。如果 `reference` 只检查计数，不检查 `rejected records` 和 `diagnostics`，优化版本就可能把坏行吞掉而不被发现。

Reference 的伪代码 原理卷只需要伪代码描述语义：

Listing 1.5 `reference` 的语义伪代码

```
initialize empty count table
initialize empty diagnostic list

for each input line with line_number:
    parse the line according to the input contract
    if parsing fails:
        append diagnostic(line_number, error_kind)
        continue
    increment count[event_name]

sort output keys by stable order
return counts and diagnostics
```

这段伪代码没有讨论哈希表选型、内存分配、SIMD 或线程。它刻意把正确性放在性能之前。稳定输出顺序也很重要：哈希表遍历顺序可能随实现、机器和插入顺序变化；若输出用于测试、缓存或下游输入，就应排序或使用稳定编码。确定性不是为了好看，而是为了让差异可比较。

Reference 和 oracle 的区别 `Reference` 生成期望结果，`oracle`（判定器）负责比较结果。二者经常被混为一谈。对整数计数，`oracle` 可以要求完全一致；对浮点归约，`oracle` 可能要求误差界；对近似 `heavy hitter`，`oracle` 必须检查误差合同而不是精确相等。第一章的计数任务选择整数结果，是为了先建立最严格的正确性边界。

1.4 记录身份：切分输入之前先知道每条记录是谁

当输入只由一个线程顺序扫描时，行号似乎已经足够。但一旦输入被切成多个范围，行号、字节偏移、文件名和输入版本就会变得重要。假设一个大文件被切成两个 `split`，`split 0` 处理前三行，`split 1` 处理后两行。若第二个 `split` 报告“第 1 行坏了”，这个“第 1 行”到底是整个文件的第 1 行，还是 `split` 内部的第 1 行？如果没有稳定身份，诊断和恢复都会含糊。

第一版记录身份可以用一个逻辑结构表示：

Listing 1.6 记录身份的伪结构

```
RecordId:
    input_name
    input_generation
    byte_offset
    line_number
```

`input_name` 区分不同输入文件；`input_generation` 区分同一路径下不同版本的输入；`byte_offset` 支持从文件定位；`line_number` 支持人读诊断。不同系统可以选择不同字段，但必须回答同一个问题：当一个错误、计数或输出被报告出来时，它能否追溯到唯一输入事实。

Split 不是简单除以线程数 `split`（输入切片）是可独立处理的输入范围。它可以按字节范围划分，但语义上必须包含完整记录。若从文件中间切开，切分点可

能落在一行内部。正确 reader 需要移动到行边界，避免同一行被两个 split 重复处理，也避免一行被截断后误报坏行。这个问题与线程无关，单线程模拟 split 也会出现。

两个 split 的手工结果如下：

Listing 1.7 两个 split 的局部贡献

```
split 0 covers records 0..2:
  login = 1
  view  = 1
  click = 1

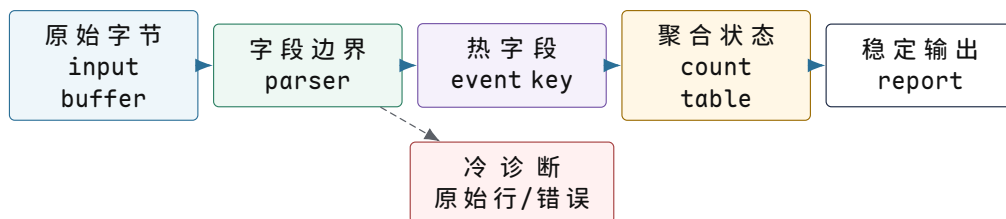
split 1 covers records 3..4:
  logout = 1
  view   = 1

merge:
  click  = 1
  login  = 1
  logout = 1
  view   = 2
```

这引出 **partial**（局部结果）。Partial 不是最终答案，而是某个输入范围对最终答案的贡献。它至少要说明来源：来自哪个 input、哪个 split、哪个输入版本，以及覆盖哪些记录。缺少来源字段时，合并阶段只能看到一堆计数，不知道它们是否重复、过期或缺失。

1.5 数据路径：从字节到热字段

现在把一条记录从输入字节追到计数表。最朴素的路径是：reader 读一行字符串，parser 拆字段，aggregator 用事件名更新计数表，writer 输出结果。小输入时这条路径很清楚；输入变大后，真正花时间的可能不是整数加法，而是读取字节、查找分隔符、比较字符串、计算哈希、分配节点、访问随机桶和写出结果。



热路径反复处理合法记录；冷路径只在错误、审计或调试时访问。
把冷数据塞进热对象，会让 CPU 搬运大量暂时用不到的字节。

图示：从字节到计数的路径可以投影到 parser 热循环、地址序列、数据布局 and 共享状态；同一条数据流在不同层暴露不同成本。

热路径（hot path）是正常记录反复经过的主流程；**冷路径**（cold path）是错误、诊断、审计或调试时才访问的流程。日志统计的热路径需要事件名、可能的 key 编码和计数槽位；冷路径需要原始行、错误详情、文件名和调试上下文。把两者混在一个巨大对象里，会让聚合循环反复带入不需要的字节。把冷数据完全丢掉，又会让错误报告无法定位。好的布局不是“只保留热字段”，而是让热字段连续、让冷字段可回查。

地址路径比类图更接近硬件 类图告诉我们对象之间的逻辑关系，地址路径告诉我们 CPU 实际访问什么。连续数组、字符串池、哈希桶、链表节点和错误列表会

形成完全不同的地址序列。后面讲 cache、TLB 和预取时，不会从抽象层级图开始，而会回到这里问：一条记录在内存中的哪些地址被访问，这些地址是否连续，是否跨很多页，是否被多个核心写。

汇编是连接语义和硬件的中间证据 系统分析不能直接从 C++ 跳到“缓存不好”或“分支不好”。中间至少要看一次机器形态。下面的聚合语义：

Listing 1.8 聚合语义

```
1 local_counts[event_type] += value;
```

在普通整数数组上常接近下面的抽象机器路径：

Listing 1.9 聚合更新的典型机器形态

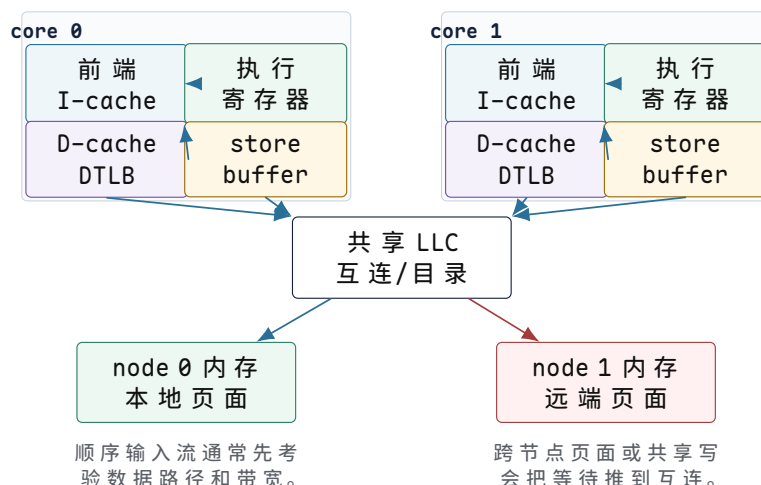
```
load  event_type from event_type column or object field
compute counter address = base_counts + event_type * sizeof(counter)
load  old counter
add   value
store new counter
```

如果 `event_type` 来自连续热列，第一条 `load` 的地址是短步长；如果它来自胖对象，地址里会出现对象大小乘法和字段偏移；如果 `local_counts` 是 worker 本地数组，`store` 多数落在本核心拥有的 line；如果它是全局原子计数器，更新可能变成带 `lock` 语义的 `read-modify-write`。源代码同样是一句加法，汇编形态却把地址、`load/store`、分支和同步边界暴露出来。通用方法是：先固定语义，再看典型机器形态，再解释硬件结构为何会让这条路径快或慢。

1.6 硬件第一层：同一行日志怎样穿过 CPU

把五行输入放大到一亿行后，瓶颈很少只来自“循环次数多”。CPU 执行一次日志解析，会同时处理两类流：指令流和数据流。指令流来自 `parser`、`hash`、`compare`、`branch`、`counter update` 这些机器指令；数据流来自输入缓冲区、字段字节、事件名字典、计数表和输出缓冲。指令流走 I-cache，数据流走 D-cache 和 TLB；分支预测决定下一段指令能否提前取入；`store buffer` 暂存尚未完全提交的写入；`coherence` 协议决定多核心写同一位置时谁拥有 `cache line`。

硬件拓扑先作为坐标系



图示：本图要证明的命题是：前端、cache/TLB、NUMA 和一致性不是并列名词，而是同一条日志热路径经过的不同位置。

读这张图时要把三件事分开。第一，指令流从每个核心的前端进入：parser 的热循环、分支目标和机器码足迹决定 I-cache、ITLB、预测器和译码是否能稳定供给。第二，数据流从 load/store 进入：输入 buffer、热字段、哈希桶和计数槽的地址序列决定 D-cache、DTLB、cache line 和 page walk 的等待。第三，多个核心共享写或跨节点访问时，等待不再只在单个核心内部：cache line 所有权、互连路径、页面 first-touch 和 NUMA 放置会进入解释。

取指：代码本身也要被搬运 程序运行前，编译器已经把 C++ 代码变成机器指令。CPU 核心执行热循环时，需要先从内存层级中取出这些指令。最靠近核心的是 I-cache (instruction cache, 指令缓存)，它缓存最近执行过的指令字节。若 parser 热循环很小、分支路径稳定，I-cache 命中率通常很高；若把大量错误处理、日志格式化、虚函数分派和罕见路径塞进同一个热函数，指令工作集会膨胀，前端可能开始等待指令供应。

I-cache 的关键事实是：CPU 不会因为代码“逻辑上属于同一个函数”就轻松执行它。它取的是指令地址附近的一段字节。热路径代码越集中，前端越容易持续供给；热路径不断跳到分散的大函数、模板展开过度或异常处理路径，就可能让前端频繁取新指令。分析流水线和分支时，I-cache 会和分支预测一起出现：分支预测不仅决定下一条路，也决定下一批指令是否能提前取来。

取数：D-cache 和 cache line 解析一行日志时，CPU 会读取输入缓冲中的字节，寻找空格，把 `event_name` 的字节范围拿出来，再更新计数表。数据读取首先查 D-cache (data cache, 数据缓存)。D-cache 不按“字段”搬运，而按 cache line 搬运；常见 line 大小是 64 字节。读一个字符，也会把包含它的一整条 line 拉进缓存。

因此顺序扫描输入缓冲很适合 CPU：第一个字节命中后，后面的字节通常落在同一条或相邻 cache line，硬件预取器还能提前请求后续 line。相反，字符串 key 查哈希表可能访问多个分散位置：哈希表桶、节点、key 字节、计数值，每一步都可能落在不同 cache line。小任务表面上是“统计事件名”，底层却是两种完全不同的数据流：输入缓冲是连续流，计数表查找是间接流。

地址翻译：TLB 为什么会参与日志解析 程序使用虚拟地址，内存控制器最终访问物理地址。虚拟页到物理页的翻译结果缓存在 TLB (translation lookaside buffer, 地址翻译缓存) 里。若热循环顺序扫描一个大缓冲，页面访问有规律，TLB 压力相对可控；若哈希表节点散落在大量页面上，每次查找可能触碰新的页，TLB

miss 会让核心等待页表遍历。

这解释了为什么数据布局要强调冷热分离和连续热列。把每条日志做成一个包含原始文本、错误详情、调试字段的大对象，聚合阶段每处理一条记录就跨更大的对象步长，每页能覆盖的有效记录数下降。把 `event_type[]` 和 `value[]` 做成连续热列，TLB 覆盖的逻辑记录数会明显增加。TLB 不是操作系统章节的孤立名词，它直接影响一条日志在热循环中要等多少地址翻译。

分支：坏行和热点 key 会改变控制流 Parser 通常包含大量判断：是不是空格，字段数够不够，value 是否是整数，event name 是否为空。若大部分输入格式一致，分支预测器能学到稳定模式，流水线可以提前沿常见路径取指和执行。若输入坏行随机出现、字段长度差异极大、错误处理路径和正常路径混在一起，预测失败会让流水线丢弃已经投机执行的工作，重新从正确路径取指。

这也是为什么错误路径要从热路径中分离。不是因为错误不重要，而是因为错误处理需要更多状态、更多字符串、更多分支和更多输出。把它们放在冷路径里，正常记录可以走短而稳定的控制流；错误记录仍然保留诊断身份，通过 `RecordId` 回查原始行和字段边界。

写入：计数表更新不是一个抽象加一 当 aggregator 执行 `count[event_name] += 1` 时，底层至少发生三件事：找到计数槽位，读取旧值，写回新值。单线程下，这通常只是 load、add、store。多线程下，如果多个核心更新同一个热门 key，包含该计数值的 cache line 必须在核心之间转移独占所有权。原子加法保证结果不丢失，却不能消除这条 line 的迁移；锁可以保护整个表，却会把热路径排队；线程本地 partial 则把高频写变成本地写，最后再合并。

store 也不会瞬间到达内存。核心通常先把写入放进 store buffer，之后再按一致性和内存层级规则提交。若写入目标是连续且由一个核心独占，store buffer 能平稳排出；若多个核心争同一条 line，store 可能等待所有权。内存序讨论会把 release、acquire、store buffer 和可见性接起来；此处先固定一点：共享写位置越热，硬件越难把它当作普通加法处理。

同一任务的硬件账本 把一条合法记录的热路径压成账本，可以得到下面的事件链：

Listing 1.10 一条合法日志记录的硬件事件链

```
fetch parser instructions from I-cache
load bytes from input buffer through D-cache
translate input addresses through TLB
branch on separators and field count
compute or lookup event key
load count-table bucket or local partial slot
update counter in register
store counter back through store buffer
```

每一行都可能成为瓶颈：I-cache 前端断供、D-cache miss、TLB miss、分支预测失败、哈希表随机访问、共享 cache line 迁移、store buffer 等待。这样概念就不是凭空出现的：I-cache 来自取指瓶颈，D-cache 来自数据搬运，TLB 来自地址翻译，分支预测来自控制流，原子和锁来自共享写，scan 和 partition 来自输出位置和恢复边界。

等待账本怎样变成优化判断 硬件账本不是为了把术语列得更完整，而是为了约束优化顺序。一个成熟的优化判断通常按三步走：先定位等待发生在哪条账本线上，再提出只改变这一条线的设计，最后用机器证据验证等待是否真的移动或减少。若没有这三步，优化很容易退化成“把代码写得更复杂，然后希望它更快”。

仍然看日志统计。假设单线程 reference 正确，但吞吐很低。第一种可能是 parser 热循环前端供给不足：反汇编显示正常路径里夹着大量错误格式化、日志

字符串构造和罕见分支，profile 显示热点分散在多个冷函数入口附近。此时合理设计不是先加线程，而是把错误路径拆到冷函数，正常路径只留下字段边界推进和少量状态转移。验证材料也不只是总时间：还要看热循环指令长度、分支目标、前端停顿相关计数，以及坏行比例变化时吞吐是否更稳定。

第二种可能是数据等待。若 parser 已经很短，但聚合阶段每条记录都追哈希表节点，硬件账本指向 D-cache/TLB miss。此时加锁、换线程池或调大队列都不能触及主因。更直接的设计是先把字符串事件名编码成小整数，再用连续数组保存局部计数。验证材料应当包括地址路径变化：从 bucket->node->keybytes->counter 变成 event_type[i]->local_counts[event_type[i]]，再配合缓存 miss、TLB miss 和吞吐变化观察。

第三种可能是共享写等待。若单线程已经接近内存带宽，多线程后吞吐反而下降，且热门 key 被多个 worker 同时更新，全局原子计数器很可能让 cache line 在核心间来回迁移。此时“原子是无锁的，所以应该更快”是错误直觉。硬件账本要求把高频写改成本地写：每个 worker 维护自己的 partial，最后合并。验证材料应当看扩展曲线、每线程 partial 的 cache line 对齐、合并阶段占比，以及热门 key 分布改变时吞吐是否仍然稳定。

账本线索	优先考虑的改造	需要补的证据
前端供给不足	热路径收缩，冷路径外提，减少间接跳转	反汇编长度、分支目标、前端停顿、坏行敏感性
D-cache/TLB miss	连续热列、小整数编码、开放寻址或数组计数	地址路径、cache/TLB 计数、每页有效记录数
分支预测失败	状态机分层、错误路径分离、批量 mask	输入分布、mispredict、正常/异常路径比例
共享写争用	线程本地 partial，按 cache line 隔离，延后合并	扩展曲线、coherence 相关事件、合并成本
输出提交不清	临时文件、checksum、fsync、manifest 提交	崩溃点、恢复扫描、重复提交语义

这张表不是优化配方，而是判断顺序。先找等待，再改数据形状或执行形状；先证明语义不变，再证明机器形态改变；先解释单机，再解释并发和分布式。第二册后面的每一章都应当回到这条方法。

为什么会有缓存层级 如果核心每条 load 都直接等 DRAM，算术单元会大部分时间空转。现代核心的频率远高于主存响应速度，流水线一次能容纳很多尚未完成的指令，乱序执行也只能在可见窗口内寻找不依赖慢 load 的工作。因此机器在核心旁边放多级缓存：L1 最近、容量小、延迟低；L2 更大但更慢；LLC 容量更大、被多个核心共享；再往外才是内存控制器和 DRAM。缓存层级不是为了抽象整洁，而是为了让“最近用过、附近会用、反复会用”的字节尽量停在离核心更近的地方。

日志统计天然同时包含适合缓存和不适合缓存的访问。顺序扫描输入时，下一条 cache line 很可能紧跟当前 line，硬件预取器可以提前发起请求；事件字典若很小，查找表也可能长期留在 L1/L2；而字符串哈希表节点若由分配器散布在堆上，访问序列就变成不可预测的地址跳转。缓存不能理解“这是同一个事件名”，它只看到地址。地址连续、复用距离短、工作集小，缓存就容易帮忙；地址分散、复用距离长、跨页多，缓存就只能不断替换。

Listing 1.11 同一个语义在缓存眼里的三种形状

```
sequential input scan:
  line 0, line 1, line 2, line 3 ...

compact event id aggregation:
```

```
event_type[i] -> local_counts[event_type[i]]

string hash table aggregation:
    bucket pointer -> node -> key bytes -> next node -> counter
```

这也解释了为什么小任务一开始就要落到硬件。数据结构在源代码里可能都叫“统计 event”，但核心看到的一个是连续字节流，一个是小整数表，一个是指针追踪。数据布局、SIMD、线程本地 partial 和分布式 partition，都从这个事实开始。

I-cache 失败不是玄学 I-cache miss 或前端供给不足，常被误解成“代码太大”。更准确的说法是：热路径需要的指令字节没有以足够稳定、紧凑、可预测的方式供应给译码和执行端。一个 parser 如果把正常路径、错误格式化、日志打印、异常构造、虚函数回调和模板展开后的分支全部挤在一起，热循环不一定能稳定留在前端的工作集里。即使 I-cache 不 miss，过多复杂分支和间接跳转也可能让取指方向不稳定，译码带宽被浪费在罕见路径上。

观察 I-cache 问题不能只看函数名。更可靠的材料包括：热循环反汇编有多长，正常路径是否连续，冷错误路径是否被内联进热函数，分支目标是否分散，profile 中热点是否落在前端停顿相关区域，输入错误率改变时吞吐是否异常下降。若把错误处理搬到冷函数后正常输入变快，而坏行密集输入没有丢诊断，就说明修改真正改变了取指和控制流形状，而不是偷偷删掉了工作。

Page walk 是 TLB miss 背后的账本 TLB 保存虚拟页到物理页的翻译。TLB miss 时，核心需要查页表，这个过程叫 page walk。页表本身也在内存中，page walk 可能访问多级页表项；这些页表项也可能命中或 miss cache。于是一次看似普通的 load，若数据地址翻译不在 TLB 中，可能先变成多次页表读取，再变成真正数据读取。大页、连续分配、减少随机页访问、提高每页有效记录数，都会改变这条账本。

日志任务里，顺序扫描 4.8 GiB 输入不代表每个字节都很贵，因为每个 4 KiB 页可提供很多连续记录；胖对象数组若每条记录 160 字节，一个页只能放几十条记录，其中热字段可能只有几个字节；哈希表节点若散在大量页上，每次聚合都可能触碰新页。TLB 的核心问题不是“地址翻译很高级”，而是“有限的翻译缓存能覆盖多少有用数据”。每页有效记录越少，TLB 覆盖越差。

Listing 1.12 TLB 覆盖的直观估算

```
4 KiB page, average record 48 bytes:
    about 85 raw text records per page during sequential scan

4 KiB page, 160-byte fat LogRecord:
    about 25 objects per page, hot fields are only a small fraction

4 KiB page, scattered hash nodes:
    possibly only a few useful counters or keys per touched page
```

这不是精确性能公式，却足以推导设计方向。若热循环只需要 event_type 和 value，把它们放成连续列，会让同一个 TLB 条目覆盖更多有效记录；若诊断字段只在错误路径回查，就不应让它们跟着热循环一起跨页移动。

投机执行为什么会惩罚 parser 分支预测器根据过去的控制流猜下一条路径，前端按猜测取指，后端按猜测执行。猜对时，流水线保持忙碌；猜错时，已经投机执行的工作被丢弃，核心从正确路径重新开始。Parser 的分支很多：字符是不是空格，字段是否结束，value 是否为负，是否越界，是否坏行。只要输入分布稳定，预测器可以学到模式；若长短行、错误行、特殊字符和罕见格式交错出现，预测状态会被打乱。

这不是说 parser 一定要消灭所有分支。分支是语义边界的一部分。真正的改造方向是把分支分层：正常格式走短路径，错误路径记录身份后转入冷处理；高频字符分类尽量用查表、位运算或 SIMD mask 变成数据流；跨块状态用明确状态机表达，避免在热循环里反复做全量判断。SIMD mask 的本质，就是把“每个字节走哪个分支”改成“先生成一组布尔事实，再批量处理”。

Store buffer 和一致性把写入变成协议 写计数器看起来比读输入简单，但多核心下往往更难。核心写入时先进入 store buffer；如果目标 line 只被本核心使用，写入可以平稳排出。若其他核心也要写同一 line，一致性协议必须让某个核心取得独占所有权，其他副本失效或等待。全局 atomic counter 语义上只是一行加法，硬件上可能是许多核心反复抢一条 line。锁也一样：锁字本身是一条热门 line，临界区保护的计数表还可能产生更多共享写。

线程本地 partial 的底层意义，是把高频写从“多核心争一条 line”改成“每个核心写自己的 line”。最后的合并阶段是读多写少，顺序可控，容易测量。这个设计并不是并行算法的装饰，而是从 store buffer、cache line 所有权和一致性流量推出的工程结论。

1.7 从五行到一亿行：规模怎样改变问题形状

五行输入能手算语义，却不能暴露系统成本。把输入放大到一亿行后，任务仍然是同一个任务：解析、按事件名计数、输出稳定结果。变化发生在每个边界的成本上。输入字节超过缓存容量，顺序扫描开始依赖内存带宽；事件名变多后，哈希表查找开始产生随机访问；线程变多后，热门 key 的计数槽位变成共享写热点；输出变大后，文件写入不再是最后一行代码，而是 page cache、writeback、fsync 和 manifest 的状态机。

规模账本 先估算一亿行的最低账本。假设平均每行 48 字节，原始输入约 4.8 GiB。若 parser 只顺序读输入，硬件主要面对连续字节流；若每行生成一个 160 字节胖对象，热路径后续又要扫描约 16 GiB 的对象数组，其中大部分字段并不参与聚合；若事件名保持字符串，每次更新可能访问字符串字节、哈希表桶、节点和计数值。此时“同一个算法”在硬件眼里已经变成三条完全不同的数据流。

数据形状	硬件主要看到什么	可能暴露的概念
顺序文本扫描	连续 cache line、稳定分支、少量页翻译	D-cache、TLB、硬件预取、分支预测
胖对象数组	大步长、冷字段、每页有效记录少	数据布局、冷热分离、有效字节比例
字符串哈希表	随机桶、节点追踪、key 比较、分配器	cache miss、TLB miss、编码、开放寻址
共享全局计数	多核心写同一 cache line	锁、原子、coherence、线程本地 partial
可靠输出	临时文件、checksum、fsync、manifest	completion、durability、commit、恢复

这张表说明一个重要事实：系统设计不是把小程序“并行一下”。规模会把隐藏在一行 C++ 语句里的硬件、运行时和 I/O 成本全部摊开。若不先写规模账本，后续很容易把慢归因到模糊的“CPU 不够”或“多线程不行”。

从字节成本推出热字段 一条日志有原始文本、字段边界、用户 id、事件名、值、错误状态、文件 offset 和诊断上下文。聚合阶段真正反复访问的只是事件 key 和 value。若每条记录都用胖对象保存所有字段，聚合循环每次都会跨过大量冷数据。硬件按 cache line 搬运，不会因为某个字段在业务上“不需要”就自动跳过它。

因此数据布局不是后期优化，而是从规模账本自然推出的结构选择。先解析输入，生成连续的热字段；同时保留 record id，让冷诊断路径能回查原始字节。热

字段服务 CPU, record id 服务语义。任何只追求吞吐而丢掉错误回查的改造, 都还没有完成系统边界。

从随机访问推出字典编码 事件名如果直接以字符串参与聚合, 热循环要反复计算哈希、比较字符串、追踪节点。若事件类型集合可枚举, 前置阶段可以把字符串映射成整数 id, 后续阶段用整数数组或本地计数表聚合。这个改造的本质不是“字符串慢”, 而是把随机、变长、分散的访问集中在解析阶段, 把聚合阶段改造成短整数、连续数组和固定宽度操作。

Listing 1.13 字典编码前后的热路径账本

```
before encoding:
  read event_name bytes
  compute hash over variable-length bytes
  find hash bucket
  compare candidate strings
  update counter

after encoding:
  parse event_name once
  map event_name to event_type id
  aggregate event_type id with integer counter
```

编码引入了新的控制面事实: 字典版本。若同一个整数 id 在不同运行中代表不同字符串, 输出就失去可比性。稳定系统必须把 dictionary id、dictionary checksum 或 schema version 写进报告。性能优化一旦改变可见身份, 就必须补上新的审计字段。

从共享写推出线程本地 partial 最直接的并行方案是多个 worker 解析不同 split, 然后都执行 `global_counts[event]+=value`。若用锁保护全局表, 所有 worker 在热门 key 上排队; 若用原子计数器, 结果不会丢, 但包含该计数器的 cache line 会在核心之间来回转移; 若用每 worker 本地 partial, 热写发生在本地内存, 最后统一合并。Partial 不是为了显得复杂, 而是从共享写的硬件成本里推出来的。

Listing 1.14 共享计数到 partial 的推导

```
global counter:
  many cores write the same counter cache line
  coherence moves ownership between cores
  atomic correctness remains, throughput collapses on hot keys

worker-local partial:
  each core writes its own counter array
  hot updates stay mostly local
  final merge reads partial arrays and produces stable output
```

这也解释了为什么 partial 必须带来源身份。一个 partial 来自哪个 split、哪个 worker、哪个 attempt、哪个输入版本, 都影响恢复和去重。只把若干计数数组相加, 短期能得到数字; 一旦重试、失败或恢复出现, 就无法判断某份贡献是否已经被接受。

从输出放大推出提交边界 小输入可以直接打印到标准输出; 大输入通常要写 block、合并、压缩、上传或给下游读取。写出路径一放大, 文件存在和结果生效就必须分开。临时文件说明某次物理执行留下了字节; checksum 说明字节看起来

完整；fsync 说明持久化边界更强；manifest 才说明某个 logical split 的某个 attempt 被接受。I/O 和分布式提交不是另起话题，它们只是把“输出稳定结果”这件事推到真实故障边界。

1.8 从 C++ 语句到执行账本

系统分析不能停在源代码语句。下面这句 C++ 看起来只有一次加法：

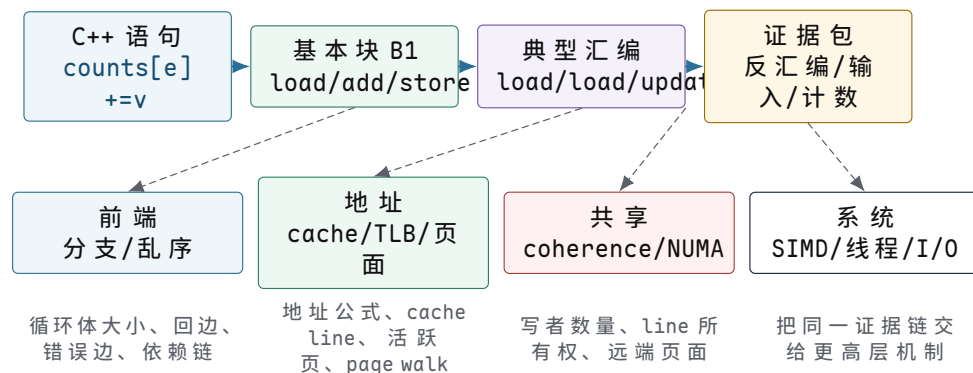
Listing 1.15 源代码里的聚合更新

```
1 counts[event_type] += value;
```

在单线程、小数组、热缓存情况下，它确实很接近一次普通更新。换成大表、多线程、热点 key 和冷缓存后，这句代码会拆成一串硬件事件：计算索引地址，翻译虚拟地址，加载计数所在 cache line，等待该 line 处于可读或可写状态，在寄存器中相加，再把结果写回 store buffer。若计数槽位被多个核心同时写，核心还要竞争该 line 的独占所有权。正确性、吞吐和尾延迟都藏在这些步骤里。

同一行代码可以从三个方向审查。第一，看它所在基本块怎样被前端取指、预测和乱序窗口消化。第二，看 `event_type[i]`、`value[i]` 和 `counts[event_type]` 这些地址怎样穿过 TLB、L1D、cache line 和页。第三，看当多个 worker 写同一张表时，cache line 所有权怎样跨核心、跨互连和跨 NUMA 节点移动。术语只有落回同一个热块，才有工程含义。

同一更新块的三个硬件审查方向



图示：本图要证明的命题是：第二册不是按术语跳转，而是把同一个更新块分别放大到前端、地址路径和多核心拓扑。

先切基本块，再贴硬件路径 第一册已经讲过基本块和控制流图。第二册继续使用同一套读法，只是每个基本块还要附上地址流和等待位置。以一条记录的聚合为例，热路径大致可以切成四个块：读取记录字段，判断记录是否有效，执行计数更新，进入下一条记录。源码里它可能只是一个 `for` 循环；机器层面却是一组普通顺序边、条件边和循环回边。

Listing 1.16 一条记录聚合的基本块形状

```
B0 loop_head:
  load event_type[i]
  load value[i]
  test valid flag or parser state
  if invalid goto B2_error
```

```

B1_update:
    counts[event_type] += value
    goto B3_next

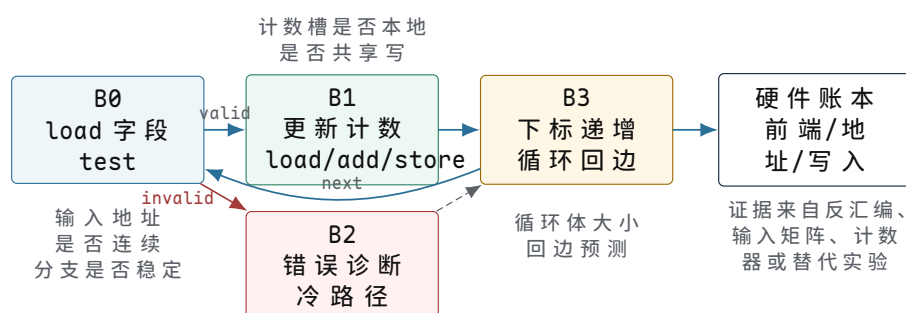
B2_error:
    append diagnostic identity

B3_next:
    i = i + 1
    if i != n goto B0_loop_head

```

这张 CFG 的价值不只是说明程序走哪条路。它还把硬件问题分组：**B0** 的输入 load 是否连续，决定 D-cache 和 TLB 是否友好；**B0** 到 **B2** 的条件边是否稳定，决定分支预测是否容易；**B1** 的写入目标是否线程私有，决定 store buffer 和 cache line 所有权是否平稳；**B3** 的回边是否规律，决定前端能否持续取回热循环。没有基本块，硬件账本会变成一串散乱名词；有了基本块，每个等待都能挂到某条边或某段顺序指令上。

从 CFG 到硬件账本



图示：本图要证明的命题是：CFG 决定执行路径，硬件账本决定每条路径在真实机器上等什么。

再把更新块落到汇编形状 **B1_update** 在一个整数编码、本地计数数组的版本中，可能接近下面这种机器形状：

Listing 1.17 本地计数更新的一种典型汇编形状

```

; rdi = event_type column
; rsi = value column
; rdx = local_counts
; rcx = i
mov    eax, dword ptr [rdi + rcx*4]    ; event_type[i]
mov    r8d, dword ptr [rsi + rcx*4]    ; value[i]
add    qword ptr [rdx + rax*8], r8     ; local_counts[event] += value

```

这段汇编不是要求编译器必须生成同样文本，而是让读者看到执行账本怎样落地。前两条 load 若来自连续列，地址是短步长；第三条更新的目标由 `rax` 决定，可能落在小型本地数组，也可能因为 key 分布变成随机槽位。若 `local_counts` 换成全局原子计数，机器形态可能出现带锁定语义的读改写写；若 `event_type` 仍是字符串，前面还会多出哈希、比较和节点追踪。第二册后面的优化都围绕这种形状变化展开，而不是围绕 C++ 源码行数展开。

Listing 1.18 同一更新块的硬件读法

```

instruction stream:
    loop body size, branch targets, decode pressure

address stream:
    event_type[i] and value[i] are stride loads
    local_counts[event] may be a small local table or a random table

write ownership:
    worker-local table stays mostly private
    global atomic table may move cache lines between cores

```

语义账本和执行账本要同时存在 语义账本回答“结果是什么”，执行账本回答“结果怎样被机器推进”。两者缺一不可。只有语义账本，优化无法判断是否破坏正确性；只有执行账本，系统可能很快地产生错误结果。对日志统计，语义账本记录 accepted、rejected、counts、diagnostics 和 manifest；执行账本记录输入字节、热字段布局、cache/TLB、任务等待、I/O completion 和提交状态。

层次	主要问题	最小证据
语义	哪些输入被接受，结果是否稳定	reference、oracle、diagnostics
地址	热路径访问哪些内存位置	layout report、有效字节比例、页覆盖
执行	CPU 在等指令、数据、分支还是写入	计数器、时间分解、硬件账本
并发	谁在等待谁，谁共享写	worker snapshot、等待图、partial 来源
提交	哪个输出对外生效	manifest、attempt、checksum、恢复审计

反例：只看平均吞吐会误导 假设版本 A 单线程处理 100 万行每秒，版本 B 使用 8 线程处理 300 万行每秒。若报告只写平均吞吐，版本 B 看起来更好。但如果版本 B 在热点 key 输入下退化到 50 万行每秒，坏行密集时吞掉 diagnostics，磁盘变慢时无界队列增长，崩溃恢复后重复读取旧 block，它就不是可接受系统。系统性能必须和输入分布、失败路径、内存峰值和提交语义一起报告。

Listing 1.19 一次运行报告至少应回答的问题

```

input:
    bytes, records, accepted, rejected, checksum
execution:
    workers, chunk size, queue wait, run time, blocked time
memory:
    hot bytes, cold bytes, peak bytes, allocation count
hardware:
    cache/TLB/branch indicators when available
output:
    blocks, attempts, manifest entries, checksum
failure:
    retries, late completions, duplicate attempts, recovery actions

```

这份报告把输入、执行、内存、输出和失败边界变成实现门禁。优化不是只让某个函数变快，而是让系统在更大规模、更差输入和可恢复故障下仍能解释自己的结果。

从一行 C++ 推到完整证据链 把 `counts[event_type]+=value` 写成执行账本以后，还要继续把证据链补全。第一层是源码语义：`event_type` 是否已经过输入合同验证，`value` 是否参与错误策略，`counts` 是全局表、worker 本地表，还是某个 shard 的局部表。第二层是编译后的机器形态：热循环里是否有边界检查、哈希函数、函数调用、容量增长、异常路径、`lock` 前缀或系统调用。第三层是地址形态：`event_type` 来自连续列还是对象字段，`counts` 的槽位是否连续，写入是否落在多个核心共享的 line。第四层是运行证据：不同输入分布、线程数和工作集规模下，阶段时间、反汇编、计数器或降级证据是否支持同一个解释。第五层是语义审计：优化后 `accepted`、`rejected`、`diagnostics`、`checksum` 和 `manifest` 是否仍然一致。

Listing 1.20 从一行更新到证据链的展开

```
source semantics:
    counts[event_type] += value

machine shape:
    load event_type
    compute counter address
    load old counter
    add value
    store new counter
    or execute locked read-modify-write if shared atomic

address shape:
    event_type column: contiguous or object field
    counter table: local array, shard, hash bucket, or global atomic
    cache line: private, false-shared, or hot shared

runtime evidence:
    stage time, scaling curve, disassembly summary,
    counter evidence when available, checksum and diagnostics
```

这条链的价值在于防止跳跃。不能从源码一行直接跳到“需要无锁”，也不能从一次 cache miss 直接跳到“数据布局错误”。若机器形态里没有共享写，原子知识暂时用不上；若地址形态是连续流式扫描，cache miss 高可能只是带宽账本；若阶段时间显示 `write+commit` 才是长尾，继续优化 `parser` 热循环就只是局部漂亮。

热循环审查模板 任何准备被优化或并行化的热循环，都可以先过一张审查表。表里的问题不是代码风格，而是机器会看见什么。

审查项	要回答的问题	常见证据
语义身份	输入、输出、错误和提交事实是否稳定	reference、oracle、checksum、RecordId
控制流	正常路径是否短而稳定，冷路径是否离开热循环	反汇编、分支输入矩阵、错误率对照
地址流	load/store 是否连续、随机、跨页或依赖前一次 load	layout report、AccessTrace、规模阶梯
写入归属	写入 line 是否被单线程、单 worker、多个核心或多个节点修改	partial 布局、padding 对照、扩展曲线
容量边界	热代码、热数据和活跃页是否超过近端结构	函数大小、工作集、TLB 覆盖估算
等待位置	时间耗在运行、锁、I/O、队列、缺页还是提交	StageReport、trace、off-CPU 指标

这张表不要求一次拿到所有硬件计数器。普通用户环境、容器或虚拟化环境中，很多 PMU 事件不可用，仍然可以通过反汇编摘要、输入对照、阶段时间、checksum 和扩展曲线建立较弱但诚实的结论。证据不足时，结论就写窄；不要把推断伪装成已经测到的硬件事实。

1.9 执行与等待：从一个循环到多个工作者

单线程 reference 的执行顺序很简单：读一行，解析一行，更新一次计数。并行化的诱惑也很直接：把输入切成多个 split，让多个 worker 同时处理。真正的问题是，共享状态和等待点会随之出现。

最坏的并行化方式是所有 worker 直接更新同一个全局计数表。若每次更新都拿一把全局锁，热路径会变成“解析一点点，等待一次锁”；若每个 key 使用一个原子计数器，热门 key 会让多个核心争抢同一条 cache line；若每个 worker 写本地 partial，热路径变快了，但最后必须合并 partial，并且要定义什么时候读取全局进度。

这不是三种 API 的选择题，而是三种成本形态：

方案	购买的能力	付出的成本
全局锁	语义直观，容易保持不变量	热路径串行化，竞争时进入等待
全局原子	消除 mutex 阻塞路径	热点 cache line 迁移，扩展性差
线程本地 partial	热写本地化，减少共享	需要合并阶段和来源身份

等待点必须命名 系统变慢时，不能只说“多线程不快”。要问等待在哪里：worker 等锁，reader 等队列空间，writer 等磁盘，driver 等所有 split 完成，还是 reducer 等某个长尾 key。等待点一旦被命名，就可以被测量：等待次数、等待时间、队列最大水位、worker 空闲比例、任务运行时间和合并时间。后续线程、锁、运行时和 I/O 章节都会反复使用这个习惯。

背压先作为现象理解 背压 (backpressure) 表示下游处理不过来时，上游必须被限速。这里先给直觉：如果 reader 和 parser 很快，writer 很慢，而中间队列无界，短时间 benchmark 可能很好看，长时间运行却会不断堆积内存。一个有界队列把“下游慢”变成显式等待。背压不是失败，而是系统保护自己。

1.10 提交事实：文件存在不等于结果生效

只要系统写输出，就必须区分临时事实和提交事实。假设一个 worker 处理 split 1，写出一个临时 block 后进程崩溃。重启后 driver 重新调度 split 1，第二次又写出一个 block。如果 reducer 只是扫描目录，它会看到两个 block，并把

同一段输入算两次。目录里有文件，只说明某次物理执行曾经写过东西；它不说明这个结果已经被业务接受。

manifest（提交清单）是记录哪些输出已经生效的控制面事实。第一版可以把它理解为一张小表：

Listing 1.21 manifest 的语义伪结构

```
ManifestEntry:
  logical_split_id
  accepted_attempt_id
  block_path
  block_checksum
  record_count
  commit_time
```

Reducer 不扫描临时目录，而是读取 manifest 中被接受的条目。旧 block 可以稍后清理；清理失败不应改变业务结果。这个边界在单节点 I/O 中表现为可靠写入、短写、fsync 和 completion；跨进程或跨机器后又会表现为 attempt、迟到响应、epoch 和分布式提交。这里先固定一句话：输出文件是数据面事实，manifest entry 是控制面事实；真正决定结果是否生效的是控制面。

Attempt 是物理执行，不是逻辑任务 attempt（执行尝试）是同一个逻辑任务的一次物理执行。Split 1 可以有 attempt 1 和 attempt 2，但逻辑贡献只能接受一次。没有 attempt id，系统无法区分“同一任务重跑”与“两份不同任务”。分布式系统里，attempt 会更重要，因为超时不代表对方没有执行，旧响应也可能在重试之后才到达。

1.11 第一层最小账本

到这里，Compute Systems Engine 的第一层账本已经足够清楚：

边界	要回答的问题	最低证据
输入合同	什么是一条合法记录	正常输入和坏行输入
Reference	什么叫算对	手工结果和稳定输出
RecordId	错误和贡献来自哪里	文件、版本、offset、行号
Partial	局部结果怎样合并	两个 split 的手工贡献
数据路径	字节怎样进入热路径	reader/parser/aggregator 图
等待点	谁可能阻塞谁	锁、队列、writer 慢的命名
Manifest	什么结果已经生效	重复 attempt 只接受一次

这张账本不是为了装饰，而是后续机制的索引。遇到硬件机制，回到数据路径；遇到同步机制，回到等待点；遇到运行时机制，回到 split、partial 和任务状态；遇到 I/O 或分布式机制，回到 manifest 和 attempt。这样内容不会散成并列概念。

1.12 版本阶梯：同一个系统怎样被迫长大

只把机制按章节排列，容易产生一种错觉：先学 cache，再学锁，再学 I/O，好像每个主题都是独立知识块。真正的系统不是这样长出来的。它通常先有一个最小正确程序，随后在规模、输入分布、线程数、故障和恢复要求下反复暴露新问题。每次升级都应由一个失败推动，并留下新的接口、报告字段和回归样例。Compute Systems Engine 的贯穿路线可以写成一条版本阶梯。

版本	新能力	暴露的失败或成本	新增证据
V0	顺序 reference	结果定义不清，坏行策略含糊	输入合同、oracle、diagnostics
V1	RecordId 和 split	切分后错误无法定位，partial 来源不明	input generation、offset、split checksum
V2	热字段 batch	胖对象搬运冷字段，TLB 覆盖差	MemoryShape、hot/cold bytes
V3	热循环审查	parser 被分支、取指和随机 key 拖住	反汇编摘要、输入族、stage time
V4	线程本地 partial	全局锁或原子让热门 key 争一条 line	partial 来源、合并 checksum
V5	同步合同	队列关闭、等待谓词和背压不明确	QueueContract、wait trace
V6	原子和任务状态	完成计数不能发布结果，CAS 失败路径不清	AtomicProtocolNote、TaskState
V7	scan/partition	输出位置靠抢，失败后不知道哪些字节有效	stage ledger、attempt、range checksum
V8	任务运行时	worker 阻塞等待后继，ready 任务无人执行	WorkerSnapshot、ready wait、helping trace
V9	可靠 I/O 和 manifest	文件存在不等于结果生效，重试会重复提交	ManifestEntry、recovery audit
V10	单节点分布式模拟	超时、迟到响应和旧 epoch 让控制面分叉	MessageEnvelope、epoch、checkpoint

这张表有两个用处。第一，它阻止概念空降。比如 `AtomicProtocolNote` 不是因为原子章节需要一个名词，而是因为 V6 中“完成计数不能发布结果”的反例要求把原子变量的角色写清；`ManifestEntry` 不是 I/O 章节的格式偏好，而是 V9 中“文件存在却不能说明结果生效”的恢复边界。第二，它让每次升级都有可验收的出口：不是“代码更复杂了”，而是某个旧失败被新证据封住。

V0：顺序 reference 必须先完整 V0 的目标不是快，而是把语义固定。它读取输入，按合同解析，统计 key，稳定排序输出，记录坏行。若这里没有定义 value 溢出、空 key、字段过多、最后一行无换行、输出排序和错误摘要，后面任何优化都会把未定义行为放大。V0 的报告必须包含 `accepted_records`、`rejected_records`、`counts_checksum` 和 `diagnostics_checksum`。这些字段以后不消失，只是被并行、I/O 和分布式路径继续携带。

Listing 1.22 V0 需要先钉住的语义

```
input contract:
  exact fields, bad-line policy, integer range, final newline rule

reference output:
  stable key order
  counts checksum
  diagnostics with record identity

oracle:
  exact match for integer counts
  exact match for diagnostics checksum
```

如果 V0 只输出一张 counts 表，而不输出 diagnostics，V4 的多线程 parser 就可能吞掉坏行；如果 V0 不规定排序，V7 的 partition 输出就可能每次顺序不同；如果 V0 不规定溢出，V13 的 reduce 合并就无法判断大输入是否仍然正确。Reference 的慢是可以接受的，含糊不可接受。

V1: 记录身份比线程更早出现 很多工程会直接从顺序程序跳到多线程。更稳的顺序是先引入 `RecordId` 和 `InputSplit`，即使仍然单线程执行。原因很简单：切分输入首先是语义问题，其次才是并行问题。`Split` 边界落在半行中间怎么办，错误行是全局第几行还是 `split` 内第几行，重试时怎样证明处理的是同一段字节，这些问题在线程出现前就已经存在。

Listing 1.23 V1 的 `split` 审计字段

```
InputSplit:
    input_generation
    file_id
    byte_begin
    byte_end
    first_record_id
    record_count_estimate
    split_checksum

Partial:
    split_id
    attempt_id
    accepted_records
    rejected_records
    counts_checksum
```

V1 的回归样例应包括：`split` 从行边界开始、`split` 从行中间开始、最后一行没有换行、同一个输入用不同 `split` 大小切分。所有样例最终 `counts` 和 `diagnostics` 必须与 V0 一致。这个阶段通过后，后面再加线程时，线程只是执行 `split` 的实体，不再决定记录身份。

V2: 热字段 `batch` 从字节预算中推出 V2 把解析结果放入 `HotBatch`。热字段是聚合阶段反复访问的事实，例如 `record_id`、`event_type`、`value`；冷字段是错误诊断、原始行、用户字符串和调试上下文。冷热分离不是为了追求某种数据结构风格，而是为了让 CPU 在聚合阶段少搬不参与计算的字节。V2 的报告要写清每条记录的热字节、冷字节、总字节、每页有效记录数和 `batch` 大小。

Listing 1.24 V2 的 `MemoryShape` 摘要

```
MemoryShape:
    raw_input_bytes
    hot_column_bytes
    cold_payload_bytes
    bytes_per_hot_record
    estimated_records_per_4k_page
    batch_record_count
    batch_hot_bytes
```

若 V2 比 V0 慢，不应立刻否定 `batch`。要先看它是否保留了太多冷字段，是否引入了额外分配，`batch` 是否太小，字典编码是否把字符串随机访问提前变成了新的瓶颈。布局实验不能只给耗时，它必须解释地址流怎样变化。

V3: 热循环必须能被机器证据解释 V3 开始审查 `parser`、`encoder` 和 `aggregator` 的热循环。这里的目标不是手写复杂汇编，而是避免从 C++ 语句直接跳到硬件结论。一个可信的热循环报告至少包含输入族、编译器选项、反汇编摘要、阶段时间、可用计数器或替代证据、结论边界。

Listing 1.25 V3 HotKernelReport 的最小字段

```
HotKernelReport:
  kernel_name
  input_family
  compiler_and_flags
  disassembly_summary
  bytes_read
  bytes_written
  branch_shape
  counter_evidence_or_reason_unavailable
  stage_time_ns
  conclusion_boundary
```

输入族必须攻击不同机器假设：全合法短行、全合法长行、随机坏行、热点 key、均匀 key、超长 key、空输入、小输入和大输入。若只测一份均匀随机输入，热循环报告无法区分分支问题、cache 问题和 key 分布问题。V3 的核心习惯是：任何性能结论都要能被输入变化推翻或支持。

V4：线程本地 partial 先于无锁结构 V4 的朴素并行方案会暴露共享写：多个 worker 更新同一张全局表。全局 mutex 正确但会排队；全局 atomic 正确但热门 key 仍争 cache line；worker 本地 partial 把高频写留在本地，最后再合并。这个改造之所以靠前，是因为它不需要复杂无锁协议，却能解决最常见的共享写成本。

Listing 1.26 V4 partial 的正确性账本

```
for each split attempt:
  build local partial
  keep split_id and attempt_id
  record accepted/rejected
  compute partial checksum

merge:
  sort or order partials deterministically
  accept each logical split once
  compare final checksum with reference
```

Partial 不是临时数组那么简单。它必须带来源身份，否则 V9 的重试会把同一个 split 的两个 attempt 都合并进去。性能结构和恢复结构在这里第一次相遇：本地 partial 减少共享写，同时让每份贡献可命名、可去重。

V5 到 V8：等待从线程栈移到状态表 有界队列、条件变量、任务图和 worker trace 的共同目标，是把等待从“某个线程卡住了”变成可观察状态。V5 的队列要写清容量、等待谓词、关闭模式和背压；V6 的原子状态要写清发布关系和生命周期；V7 的 scan/partition 要写清阶段状态和输出范围；V8 的运行时要写清 ready、running、blocked、helping 和 parked。

Listing 1.27 等待状态的统一字段

```
waiter:
  worker_id or task_id
  waiting_for
  wait_begin_ns
  wait_reason
  wake_reason
```

```

state_rechecked

task:
  predecessor_count
  ready_time
  start_time
  finish_time
  terminal_state

```

这组字段让第 10 章的条件变量、第 11 章的原子 wait、第 14 章的 worker park 都共享同一条纪律：状态才是事实，通知只是提示状态可能变化。任何等待如果不能写出谓词或目标状态，都还没有进入可靠系统。

V9 和 V10: 提交事实离开进程内存 可靠 I/O 和分布式模拟把事实推出进程内存。V9 中, write 成功不等于 durable, 不等于 manifest 接受; V10 中, 远端执行成功不等于 Driver 已接受, 超时不等于远端没有执行。于是 attempt、manifest、epoch、checkpoint 都从反例中出现。它们不属于“分布式专用名词”, 而是同一个提交问题跨过进程、文件系统和网络后的形态。

Listing 1.28 V9/V10 提交链

```

physical execution:
  worker runs one attempt
  data block is written
  completion may be delayed, duplicated, or lost

control-plane acceptance:
  driver checks task, attempt, epoch, checksum
  manifest accepts at most one attempt per logical task
  checkpoint accelerates recovery but does not override manifest

```

这条链完成后, Compute Systems Engine 才从“一个能跑的并程序”变成“一个能解释自己结果的计算系统”。它不是靠复杂度取胜, 而是靠每一层都有权威事实和可复查证据。

1.13 怎样把底层账本用于后续机制

第一层账本真正的用处, 是把后续每个新名词都接回一个已经看见的问题。若后面讨论流水线, 问题不是“流水线是什么”, 而是 parser 热循环为什么需要连续供应指令和数据; 若讨论 cache, 问题不是“缓存有几级”, 而是 event key 和计数表的地址序列为什么会让核心等待; 若讨论锁和原子, 问题不是“哪个 API 更快”, 而是多个 worker 写同一个计数槽时, 哪条 cache line 在核心之间移动; 若讨论 manifest, 问题不是“清单格式怎样设计”, 而是重复 attempt 为什么不能靠扫描目录解决。

概念进入前先问触发条件 每个机制都有触发条件。I-cache 只有在热代码取指供应跟不上时才成为主要解释; TLB 只有在页覆盖不足或随机页访问增多时才进入瓶颈; 分支预测只有在控制流不稳定时才解释吞吐波动; NUMA 只有在线程和内存页跨节点时才是核心问题; 原子只有在共享状态足够窄时才比锁更容易证明。若没有触发条件, 直接套用名词会把系统分析变成猜。

Listing 1.29 新概念进入前的三问

```

what concrete failure or cost exposed this concept?
which low-level event can observe or refute it?

```

what interface or report field must change because of it?

这三问会反复出现。比如“预取”进入之前，要先证明未来地址可预测、当前存在内存等待、预取距离有独立工作可隐藏延迟；“线程池”进入之前，要先证明每个请求创建线程或阻塞污染计算线程；“共识”进入之前，要先证明多个控制面副本可能给出互相矛盾的提交事实。这样机制不是按目录排列，而是从故障和成本中生长出来。

小任务必须保留人工可审计路径 后续系统会变大，但一百行、一千行和一亿行应共享同一条审计路径。小输入可以手算 counts；中等输入可以打印 split partial 和 manifest；大输入只保存 checksum、统计摘要和抽样 trace。规模变化后，证据形式变了，身份链不应变：input generation、record id、split、attempt、block、manifest entry、最终输出 checksum 必须仍能连起来。

若某次优化让小输入也无法解释，例如错误行只能报“某个块坏了”、重复 attempt 只能靠删除目录解决、线程卡住只能靠猜测 worker 状态，那么大输入的吞吐数字就没有工程价值。系统能力不是只在大规模时出现，它应该在最小可审计样本上就能说清楚。

1.14 从机器最小事实推导系统边界

前面的账本已经列出 I-cache、D-cache、TLB、分支、store buffer、coherence、manifest 等词。为了避免这些词变成孤立名词，需要再往下一层：CPU 不是按“日志统计”执行，也不是按“类和函数”执行；它反复推进一组非常小的机器事实。程序计数器给出下一段指令地址，寄存器保存少量当前值，load/store 在地址空间中搬运字节，分支改变下一条指令的位置，异常把控制权交给内核，外设和文件系统把持久状态放在 CPU 之外。所有高级边界都要落回这些事实。

程序计数器推出取指边界 一个核心当前执行到哪里，由程序计数器和预测路径决定。若热循环短而连续，前端能连续取入指令；若每条记录都进入多层虚调用、大 switch、异常路径或格式化日志函数，程序计数器就不断跳到分散地址。于是“热路径要短”不是风格偏好，而是从程序计数器和 I-cache 推出来的边界。热代码越集中，取指越容易；冷代码越远离热循环，正常输入越不容易被罕见路径拖累。

同一条 parser 逻辑可以有两种机器形状：

Listing 1.30 两种取指形状

```
inline-all-paths:
  scan character
  branch to normal field update
  branch to rare malformed-line formatting
  branch to allocation-heavy diagnostic object
  branch back to scan

split-hot-cold:
  scan character
  update compact parser state
  append compact error code when needed
  continue scan
  build full diagnostic after the hot scan finishes
```

两者都能表达错误处理。差别是第一种把罕见路径的指令字节塞进热区域，第二种先记录足够身份，再把完整诊断延后。由此自然推出冷路径分离：不是不处理错误，而是把错误语义和热取指成本分开。

寄存器推出批量和局部状态 寄存器非常快，但数量有限。一个循环若每条记录

只维护少量局部状态，例如当前字段起点、字段编号、局部计数和 checksum，核心可以把这些值留在寄存器里。若每条记录都调用复杂对象方法、读写全局状态、构造字符串或更新共享表，寄存器很快被地址、临时对象和调用约定挤满，热循环就转向更多内存访问。

这解释了为什么 batch 是底层概念。Batch 不是把输入随便切块，而是让一小段数据和一组局部状态在寄存器、L1/L2 和 TLB 可承受的窗口内完成足够多工作。一个 batch 太小，函数调用、任务调度和提交开销占比高；一个 batch 太大，局部状态和活跃页数膨胀，恢复边界也变粗。合理 batch 来自寄存器、缓存、页、任务和恢复共同约束。

Load/store 推出地址身份 CPU 通过 load 读地址，通过 store 写地址。源码里一个 `record.event_type` 字段，在机器层面只是某个基地址加偏移。若这个基地址属于胖对象数组，连续记录之间的有效热字段相隔很远；若这个基地址属于 `event_type[]` 热列，连续 load 可以自然前进。于是 RecordId 和 HotBatch 的组合不是抽象建模，而是在两种需求之间分工：RecordId 保留语义身份，HotBatch 提供短地址流。

地址身份还解释了为什么指针不是万能引用。一个指针只在当前进程当前地址空间中有效，难以写入 manifest、trace、文件和网络消息；一个带 generation 的索引或 record id 可以跨阶段、跨进程、跨重试保持可比较。分布式章节出现 attempt、epoch 和 manifest 时，并不是换了话题，而是把“地址不可作为长期身份”这件事推到更远边界。

分支推出输入族 分支预测器只看到历史，不理解业务。稳定格式日志会让 parser 的字段分支高度可预测；随机坏行会让错误路径变热；短行和长行混合会改变循环回边、字段结束和缓冲边界的分布。因此输入合同必须变成输入族，而不是只放一份默认样本。规则输入、随机字段、错误密集、短行、长行、热点 key、均匀 key，分别攻击不同机器假设。没有输入族，分支、缓存和并发结论都容易只适用于某个偶然分布。

异常和系统调用推出 I/O 边界 用户态热循环可以连续推进；系统调用会进入内核；page fault 会把一次普通访问变成异常处理；write 可能只把字节放入 page cache；fsync 会把部分持久化等待拉到当前路径。由此推出 I/O 边界必须单独命名。若计时范围包含 page fault 和 fsync，它测的是端到端或冷启动；若计时范围排除了这些，它测的是热循环。二者都有效，但不能互相解释。

共享写推出层级归约 单核时，计数可以留在寄存器或本地 cache；多核时，多个核心写同一位置就需要一致性协议协调。高频共享写会把热循环变成所有权迁移。于是全局计数器、全局队列、全局进度条和全局统计表都要先被怀疑。分层归约从这条事实推出：worker 本地写，节点内合并，跨节点低频合并，最后提交稳定结果。这个结构既是性能结构，也是恢复结构，因为每层 partial 都能带来源身份。

机器事实	直接问题	推出的系统边界
程序计数器取指	热代码是否连续，冷路径是否污染前端	fast path / slow path 分离
寄存器有限	局部状态能否留在核心附近	batch、局部 partial、短 kernel
load/store 地址	热字段是否形成短地址流	HotBatch、冷热拆分、RecordId
分支预测	输入历史是否稳定	输入族、错误率矩阵、branch 报告
page fault / syscall	何时进入内核和外设路径	冷/热计时边界、I/O 阶段
cache line 所有权	谁在高频写同一 line	线程本地、分片、分层归约

这张表给后续内容提供进入顺序。先观察机器事实暴露的成本，再命名机制，再改接口和报告字段。机制如果不能回到某个机器事实或语义边界，就暂时没有进入热路径的资格。

1.15 从五行输入推到资源预算

一个小任务如果只停在“能算出结果”，它无法支撑后续复杂机制。必须继续追问：同一份语义结果，需要消耗哪些机器资源；哪些资源随记录数线性增长；哪些资源随 key 基数、错误率、线程数、输出块数量或重试次数增长；哪些资源一旦超过边界，程序会从单纯计算变成等待、争用或恢复问题。这样，后续出现的 batch、partition、partial、manifest 和 backpressure 就不是抽象词，而是资源预算被逼出来的结构。

先把五行输入扩成一百万行。若每行平均 48 字节，输入字节大约 48 MiB。顺序读取时，主要成本是把文件页、用户 buffer、parser 热循环和输出状态串起来；若每条记录被解析成 160 字节胖对象，热数据立刻放大到 160 MiB，还不包括字符串堆分配、哈希表节点和错误对象。若事件名只有 32 种，聚合结果可以是一小段计数数组；若事件名有几百万种，字典、哈希表或排序临时空间会成为主体。语义任务仍然是“统计事件”，机器任务已经变成“搬运多少字节，触碰多少页，写多少 cache line，持有多少未提交输出”。

Listing 1.31 从语义量到机器预算的粗账本

```
records:
  count = 1,000,000
  average raw line = 48 bytes
  raw input ~= 48 MiB

fat object layout:
  LogRecord ~= 160 bytes
  hot fields may be less than 16 bytes
  object array ~= 160 MiB, plus heap strings and diagnostics

hot column layout:
  record_id ~= 16 bytes
  event_type ~= 4 bytes
  value ~= 4 bytes
  hot columns ~= 24 MiB, cold store kept out of aggregation

aggregation state:
  small key set -> compact counter array
  large key set -> dictionary, hash table, partition or sort
```

这份粗账本不要求完全准确，它的作用是阻止早期设计把所有东西都叫“数据”。原始输入、热字段、冷诊断、字典、partial、输出块、manifest、指标 trace，生命周期和访问方式都不同。把它们混在一个对象图里，等于要求 CPU 在每个阶段搬运所有阶段的字节。把它们拆成稳定身份和阶段化布局，后续每个阶段才有机会只处理必要事实。

预算单位必须和硬件单位相遇 资源预算不能只写“有多少条记录”。硬件的关键单位包括 cache line、页面、core、hardware thread、队列槽位、文件页和持久化块。记录数要转换成这些单位，性能判断才有落点。例子如下：

语义单位	机器单位	设计问题
一条记录	若干输入字节、字段 span、一个 record id	热字段是否能连续保存，冷字段是否可回查
一个 event key	字符串字节或整数 id	是否先编码，编码表版本如何记录
一个计数更新	一个计数槽所在 cache line	是否共享写，是否需要局部 partial
一个 split	字节范围和完整记录边界	是否能重试，partial 是否能去重
一个 output block	page cache 脏页、文件名、checksum	write 返回后是否已经可恢复
一次 retry	attempt id 和旧结果可见性	迟到结果是否会污染 manifest

这张表说明，系统边界不是先由架构图决定，而是由单位转换决定。若一个 event key 在热循环里仍是字符串，聚合阶段就要反复读取字符串字节和哈希表结构；若它已经变成整数 id，热循环主要读整数列和计数槽。若一个 output block 只是文件名，恢复阶段无法知道内容是否完整；若它带 checksum、attempt 和输入范围，manifest 才能判断是否接纳。

小输入也要保留规模变量 五行输入可以手算，但报告中仍应保留规模变量。比如 `record_count`、`raw_bytes`、`accepted`、`rejected`、`key_count`、`split_count`、`partial_count`、`output_bytes`。这些字段在五行时很小，在一亿行时变大；字段名称不变，审计路径就不变。若小输入报告只打印 counts，大输入报告另写一套吞吐摘要，后续无法确认两者是否在说同一个状态机。

Listing 1.32 最小输入也保留的规模字段

```
RunShape:
  input_generation
  record_count
  raw_bytes
  accepted_records
  rejected_records
  key_count
  split_count
  partial_count
  output_bytes
  manifest_entries
```

RunShape 不是性能模型，而是规模坐标。它可以和分支输入族、活跃页数、线程数、NUMA 节点、outstanding I/O、dirty bytes 和恢复窗口相连。缺少规模坐标，任何“变快”或“变慢”都难以复查。

从预算推出 batch 为什么需要 batch？不是因为批处理听起来高级，而是因为单条记录作为执行单位太小。每条记录都提交 task，会让队列、调度、函数调用、计时、指标和错误处理开销压过真正解析工作。把太多记录塞进一个巨大 batch，又会让热字段、错误摘要、partial 和输出 buffer 跨过缓存、TLB、恢复和延迟边界。Batch 是在两种错误之间选一个可控窗口：足够大，让固定开销摊薄；足够小，让工作集、等待时间和失败重做可控。

Listing 1.33 batch 大小的推导口径

```
batch is too small when:
  queue operations, task scheduling, and reporting dominate useful parsing

batch is too large when:
  hot columns no longer fit the intended cache/TLB window
```

```
one bad block delays too much downstream work
retry repeats too many already-parsed records
output buffers create too much dirty data
```

```
reasonable batch:
  has stable record boundaries
  has compact hot columns
  has bounded diagnostics
  has a partial that can be merged and audited
```

这个推导把 batch 同时连接到硬件、运行时和恢复。硬件侧关心热字段和活跃页；运行时侧关心 task 粒度和队列；恢复侧关心失败重做和 manifest 粒度。一个只按“每批一万条”硬编码的设计无法解释这些交换。更好的报告会说明 batch 选择来自哪条预算约束，并记录当前实验环境和输入下的实际形状。

从预算推出 partition 为什么需要 partition？当 key 集合大到本地计数表无法稳定留在缓存或 TLB 覆盖范围内，或者当多线程共享表出现高频写争用，直接聚合会把每条记录都变成随机访问和同步问题。Partition 把记录按 key 范围或 hash 分成多个较小集合，让每个集合在更短时间内面对更小工作集。它不是免费优化，因为 partition 本身需要写出 block、保存边界、处理倾斜、管理 manifest，并在后续阶段读回。

Partition 的正确入口是“工作集太大或共享写太热”，不是“所有大数据都要 shuffle”。若 key 基数很小，局部数组计数可能更好；若 key 分布极度倾斜，简单 hash partition 会制造热点 partition；若输出设备慢，过多 partition block 会把 I/O 放大。这个概念出现之前，必须先有 key 分布、局部表大小、共享写位置和输出块预算。

触发条件	partition 购买的能力	新增成本
key 工作集超出近端缓存	缩小每次聚合的活跃集合	多次写读、block 元数据、排序或合并
全局表共享写过热	让不同 worker 处理不同 key 范围	倾斜、负载均衡、最终合并
单机内存不足 需要跨节点扩展	把中间状态分段落盘 为 shuffle 和 reduce 建立边界	I/O 背压、恢复扫描、临时空间 网络、重试、attempt、manifest

从预算推出 manifest 为什么需要 manifest？因为输出不是一个瞬间事实。程序可能写出临时文件，write 返回成功，page cache 变脏，fsync 成功，rename 后文件名可见，目录项持久化，最后 run 的结果才可被恢复逻辑接受。重试又会让同一个逻辑 split 出现多个 attempt 输出。没有 manifest，系统只能扫描目录并猜测哪些文件属于有效结果；这在崩溃、迟到 completion、重复执行和部分写入面前不可靠。

Manifest 的最小职责是把“哪些输出对象被接受”写成可恢复事实。它不只是文件列表，还要包含输入 generation、split 身份、attempt、范围、字节数、checksum、状态和提交顺序。这样，恢复时可以拒绝旧 attempt、识别缺失 block、发现 checksum 不符、判断是否需要重算。Manifest 来自提交预算，不来自格式偏好。

Listing 1.34 manifest entry 的最小责任

```
ManifestEntry:
  run_id
  input_generation
  split_id
```

```

attempt_id
output_path
byte_count
checksum
commit_state
committed_at_log_position

```

这个结构的理由很简单：文件存在不等于输出已生效；attempt 成功不等于逻辑任务应被接纳；恢复扫描不应靠文件名猜测状态。

1.16 一条记录的完整剖面

把一条合法日志记录从输入文件推进到最终输出，可以得到一个比早期硬件账本更完整的剖面。这个剖面不用于精确预测纳秒，而用于检查有没有漏掉边界。

Listing 1.35 一条记录从输入到提交的剖面

```

input layer:
  file page becomes visible through read or mmap
  bytes enter input buffer or mapped page

parse layer:
  frontend fetches parser instructions
  D-cache loads input bytes
  branches classify separators and fields
  parser emits record_id, event_name span, value, error code

encode layer:
  dictionary maps event_name bytes to event_type
  report keeps dictionary version and checksum

aggregate layer:
  worker reads event_type and value from hot batch
  worker updates local partial counter
  local partial keeps split and attempt identity

merge layer:
  node or worker partials are combined in stable order
  output counts are sorted or encoded deterministically

commit layer:
  block bytes are written with checksum
  manifest accepts exactly one attempt for each logical split
  final report records input, output and commit identities

```

每一层都有可丢失的事实。输入层可能混淆冷读和热读；parse 层可能吞掉坏行；encode 层可能让同一个整数在两个字典版本中代表不同事件；aggregate 层可能重复接受同一个 attempt；merge 层可能输出不稳定顺序；commit 层可能把临时文件误当已生效结果。系统机制的价值在于保护这些事实，而不是增加术语数量。

可观察字段从剖面中来 报告字段不应凭感觉添加。剖面中每个会影响结果或解释的边界，都应有字段承接：[input_checksum](#) 保护输入身份，[accepted/rejected](#) 保护解析语义，[dictionary_checksum](#) 保护编码语义，[split_id/attempt_id](#) 保护局部贡献身份，[partial_checksum](#) 保护合并前状态，[manifest_entry](#) 保护提交事

实，`stage_time` 保护性能解释。

Listing 1.36 剖面导出的最小审计字段

```
AuditReport:
  input_name, input_generation, input_checksum
  accepted_records, rejected_records, diagnostics_checksum
  dictionary_version, dictionary_checksum
  split_count, attempt_count, accepted_attempt_count
  partial_count, partial_checksum
  output_checksum, manifest_checksum
  stage_times, unavailable_metrics
```

`unavailable_metrics` 也要记录。硬件计数器、NUMA 信息、page fault 细分、I/O 设备指标在某些环境不可用，报告不能把缺失当作零。不可用本身就是结论边界。

1.17 实验

这一组实验不要求写完整工程，但必须完成一份可复查设计。完整代码放在工程代码卷；原理部分至少应写出下列材料：

习题与作业

实验一：写出日志统计的输入合同。列出合法行、缺字段、字段过多、value 非整数、空 event name 五类输入，说明每类输入进入 `accepted` 还是 `rejected`。

实验二：手算五行正常输入和一份含坏行输入。输出 `accepted`、`rejected`、`counts` 和 `diagnostics`。

实验三：把五行输入切成两个 `split`，写出每个 `split` 的 `partial` 和合并结果。改变 `split` 边界后，解释为什么最终结果仍应一致。

实验四：设计 `RecordId`。说明每个字段解决哪个诊断或恢复问题。

实验五：设计一个重复 `attempt` 场景。写出目录中可能存在的 `block`，以及 `manifest` 中应接受哪一个。

实验六：为一条合法日志记录写硬件事件链。至少包含 `I-cache`、`D-cache`、`TLB`、分支、计数表读取和计数表写入。说明其中哪一项最可能在输入顺序扫描、字符串哈希、热门 key 三种场景中成为瓶颈。

1.18 小结

入口不是 CPU 名词表，而是一个可以手算的小任务。输入合同规定什么能进入系统；`reference` 规定什么叫算对；`RecordId` 让错误和贡献可追踪；`partial` 让输入切分后仍能合并；数据路径让硬件成本有了对象；等待点让线程和队列问题可观察；`manifest` 让输出文件和提交事实分开。后续复杂机制，都应回到这些边界上解释。

下一步把视角缩进 CPU 核心内部。`parser` 和 `aggregator` 已经被定位为热路径，接下来要问：为什么同样处理一批记录，有些循环能持续供给执行单元，有些循环却被依赖、分支预测或取数等待卡住。硬件知识不是抽象背景，它是解释这条数据路径为什么变快或变慢的第一块地基。

工程案例

知识点小案例：一行坏日志怎样逼出输入合同。特例先放到桌面上：输入共有五行，第四行只有 `10:03u003` 两个字段。若程序仍然把它计入 `logout`，说明解析器在猜；若程序直接退出，说明错误策略变成全局失败。正确

做法先手算：accepted 为 4，rejected 为 1，第四行进入 diagnostics，计数表只来自合法行。由这个特例推出规律：输入合同不是文档装饰，而是 reference、测试、错误摘要和后续 split 合并的共同前提。工程上，每个 parser 都要输出 accepted、rejected、record id、error kind 和 checksum，否则后面再谈线程、SIMD 或分布式重试都没有裁判。

第 2 章

取指前端、流水线、乱序执行与分支预测

线上日志解析器曾经遇到过一个很典型的退化：固定格式日志吞吐稳定，换成用户自定义字段后，代码没有改，单核吞吐却下降，尾延迟也抖动。源码层面仍然只是一个循环：读取字节，判断分隔符、数字、引号、转义或非法字符，推进状态，偶尔写出字段边界。若只看 C++ 行数，很容易把问题说成“字符串处理慢”；把输入分布、反汇编和性能计数器放到一起，才会看到真正的因果链：随机分支让预测失败变多，错误路径混进热循环让前端供给变差，状态变量形成循环携带依赖，某些字段统计又把加法串成一条长链。

这里不从“五段流水线”这类静态图开始，而从热循环为什么突然变慢开始。流水线解释为什么一次猜错分支会丢掉已经取入和译码的工作；乱序执行解释为什么多累加器能拆开一条依赖链；取指前端解释为什么 I-cache、ITLB、BTB、micro-op cache 和代码布局会限制一个看似很小的循环；执行端口和 load/store queue 解释为什么源码行数相同，机器看到的工作形状却不同。硬件概念只有能解释一段具体代码的性能异常，才真正有用。

分析沿六条线展开：把一段 C++ 热循环拆成取指前端、I-cache/ITLB、分支预测、依赖链、执行端口、load/store 和退休提交几个可观察问题；区分延迟、吞吐、IPC、前端受限、后端受限、错误投机和内存等待，而不是只说“CPU 没跑满”；用单累加器、多累加器、branchless 计数、指针追踪和冷路径分离解释乱序执行能隐藏什么、不能隐藏什么；根据输入分布判断分支预测是否可能是主因，并说明 branchless、查表、分桶和批处理各自付出的代价；用反汇编、编译器优化报告和 `perfstat/perfannotate` 或替代证据构造可复查的热循环证据包；在 Compute Systems Engine 中形成 `HotKernelReport`，让后续 SIMD、数据布局、线程和任务运行时复用同一套单核证据。

工程案例

核心案例是日志解析热循环。输入有三类：规则日志、随机用户字段和错误密集字段。规则日志让分支预测器容易学习；随机字段让字符类别分支接近不可预测；错误密集字段把本该很少进入的错误路径拉进热循环。每个优化版本都必须输出相同的字段数、错误摘要和 checksum，否则性能比较无效。

先把第一章的同一段更新块放在桌面上：

Listing 2.1 这段热循环的核心内部形状

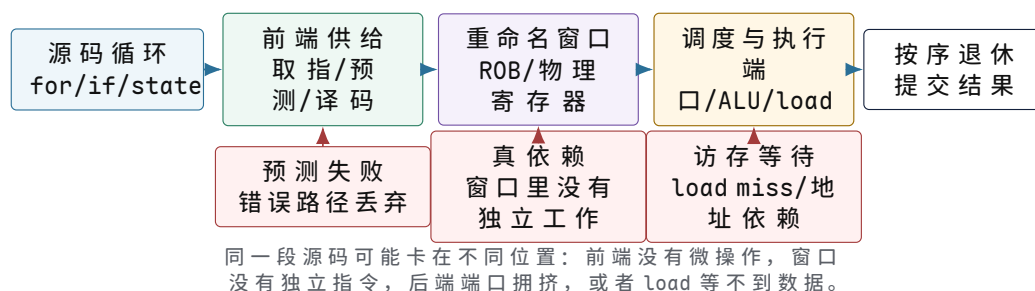
```
B0_loop:
    load input byte or event_type[i]
    classify byte, check field boundary or validity
    branch to normal update or cold error path

B1_update:
    load counter slot
    add value
    store counter slot
```

```
B3_next:
  advance i
  branch back to B0_loop
```

这里关心的不是 `counterslot` 究竟在哪一级 cache，也不是多个核心是否争同一条 line；这些属于数据路径和一致性问题。当前问题只看核心内部：这些基本块的机器码能否连续供给，`valid` 分支是否可预测，`add` 是否形成跨迭代依赖，乱序窗口里有没有独立工作可以覆盖一次慢 load，错误路径是否把冷代码拖进热循环。先把前端、分支和乱序执行与 cache/TLB 分开，才能解释“同一核心怎样消化这段基本块”。

热循环在核心内部的路径



图示：本图要证明的命题是：热循环优化不是“少写几行代码”，而是改变核心内部的供给、依赖和等待形状。

2.1 从现象到假设

性能分析的第一步不是猜硬件部件，而是把现象写成可证伪假设。日志解析器变慢时，可以先提出四个候选解释：输入更大导致内存带宽不足；字符类别更随机导致分支预测失败；错误行增加导致冷路径进入热循环；字段处理函数无法内联导致前端供给不足。四个假设都能让程序变慢，但它们要求完全不同的修复。

若瓶颈是内存带宽，减少几个整数分支可能没有端到端收益；若瓶颈是分支预测，把线程数翻倍只是把低效循环复制到更多核心；若瓶颈是错误路径，`branchless` 改写不如冷热路径分离；若瓶颈是调用边界，数据布局 and API 可能比局部表达式更重要。可靠性能判断的核心，不是知道很多技巧，而是知道哪个技巧对应哪个证据。

四类证据 第一类证据是输入分布。记录每类字符、字段长度、错误比例、短行比例、长字段比例和随机种子。没有输入分布，分支预测分析会变成猜测。第二类证据是二进制形状。反汇编回答热循环是否有调用、间接跳转、额外边界检查、向量指令、栈 spill 和冷路径代码。第三类证据是运行指标。理想情况下记录 `cycles`、`instructions`、`branches`、`branch-misses`、`cache-misses`、IPC 和耗时分布；没有 PMU 权限时，也要保留耗时阶梯、反汇编和输入对照。第四类证据是正确性。优化版本必须和 reference 的 checksum、错误摘要和输出规模一致。

不要先开线程 单核热循环没有看清之前，开线程常常会制造新的噪声。多个线程会引入线程创建、调度、cache line 迁移、内存带宽竞争和 NUMA 放置问题。若单核版本每个字符都被随机分支击穿，八个线程只是让八个核心都浪费在错误投机上；若单核版本每条记录都写共享原子，更多线程还会放大 cache line 争用。线程池和任务运行时必须建立在清楚的单核执行模型上。

核心思想

热循环报告的基本句式是：“在某类输入上，某个二进制形状导致某类硬件等待；某个改写改变了这个形状；指标支持或不支持这个因果链；新的

瓶颈迁移到哪里。”

2.2 取指供给的物理矛盾

CPU 执行的第一步不是“做加法”或“读内存”，而是找到下一批指令。程序计数器 PC 保存下一条指令的地址；若程序只有一条直线，下一条地址似乎就是当前地址加上指令长度。真实热循环不是直线：循环回边要跳回开头，条件分支可能走两边，函数调用会进入另一个地址，返回要回到调用点，异常检查和错误路径会改变控制流，虚调用和 `switch` 甚至让目标地址随数据变化。等分支真正执行完再去取下一条指令，后端会长期没有工作。现代核心必须在答案确定之前先猜下一条 PC，并沿着猜测路径取指、翻译、译码、排队。

取指前端由这个约束推导出来。它不是一个孤立部件，而是一条供给链：预测器猜下一条 PC，ITLB 翻译指令页，I-cache 提供指令字节，译码器把机器指令变成内部微操作，micro-op cache 保存已经译码过的热路径，队列把微操作送进重命名和调度阶段。任何一环供不上，后端执行端口即使空着也没有工作可做。前端性能差的程序常常看起来“算术不多、数据不大”，真正的问题却是核心没有持续拿到正确路径上的微操作。

这个约束来自硬件距离。寄存器文件和执行单元必须在很短的周期内交换操作数；一级缓存使用片上 SRAM，离核心很近，容量不能太大；更下层的缓存更大也更慢；主存容量大，却在芯片外，访问延迟和能耗都高得多。若每条指令都要等主存给出字节，再等页表翻译，再等译码完成，核心绝大多数周期都会空转。现代处理器用更多晶体管、更多预测和更多小型缓存，购买的是一件事：在后端需要工作之前，把正确路径上的内部工作单元准备好。

因此，前端不是“执行之前的准备动作”那么简单。它本身就是一条高吞吐流水线，也有容量、带宽、延迟、冲突和回滚。I-cache 的容量小，意味着热代码足迹必须受控；ITLB 的项数有限，意味着热代码页不能无限分散；BTB、RAS 和方向预测器有历史和表项限制，意味着分支目标和输入分布会改变供给稳定性；译码器有带宽限制，意味着复杂指令和过度展开会让后端拿不到足够微操作。这些限制相互耦合，不能用一个孤立名词解释性能。

时钟周期为什么逼出流水线 一个组合逻辑电路从输入稳定到输出稳定需要时间。若把取指、译码、寄存器读取、执行、访存和提交都塞进一个巨大周期，周期必须长到足以容纳最慢路径，频率会很低。流水线把长路径切成多段，中间用寄存器保存阶段结果，让不同指令处在不同阶段中。这样做提高了吞吐，却引入了新问题：后面的阶段每个周期都需要前面阶段喂给它工作；一旦前端停顿，空泡会沿流水线向后传播。

前端停顿并不总是表现为明显的 cache miss。一个条件分支预测错了，前端可能已经沿错误地址取入多条指令；一个间接跳转目标没猜中，取到的字节属于另一段代码；一个热循环跨越不利边界，每轮能取到的字节数下降；一个函数被过度内联，指令足迹超过前端结构能稳定容纳的范围。流水线让核心能同时处理许多指令，也让“拿错指令”和“拿不到指令”的成本被放大。

前端和后端之间的合同 后端不消费源码，也不直接消费文件中的机器码字节。多数现代乱序核心在内部调度的是微操作：它们描述一次算术、一次地址生成、一次加载、一次存储、一次分支检查或其他更接近执行资源的动作。前端的合同就是持续提供这些微操作，并尽量保证它们来自正确路径。若这个合同破裂，执行端口、乱序窗口、load/store queue 和退休单元都可能闲着。性能分析把这种状态叫 frontend bound，并不是说前端“更重要”，而是说当前程序的限制在供给端。

深入理解

硬件里没有一个单独开关叫“前端优化”。软件能改变的是机器码形状：热路径字节数、分支方向分布、分支目标数量、函数布局、循环体大小、调用边界、异常和日志慢路径是否混在内层。I-cache、ITLB、BTB、译码器

和 uop cache 只是把这些形状转化成等待或吞吐的具体部件。

2.3 从 C++ 到指令字节

C++ 源码不是取指单元的输入。编译器把表达式、分支、内联函数、模板实例和异常边界变成机器指令；链接器把不同目标文件、库函数、跳转表和常量区域放进可执行文件；操作系统把可执行文件的代码段映射到进程虚拟地址空间。热循环最后落在 `.text` 段中的一串机器码字节上，每个字节都有虚拟地址。PC 指向这些地址，前端按预测结果读取字节，译码器再从字节流里识别指令边界和操作含义。

这一步已经足够解释许多误判。一个源码层面的“小函数”可能被内联到十几个调用点，导致代码体积膨胀；一个模板泛型算法可能为多个类型生成多份机器码；一个 `switch` 可能变成比较链、跳转表或间接跳转；一个错误路径如果和正常路径混在一起，格式化字符串、异常对象构造和日志函数引用都会进入同一个取指区域。前端看到的是地址、字节、分支目标和译码任务，不是源码排版。

接上第一册的汇编视角 第一册已经建立了一条基本读法：寄存器保存值，内存操作数由 `base`、`index`、`scale` 和 `displacement` 形成地址，`cmp/test` 设置 `flags`，条件跳转读取 `flags` 改变 `rip`，`call/ret` 通过 ABI 和栈改变控制流。这里不重新讲这些规则，而是继续追问：这些指令字节进入现代核心以后，为什么同样正确的汇编会有完全不同的吞吐。

看一个最小字符分类循环。C++ 版本可能写成：

Listing 2.2 字符分类的源码形状

```
1 std::uint64_t count_separator(std::span<const char> input) {
2     std::uint64_t count = 0;
3     for (char ch : input) {
4         if (ch == ',' || ch == '\n') {
5             ++count;
6         }
7     }
8     return count;
9 }
```

某个优化等级、编译器和目标平台下，它可能落成类似下面的 Intel 风格汇编。这里不是规定编译器必须生成这段代码，而是给出读机器形状的入口。

Listing 2.3 同一循环可能出现的典型汇编形状

```
; rdi = input data pointer, rsi = input size
xor    eax, eax                ; count = 0
xor    ecx, ecx                ; i = 0
.Lloop:
    movzx edx, byte ptr [rdi + rcx]
    cmp    dl, 44               ; ','
    je     .Lhit
    cmp    dl, 10               ; '\n'
    jne    .Lnext
.Lhit:
    inc    rax
.Lnext:
    inc    rcx
    cmp    rcx, rsi
    jne    .Lloop
```


第一册的读法已经能解释这段代码的正确性：`rdi` 是输入基址，`rcx` 是下标，`movzx` 读取一个字节并零扩展，`cmp` 设置 `flags`，`je/jne` 根据 `flags` 改变控制流。这里要把同一段汇编继续向下拆：`jne .Lloop` 是循环回边，前端必须预测它大多数时候会跳；两个字符分类分支的方向取决于输入分布，随机输入会让预测历史更难；`movzx` 的地址是连续递增，数据预取器容易工作；整个循环的机器码如果很小，I-cache 和 uop cache 容易稳定命中。也就是说，一段汇编同时包含控制流形状、地址形状和前端足迹。

汇编不是终点，而是硬件问题的入口 只看汇编仍然不够。上面的 `cmp/jne` 在规则日志中可能非常便宜，因为预测器学到稳定模式；在随机用户字段中可能很贵，因为错误投机反复冲刷窗口。同一条 `movzx edx, byte ptr [rdi + rcx]` 在输入 `buffer` 位于 L1D、LLC、远端 NUMA 节点或触发缺页时，等待完全不同。汇编告诉我们“机器将做什么动作”，硬件章节解释“这些动作在某个输入和拓扑下要等什么”。这一层分析的基本方法就是：源码定语义，汇编定动作，微架构定等待，实验定结论。

代码段、权限和可执行字节 进程地址空间不是一段裸内存。可执行文件会被加载成若干映射区：代码段通常可读、可执行，不可写；数据段可读写，不可执行；共享库有自己的映射区；JIT 或动态加载系统还会在运行中创建新的可执行区域。PC 指向的是某个可执行映射中的虚拟地址。硬件和操作系统必须阻止把普通数据页当指令执行，也要阻止用户态跳进没有执行权限的内核地址。这个安全边界不是性能之外的附加条件，它直接进入取指路径：地址能否翻译、页是否有执行权限、目标是否落在合法代码区，都会影响前端能不能继续供给。

机器码足迹比源码行数更接近硬件成本 源码行数相同，机器码足迹可能完全不同。一个带异常和日志格式化的分支，即使极少执行，也可能把许多冷代码引用拖到同一函数附近；一个模板函数看起来只写了一份，实例化后可能有多份机器码；一个 `std::visit`、虚函数调用或函数指针表，可能把直线控制流变成间接目标集合。反过来，一个看起来较长的表驱动循环，机器码可能短而稳定，分支也少。分析前端时，第一份证据不应是“源码看起来简单”，而应是热函数大小、热循环反汇编、调用边界、跳转目标和异常/日志慢路径位置。

可以用一个小的心智转换检查代码：把函数名删掉，只保留地址区间和跳转边。若热循环每轮在一个短地址区间里顺序前进，偶尔跳回循环头，前端容易稳定供给；若它每条记录都从一个地址区间跳到另一个地址区间，再通过间接目标进入第三个区间，前端要同时解决目标预测、I-cache 足迹、ITLB 覆盖和译码供给。这个转换比争论“抽象是否有开销”更准确，因为抽象只有落成机器码形状之后才有硬件成本。

2.4 I-cache：指令字节为什么要按块缓存

若每次执行循环都从主存读取指令字节，核心会被内存延迟拖住。硬件不能为每个字节都建立一条独立快速通道，于是使用小而快的 SRAM 存最近使用的内容，用较大的下层缓存和内存存完整内容。又因为程序取指常常具有空间局部性：执行了某个地址，接下来很可能执行它后面的地址，硬件不按单个字节搬运，而按固定大小的 cache line 搬运。常见系统里 line 可以理解为几十字节量级的一块连续地址；具体大小属于平台事实，分析时要用机器资料或工具确认。

由此自然得到 I-cache：它是指令侧的一级高速缓存，保存最近取过的指令 cache line。I-cache 的目标不是保存“函数”，而是保存包含某些指令地址的一段字节。只要热循环的指令字节反复落在少数 cache line 里，前端就能从很近的地方取到它们；若热路径和冷路径混杂、过度内联、过度展开或随机分派让代码足迹变大，I-cache 就会更频繁地被迫替换内容，前端供给开始抖动。

为什么 I-cache 必须很小 片上 SRAM 越大，访问越慢、面积越大、能耗越高，也越难放在离取指单元很近的位置。一级 I-cache 必须在很短时间里给前端提供指令字节，所以它不能像主存那样大。容量小不是设计失误，而是速度、面积和能

耗交换后的结果。这个事实会反过来约束软件：热循环不能假设任意多机器码都能留在最近的指令缓存中。

容量小还意味着 I-cache 的替换成本会被热路径放大。一个大函数只要偶尔执行一次，未必重要；一个内层循环每条记录都进入许多小函数、虚调用目标或错误检查块，就会在单位时间内制造大量取指需求。前端不关心这些字节来自“漂亮的抽象”还是“粗糙的手写代码”，它只关心当前预测路径上需要哪些 cache line，以及这些 line 是否还在近处。

从顺序取指推出空间局部性 大多数程序不是每条指令都随机跳转。普通直线代码会从低地址走向高地址；循环会反复取同一段字节；函数调用会跳到一个目标再返回；条件分支通常也有常走方向。硬件利用这种空间局部性，一次搬入整条 cache line。若当前 PC 落在某条 line 的中间，后续若干条指令很可能也在这条 line 或下一条 line 中。这样，单次 miss 的成本可以服务多条后续指令。

空间局部性也有反面。若热路径每隔几条指令就跳到远处，搬入的一整条 line 只用少量字节；若冷错误路径夹在热路径中，前端可能反复把不用的字节带进来；若过度展开让循环体跨越太多 line，单轮工作可能挤掉下一轮马上要用的字节。I-cache 优化的第一原则不是盲目对齐，而是提高热路径字节的有效使用率。

I-cache 不是一个“装代码的盒子” 把 I-cache 只想成“代码缓存”，仍然太粗。取指路径在每个周期面对的是一个更尖锐的问题：当前预测 PC 所在的若干字节能否在很短时间内送到译码或 uop cache 查询位置。这个时间预算很小，因此一级指令缓存必须离前端近、端口足够宽、查找规则简单。容量、相联度和访问延迟之间存在直接交换：容量大可以保存更多代码，但 tag 比较、连线和选择器都会变重；相联度高可以减少冲突，但每次查找要比较更多候选；端口宽可以每周期给出更多字节，但面积和能耗会上升。于是硬件不会为“任意大热函数”买单，软件必须把热路径塑造成前端能连续供给的形状。

从这个物理约束出发，许多经验规则就不再神秘。内联不是绝对收益：它去掉调用和暴露优化机会，同时增加调用点处的机器码足迹。展开不是绝对收益：它减少循环分支和暴露 ILP，同时扩大循环体，消耗 I-cache 和 uop cache 容量。异常路径、日志格式化和诊断构造不能无成本地留在热循环附近：即使它们很少执行，也可能让函数布局、跳转目标和取指块变得更差。前端优化不是“把代码写短”，而是让最常走的 PC 序列短、连续、目标稳定，并让罕见路径在布局和控制流上离开内层循环。

取指缓存也有带宽，不只容量 容量问题回答“热代码能不能留下来”，带宽问题回答“每周期能不能喂饱后端”。一个循环体即使完全命中 I-cache，如果每轮需要跨多个取指块、多个分支目标和复杂译码，前端仍可能供给不足。反过来，一个函数总体很大，但真正热的直线路径很短，冷路径被放远，常走目标稳定，前端可能很好。容量、带宽、目标稳定性和译码复杂度必须一起看。

日志解析器中常见的坏形状是“每个字符都带着全套解释器”。热循环既要判断字符类别，又要处理转义、错误构造、动态字段分派、统计上报和调试开关；每种情况都引入分支或调用。更好的形状是把每字符热循环限制为少数动作：取字节，分类，更新紧凑状态，必要时记录错误事实；完整错误对象、字符串格式化、慢速校验和策略分派放到块边界或慢路径。这样做不是删除功能，而是把不同频率的代码分开，让 I-cache 的小容量服务高频路径。

Listing 2.4 把热路径和冷路径分开的前端账本

```
hot path per byte:
  load input byte
  classify byte
  update compact parser state
  append a small error code only if needed
```

```

cold path per block or per error:
  format diagnostic message
  recover original line context
  update detailed counters
  emit trace records

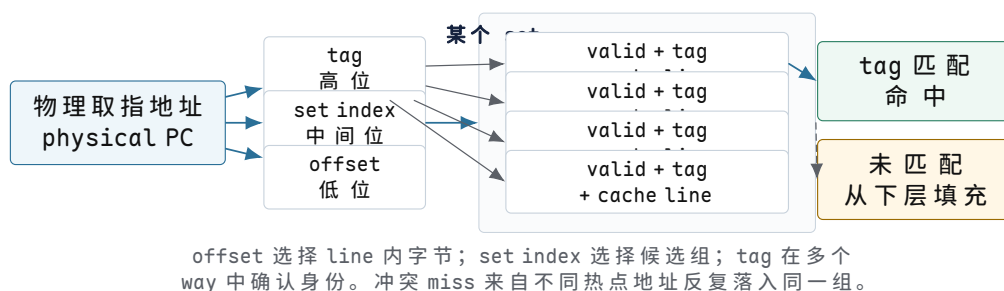
```

这份账本可以直接变成代码审查问题。每个新功能进入 parser 内层前，都要说明它属于每字节、每字段、每记录、每块还是每次作业。频率越高，越应该保持机器码短、分支少、目标稳定；频率越低，越应该从热函数中拆出去。这样定义频率边界，比空泛地说“性能敏感代码要小心”更可执行。

2.5 I-cache 的组相联结构

I-cache 的地址拆分 把 I-cache 想成一张按组存放 cache line 的小表。取指地址先被拆成三部分：低位是 line 内偏移，用来选择这条 line 里的哪几个字节；中间若干位是 set index，用来选择表中的哪一组；高位是 tag，用来确认这一组里的某个条目是不是目标地址。一个 set 里有多个 way，叫组相联。若某个 way 的 tag 匹配并且条目有效，I-cache 命中；否则就要从下层缓存或内存把包含目标地址的整条 line 填进来。

I-cache 查找为什么要拆成 offset、set 和 tag



图示：本图要证明的命题是：I-cache 的命中不是按函数名判断，而是由地址位、组、路和 tag 匹配共同决定。

Listing 2.5 I-cache 查找的抽象伪代码

```

1 CacheLine fetch_instruction_line(Address virtual_pc) {
2   Address physical_pc = itlb_translate(virtual_pc);
3   LineOffset offset = low_bits(physical_pc);
4   SetIndex set = middle_bits(physical_pc);
5   Tag tag = high_bits(physical_pc);
6
7   for (Way way : icache.sets[set].ways) {
8     if (way.valid && way.tag == tag) {
9       return way.line.bytes_from(offset);
10    }
11  }
12
13  CacheLine line = lower_cache_hierarchy.read_line(physical_pc);
14  replace_one_way(icache.sets[set], tag, line);
15  return line.bytes_from(offset);
16 }

```

这段伪代码省略了真实硬件中的并行查找、预测、权限检查和预取，但保留了最重要的推导：I-cache 不是按函数名查找，也不是按源码行查找，而是按地址位查找。于是三类问题会自然出现。第一，强制 miss：某段代码第一次执行，还没有进入 I-cache。第二，容量 miss：热代码总足迹超过 I-cache 能稳定容纳的范围。第三，冲突 miss：两个热点代码块总大小不大，却因为地址位映射到同一组，反复互相替换。性能报告不总能把三者完全分开，但代码体积、函数布局、热路径长度、输入触发的分支集合和前端受限指标能给出方向。

冲突为什么会出现 若缓存完全按一个大表任意放置，硬件查找会慢；若每个地址只能放在唯一位置，冲突又会太严重。组相联是折中：set index 快速缩小候选范围，多个 way 给同一组留出少量弹性。代价是很直观的：两个热点地址只要 set index 相同，就会争同一个 set；热点代码总大小即使小于整个 I-cache，只要集中映射到少数组，也可能反复替换。

软件一般不能手算所有 set index，因为 ASLR、链接布局、共享库、代码重排和处理器实现都会影响细节。工程上更可靠的做法是减少热路径字节、让常走路径连续、把冷函数挪出热函数、控制过度内联和展开，并用反汇编和计数器观察结果。PGO 和 LTO 的价值之一，就是把真实热路径反馈给编译器和链接器，让常走分支、热函数和冷函数在布局上分开。

I-cache miss 不等于 D-cache miss 指令 miss 的直接后果是前端拿不到字节；数据 miss 的直接后果是后端 load 等不到值。二者都可能表现为 IPC 低，但修复方向不同。若数据全在 L1D，热函数却巨大，继续改数据结构不会解决前端饥饿；若 I-cache 稳定，链表遍历每步都等数据，继续缩小代码也不会让节点地址变近。第二章把 I-cache 放在前端供给链中，第三章再系统讲数据 cache、数据 TLB 和虚拟内存地址路径，就是为了避免把两类等待混在一起。

为什么 I-cache 和 D-cache 要分开讲 指令和数据最终都来自内存，也都可以用 cache line、set、tag、way 这些结构加速；它们的访问者和约束却不同。I-cache 服务取指前端，典型访问形状是顺序读取、循环回边、函数调用和分支目标；D-cache 服务 load/store 单元，要处理读写、store buffer、写分配、原子修改、一致性和别名。把二者混在一起会造成错误归因：程序可能数据都在 L1D 里，却因为热代码太大而前端受限；也可能 I-cache 很稳，却因为随机数据访问让后端等内存。第 3 章展开数据 cache 和数据 TLB，本节只抓住指令路径的成本。

取指宽度、边界和对齐 前端通常一次取一段字节，而不是每周期只取一条完整指令。若热循环跨过 cache line 边界、取指块边界或页边界，硬件可能需要两次读取或额外对齐；若分支目标落在不利位置，前端每轮能提供的字节数或指令数可能下降。x86 指令长度不固定，译码器还要从字节流里找指令边界；RISC 风格 ISA 的指令边界更规则，但仍然受 I-cache 容量、ITLB、分支预测和取指带宽限制。不同 ISA 的细节不同，推导顺序相同：先有目标地址，再取字节，再识别指令，再生成内部工作单元。

对齐不是玄学。若一个很短的循环完全落在少数取指块内，前端每轮工作简单；若循环头、循环尾和常走分支目标分散在多个块里，每轮可能要组合更多字节。手工填充对齐不应成为默认动作，因为它会增加代码体积，也可能伤害其他路径。更稳的工程策略是先减少热路径体积、分离冷路径、避免无意义展开，再用反汇编和测量确认是否还存在边界问题。

从 PC 到微操作的取指前端



前端的输出不是源码行，而是可进入重命名窗口的微操作；I-cache、ITLB、BTB 和译码带宽共同决定供给速度。

图示：本图要证明的命题是：I-cache 不是孤立名词，它是“根据预测地址持续提供指令字节”这条硬件路径中的一个必要环节。

2.6 ITLB：PC 为什么还要经过地址翻译

用户态程序看到的是虚拟地址。PC 也保存虚拟地址；操作系统和硬件用页表把虚拟页映射到物理页，并在映射上附带读、写、执行和用户态权限。I-cache 访问下层缓存时必须知道物理位置和权限边界，所以取指路径也需要地址翻译。ITLB 保存最近使用的“指令虚拟页到物理页”的翻译结果。热循环通常落在少数页里，ITLB 命中后翻译成本很低；若代码足迹跨越许多页，大型解释器、JIT、插件系统、过度展开的模板代码或大量小函数随机跳转，就会增加 ITLB 压力。

TLB miss 不只是“查一张表”。若 ITLB 没有目标虚拟页的翻译，硬件或操作系统要沿页表层级找到映射，检查权限，再把结果填回 TLB；如果页不在内存或权限不允许执行，还会触发异常。现代处理器会让 page walk 尽量由硬件完成，也会缓存中间层级，但它仍然比普通 ITLB 命中昂贵得多。指令页使用大页、JIT 代码布局、共享库数量、ASLR 和热函数分散程度，都会影响指令侧翻译压力。前端事实是：I-cache 命中之前，PC 对应的虚拟地址必须能被翻译成可执行的物理位置。

页表项保存的不只是物理页号 页表项通常包含物理页框号和一组属性：是否存在、是否允许用户态访问、是否可写、是否可执行、是否脏、是否被访问过，以及和缓存策略相关的位。取指最关心的是“这个虚拟页是否存在、是否属于当前权限级、是否允许执行”。若系统启用了不可执行页，普通数据页即使有字节也不能被当作指令。JIT 系统在生成代码时必须小心处理写权限和执行权限的切换，否则不是性能问题，而是安全和正确性问题。

这一点会改变对“代码布局”的理解。热函数如果散落在许多虚拟页上，前端不仅要在 I-cache 中保留许多 line，还要在 ITLB 中保留许多页翻译；若频繁在共享库、插件和 JIT 代码区之间跳转，活跃代码页数会继续上升。大页可以扩大单个 TLB 项覆盖的地址范围，但也会带来权限粒度、内存碎片、部署配置和调试复杂度。把热路径聚合到较少页面里，通常比事后追求某个神奇开关更可靠。

ITLB 和 DTLB 为什么分开观察 取指和数据访问都需要地址翻译，但压力来源不同。ITLB 的访问者是前端 PC 流，典型问题是热代码页过多、间接跳转目标分散、JIT 代码碎片化、共享库调用边界频繁。DTLB 的访问者是 load/store，典型问题是数据对象跨页、随机索引、指针追踪、哈希表节点分散。两个 TLB 可能在不同结构上共享下层资源，也可能有独立一级结构；具体实现会随处理器变化。性能报告不应把“TLB 问题”写成一个模糊桶，而要说明是指令页还是数据页、是页数太多还是访问顺序太随机。

Listing 2.6 指令页翻译的抽象伪代码

```

1 ExecutablePage translate_instruction_page(Address virtual_pc) {
2     VirtualPage vpn = virtual_pc.page_number();
3 }
  
```



```

4  if (ITlbEntry entry = itlb.lookup(vpn); entry.hit) {
5      if (!entry.permissions.executable) {
6          raise_execute_permission_fault(virtual_pc);
7      }
8      return {entry.physical_page, entry.permissions};
9  }
10
11  PageTableEntry pte = walk_page_tables(vpn);
12  if (!pte.present || !pte.user_accessible || !pte.executable) {
13      raise_instruction_fault(virtual_pc);
14  }
15
16  itlb.fill(vpn, pte.physical_page, pte.permissions);
17  return {pte.physical_page, pte.permissions};
18 }

```

这段伪代码把真实硬件的并行性简化成顺序步骤，但保留了关键事实：地址翻译不是取指之后才做的簿记，而是前端能否访问指令字节的前提。若 ITLB miss 或执行权限异常发生，后面的 I-cache 命中、译码带宽和 uop cache 都暂时没有意义。许多“代码明明很短却慢”的解释，最后会落到目标分散、页覆盖不足或错误路径把冷代码页带热。

2.7 预测下一条 PC：BTB、RAS 和方向历史

直接分支的目标地址写在机器指令里，表面看译码后自然就知道下一条地址。问题在于，等译码完成再决定下一条 PC，前端已经慢了一拍；现代核心每周期需要多条指令或多个微操作，必须提前形成取指队列。BTB，即 branch target buffer，用分支指令所在地址索引，缓存“这条分支以前跳到哪里”。条件分支还需要方向预测：这次跳还是不跳。函数返回的目标来自调用栈，普通 BTB 难以稳定预测，于是硬件用返回地址栈 RAS 记录 call/return 的栈式关系。间接跳转、虚调用和解释器分派的目标随数据变化，更依赖历史和类型分布。

最简单的方向预测可以从循环推出来。若只记录“上一次是否跳转”，一个循环回边在大多数迭代中跳转，最后一次不跳；下一次重新进入循环时，上一次记录是不跳，又会预测错。两位饱和计数器解决这个小问题：状态分成强跳、弱跳、弱不跳、强不跳；一次相反结果只把状态推向中间，不会立刻翻转。这不是现代预测器的完整形态，却说明硬件为什么保存历史，而不是每次从零判断。

为什么等到执行阶段就太晚 一条条件分支要等它依赖的寄存器或标志位准备好，才能知道真实方向；一条间接跳转要等目标地址被计算出来，才能知道真实目标；一条函数返回要等返回地址可用，才能回到调用点。如果前端在这些答案出现之前停止，乱序窗口会逐渐耗尽。预测器的作用是把“未来控制流”提前变成一个可取指地址，让前端继续工作。预测越准，投机路径越像真实路径；预测越差，核心做的无效工作越多。

这个机制解释了分支预测成本的两个层面。第一层是方向和目标本身是否预测正确。第二层是发现错误后要恢复多少工作：取过的字节、译过的微操作、占用的 ROB 项、执行端口和缓存访问都可能被浪费。深流水线和宽乱序窗口能提高吞吐，也会让错误路径积累更多无效工作。因此，分支问题不是“多一条 jump 指令”，而是“错误未来占用了多少硬件预算”。

BTB 预测目标，方向预测器预测是否跳转 条件分支有两个问题：这次走跳转目标，还是继续执行 fall-through；若走跳转，目标在哪里。BTB 主要保存目标地址，方向预测器主要保存跳转倾向和历史。二者缺一不可：方向猜对但目标错，前端仍然取错地方；目标在 BTB 中但方向猜不跳，前端会继续取 fall-through。直接调用和无条件跳转也需要目标预测，因为前端希望在译码确认之前就开始取目标地址。

间接分支更难。虚函数调用、函数指针、解释器 opcode 分派、JIT stub 和 `switch` 跳转表，目标可能随数据变化。若同一分支地址在不同记录上跳到很多目标，BTB 项和历史都会变得不稳定。工程上的批处理、分桶、类型专门化和 `direct threading`，本质上都在减少每条数据上的随机目标变化，让一批相似记录走更稳定的目标集合。

从 `cmp/jcc` 读出预测问题 第一册读条件分支时，重点是 `flags` 和跳转目标。这里还要读出概率。下面两段汇编在语义上都很普通，硬件形状却不同：

Listing 2.7 规则输入上的循环回边和稀有分支

```
.Loop:
    movzx edx, byte ptr [rdi + rcx]
    cmp    dl, 10
    je     .Lnewline           ; 每几十个字节才出现一次
    ; 普通字符路径
    inc    rcx
    cmp    rcx, rsi
    jne    .Lloop             ; 除最后一次外几乎都跳
```

循环回边 `jne .Lloop` 的模式很强：大多数迭代跳转，最后一次不跳。稀有 `newline` 分支也有强倾向：大多数时候不跳，偶尔跳。预测器不需要理解“换行”这个业务含义，只要这个分支地址的历史足够稳定，就能高概率猜对。此时分支指令仍然存在，但它们不一定是瓶颈。

Listing 2.8 随机分类输入上的接近均匀分支

```
.Loop:
    movzx edx, byte ptr [rdi + rcx]
    cmp    dl, byte ptr [class_table + rcx] ; 示例：阈值随位置变化
    ja     .Lclass_a                 ; 方向接近随机
    ; class B path
    jmp    .Lnext
.Lclass_a:
    ; class A path
.Lnext:
    inc    rcx
    cmp    rcx, rsi
    jne    .Lloop
```

第二段的分类分支如果接近 50/50，且缺少稳定周期，预测器很难长期猜中。错误预测发生时，前端已经沿某条路径取了指令，译码器可能已经产生微操作，乱序窗口可能已经分配条目，后端甚至可能执行了错误路径上的 `load`。分支解析后，这些工作被冲刷，PC 重定向到真实路径。于是 `branch miss` 的成本来自“错误未来被提前执行”，不是来自 `ja` 这个助记符本身。

Fall-through 也是布局决策 条件分支只有一个显式目标，另一边是 `fall-through`，即顺序落到下一条指令。编译器和链接器会尽量让常走路径成为更便宜的布局：少跳、连续、前端容易取。`Profile-guided optimization` 的价值之一，就是用真实输入告诉编译器哪边常走。若源码把错误处理写在 `if` 内部，不代表最终机器码一定把错误路径放远；要看反汇编和布局。热路径审查时，应该标出每个条件分支的常走方向、`fall-through` 是否符合输入分布、冷标签是否远离内层循环。

Listing 2.9 从反汇编记录分支证据

```

branch address: 0x... .Lparse_loop + 37
source condition: ch == ',' or ch == '\n'
expected input: RegularLines
likely direction: not taken for ordinary byte
fall-through path: ordinary byte path
evidence:
  branch-misses available? yes/no
  disassembly saved? yes
  checksum unchanged? yes

```

这类记录把第一册的汇编阅读推进到性能判断。读懂 `cmp/jne` 只是起点；还要把它放回输入分布、代码布局和预测失败成本中。

RAS 为什么专门处理返回 函数调用和返回有栈式结构：`call` 的下一条地址就是对应 `return` 应回到的位置。若只靠普通 BTB 预测 `return`，多个调用点共享同一个返回指令时很容易混淆。RAS，即 `return address stack`，用硬件小栈保存 `call` 后的返回地址，`return` 时弹出预测目标。它适合正常的嵌套调用，却会被异常控制流、栈破坏、深递归超过容量、非局部跳转或不规则调用约定干扰。热路径中过多小函数调用不只增加 `call/return` 指令，也会消耗返回预测资源和前端带宽。

Listing 2.10 两位分支方向预测器的抽象形态

```

1 enum class DirectionState {
2     StrongNotTaken,
3     WeakNotTaken,
4     WeakTaken,
5     StrongTaken
6 };
7
8 bool predict_taken(DirectionState state) {
9     return state == DirectionState::WeakTaken ||
10         state == DirectionState::StrongTaken;
11 }
12
13 DirectionState update(DirectionState state, bool actual_taken) {
14     if (actual_taken) {
15         return saturating_increment(state);
16     }
17     return saturating_decrement(state);
18 }

```

真实预测器会使用更多历史、更复杂的索引和多级结构，但基本问题没有变：当前 PC 附近有没有分支，这个分支这次走哪边，走了以后目标地址是什么。任何一个答案错了，前端就会沿错误路径取指和译码；等后端发现错误，错误路径上的微操作被丢弃，PC 被重定向到正确路径，前端重新供给。`branch-miss` 的成本不是某条分支指令本身慢，而是一段错误路径浪费了前端、乱序窗口和后端资源。

2.8 译码器：从机器码字节到微操作

取到指令字节之后，后端仍然不能直接执行。机器指令要先被译码成内部微操作，再进入寄存器重命名、调度和执行。`x86` 这类变长指令集的第一道困难是边界：一条指令可能因为前缀、寻址模式、立即数和位移占用不同字节数；译码器必须从

连续字节流中切出一条条合法指令。定长或较规则的 ISA 边界更简单，但仍然要识别操作类型、寄存器、内存地址形式、异常条件和所需执行资源。

译码带宽是有限资源。简单指令可能每周期译码多条，复杂指令可能拆成多个微操作；某些内存操作同时需要地址生成、加载和算术；调用、返回、间接跳转和异常边界还会影响前端控制。若热路径每轮都需要复杂译码，后端端口空闲也没有用，因为进入重命名窗口的微操作不够。此时减少数据 cache miss 不是主线，真正的修复常常是缩小热路径、降低分派复杂度、避免过度展开，或者让热循环稳定命中 micro-op cache。

为什么机器指令不能直接进入调度器 机器指令是软件和硬件之间的可见合同，必须长期兼容；内部微操作是处理器实现自己的工作单元，可以随代际变化。把两者分开，硬件就可以把一条复杂指令拆成几个更适合端口和流水线的动作，也可以把某些简单组合融合成更少的内部动作。比如一次带内存操作的算术，可能需要地址生成、加载、算术和写回；一次比较加条件跳转，在某些实现上可能在前端或后端以融合形式处理。具体规则依赖微架构，原理不变：后端调度需要的是资源清晰、依赖清晰、异常边界可维护的内部工作单元。

这个分层也解释了为什么反汇编只能看到一部分事实。反汇编显示机器指令，不直接显示每条指令拆成多少微操作、占哪些端口、能否融合、是否来自 uop cache。性能分析要把反汇编、编译器报告和 PMU 指标合在一起。若热循环机器指令很少，但每条都复杂，译码和端口压力仍可能高；若机器指令较多，但来自稳定的 uop cache 并且后端资源匹配，未必慢。

变长指令让边界识别也成为工作 x86 指令长度可变，前缀、操作码、ModR/M、SIB、位移和立即数让边界识别变复杂。前端要从一串字节中找出第一条指令从哪里开始、在哪里结束，下一条又从哪里开始。分支目标必须落在合法指令边界上；跳到字节流中间会变成另一种解码结果或触发异常。定长 ISA 在边界上更简单，但仍要面对取指带宽、预测、I-cache、ITLB 和译码资源。不要把“定长指令更容易解码”误读为“前端没有瓶颈”；它只是减轻了其中一类困难。

译码带宽和后端端口是两道门 一个循环可能先被前端译码带宽限制，也可能穿过前端后被执行端口限制。二者的修复方向不同。若译码带宽限制明显，减少热路径复杂控制流、让循环命中 uop cache、控制展开规模可能有效；若端口限制明显，要看 load 端口、store 端口、整数 ALU、向量单元、地址生成单元或分支单元哪一类拥挤。只说“指令少一点”不够，因为少掉的如果不是瓶颈资源，时间可能不变。

2.9 Micro-op cache：为什么重复译码可以被缓存

如果一段热循环已经译码过，而且代码没有改变，下一轮仍然从相同地址执行，重复译码就是浪费。Micro-op cache 的动机由此产生：保存已经译码好的微操作，让前端在命中时绕过部分译码工作，直接向后续阶段供给内部工作单元。I-cache 保存的是指令字节，micro-op cache 保存的是译码结果；二者解决的是前端链路上不同位置的等待。

Micro-op cache 也不是无限空间。循环体太大、过度展开、模板实例过多、冷路径夹在热路径中、频繁跨函数、间接分派目标太多，都会降低它的稳定命中。更隐蔽的情况是：某个优化减少了指令条数，却让代码布局更分散，或者让热路径跨过更多前端结构边界，最后总时间没有下降。遇到这种现象，不能只看 instructions 计数下降，还要看分支失败、前端受限、代码体积、热点地址分布和反汇编形状。

uop cache 不是 I-cache 的替代品 I-cache 命中说明指令字节在近处；uop cache 命中说明某个地址范围的译码结果在近处。它们可以同时有用，也可以分别成为瓶颈。若 uop cache 命中，前端可能绕过复杂译码，但仍然依赖预测和地址流稳定；若代码修改、JIT 生成新代码、目标分散或热路径超过 uop cache 容量，就会退回普通取指和译码路径。把二者混成“缓存命中”会导致错误诊断。

一个简单判断是：I-cache 关注“字节是否在近处”，uop cache 关注“这些字节

是否已经变成可供供给的内部工作”。热循环很短、循环体稳定、分支目标少，最容易从 uop cache 受益；大型解释器、模板膨胀、过度展开和冷路径混入，会让它的命中稳定性下降。减少机器码字节有时会改善 I-cache，有时会改善 uop cache，有时只是把瓶颈迁移到后端。结论必须由指标和反汇编共同支持。

循环体大小的双重约束 展开循环可以减少分支次数、暴露更多独立工作，也可能增加寄存器压力和代码体积。对后端而言，展开让乱序窗口看到更多独立操作；对前端而言，展开让每轮或每个 kernel 的字节数和微操作数增加。最佳展开不是越大越好，而是让依赖、端口、寄存器和前端容量同时平衡。第 7 章讲 SIMD 和 ILP 时会再次遇到这条原则：向量化和展开必须同时接受前端和后端审计。

前端结构	保存的对象	主要解决的问题	典型失效形状
BTB/RAS/方向预测	目标和历史	提前给出下一条 PC	目标多变、返回栈异常、输入随机
ITLB	指令页翻译	虚拟地址到可执行物理页	代码页过多、JIT/插件分散
I-cache	指令 cache line	快速提供机器码字节	热代码足迹大、冲突、冷路径混入
译码器	字节到微操作的转换逻辑	识别边界和内部动作	复杂指令、调用边界、带宽不足
uop cache	已译码微操作	绕过重复译码	循环体过大、目标分散、布局不稳

Listing 2.11 取指前端的抽象伪代码

```
1 while (core_is_running) {
2   Address predicted_pc = predict_next_pc(pc, branch_history, btb, ras);
3
4   PageTranslation page = itlb.lookup(predicted_pc.page());
5   if (!page.hit) {
6     page = page_table_walk_and_permission_check(predicted_pc);
7     itlb.fill(predicted_pc.page(), page);
8   }
9
10  InstructionBytes bytes = icache.fetch(page, predicted_pc.offset());
11
12  MicroOps uops;
13  if (uop_cache.contains(predicted_pc)) {
14    uops = uop_cache.read(predicted_pc);
15  } else {
16    Instructions insts = find_boundaries_and_decode(bytes);
17    uops = translate_to_micro_ops(insts);
18    uop_cache.fill(predicted_pc, uops);
19  }
20
21  rename_queue.push(uops);
22  pc = predicted_pc;
23 }
```

这段伪代码不是某一款 CPU 的真实实现。真实硬件会并行、预测、回滚、预取、对齐、融合指令、处理异常，还会按厂商和代际变化。伪代码保留的是因果关系：下一条 PC 必须提前预测；取指地址必须能翻译并通过执行权限检查；指令字节必须从 I-cache 或下层层级到达；译码或微操作缓存必须提供后端能使用的工作单元。

I-cache、ITLB、BTB 和 uop cache 不是四个并列名词，而是同一条供给链上的四个压力点。

2.10 完整账本：一条热循环迭代怎样变成可退休结果

把前端、译码、乱序和退休分开讲，是为了降低复杂度；真正的核心在每个周期里同时推进许多半成品。要理解一个优化为什么有效，必须能把一轮循环从 PC、指令字节、微操作、寄存器重命名、调度、执行、写回和退休串成一条账本。否则很容易把局部现象误判成全局瓶颈：分支少了但端口仍满，指令少了但 load 等待仍长，循环展开了但前端供给被代码体积拖住。

仍以解析器热循环为例。源码上看，一轮迭代读取一个字节、分类、更新状态和计数。硬件账本要拆得更细：取指路径先猜下一条 PC，I-cache 和 uop cache 尝试供给微操作；重命名阶段把架构寄存器映射到物理寄存器，消除名字冲突；调度器等待操作数和执行端口；load 进入地址生成、DTLB 和 L1D；分支在执行后验证预测；最后退休逻辑按程序顺序提交结果。这里每个阶段都能等待，也都能回滚。

Listing 2.12 一轮热循环迭代的硬件账本

```
frontend:
    predict loop-back and internal branches
    fetch instruction bytes through ITLB and I-cache
    supply micro-ops from uop cache or decoders

rename and allocate:
    assign physical registers for temporary values
    allocate ROB entries, load queue entries, store queue entries
    keep precise exception and rollback metadata

execute:
    compute byte address
    translate through DTLB
    load input byte from L1D or lower cache
    classify byte with branch or table lookup
    update parser state, counters and error summary

resolve:
    verify branch prediction
    forward store data when possible
    wake dependent micro-ops

retire:
    commit completed micro-ops in program order
    expose architectural state
    handle exception or misprediction rollback precisely
```

这张账本有一个重要含义：优化不能只盯源码动作。把 if 改成查表，确实减少某些方向分支，却增加一次数据 load、一个地址生成、一个表所在 cache line 和可能的寄存器压力；展开循环确实减少回边分支，也让一轮占用更多 ROB、寄存器和前端字节；把错误对象构造移出热路径，既减少前端足迹，也减少分配器和写入路径进入热循环的机会。每个改写都必须说明它购买了哪一段硬件账本，付出了哪一段成本。

ROB 保存的是尚未提交的未来 乱序执行让后面的独立工作先做，但不能让程序对外表现成乱序。核心需要一个结构保存已经发射、可能已经执行、但还没有按

程序顺序提交的操作。这个结构通常用 reorder buffer 的概念描述。它记录每个微操作的完成状态、目标寄存器、异常信息和回滚边界。分支预测错时，预测路径之后的条目被丢弃；异常发生时，核心能把异常报告到精确的程序点；store 也要等到安全时刻再真正对外可见。

ROB 的容量是隐藏延迟的预算。连续数组求和中，核心可以让许多独立 load 同时在路上；指针追踪中，下一地址依赖当前 load 返回，ROB 再大也难以制造足够独立请求。解析器如果每个字符都依赖上一状态，状态依赖会限制乱序窗口；若把输入分成块、先做结构字符候选扫描，再做少量确认，硬件能看到更长、更规则的独立工作。

重命名解决名字，不解决数据事实 第一卷里看到的 `rax`、`rcx` 是架构寄存器名。硬件内部有更多物理寄存器，重命名把同一个架构名在不同时间的值映射到不同物理位置，避免假依赖。例如两个独立累加器都写 `rax` 的源码形态会被编译器或硬件变成不同物理寄存器上的工作，使它们可以重叠执行。

但是重命名不能消除真实依赖。若第二条指令必须使用第一条 load 得到的地址，硬件只能等；若 store 的地址可能和后面的 load 重叠，内存消歧预测错误后需要回滚；若一个原子读改写写必须拿到 cache line 独占权，寄存器再多也不能让所有核心同时修改同一条 line。把名字依赖、地址依赖和一致性依赖分清，是分析热循环的起点。

等待来源	账本中的位置	典型修复方向
前端供给不足	PC、ITLB、I-cache、uop cache、译码	冷热路径分离、缩小热循环、稳定分支目标
真实数据依赖 地址依赖	调度器等待操作数 地址生成、load queue、DTLB	多累加器、分块、改变算法数据流 连续布局、索引数组、减少指针追踪
端口拥挤	执行端口、load/store 端口、AGU	减少内存操作、调整展开、拆冷热字段
一致性等待	store buffer、原子、cache line 所有权	分片、thread-local partial、padding、批量合并
退休受阻	ROB 头部未完成或异常等待	缩短长延迟链、减少缺页和异常路径进入热循环

从汇编反推硬件账本 反汇编给出的是账本入口，不是账本终点。看到 `cmp` 和 `jcc`，要继续问分支方向是否随输入随机；看到 `mov` 内存操作数，要继续问地址是否连续、是否跨 cache line、是否跨页；看到 `lock xadd` 或 `lock cmpxchg`，要继续问 cache line 独占权在哪里迁移；看到函数调用，要继续问调用目标是否稳定、能否内联、是否把冷路径拉进热代码。

Listing 2.13 从几类汇编形状追问硬件事实

```
cmp/jcc:
    is the direction predictable for each input family?
    does misprediction flush useful work?

mov [base + index * scale]:
    is the address stream sequential, strided, random or dependent?
    how many useful bytes does each cache line provide?

call or indirect jump:
    is the target stable enough for BTB and inlining?
    does it cross many code pages or pull cold code into the hot path?

lock cmpxchg:
    which cache line becomes exclusive?
```

```
how many cores are trying to modify it?
```

这种追问把“看汇编”从背指令变成工程证据。汇编证明代码形状，硬件账本解释等待位置，输入矩阵验证等待是否随数据分布变化。三者合起来，优化才不会停在猜测。

2.11 把前端问题落到热循环

日志解析器的热循环提供了一个完整例子。固定格式日志中，字段位置稳定，分隔符出现规律，循环回边和几个常见分支容易预测；机器码足迹小，错误路径很少执行，I-cache 和 uop cache 可以稳定服务。换成用户自定义字段后，字符类别分布更接近随机，分支方向难预测；如果错误检查和格式化仍在内层循环，冷路径机器码被拖进热取指区域；若每个字段类型用虚函数处理，还会出现间接分支和难以内联的小调用。此时“同样是扫描字节”这句话掩盖了机器看到的差异：预测历史变差、目标地址变多、热代码足迹变大、译码任务变复杂。

对应的改写也要直接改变这些硬件形状。冷热路径分离让正常路径短而直；小表分类把多级分支换成数据 load 和掩码；批量扫描让一段连续字节走同一条紧凑路径；按字段类型分桶减少每条记录上的间接分派；PGO/LTO 可以帮助编译器把热函数放近、把冷函数挪远、让常走分支成为 fall-through。每个改写都必须接受反证：查表可能把问题从分支迁移到 D-cache；批处理可能伤害短行；过度内联可能提高局部优化却扩大 I-cache 足迹；PGO 依赖采样输入，输入分布变了就可能失效。

四个版本的前端形状 朴素版本通常按字符执行状态机，内层有多个 if 或 switch 分支。规则日志上它可能很好，因为分隔符位置和字符类别稳定；随机字段上它可能出现高分支失败。查表分类版本把字符类别判断变成一次 table load 和少量掩码，减少方向分支，却可能增加 D-cache 访问和寄存器压力。冷路径分离版本让正常路径只记录错误摘要，完整错误对象放到慢路径构造，减少热代码足迹和错误路径污染。批量扫描版本先找结构字符候选，再进入语义确认，目标是让普通字节走更短、更直、更可向量化的路径。

四个版本不是“越后越高级”。它们分别购买不同能力，也承担不同风险。规则输入上 branch 版本可能胜过 branchless；短行输入上批量扫描启动成本可能太高；错误密集输入会让慢路径频繁返回热路径；查表分类可能把前端问题变成数据 cache 问题。好的报告不是宣布某个版本永远最快，而是把输入族、机器码形状和指标放在一起，说明每个版本在什么条件下改变了哪一段硬件路径。

冷热路径分离的具体边界 错误处理不能被删除，只能从每字符热循环中移出去。热循环可以记录紧凑事实：记录编号、字节偏移、错误码、状态机状态、少量上下文边界。块结束后或慢路径里再根据这些事实构造完整错误信息。这样做必须保持诊断语义：错误顺序是否稳定，多个错误是否都保留，原始输入生命周期是否足够，错误上下文是否还能回查。若冷路径分离让问题难以定位，它只是把性能问题换成可维护性问题。

虚调用和按类型分桶 若每条字段都调用一次虚函数处理，前端要面对间接分支，编译器也很难看到具体循环体。按字段类型分桶后，一批同类型字段进入同一个具体内核，间接目标减少，内联和向量化机会增加。代价是需要额外分类、缓冲和顺序恢复。若业务要求严格按原始顺序产生副作用，分桶可能不可用；若处理只产生可交换的统计结果，分桶就很自然。性能设计必须先问语义是否允许重排，再谈硬件收益。

前端压力点	常见现象	可尝试的改写	需要反证的风险
I-cache 容量 或 冲突 ITLB 压力	热函数变大后耗时不降反升；前端受限增加 大型解释器、JIT 或插件 路径跨许多代码页	冷热路径分离、减少过度 内联、PGO 布局 合并热路径、减少随机小 函数跳转、检查页大小与 布局	指令数下降但代码 足迹更差 数据 TLB 或分支问 题被误判
BTB/间接目标 方向预测失败	虚调用、函数指针、解释器 分派随输入变化 随机字段、接近 50/50 的 条件分支	分桶、批量接口、专门化 热点类型 branchless、查表、按输 入族分流	类型分布变化后收 益消失 可预测输入上反而 变慢
译码 或 uop cache	复杂指令、过度展开、循环 体太大	缩小循环体、简化分派、 控制展开规模	后端端口或内存成 为新瓶颈

2.12 前端瓶颈的实验判别

前端瓶颈经常出现在“每条数据做一点点工作，但控制流很复杂”的系统里：解释器循环、正则引擎、协议解析器、数据库执行器、日志解析器、虚函数密集的对象图、带大量错误检查的热路径。它的症状是数据不大、算术不重、load miss 不明显，IPC 却低；或者某个版本指令数下降了，时间没有下降。可验证的实验有四类。第一，固定输入和 checksum，只改变代码布局或冷热路径，观察前端指标和耗时是否同向变化。第二，固定二进制，只改变输入分布，观察 branch-misses 和时间是否跟随随机性变化。第三，保留语义，把每元素虚调用改成 batch 调用，观察间接分支、调用边界和内联形状是否变化。第四，在无法读取 PMU 时，用反汇编、函数大小、热点地址、输入族耗时阶梯和重复轮次构造替代证据。

前端分析的结论不应写成“用了 I-cache 优化”。I-cache 是硬件事实，不是软件技巧。更清楚的表达是：某个改写减少了热路径机器码足迹，或者减少了随机分支目标，或者让译码后的热循环稳定复用，因而前端供给改善；如果指标不支持这条链路，就要承认瓶颈可能在依赖、端口、D-cache、TLB、同步或内存带宽上。

证据链应该能反驳自己 前端结论至少要能回答四个反问。第一，输入分布变得可预测后，所谓分支优化是否仍然有效；若仍然有效，主因可能不是分支预测。第二，函数大小和热路径地址变短后，前端指标是否改善；若没有改善，I-cache 可能不是主因。第三，指令数下降后时间是否下降；若时间不降，瓶颈可能已经迁移。第四，批处理或分桶是否改变了语义边界；若改变了 checksum、错误摘要或顺序合同，性能结果无效。

没有 PMU 时怎样保持严谨 普通环境可能没有权限读取前端细分事件。此时仍然可以做严谨对照：固定编译器和优化选项，保存反汇编；固定输入族和随机种子，保存 checksum；记录函数大小、热循环地址范围、调用和跳转形态；用规则输入、随机输入、错误密集输入和短行输入构造趋势；多轮运行记录分位数；把不可用指标写成 unavailable，而不是填零。没有 PMU 不代表可以写猜测，只是证据粒度降低。

前端供给的最小对照路径可以只选解析器的一个热循环。输入至少覆盖规则、随机、错误密集和短行四类；版本只保留朴素状态机、查表分类、冷路径分离和批量扫描四个。证据必须包含 checksum、错误摘要、函数大小、热循环反汇编、输入分布、可用计数器或不可用说明，并明确指出每个版本主要改变了 I-cache、ITLB、BTB、方向预测、译码还是 uop cache。这样对照的是机器供给路径，不是堆一组无关 benchmark。

2.13 完整案例：把逐字符解析器改造成两阶段热路径

前端问题最容易在解析器里显形，因为解析器的源码看起来只是“读一个字符，判断它属于哪一类”，机器看到的却是一串分支、跳转目标、调用边界和错误路径。先看一个朴素循环。它一边读取字节，一边维护状态，一边处理错误，一边把诊断字符串写入对象。源码短，但每个字符都可能穿过多个 if，每个分支又可能落到不

同代码块。

Listing 2.14 朴素逐字符解析器的控制流形状

```
state = Normal
for each byte ch in input:
    if ch == ',':
        finish_field()
        state = Normal
    else if ch == '"':
        toggle_quote_state()
    else if ch == '\\':
        state = Escape
    else if is_digit(ch):
        append_digit(ch)
    else if is_space(ch):
        skip_or_finish()
    else:
        build_error_message(record_id, offset, ch)
        state = BadRecord
```

这个版本的根本问题不是分支数量本身，而是分支方向和目标是否能被前端稳定预测。固定格式输入中，逗号、数字和空格的位置接近周期性，预测器会逐渐学到“下一次大概率仍然走相同方向”。一旦输入来自用户自定义字段，字符类别可能接近随机，某些条件长期停在 50/50 附近；错误路径虽然罕见，却包含格式化、分配和字符串拼接，机器码体积远大于正常路径。于是同一个循环同时破坏方向预测、BTB 目标局部性、I-cache 足迹和 uop cache 复用。

从第一卷的汇编视角看，`if/else` 通常会落成比较和条件跳转。下面的伪汇编不对应特定编译器，只保留控制流形状：

Listing 2.15 字符分类在机器层面的分支形状

```
loadzx r8d, byte ptr [input + i]
cmp r8b, ','
je finish_field
cmp r8b, '"'
je quote_path
cmp r8b, '\\'
je escape_path
call is_digit_or_table
test eax, eax
jne digit_path
jmp other_path
```

每个 `je/jne/jmp` 都给前端提出一个问题：下一条 PC 是 fall-through，还是跳转目标。预测正确时，前端持续供应；预测错误时，错误路径上已经取指、译码甚至发射的微操作要被丢弃。若 `other_path` 还会调用诊断构造，热循环附近的机器码会被冷路径拉大，I-cache 和 uop cache 也要承担不常用代码。

第一步：把状态机写成可审计的转移 改造前先把状态机从分支堆里抽出来。状态机不是为了抽象漂亮，而是为了明确“每个输入字节只改变哪些少量事实”。日志行解析常见状态可以收敛到 `Normal`、`InField`、`InQuote`、`Escape` 和 `BadRecord`。每次读字节时，程序根据当前状态和字符类别计算下一状态，并输出少量事件。

Listing 2.16 解析状态机的语义账本

```

State:
    Normal    -- not inside a field
    InField   -- reading an unquoted field
    InQuote   -- reading a quoted field
    Escape    -- next byte is escaped
    BadRecord -- record already failed

Event:
    FieldEnd(record_id, byte_offset)
    Digit(record_id, byte_offset)
    Quote(record_id, byte_offset)
    Error(record_id, byte_offset, error_code)

```

这个账本带来两个好处。第一，错误语义不会因为优化被吞掉，所有坏行仍然通过 `Error` 事件保留 `record id` 和 `offset`。第二，热路径可以只生成紧凑事件，不在每个字节处构造完整字符串。冷路径稍后再根据 `record_id`、`offset` 和原始输入切片格式化诊断。

第二步：用分类表减少方向分支 字符类别可以从多级分支变成一次表读取。表的索引是无符号字节，表项是一个小整数或 `bit mask`。这样做不会让分支消失，状态转移和错误处理仍然存在；它只是把“这个字节是不是逗号、引号、反斜杠、数字、空白”合并到一个稳定的 `load`。

Listing 2.17 分类表把多级判断换成数据读取

```

class_table[256] =
    Other, Other, ..., Space, ...
    Separator for ','
    Quote      for '"'
    Escape     for '\'
    Digit      for '0'..'9'

for each byte ch:
    cls = class_table[unsigned(ch)]
    next_state, event = transition[state][cls]
    emit compact event if needed
    state = next_state

```

这一步的硬件取舍很具体。优点是方向分支减少，循环体更规整，编译器更容易把常见路径放成 `fall-through`。代价是每个字符多了一次表 `load`，分类表虽然小，仍会占用 `D-cache` 和寄存器；若原输入本来非常规则，原始分支可能已经预测得很好，查表版反而多做工作。因此报告必须同时保留规则输入和随机输入。

第三步：把热事件和冷诊断拆开 完整错误消息不应在每个字符路径上构造。热路径只记录事实，冷路径负责解释事实：

Listing 2.18 热路径只输出紧凑解析事件

```

1 enum class ParseError : std::uint16_t {
2     None,
3     UnexpectedQuote,
4     BadEscape,
5     BadDigit,

```

```

6  UnterminatedQuote
7  };
8
9  struct ParseEvent {
10     std::uint32_t record_id = 0;
11     std::uint32_t byte_offset = 0;
12     ParseError error = ParseError::None;
13 };

```

`ParseEvent` 的字段故意很少：`record id` 连接冷表，`offset` 定位原始字节，`error` 给出可比较的错误类别。冷路径稍后把它转换成文件名、行号、上下文片段和人类可读文本。这样正常路径的代码足迹缩小，错误格式化、分配和 I/O 不再污染内层循环。

冷路径分离必须守住三个不变量。第一，事件顺序要能复原，多个错误不能因为只保留一个标志而消失。第二，原始输入或冷表生命周期要覆盖诊断阶段。第三，错误计数和 `checksum` 要进入报告，防止优化版本只处理好行而悄悄少做工作。

第四步：两阶段扫描让前端看见更直的循环 两阶段解析把“结构发现”和“语义确认”分开。第一阶段只扫描字节，找逗号、引号、反斜杠和换行等结构字符；第二阶段再按候选位置推进状态机。普通字节占多数时，第一阶段循环可以非常短，甚至以后可以替换为向量化扫描。

Listing 2.19 两阶段解析的控制流形状

```

stage 1: scan structure bytes
  for block in input:
    mask = find(',', '"', '\', '\n') in block
    append candidate offsets from mask

stage 2: validate records
  for offset in candidate_offsets:
    update parser state
    emit field boundary or compact error

```

这不是免费午餐。短行输入中，候选数组的写入和第二阶段启动成本可能超过收益；引号和转义密集输入中，第二阶段仍然复杂；错误密集输入会让冷路径频繁进入。两阶段设计成立的前提是普通字节占多数、记录足够长、候选位置远少于总字节数。这个前提必须由输入报告证明。

第五步：把证据字段固定下来 解析器优化若只报告总耗时，很容易把语义变化当成性能提升。一次合格的前端报告至少包含以下字段：

Listing 2.20 解析器前端报告字段

```

parser.bytes
parser.records
parser.accepted_records
parser.rejected_records
parser.error_rate
parser.branch_shape
parser.hot_function_bytes
parser.calls_in_loop
parser.branch_misses_per_kib
parser.frontend_bound_if_available

```

```
parser.checksum
parser.diagnostic_checksum
```

`branch_shape` 可以写成 “regular”、“random”、“sorted”、“error-heavy” 这类输入族标签；`hot_function_bytes` 来自符号大小或反汇编地址范围；`calls_in_loop` 检查热循环是否仍有虚调用、分配或格式化；两个 `checksum` 分别保护数据结果和诊断结果。没有 PMU 权限时，相关字段写 `unavailable`，但反汇编、函数大小、输入族和 `checksum` 仍然必须保留。

第六步：把源码改动映射回硬件路径 最后把改造写成硬件账本，而不是写成口号：

改造	主要改变	反证问题
分类表	减少字符类别判断的方向分支	规则输入上是否反而变慢
冷热分离	缩小热路径机器码，减少错误格式化污染	错误摘要是否仍完整
两阶段扫描	让普通字节走更短更直的循环	短行和结构密集输入是否变慢
按字段类型分桶	减少间接调用目标，扩大批量处理机会	顺序合同是否允许重排
PGO/LTO 布局	让常走路径更接近 fall-through	采样输入变化后是否失效

这个案例把第一卷的 `cmp/jcc`、函数调用、反汇编和二进制布局连接到本章的 I-cache、BTB、译码和预测。源码层面的“少几个判断”不是结论；真正的结论是某个改写让前端更容易连续取到正确机器码，并且 `checksum`、诊断和输入矩阵证明语义没有改变。

2.14 流水线不是源码顺序

处理器不会按源码行一行一行执行。前端预测下一条取指地址，把指令从 `instruction cache` 取出，译码成内部微操作，可能命中 `micro-op cache`；中间阶段做寄存器重命名，分配重排序缓冲区、`load buffer`、`store buffer` 和调度队列；后端等待操作数准备好，把微操作发射到执行端口、加载单元、存储单元或分支单元；最后退休阶段按程序顺序提交结果，保证单线程可观察语义没有被破坏。

这个结构带来一个重要事实：执行顺序和提交顺序不是一回事。乱序核心可以先执行后面已经准备好的独立指令，隐藏一部分延迟；但它必须按程序顺序提交，让异常、分支和内存语义看起来仍然正确。重排序缓冲区可以理解成一个有限的“未来窗口”：窗口里如果有独立工作，核心能继续推进；窗口里如果全是依赖同一个 `load`、同一个分支或同一个累加器，核心只能等。

延迟和吞吐 延迟是一个操作从开始到结果可用需要多久；吞吐是稳定状态下单位时间能完成多少操作。一个加法延迟可能不为零，但核心可以每周期发射多个独立加法；一个 `load` 命中 L1 也要等待若干周期，但多个独立 `load` 可以同时在路上。优化热循环时，要判断限制来自关键路径延迟还是资源吞吐。单累加器常受关键路径限制；连续数组扫描在大输入上更容易受内存带宽限制；复杂 `switch` 可能受前端和分支恢复限制。

前端供给 前端供给的判断要回到上一节的链路：预测是否稳定，ITLB 是否能覆盖热代码页，I-cache 是否能容纳热路径，译码或 `micro-op cache` 是否持续供应，间接分支目标是否可预测。前端瓶颈的典型症状是：数据不大，算术也不重，后端端口看起来没有打满，但 IPC 仍然低。常见原因包括热函数过大、过度内联导致 I-cache 压力、虚调用和函数指针导致间接分支、大 `switch` 目标随机、错误路径和日志构造混在热路径。

后端执行 后端不是“算术单元”一个盒子。整数 ALU、向量执行、加载端口、存储地址端口、存储数据端口、分支单元和地址生成单元都有不同吞吐。一个循环可能总指令数不多，却因为每轮需要多个 load 卡在加载端口；也可能因为地址表达式复杂卡在地址生成；还可能因为只有一条累加依赖链，让执行端口空着也发不出新工作。后端分析要看微操作混合、端口压力、依赖链和访存等待，而不是只数源码里的运算符。

退休和精确异常 乱序执行必须让程序看起来像按顺序执行。退休阶段按程序顺序提交结果，保证异常和分支错误能恢复到清楚边界。这就是为什么分支预测失败会浪费工作：错误路径上的微操作可能已经取指、译码甚至执行，但它们不能退休，最终要被丢弃。错误投机消耗前端、执行端口和缓存资源，却不贡献有用结果。

ROB 为什么必须按序退休 重排序缓冲区通常被称为 ROB。它保存已经进入乱序窗口、但尚未按程序顺序提交的微操作。每个条目至少要回答几件事：这条微操作对应哪条程序顺序上的指令，目标寄存器或目标 store 是什么，执行是否完成，是否产生异常，分支预测是否被证明正确，提交时需要释放哪些旧物理寄存器或写出哪些架构状态。没有这样的账本，处理器可以乱序执行，却无法把结果恢复成单线程程序应有的顺序。

精确异常从 ROB 自然推导出来。假设一段代码中，第三条指令会因为除零或缺页异常而失败，而第四、第五条指令在乱序窗口里已经算出结果。如果处理器允许第四、第五条提前修改架构寄存器或写内存，异常处理程序看到的状态就会混合“异常前”和“异常后”的世界。按序退休解决这个问题：第三条之前的指令可以提交；第三条报告异常；第三条之后的投机结果全部丢弃。异常边界因此精确地落在那条指令上。

Listing 2.21 按序退休让异常边界保持精确

```
program order:
  I1: load  a[i]
  I2: add   independent counters
  I3: divide by denominator
  I4: store result to output

out-of-order execution may finish I2 before I1 or I3
retirement still commits in order:
  commit I1
  commit I2
  I3 raises exception
  discard I4 and all younger speculative work
```

这条账本解释了为什么 store 的提交格外谨慎。寄存器结果在 ROB 中可以暂存，分支错了可以丢弃；内存写一旦对其他核心或设备可见，就更难撤回。因此 store 通常先进入 store buffer，地址和数据准备好后等待退休边界，再逐步对缓存层级和一致性协议生效。乱序核心不是随意改写世界，而是在一个可以回滚的窗口里提前做工作，最后用按序退休把世界推进到下一条确定边界。

重命名解决名字冲突，不解决真实依赖 寄存器重命名把架构寄存器映射到更多物理寄存器。源码和汇编里可能多次写 rax，但这些写不一定互相依赖；重命名可以让不同版本落到不同物理寄存器，消除写后写和写后读这类名字冲突。它不能消除真实依赖：若第二条指令确实需要第一条指令算出的值，第二条仍必须等第一条结果可用。

Listing 2.22 重命名能拆开的依赖和拆不开的依赖

```
false name conflict:
```

```

rax = load a[i]
rax = load b[i]
c[i] = rax
; 第一条写 rax 的结果没有被第二条使用，可以映射到不同物理寄存器。

```

```

true dependency:
rax = load a[i]
rax = rax + 1
c[i] = rax
; 加法必须等待 load 的值。

```

这也是多累加器、分块、批量 probe 和向量化能发挥作用的底层原因。它们把工作拆成多个真实依赖较短的链，让重命名、调度队列和执行端口有东西可发射。若算法本身每一步都依赖前一步返回的地址或状态，窗口再大也只能等待。

深入理解

“流水线越深越快”是入门阶段的过度简化。深流水线可以支持更高频率，但分支预测失败、依赖等待和前端供给不足都会放大浪费。现代核心的性能来自预测、缓存、乱序窗口、执行端口和内存层级的组合，不来自单一流水线长度。

2.15 乱序执行和依赖链

乱序执行只能重排独立工作，不能消除真实数据依赖。寄存器重命名可以消除某些假依赖，例如两次写同一个架构寄存器但并不互相需要结果；它不能让下一次加法在上一次累加结果还没出来时凭空完成。很多高性能改写，本质上就是把一个长关键路径拆成多个短路径，让乱序窗口里出现更多独立工作。

求和循环是最小例子。朴素版本清楚、正确、适合作 reference，但它把所有加法串成一条依赖链。

Listing 2.23 baseline: 单累加器顺序归约

```

1 std::int64_t sum_baseline(std::span<const std::int32_t> values) {
2     std::int64_t sum = 0;
3     for (const std::int32_t value : values) {
4         sum += value;
5     }
6     return sum;
7 }

```

多累加器版本没有改变算法复杂度，也没有减少读取字节；它改变的是依赖图。多个 partial 之间互不依赖，核心可以在一个 partial 等结果时推进另一个 partial。

Listing 2.24 多累加器：把一条长依赖链拆成多条短链

```

1 constexpr std::size_t SUM_LANE_COUNT = 4;
2
3 std::int64_t sum_multi_accumulator(std::span<const std::int32_t> values) {
4     std::array<std::int64_t, SUM_LANE_COUNT> partials{};
5     std::size_t index = 0;
6     const std::size_t limit =
7         values.size() / SUM_LANE_COUNT * SUM_LANE_COUNT;

```



```

8
9  for (; index < limit; index += SUM_LANE_COUNT) {
10     for (std::size_t lane = 0; lane < SUM_LANE_COUNT; ++lane) {
11         partials[lane] += values[index + lane];
12     }
13 }
14 for (; index < values.size(); ++index) {
15     partials.front() += values[index];
16 }
17
18 std::int64_t total = 0;
19 for (const std::int64_t partial : partials) {
20     total += partial;
21 }
22 return total;
23 }

```

这个例子的重点不是要求手写所有归约。现代编译器可能自动展开和向量化，某些平台上手写版本没有收益。判断必须来自瓶颈证据：如果 baseline 慢在累加依赖，多累加器可能有效；如果输入已经远大于缓存并接近内存带宽，多累加器收益会很小；如果是浮点求和，改变合并树会改变舍入结果，必须先写误差合同；如果展开过度，寄存器压力和代码体积会反过来伤害前端。

指针追踪是相反例子 连续数组扫描虽然会 miss，但地址规律明显，硬件预取器和乱序窗口可以让多个 load 同时在路上。链表遍历不同：下一次地址藏在当前节点里，必须等当前 load 返回。

Listing 2.25 指针追踪：下一次地址依赖当前加载结果

```

1 struct Node {
2     std::int32_t value = 0;
3     const Node* next = nullptr;
4 };
5
6 std::int64_t sum_list(const Node* head) {
7     std::int64_t sum = 0;
8     const Node* current = head;
9     while (current != nullptr) {
10         sum += current->value;
11         current = current->next;
12     }
13     return sum;
14 }

```

两个循环都是线性复杂度，但等待形状完全不同。链表可能每个节点都在不同 cache line 或不同页，TLB 和 cache 都不友好；更关键的是，核心很难提前发起下一次请求。第 3 章会从缓存和 TLB 角度展开，本章先建立流水线直觉：局部性不只是命中率，也是硬件能否提前知道下一步地址。

从两段汇编看地址依赖 第一册已经讲过 x86 内存操作数的地址公式。这里要利用这个公式判断硬件能否并行发起请求。数组求和的热循环可能长这样：

Listing 2.26 连续数组求和的典型地址形状

```
; rdi = values, rsi = count
xor    eax, eax
xor    ecx, ecx
.Larray:
    add    eax, dword ptr [rdi + rcx*4]
    inc    rcx
    cmp    rcx, rsi
    jne    .Larray
```

[rdi+rcx*4] 里的下一次地址可以由 rcx 的递增提前算出。即使某次 load 还没回来，前端和地址生成单元仍能准备后续迭代，硬件预取器也容易识别连续模式。若循环展开成多路累加，窗口里还会出现更多独立 load 和独立加法。

链表求和的热循环可能长这样：

Listing 2.27 链表求和的典型地址依赖

```
; rdi = current node pointer
xor    eax, eax
.Llist:
    test   rdi, rdi
    je     .Ldone
    add    eax, dword ptr [rdi] ; value
    mov    rdi, qword ptr [rdi + 8] ; next
    jmp    .Llist
.Ldone:
```

这里下一轮的基址 rdi 来自当前节点的 next 字段，也就是 mov rdi, qword ptr [rdi + 8] 的 load 结果。当前 load 没返回，下一地址就不存在。乱序窗口可以重排独立工作，却不能猜出链表指针值；预取器也很难从数据相关地址里稳定推出下一步。于是两段汇编都很短，复杂度都线性，但第一个给硬件一个可预测地址流，第二个给硬件一条串行等待链。

汇编形状	硬件能提前做什么	常见瓶颈
[base+i*scale]	计算后续地址，预取后续 line，并行挂起多个 load	带宽、cache 层级、累加依赖
p=[p+offset]	必须等当前节点返回后才知道下一地址	load-use 延迟、cache/TLB miss 串行化
[table+hash*stride]	hash 算出后才能访问，多个 probe 可批处理	随机 miss、TLB 覆盖、冲突

这张表就是从第一册地址公式通往缓存和乱序执行的桥。反汇编里出现内存操作数后，不只要问它读哪个变量，还要问下一次地址是否能提前计算、是否有多个独立地址、是否会跨很多 page、是否会写共享 line。

从一段汇编读出端口、AGU 和依赖链 反汇编不是把 C++ 翻译成另一种文字，而是把语义拆成机器必须调度的微操作。下面这段循环同时包含地址生成、load、整数乘、加法、分支和循环计数：

Listing 2.28 热循环的微架构判读入口

```
; rdi = values, rsi = weights, rdx = count
xor    eax, eax
xor    ecx, ecx
.Lhot:
```

```

mov    r8d, dword ptr [rdi + rcx*4]
imul   r8d, dword ptr [rsi + rcx*4]
add    eax, r8d
inc    rcx
cmp    rcx, rdx
jne    .Lhot

```

第一步看地址。两个内存操作数都用 `base+index*4`，地址生成单元可以在 `rcx` 已知后计算地址。现代核心通常有多个 AGU，AGU 是 address generation unit，负责把基址、索引、比例和位移合成有效地址。若循环里每轮有两个 load 和一个 store，AGU 数量可能成为吞吐上限；若只有一个 load，瓶颈更可能落在 load 延迟、乘法端口或分支上。AGU 不执行内存访问本身，它只是给 load/store 队列提供地址。

第二步看数据依赖。`imul r8d, dword ptr [rsi + rcx*4]` 依赖第一个 load 的结果，也依赖第二个 load 的结果；`add eax, r8d` 又依赖乘法结果。因此单个累加器 `eax` 会形成跨迭代依赖链：第 *N* 轮加法完成前，第 *N*+1 轮的 `add eax, ...` 无法退休为同一个累加器的新值。循环展开并使用多个累加器，目的不是让源码更长，而是把一条长依赖链拆成几条短链，让乱序窗口里同时存在更多独立工作。

Listing 2.29 多累加器把一条依赖链拆成多条

```

acc0 += values[i + 0] * weights[i + 0]
acc1 += values[i + 1] * weights[i + 1]
acc2 += values[i + 2] * weights[i + 2]
acc3 += values[i + 3] * weights[i + 3]
final = acc0 + acc1 + acc2 + acc3

```

第三步看前端形状。循环体很短，分支目标稳定，I-cache 和 uop cache 通常不是主要矛盾；如果把错误处理、虚调用、格式分派和统计更新全塞进同一循环，前端才会开始吃紧。第四步看内存层级。两个数组顺序读取时，硬件预取器容易跟上；若 `weights` 换成按 hash 查表，地址流就从顺序变成随机，瓶颈会从端口和乘法转向 cache、TLB 和内存级并行度。这样读汇编，才能把“优化循环”变成具体问题：是依赖链太长、端口压力太高、前端供给不足，还是地址流太差。

零 idiom、LEA 和宏融合这些小细节为什么值得知道 某些短指令背后有专门硬件处理。`xor eax, eax` 常被识别为 zero idiom，核心知道它把寄存器置零，甚至可以在重命名阶段切断对旧 `eax` 的依赖；这比 `mov eax, 0` 更常见，也解释了为什么反汇编里经常看到它。`lea` 名义上是地址计算，实际常被编译器当作不改 flags 的整数加法和乘小常数工具。若热循环中 `cmp` 紧跟条件跳转，部分微架构能把它们融合成更少的前端工作；但中间插入无关指令、布局跨边界或目标复杂时，这种融合可能消失。

这些细节不应该变成背指令表。更可靠的判读顺序是：先判断依赖链，再判断地址流，再判断前端供给，最后才讨论某条指令本身是否“快”。同一条 `imul` 在内存等待主导时可能不是瓶颈；同一条 `mov` 在指针追踪里可能决定整个循环的延迟。汇编只有放回流水线、缓存和输入分布里，才有解释力。

内存级并行度 内存级并行度 (memory-level parallelism) 指核心同时挂起多个独立内存请求的能力。数组、多个独立游标、分块处理和批量 probe 都能提高它；单链表、递归对象图和每步依赖共享原子结果都会降低它。乱序窗口再大，也不能让一个地址依赖链变成多个独立地址。软件要做的事，是在语义允许时把独立工作显式放进窗口。

2.16 Load/Store Queue、别名和发布可见性

内存操作比寄存器操作复杂。核心要判断一个 load 是否可能读取前面尚未完成的 store；store 可以先进入 store buffer，稍后再对其他核心可见；load/store queue、store buffer、TLB、cache 和一致性协议都会参与成本。硬件会做内存消歧预测，但预测有边界。编译器也会被语言别名语义约束，不能随意重排可能互相影响的读写。

Load Queue 和 Store Queue 的责任 Load queue 记录已经进入窗口的 load：地址是否算出，翻译是否完成，是否命中，是否需要等待更早的 store。Store queue 记录尚未完全对外生效的 store：地址、数据、大小、是否已经越过退休边界、目标 line 是否取得写权限。两张队列共同维护一条规则：程序顺序上更早的 store 若可能写到某个地址，后面的 load 不能随便从缓存读一个旧值当作结果。

最小例子是先写后读同一地址：

Listing 2.30 同线程内 store 后 load 的地址账本

```
store [p] = 7
load rax = [p]

if store address is known and matches load address:
    load should receive 7 from store buffer
else if older store address is unknown:
    load may have to wait or execute speculatively
```

这条规则服务的是单线程语义。即使 store 还没有写入 L1D，后面的同线程 load 也应该看到自己刚写的值。硬件通过 store-to-load forwarding 从 store buffer 直接把数据转给 load。这个转发成功时非常快；失败或不确定时，load 可能等待、重放，或在消歧预测错误后被冲刷流水线。

Store-to-load forwarding 的成功和失败 转发要求地址、大小、对齐和覆盖关系足够清楚。若前面的 store 写 4 字节，后面的 load 读同样 4 字节且地址相同，硬件容易转发。若 store 和 load 部分重叠、跨 cache line、大小不同或地址低位尚未全部确定，硬件可能无法直接转发，需要等 store 写入缓存或让 load 重放。某些看似小的类型转换、非对齐结构体字段和手写序列化代码，会在这里制造隐藏成本。

Listing 2.31 转发友好和不友好的形状

```
friendly:
    store u32 [p + 0] = value
    load  u32 [p + 0]

harder:
    store u16 [p + 1] = value
    load  u32 [p + 0]

harder:
    store u64 [p + 60] = value    ; may cross a cache-line boundary
    load  u64 [p + 60]
```

这种失败不是 C++ 语义错误，而是微架构等待。解析器、二进制协议、压缩格式和日志编码经常在字节数组里做非对齐读写；正确性可以成立，性能仍要通过反汇编和输入规模阶梯验证。若热路径依赖大量部分重叠读写，改成先按自然宽度写临时结构、再统一编码，可能比继续微调表达式更稳。

内存消歧预测的失败账本 乱序核心希望让后面的 load 提前执行，否则一个地址尚未算出的 store 会挡住窗口里很多独立工作。硬件会预测某个 load 与更早 store 不别名，于是先发起 load。若后来发现更早 store 的地址与 load 重叠，load 读到的值不可信，依赖它的年轻微操作要被重放。这叫内存消歧失败或 memory-order violation。

Listing 2.32 一次内存消歧失败的事件链

```
I1: store [base + unknown_index] = value
I2: load rax = [other_base + i]

I2 executes early because hardware predicts no alias
later I1 address resolves and overlaps I2 address
I2 and younger dependent work are replayed
```

接口清楚能减少这种不确定性。只读输入和独占输出分开，热循环中避免把同一 buffer 既当源又当目标，必要时在调试模式检查范围不重叠，都会给编译器和硬件更好的形状。这里的重点不是追求某个特殊关键字，而是让内存访问关系成为接口合同，而不是藏在调用方习惯中。

下面的循环语义清楚，却没有说明 `input` 和 `output` 是否重叠。

Listing 2.33 别名不清会让编译器和硬件更保守

```
1 void add_scaled(std::span<const std::int32_t> input,
2               std::span<std::int32_t> output,
3               std::int32_t scale) {
4     const std::size_t count = std::min(input.size(), output.size());
5     for (std::size_t index = 0; index < count; ++index) {
6         output[index] += input[index] * scale;
7     }
8 }
```

若调用方传入同一数组的重叠视图，某一轮写入可能影响后续读取。编译器为了保持 C++ 语义，必须保守。工程上解决别名问题，不应只靠“我知道不会重叠”的口头承诺，而要靠接口、所有权和测试：输入使用只读视图，输出由调用者保证独占，调试版本检查范围不重叠，或者把算法改成先读后写的两阶段结构。

store buffer 和并发语义 普通 store 进入 store buffer 后，本线程可以继续执行，但其他核心不一定按源码顺序看到写入。后面讲 C++ release/acquire 时，会把这条硬件直觉变成 happens-before 关系。此处先固定一件事：单线程流水线为了吞吐会暂存、预测和重排内部工作；多线程程序若需要可见性顺序，必须通过锁、原子和内存序把边界说清。

热原子不是好的归约 许多初学者把并行求和写成多个线程同时更新一个 `std::atomic`。结果可能正确，却经常很慢。即使用 `memory_order_relaxed`，所有线程仍在争夺同一条 cache line 的独占修改权。Relaxed 只放松顺序约束，不能消除一致性流量。正确方向通常是线程本地 partial：热路径在寄存器和本地槽位中累加，最后再合并。这个原则会在第 4 章、第 9 章和第 13 章反复出现。

2.17 分支预测来自输入分布

分支预测器不是读懂业务逻辑，它根据分支地址、历史方向、目标和上下文猜下一步。循环回边通常容易预测；数据相关、接近随机、目标多变的分支难预测。一次错误预测的成本不是“分支指令本身慢”，而是错误路径上已经取入、译码甚至执行的工作被丢弃，前端重新从正确路径供给，乱序窗口里的有用工作突然减少。

阈值计数是最小例子。

Listing 2.34 带条件分支的过滤计数

```
1 std::int64_t count_greater_branch(std::span<const std::int32_t> values,
2                                   std::int32_t threshold) {
3     std::int64_t count = 0;
4     for (const std::int32_t value : values) {
5         if (value > threshold) {
6             ++count;
7         }
8     }
9     return count;
10 }
```

如果输入已经排序，或者几乎全都大于阈值，分支很容易被猜中。若输入随机且一半大于阈值、一半不大于阈值，预测会困难得多。可以写一个无显式分支版本：

Listing 2.35 把条件转成整数贡献

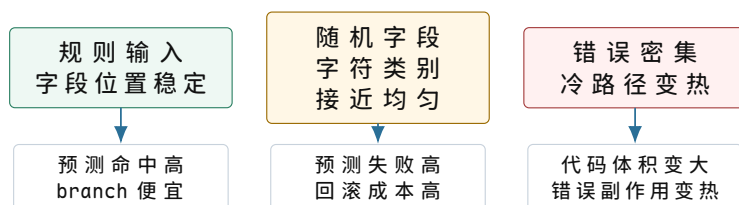
```
1 std::int64_t count_greater_branchless(std::span<const std::int32_t> values,
2                                       std::int32_t threshold) {
3     std::int64_t count = 0;
4     for (const std::int32_t value : values) {
5         count += value > threshold ? 1 : 0;
6     }
7     return count;
8 }
```

这段源码仍然包含条件表达式，编译器可能生成条件移动、集合指令、向量比较，也可能仍生成分支。这里不能保证某种机器码，实验设计必须回答具体问题：规则输入和随机输入是否表现不同；branch-misses 是否同向变化；branchless 版本 instructions 是否增加；总时间是否下降；当输入变得可预测后，收益是否消失。若这些现象吻合，分支预测才是有证据的解释。

branchless 不是银弹 Branchless 改写通常用更多算术、掩码或查表换更少控制依赖。如果原分支高度可预测，改写可能更慢；如果两边工作量差异很大，branchless 会执行本来可以跳过的工作；如果查表引入额外 load，瓶颈可能迁移到缓存或端口；如果掩码表达式增加寄存器压力，编译器可能 spill 到栈。判断 branchless 的唯一可靠方法，是在明确输入分布上测量，并阅读热循环机器码。

冷路径分离 解析器里最常见的前端污染，是把罕见错误处理写在每个字符扫描的内层循环中。错误路径可能构造字符串、记录上下文、格式化日志、分配对象或抛出异常。成熟工程代码不是不处理错误，而是让热循环只记录紧凑错误摘要，例如 record id、byte offset、错误码和少量上下文边界；完整错误对象在块结束后或慢路径中构造。这样正常路径短、直、可预测，错误语义仍然完整。

输入分布怎样改变分支成本



同一段 `if` 在不同输入上有不同硬件成本；实验必须覆盖输入族，不能只跑一种随机数据。

图示：本图要证明的命题是：分支成本来自输入历史是否可预测，不来自源码里是否出现 `if` 这个词。

2.18 前端设计：热路径要短、直、可批量

前端问题最容易被低估。数据库执行器、日志解析器、解释器、正则引擎、协议解析器和多态对象处理器，常常不是因为算术重而慢，而是因为每条记录都做复杂分派、每个元素都走虚调用、每个 `case` 都带着冷错误逻辑，导致 `instruction cache`、分支目标预测和译码供给受压。

虚函数和间接分支 面向对象接口在控制面很有价值，但每个元素一次虚调用可能让热路径变成间接分支和不可内联调用。若对象类型分布稳定，预测器还能学习；若类型随机，目标预测困难，编译器也看不到具体循环体，无法展开或向量化。优化方向通常不是全局废除抽象，而是分离控制面和数据面：控制面保留清晰接口，数据面把一批同类记录交给具体内核批量处理。

大 `switch` 和解释器循环 解释器或字节码执行器常见一个大 `switch`。若 `opcode` 分布随机，分派成本高；若每个 `case` 很小，分派占比更高；若每个 `case` 很大，`I-cache` 压力上升。工业界常用的方向包括 `direct threading`、`superinstruction`、热点 `opcode` 专门化、`JIT`、按 `opcode` 分桶和批处理。每个方向都在做同一件事：减少每个元素上的不可预测控制流，让一批同类工作走同一条紧凑路径。

解析器的批量化 日志解析器也可以借鉴这个原则。朴素状态机按字符推进，语义清楚，适合作 `reference`。优化版本可以先批量扫描结构字符，例如分隔符、换行、引号和转义候选，再让标量慢路径确认复杂语法。这样普通字节走短路径，复杂输入回到完整状态机。这个设计不是免费：短行上启动成本可能不划算，错误密集输入会频繁退回慢路径，代码也更复杂。因此报告必须写普通路径命中率、回退次数和错误摘要一致性。

工程案例

工业设计：`simdjson` 的两阶段思想。约束是 `JSON` 解析既要高吞吐，又要保持完整语义。核心原理是先快速找结构字符和索引，再按语法阶段解析，避免每个普通字节都进入复杂状态机。换来的能力是前端路径更规则、批量扫描更容易向量化；代价是中间结构索引、阶段边界和错误回退更复杂。迁移到 `Compute Systems Engine` 时，不要求照搬 `JSON` 算法，而是采用“先找结构位置，再做语义确认”的缩小版解析器实验。

2.19 工业界的诊断和内核组织

工业案例不能只列系统名。要看它面对什么约束，用什么机制购买什么能力，又付出了什么代价。这里选择四类参照：`PMU Top-Down`、编译器优化报告、数据库向量化执行和 `HPC microkernel`。它们的共同点是把“慢”拆成可观察边界，而不是凭直觉改代码。

PMU Top-Down：先分类，不直接给答案 Intel 的 Top-Down 思路把核心周期粗分为 retiring、bad speculation、frontend bound、backend bound 等方向。它的价值不是自动给出改法，而是阻止分析在错误方向上消耗时间。若 bad speculation 明显，先看输入分布和分支形状；若 frontend bound 明显，先看代码体积、I-cache、间接跳转和译码；若 backend bound 明显，再分依赖、端口、load 和内存层级；若 retiring 高，说明核心大部分时间在提交有用工作，端到端瓶颈可能在别处。

这个方法也有代价。PMU 事件与微架构相关，不同 CPU、虚拟化环境、权限设置和采样方式都会影响可用性。报告不能把某台机器上的事件名写成永久事实。Compute Systems Engine 只采用它的诊断结构：先分类，再深入；有计数器时记录事件，没有计数器时写替代证据和边界。

Top-Down 四类怎样落到热循环 Top-Down 的四个一级分类可以直接映射到日志解析热循环。Retiring 高，说明很多周期在提交有用微操作；这不等于程序已经最优，只表示当前核心内部没有明显空转。Bad speculation 高，通常意味着分支预测失败或机器清空较多；解析器里最常见的原因是字符类别分布随机、错误路径进入热循环、间接分派目标不稳定。Frontend bound 高，说明后端缺少可执行微操作；可能是热函数太大、I-cache/ITLB 供给不稳、uop cache 容不下循环体、间接跳转打乱取指。Backend bound 高，说明微操作到了后端却等资源或数据；要继续区分端口压力、依赖链、L1/L2/LLC/DRAM 等待、store buffer 和一致性等待。

分类	热循环里的典型原因	下一步证据
Retiring	微操作大多成功提交，瓶颈可能在算法字节量或外部阶段	每记录指令数、端到端阶段耗时、reference 对照
Bad speculation	随机分支、错误路径混入、间接目标不稳定	输入分布、branch miss、冷热路径版本对照
Frontend bound	代码足迹大、I-cache/ITLB/uop cache/译码供给不足	函数大小、反汇编范围、调用和跳转目标、内联对照
Backend bound	load 等待、端口拥挤、依赖链、store buffer 或一致性热点	地址序列、工作集阶梯、局部 partial、端口和 cache 证据

这张表的作用是限制推理范围。例如随机输入上 bad speculation 上升，同时 branchless 版本减少 branch miss 且 checksum 不变，可以支持“分支预测是主因”的结论；若 branchless 版本指令数大增、frontend bound 上升、总时间没有下降，就说明改写只是把瓶颈从错误投机移动到前端供给。性能分析的成熟标志不是一次就猜对，而是能解释瓶颈迁移。

同一指标不能脱离分母 PMU 计数器要和分母一起读。branch-misses 绝对值高，可能只是处理的数据更多；要看每条记录、每 KiB、每千条分支的 miss。Cache miss 也一样：一次处理 1 GiB 输入的 miss 总数必然大于处理 1 MiB；关键是每条 cache line 提供了多少有效字节，miss 是否集中在随机表、输入流、输出流或页表。IPC 低不能单独证明后端等内存，因为前端断供、同步等待和故障注入也会让 IPC 低。

Listing 2.36 HotKernelReport 中的 Top-Down 摘要字段

```
topdown:
  retiring_percent
  bad_speculation_percent
  frontend_bound_percent
  backend_bound_percent

normalization:
  records
  input_bytes
```

```

instructions_per_record
cycles_per_record
branch_misses_per_kiB
cache_misses_per_kiB

evidence_boundary:
  counters_available: true
  virtualized_pmu: false
  repeated_runs: 20

```

没有标准化字段，报告会退化成工具输出截图。标准化后的指标可以跨输入族比较：规则日志和随机日志字节数相同，但 branch miss per KiB 差异明显；对象图和列式 batch 记录数相同，但 cache miss per record 差异明显；单线程和多线程处理量相同，但 locked RMW per record 或 coherence 相关事件差异明显。并发、I/O 和分布式报告也应采用同一思想：计数必须绑定业务单位。

从 Top-Down 回到代码改写 Top-Down 只能把方向缩窄，不能替代改写设计。若分类指向 bad speculation，第一步不是立刻写无分支代码，而是构造输入族：全都命中、全都不命中、规律交替、随机、错误密集。若只有随机输入慢，分支预测假设更强；若错误密集输入慢，冷热路径分离更可能有效；若所有输入都慢，可能是前端足迹或后端访存。若分类指向 frontend bound，先比较函数大小、内联深度、调用和间接跳转，再决定是否拆冷路径、减少模板膨胀、把逐行虚调用改成 batch dispatch。若分类指向 backend bound，先画地址序列和依赖链，再决定分块、预取、局部 partial、数据布局或向量化。

这条回路让“工具输出”变成“工程动作”。每一次动作都要保留失败版本：branchless 未提升、过度展开伤害 I-cache、预取污染缓存、padding 增大 TLB 压力，这些都是有价值的负结果。负结果写清楚，下一次才能更快排除错误方向。

编译器优化报告：让编译器解释拒绝理由 LLVM、GCC 和其他编译器能输出向量化、内联、循环优化和优化失败诊断。高性能 C++ 不是和编译器对抗，而是让真实语义进入源码：连续范围、只读输入、独占输出、明确合并合同、可见的循环边界、可内联的小函数。若报告显示因为别名、无法证明依赖或循环控制复杂而不能向量化，修复点往往是 API 和数据结构，而不是手写几条指令。

数据库向量化执行：每次处理一批，而不是每次处理一行 DuckDB、ClickHouse、Velox 等系统的向量化执行器都体现同一个原则：把一批列式数据送入算子，使用 selection vector、bitmap、column vector 和 batch 级状态减少每行分派。约束是分析型查询吞吐优先、数据列式友好、算子可批量执行；能力是更好的缓存局部性、更少虚调用、更容易 SIMD；代价是批次边界、空值语义、选择向量和 pipeline breaker 更复杂。迁移到 Compute Systems Engine，就是让 HotBatch 承载热字段列，让解析、过滤和统计尽量按 batch 运行，而不是每条记录都跨层调用。

HPC microkernel：把依赖、寄存器和内存层级一起设计 BLIS、OpenBLAS 等高性能库的 GEMM 内核不是简单写三重循环。它们用 blocking 控制缓存工作集，用 packing 改变访存形状，用 microkernel 固定寄存器 tile，用 ISA dispatch 选择平台实现。约束是矩阵乘的语义稳定、算术密度高、形状可分块；能力是接近硬件峰值；代价是代码复杂、平台特化和打包开销。这里借鉴的是它的工程态度：先有 reference，再设计数据移动，再设计寄存器级独立工作，最后用报告说明适用形状。

2.20 测量闭环：源码、二进制、计数器

测量闭环有三个环节。源码表达语义；二进制显示编译器实际生成了什么；计数器和耗时显示运行时发生了什么。只看源码，会把“看起来简单”误判成“机器工作少”；只看反汇编，会忘记输入分布和语义边界；只看耗时，会得到无法解释的数字。

反汇编读什么 不需要背所有指令。先找循环体范围、回边、条件跳转、函数调用、间接跳转、load/store 数量、是否有向量指令、是否有栈访问、尾部如何处理。再把这些事实对应回源码：哪个 `if` 变成了分支，哪个条件表达式变成了条件移动，哪个小函数没有内联，哪个 `span` 访问生成了边界或别名保护，哪个错误路径还留在热函数里。

从第一册到本册的四层证据 第一册的反汇编阅读解决“这段机器码怎样实现 C++ 语义”。本册要在此基础上继续写四层证据。第一层是源码语义：reference、输入合同、错误策略和 checksum。第二层是汇编动作：循环范围、内存操作数、分支、调用、向量指令、原子指令和栈访问。第三层是硬件等待：前端供给、分支恢复、依赖链、load miss、TLB miss、store buffer、cache line 所有权或端口压力。第四层是实验边界：输入族、编译选项、机器拓扑、PMU 是否可用、多轮分布和不可验证项。

Listing 2.37 反汇编审查的四层记录

```
source semantics:
  reference checksum, accepted/rejected records, error policy

assembly actions:
  loop range, jcc targets, calls, loads, stores, atomics, vector ops

hardware hypothesis:
  frontend bound, branch recovery, address dependency, cache/TLB,
  store buffer, coherence, memory bandwidth, or port pressure

experiment boundary:
  input family, compiler flags, CPU topology, counters available,
  unavailable evidence, repeated-run distribution
```

这份记录的价值在于阻止跳跃推理。看到 `jne` 不能直接宣布分支预测是瓶颈；还要看输入分布和 branch-misses 或替代趋势。看到 `mov` 不能直接宣布内存慢；还要看地址序列、工作集、TLB 覆盖和层级拐点。看到 `lock add` 才能优先怀疑一致性热点，但仍要用线程数曲线、padding 对照和局部 partial 对照证明。

计数器怎样解释 `perfstat` 可以记录 cycles、instructions、branches、branch-misses、cache-misses 等指标；`perfrecord` 和 `perfanotate` 可以把热点关联到指令地址。它们都不是自动答案。IPC 低可能是前端饿、后端等内存、分支频繁回滚，也可能是程序本来等待同步。Branch misses 高要结合输入分布；cache misses 高要结合字节流量和地址序列；instructions 下降而时间不降，说明瓶颈可能已经迁移。报告要写“当前证据支持什么”，也要写“当前证据不能证明什么”。

受限环境的降级路径 容器、CI、虚拟机和普通用户权限下，硬件计数器可能不可用。证据链不能因此中断。降级路径包括：固定输入族和随机种子，重复多轮记录耗时分位数；保存编译器版本、优化选项和反汇编；改变输入规模做 L1/L2/LLC/内存阶梯；改变输入分布观察趋势；保存 checksum 防止被优化掉；明确写出无法读取 PMU 的原因。没有 PMU 权限，不等于可以写无证据结论。

完整诊断账本：分支改写为什么可能变快，也可能变慢 把热循环从 C++ 推到硬件等待，最适合用分支计数案例练习。朴素版本直接写业务条件：遇到分隔符就计数，遇到非法字符就记录错误。这个版本的好处是语义直观，坏处是每个字节都带着控制流。若输入规律稳定，预测器可以学到大多数分支方向；若字符类别接近随机，核心会反复沿错误路径取指、译码和投机执行，再在分支解析后丢弃这些工作。

Listing 2.38 分支版本保留了最直接的语义

```

1 for (std::uint8_t ch : bytes) {
2     if (ch == ',' || ch == '\n') {
3         ++separator_count;
4     } else if (ch < 0x20) {
5         record_error(ch);
6     }
7 }

```

一种常见改写是查表：把字符类别预先编码成小整数，热循环只做一次表访问和少量算术。它可能减少不可预测分支，却增加一次 load，增加寄存器占用，也可能把错误路径摘要变成后处理。若表留在 L1D，随机输入收益明显；若输入本来规则，分支版本已经预测很好，查表版本可能只是增加了工作。若错误密集，错误后处理仍会进入热路径，收益又会消失。

Listing 2.39 查表版本把控制流转成数据流

```

1 for (std::uint8_t ch : bytes) {
2     std::uint8_t cls = char_class[ch];
3     separator_count += (cls & ClassSeparator) != 0;
4     error_summary |= (cls & ClassError);
5 }

```

这两个循环的比较不能停在源码层。分支版本的汇编里通常会看到若干 `cmp/jcc` 或 `setcc`；查表版本会出现 `char_class+ch` 的内存读、按位操作和加法。前者的风险集中在 bad speculation，后者的风险集中在额外 load、前端指令数、寄存器压力和表的 cache 行为。若只看指令条数，容易误判；若只看 branch-misses，也会忽略查表引入的新瓶颈。

Listing 2.40 两种改写对应的硬件假设

```

branch version:
  expected risk:
    unpredictable jcc, bad speculation, cold error path mixed in loop
  evidence:
    branch_misses per KiB grows on random input
    regular input remains stable

table version:
  expected risk:
    extra data load, register pressure, possible frontend growth
  evidence:
    branch_misses falls, instructions or cache misses may rise
    short inputs may lose to fixed setup cost

```

成熟报告要写“瓶颈是否迁移”。如果查表版本让 branch miss per KiB 大幅下降，但 cycles per KiB 不降，说明原瓶颈已经移动。可能是表读和输入读争用 load 资源，可能是错误摘要仍在热循环里，可能是编译器没有把循环收紧，可能是短输入固定成本占主导。此时继续减少分支没有意义，下一步应看反汇编动作、输入长度矩阵和 cache/TLB 证据。

Listing 2.41 一次分支改写实验的解释模板

```

input family:
    RegularFields, RandomFields, ErrorDense, ShortLines

source change:
    branch classifier -> table classifier

assembly change:
    fewer conditional jumps
    one additional load from char_class
    error construction removed from hot loop

hardware hypothesis:
    random input should reduce bad speculation
    regular input may not improve
    short input may lose to dispatch and setup

decision:
    keep table path for random and long inputs
    keep branch path as reference and short-input fallback

```

这段账本也说明反汇编和计数器的分工。反汇编证明改写确实改变了机器动作：分支少了、load 多了、冷路径是否离开。计数器和耗时证明这些动作在某类输入上是否改善等待。checksum 和错误摘要证明语义没有被偷换。三者缺一项，结论都不完整。

源码阅读任务

源码阅读建议从三个入口任选一个。第一，阅读 Linux `kernel/events/` 和 `tools/perf/` 附近路径，解释用户态 `perfstat` 如何请求事件、内核如何管理计数器、权限如何限制结果。第二，阅读 LLVM `LoopVectorize` 或 `SLP vectorizer` 的文档和一条小循环诊断，解释为什么某个循环没有向量化。第三，阅读一个工业解析器或数据库执行器的热路径，找出 `reference`、`fast path`、`slow path`、`batch` 边界和错误回退。报告必须把源码路径映射回一个实际实验现象。

2.21 项目交付：HotKernelReport

Compute Systems Engine 在这里形成的对象是 `HotKernelReport`。它不是热路径对象，而是实验和回归对象。每一个热内核优化提交，都必须把语义、输入、二进制和运行证据放进同一份报告。后续 SIMD、内核套件、归约和最终项目都复用它。

Listing 2.42 HotKernelReport 的最小形态

```

1 enum class CounterState {
2     Available,
3     Unavailable
4 };
5
6 struct CounterValue {
7     CounterState state = CounterState::Unavailable;
8     std::uint64_t value = 0;
9     std::string note;
10 };

```

```

11
12 struct HotKernelReport {
13     std::string kernel_name;
14     std::string variant_name;
15     std::string input_family;
16     std::string compiler_flags;
17     std::string disassembly_note;
18     std::string loop_address_range;
19     std::string assembly_actions;
20     std::string hardware_hypothesis;
21     std::string unavailable_evidence;
22     std::uint64_t input_bytes = 0;
23     std::uint64_t record_count = 0;
24     std::uint64_t output_checksum = 0;
25     std::chrono::nanoseconds elapsed{};
26     CounterValue cycles;
27     CounterValue instructions;
28     CounterValue branches;
29     CounterValue branch_misses;
30     CounterValue cache_misses;
31 };

```

这个结构故意把 `unavailable` 当成显式状态。没有硬件计数器时，字段不能默填零，因为零会被误读成事件不存在。报告还要包含输入族和反汇编摘要，避免把单次耗时当成结论。若一个优化版本只在 `RandomFields` 上快，在 `RegularFields` 上慢，它不是失败，而是结论边界更清楚；若 `checksum` 变了，则无论耗时多漂亮都必须回滚或修复。

解析器四版本对照 四个版本足以解释前端供给变化。第一版是朴素 `switch` 状态机，作为 `reference`。第二版把字符类别判断改成小表，只改变分支形态。第三版把错误构造移出热循环，主循环只记录错误摘要。第四版做块级扫描，先找分隔符和特殊字符候选，再让标量状态机确认复杂语法。四个版本使用同一输入族，输出同一 `checksum`、字段数和错误摘要。

版本	改变的主要形状	预期收益	主要风险
朴素状态机 查表分类	无，语义最直观 控制依赖换成 <code>table load</code>	<code>reference</code> 可信 随机字符更稳	随机输入分支多 增加 <code>load</code> 和寄存器压力
冷路径分离 块级扫描	正常路径更短更直 每字节状态机变成批处理	前端和预测更干净 为 <code>SIMD</code> 留边界	错误摘要可能不足 短行和错误密集输入可能退化

输入族 至少准备四类输入。第一，规则日志：字段位置稳定，错误率接近零。第二，随机字段：字段长度和字符类别随机。第三，错误密集：非法字节、未闭合引号、连续分隔符和超长字段增多。第四，短行输入：每行很短，批处理启动成本更明显。每类输入都要记录随机种子、字节数、记录数、错误数和字符类别分布。

解析热循环的证据可以汇总成 `HotKernelEvidence`：先运行朴素状态机，保存 `checksum`、错误摘要、反汇编和耗时；再分别运行查表分类、冷路径分离和块级扫描版本。判读必须解释每个版本改变的是分支形态、前端体积、依赖链、`load` 形状还是错误路径。若无法使用 `perf`，写清不可用原因，并用反汇编、输入分布和多轮耗时替代。

2.22 流水线实验门禁和习题

复查必须覆盖五项事实：reference、checksum、错误摘要和边界输入是否完整；编译器、选项、热循环反汇编和优化报告是否可追溯；规则、随机、错误密集和短行输入是否都被覆盖；旧版本卡在哪里、新版本改变了什么、瓶颈迁移到哪里是否有因果解释；机器、编译器、权限和输入分布对结论的限制是否写清。

习题与作业

习题一：为 `sum_baseline` 和 `sum_multi_accumulator` 写同一组正确性测试。输入必须包含空数组、单元素数组、长度不是 `SUM_LANE_COUNT` 倍数的数组、负数数组和大规模随机数组。报告要说明每个测试覆盖哪一种语义边界。

习题二：构造四组 `count_greater` 输入：全真、全假、周期性和随机。比较 `branch` 版本和 `branchless` 版本。若可以使用 `perfstat`，记录 `branches` 和 `branch-misses`；若不能使用，保存命令、错误原因、反汇编和耗时趋势。

习题三：把一个链表求和改写成连续数组求和。不要只写“数组更快”，要从地址依赖、预取、TLB、cache line、分配器和对象生命周期解释两者差异，并说明哪些业务语义会阻止替换。

习题四：选择项目中的一个解析或统计热循环，提交一份 `HotKernelReport`。报告必须包含输入族、reference、checksum、反汇编摘要、计数器或替代证据、结论边界和下一步实验。只提交优化版代码不合格。

视角从核心内部推进到数据路径之后，很多等待会暴露出不同来源。乱序窗口可以隐藏一部分延迟，但它需要独立请求；分支预测可以保持前端供给，但它不能修复随机访存；多累加器可以拆依赖链，但它不能让远端内存变近。cache line、TLB、页表和地址序列解释的是这些等待如何进入核心。

工程案例

知识点小案例：随机分支为什么会把热循环拖慢。先看一个二值分类循环：统计数组中大于阈值的元素。若输入全小于阈值，分支方向稳定；若输入按 0、1、0、1 交替，预测器仍可能学到模式；若输入由随机数生成，分支方向接近不可预测。三组输入做同一件事，结果完全一样，但流水线看到的是三种前端和控制流形状。由特例推出规律：分支预测不是“if 快不快”的抽象问题，而是输入分布能否给预测器稳定历史。工程报告必须同时保存输入分布、checksum、分支版和 `branchless` 版反汇编；没有这些证据，不能把波动简单归因于“CPU 慢”。

第 3 章

缓存、TLB 与虚拟内存的地址路径

前面把视角放在核心内部：前端怎样供给微操作，乱序窗口怎样寻找独立工作，分支预测失败为什么会丢掉已经做过的事。接下来继续追问一个更基本的问题：一条 load 指令真正要访问哪个地址，那个地址附近还有多少有用数据，地址翻译能不能命中，第一次访问页面时操作系统要做什么，多个核心写同一条 cache line 时会发生什么。

仍然从 Compute Systems Engine 的日志统计任务开始。第一版实现很自然：每读一行日志，就分配一个 `LogRecord` 对象，里面保存原始文本、事件名字符串、用户字符串、时间戳、错误信息和若干指针；随后把对象指针放进容器，聚合阶段再遍历这些对象并更新哈希表。它在小输入上容易写、容易调试，也能和顺序 reference 对齐。输入放大后，另一个版本却明显更快：它把记录的稳定身份保存为 `RecordId`，把热字段压成连续数组，把事件名编码为整数，错误文本和原始行只留在冷存储中，需要诊断时再回查。

两个版本的算法复杂度都可以写成线性。差别不在大 O，而在地址路径。对象版让核心追踪堆节点、字符串、桶数组和错误对象；列式热字段版让聚合阶段顺序扫描事件 id 和计数槽。前者每处理一条记录可能触碰多条 cache line、多个页面和多次指针依赖；后者让硬件预取器看到稳定步长，让 TLB 用较少翻译项覆盖更多有效数据。缓存、TLB 和虚拟内存不是背景知识，它们直接决定“线性扫描”到底像顺着连续道路前进，还是像在城市里按纸条找下一扇门。

分析沿六条线展开：把 C++ 对象图翻译成 CPU 看到的地址序列，区分连续扫描、固定步长、随机索引、指针追踪和冷热字段混读；解释 cache line 为什么既是空间局部性的机会，也是 false sharing 和写共享的争用单位；用工作集、活跃页数、TLB 覆盖、page walk 和 first-touch 解释同一算法在不同输入规模下的性能拐点；区分分配、触页、预热、热循环、mmap 冷读、page cache 热读和持久化写回，避免把系统状态混进算法结论；为日志统计、哈希聚合、partial 数组、输入读取和 checkpoint 写出地址路径实验，而不是只给容器名和耗时；在 Compute Systems Engine 中形成 `MemoryShape` 和 `AccessTrace` 报告，让后续数据布局、线程、I/O 和分布式机制复用同一张地址地图。

工程案例

核心案例是同一份日志输入的四种地址形状：胖对象数组、指针对象图、热字段列式 batch、按 key 分区后的局部聚合。四种形状必须输出同样的统计结果、错误摘要和 checksum。比较它们时，不只记录总时间，还要记录热字段字节数、冷字段是否进入热路径、活跃页面、访问顺序、是否预热、是否存在共享写和可用的 cache/TLB 证据。

沿用同一段热块时，这里只把其中的 load/store 地址拿出来。核心内部问题问“这些指令能否被持续消化”；地址问题问“这些指令访问的地址到底是什么形状”。例如：

Listing 3.1 同一更新块在本章中的地址账本

```
event_type[i]:
    address = base_event_type + i * 4
    usually a stride load over a hot column

value[i]:
```



```

address = base_value + i * sizeof(value)
another stride load if values are columnar

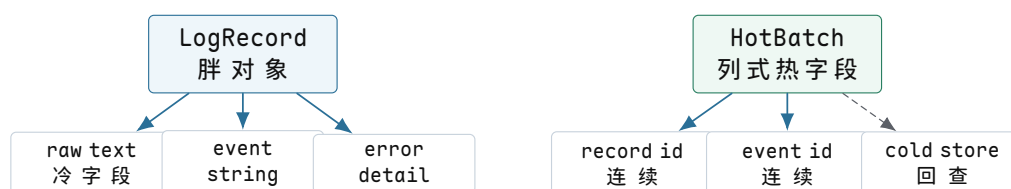
counts[event_type]:
    address = base_counts + event_type * sizeof(counter)
    small local array, hash bucket, shard, or global counter

diagnostic path:
    record id -> cold storage -> original bytes or error context
    should not be touched by every normal record

```

这份地址账本把本章边界切清楚。若 `event_type[i]` 和 `value[i]` 是连续列，本章会解释为什么 cache line、TLB 覆盖和预取器容易工作；若它们藏在胖对象或链表里，本章会解释为什么每条记录会多碰 cache line 和页面；若 `counts[event_type]` 是本地小表，本章先把它当作普通地址路径分析；若它被多个核心高频写，第四章再把同一地址放进一致性和 NUMA 拓扑里。

同一条记录在对象图和地址图中的差异



对象图说明程序员看到的关系；地址图说明热循环实际读取哪些连续字节、跳转到哪些堆对象、跨越多少页。性能判断必须从地址图开始。

图示：本图要证明的命题是：同样的业务记录可以产生完全不同的地址路径；优化首先改变的是机器要走的路。

3.1 地址首先是硬件选择问题

程序里的指针看起来像一个整数，源码里写的是 `records[i].event_id` 或 `node->next`。硬件面对的不是字段名，而是一组地址位。地址位进入译码电路，选择某个 cache set、某个 TLB 项、某条总线请求、某个内存控制器队列，最后落到 DRAM 的行、列和 bank。缓存、TLB、预取器和页表之所以存在，是因为核心不能每次都把完整地址请求送到远处主存再等待答案。地址必须先在近处被筛选、缓存和预测。

从这个角度看，内存访问不是“读一个变量”这么简单。一次 load 至少包含五个问题：虚拟地址是否合法；虚拟页能否翻译成物理页；目标 cache line 是否在 L1D；若不在，是否在 L2、LLC 或其他核心的缓存中；取回 line 后，指令需要的是其中哪些字节。写入还要多问几个问题：是否需要先取得独占权限；是否需要把旧 line 读入；其他核心是否持有这条 line；写入何时对其他核心可见。先把单核地址路径讲清，一致性和内存序才有落点。

为什么不能直接访问主存 执行单元附近的寄存器很小但极快；片上 SRAM 可以做成 L1/L2 cache，容量较小但延迟低；主存容量大，距离远，延迟和能耗都高。若一个循环每次 load 都等待主存，乱序窗口很快被未完成 load 填满，执行端口没有可发射的独立工作。缓存层级的动机不是“多做几层表”，而是在核心附近保存最近和附近的字节，让常见访问不必走到远处。

这个推导也解释了为什么缓存不能无限大。缓存越大，查找路径、连线、面积和能耗越高；离核心越近，越需要小而快。L1D 通常追求低延迟和高带宽，容量有限；

L2 稍大稍慢；LLC 更大，也要服务多个核心。不同处理器参数会变，但交换不变：越近越快也越小，越远越大也越慢。软件的局部性，就是让正在使用的工作集尽量停留在更近的层级。

地址序列决定硬件能否提前工作 连续数组扫描时，下一次地址可以从当前地址加固定步长得到。硬件预取器可以在程序真正请求之前，把后续 cache line 提前拉近；乱序窗口也可以同时挂起多个独立 load。指针追踪不同，下一地址藏在当前节点内容里，当前 load 没回来就不知道下一步去哪。随机索引数组介于二者之间：索引流本身可能连续，但数据地址由索引决定，预取器未必能看出规律。

因此，地址序列比算法复杂度更接近硬件等待。两个 $O(n)$ 循环，一个是 `base+i*4`，另一个是 `p=p->next`，大 0 相同，硬件并行度、cache line 利用率、TLB 覆盖和预取效果都不同。性能报告必须把“访问什么地址”写出来，否则后面的缓存名词没有落点。

核心理想

内存层级的第一原理是距离和选择：核心需要的字节可能在寄存器旁边，也可能在芯片外；地址位决定先查哪里，命中在哪里，失败后向哪里请求。

一个字节从文件到寄存器的生命周期 为了避免把 cache、TLB 和 page cache 讲成孤立部件，先追踪日志文件中的一个字符。它最初在存储设备或内核 page cache 中；通过 `read` 时，内核把它复制进用户 buffer；通过 `mmap` 时，用户虚拟地址在访问时映射到对应文件页。随后 parser 执行一条 load 指令，虚拟地址进入 DTLB 翻译；翻译成功后，物理地址进入 L1D cache 查找；L1D miss 时向 L2、LLC 或内存请求整条 cache line；line 到达后，load 从 line 中抽出目标字节放进寄存器；分支或查表逻辑再使用这个寄存器值判断字段边界。

Listing 3.2 一个输入字节到达寄存器的事件链

```
storage or page cache contains file bytes
read copies bytes to a user buffer
or mmap maps a file page into the process address space

parser computes virtual address of byte i
DTLB translates virtual page to physical page
L1D cache looks up the physical cache line
if miss, lower cache or memory supplies the whole line
load extracts one byte from the line
register receives the byte
branch or table lookup classifies the byte
```

这条链说明“读一个字符”不是单层动作。若文件页尚未驻留，等待可能在 I/O；若虚拟页尚未映射，等待在 page fault；若页表翻译不在 TLB，等待在 page walk；若数据 line 不在 cache，等待在内存层级；若分支预测错了，刚读到的字节可能属于错误投机路径，后续工作会被丢弃。不同等待有不同修复方向，不能全部写成“内存慢”。

虚拟地址、物理地址和文件偏移不是同一个身份 文件偏移描述字节在文件中的位置；虚拟地址描述进程访问它的位置；物理地址描述内存控制器最终看到的位置。通过 `read` 读取时，同一个文件偏移被复制到用户 buffer 的某个虚拟地址；通过 `mmap` 读取时，文件偏移和虚拟页之间建立映射；同一个文件页还可能被多个进程映射到不同虚拟地址。程序指针只能表示当前进程的虚拟地址，不能作为持久身份写进诊断、manifest 或网络消息。

这解释了为什么第一章使用 `RecordId` 而不是裸指针。裸指针可以让当前进程快速定位内存，却无法在重启后定位文件内容，也无法证明另一个 attempt 处理的

是同一输入 generation。文件偏移和 checksum 才能跨进程、跨重试、跨持久化边界保持意义。地址路径优化如果丢掉这种身份，后续恢复会失去依据。

身份	由谁解释	适合解决的问题
文件偏移 虚拟地址	文件系统和输入合同 当前进程页表	诊断回查、split 边界、恢复验证 用户态 load/store、指针和视图 生命周期
物理地址	硬件缓存、TLB 和内存控制器	cache set、NUMA 位置、互连和带宽
逻辑 record id	项目状态机	重试、partial、manifest 和审计

地址生命周期推出视图生命周期 很多解析器为了避免拷贝，会让字段视图直接指向输入 buffer 或 mmap 区域。这样做可以减少字符串分配，却把字段生命周期绑定到底层存储。若 buffer 被复用，视图会指向新内容；若 mmap 被解除，视图变成悬空引用；若文件被截断，后续访问可能触发错误；若字段视图跨线程传递，释放时机必须被同步保护。零拷贝不是免费优化，它只是把拷贝成本换成生命周期证明。

因此，一个阶段如果只需要临时比较 event name，可以使用 span；如果结果要跨 batch、跨线程、跨 checkpoint，就需要编码、复制或持久化身份。第六章的数据布局会继续展开这一点：热路径可以用短生命周期视图减少搬运，系统边界必须用稳定身份防止悬空。

3.2 取指路径和取数路径必须同时看

很多性能分析把缓存简化成“数据是否在 cache 中”。实际热循环同时消耗两条流：指令流和数据流。指令流由程序计数器驱动，经过 ITLB、I-cache、译码器或 micro-op cache，最后变成后端可发射的微操作；数据流由 load/store 地址驱动，经过 DTLB、L1D/L2/LLC、store buffer、一致性协议和内存控制器。两条流任何一条断供，核心都可能等待。

为什么 I-cache 会影响数据处理 解析日志看似是数据问题，但 parser 的控制逻辑本身也要被取入。若热函数包含大量模板实例、异常处理、格式化日志、虚调用和罕见错误路径，热循环的指令工作集会膨胀。I-cache miss 或译码供给不足时，即使数据已经在 L1D 中，执行单元也可能没有足够微操作可做。前端瓶颈常见于解释器、正则引擎、协议解析器、数据库表达式执行器和复杂状态机。

把错误路径移出热函数、把多态逐行调用改成批量 dispatch、把稀有 case 放到慢路径，不只是减少分支，也是在缩小指令地址工作集。I-cache 的关键问题是“下一批要执行的指令字节是否已经在近处”，不是“代码行数是否少”。

ITLB 和 DTLB 是两张不同账本 指令地址也要翻译。PC 给出的是虚拟地址，取指路径需要 ITLB；数据 load/store 需要 DTLB。一个大程序可能数据局部性很好，却因为热代码分散、插件化、JIT 代码过多或模板展开导致 ITLB/I-cache 压力。反过来，一个短小循环可能取指稳定，却因随机数据访问导致 DTLB 和 D-cache 压力。报告中把所有 miss 都叫“缓存不好”，会掩盖修复方向。

路径	访问对象	常见修复方向
取指路径	热代码字节、分支目标、译码结果	冷路径分离、批量 dispatch、减少间接跳转、缩小热函数
取数路径	输入字节、热字段、桶、节点、计数槽	连续布局、分块、字典编码、局部 partial、减少随机页
写入路径	目标 cache line、store buffer、所有权状态	独占写区间、padding、分片、批量提交

一条 `load` 之前，前端已经做了很多事。当源码写下 `value=event_types[i]`，真正的数据 `load` 发生之前，前端必须先取得并译码对应指令。若分支预测错误，后端可能已经发起了错误路径上的 `load`，之后又被冲刷；若取指 `miss`，正确路径上的 `load` 甚至还没机会进入后端。于是“数据 `cache` 命中”并不保证循环快。第 2 章讨论前端，本章讨论数据地址，实际判读要把二者相乘：热循环要既能持续供给指令，又能让 `load/store` 地址可预测、可翻译、可缓存。

取指和取数的共同敌人：分散 取指分散表现为代码块多、跳转目标多、内联过度或解释器分派频繁；取数分散表现为堆节点、字符串、随机桶、跨页对象和远端页面。二者的底层相似：有限的近端缓存装不下当前工作集，下一步位置难以提前知道。批量化能同时改善两者：一批同类记录走同一个小内核，指令路径稳定；同一批热字段连续存放，数据路径稳定。

3.3 数据 `cache` 的组相联结构

第二章用 `I-cache` 解释了指令字节怎样被缓存。数据 `cache` 的基本结构相似，也要把地址拆成 `offset`、`set index` 和 `tag`；不同之处在于，数据 `cache` 还要处理读写、`store buffer`、写分配、原子修改和多核一致性。先看单核 `load`。处理器得到一个物理地址后，用低位 `offset` 选择 `cache line` 内字节，用中间位选择 `set`，用高位 `tag` 判断某个 `way` 是否正好保存目标 `line`。命中则返回 `line` 中需要的字节；未命中则向下层缓存或内存请求整条 `line`。

从汇编内存操作数进入地址路径 第一册读过这样的指令：

Listing 3.3 一个普通数组 `load` 的汇编形状

```
mov eax, dword ptr [rdi + rcx*4]
```

从 `ISA` 语义看，它把地址 `rdi+rcx*4` 处的 4 字节读入 `eax`。这里要继续拆这条指令在硬件里的路径。首先，地址生成单元读取 `rdi` 和 `rcx`，计算虚拟地址；其次，`DTLB` 用虚拟页号查物理页号和权限；再次，`L1D` 用物理地址中的 `set/tag` 查找 `cache line`；命中后，用 `line` 内 `offset` 选择 4 字节；若这些字节跨 `line`，还要处理第二条 `line`；若 `DTLB` 或 `L1D miss`，就把等待推向 `page walk` 或更低缓存层级。

Listing 3.4 把一条 `load` 汇编展开成硬件问题

```
mov eax, dword ptr [rdi + rcx*4]

address generation:
    virtual = rdi + rcx * 4
translation:
    virtual page -> physical page, permission check
cache lookup:
    physical line -> L1D set/tag/way
byte selection:
    line offset -> four bytes -> eax
```

这就是为什么“汇编相同，速度不同”并不矛盾。同一条 `mov`，当 `rdi` 指向连续热数组时，可能每条 `line` 服务多个元素；当 `rdi` 指向分散对象图时，可能每次都等待新 `line` 和新页；当它跨页访问尾部时，还可能触发第二次翻译甚至异常。汇编给出地址公式，缓存和 `TLB` 给出等待位置。

地址生成本身也有资源 现代核心通常有专门的地址生成单元处理 `base+index*scale+displacement`。简单连续数组访问容易被硬件快速计算；复杂索引、多

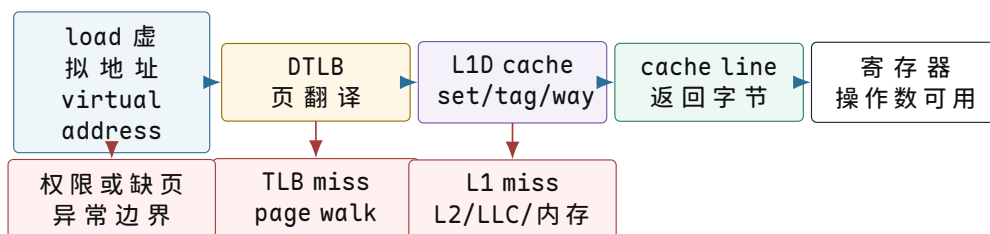
个地址流和同时发生的 load/store 会争用地址生成资源。比如一次聚合更新可能先读 `event_ids[i]`，再按 event id 读桶，再写计数槽；源码上是几行，汇编上是多条内存操作和多个地址公式。若每轮有多个 load 和 store，瓶颈可能不是 ALU，而是地址生成、load 端口、store 地址端口或 cache miss。

Listing 3.5 一次哈希更新可能展开成多条地址路径

```
mov    edx, dword ptr [event_ids + rcx*4]    ; read event id
imul   r8d, edx, hash_multiplier           ; hash/index math
mov     r9, qword ptr [buckets + r8*8]      ; read bucket pointer
add     qword ptr [r9 + count_offset], 1    ; update counter
```

读这段汇编时，不能只数指令条数。第一条是连续数组 load，第二条是整数计算，第三条可能是随机 bucket load，第四条是写共享或本地计数槽。每条地址路径都有自己的 cache line、TLB 和一致性问题。数据布局优化的目标，就是把随机、跨页、共享写的路径尽量移出高频循环，或把它们批量化。

一次数据 load 的地址路径



一次 load 不是单个动作：先证明虚拟地址能翻译，再查数据 cache；任一层 miss 都会把等待推向更远的结构。

图示：本图要证明的命题是：数据访问的成本来自地址翻译、cache 查找和 miss 恢复的串联，而不是来自变量名本身。

为什么按 cache line 搬运 若硬件只搬运一个字节，每次顺序扫描都要产生大量小请求，控制成本和总线利用率都会很差。把连续地址按固定大小分成 cache line 后，一次 miss 可以服务后续多个相邻元素。扫描 `std::int32_t` 数组时，一条 64 字节 line 可以包含 16 个整数；处理器取第一个整数时带来的 line，后续若干元素可以直接命中。对象图或随机访问的低效，也正是在这里出现：一条 line 被搬进来，却只用其中几个字节。

cache line 是搬运单位，不是 C++ 对象单位。一个对象可能跨两条 line；多个对象可能共享一条 line；一个字段可能和完全不相关的冷字段同处一条 line。结构体布局、字段顺序、对齐、padding、数组元素大小和分配器返回地址，都会改变 line 内哪些字节被一起搬运。性能分析要问“这条 line 里有多少字节被当前阶段使用”，而不是只问“对象是否连续”。

set、way 和冲突 数据 cache 也会有冲突。若两个热数组的地址间隔让它们反复映射到同一组，即使总工作集小于 cache 容量，也可能互相替换。步长访问尤其容易暴露这个问题：每次跨固定大步长，低位 offset 不变，中间 set 位按某种周期变化，可能只打到少数 set。实际处理器有复杂索引和替换策略，不应手算成绝对结论；但理解 set/way 能解释为什么“数据总量不大”仍可能 miss 高。

工程上常见的缓解包括改变布局、分块、给数组加少量 padding、把多个并行数组合并或拆分、调整 batch 大小、避免多个大表以相同步长同步访问。每个办法都可能反噬：padding 增大内存占用，合并字段可能把冷字段带热，拆分字段可能增加地址流。正确做法不是套公式，而是先画访问序列，再用规模阶梯和计数器验证。

一次 store 比一次 load 多出什么 Load 需要找到最新可见数据,store 还要获得修改权。单线程、线程私有数据上,这个差别常被缓存和 store buffer 隐藏;多核心共享写时,差别会变成主要成本。处理器不能只把一个字节写到某个抽象变量里,它要把包含目标字节的整条 cache line 置于可写状态。若其他核心持有 Shared 副本,需要让它们失效;若另一个核心持有 Modified 副本,需要拿到最新 line。只有这些动作完成后,store 才能安全修改本地副本。

汇编上,store 看起来也许只是反方向的 mov:

Listing 3.6 普通 store 的汇编形状

```
mov dword ptr [rdi + rcx*4], eax
```

这条指令至少带来两条依赖:地址依赖 rdi/rcx,数据依赖 eax。硬件可以先把 store 放入 store buffer,让后续指令继续;但它最终仍要让目标 line 进入可写状态,并把字节写到正确 offset。若该 line 不在本地缓存,很多实现会先按 write-allocate 拉入整条 line,再修改其中 4 字节。若目标 line 被其他核心共享,还需要一致性协议参与。于是普通 store 在私有、连续、热输出上可以很快;在共享、随机、跨核心写上可能很贵。

内存读改写比普通 store 更强 有些汇编把读和写合成同一条内存操作,例如:

Listing 3.7 内存读改写的汇编形状

```
add qword ptr [rdi], 1
```

从 ISA 语义看,它读取 [rdi] 的旧值,加一,再写回同一地址。单线程时它仍可能由多个内部动作完成;多核心时,如果它是普通非原子访问,C++ 同步语义仍然不成立。若编译器为 std::atomic::fetch_add 生成带锁定语义的读改写写,硬件还必须保证该操作在线性化点上不可被其他核心穿插。这里先固定区分:mov [mem], reg 是普通 store,add [mem], imm 是读改写写形状,带锁定语义的原子读改写写还会加入跨核心排他要求。

汇编形状	基本动作	容易出现的成本
mov reg, [addr]	load	TLB miss、cache miss、地址依赖 store buffer、写分配、写回带宽 load/store 依赖、line 修改权 独占所有权、一致性流量、竞争等待
mov [addr], reg	store	
add [addr], imm	读旧值、计算、写回	
lockadd [addr], imm	原子读改写写	

Listing 3.8 一次 store 的额外问题

```
compute virtual address
translate address through DTLB
find cache line
obtain writable ownership for the line
place store into store buffer or write cache line
make the write visible according to memory ordering rules
```

这解释了为什么“每线程写自己的计数器”仍可能慢。如果多个计数器落在同一条 line,worker 0 写 counter[0],worker 1 写 counter[1],硬件看到的仍是同一条 line 被两个核心反复修改。第三章先从 cache line 单位解释这个事实,第四章会把它放进一致性协议和互连消息里,第十一章再讨论原子和内存序。

写分配为什么会读旧 line 许多缓存采用 write-allocate 策略：写一个当前不在缓存中的地址时，先把整条 line 读入缓存，再在本地修改它。这对“读后写”很合理，因为后续可能继续访问同一 line；对一次性大输出，则可能浪费带宽。比如写一个从未读取、不会很快再读的大数组，每次 store miss 都可能先拉入旧 line，再写新数据。某些平台和编译器可以使用 streaming store 或 non-temporal store 提示，绕开部分缓存污染，但它们有对齐、大小、顺序和可移植性边界，不能当作默认选择。

日志系统写 output block 时也会遇到这件事。若先在用户 buffer 中顺序生成，再一次性写出，用户态 store 仍会经过 cache；若 output 很大，它可能挤掉 parser 和聚合的热数据。若使用 direct I/O 或注册 buffer，又会引入对齐、生命周期和错误处理约束。I/O 章节讲异步写之前，必须先知道普通 store 和普通 write 已经在内存层级中制造状态。

Listing 3.9 写入策略的粗分类

```
ordinary cached store:
    good for data that will be read or modified soon
    may allocate cache lines and evict other hot data

streaming or non-temporal store:
    useful for large one-way output on suitable platforms
    requires platform-specific validation and careful ordering

kernel write path:
    copies or maps user bytes into kernel-managed I/O state
    completion and durability are separate questions
```

非对齐和跨 line 访问 若一个字段跨越 cache line 边界，一次 load 可能需要读取两条 line；若跨越页面边界，还可能涉及两次地址翻译和两种权限结果。多数现代处理器能处理许多非对齐访问，但成本和异常边界会变复杂。数据结构设计不必把所有字段都过度对齐，却要避免让热路径中高频访问的多字节字段反复跨 line 或跨页。

字符串解析尤其容易忽略边界。一个 SIMD 扫描可能一次读取 16、32 或 64 字节，若读取越过 buffer 末尾，哪怕后续用掩码丢掉多余字节，也可能触碰无效页面。安全实现要么保证输入末尾有 padding，要么在尾部使用安全标量路径，要么使用不会越界 fault 的加载策略。第七章讲 SIMD 时会回到这里：向量宽度不是只影响速度，也影响边界安全。

Listing 3.10 数据 cache 查找的抽象伪代码

```
1 LoadedBytes load_bytes(Address virtual_address, std::size_t width) {
2     PageTranslation page = dtlb_translate(virtual_address);
3     Address physical = page.physical_base + virtual_address.page_offset();
4
5     LineOffset offset = low_bits(physical);
6     SetIndex set = middle_bits(physical);
7     Tag tag = high_bits(physical);
8
9     for (Way way : l1d.sets[set].ways) {
10         if (way.valid && way.tag == tag) {
11             return way.line.bytes(offset, width);
12         }
13     }
```

```

13 }
14
15 CacheLine line = lower_cache_hierarchy.read_line(physical);
16 replace_one_way(l1d.sets[set], tag, line);
17 return line.bytes(offset, width);
18 }

```

这段伪代码省略了乱序执行、store-to-load forwarding、非对齐访问、权限异常、硬件预取、多核一致性和 speculation，但它保留了关键因果：虚拟地址先翻译，物理地址再查 cache，miss 后按 line 填充。后续所有布局优化都可以回到这个伪代码提问：它减少了翻译次数，减少了 line 数，减少了冲突，还是让更多 load 能并行挂起。

3.4 先画地址序列，再谈缓存层级

缓存章最常见的坏写法，是从 L1、L2、L3 的容量和延迟列表开始。参数有用，但它们不是分析起点。起点应当是一串地址：base+0、base+4、base+8 这样的连续扫描；每次跨 64 字节或 4096 字节的固定步长；按随机索引访问数组；从节点里读出下一节点地址的指针追踪；先访问 bucket 再访问节点再访问字符串的哈希查询。硬件看到的是这些地址，不是变量名、类名或容器名。

连续扫描和指针追踪不是同一种线性 数组求和和链表求和都是 $O(n)$ ，但它们让核心等待的方式不同。数组扫描的下一地址可以由当前地址加固定步长得到，cache line 里通常包含多个后续元素，硬件预取器有机会提前请求后面的 line。链表追踪必须先等当前节点加载回来，才能知道下一节点地址。乱序执行可以并行多个独立 miss，却很难提前追一条单独的 next 指针链。

这就是为什么前面的“内存级并行度”会在地址路径里继续出现。连续数组可以让多个独立 load 同时在路上；随机指针链常把关键路径变成“等一个节点，再等下一个节点”。若链表节点还由普通分配器分散到许多页面，TLB miss 和 cache miss 会叠加。复杂度描述增长阶，地址序列描述每一步要等什么。

容器名不是访问模式 `std::vector` 连续存储，但如果用随机索引访问，它仍然可能局部性很差。`std::deque` 分段连续，适合两端增长，但长距离扫描不如单个连续数组直接。`std::list` 插入删除稳定，却让遍历变成节点追踪。`std::unordered_map` 平均查找是常数，但一次查询可能包含 bucket、节点、key 字节和 value 的多次随机访问。不能只背容器复杂度表，还要问业务热路径怎样访问它。

日志统计中的事件名聚合就是典型例子。直接用字符串作为 key 更新 `std::unordered_map`，一次记录可能要读字符串、计算哈希、访问 bucket、追节点、比较字符串，再写计数。若先把事件名字典编码为整数，后续聚合就可以更新紧凑数组或局部整数哈希表。编码阶段并没有消失，它只是把昂贵的字符串地址路径集中在前面，让后续多次聚合使用更短、更可预测的地址路径。

对象图和地址图 对象图描述业务关系：记录有字段，字段有字符串，错误对象引用原始行。地址图描述硬件路径：某个字段和哪些冷字段落在同一条 cache line；字符串内容是否在另一块堆内存；哈希节点是否跨页；同一线程下一步能否预取；不同线程是否写同一条 line。严谨的性能报告必须把对象图翻译成地址图。

这个翻译常常暴露接口问题。若聚合阶段只需要 `event_id`，却必须拿到完整 `LogRecord` 才能调用接口，冷字段就会被迫进入热路径。若错误报告只需要少数坏行，却每条正常记录都构造完整错误上下文，前端和缓存都会被污染。把热字段和冷字段分开，不是追求某种架构潮流，而是让每个阶段只移动自己真正需要的字节。

核心思想

局部性不是容器属性，而是布局、访问顺序、字段冷热、生命周期和共享写共同产生的结果。说“vector 更快”不够，要说它让哪一段地址变连续，

让哪些 cache line 的有效字节比例提高。

3.5 Cache line：搬运单位也是争用单位

处理器通常按 cache line 搬运数据，常见大小是 64 字节，具体平台要查询。程序读取一个 4 字节整数时，硬件通常把它所在的整条 line 带入缓存。顺序扫描数组时，这很有利，因为一条 line 包含多个相邻元素；扫描胖对象的单个小字段时，这可能浪费，因为同一条 line 里大部分字节不是热路径需要的数据。

从总线效率推出整行填充 下层缓存和内存不是为每个字节单独服务的。如果核心要读一个 4 字节字段，硬件仍然倾向于把包含它的一整条 line 拉近，因为控制一次请求、仲裁一次总线、检查一次 tag 和维护一次一致性状态都有固定成本。整行填充把固定成本摊到多个相邻字节上。这个设计依赖一个假设：程序访问一个地址后，很可能访问附近地址。连续数组、结构化扫描和紧凑 batch 会兑现这个假设；随机指针图和只读胖对象中的一个小字段会浪费这个假设。

整行填充也解释了为什么“读少量字段”可能并不便宜。假设一个对象 160 字节，热阶段只读其中 8 字节。即使对象数组完全连续，每条记录仍可能带来多条 line 的搬运；如果这些对象里还混有字符串指针和错误详情，后续访问会跳到更多 line。列式布局不是因为名字高级，而是把同一阶段真正要读的字段压到相邻 line，让一次搬运服务更多有效字节。

有效字节比例 设一个 `LogRecord` 对象有 128 字节，聚合阶段只需要其中 4 字节事件 id 和 4 字节计数输入。即使对象数组连续，硬件仍可能为每条记录搬运多条 cache line，而热路径只使用很少字节。列式 `HotBatch` 把事件 id 连续存放后，一条 cache line 可以携带多个事件 id，有效字节比例明显提高。这个变化比“结构体数组还是列式数组”的名字更重要。

有效字节比例也解释了为什么 map 融合和冷热拆分有边界。若多个字段总是一起使用，把它们拆得太散会增加多条独立地址流；若冷字段很少访问，把它们和热字段绑在一起就浪费带宽。布局优化的第一句话应当是“本阶段每条记录实际读取哪些字段”，而不是“我要使用 SoA”。

False sharing Cache line 也是多核一致性的基本单位。两个线程写不同变量，如果变量落在同一条 cache line 上，硬件仍然要让这条 line 的独占所有权在核心之间迁移。这叫 **false sharing**（伪共享）：软件层面没有共享同一个字段，硬件层面却共享了同一个搬运和一致性单位。它不会让结果错误，却会让扩展性突然崩掉。

并行计数很容易中招。源码里每个 worker 写自己的 `partial[worker]`，逻辑上互不干扰；如果这些 8 字节计数器紧挨着，多个 worker 可能反复写同一条 64 字节 line。对齐和填充能把高频写槽位分到不同 line，但它也增加内存占用。百万个低频计数器全部填充到 64 字节并不划算。系统设计要问写入频率、线程数、读取频率和空间预算，而不是见到计数器就填充。

一致性为什么以 line 为单位 多核系统必须回答一个问题：某个核心写了一个地址，其他核心什么时候不能继续使用旧值。若一致性按单个字节维护，状态量和协议流量会非常大；按 cache line 维护则把一小段连续字节作为一致性单位。于是一个核心想写某个字节，通常要取得包含它的整条 line 的独占或修改权限。另一个核心哪怕只写同一 line 上另一个变量，也会争夺同一份权限。这就是 false sharing 的硬件根源。

这个机制和语言层面的数据竞争不是一回事。两个线程写同一普通变量且没有同步，可能是 C++ 数据竞争，结果未定义；两个线程写不同变量但落在同一 line，语义可能完全正确，只是硬件来回转移 line 权限而变慢。正确性和扩展性要分开分析。第 11 章讨论原子时会看到，`memory_order_relaxed` 可以放松顺序要求，却不能消除同一 line 的独占权限迁移。


```

1 constexpr std::size_t CACHE_LINE_BYTES = 64;
2
3 struct alignas(CACHE_LINE_BYTES) WorkerPartial {
4     std::uint64_t valid_records = 0;
5     std::uint64_t parse_errors = 0;
6 };
7
8 static_assert(sizeof(WorkerPartial) <= CACHE_LINE_BYTES);

```

这段代码只是示例形态。真实工程中还要防止两个相邻对象被分配器放回同一 line，要确认结构体大小和对齐是否达到预期，还要避免把只读字段和高频写字段混在一起。第 9 章讨论线程亲和性，第 11 章讨论原子和内存序，第 12 章讨论队列 head/tail 布局时，都会回到这条 line 边界。

写分配和一次性输出 写入也会消耗内存带宽。普通写 miss 往往采用 write allocate：先把目标 line 读入缓存，再修改，之后再写回。若程序写的是一次性大块输出，旧内容没有价值，这个读入可能浪费带宽。某些架构提供 non-temporal store，用于减少缓存污染和写分配，但它不是普遍优化；如果输出马上被下一阶段读取，绕过缓存可能更慢。

Compute Systems Engine 的 `ShuffleBlock` 就有这个取舍。若写出后立刻在同一节点 reduce，普通缓存路径可能有用；若写完交给磁盘或网络，缓存污染可能更明显。这里不手写特殊指令，但问题必须说清：写也有地址路径，写入目标是否会再读，决定了缓存策略是否值得讨论。

store buffer 让写入不等于已经全局可见 核心执行 store 时，常常先把写入放进 store buffer，让本线程继续向前执行。这样可以隐藏一部分写延迟，也能把多个写合并到同一 line。可是 store buffer 里的内容什么时候对其他核心可见，和本线程什么时候继续执行，不是同一件事。单线程程序通常看不见这个差异；多线程程序若用普通变量通信，就会踩到可见性边界。锁、原子和内存序的价值，正是把这些硬件缓冲和重排边界变成语言层面的 happens-before。

写路径也会占用 cache line、store buffer、一致性权限和写回带宽。并行程序里一个热计数器可能比一大段只读输入更贵，因为它把许多核心聚集到同一条 line 上反复争夺修改权。线程本地 partial、分区聚合和 lock-free 队列的性能边界，都会回到这条写路径。

3.6 工作集、预取和分块

工作集 (working set) 是某一时间窗口内会被反复访问的数据集合。它可能是一个热字段数组，一个局部哈希表，一个 batch，一个队列，也可能是任务图的 ready 状态。工作集小到能留在 L1/L2，和大到超过 LLC、甚至跨越大量页面，是完全不同的问题。性能曲线上的拐点，往往来自工作集越过某个层级。

工作集为什么不是总数据量 一个作业可能处理 100 GiB 输入，但某一瞬间真正反复使用的状态只有一个 256 KiB 局部哈希表和几个输入缓冲；另一个作业总输入只有 200 MiB，却在每条记录上随机访问一个 50 MiB 字典。前者总数据量巨大，热工作集可能可控；后者总数据量较小，热窗口却超过缓存层级。问“数据集多大”不够，要问“这一段循环的活跃字节有多少，多久会再次访问，访问前是否已经被驱逐”。

工作集还包括写入目标。一个局部 histogram 如果每个 bucket 高频更新，它既占缓存容量，也占写权限；一个输出缓冲如果只写一次不再读，它的缓存价值和局部表不同；一个队列如果生产者和消费者同时读写 head/tail，它的工作集还包括一致性状态。把这些状态混成一个“内存占用”数字，会掩盖真正的等待来源。

层级拐点怎样出现 规模阶梯实验常看到几个台阶：小数组很快，变大后突然慢

一些，再变大又慢一截，最后接近内存带宽或随机访问延迟。这不是算法复杂度突然改变，而是工作集从 L1D 溢出到 L2，从 L2 溢出到 LLC，从 LLC 溢出到内存，或者活跃页超过 TLB 覆盖。每跨一个层级，命中延迟、带宽和可并行 miss 数量都会变化。

拐点比单点耗时更有解释力。若 32 KiB 附近出现变化，可能与 L1D 容量或冲突有关；若几百 KiB 或数 MiB 附近变化，可能与 L2/LLC 或 TLB 有关；若步长变成每页一次后突然退化，TLB 和 page walk 可能参与。具体容量要看平台资料，不能把某台机器的数字写成普遍规律，但“跨层级出现拐点”这个分析方法稳定。

分块的本质 矩阵乘法用 blocking 控制缓存工作集。日志统计也需要同样思想：一次处理一个 batch，让热字段、局部 histogram、错误摘要和输出缓冲保持在可控范围内。Batch 太小，函数调用、调度和 I/O 边界成本占比高；batch 太大，cache、TLB 和恢复范围都变差。一个只看算法循环的最佳 batch size，不一定是系统最佳 batch size。

分块不是为了好看，而是让昂贵搬运服务更多有用工作，让局部状态在被驱逐前完成足够计算。单机 cache block、I/O buffer、网络 batch 和分布式 shuffle block 在不同尺度上解决同一个问题：控制工作集和边界成本。

分块同时控制容量和复用距离 复用距离是同一数据两次使用之间，程序访问了多少其他数据。若复用距离超过某个缓存层级容量，第一次搬入的数据在第二次使用前可能已经被驱逐。分块通过改变循环顺序或阶段边界，把会互相复用的数据放在较短时间窗口内。矩阵乘法里，tile 让一小块 A/B/C 在缓存中反复使用；日志聚合里，batch 让事件 id、局部计数表和错误摘要在可控窗口内完成足够工作。

分块也控制 TLB。一个 batch 若跨越太多页面，即使每页只访问少量字节，也会增加翻译压力；一个局部表若刚好能留在少量页面中，TLB 覆盖就更稳定。于是 batch size 不是越大越好。系统还要考虑恢复边界：batch 太大，失败重试成本高；batch 太小，调度和提交开销高。好的分块大小来自 cache、TLB、调度、I/O 和恢复的共同约束。

硬件预取器能看懂什么 硬件预取器擅长顺序访问和某些固定步长访问。它不理解业务对象，也很难理解复杂指针图。若程序按 `event_ids[i]` 顺序扫描，预取器容易提前请求后续 line；若程序从哈希节点跳到字符串，再跳到下一个节点，下一地址依赖当前 load 结果，预取困难。软件预取只有在地址能提前算出、预取距离合适且不会污染缓存时才可能有效；更可靠的优化通常是先改变布局 and 访问顺序。

预取为什么会失败 预取器需要在“太早”和“太晚”之间命中窗口。太晚，数据还没回来，load 仍然等待；太早，预取来的 line 可能在使用前被驱逐，还会污染缓存；预取错了，会占用带宽和 cache 空间。复杂对象图、数据相关分支、哈希表探测、压缩格式解析和不规则图遍历，都可能让预取器无法稳定预测。软件预取也不是万能，它增加指令、占用前端和寄存器，还要求程序能提前算出未来地址。

更稳的办法通常是改变地址序列。把链表节点放进 arena，至少能减少页分散；把字符串 key 编码成整数，能把随机字符串比较变成紧凑数组访问；把每条记录的虚调用改成 batch 内核，能让一批相似地址连续处理。预取优化若不改变不可预测地址的来源，收益常常很脆弱。

哈希表的内存成本 哈希表是工作集分析的好对象。小 key 集合下，bucket、节点和字符串可能大多留在缓存里；key 基数扩大后，每次更新可能访问随机 bucket，装载因子、探测长度、节点分配和字符串比较都会进入成本。热点 key 又有双重性：读多时提高命中，写多并发时制造共享写热点。

项目报告不能只写“使用 unordered map 统计”。它至少要记录 key 数量、最大 key 占比、前十 key 占比、装载因子、是否发生 rehash、节点分配次数、局部表大小和合并成本。若 key 空间可以编码为小整数，数组桶可能更好；若 key 稀疏且一次性处理，哈希可能更合适；若输入要分布式 shuffle，编码表是否稳定又会变成协议问题。

3.7 TLB：页级覆盖也是性能资源

程序使用虚拟地址，硬件执行时要把虚拟页翻译成物理页。TLB 缓存最近使用的翻译。TLB 命中时，翻译成本很低；TLB miss 时，处理器需要进行 page walk，读取多级页表项。页表项本身也在内存层级中，可能命中缓存，也可能继续等待内存。一个程序即使数据 cache 行为尚可，只要活跃页数太多，地址翻译也可能成为瓶颈。

为什么需要虚拟地址 如果所有程序直接使用物理地址，进程之间很难隔离，装载位置也会互相冲突，操作系统难以按需分配和回收页面。虚拟地址给每个进程一套看起来连续的地址空间，操作系统用页表把虚拟页映射到物理页，并在映射上附带权限。这样，两个进程可以都使用同一个虚拟地址而指向不同物理页；只读共享库可以映射到多个进程；写时复制可以让 fork 后的页面在真正写入时再复制；未使用的虚拟范围可以暂时不占物理内存。

虚拟地址带来的代价是每次内存访问都需要翻译。若每个 load/store 都完整遍历页表，成本会不可接受，于是有了 TLB。TLB 不是普通数据缓存，而是页表翻译缓存：用虚拟页号查到物理页号和权限。它缓存的是“这一页怎样映射”，不是这一页的数据字节。数据字节仍由 L1D/L2/LLC/内存层级负责。

页内偏移为什么不需要翻译 页面大小固定时，虚拟地址可以拆成虚拟页号和页内偏移。页表翻译虚拟页号到物理页号，页内偏移原样保留。若页面大小是 4 KiB，低 12 位就是页内偏移；虚拟页和物理页的这 12 位位置相同。这个事实让硬件可以并行做一些事情，例如用低位选择 cache line 内偏移，或在某些设计中让 TLB 查找和 cache 索引部分重叠。具体实现复杂，但基本拆分很重要：TLB 覆盖的是页，不是单个字节。

活跃页数比元素数更接近问题 假设页面大小是 4 KiB。连续数组顺序扫描一百万个整数，会跨越很多页，但访问顺序稳定，一页内使用大量元素，TLB 和预取器都有机会工作。若一百万个小对象随机散在堆页上，每处理一个对象可能跳到新页，每页只用少量字节。两者元素数相同，活跃页形状不同。

TLB 覆盖范围可以粗略理解为“当前 TLB 能同时记住多少页面的翻译”。当热窗口跨越的页面远超过覆盖范围，page walk 会频繁出现。对象池、arena、冷热拆分、排序索引和分块，都可能减少单位时间内的活跃页数。大页把页粒度从 4 KiB 扩大到 2 MiB 或更大，能显著扩大覆盖范围，但它会影响内存碎片、权限粒度、NUMA 放置和部署配置。大页是取舍，不是魔法开关。

page walk 为什么昂贵 多级页表可以避免为整个巨大虚拟地址空间一次性分配页表。虚拟页号被拆成多段，每段索引一层页表；最后一层页表项给出物理页号和权限。TLB miss 时，硬件 page walker 需要读取这些页表项。页表项本身在内存里，也要经过 cache 层级；若页表项不在 cache 中，page walk 会继续等待内存。一次 TLB miss 可能不只是一次内存访问，而是一串相关的内存访问。

这就是为什么活跃页数过多会让程序变慢，即使数据 cache line 看起来利用率不错。每次访问的数据可能很少，但每个新页都需要翻译；翻译缺失又要读取页表；页表读取还会占用 cache 和内存带宽。大型哈希表、对象图、稀疏矩阵、图算法和随机采样，都容易同时面对数据 miss 和 TLB miss。

Listing 3.12 多级页表查找的抽象伪代码

```
1 PageTableEntry walk_page_tables(VirtualAddress address) {
2   PageTable* table = current_process.root_page_table;
3   for (Level level : page_table_levels) {
4     std::size_t index = address.index_bits(level);
5     PageTableEntry entry = table->entries[index];
6
7     if (!entry.present) {
8       raise_page_fault(address, level);
9     }
10  }
```

```

9     }
10    if (level == Level::Leaf) {
11        return entry;
12    }
13    table = entry.next_level_table();
14 }
15 unreachable();
16 }

```

这段伪代码保留了 page walk 的根本形状：页表是由地址位逐层索引的数据结构；每一层都可能因为页表项不在 cache 中而等待；叶子项还要检查权限。真实处理器会有 page walk cache、并行 walker 和各种优化，但不能把 TLB miss 想成“查一个寄存器”。它是可能走到内存层级深处的一组相关访问。

TLB miss 和 page fault 不是一回事 TLB miss 表示翻译缓存里没有目标页的映射；page walk 如果找到有效页表项，就可以填回 TLB 后继续执行。Page fault 表示页表项不存在、权限不允许，或者需要操作系统介入，例如按需分配、从文件读取、写时复制或报错。前者是硬件可恢复的缓存缺失；后者是异常边界，控制权进入内核，处理完后再回到用户态重试或终止。把二者混在一起，会误判成本和修复方向。

例子很直接。顺序扫描一个已经初始化的大数组，可能出现 TLB miss，但通常不会 page fault；第一次写一个刚 `resize` 或 `mmap` 后尚未触碰的页面，可能出现 minor fault，内核分配并清零物理页；访问被换出或文件尚未读入的页面，可能出现 major fault，需要 I/O；写一个共享的 COW 页面，可能触发复制。它们都和“页”有关，但等待形状完全不同。

随机页 load 的完整事件链 现在把一条随机页访问拆开。假设 `indices[i]` 给出一个随机页面号，循环读取每页中的一个 `std::uint32_t`。汇编层面仍然是一条普通 load；硬件层面，先要读取索引数组，再由索引生成目标虚拟地址，再翻译目标页，再取目标 cache line。若目标页翻译不在 DTLB，page walker 还要读取页表项；若目标数据 line 不在 cache，内存层级还要取数据；若页面第一次被访问，还可能进入内核处理缺页。一个源码 load 因此展开成多条相关等待。

Listing 3.13 随机页 load 的底层事件链

```

source:
    value += base[indices[i] * page_stride]

event chain:
    load indices[i] from a mostly sequential index array
    compute target virtual address
    look up target virtual page in DTLB
    if DTLB miss:
        read page-table entries through page walk
        fill translation or raise page fault
    look up target data cache line
    if data miss:
        request line from lower cache or memory
    extract value and wake dependent add
    retire only after older operations are complete

```

这条链解释了为什么随机页访问常常比“随机 cache line”更糟。随机 cache line 主要攻击数据 cache 和预取器；随机页访问同时攻击 TLB 覆盖、页表项局

部性和数据 cache。若输入规模再超过物理内存或 page cache 状态不稳定，还会把缺页和 I/O 混入。报告必须写清测试期间是否已经触页、是否预热、是否能读取 minor/major fault、是否能读取 dTLB miss。没有这些字段，很难判断慢来自翻译、数据还是 I/O。

证据	支持的解释	不能单独证明的事
每页访问曲线出现拐点	TLB 覆盖或 page walk 可能成为因素	不能证明数据 cache 不重要
minor fault 在测量期增长	首次触页混入热循环	不能说明磁盘 I/O 存在
major fault 增长	冷页或文件页需要 I/O	不能说明算法热路径慢
dTLB miss 高	活跃页翻译压力大	不能说明每条数据 line 都 miss
函数反汇编不变	代码形状相同，等待来源来自地址流或系统状态	不能代替硬件计数器

把首次触页移出热循环 许多错误实验把分配、首次触页和计算混在一起。比如先 `resize` 一个大数组，然后立即在测量区间内并行写入。第一次写每个页面时，内核可能分配物理页、清零、建立页表项、更新 NUMA 放置；这些成本属于初始化和页面承诺，不属于后续稳定计算。若候选版本提前触页而 baseline 没有，候选版本可能只是把成本移到了测量外；若 baseline 在热循环里触页，优化结论也会被污染。

Listing 3.14 把页级生命周期拆成四段

```
allocate:
    reserve virtual address range or container capacity

commit or first touch:
    write one byte per page
    record minor and major faults
    establish intended NUMA placement when relevant

warm:
    run a checksum pass if the experiment is hot-cache or hot-page-cache
    skip this step for cold-start experiments, but mark it explicitly

measure:
    run the target loop
    assert checksum and record page-fault counters again
```

冷启动实验当然可以把缺页和 I/O 计入端到端；关键是命名清楚。热循环实验要尽量排除首次触页；端到端恢复实验要明确包含 page cache、mmap、read、writeback 和 fsync。不同问题使用不同边界，不能拿热循环结论替代冷启动结论，也不能拿冷读总时间证明某个数据布局更差。

页级实验 一个好的 TLB 实验要固定逻辑工作量，同时改变页形状。连续扫描、每 cache line 访问一个元素、每页访问一个元素、随机页访问，四者处理的元素数可以相同，但有效字节比例、TLB 压力和预取能力完全不同。若平台能读取 dTLB miss 事件，就记录；若不能，就用访问规模阶梯、步长曲线和页数估算做替代证据，并明确写出证据边界。

实验还要避免编译器把访问优化掉。每轮访问应当贡献到 checksum 或累加结果，并在报告中保存结果。输入规模要跨过 L1、L2、LLC 和内存，不要只测一个点。性能拐点比单次耗时更能说明层级行为。

四个页级对照 页级实验最少要有四个对照。第一，顺序扫描，每个 cache line 尽量使用连续字节，观察带宽和层级拐点。第二，固定步长扫描，例如每 64 字节、256 字节、4 KiB 取一个元素，观察有效字节比例怎样下降。第三，每页访问一个元素，让 TLB 覆盖成为主要变量。第四，随机页访问，打断预取和页表访问局部性。四者的循环体可以几乎相同，改变的只有索引生成。

访问形状	放大的硬件问题	判读方式
顺序扫描	cache line 利用率、带宽、预取	看规模阶梯和每字节耗时
固定步长	有效字节比例下降、set 冲突风险	改变 stride，观察周期性拐点
每页一点	TLB 覆盖、page walk 成本	固定访问次数，改变活跃页数
随机页	TLB、cache、预取同时恶化	固定页集合，改变随机种子和工作集

关键是固定语义结果。每次访问都把读到的值混入 checksum，避免编译器删除循环；索引数组本身要单独预热，否则测试到的是索引流成本；随机顺序要保存 seed，保证回归可复现。若平台允许读取 `dTLB-load-misses`、`cache-misses` 或等价事件，就记录；若不允许，就用页数、步长、耗时分位数和反汇编作为替代证据。

Listing 3.15 页级实验的伪代码结构

```
prepare data buffer with N pages
prepare index stream according to pattern
touch or do not touch pages according to experiment mode

for index in index_stream:
    checksum = mix(checksum, data[index])

record:
    pattern, page_count, stride, random_seed
    warmed_pages, checksum
    elapsed distribution
    minor_faults, major_faults if available
    dtlb/cache counters if available
```

这段结构能直接迁移到代码实践卷。顺序扫描慢，可能是带宽或编译器没有向量化；每页一点突然变慢，才更像 TLB 覆盖问题；随机页比每页一点更慢，说明预取和 cache 也被破坏。对照判读存在的意义，就是防止把所有慢都归因到同一个名词。

Listing 3.16 AccessTrace 的示例形态

```
1 enum class AccessPattern {
2     Sequential,
3     FixedStride,
4     RandomIndex,
5     PointerChase
6 };
7
8 struct AccessTrace {
9     std::string structure_name;
10    AccessPattern pattern = AccessPattern::Sequential;
11    std::uint64_t element_count = 0;
12    std::uint64_t element_bytes = 0;
```



```

13  std::uint64_t touched_bytes = 0;
14  std::uint64_t estimated_pages = 0;
15  bool writes_data = false;
16  bool warmed_pages = false;
17  std::uint64_t checksum = 0;
18  };

```

这个结构不会预测性能，它只是强迫报告说清地址形状。报告可以按系统层次扩展：线程分析加入 worker id 和 cache line 写共享，NUMA 分析加入节点放置，I/O 分析加入 mmap 和 page cache 状态，分布式分析加入 block identity 和 manifest。

从 AccessTrace 到 MemoryShape AccessTrace 记录动态访问；MemoryShape 记录静态布局。两者必须配套。一个结构体可能静态上有 128 字节，但热循环只读其中 8 字节；一个数组可能静态连续，但访问索引是随机；一个哈希表可能元素总数不大，却跨很多页。MemoryShape 解释“字节在哪里”，AccessTrace 解释“循环怎样走过这些字节”。

Listing 3.17 MemoryShape 的示例形态

```

1  struct MemoryShape {
2      std::string object_name;
3      std::uint64_t object_count = 0;
4      std::uint64_t bytes_per_object = 0;
5      std::uint64_t hot_bytes_per_object = 0;
6      std::uint64_t cold_bytes_per_object = 0;
7      std::uint64_t estimated_cache_lines = 0;
8      std::uint64_t estimated_pages = 0;
9      bool contains_pointers = false;
10     bool shared_between_threads = false;
11 };

```

胖对象版本的 bytes_per_object 可能很高，hot_bytes_per_object 却很低；这说明 cache line 里大量冷字段被带入。链式对象的 contains_pointers 为 true，AccessTrace 又显示 pointer chase；这说明下一地址依赖当前 load。按 worker 分片后的 partial 数组 shared_between_threads 为 true，但若每个 worker 只写自己的 cache-line-aligned 槽位，一致性风险低；若多个 worker 写同一 line，第四章的 false sharing 就会出现。用两个报告结构连接章节，比孤立讲“局部性”更稳。

页表是数据结构，不是抽象开关 页表经常被描述成“虚拟地址映射到物理地址”。这句话正确，但太短。页表本身是内核维护的一组内存中的表；每个进程有自己的根页表；页表项记录物理页号、权限、是否 present、是否 dirty、是否 accessed、是否可执行等状态。TLB miss 时，硬件 page walker 要读取这些页表项；缺页时，内核要修改这些页表项；上下文切换、页面回收、COW、mmap 和 NUMA 放置都会改变页表或它指向的物理页。

因此虚拟内存不是一层透明薄膜。它同时提供隔离、懒分配、文件映射、共享库、写时复制和权限控制，也引入页级状态和页级成本。一个系统若大量创建短生命周期映射、频繁触发 COW、随机访问巨量页或在多核间迁移页面，就会把页表和 TLB 变成热问题。

从页表状态推出缺页分类 访问一个虚拟地址时，硬件先检查当前页是否已经有有效翻译。若 TLB 没有，就 page walk。Page walk 可能找到 present 页表项，说明只是 TLB miss；也可能发现页不存在或权限不允许，触发 page fault。进入

内核后，内核会根据 VMA 和页表状态决定动作：给匿名页分配并清零物理页，给文件页从 page cache 建立映射，从磁盘读入缺失文件页，复制 COW 页，或者发送错误信号。

Listing 3.18 从页表状态到缺页处理的分类

```

TLB miss:
    page table entry is present
    hardware fills TLB and retries the access

minor fault:
    virtual range is valid
    no disk read is needed
    kernel allocates/maps a physical page or resolves COW

major fault:
    virtual range is valid
    file page is not resident
    kernel waits for storage I/O, then maps the page

protection fault:
    access violates permission
    kernel reports error or terminates the process

```

这四类等待形状完全不同。TLB miss 仍主要在硬件和页表读取中恢复；minor fault 进入内核但通常不等磁盘；major fault 进入 I/O；protection fault 是语义错误。把它们都叫“内存慢”，无法指导修复。

虚拟连续不等于物理连续 `std::vector` 给程序一个连续虚拟地址区间，但它背后的物理页可以来自不同物理位置，甚至不同 NUMA 节点。对顺序扫描来说，虚拟连续已经足够让地址递增和预取工作；对 NUMA 和 I/O 来说，物理页归属仍然重要。若主线程先触碰所有页，物理页可能集中在一个节点；若 worker 分别 first-touch 自己的范围，物理页更可能贴近使用者。

这条事实解释了为什么布局分析要分两层。虚拟地址形状决定程序能否形成连续 load/store、预取和 TLB 局部性；物理页放置决定访问距离、内存控制器和 NUMA 带宽。前者主要由数据结构和访问顺序控制，后者还受操作系统策略、线程位置和缺页时机影响。

3.8 完整案例：同一批记录的四条地址路径

很多内存优化讨论会把“数据量很大”当作唯一变量。硬件真正看到的是地址序列：下一次 load 的地址能否提前算出，连续字节是否被充分使用，当前窗口跨越多少 cache line 和页面，是否第一次触碰页面，页表项是否已经在 TLB 或缓存中。下面用同一批逻辑记录构造四条地址路径，把 cache、TLB、page fault 和指针依赖拆开。

设每条记录有一个 `value` 字段。四个版本都把相同数量的 `value` 混入 checksum，逻辑工作量相同；改变的只是访问地址的生成方式。

Listing 3.19 四条地址路径的共同语义

```

input:
    records[0..N)
    each record contributes exactly one value

```

```
result:
    checksum = mix(checksum, value)
    visited_count == N
```

共同语义非常重要。若随机版本少访问一部分记录，或指针版本因为坏链提前结束，耗时就失去可比性。每个循环都要返回 `checksum` 和 `visited_count`，报告还要保存随机 `seed`、页面数量、步长和预热策略。

路径一：顺序扫描 顺序扫描的地址由基址加循环计数器生成。处理器很早就能知道后续地址，硬件预取器也容易跟上。若元素布局紧凑，每条 `cache line` 上有大量有效字节，TLB 也能按递增页号工作。

Listing 3.20 顺序扫描的地址生成

```
for i in 0..N:
    address = base + i * sizeof(Record)
    checksum = mix(checksum, load value at address)
```

这条路径慢时，首先怀疑带宽、向量化、字段间距或循环体额外工作。若 `Record` 很胖，顺序仍然可能慢，因为每次搬入的 `cache line` 中只有很少字节被使用；这时问题不是顺序性，而是有效字节比例低。

路径二：固定步长 固定步长让地址仍可预测，但每条 `cache line` 的有效字节下降。步长等于 64 字节时，可能每条 `line` 只用一个小字段；步长等于 4096 字节时，每次都跨到下一页，TLB 覆盖很快成为变量。固定步长还可能暴露 `set conflict`：若地址低位模式总是落到同一 `cache set`，容量还没用完就发生冲突驱逐。

Listing 3.21 固定步长扫描

```
for k in 0..N:
    offset = (k * stride_bytes) % data_bytes
    checksum = mix(checksum, data[offset])
```

固定步长实验要画步长曲线，而不是只测一个点。64、128、256、512、4096、8192 字节可能对应不同现象：`line` 利用率下降、预取距离变化、页覆盖变化、`set` 冲突变化。曲线的拐点比单个数字更能说明层级边界。

路径三：随机页访问 随机页版本把记录按页面打散，每页只取一个或少量值。这样能把“元素数量”从“活跃页数”中拆出来。若每页只用 8 字节，绝大多数搬入的 `cache line` 和页面翻译都没有被充分利用。

Listing 3.22 随机页访问

```
prepare page_order[0..page_count)
shuffle page_order with fixed seed

for p in page_order:
    offset = p * page_size + value_offset_in_page
    checksum = mix(checksum, data[offset])
```

这条路径慢，不能马上写成“`cache miss`”。它同时破坏预取、`cache` 局部性、TLB 局部性和页表项局部性。要区分原因，需要和固定每页访问、顺序每页访问、预热与未预热运行对照。如果冷运行远慢于热运行，并且 `minor fault` 增多，首次触页参与了成本；如果热运行仍在活跃页数超过某个范围后变慢，TLB 覆盖或 `page walk`

更可疑。

路径四：指针追踪 指针追踪最难隐藏延迟，因为下一地址依赖当前 load 的结果。顺序数组中，地址 `i+1` 在本轮 load 前已经知道；链表中，下一节点地址只有当前节点到达后才知道。乱序窗口可以推进独立工作，却很难跨过这条地址依赖链。

Listing 3.23 指针追踪的地址依赖

```
node = head
while node != null:
    checksum = mix(checksum, node->value)
    node = node->next
```

指针追踪的慢可能来自三层。第一，节点分散导致 cache miss 和 TLB miss；第二，下一地址依赖当前结果，内存级并行度低；第三，节点含有冷字段或虚函数，使热路径继续扩大。把链表节点放入 arena 可以改善页分散，却不能完全消除 next 指针依赖；把链表转换成索引数组或分块邻接表，才可能让地址提前计算。

把四条路径放进同一张审计表 同一批记录的四条路径给出一组判读规则：

现象	更可能的原因	下一步对照
顺序也慢	带宽、字段间距、额外计算或未向量化	缩小字段、看反汇编、比较字节账本
顺序快，固定步长慢	line 利用率下降或 set conflict	扫 stride 曲线，改变对齐和容量
每页访问出现拐点	TLB 覆盖或 page walk 成本	改 <code>page_count</code> ，记录 dTLB 或替代证据
冷运行慢很多	minor fault、major fault 或 page cache 状态	记录 faults，拆分预热和正式测量
指针版最慢	地址依赖链和分散节点	arena、索引化、分块邻接表对照

这张表不会自动给出答案，但能防止把所有慢都归因到 cache。Cache line、TLB、page fault 和指针依赖都发生在“内存访问”附近，修复方向却完全不同：line 利用率靠布局，TLB 覆盖靠页形状和分块，fault 靠测量边界和驻留状态，指针依赖靠表示法改变。

报告字段要覆盖页级状态 一次地址路径实验至少保存下面字段：

Listing 3.24 地址路径审计报告字段

```
access.pattern
record_count
page_count
stride_bytes
random_seed
warmed_pages
first_touch_inside_timing
minor_faults
major_faults
dtlb_misses_available
checksum
visited_count
elapsed_ns_median
```

`first_touch_inside_timing` 用来说明首次触页是否被计入；`minor_faults` 和 `major_faults` 来自系统资源统计或工具输出；`dtlb_misses_available` 表示硬件计

数器是否真的可用，而不是把不可用写成零。`checksum` 和 `visited_count` 是语义护栏，没有它们，任何地址实验都可能被编译器优化或数据缺失污染。

RecordId 不是虚拟地址 布局改造常犯的误差是把指针或虚拟地址当成长期身份。虚拟地址只是当前进程当前映射中的位置；`vector` 重新分配、`mmap` 重新映射、跨进程传输、文件落盘和分布式 `shuffle` 都会让它失去意义。稳定身份应由 `RecordId`、`block id`、`offset` 或 `manifest entry` 表达，再由当前布局把身份映射到热列或冷表位置。

这个区分会影响后续所有章节。并发阶段需要知道哪个 `worker` 拥有哪个 `record` 范围；I/O 阶段需要从文件 `offset` 重建记录；分布式阶段需要跨机器传递逻辑 `block`；故障恢复需要根据 `manifest` 判断哪些输出已经提交。虚拟地址可以服务当前热循环，不能充当系统合同。

3.9 首次触页、NUMA 与测量边界

分配内存不等于物理页面已经就位。许多系统采用按需分配：程序拿到虚拟地址范围后，第一次写某个页面时才真正分配物理页、清零页面并建立页表项。这次 `page fault` 可能是 `minor fault`，不涉及磁盘，但它仍然远比普通 `load/store` 昂贵。若 `benchmark` 把首次触页放进热循环，就会把页面分配成本误写成算法成本。

reserve、commit 和物理页不是同一件事 C++ 容器的 `reserve`、运行库的堆分配、操作系统的虚拟地址范围和物理页面分配，不在同一层。一个 `vector` 预留容量，可能只是让分配器拿到一段虚拟地址；操作系统可能还没有为每个页面分配物理页。第一次写入某页时，CPU 发现页表项尚未指向可写物理页，触发缺页异常；内核检查这段地址是否属于合法 `VMA`，分配或找到物理页，清零或建立文件映射，更新页表，再回到用户态重试那条指令。

这个过程通常比普通 `cache miss` 重得多。普通 `miss` 仍在硬件数据路径里恢复；缺页进入内核，涉及权限检查、页表修改、可能的锁、内存分配和 `TLB shutdown` 等动作。若需要从磁盘读文件页，成本更高。性能实验必须区分“算法处理已驻留数据”和“端到端包含分配与触页”。两者都可以测，但不能混写成同一个结论。

分配、初始化、预热、测量 严谨实验至少区分四段。第一，分配虚拟地址或容器容量。第二，初始化或生成输入，此时可能触碰页面。第三，预热，让 `page fault`、`cache` 状态或 `JIT/动态链接` 等一次性成本离开被测热循环。第四，正式测量核心循环。若研究冷启动或端到端批处理，可以故意包含前几段，但报告必须说明。若研究热循环吞吐，就不能让首次触页悄悄混进来。

日志输入生成尤其容易犯错。一个版本在生成时顺序写满所有页面，另一个版本只 `reserve` 容量后开始计时解析；后者第一次解析就承担 `page fault`。两者总时间不同，不一定是解析算法不同。测量框架会把这些字段制度化；每个内存实验都必须写清触页策略。

预热不是作弊 如果目标是热循环吞吐，预热是为了移除与该问题无关的一次性成本：页面首次分配、动态链接、文件页进入 `page cache`、分支预测器和缓存初始状态等。预热不是把程序变快的作弊，而是把问题边界写清。相反，如果目标是冷启动延迟、批处理端到端时间或失败恢复时间，就必须包含这些成本，并且把它们单独分项。真正的问题不是是否预热，而是报告有没有说明测的是哪一种语义。

缺页计数怎样读 `minor fault` 通常表示不需要磁盘 I/O 的缺页，例如匿名页首次触碰、`COW` 或已经在 `page cache` 中的文件页建立映射；`major fault` 通常涉及阻塞 I/O。这个区分不是绝对性能公式，但能帮助解释冷读和首次触页。一个解析器版本如果 `minor faults` 明显更多，可能是触页策略不同；`major faults` 更多，可能是文件页没有驻留或系统内存压力更大。没有这些计数时，至少要记录输入是否预热、是否重复运行、系统内存压力和文件缓存状态。

NUMA first-touch 在 `NUMA` 机器上，页面常按 `first-touch` 策略放置到首次写入它的线程所在节点。若主线程初始化整个大数组，再把计算分给另一个 `socket` 的 `worker`，`worker` 可能大量访问远端内存。源码里数组连续、分块也正确；硬件

拓扑里却变成远端访问。正确实验通常让每个 worker 初始化自己负责的分块，使物理页放置和计算线程一致。

不是每台实验设备都有 NUMA 拓扑，但这个变量仍然必须进入设计，因为服务端系统、数据库和 HPC 程序经常遇到它。在单 socket 或无法控制亲和性的环境中，报告应写明“未验证 NUMA 放置”，并保留 first-touch 的设计说明。未验证边界写清，比给出虚假的通用结论更专业。

内存压力会改变所有结论 当工作集接近或超过物理内存，系统开始回收 page cache、换出匿名页或触发更多缺页。一个本来受 cache miss 限制的程序，会变成回收和 I/O 问题。Benchmark 应至少记录输入大小、进程峰值内存、可用内存、是否启用 swap、是否有其他大进程干扰。Compute Systems Engine 的 batch size、队列容量、block cache 和 checkpoint 保留数量，都会影响内存压力。内存不是无限背景资源，而是所有章节共享的预算。

3.10 mmap、page cache 和持久化边界

mmap 可以把文件映射到进程地址空间，让程序像访问内存一样读取文件。它简化了某些随机读取和共享映射场景，但不等于文件已经在内存里，也不等于错误处理消失。第一次访问未驻留页面时仍可能 page fault；文件被截断或读取失败时，错误可能出现在后续访问处；映射生命周期必须覆盖所有指向映射区域的视图。

page cache 是内核维护的文件页缓存 文件数据常先进入 page cache。普通 read 会把 page cache 中的字节复制到用户 buffer；mmap 让用户虚拟页直接映射到文件页，访问时通过缺页把对应文件页带入内存。两者都可能受 page cache 影响。第二次读取同一文件快，不一定是解析器优化，而可能是文件页已经驻留；第一次读取慢，也可能是存储设备、readahead、压缩文件系统或内存压力的结果。

page cache 属于系统共享状态，不是某个进程私有缓存。另一个进程读过同一文件，当前实验可能受益；系统内存压力大，page cache 可能被回收；写入文件后，page cache 中的页可能变脏，稍后异步写回。文件 I/O benchmark 若不记录冷/热状态，数字很容易不可复现。

read 和 mmap 是不同状态机 普通 read 把 I/O 边界显式放在系统调用处：读一个 buffer，解析，释放或复用 buffer。它可能多一次拷贝，但错误处理、背压和生命周期更直观。mmap 把读取延迟分散到页面访问处，代码短，但 page fault、readahead、信号错误和映射生命周期都要进入设计。Compute Systems Engine 可以同时保留 read 后端和 mmap 后端，用实验说明二者差异，而不是把 mmap 神化成永远更快。

日志解析若把 raw_offsets 指向 mmap 区域，后续错误报告必须保证映射仍存在。若为了节省内存提前解除映射，冷字段回查会失效。地址路径优化不能破坏诊断语义。

冷读、热读和污染状态 文件实验至少区分三种状态。冷读尽量减少 page cache 命中，用来观察缺页、磁盘和预读。热读重复读取同一文件，用来观察 page cache 命中后的解析和内存路径。污染状态在两次运行之间读取更大的无关文件，用来观察缓存被挤出后的行为。这三种状态都不完美，但能防止把 page cache 命中误写成算法优化。

报告不要只写“mmap 比 read 快”。要写文件大小、访问顺序、是否预热、major/minor faults、解析 checksum、错误处理边界和冷/热差异。若当前环境无法清理系统 page cache，就明确说明，并用热读、污染读和输入变更做近似对照。

写文件不等于提交事实 写入文件成功，数据可能只在 page cache 中变脏，尚未持久化到设备。内核会异步写回；fsync 或 fdatasync 把部分持久化成本拉到当前路径；写临时文件、fsync、rename、再 fsync 目录，才更接近可靠提交协议。文件系统和设备仍有自己的语义边界。可靠写、背压和 manifest 都建立在同一个原则上：写字节、页面变脏、文件名可见、崩溃后可恢复，不是同一件事。

Compute Systems Engine 的 checkpoint 实验要显式比较三种语义：只

write, write 后 fsync, 临时文件 fsync 后 rename 并 fsync 目录。最快路径不一定是正确提交路径。性能和恢复能力是必须声明的交换。

mmap 的生命周期陷阱 如果解析阶段把字段视图、错误上下文或原始行偏移指向 mmap 区域, 后续阶段就依赖映射继续存在。解除映射后, 这些视图不再有效; 文件被截断后, 访问某些页可能触发信号; 映射文件被替换后, 旧映射和新文件名之间的关系也需要说清。普通 read 把数据复制到自有 buffer, 生命周期更显式; mmap 减少拷贝, 却把生命周期和错误边界变得分散。

因此, mmap 不是“更底层所以更快”的同义词。顺序大文件读取中, read 加合理 buffer 可能已经很好; 随机读取或多进程共享同一文件页时, mmap 可能更方便; 错误处理和背压复杂时, 显式 read 状态机更容易控制。选择哪一个, 取决于访问顺序、生命周期、错误语义和系统状态, 而不是 API 名字。

3.11 案例走查：哈希聚合怎样从地址问题变成系统问题

现在把前面的概念连成一条完整路径。任务是统计日志中每个事件名出现次数。朴素版本直接用 `std::unordered_map<std::string, std::uint64_t>` 更新全局表。单线程小数据下, 它足够清楚, 也适合作 reference 的一部分。输入变大、key 变多、线程加入后, 问题开始分层暴露。

第一层：字符串和节点让地址随机 每条记录的事件名可能在原始行中, 也可能复制成字符串对象。计算哈希要读字符串字节; 访问哈希表要读 bucket; 节点式实现还要追节点指针; 发生冲突时继续比较 key。若这些对象分散在堆页上, cache miss 和 TLB miss 会同时上升。优化方向可以是字符串 interning、字典编码、开放寻址、排序后线性合并或按 batch 局部聚合。每个方向改变的是地址路径和语义合同。

这里的关键不是“哈希表慢”, 而是一次更新包含多条地址流。字符串字节是一条流, bucket 数组是一条流, 节点对象是一条流, 计数槽写入又是一条流。若 key 很短且重复度高, 字符串可能很快被编码掉; 若 key 很长且多数只出现一次, 哈希和比较成本会放大; 若节点分配分散, TLB 覆盖也会变差。报告应把一次更新拆成这些地址动作, 而不是只给容器名称。

第二层：共享写让 cache line 迁移 多线程直接更新全局哈希表会引入锁、原子或细粒度分片。即使使用 relaxed 原子计数, 热点 key 的计数槽仍可能在核心间来回迁移。更稳的方向是每个 worker 先写局部 histogram, 再合并。这样热路径从共享写变成本地写, 代价是内存峰值和合并阶段增加。若 key 基数巨大, 局部表也可能超过 cache, 需要继续分区。

局部 histogram 也不是免费。每个 worker 一份表会把内存占用乘以 worker 数; key 空间大时, 局部表可能超过 L2/LLC; 合并阶段可能变成新的随机访问; 如果热点 key 很少, 局部表加合并的成本可能高于少量同步。工程上需要比较至少三种形状: 全局共享表、按 worker 局部表、按 key 分区表。三者分别购买的是实现简单、本地写入、工作集缩小, 代价也不同。

第三层：分区把工作集缩小, 但引入 manifest 按 key hash 把记录分成多个 partition, 可以让每个 partition 的聚合工作集更小, 也为分布式 shuffle 铺路。可是 partition 不只是写几个输出数组。系统要记录每个 bucket 的范围、hash seed、partition 版本、记录数和 checksum。这个记录就是后面反复出现的 manifest (清单): 描述输出对象身份和边界的事实。没有 manifest, 重试、恢复和分布式 reduce 都无法证明自己读的是同一批数据。

分区把一个大随机问题变成多个小随机问题。若每个 partition 的 key 集合能留在 cache 或 TLB 覆盖范围内, 聚合会稳定很多; 若 partition 太多, 调度、文件句柄、buffer 和 manifest 管理成本会反过来上升。分区数不是越多越好。它由 key 基数、倾斜程度、局部表大小、输出 block 大小、I/O 合并成本和恢复粒度共同决定。

第四层：字典编码改变协议 把事件字符串编码为整数能让聚合更快, 但编码表

的生命周期必须清楚。它是每个 batch 独立，还是整个 run 共享？不同 worker 是否使用同一编码？checkpoint 是否保存编码表版本？输出时如何反查字符串？若分布式阶段每个 worker 独立给 login 编成不同 id，后续聚合就会错。一个看似局部的内存优化，最终会变成协议和恢复问题。

第五层：排序把随机写变成线性合并 另一条路线是先收集 event_id，排序后线性计数。排序会增加 $O(n \log n)$ 或 radix pass 成本，却把随机哈希更新变成顺序扫描和相邻相等合并。小数据或高重复 key 上，哈希可能更直接；超大数据、需要外部排序或分布式 shuffle 时，排序路线更容易控制工作集和输出顺序。这里不能只背复杂度：哈希的随机地址和排序的顺序 pass 是完全不同的硬件形状。

方案	地址形状	购买的能力	主要代价
全局字符串哈希	字符串、bucket、节点随机访问	实现直接，reference 清楚	cache/TLB miss、共享写、锁或原子
字典编码 + 数组计数	字符串成本前置，热路径整数索引	热循环紧凑，计数槽连续	编码表生命周期和版本管理
worker 局部表	每个 worker 本地写	减少 line 迁移和锁竞争	内存峰值、合并阶段、key 基数压力
按 key 分区	每个 partition 工作集变小	cache/TLB 更稳定，便于 shuffle	partition 元数据、倾斜、I/O 合并
排序后线性合并	多次顺序 pass	随机写变少，输出有序	排序成本、临时空间、批次边界

心智模型

内存优化不是把容器换成另一个容器，而是改变地址路径、共享形状和状态身份。若优化让结果更难复查、重试或恢复，它只是把成本推迟到了后面的系统边界。

完整地址账本：一次聚合更新到底走了哪些路 把哈希聚合讲透，不能停在“哈希表随机访问”这句话。以 counts[event_id]++ 为目标，输入如果已经完成字典编码，热路径看起来很短：读一个 event_id，计算槽位，读旧计数，写回新计数。若 counts 是线程私有连续数组，这条路径主要消耗顺序读和局部写；若 event_id 仍是字符串，路径会多出字符串内容、长度、哈希、bucket、节点和比较；若 counts 被多个线程共享，写回会变成 cache line 所有权争用。

Listing 3.25 同一个业务更新的三种地址形状

```
1 // string hash path
2 auto& slot = global_table[record.event_name()];
3 ++slot.count;
4
5 // encoded dense counter path
6 std::uint32_t id = event_ids[i];
7 ++local_counts[id];
8
9 // partitioned path
10 std::uint32_t part = route_partition(event_ids[i]);
11 partition_buffers[part].append(record_id);
```

第一条路径的地址图最复杂。record.event_name() 可能指向冷字符串；哈希函数要顺序读字符串字节；哈希表 bucket 数组是一次随机索引；冲突链或开放寻址探测又会访问更多 cache line；比较 key 时可能再次读取字符串；最后更新计数槽。若多个 worker 共享同一个表，计数槽或桶元数据还会在核心间迁移。它容易

写成 reference，却把输入字符串、哈希表结构、分配器、TLB 和一致性全部混进热循环。

第二条路径把字符串成本前移到字典编码阶段。热循环只读连续的 `event_ids`，再写本地 `local_counts`。如果 `event_id` 范围不大，`local_counts` 可以留在 cache 中，TLB 覆盖也稳定。代价是字典表需要版本、checksum 和恢复策略；不同 worker 不能私自给同一个字符串分配不同 id。这里出现一个重要边界：热路径变快，是因为协议承担了身份管理。

第三条路径先把记录分区。它让每个后续聚合阶段面对更小的 key 集合和更局部的输出，却引入 partition buffer、bucket range、block checksum、manifest 和 I/O。它不是单纯的内存优化，而是把内存工作集、任务调度和分布式 shuffle 放到同一条路上。

Listing 3.26 一次更新的地址账本

```
for one input record:
  read event_ids[i]
  pattern: sequential, one narrow column
  expected: L1/L2 friendly, prefetchable

  update local_counts[event_id]
  pattern: indexed write inside worker-private table
  expected: good if key range fits cache/TLB

  if using global hash table:
    read string bytes, bucket array, node, key bytes
    write shared slot or metadata
    expected: random cache/TLB misses and possible line migration

  if using partition buffer:
    append record id to partition output
    update manifest candidate later
```

这份账本能直接指导实验。若 `event_ids` 顺序读很快，但整体仍慢，问题可能在 `local_counts` 工作集过大、key 倾斜、共享写或后续合并。若局部计数版快，分区版慢，可能是 partition buffer 太多、manifest 元数据过重或输出写出受限。若字符串哈希版在小输入上不慢，不说明它适合大输入；小输入可能全部停在 cache 里，尚未暴露 TLB、分配器和 page walk。

把 RFO 和共享写提前写进报告 写一个计数槽之前，核心通常需要拥有包含该槽的 cache line。若该 line 不在本地可写状态，硬件会发起读旧 line 或取得所有权，这类读改写前的流量常被称为 RFO 思路：即使程序只想写几个字节，也可能先搬来整条 line。线程私有计数槽的 RFO 成本很快被局部性摊薄；多个核心写同一条 line 时，所有权会反复迁移，写入延迟和互连流量都会上升。

Listing 3.27 共享计数槽的硬件事实

```
worker 0 writes counts[login]
  obtains line containing counts[login]

worker 1 writes counts[login]
  must obtain the same line
  worker 0's writable copy is invalidated or downgraded

worker 0 writes again
```



```
repeats ownership transfer
```

这就是为什么局部 `partial` 不只是减少锁，而是改变 `cache line` 所有权路径。报告字段应明确写 `shared_write_sites`、`worker_private_bytes`、`merge_bytes`、`hot_key_share` 和 `line_padding_policy`。否则线程扩展性差时，很难判断是锁实现问题、原子指令问题、`false sharing`，还是 `key` 倾斜导致真正同槽竞争。

Listing 3.28 聚合地址报告字段

```
address_shape:
  input_column_bytes
  dictionary_bytes
  local_counter_bytes_per_worker
  estimated_active_pages
  shared_write_sites
  line_padding_policy
  hot_key_share
  partition_count

observed_or_fallback:
  cache_misses_per_record
  dtlb_misses_per_record
  page_faults_during_measurement
  merge_time_ns
  output_manifest_entries
```

字段名本身不是重点，重点是报告必须能回答：热路径每条记录搬了多少有效字节，多少写入是 `worker` 私有，多少状态跨页，多少输出进入 `manifest`。这样第 6 章讨论布局、第 10 章讨论锁、第 11 章讨论原子、第 18 章讨论 `shuffle` 时，所有章节都能回到同一张地址账本。

3.12 MemoryShape 和 AccessTrace

Compute Systems Engine 在这里形成两个报告对象。第一个是 `MemoryShape`，描述数据结构的静态形状：元素大小、元素数量、热字段、冷字段、对齐、页数估计、生命周期和所有权。第二个是 `AccessTrace`，描述某次实验的动态访问：顺序、步长、随机、指针追踪、是否写入、是否预热、`checksum` 和可用指标。它们不是为了搭建复杂框架，而是让后续每次优化都有共同语言。

静态形状回答“字节在哪里” `MemoryShape` 应回答一组静态问题：对象有多大，哪些字段在热路径，哪些字段只用于诊断，元素是否连续，结构是否有指针外跳，分配器是否把对象集中在 `arena`，字段是否按 `cache line` 对齐，生命周期跨越一个 `batch`、一个 `stage`、整个 `run` 还是持久化文件。没有这些字段，性能讨论会退回“这个结构体大概不太好”。

动态访问回答“字节怎样被走过” `AccessTrace` 应回答动态问题：访问是顺序、步长、随机索引还是指针追踪；每次访问读写多少字节；同一 `line` 是否复用；活跃页数大约多少；是否有共享写；是否预热；`checksum` 是否证明访问没有被优化掉。静态形状相同，动态访问可以完全不同：同一个 `std::vector` 顺序扫很好，随机索引可能很差；同一个哈希表小 `key` 集合很好，大 `key` 集合可能变成随机内存测试。

Listing 3.29 MemoryShape 的示例形态

```
1 enum class LifetimeKind {
2   BatchLocal,
```



```

3   StageLocal,
4   RunGlobal,
5   Persistent
6 };
7
8 struct MemoryShape {
9     std::string name;
10    std::uint64_t element_count = 0;
11    std::uint64_t element_bytes = 0;
12    std::uint64_t hot_field_bytes = 0;
13    std::uint64_t cold_field_bytes = 0;
14    std::uint64_t alignment_bytes = 0;
15    std::uint64_t estimated_pages = 0;
16    LifetimeKind lifetime = LifetimeKind::BatchLocal;
17    bool has_shared_writes = false;
18    bool owns_storage = true;
19 };

```

写这类报告时，要避免过度精确的假象。估算页数不等于知道物理页放置；记录 alignment 不等于证明 false sharing 消失；写 `has_shared_writes=false` 不等于没有并发 bug。报告的价值是把问题显式化，让实验和代码审查知道该看哪里。

项目最小交付 Compute Systems Engine 的最小切片包括三组对照。第一，胖对象或对象指针版本对热字段列式版本，比较有效字节和扫描吞吐。第二，紧凑 partial 数组对 cache-line 对齐 partial，比较多线程写入扩展性。第三，read 输入后端对 mmap 输入后端，比较冷读、热读和错误生命周期。三组对照都必须和顺序 reference 输出同样 checksum 和错误摘要。

这些实验的完整 C++ 实现属于代码实践卷。原理卷保留伪代码、报告字段和判读口径。这样分册以后，正文可以把因果链讲透，代码卷可以把输入生成器、测试、benchmark harness 和平台降级路径做完整。

观察入口 理想环境下，可以使用 `perfstat` 观察 cache、dTLB、page-faults 等事件，用 `/proc/self/maps` 和 `/proc/self/smaps` 检查映射，用 `numastat` 或 `numactl` 观察 NUMA 放置，用编译器和反汇编确认循环没有被优化掉。受限环境下，仍可使用规模阶梯、步长曲线、冷/热运行、checksum 和内存峰值作为替代证据。报告必须把 unavailable 写成显式状态，不能把读不到的指标填零。

3.13 源码阅读：从 Linux 页表和页缓存看内存成本

源码阅读不要求把 Linux 内存子系统读成百科，重点是把用户态实验现象和内核维护的状态连接起来。可以固定一个内核版本，选择三条窄路径：缺页怎样建立映射，文件页怎样进入 page cache，脏页怎样写回。报告中写概念、状态和实验对应关系，不要罗列无关函数。

缺页路径 用户程序访问尚未映射的虚拟页时，会触发 page fault。内核检查 VMA，判断权限，找到或分配物理页，建立页表项，必要时清零页面，然后返回用户态重试指令。VMA 描述虚拟地址区间的来源和权限；页表项描述虚拟页到物理页的映射；匿名页、文件页和写时复制页的处理不同。把这个路径读清后，首次触页为什么贵就不再神秘。

页缓存路径 文件读取常通过 page cache。第二次读取同一文件可能很快，因为页面已经在内存；第一次读取可能受磁盘、readahead 和缺页影响；内存压力下，page cache 会被回收。阅读 `mm/filemap.c` 或对应版本的文件映射路径时，要关注 page cache 是以页为单位管理共享状态，不是某个进程私有的普通数组。

脏页和回写 写文件后，页面可能处于 dirty 状态，稍后由内核回写。脏页太多

会触发回写压力，普通写路径可能突然变慢。`fsync` 把提交语义和等待成本显式拉到调用处。源码阅读报告要把 `dirty page`、`writeback`、`fsync` 和项目 `checkpoint` 对应起来：什么时候只是写入内存，什么时候才可以宣布提交。

大页扩大 TLB 覆盖，但不会消灭地址问题 4 KiB 页下，512 MiB 连续虚拟空间包含 131072 个页。即使数据在物理内存中，DTLB 也不可能同时覆盖这么多页；顺序扫描时硬件页表遍历和页表缓存可以缓解压力，随机页访问则会反复暴露覆盖不足。2 MiB 大页把同样 512 MiB 空间压成 256 个页，TLB 覆盖立刻改变。这就是大页对数据库 `buffer pool`、向量化执行和大数组扫描有吸引力的底层原因：它减少的不是数据 `cache line` 数量，而是地址翻译对象数量。

大页不是免费开关。首先，系统要找到或整理足够连续的物理页；内存碎片、透明大页折叠和拆分都会引入不可预测延迟。其次，大页会改变 COW 和缺页粒度：原本只写 4 KiB 的复制，可能让 2 MiB 映射拆分或复制更多元数据。再次，大页会让小对象混在一个更大的翻译单位里，若生命周期和访问模式不一致，回收、NUMA 放置和 `page cache` 行为会更难解释。工程上使用大页时，要说明对象大小、访问模式、生命周期和是否允许启动阶段预分配；不能只写“开启 `huge page` 后更快”。

页大小	购买的能力	付出的成本
4 KiB	粒度小，分配灵活，COW 和回收细	TLB 覆盖有限，随机页访问容易触发翻译压力
2 MiB	TLB 覆盖大，适合大连续工作集	预留和碎片更敏感，拆分和 NUMA 放置更重
1 GiB	极大覆盖，适合少数固定大区间	配置成本高，灵活性低，错误放置代价很大

TLB shutdown 把页表修改变成跨核心协议 页表不是只有当前核心看见。一个进程的多个线程可能在不同核心上运行，它们的 TLB 都可能缓存了同一虚拟页的翻译结果。当内核修改某个已经被使用的页表项，例如取消映射、改变权限、处理 COW 或迁移页面时，旧翻译不能继续留在其他核心的 TLB 里。内核需要让相关核心失效对应 TLB 项，这个过程常被称为 `TLB shutdown`。

`Shutdown` 的成本来自跨核心协调。发起核心更新页表后，要向可能持有旧翻译的核心发送中断或等价通知；远端核心在安全点执行失效操作；之后发起方才能确信旧翻译不会继续被使用。频繁 `mmap/munmap` 小区间、频繁改变页权限、过度依赖 COW 写放大、在热路径中反复创建和销毁大映射，都可能把本来局部的内存管理变成全机同步成本。第 4 章讲 NUMA 和互连时，会看到这种成本不仅是内核函数慢，还会扰动远端核心的执行。

COW 让一次普通写进入缺页和复制路径 写时复制 COW 常出现在 `fork`、文件私有映射和某些快照机制里。两个地址空间最初共享同一物理页，页表项标为只读；当其中一个写入时，CPU 触发权限 `fault`，内核分配新页，复制旧内容，更新写入方页表，再返回用户态重试写指令。用户态看到的只是一次普通 `store`，底层却经历了异常、内核分配、内存复制、页表更新和可能的 TLB 失效。

这条路径解释了两个现象。第一，`fork` 后立刻大量写父进程已有内存，会把 COW 优势变成复制成本；第二，用私有 `mmap` 修改大文件时，写入并不直接改变文件页，而是生成进程私有匿名页。理解 COW 后，`mmap` 就不再是“文件变成数组”的简单比喻，而是一组映射、权限、缺页和可见性规则。

源码阅读任务

源码阅读交付一张表即可。行包括 `page fault`、`TLB miss`、`page cache hit`、`major fault`、`dirty page`、`writeback`、`reclaim`、COW。列包括用户态现象、内核维护的状态、项目中可能出现的位置、观测方法和未验证边界。重点是把内存成本变成可追踪事实，而不是把 Linux 源码读成函数名清单。

3.14 实验矩阵：把 cache、TLB 和 page fault 分开

只讲 cache line、TLB 和 page fault 还不够。工程报告必须能把它们从同一个慢现象里拆开。一个随机访问循环可能同时有 cache miss、DTLB miss、page walk、分支开销和预取失败；一次首次扫描大数组可能既包含缺页清零，也包含真实内存带宽。若实验矩阵没有设计好，所有解释都会混在一起。

本节给出一个标准矩阵。它不是为了追求每台机器输出相同数字，而是训练读者用单变量对照把候选瓶颈逐步排除。

四个访问模式

最小 benchmark 只需要一个数组和四种访问模式：

Listing 3.30 四种地址模式的伪代码

```
1 // 1. contiguous: cache line 利用率最高，预取器友好
2 for (std::size_t i = 0; i < n; ++i)
3     sum += a[i];
4
5 // 2. stride: 控制每次跨过多少元素
6 for (std::size_t i = 0; i < n; i += stride)
7     sum += a[i];
8
9 // 3. page_stride: 每页触碰一个元素，放大 TLB 压力
10 for (std::size_t p = 0; p < page_count; ++p)
11     sum += a[p * elements_per_page];
12
13 // 4. pointer_chase: 下一个地址依赖当前 load
14 for (std::uint32_t x = seed; steps--;)
15     x = next[x];
```

这四种模式回答不同问题。连续扫描用于估算可持续带宽；stride 访问用于观察 cache line 内有效字节比例；每页一个元素用于放大翻译覆盖问题；指针追踪用于限制内存级并行度，因为下一次 load 的地址要等本次 load 返回。

必须分开的阶段

实验代码要把分配、初始化、预热和测量分成四个阶段。否则首次触页和热循环会混在一起。

Listing 3.31 内存实验阶段

```
allocate:
    reserve virtual address space or allocate vector

initialize:
    touch pages and fill data
    record which thread performs first touch

warmup:
    run untimed pass if measuring hot-cache/hot-TLB behavior

measure:
    run timed kernel
    consume checksum
```

如果目标是测首次触页，测量范围应故意包含 initialize 的第一次写；如果目标是测 hot loop，initialize 和 warmup 必须在计时外完成。报告不写计时边界，结论没有意义。

输入规模阶梯

规模阶梯要覆盖 L1、L2、LLC、DRAM 和大量页。具体容量随机器不同，报告至少要写实验环境中的 cache 信息来源。一个通用阶梯可以先这样设计：

规模	主要观察	典型误读
32 KiB	接近 L1 数据 cache	过小，循环开销可能占比高
256 KiB–1 MiB	L2 或私有 cache 区域	不同 CPU 层级差异很大
8–64 MiB	LLC 压力	多核共享和系统干扰会参与
512 MiB–2 GiB	DRAM 和 TLB 压力	首次触页会污染结果
每页一个元素	DTLB 覆盖与 page walk	不能直接代表真实应用

规模阶梯的意义是寻找拐点。若 stride=1 在 32 KiB 到 64 MiB 都平稳，512 MiB 后下降，可能是 DRAM 或 TLB 参与；若每页一个元素在小总字节下已经慢，翻译路径更可疑；若指针追踪比随机索引更慢，可能是内存级并行度被依赖链压住。

PMU 事件和降级路径

理想情况下，报告保存下面的命令输出：

Listing 3.32 cache/TLB 实验的 perf 命令形态

```

1 taskset -c 2 ./mem_shape_bench \
2   --mode stride --bytes 1073741824 --stride 64 \
3   --warmup 1 --repeat 10 --csv mem.csv
4
5 perf stat -r 5 \
6   -e cycles,instructions,cache-references,cache-misses,page-faults \
7   -- taskset -c 2 ./mem_shape_bench --mode stride \
8     --bytes 1073741824 --stride 64 --repeat 1
9
10 perf stat -r 5 \
11   -e dTLB-loads,dTLB-load-misses,LLC-loads,LLC-load-misses \
12   -- taskset -c 2 ./mem_shape_bench --mode page_stride \
13     --bytes 1073741824 --repeat 1

```

事件名会随平台变化。Intel、AMD、ARM、虚拟机、容器和受限环境都可能不支持这些名字。报告的第一行应该保存 `perflist` 或至少保存不可用错误。若 DTLB 事件不可用，可以用 page size、访问页数、stride 曲线、huge page 对照和 wall time 作为替代证据；但不能写“DTLB miss 下降了”。

想证明的事	首选证据	降级证据
cache line 利用率	cache miss、带宽、stride 曲线	有效字节/line 计算、规模阶梯
DTLB 覆盖不足	DTLB miss、page walk 事件	每页访问、huge page 对照、页数账本
首次触页进入计时	page-faults、minor/major fault	initialize/measure 分阶段时间
DRAM 带宽限制	带宽、uncore、LLC miss	bytes/time 与机器带宽数量级
依赖链限制 MLP	pointer chase 慢于随机批量	多路 chase 对照

page fault 实验：第一次写和第二次写不是一回事

首次触页实验要明确测的是虚拟内存状态机，而不是普通 store 吞吐。最小对照如下：

Listing 3.33 首次触页对照

```
case A:
    allocate 1 GiB
    timed memset/write once

case B:
    allocate 1 GiB
    untimed memset/write once
    timed memset/write second time

case C:
    mmap file
    timed sequential read first pass
    timed sequential read second pass
```

若 A 明显慢于 B，差异可能来自缺页、清零、页表建立和 THP 行为。若 C 的第二次读快很多，可能是 page cache，而不是解析器突然变快。报告中要保存 `/usr/bin/time-v` 的 minor/major faults，并说明是否能控制 page cache。普通用户通常不能安全清空全局 page cache；这不是问题，问题是没有说明。

huge page 对照：减少页数，不减少数据字节

大页实验要写清目标。启用 huge page 后，同样 512 MiB 数据仍然要读取 512 MiB cache line；改变的是地址翻译对象数量和某些 page walk 成本。若连续流式扫描本来已经接近带宽上限，大页收益可能很小；若随机每页访问很多，大页可能显著改变 TLB 行为。

Listing 3.34 huge page 实验报告字段

```
hugepage_experiment:
    allocation_method: transparent_hugepage | mmap_hugetlb | madvise
    page_size_observed
    bytes
    active_pages_4k_equivalent
    active_pages_actual
    access_mode
    fault_count
    timing
    unavailable_boundaries
```

不要只写“开启 THP 后快了”。要说明实际是否使用了大页。Linux 上可以通过 `/proc/<pid>/smaps` 中的 huge page 相关字段、`madvise` 返回值和系统配置辅助判断。若无法证明实际分配到大页，就只能写“请求大页策略后观察到某变化”，不能写“大页导致变化”。

NUMA first-touch 和 cache/TLB 的交叉

cache/TLB 报告也需要给出 NUMA 边界：页面放置会改变同一个地址模式的物理距离。first-touch 实验至少有两个版本：

Listing 3.35 first-touch 最小对照

```

main_touch:
    main thread allocates and initializes all pages
    workers pinned to cores process their ranges

worker_touch:
    each worker initializes its own range after pinning
    workers process the same ranges

```

若多 socket 机器上 `worker_touch` 明显更快，说明页面位置可能参与；若单 socket 机器上无差异，不能推出 NUMA 不重要，只能说明当前设备无法验证远端路径。报告要记录 CPU 拓扑、绑定策略、初始化线程、计算线程和是否开启自动 NUMA balancing。

一份合格的内存层级报告

内存报告至少包含这些字段：

Listing 3.36 cache/TLB/page fault 报告 schema

```

environment:
    cpu_model
    cache_summary
    page_size
    numa_nodes
    kernel
    perf_availability

input:
    bytes
    element_size
    access_mode
    stride
    active_pages
    checksum

phases:
    allocate_ms
    initialize_ms
    warmup_ms
    measure_ms

counters_or_fallback:
    cycles
    instructions
    cache_misses
    dtlb_misses
    minor_faults
    major_faults
    unavailable

conclusion:
    supported_explanation

```

```
rejected_explanations
unverified_boundaries
```

这个 schema 的价值是让“慢”变成可审计事实。若 cache miss 高但 active pages 很少，就优先查工作集和 stride；若 page faults 只在 initialize 出现，hot loop 结论不能继续拿 fault 解释；若 huge page 改善只出现在 page-stride 而不出现在 contiguous，说明改善更像翻译覆盖而不是数据带宽。

反例：一个看起来像 cache miss 的 page fault

坏报告常写：“第一次运行慢，第二次运行快，说明 cache 命中更好。”这句话不够。第一次运行慢可能是 page cache 冷、匿名页首次触发 minor fault、动态链接 lazy binding、THP 整理、CPU 频率爬升或输入生成器初始化。合格报告要用分阶段和反证输入拆开：

Listing 3.37 拆开首次慢的反证路径

```
if first pass slow:
    compare initialized vs uninitialized allocation
    compare first file read vs second file read
    record minor/major faults
    run same hot loop after explicit warmup
    record dynamic linking and timer boundary if relevant
```

只有当 page fault、初始化、I/O 和动态链接都排除后，才能把差异主要归因到 cache 热度。工程上，少说一句“显然是 cache”，多保存一张阶段表，能避免很多错误优化。

3.15 内存实验门禁和习题

复查必须覆盖六项事实：所有布局版本是否和 reference 的 checksum、错误摘要和输出规模一致；对象图是否已经翻译成地址图，热字段、冷字段、访问顺序、页数估计和共享写是否写清；实验是否覆盖连续、步长、随机、指针追踪、冷读和热读，并区分初始化、预热和测量；cache/TLB/page-fault 指标或替代证据是否保存并标记不可用边界；padding、列式化、字典编码、mmap、大页和分区各自的成本是否说明；MemoryShape 和 AccessTrace 是否能被后续线程、I/O 和分布式实验复用。

习题与作业

习题一：为 Compute Systems Engine 的 partial 数组设计两个布局：紧凑布局和按 cache line 隔离布局。写出每个字段大小、结构体大小、对齐、每条 cache line 可能容纳几个 partial，以及为什么某些场景不应该使用填充。

习题二：实现或设计三种遍历对照：连续数组、随机索引数组、链表或索引模拟链表。三者处理相同数量的逻辑元素，输出 checksum。报告必须分别分析 cache line 利用率、TLB 覆盖、预取器和内存级并行度，不允许只写“随机慢”。

习题三：设计首次触页实验。说明如何分离分配、初始化、预热和计算；如何记录 page-faults；若没有 NUMA 机器，哪些结论不能验证，哪些仍可通过 page fault 和规模阶梯观察。

习题四：比较 read 后端和 mmap 后端读取同一份日志输入。报告至少包含冷读、热读、污染状态、错误处理边界、映射生命周期、checksum 和不可观测指标。若当前环境不能清理 page cache，写出替代对照。

习题五：阅读一个 Linux 内存路径入口，例如缺页、mmap 或 page cache。写一份不超过两页的报告，把一个用户态实验现象映射到内核维护的状态，不要罗列无关函数。

下一步进入 NUMA、互连和缓存一致性。前面的地址路径已经看到 cache line 是搬运和争用单位，也看到页面放置会影响内存距离。单核地址路径一旦加入多个核心、多个 socket 和一致性协议，问题会从“当前核心怎样访问数据”升级为“哪些核心以什么权利访问同一份数据”。这正是 NUMA 和一致性要解决的边界。

工程案例

知识点小案例：同样读一百万个元素，地址顺序决定瓶颈。特例是三段循环：连续读 `a[i]`，按固定步长读 `a[i*64]`，按随机索引读 `a[index[i]]`。三者逻辑上都是累加一百万个值，但机器看到的 cache line 利用率、TLB 覆盖和预取可能完全不同。连续读能把一条 cache line 的多个元素都用掉；大步长可能每次只用一个元素；随机索引还会让预取器失效。由此推出规律：复杂度相同不等于地址流相同。工程上分析内存问题时，先写出地址公式、工作集大小、页数和访问顺序，再看 cache/TLB 指标；不能只用 $O(n)$ 解释内存性能。

第 4 章

NUMA、互连与缓存一致性

先看一个很小的并程序：输入是一批日志记录，每个 worker 解析自己负责的片段，然后把错误行数量加到一个全局计数器里。单线程时吞吐稳定；两线程时略有提升；四线程以后提升变小；八线程甚至变慢。代码没有复杂锁，只是每条记录执行一次 `std::atomic<std::uint64_t>::fetch_add`。从 C++ 语义看，结果正确，没有数据竞争；从硬件看，所有核心在反复争夺同一条 cache line 的修改权。

把程序改成每个 worker 写自己的局部计数，阶段结束后再合并，吞吐突然恢复。这个现象比任何协议名都重要：共享内存并不等于共享写没有成本。多个核心读同一份数据时，硬件可以复制 cache line；多个核心写同一条 line 时，硬件必须决定当前谁拥有写权限，并让其他副本失效或更新。写权限在核心之间来回移动，就会占用互连、缓存层级和等待时间。

本章从这个共享写热点推导缓存一致性、片上互连和 NUMA。多核心处理器不是把许多核心简单摆在一起；每个核心有私有缓存，多个核心共享更大的缓存层级，通过环、mesh、交叉开关或 socket 间链路通信，连接一个或多个内存控制器，并用一致性协议维护共享内存语义。多 socket 或多 chiplet 机器还会出现 NUMA：不同核心访问不同内存节点的延迟和带宽不同。只有把“哪条 cache line 被谁写、哪个页面属于哪个节点、合并阶段穿过哪条互连”说清楚，多线程程序的扩展曲线才有解释。

前两章已经分别放大了同一段热块的核心内部形状和地址路径。本章继续沿同一个更新块推进，只新增一个条件：写者不再只有一个核心。

Listing 4.1 同一更新块进入多核心后的新问题

```
single worker:
    load event_type[i]
    update local_counts[event_type]
    most hot stores stay in this worker's cache lines

many workers with global_counts:
    all workers update global_counts[event_type]
    hot keys make many cores request the same counter line

many workers with partials:
    each worker updates worker_counts[worker_id][event_type]
    merge phase later reads partials and produces one result
```

这三种写法的语义都可以得到同一张 counts 表，但硬件路径完全不同。单 worker 主要是第 3 章的地址和 cache/TLB 问题；全局表把每次更新变成 cache line 所有权问题；worker partial 把高频写留在本地，把跨核心通信推迟到合并阶段。NUMA 也是同一条线的延伸：partial 的页面若在远端节点，或者最终合并跨 socket 读取大量 partial，等待会从单个核心内部移动到互连和内存控制器。

4.1 从一个 atomic 事故推导共享写成本

单线程计数器的成本很容易理解：寄存器里保存当前值，循环里递增，最后写回内存。多线程全局计数器看起来只是把普通加法换成原子加法，但硬件工作已经变了。原子读改写写必须在线性化点上表现得像不可分割操作；处理器不能允许两个

核心同时拿着旧值各自加一再覆盖。于是原子变量所在 cache line 需要被某个核心独占修改。下一个核心要修改时，又要拿走这条 line 的修改权。

注意这里的单位是 cache line，不是变量。一个 8 字节计数器通常落在一条几十字节量级的 line 中；一致性协议管理的是整条 line 的副本状态。若全局计数器非常热，处理器做的算术很少，等待所有权迁移很多。线程数增加后，程序越来越不像并行计算，越来越像许多核心排队修改同一小块内存。

原子读改写的硬件路径 原子自增可以拆成三件事：读旧值、计算新值、把新值写回同一地址。普通单线程加法可以让这些动作在核心内部快速完成；多核心共享时，处理器还必须保证没有另一个核心同时改写同一位置。于是原子操作在执行前要让目标 line 进入可修改状态。若这条 line 当前在别的核心处于 Modified，当前核心要等待对方交出或写回最新数据；若多个核心都有 Shared 副本，当前核心要让这些副本失效。只有这些协议动作完成后，核心才能在本地产执行加法并写回 cache line。

接上第一册的汇编视角，`std::atomic<std::uint64_t>::fetch_add(1)` 在 x86-64 上常见的机器形状可能是带 `lock` 前缀的读改写写，例如：

Listing 4.2 原子加法可能出现的汇编形状

```
lock add qword ptr [rdi], 1

; 或者需要旧值时：
mov  eax, 1
lock xadd qword ptr [rdi], rax
```

具体编译器、目标 ISA 和返回值需求会影响生成的指令。这里要抓住的是形状，而不是背某个固定助记符。没有 `lock` 的 `add qword ptr [rdi], 1` 已经是读旧值、加一、写回；加上原子语义后，硬件还必须保证其他核心不能同时对同一位置完成另一个不可分割修改。在现代缓存一致性机器上，这通常意味着目标 cache line 要进入当前核心的可写独占状态。若多个核心反复执行同一条 locked RMW，真正忙的是 line 所有权迁移和失效确认，不是整数加法器。

Listing 4.3 从 locked RMW 读出 cache line 账本

```
instruction:
    lock add qword ptr [counter_address], 1

line-level questions:
    which cache line contains counter_address?
    which core currently owns the line in Modified/Exclusive state?
    which other cores hold Shared copies that must be invalidated?
    how often does ownership move per input record?
```

这就是为什么反汇编报告不能只写“用了 `atomic`”。它要写原子对象所在字段、cache line 邻居、更新频率、写者数量和是否有局部化替代。若 `lock add` 每条记录出现一次，线程数增加后大概率会看到扩展性问题；若它只在阶段结束时出现一次，成本通常可以接受。

Listing 4.4 原子读改写的抽象硬件流程

```
1 void atomic_fetch_add_like(Address addr, Value delta) {
2     CacheLine line = request_line_with_write_permission(addr);
3     wait_until_other_writers_are_excluded(line);
4 }
```



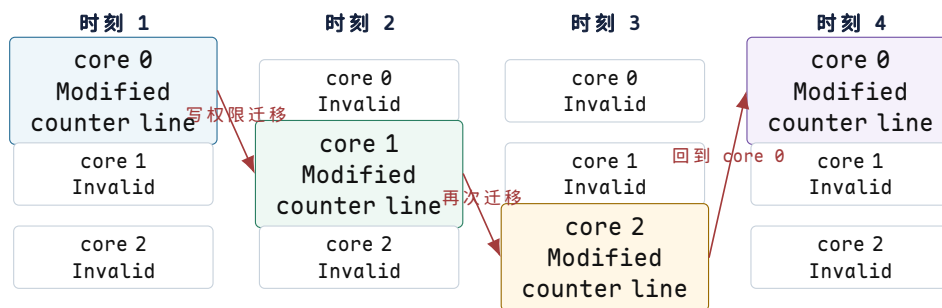
```

5 Value old_value = line.load(addr.offset());
6 Value new_value = old_value + delta;
7 line.store(addr.offset(), new_value);
8
9 keep_line_in_modified_state_until_evicted_or_requested();
10 }

```

真实硬件会把许多步骤并行化，也会用 store buffer、锁前缀、LL/SC、缓存锁定或内部微操作处理不同 ISA 的细节；但抽象流程保留了关键成本：算术发生在核心里，写权限来自一致性协议。若每条记录都执行一次原子加法，热路径就把协议等待放进了每次循环。把更新改成局部累加，再在阶段边界合并，是把协议等待从高频路径移到低频路径。

共享写热点的所有权迁移



每次 `fetch_add` 的算术都很小；真正反复发生的是同一条 cache line 的修改权从一个核心转移到另一个核心。

图示：本图要证明的命题是：热点 atomic 的瓶颈不是加法本身，而是同一条 cache line 的写权限在核心之间迁移。

从全局写到局部 partial 线程本地 partial 的本质是改变写路径。热循环里，每个 worker 只写自己的计数槽位；全局结果只在阶段边界合并。这样高频写从“所有线程争一条 line”变成“每个线程写自己的 line”。合并仍然有成本，但成本被移到低频阶段，并且可以按 worker、NUMA 节点或树形结构进一步组织。

Listing 4.5 从共享写热点到局部 partial 的伪代码

```

1 // baseline: 每条记录都写同一个一致性热点
2 for (Record r : worker_input) {
3     if (is_error(r)) {
4         global_error_count.fetch_add(1, std::memory_order_relaxed);
5     }
6 }
7
8 // 改造后：热路径只写本 worker 的局部状态
9 std::uint64_t local_error_count = 0;
10 for (Record r : worker_input) {
11     if (is_error(r)) {
12         ++local_error_count;
13     }
14 }
15 worker_partials[worker_id] = local_error_count;

```

`memory_order_relaxed` 只能放松原子操作和其他内存操作之间的排序约束，不能让同一条 `cache line` 同时被多个核心修改。它可能减少屏障类成本，却不会消除一致性所有权迁移。判断一个 `atomic` 是否合适，不能只看内存序；要问它是否位于热路径、更新频率多高、写者分散到多少核心、是否可以批量或分层合并。

为什么共享读容易，共享写困难 共享读没有破坏副本一致性。`core 0`、`core 1` 和 `core 2` 都读取同一张只读表时，每个核心缓存一份相同 `line`，后续 `load` 可以在本地命中。只要没有核心写入，这些副本可以并存。共享写会破坏这个条件：一旦 `core 0` 写了 `line`，`core 1` 的旧副本就不能再被当作最新值。硬件必须在写发生前或写传播时处理其他副本，否则同一地址会在不同核心看到不同新旧值。

这个推导能解释很多工程判断。只读配置快照适合被多个线程共享；偶尔替换配置时，可以用版本指针或阶段切换降低写频率；每个请求都写全局指标，则会把所有核心拉到同一条 `line` 上。不要把“共享”当作一个词，要分成共享读、低频共享写和低频共享写。

正确性同步和扩展性结构不是一回事 `Atomic`、`mutex` 和条件变量首先解决的是程序语义：是否有数据竞争，某个不变量是否被保护，某次发布是否被另一个线程可靠观察。扩展性结构解决的是另一个问题：同一条 `cache line` 是否被高频写，远端节点是否反复访问同一批页面，合并阶段是否串行化。一个程序可以同步正确但扩展性很差，也可以局部跑得很快但同步语义错误。分析时要把这两层分开。

同步原语不是魔法。它们必须落到 `load`、`store`、原子读改写写、缓存行所有权、`store buffer`、内存序和等待队列上。低竞争时这些路径很短；高竞争时，锁变量、队列 `head/tail`、全局状态位和统计计数器都会变成共享写热点。

共享写的三个等级 共享状态可以按写入频率和共享范围分级。第一类是只读共享，例如只读配置表、编码字典、不可变输入块和常量权重。多个核心可以复制这些 `line`，通常扩展性很好。第二类是低频写共享，例如阶段结束时写一次完成标志、每秒刷新一次指标、偶尔替换配置快照。它需要同步，但不一定主导性能。第三类是高频写共享，例如每条记录更新全局计数器、所有生产者争一个队列 `tail`、多个 `worker` 写同一条 `line` 上的不同字段。这个等级最危险，常见表现是线程数越多越慢。

共享等级	常见形态	首要问题
只读共享	配置、字典、只读输入块	代码和数据是否能稳定留在缓存中
低频写共享	完成标志、阶段状态、低频指标	同步语义是否清楚，发布和观察是否可靠
高频写共享	热 <code>atomic</code> 、全局队列、共享统计数组	<code>cache line</code> 所有权迁移和互连流量是否支配热路径

优化时要先判断共享等级，再选择 `mutex`、`atomic`、分片、线程本地、批量提交或分层合并。`mutex` 保护低频复杂状态可能完全合理；`atomic` 保护高频热点计数器仍然可能很慢；无锁队列如果所有生产者争同一个 `tail`，也逃不开一致性流量。

4.2 缓存一致性从哪里来

第 3 章已经说明 `cache line` 是数据搬运单位。多核心一加入，就多出一个必须回答的问题：若 `core 0` 和 `core 1` 都缓存了同一个地址，`core 0` 写入以后，`core 1` 还能不能继续读自己的旧副本？如果允许旧副本无限期存在，共享内存程序就会得到混乱结果；如果每次读写都绕过缓存去内存，性能又会崩掉。缓存一致性协议就是在这两个极端之间建立规则：哪些副本有效，谁有权写，写入后其他副本怎样失效或更新。

最重要的推导是：一致性不能按 C++ 对象维护，也不能按变量名维护。硬件只看到物理地址和 `cache line`。若两个变量落在同一条 `line`，即使它们属于不同对象、

由不同线程独占使用，硬件仍把它们当作同一个一致性单位。这就是 false sharing 的根。

一致性要解决的最小反例 先不用任何协议名，只看一个两核心反例。内存中 $x=0$ 。core 0 读 x ，把包含 x 的 line 放入自己的 cache；core 1 也读 x ，也缓存一份。现在 core 0 写 $x=1$ 。如果 core 1 继续读自己的旧副本，就会得到 0；如果 core 0 的写必须立刻穿透到内存并清理所有缓存，单核写性能会被远端内存拖垮。一致性协议必须同时满足两件事：读多时允许副本存在，写时又能让旧副本失效或得到更新。

Listing 4.6 没有一致性规则时的反例

```
initial memory: x = 0

core 0 loads x -> caches line X, sees 0
core 1 loads x -> caches line X, sees 0
core 0 stores x = 1 in its cached copy
core 1 loads x from its old cached copy

without coherence:
    core 1 may keep seeing 0 forever
```

真实程序当然还要受语言内存模型约束。这里的反例只解释硬件为什么要维护 line 状态。没有一致性，普通共享内存就无法提供“同一地址的写最终能被其他核心观察到”的基础；有了一致性，普通无同步读写仍可能在 C++ 层面成为数据竞争。两个层次不能混。

为什么粒度只能粗到 cache line 硬件理论上可以按字节维护一致性，但成本会过高。每个 cache line 已经是缓存标签、有效位、脏位、替换和互连传输的单位；若按每个变量维护状态，硬件就需要理解语言对象边界、结构体字段、分配器布局 and 类型别名，这不现实。按 line 维护一致性，是硬件复杂度和空间局部性之间的取舍。代价就是 false sharing：软件语义上不共享，硬件粒度上仍共享。

这条推导直接影响结构体布局。把多个 worker 的高频计数放在相邻数组元素里，源码上看每个 worker 独占自己的元素；硬件上它们可能同属一条 line。把一个锁、一个统计计数、一个队列 head 和一个取消标志塞进同一结构体，低频字段也可能被高频字段拖进所有权迁移。布局审查必须标出每个字段的写者和频率，而不仅是字段含义。

MESI 的直觉 许多协议可以用 MESI 直觉入门。Modified 表示当前核心有被修改且尚未写回的独占副本；Exclusive 表示当前核心有未修改的独占副本；Shared 表示多个核心可以读；Invalid 表示副本无效。一个核心想写 Shared line，需要先取得独占或修改权限，并让其他核心副本失效。其他核心随后再读，就要重新获取有效副本。

真实处理器可能使用 MESIF、MOESI 或更复杂的变体，也可能在不同缓存层级使用不同优化。软件不需要背完所有状态名，但必须抓住状态迁移的成本：共享读容易扩展，共享写需要协调所有权，频繁写同一条 line 最贵。Modified、Owned、Forward 这些状态名称的差异会影响具体路径，不改变这条基本判断。

为什么不每次都写回内存 如果每个 store 都立刻写到内存，再让其他核心从内存读最新值，一致性看起来会简单，但性能会非常差。内存比 L1/L2 缓存远得多，内存控制器带宽也有限；每次写都外推到内存，会让热循环被最慢路径支配。Modified 状态存在的意义，就是允许一个核心在拥有 line 的修改权时多次本地写入，等 line 被替换或其他核心请求时再向外传播。这样单核心或线程私有写非常快，代价是共享写必须通过协议协调所有权。

这也解释了 per-core、per-thread、per-NUMA 结构为什么常见。它们不是为了

让代码看起来复杂，而是让热写长期停留在一个核心或一个较近范围内，避免每次操作都把状态推出去协调。

Listing 4.7 MESI 直觉下的两类共享

```
read-mostly table:
  core0 Shared
  core1 Shared
  core2 Shared

hot counter:
  core0 Modified -> core1 Modified -> core2 Modified -> core0 Modified
```

这段文本图的重点不在状态名，而在共享形态。只读表可以同时停留在多个核心的缓存里；热点计数器只能让写权限轮流移动。很多性能问题就藏在这条差异里。

状态迁移账本：读、写、再读 把 MESI 直觉写成账本，比记状态名更有用。假设 core 0 和 core 1 轮流访问同一条 line：

Listing 4.8 同一条 line 的状态迁移账本

```
step 1: core 0 reads line X
  core 0: Exclusive or Shared
  core 1: Invalid

step 2: core 1 reads line X
  core 0: Shared
  core 1: Shared

step 3: core 0 writes line X
  core 0 requests write ownership
  core 1's Shared copy becomes Invalid
  core 0: Modified

step 4: core 1 writes line X
  core 1 requests write ownership
  core 0 must supply or write back the latest data
  core 0: Invalid
  core 1: Modified
```

这个账本解释了扩展曲线。只读阶段，副本可以扩散，多个核心本地命中；写阶段，写权限只能被一个核心持有；轮流写阶段，所有权反复迁移。高频 atomic、短临界区锁、共享队列 tail、false sharing 计数槽，都容易落入第四步的循环。

一致性不保证立即全局同时可见 一致性保证同一地址的副本受规则约束，不等于每个 store 在同一时刻被所有核心同时看到。store buffer、写回、失效确认、内存序和编译器重排都会影响程序能否把多个地址的更新当作一个有序协议。一个核心写 data 再写 flag，另一个核心看到 flag 后是否一定看到 data，需要 release/acquire、锁或其他同步关系来保证。硬件一致性是地基，语言同步是协议。

这个区别非常关键。第四章解释为什么共享写慢；第十一章解释怎样让共享状态有定义、可证明。若只学一致性，容易写出数据竞争；若只学原子 API，容易忽略 cache line 迁移。正确系统同时满足两件事：语义上有 happens-before，结构上没有把高频路径压到同一条 line。

False sharing 是布局层面的共享写 False sharing 指的是软件层面没有共

享同一个变量，硬件层面却共享同一条 cache line。最容易出错的例子是每线程计数槽位数组：`partials[0]` 由 worker 0 写，`partials[1]` 由 worker 1 写，源码上看互不干扰；如果两个槽位相邻并落在同一条 line，上述写入仍会让同一条 line 的所有权在核心之间迁移。

Listing 4.9 按 cache line 隔离高频写槽位的形态

```
1 struct alignas(64) WorkerCounter {
2     std::uint64_t value = 0;
3     std::byte padding[64 - sizeof(std::uint64_t)] {};
4 };
5
6 std::vector<WorkerCounter> counters(worker_count);
```

Padding 不是默认答案。它增加对象大小，降低缓存容量利用率，扩大合并阶段扫描的字节数，也可能增加 TLB 压力。只有被不同线程高频写的字段才值得隔离；只读字段、低频状态、同一线程访问的热字段不应被无意义地填满整条 line。布局判断要看写者、频率、共享范围和对象数量。

一致性和 C++ 内存模型的分工 硬件一致性解决的是“同一内存位置的多个缓存副本怎样协调”。C++ 内存模型解决的是“哪些跨线程访问有定义，哪些同步建立 happens-before，哪些原子操作提供怎样的排序”。二者不能互相替代。普通变量被多个线程无同步读写，即使底层 CPU 有缓存一致性，C++ 层面仍然是数据竞争。反过来，把操作改成原子，只能修复数据竞争和同步语义，不保证这个共享位置能扩展到很多核心。

因此不能用“CPU 有一致性”来为未同步普通变量辩护，也不能用“我用了 atomic”来推断性能良好。语言规则先决定程序是否有定义，硬件规则再决定有定义的同步怎样落到缓存、互连和等待路径上。

Store buffer 和可见性 核心执行 store 后，写入可能先进入 store buffer。对本线程来说，后续 load 可能通过 store forwarding 看到自己的写；对其他核心来说，这个写何时可见还要经过缓存一致性和内存序约束。释放锁、release store 和内存屏障会把这种可见性变成可依赖的同步关系。

这解释了为什么“我先写 data，再写 flag”在并发中不够。编译器和硬件都可能在允许范围内重排或延迟可见性。正确发布对象要用 mutex、release/acquire、condition variable 或其他同步机制建立明确关系。性能分析时也要记住：锁释放不是普通 store，获取锁也不是普通 load；它们都可能要求某条 cache line 的最新状态，并在竞争时等待所有权迁移。

锁变量和队列元数据也是一致性对象 一个 mutex 内部会有原子状态、等待队列和可能的 futex 系统调用。低竞争时，获取和释放可能主要发生在用户态，成本较低；高竞争时，锁状态所在 line 会在核心之间移动，等待线程可能进入内核睡眠和唤醒。临界区很短但竞争极高时，锁变量本身的所有权迁移可能主导；临界区很长时，等待时间和调度开销可能主导。

队列也类似。生产者主要写 tail 和槽位状态，消费者主要写 head；如果 head、tail、size 和高频统计字段挤在同一条 line，不同角色会互相干扰。SPSC ring 通常把 head 和 tail 分开对齐；MPMC 队列还会给每个槽位序号，减少集中争用。结构体布局不是装饰，而是把不同写者和不同频率的字段映射到合适的硬件单位。

4.3 互连：所有权消息走在哪里

当一个核心发现自己没有某条 line 的写权限时，它必须向别处请求。请求可能到达共享缓存、目录、拥有者核心或内存控制器；其他副本可能收到失效消息；拥有者可能提供最新数据；目录可能更新 sharer 集合。软件不会直接调用“互连”，但互连会出现在 cache line 迁移、远端内存访问、跨 socket 锁竞争、共享 LLC 争

用和 I/O 路径上。

小规模系统可以用广播窥探的直觉理解：一个核心要写某条 line，就通知其他核心检查自己是否有副本，并让副本失效。核心数量增加后，纯广播会变贵，因为每次写请求都可能触达许多参与者。于是很多系统使用目录式信息，记录某条 line 可能在哪些核心或节点有副本，再有针对性地发送消息。真实实现很复杂，但软件层面的结论非常朴素：共享范围越大，维护一致性的通信越难便宜；写者越分散，所有权迁移越频繁。

Listing 4.10 一次写权限请求的消息形态

```
core A wants to write line X
-> send Read-For-Ownership request for X
directory or snoop fabric finds sharers/owner
-> invalidate shared copies, or request owner to supply data
other cores acknowledge invalidation
-> core A receives writable copy
core A stores new bytes and holds Modified ownership
```

这段流程故意没有写具体厂商协议名。不同处理器会在缓存层级、目录位置、响应合并和数据转发上做大量优化，但软件能依赖的判断是稳定的：一次写热点不是单个核心内部事件，而是可能涉及请求、失效、确认、数据转发和目录更新。若写者在多个 socket 间来回切换，这些消息就会走更远路径。

环、mesh 和 socket 间链路 多核心处理器内部可以使用环形互连、mesh 网络或其他片上网络连接核心、缓存切片、目录和内存控制器。单 socket 内部已经不是“零距离”；多 socket 还会通过 UPI、Infinity Fabric 或类似链路连接。不同平台名称不同，基本成本模型相同：消息要在拓扑中走路径，占用带宽，经历排队，并可能经过多个中间结构。

这就是为什么一个全局队列在四核心开发机上看不出问题，在双 socket 服务器上可能突然变差。四核心机器上，head/tail 的 line 迁移还勉强能承受；跨 socket 后，每次迁移都可能穿过更远链路，延迟和带宽成本都更高。工程结论不能只来自最小机器上的正确性测试，还要在目标拓扑上观察扩展曲线。

目录协议的工程影响 目录协议维护某条 line 的副本位置或所有权信息。软件不需要实现目录协议，却要理解它给程序留下的影子：把数据保持私有，成本最低；在一个 core group 或 socket 内共享，成本较低；跨 socket 高频写共享，成本高；跨机器共享，成本更高。线程本地 partial、per-NUMA partial、socket 内合并、跨 socket 合并、跨机器 reduce，就是把这个成本阶梯显式写进算法。

这个阶梯还能解释许多数据结构选择。全局 hash table 如果所有线程争同一组 bucket，热点 line 会集中；分片 hash table 把写者分散到多个 shard，但全局快照和 rehash 变复杂；全局 MPMC 队列简单，但 head/tail 容易成为热点；本地队列加工作窃取让高频路径局部化，低频失衡才访问远处。所谓“更高级”的数据结构并不一定更快，关键是它是否减少了高频共享写和远距离通信。

互连上的数据和权限是两类流量 一次远端读取大数组，主要传输的是数据字节；一次共享写热点，传输的不只是数据，还有权限和失效消息。二者都会占用互连，却有不同表现。只读流式扫描可能随着线程数增加撞上带宽上限；热点原子可能在数据字节很少的情况下仍然扩展很差，因为每次修改都要重新安排所有权。看到“处理的数据不大但很慢”时，尤其要怀疑共享写，而不是只看内存带宽。

内存控制器带宽也是共享资源 互连成本不只来自一致性消息。内存控制器本身也有带宽上限。若所有线程都扫描同一内存节点上的大数组，即使核心分布在多个节点，数据仍从少数控制器出来；线程数增加后，总吞吐可能到顶，每线程吞吐下降。此时继续增加线程只会让每个线程分到更少带宽。

带宽饱和有时和共享写很像：线程数增加后曲线停住。但修复方向不同。共享写

热点要分片、局部化或批量合并；带宽饱和要减少搬运字节、改善局部性、压缩数据、分散到多个控制器或承认机器资源已经到顶。实验必须把只读扫描、局部写、共享写和合并阶段分开测，不能把所有扩展失败都归因于“线程太多”。

互连瓶颈的表现 互连或控制器瓶颈常见表现是“每个线程单独跑都快，一起跑都慢”。可能原因包括多个核心争同一内存控制器、跨 socket 远端访问过多、共享写导致 line 在互连上来回移动、多个任务争同一共享 LLC 切片或设备路径。排查时要把线程数、绑定位置、数据放置、访问模式、共享写位置和合并阶段放进同一张对照判读表。

现象	可能原因	需要的对照
全局 atomic 随线程数变慢	同一 line 所有权迁移	per-worker partial、padding、低频批量提交
只读大数组到某线程数后停住	内存带宽或控制器饱和	改变数据规模、节点放置、线程绑定
绑定后波动变小但均值不变	调度迁移是噪声，不是主瓶颈	记录迁移次数、分段计时
最终合并时间很长	串行远端读或合并树不合适	节点内合并、树形合并、合并字节统计

4.4 NUMA：内存不是无位置的池

NUMA 是 Non-Uniform Memory Access，意思是不同核心访问不同内存节点的成本不一样。最常见的服务器形态是每个 socket 或 die 连接自己的内存控制器和一部分物理内存；本 socket 核心访问本地内存延迟更低、带宽更高，访问其他 socket 的内存要经过互连。某些 chiplet、工作站、云实例和大型桌面平台也会呈现非均匀拓扑，只是操作系统暴露方式不同。

NUMA 不是“内存慢一点”的口号，而是页面有物理归属。虚拟地址由页表映射到物理页；物理页来自某个节点的内存。线程在 node 0 上运行，却频繁访问 node 1 的页面，就会产生远端访问。若所有页面都集中在 node 0，node 1 上的核心再多，也可能被远端链路和 node 0 内存控制器限制。

First-touch 从缺页路径推导出来 Linux 常见策略是 first-touch：页面通常在第一次实际写入或缺页分配时落到触碰它的 CPU 所在 NUMA 节点。调用 malloc 或 reserve 往往只获得虚拟地址范围，不一定立刻分配物理页。第一次写入触发缺页，内核选择物理页。于是初始化代码不再是无关准备工作，它决定页面最初属于哪个节点。

若主线程在 node 0 上单线程初始化一个大数组，页面可能集中在 node 0。随后 worker 分布到 node 0 和 node 1 处理这个数组，node 1 上的 worker 就要远端访问。更好的实验版本是让每个 worker 初始化自己后续要处理的块，并在计算阶段使用同样的 worker-to-range 映射。这样页面归属和热访问者一致，多个内存控制器也能共同工作。

Listing 4.11 first-touch 缺页放置的抽象流程

```
1 PhysicalPage resolve_write_fault(Thread t, VirtualPage vp) {
2   VmaPolicy policy = lookup_memory_policy(vp);
3   NumaNode preferred = policy.preferred_node_or(t.current_cpu.node);
4
5   PhysicalPage page = allocate_page_from(preferred);
6   map_virtual_page_to_physical_page(vp, page);
7   return page;
8 }
```

这段伪代码省略了 VMA、页表锁、透明大页、回收、迁移和安全检查，但保留了实验最关心的因果：第一次写入发生在哪个执行位置，会影响物理页面来自哪个节点。主线程清零、worker 本地初始化、交错分配和显式 membind，都会通过这条路径改变后续访问距离。

Listing 4.12 NUMA 实验的阶段分解

```
allocate virtual address range
bind workers or record actual CPU placement
initialize pages by the worker that will later use the range
warm up if the experiment needs warm cache or stable placement
compute with the same worker-to-range mapping
merge worker partials by node, then merge node results
```

从虚拟地址推到内存节点 NUMA 不是在指针上贴一个节点标签。用户态指针仍然是虚拟地址；页表项把虚拟页映射到物理页框；物理页框属于某个内存节点；内存节点连接到某个内存控制器和拓扑位置。一次 load 的路径因此可以写成：虚拟地址经过 DTLB 得到物理页号，物理地址进入 cache 层级，miss 后请求被路由到拥有该 line 的缓存、LLC 切片或内存控制器；若目标物理页在远端节点，请求和数据要穿过 socket 或 die 间互连。

Listing 4.13 远端内存访问的身份链

```
user pointer:
    virtual address

page table:
    virtual page -> physical page frame

physical placement:
    physical page frame belongs to NUMA node N

execution placement:
    current thread runs on CPU in NUMA node M

access cost:
    local if M == N
    remote if M != N, with interconnect latency and bandwidth cost
```

这条链说明为什么单纯打印指针值没有诊断力。同一个虚拟地址范围，在不同运行中可能因 first-touch、自动迁移、membind 或内存压力落到不同节点；同一线程 id 也可能因调度迁移跑到不同 CPU。报告需要同时记录虚拟范围、初始化线程、实际 CPU、可见节点、页面放置线索和绑定策略。否则只能看到“同一程序有时快有时慢”，看不到位置身份改变。

页面迁移不是免费修复 自动 NUMA balancing 或手工迁移可以把热页挪近执行线程，但迁移本身要复制页面、更新页表、处理 TLB shutdown，并可能扰动缓存。长时间稳定访问的服务可能从迁移中受益；短小 benchmark 或快速阶段切换的任务可能还没收回迁移成本就结束。若一个数组先由 node 0 解析、随后由 node 1 聚合、最后又由 node 0 写出，页面来回迁移可能比远端访问更糟。

因此 NUMA 设计优先追求“让数据从一开始就在正确位置，且阶段之间位置变化少”。Worker 本地初始化、按 split 固定 worker-to-range、节点内合并和低频跨节点摘要，都是为了减少后续迁移需求。自动机制可以作为生产系统的一部分，但

实验报告要写清它是否参与；否则一轮运行的改善可能来自内核后台迁移，而不是算法结构。

线程绑定和 first-touch 是两件事 线程绑定约束执行位置；first-touch 影响页面归属。绑定线程不会自动移动页面，移动页面也不会自动固定线程。一个实验若只绑定线程，却让主线程初始化所有数据，仍然可能让远端访问进入热循环。反过来，只做并行初始化但计算时线程被调度到其他节点，也会破坏局部性。

绑定的价值首先是减少变量。未绑定运行可能因为调度迁移导致样本波动；绑定后更容易解释线程和数据位置。但绑定也可能伤害系统：把多个 heavy worker 绑到同一核心或同一 LLC 组会变慢；容器、云实例或受限环境可能限制可用 CPU；长期固定高性能核心可能引发频率和温度变化。实验报告要写绑定策略、可见 CPU mask 和失败时的降级，而不是把“绑定”写成绝对好事。

自动 NUMA balancing 是隐藏变量 Linux 自动 NUMA balancing 会采样页面访问，并可能把页面迁移到更常访问它的节点。它能改善一些长期运行程序，但也会引入额外 page fault、页面迁移成本和实验不确定性。短 benchmark 可能还没等自动机制收敛就结束；长服务可能在负载变化时发生页面迁移。

做可控实验时，要记录自动 NUMA balancing 是否启用；若使用 numactl、线程绑定和明确 first-touch，就要说明这些策略是否真正生效。评估生产行为时，则应在接近真实配置下运行，并把自动迁移视为系统的一部分。关键不是永远关闭或永远依赖自动机制，而是不要让它成为看不见的变量。

大小核、容器和虚拟化的边界 异构核心和 NUMA 都意味着执行资源不均匀，但原因不同。异构核心是核心性能和能耗不同，同一内存访问拓扑未必像多 socket 那样分节点；NUMA 是内存位置和访问成本不同，核心本身可能同构。线程数增加后变慢，可能是低性能核心加入、频率下降、温度限制、调度迁移或远端内存，不能直接归因于 NUMA。

容器和云环境也会改变可见拓扑。cgroup 可能限制 CPU 集合、CPU 配额、内存节点和性能计数器权限；std::thread::hardware_concurrency 看到的数量未必等于实际可用核心数；numactl--hardware 可能不可用或只显示虚拟化后的视图。实验输出至少要记录可见 CPU 数、cpuset、worker 数、是否能读 /sys/devices/system/node/，以及哪些结论无法在当前设备验证。

4.5 把拓扑写进算法：分层 partial 和本地优先

从全局计数器改成线程本地 partial，解决的是共享写热点；从线程本地 partial 改成按 NUMA 节点分层合并，解决的是远端合并和页面归属。算法结构自然长成三层：worker 本地处理，节点内合并，跨节点合并。这个结构和分布式系统中的 local combine、node-level reduce、global reduce 是同一条思想，只是“远处”的尺度不同：单机里远处是另一个 NUMA 节点，集群里远处是另一台机器。

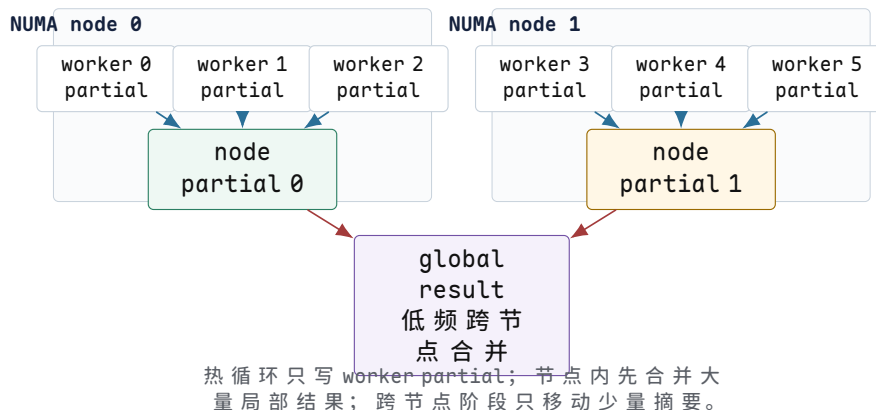
Listing 4.14 分层归约的抽象伪代码

```
1 for (Worker w : workers) {
2   w.partial = compute_local_partial(w.input_range);
3 }
4
5 for (Group g : topology.groups) {
6   g.partial = merge_partials(g.workers);
7 }
8
9 Result result = merge_group_partials(topology.groups);
```

这段伪代码看起来简单，真正的设计点在数据归属。第一层要求 worker partial 由 worker 高频写；第二层要求同 group 的 partial 尽量在本地合并；

第三层只处理压缩后的 group partial。若第一层 partial 全由主线程分配并触碰，页面可能已经错位；若第二层合并线程跑到远端 CPU，本地合并也会变成远端访问；若第三层合并数据很大，说明前两层没有充分压缩。

NUMA 分层归约



图示：本图要证明的命题是：分层归约把高频写留在近处，把远端通信压缩到低频摘要合并。

合并树改变的不只是速度 全局 atomic 的读取很简单：load 一个值即可。拆成 partial 后，读取全局值需要合并多个槽位；运行中展示进度时，可能只能得到近似快照。若业务要求严格单调、严格实时的全局进度，就要定义读取协议。优化不会只改变性能，也会改变可见状态的语义。

浮点归约还会改变数值顺序。线程本地 partial、树形合并和 NUMA 分层会以不同顺序相加，浮点舍入误差可能不同。整数计数通常没有这个问题；浮点求和要写清误差合同、固定合并树或使用补偿求和。系统优化不能偷偷改变语义。

队列的本地优先设计 全局 MPMC 队列常成为跨节点热点。所有生产者争 tail，所有消费者争 head 或槽位状态；队列锁、head/tail 和统计字段可能在节点之间迁移。拓扑感知线程池通常会拆成本地队列或节点队列：worker 优先从自己的 deque 取任务，空闲时先在同节点窃取，再跨节点窃取。这样高频路径局部化，低频失衡才付远端成本。

本地优先不是越强越好。若任务粒度极小，频繁窃取会放大调度成本；若负载高度不均，过度坚持本地会让部分 worker 空闲；若任务访问的数据没有位置偏好，全局队列可能足够简单。设计要通过队列长度、窃取次数、远端执行次数、任务耗时分布和合并阶段来判断，而不是凭口号拆队列。

分片锁和分片表 哈希表、计数表和状态表可以用 shard 降低协调粒度。每个 shard 有自己的锁、局部桶和统计字段，线程只竞争相关 shard。这样减少一致性流量，但引入跨 shard 操作复杂性：全局快照、关闭、rehash、迁移 key 和一致性提交都可能阶段协议。

Shard 太少，竞争仍高；shard 太多，内存和遍历成本上升；key 倾斜会让某个 shard 仍然热点。报告要记录每个 shard 的请求分布、最大 shard 负载、锁等待时间和跨 shard 操作次数。只看总吞吐会掩盖局部热点。

内存池和远端释放 NUMA 不只影响分配，也影响释放。对象在 node 0 分配，由 node 1 释放，分配器元数据和对象 line 可能在节点间移动。高性能分配器常使用线程本地或节点本地缓存，但跨线程释放仍然需要特殊路径。对象池可以让 owner 回收自己分配的对象；远端释放先进入 remote free 队列，再由 owner 批量回收。

无锁数据结构的内存回收也会碰到这个问题。Hazard pointer 或 epoch 延迟释放后，最终由哪个线程 delete 节点？如果所有删除都集中到一个线程，可能造成远端释放和内存热点。内存管理和拓扑不能完全分开。

I/O 设备也有位置 网卡、NVMe 和 GPU 等设备通常连接到某个 PCIe root complex，离某些 CPU/NUMA 节点更近。高性能网络和存储系统会把中断、设备队列、接收 buffer 和处理线程尽量放在同一节点。否则线程在远端节点准备 buffer，再交给近端设备 DMA，路径会绕远，cache、IOMMU 和互连成本都会增加。

日志处理和 shuffle 程序同样受影响。若网卡在 node 0，接收缓冲在 node 0，但 reduce worker 在 node 1 上读取，远端内存流量会进入数据面。拓扑意识不只属于计算内核，也属于 I/O 管线、内存池和任务调度。

4.6 从对照判读区分一致性、带宽和 NUMA

线程数增加后吞吐不再提升，只是现象，不是原因。可能是全局锁，也可能是 cache line 所有权迁移、内存带宽饱和、远端内存、任务粒度过小、调度迁移、SMT 竞争或 I/O 等待。若不做对照判读，NUMA 和一致性很容易被当成万能解释。

四个最小对照 第一组对照是只读扫描。每个 worker 读取自己范围，不写共享状态。若只读扫描随线程数扩展到某点后停住，优先怀疑内存带宽、内存控制器或 NUMA 放置，而不是锁。第二组对照是本地写扫描。每个 worker 写自己的输出范围或本地 partial。若本地写扩展正常，说明 store buffer、写分配和带宽尚可。第三组对照是共享写热点。所有 worker 更新同一个 atomic 或同一条 line 上的槽位。若这里崩掉，说明一致性所有权迁移很可能参与。第四组对照是分层 partial。若它恢复扩展性，说明把高频写本地化是正确方向。

对照	热路径形状	主要观察
只读扫描	多 worker 读独立范围	带宽上限、远端读、预取能力
本地写扫描	每个 worker 写独占范围	写分配、store buffer、输出带宽
共享写热点	多 worker 写同一 line 或同一 atomic	cache line 所有权迁移、一致性流量
分层 partial	worker 本地写，节点内合并	本地化是否消除热点，合并是否变成新瓶颈

这些实验都要保持逻辑工作量和 checksum 可比。若共享写版本结果正确但慢，问题是扩展性；若分层 partial 结果变了，问题先回到语义。性能实验不能绕过正确性。

把共享写拆到一次 store 的消息链 共享写问题常被一句“cache line 来回跑”概括，但工程判断需要更细的账本。一次普通 store 先在核心内计算地址，经过 DTLB 翻译，查找 L1D；若目标 line 已经在本核心 Modified 或 Exclusive 状态，写入可以进入 store buffer 并较快完成。若目标 line 不在本核心，或者只以 Shared 状态存在，核心就要发出 Read-For-Ownership 请求。RFO 的含义是：不仅要读到这条 line，还要取得未来写它的权限。

Listing 4.15 一次共享 store 可能触发的消息链

```

core A wants to store to address x
compute address and find cache line X
L1D does not hold X in writable state
send Read-For-Ownership request for X
directory or snoop fabric finds owner and sharers
invalidate shared copies or request owner to supply modified data
wait for required acknowledgements
receive writable copy of X
commit store into local cache line through store buffer

```

这条链解释两个看似矛盾的现象。第一，线程私有写可以很快，因为 writable line 长期停在本核心，store buffer 能平稳排出。第二，热点共享写可以在数据量极小的情况下很慢，因为每次写入前都可能要经过权限请求、失效、确认和数据转发。整数加法本身仍然很便宜，昂贵的是把“谁有权写这条 line”在多个核心之间重新安排。

store buffer 不能消除一致性等待 Store buffer 让核心把尚未完全写回缓存层级的 store 暂存起来，从而减少后续指令被普通写入阻塞的机会。它不是一致性协议的替代品。若目标 line 还没有可写所有权，store buffer 里的条目不能凭空对其他核心生效；若 store buffer 被未完成共享写填满，后续 store 也会反过来阻塞前端和后端。高频原子、锁释放、共享队列 head/tail 和 false sharing 槽位，都可能把 store buffer 从隐藏延迟的结构变成等待暴露的位置。

这也解释了为什么 `memory_order_relaxed` 只能解决排序成本的一部分。Relaxed 原子仍然要对同一原子对象建立不可分割的修改顺序，仍然要让目标 line 在执行 RMW 时处于合适的独占状态。若瓶颈是所有权迁移，放松 acquire/release 关系可能改善有限；若瓶颈是多余屏障或同步边界，内存序选择才可能明显影响热路径。先分清“排序等待”和“所有权等待”，再讨论内存序。

等待来源	底层问题	常见修复方向
line 所有权等待	多核心高频写同一 line	thread-local partial、shard、padding、批量提交
排序或屏障等待	需要发布/观察顺序或强同步边界	缩小同步范围、使用恰当 memory order、阶段协议
store buffer 压力	未排出的 store 过多或目标 line 未就绪	连续私有写、减少共享写、分块输出
锁等待	临界区被占用或等待线程进入内核	缩短临界区、拆锁、局部化状态、队列化

False sharing 的汇编视角 False sharing 不会在汇编里显示为特殊指令。两个 worker 可能都执行普通的 `mov` 或 `add` 到不同地址，源码也明确写的是不同数组元素。问题藏在地址低位：两个地址落在同一条 cache line。硬件的一致性单位是 line，于是两个不同变量共享同一份所有权。

Listing 4.16 两个独立槽位仍可能落在同一条 line

```
worker 0:
    add qword ptr [partials + 0], 1

worker 1:
    add qword ptr [partials + 8], 1

if both addresses are inside the same cache line:
    the line ownership moves between worker 0 and worker 1
```

反汇编只能告诉写入地址公式，不能单独告诉这两个地址是否同线。还要结合对象布局、对齐、元素大小、分配器返回地址和运行时 worker 数。紧凑 partial 和 aligned partial 的对照实验，就是把这件事变成可观察差异：若仅改变槽位间距就明显改善多线程扩展，false sharing 假设得到支持；若没有改善，瓶颈可能在全局合并、带宽、任务调度或其他共享结构。

线程绑定不是目的，是诊断工具 绑定线程到核心或节点，可以减少调度迁移变量，让页面 first-touch 和计算位置更容易对齐。诊断时可比较三种状态：不绑定、只绑定计算、绑定初始化和计算。若只绑定计算无改善，绑定初始化后改善，说明页面放置参与；若两者都无改善，可能是带宽、共享写、任务粒度或其他问题。若绑定后波动下降但中位数不变，调度迁移只是噪声来源，不是主瓶颈。

Listing 4.17 NUMA 对照运行矩阵

```

run A: no binding, main thread initializes all pages
run B: compute workers bound, main thread initializes all pages
run C: workers bound, each worker first-touches its own range
run D: workers bound, memory interleaved across nodes

record:
    throughput, per-stage time, page faults, node placement if available,
    worker CPU placement, checksum, remote/local evidence if available

```

当前设备可能没有 NUMA 或无法读取节点信息。此时仍可保留矩阵设计，并把 NUMA 证据标为 unavailable；不要把单 socket 结果写成“NUMA 无影响”。单 socket 能验证共享写和 false sharing，不能完整验证远端内存。

用输入形状放大或缩小问题 共享写热点可以用单热点 key 放大：90% 记录落到同一个 event id，全局 atomic 会被迫争同一计数槽。内存带宽可以用大只读数组放大：每个元素只读一次，几乎没有计算。TLB 可以用每页一个元素放大。NUMA 可以用大数组和跨节点 worker 放大。不同输入形状像显微镜，分别放大不同底层成本。

因此第 5 章的输入矩阵不是测量章独有工具，也是 NUMA 和一致性的诊断工具。若一个优化只在单热点 key 上提升，结论是“修复共享写热点”，不是“所有输入都更快”；若只在大只读输入上停滞，结论可能是“带宽已满”，不是“锁有问题”。

扩展曲线要按阶段解释 端到端吞吐随线程数变化时，至少拆成 parse、aggregate、merge、write、commit。共享写问题通常集中在 aggregate 或队列阶段；NUMA 远端读可能同时影响 parse 和 aggregate；内存带宽饱和会让多个流式阶段一起到顶；writeback 或 fsync 会让 write/commit 阶段长尾。只看总吞吐，会把完全不同的瓶颈折叠成一条曲线。

Listing 4.18 扩展曲线报告的最小字段

```

threads, binding_policy, first_touch_policy,
parse_time, aggregate_time, merge_time, write_time, commit_time,
records_per_second, p95_task_time, max_partition_time,
checksum, unavailable_metrics

```

p95_task_time 和 max_partition_time 用来观察长尾。NUMA 和互连问题常常不是平均慢，而是某些远端任务、热点 shard 或跨节点合并特别慢。平均值会掩盖这些尾部。

4.7 完整案例：一条 cache line 从本地写到跨 socket 争用

把一致性、互连和 NUMA 讲清，最有效的办法是只跟踪一条 cache line。假设日志系统有一个 GlobalStats，其中 accepted、rejected、bytes 和 slow_path 四个计数器相邻。worker 0 主要更新 accepted，worker 1 主要更新 rejected，worker 2 更新 bytes，worker 3 更新 slow_path。源码层面看，四个线程写四个不同变量；硬件层面看，它们可能写同一条 cache line。

Listing 4.19 软件不同变量可能落在同一条 cache line

```

1 struct GlobalStats {
2     std::atomic<std::uint64_t> accepted;
3     std::atomic<std::uint64_t> rejected;
4     std::atomic<std::uint64_t> bytes;

```

```

5  std::atomic<std::uint64_t> slow_path;
6  };

```

若这些字段同处一条 64 字节 line, worker 0 的 `accepted.fetch_add` 要拿写权限; worker 1 的 `rejected.fetch_add` 随后也要拿同一条 line 的写权限; worker 2 和 worker 3 继续重复这件事。变量没有共享, line 共享了。这个案例比单一全局计数器更隐蔽, 因为代码审查时很容易认为“每个线程写自己的字段”已经足够。

第一阶段：单 socket 内的所有权迁移 在单 socket 内, 写权限请求通常通过共享缓存、目录或窥探结构找到当前拥有者。一次写可以抽象成下面的消息链：

Listing 4.20 单 socket false sharing 的消息账本

```

worker 0 writes accepted:
  request ownership of line L
  invalidate other cached copies of L
  update accepted inside L

worker 1 writes rejected:
  request ownership of the same line L
  worker 0's copy becomes invalid or supplies data
  update rejected inside L

worker 2 writes bytes:
  request ownership of the same line L again
  previous owner loses write permission

```

这里最重要的事实是：每次更新的算术很轻，但所有权迁移很重。若每条记录都更新一次统计，程序就把每条记录都变成一次互连协议动作。反汇编中可能只看到短短的 `lock add` 或 `lock xadd`，但这条指令背后的 line 状态在核心间来回移动。

第二阶段：跨 socket 以后等待更长 若 worker 分布在两个 socket 上，同一条 line 的所有权可能跨 socket 链路迁移。跨 socket 通信通常比同 socket 更慢，也更容易挤占内存控制器和互连带宽。此时线程数增加不仅不能线性扩展，还可能让远端链路成为主路径。

Listing 4.21 跨 socket 写热点的等待形状

```

socket 0, core 3 owns line L in Modified state
socket 1, core 9 wants to update another field in L

core 9:
  sends ownership request through socket interconnect
  waits for current owner or directory response
  receives newest line data or permission
  performs the atomic update

next update from socket 0 repeats the reverse trip

```

这解释了为什么“绑定线程”有时能改善结果，有时没有效果。若所有写者仍然争同一条 line，绑定只改变争用发生在哪里；若先把写路径分片，让每个 socket 写自己的 local stats，再按 socket 合并，绑定才有结构意义。位置策略必须和

布局策略一起出现。

第三阶段：按写者重排结构 修复不是把所有字段都 `alignas(64)`。修复要先问写者是谁、写频率多高、读取方什么时候读。若四个字段都被每个 worker 高频更新，应该做 per-worker 或 per-socket partial；若字段各有固定写者，可以按写者分组；若只是低频指标，保持紧凑可能更好。

Listing 4.22 按 worker 隔离高频统计，再按阶段合并

```

1 struct alignas(64) WorkerStats {
2     std::uint64_t accepted = 0;
3     std::uint64_t rejected = 0;
4     std::uint64_t bytes = 0;
5     std::uint64_t slow_path = 0;
6 };
7
8 std::vector<WorkerStats> worker_stats(worker_count);
9
10 void process_worker(std::uint32_t worker_id, BatchView batch) {
11     WorkerStats& local = worker_stats[worker_id];
12     for (RecordView record : batch) {
13         local.bytes += record.bytes;
14         if (record.valid) {
15             ++local.accepted;
16         } else {
17             ++local.rejected;
18             local.slow_path += record.needs_diagnostic;
19         }
20     }
21 }
```

这段结构让热循环只写本 worker 的 line。合并阶段仍要读所有 partial，但合并是低频、顺序、可分层的。若在 NUMA 机器上，还可以把 `worker_stats` 按 socket 分配，先 socket 内合并，再跨 socket 合并。这样跨 socket 流量从“每条记录一次”降到“每阶段一次或每批一次”。

第四阶段：把页面归属也写进账本 布局修复后还可能慢，因为 partial 数组的物理页归属不对。若主线程一次性初始化所有 `worker_stats`，页面可能集中在主线程所在节点；远端 worker 每次写自己的 partial，仍然是在写远端内存。更稳的策略是让每个 worker first-touch 自己的区域，或由 NUMA-aware allocator 按节点分配。

Listing 4.23 NUMA 初始化和计算要匹配

```

bad:
    main thread allocates and writes all partial pages
    worker threads on other sockets update remote pages

better:
    each worker initializes its own partial storage
    computation runs on the same or nearby CPU set
    merge stage records local and remote bytes separately
```

这里的“页面归属”来自第 3 章的缺页路径：第一次触碰页面时，内核选择物理

页；第 4 章把这个物理页放置接到线程位置；后续并发章节再把这个位置接到调度和运行时。若报告只写 `std::vector<WorkerStats>`，没有写 first-touch 和 CPU 绑定，NUMA 结论不完整。

第五阶段：实验字段 这个案例的报告不应只写“padding 后快了”。它至少要记录：

Listing 4.24 一致性和 NUMA 案例报告字段

```
stats.layout
stats.bytes_per_worker_slot
stats.cache_line_isolation
stats.writer_count_per_line
stats.updates_per_record
stats.first_touch_policy
stats.cpu_binding_policy
stats.socket_count
stats.local_merge_ns
stats.cross_socket_merge_ns
stats.remote_access_evidence
stats.checksum
```

`remote_access_evidence` 可以来自平台计数器、NUMA 工具、页放置记录或替代证据；不能得到硬件计数时，也要写明不可用。关键是把结论落到可验证事实：更新频率下降了吗，写者是否真的分离到不同 line，页面是否贴近写者，跨 socket 合并是否被限制到低频阶段。

第六阶段：什么时候不应该优化 如果统计只在每秒刷新一次，或者 worker 数很小，padding 和 per-worker partial 可能浪费内存并增加合并复杂度。如果字段需要频繁被监控线程读取，把每个 worker 的计数分散到许多页面，监控读取也会变重。成熟设计要允许不同等级：热路径使用 worker-local；低频控制面保持紧凑；导出指标时批量快照。所有权迁移是成本，结构复杂度也是成本，二者都要写进取舍。

4.8 C++ 接口怎样表达位置

C++ 标准库不直接暴露 NUMA 拓扑，但工程代码可以在接口层保留位置语义。线程池可以知道 worker 所属的逻辑组；任务可以带有 preferred group；内存池可以按组创建 arena；队列可以按 shard 或 node 分片；归约可以输出分层 partial。接口不需要一开始绑定某个 Linux API，但不能把任务看成完全无位置的函数。

Listing 4.25 任务元数据中保留位置偏好

```
1 struct PlacementHint {
2     std::int32_t preferred_worker = -1;
3     std::int32_t preferred_group = -1;
4     std::uint64_t data_block_id = 0;
5     std::uint64_t expected_bytes = 0;
6     bool can_migrate = true;
7 };
8
9 struct ComputeTask {
10     std::size_t begin = 0;
11     std::size_t end = 0;
12     PlacementHint placement;
```

```
13 };
```

这段结构不是要求立刻实现完整 NUMA runtime，而是让接口保留“任务最好靠近哪批数据”的表达能力。数组块任务的 `begin/end` 暗含数据位置；分区聚合的 `partition id` 暗含目标 `shard`；线程池本地队列暗含 `worker` 位置。位置语义一旦完全丢失，后面很难再补回来。

拓扑抽象不要硬编码机器 项目可以定义抽象拓扑，而不是直接把某台机器的 CPU 编号写进算法。单 `socket` 或受限环境返回一个逻辑 `group`；双 `socket` 服务器返回多个 `group`；异构核心设备可以按性能等级分组；容器环境按可见 CPU 集合降级。算法只依赖 `group` 抽象：组内先合并，组间再合并；调度器尽力满足 `hint`，无法满足时记录 `fallback`。

Listing 4.26 拓扑描述的最小形态

```
1 struct WorkerTopology {
2     std::int32_t worker_count = 1;
3     std::int32_t group_count = 1;
4     std::vector<std::int32_t> worker_to_group;
5     bool binding_supported = false;
6 };
```

这个结构不直接要求 CPU `id`，是为了先表达策略，再逐步接入真实 `affinity`。没有 NUMA 信息时，所有 `worker` 属于 `group 0`；绑定不可用时，仍然可以做逻辑分层实验，并在报告中记录未绑定。平台能力强时，`group` 可以映射到真实 NUMA 节点；平台能力弱时，`group` 只是逻辑分组。

trace 里必须有拓扑事实 一次运行突然变慢，如果 `trace` 里只有线程 `id`，很难判断原因。更有用的字段包括 `worker id`、实际 CPU、逻辑 `group`、建议 `group`、数据块 `id`、`partial` 地址范围、初始化线程、合并层级、远端执行次数和绑定失败原因。这样报告能回答：慢线程是否迁移，页面是否集中在一个节点，某个 `partial` 是否可能 `false sharing`，最终合并是否把远端读集中到一个线程。

Listing 4.27 拓扑运行摘要的示例形态

```
1 struct PlacementTrace {
2     std::int32_t worker_id = -1;
3     std::int32_t actual_cpu = -1;
4     std::int32_t actual_group = -1;
5     std::int32_t preferred_group = -1;
6     std::uint64_t data_block_id = 0;
7     std::uint64_t processed_bytes = 0;
8     std::uint64_t local_tasks = 0;
9     std::uint64_t remote_tasks = 0;
10    std::uint64_t steal_count = 0;
11 };
```

拓扑只影响性能和局部性，不应改变任务语义。任务无论在哪个 `worker` 上执行，`checksum`、错误集合和输出合同都必须相同。测试可以先随机打乱 `placement`，确认结果一致；再启用本地优先策略，测性能和局部性。这样调度策略和业务正确性不会纠缠。

降级设计是接口质量的一部分 很多运行环境没有多 NUMA 节点，或没有权限设置 `affinity`，或不能读取性能计数器。代码必须能降级：读取不到真实 `node` 时退化成一个 `group`；绑定失败时记录失败并继续；不能控制内存策略时仍可做普通

分块；不能读取 cache-to-cache 事件时用扩展曲线、padding 对照和合并耗时代替。

降级不等于忽略拓扑。即使只有一个 group，程序仍然可以保留 worker-to-group 映射、分层 partial 接口、队列分片和 trace 字段。这样同一份代码在普通设备上可运行，在服务器上能启用更多策略。更重要的是，报告会诚实说明哪些策略真的生效，哪些只是逻辑结构。

4.9 从共享计数走到分层归约

这一节使用同一个日志统计任务推导共享写的迁移路径。输入固定，语义固定，逐步改变共享形态。版本一使用全局 atomic 计数；版本二使用每 worker 计数；版本三把每 worker 计数按 cache line 隔离；版本四按 group 做分层 partial；版本五把最终合并改成树形。每个版本输出相同 checksum 和计数结果，只改变写路径和合并路径。

版本	改动	要观察的现象
全局 atomic	每条记录更新同一计数器	线程数增加后共享写是否饱和
worker partial	热路径写本地计数	热循环是否加速，合并是否出现
aligned partial	隔离高频写槽位	false sharing 对照是否明显
group partial	先组内合并，再全局合并	远端合并和页面放置是否影响结果
tree merge	固定合并树	最终合并长尾是否下降，语义是否保持

对照判读不要只跑一个数字 每个版本至少改变线程数、输入规模和任务粒度。线程数展示扩展曲线；输入规模决定数据是否超出缓存和内存带宽上限；任务粒度决定队列和调度成本是否显著。若只有一个线程数和一个输入规模，结果很容易被偶然因素支配。

分段计时必须保留：读取或生成输入、初始化页面、热循环、本地合并、组内合并、跨组合并、写出结果。线程本地 partial 常让热循环变快，却可能增加合并阶段；padding 可能减少 false sharing，却增加扫描字节；NUMA 分层可能减少远端访问，却在负载不均时增加等待。没有分段计时，优化只是把成本藏到别处。

共享写和远端读要分开测 一个扩展曲线停住，可能是共享计数器在争 line，也可能是远端读占用互连，也可能是最后合并串行化。对照版本要能拆分原因：只读本地数据观察带宽和页面放置；线程本地 partial 观察本地写和合并；全局 atomic 观察一致性流量；故意交叉 worker 和数据块观察远端访问。

若有 perfc2c、uncore 计数器、numastat 或平台 PMU，可以记录更强证据。若没有权限，不应写具体互连流量数字，而应写“结果符合共享写热点假设”或“当前设备无法直接验证远端访问”。结论的强度必须匹配证据。

拓扑记录优先于事后猜测 实验开始前记录 lscpu、numactl--hardware、/sys/devices/system/node/、可见 CPU mask、容器 cpuset、worker 数、绑定策略、自动 NUMA balancing 状态和输入规模。受限环境中至少记录可见 CPU 数、调度限制和缺失项。没有背景材料，后面看到曲线变化只能猜。

最小实现保留全局 atomic、worker partial、aligned partial 和 group partial 四个版本。每个版本输出相同 checksum、错误计数、线程数、worker 到 group 映射、初始化策略、热循环耗时、合并耗时和未验证边界。若当前设备没有可观察 NUMA 节点，仍然完成前三个版本，并把 group partial 作为逻辑分组运行。

判读扩展曲线 全局 atomic 若在少线程时足够快，不说明它永远合适；若线程增加后吞吐停住，说明共享写假设值得检查。Worker partial 若大幅改善，说明热共享写很可能参与成本。Aligned partial 若比紧凑 partial 快，说明 false sharing 可能存在；若不变，瓶颈可能不在相邻槽位。Group partial 若只在多节点机器上改善，说明远端放置和合并路径参与；若在单节点机器上无收益，这是合理结果。

判读还要写成本迁移。优化后瓶颈可能从全局写迁移到内存带宽，从热循环迁移到最终合并，从共享 line 迁移到队列调度。好的报告不只写“版本四最快”，还要写“版本二消除了热路径共享写，版本三排除了相邻槽位争用，版本四在当前实验环境没有额外收益，因为只有一个可见 NUMA 节点”。

4.10 Linux 观察入口和源码阅读

Linux 上可以从用户态观察入口开始：`lscpu` 查看 CPU、socket、core、SMT 和 cache 信息；`numactl--hardware` 查看 NUMA 节点和距离；`/sys/devices/system/cpu/` 和 `/sys/devices/system/node/` 提供拓扑文件；`/proc/self/status` 可以看到部分 CPU mask；`/proc/self/numa_maps` 在可用时显示页面放置线索；`perfstat`、`perfc2c` 和 `numastat` 在权限允许时提供计数证据。

线程绑定可以从 `taskset`、`numactl--physcpubind`、`numactl--cpunodebind` 或程序内 affinity API 开始。内存策略可以用 `numactl--membind`、`numactl--interleave` 和 `first-touch` 对照观察。工具只是入口，真正的问题仍然是：线程在哪，页面在哪，谁高频写，合并怎样跨越拓扑。

源码阅读从窄问题进入 不要试图一次读懂完整调度器或完整内存管理。更稳的入口是带着一个实验问题追踪对象：线程迁移时哪些 mask 和调度域参与；`first-touch` 触发缺页后，VMA 策略怎样影响页面分配；自动 NUMA balancing 怎样采样访问并迁移页面；`per-cpu` 变量和 slab 本地缓存怎样减少共享写。

可以从 `kernel/sched/topology.c` 理解调度域和 CPU 层级，从 `mm/mempolicy.c` 理解 NUMA 策略，从 `page fault` 和页面分配路径理解 `first-touch`，从 `per-cpu` 相关代码理解“把共享状态拆到局部”的内核版本。阅读报告不需要复制所有函数名，应回答一个具体问题：某个用户态实验现象能否映射到内核对象和状态变化。

证据等级 报告可以把证据分成三层。第一层是直接测到的数字：耗时、吞吐、线程数、绑定、合并时间、计数器事件。第二层是从源码或硬件模型推导出的解释：某条 line 可能发生所有权迁移，某个页面可能在 `first-touch` 时落到 node 0。第三层是当前设备无法验证的合理假设：没有 uncore 计数器时，不能写互连字节，只能写结果符合或不符合某个假设。

这不是形式主义。NUMA 和一致性问题很容易被机器差异放大。把证据等级写清，别人才能知道哪些结论可复现，哪些需要换机器继续验证。

4.11 PMU 事件怎样映射到硬件结构

一致性和 NUMA 很容易被写成“感觉像远端访问”。要把它变成工程证据，必须把候选瓶颈映射到可观察指标。PMU 事件不是魔法真相；它们只是硬件在某些结构上计数。读报告时要问：这个事件对应哪个结构，能证明什么，不能证明什么。

候选瓶颈	可能观察的指标	不能直接证明的事
共享写热点	CAS/locked 指令次数、cache-to-cache、LLC miss、扩展曲线	不能只凭一个 cache miss 断言 false sharing
远端 NUMA	remote node 访问、numastat、uncore 带宽、绑定对照	不能在无拓扑记录时断言远端
false sharing	对齐前后吞吐、c2c line、写者字段布局	不能只看 CPU 利用率
互连饱和	socket 间带宽、LLC miss、remote HITM、线程数扫描	普通 <code>perfstat</code> 常不够
内存带宽	sustained bandwidth、LLC miss、bytes/op 账本	不能解释所有延迟长尾

不同平台事件名差异很大。Intel、AMD、ARM、虚拟机和受限设备暴露的 PMU 不同，容器权限也会限制访问。因此书中不能把某个事件名当成唯一标准。更稳的写法是先给硬件结构假设，再列出当前环境可用证据和降级证据。

从现象到计数器的三步 第一步，先用结构性对照证明方向。例如全局 `atomic` 随线程数增加吞吐下降，`worker partial` 改善，`aligned partial` 再改善，这比单独一个 PMU 事件更强。第二步，再用可用计数器补证据，例如 `cache miss`、`context switches`、`locked` 指令、`remote access` 或 `c2c`。第三步，写出不可验证部分：若没有 `uncore` 权限，就不能写“互连带宽已饱和”，只能写“扩展曲线和布局对照支持共享写解释”。

Listing 4.28 NUMA/coherence 报告的指标降级路径

```
best case:
  perf c2c + remote HITM + numastat + topology + timing

common server case:
  perf stat + numactl binding + aligned/unaligned variants + timing

restricted case:
  wall time + thread sweep + layout variants + checksum + topology notes
```

把事件放回代码位置 计数器是进程或线程级聚合时，必须再用阶段时间或 `profile` 把它放回代码区域。若全程序 LLC miss 上升，不一定是 hot counter；可能是输入扫描、日志输出或合并阶段。最小报告要拆热循环、合并、I/O 和调度。若能用 `perfrecord` 或应用 `trace` 标出热点函数，再把事件和字段布局对上，证据才闭合。

MESI 状态不是 PMU 输出 很多报告会写“这里发生了 MESI 从 S 到 M”。这通常是推断，不是直接观察。PMU 可能提供的是某类 `miss`、`snoop`、`remote HITM` 或 `cache-to-cache` 事件；从这些事件到 MESI 状态迁移，中间还隔着架构实现和事件定义。严谨写法是：“该 line 是多写者热点；对齐和 `partial` 对照改善吞吐；`c2c` 显示该 line 上存在大量远端修改命中；因此共享写所有权迁移是强解释。”不要把推断写成硬件日志。

4.12 NUMA 的定量模型：远端不是慢一点这么简单

NUMA 分析至少有两个量：延迟和带宽。远端 load 的延迟更高，跨 socket 带宽也可能更低；更重要的是，远端访问会占用互连，影响其他核心。一个线程偶尔

读远端配置几乎无害；所有 worker 在热循环里随机访问远端大表，就会让互连成为共享瓶颈。

可以先写一个粗模型：

$$T \geq \max \left(\frac{B_{local}}{BW_{local}}, \frac{B_{remote}}{BW_{remote}}, \frac{B_{interconnect}}{BW_{interconnect}} \right) + N_{remote} \cdot L_{extra}$$

这里 `B_local` 是本地节点字节流量，`B_remote` 是远端节点字节流量，`B_interconnect` 是跨节点互连承载的字节，`N_remote` 是对延迟敏感的远端访问次数，`L_extra` 是远端相对本地的额外延迟。这个公式不追求精确预测，它的作用是防止一句“NUMA 慢”掩盖不同原因。

工作负载形态	主要量	更可能的优化
大块顺序扫描远端内存	远端带宽、互连带宽	first-touch、membind、分片扫描
随机指针追踪远端对象	额外延迟、TLB/cache miss	数据局部化、压缩对象图、批量化
热共享写跨 socket	所有权迁移、remote HITM	per-node partial、分层合并
低频配置读取	几乎不主导	保持简单，避免过度分片

first-touch 是页面放置算法的一部分 Linux 常见策略下，匿名页面可能在第一次写入时分配到执行该写入的 CPU 所属节点。若主线程统一初始化所有 worker buffer，页面可能集中在主线程节点；随后其他节点 worker 访问就变成远端。把初始化分给各 worker，不只是预热 cache，而是在控制物理页面归属。报告必须把“分配”“初始化”“计算”分成不同阶段，否则无法解释页面放置。

分层 partial 的成本账本 per-thread partial 消除了线程间热共享写，但最终合并仍要读所有 partial。如果 partial 很大，合并可能变成内存带宽路径；如果跨 socket 合并集中到一个线程，可能变成远端读路径。因此分层 partial 要按拓扑合并：先本 core 或本 NUMA node 合并，再跨节点合并。优化目标不是让所有通信消失，而是把高频细粒度通信变成低频粗粒度通信。

4.13 案例：日志统计的拓扑化改造

把前四章连起来看，一个日志统计程序的瓶颈会沿着硬件路径迁移。第二章先看热循环：解析器的分支、指令足迹和依赖链是否让单核心前端或乱序窗口受限。第三章看地址：热字段是否连续，cache line 里有效字节比例如何，DTLB 是否被随机对象图击穿。第四章加入多核心：热计数器是否共享写，partial 是否 false sharing，页面是否在使用者附近，合并是否跨节点集中。

第一步：固定语义 reference 优化前先有 reference。输入记录、错误判断、事件类型统计、输出顺序和 checksum 必须固定。后续把全局计数器拆成 partial、把合并变成树形、把队列分片，都不能改变语义。若浮点或顺序敏感结果会变化，必须先写合同，而不是用性能改动掩盖差异。

第二步：把热写移出主循环 朴素版本每条记录更新全局 `processed_count` 和 `error_count`。改造后，每个 worker 在局部变量里累加，批次结束写入自己的 partial。若需要运行中进度，可以让监控线程低频汇总 partial，并明确这只是近似快照。这样热路径不再为每条记录争同一条 line。

第三步：按写者设计布局 WorkerStats 不应把所有字段随意放在一个结构体数组里。每个 worker 高频写的计数器应与其他 worker 隔离；只读配置可以紧凑；低频诊断字段可以单独存放；合并线程读的字段要考虑扫描成本。布局不是按对象归属决定，而是按写者、频率和阶段决定。

第四步：把页面初始化纳入算法 输入 buffer、partial 表、临时聚合表和输出 block 都有页面归属。大数组若由主线程统一清零，后续 worker 可能远端访问。更稳的做法是按 worker 或 group 初始化自己将要使用的范围。初始化耗时可以单独计入，也可以作为预处理阶段报告；不能既让初始化影响页面，又在报告里假装它不存在。

第五步：分层合并并记录成本 每个 worker 先生成本地统计；同 group worker 先合并成本组统计；最后合并全局结果。报告拆分热循环、组内合并、跨组合并和写出。若跨组合并成为长尾，可以改成树形；若组内负载不均，要调整分块或窃取策略；若输入很小，回到更简单版本。拓扑化改造不是单向加复杂度，而是在证据下选择合适层级。

第六步：把 trace 写成稳定接口 任务运行时、工作窃取、原子内存序、lock-free 结构、并行 reduce 和分布式 shuffle 都需要这些事实。PlacementHint、WorkerTopology、PlacementTrace 和分层 partial 不是孤立设计，而是用来回答“任务在哪里、数据在哪里、结果怎样提交”。单机里的远端 group 到集群里会变成远端机器；本地 partial 到 shuffle 里会变成 map-side combine。

4.14 展开：MESI 状态机和 false sharing 的证据边界

一致性协议的细节随处理器不同而不同，但工程分析需要一个足够准确的状态机。以常见 MESI 直觉为例，一条 cache line 可以处在 Modified、Exclusive、Shared、Invalid 等状态。软件不直接控制这些状态，却通过读写模式制造状态迁移。

状态	直觉含义	软件常见来源
M	本核心有最新副本，内存可能旧	某线程独占写热字段
E	本核心独占干净副本	只被一个核心读过的 line
S	多核心共享干净副本	多线程读同一只读表
I	本核心副本无效	其他核心写入后被失效

读 miss 可能把 line 从内存或其他 cache 带到本核心；写 miss 或写共享 line 需要拿到所有权，并让其他副本失效。false sharing 的本质是：两个线程写不同字段，但字段落在同一 cache line 上，硬件只能以 line 为单位迁移所有权。

Listing 4.29 两个核心写同一 line 的状态直觉

```
Core 0 writes field a in line L:
  Core 0 needs L in Modified
  Core 1 copy of L becomes Invalid

Core 1 writes field b in the same line L:
  Core 1 needs L in Modified
  Core 0 copy of L becomes Invalid

repeat:
  ownership of L bounces even though a and b are different fields
```

这就是为什么 padding 能改善某些场景：它不是让写入变少，而是让不同写者落到不同 line，减少所有权迁移。但 padding 不能改善同一线程的大量内存读，不能降低算法字节量，也不能解决负载不均。

HITM、c2c 和“强解释”

若平台能提供 HITM 或 cache-to-cache 证据，它们可以加强共享写解释。HITM 的直觉是：一个核心请求的数据在另一个核心的 modified line 中，说明数据在 cache 间转移，而不是简单从内存读取。但事件名、精度和采样能力因平台而异，报告必须写清工具和事件定义。

证据	支持的结论	仍不能证明
padding 后 吞吐 提升	line 级共享写是候选瓶颈	一定是唯一瓶颈
c2c 指向同一地址	某 line 发生 cache 间竞争	源码层哪个字段负责，需结合布局
remote HITM 增加 线程扩展曲线崩坏	跨 socket 所有权迁移严重 共享资源随线程数恶化	具体 MESI 迁移细节 不区分锁、atomic、内存带宽

严谨措辞应是“证据支持共享写所有权迁移”，而不是“我证明了 MESI 状态从 S 到 M”。硬件协议是实现细节；软件报告要停在可验证事实和合理推断之间。

NUMA 页面迁移和 first-touch 的时间线

first-touch 不是一句“谁先摸页面归谁”就结束。真实程序里常有分配、清零、初始化、计算、回收多个阶段。若主线程分配并 `memset` 大数组，页面可能落在主线程所在 NUMA 节点；随后 worker 绑定到其他节点访问，就产生远端流量。若自动 NUMA balancing 打开，内核还可能根据采样迁移页面，导致中途性能变化。

Listing 4.30 页面放置时间线

```
allocate virtual range:
  no physical page may be allocated yet

main thread memset:
  page fault allocates physical pages near main thread

workers run on other nodes:
  remote reads/writes appear

kernel balancing may sample:
  migrate pages if policy and access pattern allow
```

因此 NUMA benchmark 必须记录阶段。只报总耗时，看不到页面到底在谁手里；只绑定计算线程，不控制初始化线程，结论也不稳。更好的报告拆成 allocation、touch/init、compute、merge，并记录每阶段的 CPU/node。

定量账本：一次远端扫描到底多贵

假设每个 worker 扫描 1GiB 数据，本地可持续带宽 100GiB/s，远端可持续带宽 50GiB/s，跨 socket 互连可持续 80GiB/s。如果 4 个远端 worker 同时扫描，总远端需求是 4GiB，单看远端内存下限是：

$$T_{remote_mem} \geq \frac{4 \text{ GiB}}{50 \text{ GiB/s}} = 80 \text{ ms}$$

单看互连下限是：

$$T_{link} \geq \frac{4 \text{ GiB}}{80 \text{ GiB/s}} = 50 \text{ ms}$$

本地扫描若有足够带宽，下限可能是：

$$T_{local} \geq \frac{4 \text{ GiB}}{100 \text{ GiB/s}} = 40 \text{ ms}$$

这些数字不是预测器，只是 sanity check。若实测远端扫描 400ms，要继续查 TLB、随机访问、线程迁移、合并、频率和 page placement。若实测 10ms，则说明账本口径错了：可能数据没有真正读取、cache 已热、输入规模不是 GiB，或者计时范围不包含完整工作。

核心想

NUMA 优化的第一步不是写 numactl，而是把字节、页面、线程位置和阶段时间写成账本。账本不精确也比没有账本强，因为它能排除数量级错误。

4.15 状态机和事故：MESI 直觉如何变成生产诊断

把一致性协议写成完整硬件实现会很复杂，但软件工程需要的最小状态机并不大。对一条 cache line，读者至少要能跟踪四种抽象状态：

状态	软件直觉	性能含义
Invalid	本核心没有可用副本	下次访问要向外请求
Shared	多核心可读，当前核心不能直接写	读可扩展，写要升级权限
Exclusive	只有当前核心有干净副本	写可本地转 Modified
Modified	当前核心有最新且已修改副本	其他核心读写要经过当前拥有者或目录

一次只读共享通常是 Invalid->Shared；一次私有写可能是 Invalid->Exclusive->Modified；一次共享写热点则反复在不同核心之间迁移 Modified 所有权。这个状态机能解释两个看起来矛盾的事实：不可变大权重可以被许多核心同时读而扩展不错；一个 8 字节 atomic 计数器却能让 32 个核心排队。

工程事故：指标系统拖垮热路径

某个日志服务上线后，CPU 使用率升高但吞吐下降。火焰图显示业务函数仍在热路径，perfstat 显示 cycles 增加，context switch 不高，锁等待也不明显。代码审查发现每处理一条记录都会更新四个全局 atomic 指标。指标系统本来是为了可观测性，却把每条记录变成跨核心共享写。

正确复盘不能只写“atomic 慢”。要按状态机写证据：

Listing 4.31 共享写事故复盘字段

```
symptom:
  throughput drops after metrics patch
  CPU busy but useful records/s lower

code evidence:
  hot loop updates global atomic counters per record
  counters share one cache line or one shard

model evidence:
  every update requires writable ownership of line L
  multiple cores rotate ownership

experiment:
```

```
disable metrics
per-worker metrics
compact partial vs alignas(64) partial
hot-key input vs uniform input

fix:
worker-local counters
batch export every N milliseconds
control-plane snapshot accepts approximate value
```

这类事故的教训是：可观测性字段也属于数据布局。指标、日志、trace id、引用计数和状态位若进入每条记录热路径，就要像业务数据一样做 cache line 和写者分析。

目录协议和 HITM：证据边界

多 socket 机器常用目录或类似机制追踪哪条 line 在哪里。软件通常看不到完整协议状态，只能通过替代证据推断，例如 remote HITM、c2c 报告、uncore 计数器、线程绑定对照、跨 socket 和同 socket 对照。报告中的严谨措辞应是“证据支持 line 所有权跨 socket 迁移”，而不是“我证明了某条 line 一定处于 M 状态”。

证据	支持的解释	不能单独证明
per-worker padding 后改善	false sharing 或共享写参与	具体 MESI 转移
同 socket 快于跨 socket	远端路径或互连参与	所有慢都来自 NUMA
remote HITM 上升	远端核心持有修改 line	C++ 层语义正确
global atomic 扩展差	共享写热点参与	memory order 选择就是根因

实验：四种布局的扩展曲线

习题与作业

实验一：实现四个版本的计数器：单全局 atomic、紧凑 per-worker partial、alignas(64) per-worker partial、per-socket partial。输入包含均匀 key 和 90% 热点 key。记录 1、2、4、8、16 线程吞吐、checksum、合并时间和可用 PMU/c2c 指标。

实验二：在支持 NUMA 的机器上分别测试主线程初始化和 worker first-touch。记录页面放置证据、绑定策略和远端访问替代指标。若当前实验环境没有 NUMA，报告必须写清“无法验证远端路径”，并保留单 socket false sharing 对照。

实验三：把指标导出从每条记录更新改为每批次合并。比较吞吐、指标新鲜度和监控读取成本，说明性能收益换来了哪种可观测性延迟。

4.16 常见误区和边界

误区一：atomic 一定比锁快 Atomic 的路径短，不代表适合高频共享写。低频状态位、任务完成标志、引用计数和线性化点可能适合 atomic；每条记录更新一次的全局计数器通常不适合。锁也不是一概错误：低频复杂不变量用 mutex 可能更清楚、更快。选择同步原语前，先判断热路径频率和共享范围。

误区二：padding 越多越好 Padding 只解决特定 false sharing，不解决内存带宽、TLB、合并长尾或负载不均。过度 padding 会扩大对象，降低缓存容量利用

率，增加合并扫描字节。只隔离不同线程高频写的字段；对只读或低频字段保持紧凑。

误区三：绑定线程一定提升性能 绑定能稳定位置，也可能破坏调度灵活性。移动设备上，固定大核可能触发降频；服务器上，错误绑定可能把 worker 集中到同一 LLC 或同一内存节点；容器中，绑定可能和 cpuset 冲突。绑定是实验变量和部署策略，不是无条件优化。

误区四：NUMA 优化永远值得 若输入很小，数据在 LLC 内，NUMA 策略可能没有收益；若任务高度不均，本地优先可能让 worker 空闲；若只有单节点机器，复杂分层只会增加代码；若热路径很快压缩成小 partial，远端访问时间占比可能很低。先测远端成本和共享写成本，再引入拓扑策略。

误区五：工具数字可以脱离模型 计数器事件、cache miss、LLC miss、remote access 和 c2c 报告都需要上下文。一次 miss 可能有用，因为它带回后续许多字节；一次远端访问可能无害，因为发生在低频合并；一个 atomic 事件可能很贵，因为发生在每条记录。工具提供证据，不替代因果链。

4.17 习题

习题与作业

习题一：画出全局 atomic 计数器、紧凑 worker partial、按 cache line 隔离 worker partial、按 NUMA group 分层 partial 四种布局。分别标出哪些 line 被谁写，哪些阶段需要合并，读取全局值的成本在哪里。

习题二：设计一个 first-touch 实验。要求写清分配、初始化、绑定、计算、合并五个阶段；记录输入规模、页面大小、worker 到 CPU 的映射和不可验证边界。若当前设备没有多个 NUMA 节点，写出可运行替代实验。

习题三：为线程池设计本地队列加工作窃取方案。说明 worker 本地 pop、同 group steal、跨 group steal、全局溢出队列和关闭协议分别保护哪些状态，哪些字段可能成为共享写热点。

习题四：给 WorkerStats 设计两种布局：紧凑布局和隔离高频写布局。计算结构体大小、对齐、每条 cache line 可能容纳几个 worker 的字段，并说明何时不应使用隔离布局。

习题五：阅读 Linux 调度拓扑、NUMA 策略或 page fault 路径的一个窄入口。报告必须从一个用户态实验问题出发，例如“为什么主线程初始化会影响页面节点”或“为什么线程迁移会破坏 first-touch 假设”，不允许只罗列函数名。

硬件侧因果链已经足够提出候选解释：指令怎样供给，数据怎样到达核心，页面怎样翻译和放置，多核心怎样协调共享写。下一步必须把候选解释变成可复查证据：哪一段耗时，哪一类资源饱和，哪个对照实验能排除错误假设。

工程案例

知识点小案例：四个线程递增一个计数器为什么扩展性变差。特例很小：四个 worker 每处理一条记录都执行一次全局 `atomic_fetch_add`。内存序即使是 relaxed，目标 cache line 仍然要在多个核心之间转移所有权；如果线程跨 NUMA 节点，转移路径更长。把它改成 worker-local partial 后，热路径只写本地字段，最后再合并，计数语义没变，写共享线的频率却大幅下降。由此推出规律：NUMA 和一致性不是只有大对象才相关，单个热点 cache line 也能成为全局瓶颈。工程上先画谁写哪条 line，再决定全局 atomic、per-worker partial 还是 per-NUMA 合并。

第 5 章

可信测量、Roofline 与性能计数器

硬件知识必须落到证据。前四章已经把一条执行路径拆成取指、译码、load/store、cache line、TLB、页面放置、互连和一致性所有权。现在换一个问题：当一次运行显示“快了 30%”，这个数字到底来自哪里？可能是热循环真的少等了内存；也可能是输入已经进入 page cache，首次触页被排除，线程少了一次迁移，CPU 频率更高，错误路径被跳过，或者优化版少做了某个提交动作。性能测量不是把秒表放在程序外面，而是把“同一个语义、同一个输入、同一个机器状态、同一个计时边界”固定下来，再解释剩下的差异。

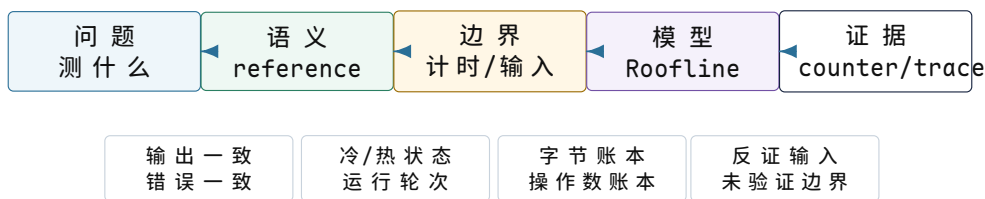
先看一个常见争论。Compute Systems Engine 的归约热路径被改成了多累加器版本，有人测出三倍加速，有人测出几乎没提升。两个人可能都没有撒谎，只是测量对象不同：第一个人测的是 L1/L2 能容纳的小数组，瓶颈主要是累加依赖；第二个人测的是远超 LLC 的大数组，瓶颈已经变成内存带宽。再换一个环境，第三个人把输入生成、首次触页、线程创建和输出校验都放进计时区，测到的又是端到端流程。没有测量边界，性能数字会变成口水仗。

可信测量的目的不是让报告变厚，而是让每一个性能结论都有可复查的证据链：被测问题是什么，正确性怎样固定，输入和机器状态是什么，计时范围在哪里，模型怎样约束可能上限，计数器或 trace 支持哪种解释，哪些结论还不能说。数据布局、SIMD、线程、锁、原子、运行时、I/O 和分布式章节，都会复用这条证据链。

工程案例

核心案例是“归约和日志统计为什么测出相互矛盾的数字”。同一个项目至少要同时保留三类测量：端到端 run，包括读取、解析、聚合、写出和提交；阶段测量，分别记录 parse、filter、histogram、partition、reduce 和 manifest commit；热内核测量，只测某个 loop 或 kernel 的吞吐。三类数字都重要，但回答的问题不同。

性能结论必须经过的证据链



一个耗时数字只有穿过语义、边界、模型和证据四层，才有资格成为工程结论。

图示：本图要证明的命题是：性能测量不是跑分截图，而是从问题到证据的论证链。

5.1 先固定语义，再比较速度

性能测量的第一条纪律是：先证明两个版本解决的是同一个问题。优化版可能跳过错误行、忽略尾部、改变浮点合并顺序、提前丢弃 top-k 候选、少写 manifest、没有等待持久化，都会得到漂亮时间。这样的数字不是优化结果，而是语义退化的症状。

Reference 是性能实验的锚点 reference（参考实现）指一段尽量简单、可信、

可复查的实现。它可以慢，但要把语义讲清。归约的 `reference` 是顺序求和；`stable filter` 的 `reference` 是串行保序输出；日志统计的 `reference` 是顺序解析、错误分类和最终计数；`shuffle` 的 `reference` 则要包含每个 `key` 的最终归属和 `block` 校验。没有 `reference`，`benchmark` 只是在比较两个黑盒。

`Reference` 不等于 `oracle`。`oracle`（判定器）是比较 `reference` 和候选版本结果的规则。整数计数通常要求精确一致；浮点归约可能允许误差，但必须写绝对误差、相对误差或固定合并树；`top-k` 必须写 `tie-break`；`partition` 必须写 `bucket` 范围、路由函数和 `checksum`；I/O 提交必须写哪些文件或 `manifest` 条目算作已生效。`Reference` 生成正确结果，`oracle` 判断候选结果是否可接受。

每个样本都要带正确性状态 性能样本不能只保存时间。每一轮运行都应返回 `correct`、`checksum`、`records_in`、`records_out`、`error_count` 和关键不变量。校验失败的样本不能混进性能统计后继续谈加速。并行程序尤其要这样做：某些数据竞争只在高线程数、特定输入或特定调度下出现；如果错误样本被当成最快样本，报告会奖励 `bug`。

Listing 5.1 `BenchmarkSample` 的最小形态

```
1 enum class SampleStatus {
2     Passed,
3     Failed,
4     Skipped
5 };
6
7 struct BenchmarkSample {
8     std::string case_name;
9     std::string variant_name;
10    std::uint64_t input_bytes = 0;
11    std::uint64_t records_in = 0;
12    std::uint64_t records_out = 0;
13    std::uint64_t checksum = 0;
14    std::chrono::nanoseconds elapsed{};
15    SampleStatus status = SampleStatus::Failed;
16    std::string note;
17 };
```

这个结构故意把 `Skipped` 写进状态。某些实验依赖 `NUMA`、`PMU`、特定 `SIMD` 或文件系统能力，当前设备不支持时，应记录跳过原因，而不是静默消失。受限环境中的诚实边界，是系统工程能力的一部分。

防止编译器优化掉 `benchmark` 优化器会删除不可观察的计算。若循环求和结果没有被使用，编译器可能删掉整个循环；若输入是编译期常量，计算可能被常量折叠。防优化的方法包括让结果参与 `checksum`、返回给调用者、写入专用 `sink`，或使用成熟 `benchmark` 框架的接口。但防优化不能污染测量对象：在热循环中打印每个元素，测到的是 I/O；每轮写全局原子 `sink`，可能制造新的共享写瓶颈。

热循环只做被测工作，循环外做轻量校验。端到端实验可以包含输出和提交，但要明确命名为端到端，不要再把它解释成单个 `kernel` 的吞吐。

语义边界来自状态机 很多“优化后更快”的争议，本质是状态机边界改变了。日志统计不是只把整数加起来，还包括输入记录是否被接受、错误行如何分类、输出 `block` 何时可见、`manifest` 是否提交、失败后能否恢复。若旧版本等待 `manifest` 落盘，新版本只写内存缓冲，二者不是同一个状态机。若旧版本 `stable filter` 保持输入顺序，新版本允许乱序输出，二者的 `oracle` 就不同。性能样本必须携带状态，而不是只有耗时。

Listing 5.2 一次运行至少要固定的语义状态

```
input identity and seed
reference output identity
candidate output identity
checksum and error summary
commit or visibility boundary
accepted oracle and tolerance
```

5.2 计时边界：端到端、阶段和热循环

同一个程序可以有三种合法时间。端到端时间包含用户真实承受的全流程：读取、解析、计算、写出、提交、恢复检查。阶段时间把流程拆成 parse、filter、histogram、partition、reduce、write、commit。热循环时间只测一个明确 kernel。三者都重要，但不能互相替代。

计时器测的是时间，不自动测原因 计时器读数来自某个时钟源，例如单调时钟或硬件计数器。它能告诉两个时间点之间经过了多久，却不会说明线程是在 CPU 上执行、等待内存、睡眠、被抢占、等待 I/O，还是被 cgroup 节流。热循环里读计时器还会带来额外指令和序列化成本；多线程中，不同核心的时间戳、频率状态和调度迁移也会引入解释边界。因此计时范围要足够大，重复次数要足够多，并且要记录线程位置、频率和系统限制的可观察信息。

这不是追求完美测量，而是避免把计时器当成因果解释。秒表只能说“这段时间变了”，不能直接说“cache 变好了”或“SIMD 生效了”。后者需要阶段拆分、字节账本、反汇编、计数器、trace 或对照实验。

端到端时间回答“系统是否真的变快” 如果一个解析热循环快了两倍，但端到端只快了 5%，说明瓶颈可能迁移到 I/O、队列、写出或提交。局部优化仍有价值，但下一步不能继续盯着解析器。端到端时间是用户视角和系统视角，适合判断优化是否值得进入默认路径。

端到端时间也会包含系统慢路径。第一次读取文件可能触发 page cache miss，第一次触碰数组可能触发 minor fault，第一次写 checkpoint 可能暴露 fsync 成本。这些成本不是噪声，而是端到端场景的一部分。关键是报告要写清“这是冷启动端到端”，不要把它拿去解释热循环指令效率。

冷状态和热状态是两种输入 同一份数据在冷 page cache 和热 page cache 下不是同一个测量状态。第一次触碰匿名页会触发缺页和物理页分配，第二次扫描可能只是在缓存和内存之间移动数据；第一次打开大量文件会走目录和元数据路径，后续可能命中内核缓存。若目标是用户首次运行，冷状态就是被测对象；若目标是服务稳态，热状态才接近问题。报告要把冷/热状态写进输入合同，而不是在结果不好看时把冷启动称为噪声。

阶段时间回答“成本迁移到哪里” 阶段时间要按语义边界切，不要只按函数名切。Compute Systems Engine 可以至少分成：input、parse、filter、local histogram、partition、merge reduce、top-k、write、manifest commit。每个阶段记录 records in/out、bytes in/out、temporary bytes、state bytes、error count、retry count 和 checksum。这样某次优化后，即使总时间不变，也能看到成本是否从 parse 转移到 write，或从 local aggregate 转移到 merge。

Listing 5.3 阶段报告的最小字段

```
1 struct StageReport {
2     std::string stage_name;
3     std::uint64_t records_in = 0;
4     std::uint64_t records_out = 0;
```



```

5  std::uint64_t bytes_in = 0;
6  std::uint64_t bytes_out = 0;
7  std::uint64_t temporary_bytes = 0;
8  std::uint64_t state_bytes = 0;
9  std::uint64_t checksum = 0;
10 std::chrono::nanoseconds elapsed{};
11 };

```

这不是完整观测系统，却能防止最常见的误判：局部内核变快，但下游队列积压；过滤更激进，导致输出少了；partition 写得更快，却没有提交 manifest；top-k 候选减少，漏掉全局强 key。阶段报告把性能和语义绑在一起。

热循环时间回答“机器等待在哪里” 热循环测量适合分析核心、cache、TLB、分支、SIMD 和端口。它要求输入已经准备好，输出预分配，校验在计时外，重复次数足够多。热循环结果不能直接代表端到端系统，但能解释某个 kernel 的瓶颈。HotKernelReport、AccessTrace 和 BenchmarkCase 应互相引用。

常见误区

不要用热循环时间证明端到端收益，也不要端到端时间证明某个指令级优化有效。先命名测量对象，再解释数字。

5.3 输入矩阵和统计方法

一次运行不是证据。一个默认输入也不是 workload。可信测量需要输入矩阵、重复运行、统计摘要和异常解释。输入矩阵不是为了跑很多数据，而是为了攻击假设：小工作集、大工作集、随机访问、热点 key、错误密集、短行、长行、冷读、热读、多线程、过度订阅，每一类输入都能暴露不同瓶颈。

输入要能证伪假设 怀疑分支预测，就准备可预测和不可预测输入：全真、全假、排序、随机。怀疑 cache/TLB，就准备连续、固定步长、随机索引和指针追踪。怀疑 false sharing，就比较紧凑 partial 和 cache-line 对齐 partial。怀疑 NUMA，就比较主线程初始化和 worker first-touch。怀疑 I/O，就比较冷读、热读和 page cache 污染状态。没有反证输入，性能报告很容易只讲喜欢的故事。

运行顺序和预热 预热不是作弊，而是定义状态。若测量对象是冷启动，第一轮就是被测对象；若测量对象是稳态吞吐，应在正式计时前预热。运行顺序也会影响结果：固定先跑 baseline 再跑 optimized，可能让后者总是享受热 cache 或更高温度。更稳妥的做法是保存原始样本，必要时随机化版本顺序，记录每轮状态。

机器状态也是实验输入 CPU 频率、温度、SMT、后台任务、容器配额、内存压力、透明大页、自动 NUMA balancing、page cache 状态和 I/O 写回都会改变结果。它们不是“环境噪声”四个字可以抹掉的东西，而是实验输入的一部分。无法完全控制时，也要记录可观察部分：可见 CPU、绑定策略、是否充电、温度风险、系统负载、是否发生 page fault、是否出现 context switch 或 throttling。受限设备上的结论必须保守。

频率和温度会改变每秒能做什么事 现代 CPU 不以一个固定频率运行。负载、温度、功耗、电源策略、核心数量和指令类型都会影响频率。单线程短 benchmark 可能得到较高 boost；多线程长时间运行可能降低频率；受限设备、笔记本或共享环境在温度上升后可能限频；使用宽 SIMD 或高功耗指令时，某些平台会降低频率。若一个版本运行时间更短，它可能一直处在更高频率窗口；另一个版本较慢，可能被热限制拖入低频。

这并不意味着性能测量无法进行，而是报告要记录持续时间、轮次、是否长时间预热、是否充电、是否存在温度风险，以及能否读取频率。对比短热循环时，不应只跑一次最快样本；对比端到端长任务时，要让运行时间足够覆盖稳态。若频率不可观测，结论应写窄，避免把频率差异误写成 cache 或 SIMD 改善。

SMT 会让两个线程共享同一个核心资源 SMT 或超线程让一个物理核心同时运行多个硬件线程。它能在一个线程等待时利用空闲资源，但两个线程会共享前端、执行端口、L1/L2、TLB 和带宽。内存等待型任务可能受益，前端或端口已经饱和的任务可能互相干扰。把 worker 数从物理核心数增加到逻辑线程数，吞吐可能提升、停滞或下降。

因此线程数矩阵要标出物理核心和逻辑线程的边界。若 8 物理核心、16 逻辑线程的机器在 8 到 16 线程没有提升，不必立即寻找锁；可能只是核心内部资源已经满。若 16 线程反而下降，再看共享写、缓存容量、调度和频率。报告写“worker=16”不够，要写可见拓扑或至少写 `hardware_concurrency` 之外的实际绑定策略。

调度迁移和抢占会改变样本尾部 一个线程在运行中被迁移到另一个核心，会带来 cache/TLB 热状态丢失，也可能改变 NUMA 访问距离。被后台任务抢占，会让 wall time 增加但 CPU 执行时间不一定增加。服务型路径的 P95/P99 很容易受这些事件影响。若工具可用，应记录 context switches、CPU migrations、cgroup throttling；若不可用，应用 trace 至少记录任务开始结束、worker id 和长尾样本。

绑定线程能减少迁移变量，但绑定也会带来新的风险：错误绑定到同一核心、忽略系统保留核心、触发温度问题或和容器 cgroup 冲突。绑定是实验变量，必须写进报告。

page cache 和 writeback 会污染 I/O 结论 读取同一文件第二次快，不一定是解析器优化；写文件后一段时间突然变慢，也可能是脏页回写压力。Page cache、readahead、dirty page、writeback、fsync 和存储设备队列都会进入端到端时间。测 parser 热循环时应把输入放在内存中并说明状态；测真实文件读取时要区分冷读、热读和污染状态；测可靠写时要把 write、fsync、rename 和 manifest commit 分开。

如果当前环境不能清理系统 page cache，不要写“冷读已验证”。可以采用替代对照：改变输入文件名和内容，读取大于内存的污染文件，记录 major/minor faults，或者只声明热读结果。诚实边界比不可复现的冷读数字更有价值。

受限环境下的最小机器记录 普通用户、容器、虚拟机和云环境常常不能读取完整 PMU、NUMA、频率或系统缓存状态。最小机器记录仍应包含：操作系统、CPU 型号或可见架构、可见 CPU 数、内存大小或限制、编译器和选项、是否容器、是否能设置 affinity、是否能使用 `perfstat`、输入是否在文件系统上、是否存在温度或配额风险。缺失项写 unavailable。

Listing 5.4 受限环境的机器记录模板

```
machine:
  os, kernel, architecture, cpu_model_if_available
  visible_cpus, affinity_policy, smt_unknown_or_enabled
  memory_limit_if_available, container_or_native
  compiler, flags, build_type
  perf_status, numa_status, frequency_status
  storage_path, page_cache_control_status
  thermal_or_power_notes
```

这份模板不会让实验变完美，但能阻止最严重的误读：把权限不可用当成事件为零，把容器限制当成硬件上限，把热 page cache 当成解析优化，把单次 boost 当成稳定吞吐。

测量本身会改变系统 测量不是透明观察。插入计时器会增加指令和分支；打印日志会改变 I/O 和锁；采样 profiler 会产生中断；PMU multiplex 会降低事件精度；过细的 trace 会污染 cache；在热循环里读取高精度时钟可能比被测操作还贵。测量设计要把观察成本写出来，并尽量让观察发生在阶段边界而不是每个元素

内部。

Listing 5.5 测量扰动的常见来源

```
timer in inner loop:
  adds instructions, serializing effects, and branch noise

logging in hot path:
  adds formatting, allocation, locks, and I/O

sampling profiler:
  interrupts execution and observes only sampled locations

full trace:
  changes cache behavior and may create synchronization

PMU multiplex:
  estimates events when hardware counters are insufficient
```

这不是说不要测，而是要分层测。热循环用外层计时包住大量重复；阶段级 trace 记录开始/结束和少量统计；详细 trace 只在小输入或采样模式开启；日志不要进入主循环。若为了定位罕见错误必须记录每条事件，就要承认这已经不是原始性能实验，而是诊断模式。

计时器也有语义边界 墙钟时间包含 CPU 执行、被抢占、等待 I/O、等待锁、page fault 和系统干扰；CPU time 更接近线程实际运行时间，却可能漏掉阻塞等待；周期计数受频率、迁移和权限影响；硬件时间戳在不同核心之间的同步也有平台条件。选择计时器前要先问测量对象是什么：热循环占用核心多久，还是用户请求等了多久；批处理实际完成多久，还是某个线程实际跑了多久。

时间口径	适合回答	容易误读
Wall time	用户或批处理实际等待时间	把抢占、I/O 和后台干扰误写成计算成本
Thread CPU time	线程真正占用 CPU 的时间	看不到睡眠、I/O 和队列等待
Process CPU time	进程总体 CPU 消耗	多线程时可能大于 wall time，不能直接当延迟
Cycle counter	核心周期级观察	受频率、迁移、权限和序列化影响
Stage timestamp	队列和流水线顺序	需要处理时钟来源和 trace 扰动

Compute Systems Engine 应同时保留 wall time 和阶段时间；在能获取时再记录 CPU time、cycles 和 PMU。若 worker 阻塞在 I/O，wall time 上升但 CPU time 不一定上升；若热循环被抢占，wall time 样本变长，CPU time 可能保持；若原子争用忙等，wall time 和 CPU time 都上升。不同口径合起来才有诊断力。

重复运行不是简单求平均 同一程序的多次运行不是独立同分布的理想样本。前一次运行会预热 page cache、改变分支历史、改变温度、触发写回、影响内存分配器状态；后一次运行可能在更高温度或更脏的 page cache 下执行。重复运行的目的不是机械平均，而是观察分布和状态迁移。报告最好保留每轮原始样本和轮次顺序，而不是只留下一个聚合数字。

如果样本呈现趋势，例如前几轮越来越快，可能是预热；后几轮越来越慢，可能是温度或 writeback；周期性尖峰可能是后台任务或 GC/回收；单次异常可能是 page fault 或调度干扰。把这些样本全平均会掩盖信息。性能报告应允许写“首轮冷启动”“稳态中位数”“长尾样本原因未明”这些边界，而不是强行给一个单值结论。

不要只报平均值 平均值容易被少数异常样本拉偏，最小值可能过于乐观，最大值可能受偶然干扰支配。报告至少应给出运行次数、最小值、中位数和尾部；服务型路径还要关注 P95/P99；批处理路径关注吞吐、中位数和阶段占比。若样本波动很大，先解释波动来源：CPU 频率、温度、后台任务、page fault、I/O 写回、cgroup throttling、调度迁移，还是程序内部长尾。

指标	适合回答的问题	常见误用
最小值	干扰较少时的理想近似	当成真实用户稳定体验
中位数	常态吞吐或延迟	掩盖少数严重长尾
P95/P99	尾部稳定性、服务体验	样本太少时解释过度
平均值	总资源消耗或批处理汇总	被少数异常样本拉偏
扩展曲线	线程数、worker 数、负载变化	只看单点 speedup

强扩展、弱扩展和尾延迟扩展 强扩展固定总工作量，观察线程数增加后耗时是否下降；弱扩展固定每个 worker 的工作量，观察总体规模增长后效率是否保持；尾延迟扩展关注并发请求或任务增多后 P95/P99 怎样变化。归约适合强扩展和弱扩展；线程池和队列适合吞吐和尾延迟；分布式 shuffle 要同时看吞吐、长尾 partition 和重试成本。只说“多线程加速比”太粗。

5.4 Roofline：用字节账本约束优化方向

Roofline（屋顶线模型）把内核性能放在两个上限之间：计算吞吐上限和内存带宽上限。**算术强度**（arithmetic intensity）是每搬运一个字节做多少计算。数组求和每读取一个元素只做一次加法，算术强度低，容易受内存带宽限制；分块矩阵乘让同一数据多次参与乘加，算术强度高，才可能接近计算峰值。

Roofline 不是为了画漂亮图，而是为了阻止错误优化。若内核已经接近可持续内存带宽，继续减少几条算术指令通常收益很小；若内核远低于带宽上限，则要检查访问是否连续、是否有指针依赖、TLB miss、分支失败、写分配或同步等待。若内核算术强度很高却远低于计算上限，则要看向量化、依赖链、执行端口和前端供给。

Roofline 从两个守恒账本推出 处理器执行一个内核时，至少有两类资源账本。第一类是数据移动账本：有多少字节必须从缓存层级或内存系统流过，是否有额外写分配、临时数组、冷字段、随机节点和一致性流量。第二类是操作账本：有多少整数操作、浮点操作、比较、分支、地址计算和同步操作。给定机器在一段时间内能提供的内存带宽和计算吞吐都有上限；于是内核运行时间不可能同时小于“必要字节 / 可持续带宽”和“必要操作 / 可持续吞吐”。Roofline 的屋顶就是这两个下界叠出来的。

Listing 5.6 Roofline 判断的抽象流程

```
bytes = estimate_required_data_movement(kernel, input)
ops = estimate_required_operations(kernel, input)
time_by_bandwidth = bytes / sustained_bandwidth
time_by_compute = ops / sustained_compute_throughput
lower_bound = max(time_by_bandwidth, time_by_compute)
compare(measured_time, lower_bound)
```

如果实测时间接近带宽下界，优化方向应优先减少字节、改善局部性或提高复用；如果远高于两个下界，说明还有其他等待，例如指针依赖、分支失败、TLB miss、锁、原子、系统调用或 I/O。Roofline 给的是排除法，不是自动诊断。

字节账本先于公式 很多报告只数源码里显式读写的数组，漏掉实际流量。简单 copy 逻辑上读 4 字节、写 4 字节，但普通写 miss 可能触发 write allocate，先

读入目标 cache line 再修改；哈希表查询不只读 key 和 value，还读 bucket、节点、控制字节和字符串；对象数组扫描一个热字段，cache line 可能带入冷字段；全局原子计数器逻辑上只写 8 字节，硬件上可能让整条 cache line 在核心之间迁移。Roofline 的第一步是把这些字节写出来。

Listing 5.7 字节账本的最小形态

```
1 struct ByteLedger {
2     std::string kernel_name;
3     std::uint64_t logical_read_bytes = 0;
4     std::uint64_t logical_write_bytes = 0;
5     std::uint64_t temporary_bytes = 0;
6     std::uint64_t estimated_extra_bytes = 0;
7     std::uint64_t integer_ops = 0;
8     std::uint64_t floating_ops = 0;
9 };
```

`estimated_extra_bytes` 不是精确真理，而是把 cache line、写分配、临时材料化和结构体冷字段纳入讨论。估算错了也有价值：如果实测和估算差距很大，说明存在隐藏拷贝、随机访问、预取失败、page fault 或同步流量。

写分配为什么会改变字节数 许多缓存采用 write allocate：写一个尚未在缓存中的地址时，硬件先把目标 line 读入缓存，再修改其中几个字节，之后再写回或传播。于是一次看似纯写的流式 store，可能包含读入目标 line 的流量。非临时写、批量初始化、memset、写合并缓冲和平台特定指令可以改变这条路径，但不能在账本里假装写入免费。对 histogram 这类随机写，写分配还会和缓存容量、冲突、TLB 和一致性叠加。

算术强度不是源码复杂度 源码看起来复杂，不代表算术强度高。一个哈希表查询可能有很多分支和指针，却每次只做少量整数操作，主要等待随机内存；一个矩阵乘源码很短，却让同一数据参与大量乘加，算术强度高。算术强度只关心“每移动一个字节做多少有用操作”，不是代码行数或算法名字。报告中应写操作数账本，而不是只写 $O(n)$ 。

三个基准内核 三个基准内核可以覆盖三类常见瓶颈。第一，sum：大数组只读一次，低算术强度，常用于观察带宽和依赖链。第二，saxpy 或 scale-and-add：读两个数组、写一个数组，展示流式读写和 write allocate。第三，histogram：每条记录读 key、写 bucket，展示随机写、热点 key、false sharing 和合并成本。三者都简单，但瓶颈模型不同。

Compute Systems Engine 的日志统计会同时遇到这三类形状。解析后的 event id 列适合 sum/map 式扫描；字段变换适合 saxpy 式字节账本；事件计数则变成 histogram，受 key 分布和共享写影响。Roofline 能解释规则流式内核，但对 histogram、锁等待、I/O 和分布式重试不够直接，需要配合计数器和 trace。

dot product 的完整手算 Roofline 必须能落到一个小 kernel。考虑最小 dot product：

Listing 5.8 dot product 的标量语义

```
1 float dot_product(std::span<const float> a, std::span<const float> b)
2 {
3     float sum = 0.0F;
4     for (std::size_t i = 0; i < a.size(); ++i) {
5         sum += a[i] * b[i];
6     }
7     return sum;
```



```
8 }
```

每个元素做一次乘法和一次加法，通常按 2 FLOPs 计；每个元素至少读取两个 float，共 8 字节。忽略 `sum` 在寄存器中的归约和最终写回：

$$AI = \frac{2 \text{ FLOPs}}{8 \text{ bytes}} = 0.25 \text{ FLOP/byte}$$

若可持续内存带宽是 50GB/s，带宽 roof 是：

$$50 \text{ GB/s} \times 0.25 \text{ FLOP/byte} = 12.5 \text{ GFLOP/s}$$

若 CPU 峰值浮点吞吐是 500GFLOP/s，这个 kernel 也不会因此接近 500 GFLOP/s；它先被带宽 roof 压住。实测若只有 8 GFLOP/s，不应直接说“SIMD 不够强”，而要问：实际 GB/s 是多少，是否接近 50 GB/s；循环是否向量化；归约依赖是否被多累加器打散；输入是否对齐；工作集是在 L3、DRAM 还是 page fault 路径上。

账本项	dot product	解释
FLOPs/element	2	multiply + add
bytes/element	8	读两个 float 输入
AI	0.25 FLOP/byte	低算术强度
50 GB/s 带宽	12.5 GFLOP/s	memory-bound 上限
roof		
500 GFLOP/s	不主导	计算峰值高于带宽约束
compute roof		

矩阵遍历的 cache/TLB 账本 二维数组遍历可以把 cache line 和 TLB 压力讲清。假设 C/C++ row-major 存储 `floata[M][N]`：

$$\text{addr}(i, j) = \text{base} + (i \times N + j) \times 4$$

行优先遍历中，内层 `j` 每次让地址增加 4 字节。若 cache line 是 64 字节，一条 line 覆盖 16 个 float，硬件预取也容易工作。列优先遍历中，内层 `i` 每次让地址增加 `N*4` 字节。若 `N=4096`，步长是 16 KiB；相邻访问通常落在不同 cache line，甚至不同页面，TLB 覆盖压力也更大。

Listing 5.9 两个遍历方向的地址差异

```
1 // row-major friendly: inner stride = 4 bytes
2 for (std::size_t i = 0; i < M; ++i) {
3     for (std::size_t j = 0; j < N; ++j) {
4         sum += a[i][j];
5     }
6 }
7
8 // row-major hostile when N is large: inner stride = N * 4 bytes
9 for (std::size_t j = 0; j < N; ++j) {
10     for (std::size_t i = 0; i < M; ++i) {
11         sum += a[i][j];
12     }
13 }
```


这不是死记“行优先快”。如果 layout 是 column-major、blocked layout 或 view/stride 张量, 结论会跟地址公式一起改变。报告必须写 layout、shape、stride 和 element size, 否则 cache/TLB 分析没有对象。

遍历方式	内层地址步长	机器后果
row-major 行优先	4 bytes	spatial locality 好, 预取容易
row-major 列优先	$N \times 4$ bytes	跨 line/跨页, cache/TLB 压力高
blocked traversal	tile 内连续	用小工作集换复用

perf stat 要接回 Roofline perfstat 输出不能孤立解释。先把计数器转成派生指标:

$$IPC = instructions/cycles$$

$$branch\ miss\ rate = branch_misses/branches$$

$$cache\ miss\ rate = cache_misses/cache_references$$

再把它接到字节账本和反汇编。dot product 若 measured GB/s 接近带宽 roof、IPC 偏低、cache miss 随工作集变大而上升, memory-bound 解释较强。若 instructions 很多、GFLOP/s 低、反汇编没有向量指令, 优化方向是 SIMD 和多累加器。若 branch misses 高, 说明被测代码可能不是纯 dot, 而是含数据相关分支或错误路径。

观察	可能解释	需要反证
IPC 低、cache miss 高	memory-bound、TLB 或随机访问	工作集阶梯、stride 对照
IPC 高、GB/s 低	数据在 cache 或字节账本错	冷/热 状态、bytes moved 核对
branch miss 高	输入相关分支或错误路径热	输入分布、CFG/反汇编
instructions 暴涨	未向量化、额外检查、函数调用	objdump、优化报告
context switches 高	调度干扰或线程等待	affinity、trace、CPU time

如果 PMU 不可用, 报告要降级而不是伪造。可以使用输入规模阶梯、stride 对照、阶段时间、反汇编、checksum、线程扩展曲线和应用 trace。证据等级降低, 结论就要写窄: 可以说“结果符合内存带宽瓶颈假设”, 不能说“cache miss 已下降”。

RooflineReport 最小字段 每个核心 kernel 都应能产出一份小报告:

Listing 5.10 RooflineReport 最小字段

```
kernel_name
input_shape
dtype
semantic_oracle
flops_formula
```

```

bytes_formula
arithmetic_intensity
measured_elapsed_ns
measured_gflops
measured_gb_per_s
machine_bandwidth_roof
machine_compute_roof
roofline_bound: memory_bound or compute_bound or mixed
perf_counters_available
counter_summary
gap_explanation
unverified_boundaries

```

这份结构迫使优化结论回答“改变了哪条上限”。减少几条指令不会让 DRAM-bound kernel 大幅变快；改变 layout 可能让 cache/TLB 证据变化；SIMD 提升 GFLOP/s 时也要看 GB/s 和 correctness oracle 是否保持。

单线程 Roofline 和多线程 Roofline 单线程可能受加载端口、预取能力、乱序窗口或依赖链限制，远低于平台总内存带宽；多线程逐步增加后，可能接近内存控制器带宽，然后加线程不再提升。报告应同时展示单线程曲线和多线程曲线。若 1 到 4 线程提升明显，8 线程后停滞，可能是带宽、LLC、NUMA 或同步热点；若 2 线程就不提升，可能是串行部分、任务粒度或全局锁。

5.5 计数器、Profile 和 Trace

计时告诉现象，计数器提供线索，profile 定位热点，trace 保留顺序。四者回答不同问题。单核热循环慢，先看时间、反汇编、IPC、branch、cache 和 TLB；多线程不扩展，看扩展曲线、context switches、锁等待、false sharing 对照；I/O 慢，看队列长度、page faults、write/fsync 时间；分布式慢，看 request timeline、attempt、timeout、partition 和重试。

perf stat 的位置 perfstat 适合整体计数，例如 cycles、instructions、branches、branch-misses、cache-misses、page-faults、context-switches。它能帮助区分趋势，但不能自动给答案。IPC 低可能是内存等待、分支失败、前端供给不足、锁等待或系统调用阻塞；cache miss 高可能只是大数组流式扫描的正常现象；context switches 多可能来自过多线程、阻塞队列或系统干扰。

报告必须保存命令、事件列表、运行时间、权限限制和是否出现 multiplex。不同 CPU 上事件名和含义可能不同，云环境、容器和虚拟化环境可能限制 PMU。不可用时，不要把缺失事件填成 0；应写 Unavailable，并使用输入规模阶梯、反汇编、阶段时间和应用指标作为降级证据。

从退休指令到 IPC 性能计数器首先要回到一条具体循环。考虑最小整数求和：

Listing 5.11 整数求和的 C++ 形态

```

1 std::int64_t sum_values(std::span<const std::int32_t> values) {
2     std::int64_t sum = 0;
3     for (const std::int32_t value : values) {
4         sum += value;
5     }
6     return sum;
7 }

```

在未向量化、未展开的简化路径里，热循环可能接近下面的形状：

Listing 5.12 求和循环的一种典型汇编形态

```

xor     eax, eax                ; sum = 0
xor     ecx, ecx                ; i = 0
.Lloop:
movsxd  rdx, DWORD PTR [rdi+rcx*4] ; load values[i]
add     rax, rdx                ; sum += value
inc     rcx                     ; ++i
cmp     rcx, rsi                ; i != size
jne     .Lloop

```

退休指令 (retired instruction) 是已经走到提交点、改变了架构可见状态的指令。乱序核心内部可能执行了更多微操作，也可能沿错误分支执行了一段后来被丢弃的路径；这些工作消耗周期和能量，却不一定退休。于是 `instructions` 不是“硬件实际忙了多少”的完整描述，`cycles` 也不是“做了多少有效工作”的完整描述。二者相除得到的 IPC 只是一个入口：IPC 低可能是内存等待、分支失败、前端断供、锁等待、系统调用睡眠或频率变化；IPC 高也不保证程序好，因为它可能高效地执行了大量无用拷贝。

这条循环还说明，计数器不能脱离反汇编解释。若编译器把循环向量化，单个向量 `load` 会覆盖多个元素，退休指令数下降；若编译器展开四路累加，分支指令变少，寄存器压力上升；若输入大小很小，函数入口、边界检查和调用开销占比上升。报告写 `instructions` 之前，必须知道被计数的到底是哪条机器路径。

load 指令背后的事件链 `movsxd rdx, DWORD PTR [rdi+rcx*4]` 看起来是一条指令，硬件内部至少拆出几件事：用 `rdi`、`rcx` 和 `scale` 计算有效地址；用虚拟页号查 DTLB；命中后得到物理地址；用物理地址的 `index` 查 L1D set；比较 `tag`；命中则取出对应 4 字节并符号扩展；未命中则继续去 L2、LLC 或内存。若地址跨页或 TLB 未命中，还会走页表遍历；若页不在内存，还会进入 `page fault` 和调度等待。

因此 `cache-misses`、`LLC-load-misses`、`dTLB-load-misses` 这些事件并不是源码语句级的事实，而是某个采样窗口里硬件路径的侧影。连续数组求和的 LLC miss 可能很高，却是流式扫描的正常代价；随机指针追踪的 miss 也可能很高，但它表示每一步都在等待下一地址。两者数字接近，优化方向完全不同。必须把事件和地址形状绑定：连续、固定步长、随机索引、链式依赖、共享写、系统调用，分别对应不同解释。

分支计数器要结合控制流 循环尾部的 `cmp/jne` 通常非常可预测：大部分迭代跳回，最后一次不跳。条件过滤则不同：

Listing 5.13 可预测性取决于输入分布的分支

```

1 if (event_type == target) {
2     ++count;
3 }

```

如果输入先按 `event_type` 排序，分支结果会成段出现；如果输入完全随机，结果可能接近抛硬币。CPU 的分支预测器只能利用历史模式，不能提前知道随机数据。一次 `branch miss` 会让前端和乱序窗口丢弃错误路径上的工作，重新从正确目标取指。报告中看到 `branch-misses` 升高时，不能立刻写“分支是瓶颈”；还要看这个分支是否在热循环、miss 数量相对元素数是否大、改成 `mask` 或分桶后时间是否下降，以及新路径是否增加了额外 `load/store`。

反汇编是计数器的坐标系 同一段 C++ 可以生成完全不同的机器路径。打开优化后，`std::accumulate` 可能内联成普通循环，也可能因为迭代器、别名或异常边

界留下额外调用；简单 `map` 可能自动向量化，也可能因为指针重叠假设生成运行时检查；一次 `push_back` 可能在热路径里只写数组，也可能触发容量检查、分配和搬迁。计数器只能说明“这次机器路径发生了什么”，不能说明“源码理论上应该怎样”。

因此性能报告应保存关键反汇编摘要，而不是只保存源码片段。摘要不必贴完整二进制，但要说明：主循环是否存在；是否向量化；是否展开；是否有函数调用；`load/store` 地址形态是什么；是否有 `lock` 前缀、系统调用、异常慢路径或运行时别名检查。没有这个坐标系，`cycles`、`instructions` 和 `cache-misses` 很容易被解释到错误源码位置上。

Listing 5.14 计数器报告的反汇编摘要字段

```
hot_loop:
  vectorized: yes/no
  unroll_factor: known/unknown
  main_load_shape: [base + index * scale + displacement]
  main_store_shape: contiguous/random/shared
  calls_in_loop: none/list
  atomic_or_lock_prefix: yes/no
  syscall_in_timed_region: yes/no
  tail_strategy: scalar/masked/unknown
```

事件解释的最小对照 单个计数器很少能单独证明原因。更稳的做法是为每个假设准备一个对照。如果怀疑内存带宽，用 `stream copy`、数组求和、不同输入规模和线程数扫描建立带宽边界；如果怀疑分支，用全真、全假、排序和随机输入比较；如果怀疑 TLB，用连续、固定大步长和随机页访问比较；如果怀疑同步，用全局原子、`thread-local partial`、`padding partial` 对照。计数器负责给线索，对照实验负责排除错误解释。

观察	不能直接得出的结论	需要的对照
IPC 低	一定是内存慢	分支、前端、锁、I/O、off-CPU 时间
cache miss 高	cache 优化一定有效	流式带宽基准、随机访问对照、字节账本
branch miss 高	应改成无分支	输入分布矩阵、mask 版本、额外字节成本
instructions 少	程序一定快	周期、带宽、尾部、系统调用和语义校验
context switch 多	调度器一定是主因	队列等待、I/O 等待、锁等待和线程过量对照

计数器是采样窗口里的硬件事件 性能计数器通常统计某类硬件事件在一段时间窗口内发生了多少次，或者按采样频率记录部分事件的位置。它们不是语义级事实。一次 `cache miss` 可能是流式扫描的正常成本，也可能是随机指针追踪的问题；一次 `branch miss` 可能发生在冷错误路径，也可能发生在热循环；`context switch` 可能来自程序阻塞，也可能来自系统干扰。计数器必须和计时边界、输入、阶段和源码位置绑定，才有解释力。

还有两个常见陷阱。第一，事件名不等于事件定义。不同微架构上 `cache-misses`、`LLC-load-misses`、`branch-misses` 可能映射到不同硬件事件或近似事件。第二，`multiplex` 会让工具在多个事件之间轮换，输出按时间比例估算；事件太多时，误差可能变大。报告要保存原始输出，不要只摘一个百分比。

计数器必须归一化 原始事件数本身很少能比较。处理 10 GiB 输入的 `cache miss` 数当然可能高于处理 1 GiB 输入；线程数翻倍后 `cycles` 总数也可能上升，即使 `wall time` 下降。计数器要和语义单位、数据单位或时间单位绑定：每条记

录多少 instructions, 每 KiB 输入多少 branch miss, 每次 accepted record 多少 LLC miss, 每个 worker 每秒多少 context switch, 或者每个输出 block 多少 page fault。归一化以后, 实验才知道是在“同样工作更便宜”, 还是“工作量变了”。

原始事件	更有用的口径	解释边界
instructions	instructions per record / per byte	说明机器路径长短, 不直接说明等待
branch-misses	misses per KiB / per branch / per record	必须结合输入分布和热分支位置
LLC misses	misses per MiB / per random lookup	流式扫描和随机访问含义不同
dTLB misses	misses per page / per record	要结合页面大小和有效记录数
cycles	cycles per record / per byte / per stage	受频率、迁移和等待影响
context switches	switches per second / per request	要区分阻塞、抢占和过量线程

归一化还要和 checksum 一起保存。若 candidate 每条记录 instructions 大幅下降, 但 accepted record 也下降, 可能是错误地跳过了输入。若每字节 branch miss 下降, 同时坏行 diagnostics checksum 不一致, 可能是把错误路径删除了。性能单位必须以语义单位为分母, 而不是只以“跑过的循环次数”为分母。

Top-Down 指标怎样落回代码 许多平台提供 Top-Down 风格的分类, 把 pipeline slot 粗分为 retiring、bad speculation、frontend bound、backend bound 等方向。它的价值不在名词本身, 而在把下一步实验缩小。Retiring 高并不表示无需优化, 可能只是有效地执行了很多多余工作; bad speculation 高, 优先看热分支和输入分布; frontend bound 高, 优先看热代码足迹、分支目标、译码、uop cache 和冷路径布局; backend bound 高, 再分内存等待、执行端口、依赖链、store buffer 和同步等待。

Top-Down 方向	先怀疑的问题	下一份证据
Retiring 高	有效退休多, 但可能做了多余字节或多余指令	字节账本、反汇编、算法替代版本
Bad speculation 高	分支预测失败或错误路径投机多	分支输入矩阵、热分支定位、branchless/mask 对照
Frontend bound 高	取指、译码、I-cache、ITLB 或 uop cache 供给不足	函数大小、冷路径分离、反汇编布局、调用边界
Backend bound 高	load/store、内存层级、执行端口、依赖或同步等待	AccessTrace、Roofline、依赖拆分、partial/padding 对照

这些分类不能替代实验。比如 frontend bound 可能来自真正 I-cache 压力, 也可能来自分支目标不稳定让前端反复走错; backend bound 可能来自 DRAM 带宽, 也可能来自一条指针依赖链或全局 atomic。正确用法是: Top-Down 给第一批嫌疑人, 输入矩阵、反汇编和替代实现负责筛掉错误嫌疑人。

HotKernelReport 要同时写事实和推断 项目里的热内核报告应该把直接事实、工具线索和工程推断分开。直接事实包括输入、checksum、阶段时间、反汇编摘要和样本统计; 工具线索包括计数器事件、profile 热点和 Top-Down 分类; 工程推断包括“瓶颈可能是分支预测”“可能是共享写热点”。推断后面必须跟随下一步反证实验, 不能直接变成结论。

Listing 5.15 HotKernelReport 的扩展字段

```
1 struct HotKernelReport {
2     std::string kernel_name;
3     std::string input_family;
```



```

4  std::uint64_t input_bytes = 0;
5  std::uint64_t records = 0;
6  std::uint64_t checksum = 0;
7
8  double seconds_median = 0.0;
9  double cycles_per_record = 0.0;
10 double instructions_per_record = 0.0;
11 double branch_misses_per_kib = 0.0;
12 double llc_misses_per_mib = 0.0;
13
14 bool counters_available = false;
15 std::string disassembly_summary;
16 std::string topdown_summary;
17 std::string primary_hypothesis;
18 std::string falsification_plan;
19 };

```

`primary_hypothesis` 只能写当前最强解释，例如“随机字符输入下 bad speculation 上升，怀疑 parser 热分支不可预测”。`falsification_plan` 要写下一组对照，例如“构造全真、全假、排序和随机输入；比较 branchless table 分类；确认 diagnostics checksum”。这样报告不会停在“看起来是分支问题”，而会推动代码和实验继续收敛。

Profile 定位时间，Trace 解释顺序 Profile 聚合“时间花在哪里”，例如 `perfrecord/report` 或其他采样工具。它适合找热点函数和指令范围，但不天然解释排队和顺序。Trace 记录“事件按什么顺序发生”，适合线程池、队列、异步 I/O、分布式请求和 checkpoint。一个线程池 CPU profile 可能显示执行函数很热，但 trace 才能看到 worker 大量空闲、某个 partition 长尾或 queue push 长时间等待。

Trace 也有成本。热路径每条记录都写详细日志，会改变程序行为。更稳妥的是记录阶段边界、任务开始/结束、队列满、队列空、消息发送/完成、重试、提交和错误摘要。需要细节时采样或打开实验模式。观测系统本身也是计算系统，仍然受数据布局、锁、I/O 和背压约束。

Off-CPU 时间 很多系统慢在 off-CPU 时间，也就是线程没有在 CPU 上运行。原因可能是等待锁、等待 I/O、等待条件变量、被调度器抢占、cgroup 节流或睡眠。On-CPU profile 看不到完整等待。工具不足时，应用内部也可以记录 push 等待时间、pop 空队列时间、worker 空闲时间、I/O completion 等待时间和 fsync 时间。后续第 15 章讲背压时，会把这些字段变成队列合同的一部分。

5.6 反汇编、计数器和输入矩阵的三角校验

性能分析最危险的时刻，是某个指标刚好支持已有猜想。看到 `branch-misses` 下降，就宣布分支问题修好；看到 `instructions` 下降，就宣布代码更高效；看到端到端快了，就宣布算法更好。这些结论都可能错。反汇编只能证明机器动作形状，计数器只能给出某类硬件事件线索，输入矩阵只能说明现象是否随数据分布变化。三者互相约束，才形成可靠证据。

例如把解析器从多分支分类改成查表分类。反汇编应能看到条件跳转减少、表 load 增加、错误构造是否离开热循环；计数器应观察 `branch`、`branch-misses`、`instructions`、`cycles`、`cache` 或前端受限指标是否变化；输入矩阵应覆盖规则字段、随机字段、错误密集和短行。若随机字段变快、规则字段不变或略慢，且分支失败下降，解释较强。若所有输入都变快但分支指标不变，收益可能来自冷路径分离或代码体积；若 `branch-misses` 下降但 `cycles` 不降，瓶颈已经迁移。

Listing 5.16 三角校验的最小材料

```

assembly evidence:
  hot loop address range
  conditional branches removed or added
  loads, stores and calls added or removed
  cold path placement changed or unchanged

counter evidence:
  cycles and instructions normalized by input bytes
  branches and branch_misses normalized by records
  cache/TLB/frontend metrics when available
  unavailable events recorded explicitly

input evidence:
  regular fields
  random fields
  error-dense input
  short records
  large cold input

```

反汇编回答“变成了什么机器动作” 反汇编不是为了炫耀指令名。它回答几个具体问题：热循环是否真的是被测函数；编译器有没有把循环优化掉；分支、load、store、调用和原子指令分别在哪里；循环是否展开；冷路径是否仍然和热路径混在一起；地址公式是否从连续变成随机；SIMD 指令是否真的出现。没有这些事实，计数器变化很难解释。

同一个源码改动，编译器可能生成完全不同的机器码。打开 LT0、PG0、`-march=native`、异常开关、断言开关、日志宏，都可能改变热循环。报告保存编译命令和反汇编，是为了让结论能被复查，而不是为了把每条指令都讲完。

计数器回答“哪些等待可能变了” 计数器有硬件、内核权限和事件定义的限制。不同平台的事件名不一定等价，容器中可能读不到 PMU，复用 multiplexing 会降低精度，短程序的噪声可能很大。因此计数器不能当判决书，只能当证据。使用它们时要归一化：每字节 cycles、每记录 instructions、每 KiB branch-misses、每任务 context switches。没有归一化，输入规模和错误率变化会伪造趋势。

指标变化	合理推断	必须继续检查
instructions 下降	机器动作变少或循环被优化	checksum、反汇编、阶段边界是否一致
branch-misses 下降	预测失败减少或分支总数变化	cycles 是否下降，load 或前端是否成为新瓶颈
cache-misses 上升	数据路径更分散或工作集变大	是否换来更少同步，是否输入已冷读
context-switches 上升	阻塞、抢占或线程过多	voluntary/involuntary、futex、I/O 和调度 trace
cycles 不变	瓶颈未动或收益被新成本抵消	阶段表、反证输入和端口/内存线索

输入矩阵回答“解释能否被反驳” 如果某个解释正确，改变输入分布应产生可预测的变化。分支预测问题应随随机性增强而放大；cache line 有效字节问题应随步长增大而放大；TLB 问题应随活跃页数增大而放大；共享写问题应随热点 key 和线程数增大而放大；I/O 问题应在冷读和热读之间明显不同。输入矩阵不是附属测试，而是性能因果的反证装置。

Listing 5.17 按瓶颈假设设计输入矩阵

```

branch hypothesis:
    regular, periodic, random, adversarial 50/50

cache-line hypothesis:
    contiguous, stride 64B, stride 256B, random line

TLB hypothesis:
    one element per cache line, one element per page, random page

coherence hypothesis:
    uniform key, single hot key, skewed Zipf-like key

I/O hypothesis:
    hot page cache, cold page cache, mmap first touch, read buffer

```

错误结论怎样被三角校验拦住 假设候选版本耗时从 12 秒降到 5 秒。三角校验可能发现：反汇编里错误诊断构造被整个优化掉，checksum 没有覆盖错误摘要；计数器里 instructions 下降巨大，但坏行输入输出不同；输入矩阵里只有无错误输入变快，错误密集输入失败。这个候选不是优化，而是语义缩水。

另一个候选版本 branch-misses 从每 KiB 200 次降到 20 次，耗时却不变。反汇编显示查表分类引入额外 load 和更多寄存器压力；计数器显示 cache-misses 和 cycles 不降；规则输入和随机输入差距缩小但总时间没有改善。结论应写成“分支瓶颈被削弱，瓶颈迁移到数据 load 或后端资源，当前版本不应替换 baseline”，而不是继续堆更多 branchless 技巧。

一个热循环的最小三角校验路径可以这样组织：选择一个明确改写，例如分支分类改查表、链表改数组、全局 atomic 改 thread-local partial；同时保留热循环反汇编摘要、归一化计数器或不可用说明、至少四组输入矩阵、checksum 和错误摘要；最后写出被支持的解释、被排除的解释和仍未验证的边界。这样做的目的不是增加文档负担，而是防止“快了”掩盖语义缩水或瓶颈迁移。

5.7 反证判读：不要只支持单一解释

可信报告不只支持一个假设，还要主动排除其他解释。若瓶颈被判断为内存带宽，就要说明为什么不是分支、锁、I/O 或调度；若锁被判断为主瓶颈，就要说明临界区、futex、等待时间或全局共享写的证据；若 SIMD 被判断为有效，就要说明尾部、错误路径和数值语义仍正确。反证让结论更可信。

单变量原则 一次实验尽量只改变一个变量。若同时改数据布局、线程数、编译选项、输入规模和算法，变快也无法归因。实际工程修改可能较大，但实验应拆解：先只换布局，再只换线程，再只换 partial 对齐，再只换输入分布。失败实验同样要保留，因为它说明瓶颈可能不在这个方向。

案例：parallel histogram 为什么不扩展 假设 parallel histogram 在 8 线程后不再提升。候选解释包括全局锁、全局原子、false sharing、key 倾斜、哈希分配、内存带宽、线程迁移和任务粒度。反证矩阵可以这样设计：全局 mutex 对 thread-local partial，排除锁；紧凑 partial 对 padded partial，检查 false sharing；均匀 key 对单热点 key，检查倾斜；预分配 bucket 对动态分配，检查分配；固定线程绑定对默认调度，检查迁移；改变 chunk size，检查任务粒度。每组实验都必须保持 checksum 和 key 计数一致。

统计显著和工程显著 一个 1% 的提升即使多轮稳定，也未必值得引入复杂手写 SIMD 或无锁结构。工程显著性要考虑代码复杂度、平台差异、测试成本、错误风险和可维护性。一个 5% 的简单布局调整可能比 8% 的难维护汇编更值得保留。性

能工程不是只添加技巧，也是控制复杂度。

深入理解

负结果也是结果。手写预取没有收益，可能说明硬件预取已经足够或访问模式不适合；手写展开让短行变慢，可能说明代码体积或启动成本增加；无锁队列提高平均吞吐却恶化 P99，说明尾部路径变差。把这些写进报告，方法边界才清楚。

5.8 测量审计：从错误样本追到机器证据

性能报告最危险的形态是“候选版本快了很多”，但没有说明它是否仍在做同一件事。设日志聚合优化版声称加速 2.7 倍。复查样本时发现 malformed 输入下 rejected 计数少了一部分，错误诊断 checksum 也不同。此时不能把这些样本删掉后继续算中位数，因为候选版本已经不是同一个程序。测量审计的第一条原则是：语义失败的样本没有资格进入性能摘要。

第一步：把样本分成 valid 和 invalid 每个样本都必须先通过 reference 校验。校验内容至少包括输出计数、accepted/rejected、错误摘要、manifest 候选项和 checksum。通过校验的样本进入性能统计；失败样本进入错误列表，保留命令、输入、seed、版本和失败差异。

Listing 5.18 样本进入统计前的门槛

```
for sample in measured_samples:
    if sample.output_checksum != reference.output_checksum:
        sample.valid = False
        sample.reason = "output checksum mismatch"
        continue
    if sample.diagnostic_checksum != reference.diagnostic_checksum:
        sample.valid = False
        sample.reason = "diagnostic checksum mismatch"
        continue
    sample.valid = True
```

这样做会让某些漂亮数字消失，但这是正确结果。错误样本揭示的是候选实现破坏了语义边界，下一步应修实现或缩小优化适用范围，而不是继续解释速度。

第二步：把阶段报告和总时间分开看 候选版本通过语义后，仍不能只比较总时间。总时间下降可能来自解析、聚合、写出、诊断、提交或预热边界的任一变化。阶段报告把端到端路径拆开，避免在错误位置优化。

Listing 5.19 阶段报告的最小字段

```
StageReport:
    parse_ns
    encode_ns
    aggregate_ns
    reduce_ns
    diagnostics_ns
    write_ns
    commit_ns
    accepted_records
    rejected_records
    bytes_read
    bytes_written
```

如果优化版只在错误稀少输入上快，而错误密集输入的 `diagnostics_ns` 激增，说明冷路径分离的回查成本需要进入结论；如果 `aggregate_ns` 下降但 `write_ns` 不变，总体加速会受输出边界限制；如果 `parse_ns` 下降同时 `rejected` 下降，语义仍可疑。

第三步：反汇编检查热循环是否真的改变 计数器之前，先看二进制。候选版本若宣称减少分支，热循环中应看到较少的条件跳转，或分支集中到更可预测的位置；若宣称变成连续列读取，应看到对 `span` 基址加索引的 `load`，而不是追踪对象指针；若宣称减少虚调用，应看不到每条记录一次间接 `call`。

Listing 5.20 热循环反汇编审计问题

```
Does the loop contain:
  a call inside every record iteration?
  indirect branch targets depending on input data?
  loads through LogRecord pointer fields?
  allocations or exception paths in the hot range?
  stores to a shared global counter?
```

反汇编不能替代运行数据，但能快速打断错误叙事。源码层面已经改成 `batch`，不代表编译后的热循环真的没有对象回查；源码层面移走错误路径，不代表链接后冷函数没有因为内联或模板膨胀留在热区域附近。

第四步：计数器必须归一化 原始计数器不是事实本身。一个版本处理了更多记录，自然会有更多指令；一个版本写出更多错误诊断，自然会有更多 `cache miss`。计数器要除以共同语义单位。常见归一化包括：

指标	归一化单位	用途
instructions	每条 accepted record 或每输入字节	判断是否少做工作或控制流更短
cycles	每条 accepted record 或每输入字节	对比总体机器成本
branch misses	每 KiB 输入或每千记录	观察预测失败是否随输入随机性变化
LLC misses	每 MiB 输入或每千记录	观察工作集和布局变化
minor faults	每次运行和每 GiB 输入	判断首次触页是否混入
lock wait	每任务或每输出 block	分离 CPU 热循环和等待时间

归一化后仍要看分母是否合理。错误密集输入中，用 `accepted record` 作分母会夸大成本，因为大量 `rejected` 记录仍被解析；此时还要给出每输入字节指标。聚合章节常用每 `accepted record`，解析章节常用每输入字节，I/O 章节还要给出每输出字节。

第五步：构造反证计划 假设候选版本快是因为“减少分支失败”。反证计划不能只重复同一个随机输入，而要构造输入族：全可预测、全不可预测、排序后可预测、错误密集、短行、长行。若只有随机输入快，全可预测输入不快，分支解释更可信；若所有输入都快，可能是指令数、内存布局或少做工作；若错误密集输入失败，冷路径语义没有守住。

Listing 5.21 候选解释和反证实验

```
hypothesis:
  branch misses dominate parser time

falsification:
```



```

run predictable input
run random input
run sorted-by-class input
run error-heavy input
compare checksum and diagnostics
normalize branch misses per KiB

```

反证计划写进报告，是为了让后来的人知道结论还能怎样被推翻。没有反证的报告只能说明“这组数据支持当前说法”，无法说明其他解释已经被排除。

MeasurementAudit 结构 工程上可以把审计结果保存成结构化对象，和 **HotKernelReport** 一起落盘。

Listing 5.22 MeasurementAudit 保存语义和机器证据

```

1 struct MeasurementAudit {
2     std::string case_name;
3     std::string candidate_name;
4     std::uint64_t input_seed = 0;
5     std::uint64_t input_bytes = 0;
6     std::uint64_t reference_checksum = 0;
7     std::uint64_t candidate_checksum = 0;
8     std::uint64_t diagnostic_checksum = 0;
9     bool semantic_valid = false;
10
11     std::uint64_t accepted_records = 0;
12     std::uint64_t rejected_records = 0;
13     double seconds_median = 0.0;
14     double instructions_per_record = 0.0;
15     double branch_misses_per_kib = 0.0;
16     double llc_misses_per_mib = 0.0;
17     double cycles_per_accepted_record = 0.0;
18
19     std::string disassembly_note;
20     std::string primary_hypothesis;
21     std::string falsification_plan;
22 };

```

这个结构不是为了把所有工具输出塞进内存，而是固定报告边界。原始 **perf** 输出、反汇编、样本 CSV 和环境信息可以作为文件保存；结构里只放审计摘要和关键归一化指标。后续 CI 或回归脚本可以先检查 **semantic_valid**，再看性能阈值。

错误报告和合格报告的差别 错误报告常见写法如下：

Listing 5.23 不可接受的性能结论

```

new parser is 2.7x faster
tested on 10M rows
perf looks better

```

它缺少 reference、输入族、错误比例、阶段归因、归一化指标和反证。更合格的写法应该保留可复查事实：

Listing 5.24 可复查的性能结论

```

case: parser-random-fields-10M
semantic_valid: true
input: 1.8 GiB, seed 9127, error_rate 0.3 percent
speedup: 2.7x median over 20 valid samples
parse_ns: -61 percent, diagnostics_ns: +4 percent
branch_misses_per_kib: 18.4 -> 3.1
hot_function_bytes: 1568 -> 736
checksum and diagnostic_checksum match reference
falsification: predictable input shows only 1.05x

```

这段结论并不夸张，但可复查。它说明加速主要在 `parse` 阶段，分支失败和热函数体积下降支持前端解释；同时可预测输入收益很小，给出了解释边界。这样的报告才能作为后续布局、SIMD、线程和 I/O 改造的基础。

5.9 BenchmarkCase：把测量变成项目基础设施

如果每章都手写不同 `benchmark`，结果无法比较，报告也无法复查。`Compute Systems Engine` 应定义一个最小 `BenchmarkCase`：它描述输入生成、reference、候选版本、预热轮数、测量轮数、线程数、绑定策略、可用计数器、阶段报告和输出路径。完整实现放在代码实践卷，原理卷给出结构和纪律。

Listing 5.25 `BenchmarkCase` 的最小形态

```

1 struct BenchmarkCase {
2     std::string name;
3     std::string input_family;
4     std::uint64_t input_bytes = 0;
5     std::uint64_t seed = 0;
6     std::int32_t worker_count = 1;
7     std::int32_t warmup_rounds = 1;
8     std::int32_t measured_rounds = 10;
9     bool requires_perf = false;
10    bool requires_numa = false;
11 };

```

这个结构不是通用 `benchmark` 框架，只是固定最重要的上下文。真正的结果还要包含 `BenchmarkSample`、`StageReport`、`ByteLedger`、机器信息、编译命令和原始工具输出。项目可以把每次运行保存为目录：

Listing 5.26 一次实验归档目录的建议形态

```

1 results/2026-06-21-reduce-baseline/
2   env.txt
3   commands.txt
4   input.json
5   samples.csv
6   stages.csv
7   notes.md

```

书中不需要展示所有原始文件，但工程上必须知道证据在哪里。几周后发现性能回归时，只有最终图表不够，必须能回到输入、命令、样本和校验。

判读结构 性能判读可以固定为以下结构：

1. 问题：本次对照回答哪个具体问题。
2. 语义：reference、oracle、checksum、错误摘要和边界输入。
3. 环境：机器、系统、编译器、编译选项、线程绑定、权限限制。
4. 输入：规模、分布、seed、冷/热状态、错误比例。
5. 矩阵：版本、线程数、batch size、队列容量、运行轮次。
6. 结果：原始样本、统计摘要、阶段时间、字节账本、可用计数器。
7. 分析：瓶颈假设、支持证据、反证、成本迁移。
8. 边界：哪些机器、输入和故障模型尚未验证。

固定结构不是为了填表，而是让缺失项快速暴露。若缺 reference，速度无意义；若缺输入，结论不可复现；若缺字节账本，Roofline 站不住；若缺边界，局部结果很容易被误用到其他机器。

性能回归说明 项目变大后，性能回归不可避免。好的回归说明包含触发版本、输入、环境、变化幅度、阶段归因、最可能原因和下一步验证。先看阶段表：如果总时间慢 20%，parse 阶段不变，write 阶段慢 80%，不要去优化解析器；如果所有阶段都慢，先检查环境、频率、输入和编译选项。修复后不仅看总时间恢复，还要看原阶段指标和 checksum 是否恢复。

5.10 完整案例：三倍加速为什么不能直接相信

现在把前面的规则合成一次完整审查。某个版本把日志聚合从全局哈希表改成线程本地 histogram，报告写着“8 线程三倍加速”。这句话不能直接进入结论，因为它至少缺四层事实：两个版本是否输出同一结果，输入是否覆盖热点和坏行，计时范围是否包含写出和提交，三倍来自减少锁等待、减少随机访问、减少字符串比较，还是只是少做了错误诊断。

第一步：固定语义再看时间 先要求两个版本在同一输入上输出相同 counts、accepted、rejected、diagnostics、manifest entry 和 checksum。若优化版只统计合法行但不保存错误记录身份，它可能会更快，但这不是同一个程序。若优化版把字符串 event name 编码成整数 id，还要检查 dictionary version 和输出解码是否稳定。若优化版把输出排序去掉，测试可能因为哈希遍历顺序不同而偶尔失败，也不能把这个差异当成性能噪声。

Listing 5.27 性能样本先过语义闸门

```
case hot-key-10m:
    baseline checksum = 0x7e34...
    candidate checksum = 0x7e34...
    accepted = 10000000
    rejected = 120
    diagnostics checksum equal
    manifest entry equal
    status = Passed

case malformed-lines:
    candidate diagnostics checksum mismatch
    status = Failed
    elapsed is not used in performance summary
```

这一步会淘汰很多漂亮数字。被淘汰的样本不是“影响平均值的异常值”，而是没有资格参与性能统计的错误运行。后续所有阶段表和计数器，都只对 Passed 样本解释。

第二步：拆成阶段看成本迁移 如果端到端从 30 秒降到 10 秒，阶段表可能

有三种解释。第一，parse 仍然 8 秒，histogram 从 18 秒降到 2 秒，这是共享写和哈希表路径改善；第二，histogram 变快，但 write 从 2 秒涨到 9 秒，说明 partial 合并或输出放大把成本推到下游；第三，所有阶段都变快，可能是输入已经热在 page cache，或优化版少做了校验。阶段表的价值在于让“更快”变成可定位的变化。

阶段	baseline	candidate	初步解释
read	3.1 s	3.0 s	输入路径基本不变
parse	7.8 s	7.6 s	parser 不是主要收益来源
histogram	17.4 s	2.4 s	共享写或哈希路径被移除
merge	0.6 s	1.1 s	partial 合并成本增加但可接受
write+commit	1.1 s	1.2 s	输出语义未明显缩水

阶段表仍不是最终解释。它只能说明收益集中在 histogram。要进一步判断收益来自哪里，需要构造反证输入。

第三步：用输入矩阵排除错误解释 同一个优化在不同输入下应有可解释形状。若 key 均匀分布，全局 atomic 可能已经能扩展一段；若单个 key 占 90%，全局 atomic 或锁会急剧退化，thread-local partial 应保持稳定；若 event name 很长且种类多，字符串哈希和比较可能占主导；若 dictionary 已经把 key 变成整数，收益应更多来自共享写本地化。输入矩阵要故意改变这些因素。

输入族	攻击的假设	预期观察
均匀短 key	检查普通并行开销	partial 仍快，但倍率有限
单热点 key	检查 cache line 所有权竞争	全局 atomic 退化，partial 稳定
长字符串 key	检查哈希和比较成本	字典编码收益明显
坏行密集	检查错误路径是否被偷删	candidate 必须保持 diagnostics
超大输入冷读	检查 I/O 是否主导	histogram 改造对端到端收益变小

如果优化只在一个精心挑选的输入上三倍加速，报告应把结论写窄；如果在热点 key、均匀 key、长字符串和坏行输入上都保持语义正确，并且阶段变化符合预期，结论才开始稳固。

第四步：把硬件线索和模型接上 对 histogram 来说，Roofline 只能提供部分约束，因为随机写、同步和一致性流量很难用简单字节账本完全表示。但仍然可以写最小账本：每条记录读取一个 event id，写一个计数槽；全局 atomic 还会引入 cache line 所有权迁移；字符串版本还会读 key 字节和哈希桶；partial 版本把热写变成本地数组更新。计数器若可用，可以观察 instructions、cycles、cache misses、branch misses、context switches；若不可用，也可以用线程数扩展曲线和输入对照支持解释。

Listing 5.28 histogram 优化的证据链摘要

```
semantic evidence:
  reference checksum equal
  diagnostics checksum equal
  manifest equal
```

```

stage evidence:
  histogram stage drops from 17.4s to 2.4s
  merge cost increases from 0.6s to 1.1s

model evidence:
  global atomic writes one hot shared cache line
  partial writes per-worker local lines
  final merge is sequential read-mostly

counter or fallback evidence:
  thread scaling improves under hot-key input
  padded partial beats compact partial under multi-core input
  unavailable PMU events are recorded as Unavailable, not zero

```

第五步：写出不能说的部分 好的性能判读必须写边界。若只在 x86-64 台式机上测过，不能宣称所有 ARM 移动设备都相同；若只测热 page cache，不能说明冷启动端到端；若 PMU 不可用，不能声称 cache miss 已经下降；若只测整数计数，不能迁移到浮点归约；若没有故障注入，不能证明 manifest 恢复路径不变。边界不是削弱结论，而是防止局部证据被错误使用。

最后的结论可以这样写：在给定机器、给定输入矩阵和热 page cache 条件下，thread-local histogram 保持 reference 语义，主要把 histogram 阶段从共享写和字符串哈希路径转成 worker-local 整数计数；端到端中位数从 30 秒降到 10 秒，收益集中在 histogram 阶段；热点 key 输入下扩展曲线支持 cache line 所有权竞争解释；坏行密集输入通过 diagnostics checksum；冷读、ARM 设备和故障恢复路径尚未验证。这样的结论才配得上“三倍加速”四个字。

5.11 工业工具和源码阅读

Linux 工具链很强，但不能把它写成命令大全。工具要按问题选择。单核热循环可从 `perfstat`、`perfrecord`、反汇编和编译器优化报告开始；调度问题看线程状态、context switches、`perfsched` 或应用 trace；锁问题看等待时间、futex 路径和临界区；I/O 问题看 page faults、write/fsync 时间、队列水位和设备指标；分布式问题看 request id、attempt、deadline、重试和提交 timeline。

perf 源码阅读入口 源码阅读可以从 `tools/perf/` 的事件选择和输出路径进入，再对照 `kernel/events/core.c` 中 perf event 的创建、权限检查和计数路径。阅读目标不是背函数，而是回答三个问题：用户态工具怎样请求事件；内核怎样判断权限和事件支持；为什么某些事件在当前设备不可用。报告要把源码路径和一个实际实验联系起来。

证据等级 性能报告应区分三层证据。第一层是直接测得的结果，例如 wall time、checksum、样本、stage time。第二层是工具和计数器提供的线索，例如 branch-misses、cache-misses、context-switches。第三层是从源码、反汇编或模型推导的解释。三层证据互相支持时，结论较强；只有第三层时，结论必须保守。

源码阅读任务

源码阅读任务：选择 `perfstat` 的一次实际运行，保存命令和输出。阅读用户态 `tools/perf/` 中事件解析或输出路径，再阅读内核 perf event 的权限或创建路径。报告必须回答：本次实验哪些事件可用，哪些不可用；不可用的原因来自权限、硬件、内核还是容器；若不可用，项目如何降级为阶段时间、输入阶梯或应用 trace。

5.12 实现边界：测量对象

Compute Systems Engine 在这里形成三个对象：`BenchmarkCase` 描述实验设计，`BenchmarkSample` 描述一次运行结果，`StageReport` 描述阶段成本。代码实践卷负责完整实现；原理部分解释这些字段为什么存在。

最小对照组 至少保留四组对照。第一，顺序归约、多累加器归约、线程本地 `partial` 归约，覆盖小输入、中输入、大输入和线程数扫描。第二，热原子计数器、`mutex` 计数器、`thread-local histogram`，观察共享写和同步等待。第三，`read` 后端和 `mmap` 后端，比较冷读、热读、`page fault` 和错误生命周期。第四，一个坏结论审稿练习，把不可复查的“三倍加速”改写成完整证据链。

移动和受限环境 若在容器、虚拟机、云沙箱或普通用户权限下运行，报告要记录可见 CPU、是否能设置 `affinity`、是否存在配额或温度风险、哪些 `perf` 事件不可用。无法读取 PMU 不是失败，但必须降级：使用多轮 `wall time`、输入规模阶梯、`checksum`、阶段时间、反汇编和源码解释。不能把推测写成硬件事实。

从证据到改法 测量不会直接给出改法，它给出瓶颈位置、证据强度和被排除的解释。若字节账本显示冷字段被大量搬运，就检查数据布局；若反汇编和计数器显示分支和指令供给问题，就检查 `SIMD`、`ILP` 和热路径形状；若扩展曲线显示同步热点，就检查线程、锁、原子和无锁结构；若阶段报告显示 `write` 和 `commit` 长尾，就检查异步 I/O 和背压；若 `trace` 显示 `partition` 长尾和重试，就检查分布式提交边界。每一次优化，都应回到同一套证据链。

5.13 PMU 指标的因果表：从数字回到结构

性能计数器最危险的用法，是看到一个数字高就直接下结论。正确做法是把指标放回硬件结构、代码区域和对照实验。下面这张表给出本书后续反复使用的读法。

指标或派生量	第一层含义	必须补的上下文
<code>cycles</code>	CPU 周期消耗	频率、是否被调度走、计时边界
<code>instructions</code>	<code>retired</code> 指令数量	CPI、是否优化掉工作、是否路径改变
<code>IPC</code>	每周期退休指令数	前端/后端/内存瓶颈不能只靠 <code>IPC</code> 判定
<code>branches</code>	分支数量	分支是否在 <code>hot loop</code> ，是否可预测
<code>branch-misses</code>	分支预测失败线索	输入分布、分支位置、错误路径
<code>cache-misses</code>	某级 <code>cache</code> 未命中线索	<code>miss</code> 是否有用、字节账本、访问模式
<code>dTLB-load-misses</code>	地址翻译压力线索	<code>page size</code> 、工作集、随机访问
<code>context-switches</code>	调度干扰或阻塞线索	线程状态、锁/I/O/睡眠路径
<code>cpu-migrations</code>	线程位置变化	<code>affinity</code> 、 <code>NUMA</code> 、 <code>cache</code> 热度丢失

`IPC` 低不自动等于“代码差”。内存带宽受限的流式扫描，`IPC` 可能低；分支错误多的 `parser`，`IPC` 也可能低；系统调用和调度干扰也会让 `IPC` 难看。反过来，`IPC` 高也不代表程序接近最优：一个无用循环可能退休很多简单指令。PMU 只能和 `correctness`、字节账本、阶段拆分、反汇编一起解释。

派生指标比原始数字更接近模型 原始 `cache-misses=1000000` 没有独立意义。更有用的是每处理一个元素多少 `miss`、每个 `miss` 带回多少有效字节、`miss` 增加

是否对应运行时间增加、改变 layout 后 miss 是否随预期变化。例如矩阵按列扫描若每次只使用 cache line 中一个 double，有效利用率只有 8/64=12.5%。这比“cache miss 很多”更能指导改法。

Listing 5.29 报告中更有解释力的派生字段

```
cycles_per_element
instructions_per_element
bytes_moved_per_element
misses_per_ki_elements
branch_miss_rate
tlb_misses_per_mib
effective_bandwidth_gib_s
attained_gflop_s
arithmetic_intensity_flop_per_byte
```

Top-down 不是口号 许多现代 CPU 可以把周期粗分成 frontend bound、bad speculation、backend bound 和 retiring。即使当前设备没有完整 top-down 事件，也可以用同样思路组织证据。前端问题看 I-cache、指令体积、分支目标和 decode；错误推测看 branch miss 和输入分布；后端问题再分内存、端口、依赖链；retiring 高说明大量周期在提交有用或至少可退休的指令，但仍要看这些指令是否必要。

分类	常见原因	对照实验
frontend bound	代码体积、I-cache、decode、间接跳转	缩小热路径、减少模板膨胀、看反汇编
bad speculation	分支难预测、错误路径多	构造可预测/随机输入对照
backend memory	cache/TLB/DRAM/NUMA 等待	改布局、改变工作集、测带宽下限
backend core	端口、依赖链、除法、shuffle	多累加器、改指令形态、看 uops
retiring	指令被顺利提交	检查是否做了多余工作

不能用 PMU 替代 reference 如果候选版本指令数下降 40%，但 checksum 也变了，这不是优化。若候选版本 cache miss 下降，因为它跳过了错误记录或少写了 manifest，也不是优化。性能证据永远排在 correctness 证据之后。报告结构应先写 reference/oracle/checksum，再写时间和 PMU。

PMU 判读树：从症状到验证实验

PMU 指标最适合做假设生成，不适合单独当判决。一个可复用的判读树应该先问阶段，再问数量级，再问对照实验。下面这张表是本书后续性能报告的最小判读框架。

症状	候选瓶颈	验证实验
IPC 低、LLC miss 高	DRAM 或 LLC miss 等待	改工作集大小、顺序/随机对照、带宽下限
IPC 低、branch miss 高	错误推测	构造可预测输入、拆热分支、看错误路径占比
IPC 低、指令少但周期多	长延迟依赖链或内存等待	多累加器、prefetch、load-use 距离对照
instructions 高、IPC 正常	多做了工作或前端压力	反汇编、阶段拆分、去掉 diagnostics 对照
dTLB miss 高	页覆盖或随机访问	huge page、压缩对象、按页局部性分组
context switch 高	阻塞、抢占或同步等待	绑定、减少线程数、记录等待谓词
cycles 波动大	频率、调度、热状态、I/O	固定输入、多轮样本、温度/频率记录

这张表的重点是“验证实验”。例如 IPC 低不能自动归因到内存；如果把循环改成多累加器后显著变快，可能是依赖链限制；如果随机输入变慢而顺序输入不慢，才更像地址局部性；如果减小工作集到 L2 后变快，说明层级位置影响明显。

Top-down 的四类周期怎样落到代码

Top-down 分类常把周期分成 retiring、bad speculation、frontend bound、backend bound。入门时可以把它们映射成四个问题：

1. 退休了很多有用指令吗？如果是，先查算法和多余工作。
2. 取错路了吗？如果是，查分支输入分布和错误路径。
3. 指令供不上吗？如果是，查 I-cache、decode、间接跳转和代码体积。
4. 后端等资源吗？如果是，再拆内存、端口、依赖链、除法和同步。

Listing 5.30 一个热循环报告的 top-down 叙述模板

```
stage: parse_hot_loop
correctness: checksum matched reference
time share: 72% of end-to-end
top-down:
  bad speculation high on random input
  frontend acceptable after code-size audit
  backend memory moderate, not bandwidth-saturated
evidence:
  predictable input branch_miss_rate drops
  branchless version reduces p95 but adds instructions
decision:
  keep branchy path for common input
  add branchless backend for adversarial input bucket
```

注意最后的 decision。性能分析不是为了把所有指标变漂亮，而是为了根据输入分布选择工程策略。一个 parser 对常见日志格式分支预测很好，对恶意随机输入很差，可能需要分 bucket，而不是无条件改成更重的 branchless 路径。

一份成功 PMU/Top-Down 报告应该长什么样

Top-Down 采集成功时，报告不能只贴一屏 `perfstat`。它至少要证明四件事：事件确实可用，采样窗口覆盖了正确阶段，计数器和输入规模归一化，Top-Down 分类能被反汇编或对照实验支持。下面是本书建议的成功报告结构。

Listing 5.31 PMU/Top-Down 成功报告字段

```

environment:
  cpu_model
  microarchitecture_if_known
  kernel_version
  perf_version
  native_or_container
  frequency_policy
  affinity_policy

command:
  build command and commit
  perf stat command
  event list or metric group
  repetitions and warmup policy

workload:
  stage name
  input shape and checksum
  bytes moved and operations
  reference checksum

raw counters:
  cycles
  instructions
  branches
  branch-misses
  cache and TLB events if available
  top-down slots or metric percentages if available

derived:
  ipc
  cycles_per_element
  instructions_per_element
  branch_miss_rate
  effective_bandwidth
  attained_gflops
  top-down interpretation

evidence boundary:
  multiplex status
  unavailable events
  uncore/kernel/user filtering
  unsupported conclusions

```

在支持 Top-Down 的 x86-64 Intel 环境中，命令可能长这样：

Listing 5.32 Top-Down 成功采集的命令形态

```

1 taskset -c 2 ./bench_reduce --case stride1 --size 268435456 \
2   --warmup 3 --repeat 20 --csv out.csv
3

```

```

4 perf stat -r 10 -- \
5   ./bench_reduce --case stride1 --size 268435456 --repeat 1
6
7 perf stat -r 10 --topdown -- \
8   ./bench_reduce --case stride1 --size 268435456 --repeat 1

```

不同 `perf` 版本和 CPU 暴露的 Top-Down metric 名字不完全相同。有的平台使用 `--topdown`，有的平台需要从 `perflist` 中选择 TMA 或 topdown metric group。报告必须保存实际命令和 `perflist` 证据，不能把别的机器上的事件名复制过来。

Top-Down 结果	可以优先怀疑	必须继续验证
frontend bound 高	I-cache、decode、uop cache、间接跳转	热循环反汇编、代码体积、分支目标
bad speculation 高	分支输入难预测、错误路径混入	all-true、all-false、random 输入矩阵
backend memory 高	L1、L2、LLC、DRAM、TLB、NUMA 等待	工作集阶梯、stride、带宽 roof
backend core 高	端口、依赖链、shuffle、除法、store buffer	多累加器、指令替换、uop/端口审计
retiring 高	指令能顺利提交	是否多做工作，是否语义缩水

这张表有意把“结果”和“结论”分开。Top-Down 只能告诉第一批嫌疑人。若 `backendmemory` 高，但把工作集缩到 L2 后时间不变，就不能继续说 DRAM 是根因；若 `badspeculation` 高，但随机输入和真实输入没有差异，可能是错误路径布局或间接跳转，而不只是某个 `if`。

当前受限环境的真实降级证据

很多容器、云沙箱、虚拟机和普通用户环境即使 `perf` 存在，也拿不到 PMU。一次真实受限输出可能是：

Listing 5.33 PMU 不可用时的输出形态

```

Error:
No supported events found.
Access to performance monitoring and observability operations is limited.
perf_event_paranoid setting is -1

```

这段输出的读法很重要。`perf_event_paranoid=-1` 表面上放开了很多权限，但容器能力、内核配置、平台 PMU 支持或用户空间环境仍可能让事件不可用。此时报告可以继续做性能分析，但必须降级：

缺失证据	可替代证据	结论边界
cycles/instructions	wall time、CPU time、反汇编、输入阶梯	不能声称 IPC 改善
cache miss	工作集大小、stride 对照、有效带宽	不能声称 miss 下降
branch miss	可预测/随机输入、CFG、反汇编	只能说符合分支假设
Top-Down	Roofline、阶段时间、源码对照、trace	不能写 frontend/backend 百分比
NUMA/uncore	线程绑定、first-touch、扩展曲线	不能证明 remote HITM

降级报告不是低质量报告。低质量是把缺失事件填成 0，或把“符合假设”写成“已经证明”。如果 PMU 不可用，一份合格报告仍应保存：原始错误输出、环境记录、反汇编摘要、输入矩阵、checksum、阶段时间、字节账本和未验证边界。

端口压力和 uops：当 backend 不是内存

若数据都在 L1，branch miss 很少，但周期仍高，就要看核心执行资源。现代核心有多个执行端口，不同指令占用不同端口；load、store、整数 ALU、浮点 FMA、shuffle、branch、除法等资源不一样。没有具体 uops 工具时，也可以用指令形态做保守判断。

代码形态	可能资源	改写方向
每元素多次 shuffle	shuffle/permute 端口	改布局，减少跨 lane 重排
单累加变量归约	加法依赖链	多累加器，树形归约
大量整数除法/取模	长延迟除法单元	strength reduction、容量取 2 的幂
load 后立刻使用	load-use 延迟	展开，提前 load，软件预取
store 很多且写分散	store buffer、写合并差	批量写、连续输出、分阶段 compact

向量化并不只增加 lane；它也可能增加 shuffle、mask、tail 和寄存器压力。若 bottleneck 是 shuffle 端口而不是算术吞吐，换更宽向量未必改善。

性能报告里的反证输入

一个结论至少要有反证输入。若你说“瓶颈是内存带宽”，要有小工作集输入证明数据进 cache 后变快；若说“瓶颈是 branch miss”，要有可预测输入证明 branch miss 和时间一起下降；若说“瓶颈是 TLB”，要有 huge page、页内连续或压缩对象的对照；若说“瓶颈是锁竞争”，要有单线程、分片、thread-local 或无共享写版本。

Listing 5.34 反证输入矩阵

```
memory hypothesis:
  tiny_in_L1
  fits_L2
  exceeds_LLC
  streaming_large
  random_large

branch hypothesis:
  all_true
  all_false
  periodic
  random
  real_trace

contention hypothesis:
  one_thread
  per_thread_partial
  padded_partial
  global_atomic
```

没有反证输入的报告很容易变成故事。经典性能教材厚，不是因为术语多，而是因为每个判断都留下“如果我错了，哪个输入会推翻我”的入口。

5.14 测量报告的反面案例

坏报告最常见的形式是：“优化后耗时从 12 秒降到 4 秒，提升三倍。”这句话缺少语义、输入、环境、统计、边界和原因。它可能是真的，也可能来自 page cache 预热、少写了 manifest、跳过了坏行、线程数不同、编译选项不同、CPU 频率不同、输出校验关闭，或者只挑了最好的样本。测量章要建立的不是写报告格式，而是把这种句子拆到无法藏错。

把一句话拆成可复查事实 同一个结论应至少改写成：在同一 commit、同一输入 checksum、同一 reference checksum、同一编译命令、热 page cache 状态下，候选版本在 20 轮测量中中位数从 12.1 秒降到 4.0 秒；parse 阶段不变，histogram 阶段下降；坏行 diagnostics checksum 一致；write+manifest 阶段仍被计入；PMU 事件不可用，因此 cache miss 下降只是推断，不能写成事实；冷启动、ARM 平台和故障恢复尚未验证。这样的句子变长了，但每个部分都能被检查。

Listing 5.35 坏报告到好报告的字段展开

```
bad:
  3x faster

checkable:
  same input checksum
  same reference checksum
  same commit and build flags
  same timing boundary
  passed samples only
  median/p95 and raw samples saved
  stage movement explained
  unverified boundaries listed
```

证据缺口要显式留下 无法使用 perf、无法控制 CPU 频率、无法清空 page cache、无法设置 affinity，都不是失败；把这些缺口伪装成已验证事实才是失败。受限环境中的报告仍然可以有价值：用阶段时间、输入阶梯、反汇编、checksum、trace 和反证输入支持保守结论。证据等级越低，结论越窄；这条规则比任何具体工具都重要。

把测量变成持续回归门 性能报告不应只在优化完成时写一次。项目应保留一组小而稳定的回归 benchmark：一个流式读写内核，一个 histogram 热点输入，一个坏行密集 parser，一个可靠写出场景，一个线程池阻塞场景。每次改动至少记录 checksum、阶段时间和关键资源指标。阈值不必过度敏感，可以先用“超过历史中位数 15% 且连续多轮出现”触发人工复查。这样性能问题会在改动附近暴露，而不是在几章之后变成无法定位的慢。

回归门还要保存失败样本。一次性能失败如果只打印“超过阈值”，没有输入 seed、机器状态、阶段拆分和原始样本，就无法复现。性能回归和正确性回归一样，都需要最小可重放证据和审计记录。

5.15 状态机：一次性能结论怎样从假设走到验收

性能分析也应该有状态机。否则报告很容易从“观察到变快”直接跳到“知道原因”。本书建议把每次优化结论固定成下面的状态转移：

状态	进入条件	退出条件
Observation Hypothesis	看到耗时、吞吐或延迟变化 提出一个可证伪瓶颈解释	保存原始样本和环境 写出会推翻它的输入或指标
Model	建立 FLOPs、bytes、同步或排队账本	数量级和实测不矛盾
Experiment Evidence	运行单变量对照和反证输入 阶段、计数器、反汇编或 trace 支持假设	correctness 全部通过 替代解释被逐项排除
Decision	写出上线、保留、回退或继续验证	边界和回归门明确

这台状态机能防止两类常见错误。第一类是“测到了变化但没有模型”：比如三倍加速，却不知道是 I/O、parser、histogram 还是写出阶段变化。第二类是“有模型但没有反证”：比如声称 cache miss 下降，却没有输入阶梯、layout 对照或计数器证据。

数学模型：Roofline 只是下限，不是预言

对一个 kernel，最基本的下限是：

$$T \geq \max \left(\frac{\text{bytes}}{BW_{\text{sustained}}}, \frac{\text{ops}}{\text{Throughput}_{\text{sustained}}} \right)$$

但真实程序还要加上固定成本和系统成本：

$$T_{\text{observed}} = T_{\text{kernel}} + T_{\text{launch}} + T_{\text{sync}} + T_{\text{alloc}} + T_{\text{io}} + T_{\text{noise}}$$

因此 Roofline 的正确用法不是预测精确耗时，而是做数量级审计。若模型说流式读写至少 80ms，报告声称 5ms 完成，就要先怀疑 bytes 口径、cache 热度、计时范围或编译器删除。若模型说算力上限 1000GFLOP/s，实测 50GFLOP/s，则要查访存、依赖链、向量化、分支、线程和调度，而不是直接说“机器不行”。

PMU 判读的反例矩阵

同一个 PMU 现象可能有多个原因。下面的矩阵用于训练“不要只看一个指标”。

现象	可能原因	反证实验
IPC 低	内存等待、分支错误、长依赖链、调度干扰	固定输入、减少工作集、看分支输入、检查 context switch
cache miss 高	工作集大、stride 差、无用预取、随机访问	改连续输入、缩小到 L2、计算 cache line 利用率
branch miss 高	输入随机、错误路径混入、状态机分支复杂	all-true/all-false/周期输入对照
instructions 高	多做工作、未内联、未向量化、诊断路径进入热区	反汇编、阶段计时、关闭诊断对照
cycles 高 但 PMU 不完整	频率、抢占、I/O、page fault、权限限制	trace、fault 计数、raw samples、环境记录

这张表要求每个结论至少有一个“替代解释排除”。如果不能排除，就把结论写窄：不是“branch miss 是根因”，而是“随机输入下 branch miss 与阶段耗时同时上升，证据支持分支预测参与”。

工程事故：把 debug 日志留在 timed region

一个常见事故是优化版本看起来变慢，最后发现候选代码在热循环中保留了 debug 计数和日志格式化。日志在默认级别下不输出，但字符串拼接、状态检查、原子计数或分支仍在计时范围内。修复不是简单删除日志，而是把观测点移到阶段边界，热路径只写低成本 counters，采样或批量导出。

Listing 5.36 测量事故复盘模板

```
incident:
  candidate slower by 18% in hot-key benchmark

semantic:
  checksum equal
  diagnostics equal

stage:
  parse unchanged
  histogram slower

binary evidence:
  extra formatting call remains in hot loop
  branch to logging guard is executed per record

fix:
  move diagnostic string construction out of hot loop
  keep numeric counters only
  add regression check for instructions_per_record
```

实验：用反证输入训练 PMU 判读

习题与作业

实验一：选择一个分支密集循环，构造 all-true、all-false、周期、随机和真实 trace 五种输入。记录 branch miss rate、cycles/item、checksum。说明哪种输入推翻“算法本身慢”的解释。

实验二：选择一个 stride 访问循环，构造 stride=1、cache line、page、随机四种访问。记录有效字节利用率、TLB 替代证据和 wall time。用公式说明为什么每次只用 8/64 字节会浪费带宽。

实验三：写一份“证据不足”的性能报告，然后按 Observation -> Hypothesis -> Model -> Experiment -> Evidence -> Decision 状态机补全。最终报告必须列出至少两个未验证边界。

5.16 性能报告门禁和习题

复查必须覆盖六项事实：reference、oracle、checksum、错误摘要和边界输入是否完整；端到端、阶段和热循环是否分清，预热、冷启动和计时范围是否写明；输入矩阵是否能证伪假设，单变量对照、运行轮次和统计方法是否合理；字节账本、操作数账本和 Roofline 判断是否有数量级依据；计数器、profile、trace 或降级证据是否能支持瓶颈解释，并写出不可用边界；原始命令、输入、样本、环境和报告是否可追溯。

习题与作业

习题一：为顺序归约、多累加器归约和线程本地 `partial` 归约设计对照判读表。要求列出输入规模、线程数、预热轮数、测量轮数、计时范围、正确性校验、字节账本和需要记录的计数器。每个变量都要说明用于排除哪一种错误解释。

习题二：写一份报告解释为什么热原子归约即使使用 `memory_order_relaxed` 也可能扩展性很差。报告必须同时使用 C++ 语义、cache line 所有权、线程数扩展曲线和对照实验四类证据。

习题三：设计一个队列 benchmark。区分 SPSC、MPSC 和 MPMC 三种竞争模式，记录吞吐、队列长度、push 等待、pop 等待和 P95/P99。说明为什么单生产者单消费者结果不能证明一个队列适合线程池全局队列。

习题四：在当前设备上运行或设计 `perfstat` 可用性检查。记录哪些事件可用，哪些因权限、平台或容器不可用。对不可用事件，写出替代证据方案。

习题五：把一句“不完整报告：优化后三倍加速”改写成完整性能报告摘要。必须包含问题、reference、输入、环境、命令、样本、统计、字节账本、计数器或降级证据、反证输入和未验证边界。

字节账本和阶段报告会直接决定是否应该做冷热拆分、列式 batch、selection vector、padding、预取或分块。没有可信测量，数据布局优化只是猜；有了证据链，布局改造才有进入条件。

工程案例

知识点小案例：三倍加速为什么还不是性能结论。特例是一句报告：“优化后从 30 ms 变成 10 ms”。这句话缺少问题、输入、环境、预热、计时范围、正确性校验和字节账本。若 30 ms 包含文件读取而 10 ms 只测热循环，比较无效；若优化版漏掉错误行，速度更快也不能接受；若输入只放在 L3 里，不能推出 DRAM 场景也快。由此推出规律：性能数字只有放进 Observation、Hypothesis、Model、Experiment、Evidence、Decision 状态机才有意义。工程上每个 benchmark 至少带 checksum、输入规模、阶段计时、样本统计和未验证边界。

第 6 章

PMU、Top-Down 与 SIMD 资源账本

6.1 从耗时数字走向硬件归因

上一章建立了可信测量和 Roofline 方法，但真实性能工程还需要继续下沉：当一个 kernel 慢时，慢在前端取指、分支错误、执行端口、load/store、cache miss、TLB miss、内存带宽，还是调度等待？单个 elapsed time 无法回答。PMU 和 Top-Down 方法的价值，是把“慢”拆成更接近硬件结构的证据。

Top-Down 不应被理解成一组固定百分比截图，而是一棵诊断树。它先问每个 pipeline slot 最终归到哪里：retiring、bad speculation、frontend bound、backend bound。然后继续下钻：backend bound 是 core bound 还是 memory bound；memory bound 是 L1、L2、LLC、DRAM、DTLB 还是 store 相关等待；core bound 是端口压力、除法、shuffle、FMA、load/store 单元还是寄存器依赖。

Listing 6.1 Top-Down 诊断树的报告形态

```
topdown:
  level_1:
    retiring
    bad_speculation
    frontend_bound
    backend_bound
  level_2:
    frontend_latency
    frontend_bandwidth
    core_bound
    memory_bound
  drilldown:
    branch_miss
    icache_or_itlb
    l1_l2_llc_miss
    dtlb_miss
    port_pressure
    store_buffer_or_load_block
```

如果环境没有权限采集这些事件，报告必须降级：可以写“无法证明 IPC 改善”，不能把 wall time 变化解释成 IPC 改善；可以写“耗时下降且字节账本不变”，不能声称 LLC miss 下降。严肃的性能书必须教会读者区分可证据支持的结论和合理猜测。

6.2 PMU 事件到瓶颈类型的映射

PMU 事件名称随架构变化，但诊断逻辑稳定。下面的表不是某个平台的完整事件清单，而是报告里的解释模板。

观察	可能解释	下一步验证
IPC 低、cache miss 高	内存等待或工作集过大	改工作集、stride、prefetch、带宽账本
IPC 低、branch miss 高	分支预测失败	构造可预测/随机输入，对照分支消除
IPC 低、L1 命中但周期高	core bound 或依赖链	看端口压力、uops、latency、反汇编
instructions 大幅增加	算法或编译路径变了	对照汇编、循环体、函数调用和 tail
cycles 高 但 CPU time 低	可能有睡眠或抢占	看 context switch、off-CPU、trace
LLC miss 低但带宽高	streaming 命中预期	Roofline 字节账本和内存带宽
DTLB miss 高	页面跨度大或随机访问	huge page、blocking、访问顺序、页数估计

事件不是定罪证据。比如 LLC miss 高可能是坏事，也可能是 streaming workload 的正常结果；IPC 高也不一定端到端快，可能做了更多无用指令。每个解释都要回到语义、字节、操作数和反例输入。

6.3 完整 PMU 报告模板

高质量 PMU 报告至少包含下面字段：

Listing 6.2 PMU/Top-Down 报告字段

```
pmu_report:
  environment:
    os_kernel
    cpu_model
    microarchitecture_if_known
    perf_or_profiler_version
    permission_status
    frequency_governor
    affinity_policy
    smt_status
  binary:
    compiler
    flags
    target_cpu
    symbol_under_test
    objdump_path
  workload:
    input_shape
    input_bytes
    operations
    checksum
    warmup
    repetitions
  counters:
    cycles
    instructions
    branches
    branch_misses
    cache_references
```

```
cache_misses
l1_l2_llc_if_available
dtlb_if_available
topdown_if_available
derived:
  ipc
  cycles_per_element
  bytes_per_second
  flops_or_ops_per_second
  arithmetic_intensity
conclusion:
  supported_claims
  rejected_claims
  evidence_boundary
```

报告里最重要的是 `rejected_claims`。如果没有 `branch event`，就不能写“分支预测改善”；如果没有 `cache event`，就不能写“缓存 miss 降低”；如果没有反汇编，就不能写“AVX2 路径被使用”。工程上，知道不能证明什么，和证明了什么一样重要。

6.4 SIMD 资源账本：宽寄存器只是起点

SIMD 不是“一个指令处理 8 个 float 所以快 8 倍”。一个 AVX2 `ymm` 寄存器确实能放 8 个 `float`，但 `kernel` 还要读取内存、做地址计算、处理 `tail`、归约 `lane`、可能 `shuffle`、写回结果。真实瓶颈可能是 `load/store` 端口、`FMA` 端口、`shuffle` 端口、前端 `uop` 带宽、寄存器压力、数据依赖或内存带宽。

资源	典型成本	常见误判
load/store	每个向量仍要搬字节	只数 FLOPs，不数 bytes
FMA/ALU	throughput 和 latency 不同	把延迟当吞吐
shuffle/permute	lane 重排可能很贵	忽略 layout 对指令的影响
horizontal	跨 lane 合并需要额外步骤	以为向量加完就得到标量
reduce		
gather/scatter	地址分散，吞吐常差	把它当普通 load/store
mask/tail	AVX-512 mask 方便但不免费	认为 tail 完全无成本
前端 uops	宽向量指令也要解码和发射	只看寄存器位宽

6.5 dot product 的四级资源账本

以 `dot product` 为例。标量 C++ 看起来很简单：

```
1 float dot(const float* a, const float* b, int n)
2 {
3     float s = 0.0f;
4     for (int i = 0; i < n; ++i) {
5         s += a[i] * b[i];
6     }
7     return s;
8 }
```

从资源账本看，每个元素至少读 `a[i]` 和 `b[i]` 两个 `float`，也就是 8 字节，做一个乘法和一个加法，约 2 FLOPs。算术强度约为 $2/8 = 0.25$ FLOP/byte。若数据远超 `cache`，内存带宽很容易成为上限。

标量路径

标量路径每轮通常有两个 load、一个乘、一个加、一个 loop branch。若编译器不重关联，累加变量形成长依赖链：下一次加法依赖上一次结果。即使所有数据在 L1，add latency 也会限制吞吐。多累加器可以把依赖链拆开，让 CPU 同时推进多个 partial sum。

AVX2 路径

AVX2 每次处理 8 个 float。热循环可能形如：

Listing 6.3 AVX2 dot 的资源账本形态

```
loop:
    load 32B from a
    load 32B from b
    fused multiply-add into vector accumulator
    advance pointers
tail:
    handle n % 8
reduce:
    horizontal reduce vector lanes
```

这减少了循环控制开销，并提高了乘加吞吐，但每 8 个元素仍要读 64 字节。若数据来自 DRAM，带宽上限仍然决定最终吞吐。若数据在 L1，FMA 端口、load 端口和 reduction 成本更重要。

AVX-512 路径

AVX-512 每个 zmm 可以处理 16 个 float，并能用 mask 处理 tail。优势是更宽、更自然的 mask 和一些更强的整数/低精度指令；代价是可能增加寄存器压力、代码体积、前端压力，并在某些平台触发频率变化。报告不能只写“AVX-512 更宽”，必须保存实际路径、频率状态、吞吐和端到端收益。

VNNI/AMX 与 int8 dot

AI 推理常见的 int8 dot 不只是把 float 换成 int8。VNNI 把多个低精度乘加合并到更少指令，AMX 进一步引入 tile 资源。它们的资源账本包括：码点读取、scale metadata、累加器宽度、requantization、tile 配置、tile load/store、对齐和 fallback。若模型格式是 int4，还要加 nibble 解包和 group scale 读取。低比特的瓶颈常从“读权重字节”转移到“解包、scale 和指令支持”。

6.6 反汇编证据：证明 fast path 真的执行

SIMD 报告必须证明二进制里有目标指令，还要证明运行时真的走到这条路径。仅仅打开 -mavx2 不够；编译器可能因为别名、对齐、浮点语义或目标函数不可见而未向量化。仅仅在文件里找到 vaddps 也不够；运行时 dispatch 可能进入 scalar fallback。

Listing 6.4 SIMD 证据字段

```
simd_evidence:
    requested_backend: scalar / auto / avx2 / avx512
    compiled_flags
    cpu_feature_detection
    dispatch_decision
    symbol_name
    objdump_excerpt
```

```
vector_instructions_seen
tail_path_seen
forced_backend_test
fallback_test
```

如果输入长度很短，dispatch 可能故意选择 scalar；如果指针不满足对齐，kernel 可能转入 unaligned path；如果 CPU 不支持目标 ISA，程序必须安全 fallback。所有这些都要进入测试矩阵。

6.7 端口压力和 uop 思维

现代核心有多个执行端口。load、store、FMA、整数 ALU、branch、shuffle 和除法不是使用同一资源。一个 kernel 即使没有 cache miss，也可能因为某类端口饱和而慢。比如大量 shuffle 的 AoS 到 SoA 转换可能受 shuffle/permute 限制；大量 gather 可能受内存地址生成和 load 端口限制；horizontal reduction 可能受跨 lane 合并和依赖链限制。

报告不要求读者背每代微架构端口表，但要形成资源账本：

Listing 6.5 端口压力账本

```
per_vector_iteration:
  loads: count and bytes
  stores: count and bytes
  fma_or_alu_ops: count
  shuffles_or_permutes: count
  branches: count
  address_generation: simple / indexed / gather
  dependency_chains:
    accumulator
    pointer_updates
    reduction
  likely_bottleneck:
    memory_bandwidth / load_ports / fma_ports / shuffle / frontend
```

这个账本的作用是提出可验证假设。若怀疑 shuffle 过重，可以改 layout 或预打包；若怀疑 load 端口，可以减少读流或 blocking；若怀疑前端，可以看代码体积和 uop cache；若怀疑依赖链，可以多累加器或重排归约。

6.8 实验矩阵

实验一：对同一个 dot kernel 生成 scalar、auto-vectorized、AVX2 或平台可用 SIMD 版本。每个版本保存汇编、输入长度、checksum、耗时、cycles、instructions 和可用 cache 事件。

实验二：构造 L1-fit、LLC-fit、DRAM-stream 三类输入。用相同代码比较 IPC、有效带宽和 cycles/element。解释为什么同一 SIMD 优化在不同工作集上收益不同。

实验三：构造需要 horizontal reduction 的 kernel 和纯 elementwise kernel。比较向量化收益，指出 reduction 成本来自哪里。

实验四：构造连续 load 和 gather-like 访问。即使操作数相同，也要报告地址流、cache/TLB 事件和吞吐差异。

验收标准：报告必须包含语义 oracle、反汇编、运行时 dispatch、字节账本、PMU 可用性和未验证边界。没有这些字段的“SIMD 加速表”，不能作为本书的性能证据。

工程案例

知识点小案例：SIMD 版本快了，瓶颈也可能已经换了。特例是 dot product：标量版本每次处理一个元素，AVX2 版本一次处理八个 float。小数组时，函数调用、分支和归约开销可能占主导；L1 内数据时，FMA 端口可能可见；大数组流式读时，内存带宽可能成为上限。由此推出规律：SIMD 不是一个单一结论，而是改变了指令、load、shuffle、归约和带宽账本。工程报告要同时记录反汇编、输入规模阶梯、cycles、instructions、有效带宽和可用 PMU 事件；若 PMU 不可用，也要用字节账本和版本对照说明瓶颈假设。

第 7 章

数据布局、局部性与预取

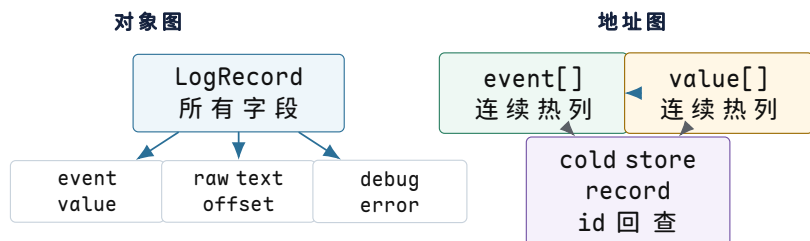
先看一个日志系统事故。最初的 `LogRecord` 很小，只保存时间戳、级别、事件类型和一两个整数值。后来为了排查线上问题，结构体里逐渐加入原始文本、解析状态、字段边界、错误详情、调试标签、文件 `offset` 和若干统计字段。聚合热路径仍然只读取 `event_type` 和 `value`，吞吐却明显下降。算法复杂度没有变，分支也没有变，慢下来的原因是硬件每次搬运的是 `cache line`：热字段旁边塞满冷字段后，每条 `line` 里真正有用的字节比例下降，更多 `line` 被搬进来，`TLB` 覆盖的记录数也减少。

把热字段拆成连续数组，冷诊断信息放到旁路表，通过稳定 `record_id` 回查，聚合吞吐恢复，错误报告能力也没有丢。这个改造不是“面向对象不好”或“列式一定好”，而是一个更底层的事实：CPU 执行 `load/store` 时看到的是地址序列、`cache line`、页和写者，不是类名。数据布局就是把业务对象翻译成硬件更容易消费的地址形状。

7.1 先从 `cache line` 和地址序列推导布局

第 3 章已经说明，处理器通常按 `cache line` 搬运数据。程序读一个 4 字节字段，硬件会把包含它的一整条 `line` 放进缓存；程序跨页随机访问，`TLB` 要为更多页面保存翻译；程序沿连续地址扫描，预取器可以提前请求后续 `line`。布局问题因此不是“字段放在哪里更漂亮”，而是“热路径每处理一个逻辑元素，要搬多少 `line`、跨多少页、能不能提前知道下一地址、有没有不同线程写同一 `line`”。

同一条业务记录的对象图和地址图



对象图表达业务归属；地址图表达热路径实际搬运的字节、页和回查关系。布局优化要保护二者之间的映射。

图示：本图要证明的命题是：布局优化不是丢掉对象语义，而是把热路径需要的字段翻译成更短、更连续的地址流。

访问矩阵先于类型设计 设计布局前先写访问矩阵。列是字段，行是阶段，格子标出读、写、只读配置、冷路径、并发写或错误回查。一个字段是否属于同一个结构体，不应由业务对象名字决定，而应由“哪些阶段同时访问它、访问频率多高、是否同一线程写、生命周期是否一致”决定。

Listing 7.1 日志统计的访问矩阵示例

```
parse stage:
  reads: raw_line
  writes: record_id, event_type, value, parse_status, field_offsets
```

```

aggregate hot path:
  reads: event_type, value
  writes: worker-local counters

error report cold path:
  reads: record_id, raw_line, field_offsets, parse_status

```

这个矩阵自然推出三个布局决定。第一，`event_type` 和 `value` 应该形成连续热列，服务聚合扫描。第二，`raw_line`、字段边界和错误详情不能丢，但可以放在冷存储里，通过 `record_id` 回查。第三，`worker` 高频写的 `counters` 不应和其他 `worker` 的 `counters` 同处一条 `line`。布局是访问模式的结果，不是容器名字的结果。

有效字节比例 假设一个原始 `LogRecord` 是 160 字节，聚合阶段只用 4 字节 `event_type` 和 8 字节 `value`。每处理一条记录，硬件可能搬入三条 64 字节 `line`，热路径只使用其中 12 字节。列式热字段后，一条 64 字节 `line` 可以容纳 16 个 `event_type` 或 8 个 `value`。同样逻辑工作量，数据移动形状完全不同。

布局	聚合热路径看到的地址	典型后果
胖对象数组	每条记录跨多条 <code>line</code> ，热字段间隔大	冷字段被反复搬运，TLB 覆盖下降
热列数组	<code>event_type[]</code> 和 <code>value[]</code> 连续扫描	<code>line</code> 利用率高，预取和 SIMD 更容易
AoSoA 块	块内热列连续，块间保持批次关系	适合 SIMD 宽度和缓存块，尾部处理稍复杂

TLB 覆盖也会被对象大小影响 页面翻译按页工作。若每页 4 KiB，一个 160 字节对象每页只能放大约 25 条记录；若热列只保存 12 字节热数据，每页可覆盖数百条逻辑记录。热循环跨越的页面越多，DTLB 压力越大；随机访问冷字段时，回查路径再承担额外成本。冷热分离的收益不只来自 `cache line`，也来自更好的页覆盖。

地址模式比类名更接近硬件 把一个字段读取写成 C++：

Listing 7.2 聚合只读两个热字段

```

1 for (std::size_t i = 0; i < records.size(); ++i) {
2   local_counts[records[i].event_type] += records[i].value;
3 }

```

若 `records` 是 `std::vector<LogRecord>`，编译器计算 `records[i].event_type` 时必须把对象大小乘进地址。典型形态可以抽象成：

Listing 7.3 AoS 字段访问的地址形态

```

address_of_event = base_records + i * sizeof(LogRecord) + offset_event_type
address_of_value = base_records + i * sizeof(LogRecord) + offset_value
load event_type from address_of_event
load value      from address_of_value

```

若 `sizeof(LogRecord)=160`，相邻两次 `event_type` `load` 的地址相差 160 字节。CPU 看到的是一个大步长流，而不是“同一类对象”。即使预取器识别出固定步长，每条 `cache line` 里也只用少数字段，TLB 每页覆盖的逻辑记录数也很少。若对

象里含有 `std::string`，热循环还可能因为错误统计或调试分支追到额外指针，地址形状进一步散开。
热列版本不同：

Listing 7.4 SoA 热列访问的地址形态

```
address_of_event = base_event_types + i * sizeof(uint32_t)
address_of_value = base_values      + i * sizeof(int64_t)
load event_type from address_of_event
load value      from address_of_value
```

这里 `event_type` 地址每次只前进 4 字节，`value` 地址每次前进 8 字节。硬件可以把后续 line 预取进来，SIMD 也能用一条向量 load 取多个相邻元素。AoS 和 SoA 的区别因此不是“面向对象”和“面向数组”的风格之争，而是热循环里地址表达式从“大对象步长 + 字段偏移”变成了“窄元素步长 + 连续列”。
把 **cache line 账本算到每次迭代** 同一段聚合逻辑在两种布局下的 cache line 账本完全不同。假设 cache line 为 64 字节，`LogRecord` 为 160 字节，热字段总共 12 字节。AoS 扫描 64 条记录时，热路径可能触碰约 160 条 cache line 量级的对象字节；真正使用的热字节只有 $64 \times 12 = 768$ 字节。SoA 扫描同样 64 条记录时，`event_type[]` 需要 256 字节，约 4 条 line；`value[]` 需要 512 字节，约 8 条 line。数量级差异来自 line 利用率，不来自算法复杂度。

布局	每 64 条记录的热路径字节形状	推导结果
AoS 胖对象	约 64×160 字节对象跨度	line 有效字节低，页覆盖低，冷字段被拖入
SoA 热列	$64 \times 4 + 64 \times 8$ 字节热列	连续、短步长、预取和 SIMD 更自然
AoSoA 块	块内按列连续，块间带元数据	可按 SIMD 宽度和 cache 工作集调块宽

这只是估算，不是替代测量。真实机器还会受到对齐、预取、硬件 stream 数量、cache set 冲突、TLB、写分配和下游 partial 写入影响。但估算足以解释为什么“只读两个字段”的热循环会因为对象膨胀而变慢，也能指导下一步实验该看哪些计数器 and 输入规模。
从**字节账本推导布局** 布局改造应从一张字节账本开始，而不是从“换成 SoA”开始。假设聚合阶段只需要 `event_type` 和 `value`，错误报告阶段才需要 `raw_line`、`field_offsets` 和 `parse_status`。原始对象如下：

Listing 7.5 膨胀后的 LogRecord 形态

```
1 struct LogRecord {
2     std::uint64_t timestamp_ns;
3     std::uint32_t user_id;
4     std::uint32_t event_type;
5     std::int64_t value;
6     std::uint64_t file_offset;
7     std::uint16_t field_begin[4];
8     std::uint16_t field_end[4];
9     std::uint32_t parse_status;
10    std::string raw_line;
11    std::string debug_tag;
12 };
```

即使忽略 `std::string` 指向的外部字节，这个对象自身也会包含指针、长度、容量、数组和对齐空洞。聚合热路径真正需要的只有 `event_type` 和 `value`。如果每条记录要跨两到三条 `cache line`，热路径搬入的大部分字节都不会被使用。由此自然推出第一步拆分：把热字段和稳定身份变成连续 `batch`，把冷字段放入可回查存储。

Listing 7.6 从胖对象拆出的热 `batch` 和冷表

```
1 struct HotLogBatch {
2     std::span<const std::uint32_t> record_ids;
3     std::span<const std::uint32_t> event_types;
4     std::span<const std::int64_t> values;
5 };
6
7 struct ColdRecordInfo {
8     std::uint64_t file_offset = 0;
9     std::uint16_t field_begin[4] = {};
10    std::uint16_t field_end[4] = {};
11    std::uint32_t parse_status = 0;
12 };
```

这个拆分保护了两个目标：聚合热路径看到短而连续的地址流；错误报告仍然能通过 `record_id` 找回原始行、`offset` 和字段边界。若改造后无法回查错误，说明布局优化破坏了语义身份；若改造后热路径仍然需要追 `raw_line` 指针，说明冷热边界没有真正落地。

写入路径也要进账本 很多布局讨论只看读取，忽略写入。解析阶段写 `record_ids/event_types/values`，聚合阶段写 `worker-local counters`，错误阶段写 `diagnostics`。写入一个尚未在 `cache` 中的 `line` 时，处理器通常需要取得该 `line` 的独占所有权；这会产生写分配或等价的所有权请求。若多个 `worker` 写相邻小槽位，边界 `cache line` 会在核心间来回转移；若每个 `worker` 写独立、对齐的 `batch` 或 `partial`，写入路径更顺。

阶段	主要写入	布局要求
parse	热列、冷表、诊断列表	每个 worker 写自己的 batch；错误冷路径可追加但需有边界
aggregate	worker-local counters	高频写槽位按 worker 或 NUMA 隔离
filter	selection vector 或 紧凑输出	先 count/scan 再写独占区间
report	稳定输出和 manifest	输出顺序和提交事实分离

写入账本会影响 API。若 `parse worker` 直接把记录 `push` 到全局 `std::vector<LogRecord>`，全局 `size`、`capacity` 和 `allocator` 元数据会成为共享写热点。若 `worker` 先写本地 `batch`，再由 `scan` 或 `manifest` 合并，热路径写入就是连续的。第 13 章的 `scan` 并不是独立算法，它是布局写入路径的自然结果：想让每个 `worker` 连续写，就必须先知道它的输出区间。

7.2 AoS、SoA 和 AoSoA 从哪里来

AoS (Array of Structures) 把一个对象的字段放在一起，适合经常同时处理完整对象的场景。**SoA** (Structure of Arrays) 把同类字段连续存放，适合批量处

理少数字段的场景。**AoSoA** (Array of Structures of Arrays) 把数据按小块组织，块内使用 SoA，块间保持批次关系，常用于适配 SIMD lane、cache line 和尾部处理。

Listing 7.7 AoS、SoA 与 AoSoA 的基本形态

```

1 struct Particle {
2     float x;
3     float y;
4     float z;
5     float vx;
6     float vy;
7     float vz;
8 };
9
10 struct ParticleSoA {
11     std::vector<float> x;
12     std::vector<float> y;
13     std::vector<float> z;
14     std::vector<float> vx;
15     std::vector<float> vy;
16     std::vector<float> vz;
17 };
18
19 constexpr std::int32_t kBlockWidth = 8;
20
21 struct ParticleBlock {
22     std::array<float, kBlockWidth> x;
23     std::array<float, kBlockWidth> y;
24     std::array<float, kBlockWidth> z;
25     std::array<float, kBlockWidth> vx;
26     std::array<float, kBlockWidth> vy;
27     std::array<float, kBlockWidth> vz;
28 };

```

如果热路径总是同时读取 `x/y/z/vx/vy/vz`，AoS 的对象局部性很自然。如果某个阶段只扫全部 `x` 或只读 `event_type`，SoA 更容易形成连续 load、SIMD 和稳定预取。AoSoA 的块宽度不是魔法数字，它来自 SIMD 宽度、cache line、对齐、尾部浪费、块内复用和代码复杂度。

字段共现决定布局 字段共现指多个字段是否总是在同一阶段一起访问。共现强的字段适合放近；共现弱但某个字段非常热，就适合拆出热列。调试字段、错误上下文、原始文本、统计标签通常和主计算不共现，应进入冷路径。只在提交阶段使用的 manifest 字段，不应污染 parse 或 aggregate 热循环。

从字段共现到布局图 字段共现最好不要只用文字判断。可以画一张字段到阶段的矩阵，再把阶段频率、访问类型和写者标出来。若两个字段都在每条记录的聚合热路径中被同一线程只读，它们可以放近；若一个字段每条记录读，另一个字段只在坏行中读，它们不该共享热 line；若两个字段都高频写但写者不同，它们更不该靠近。

字段	parse	aggregate	diagnose	写者
record id	写	读	读	parser worker
event type	写	高频读	不读	parser worker
value	写	高频读	不读	parser worker
raw line	读	不读	冷读	input owner
parse status	写	不读	冷读	parser worker
local count	不读	高频写	汇总读	aggregate worker

这张表直接推出三类存储：热列、冷表、worker-local partial。热列服务顺序读和 SIMD；冷表服务诊断回查；partial 服务本地高频写。若为了“对象完整”把它们重新塞进 `LogRecord`，就等于放弃矩阵给出的底层证据。

同一字段在不同阶段可能需要两种形态 字段不是永远热或永远冷。`record_id` 在解析输出时是热写字段，在聚合时可能只用于 `trace` 或选择向量，在诊断时又成为冷回查入口。`event_name` 在字典编码前是变长字节，在编码后是整数 `id`，在最终报告时又要还原为字符串。布局设计要允许阶段形态转换，而不是强迫一个字段用一种表示贯穿全程。

Listing 7.8 字段形态随阶段变化

```

parse:
    event_name_span -> dictionary lookup

aggregate:
    event_type_id -> local counter index

report:
    event_type_id + dictionary_version -> stable event_name

```

这个转换链要求字典版本、`record id` 和 `checksum` 进入控制面。否则整数 `id` 虽然让热路径变快，却可能让输出身份变模糊。

AoSoA 是可调块，不是妥协口号 完全 SoA 可能让业务对象关系变得分散，完全 AoS 可能浪费带宽。AoSoA 提供块级折中：一个块内的同类字段连续，适合 SIMD；块作为一个单位，方便调度、缓存复用和错误定位。块大小太小，循环层次和元数据成本上升；块大小太大，尾部处理和缓存压力上升。选择块大小要用第 5 章的字节账本和对照判读验证。

块宽度可以从三个底层约束推出来。第一，SIMD lane 数决定一次向量操作自然处理多少元素，例如 256-bit 向量可装 8 个 `float` 或 4 个 `double`。第二，cache line 决定一个字段块跨多少条 line，过小会浪费 line，过大会增加 L1 工作集。第三，调度粒度决定一个 batch 是否足够大到摊薄任务开销，又足够小到避免长尾。由这三个约束得到候选块宽，再用实验选定，而不是先写一个看起来整齐的常数。

Listing 7.9 AoSoA 块宽选择的推导账本

```

candidate block widths:
    SIMD width:      8 x int32 per 256-bit vector
    cache line:      16 x int32 per 64-byte line
    scheduler chunk:  at least thousands of records per task
    cold lookup need: keep block id + row id for diagnostics

try block width = 8, 16, 32
measure: hot scan throughput, tail handling, cache misses, code complexity

```

Selection vector 保护语义身份 过滤和压缩热数据时，不能丢掉记录身份。一个 **selection vector**（选择向量）可以保存有效记录的 **record_id** 或原始位置。热路径只处理紧凑热列，错误报告、稳定排序、join 和输出阶段通过 id 回查。这比复制完整冷对象更省，但要求 id 生命周期稳定。

Listing 7.10 热列和选择向量

```
1 struct HotLogBatch {
2     std::vector<std::uint32_t> record_ids;
3     std::vector<std::int32_t> event_types;
4     std::vector<std::int64_t> values;
5 };
6
7 struct SelectionVector {
8     std::vector<std::uint32_t> selected_rows;
9 };
```

选择向量会增加一次间接访问；若后续阶段只处理少量有效行，这个代价通常值得。若所有行都有效且每行都需要完整记录，选择向量可能只是额外层次。布局结构必须跟输入分布一起评估。

Bitmap、selection vector 和材料化 batch 过滤结果常见三种表示。Bitmap 每个输入元素用 1 bit 表示是否保留，空间小，适合与位运算、popcount 和批量 skip 结合；selection vector 保存被保留下来的行号，适合选择率低或后续仍需回查原始 batch；材料化 batch 直接把保留元素写成新的紧凑热列，适合后续要顺序扫描大量字段。三者没有绝对优劣，取决于选择率、后续算子、字段大小和稳定性要求。

表示	适合场景	主要代价
bitmap	高选择率、批量布尔组合、延迟材料化	后续要做 rank 或扫描 bit，访问字段仍需原 batch
selection vector	低选择率、保留 record id、避免复制大记录	间接访问，index 本身占带宽
materialized batch	后续连续处理多个热字段	写出成本和临时空间，需要 scan 分配位置

这个选择和硬件路径直接相关。Bitmap 让控制信息非常紧凑，但每次真正读取字段还要回到原 batch；selection vector 减少被访问的行数，却引入不连续 load；材料化 batch 增加一次写出，却把后续读变成连续流。性能报告必须把选择率作为输入维度，否则“哪种表示更快”没有意义。

7.3 预取：让未来地址可预测

硬件预取器擅长识别顺序访问和固定步长访问。连续数组扫描时，处理器看到 **base+0**、**base+4**、**base+8** 这样的规律，很容易提前请求后续 line。链表追踪时，下一地址藏在当前节点里，必须等当前 load 返回后才知道，预取器很难提前工作。预取不是独立技巧，而是地址序列可预测性的结果。

最好的预取常来自布局改造 把随机对象压缩进连续数组、把节点按遍历顺序重排、把哈希桶控制字节紧凑存放、把图邻接表改成 CSR，这些改造往往比手写 **prefetch** 更有效。它们让硬件自然看到规律地址流。软件预取只有在地址能提前算出、未来确实会访问、当前有足够独立工作、预取距离合适时才可能有效。

Listing 7.11 预取是否可能有效的判断

```

address known early enough?
future access highly likely?
enough independent work before use?
prefetch distance tuned for latency?
cache pollution acceptable?

```

如果五个问题答不上来，手写预取很可能只是噪声。预取太早会把真正热的数据挤出去，太晚来不及隐藏延迟，预取错误路径会浪费带宽。第 5 章的计数器和输入矩阵应先证明存在内存等待或预取失败，再尝试预取指令。

多路游标可以制造可预取性 单条指针链每一步依赖上一节点，难以预取。若同时处理多条独立链，当前等待链 A 的节点时，可以推进链 B、C、D，并为下一批地址发出请求。这叫增加内存级并行度。布局上，批量化和分块能提供多个独立地址流；算法上，避免单一长依赖链。预取和乱序执行都需要“未来有独立工作”。

软件预取失败的常见原因 软件预取经常失败，不是因为指令无效，而是因为使用条件不成立。第一，预取距离不对：距离太短，数据还没回来就被使用；距离太长，数据回来后被别的访问挤出 cache。第二，地址不确定：分支之后才知道是否访问，错误路径预取会浪费带宽。第三，工作集太大：预取把更多 line 提前拉入，却超过 cache 容量，造成污染。第四，瓶颈并不在内存等待，而在分支、前端、锁或 store buffer。

因此预取实验要写两套代码：布局改造前后各一套。若连续热列版本已经接近顺序带宽，手写预取可能没有收益；若哈希表版本仍然随机，预取也可能因为地址太晚才知道而无效。最稳的优化顺序是先改地址形状，再考虑预取指令。

Listing 7.12 预取实验应记录的字段

```

layout,records,bytes_read,bytes_written,hit_rate,selectivity,
prefetch_distance,cache_misses,dtlb_misses,branch_misses,
elapsed_ns,checksum

```

7.4 生命周期、分配器和容器形状

布局还由生命周期决定。生命周期相近的对象适合放在同一个 arena 或 batch 中，批量释放；生命周期差异很大的对象混在一起，会导致碎片、悬空引用、过早保留冷数据或复杂析构。分配器成本也不只在 `new/delete` 调用处出现：分散节点会降低遍历局部性，分配器元数据占用 cache，跨线程释放会引入同步和远端缓存流量。

Arena 的收益来自生命周期和局部性 Arena 把同一阶段创建、同一阶段销毁的对象放在连续区域里。解析一批日志时，临时 token、字段边界和错误上下文可以随 batch 一起释放；任务运行时中长期存在的 task 不适合放进只服务单个 batch 的 arena。Arena 适合批量释放，不适合精确析构每个对象；对象池适合复用固定形状对象，但可能保留空闲内存。

常见误区

不要把“使用堆”简单等同于慢。真正的问题是生命周期是否清楚、分配是否在热路径、对象是否分散、是否跨线程释放、是否破坏局部性。

生命周期分层 Compute Systems Engine 至少有五种生命周期。第一，记录级临时状态，只在解析一行时存在，最好留在栈或寄存器。第二，batch 级热列，服务一个分块内的聚合、过滤和错误摘要。第三，stage 级局部表，例如 worker-local

histogram。第四，run 级控制面，例如字典、输入版本、schema 和 manifest 草稿。第五，持久化输出，例如 block、checkpoint 和最终 manifest。把不同生命周期的对象混在一个分配器或一个结构体中，会导致释放边界不清、内存峰值升高和冷数据滞留。

生命周期	例子	合适形态
record 临时 batch 局部	当前字段起点、字符状态 event type 列、value 列、 field offset 摘要	栈变量、寄存器、短状态机 arena、连续 vector、 BatchView
stage 局部	worker histogram、 selection vector	worker-local 存储、可复用 buffer
run 全局	字典、schema、输入 generation	不可变快照、版本化句柄
持久化	block、checkpoint、 manifest	文件、checksum、提交协议

这张表能解释很多内存泄漏和峰值问题。若 raw line 被 run 全局对象长期持有，冷数据会跨过本该释放的 batch 边界；若 manifest 草稿放在 batch arena 中，batch 释放后提交路径会悬空；若 worker-local histogram 每个 batch 都重新分配，分配器成本和触页成本会进入热路径。

跨线程释放要进入布局账本 对象在哪个线程分配、在哪个线程释放，也会影响性能。线程本地分配器通常让同线程分配释放很快；跨线程释放可能把对象放进远端队列，修改分配器元数据，并让 cache line 在核心之间迁移。任务系统里，一个 producer 创建 task，consumer 执行并删除 task，是典型跨线程生命周期。若 task 很小且频繁，分配释放路径可能比任务本身还贵。

解决方向包括：固定 owner，由 owner 批量回收；使用任务池并让远端释放进入 owner 的回收队列；把小 task 描述符放进 ring buffer 槽位，避免每次堆分配；或者把多个小任务合成 batch。选择哪种方式，取决于任务粒度、所有权语义、内存峰值和实现复杂度。

标准容器的内存形状 `std::vector` 连续存放元素，适合扫描、索引、批量复制和 SIMD，但扩容会搬迁元素。`std::deque` 分块存储，适合两端增长，但长距离扫描不如单连续数组直接。`std::list` 节点独立分配，插入和 splice 有语义价值，但遍历常变成指针追踪。`std::unordered_map` 平均查找是常数，但一次查询可能访问 bucket、节点、key 字节和 value，常数并不是一个数。

复杂度表描述增长阶，地址图描述每一步等什么。一个 $O(1)$ 链表插入可能伴随分配、指针追踪和 cache miss；一个 $O(n)$ vector 扫描可能被预取和 SIMD 高效处理。容器选择要同时写访问频率、元素大小、生命周期、引用稳定性、迭代模式和并发写者。

索引和句柄常比指针更适合批量系统 指针直接表达关系，但它绑定当前进程地址，难以序列化、搬迁和压缩。索引可以更小，能写进 trace、manifest、文件和网络消息，也方便把对象放在连续数组里。删除和复用位置时，索引需要 generation，避免旧索引误指向新对象。

Listing 7.13 索引加 generation 的稳定句柄

```
1 struct RecordId {
2     std::uint32_t index = 0;
3     std::uint32_t generation = 0;
4 };
5
6 struct RecordSlot {
```



```

7  std::uint32_t generation = 0;
8  bool alive = false;
9  };

```

明确位宽不是细节。索引宽度影响结构体大小、cache 密度、文件格式、网络协议和可处理规模。布局设计和类型选择是一件事。

句柄设计要回答复用和持久化 索引一旦可复用，就必须防止旧句柄误指向新对象。Generation 是常用办法：槽位每次释放或重新分配时增加 generation，外部句柄携带旧 generation，访问时比较。不需要长期持有的 batch row id 可以只用 32 位行号；跨 batch、跨文件、跨重试的 record id 则要包含 input generation、split 或 block 信息；持久化 manifest 中的 id 还要能在进程重启后解释。

Listing 7.14 按范围选择身份强度

```

row index:
    valid only inside one HotBatch

record id:
    input generation + byte offset + row number

task attempt id:
    logical task id + attempt number + worker/run identity

manifest entry id:
    logical split id + accepted attempt + block checksum

```

身份越强，字段越多，热路径成本越高。布局设计要把强身份放在控制面和冷路径，把热路径保留必要的短 id。比如聚合热列可以保存 32 位 row id，诊断阶段再通过 batch metadata 扩展成完整 record id。

7.5 图、哈希表和稀疏结构

不是所有数据都能变成简单连续数组。图、哈希表、稀疏矩阵和索引结构都有间接访问。但间接访问也有布局优劣。链式哈希表把节点散在堆上，查询需要追指针；开放寻址哈希表把控制字节、key 和 value 放在连续区域，更容易批量探测和利用 cache。图如果用每个节点一个 `std::vector`，邻接表分散；CSR 把所有边压成连续数组，并用 offset 指示每个顶点的边范围。

CSR 让图遍历变成区间扫描 CSR (Compressed Sparse Row) 用 `offsets[v]` 到 `offsets[v+1]` 表示顶点 `v` 的邻居区间，所有邻居保存在一个连续 `edges[]` 数组里。遍历单个顶点的邻居变成连续扫描，适合预取；遍历多个顶点时，offset 和 edge 数组也比堆节点更紧凑。

Listing 7.15 CSR 图的基本形态

```

1 struct CsrGraph {
2     std::vector<std::uint32_t> offsets;
3     std::vector<std::uint32_t> edges;
4 };
5
6 std::span<const std::uint32_t> neighbors(const CsrGraph& graph,
7                                         std::uint32_t vertex) {
8     const std::uint32_t begin = graph.offsets[vertex];
9     const std::uint32_t end = graph.offsets[vertex + 1];

```

```

10 return std::span<const std::uint32_t>(&graph.edges[begin], end - begin);
11 }

```

CSR 的代价是更新困难。增删边可能需要重建或维护增量结构。布局优化常在“查询快”和“更新灵活”之间取舍。若图阶段是批处理，CSR 很合适；若图频繁在线更新，可能需要分层结构：热查询用压缩快照，更新写入 `delta`，阶段边界再合并。

哈希表的常数来自地址形状 `std::unordered_map` 的平均 $O(1)$ 隐藏了 bucket 访问、节点分配、哈希计算、key 比较和 value 写入。对于小整数 key 的 histogram，用连续数组或局部桶通常更快；对于字符串 key，先字典编码成整数，后续阶段用整数 key，可以把随机字符串路径集中到前置阶段。哈希表不是错，问题是它是否位于热循环、key 是否可编码、写入是否共享、bucket 是否倾斜。

7.6 并发写布局：先标出写者

多线程布局要先标出写者。只读共享可以复制；低频写共享需要同步；高频写共享要避免多个线程写同一 line。一个常见错误是每个 worker 写 `stats[worker_id]`，看似各写各的，实际相邻槽位落在同一 line，形成 false sharing。修复方式是隔离高频写字段、按 worker 或 NUMA 节点分层、低频汇总。

Listing 7.16 线程本地统计槽位

```

1 struct alignas(64) WorkerStats {
2     std::uint64_t tasks_executed = 0;
3     std::uint64_t steal_attempts = 0;
4     std::uint64_t empty_polls = 0;
5 };
6
7 std::vector<WorkerStats> stats(static_cast<std::size_t>(worker_count));

```

这个布局适合 worker 高频写自己的统计。它不适合每个请求都读取全局精确值。如果业务需要实时限流，统计槽位还要配合周期性汇总或原子发布。并布局总是在写入吞吐、读取成本、内存占用和语义新鲜度之间取舍。

读取方和写入方不能共享同一控制字段 队列、环形缓冲和任务池里，生产者主要写 tail，消费者主要写 head；监控线程可能读 size 和统计字段。把 head、tail、size 和热指标放在同一 line，会让不同角色互相干扰。SPSC ring 通常隔离 head 和 tail；MPMC 队列会让槽位状态分散，减少集中争用。布局需要按访问者和写入频率组织，而不是按“这些都是队列字段”组织。

发布字段和统计字段要分冷热 队列槽位通常有两类字段。发布字段决定协议正确性，例如 sequence、state、head、tail、ready flag；统计字段只用于观测，例如 push 次数、空轮询次数、最长等待。发布字段需要精确同步，统计字段可以延迟汇总。把它们放在同一结构中，会让观测写入干扰协议热路径。

Listing 7.17 队列控制字段的冷热分离

```

1 struct alignas(64) QueueProducerCursor {
2     std::atomic<std::uint64_t> tail{0};
3 };
4
5 struct alignas(64) QueueConsumerCursor {
6     std::atomic<std::uint64_t> head{0};
7 };
8
9 struct alignas(64) QueueStats {

```

```

10 std::uint64_t pushes = 0;
11 std::uint64_t empty_polls = 0;
12 std::uint64_t slow_path_waits = 0;
13 };

```

这个示例不是要求所有结构都机械 padding 到 64 字节，而是提醒先标出写者和频率。高频原子 `cursor` 和低频统计不应互相污染；生产者写的 `tail` 和消费者写的 `head` 不应落在同一条 line；只在调试模式下写的字段不应进入 `release` 热路径。并发数据结构的性能常常先由字段布局决定，再由算法细节决定。

7.7 压缩、编码和计算成本交换

减少字节有时比减少计算更重要。列式数据库、搜索系统、日志系统和 `shuffle` 管线常把字符串编码成整数，把整数差分或位压缩，把稀疏布尔条件变成 `bitmap`。这样每个元素搬运的字节减少，但解码、分支和错误处理增加。是否划算取决于瓶颈：内存带宽受限时，压缩可能更快；计算受限时，压缩可能更慢。

编码会改变错误路径和调试路径 把事件名编码成整数后，热聚合更快；但错误报告还需要把整数映射回原字符串，字典版本也要稳定。若编码表在运行中更新，`record id`、`dictionary id` 和 `version` 都要进入 `trace`。压缩和编码不是只改变性能，还改变可见状态和调试能力。

压缩和 SIMD 的关系 压缩数据可能更适合 SIMD，也可能更难 SIMD。固定宽度 `int32` 列容易向量化；变长编码节省字节，却引入分支和边界解析；`bitset` 对过滤很高效，却需要位运算和 `popcount`。选择编码前，要写字节账本和操作账本：减少了多少字节，增加了多少操作，是否改变分支可预测性。

7.8 迁移旧代码：保护接口和证据

把成熟代码从胖对象改成热列，风险不在“写几个 `vector`”，而在接口、身份、生命周期和调试能力。旧代码可能到处保存 `LogRecord*`，错误报告依赖原始行，异步任务跨越 `batch` 生命周期，测试只比较最终计数。直接替换布局会让很多隐含合同破裂。

迁移步骤 第一步，写访问矩阵和字节账本，证明布局是瓶颈。第二步，建立 `reference` 和 `oracle`，固定输出、错误摘要和回查能力。第三步，引入新布局的内部表示，通过 `BatchView` 或 `adapter` 保持旧接口。第四步，只迁移热路径，保留旧路径作对照。第五步，逐步让调用方依赖稳定 `id` 和视图，而不是指针。第六步，删除旧布局前保留回归测试。

Listing 7.18 `BatchView` 把接口和布局分开

```

1 struct BatchView {
2     std::span<const std::uint32_t> record_ids;
3     std::span<const std::int32_t> event_types;
4     std::span<const std::int64_t> values;
5 };
6
7 struct ColdRecordStore {
8     std::string_view raw_line(RecordId id) const;
9     std::uint32_t file_offset(RecordId id) const;
10 };

```

热路径只需要 `BatchView`；错误路径通过 `ColdRecordStore` 回查。接口表达的是阶段需要什么，而不是暴露底层容器。这样未来从 `SoA` 改成 `AoSoA`、从 `vector` 改成 `mmap block`，调用方也不必重写。

双写和缓存一致性 迁移期间可能同时保留旧对象和新热列。若二者都可写，就会出现双写不一致。更稳妥的策略是明确权威数据：热列是聚合权威，冷表是诊断权威；或者旧对象只作为 reference，不进入生产路径。若需要缓存，就要有版本号和失效策略。布局重构是数据所有权迁移，不是简单复制字段。

7.9 布局判读

布局判读必须保持语义不变，只改变地址形状。对 LogRecord 案例，可以比较胖对象数组、热列 SoA、AoSoA block、热列加选择向量四个版本。输入包含短行、长行、错误稀疏、错误密集、事件类型均匀、事件类型热点。每个版本输出相同 checksum、错误摘要和可回查 record id。

对照项	观察内容	反证问题
只读热字段扫描 完整记录回查	吞吐、cache/TLB、字节账本 错误报告、record id、冷表 访问	是否真的少搬冷字段 是否破坏诊断能力
AoS 访问全部字 段	AoS 是否反而更快	SoA 是否只适合字段子集 访问
紧凑 stats 对隔 离 stats	多线程扩展曲线	false sharing 是否参与 成本
不同块大小 AoSoA	SIMD、尾部、缓存压力	块宽度是否合理

MemoryShape 证据 第 3 章的 MemoryShape 可以在这里扩展为布局证据。它应写清对象大小、字段大小、对齐、每条 cache line 容纳的逻辑元素数、每页容纳的逻辑元素数、热字段有效字节比例、冷字段回查方式、写者分布、分配器和生命周期。只有耗时数字不够；要说明为什么变快，以及哪些语义能力仍然保留。

反例必须保留 SoA 不总是更快。若每次都处理完整对象，AoS 可能更好；若输入很小，布局差异可能被函数调用和调度成本掩盖；若热列导致更多间接回查，错误密集输入可能变慢；若 padding 过度，合并阶段扫描更多字节。判读要写这些边界，避免把局部改造变成通用口号。

7.10 案例：LogRecord 到 BatchView

初始版本保存 `std::vector<LogRecord>`。聚合阶段遍历每条记录，读取 `event_type` 和 `value`，更新本地 histogram。第一轮测量显示对象变大后吞吐下降。字节账本说明每条记录热路径只用少数字节，cache line 里大部分是冷字段；AccessTrace 说明地址步长等于胖对象大小，热字段之间间隔大；MemoryShape 说明每页覆盖的逻辑记录数下降。

改造版本引入 `HotLogBatch` 和 `ColdRecordStore`。解析阶段从原始行生成 `record_id/event_type/value` 热列，同时把原文、字段边界和错误状态写入冷表。聚合阶段只读热列和线程本地 partial。错误报告阶段根据 `record_id` 回查冷表。这样热路径字节下降，诊断路径仍可用。

迁移判据 迁移通过的条件不是“快了”。至少要满足五个判据：reference 输出一致；错误报告能回到原始行和 offset；热路径字节账本下降；完整字段访问场景没有被误判；旧接口通过 adapter 仍能工作或明确废弃。若任何一项缺失，布局优化还没有完成。

布局产生的系统接口 热列为 SIMD 提供连续输入；BatchView 为线程池提供可切分任务；record id 为 trace、manifest 和分布式 shuffle 提供稳定身份；冷热分离为 I/O 和错误报告提供不同生命周期。数据布局不是孤立优化，它决定并行、向量化、异步和分布式边界能否清楚表达。

7.11 完整改造账本：从 LogRecord 到可审计热路径

把布局改造写完整，才能避免把 SoA 误解成一个机械替换。下面以日志聚合为例，目标不是把所有字段拆成数组，而是在不丢诊断能力的前提下，让聚合热路径只面对短、连续、可预取、可切分的地址流。

第一步：冻结语义合同 改布局前先冻结可观察结果。输入合同定义合法行和坏行；reference 输出 accepted、rejected、按 key 排序的计数和 diagnostics；oracle 比较计数、错误位置和 checksum。没有这一步，布局改造后即使吞吐提升，也不知道是否悄悄丢掉了坏行、改变了输出顺序或把两个不同输入版本混在一起。

Listing 7.19 布局改造前必须固定的语义字段

```
reference report:
  input_checksum
  accepted_records
  rejected_records
  sorted(event_type -> total)
  diagnostics(record_id, error_kind, byte_offset)
```

这份报告会约束后续所有中间表示。热列可以不保存原始字符串，但必须能通过 `record_id` 回到错误行；字典编码可以把事件名变成整数，但必须记录字典版本；selection vector 可以跳过未选中的行，但不能让 record id 与原始 offset 断开。

第二步：写阶段访问矩阵 访问矩阵要细到阶段。Parser 需要原始字节、字段边界和错误上下文；aggregator 需要 event type 和 value；reporter 需要稳定输出和错误回查；manifest 需要 block checksum、attempt 和记录数量。字段如果只在冷路径使用，就不应进入聚合热对象；字段如果被多个线程高频写，就要先考虑隔离写者。

阶段	热读写	冷读写
parse	写 record id、event type、value	写 raw line span、field offsets、parse error
aggregate	读 event type/value，写 local counts	不访问 raw line
diagnose	读 record id	回查 raw line、offset、错误类别
write block	读 partial counts	写 block metadata、checksum、manifest candidate

这张矩阵导出一个明确结论：`event_type[]`、`value[]` 和 `record_id[]` 是热 batch；`raw_line`、`field_offsets`、`parse_status` 是冷表；`partial_counts` 是 worker-local 写热点；`ManifestEntry` 是提交控制面。不同生命周期的对象不应塞进一个结构体。

第三步：把字符串 key 从热循环中搬走 若聚合阶段还在处理字符串 key，热路径会被变长比较和哈希表随机访问支配。更稳的做法是解析阶段做字典编码：把事件名映射到整数 `event_type`，聚合阶段只处理整数。字典本身是控制面对象，必须有版本和 checksum。

Listing 7.20 字典编码的语义伪代码

```
dictionary = load or build dictionary for this input generation
```



```

for each parsed record:
    if event_name is unknown and dictionary is closed:
        emit diagnostic(record_id, UnknownEvent)
        continue
    event_type = dictionary.lookup_or_insert(event_name)
    hot_batch.append(record_id, event_type, value)

report.dictionary_checksum = dictionary.checksum()

```

开放字典和封闭字典语义不同。开放字典允许新事件名进入，适合探索性日志；封闭字典把未知 key 当错误，适合严格 schema。无论哪种策略，reference 和优化路径必须一致。否则不同运行中 `event_type=7` 可能代表不同字符串，输出就失去可比性。

第四步：生成 BatchView 而不是暴露容器 热路径函数不应依赖具体容器类型，也不应知道冷表结构。它只接收 `BatchView`，其中每个 span 长度一致，`record_ids[i]`、`event_types[i]`、`values[i]` 描述同一条逻辑记录。这个约束要在构造 batch 时检查，而不是让聚合函数靠约定猜。

Listing 7.21 BatchView 的不变量

```

1 struct BatchView {
2     std::span<const std::uint32_t> record_ids;
3     std::span<const std::uint32_t> event_types;
4     std::span<const std::int64_t> values;
5 };
6
7 bool is_well_formed(const BatchView& batch) {
8     return batch.record_ids.size() == batch.event_types.size() &&
9           batch.record_ids.size() == batch.values.size();
10 }

```

有了视图，底层布局可以从 SoA 换成 AoSoA，也可以从内存 vector 换成 mmap block 或 arena 里的连续区域。聚合核只依赖“同一行索引对应同一记录”这个不变量。接口越接近阶段需求，布局迁移越可控。

第五步：把局部计数写成 worker-local 聚合阶段的高频写不能直接落到全局计数表。每个 worker 写自己的 `partial_counts[worker_id][event_type]`，最后 reduce 合并。若 event type 数量小，二维连续数组很直接；若 event type 很大，可以按 partition 或稀疏 map 存 partial，但仍要保持每个 worker 的写路径独立。

Listing 7.22 worker-local partial 的执行账本

```

worker 0:
    scan batch 0
    write partial[0][event_type]

worker 1:
    scan batch 1
    write partial[1][event_type]

reduce:

```

```
for each event_type:
    total = sum(partial[worker][event_type])
```

这个方案购买的是热写本地化，付出的成本是 partial 内存和最终合并。若 key 基数很小、热点严重，它通常划算；若 key 基数巨大且每个 worker 只见到少量 key，稠密 partial 可能浪费内存，需要稀疏 partial 或分区合并。布局选择仍然要回到字节账本。

第六步：让诊断走回查路径 冷热分离后，错误报告不能再假设 LogRecord 里有完整原始行。诊断要通过 record_id 进入 ColdRecordStore，取回 file id、offset、字段边界、原始切片和错误类别。这个回查路径要有测试，因为它不在主吞吐路径里，最容易在重构中坏掉。

Listing 7.23 诊断回查的语义伪代码

```
for each diagnostic in diagnostics:
    info = cold_store.lookup(diagnostic.record_id)
    print file_id, byte_offset, line_number, error_kind
    if debug mode:
        print raw_line_slice(info)
```

冷路径可以慢，但不能错。若回查失败，报告里要出现 missing cold record，而不是静默省略错误。工程上可以把冷表按 input split 分段保存，减少长期内存占用；只要 record_id 能定位到分段和行内位置，语义就不变。

第七步：用反例检查布局选择 改造完成后要保留反例。输入很小，SoA 可能被构造 batch 的成本抵消；错误行密集时，冷表回查可能成为主成本；每条记录后续都要完整输出时，AoS 可能更自然；event type 数量巨大时，稠密 partial 可能爆内存；字典编码若每批都重建，控制面成本会很高。成熟报告要写这些边界，而不是把某次收益推广成普遍规律。

反例场景	可能失败点	需要记录的证据
小输入	batch 构造和调度成本压过收益	元素数、固定开销、端到端时间
错误密集	冷表回查变成热路径	rejected 比例、diagnostic 时间
完整对象消费 巨大 key 空间	SoA 增加多列同步成本 稠密 partial 浪费内存	字段共现矩阵、访问比例 key 基数、非零计数、峰值内存
动态字典	编码表版本和锁竞争复杂	dictionary checksum、更新时间、等待时间

第八步：保留布局证据 布局改造的结果必须进入运行证据，否则后续性能变化无法解释。至少包含：对象大小、热列字节数、冷表字节数、每行有效热字节、record id 宽度、字典 checksum、partial 形状、selection 表示、batch 大小、峰值内存和回查失败数。这样当吞吐变化时，能知道是地址形状、输入分布还是语义路径变了。

第九步：把布局选择写成状态迁移，而不是一次重构 真实项目很少能一次从胖对象切到完美列式。更稳的做法是把布局迁移写成几个可回滚阶段，每个阶段只改变一个地址事实，并保留同一份 reference。第一阶段只给旧 LogRecord 增加 record_id 和 checksum，让输出可比较；第二阶段抽出 event_id 热列，但错误报告仍回查旧对象；第三阶段把原始文本和错误上下文移入 ColdRecordStore；

第四阶段让聚合热循环只接收 `BatchView`；第五阶段把 `worker-local partial` 和 `manifest` 写入同一份运行证据。

Listing 7.24 布局迁移的可回滚阶段

```
stage 0:
    LogRecord reference, full diagnostics, checksum

stage 1:
    add stable RecordId and input offset

stage 2:
    create event_id and value hot columns
    keep old object path for diagnostics

stage 3:
    move raw line and parse errors into ColdRecordStore

stage 4:
    aggregate(BatchView) reads only hot columns

stage 5:
    report MemoryShape, AccessTrace, partial layout, manifest candidate
```

这样写的价值是把“改成列式”拆成可验证事实。若 stage 2 以后 checksum 变了，问题在编码或热列生成；若 stage 3 以后错误报告缺字段，问题在 cold store 生命周期；若 stage 4 以后吞吐没变，问题可能是热循环仍旧回到了对象图；若 stage 5 后内存峰值过高，问题在批次生命周期或下游背压。每一步都有退路，而不是把所有风险压到一次大重构里。

布局选择要从 cache line 和 TLB 同时推导 AoS、SoA、AoSoA 的取舍不能只看“字段是否连续”。还要问每条 cache line 中有效热字节比例是多少，活跃页数是多少，是否同时访问多个列，是否要跨线程写 partial，是否会把 cold store 长期保留。一个 `LogRecord` 如果 160 字节，其中热路径只读 8 字节，那么 AoS 扫描时每搬入一条或多条 cache line，只使用很少部分；SoA 可以把热字段压紧，减少 cache line 和页覆盖。但如果后续阶段几乎每条记录都要回查完整文本，SoA 会引入大量间接回查。

Listing 7.25 布局判断的硬件账本

```
AoS candidate:
    object_bytes = 160
    hot_bytes_per_record = 8
    cold_bytes_per_record = 120
    pointer_fields = yes
    hot loop line_utilization is low

SoA candidate:
    event_id column = 4 bytes per record
    value column = 4 bytes per record
    cold lookup by RecordId only on diagnostics
    active pages for hot loop are lower

AoSoA candidate:
```

```

block records by cache-friendly tile
keep several hot columns adjacent per block
align tile with SIMD and task grain

```

TLB 也要进入判断。SoA 把热字段集中到少数连续页，通常提高页级覆盖；但如果列很多、每列都很窄，单次处理同时跨多个列数组，活跃页数也可能上升。AoSoA 常用于折中：一个 block 内保存若干列，既让 SIMD 看到连续 lane，又把同一批记录的热字段放在相近页面中。它比纯 SoA 复杂，但能同时服务 SIMD、cache 和任务调度粒度。

BatchView 是硬件合同，不只是接口包装 BatchView 的意义不是把参数变漂亮，而是向编译器和后续阶段表达一组硬件事实：输入连续、长度明确、热字段分开、生命周期受 batch 控制、错误回查通过 record_id。如果 BatchView 内部仍然保存 `std::vector<LogRecord*>`，热循环仍然要追指针；如果它暴露可变全局状态，worker-local partial 的边界又会被打破。接口形状必须服务地址形状。

Listing 7.26 BatchView 应表达连续热列和稳定身份

```

1 struct BatchView {
2     std::span<const RecordId> record_ids;
3     std::span<const std::uint32_t> event_ids;
4     std::span<const std::int64_t> values;
5     std::uint64_t dictionary_generation = 0;
6     std::uint64_t batch_checksum = 0;
7 };

```

这个结构还没有实现完整解析器，但它已经把第 3 章的地址账本落到了接口上。聚合函数接收 BatchView 后，反汇编中应出现对 `event_ids[i]` 和 `values[i]` 的连续 load，而不是对 LogRecord 指针和字符串的追踪。若反汇编与这个预期不一致，说明接口没有真正改变硬件路径。

7.12 完整迁移案例：从对象接口到 BatchView 内核

布局迁移最难的部分不是把字段拆成数组，而是把旧接口、语义合同、生命周期和热循环边界同时迁过去。旧代码通常从一个对象接口开始：

Listing 7.27 旧接口把完整对象带进热循环

```

1 void aggregate(std::span<const LogRecord> records, Counts& counts) {
2     for (const LogRecord& record : records) {
3         if (!record.valid()) {
4             continue;
5         }
6         counts.add(record.event_name(), record.value());
7     }
8 }

```

这段代码的问题不在函数名，而在接口形状。热循环拿到完整 LogRecord，于是字符串、错误状态、原始行、字段 offset、时间戳、调试标记都可能被带进同一个对象步长。即使当前只读 event_name 和 value，编译器也只能围绕对象布局生成地址；若 event_name() 返回字符串视图或触发字典查询，热循环还会追到冷表。接口已经把硬件路径锁住。

第一阶段：引入 adapter，但保留 reference 第一步不直接删旧接口，而是增

加 adapter。Adapter 从旧 `LogRecord` 生成 `BatchStorage`，再把只读视图交给新内核。旧 `aggregate` 仍作为 reference 存在，用来比较输出和诊断。

Listing 7.28 adapter 把旧对象转换为热列存储

```
1 struct BatchStorage {
2     std::vector<RecordId> record_ids;
3     std::vector<std::uint32_t> event_ids;
4     std::vector<std::int64_t> values;
5     std::uint64_t dictionary_generation = 0;
6     std::uint64_t checksum = 0;
7 };
8
9 BatchView view_of(const BatchStorage& storage) {
10     return BatchView{
11         .record_ids = storage.record_ids,
12         .event_ids = storage.event_ids,
13         .values = storage.values,
14         .dictionary_generation = storage.dictionary_generation,
15         .batch_checksum = storage.checksum
16     };
17 }
```

Adapter 阶段的收益可能很小，甚至更慢，因为它同时保留旧对象和新列。它的价值是建立等价性：同一输入经过旧路径和新路径，counts、diagnostics、dictionary checksum 和 record id 映射必须一致。等价性稳定后，才能把热循环迁到新内核。

第二阶段：把聚合核缩成连续读取和本地写 新聚合核不再接收 `LogRecord`，也不再接收全局 `Counts`。它只读 `BatchView`，写 `WorkerPartial`。这样地址形状非常明确：两个或三个连续列读，一个 worker-local 计数表写。

Listing 7.29 BatchView 聚合核只保留热路径依赖

```
1 struct WorkerPartial {
2     std::span<std::uint64_t> counts;
3     std::uint32_t worker_id = 0;
4 };
5
6 void aggregate_batch(BatchView batch, WorkerPartial partial) {
7     for (std::size_t i = 0; i != batch.event_ids.size(); ++i) {
8         const std::uint32_t event = batch.event_ids[i];
9         const std::int64_t value = batch.values[i];
10        partial.counts[event] += static_cast<std::uint64_t>(value);
11    }
12 }
```

这段代码有意不处理错误诊断、不查字符串、不提交 manifest、不写全局计数。聚合核越窄，反汇编越容易和硬件假设对应：连续 load、简单索引、worker-local store。若需要过滤坏行，最好在构造 batch 时只放 accepted 记录，或给出紧凑 selection；不要让每条记录在聚合核里回查完整状态。

第三阶段：把不变量写在构造边界 `BatchView` 是视图，不拥有内存；因此它必须由拥有者 `BatchStorage` 或 mmap block 保证生命周期。构造边界要检查 span

长度、字典 generation、record id 映射和 checksum。聚合核内部不反复检查这些条件，否则热循环又被控制面污染。

Listing 7.30 BatchView 构造边界的不变量

```
invariants:
  record_ids.size == event_ids.size
  record_ids.size == values.size
  event_ids refer to dictionary_generation
  record_ids map to ColdRecordStore entries
  batch_checksum covers accepted hot records
  storage lifetime outlives BatchView
  each WorkerPartial is owned by one worker
```

这些不变量分别服务不同风险。长度一致防止列错位；字典 generation 防止整数 id 解释错；record id 映射保护诊断；checksum 保护语义等价；生命周期防止悬垂视图；partial 所有权防止共享写和 false sharing。

第四阶段：冷表从热路径断开，但可回查 冷表通常保存原始行、文件 id、字节 offset、字段边界、错误类别和必要的上下文。它不应该被聚合核访问，但必须能通过 RecordId 回查。

Listing 7.31 冷热路径的生命周期划分

```
input buffer:
  owns raw bytes until parse finishes or cold slices are persisted

hot columns:
  record_ids, event_ids, values
  live for parse -> aggregate -> reduce

cold store:
  raw slices, offsets, parse errors
  live until diagnostics and retry decisions finish

worker partial:
  one writer during aggregate
  read during reduce
```

生命周期划分影响内存峰值。若输入 buffer、旧对象、热列和冷表同时保留，吞吐可能提升而峰值内存爆炸；若过早释放输入 buffer，诊断回查会失败。布局报告必须同时写热字节、冷字节和生命周期，而不是只写热循环速度。

第五阶段：用 report 证明迁移真的完成 迁移完成的标志不是函数签名变了，而是报告能证明热路径地址形状已经改变，并且语义能力仍然存在。

Listing 7.32 布局迁移报告字段

```
layout.object_bytes_before
layout.hot_bytes_after
layout.cold_bytes
layout.batch_records
layout.partial_bytes
layout.dictionary_generation
layout.record_id_width
```

```
layout.cold_lookup_failures
layout.hot_loop_calls
layout.hot_loop_load_shape
layout.checksum
layout.diagnostic_checksum
```

`hot_loop_calls` 用来确认内层是否仍有每记录调用；`hot_loop_load_shape` 可以写成 “contiguous columns”、“indexed selection” 或 “pointer chase”；`cold_lookup_failures` 必须为零。若 object bytes 下降但 diagnostic checksum 不一致，迁移失败；若 checksum 一致但 `hot_loop_load_shape` 仍是 pointer chase，接口没有真正改变地址路径。

第六阶段：把迁移接到系统接口 `BatchView` 不是布局改造的终点。向量化需要 `event_ids` 和 `values` 的连续性；线程池需要可切分 batch；锁和 atomics 需要比较全局计数、worker-local partial 和分区 reduce；运行时需要把 batch 变成任务；I/O 需要决定热列来自 read buffer 还是 mmap block；分布式提交需要把 block id、dictionary generation 和 manifest entry 写成跨机器合同。

因此迁移时要避免把当前实现细节写死在接口里。聚合核需要的是连续热列和稳定身份，不需要知道底层来自 vector、arena、mmap 还是网络 buffer。接口越准确表达阶段需求，后续从单线程到多线程、从内存到 I/O、从单机到分布式的移动越少。

常见失败模式 最后用失败模式检查迁移深度：

失败模式	机器后果	修复方向
BatchView 保存对象指针	仍然 pointer chase, TLB 和 cache 分散	改成连续热列或 AoSoA block
聚合核查询字符串字典	变长比较和随机哈希回到热路径	解析阶段编码, 聚合只用整数 id
所有 worker 写全局 Counts	cache line 争用和同步等待	worker-local partial 后 reduce
冷表生命周期短于诊断	错误回查失败或悬垂视图	record id 加分段所有权
selection 接近全量仍间接读	多读索引列且打散连续访问	materialize 成新的连续 batch

这些失败模式说明，布局不是局部字段排列，而是阶段合同、地址序列、生命周期和并发写者共同决定的系统形状。

7.13 排障案例：改成列式以后为什么仍然慢

有时把胖对象拆成热列以后，吞吐仍然没有明显提升。这个结果不能简单归因于“列式不适合”。更常见的原因是某条隐含地址路径没有被拆掉，或者新的表示引入了额外间接层。排障要沿着硬件账本逐层检查：热循环是否真的只读连续列，是否仍在查字符串字典，是否每条记录都回查冷表，是否 selection vector 造成随机行号访问，是否 worker-local partial 发生 false sharing，是否 batch 太小导致调度和函数调用成本压过缓存收益。

先确认热路径没有回到对象图 第一层检查是反汇编和访问记录。聚合函数的入参如果仍然是 `LogRecord&`，即使内部只读取 `event_type`，编译器和调用方也可能保留对象步长；若热循环调用 `record.event_name()`，很可能又追到字符串或冷表。真正的热路径应只接收 `BatchView`，并且在循环内只读 `event_types[i].values[i]` 和局部计数数组。错误回查、格式化日志、字典插入、manifest 写入都不应出现在这段循环里。

Listing 7.33 热路径审计的最小清单

```
inside aggregate loop:
  allowed:
    load event_types[i]
    load values[i]
    update worker-local partial
  suspicious:
    lookup cold_store by record_id
    hash event_name string
    allocate diagnostic object
    push to global vector
    update shared stats line
```

这份清单看起来朴素，但能抓住大量失败改造。很多代码“数据已经列式”，却在热循环里为了统计、调试或日志继续访问冷路径。布局优化不能只改存储结构，还要改阶段边界：聚合就是聚合，诊断就是诊断，提交就是提交。

再看 selection vector 是否把连续读打散 过滤后使用 selection vector 时，后续阶段读 event_types[selected_rows[j]]。如果选择率很低，这很好，因为少读了很多行；如果选择率接近 100%，它就把原本连续读变成了带索引的间接读，还多读了一列索引。此时材料化成新的连续 batch 可能更快。选择率不是业务备注，而是布局决策输入。

选择率	selection vector 的表现	更可能的替代方案
低于 5%	少量间接读，避免复制大量无效行	保留 selection vector
20% 到 70%	取决于后续字段数和缓存形状	对照 bitmap、selection、材料化
接近 100%	多读 index，破坏连续扫描	直接保留原 batch 或材料化连续输出

这个判断还要乘以后续字段数。若后续只读一个窄字段，selection vector 的间接成本可能仍可接受；若后续连续读取多个热字段，材料化一次再顺序扫描常更稳。第 13 章的 count/scan/write 正是为了让材料化输出也能并行且稳定。

检查局部计数是否真的局部 worker-local partial 如果紧挨着放在一个二维数组中，partial[0][0]、partial[1][0]、partial[2][0] 可能落在相邻 cache line，甚至同一 line 的不同位置。每个 worker 看似写自己的数组，硬件却看到多个核心反复写相邻 line。若 key 基数很小，所有 worker 都频繁写 event_type=0，false sharing 或同组 line 竞争仍可能发生。解决方式是按 worker 分配独立块、对齐每个 worker 的热区域，或按 NUMA 节点先局部合并。

Listing 7.34 partial 布局的两种地址形状

```
bad for hot key:
  partial[event_type][worker_id]
  workers write adjacent slots for the same event

better for worker-local writes:
  partial[worker_id][event_type]
  each worker writes a contiguous private region
```

数组下标顺序不是小事。它决定同一时刻多个核心写的是相邻地址还是私有区域。布局报告必须写清 partial 的主维度、对齐、padding 和合并顺序，否则无法解释热点 key 下的扩展曲线。

最后检查 batch 粒度和生命周期 过小 batch 会让每个 batch 的元数据、函数调用、队列 push/pop、字典版本检查和 cold store 分段成本占比过高；过大 batch 会降低调度灵活性、增加尾部等待、让错误回查保留太多冷数据。合适 batch 大小要同时看 L2/L3 容量、SIMD 主循环、线程池粒度、错误报告延迟和 manifest block 大小。布局不是只为单核热循环服务，它还要服务运行时和 I/O。

Listing 7.35 batch 大小选择要同时回答的问题

```
hot columns fit in which cache level?
does one task run long enough to amortize queue cost?
can slow batches create tail latency?
how long must cold records remain available?
does output block size match manifest and I/O policy?
```

当列式改造没有提速时，这五个问题常能找到原因。比如热列确实连续，但 batch 只有几百条记录，线程池开销超过节省的 cache miss；或者 batch 很大，最后一个热点 partition 拖住 reduce；或者 cold store 生命周期绑定整个作业，导致内存峰值过高。布局设计必须和任务、I/O、提交一起审查。

布局改变要有回滚路径 布局改造可能让某些输入变快，也可能让另一些输入变慢。工程上不应把旧路径立刻删除。更稳的做法是保留锁版或胖对象 reference，新增 MemoryShape 报告和运行时开关，在同一输入矩阵上比较。若错误密集输入显示冷表回查成为主成本，可以按错误率切换策略；若小输入固定开销过大，可以走轻量 AoS 路径；若长输入热循环占主导，再启用 SoA/AoSoA。回滚路径不是保守，而是让优化可验证、可撤销、可分阶段上线。

Listing 7.36 布局策略选择的运行时依据

```
if input_records < small_threshold:
    use simple row layout
elif rejected_ratio is high and diagnostics are required immediately:
    use row layout or eager diagnostic materialization
else:
    use hot-column batch with cold lookup
```

这个伪代码不是要求实现动态策略，而是说明布局选择应由输入、语义和证据驱动。没有报告字段时，系统连何时切换都不知道。

布局报告要能解释内存峰值 吞吐提升但内存峰值翻倍，也必须进入判断。热列、冷表、selection vector、materialized batch、worker-local partial 和临时输出都占空间；如果每一层都保留整批数据，流水线会变成内存堆积。报告应写出每个阶段持有多少 batch、每个 batch 的热字节和冷字节、何时释放冷表、何时释放 selection vector。生命周期清楚后，才能判断是布局本身占内存，还是下游背压让 batch 不能释放。

Listing 7.37 布局内存峰值分解

```
hot_columns_bytes
cold_store_bytes
selection_vector_bytes
materialized_batch_bytes
worker_partial_bytes
pending_output_bytes
```

这份分解会在第 15 章的背压中继续使用。数据布局不是单核局部性问题，它也决定系统能承受多少 in-flight 工作。

因此，布局报告要同时给出“每条记录热路径搬多少字节”和“整个流水线最多同时持有多少字节”。前者解释单核吞吐，后者解释系统稳定性、恢复压力和背压触发点。两者只看一个，都会把问题讲偏，也无法解释为什么同一布局在短任务、长任务和错误密集输入中表现不同。

7.14 习题

习题与作业

习题一：为一个包含 `timestamp`、`user_id`、`event_type`、`value`、`raw_line`、`parse_status`、`debug_tags` 的日志记录写访问矩阵。根据矩阵设计 AoS、SoA 和冷热分离三种布局，并估算每条 cache line 的有效字节比例。

习题二：设计 `BatchView` 和 `ColdRecordStore` 的接口。要求聚合热路径不依赖原始文本，错误报告仍能根据 `record_id` 找回原始行、文件 offset 和解析状态。

习题三：比较 `std::vector`、`std::deque`、`std::list` 和索引表在 ready task 队列中的内存形状。不要只写复杂度，要写元素是否连续、引用是否稳定、遍历是否热、分配是否在热路径、并发写字段在哪里。

习题四：把一个链式图结构改写成 CSR。说明查询、遍历、插入、删除、序列化和调试路径分别变好或变差在哪里。

习题五：设计一个 false sharing 布局实验。比较紧凑 `WorkerStats`、按 cache line 隔离 `WorkerStats` 和 per-NUMA partial。记录线程数扩展、合并成本、内存占用和不可验证边界。

习题六：为 bitmap、selection vector 和 materialized batch 各设计一个输入场景。要求写出选择率、后续算子、读写字节账本和预期瓶颈，并说明哪一种表示最合适。

习题七：设计一个软件预取实验。先证明存在内存等待，再选择预取距离。报告必须包含预取无收益或负收益时的解释。

习题八：按“冻结语义、访问矩阵、字典编码、`BatchView`、worker-local partial、冷表回查、反例检查、RunReport”八步，写一份日志布局迁移方案。要求每一步都给出一个能失败的反例和对应测试。

连续热列、AoSoA 块、选择向量、对齐和批量视图，都是向量化的入口。若数据仍然散在对象图和指针链中，指令再强也很难喂饱执行单元。

工程案例

知识点小案例：日志对象为什么要冷热拆分。特例是一条日志记录同时保存 `timestamp`、`user_id`、`event_type`、`value`、`raw_line` 和 `debug_tags`。聚合热路径只需要 `event_type` 和 `value`，但 AoS 布局会把原始文本指针和调试字段一起拖进 cache line。改成热列 `event_type[]`、`value[]` 加冷表回查后，热循环读字节减少，错误报告仍能通过 record id 找回原始行。由此推出规律：布局优化不是把结构体换成数组，而是用访问矩阵证明哪些字段同频访问。工程上任何冷热拆分都要同时保留语义、错误身份和回查路径。

第 8 章

SIMD、指令级并行与向量化

日志解析器曾经出现过一个很隐蔽的回归：标量版本逐字节扫描字段，遇到第一个非法字符时返回精确的 `record_id`、字段号和字节偏移；优化版本一次处理一整块字节，规则输入快了很多，错误输入却只报告“某个块中存在非法字符”。更糟的是，块尾的非法字节有时被归到下一条记录。主循环看起来很漂亮，错误语义却退化了。

这个事故把 SIMD 的入口钉在正确位置：宽寄存器和指令名不是起点。起点是机器每个周期能取多少指令、解码多少微操作、发射到哪些执行端口、从 cache line 中取回多少字节；同时也是程序语义能否把多个元素放进同一个批次。只有当地址规律、输出位置、错误路径、尾部元素、数值语义和平台回退都被定义清楚，SIMD 才是优化。否则，它只是把错误批量化。

8.1 从一条标量循环看机器瓶颈

考虑最普通的数组变换：

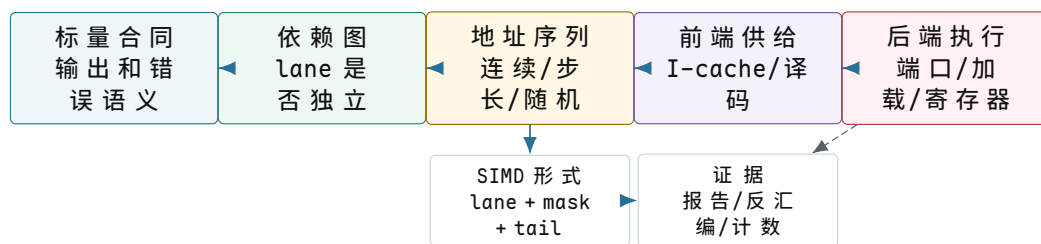
Listing 8.1 标量一对一变换的语义骨架

```
1 for each index in [0, count):
2   output[index] = input[index] * scale + shift
```

源代码只有一行表达式，CPU 实际执行的不是“表达式”，而是一串指令字节。指令字节先从内存层次进入指令缓存，通常称为 I-cache；前端取指、译码，把复杂指令变成一个或多个微操作；乱序核心把微操作放入窗口，等待输入寄存器、地址、cache 数据和执行端口可用；完成后结果暂存在重排序缓冲，最后按程序顺序提交。一个循环是否快，不只取决于每轮做了几次乘加，也取决于前端能否持续供给微操作、后端是否有足够端口、加载路径是否等得到数据、寄存器是否足够容纳临时值。

I-cache 的存在来自一个基本事实：取指也要走内存层次。CPU 每周期想执行很多微操作，但主存距离太远，不能每条指令都从主存取。于是热代码的指令字节会进入较小、较快的指令缓存。循环展开、模板膨胀和手写多版本内核会增加代码体积；当热路径指令字节超过前端缓存能力时，后端执行单元可能空着等待取指和译码。宽指令减少了每个元素对应的指令条数，却也可能引入更大的代码和更多特殊路径。SIMD 要同时接受前端和后端审计。

标量循环到 SIMD 的约束链



向量化不是跳过语义直接选指令，而是把标量合同、依赖、地址形状和机器供给同时对齐。

图示：本图要证明的命题是：SIMD 是语义批量化和硬件供给共同允许的结果，不是孤立的指令替换。

前端：指令字节也会成为瓶颈 热循环的指令首先要被取到 I-cache，再进入译码和微操作缓存等前端结构。若循环体很短，前端可以反复供给同一小段指令；若展开过度、模板实例太多、分支路径太散，前端会在取指、译码和预测上消耗更多预算。手写 SIMD 常伴随多版本派发和尾部路径，代码体积上升后，不能只盯着主循环的后端吞吐。

I-cache 不是“装代码的普通缓存”这么简单。数据 cache 处理 load/store，I-cache 处理即将执行的指令字节。二者可能共享更低层缓存，却在核心前端承担不同职责。一次 I-cache miss 会让前端取不到下一批指令，后端即使有空闲乘法和加法器也无法持续开工。代码膨胀导致的前端停顿，常在微基准中不明显，却会在真实系统的多个热路径交替执行时出现。

后端：延迟、吞吐和执行端口 一条加法指令有延迟，也有吞吐。延迟是一个结果从输入到可被后续指令使用需要等多久；吞吐是执行单元在流水线填满后每周期能接收多少条同类操作。单一累加器求和受延迟限制，因为下一次加法必须等上一次结果；一对一数组变换更容易受吞吐限制，因为不同元素之间独立，乱序核心可以把多个加载、乘法和存储同时排队。

执行端口是后端资源的入口。加载使用加载端口和地址生成单元，存储使用存储数据和存储地址路径，整数和浮点计算使用相应算术端口。若每个元素只做一次简单比较却要读写很多字节，瓶颈可能在加载/存储；若每个元素有多个 FMA 且数据复用高，瓶颈可能在浮点执行端口；若循环控制和分支太复杂，前端和分支预测可能先倒下。向量化改变的是每条指令覆盖的元素数，但它不会凭空增加内存通道、执行端口和寄存器文件。

8.2 批量语义先于向量指令

SIMD 表面上是一条指令处理多个 lane。更准确地说，是在同一条指令语义下同时处理多个元素。一个 lane 可以看成向量寄存器中的一个槽位，例如 256 位寄存器可以容纳 8 个 float 或 32 个字节。lane 不是线程：它们共享同一条指令、同一次控制选择、同一组寄存器和同一个提交路径。遇到条件时，不是每个 lane 任意分叉，而是用 mask 表示哪些 lane 有效。

批量语义需要回答五个问题。第一，某个元素的计算是否依赖前一个元素的输出。第二，输入和输出是否可能破坏性重叠。第三，输出位置在处理当前元素时是否已知。第四，遇到异常、非法字符或边界值时，是否仍能保持标量合同。第五，浮点重排、整数溢出、饱和和舍入是否被允许。任何一个问题含糊，向量化都会把复杂性藏进更难调试的路径。

三种循环形状 一对一 map 最容易批量化。位置 i 的输出只依赖位置 i 的输入，不同元素之间没有语义通信。数组缩放、阈值比较、字节是否为数字，通常属于这种形状。

归约需要把多个输入合成一个输出。它可以向量化，但不是简单地把所有元素并排处理完就结束。向量寄存器内部要做水平合并，多个向量 `partial` 还要合并到最终结果。整数计数通常容易固定语义；浮点求和会改变加法顺序，必须说明误差或固定合并树。

前缀和、状态机解析和递推滤波含有循环携带依赖。位置 `i` 的结果依赖位置 `i-1` 的结果，普通 SIMD 不能直接把所有 lane 当作独立元素。可行路线通常是算法重写：分块 `scan`、分块状态摘要、候选位置批量扫描后再由标量状态机确认。这里的关键不是放弃 SIMD，而是只把真正独立的子问题批量化。

循环形状	依赖事实	典型机制	SIMD 风险
一对一 map	每个输出位置独立	连续 load、向量计算、连续 store	别名、尾部、带宽上限
归约	多个输入合成 <code>partial</code>	多累加器、水平合并、树形合并	浮点误差、溢出、合并树未声明
稳定 filter	输出位置取决于前面保留数	<code>mask</code> 、 <code>count</code> 、 <code>scan</code> 、 <code>compact</code>	顺序、输出容量、选择率变化
状态机解析	状态跨字节或跨记录传播	SIMD 找候选，标量或块状态确认	跨块状态、错误位置、恢复策略
随机写 histogram	多个元素写数据相关桶	本地 <code>partial</code> 、冲突检测、分阶段合并	lane 冲突、热点桶、原子退化

别名是语义问题，不只是编译器问题 若函数接收三个裸指针 `a`、`b`、`out`，编译器必须考虑它们可能指向重叠区域。某些重叠是安全的，例如 `out[i]` 就地覆盖 `a[i]`；某些重叠会破坏语义，例如 `out` 指向 `a+1`，写当前输出会影响下一轮读取。编译器不能随意重排这样的循环。

C++ 没有标准 `restrict` 关键字。接口仍可以表达更清楚的合同：只读输入用 `std::span<const T>`，输出用独占缓冲，debug 模式检查区间是否破坏性重叠，必要时把输入复制到稳定临时区。别名约束不是为了讨好编译器，而是为了让程序语义允许重排、展开和向量化。

8.3 lane、mask 和 tail 从哪里来

当多个元素能共享同一条操作语义时，硬件可以用更宽的数据路径承载它们。向量寄存器把多个 lane 放在一起；向量加载一次取回一段连续字节；向量比较产生一个 `mask`；向量算术对每个 lane 做同类运算；向量存储把结果写回连续区域。不同 ISA 的寄存器宽度和指令细节不同，但这套抽象稳定：lane 承载并行元素，`mask` 承载条件，`tail` 处理不足一整个向量的末尾。

mask 是数据化的分支 标量代码中常写：

Listing 8.2 标量条件的语义

```
1 if value >= threshold:
2     keep value
```

向量代码不能让每个 lane 随意走不同指令流。它通常先比较一组 lane，得到一个 `mask`：第 3 位为 1 表示第 3 个元素满足条件。之后可以用 `mask` 选择结果、统计命中数、找第一个失败位置，或者把保留元素压缩写出。`mask` 的好处是减少大量短分支；代价是需要额外寄存器、逻辑操作、压缩和输出位置计算。

`mask` 的语义必须精确。一个 bit 到底对应输入中的哪个字节，bit 为 1 表示“分隔符”“非法字符”还是“保留记录”，后续如何从块内偏移恢复全局偏移，这些都不是实现细节。错误报告、selection vector、stable filter 和解析状态机都依赖同一份 `mask` 合同。

从标量比较到 `movemask` 用 ASCII 数字分类可以看到 `mask` 从哪里来。标量循环每次读取一个字节，做两次比较：

Listing 8.3 标量数字分类

```
1 bool is_digit(unsigned char b) {
2     return b >= '0' && b <= '9';
3 }
```

典型标量机器路径是：load 一个字节，和常量 `'0'`、`'9'` 比较，生成条件码，再用条件跳转或条件设置得到布尔结果。若每个字节都要分支，随机输入会给分支预测制造压力；若改成条件设置，分支少了，但仍然是一字节一字节推进。SIMD 版本把 16、32 或更多字节放进一个向量寄存器，使用同一条比较语义同时作用在多个 lane 上。抽象形态如下：

Listing 8.4 向量数字分类的机器动作抽象

```
v = vector_load(bytes + block_begin)
ge_zero = compare_each_lane(v, '0', >=)
le_nine = compare_each_lane(v, '9', <=)
digit_lanes = bitwise_and(ge_zero, le_nine)
digit_mask = extract_lane_sign_bits(digit_lanes)
```

在 x86 SSE/AVX 路径里，字节比较常产生每个 lane 全 0 或全 1 的字节；随后类似 `pmovmskb/vpmovmskb` 的动作把每个 lane 的最高位提取成整数 `mask`。这样一个整数的第 `k` 位就对应块内第 `k` 个字节。不同 ISA 的指令名不同，但核心形状稳定：向量比较产生 lane 结果，`mask` 提取把 lane 结果变成可用于分支、计数和位置恢复的标量控制字。

Listing 8.5 `mask` 到位置的映射合同

```
block_begin = 128
digit_mask bit 0 -> input byte 128
digit_mask bit 7 -> input byte 135
digit_mask bit 31 -> input byte 159

first matching byte:
    lane = index_of_lowest_set_bit(digit_mask)
    absolute_offset = block_begin + lane
```

这一步解释了为什么 SIMD parser 常常先做“结构字符扫描”。它不是直接完成全部解析，而是用宽比较快速生成候选位置：逗号、换行、引号、非法字节、数字范围。后续语义状态机再按 `mask` 中的位置推进。SIMD 负责把大量独立字节分类成 `mask`，状态机负责解释这些位置在记录、字段和错误合同中的含义。

mask 消除了短分支，不消除输出位置问题 向量比较可以把一组 `value>=threshold` 变成一个 `mask`，但 `mask` 只回答“哪些 lane 通过”。Stable filter 还必须回答“通过的元素写到 output 的哪一个位置”。这需要 `popcount`、块内 `rank` 和块间 `scan`。若只用一个全局 `fetch_add(popcount(mask))` 给每个向量块抢输出区间，块的顺序可能变成调度顺序；若每个 lane 再单独 `fetch_add`，又回到共享计数器热点。正确结构仍然是第 13 章的 `count/scan/write`：SIMD 加速 `count` 和块内压缩，`scan` 保证全局稳定位置。

因此 `mask` 不是并行输出的完整答案，而是把控制流数据化之后交给后续位置

分配。这个边界讲清楚，才能避免把“SIMD 比较很快”误写成“SIMD filter 已经完成”。宽比较降低了分支和比较指令成本，scan 和写出合同解决输出顺序、独占区间和恢复。

尾部不是小角落 输入长度很少刚好是向量宽度的倍数。尾部策略通常有三种。第一，主体向量循环处理完整块，剩余元素用标量循环处理；它最清楚，适合公开 API 和不可信输入。第二，使用 mask load/store 处理尾部；它可以减少分支，但依赖 ISA，且必须确认被 mask 掉的 lane 不会触发非法访问。第三，在容器末尾保留 padding，让主体循环可以安全读到逻辑末尾之后；它适合内部 batch 或张量缓冲，不适合任意外部 span。

越界读即使结果被丢弃，在 C++ 语义中也可能是未定义行为；跨过映射页还可能直接触发 page fault。尾部策略必须写进接口和测试，而不是只在主循环旁边加一句注释。

常见误区

SIMD 主循环通过大输入测试，不代表实现正确。长度为 0、1、向量宽度减 1、向量宽度、向量宽度加 1、非对齐起点、跨页边界和输入输出重叠，才是尾部和边界语义的压力输入。

异常语义不能被 lane 吞掉 标量 parser 遇到第一个非法字节，可以报告精确 offset。向量 parser 一次发现多个 lane 异常时，必须定义报告哪个错误：最小 offset、全部错误、每条记录第一个错误，还是只输出块级错误码。不同选择会影响 mask 处理、排序、诊断容量和冷路径回查。若标量 reference 报告第一个错误，SIMD 版本也必须从 mask 中恢复最小 set bit 对应的全局偏移。

Listing 8.6 从 mask 恢复精确错误位置

```
block_start = absolute byte offset of the vector block
invalid_mask = compare_lanes_for_invalid_bytes(block)

if invalid_mask != 0:
    lane = index_of_lowest_set_bit(invalid_mask)
    error_offset = block_start + lane
    emit diagnostic(record_id, field_id, error_offset)
```

这段伪代码显示，SIMD 不是把错误语义降级成“块有错”。Mask 是一组候选事实，冷路径必须把它映射回记录身份、字段身份和字节偏移。若块跨记录边界，还要知道每个 lane 属于哪条记录；若错误报告要求稳定顺序，就不能让不同 worker 的异常无序写入同一诊断列表。向量化后的诊断语义要和标量 reference 一样可审计。

跨页安全来自加载合同 向量加载一次可能覆盖多个 cache line，甚至跨过页面边界。若最后几个 lane 超过有效输入范围，即使用 mask 丢弃，也可能已经触碰未映射页面。某些 ISA 的 mask load 可以保证被 mask 掉的 lane 不访问内存，某些写法只是编译器层面的选择，不能靠想象。公开 API 处理任意 std::span 时，最稳的策略仍是主体循环只处理完整安全范围，尾部走标量或明确安全的 mask load。

尾部策略	安全前提	适合场景
标量尾部 mask load/store	主循环不越过 count ISA 保证被 mask lane 不访问	公共库、不可信输入、最易审查 已验证平台路径，尾部频繁且成本可见
padding sentinel overread then discard	分配器保证末尾有可读安全字节 只在能证明地址仍合法时成立	内部 buffer、解析器专用输入块 极少数受控内存布局，不适合通用 span

这张表把“能跑”拆成“语义安全”和“平台安全”。跨页错误常常只在输入刚好落

到映射尾部时暴露，普通大数组测试不会覆盖。因此 SIMD 单元测试必须刻意构造靠近页尾的 buffer，检测长度 0、短尾和非对齐起点。

平台派发是语义的一部分 同一份程序可能在 SSE、AVX2、AVX-512 或没有目标 SIMD 的机器上运行。若编译期只生成一个最高 ISA 版本，旧机器可能无法启动；若运行时多版本派发，派发函数本身要保证所有版本输出一致。平台派发不能只比较吞吐，还要比较错误报告、尾部、浮点误差、NaN、溢出和别名行为。

Listing 8.7 SIMD 多版本派发的报告字段

```
SimdDispatchReport:
  selected_isa
  scalar_fallback_available
  vector_width_bytes
  alignment_requirement
  tail_strategy
  checksum_matches_scalar
  diagnostics_match_scalar
  unsupported_reason_if_fallback
```

这个报告会防止两个常见问题。第一，在开发机上只验证 AVX2，部署到无 AVX2 环境时 silently fallback 到未经测试路径。第二，标量 fallback 只在短输入上跑过，没有覆盖错误语义。高性能路径和回退路径都属于同一个接口合同；回退不应被当成“慢但无所谓”的角落。

8.4 端口、lane 利用率和 reduction 成本

SIMD 宽度只是上限，不是实际性能。一个 AVX2 寄存器能放 8 个 float，不等于吞吐一定 8 倍。真实速度还受 load/store 端口、FMA/ALU 端口、shuffle 端口、前端 uop 供给、寄存器压力和尾部 mask 控制。向量化报告必须写机器资源账本。

每元素资源账本 考虑 $out[i]=a[i]*scale+b[i]$ 。标量每元素大致需要两个 load、一个乘加、一个 store。AVX2 每条向量指令处理 8 个 float，但每个向量仍要读取 32 字节 a、32 字节 b，写 32 字节 out。若数据不在 cache 中，主瓶颈可能仍是字节流量，而不是乘法器。

Listing 8.8 一对一 float 变换的向量账本

```
per 8 float elements:
  load a:    32 bytes
  load b:    32 bytes
  store out: 32 bytes
  compute:   8 multiply-add operations

per element:
  bytes ~= 12
  flops ~= 2
  AI  ~= 0.167 FLOP/byte
```

这个 AI 很低。若没有数据复用，它更像带宽 kernel。把标量换成 SIMD 可以减少指令数和分支成本，但不会减少必要字节。若 benchmark 显示接近内存带宽上限，再继续手写复杂 intrinsic 可能收益很小。

lane 利用率 向量宽度为 8，但尾部、mask、分支和数据选择可能让有效 lane 远小于 8。若每 8 个 lane 只有 1 个满足条件，向量比较仍然很快，但后续

compact、scatter 或随机写可能主导成本。报告要区分“比较阶段 lane 满载”和“写出阶段有效 lane 稀疏”。

形态	lane 利用率风险	常见补救
连续 map filter	尾部和对齐 选择率低或波动大	标量尾部或安全 mask count/scan/write, 分 块压缩
histogram	lane 写同一桶	局部 partial, 冲突检 测
parser reduction	分支和跨块状态 水平合并成本	SIMD 找候选, 标量确认 多 accumulator, 树形 合并

horizontal reduction 不是免费 向量寄存器内部求和最终要把多个 lane 合成为一个标量。这个 horizontal reduce 需要 shuffle、permute 或专门水平加法指令。若每处理很少元素就做一次水平合并, shuffle 成本会吃掉向量化收益。正确形态通常是多个向量 accumulator 在长循环中累加, 循环结束后再少量水平合并。

Listing 8.9 reduction 的成本边界

```
bad shape:
load vector
reduce vector to scalar immediately
add scalar to sum

better shape:
keep several vector accumulators
accumulate many vectors
reduce accumulators at the end
```

浮点 reduction 还要声明误差合同。多 accumulator 和树形合并改变加法顺序, 结果可能与标量逐项求和最后几位不同。若业务要求 bitwise identical, 就要固定合并树或保留标量 reference; 若只要求误差边界, 就要写出绝对/相对误差阈值和测试输入。

端口压力来自指令混合 同一个 vaddps、vmulps、vpermps、vmovups 不占用同一种资源。连续 load/store 受 load/store 端口和缓存带宽限制; shuffle/permute 常走专门端口; FMA 走浮点执行端口; 地址计算需要 AGU。一个 kernel 若每轮有大量 gather、permute 和 masked store, 就不能用“FMA 峰值”估算。

Listing 8.10 SIMD 报告必须写的资源问题

```
instruction mix:
vector loads/stores
arithmetic ops
shuffles/permutations
masks/blends
gathers/scatters

resource questions:
are loads contiguous?
do shuffles dominate?
is tail path frequent?
```

```
do accumulators exceed registers?
does unrolling overflow I-cache or registers?
```

8.5 指令级并行：不用 SIMD 也能变快的原因

SIMD 关注一条指令覆盖多个元素，指令级并行关注多个互不依赖的指令同时推进。现代乱序核心不会机械地按源代码逐行等待；只要依赖允许，它会让加载、算术、地址生成和存储在不同端口上重叠。代码要给乱序窗口足够多的独立工作。

单累加器求和是最经典的反例：

Listing 8.11 单累加器形成一条长依赖链

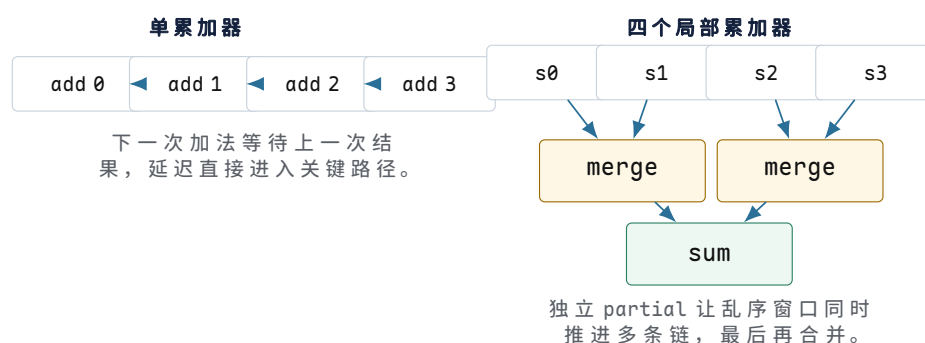
```
1 sum = 0
2 for value in values:
3     sum = sum + value
```

每次加法都要等上一轮 `sum` 出来。这条依赖链限制了乱序核心能看到的独立工作。改成多个局部累加器后，语义仍然是求和，但机器看到多条较短的依赖链：

Listing 8.12 多个累加器暴露指令级并行

```
1 s0 = 0; s1 = 0; s2 = 0; s3 = 0
2 for i in blocks of 4:
3     s0 += values[i + 0]
4     s1 += values[i + 1]
5     s2 += values[i + 2]
6     s3 += values[i + 3]
7 sum = ((s0 + s1) + (s2 + s3))
```

单依赖链与多累加器



图示：本图要证明的命题是：多累加器通过拆短依赖链提高指令级并行，但同时改变了归约合并树。

多累加器的代价 累加器不是越多越好。更多累加器需要更多寄存器；寄存器不足时，编译器会 spill 到栈，反而增加 load/store。循环展开也会增加代码体积，可能加重 I-cache 和译码压力。对浮点求和，多累加器还改变合并顺序，必须进入误差合同。性能提升来自依赖链变短，不来自语义免费变化。

SIMD 内核也需要 ILP 一个向量累加器同样可能形成依赖链。高性能 dot product 往往同时使用多个向量累加器，让加载、FMA 和水平合并更容易重叠。反

过来，只有 SIMD 没有足够 ILP，也可能被 FMA 延迟、加载延迟或水平合并拖住。单核优化经常是 SIMD、展开、多累加器、预取和寄存器预算共同决定的结果。

8.6 内存路径决定向量质量

SIMD 喜欢连续数据，因为连续地址可以让 cache line、预取器、TLB 和加载端口一起工作。一个 64 字节 cache line 可以容纳 16 个 `int32_t`、8 个 `double` 或 64 个字节。如果热字段被胖对象隔开，向量加载会拖入大量冷字段；如果热字段被组织成列式数组，一个向量加载就能带回多个有用元素。第 6 章的数据布局是 SIMD 的前置条件。

连续、步长、gather 连续 load/store 是最理想的形状。固定步长访问仍可能被预取器识别，但有效 lane 利用率取决于步长和字段大小。gather 从多个不连续地址收集 lane，scatter 向多个不连续地址写回。它们提高表达能力，却不把随机访问变成连续访问；每个 lane 仍可能等待不同 cache line、不同页和不同地址生成路径。

遇到 gather/scatter，先问能否改变数据形状：按 key 分区、排序索引、把热字段提成连续数组、把链式结构改成 CSR、先构造 selection vector 再顺序处理。直接使用 gather 常是最后手段，不是第一反应。若随机访问不可避免，可以按页或 cache line 分组请求，提高内存级并行，减少完全随机的抖动。

对齐的真实边界 现代 CPU 通常能执行非对齐连续 load/store。非对齐不等于错误，也不必为了对齐把接口复杂化。但跨 cache line、跨页、跨分配边界的访问成本更高，尾部处理也更危险。若决定要求对齐，分配器、容器、API 和 fallback 都要共同保证；只在注释里写“假设对齐”会制造未定义行为和移植风险。

预取不能修复错误布局 硬件预取器擅长发现顺序或固定步长访问。手工预取可以提示未来地址，但它需要可预测的未来地址，并且不能太早或太晚。随机链表、哈希表节点和分支驱动访问很难靠预取彻底修复。SIMD 同理：若数据仍散在对象图中，宽寄存器只是在等待多个随机 miss。先写地址账本，再谈指令。

8.7 自动向量化是编译器在做证明

自动向量化不是玄学。编译器尝试证明循环满足一组条件：没有破坏性别名，没有不可处理的循环携带依赖，内存访问足够规则，控制流可以变成 mask 或分支外提，数值语义允许重排，目标机器有合适指令，并且成本模型认为收益超过准备和尾部成本。失败报告不是坏消息，而是设计反馈。

报告线索	含义	先尝试的改法
可能别名	输出写可能影响后续输入读	缩窄接口、独占输出、debug 区间检查
循环携带依赖	本轮需要上一轮结果	多 partial、scan、分块状态摘要
调用不可内联	编译器看不到副作用边界	拆纯函数、查表、把慢路径移出热循环
控制流复杂	分支难以转成 mask 或成本太高	分离正常路径和异常路径，按类型分桶
浮点语义严格	重排可能改变结果	写误差合同，决定是否允许固定树形合并
成本模型不值得	输入短、尾部重或指令代价高	按长度分派，保留标量短路径

报告要和反汇编互相验证 编译器说 vectorized，不代表得到理想代码。可能生成了运行时别名检查，可能只用了窄向量，可能尾部路径很重，可能因为目标设置没启用预期 ISA。编译器说 not vectorized，也不代表函数慢，瓶颈可能在内存

带宽、分支、I/O 或调用层。反汇编要看加载/存储形态、向量宽度、主循环、peel loop、尾部、运行时检查和函数调用。

性能证据也要放回第 5 章的字节账本。如果每个元素只做一次比较却读写几十字节，SIMD 可能减少指令数，却不能突破内存带宽。若计数器显示前端停顿上升，可能是过度展开或多版本代码让指令供给变差。若 branch misses 下降但 cache misses 上升，说明瓶颈迁移了。向量化报告只是证据链的一环。

8.8 三个案例

一对一数组 map 数组 map 的合同最简单：第 i 个输出只依赖第 i 个输入和少量标量参数。优化路线也最清楚。先写标量 reference，检查输入输出长度和别名；再把热字段组织成连续数组；然后读自动向量化报告；最后按需要考虑手写平台版本。

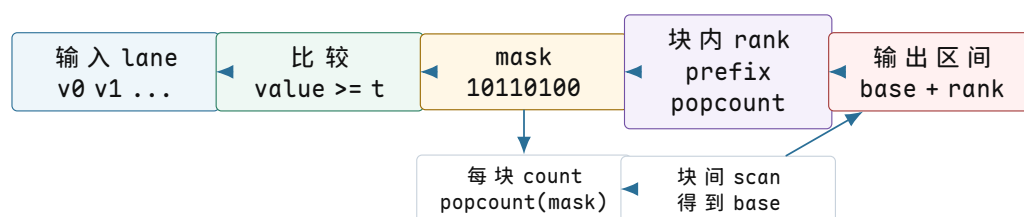
这个案例的性能判断来自字节和操作。若 float map 每个元素读 4 字节、写 4 字节，只做一次乘加，它很容易受内存带宽限制。融合两个相邻 map 可能减少中间数组读写，比换更宽向量更有效。若 map 中有昂贵数学函数，瓶颈可能转向函数调用、近似算法和前端供给。数组 map 不是简单题，它是把 Roofline、布局、SIMD 和前端连起来的最小样本。

ASCII 字节分类 日志解析常要找逗号、换行、空白、数字或非法字节。单个字节是否为数字，只依赖它自己，很适合批量比较；CSV 引号状态、转义和 UTF-8 边界可能依赖前文状态，不能直接当作独立 lane。稳妥结构是：SIMD 先生成候选 mask，例如分隔符 mask、引号 mask、非法字节 mask；标量或块状态机再消费这些候选位置，恢复精确错误偏移。

错误报告决定输出结构。如果只需要“是否存在非法字节”，mask reduce 成一个布尔值即可；如果需要第一个非法位置，必须用 mask 找到最低位 set bit 并加上块起点；如果需要所有错误位置，输出就是 position list 或 side buffer。不同语义对应不同成本，不能把“块中有错”冒充“第一个错误位置”。

稳定 filter 阈值过滤可以用向量比较快速生成 mask，但紧凑输出位置不是天然存在的。保留元素要写到连续输出数组中，第 k 个保留元素的位置取决于前面有多少元素保留。这就是 scan 的问题。

SIMD filter: mask 到输出位置



比较只解决“哪些 lane 保留”；stable filter 还必须用块内 rank 和块间 scan 解决“写到哪里”。

图示：本图要证明的命题是：SIMD filter 的核心不只是向量比较，而是 mask、rank、scan 和稳定输出合同的组合。

选择率会改变策略。全保留时，filter 接近 copy，压缩写可能多余；全丢弃时，输出很小但仍要读完整输入；随机 50% 命中时，分支难预测，mask 可能明显受益；低选择率时，selection vector 可能比复制整条记录更好。验证必须覆盖选择率矩阵，否则无法判断策略边界。

8.9 数值语义、溢出和特殊指令

SIMD 指令不只影响速度，也可能携带特定数值语义。整数可以有回绕、饱和、拓宽累加和窄化存储；浮点可能涉及 FMA、舍入模式、NaN、Inf、有符号零和 fast math。只要向量版本和标量 reference 的数值语义不同，就必须写明这是允许的改变，还是实现错误。

浮点归约 浮点加法不满足数学意义上的结合律。串行左折叠、四累加器、pairwise sum、SIMD 水平归约、多线程树形合并，可能得到不同低位结果。若业务要求逐 bit 可复现，就要固定合并树并控制编译选项；若允许误差，就要写绝对误差、相对误差或 ULP 边界。不能把 fast math 当作默认性能开关。

整数宽度 窄整数 lane 很容易在中间结果中溢出。8 位或 16 位数据常用于压缩和量化，但累加时可能需要拓宽到 32 位或 64 位。饱和加法和普通无符号回绕不是同一语义。解析、计数、图算法和哈希都要明确宽度：std::uint32_t 的回绕、std::int32_t 的溢出、宽累加后的窄化规则，不能靠平台习惯猜。

特殊指令要有 reference SIMD 常提供饱和加法、比较打包、字节 shuffle、popcount、compress store 和近似倒数。这些指令能明显提速，也更容易改变边界行为。每个特殊指令路径都要有慢速 reference 和压力输入：最大值、最小值、负数、NaN、舍入边界、非法字符、尾部和非对齐。reference 不是低级版本，而是语义锚点。

8.10 平台分派和回退

不同机器支持不同向量能力。x86 有 SSE、AVX、AVX2、AVX-512；同一 ISA 在不同微架构上的吞吐、延迟、降频和 cache 行为也不同。公开 API 不应暴露某个平台的寄存器类型。业务层需要的是同一个语义合同，底层可以有 scalar、auto-vectorized、native backend 多个实现。

Listing 8.13 VectorKernelPlan 的接口形态

```

1 enum class TailPolicy {
2     ScalarRemainder,
3     MaskedTail,
4     PaddedInput,
5 };
6
7 enum class NumericPolicy {
8     BitwiseStable,
9     FixedReductionTree,
10    BoundedError,
11    WrappingInteger,
12    SaturatingInteger,
13 };
14
15 struct VectorKernelPlan {
16     TailPolicy tail = TailPolicy::ScalarRemainder;
17     NumericPolicy numeric = NumericPolicy::BitwiseStable;
18     bool requires_aligned_input = false;
19     bool allows_destructive_overlap = false;
20     bool has_scalar_fallback = true;
21     bool reports_exact_error_position = true;
22 };

```

这个接口形态把 SIMD 的前置条件变成可审查对象。若某个 backend 只能处理

对齐输入，它就只能作为 fast path，并必须有 fallback；若某个 backend 只返回块级错误位置，它不能替代要求精确错误的 reference；若某个浮点 backend 允许重排，它要标注误差策略。

按长度分派 短输入不一定适合 SIMD。函数调用、dispatch、对齐检查、mask 准备和尾部处理可能比标量循环还贵。长输入更容易摊薄固定成本。长度阈值应由输入分布和 benchmark 决定，而不是由向量宽度拍脑袋决定。证据要区分小输入 p99 和大输入吞吐，防止优化热 batch 的同时伤害短请求。

强制 backend 测试 运行时派发不能只测试当前机器最快路径。测试应能强制 scalar、auto-vectorized、AVX2、AVX-512 或其他可用 backend，所有 backend 使用同一组 reference 输入。否则 fallback 长期无人执行，一旦部署到不同机器就会出错。dispatch 层本身也要测试：不支持目标指令时不能误入 native 路径。

8.11 从标量循环推到向量执行

SIMD 不应先从 intrinsics 开始。更可靠的推导顺序是：先写清标量语义，再确认循环迭代之间是否独立，接着分析地址流是否连续，最后才看编译器是否能把多次标量操作合并成一次向量操作。这个顺序能把“用了向量指令”还原成机器真正执行的动作：每轮读取多少字节、处理多少 lane、怎样处理 mask、尾部是否回到标量、结果是否仍满足原来的合同。

数组 map：为什么能批量执行 一对一 map 最容易向量化，因为第 *i* 个输出只依赖第 *i* 个输入。若输入输出不破坏性重叠，编译器可以把连续四个或八个元素合成一个向量 load，执行一组并行乘加，再用一个向量 store 写回。这里成立的不是“SIMD 魔法”，而是三个底层事实同时成立：迭代没有循环携带依赖；地址是等步长连续访问；每个 lane 做同样运算。

Listing 8.14 标量循环到向量循环的机器形状

```
scalar:
    load input[i]
    multiply
    add
    store output[i]
    i += 1

vector:
    load input[i ... i+lanes-1]
    multiply lanes in one vector register
    add lanes in one vector register
    store output[i ... i+lanes-1]
    i += lanes
```

反汇编要验证的正是这件事：主循环是否出现连续 vector load/store，步长是否等于向量宽度，循环体里有没有意外调用、标量分支、gather 或栈 spill。若只是源码看起来“批量”，机器码仍是一元素一支，就没有真正获得 SIMD 的执行形态。

ASCII 分类：mask 是状态压缩，不是语义结果 字节分类可以把 16、32 或 64 个字节同时比较，得到一个 bit mask。mask 只是把“哪些 lane 满足条件”压缩成位图；后续还要把位图转回绝对偏移，维护第一个错误位置，累积分隔符数量，并处理尾部和跨块状态。也就是说，SIMD 只加速了宽扫描，不能替代 parser 的语义状态机。

Listing 8.15 mask 到语义结果的推导

```

vector load bytes[i ... i+31]
separator_mask = compare(byte == ' ' or byte == '\n')
illegal_mask   = compare(byte not allowed)

while separator_mask != 0:
    bit = ctz(separator_mask)
    emit separator position i + bit
    clear bit

if illegal_mask != 0:
    first_error = min(first_error, i + ctz(illegal_mask))

```

这段伪代码显示了 SIMD 内核最容易被忽略的成本：`ctz` 枚举 `mask`、位置恢复、错误路径、`tail` 处理和跨块状态合并。若输入很短，这些固定成本可能超过向量主循环收益；若错误很多，冷路径会变热；若 API 要求精确错误位置，就不能只返回“本块有错”。

稳定 filter：比较容易，写出位置难 Filter 的第一步是判断每个元素是否保留，这一步天然适合向量比较。真正困难的是稳定紧凑输出：保留下来的元素要按原顺序写到连续位置。每个元素的输出位置等于它之前被保留元素的数量，这就是循环携带依赖。向量比较只能生成 `mask`，不能凭空知道全局 `rank`；因此稳定 filter 往往需要块内 `rank`、块间 `count/scan`、最后 `write` 三阶段。

浮点归约：向量化会改变合并树 浮点加法不是实数加法，重排合并顺序可能改变舍入结果。标量左折叠、四累加器、`pairwise sum`、自动向量化和 `fast math` 生成的是不同合并树。向量化归约不能只看速度，要先确定数值合同：是否要求 `bitwise` 稳定，是否允许固定误差界，`NaN` 和 `Inf` 怎样传播。若合同要求逐元素左折叠结果，编译器不能随意重排；若合同允许固定树形，SIMD 才有更大空间。

最小证据链 每个向量候选内核都应能留下同一条证据链：标量 `reference` 是什么；别名、尾部、对齐和数值前提是什么；选中的 `backend` 是什么；反汇编里的主循环长什么样；`checksum`、错误位置或误差是否和合同一致；瓶颈是否从计算移动到内存、前端、`mask` 枚举或尾部。证据链越短越好，但不能断在“跑得更快”。

8.12 源码阅读路线

源码阅读适合围绕一个具体现象展开。若编译器拒绝向量化，阅读 LLVM `LoopVectorize` 或 GCC 向量化诊断文档时，只追这一条拒绝原因：别名、依赖、控制流还是成本模型。若某个成熟解析器很快，阅读它如何组织 `scalar fallback`、`mask`、`tail`、`dispatch` 和错误路径，而不是复制整段 `intrinsics`。若某个数据库执行器使用 `selection vector`，重点看 `filter` 输出身份如何传给后续算子。

源码阅读任务

可选入口：

1. LLVM `LoopVectorize` 或 `SLP vectorizer`：从一条未向量化报告追到源码诊断，解释编译器需要哪些证明。
2. `simdjson`、成熟 CSV/JSON 解析器或数据库执行器：找出字节分类、结构字符 `mask`、错误回退和批次边界。
3. `DuckDB`、`ClickHouse` 或 `Velox`：阅读 `selection vector`、`column vector` 和 `filter` 的实现边界，说明 `mask` 怎样转成后续算子的输入身份。
4. 标准库或数学库的多版本派发：观察 `scalar reference`、平台 `backend`、`feature detection` 和测试组织。

源码阅读要从一个具体现象开始，最后回到当前项目中应采用、简化或拒绝的设计。

8.13 实现边界

Compute Systems Engine 在这一阶段留下一个可替换内核族，而不是把平台代码散落到业务逻辑中。最低形态包括三个对象：`VectorKernelPlan` 描述语义和平台前提；`KernelVariant` 注册 scalar、auto 和 native backend；`VectorKernelEvidence` 保存输入、反汇编摘要和判读结论。业务层调用稳定接口，验证层强制所有 backend，证据层解释为什么选择默认路径。

ASCII 分类切片 实现切片可以只处理一块字节并输出三类信息：分隔符位置、非法字节总数、第一个非法字节偏移。标量 reference 逐字节生成完整结果；自动向量化版本保持普通 C++ 循环但简化控制流；native backend 可选。尾部先用标量处理。验证重点放在长度矩阵、非对齐起点、页尾安全和强制 backend 上；性能数字只在语义边界全部通过后才具有解释价值。

从单核到线程 SIMD 和多线程不是替代关系。SIMD 提高单核每条指令处理的数据量，多线程使用多个核心。稳妥顺序是先固定标量语义，再让单线程内核批量友好，再扩展到线程池。若同时改变布局、SIMD 和线程，速度变化难以归因，错误也更难缩小。单核向量化后，瓶颈可能转向内存带宽；再加线程时扩展曲线可能提前变平，这不是失败，而是系统抵达新的资源边界。

8.14 完整执行账本：ASCII 分类从标量到 SIMD

把一个字节分类内核写完整，能把 SIMD 的所有关键边界串起来：标量 reference、批量 mask、尾部、错误位置、跨块状态、平台回退和证据字段。这里的目标不是展示某个 ISA 的 intrinsics，而是证明同一个语义怎样被批量执行。

标量合同 输入是一段日志字节，输出三个事实：分隔符位置列表、非法字节总数、第一处非法字节偏移。非法字节定义为既不是可打印 ASCII，也不是换行、回车、制表或空格的字节。分隔符定义为空格和换行。这个合同故意简单，但已经包含 SIMD 常见难点：需要所有分隔符位置，需要第一个错误位置，需要处理尾部和空输入。

Listing 8.16 ASCII 分类的标量语义伪代码

```
first_error = none
error_count = 0
separator_positions = []

for i in [0, bytes.size):
    b = bytes[i]
    is_separator = (b == ' ' or b == '\n')
    is_allowed = is_printable_ascii(b) or b in [' ', '\n', '\r', '\t']

    if is_separator:
        separator_positions.append(i)

    if not is_allowed:
        error_count += 1
        if first_error is none:
            first_error = i
```

标量 reference 的价值在于明确三个输出的精度。优化版本不能把 `first_`

`error=137` 退化成“第 128..159 块有错误”，也不能只返回“存在错误”。如果业务只需要块级错误，可以另设合同；不能在优化时悄悄降低诊断精度。

批量分类的状态 向量版本按块读取字节。每个块产生两个 mask：separator mask 和 illegal mask。mask 的第 k 位对应块内第 k 个字节。全局位置等于 `block_begin+k`。这条映射是恢复错误位置的核心。

Listing 8.17 向量块处理的语义伪代码

```
for each full vector block:
    bytes = load block
    separator_mask = classify_separators(bytes)
    illegal_mask = classify_illegal(bytes)

    while separator_mask has set bit:
        k = index_of_lowest_set_bit(separator_mask)
        separator_positions.append(block_begin + k)
        clear bit k

    if illegal_mask != 0:
        error_count += popcount(illegal_mask)
        if first_error is none:
            first_error = block_begin + index_of_lowest_set_bit(illegal_mask)

process tail with scalar reference logic
```

这个伪代码没有依赖具体指令，但已经固定了 SIMD 后端必须满足的语义。后端可以用比较、查表、饱和运算或范围判断生成 mask；只要 mask 与标量分类一致，后续位置恢复就稳定。若某个平台没有快速 `compress` 或 `movemask`，也可以退回标量枚举 mask 位。

尾部策略 第一版使用标量尾部。主体循环只处理完整向量块，剩余不足一块的字节交给 reference 逻辑。这样最容易证明不越界，也容易和任意 `std::span<const std::byte>` 接口兼容。后续若改成 masked tail 或 padding，需要重新证明被 mask 掉的 lane 不会触发非法访问，并把策略写进 `VectorKernelPlan`。

Listing 8.18 尾部处理账本

```
vector_width = 32
count = 79

full blocks:
    block 0 covers [0, 32)
    block 1 covers [32, 64)

tail:
    scalar covers [64, 79)
```

尾部测试必须覆盖 0、1、31、32、33、63、64、65。长度为 31 时主体循环不应执行；长度为 32 时尾部为空；长度为 33 时主体和尾部都执行。很多越界和重复统计 bug 都在这些长度出现。

非对齐和跨页边界 外部输入的起点不一定按向量宽度对齐。第一版应允许非对齐 load，或者在检测到不支持时走标量 fallback。更危险的是跨页边界：即使 CPU 支持非对齐访问，错误的 masked tail 或过读 padding 也可能跨到未映射页。安

全策略是：公开 API 不允许读取 `span` 边界之外的任何字节；只有内部 `batch` 明确分配了 `padding`，才能启用 `padded input backend`。

跨块状态 ASCII 分类本身没有复杂跨块状态，但真实解析有。例如 UTF-8 多字节序列可能跨块，CSV 引号状态可能跨块，Windows 换行 `\TU\textbackslashashr\TU\textbackslashashn` 可能跨块。块级 SIMD 只能负责找候选和局部分类；跨块状态要保存在 `block state` 中，由下一个块继续。

Listing 8.19 块状态摘要的语义形态

```
BlockState:
  previous_was_cr
  inside_quote
  utf8_continuation_needed
  absolute_offset_of_block
  first_error_offset_or_none
```

如果优化版本不保存这些状态，错误会集中出现在块边界。比如 `\TU\textbackslashashr` 在块末尾、`\TU\textbackslashashn` 在下一块开头时，若两块互不认识，就可能把合法换行拆成两个事件。向量化解析器的难点往往不在单块比较，而在块与块之间的语义接缝。

证据字段 ASCII 分类至少保留：`input bytes`、`alignment offset`、`vector width`、`backend`、`tail policy`、`separator count`、`error count`、`first error`、`mask extraction method`、`scalar fallback count`、`checksum`、短输入 `p99` 和长输入吞吐。这样才能判断速度变化是主循环变快、尾部变慢、错误路径退化还是 `dispatch` 成本上升。

Listing 8.20 ASCII 分类证据摘要

```
backend=avx2
vector_width=32
tail_policy=ScalarRemainder
alignment_offset=7
input_bytes=1048576
separator_count=223141
error_count=3
first_error=917
scalar_tail_bytes=17
checksum=0x92df4120
```

失败路径分类 向量 `backend` 可能失败在三类地方。第一，语义失败：`mask` 和 `reference` 不一致，`first error` 或 `separator positions` 错。第二，边界失败：尾部、非对齐、跨页或空输入出现越界。第三，证据失败：速度数字有了，但反汇编、`backend`、输入分布和 `checksum` 没保存。三类失败都要阻止优化进入默认路径。

从分类到完整 parser 字节分类只是 `parser` 的前半段。后半段要根据分隔符位置切字段、根据错误 `mask` 生成诊断、根据跨块状态处理引号、转义、UTF-8 或 `schema`。更成熟的结构是两阶段：SIMD 阶段生成候选 `mask` 和块摘要；语义阶段消费候选，生成 `RecordId`、`field offsets` 和 `diagnostics`。这样 SIMD 负责宽扫描，语义状态机负责精确合同。

常见错误 第一，把完整解析器一次性 SIMD 化，忽略跨块状态和错误恢复。第二，只测长度刚好是向量宽度倍数的大输入。第三，手写 `native backend` 后删除 `scalar reference`。第四，浮点归约打开 `fast math` 却不写误差合同。第五，把

gather 当作随机访问的万能修复。第六，过度展开导致前端供给变差。第七，让平台寄存器类型泄漏到业务接口。每一个错误都不是语法问题，而是合同、硬件和证据链断裂。

8.15 排障案例：向量版本为什么没有变快

向量化失败不只表现为结果错误，也可能表现为“代码用了 SIMD，但时间没有下降”。这时不要先换更宽 ISA，而要沿着机器瓶颈回查。向量版本可能减少了算术指令，却被内存带宽限制；可能生成了 gather，仍在等随机地址；可能主循环很快，但尾部、派发和对齐检查占据短输入时间；可能过度展开让 I-cache 和译码压力上升；可能平台进入较低频率状态，使 AVX 宽指令的收益被频率变化抵消。每一种原因都需要不同证据。

第一层：确认实际运行的是哪个 backend 运行时派发会让排障复杂。只写 `backend=avx2` 不够，还要证明这次调用真的进入 AVX2 路径：保存 CPU feature 检测结果、dispatch 决策、强制 backend 路径和关键反汇编摘要。若输入很短，按长度分派可能进入 scalar 路径；若对齐或尾部策略不满足，native backend 可能拒绝执行；若编译选项没有启用目标 ISA，所谓向量版本可能只是普通标量代码。

Listing 8.21 backend 排障字段

```
requested_backend = native
selected_backend = avx2
cpu_features = avx2,bmi2,popcnt
length_threshold = 256
input_length = 1048576
alignment_offset = 7
fallback_reason = none
disassembly_summary = vector loads + movemask path present
```

这些字段能避免一个常见误判：以为 SIMD 慢，其实 SIMD 根本没有跑；或者以为 scalar 快，其实短输入派发开销被算进 native 路径。

第二层：用 Roofline 判断是否被带宽封顶 数组 map 若每个元素只做一次简单运算，SIMD 可能把指令数降下去，但每个元素仍要读写相同字节。当实测吞吐已经接近可持续内存带宽，再换更宽向量通常收益很小。此时更有效的方向可能是融合相邻算子、减少中间数组、改数据类型、压缩输入或改变布局，而不是继续写 intrinsics。

Listing 8.22 带宽封顶的判断形状

```
kernel:
  read input: 4 bytes per element
  write output: 4 bytes per element
  ops: 1 multiply + 1 add

observed:
  scalar uses fewer than expected compute resources
  vector version improves instruction count
  total bytes per second close to measured stream bandwidth

next direction:
  fuse producer/consumer or reduce bytes
```

相反，如果向量版本远低于带宽和计算下界，就要找其他等待：别名检查、尾部路径、分支、gather、TLB、前端、寄存器 spill 或同步。

第三层：检查编译器插入的运行时检查 为了证明别名安全，编译器可能在主循环前插入运行时区间检查：如果输入输出不重叠，走向量路径；否则走标量路径。长输入时这点成本可以忽略，短输入时检查和派发可能成为主成本。若接口能从类型层面表达只读输入和独占输出，检查可以减少；若无法保证，就应该保留检查并按长度选择路径。

反汇编里要看三件事：主循环是否使用预期向量宽度；循环前是否有大量地址比较和分支；尾部是否有多个 peel loop。自动向量化报告和反汇编必须一起读，不能只看“vectorized”一个词。

第四层：看寄存器压力和 spill 手写 SIMD 容易贪多：同时保留多个向量累加器、mask、shuffle 常量、临时索引和输出缓冲指针。寄存器不足时，编译器会把部分临时值 spill 到栈上，主循环反而增加额外 load/store。过度展开也会让代码体积变大，前端供给变差。排障时要在反汇编中查栈访问、循环体长度和不必要的保存恢复。

症状	可能原因	优先检查
短输入变慢	dispatch、尾部、运行时检查成本高	长度分派阈值、短输入 p99
长输入不变快	内存带宽封顶或 gather 等待	字节账本、访问形状、带宽基准
热点函数变大	展开过度或多版本膨胀	I-cache、反汇编长度、前端指标
向量主循环有栈访问	寄存器压力或 spill	临时变量数、累加器个数
错误输入慢很多	冷路径混进热循环	错误率矩阵、错误报告阶段时间

第五层：把速度问题重新归入合同 有些向量版本慢，是因为它保留了完整合同；有些向量版本快，是因为它偷删了合同。排障不能只问“为什么没快”，也要问“慢的部分是不是我们必须购买的语义”。精确 first error、稳定输出位置、浮点 bitwise 可复现、任意非对齐 span、安全尾部、fallback backend，这些都是真成本。若业务愿意放松合同，例如只报告块级错误或允许固定误差，可以另设接口；不能在同一个接口下用速度压力悄悄改变语义。

最后的优化判读应把决策写清：当前向量版本在长输入上受内存带宽限制，短输入由 dispatch 和尾部主导；保留 scalar 短路径、融合相邻 map、维持精确错误位置；暂不引入更宽 backend，因为证据显示瓶颈不在算术吞吐。这样的结论比盲目追求更宽指令更有工程价值。

完整向量化账本：从编译器拒绝理由到接口改造 自动向量化失败时，最差的反应是直接改写成 intrinsics。更稳的路径是先让编译器说明拒绝理由，再判断它拒绝的是语义问题、接口问题还是成本模型问题。比如一个 stable filter 循环没有向量化，可能因为输出位置依赖前面保留数量；一个数组 map 没有向量化，可能因为输入输出可能别名；一个 parser 循环没有向量化，可能因为循环携带状态跨字节传播。三种原因对应完全不同的改法。

Listing 8.23 自动向量化诊断要翻译成工程原因

```
compiler report:
  loop not vectorized: unsafe dependent memory operations

engineering interpretation:
```

input and output ranges may overlap
 compiler cannot prove independent lanes

possible fixes:

split API into non-overlap fast path and safe fallback
 materialize output positions by count/scan/write
 keep scalar reference for overlap and boundary cases

这个翻译过程比“编译器不够聪明”更有用。若接口接收两个普通指针，编译器确实难以证明它们不重叠；若业务合同本来就允许原地修改，那么强行声明 non-overlap 会制造 bug；若业务合同不允许重叠，就应该把这个事实写进类型、断言或派发层。向量化从这里开始变成接口设计，而不是指令技巧。

Listing 8.24 把别名事实写进接口边界

```
1 enum class OverlapPolicy {
2     MayOverlap,
3     NonOverlapping
4 };
5
6 struct TransformView {
7     std::span<const std::int32_t> input;
8     std::span<std::int32_t> output;
9     OverlapPolicy overlap = OverlapPolicy::MayOverlap;
10 };
```

当 `overlap=NonOverlapping` 时，fast path 可以走向量版本；当调用方无法证明这一点时，走保守 reference 或先拷贝。这个设计购买的是可审计的前提，而不是把 `restrict` 一类假设藏进注释。证据中也要记录 fast path 被拒绝多少次，拒绝原因是什么，否则 fallback 会在生产路径里悄悄吞掉收益。

标量汇编和向量汇编要对照读 读向量化结果时，先找标量循环的动作：每轮 load 几次、compare 几次、branch 几次、store 几次、是否有调用和栈访问。再找向量循环的动作：向量 load 宽度、比较或算术指令、mask 提取、循环步长、尾部路径、是否出现 gather、是否有 peel loop 和运行时别名检查。只有两份对照同时存在，才能说清优化改变了什么。

Listing 8.25 反汇编对照记录

```
scalar loop:
    loads per element
    branches per element
    calls in loop
    store pattern

vector loop:
    vector width and step
    vector load/store instructions
    compare/mask/extract path
    tail handling
    runtime alias checks
    scalar fallback range
```

如果向量循环有 `gather`，要回到第 3 章的地址账本：`gather` 只是把多个随机地址放进一条指令语义里，不会让随机内存变成连续内存。若向量循环有很多栈访问，要回到寄存器压力；若主循环很短但前置检查很长，要回到短输入阈值；若尾部代码复杂，要回到 `tail` 策略。汇编对照不是为了炫技，而是把“用了 SIMD”拆成实际机器动作。

mask/tail/dispatch 三件事必须进入同一份测试 很多向量 bug 分别藏在三处：`mask` 位和元素位置映射错，`tail` 越界或漏元素，`dispatch` 在某个平台或短输入上选错 backend。测试如果只覆盖大数组合法输入，很容易全部漏掉。最小测试矩阵应把长度、对齐、页尾、错误位置和 backend 强制组合起来。

Listing 8.26 向量测试矩阵的最小维度

```
length:
    0, 1, lane-1, lane, lane+1, 2*lane-1, 2*lane, 2*lane+1

address:
    aligned, unaligned, near page end, padded internal batch

error position:
    first byte, middle byte, last byte, multiple bytes, no error

backend:
    force scalar, force auto-vectorized, force native if available
```

每个组合都不一定要跑巨大输入；边界矩阵关注正确性和越界，长输入矩阵关注吞吐和瓶颈。证据要把两者分开。若一个 AVX2 backend 长输入很快，但 `near page end` 输入失败，它不能进入默认路径；若短输入 `p99` 变差，可以保留长输入 `fast path` 和短输入 `scalar path`。

把向量化收益绑定到阶段占比 一个内核快 3 倍，不等于端到端快 3 倍。若该内核只占阶段 10%，理论上最多给端到端带来有限收益；若它改变输出布局，可能让后续阶段更快或更慢。向量化判读必须写优化前后阶段占比、输出格式是否改变、后续 I/O 和 `manifest` 是否受影响。这样才能防止把精力花在已经不是瓶颈的局部内核上。

Listing 8.27 向量化收益的端到端账本

```
before:
    classify_bytes = 38% of stage time
    parse_semantics = 44%
    write_manifest_candidate = 8%
    other = 10%

after:
    classify_bytes = 15%
    parse_semantics = 59%
    write_manifest_candidate = 12%
    other = 14%

interpretation:
    SIMD moved bottleneck to semantic parser
    next work should reduce state-machine cost or batch boundaries
```


这类账本会让优化方向更清楚。字节分类变快后，语义 parser 成为瓶颈，不代表 SIMD 失败；它说明下一步要研究跨块状态摘要、字段边界表示或错误回查，而不是继续扩大向量宽度。

SIMD 接口要能拒绝不满足前提的输入 向量 backend 不应假装能处理所有输入。如果某个实现要求内部 padding，就只接受带 padding 的内部 batch；如果它只支持 ASCII，就不能被 UTF-8 parser 无条件调用；如果它只返回块级错误位置，就不能替换要求精确偏移的接口；如果它依赖非重叠输出，就要在 debug 或派发层检查。拒绝 fast path 不是失败，而是保持合同。

Listing 8.28 向量 fast path 的前置检查

```
can_use_native_backend(input):
    require backend feature available
    require input length >= threshold
    require tail policy can safely handle this span
    require numeric policy matches requested contract
    require error reporting precision is sufficient
    require destructive overlap is absent or explicitly allowed
```

这些检查会带来固定成本，所以短输入可能走标量路径。把前置条件写成接口，比在注释里说“调用者保证”更容易维护，也能让测试强制覆盖 fallback。

8.16 ISA 现实：宽向量不是单调收益

SIMD 常被误解成“寄存器越宽越快”。真实机器上，向量宽度只是一个变量。宽向量会改变 load/store 带宽、执行端口、寄存器压力、前端代码体积、频率策略、尾部处理和平台覆盖范围。一个 AVX2 内核可能比标量快很多，也可能被内存带宽封顶；一个 AVX-512 内核可能让 mask 处理更自然，也可能因为频率和端口压力让端到端收益变小。

从寄存器宽度推导 lane 数 假设处理 32 位整数。128 位寄存器一次容纳 4 个 lane，256 位寄存器一次容纳 8 个 lane，512 位寄存器一次容纳 16 个 lane。lane 数增加后，主循环迭代次数减少；但每次迭代要搬运的字节更多，尾部可能更复杂，mask 提取和输出压缩也可能更贵。若内核每个元素只做一两次简单操作，lane 数增加很快把瓶颈推到 load/store。

Listing 8.29 向量宽度推导的第一张账本

```
element type: int32

128-bit vector:
    lanes = 4
    bytes per load = 16

256-bit vector:
    lanes = 8
    bytes per load = 32

512-bit vector:
    lanes = 16
    bytes per load = 64

question:
    does the kernel wait for arithmetic, load/store, shuffle, or memory?
```

因此平台分派不能只按“支持最宽 ISA”选择。更稳的策略是按内核类型、输入长度、数据布局和机器特征选择 backend：小输入走标量；中等输入走低开销向量；长输入且计算密度足够时才启用更宽 backend；随机访问或 gather 密集内核优先改布局。

AVX 降频和阶段收益 某些平台执行宽向量指令时可能降低核心频率，或者让同一核心上的其他线程受影响。频率策略随架构和指令类型变化，软件不能假定所有机器一样。不必夸大这个问题，但必须把它纳入证据：如果 AVX-512 版本每条输入指令更少，耗时却没有下降，要检查频率、端口、内存和阶段占比。

Listing 8.30 宽向量收益审计字段

```
backend
vector_width_bits
input_bytes
stage_percent_before
stage_percent_after
cycles_per_byte
instructions_per_byte
bytes_per_second
frequency_observed_if_available
tail_policy
fallback_count
```

若无法读取频率，也不能直接断言降频。可以写“frequency evidence unavailable”，再用指令数、周期、带宽和阶段占比判断。性能工程要区分证据和推测。

gather 和 scatter 不是随机访问解药 Gather 把多个地址的 load 放进一条向量指令语义里；scatter 把多个地址的 store 放进一条向量指令语义里。它们简化了代码形状，却不会把随机地址变成连续地址。若每个 lane 指向不同 cache line，硬件仍要等待多个独立地址；若每个 lane 指向不同页面，TLB 压力仍存在；若 scatter 写到共享 line，还会碰到一致性所有权。

Listing 8.31 gather 账本必须回到地址序列

```
for vector lanes:
    lane 0 loads base[index0]
    lane 1 loads base[index1]
    lane 2 loads base[index2]
    ...

if indices are random:
    line utilization may be low
    prefetch may be weak
    TLB coverage may dominate
    gather instruction does not remove these costs
```

这就是为什么第 6 章的数据布局优先于第 7 章的指令技巧。连续热列让普通 vector load 足够有效；对象图或随机索引即使用 gather，也可能只是把等待包装成更复杂的指令。

shuffle、permute 和 compress 的真实成本 很多非平凡 SIMD 内核不只是加

减乘除，还要重排 lane。字节分类可能需要 `movemask`；stable filter 可能需要 `compress` 或查表压缩；UTF-8 检查可能需要跨 lane 移位；解析器可能需要把 mask 位转换成位置列表。这些动作消耗执行端口、常量表、寄存器和前端指令。若一个内核的 shuffle 成本接近或超过算术成本，继续扩大向量宽度未必有效。

动作	购买的能力	常见风险
wide load/store	一次搬运更多连续元素	内存带宽、对齐、尾部
movemask	把 lane 判断变成位图	mask 枚举、位置恢复成本
shuffle/permute	重排 lane 或字节	端口压力、常量和寄存器压力
compress	把保留元素压紧输出	平台差异、输出位置合同
gather/scatter	表达多地址访问	随机地址等待仍在

一个后端选择矩阵 向量 backend 应像运行时策略一样可观测，而不是散在条件编译里。一个实用矩阵可以这样写：

Listing 8.32 SIMD backend 选择矩阵

```
if input length < short_threshold:
    use scalar
else if contract requires exact ordered output and compress is unavailable:
    use scalar or two-pass count/scan/write
else if addresses are contiguous and backend feature is available:
    use native vector backend
else if compiler auto-vectorized loop passes disassembly audit:
    use auto-vectorized backend
else:
    use scalar reference
```

这个矩阵把性能前提和语义前提写在一起。它允许快路径拒绝不合格输入，也允许后续添加平台 backend。默认路径不应因为支持某个 ISA 就无条件进入 native backend；默认路径应该选择当前输入和合同下最可靠的实现。

从 ISA 到实现判读 最终判读要能回答四个问题。第一，这次运行实际选择了哪个 backend。第二，选择理由是什么：长度、特性、对齐、合同、fallback。第三，主循环机器形状是否符合预期：连续 load、mask、tail、无意外调用。第四，端到端瓶颈是否真的在这个内核。若这四个问题没有答案，SIMD 代码仍是孤立片段，不应成为系统默认路径。

8.17 默认路径：向量内核怎样进入系统

向量内核进入默认路径之前，必须经过一组固定审查。审查不是为了拖慢优化，而是为了防止一个只在理想输入上很快的内核破坏边界输入、短输入、错误诊断、跨平台回退和数值语义。SIMD 代码一旦散入业务层，后续修复成本很高；因此默认路径应只接受语义合同、平台前提和证据链都完整的内核。

审查一：语义同一性 第一项审查是语义同一性。Scalar reference、自动向量化版本、native backend 必须在同一输入矩阵上输出同样结果，或在明确误差合同内输出可接受结果。比较对象不能只包含最终 checksum，还要包含错误位置、separator count、selection vector、输出顺序、尾部元素和 diagnostics checksum。若某个 backend 只在“无错误大输入”上通过，它只能作为实验路径，不能成为默认路径。

审查二：边界输入 边界输入要比正常输入更重要。长度矩阵覆盖 0、1、lane-1、lane、lane+1、2*lane-1、2*lane、2*lane+1；地址矩阵覆盖对齐、非对齐、接近页尾、内部 padding 和无 padding；语义矩阵覆盖首元素错误、尾元素错误、全合

法、全非法、短行、长行和跨块状态。边界输入的目标不是提高覆盖率数字，而是攻击 SIMD 最容易出错的三个位置：尾部、mask 映射和跨块接缝。

审查三：平台前提 平台前提必须在运行时和构建时都可见。构建时记录编译器、目标 ISA、优化选项和是否启用 fast math；运行时记录 CPU feature、selected backend、fallback reason、vector width 和长度阈值。若 native backend 不可用，fallback 必须通过同一语义测试。若 dispatch 只在当前机器上验证，要写明未验证平台。平台前提是合同的一部分，不是编译脚本细节。

审查四：短输入和长输入分别看 很多向量内核在长输入上快，在短输入上慢。短输入受函数调用、dispatch、尾部、运行时检查和冷代码影响；长输入受内存带宽、前端供给和端口吞吐影响。默认策略可以是短输入走 scalar，长输入走 native。阈值必须有来源，不能把一个大输入 benchmark 推广到所有调用。

审查五：复杂度预算 向量路径会增加代码体积、测试矩阵、平台分支和排障成本。若收益只在极窄输入上出现，或者端到端阶段占比很小，就不应进入默认路径。复杂度预算可以写成三行：该内核占端到端多少比例，native backend 在目标输入上带来多少阶段收益，维护成本和 fallback 风险是否可接受。性能工程不只是添加快路径，也包括拒绝不值得维护的快路径。

Listing 8.33 向量内核上线审查摘要

```
semantic:
  scalar reference passed
  native backend passed full boundary matrix
  diagnostics checksum equal

platform:
  feature detection recorded
  scalar fallback forced and passed
  dispatch threshold measured

performance:
  short input p99 not worse than scalar threshold
  long input throughput improves target stage
  end-to-end bottleneck movement explained

decision:
  default for large ASCII blocks
  scalar for short blocks and unsupported platforms
```

上线后仍要保留 reference Native backend 进入默认路径后，reference 不能删除。Reference 用于回归、故障复现、跨平台验证和语义变更审查。若后来输入合同从 ASCII 扩展到 UTF-8，或错误报告从第一个错误扩展到全部错误，reference 是判断 native backend 是否仍可使用的锚点。删除 reference 等于删除对优化路径的约束。

8.18 展开：SIMD 的指令级资源账本

SIMD 不是“一个指令顶多个指令”这么简单。每个向量指令仍然要经过前端取指和解码，占用执行端口，读写寄存器，访问 load/store 单元，处理跨 lane 重排和尾部。高质量 SIMD 报告至少要把每元素资源账本写出来。

latency 和 throughput 不是同一个量

指令 latency 是一个结果从输入可用到输出可用的时间；throughput 是流水线稳定后每周期能发射或完成多少同类指令。一个内核如果有长依赖链，就受 latency 限制；如果有很多独立操作，就更接近 throughput 限制。

Listing 8.34 单累加和多累加的差异

```
single accumulator:
    s = s + a[i]
    next add depends on previous s
    latency chain is exposed

four accumulators:
    s0 += a[i+0]
    s1 += a[i+1]
    s2 += a[i+2]
    s3 += a[i+3]
    independent chains overlap
```

向量化归约也有同样问题。向量寄存器内先做多 lane 累加，最后 horizontal reduce。主循环可能很快，尾部和最终横向归约却不可忽略。短输入上，横向归约和 dispatch 成本可能吃掉收益。

load/store 吞吐和 cache line 账本

对简单 map 内核，例如 $y[i]=a*x[i]+b$ ，每个元素读 4 字节、写 4 字节，做 1 次乘法和 1 次加法。AVX2 每次 8 个 float：

Listing 8.35 AVX2 float map 的单迭代账本

```
per vector iteration:
    load x: 32 bytes
    store y: 32 bytes
    arithmetic: 8 mul + 8 add

per element:
    bytes moved: 8 bytes
    flops: 2
    arithmetic intensity: 0.25 flop/byte
```

若目标机器持续内存带宽是 50GB/s，这个内核的带宽上限约为 12.5GFLOP/s。即使用再宽的向量，只要每元素字节量不变、数据来自内存，它也会碰到带宽上限。宽向量能降低指令开销，但不能把 8 字节/元素变成 0。

shuffle 和 gather 是常见隐藏成本

许多 SIMD 代码慢在非算术指令。AoS 到 SoA 的字段抽取需要 shuffle；stable filter 需要 mask、rank、compress；随机索引需要 gather；稀疏输出需要 scatter。它们的吞吐、延迟和端口占用通常比普通 add/mul 更难看。

操作	资源风险	更好的上游改法
AoS 字段抽取	shuffle 多, load 有效载荷低	改 SoA 或 AoSoA
gather 随机读	多 cache line、多 TLB miss	重排索引, 批量化, 压缩工作集
scatter 写	store 合并差, 可能 false sharing	先生成局部 buffer, 再合并
horizontal reduce	跨 lane 合并成本	多累加器, 树形归约, 批量输出
mask compress	平台差异和输出位置计算	count/scan/write 两阶段

这就是为什么数据布局章节先于 SIMD 章节。若 layout 错, SIMD 只能用昂贵指令弥补; 若 layout 对, 普通连续 load/store 就能吃到大部分收益。

AVX2、AVX-512、VNNI、AMX 的正确位置

ISA 名词要落到 kernel 形态。AVX2 提供 256 位向量, 适合通用 float/int 处理; AVX-512 提供更宽寄存器和 mask 机制, 尾部和 compress 更方便, 但可能带来频率、代码体积和平台覆盖问题; VNNI 把低精度 dot-product 形态做成更合适的指令, 适合 int8 推理; AMX 更像 tile 级矩阵单元, 适合足够大的块乘, 编程模型和状态保存成本都不同。

能力	适合的问题	不适合被神化的原因
AVX2	连续向量 map、reduce、基础 GEMV	mask/tail 和 int8 dot 不如新 ISA 直接
AVX-512	宽向量、mask、compress、更多寄存器	平台覆盖、频率、短输入成本
VNNI	int8 dot-product、量化 linear	仍受权重读流和 packing 影响
AMX	大 tile GEMM、int8/bf16 矩阵块	小 batch decode GEMV 利用率可能低

第三册的 int8 GEMV/GEMM 会复用这张表。decode 单 token 常是 GEMV, 权重读流大、复用弱; prefill 多 token 更像 GEMM, 能更好利用 VNNI/AMX/tensor core 类单元。若报告只写“用了 AVX-512/VNNI 所以快”, 证据不够; 必须写 shape、bytes、packing、tail、线程和 roofline。

用静态资源模型补上端口账本

在没有 PMU 或 Top-Down 事件的机器上, 也不能完全凭感觉说“端口压力很大”。可以先用 `llvm-mca`、Intel Architecture Code Analyzer 的历史报告、`uops.info`、Agner Fog 指令表或厂商优化手册做静态资源模型。静态模型不能替代实测, 但它能回答一个很重要的问题: 如果数据都在 L1、分支预测完美、没有频率变化, 这段循环理论上会压在哪些执行资源上。

考虑一个 AVX2 `float` map 主循环, 每轮处理 8 个元素:

Listing 8.36 AVX2 map 主循环的资源建模输入

```
.intel_syntax noprefix
.text
.globl kernel
kernel:
.Lloop:
```

```

vmovups ymm0, ymmword ptr [rdi]
vfmadd213ps ymm0, ymm1, ymm2
vmovups ymmword ptr [rsi], ymm0
add rdi, 32
add rsi, 32
dec rcx
jne .Lloop
ret

```

一份静态报告可以用下面的命令生成：

Listing 8.37 llvm-mca 静态资源建模命令

```

llvm-mca \
-mtriple=x86_64-unknown-linux-gnu \
-mcpu=skylake \
-iterations=100 \
avx2_map.s

```

核心输出如下。这里保留 `ret` 是为了说明函数边界存在固定成本；分析长循环时应主要看循环体，短输入时则不能忽略边界成本。

Listing 8.38 llvm-mca 的静态吞吐和 uop 摘要

```

Iterations:      100
Instructions:     800
Total Cycles:    212
Total uOps:      1100

Dispatch Width:  6
uOps Per Cycle:  5.19
IPC:              3.77
Block RThroughput: 1.8

[uOps Lat RThroughput] Instructions:
1    7    0.50    vmovups ymm0, ymmword ptr [rdi]
1    4    0.50    vfmadd213ps ymm0, ymm1, ymm2
2    1    1.00    vmovups ymmword ptr [rsi], ymm0
1    1    0.25    add rdi, 32
1    1    0.25    add rsi, 32
1    1    0.25    dec rcx
1    1    0.50    jne .Lloop

```

这段输出不是为了炫工具，而是把口头判断变成资源账本。每个向量迭代做 8 个元素，读 32 字节、写 32 字节，做 8 个 fused multiply-add 语义。静态吞吐 `BlockRThroughput=1.8cycles` 表示在理想条件下每 8 个元素至少约 1.8 周期，折算约 `0.225cycles/element`。如果实测远高于这个数，原因通常不在 FMA 指令本身，而在内存层次、尾部、别名检查、前端、频率、线程或测量方式。

资源压力输出还会告诉我们哪些端口更紧张：

Listing 8.39 端口压力摘要，列名随 microarchitecture 变化

```

Resource pressure per iteration:

```

Port0	Port1	Port2	Port3	Port4	Port5	Port6	Port7
1.51	1.51	1.01	1.02	1.00	1.98	2.00	0.97

不要把这些数字机械解释成“Port6 一定是瓶颈”。静态模型默认数据可用、没有 cache miss、没有乱序窗口外的等待，也不建模 AVX 频率策略。正确用法是形成假设：这个循环除了 load/store 和分支控制外，FMA 本身不是明显瓶颈；若换成 gather、多次 permute 或 horizontal reduction，静态端口压力会转向 shuffle、load 和前端。接下来再用真实 benchmark、perfstat、反汇编和输入阶梯验证。

证据	能说明什么	不能说明什么
llvm-mca 吞吐指令 latency/throughput 表	理想 L1/无 miss 下的资源下限 单指令资源性质	真实 cache、TLB、频率、NUMA 编译器实际是否生成该指令
反汇编主循环 perf 计数 长度阶梯 benchmark	真实指令、tail、spill、调用 真实周期、指令、miss、分支 dispatch/tail/短输入成本	动态 miss 和分支概率 没有源码语义和地址账本 单独解释硬件端口归因

AVX2、AVX-512 和 AMX 的选择要看 shape

同一段业务不应默认选择“最宽 ISA”。可以把典型 kernel 按 shape 分成四类：

kernel 形态	首要资源	更合适的实现	主要风险
连续 map	load/store 带宽、前端	scalar/AVX2/AVX-512 分长度	短输入和频率
dot/reduce	依赖链、horizontal reduce	多累加器 AVX2/AVX-512	误差和尾部
int8 GEMV	权重读流、dot 指令	AVX2 packed、AVX-512 VNNI	packing 和带宽
大块 GEMM	数据复用、tile	VNNI/AMX/库调用	shape 不够大时 利用率低
随机 gather	cache/TLB/地址生成	改 layout、排序、分块	gather 只是最后手段

这张表和第三册的 decode/prefill 直接相连。decode 单 token 的 linear 常接近 GEMV：权重每步读一次，batch 小，AMX 或大 tile GEMM 的固定成本不一定划算。prefill 有多个 token，能把 $[T,H] \times [H,O]$ 做成更像 GEMM 的形态，数据复用更强，VNNI/AMX 或成熟 BLAS 后端才更容易吃满。评价一个 SIMD/AMX 路径时，必须同时报告 T,H,O,K、batch、数据布局、packing 是否复用、tail 比例和 fallback 规则。

反汇编审查清单

编译器报告说 vectorized 不等于最终运行路径满意。最小反汇编审查包括：

- 主循环是否存在连续 vector load/store。
- 是否有意外函数调用、除法、复杂 gather 或频繁 shuffle。
- 是否存在 runtime alias/alignment check，失败路径是否可能常走。
- tail path 是否正确处理 0、lane-1、lane、lane+1。
- 多 backend dispatch 是否真的选择到目标实现。

Listing 8.40 SIMD 报告的最小证据字段

```
source contract:
  no destructive overlap
  alignment guarantee or unaligned path
```

```

tail policy

assembly audit:
    vector loop instruction shape
    load/store count
    shuffle/gather/scatter count
    fallback branches

performance audit:
    bytes per element
    cycles per element
    bandwidth
    stage share
    scalar/reference comparison

```

SIMD 的工程成熟度不取决于写了多少 intrinsic，而取决于能否证明：语义相同，地址流更好，资源账本合理，端到端真的移动了瓶颈。

证据包：x86-64 AVX2 dot-product 的真实反汇编

下面这段 evidence 不是伪代码，而是在 x86-64 目标上编译一个 `dot8_avx2` 后得到的关键片段。输入长度为 8 时，程序输出 `240.0`，先证明语义没有丢。

Listing 8.41 AVX2 dot-product 的源码轮廓

```

extern "C" __attribute__((noinline))
float dot8_avx2(const float* a, const float* b, std::size_t n) {
    __m256 acc = _mm256_setzero_ps();
    for (; i + 8 <= n; i += 8) {
        __m256 va = _mm256_loadu_ps(a + i);
        __m256 vb = _mm256_loadu_ps(b + i);
        acc = _mm256_fmadd_ps(va, vb, acc);
    }
    float tmp[8];
    _mm256_storeu_ps(tmp, acc);
    float sum = horizontal_sum(tmp);
    for (; i < n; ++i) sum += a[i] * b[i];
    return sum;
}

```

反汇编中能看到三个事实：主循环连续读取两个 256-bit 向量，使用向量 FMA，最后做水平归约和 tail。

Listing 8.42 objdump 关键片段

```

0000000000401180 <dot8_avx2>:
4011a0: vmovups    (%rdi,%rax,4), %ymm1
4011a5: vfmadd231ps (%rsi,%rax,4), %ymm1, %ymm0
4011ab: add        $0x8, %rax
4011af: cmp        %rax, %rdx
4011b2: jae        4011a0 <dot8_avx2+0x20>
4011b4: vextractf128 $0x1, %ymm0, %xmm1
4011ba: vaddps     %xmm1, %xmm0, %xmm0

```

这段反汇编比“用了 AVX2”更有信息量。每轮读取 2×32 字节，做 8 个乘加，若按 multiply 和 add 分开算是 16 FLOPs。字节账本约为 $16/64 = 0.25$ FLOP/byte，因此大数组上仍可能受内存带宽限制。向量指令降低的是循环和指令开销，不会消除输入数组的读流。

tail path 也值得看。当前实现对剩余元素回到标量循环：

Listing 8.43 tail path 证据片段

```
4011d0: vmovss (%rdi,%rax,4), %xmm1
4011d5: vmulss (%rsi,%rax,4), %xmm1, %xmm1
4011da: vaddss %xmm1, %xmm0, %xmm0
4011de: inc    %rax
```

这说明“AVX2 版本”不等于整段函数没有标量路径。报告必须记录编译命令、ISA flag、主循环和 tail 反汇编，而不是只写源码用了哪个 intrinsic。

PMU 降级：没有 counters 时怎样保留证据

同一环境下执行 perfstat-ecycles,instructions 返回：

Listing 8.44 perf 不可用时的报告方式

```
Error:
No supported events found.
Access to performance monitoring and observability operations is limited.
```

这不是“无法写报告”的理由。可用证据仍包括：输出 checksum、长度阶梯、反汇编指令形态、每轮 bytes/FLOPs 账本、wall time 多轮样本、编译命令和机器特征。结论要改写成“从反汇编和字节账本看，该 kernel 对大输入可能接近带宽路径；当前环境无法用 PMU 验证 IPC/cache miss”。这比伪造 IPC 更接近工程事实。

8.19 状态机：从标量循环到 SIMD kernel

向量化也可以写成状态机。这样能把“编译器是否向量化”和“手写 intrinsic 是否正确”放到同一张验收表里。

状态	需要证明	失败模式
ScalarContract	标量 reference 语义、边界和错误报告稳定	优化目标本身语义不清
IndepOK	迭代间无破坏性依赖，别名合同明确	输入输出重叠、写后读依赖
VectorBody	主循环地址连续，lane 映射正确	shuffle/gather 太多，地址流变差
TailHandled	0 到 lane-1 的剩余元素被合法处理	越界宽读、漏处理末尾
DispatchSelected	运行时选择正确 ISA，fallback 存在	不支持平台崩溃或走错路径
ResourceAudited	bytes、uops、端口、带宽和阶段收益可解释	只看 microbenchmark，不看端到端

这台状态机说明，SIMD kernel 不是从 intrinsic 开始，而是从标量合同开始。若 scalar reference 没有定义 NaN、overflow、tail、错误位置和重叠输入，后面的向量化无法证明正确。

数学模型：lane 利用率和尾部成本

设向量宽度为 W ，输入长度为 N ，完整向量次数为 $\lfloor N/W \rfloor$ ，尾部为 $N \bmod W$ 。主循环 lane 利用率可粗略写成：

$$util = \frac{N}{W \times \lceil N/W \rceil}$$

当 N 很小或常落在 $W+1$ 附近时，尾部、dispatch 和 horizontal reduction 的固定成本会吞掉收益。短字符串分类、小 batch decode、很短的 tensor tail 都是这种形状。报告不能只给大输入吞吐，还要给长度阶梯：0、1、 $W-1$ 、 W 、 $W+1$ 、 $2W-1$ 、大输入。

反例：向量化后更慢

一个 AoS 结构体包含 `x, y, z, flag` 四个字段，kernel 只需要 `x`。手写 AVX2 后，代码每次载入多个结构体，再用 shuffle 抽取 `x`。反汇编看起来有很多向量指令，但每条 cache line 里只有一部分字节有用，shuffle 占用额外资源。把数据改成 SoA 后，即使标量代码也可能先变快，向量化才有更好上限。

Listing 8.45 SIMD 变慢事故复盘字段

```
symptom:
  AVX2 path slower than scalar for medium input

semantic:
  checksum equal
  tail tests pass

address evidence:
  AoS load brings unused y/z/flag bytes
  vector path uses multiple shuffle instructions

model:
  useful_bytes_per_cache_line is low
  shuffle cost replaces scalar load simplicity

fix:
  convert hot path to SoA or AoSoA
  rerun scalar, auto-vectorized, intrinsic variants
```

这个反例训练一个重要判断：SIMD 不是替代数据布局的工具。布局错误时，SIMD 可能只是更快地搬运无用字节。

工业取舍：多 ISA 分发不能破坏可复现性

生产库常同时提供 scalar、SSE、AVX2、AVX-512、VNNI 或 AMX 路径。分发层必须记录选择结果，否则一次性能回归无法判断走的是哪个 backend。更麻烦的是数值可复现性：浮点归约在不同 lane 宽和合并树下可能不 bitwise 相同。工程上要把“允许误差”和“必须 bitwise 一致”写进合同。

Listing 8.46 SIMD backend dispatch report

```
input_shape
dtype
required_semantics: bitwise | tolerance
selected_backend
```

```
isa_features_detected
alignment_path: aligned | unaligned
tail_path
scalar_reference_status
max_abs_error
cycles_per_element
bytes_per_element
```

实验：指令资源账本

习题与作业

实验一：实现数组 map 的 scalar、auto-vectorized、AVX2 或 AVX-512 版本。记录每元素 bytes、FLOPs、cycles、反汇编中的 load/store/shuffle 数量。说明它更像带宽瓶颈还是算术瓶颈。

实验二：实现 float 求和的单累加器、四累加器、向量归约和 pairwise 版本。报告速度和误差，并说明为什么合并树改变会改变浮点结果。

实验三：为一个 ASCII 分类 kernel 写长度阶梯 benchmark。长度覆盖 0、1、 $W-1$ 、 W 、 $W+1$ 、 $2W-1$ 、大输入；报告 scalar fallback 阈值。

8.20 习题

习题与作业

习题一：选择数组加法、前缀和、直方图三个循环，分别写出迭代依赖、输出位置、共享写和向量化边界。说明哪个可以直接批量化，哪个需要 scan，哪个需要本地 partial 或冲突处理。

习题二：为一个 `std::span<constfloat>` 到 `std::span<float>` 的数组 map 写别名审查。列出安全重叠、破坏性重叠和 debug 检查策略。

习题三：设计 ASCII 分类的测试矩阵。长度覆盖 0、1、lane-1、lane、lane+1、 $2*\text{lane}-1$ ；输入覆盖首字节非法、尾字节非法、非对齐起点、连续分隔符和无换行结尾。说明每个输入攻击哪条假设。

习题四：比较串行求和、四累加器、pairwise sum 和 fast math 下的浮点结果。写出合并树、误差度量和是否允许 bitwise 不一致。

习题五：阅读一次编译器向量化报告。记录源码、编译命令、报告中的接受或拒绝原因、关键反汇编和后续改写方案。若当前环境无法生成报告，用反汇编和运行对照说明替代证据。

习题六：为稳定 filter 画出 mask、块内 rank、块间 scan 和输出区间。解释为什么只做向量比较不能得到紧凑稳定输出。

习题七：为 ASCII 分类写完整执行账本。要求包含标量合同、separator mask、illegal mask、first error 恢复、尾部策略、非对齐输入、跨块状态、报告字段和三类失败路径。

工程案例

知识点小案例：数组加法容易向量化，前缀和不能直接照搬。特例比较两段循环：`out[i]=a[i]+b[i]` 和 `prefix[i]=prefix[i-1]+a[i]`。第一段每个输出只依赖同一位置输入，多个 lane 可独立计算；第二段当前元素依赖前一个输出，存在循环携带依赖。由此推出规律：SIMD 首先问的是迭代之间是否独立，而不是循环看起来是否简单。工程上审查一个 kernel，要写出读集合、写集合、跨迭代依赖、别名关系和尾部策略；能直接向量化的是 map 类模式，scan、histogram、stable filter 需要额外算法结构。

第 9 章

典型计算内核模式

先看一个很容易被忽略的事故。团队要统计日志中最活跃的一百个用户，于是把输入切成多个 chunk，每个 worker 在自己的 chunk 中维护局部 top-k，最后只合并这些局部候选。小数据测试通过，线上却漏掉了一个总量很高的用户。这个用户在每个 chunk 中都排在第一百名之外，所以局部 top-k 阶段把它丢掉了；全局合并阶段再也没有机会看见它。

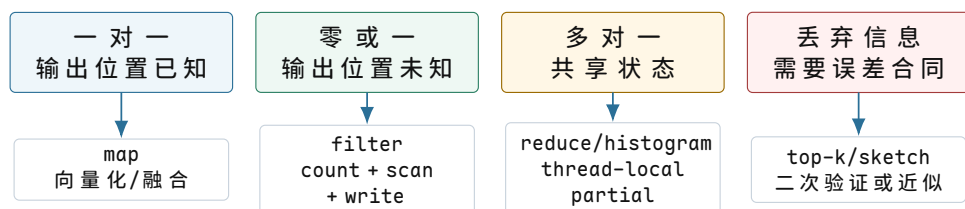
这个错误不是某个堆操作写错了，而是内核组合的合同错了。局部 top-k 丢弃了后续精确全局 top-k 仍然需要的信息。单个内核在局部语义下正确，不代表它放进流水线后仍然正确。遇到一个计算任务时，先问输入和输出的关系、输出位置如何确定、是否有共享写、数据访问是否规律、是否允许丢弃信息，然后再选择 map、reduce、scan、filter、partition、histogram、join、sort、top-k 或 graph frontier。

这里的 **内核** (kernel) 指一段高频、边界清楚、可以独立测量的计算片段。它不一定是操作系统内核，也不一定是 GPU kernel；在 Compute Systems Engine 中，解析一批日志字段、过滤有效记录、统计事件类型、合并 partial、生成 top-k 候选，都可以是计算内核。每个内核必须有 **reference** (朴素但可信的对照实现)，用来固定语义；优化版本先和 reference 对齐，再谈吞吐。

内核模式不是为了给算法起名字，而是因为机器只看见几类低层事实。第一类事实是地址：下一个 cache line 在哪里，能否预取，是否要追指针，是否会跨页。第二类事实是输出位置：处理当前元素时，目标地址已经确定，还是要先知道前面保留了多少元素。第三类事实是共享状态：多个输入是否会修改同一个桶、同一个计数器、同一个堆或同一个输出队列。第四类事实是信息是否被丢弃：局部 top-k、sketch、采样和近似聚合一旦提前丢掉候选，后续精确阶段可能永远无法恢复。

从这些事实自然推导出常见内核。输出位置天然确定，地址连续，通常得到 map；输出数量未知，需要先数再写，得到 filter 和 scan；多个输入合成一个状态，得到 reduce 和 histogram；输出要按 key 进入不同区域，得到 partition 和 shuffle；输出规模可能放大，得到 join；必须重排全局顺序，得到 sort；只保留少量候选并可能丢弃信息，得到 top-k 和 sketch。名字只是后来的标签，真正决定实现的是地址、输出、共享写和信息保留合同。

内核选择的第一张图：输出形状决定并行边界



先判断输出形状，再决定是否可以直接并行；否则优化会把语义错误藏进更快的代码。

图示：本图要证明的命题是：内核优化的第一步不是选择线程数，而是确认输出位置、共享状态和信息保留合同。

工程案例

核心案例是 Compute Systems Engine 的日志批处理内核套件。基础输入仍然是第一章定义的日志记录和 `RecordId`；后续阶段会逐步加入 `event_time`、`ingest_time`、`session_id`、`event_type` 和 `object_id` 等字段，用于引出窗口、join 和背压问题。输出包括有效记录、事件类型直方图、按 key 分区的 partial、维表 join 结果和热门 key。这个案例故意包含规则扫描、输出未知、共享写、key 倾斜和候选丢弃，可以把每一种内核模式都落到同一个工程里。

两条分析线 第一条线是识别常见内核：`map`、`filter`、`reduce`、`scan`、`histogram`、`join`、`top-k` 和 `graph frontier`。先抓住每种内核的输入输出关系，再讨论具体实现策略。第二条线是工程判断：为什么同样叫 `filter`，有的可以原地写，有的必须 `scan`；为什么同样叫 `top-k`，有的可以局部候选，有的会漏掉全局强 key；为什么同样叫 `histogram`，小范围整数 key 和稀疏字符串 key 完全不是一个系统问题。

每看到一个内核，都要问四个问题。第一，输出位置在处理当前元素时是否已经知道。第二，多个输入是否会写同一份状态。第三，内核是否改变顺序、丢弃信息或放大输出。第四，后续 stage 需要哪些身份信息才能复查、重试或恢复。能稳定回答这四个问题，才算真正读懂“典型内核模式”。

深入理解

真正的工程判断体现在负空间里：知道什么时候不该并行、什么时候不该融合、什么时候不该提前丢候选、什么时候不该把中间结果藏起来。后续每个模式都不只讲“怎样做”，也讲“什么条件下这样做会错”。

9.1 先写内核合同，而不是先写并行代码

典型计算内核不能写成算法名清单。一个名字背后可能有完全不同的系统问题。`Histogram` 可以是 256 个小桶，也可以是亿级用户 ID 的稀疏聚合；`filter` 可以只输出计数，也可以要求稳定保留原始顺序；`join` 可以是一张表查大表，也可以是两张大表产生巨大输出。若不写清输入形状和输出语义，任何“最佳实现”都没有意义。

合同 (contract) 是内核对上下游作出的承诺。它至少回答九个问题：输入字段是什么，输出字段是什么，是否保持顺序，输出位置是否天然确定，是否丢弃信息，是否精确，是否可重试，是否有外部副作用，是否需要稳定的 tie-break 或随机种子。合同越早写清，后面的并发、I/O 和分布式章节越容易复用这一段设计。

Listing 9.1 `KernelContract` 的最小形态

```
1 enum class KernelOutputShape {
2     OneToOne,
3     ZeroOrOne,
4     ManyToOne,
5     OneToMany,
6     Reordered,
7     Approximate
8 };
9
10 struct KernelContract {
11     KernelOutputShape output_shape = KernelOutputShape::OneToOne;
12     bool preserves_input_order = true;
13     bool writes_shared_state = false;
14     bool may_drop_information = false;
```

```

15 bool requires_stable_tiebreak = false;
16 bool materializes_output = false;
17 };

```

这段结构不是立即实现框架的要求，而是把思考顺序固定下来。Map 多数是一对一输出；filter 是一对零或一；reduce 是多对一；join 可能一对多；sort 和 partition 会重排；sketch 和近似 heavy hitter 可能丢弃信息。合同让这些差别变成代码边界，而不是藏在函数名里。

核心理想

分析计算内核时，先问四件事：每个输入产生多少输出，输出位置是否已知，多个输入是否会写同一状态，内核是否会丢弃后续仍需要的信息。

内核形态	主要合同	常见错误	第一组证据
Map	一对一输出，位置确定，通常保持顺序	忽略输入输出重叠；融合后错误路径混进热循环	输出 checksum、反汇编、字节流量
Filter	一对零或一，输出位置由前缀计数决定	并行 <code>push_back</code> 争用；原子下标破坏稳定顺序	选择率矩阵、输出顺序校验、临时字节
Reduce	多对一，合并操作要有语义边界	全局原子热写；浮点合并树未声明	partial 数量、合并树、误差或整数溢出检查
Histogram	多对多桶，key 分布决定热点	小桶全局共享；稀疏 key 误用巨大数组	最大 bucket、key 分布、合并耗时
Partition	多路输出，manifest 描述输出身份	只写数据不写 bucket 边界；顺序假设丢失	bucket range、block checksum、路由函数
Join	输出可能放大，构建侧要受控	低估重复 key；把输出放大归因给实现慢	左右 key 频次、输出上界、热点 key 列表
Top-k	精确或近似必须声明	局部 top-k 提前丢掉全局强 key	二次验证、tie-break、候选覆盖率

Reference 和 oracle 不是同一件事 reference（参考实现）是一段尽量简单、可信、可读的程序；oracle（判定器）是比较参考结果和待测结果的规则。二者经常被混在一起，但在高性能内核中必须分开。Reference 可以生成完整输出，oracle 可以只检查 checksum、计数、不变量或误差界。对浮点归约，oracle 不应要求逐 bit 一致，而应写清误差合同；对 top-k，oracle 必须检查 tie-break 和候选覆盖；对 partition，oracle 不仅检查元素集合，还要检查每个 bucket 的边界和路由函数。

Listing 9.2 KernelOracle 的最小形态

```

1 struct KernelOracle {
2     bool requires_exact_order = true;
3     bool allows_floating_error = false;
4     double absolute_error = 0.0;
5     double relative_error = 0.0;
6     std::uint64_t expected_records = 0;
7     std::uint64_t expected_checksum = 0;

```



```
8 };
```

这个结构也不是通用测试框架，而是强调不同内核的“正确”含义不同。Stable filter 要求输出顺序和 reference 一致；非稳定 filter 可以只要求输出集合一致；histogram 要求每个 key 的计数一致；top-k 要求顺序、分数和 tie-break 一致；近似 heavy hitter 则必须写明误差上界、漏报概率或二次验证策略。

错误输入要比随机输入更重要 随机输入能发现一部分错误，却很少正好击中合同边界。输入生成器要专门制造坏情况：空输入、单元素、最后块不满、全部通过、全部过滤、热点 key、全唯一 key、全相同 key、tie 分数、join 输出放大、图中超级节点、浮点 NaN 和整数溢出边界。可靠测试不是“规模很大”，而是“刚好攻击实现的假设”。

Listing 9.3 压力输入族的接口形态

```
1 enum class InputFamily {
2     Uniform,
3     AllPass,
4     AllReject,
5     SingleHotKey,
6     ZipfSkew,
7     AllUniqueKeys,
8     DistributedStrongKey,
9     JoinAmplification,
10    FrontierLongTail
11 };
```

这个枚举把实验从“跑一个随机数组”提升为“覆盖语义风险”。每次报告都要写明使用了哪些输入族。若某个优化只在 Uniform 上快，却在 SingleHotKey 上退化或错误，它不是通用结论，只是特定输入分布下的结果。

材料化、流水线和 pipeline breaker 材料化(materialization)是把中间结果写成数组、文件或其他可复查对象。它增加内存或 I/O 成本，却换来调试、复用、恢复和阶段测量。流水线(pipeline)则让上游输出直接进入下游，减少中间写入，但背压、生命周期和错误传播更复杂。

pipeline breaker（会打断流水线的阶段）是必须等待一批数据、重排数据、聚合状态或持久化边界后才能继续的内核。Sort、global reduce、join build、shuffle、checkpoint 常常是 breaker；简单 map 和普通 filter 更容易流水线。识别 breaker 不是运行时的附加工作，而是内核合同的一部分。

内核的机器形状要写在合同旁边 合同回答语义，机器形状回答执行成本。两个内核都叫 filter，一个可能只是标量判断后写 selection vector，另一个可能要解析字符串、查字典、写诊断、保持稳定顺序。合同旁边应写最小机器形状：主要地址序列、输出位置、写者、是否有共享写、是否需要两遍、是否材料化、是否可能成为 pipeline breaker。这样，优化讨论才不会只停在算法名。

Listing 9.4 KernelShape 的抽象字段

```
KernelShape:
  input_address_pattern: sequential | stride | random | pointer_chase
  output_position: known | prefix_count | key_route | append | external
  writes_shared_state: yes | no
  requires_two_pass: yes | no
  materializes_intermediate: yes | no
  may_block_pipeline: yes | no
```

dominant_risk: bandwidth | branch | tlb | contention | output_amplification

例如稳定 filter 的 `output_position` 是 `prefix_count`，通常需要 `count + scan + write`；全局 histogram 的 `writes_shared_state` 若为 `yes`，就要警惕 `atomic` 热点；partition 的 `output_position` 是 `key_route`，必须有 bucket 边界和 `manifest`；join 的 `dominant_risk` 可能是 `output_amplification`。这份形状不会替代 benchmark，但能提前排除明显错误实现。

输出位置是并行安全的第一问题 多个 worker 并行写输出时，最先要回答的不是“用几把锁”，而是每个 worker 是否在写独占地址范围。如果输出位置已知，按区间切分即可；如果输出位置未知，需要先计算位置；如果位置由 key 决定，需要为每个 bucket 分配空间；如果输出会放大，需要先估算或动态扩容并处理背压。所有这些问题都比“并行 for”更基础。

输出形状	位置分配方式	常见错误
一对一 filter	<code>out[i]</code> 已知 先 count，再 scan 得到起点	忽略输入输出别名和尾部 并行 <code>push_back</code> 或原子下标破坏顺序
partition histogram	每 bucket <code>count + scan</code> key 映射到 bucket 状态	桶边界未记录，恢复无法复查 所有线程争全局 bucket 或热点 <code>atomic</code>
join	每输入可能产生多输出	低估输出放大，写爆内存或下游队列
top-k	候选集合而非最终数组位置	局部候选提前丢失全局信息

这张表给内核设计一个直接规则：先让写地址独占，再让计算并行。若做不到独占，就把共享写移到低频合并，或把输出位置预先分配。锁和原子是正确性工具，不是输出位置设计的替代品。

manifest 让中间结果可复查 当 partition 或 shuffle 生成多个输出块时，只返回一个 `std::vector` 不够。系统还需要知道每个块属于哪个 bucket、包含多少记录、写到哪里、校验和是什么、是否已经提交。**manifest**（清单）就是描述这些输出对象身份和边界的记录。单机 partition 的 manifest 到第 18 章会自然变成分布式 shuffle block 的 manifest。

9.2 完整案例：从内核合同推导机器形状和恢复边界

为了把内核模式从名词表变成设计方法，固定一个小流水线：输入日志先做 `valid filter`，保留合法记录；再按 `event id` 做 `histogram`；最后输出 `top-k event`。三个阶段都很常见，但每个阶段的底层问题不同。Filter 的难点是输出位置未知；histogram 的难点是共享写和热点 key；top-k 的难点是信息是否可以提前丢弃。若只写“filter、histogram、top-k 三个内核”，什么都解释不了；必须逐个写合同、机器形状和恢复边界。

第一步：valid filter 的核心是输出位置 串行版本可以直接 `push_back` 合法记录。并行版本若让所有 worker 共享一个输出 `vector`，就会把尾指针、容量和分配器变成共享热点；若每个 worker 写本地 `vector`，再按 worker 顺序拼接，能保持稳定顺序，但需要保存每段长度和起点；若选择率极低，`selection vector` 可能比材料化记录更省；若后续阶段需要连续热列，材料化 `batch` 更合适。因此 filter 的合同不能只写谓词，还要写稳定性、record id、错误记录是否保留、输出表示和后续字段访问。

Listing 9.5 `valid filter` 的合同和机器形状

```
semantic contract:
  keep records whose parse_status is Valid
```

```
preserve input order among kept records
keep RecordId for diagnostics and manifest
```

machine shape:

```
input read: sequential parse_status column
output position: count + exclusive scan
writes: exclusive ranges after scan
recovery object: FilterBlock with input range, output range, checksum
```

第二步：histogram 的核心是写者归属 Histogram 的输入位置已知，但输出状态共享。全局 `counts[event_id]++` 在单线程里很自然；多线程下，即使用 `memory_order_relaxed`，多个核心也会争同一批 cache line 的独占权。正确改造不是先问“用哪种原子”，而是先问每个计数槽由谁写。常见路线是 worker-local partial：每个 worker 写自己的计数表，最后固定顺序合并。若 event id 很多，完整本地表可能太大，需要稀疏 partial 或分区；若热点 key 很集中，局部表能显著降低共享写；若 key 均匀且表巨大，合并成本和内存预算会成为新瓶颈。

Listing 9.6 histogram 的合同和机器形状

```
semantic contract:
  count every kept record exactly once
  produce deterministic count per event id

machine shape:
  input read: sequential event_id column
  hot write: worker-local partial[worker][event_id]
  merge: fixed order over worker partials
  dominant risk: partial bytes, hot-key skew, false sharing
```

第三步：top-k 的核心是信息丢弃 Top-k 看起来只是对 histogram 排序取前 k。优化时常见错误是每个 worker 先取局部 top-k，再合并这些候选。这个策略对近似结果可能有价值，对精确结果却不一定正确：一个 event 在每个 worker 的局部排名都在 k 之后，但全局加起来可能进入前 k。精确 top-k 要么先得到全局 count，再按稳定 tie-break 取前 k；要么证明局部候选集有覆盖性质。没有覆盖证明，就不能丢弃候选。

Listing 9.7 分散强 key 反例

```
k = 2
worker 0 local counts: A=100, B=90, X=80
worker 1 local counts: C=100, D=90, X=80
worker 2 local counts: E=100, F=90, X=80

local top-2 candidates:
  {A, B}, {C, D}, {E, F}

global:
  X = 240, but X was discarded by every local top-2
```

这类反例说明，内核合同和信息论边界先于优化。若业务允许近似 heavy hitter，报告要写误差和漏报概率；若业务要求精确 top-k，局部裁剪必须有证明，或只能作为加速候选而不能作为最终裁剪。

把三个阶段接成可恢复流水线 一旦三个内核组成流水线，就会出现 pipeline breaker。Filter 需要 count/scan/write 后才能得到稳定输出；histogram 需要所有 partial 合并后才能得到全局计数；top-k 需要全局计数后才能排序或选择。这些 breaker 都应输出可复查对象：FilterBlock、HistogramPartial、HistogramMergeReport、TopKResult。每个对象要有输入身份、输出规模、checksum、attempt 和状态。这样失败时可以重试某个 block，而不是从头猜。

Listing 9.8 三阶段流水线的恢复对象

```
FilterBlock:
    input_range, output_range, kept_count, checksum, attempt

HistogramPartial:
    worker_id, input_block_id, event_id_count, checksum

HistogramMergeReport:
    partial_ids, merge_order, total_count, checksum

TopKResult:
    k, tie_break_rule, candidate_count, result_checksum
```

这些对象也帮助性能分析。若 filter 输出很少而 histogram 仍慢，问题可能在 selection vector 间接访问或错误回查；若 histogram partial 很大，内存预算可能压垮 cache；若 top-k `candidate_count` 爆炸，说明 key 空间和 tie-break 需要重新设计。恢复字段和性能字段不是两套东西，它们都在描述同一条数据路径。

最小实现不需要把这些内容做成一组孤立实验，而应把三条路径放进同一条流水线判读中：Filter 同时保留稳定材料化版本和 selection vector 版本，用来观察“复制记录”和“保存身份”的代价；histogram 同时保留全局 atomic、本地 dense partial 和本地 sparse partial，用来判断共享写、桶数量和 key 稀疏性；top-k 必须包含分散强 key 反例，用来证明局部裁剪是否丢失全局事实。每一阶段只需要保留能解释机器行为的字段：KernelShape、records in/out、state bytes、temporary bytes、checksum、dominant risk 和不可验证边界。

9.3 一对一和多对一：Map 与 Reduce

Map 是最容易理解的内核：第 *i* 个输入通常产生第 *i* 个输出。它没有跨元素依赖，适合顺序扫描、SIMD 和线程切分。可是 map 也最容易被低估。若每个元素只做一次乘法和一次加法，它很快会受内存带宽限制；若每个元素调用复杂函数、访问冷字段或分配字符串，瓶颈又会转移到前端、分支或分配器。

Listing 9.9 一对一 map 的 reference

```
1 void scale_and_shift(std::span<const float> input,
2                     std::span<float> output,
3                     float scale,
4                     float shift) {
5     const std::size_t count = std::min(input.size(), output.size());
6     for (std::size_t index = 0; index < count; ++index) {
7         output[index] = input[index] * scale + shift;
8     }
9 }
```

这个 reference 的正确性来自输出位置合同：每个输出位置只依赖同一位置输入。优化时可以切块、向量化或融合相邻 map，但这些优化不能改变位置关系。若输入输出区间可能破坏性重叠，还要把别名语义写进接口；否则编译器和维护者都无法确定循环能否安全重排。

为什么一对一内核适合优化 一对一内核最容易优化，是因为它同时满足三个条件：读写地址可以从索引直接算出，迭代之间没有语义依赖，输出规模等于输入规模。CPU 喜欢这种形状，因为硬件预取器容易看到线性访问，编译器容易展开或向量化，多个线程也容易按连续区间切分。它的难点通常不在算法，而在字节流量、别名、尾部、错误路径和内存带宽。

这也是为什么一对一内核很容易给人错觉。代码看起来只有一行表达式，这种写法很容易被误判为 compute-bound；实际却可能每个元素只做两次浮点运算，却读写十几个字节，很快受内存带宽限制。优化这种内核时，先估算每个元素读多少字节、写多少字节、做多少运算，再判断融合、SIMD 或线程是否有意义。若瓶颈已经是内存带宽，把线程数翻倍可能只会让多个核心争同一条内存通道。

Map 融合减少字节，不是为了让代码更复杂 连续三个 map 若每一步都写出中间数组，会产生多轮内存读写。融合可以把这些逐元素变换放进一次循环，减少中间材料化。可是融合后循环体更大，寄存器压力更高，错误路径也可能混进热循环。融合是否值得，要比较未融合耗时、融合耗时、中间数据大小、错误报告是否变难，而不是凭直觉判断。

日志项目中，timestamp 归一化、event type 编码、用户 ID 映射都像 map。若解析器输出对象数组，map 会拖着冷字段一起走；若解析器输出列式 [HotBatch](#)，map 可以只扫热字段。第 6 章的数据布局因此不是孤立优化，而是 map 内核能否真正顺序扫描的前提。

心智模型

Map 的核心问题不是“能不能写成一循环”，而是“这一行循环每处理一个元素让机器移动了多少字节，产生了多少临时状态，是否把冷数据拖进热路径”。

Reduce 的第一问题是合并语义 Reduce 把多个输入合成一个输出。它看起来像“加起来”，但真正的问题是合并操作的语义。整数最大值有清楚的单位元和结合律；浮点求和不严格结合，合并树会影响舍入；字符串拼接可能满足语义结合，却被分配成本支配；错误列表合并可能要求稳定顺序。没有合并语义，就没有正确的并行 reduce。

并行 reduce 的反例，是所有线程直接更新一个共享结果。下面的代码可能得到正确计数，但每个命中元素都要修改同一条 cache line，扩展性会很差。

Listing 9.10 反例：高频共享写的 reduce

```
1 void count_positive_bad(std::span<const std::int32_t> values,
2                         std::atomic<std::int64_t>& total) {
3     for (const std::int32_t value : values) {
4         if (value > 0) {
5             total.fetch_add(1, std::memory_order_relaxed);
6         }
7     }
8 }
```

`memory_order_relaxed` 只放松了顺序约束，并没有让共享写消失。更可靠的起点是每个 worker 写自己的 partial，最后合并。

Listing 9.11 线程本地 partial，把高频共享写移出热路径


```

1 constexpr std::size_t CACHE_LINE_BYTES = 64;
2
3 struct alignas(CACHE_LINE_BYTES) CountPartial {
4     std::int64_t positive = 0;
5 };
6
7 void count_positive_range(std::span<const std::int32_t> values,
8                           CountPartial& partial) {
9     std::int64_t local_count = 0;
10    for (const std::int32_t value : values) {
11        if (value > 0) {
12            ++local_count;
13        }
14    }
15    partial.positive = local_count;
16 }

```

这个版本的原则比代码更重要：热路径写寄存器和本地 `partial`，不写全局共享对象。它把问题从“每个元素一次同步”变成“每个 worker 一次合并”。当 `partial` 很大时，合并本身也要继续设计，可能需要树形合并、按 NUMA 节点合并或按 bucket 并行合并。

Reduce 的深水区：合并树就是语义 并行 reduce 必须选择合并树。整数计数通常允许任意合并树，只要不溢出；浮点求和不同，左折叠、pairwise sum、多线程树形合并和 SIMD 水平合并都可能得到不同低位结果。若业务要求 bitwise reproducibility，就要固定合并树，甚至牺牲一部分性能；若业务允许误差，就要写明绝对误差、相对误差或 ULP 边界。

合并树还影响性能。扁平合并让一个线程扫描所有 `partial`，简单但可能成为瓶颈；树形合并能并行推进，但需要更多同步和中间状态；按 NUMA 节点先本地合并可以减少跨节点流量；按 bucket 合并可以让热点桶和普通桶分开处理。Reduce 的“高性能”不是一句 `parallel_for`，而是把语义合并树和硬件拓扑一起设计。

9.4 输出位置未知：Scan、Filter 与 Partition

Filter 选择满足谓词的元素。串行版本可以直接 `push_back`，因为当前输出位置由前面已经保留了多少元素决定。并行后，这个隐含状态变成共享资源。多个线程同时 `push_back` 会争用；使用原子下标可以分配位置，却可能破坏稳定顺序；加锁保持顺序又回到瓶颈。

稳定 filter 的推导路线是三阶段。第一，每个块统计自己会保留多少元素。第二，对块计数做 scan，得到每个块的全局输出起点。第三，每个块再次扫描输入，写入自己的连续输出区间。Scan 在这里不是抽象算法题，而是输出位置分配器。

为什么串行 `push_back` 到并行会变难 串行 filter 中，`push_back` 隐含维护了一个变量：已经输出了多少个元素。每遇到一个保留元素，这个变量加一，并决定新元素写到哪里。单线程时这个状态简单；多线程时，所有 worker 都想修改同一个“下一个输出位置”。如果用锁保护它，热路径每个命中元素都要同步；如果用原子 `fetch_add`，位置唯一但顺序变成到达顺序，不再是输入顺序；如果每个线程写本地 vector，最后拼接，仍然需要知道每个本地 vector 放到全局输出的哪个区间。

Scan 的作用就是把“运行时抢位置”改成“写出前先算位置”。每个块的保留数量是局部事实，块计数的前缀和给出全局起点。写出阶段不需要锁，也不需要全局原子，因为每个 worker 已经拿到自己的独占区间。这个推导是并行算法里非常重要的套路：先把未知输出位置转成计数问题，再用前缀和分配位置。

Listing 9.12 过滤前先统计每个块的输出数量

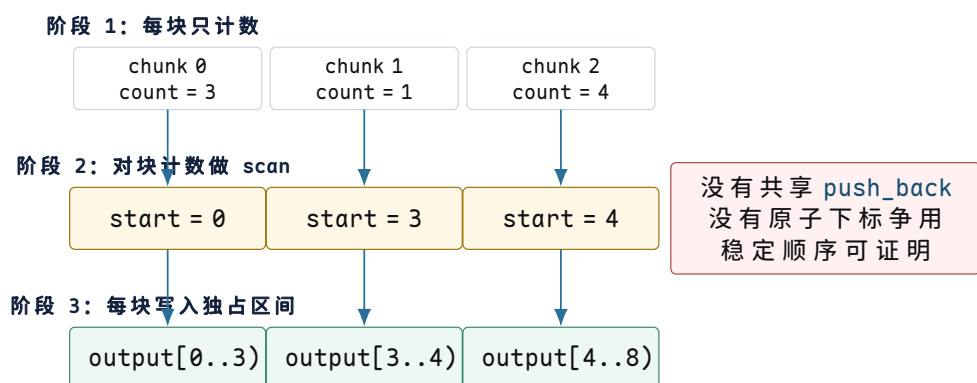
```

1 std::int64_t count_selected(std::span<const std::int32_t> values,
2                             std::int32_t threshold) {
3     std::int64_t selected_count = 0;
4     for (const std::int32_t value : values) {
5         if (value >= threshold) {
6             ++selected_count;
7         }
8     }
9     return selected_count;
10 }

```

这个函数只统计，不写输出，因此没有共享写。对所有块的 `selected_count` 做前缀和，就能得到每块的输出起点。后续写出时，每个 worker 都只写自己负责的区间。它多读了一遍输入，却消除了共享 push、动态扩容和顺序混乱。

Stable filter 的三阶段结构



图示：本图要证明的命题是：stable filter 的并行化核心不是并发容器，而是先用 scan 把输出位置变成每个 worker 的独占区间。

选择率决定 filter 的成本模型 Filter 的性能高度依赖选择率。选择率为 0 时，输出很小，但仍要读完整输入；选择率为 100% 时，filter 接近 copy；选择率在中间且随机时，分支和输出位置都复杂。低选择率可能适合 selection vector；高选择率可能适合 bitmap 或直接保留原数组加 mask；记录很大时，先保存 record id 而不是复制整条记录可能更好。

测试 filter 不能只测 50% 随机选择。至少要覆盖全不通过、全通过、低选择率、高选择率、长记录、错误记录和最后一块不满。否则实现很容易只对某个友好输入成立。

选择向量、bitmap 和材料化输出 Filter 不一定要立刻复制整条记录。若记录很大，直接复制有效记录会产生大量内存流量；另一种做法是输出 selection vector，只保存通过记录的下标；也可以输出 bitmap，每个 bit 表示一条记录是否通过。Selection vector 适合低选择率和后续随机访问可接受的场景；bitmap 适合需要快速 membership test 或保留原数组的场景；材料化输出适合后续要顺序扫描有效记录的场景。

这个选择会影响后续所有 stage。Histogram 如果只需要 event_type，可以用 selection vector 扫热字段列；Join 如果需要整条记录，可能迟早要材料化；错误报告如果需要原始 record id，就不能在 filter 时丢掉身份。Filter 的输出格式不是局部实现细节，而是管线合同。

Partition 是多路输出位置分配 Partition 把元素按 bucket 或 key 放到不同区域。二路 partition 是 filter 的近亲，多路 partition 则是 shuffle 的前置形态。可靠做法仍然是先统计每个 worker 到每个 bucket 的数量，再做二维前缀和，最后让每个 worker 写入每个 bucket 中自己的区间。

分区不只是为了并行写输出，也可以为了改变后续访问形态。把记录按 event type 分桶，后续每个桶里的分支更稳定；把记录按 user id 分区，后续 group-by 的工作集更小；把记录按 reduce target 写成 block，分布式阶段就可以直接传输。分区的成本是额外扫描、输出写入和可能打乱顺序；收益是减少共享写、控制工作集或形成 stage 边界。

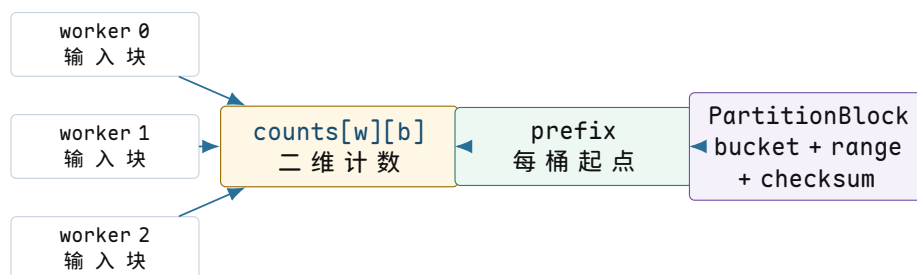
二维前缀和给每个 worker 分配每个 bucket 的独占区间 多路 partition 的核心数据结构是 `counts[worker][bucket]`。第一遍扫描时，每个 worker 只统计自己的输入会落到哪些 bucket；第二步对这个二维计数做前缀和，得到每个 worker 在每个 bucket 中的写入起点；第三遍扫描时，worker 按起点写入输出。这样每个输出位置只由一个 worker 写，且每个 bucket 的范围可以被 manifest 描述。

Listing 9.13 PartitionBlock 描述输出块身份

```
1 struct PartitionBlock {
2     std::int32_t bucket = 0;
3     std::uint64_t begin = 0;
4     std::uint64_t end = 0;
5     std::uint64_t records = 0;
6     std::uint64_t checksum = 0;
7 };
```

PartitionBlock 的价值不只是为了调试。后续分布式 shuffle 要把 block 发送给不同 reducer；checkpoint 要记录哪些 block 已经提交；重试要判断重复发送是否安全；背压要知道哪个 bucket 正在变成热点。若 partition 只返回一段数组而没有 block 身份，后续系统很难恢复和复查。

Partition 从本地分桶自然变成 shuffle manifest



本地 partition 只是在内存中重排记录；一旦 block 带有 bucket、范围和 checksum，它就具备了成为分布式 shuffle 单元的身份。

图示：本图要证明的命题是：partition manifest 是单机高性能内核和分布式 shuffle 之间的接口，不是附加日志。

稳定分区和非稳定分区 Partition 是否保持原始相对顺序，是合同的一部分。稳定分区让同一 bucket 内的记录按输入顺序保留，便于调试和复现，但通常需要更明确的计数和写出顺序。非稳定分区可以更自由地交换元素，适合某些原地算法，但后续 stage 不能假设记录顺序。Compute Systems Engine 若要求错误报告保留原始 `RecordId` 和输入顺序，稳定性和身份字段都很重要；若只是做无序 group-by，非稳定分区可能足够。

分区函数必须可复现 分区函数不能依赖进程随机盐、地址、非确定性时间或不稳定哈希，除非 manifest 记录了足够信息让重试时复现同样路由。分布式阶段尤其如此：同一条记录在重试时必须落到同一 reducer，否则幂等提交和 checkpoint 会失效。即使在单机阶段，也应把 hash seed、bucket 数和路由版本写进报告。

9.5 共享写和数据倾斜：Histogram 与 Top-k

Histogram 是共享写问题的典型场景。它的计算很短：读 key，找到 bucket，计数加一。系统成本却取决于 bucket 数量、key 分布、线程数和内存布局。桶很少时，全局原子会形成热点；桶很多时，每线程完整桶数组可能占用大量内存；key 极度倾斜时，一个热点 bucket 会支配合并阶段。

Listing 9.14 稠密 key 的本地直方图

```

1 constexpr std::int32_t EVENT_BUCKET_COUNT = 1024;
2
3 struct LocalHistogram {
4     std::array<std::int64_t, EVENT_BUCKET_COUNT> buckets{};
5 };
6
7 void histogram_dense(std::span<const std::int32_t> event_types,
8                     LocalHistogram& histogram) {
9     for (const std::int32_t event_type : event_types) {
10         if (0 <= event_type && event_type < EVENT_BUCKET_COUNT) {
11             ++histogram.buckets[static_cast<std::size_t>(event_type)];
12         }
13     }
14 }

```

这个版本速度可能很高，因为 bucket 连续、无哈希、无共享写。但它要求 key 是小范围整数。若 key 是 64 位用户 ID，直接分配巨大数组不现实；若 key 是字符串，哈希、比较、分配和字典编码都会进入成本。工程中常见的路线是：小范围 key 用数组桶，稀疏 key 用本地哈希表或排序聚合，热点 key 用采样、隔离或二级聚合。

哈希聚合和排序聚合没有固定胜负 Group-by 聚合通常有两条路线。哈希聚合边读边更新表，平均复杂度好，适合无序输入和点更新；排序聚合先按 key 排序，再顺序扫描相同 key，适合需要有序输出、key 可比较、内存局部性重要或哈希冲突严重的场景。排序聚合多了排序成本，却可能减少随机访问和分配。

选择时要看 key 数量、输入大小、输出是否需要排序、内存预算、倾斜程度和是否已有顺序。若输入已经按 key 近似有序，排序聚合可能很有吸引力；若 key 几乎全唯一，局部 combine 的收益很小；若 key 先做字典编码能变成稠密整数，数组桶可能比通用哈希表更合适。

本地 partial 不是免费午餐 线程本地桶把热路径共享写移走，但它会增加内存和合并成本。假设有 worker_count 个 worker、bucket_count 个桶、每个桶 8 字节，partial 内存大约是 worker_count*bucket_count*8 字节。若桶数是 1024、worker 是 16，这很小；若桶数是一千万，本地完整桶数组会直接不可接受。此时要改用稀疏本地 map、排序聚合、两级分桶或先做字典编码。

合并也可能成为瓶颈。若每个 worker 都有完整桶数组，最后要扫描 worker_count*bucket_count 个计数，即使很多桶为 0。稀疏结构减少空桶扫描，却引入哈希、分配和随机访问。排序聚合让相同 key 连续，合并更顺序，但要付排序成本。热路径成本、partial 内存、合并成本和输出需求必须一起比较。

策略	适用条件	主要风险
全局原子桶	桶多、冲突低、实现简单优先	热点 key 下 cache line 抖动严重
线程本地稠密桶	key 小范围、桶数可控、合并可接受	桶数大时内存和合并扫描爆炸
线程本地稀疏 map	key 稀疏、每 worker 命中 key 少	哈希和分配成本高，合并复杂
排序聚合	需要有序输出、随机访问昂贵、输入近似有序	排序是 pipeline breaker，临时空间大
两级聚合	key 倾斜明显、热点需要隔离	采样和分桶策略错误会制造新热点

热点 key 要单独观察 平均吞吐会掩盖热点。一个 key 占 40% 输入时，全局原子桶会集中争用；本地桶热路径不争用，但合并时这个 key 的所有 partial 都要汇总；分布式阶段这个 key 可能把单个 reducer 拖成长尾。报告至少要记录最大 bucket、前十个 bucket、最大 bucket 占比和合并阶段耗时。没有这些字段，报告无法区分“算法慢”和“数据倾斜”。

Top-k 不能随便提前丢信息 Top-k 的目标是减少数据移动：只保留最有可能进入最终结果的候选，而不是全量排序。每个 worker 维护局部小堆，最后合并候选，是常见结构。但它只有在合同允许时才正确。若局部阶段按 chunk 丢弃了全局精确结果仍然需要的 key，就会重演开头的 top-k 事故。

精确全局 top-k 通常要先完成按 key 的完整聚合，再从全局计数中选择 top-k。若 key 基数太大，可以使用近似 heavy hitter 或更大的局部候选集，但必须写清误差合同：可能漏报什么，是否做二次精确验证，输出是否标记为近似。监控趋势可以接受近似，计费 and 审计通常不能。

精确 top-k 的最小正确路线 精确 top-k 的安全路线是两步：先得到每个 key 的全局精确 score，再从全局 score 集合中选前 k。第一步可以用本地 hash aggregate、排序聚合或分区后聚合；第二步可以用大小为 k 的堆、选择算法或排序。无论第二步多快，只要第一步丢了 key，最终结果就无法保证精确。很多线上事故正是把“减少候选”误当成“减少中间数据但不改变语义”。

局部 top-k 并非永远错误。若每个 chunk 的划分和业务语义保证全局强 key 必定在某个 chunk 里也强，局部候选可以安全；若只是随机切块或按时间切块，就没有这个保证。另一种安全做法是局部阶段只做预筛选，后面必须对候选覆盖之外的风险做二次验证，例如使用更大的候选集、sketch 估计误差上界，或者重新扫描原始数据验证临界 key。

近似 heavy hitter 要诚实 近似 heavy hitter 算法可以用较小内存发现高频 key，但它输出的是带误差合同的结果，不是精确 top-k。报告必须说明算法可能产生 false positive、是否可能 false negative、误差上界如何计算、是否需要二次精确计数。监控报警、推荐系统候选召回、流量趋势分析常能接受近似；账单、审计、权限、安全策略通常不能接受无标记近似。

工程上常见的折中是“近似召回 + 精确验证”：第一阶段用 sketch 或局部大候选集召回可能的 heavy hitters；第二阶段只对这些候选做精确计数；第三阶段输出带 tie-break 的精确 top-k。这个方案仍有前提：召回阶段必须有足够高的覆盖保证，否则会漏掉真实 top-k。top-k 实验要故意构造分散强 key，迫使候选覆盖证明暴露出来。

Listing 9.15 确定性 top-k 候选的接口形态

```
1 struct TopCandidate {
2     std::int32_t key = 0;
3     std::int64_t score = 0;
```



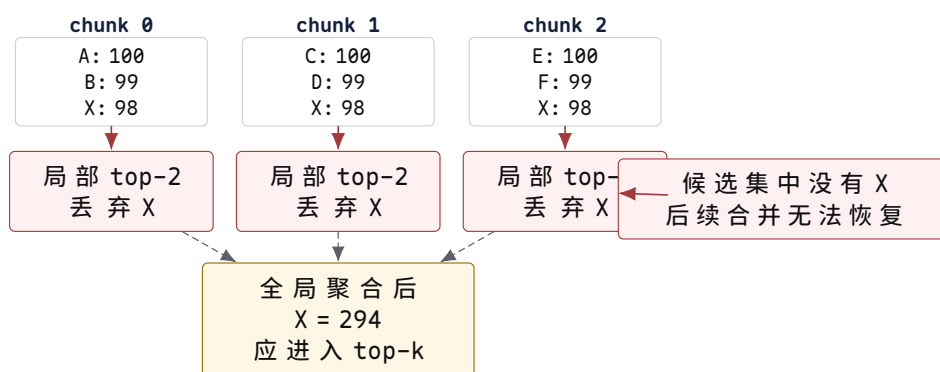
```

4 };
5
6 bool comes_before(const TopCandidate& left,
7                  const TopCandidate& right) {
8     if (left.score != right.score) {
9         return left.score > right.score;
10    }
11    return left.key < right.key;
12 }

```

Tie-break 必须成为接口语义。若两个 key 分数相同，输出按 key 升序还是任意？不确定顺序会让测试、缓存和分布式重试难以复现。可靠 top-k 内核既要快，也要把稳定性写进合同。

局部 top-k 反例：全局强 key 可以在每个 chunk 中被丢弃



图示：本图要证明的命题是：局部候选阶段一旦丢掉精确全局 top-k 仍需要的信息，后续合并阶段没有任何算法能恢复这个 key。

9.6 数据形状改变策略：Sort、Join、Stencil 与 Graph Frontier

Sort 不只是算法题，也是系统工具。排序可以把相同 key 聚到连续区间，便于聚合、join、压缩和确定性输出。外部排序先生成有序 run，再多路归并；分布式排序先采样选择分区边界，再 shuffle 到 reducer。排序常常是 pipeline breaker，但它换来顺序访问、稳定输出和更容易复查的阶段边界。

Join 的成本在输出规模和构建侧 Join 最容易低估的是输出放大。两个输入各一百万行，若某个 key 在左表出现一千次，在右表也出现一千次，该 key 的 join 输出就是一百万行。即使输入不大，输出也可能直接把内存打爆。Join 内核必须估计 key 分布、匹配度、输出上界和背压策略。

Hash join 通常把较小表构建成哈希表，再扫描较大表 probe；sort-merge join 先按 key 排序，再线性合并；nested loop 只适合极小输入或有强索引过滤的场景。选择不是固定的。若小表 key 极端倾斜，哈希 bucket 会很长；若大表过滤后很小，先 filter 再 join 更好；若最终输出需要按 key 有序，sort-merge join 可能减少后续成本。

先估算 join 输出，再选择实现 Join 输出规模由每个 key 在左右两侧的出现次数相乘决定。若 `left_count[k]` 是 key 在左表出现次数，`right_count[k]` 是右表出现次数，那么该 key 的输出数是 `left_count[k]*right_count[k]`，总输出是所有 key 的乘积求和。这个公式比“左表一百万、右表一百万”更有解释力，因为真正危险的是重复 key 和热点 key。

Listing 9.16 Join 输出上界估算的最小形态

```

1 std::uint64_t estimate_join_rows(
2     const std::unordered_map<std::int64_t, std::uint64_t>& left_counts,
3     const std::unordered_map<std::int64_t, std::uint64_t>& right_counts) {
4     std::uint64_t rows = 0;
5     for (const auto& [key, left_count] : left_counts) {
6         const auto found = right_counts.find(key);
7         if (found != right_counts.end()) {
8             rows += left_count * found->second;
9         }
10    }
11    return rows;
12 }

```

这段代码没有处理溢出，也不是生产实现；它把输出规模提前暴露出来。真实系统要处理 `rows` 溢出、内存预算、批量输出、背压和取消。若估算输出已经超过内存预算，继续写一个更快的 hash join 没有意义，应该先改变执行计划：预过滤、限制输出、分批 `materialize`、热点 `key` 单独处理或让下游具备流式消费能力。

构建侧和 probe 侧的选择 Hash join 通常选择较小的一侧作为 build side，因为哈希表状态需要常驻内存。但“小”不是只看行数，还要看 `key` 宽度、payload 宽度、重复 `key`、过滤后大小和是否已经编码。若维表很小，build/probe 清楚；若两侧都大，可能需要 `partitioned hash join`，先按 `key` 分区，再对每个分区做 build/probe；若两侧已经按 `key` 排序，`sort-merge join` 可以避免哈希表随机访问。

Compute Systems Engine 的维表 join 很适合作为这个判断的实验。维表可以是 `object_id` 到类别、权重或业务分组的映射；日志主表很大但只读几列；某些 `object` 可能极热。报告要记录 build 表大小、probe 行数、匹配率、输出行数、最大 `key` 放大倍数和内存峰值。若只写“hash join 更快”，说明没有真正分析 join。

Join 也是背压源 Join 输出可能比输入大得多，因此它天然可能制造背压。若下游是 `top-k` 或聚合，可以考虑在 join 后立即 `combine`，减少材料化；若下游需要完整明细，必须有有界缓冲、分批输出或落盘策略。这个问题会在第 15 章异步 I/O 和背压中继续展开。这里的关键是：join 不是一个孤立函数，它会改变后续 stage 的数据量和故障恢复成本。

Stencil 从邻域复用开始 Stencil 每个输出读取邻域输入，例如一维平滑、二维卷积和网格有限差分。它的关键是复用邻域数据，避免重复从内存加载。内部区域和边界区域最好分开：内部热循环没有复杂分支，边界单独处理；或者使用 `padding` 和 `ghost cell` 统一访问规则。

并行 stencil 还要处理块之间的 halo 数据。单机上，halo 是相邻块共享的边界输入；分布式上，它会变成节点之间交换的消息。这个例子说明，cache blocking 和分布式 halo exchange 是同一个思想在不同尺度上的表现。

Graph frontier 暴露不规则访问 图遍历是规则数组内核的反面。邻接表访问可能随机，顶点度数差异巨大，frontier 大小每层变化，写 `visited` 状态可能有竞争。CSR 把所有边放进连续数组，能改善邻接扫描；但顶点属性访问、frontier 去重和负载均衡仍然复杂。

BFS 的 `top-down` 策略从当前 frontier 扩展邻居；`bottom-up` 策略从未访问顶点检查是否有邻居在 frontier。当前沿很小时 `top-down` 更好，当前沿很大时 `bottom-up` 可能减少边访问。方向优化的本质，是根据数据形状切换内核，而不是固定套一个并行模板。

心智模型

图遍历把这些内核问题集中在一起：随机读、共享写、负载倾斜、输出去重、frontier 表示切换和任务调度。读懂图内核，后面看任务运行时的 ready set 会更自然。

Frontier 表示决定访问成本 Frontier 可以用 vector 表示，也可以用 bitmap 表示。Vector frontier 只存当前活跃顶点，当前沿很小时扫描成本低；但 membership test 需要额外结构，去重也要处理。Bitmap frontier 用一个 bit 表示顶点是否在当前沿，membership test 快，适合当前沿很大时做 bottom-up 检查；但扫描整个位图可能浪费。表示不是美学选择，而是由 frontier 密度、membership 查询次数、去重成本和缓存局部性决定。

表示	优点	风险
Vector frontier	稀疏当前沿扫描少，容易按顶点列表切分	超级节点造成负载长尾，去重和 membership 额外复杂
Bitmap frontier	membership 快，适合 dense frontier 和 bottom-up	稀疏时扫描浪费，更新 bit 可能有共享写
Queue frontier	适合动态发现和流式扩展	顺序和去重难控制，队列争用明显
分桶 frontier	可按度数、社区或分区组织负载	预处理复杂，错误分桶会造成倾斜

图遍历的指标不能只看总时间 图遍历要记录每层 frontier 大小、边访问数、重复发现次数、最大顶点度数、worker 空闲时间和同步次数。总时间只能说明慢，不能解释慢在哪里。若某层 frontier 很小但有一个超级节点，静态按顶点数切分会负载不均；若 frontier 很大，top-down 会访问大量边，bottom-up 可能更好；若重复发现很多，visited 更新和去重策略就是重点。

Graph frontier 还会暴露对象身份问题。一个顶点被多个 worker 同时发现时，谁负责提交？若只需要 visited 集合，原子 bit 或 CAS 可能足够；若需要记录父节点用于最短路径树，就必须定义 tie-break，例如最小父 id 或最早层次。没有 tie-break，结果可能正确地“到达”了顶点，却不能复现同一棵树。

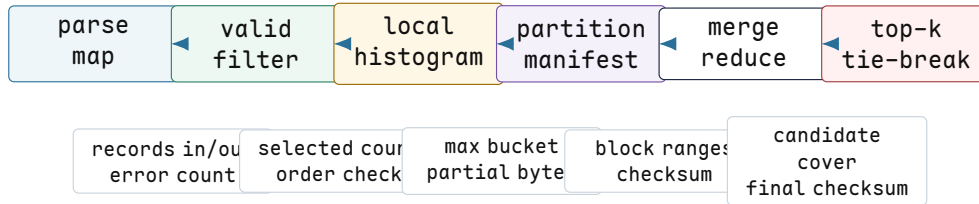
图内核的最小对照是 CSR 上的 BFS reference、vector frontier 和 bitmap frontier。链式图暴露层数和同步次数；二维网格暴露规则局部性；随机图暴露不可预测访问；幂律图暴露超级节点长尾。判读时关注每层 frontier 大小、边访问数、重复发现次数、最大 worker 耗时和输出 checksum。这样才能解释何时 vector frontier 更好，何时 bitmap frontier 更好，而不是把总耗时当成答案。

9.7 把内核组合成可分析流水线

真实系统很少只运行一个内核。日志统计可以拆成一条流水线：parse map 把原始文本变成字段列；valid filter 生成选择向量和错误列表；event histogram 对有效记录做本地计数；partition 把 partial 或记录按 reduce target 分桶；merge reduce 合并所有 worker 的 partial；top-k 输出热门事件。每一步都有独立 reference，也有组合后的不变量。

组合后的不变量比单内核更严格。Filter 后记录数加被过滤数应等于输入数；histogram 所有桶计数总和应等于 filter 输出数；partition 每个 key 的目标必须和 reduce 路由一致；top-k 的候选策略不能丢失精确结果需要的 key；最终 checksum 要和串行 reference 对齐。若单步都正确但组合失败，通常是顺序、身份、材料化边界或信息丢弃出了问题。

Compute Systems Engine 管线的阶段账本



图示：本图要证明的命题是：组合管线的每个 stage 都要留下规模、身份和校验证据，否则最终错误无法定位。

沿一条记录追踪 调试组合流水线时，选一条记录从输入追到输出。它是否通过 filter，进入哪个 partition，更新哪个局部桶，在 join 中匹配几条维表记录，最后是否进入 top-k 候选。沿单条记录追踪，比只看每个函数的单元测试更容易发现接口错误，例如 filter 后顺序改变导致 join 假设失效，或者局部 combine 丢失错误报告所需的 record id。

规模账本比单点耗时更重要 每个 stage 都要记录 records in、records out、bytes out、temporary bytes、state bytes、最大 bucket、join 放大倍数、错误数和 checksum。局部内核变快不代表系统变快。若 filter 选择率突然升高，join 输出会膨胀；若 partition 出现热点桶，下一阶段会长尾；若 top-k 为了保险保留太多候选，内存带宽可能被中间数据吃掉。

Listing 9.17 阶段报告的最小字段

```
1 struct StageReport {
2     std::string_view stage_name = {};
3     std::uint64_t records_in = 0;
4     std::uint64_t records_out = 0;
5     std::uint64_t temporary_bytes = 0;
6     std::uint64_t state_bytes = 0;
7     std::uint64_t checksum = 0;
8 };
```

这不是完整观测系统，却表达了算子判读的底线：每次优化都要能回答输入规模、输出规模、临时数据、状态大小和结果校验。没有这些字段，吞吐数字很容易误导性能判断。

内核形状	风险输入族	判读证据
Map 融合	小数组、大数组、冷字段对象、只扫热字段列	字节流量估计、反汇编、吞吐和 checksum
Stable filter	全通过、全不通过、1% 通过、50% 随机、最后块不满	records out、顺序校验、temporary bytes、分支行为解释
Histogram	均匀 key、单热点 key、Zipf 倾斜、稀疏 64 位 key	最大 bucket、partial 内存、合并耗时、全局原子对照
Top-k	无 tie、全 tie、分散强 key、候选数不足、二次验证	tie-break、候选覆盖率、漏报反例、精确/近似声明
Join	唯一 key、重复 key、热点 key、空匹配、输出爆炸	key 频次、输出上界、实际输出、背压策略
Graph frontier	规则图、随机图、幂律图、空 frontier、超大 frontier	每层 frontier、边访问数、重复发现数、worker 空闲时间

常见误区

所有性能对照都必须先通过串行 reference。若优化版只在随机输入上通过，却没有覆盖低选择率、热点 key、输出放大、分散强 key 和空输入，不能把它写成“正确实现”。

判读顺序 算子优化的判读顺序固定而简洁。先写任务合同：输入、输出、顺序、精确性、共享写和 tie-break。再写 reference 和 oracle：正确结果怎样生成，怎样比较。接着解释实现版本：baseline、优化版、没有采用的方案。然后列风险输入：规模、分布、随机种子和压力输入。最后进入阶段账本：records in/out、temporary bytes、state bytes、最大 bucket、join 放大和 checksum。性能数字只能放在这些事实之后，否则无法区分计算、内存、分支、同步、数据倾斜和材料化成本。

Listing 9.18 阶段账本 CSV 的建议字段

```
1 stage,records_in,records_out,bytes_in,bytes_out,temporary_bytes,
2 state_bytes,max_bucket,checksum,elapsed_ns,notes
```

CSV 字段看起来朴素，却能强迫优化把“快了多少”放进上下文。若某个版本耗时下降，但 `temporary_bytes` 翻倍，后续大输入可能变慢；若 `records_out` 因 filter 选择率改变而下降，吞吐提高不一定来自内核优化；若 `max_bucket` 飙升，后续分区和 reduce 可能出现长尾。

9.8 工业界设计参照

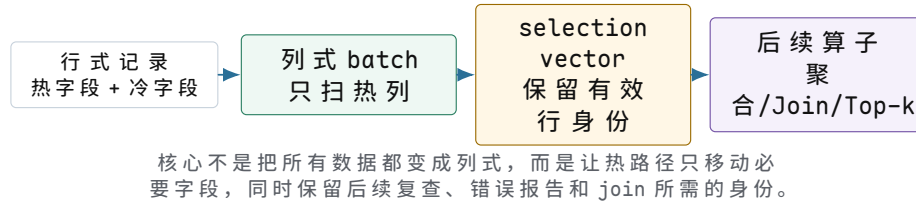
工业系统里的成熟设计，很少是某个孤立技巧。它们通常把语义合同、数据布局、批处理、状态边界和故障恢复放在一起设计。学习这些系统时，不要只记住“某数据库是列式的”“某框架有 shuffle”“某库有 vectorized execution”。真正值得迁移的是背后的约束和取舍：为什么它要用列式 batch，为什么要保留 selection vector，为什么 sort 和 join 会成为 pipeline breaker，为什么 shuffle block 必须带 manifest，为什么 top-k、limit 和聚合必须声明确定性输出。

列式执行器：batch、selection vector 和热字段 DuckDB、ClickHouse、Velox 一类向量化执行器的共同思想，是让算子处理一批连续值，而不是一行一行解释执行。列式 batch 让同一字段连续存放，map、filter、hash 和聚合可以只扫热字段，CPU 更容易预取，SIMD 更容易介入，分支也更容易按列处理。关键不只是“列式快”，而是把执行单位、内存布局和算子合同统一起来。

Selection vector 是其中一个关键设计。Filter 不一定立刻复制有效行，而是输出一组有效行下标；后续算子根据 selection vector 只访问通过过滤的行。这

样可以减少材料化和拷贝，但也带来代价：后续访问可能变成间接访问，选择率高时 selection vector 本身也会占带宽，错误报告还要保留原始行号。成熟实现不是永远使用 selection vector，而是在选择率、字段宽度、后续算子和错误语义之间做判断。

列式 batch 的核心取舍：少搬冷字段，多保留身份



图示：本图要证明的命题是：工业列式执行器的关键价值在于统一数据局部性和语义身份，而不是简单替换存储格式。

Pipeline breaker: 为什么 sort、join build 和 global aggregate 特别重要 数据库执行器和流处理系统都会区分能流水线推进的算子和会打断流水线的算子。简单 map 和 filter 可以一批接一批向下游传递；sort 必须看到足够数据才能决定顺序；hash join 的 build side 必须先构建状态再 probe；global aggregate 必须合并局部状态后才能输出最终结果。这些阶段就是 pipeline breaker。

Pipeline breaker 的成熟设计在于承认边界，而不是假装一切都能流式。它通常会明确材料化对象、内存预算、spill 策略、状态大小和恢复边界。若内存不足，sort 可以写出 run 再归并；hash aggregate 可以分区或 spill；join 可以分批 build/probe；shuffle 可以把输出写成 block manifest。这里的共性是：当一个 stage 必须停下来积累状态时，系统要把状态的身份、大小、生命周期和失败恢复讲清楚。

Shuffle 和 manifest: 从单机 partition 到分布式恢复 Spark、MapReduce 和很多分布式执行器都把 shuffle 当成核心边界。Shuffle 不只是“把相同 key 发到同一台机器”。它需要分区函数、block 文件、校验和、大小、位置、提交状态、重试和清理策略。没有 manifest，系统不知道一个 block 是否完整、是否已经提交、失败后能否重读、重复提交是否安全。

这里的 `PartitionBlock` 是工业 shuffle manifest 的缩小版。单机里它帮助调试 bucket 范围和 checksum；分布式里它变成 reducer 拉取数据、checkpoint 记录状态和失败重试的证据。核心原理是：一旦数据跨越 stage 边界，就必须拥有可命名、可校验、可恢复的身份。

Top-k、limit 和确定性输出 数据库和流处理系统经常支持 `ORDERBY... LIMITk`、top-k 聚合或近似 heavy hitter。工业实现必须非常小心确定性输出。若没有稳定排序键或 tie-break，同一个查询在不同并行度、不同分区或不同重试下可能输出不同结果。对用户来说，这不是“性能细节”，而是结果语义不稳定。

因此成熟实现会把排序键、tie-break、null 排序、稳定性、近似误差和二次验证写进语义。精确 top-k 需要全局可比较 score；近似 top-k 需要误差合同；分布式 top-k 需要保证局部候选不会漏掉全局结果，或者明确它只是近似召回。开头的事故，本质上就是把工业系统必须声明的语义边界藏进了局部优化。

从工业设计迁移到项目对象 Compute Systems Engine 不需要复制 DuckDB、ClickHouse、Spark 或 Kafka 的完整实现，但必须吸收它们的设计原则。列式 batch 迁移为 `HotEventBatch`；selection vector 迁移为 stable filter 的输出身份；pipeline breaker 迁移为 sort、join build、global reduce 和 checkpoint 的材料化边界；shuffle manifest 迁移为 `PartitionBlock`；确定性 top-k 迁移为稳定 tie-break 和候选覆盖测试；背压和重试迁移为有界队列、watermark、幂等

提交和 checkpoint。

源码阅读任务

工业案例阅读可以选择 DuckDB、ClickHouse、Velox、Spark 或类似成熟系统中的一个算子实现，追踪 hash aggregate、sort、top-k、selection vector、pipeline breaker 或 shuffle manifest 之一。重点不是复述系统名字，而是写清它面对什么约束，核心数据结构是什么，怎样保留输出身份，怎样处理内存预算或失败，以及哪一段本地流水线能迁移同样原则。不要写“某系统使用列式执行所以快”这种空泛结论。

9.9 实现路径：KernelPatternSuite

Compute Systems Engine 应形成一个小而完整的 `KernelPatternSuite`。它不需要一次写成通用框架，但必须覆盖几种不同形状：一对一 map、输出未知的 stable filter、共享写明显的 histogram、局部候选的 top-k、会输出放大的 join 或替代案例、frontier 或不规则负载切片。每个内核都保留 reference、优化版本、风险输入生成器、校验器和 stage ledger。

输入族要攻击假设 压力输入不是简单放大规模，而是攻击内核假设。Filter 要测试全通过、全不通过、低选择率和高选择率；histogram 要测试均匀 key、Zipf 分布和单热点 key；top-k 要测试“分散强 key”，即每个 chunk 中不进局部 top-k、全局却很强的 key；join 要测试重复 key 导致的输出放大；graph 要测试规则网格图、随机图和幂律图。

源码阅读入口 Linux 源码阅读可以从 `lib/sort.c`、`include/linux/rhashtable.h` 和若干 `bitmap/hashtable` 辅助实现附近进入。阅读目标不是把整套内核代码抄进书里，而是追一条和当前算子形状相关的路径：排序怎样处理元素移动，哈希表怎样处理扩容和桶，bitmap 怎样表达集合。源码判读要区分证据等级：直接测得的数字、从控制流推导的解释、当前设备无法验证的合理猜测。

最终实现只保留三条关键对照。第一，线程本地 histogram 对照全局 mutex、全局 relaxed atomic 和本地 partial，用来说明共享写如何变成 cache line 争用。第二，stable filter 用块计数和 scan 分配输出位置，用来说明输出未知时为什么要显式化 rank。第三，top-k 构造分散强 key，用来证明局部 top-k 不能作为精确全局 top-k 的前置。三条对照都服务于原理判读，不应扩展成大量互不相连的任务。

9.10 内核模式实验门禁

一、reference 是否仍然存在 没有 reference，后面的 SIMD、多线程、异步或分布式改造都无法证明语义没有变。Reference 可以慢，但必须简单、边界清楚、错误路径可解释。

二、内核合同是否写清 每个内核都要写输入、输出、顺序、精确性、信息丢弃、外部副作用、可重试性和 tie-break。若合同写不出来，先不要优化。

三、压力输入是否攻击核心边界 只扩大数据量不够。要构造低选择率、高选择率、热点 key、稀疏 key、分散强 key、输出放大、frontier 长尾和边界长度输入。

四、状态是否可观察 报告中应能看到关键对象的身份、大小、状态、错误数和校验结果。对 partition 而言，要有 bucket 范围；对 shuffle 而言，要有 manifest；对 top-k 而言，要有候选数量和 tie-break。

五、组合不变量是否成立 单个内核正确不代表流水线正确。Filter 输出数、histogram 总和、partition 路由、join 输出规模、top-k 候选覆盖和最终 checksum 都要对齐 reference。

六、是否说明不采用更复杂方案的理由 没有证据证明队列是瓶颈，就不应把锁队列换成无锁；没有证明输出顺序可变，就不应随意重排；没有证明错误路径可恢

复，就不应只提交正常路径。能解释不做什么，说明工程判断开始成形。

源码阅读任务

源码阅读只读窄入口，不做泛泛浏览。第一组入口是 Linux 的排序和集合辅助实现：阅读 `lib/sort.c`，关注元素移动、比较函数和临时空间边界；阅读 `bitmap` 相关辅助函数，关注集合如何用位表达和扫描。第二组入口是哈希结构：围绕 `include/linux/rhashtable.h` 和对应实现搜索扩容、bucket、冲突和遍历边界。第三组入口是生产库或数据库执行器中的 top-k、hash aggregate 或 sort aggregate，实现任选一个成熟项目，但报告必须把 reference、优化策略和错误边界分开。

源码报告必须回答五个问题：它解决的内核合同是什么；它怎样表示输入、输出和状态；它怎样处理错误或边界；它牺牲了什么换取性能；它和当前实验中的哪个实现可以对照。不要复制源码大段内容，重点画出数据流和状态边界。

9.11 接口延伸

这些内核模式不是清单，而是后续并发、运行时和分布式章节的接口层。Stable filter 里的 count + scan + write，会在第 13 章归约、扫描和分区中展开成并行算法；thread-local histogram 会在锁、原子和 false sharing 章节里变成共享状态设计问题；partition manifest 会在第 18 章变成 shuffle block 和 checkpoint manifest；graph frontier 会在任务运行时章节里变成 ready set、负载均衡和工作窃取问题；top-k 的信息丢弃合同会在分布式重试、近似计算和观测性里反复出现。

心智模型

后续复杂系统问题，很多都可以回到四个问题：输出位置是否确定，是否共享写，是否丢信息，是否留下可恢复的身份。

这里呈现出一个更深的结构：所谓高性能计算，不是把每个函数单独加速，而是让每个 stage 的语义、数据移动和恢复边界都清楚。一个内核快但破坏顺序，可能让后续 join 错；一个 filter 快但丢掉 record id，可能让错误报告不可用；一个 top-k 快但候选覆盖不足，可能让业务结果错；一个 partition 快但没有 manifest，可能让分布式重试无法幂等。成熟工程判断，就在这些边界上。

9.12 习题

习题与作业

习题一：把一个日志统计任务拆成 parse map、valid filter、partition、histogram、top-k 五个阶段。每个阶段写出输入、输出、是否保持顺序、是否有共享写、是否可能丢弃信息和主要瓶颈。

习题二：设计 stable filter。要求先写串行 `push_back` reference，再写共享锁反例，最后写块计数、scan、写出三阶段方案。测试覆盖选择率为 0、选择率为 100%、随机选择、最后一块不满和错误记录。

习题三：为 histogram 选择三种策略：全局原子、本地桶、稀疏本地 map。分别说明适用的 bucket 数量、key 分布、线程数、内存占用和合并成本。

习题四：构造开头的 top-k 反例。让一个 key 在每个 chunk 中都不进入局部 top-k，但全局总和进入前 k。说明怎样修改流水线才能得到精确结果，怎样声明近似结果才算诚实。

习题五：为一个 join 任务估算输出放大。给定每个 key 在左右输入中的出现次数，计算最坏输出规模，并说明何时应选择 hash join、sort-merge join、预过滤或热点 key 单独处理。

习题六：用一个小图实验比较 vector frontier 和 bitmap frontier。记录每层 frontier 大小、边访问数、重复发现次数和 worker 空闲时间，说明何时切换表示。

工程案例

知识点小案例：局部 top-k 不能推出全局 top-k。把输入切成 10 个 chunk，每个 chunk 只保留局部 top 3。构造一个 key：它在每个 chunk 都排第 4，但全局累加后超过所有局部 top key。若 reduce 阶段只看局部 top 3，这个 key 永远消失。由此推出规律：内核模式必须说明是否丢信息，不能只说明“减少数据量”。工程上 top-k 可以是精确算法，也可以是近似算法，但必须写明候选覆盖规则、误差合同和反例输入；否则一个很快的局部优化会破坏全局语义。

第零部分

并发、同步与运行时

线程、调度与亲和性

Compute Systems Engine 的日志解析器曾经使用 8 个 worker：大多数任务解析 `InputSplit`，少数任务把中间结果写成 `ShuffleBlock`。正常磁盘下吞吐稳定；磁盘变慢后，几个写出任务阻塞在同步 `write/fsync`，仍然占着 compute worker。ready 队列里堆着解析任务，CPU 使用率却下降，p99 排队时间上升。把 worker 数从 8 改到 32 后，吞吐短期略有恢复，context switches、migrations、内存占用和尾延迟同时上升。问题不在“线程不够”，而在长等待占用了本应执行计算的预算。

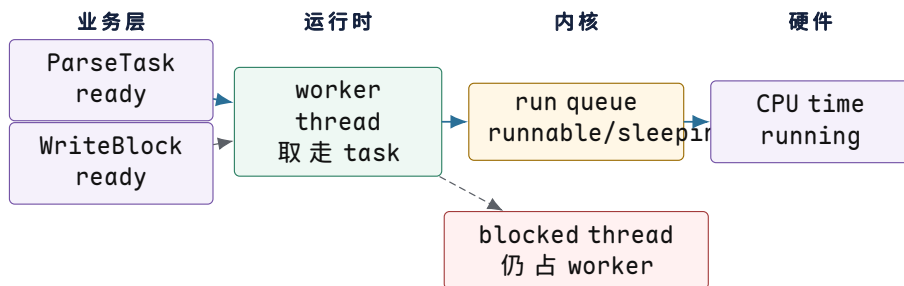
线程问题必须从硬件事实开始。CPU 核心是有限执行资源：一个核心同时只能运行有限数量的硬件线程，SMT sibling 还会共享前端、执行端口、cache 和内存带宽。操作系统不能调度业务 task，它只能调度内核可见的 thread。用户态运行时知道 task 的类型、依赖、优先级和输入 split，却不能直接让某个函数占用 CPU；它只能把 task 交给 worker thread，再由内核把 thread 放到某个 run queue。线程池、亲和性、阻塞隔离和 work stealing 都是围绕这条边界展开：业务想推进逻辑工作，内核只分配 CPU 时间。

10.1 从执行预算推导 task、thread 和 worker

一个任务没有天然的执行能力。解析一个 split、压缩一个 block、提交一个 manifest，这些都是逻辑工作描述，称为 task。Task 要前进，必须被某个 thread 执行。Thread 是操作系统调度实体，拥有栈、寄存器上下文、调度状态、优先级、CPU 时间账本和阻塞状态。Worker 是运行时期长期持有的一类 thread，它反复从用户态队列取 task 并执行。CPU 是硬件执行资源，最终只看见可运行 thread，而不是业务 task 名称。

这四个词不能混用。一个 task 可以 ready 但没有 worker 领取；一个 worker 可以领取 task 后阻塞在 I/O；一个 thread 可以 runnable 但排在 run queue 中没有真正 running；一个 CPU 可以被其他进程、内核线程、中断、cgroup 配额或温控策略占用。报告只写“有 32 个线程”没有解释力，必须写清：提交了多少 task，有多少 worker，内核看到多少 runnable thread，机器实际有多少可用 CPU。

执行预算的四层边界



用户态只能管理 task 到 worker 的映射；内核只调度 thread；CPU 只提供时间片和执行资源。

图示：本图要证明的命题是：线程池排障必须把业务 ready 队列、worker、内核 run queue 和 CPU 时间分开。

Runnable 不等于 running Runnable 表示线程具备运行条件，已经在某个运行

队列中等待调度；`running` 表示它此刻真的占用 CPU。若 `runnable` 线程数长期大于可用 CPU，调度器只能轮流运行它们，`context switch` 增加，每个线程获得的连续 CPU 时间下降。若线程处于 `sleeping` 或 `blocked`，它不消耗 CPU，却可能占住 worker，导致用户态 `ready` 任务无人执行。

这一区分解释两类相反现象。CPU 使用率很高但吞吐不涨，常见原因是线程过多、锁竞争、任务过细、内存带宽饱和或缓存行争用。CPU 使用率很低但队列增长，常见原因是 worker 睡在 I/O、future、条件变量或下游背压上。前者要减少无效竞争或改善算法，后者要隔离阻塞或改变等待模型。

线程创建成本只是表层 每个 `task` 创建一个 `std::thread` 很直观，却把栈空间、内核登记、调度对象、`join`、异常传播和关闭语义都放进热路径。线程池复用 `thread`，减少创建和销毁成本，但引入了新问题：用户态队列争用、任务粒度、隐藏阻塞、提交失败、关闭 `drain`、异常返回、worker 内等待同池任务、队列容量和背压。线程池不是“队列加几个线程”，而是执行预算的管理器。

Thread 的生命周期账本 一个 C++ `std::thread` 创建后，底层会对应一个内核可调度实体。它需要栈空间，保存寄存器上下文，拥有线程本地存储、调度状态、信号/取消相关状态和内核 bookkeeping。线程从创建到退出，至少经过以下阶段：

Listing 10.1 thread 生命周期的粗粒度事件链

```
create thread object
allocate stack and runtime bookkeeping
create kernel schedulable entity
enter runnable state on a run queue
run user code on a CPU
block, sleep, yield, or get preempted many times
return from thread function or terminate on fatal error
publish completion state to joiner
join and release thread resources
```

这条链解释了为什么“每个小任务一个线程”很快会失控。即使线程函数只做几百纳秒计算，栈、内核对象、调度、`join` 和异常路径仍然存在。线程池把这条生命周期从每个 `task` 移到 worker 初始化和关闭阶段，但 `task` 自身的生命周期并不会消失：它仍然要有提交、入队、执行、完成、失败和取消状态。线程池优化的是执行实体复用，不是取消了状态管理。

`std::thread`、`pthread` 和内核线程不是同一层对象 初学线程时最容易混淆三层对象。`std::thread` 是 C++ 标准库对象，负责表达“这个 C++ 程序拥有一个可 `join` 或可 `detach` 的执行句柄”；`pthread` 是 POSIX 线程 API，负责创建线程、设置属性、同步和 TLS 等平台接口；Linux 内核调度的是 `task` 结构里的线程执行上下文，常由 `clone` 或 `clone3` 创建，并出现在 `/proc/<pid>/task/<tid>` 下。三者通常一一相关，但语义重点不同。

层次	它回答的问题	典型证据
<code>std::thread</code>	C++ 对象是否 <code>joinable</code> ，析构时合同是什么	<code>joinable()</code> 、析构行为、异常路径
<code>pthread</code>	栈大小、 <code>detach</code> 状态、线程名、亲和性、TLS 等属性	<code>pthread_attr_*</code> 、 <code>pthread_setname_np</code>
Linux thread	是否 <code>runnable/running/sleeping</code> ，调度到哪个 CPU，阻塞在哪里	<code>proc/<pid>/task</code> 、 <code>perfsched</code> 、 <code>strace-f</code>

这张表能解释一个常见误判：`std::thread` 对象销毁了，不等于底层线程已经安全完成；只有 `join` 或 `detach` 才把所有权合同说清楚。反过来，`/proc/<pid>/task` 里看到一个 TID，也不代表业务 `task` 正在运行，它可能睡在 `futex`、I/O、page

fault 或条件变量上。写线程报告时，必须同时记录 C++ 层的 join/detach 合同、pthread 属性和内核状态。

std::thread 创建到底发生什么 `std::threadt(f, arg)` 看起来像一次函数调用，实际是一条跨 C++ 运行时、pthread、C 库和内核的创建链。标准只规定 C++ 可见语义：新线程会执行给定 callable，`std::thread` 对象变成 joinable；具体如何分配栈、建立 TLS、调用 `clone`、等待退出，是平台实现细节。工程上不需要背 glibc 源码每个字段，但必须知道这条链上有哪些成本和失败点。

Listing 10.2 从 `std::thread` 到内核线程的典型事件链

```

caller constructs std::thread
  store callable and arguments in a runtime state block
  call pthread_create or an equivalent platform primitive

pthread_create
  choose stack size and guard page policy
  allocate or reuse stack mapping
  allocate thread control block and TLS area
  prepare a startup trampoline
  enter kernel through clone or clone3

clone/clone3
  create a schedulable thread in the same process address space
  share file table, signal handlers, VM, and other selected resources
  install TLS pointer and child tid address
  put the new thread on a run queue

new thread starts
  run C library thread startup code
  invoke the C++ callable
  if callable returns, run thread exit path
  if callable throws through the thread boundary, call std::terminate

```

这条链解释了为什么线程创建不是廉价的普通函数调用。它要准备用户态状态块，可能创建或复用栈映射，初始化 TLS 和线程控制块，进入内核创建调度实体，并让新线程经历 runnable 到 running 的调度过程。调用 `std::thread` 构造函数返回时，新线程可能已经运行了一段，也可能还在 run queue 里等 CPU；C++ 标准不承诺“构造返回后新线程一定已经执行到哪一行”。

Listing 10.3 创建线程的最小观察程序

```

1 #include <thread>
2 #include <cstdio>
3 #include <unistd.h>
4
5 int main() {
6     std::thread t([] {
7         std::printf("worker tid candidate pid=%d\n", getpid());
8     });
9     t.join();
10 }

```

可以用下面的证据包观察这条链。不同 glibc 和内核版本可能使用 `clone` 或 `clone3`，字段顺序也可能不同；结论应写成“观察到当前运行环境通过这些系统调用创建和回收线程”，而不是把输出当成 C++ ABI。

Listing 10.4 观察 `std::thread` 创建链路

```
1 strace -f -e trace=clone,clone3,mmap,mprotect,munmap,futex ./↵
   ↵ thread_create_probe
2 ls /proc/$PID/task
3 cat /proc/$PID/task/$TID/status
4 cat /proc/$PID/task/$TID/maps 2>/dev/null || cat /proc/$PID/maps
```

典型输出会出现三类线索。第一，线程创建前后可能出现 `mmap/mprotect`，对应栈映射和 guard page；但实现也可能复用缓存的栈，所以没有每次都映射。第二，`clone/clone3` 参数会带一组共享标志，说明新线程和原线程共享地址空间、文件表和信号处理等进程资源，同时有自己的栈、寄存器和 TLS。第三，`join` 路经常能看到 `futex` 等等待，说明 joiner 等待退出状态，而不是忙等轮询。

现象	底层含义	常见误判
<code>clone</code> 返回后构造函数返回	创建请求成功，线程已成为内核调度对象	以为 worker 已经执行完启动代码
<code>mmap</code> 出现或消失	栈可能新建，也可能来自缓存或不同实现路径	以为没有 <code>mmap</code> 就没有线程栈
<code>futex</code> 出现在 <code>join /proc/<pid>/task</code> 新增 TID	joiner 等待线程退出状态变化 内核可见线程存在	以为 <code>futex</code> 一定代表锁竞争 以为业务 task 已开始运行

pthread_create 的关键参数：栈、属性和启动函数 `pthread_create` 比 `std::thread` 更接近平台事实。它接收线程属性、启动函数和一个 `void*` 参数。属性可以指定 detach state、栈地址、栈大小、guard size、调度策略等。C++ `std::thread` 通常把 callable 包进一个运行时对象，再把一个 trampoline 作为 pthread 启动函数；trampoline 解包 callable 并调用它。

Listing 10.5 pthread create 暴露出的线程创建属性

```
1 pthread_attr_t attr;
2 pthread_attr_init(&attr);
3 pthread_attr_setstacksize(&attr, 2 * 1024 * 1024);
4 pthread_attr_setguardsize(&attr, 4096);
5
6 pthread_t th;
7 int rc = pthread_create(&th, &attr, &entry, arg);
8 pthread_attr_destroy(&attr);
9 if (rc != 0) {
10     // rc is the error number; pthread APIs often return errors directly.
11 }
12 pthread_join(th, nullptr);
```

这段代码暴露出两个基础事实。第一，线程创建可能失败：资源限制、栈大小非法、进程或 cgroup 限制、内存不足、权限和调度属性都可能让创建失败。第二，pthread API 常直接返回错误码，而不是只通过全局 `errno` 报告；写错误处理时不能套用所有系统调用的习惯。C++ `std::thread` 构造失败会抛 `std::system_error`，它把底层错误转换成 C++ 异常语义。

clone 标志解释了“同进程多线程” Linux 线程本质上是共享一组资源的内核 task。创建线程时，C 库通常通过 `clone/clone3` 请求共享地址空间、文件描述符表、文件系统上下文和信号处理等资源，同时给新线程独立的栈和寄存器上下文。典型线程创建会涉及类似含义的标志：

标志含义	为什么重要
共享 VM	所有线程看到同一堆、全局变量、mmap 区域，因此 data race 会破坏同一份对象
共享文件表	一个线程 close 或 dup 可能影响别的线程通过同一 fd 做 I/O
共享信号处理 设置 TLS	信号、取消和进程级状态不是每个线程完全独立的世界 <code>thread_local</code> 、 <code>errno</code> 、 <code>tcache</code> 等每线程状态需要能找到自己的区域
child tid 清理	线程退出时内核清除 tid 地址并唤醒等待者，join 可据此等待

这张表把线程创建和后面几章连起来：共享 VM 解释了为什么要讲 memory model；共享文件表解释了为什么文件 I/O 章节要区分 fd 和 open file description；TLS 解释了 `tcache`、`errno` 和 `worker-local` 统计；`child tid/futex` 解释了 `join`、条件变量和低层等待原语之间的关系。

线程退出：不是从函数返回就万事结束 线程函数返回后，还要经过运行时退出路径。C++ callable 返回，`pthread` 启动代码接管，运行 TLS 析构，处理 `pthread cleanup`，更新退出状态，释放或缓存线程栈相关资源，通知 `joiner`。Linux 退出路径还会处理 `child tid` 清零和 `futex wake`。`Joiner` 观察到的是“线程已经进入可 `join` 的完成状态”，不是业务函数最后一行的瞬间。

Listing 10.6 线程退出和 `join` 的事件链

```
worker callable returns
C++ and pthread startup wrapper begins thread exit
thread-local destructors run
runtime records return value or exit status
kernel clears child tid address during thread exit
kernel wakes futex waiters for that address
joining thread returns from pthread_join/std::thread::join
owner can safely destroy thread-owned resources
```

这条链也解释了为什么线程退出期 bug 难查。TLS 析构函数可能访问已经关闭的日志器；后台线程可能在 `owner` 析构后才运行最后一段 `cleanup`；`detach` 线程可能继续持有指向对象内部的裸指针；`joiner` 等到线程退出时，业务错误已经发生在更早的位置。线程创建和退出都必须进入状态机和报告，而不是只看“线程数”和“CPU 使用率”。

`join.detach` 和析构是所有权合同 `std::thread` 的危险之处不在创建语法，而在生命周期。一个 `joinable` 的 `std::thread` 对象析构时会调用 `std::terminate`，因为标准库不能替你猜测：是应该阻塞等待线程结束，还是把线程留在后台继续跑。C++ 把这个决定强制交给程序员：要么 `join`，明确等待完成并回收线程资源；要么 `detach`，明确放弃句柄，让线程独立结束。

Listing 10.7 线程所有权错误：析构 `joinable` thread

```
1 void bad() {
2     std::thread t([] {
3         do_work();
4     });
```



```

5     if (early_error()) {
6         return;           // t is still joinable: std::terminate
7     }
8     t.join();
9 }

```

正确设计不是到处补 `detach`。Detach 适合极少数真正独立、生命周期不依赖调用者、且有自己关闭协议的后台线程。大多数系统线程都应该有明确 owner：线程池对象、service 对象或 runtime 对象。Owner 负责请求停止、通知 worker、drain 队列、join 线程，并保证线程函数不再访问已经析构的队列、allocator、日志器或配置对象。第 10 章的条件变量和第 14 章的运行时关闭协议，都建立在这个所有权合同上。

Listing 10.8 join/detach 的最小状态机

```

NoThread
  -> JoinableRunning      create succeeds
JoinableRunning
  -> JoinableFinished     thread function returns
JoinableRunning
  -> Detached             detach, owner gives up handle
JoinableFinished
  -> Joined               join observes completion and releases handle
JoinableRunning
  -> Terminate            std::thread object destructor while joinable

```

这不是语言细节，而是工程边界。若线程函数捕获 `this`，而对象析构时没有先停止并 join，后台线程可能访问已经释放的内存。若线程池析构时只设置 `stop=true`，却不通知条件变量，worker 可能永远睡着，析构线程卡在 join。若异常路径提前返回，joinable 线程会触发 terminate。线程基础章节必须先把这些合同讲清楚，否则后面讨论调度和亲和性会掩盖更基本的生命周期错误。

每个线程都有自己的栈、TLS 和退出清理 线程不是“多一个函数在跑”这么简单。每个线程有自己的栈，用来保存函数调用链、局部变量、返回地址、异常展开信息和一部分运行时状态；还有线程本地存储 TLS，用来放 `thread_local` 对象、errno、某些库的线程状态和 allocator 的线程缓存。pthread 创建线程时可以设置栈大小和 guard page；`std::thread` 通常使用平台默认属性。栈太小会在深调用、巨大局部数组或递归中撞到 guard page；栈太大又会消耗虚拟地址空间，并在大量线程时加重页表和映射压力。

Listing 10.9 线程栈和 TLS 的两个基础事实

```

1 thread_local std::uint64_t worker_events = 0;
2
3 void worker_entry() {
4     std::array<char, 64 * 1024> scratch{}; // lives on this thread's stack
5     ++worker_events;                       // lives in this thread's TLS
6     run_worker_loop(scratch);
7 }

```

`scratch` 的地址随线程栈变化，不应逃逸到线程结束之后；`worker_events` 看起来像全局变量，却是每个线程一份。线程退出时，TLS 对象会按实现规定的顺序析构。若 TLS 析构函数访问已经关闭的日志器、allocator、runtime 或动态库状态，

就可能出现很晚才爆的退出期错误。工程上应避免让复杂拥有关系藏进 TLS；若必须使用 TLS 缓存，要明确它在线程退出、fork、动态库卸载和运行时 shutdown 时的行为。

对象	常见错误	证据或防线
线程栈	大局部数组、深递归、栈地址逃逸	/proc/<pid>/maps、guard page、ASan
TLS	退出期析构访问已销毁对象	简化 TLS、显式 shutdown、TSan/ASan
tcache	线程退出时缓存释放路径被误判为泄漏	allocator 统计、退出前后 RSS 对照
errno	跨线程读取错误码，以为是全局状态	每线程错误码，立即保存

第一册讲 allocator 时说过 tcache 是线程本地小块缓存，这里要把它接回来：线程退出不是只从 run queue 消失，还会触发线程相关 runtime 状态清理。大量短生命周期线程会反复创建栈、TLS、tcache 和内核调度对象；即使业务函数很短，系统开销也会变成主成本。这是线程池存在的第一性原因之一。

线程基础证据包：先证明线程事实，再解释性能 线程报告不应只写“开了 16 个线程”。最低限度要能证明：进程里有多少内核线程，每个线程叫什么，线程是否在睡眠，是否发生过 futex 等待，是否频繁迁移，是否有大量 voluntary/involuntary context switch。没有这些事实，后面的亲和性、阻塞隔离和任务粒度结论都容易变成猜测。

Listing 10.10 线程基础证据包

```
1 # 1. 看内核线程身份。每个目录名是一个 TID。
2 ls /proc/$PID/task
3 cat /proc/$PID/task/$TID/status
4 cat /proc/$PID/task/$TID/sched
5
6 # 2. 看创建、等待和唤醒路径。
7 strace -f -e trace=clone,clone3,futex,nanosleep,read,write ./thread_probe
8
9 # 3. 看线程栈和映射。
10 cat /proc/$PID/maps | grep -E 'stack|vdso|vvar'
11
12 # 4. 看调度时间线，需要权限和内核支持。
13 perf sched record -- ./thread_probe
14 perf sched latency
```

证据边界也要写清。看到 clone 只能说明创建了新的内核执行实体，不能说明它已经运行了业务 task；看到 futex 只能说明线程进入了或唤醒了某类等待路径，不能自动证明锁竞争就是瓶颈；看到 /proc/<tid>/status 里的 state 是 sleeping，只是采样瞬间，不是完整时间线；perfsched 能解释 run queue 延迟和迁移，但它需要足够权限，且会引入采集开销。严肃报告要把命令、权限、采样窗口、负载输入和未验证边界一起写出来。

Listing 10.11 线程基础报告字段

```
thread_foundation_report:
  process_id
  thread_count
  thread_names
```

```

stack_size_policy
join_detach_policy
worker_owner_object
shutdown_protocol
clone_count
futex_wait_count
voluntary_context_switches
involuntary_context_switches
migrations
max_runnable_threads
evidence_limitations

```

这份报告的目的不是炫耀工具，而是防止三类误判。第一，把 `std::thread` 对象数量当成真实运行线程数量。第二，把线程 `sleeping` 当成没有工作，而不检查 `worker` 是否被错误等待占住。第三，把线程创建成功当成任务已经开始运行，而不记录从 `submit` 到 `running` 的中间状态。线程基础做扎实之后，后面的调度章节才有可验证的地基。

Worker loop 的底层事件链 一个最小 `worker` 循环表面上只是“取任务并运行”，底层其实跨过用户态队列、条件变量、内核等待和 CPU 调度。可以先写成事件链：

Listing 10.12 `worker` 从空闲到执行任务的事件链

```

worker locks queue mutex
queue is empty, worker checks runtime state
worker waits on condition variable
kernel parks the thread or keeps it out of the run queue
producer pushes a task and updates queue state
producer notifies one sleeping worker
kernel marks worker runnable
scheduler chooses a CPU for the worker
worker wakes, re-locks queue mutex
worker pops task and releases mutex
worker runs user function

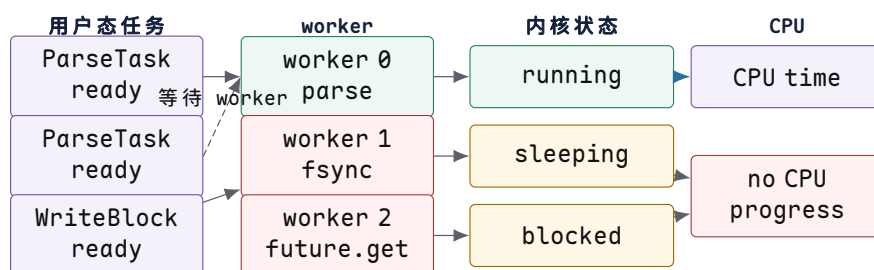
```

这条链里，通知不等于任务已经运行。被唤醒的 `worker` 需要重新进入 `runnable` 集合，等待调度器分配 CPU，再重新竞争队列锁。若生产者一次提交大量短任务，每个任务都唤醒/睡眠，固定成本会吞掉计算；若 `worker` 数多于 CPU，唤醒后还要排队；若同一把队列锁竞争激烈，线程醒来后可能又睡回去。第 14 章的本地 `deque` 和工作窃取，就是从减少全局队列热路径竞争推出来的。

10.2 事故复盘：CPU 空闲，任务却不前进

磁盘变慢后的事故现场，最容易误判的指标是 CPU 使用率低。若只看 CPU 百分比，会以为 `worker` 少；若把任务、`worker`、内核状态分开，会看到 `ready` 任务很多，但 `worker` 被写出任务占住。此时增加 `worker` 只是把错误模型扩容：更多栈、更多调度对象、更多切换，等待点仍然存在。

两层队列：ready 任务不等于正在运行



ready 队列、worker 状态、内核 run queue 和 CPU 时间是四个不同事实。CPU 空闲时，先找 worker 是否被隐藏等待占住。

图示：本图要证明的命题是：线程池慢不能只看 CPU 使用率，必须把用户态 ready 队列和内核运行状态分开。

等待图比线程列表更有解释力 线程池排障应画等待图。节点可以是 task、worker、queue、mutex、condition variable、future、file descriptor、disk、network service；边表示“谁在等谁”。Worker 等 I/O，I/O 等设备；worker 等 future，future 等队列中的 child task；producer 等下游队列容量，consumer 又等 producer 持有的锁，就可能形成等待环。线程列表只能说明 thread 当前在哪里，等待图能说明执行预算为什么无法推进业务工作。

同池等待会制造饥饿 一个常见错误是 worker 执行父 task 时提交子 task，然后同步等待子 task 完成。如果所有 worker 都这样做，子 task 还在队列里，worker 却都在等待，系统没有 mutex 死锁，却没有可用 worker 推进子 task。这叫同池饥饿。解决方式不是盲目加线程，而是禁止 worker 同步等待同池任务、把依赖交给任务图、使用 continuation，或者让运行时具备 managed blocking 语义。

状态样本如何解释事故 事故发生时，应该能采到类似下面的快照。它比“CPU 使用率 30%”更有解释力：

Listing 10.13 阻塞污染事故的状态样本

```

time=12.530s
ready.cpu_tasks=184
ready.io_tasks=0
compute.workers=8
workers.running_cpu=2
workers.blocked_io=4
workers.blocked_future=1
workers.idle=1
kernel.runnable_threads=3
cpu.utilization=34%
queue.cpu.p99_wait_ms=87
  
```

这组数字说明 ready CPU task 很多，但 compute worker 大多被 I/O 或 future 等待占住；内核 runnable thread 只有 3 个，所以 CPU 低不是因为没工作，而是因为用户态执行槽被隐藏等待耗尽。修复方向是阻塞隔离、任务图依赖或 managed blocking，而不是简单把 worker 数翻倍。

10.3 操作系统调度器看到的不是业务优先级

用户态可以给 task 标记优先级、deadline、输入 split 和资源需求；内核调度器看到的是 thread。一个 worker 正在执行高优先级 task 还是低优先级 task，内核通常不知道；一个 worker 阻塞在文件 I/O 还是等待同池 future，内核只看到 sleeping 或 blocked；一个用户态 ready 队列里有多少任务，内核也看不到。运行时要把业务事实翻译成线程行为，否则调度器无法替它做正确决定。

Run queue 是线程队列，不是任务队列 每个可运行 thread 会进入某个 CPU 或调度域的运行队列。调度器根据优先级、时间片、亲和性、负载均衡、NUMA 和其他策略选择下一个 running thread。用户态线程池中的 ready task 如果没有 worker 领取，不会进入内核 run queue；worker 如果睡在条件变量上，也不会因为用户态队列新增 task 自动运行，除非生产者通知并让它变成 runnable。

这解释了一个常见误区：ready task 多不等于 runnable thread 多。前者说明运行时有逻辑工作，后者说明内核有可调度实体。若 ready task 多而 runnable thread 少，问题在 worker 被阻塞、通知丢失、队列关闭协议或运行时调度；若 runnable thread 多而 CPU 饱和，问题可能是过度订阅、任务太细、锁竞争或外部干扰。

从时钟中断到时间片 线程不会因为业务代码“应该让出”就自动停下。内核依靠时钟中断、系统调用、阻塞点和抢占检查，周期性获得控制权，更新当前线程的运行时间账本，并决定是否切换到另一个线程。时间片不是一个固定到处相同的常数，而是调度策略、优先级、负载、cgroup、交互性和内核版本共同作用的结果。原理上必须抓住一点：CPU 时间是被内核分配给线程的离散执行窗口，不是用户态 task 自己持有的资源。

Listing 10.14 一次抢占式调度的粗事件链

```
thread A is running on CPU 3
timer interrupt or kernel preemption point occurs
kernel saves enough state for thread A
scheduler updates runtime accounting
scheduler chooses thread B from the run queue
kernel restores thread B's context
CPU returns to user mode and runs thread B
```

这条链解释了任务粒度为什么重要。一个 task 太短，排队、唤醒、锁、调度和计时成本会吞掉计算；一个 task 太长，又可能长时间不检查取消、不释放局部队列、不让更高优先级控制任务前进。运行时设计通常会让长计算 task 定期到达安全点：检查取消、刷新指标、提交 partial、处理窃取请求或让出执行权。安全点不是形式主义，而是把用户态长循环重新接回内核和运行时的调度事实。

上下文切换不是只换一个函数指针 上下文切换至少要保存和恢复寄存器状态、栈指针、程序计数器、线程本地状态和内核调度账本；还可能影响 TLB、分支预测器、cache 热状态和功耗状态。现代内核和硬件会做大量优化，切换成本仍然不是零。更重要的是，切换会破坏连续性：刚被解析器带热的输入页、字典、partial 和分支历史，可能在下次运行时已经被别的线程挤掉。

因此，context-switches 不能孤立解释。切换增加可能是任务太细、线程过多、锁竞争导致睡眠/唤醒、I/O 阻塞、cgroup 配额、系统干扰或主动让出。报告要同时记录 task 粒度、worker 数、runnable thread、阻塞原因和吞吐。只看到切换多就“绑核”或“加线程”，通常会把问题推到别处。

SMT sibling 共享的不是抽象核心 同时多线程 SMT 让一个物理核心暴露为多个硬件线程。它可以在一个线程等待 cache miss 或分支恢复时，让另一个线程使用部分空闲资源；但两个 sibling 共享前端、执行端口、L1/L2 的某些结构、TLB 资源和内存带宽。若两个 worker 都是计算密集、前端密集或内存带宽密集，它们

放在同一个物理核心上可能互相挤压；若一个等待内存、另一个做计算，SMT 可能提升利用率。

线程池报告不能只写逻辑 CPU 数。至少要记录物理核心数、SMT 是否开启、worker 是否可能落在 sibling 上、输入是否内存等待严重。某些实验应分别跑“只用物理核心数量 worker”和“使用所有逻辑 CPU worker”，观察吞吐、尾延迟、cache/TLB 线索和功耗/频率变化。SMT 是资源共享策略，不是免费的核心翻倍。

抢占和时间片会打断局部性 线程运行一段时间后可能被抢占。抢占本身是必要的公平机制，但它会打断热循环的 cache、TLB、分支预测和寄存器状态。一个短任务若刚把热字段带入 cache 就被换出，下一次回来可能要重新预热。任务粒度太细、线程数太多、后台干扰或 cgroup 配额都可能增加这种打断。测量时看到 context switches 上升，要结合任务粒度和 runnable 数解释。

优先级反转的运行时版本 低优先级或后台 task 持有全局队列锁，高优先级 task 提交或领取时被迫等待，这就是运行时层面的优先级反转。内核 mutex 可能有优先级继承，但用户态队列和自定义锁未必有。更常见的不是严格优先级，而是控制任务被数据任务淹没：关闭、取消、checkpoint commit、manifest 发布等短控制任务排在大量长数据任务后面，导致系统无法及时转入新状态。

解决方向包括控制队列和数据队列分离、控制任务使用独立 worker 或抢占点、长任务定期检查取消、提交路径不依赖被数据任务占满的池。优先级不是越多越好；每个优先级都要有防饥饿和关闭语义。

亲和性从两个事实推出 亲和性有两个来源。第一，线程最近在某个核心或 LLC 上运行，那里可能保留它的代码、栈和工作集。第二，它要处理的数据页可能靠近某个 NUMA 节点或设备。把 worker 固定在一个位置，可以保护这些局部性；把 task 任意迁移，可以改善负载均衡却破坏局部性。运行时要在二者之间取舍。

亲和性也可能制造空闲 绑定线程常被当作性能技巧，但它首先是约束。若某个 worker 被固定到 CPU 2，而 CPU 2 同时承担中断、内核线程或另一个进程的负载，这个 worker 会慢；若某个 NUMA 组的队列暂时为空，绑定策略可能让其他组繁忙而本组 CPU 空闲；若输入分片不均，严格本地优先可能保留局部性却损失总吞吐。亲和性只有和负载均衡、窃取、first-touch、设备位置和队列状态一起看，才是工程设计。

更稳的设计是分层亲和性：默认本地优先，超过等待阈值才跨核心或跨 NUMA 组窃取；控制任务可以绕开数据队列；I/O completion 可以尽量回到拥有 buffer 的运行时组，但不能让完成事件长期无人处理。这样做把“保护局部性”和“避免空闲”同时写进协议，而不是用一个固定绑核掩码替代调度设计。

策略	购买的能力	主要代价
固定 worker 到核心	cache/TLB 热状态更稳定，样本波动小	负载不均时空闲核心不能自动接管
按 NUMA 组调度	数据和线程更接近，远端访问少	需要 first-touch、组内队列和跨组窃取
完全自由迁移	调度器能平衡 CPU 使用	数据局部性和实验可解释性变差
本地优先窃取	高频路径局部，低频失衡时迁移	实现更复杂，窃取策略需要度量

迁移不是永远坏 如果某个 worker 的本地队列很长，而同组或其他组有空闲 CPU，迁移 task 可能比保持局部性更好。数据本地性是一种成本，空闲 CPU 和尾延迟也是成本。工作窃取的意义就在这里：默认保护本地性，失衡时付出一次远端成本换取整体进展。窃取次数、远端执行字节、被窃任务耗时和尾部缩短，都应写进 trace。

10.4 任务粒度：队列成本、局部性和负载均衡的折中

任务太小，每个 task 的提交、入队、出队、唤醒、计数、trace 和 cache 污染会超过有效计算；任务太大，少数长 task 拖住尾部，worker 空闲增加，取消和重试延迟变长。合理粒度来自测量曲线，不来自固定经验数。连续数组归约可以使用较大的 contiguous chunk，因为工作均匀且局部性好；日志解析遇到长行、异常行、热点 key 或压缩块时，chunk 耗时会明显不均。

粒度区域	主要现象	低层根因	调整方向
过细	吞吐低、队列热点、切换多	固定调度成本大于计算	批量提交，合并 task，减少 trace
适中	worker 忙碌均衡，队列可控	计算能摊薄固定成本	作为默认范围
过粗	尾部长、空闲 worker 多、取消慢	负载不均和关键路径变长	拆分、动态调度、work stealing

粒度和 cache 一起决定 任务切分不是只按记录数。一个 chunk 应尽量形成连续地址区间，让硬件预取和 cache line 利用稳定；也要控制状态大小，避免每个 task 携带过多临时对象。若一个 split 太小，cache 局部性还没发挥，队列成本已经主导；若太大，某个长行或热点 key 会让一个 worker 独占关键路径。粒度实验至少记录 task 数、平均执行时间、p95/p99、worker idle time、队列长度和 checksum。

嵌套并行会放大 runnable 数 上层服务可能已经为请求开了 8 个 worker，每个请求内部又调用默认 8 线程的压缩库、矩阵库或并行 STL，瞬间制造 64 个 runnable thread。操作系统只看到 thread 竞争 CPU，不知道哪些 thread 属于同一个业务请求。工业系统常用共享 executor、限制库内线程数、按 stage 设置资源预算、禁止 worker 内部隐式并行等方式控制超额订阅。

粒度账本要同时看硬件和运行时 任务粒度选择必须同时摊薄运行时固定成本和保护硬件局部性。一个解析 task 如果只处理 16 条日志，队列锁、时间戳、trace、唤醒和函数调用可能比解析本身更贵；如果一次处理 100 万条日志，cache 局部性很好，但某个长行或坏块会让一个 worker 独占关键路径，取消也要等很久才响应。

字段	过细时	过粗时	适中目标
task count	很多，队列热点	很少，worker 空闲	足以覆盖 worker 并摊薄开销
cache locality	预热不足	单任务工作集可能太大	连续 chunk，热字段可预取
tail latency	固定成本主导	长任务拖尾	p95/p99 可控
cancel/retry	快但任务太多	响应慢，重做范围大	chunk 有明确恢复边界

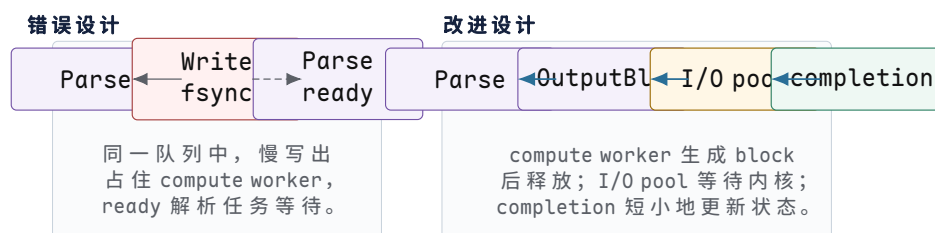
实验应扫 `chunk_records`，而不是只测一个点。每个点输出 task 数、平均运行时间、p95/p99 运行时间、ready 等待、worker idle、context switch、cache/TLB 指标或替代指标。只有把这些字段放在一起，才能区分“任务太小”与“任务太大”。

10.5 阻塞隔离：把等待预算和计算预算分开

阻塞不是错误。等待磁盘、网络、锁、future、条件变量、page fault 或下游背压，是系统运行的一部分。错误在于把长等待放到最稀缺的 compute worker 上，

并让 ready 计算任务排队。线程池设计的第一条工程边界，是 CPU 执行预算和阻塞等待预算要分开。

阻塞隔离：compute worker 不长期等待 I/O



图示：本图要证明的命题是：分池不是为了形式复杂，而是让长等待不占用计算执行预算。

任务类型要进入接口 线程池若只接受 `submit(function)`，调用者很容易把任意工作塞进去。更可靠的接口至少标记任务类型：CPU、BlockingIo、Control、Completion、Background。类型不是装饰，它决定进入哪个队列、是否允许阻塞、是否有 deadline、能否取消、是否计入 compute worker 饱和度。

Listing 10.15 线程池配置和任务类型的接口形态

```
1 enum class TaskClass : std::uint8_t {
2     Cpu,
3     BlockingIo,
4     Control,
5     Completion,
6     Background,
7 };
8
9 enum class ShutdownMode : std::uint8_t {
10     DrainAccepted,
11     CancelNotStarted,
12 };
13
14 struct ThreadPoolPlan {
15     std::uint32_t compute_workers = 0;
16     std::uint32_t blocking_workers = 0;
17     std::uint32_t queue_capacity = 0;
18     std::uint32_t chunk_records = 0;
19     ShutdownMode shutdown = ShutdownMode::DrainAccepted;
20     bool collect_timeline = false;
21 };
```

这段结构的重点不是实现线程池，而是把执行域写进类型。第 10 章的 mutex、condition variable 和 semaphore 会处理等待谓词；第 14 章会把 task 依赖和 work stealing 引入运行时；第 15 章会把 I/O completion 变成可靠写出路径。这里先建立边界：任务类型不能被 `std::function<void()>` 抹掉。

Completion 必须短小 I/O 完成后通常需要更新状态、减少 unfinished 计数、通知 driver、提交 manifest 或唤醒下游任务。completion 线程不应执行长计算，也不应在持有全局锁时调用未知回调。稳妥做法是把 completion 当作控制面事件：快速记录事实，必要时把后续 CPU 工作重新投递到 compute pool。

加线程和隔离阻塞购买的能力不同 阻塞任务混入 compute pool 时，增加 worker 有时能短期改善，因为多出来的 worker 仍能处理计算任务。但长等待仍然占用线程栈和调度对象，系统过载时队列继续增长，尾延迟仍不稳定。实验必须比较三组：单池加线程、分离 compute/I/O、异步 completion。只有三组曲线一起出现，才能说明问题到底来自线程数、等待点还是下游服务。

page fault 也是阻塞 线程不只会显式 I/O 上等待。首次访问尚未映射的文件页、匿名页分配、缺页从磁盘读入、写时复制、内存回收和 major page fault 都可能让 compute worker 进入内核等待。解析 mmap 文件时，代码表面上只是读取内存，底层可能触发 page fault；如果所有 compute worker 都在等缺页，ready 队列同样无法推进。

因此大文件处理需要把“内存访问”和“I/O 等待”放在同一张图里。预读、分批 mmap、顺序 read、固定 batch 大小、I/O pool 和 page cache 行为都会影响 worker 是否被长等待污染。性能报告若只写 `read()` 耗时，漏掉 page fault，可能误判为 CPU 解析慢。

10.6 内核调度器看到的是 thread

用户态运行时决定 task 到 worker 的映射；内核调度器决定 worker thread 到 CPU 的映射。二者信息不同：运行时知道业务优先级、任务依赖、输入 split 和错误语义；内核知道 CPU、run queue、优先级、睡眠、唤醒、迁移、cgroup 和系统负载。线程调优经常失败，是因为用户态以为自己控制了全部调度，或者把内核现象误解释成业务现象。

上下文切换的直接和间接成本 直接成本包括保存/恢复寄存器、切换内核调度结构、更新运行队列和进入/退出内核路径。间接成本更常见：cache 热度下降、TLB 和分支预测状态受影响、锁持有者被抢占导致其他线程等待、线程迁移后访问远端数据。context switch 没有一个固定纳秒数，必须放到 workload 中解释。

报告要区分 voluntary 和 involuntary context switch。自愿切换常来自等待条件变量、I/O、futex 或 sleep；非自愿切换常来自时间片耗尽、抢占、更高优先级任务或 CPU 竞争。前者追等待点，后者追线程过多、系统负载、优先级和 cgroup 配额。

唤醒不是立即运行 `notify_one`、I/O 完成、futex wake 或信号到来，只是让线程有机会进入 runnable 状态。被唤醒的线程还要排队等待 CPU，拿到 CPU 后还可能重新竞争 mutex，竞争失败又睡回去。因此高竞争条件变量和锁会放大尾延迟。正确性依靠谓词循环，性能依靠减少竞争、缩短临界区、控制队列容量和合适粒度。

futex 和条件变量的共同原则 Linux 上许多用户态等待最终会落到 futex 或类似机制：用户态先检查一个内存条件，不满足时才让内核把线程挂起；唤醒方先改变内存状态，再通知等待者。条件变量提供了更高层接口，但原则相同：状态是事实，通知只是让等待者更快重新观察事实。

Listing 10.16 等待谓词的正确顺序

```
waiter:
    lock mutex
    while queue is empty and runtime is open:
        wait on condition variable
    if queue has task:
        pop task
    else:
        observe shutdown

producer:
    lock mutex
```



```
push task
unlock mutex
notify one waiter
```

若等待端只相信“被通知了”，会被虚假唤醒、竞争唤醒和 shutdown 竞态击穿。若生产者先通知后入队，worker 可能醒来看到空队列又睡下，任务长期滞留。第 10 章会从锁和条件变量层面展开这些细节；这里先把调度事实固定下来：睡眠和唤醒必须围绕可检查的状态谓词设计。

cgroup、容器和移动设备会改变可用 CPU 在容器中，`std::thread::hardware_concurrency()` 不一定等于实际可用 CPU；cpuset、CPU quota 和调度周期可能让线程周期性 throttled。移动设备还有大小核、动态频率、温控和前后台策略。benchmark 若不记录这些环境事实，结论很难复现。可用 CPU 是实验输入，不是标准库函数返回值。

10.7 亲和性：把线程、数据和拓扑放在一起

亲和性让线程倾向或固定在某些 CPU 上运行。它能减少迁移、保护 cache 热度、对齐 NUMA 节点或设备队列；也会降低调度器自由度，让负载不均更严重，或者在移动设备上被系统策略覆盖。亲和性不是默认优化按钮，而是一个要用证据检验的局部性假设。

拓扑先于绑定 绑定前必须记录拓扑：逻辑 CPU 数、物理核心数、SMT sibling、共享 L1/L2/L3 的范围、NUMA 节点、cpuset、CPU quota、大小核信息和当前频率策略。没有拓扑记录，亲和性结论不可审查。把两个重计算 worker 绑到同一个物理核心的两个 SMT sibling 上，可能看似“两个 CPU”，实际共享前端、执行端口和 cache；把 worker 绑在某个 NUMA 节点而数据页在远端，会稳定地产生远端访问。

策略	可能收益	主要风险	默认
不绑定	保留调度器自由度，适合普通负载	迁移和 cache 热度不稳定	是
按物理核心绑定	减少迁移，避免 SMT 资源互抢	负载不均时核心空闲	实验
按 NUMA 绑定线程和内存	数据本地，远端访问少	迁移和分片策略复杂	实验
绑定大核或高性能核	短期峰值高，延迟稳定	温控、耗电、系统策略干扰	谨慎

亲和性必须绑定数据策略 若 worker 0 固定在 CPU 0，却随机处理所有输入 split，绑定价值有限。更有意义的做法是把 worker、本地队列、输入 split、partial buffer 和内存节点对齐：本节点 worker 优先处理本节点数据，跨节点窃取作为兜底。第 4 章 NUMA 放置、第 6 章 batch 布局、第 8 章 partition 和这里的 affinity 是同一个问题在不同层次上的表现：计算尽量靠近数据，但必须保留负载均衡和失败恢复路径。

迁移的代价不是只有调度时间 线程从 CPU A 迁移到 CPU B 后，寄存器状态可以恢复，但 cache、TLB、分支预测器和 NUMA 本地性不会免费跟着迁移。若 B 和 A 不共享 L2 或 L3，热数据需要重新进入缓存；若数据页在 A 所在 NUMA 节点，B 访问会走远端链路；若锁持有者迁移或被抢占，等待者会看到更长尾延迟。迁移不是总坏，调度器需要用它均衡负载；但对热循环和内存密集任务，迁移应被记录。

Listing 10.17 亲和性实验需要记录的拓扑字段

```
logical_cpus
physical_cores
```



```
smt_siblings
l1_l2_l3_sharing
numa_nodes
cpuset_allowed
cpu_quota
migrations_per_worker
last_cpu_per_worker
input_split_numa_node
```

绑定线程但不绑定内存，只解决了一半问题；绑定内存但不处理负载倾斜，会让某些节点空闲而另一些节点排队。亲和性实验要和 work stealing、partition 和 NUMA 分配一起看。默认不绑定通常是合理起点，因为它保留调度器自由度；只有证据显示迁移或远端访问是瓶颈时，绑定才有购买理由。

移动大小核的特殊边界 异构核心设备上，核心性能不等价，频率会随温度和电源状态变化。线程数等于核心数并不等于拥有同等执行预算。短时间 benchmark 可能在高性能核心上获得峰值，长时间运行后因温控或配额而变慢。受限环境还可能缺少设置 affinity 的权限。报告应记录可见 CPU、绑定是否成功、温度或配额风险；无法验证的结论要标注。

10.8 线程迁移的硬件后果：从 run queue 到 cache 热度

调度器迁移线程时，用户态看到的只是同一个 worker 继续执行任务；硬件看到的却是另一组近端状态。寄存器可以由内核保存和恢复，栈地址也仍然有效，但 L1D、L1I、L2、TLB、分支历史、uop cache 和 NUMA 本地性不一定跟随迁移。若两个 CPU 共享 LLC，部分数据可能仍在共享缓存；若跨 NUMA 节点，访问路径可能变成远端；若迁移到 SMT sibling，前端和执行端口会和另一个线程共享。线程调度因此不是单纯的队列问题，它会改变第 2、3、4 章讲过的硬件路径。

迁移不是绝对错误。调度器用迁移平衡负载、避开空闲、响应优先级和处理系统其他工作。工程问题是：当前 workload 是否依赖短期局部性，迁移是否频繁到足以破坏它，绑定是否会让负载均衡更差。亲和性要从这三个问题推出，而不是从“绑定更专业”推出。

Listing 10.18 一次 worker 迁移后的状态账本

```
before migration:
  worker_id = 3
  cpu = 7
  hot input range = split 42
  local partial buffer recently written
  parser code and data lines warm near cpu 7

after migration:
  worker_id = 3
  cpu = 14
  registers restored
  stack virtual addresses unchanged
  L1/L2/TLB/branch history near cpu 14 may be cold
  local partial pages may be remote if first-touched near cpu 7 node
```

迁移成本取决于共享层级 从 CPU 7 迁移到同一物理核心的 SMT sibling、同一 L2 的另一个逻辑 CPU、同一 LLC 的另一个核心、另一个 NUMA 节点，成本不同。共享层级越近，热数据越可能仍在较近位置；越远，越可能重新填充 cache、重新

建立 TLB 热度或访问远端内存。拓扑报告中的 `l1_l2_l3_sharing` 不是装饰字段，它决定迁移事件怎样解释。

迁移范围	可能保留的状态	主要风险
同 SMT sibling	部分核心前端和缓存层级共享	执行端口和前端资源互抢
同 L2/L3 范围	部分数据仍在共享缓存	L1、TLB 和分支状态重建
跨 LLC 范围	虚拟地址仍有效，数据可能在远处	cache 重新填充，更多互连流量
跨 NUMA 节点	任务身份仍在，页面可能远端	远端内存、互连等待、first-touch 失配

把迁移写进任务 timeline 只记录 worker 状态不够，还要记录 worker 在哪个 CPU 上运行。若同一任务从 start 到 finish 中间跨多个 CPU，可以把 CPU 变化写成采样字段或事件。若无法低成本记录每次迁移，至少记录任务开始和结束 CPU、worker 的 migration 计数、最后 CPU 和阶段耗时。这样，p99 变慢时能判断它是否集中在迁移、远端访问或 page fault 上。

Listing 10.19 迁移相关 trace 字段

```
task_id
worker_id
start_cpu
finish_cpu
migration_count_delta
numa_node_hint
input_split_id
partial_buffer_node
ready_wait_ns
run_ns
blocked_ns
```

这些字段能回答一类常见事故：worker 没有明显阻塞，ready 队列也不长，但某些任务运行时间显著变长。如果慢任务同时有跨节点迁移、远端 partial buffer 或 page fault 增长，就应检查数据放置和调度；如果慢任务没有迁移，问题可能在输入倾斜、锁、I/O 或算法长尾。

绑定策略要和回退一起提交 绑定线程属于强假设。它假设固定位置带来的局部性收益超过调度器失去自由度的成本。任何绑定策略都应有回退：关闭 affinity、按物理核心绑定、按 NUMA 节点分组、不绑定但按 split 保持软本地性。报告必须比较默认调度和绑定策略，而不是只展示绑定后的最好样本。

Listing 10.20 亲和性实验的对照顺序

```
baseline:
  no affinity
  record migrations, run_ns, ready_wait_ns, checksum

soft locality:
  keep split and partial buffer near the same worker when possible
  allow migration and stealing

core binding:
  bind workers to physical cores
  avoid SMT siblings for heavy compute when evidence supports it
```

```

numa binding:
    first-touch worker-owned pages on the target node
    prefer local work, allow bounded cross-node stealing

```

如果 no affinity 已经稳定，绑定只降低调度器自由度，就不应引入。若 NUMA binding 提升热点输入却让倾斜输入尾延迟变差，可能需要软本地性加 work stealing，而不是硬绑定所有任务。亲和性不是目标，稳定吞吐、低尾延迟和可恢复性才是目标。

10.9 可观测性：WorkerSnapshot 和 ThreadTimeline

没有状态记录，线程池问题只能靠猜。Compute Systems Engine 从这里开始加入 worker 级观测。观测不是为了做监控界面，而是为了把“CPU 低”“吞吐差”“尾延迟高”拆成排队、运行、阻塞、迁移、队列满、关闭等待和异常路径。

Listing 10.21 WorkerSnapshot：线程池排障的最小证据

```

1 enum class WorkerState : std::uint8_t {
2     Starting,
3     Idle,
4     RunningCpu,
5     RunningCompletion,
6     BlockedIo,
7     BlockedFuture,
8     BlockedQueue,
9     Stopping,
10    Joined,
11 };
12
13 struct WorkerSnapshot {
14     std::uint32_t worker_id = 0;
15     std::uint32_t last_cpu = 0;
16     WorkerState state = WorkerState::Starting;
17     TaskClass current_class = TaskClass::Cpu;
18     std::uint64_t current_task_id = 0;
19     std::uint64_t executed_tasks = 0;
20     std::uint64_t queue_pops = 0;
21     std::uint64_t idle_ns = 0;
22     std::uint64_t run_ns = 0;
23     std::uint64_t blocked_ns = 0;
24     std::uint64_t steal_count = 0;
25 };

```

这些字段都服务于问题判断。若 idle_ns 高且 ready 队列空，可能是上游供给不足；若 idle_ns 低但 blocked_ns 高，worker 被等待占住；若 last_cpu 频繁变化且 migrations 高，亲和性或系统干扰值得检查；若少数 worker 的 executed_tasks 和 run_ns 明显更高，可能有任务倾斜或窃取不足。

ThreadTimeline 记录事件，不记录故事 Timeline 至少记录 submit、enqueue、dequeue、start、block begin、block end、finish、cancel、fail、shutdown begin、worker joined。每个事件包含 task id、worker id、task class、时间戳、可选 CPU、错误码和队列长度。热路径不应打印长文本日志，应写结构化事件或采样，最

后统一输出 CSV、JSON 或 key-value 摘要。

Listing 10.22 ThreadTimeline 事件的示例形态

```

1 enum class ThreadEventKind : std::uint8_t {
2     Submit,
3     Enqueue,
4     Dequeue,
5     Start,
6     BlockBegin,
7     BlockEnd,
8     Finish,
9     Fail,
10    Cancel,
11    ShutdownBegin,
12    WorkerJoined,
13 };
14
15 struct ThreadEvent {
16     std::uint64_t timestamp_ns = 0;
17     std::uint64_t task_id = 0;
18     std::uint32_t worker_id = 0;
19     std::uint32_t queue_size = 0;
20     std::uint32_t cpu_id = 0;
21     TaskClass task_class = TaskClass::Cpu;
22     ThreadEventKind kind = ThreadEventKind::Submit;
23 };

```

观测也有成本 每个 task 都写全局锁日志，会改变调度行为。高频指标应使用每 worker 槽位、批量写入、采样或实验模式开关；低频控制状态可以用锁保护。ThreadSanitizer 能发现许多数据竞争，但会改变性能，不能用于性能报告。并发测试可以通过固定 seed、缩小队列容量、减少 worker 到 1 或 2、插入 yield、随机 sleep 来暴露时序 bug。

从 timeline 还原等待路径 ThreadTimeline 的价值在于能还原一条 task 的等待路径。假设 task 42 的事件是：

Listing 10.23 task 42 的 timeline 摘要

submit	t=0.000 ms	queue=cpu
enqueue	t=0.002 ms	queue_size=41
dequeue	t=3.870 ms	worker=3 cpu=7
start	t=3.872 ms	worker=3 cpu=7
block_begin	t=4.120 ms	kind=BlockedIo
block_end	t=18.700 ms	kind=BlockedIo
finish	t=18.960 ms	worker=3 cpu=2

这条记录同时暴露三件事：ready 等待约 3.87 ms，运行中 I/O 阻塞约 14.58 ms，worker 从 CPU 7 迁移到 CPU 2。若类似记录大量出现，说明任务类型标注错误或 I/O 没有隔离；若只发生在少数任务，可能是输入块、page fault 或下游服务长尾。timeline 不必华丽，但必须能把总耗时拆成排队、运行、阻塞和迁移。

10.10 完整排障路径：从 ready 队列增长到调度结论

线程章节最容易停在原则上：不要线程太多、不要阻塞 worker、注意亲和性。真正的系统需要一条可重复的排障路径。下面把一次事故从现象追到结论：CPU 使用率低，ready 队列增长，吞吐下降，p99 排队时间上升。

第一层：先分清工作是否存在 第一步不是加线程，而是问业务工作是否已经 ready。若 input reader 没有生成 task，CPU 低是上游供给问题；若 ready 队列有大量 CPU task，说明业务工作存在但没有被执行。RunReport 要同时写 submitted_tasks、ready_cpu_tasks、running_cpu_tasks、blocked_workers 和 idle_workers。

Listing 10.24 第一层快照：业务工作存在但没有执行槽

```
submitted_tasks=12000
ready_cpu_tasks=840
running_cpu_tasks=2
blocked_workers=5
idle_workers=1
compute_workers=8
cpu_utilization=31%
```

这组数字排除了“没有工作”的假设。ready CPU task 很多，running CPU task 很少，worker 大多 blocked。下一步应追 worker 在等什么，而不是把 compute worker 改成 64。

第二层：把 blocked 拆成等待原因 Blocked 不是一个原因。Worker 可能等 I/O、等 future、等队列空间、等 mutex、等条件变量、等 page fault、等内存回收或等下游服务。若所有等待都记成 Blocked，报告仍然无法行动。ThreadTimeline 的 BlockBegin 应带 block_reason，并记录等待对象的身份。

Listing 10.25 等待原因分解示例

```
blocked_workers:
  worker 1: BlockedIo      request_id=77 block_id=B12
  worker 2: BlockedIo      request_id=81 block_id=B13
  worker 3: BlockedFuture  waits_for_task=T980
  worker 4: BlockedQueue   queue=write_queue reason=high_water
  worker 5: BlockedPageFault input_split=S42
```

这里已经能导出不同修复：I/O 等待要分池或异步 completion；future 等待要改成任务依赖；write queue 高水位是背压正在工作；page fault 要检查输入读取策略和 page cache。把它们混成一个“阻塞”会导致错误优化。

第三层：确认内核是否真的缺 CPU 如果 worker runnable 很多但 CPU 使用率已经接近上限，问题是超额订阅、锁竞争或硬件瓶颈；如果 runnable 很少且大多数 worker sleeping，问题是用户态把执行槽占在等待上。需要采集 voluntary/involuntary context switch、runnable threads、run queue 延迟和 CPU quota。没有系统权限时，也要用 worker timeline 近似推断。

观测组合	含义	优先方向
runnable 多、CPU 高、切换多	线程竞争 CPU 或锁反复唤醒	减线程、调粒度、减少共享
runnable 少、CPU 低、blocked 多	worker 被等待占住	阻塞隔离、异步化、任务依赖
CPU 高、cache/TLB miss 高	执行在等内存层级	布局、分块、NUMA、预取实验
CPU 低、ready 少、input 慢	上游供给不足或 I/O 慢	reader、page cache、backpressure

第四层：检查同池依赖 同池等待是隐藏得很深事故。父 task 在 worker 内提交子 task，然后同步等待；所有 worker 都进入等待后，子 task 留在 ready 队列。此时 CPU 可能低，mutex 没有死锁，日志也只显示 worker 在等 future。Runtime 应记录 `waits_for_task` 和 `waiting_pool`，并在 worker 内等待同池任务时输出警告或拒绝。

Listing 10.26 同池等待的最小反例

```
compute_workers = 2

task A:
    submit child A1 to compute pool
    wait for A1

task B:
    submit child B1 to compute pool
    wait for B1

ready queue:
    A1, B1

workers:
    worker 0 waits for A1
    worker 1 waits for B1
```

修复不是简单加 worker。若有 100 个父 task，增加到 64 个 worker 仍可能耗尽。正确做法是把依赖交给任务图：父 task 完成准备工作后返回，child 完成时触发 continuation；或者运行时提供 managed blocking，临时补偿执行槽，并限制补偿上限。

第五层：把 page fault 当作 I/O 等待处理 很多 mmap 读取事故被误判成解析慢。Worker 表面上执行普通 load，实际上触发 major page fault，进入内核等待磁盘和 page cache 填充。ThreadTimeline 如果只记录显式 `read`，就看不到这类阻塞。至少应在实验报告中记录 cold cache/warm cache、major/minor faults、input split、是否预读和 mmap/read 路径。

Listing 10.27 mmap 冷页路径的事件链

```
worker starts parse task
load from mapped input address
CPU cannot translate to resident physical page
page fault enters kernel
kernel schedules disk read or waits for page cache
```

```
thread sleeps, compute worker is occupied
page becomes resident
thread becomes runnable and resumes parsing
```

如果 page fault 占主要等待，修复方向可能是顺序 read 到显式 buffer、预读、分离 I/O worker、调整 split 大小或控制并发读取，而不是改 parser 分支或增加 compute worker。

第六层：亲和性只在证据成立时使用 排障最后才考虑绑定线程。若 timeline 显示 worker 频繁迁移，且迁移后 cache/TLB 或 NUMA 指标恶化，才尝试亲和性。绑定前先记录拓扑，绑定后再比较 worker 负载均衡和尾延迟。若绑定导致某些核心空闲、某些核心排队，说明负载均衡损失大于局部性收益。

Listing 10.28 亲和性决策账本

```
evidence before binding:
  migrations_per_worker high
  input data mostly local to NUMA node
  worker tasks are long enough to keep cache hot
  load is balanced or work stealing can cross nodes

after binding:
  compare throughput, p99, migrations, remote accesses, idle time
```

亲和性是把调度自由度换成局部性。没有迁移或远端访问证据时，默认不绑定通常更稳；有证据时，也要给 work stealing 留出跨节点兜底，否则热点 split 会让绑定策略放大长尾。

第七层：形成修复工单 一次线程事故最后要落成工单，而不是停留在分析。若根因是阻塞 I/O 污染 compute pool，工单应包括：任务类型标注、blocking pool、CompletionRecord、WorkerSnapshot 中的 BlockedIo、背压容量、回归测试。若根因是同池等待，工单应包括：禁止 worker 同步等待同池 future、任务图 continuation、runtime 断言、最小反例测试。每个工单都要补报告字段，确保下次同类事故更快定位。

10.11 工业设计参照

工业案例不是项目名列表，而是约束、机制、能力和代价的组合。读源码或论文时，要问它解决的 workload 是什么，控制了哪些状态，放弃了哪些灵活性，哪些思想能迁移到当前项目，哪些复杂度不该照搬。

工程案例

Linux CFS 和调度器。 Linux 调度器面向全系统公平、优先级、NUMA、cgroup、实时任务和电源管理，不理解业务 task。它通过 task state、runqueue、wakeup、preemption、migration 和 context switch 让内核线程共享 CPU。可迁移原则是：线程状态必须显式，唤醒只是进入可运行集合，绑定和迁移都要由证据驱动。Compute Systems Engine 不需要仿写 CFS，但要把 worker 状态和队列等待记录清楚。

Linux workqueue。 内核 workqueue 把“工作项”和“执行线程”分离，并处理并发度、CPU 绑定、救援线程和内存回收压力等问题。它提醒系统设计：即使在内核里，工作项也不等于线程；可能阻塞或参与回收路径的工作要有特殊策略。Compute Systems Engine 可借鉴工作项分类和执行域隔离，不需要复制内核复杂策略。

oneTBB、Cilk 和 OpenMP task。这些运行时把大量 task 映射到有限 worker，并通过本地队列、工作窃取、task group 或 fork-join 结构减少全局竞争。它们购买的能力是负载均衡和嵌套并行控制；代价是运行时状态、调试复杂度和任务粒度敏感。第 14 章会深入 work stealing，这里先保留原则：worker 不应长期等待同池 future，任务依赖应进入运行时。

Folly executor、Java ForkJoinPool 和 Go scheduler。这些系统都把执行资源抽象成 executor/scheduler，并围绕阻塞、队列、优先级、timer、I/O 和 continuation 设计边界。Go 通过 M:N 调度让 goroutine 数量远大于 OS thread，但 CPU 密集工作仍受实际线程和核心限制；Java ForkJoinPool 对 worker 内等待有 managed blocking 等机制；Folly 强调 executor 语义和异步链。可迁移原则是：阻塞必须被运行时知道，或者被隔离。

移动大小核调度。异构核心调度器能根据负载、功耗和温度在不同性能等级的核心之间迁移任务。它购买的是能效和交互响应，代价是 benchmark 波动和用户态控制受限。Compute Systems Engine 在这类环境运行时，应把亲和性当作可选实验，并把持续吞吐、温度、频率或配额写进报告。

10.12 Compute Systems Engine 实现边界

这一阶段不要求一次写出完整任务运行时，但线程策略必须从隐式经验变成可观察对象。实现边界包括四部分：保留 reference，增加线程池配置，记录 worker/timeline，构造阻塞污染对照。

最小实现边界 第一，保留每次创建线程的 reference，用它固定输出语义。第二，实现固定 compute pool，只接受 `TaskClass::Cpu` 和短 completion。第三，把阻塞写出移到 blocking pool 或模拟异步 I/O，compute worker 只生成 `ShuffleBlock` 描述并返回。第四，输出 `WorkerSnapshot`、队列长度、任务排队时间、执行时间和阻塞时间。第五，扫描 worker 数和 chunk size，形成曲线。

关闭语义 关闭语义必须写进接口。Drain 模式表示拒绝新任务，等待已接纳任务完成；CancelNotStarted 表示未开始任务被取消，正在运行任务只能协作结束；无论哪种模式，等待中的 worker 都要被唤醒，析构都不能静默丢失异常。第 10 章会通过条件变量谓词落地，第 14 章会通过任务状态机扩展。

阻塞污染的最小对照可以构造 1000 个短 CPU task 和 8 个慢 I/O 模拟 task。比较三种设计：单池 8 worker、单池 32 worker、compute pool 8 worker 加 blocking pool 4 worker。记录吞吐、CPU task p95/p99 排队时间、context switches、worker blocked time、队列最大长度和输出校验。判读必须解释“加线程”与“阻塞隔离”购买的能力和付出的代价。

调度判读矩阵 线程实验至少覆盖五组变量：worker 数从 1 到逻辑 CPU 数的两倍，chunk size 从很小到很大，是否混入阻塞任务，是否设置亲和性，是否在移动或容器环境运行。每组输出相同字段，避免只展示最好的图。若无法获得 `perf`、`pidstat` 或 `affinity` 权限，应在项目 trace 中给出替代指标，并写明未验证边界。

现象	优先检查	常见修复
CPU 低、队列涨	worker 是否 blocked, I/O 是否慢, 下游是否背压	分池、异步 completion、容量队列
CPU 高、吞吐不涨	任务粒度、锁竞争、context switch、内存带宽	批量、减少共享写、调低 worker
p99 高、平均正常	迁移、长 task、锁持有者抢占、后台任务	拆任务、隔离资源、记录关键路径
绑定后更慢	负载不均、SMT sibling、远端内存、系统干扰	放松绑定、按拓扑分组、绑定内存
关闭卡住	条件变量谓词、未完成计数、worker 内等待	明确 drain/cancel, 唤醒所有等待者

源码阅读任务

源码阅读按事故驱动，不按目录背诵。

1. Linux: 从 `kernel/sched/core.c`、`kernel/sched/fair.c`、`kernel/sched/topology.c` 追一条现象路径，例如 wakeup 后如何进入 runqueue, migration 为什么发生, cgroup quota 如何限制运行时间。
2. Linux workqueue: 阅读 work item 与 worker pool 的边界，重点看“工作项不是线程”和“可能阻塞的工作如何不拖垮全局进度”。
3. oneTBB 或 OpenMP task: 找 task graph、arena、worker 和 task group 的责任分界，说明它们如何控制嵌套并行。
4. Folly executor 或 Java ForkJoinPool: 观察 executor 如何表达执行域、队列、阻塞和 continuation，避免只列 API。
5. 移动设备资料: 记录大小核、频率和温控对一次线程实验的影响，标注哪些结论无法在服务器上直接迁移。

阅读报告必须从一个可复现实验或事故开始，最后回到 Compute Systems Engine 中应采用、简化或拒绝的设计。

10.13 完整案例：一个任务从提交到真正运行

线程池最容易被误解的地方，是把 submit 当成“任务已经在跑”。实际上，任务要穿过用户态队列、worker 睡眠状态、条件变量或 futex、内核 run queue、CPU 时间片和运行时自己的状态机，才会真正消耗核心时间。只要其中任何一段被阻塞，ready 队列就可能增长，而 CPU 仍然不满。

T0: 提交者只制造可运行工作 提交线程构造 task，获得运行时锁，把 task 放入 ready queue，增加 unfinished count，然后通知一个 worker。这个瞬间，业务语义只是“运行时已经接纳工作”。它不保证 worker 已经醒来，不保证内核马上调度，不保证 task 已经开始，更不保证 task 会成功完成。若报告只记录 submit 时间和 finish 时间，中间所有等待都会被压成一个黑盒。

Listing 10.29 submit 的状态变化

```
submit thread:
  lock runtime
  ready_queue.push(task)
  unfinished_count += 1
  notify one sleeping worker
  unlock runtime
```

observable facts:

```
accepted_tasks += 1
ready_queue_size may increase
no CPU work has necessarily started
```

unfinished count 在这里增加，是因为 task 已被接纳。即使之后 worker 执行失败、抛异常或被取消，运行时也必须用终态减少 unfinished count。否则 wait_idle 会永久等待。

T1: worker 醒来只是进入竞争 被通知的 worker 可能正睡在条件变量上。通知会让它有机会醒来，但它醒来后还要重新抢运行时锁，重新检查谓词。若有多个 worker 同时醒来，可能只有一个拿到 task，其余重新睡眠；若 close 标志被设置，worker 可能直接退出；若 task 已被别的 worker 取走，当前 worker 仍然不能假设有工作。条件变量等待的核心是“醒来后检查状态”，不是“通知等于任务到手”。

Listing 10.30 worker pop 的状态变化

```
worker:
    lock runtime
    wait until ready_queue not empty or closing
    if ready_queue empty and closing:
        exit
    task = ready_queue.pop()
    running_count += 1
    unlock runtime
    run task outside runtime lock
```

这里把运行时锁释放后再执行 task，是为了避免任务函数反过来提交新任务、等待子任务或记录指标时与运行时锁相互卡住。线程池锁保护的是队列和计数不变量，不是业务计算本身。

T2: 内核调度决定 running 时刻 worker 从用户态角度“准备执行”后，仍可能不在 CPU 上 running。它可能刚从 sleep 变成 runnable，等待内核调度；可能受 cgroup quota 限制；可能被更高优先级线程抢占；可能迁移到另一个核心后重新填充 cache/TLB。应用层 timeline 若只记录 runtime lock 内事件，就看不到 runnable 到 running 的延迟。至少要把 task start 事件和 worker state 结合系统指标观察。

Listing 10.31 ready 到 running 的层次

```
runtime ready:
    task is in ready queue

worker runnable:
    OS thread can run but may wait in run queue

worker running:
    OS thread is actually executing on a CPU

task running:
    worker has popped the task and entered task body
```

这四层经常被混成“ready”。排障时要逐层分开。ready queue 增长但 runnable worker 很少，可能是 worker 睡眠谓词或通知问题；runnable 很多但 running 少，可能是 CPU 配额、优先级或过度订阅；task running 很多但吞吐不涨，可能是锁、

内存带宽或 I/O 阻塞。

T3: 完成路径必须覆盖异常 任务完成后, worker 回到运行时锁, 减少 running count 和 unfinished count, 记录结果, 必要时通知 idle waiters。若任务抛异常, 仍然要执行同样的计数和通知, 只是结果进入 failed 状态。工程实现通常用 RAII guard 或运行时边界捕获异常, 避免异常绕过计数。

Listing 10.32 finish 的状态变化

```
run task:
    status = Succeeded or Failed

finish:
    lock runtime
    running_count -= 1
    unfinished_count -= 1
    record terminal status
    if unfinished_count == 0:
        notify idle waiters
    unlock runtime
```

这个账本解释了为什么 `wait_idle` 不能只等待队列为空。任务离开 ready queue 后, 队列可能为空, 但 running count 和 unfinished count 仍然非零。真实空闲是“没有 ready、没有 running、没有未完成终态”。运行时报告应同时记录这些字段。

T4: 一条任务 timeline 的最小格式 可观测性不需要每条记录都写海量日志, 但任务生命周期必须可重建。最小事件包括 accepted、queued、dequeued、started、blocked、resumed、finished、failed、canceled。每个事件带 task id、worker id、thread id、timestamp、state 和可选等待原因。这样一次事故发生时, 可以问: 任务是在队列里等, 在线程上等, 还是在 I/O 上等。

Listing 10.33 任务生命周期事件

```
accepted(task=42, worker=none)
queued(task=42, queue=compute)
dequeued(task=42, worker=3)
started(task=42, worker=3)
blocked(task=42, reason=IoWrite)
resumed(task=42)
finished(task=42, status=Succeeded)
```

这条 timeline 把线程池从黑盒变成状态机。后续第 14 章的 work stealing 会让 ready 位置从全局队列扩展到 worker 本地 deque, 但这些生命周期事件仍然成立。

线程池配置也要有版本和报告 线程池不是一个静态常量。Worker 数、队列容量、chunk size、是否允许阻塞、是否绑定核心、是否启用 work stealing, 都会改变任务生命周期。一次运行必须记录 `ThreadPoolPlan`: 可见 CPU 数、worker 数、blocking worker 数、queue capacity、chunk size、affinity policy、shutdown mode 和采样策略。否则同一份性能报告无法解释为什么今天 ready 队列增长、昨天没有增长。

Listing 10.34 ThreadPoolPlan 摘要

```

visible_cpus=8
compute_workers=8
blocking_workers=2
queue_capacity=1024
chunk_records=65536
affinity_policy=none
shutdown_mode=drain
timeline_sampling=task-boundary

```

这些字段不是配置炫技，而是排障入口。若 `blocking_workers=0`，I/O 阻塞污染 `compute pool` 是合理怀疑；若 `queue_capacity` 很大，无界压力可能被隐藏；若 `chunk_records` 很大，长尾 `task` 会更明显。线程调度问题必须能回到计划对象和实际运行报告。

把 worker 状态压成可采样枚举 线程运行时不应只暴露“活着/死了”。更有用的状态是：`IdleWait`、`RunnableInPool`、`RunningTask`、`BlockedOnQueueSpace`、`BlockedOnIo`、`BlockedOnFuture`、`BlockedOnMutex`、`Stopping`、`Stopped`。采样频率可以很低，但状态必须互斥且有进入/退出事件。这样报告可以说“8 个 worker 中 5 个在 I/O 等待，2 个在 future 等待，1 个运行 CPU task”，而不是只说 CPU 使用率低。

Listing 10.35 WorkerState 枚举示例

```

IdleWait
RunningTask
BlockedOnQueueSpace
BlockedOnIo
BlockedOnFuture
BlockedOnMutex
Stopping
Stopped

```

状态枚举还会约束代码结构。进入阻塞 I/O 前必须把状态切到 `BlockedOnIo`，返回后恢复 `RunningTask`；等待同池 future 前必须记录 `BlockedOnFuture`，甚至触发运行时断言。可观测性不是事后日志，而是反过来迫使运行时把等待路径写清楚。

调度结论必须能落成改动 线程排障的最终产物不是“调度器有问题”，而是一张改动清单。若根因是同池等待，改动是禁止 worker 同步等待同池 future、改 continuation；若根因是 I/O 阻塞，改动是分离 blocking pool 和 completion；若根因是任务粒度太细，改动是增大 chunk 或批量提交；若根因是长尾 split，改动是拆分热点或 work stealing。每个结论都应能对应一个测试和一个报告字段，否则它只是经验判断。

修复后要证明等待形状改变 线程修复不能只看总吞吐。阻塞隔离修复后，`BlockedOnIo` 应从 compute worker 转移到 blocking worker，CPU task 的排队 p99 应下降；chunk 调整后，task 数应下降，单 task 运行时间上升但尾部不过度拉长；亲和性修复后，迁移次数应下降，同时 idle imbalance 不能恶化。每个修复都要有“等待形状前后对比”，否则无法判断是根因被修复，还是偶然环境变好。

Listing 10.36 线程修复验收摘要

```

before:
  compute.BlockedOnIo.high
  cpu_task.queue_p99.high

```

```
after:
  compute.BlockedOnIo.low
  blocking.BlockedOnIo.expected
  cpu_task.queue_p99.lower
  final checksum unchanged
```

这类验收把调度问题重新接回 correctness 和 report。线程池不是为了“用满 CPU”而存在，而是为了让任务在正确的执行域中推进，并且在等待时能解释原因。

如果修复只让平均吞吐上升，却让 p99 排队时间、关闭时间或阻塞状态更差，就不能直接合入默认路径。线程策略购买的是可解释推进，不是单点跑分。

调度修复还要保留失败输入。父任务等待子任务、慢 I/O 混入计算池、冷页 mmap、超细 chunk、热点 split，这些都应成为固定回归。只有这些反例长期存在，线程池才不会在后续重构中退回“看起来能跑”的黑盒，也不会把等待原因重新藏进平均吞吐数字里。每个反例都应带固定 seed、任务数量、worker 数、输入规模和期望状态分布。

10.14 习题

习题与作业

1. 构造一个 4 worker 线程池，提交 4 个父任务，每个父任务再提交一个子任务并等待。解释为什么这不是 mutex 死锁，却会造成同池饥饿。
2. 对同一批解析任务扫描 chunk size。画出吞吐、p95 排队时间、worker idle time 和任务数曲线，指出过细、适中、过粗三个区域。
3. 在单池中混入慢写出任务，记录 CPU task 的 p99 排队时间。再实现 compute pool 与 blocking pool 分离，比较收益和代价。
4. 设计 WorkerSnapshot 的扩展字段，用来区分锁等待、I/O 等待、future 等待和队列满等待。说明哪些字段适合高频采集，哪些字段只能采样。
5. 在一台支持 affinity 的机器上比较不绑定、绑定物理核心、绑定 SMT sibling、按 NUMA 分组四种策略。若机器不支持 NUMA，写出替代实验和未验证边界。
6. 阅读一个工业 executor 或 scheduler，写一页事故驱动报告：它遇到的约束是什么，核心机制是什么，买到什么能力，付出什么代价，Compute Systems Engine 应采用哪一部分。
7. 为一次线程实验记录 thread 生命周期和 worker loop 事件链。解释 task 提交、worker 睡眠、futex/条件变量唤醒、run queue、CPU running 之间的先后关系。
8. 构造一个 mmap 读取大文件的 page fault 实验。比较预热 page cache、冷 page cache、顺序 read、mmap 顺序访问四种路径下 compute worker 的阻塞时间。
9. 针对“CPU 低、ready 队列增长”的事故写一份排障报告。要求依次给出业务工作是否存在、blocked 原因分解、runnable/running 判断、同池依赖检查、page fault 检查、亲和性证据和最终修复工单。

工程案例

知识点小案例：线程数等于核心数也可能跑不满。特例是 8 个 worker 处理 8 个 chunk，但其中一个 chunk 是热点 key 密集区，处理时间是其

他 chunk 的 5 倍。即使线程数刚好等于核心数，7 个 worker 也会提前空闲，最后等待最慢任务。再叠加线程迁移，worker 的热数据可能离开原 cache 或 NUMA 节点。由此推出规律：调度分析不能只看线程数，要看 task 粒度、负载分布、worker state、迁移和等待点。工程报告至少输出 submit、start、finish、worker id、CPU id 和输入规模，才能解释空闲来自调度、数据倾斜还是同步。

第 11 章

Linux 调度、内存管理与系统瓶颈边界

11.1 为什么性能书必须讲内核边界

第二册前面的章节已经讲了线程、亲和、NUMA、cache 和同步。但真实系统里，用户态线程不是直接占有 CPU，用户态地址也不是直接等于物理内存。调度器决定线程何时运行、在哪个 CPU 上运行；内存管理决定虚拟地址何时获得物理页、页面何时回收、脏页何时写回；cgroup 和容器可能改变可见资源；I/O、page fault、signal 和内核线程会改变尾延迟。若只看用户态代码，很多瓶颈会被误判成“算法慢”。

本章不试图把 Linux 内核写成一本书，而是给性能工程建立必要边界：哪些现象属于用户态 runtime 能直接控制，哪些现象必须通过内核证据解释，哪些结论在没有权限时只能保守表达。

11.2 调度器看到的是 thread，不是业务 task

用户态 runtime 看到的是 task、future、queue、deadline、priority 和 dependency；内核调度器看到的是 schedulable entity、线程状态、runqueue、优先级、时间账本、CPU 拓扑、NUMA、cgroup 和负载均衡。两者信息不同，所以调优必须把两张图接起来。

Listing 11.1 用户态事实和内核事实的对应关系

```
runtime:
  task ready
  worker idle
  queue length
  blocking reason
  deadline and priority

kernel:
  thread runnable
  thread running
  sleeping on futex or I/O
  runqueue delay
  CPU migration
  cgroup throttling
```

一个 task ready，不代表正在运行；一个 worker thread runnable，不代表已经拿到 CPU；一个 CPU 使用率低，不代表没有业务工作，可能是 worker 全在 I/O、future 或 page fault 上睡眠。反过来，CPU 使用率高也不代表有效吞吐高，可能是 CAS 重试、锁竞争、线程过多或 busy poll。

11.3 CFS/EEVDF、runqueue 和 cgroup 的最小模型

Linux 调度策略会随版本演进，CFS 和 EEVDF 的具体实现细节也不是应用程序应仿写的对象。性能工程需要抓住稳定模型：每个 CPU 有运行队列；可运行线程进入某个 runqueue；调度器按策略选择下一个线程；线程可能被抢占、唤醒、迁移或受 cgroup 配额限制；负载均衡尝试把工作分布到合适 CPU。

内核对象/现象	工程含义	常见误判
runqueue	runnable 线程在等待 CPU	只看 worker 数，不看 runnable 延迟
wakeup	睡眠线程变得可运行	以为 notify 后立刻 running
preemption	当前线程被切走	把长尾全归因于代码慢
migration	线程换 CPU	忽略 cache/TLB/NUMA 热状态丢失
cgroup quota	容器或服务 CPU 配额	误以为硬件不够强
load balance	跨 CPU 调整 runnable 工作	错误绑核导致局部拥塞

调度报告不需要复现内核源码，但要能回答：线程有没有 runnable 但没 running；是否频繁迁移；是否发生大量自愿或非自愿 context switch；是否被 cgroup 限流；是否某些 worker 长时间睡在 futex、I/O 或 page fault。

11.4 上下文切换和 off-CPU 时间

热循环慢通常看 on-CPU 指标，服务尾延迟则常常需要 off-CPU 视角。一个请求的 wall time 可能大部分不在 CPU 上执行，而是在等待锁、条件变量、磁盘、网络、page fault、下游服务或 cgroup 配额。若只看 `perfrecord` 的 CPU 采样，可能只看到请求真正运行的短窗口，完全漏掉等待原因。

Listing 11.2 请求延迟分解

```
request wall time:
  queue_wait
  runnable_wait
  on_cpu_execute
  blocked_futex
  blocked_io
  page_fault
  downstream_wait
  cancellation_or_cleanup
```

运行时应在业务 trace 中记录 block reason。系统工具若可用，可以用调度 trace、off-CPU profiling 或 eBPF 辅助定位。若工具不可用，至少要在应用层记录 worker state: RunningTask、BlockedOnMutex、BlockedOnFuture、BlockedOnIo、BlockedOnPageFault、Stopped。把所有等待都写成 Blocked，仍然无法行动。

11.5 虚拟内存：VMA、PTE 和物理页

用户态指针是虚拟地址。一个虚拟地址属于某个 VMA；访问时 MMU 通过页表项找到物理页；TLB 缓存最近翻译；没有有效 PTE 时进入 page fault；权限不匹配时产生保护错误；匿名页第一次写入可能分配物理页；文件 mmap 第一次访问可能从 page cache 或存储中建立映射。

Listing 11.3 一次用户态 load 背后的内存边界

```
virtual address
-> VMA lookup and permission
-> page table entry
-> TLB hit or page walk
-> physical page
-> cache hierarchy
-> register value
```

性能报告要区分四件事：分配虚拟地址范围、建立页表映射、真正分配物理页、数据进入 cache。一个 `malloc` 很快，不代表物理内存已经准备好；一次 `mmap` 成功，不代表文件内容已经读入；RSS 不下降，不一定是泄漏，可能是 allocator 缓存、page cache 或映射仍在。

11.6 buddy、SLUB、reclaim、swap 和 OOM 的工程直觉

内核物理内存管理通常通过 buddy allocator 管理页框，通过 slab/slub 管理小内核对象。用户态不直接调用这些结构，但会受到它们影响：page fault 需要物理页；网络 and 文件系统需要内核对象；大量小对象、socket、buffer、page cache 和匿名页共同竞争内存。

当内存紧张时，内核可能回收 page cache、写回脏页、回收匿名页、触发 swap、压缩内存或最终 OOM。应用看到的可能是突然变慢、major fault 增加、writeback 抖动、分配失败或进程被杀。把这些都归因于“算法慢”是错误的。

现象	可能内核路径	应用层对策
minor fault 多	首次触页、COW、匿名页建立	预触页、分阶段计时、减少突发分配
major fault 多	文件页不在内存或 swap-in	改读取策略、预取、控制内存压力
dirty writeback 抖动	脏页超过阈值或 fsync 集中	批量写、限速、分离提交路径
RSS 不降	allocator 保留或 mmap 未释放	区分 live set、arena、page cache
OOM 或 kill	总内存/限制耗尽	admission、内存预算、降级策略

11.7 NUMA migration 和 first-touch

NUMA 章节已经讲了远端访问。本章补内核边界：页面初次由哪个线程写入，通常会影响它被放在哪个 NUMA node；自动 NUMA balancing 可能迁移页面；线程迁移可能让原本本地页面变成远端访问。性能报告若只写“用了 32 线程”，没有写 first-touch、线程绑定和内存绑定，就无法解释跨 socket 扩展性。

Listing 11.4 NUMA 实验报告字段

```
numa_report:
  topology
  thread_affinity
  memory_policy
  first_touch_thread
  input_size_per_node
  local_vs_remote_case
  migrations_if_observable
  bandwidth_and_latency
```

若无法读取 uncore 或 NUMA counters，也可以做对照：主线程初始化再多 worker 读取；每个 worker first-touch 自己分片；绑定线程不绑定内存；绑定线程并绑定内存。结论要写成“对照支持远端访问解释”，而不是伪装成证明了具体硬件事务。

11.8 page cache、direct I/O 和可靠写

文件 I/O 性能不能只看 read/write 返回时间。Buffered I/O 可能只把数据复制到 page cache；后续 writeback 才真正写设备。Direct I/O 绕过 page

cache，但需要对齐和更严格的 buffer 管理；它减少缓存污染，却不自动更快。可靠写还要区分 `write`、`fdatasync`、`fsync`、临时文件 `rename` 和 `manifest commit`。

Listing 11.5 写出路径状态机

```
UserBuffer
-> KernelPageCache
-> DirtyPage
-> WritebackSubmitted
-> DeviceCompleted
-> DirectoryOrManifestCommitted
```

Compute Systems Engine 的输出报告必须写清可见性边界：只返回给用户态、进入 page cache、设备完成、还是 manifest 已提交。否则“写很快”可能只是把风险推给了崩溃恢复。

11.9 源码阅读路径

本章的源码阅读不要追求一次读完整内核，而是从现象入口读局部路径：

- 调度：从 `wakeup`、`runqueue`、`context switch`、`cgroup quota` 入口读起。
- 内存：从 `page fault`、`VMA`、匿名页、`file-backed page`、`reclaim` 入口读起。
- I/O：从 `page cache`、`writeback`、`fsync` 和 `block layer completion` 入口读起。
- `futex`：从 `wait/wake`、哈希队列和用户态原子 `fast path` 的边界读起。

源码阅读报告不要求解释每一行，而要回答：这个内核路径接收什么状态，改变什么状态，对用户态暴露什么现象，哪些指标能观察到，哪些指标观察不到。

11.10 实验与验收

实验一：线程调度报告。固定一个线程池 workload，记录 `worker state`、`context switch`、迁移、`runnable/running` 边界和队列长度。解释 CPU 使用率低或高的具体原因。

实验二：page fault 报告。比较匿名大数组首次写、第二次写、文件 `mmap` 首次读和热读。记录 `minor/major fault`、耗时和字节吞吐。

实验三：page cache 写出报告。比较普通 `write`、`write` 加 `fsync`、临时文件 `rename` 加 `manifest` 的耗时和崩溃语义。

实验四：NUMA `first-touch` 对照。比较主线程初始化和 `worker` 分片初始化，记录带宽、线程绑定和未验证边界。

验收标准：报告必须把用户态现象映射到内核边界，不能只写“系统有波动”。每个结论都要说明它依赖哪些可观察证据，哪些还只是合理假设。

工程案例

知识点小案例：CPU 使用率低不等于程序没活干。特例是一个写文件管线：业务线程把 `block` 写入 `page cache` 后很快返回，但随后 `fsync` 阶段 p99 很高。观察上 CPU 使用率可能不高，系统却在等存储设备和内核 `writeback`。由此推出规律：用户态 `runnable`、内核等待、`page fault`、`page cache` 和设备 `completion` 是不同边界。工程上不要只用 CPU 利用率判断瓶颈，要把线程状态、系统调用、`page fault`、`dirty bytes`、`fsync` 耗时和设备队列放到同一张阶段表里；不可观测的内核指标要明确降级。

第 12 章

同步原语：锁、条件变量、信号量、阶段和结果

第 9 章让线程池问题变得可见：ready 任务排队、worker 阻塞、CPU 空闲、关闭卡住。现在看一个更小但更本质的事故。线程池 worker 在空队列上等待，主线程调用 `shutdown`，设置 `closed=true` 后只通知了 `not_full`。生产者被唤醒并退出，消费者仍睡在 `not_empty` 上，析构时 `join` 永远不返回。代码里有 `mutex`，有 `condition variable`，也有 `notify_one`，但同步语义仍然错，因为等待谓词没有覆盖关闭状态，通知对象也没有覆盖所有等待者。

同步原语必须从共享状态的不变量和等待谓词推出来。多个核心同时修改同一组字段时，硬件只能保证单条 `load/store` 或特定原子操作的局部语义；它不知道 `head`、`tail`、`size`、`closed` 之间应该满足什么关系。`Mutex` 的作用，是给一组字段的 `状态转换` 建立独占区间，让一个线程能在不变量被临时打破时完成修改，再把一致状态交给其他线程。`Condition variable` 的作用，是在谓词不满足时不再浪费 CPU 自旋，而是释放锁、进入内核或运行库的等待队列，等状态可能变化后再醒来重新检查。`Semaphore` 表达的是可等待的资源额度，例如空槽数量、I/O `in-flight` 许可和连接池令牌。

有界队列足够小，却同时包含互斥、等待、背压、关闭、异常、安全析构和测试。它能把同步的底层来源讲清：共享字段需要锁，因为字段关系不能被并发修改拆碎；空队列需要等待，因为消费者继续自旋会浪费 CPU；满队列需要背压，因为生产者不能无限分配内存；关闭需要广播，因为等待者睡在不同谓词上，且所有等待者都要有退出路径。

12.1 同步原语的统一坐标系

同步原语不能按“哪个 API 新”“哪个 API 快”来学。系统化的学法是先问等待对象是什么，再问这份等待对象由谁维护、状态在哪里、谁改变状态、谁唤醒等待者、错误路径如何退出。`Mutex`、`condition variable`、`semaphore`、`latch`、`barrier`、`future/promise`、`atomic wait`、`eventfd` 和 `futex` 都可以落到同一套坐标系里。

六个审查维度 每个同步设计先填写六个字段。第一，**状态位置** (`state location`)：业务事实放在受 `mutex` 保护的字段、原语内部计数、原子值、`future shared state`、`barrier phase`，还是内核 `fd` 里。第二，**状态转移** (`transition`)：哪个操作让状态改变，例如 `push`、`pop`、`close`、`release`、`count_down`、`set_value`。第三，**等待对象** (`wait object`)：等待的是不变量可修改、谓词变真、许可可用、阶段结束、一次结果 `ready`，还是 `fd` 可读。第四，**唤醒来源** (`wake source`)：哪个状态变化会唤醒哪个等待者。第五，生命周期和失败边界：关闭、取消、超时、异常、析构和 `broken promise` 怎么表达。第六，证据字段：运行时如何证明等待发生在哪里。

Listing 12.1 同步原语统一审查模型

```
SyncModel:
  state_location:
    protected_fields / primitive_counter / atomic_value
    phase_generation / future_shared_state / kernel_fd

  transitions:
```

```
who changes the state
linearization point
whether rollback is possible

wait_object:
  invariant / predicate / permit / phase / one_shot_result
  atomic_version / event_fd / kernel_wait_address

wake_source:
  which transition may make the wait object ready
  notify_one / notify_all / release / count_down
  arrive / set_value / set_exception / write_fd / futex_wake

failure_boundary:
  close / drain / cancel / timeout / exception
  broken_promise / missing_arrival / object_destruction

evidence:
  wait_count, wait_ns, wake_count, timeout_count
  phase_wait_ns, result_ready_ns, permit_leak_check
```

这份模型的目的是不是把所有原语变成同一个东西，而是防止“都能等一下”这种模糊认识。一个原语只表达一部分状态；真实系统经常组合多个原语，但每个原语必须只负责自己能表达的那层事实。

等待对象分类表 下面这张表是本章的总索引。后面所有小节都只是把某一行展开成状态机、代码和错误案例。

等待对象	状态放在哪里	合适原语	典型错误
多字段不变量	对象字段，mutex 保护	<code>std::mutex</code>	用 atomic flag 代表队列状态
谓词变真	mutex 下的谓词	<code>condition_variable</code>	<code>wait</code> 不写完整谓词
资源许可	原语内部许可计数	<code>counting_semaphore</code>	当成对象所有权或 <code>close</code> 广播
一次性归零	latch 内部计数	<code>std::latch</code>	当成可重置阶段同步
多轮阶段	barrier phase 和参与者数	<code>std::barrier</code>	动态任务队列跳过到达
一次结果或异常	future shared state	<code>future/promise</code>	worker 等同池 future
单原子版本	一个 atomic 值	<code>atomic::wait</code>	等跨字段复杂谓词
事件循环唤醒	内核 fd 计数或 readiness	<code>eventfd/pipe</code>	把 payload 塞进 fd
睡眠慢路径	用户态地址和内核等待队列	<code>futex</code>	直接当业务状态机

选择算法 代码审查时可以按下面顺序走。只要某一步回答不清，就不要急着换成更底层的原语。

Listing 12.2 同步原语选择算法

1. List the business facts.
Examples: size, capacity, closed, running_count, phase, result.

2. Group facts into invariants.
If several fields must agree, start with a mutex.
3. Write each wait condition as a sentence.
"Wait until size > 0 or closed" is a predicate.
"Wait until one permit is available" is a permit.
"Wait until all fixed participants reach phase p" is a phase.
4. Add exit edges before choosing an API.
close, drain, cancel, timeout, exception, broken promise, destruction.
5. Choose the narrowest primitive that fully expresses the wait object.
If no primitive fully expresses it, keep explicit state plus mutex/CV.
6. Add evidence fields.
If you cannot measure wait reason, you cannot debug the design.

这个算法会导出一个重要结论：`mutex+condition_variable` 不是落后选择，而是复杂状态的默认 reference。只有状态收缩到“许可”“阶段”“一次结果”“单版本号”这种窄对象时，`semaphore`、`barrier`、`future` 或 `atomic wait` 才是更准确的表达。

组合不是混乱 真实系统往往同时用多种原语。连接池可以用 `semaphore` 限制最多借出连接数，用 `mutex` 保护 `free list`，用 `condition variable` 管理关闭 `drain`，用 `future` 返回一次后台健康检查结果。I/O 子系统可以用 `bounded queue` 保存请求，用 `semaphore` 限制 `in-flight`，用 `eventfd` 唤醒 `epoll` 线程，用 `promise` 把一次提交结果交给调用者。组合本身没有问题；问题是每个原语的责任边界是否写清。

Listing 12.3 一个系统里多个原语的责任分层

```

ConnectionPool:
  mutex:
    protects free_list, closing, broken_connections
  counting_semaphore:
    limits borrowed connections
  condition_variable:
    waits for drain during shutdown
  future/promise:
    reports one background health-check result

Invalid design:
  semaphore protects vector lifetime
  future represents a stream of connections
  barrier waits for dynamic borrowers

```

这就是“系统”的含义：不是把 API 名字列全，而是能把每个等待事实放到正确层级，并解释为什么别的原语不合适。

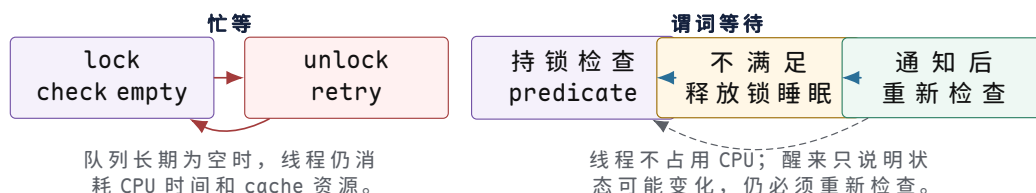
12.2 从自旋到睡眠：为什么需要条件变量

没有条件变量时，消费者可能写出这样的循环：反复拿锁，检查队列是否非空，释放锁，再继续检查。队列短时间为空时，这种自旋可以避免睡眠唤醒成本；队列长时间为空时，它会持续占用 CPU，干扰真正能运行的线程。操作系统调度的基本

单位是 thread，如果一个 thread 只是在检查“还没有数据”，它仍然可能消耗时间片、污染 cache、增加互连流量。

条件变量把这件事改成三步。第一，等待者持锁检查共享状态。第二，谓词不满足时，`wait` 原子地释放锁并让线程睡眠。第三，状态变化后通知等待者，等待者醒来重新持锁检查谓词。数据仍然在共享状态里，通知只是提示“状态可能变了”。这就是为什么条件变量必须和 mutex 一起使用：没有同一把锁，检查谓词和进入等待之间会出现丢失唤醒窗口。

从忙等到谓词等待



图示：本图要证明的命题是：条件变量不是消息队列，而是把“谓词暂不满足”从忙等变成可唤醒睡眠。

12.3 互斥锁保护的是不变量

很多并发错误来自一个误解：mutex 保护某个变量。更准确地说，mutex 保护一组状态之间的不变量。环形队列的 `head`、`tail`、`size`、`buffer` 和 `closed` 不是孤立字段，它们共同说明哪些槽位可读、哪些槽位可写、队列是否还接收新元素。只把 `size` 改成 `atomic`，不能让队列线程安全，因为 `size` 与 `head/tail/buffer` 的关系仍可能被并发修改破坏。

先写不变量表 有界队列的不变量至少包括：容量固定且大于零；`0 <= size <= capacity`；`head` 和 `tail` 总在数组范围内；`size == 0` 表示没有可读槽；`size == capacity` 表示没有可写槽；`head` 指向下一个可读槽；`tail` 指向下一个可写槽；`closed` 后不再接受新 `push`。所有会改变这些事实的代码，都必须在同一互斥边界内完成状态转换。

Listing 12.4 有界队列的不变量不是注释装饰，而是代码审查依据

```
bounded queue invariant:
capacity > 0
0 <= size <= capacity
head and tail are valid slots
size == 0 means no readable slot
size == capacity means no writable slot
closed means future push must fail
```

不变量表把代码审查变得具体。若 `push` 写入槽位后忘记更新 `size`，违反容量关系；若 `pop` 先减 `size` 再移动元素而移动抛异常，状态可能提交一半；若 `close` 在锁外设置 `closed`，等待者可能看见旧状态；若 `head` 在锁内改、`tail` 在锁外改，环形区间关系被拆碎。

临界区越短越好，但不能拆散不变量 临界区应该短，是因为持锁时间越长，其他线程等待越多，尾延迟越不稳定。但“短”不能优先于“完整”。把队列是否满的检查放在锁外，看似减少持锁时间，实际上让检查和写入之间出现竞态。正确原则是：锁内完成最小的、不可拆分的状态转换；锁外执行耗时且不改变受保护不变量的工作。

锁内尤其不能执行未知代码：用户回调、任务函数、磁盘 I/O、网络请求、复

杂日志格式化、可能很慢的析构、会再进入运行时的函数。C++ 中最后一个 `std::shared_ptr` 在锁内析构，也可能触发复杂逻辑。可靠代码常在锁内把对象移动到局部变量，释放锁后再处理或析构。

锁的硬件路径：低竞争和高竞争不是一种东西 低竞争 mutex 常常只是一段很短的用户态原子状态转换：线程尝试把锁状态从 `unlocked` 改成 `locked`，成功后进入临界区；释放时把状态改回 `unlocked`。高竞争时，事情完全不同：多个核心反复读改写同一锁变量所在 `cache line`，失败线程可能自旋一段时间，然后进入内核等待队列；释放锁的线程还要唤醒等待者。于是同一个 `std::mutex` 在低竞争和高竞争下，成本模型完全不同。

接上第一册的汇编读法，低竞争锁的快路径可以理解成“对一个锁字做原子状态转换”。不同 C++ 标准库、运行库、操作系统和架构实现不同，但机器形状常围绕 CAS、交换或带锁定语义的读改写写展开。例如一个抽象锁字从 0 表示 `unlocked`，1 表示 `locked`，快路径可能类似：

Listing 12.5 mutex fast path 的典型汇编形状

```
; rdi = address of lock word
mov    eax, 1
xor    ecx, ecx
lock cmpxchg dword ptr [rdi], eax
jne    .Lcontended
; acquired
```

第一册可以解释这几条指令的局部语义：`cmpxchg` 比较 `eax` 与内存旧值，匹配时写入新值，不匹配时把旧值带回 `eax`；`jne` 根据 `flags` 进入竞争路径。这里继续向下看：带 `lock` 的 CAS 需要让锁字所在 `cache line` 进入可修改状态；若锁空闲且 `line` 已在本核心附近，成本很低；若许多核心同时抢锁，所有核心都围绕同一条 `line` 反复读改写写，失败路径还会污染互连和前端。锁的快路径短，不代表高竞争锁便宜。

Listing 12.6 unlock 和等待路径的粗略机器形状

```
unlock fast path:
    store lock word = 0 with release semantics
    if no waiter is recorded, return

contended path:
    mark lock as contended
    spin or enter futex/runtime wait
    sleep until unlock wakes a waiter
    retry acquisition after scheduled
```

这段不是某个库的源码，而是成本地图。低竞争时，锁主要是一个原子状态转换；竞争出现后，问题扩展成 `cache line` 所有权、等待队列、系统调用、调度唤醒和重新抢锁。报告中看到 `lock cmpxchg` 或类似形状，只能说明存在原子快路径；是否慢，要看竞争率、持锁时间、等待路径和锁字共享范围。

Listing 12.7 mutex 获取的粗粒度事件链

```
fast path:
    atomic compare-exchange lock word from unlocked to locked
    enter critical section
```

```

contended path:
  fail to acquire lock word
  optionally spin while owner may release soon
  enter kernel or runtime wait path
  sleep until unlock wakes a waiter
  become runnable, get scheduled, retry acquisition

```

这条链解释了为什么“锁慢”不是可靠结论。保护低频、复杂、多字段不变量时，mutex 往往是最清楚也足够快的选择；把每条记录的计数更新放进同一把锁里，锁变量、等待队列和临界区都会变成热点。性能报告要区分锁获取次数、竞争次数、临界区耗时、等待时间和持锁期间是否执行未知代码。没有这些分项，无法判断应缩短临界区、分片、批量、换原子，还是重新设计数据流。

自旋、睡眠和混合锁 自旋等待适合预计锁很快释放的极短临界区，因为它避免睡眠和唤醒成本；睡眠适合等待时间可能较长的场景，因为它释放 CPU 给其他线程。混合锁先自旋少量轮次，失败后睡眠。选择哪种等待方式不是 API 品味，而是由临界区时间、线程数、CPU 预算、优先级和是否可能被抢占决定。

在 compute worker 中长时间自旋会浪费核心；在持锁线程被抢占时，其他线程继续自旋只能烧 CPU；在单核或过度订阅环境中，自旋甚至会阻止持锁线程获得时间片。条件变量把长等待转成睡眠，信号量把资源额度变成可等待计数，运行时 managed blocking 则让 worker 等待时补充执行资源。同步结构必须把等待时长当作一等变量。

完整竞争账本：从一把锁看 cache line、futex 和调度 现在把一把 mutex 的获取过程拆成可以写入报告的事件链。低竞争路径中，线程在用户态对锁字执行一次 CAS、exchange 或等价原子操作，成功后进入临界区。这个锁字通常只有几个字节，但硬件以 cache line 为单位维护所有权；锁字所在的整条 cache line 会被获取锁的核心拉到可修改状态。若同一条 cache line 上还放了别的频繁写字段，即使这些字段和锁语义无关，也可能被拖进同一条一致性流量里，这就是 false sharing 的一个来源。

竞争路径从 CAS 失败开始。失败说明另一个线程已经拥有锁，当前线程可以选择短暂自旋、让出处理器、进入运行库等待队列，或通过 Linux futex 等机制进入内核睡眠。futex 的直觉是“先在用户态检查一个整数，只有确实需要睡眠时才请求内核把线程挂到等待队列”。它不是锁本身，而是用户态同步原语在竞争路径上使用的睡眠和唤醒接口。释放锁时，如果运行库知道存在等待者，就需要把锁字改成未持有状态，并唤醒一个或多个等待线程；被唤醒的线程只是变成 runnable，随后还要等调度器把它放回 CPU，再重新竞争锁。

Listing 12.8 锁竞争的完整事件账本

```

acquire attempt:
  read or modify lock word
  if unlocked, publish ownership and enter critical section

contention:
  fail atomic transition
  record waiter or contended state
  spin for a bounded budget if owner is expected to release soon
  enter futex or runtime wait when spinning is no longer profitable

unlock:
  finish updates protected by the mutex
  release lock word
  wake a waiter when contended state says someone may sleep

```

```

resume:
    waiter becomes runnable
    scheduler chooses a CPU
    waiter retries acquisition and may still lose the race

```

这条账本能解释很多“偶发慢”。等待线程被唤醒，不等于它立刻获得锁；它可能在 `runnable` 队列里等待，也可能醒来后发现另一个线程已经抢先进入临界区。释放线程调用 `wake`，不等于等待者一定仍在睡眠；等待者可能刚好超时、被取消、被信号打断或已经重新检查状态。同步实现必须允许这些竞态，把正确性建立在共享状态和谓词上，而不是建立在通知次数上。

性能报告因此不能只写一个总耗时。至少要拆出以下字段：

Listing 12.9 mutex 竞争报告字段

```

lock.acquire.count
lock.contended.count
lock.spin.iterations
lock.futex.wait.count
lock.futex.wake.count
lock.hold_ns.p50
lock.hold_ns.p99
lock.wait_ns.p50
lock.wait_ns.p99
lock.owner_preempted.count
critical_section.calls_unknown_code
critical_section.bytes_touched

```

`lock.hold_ns` 说明持锁线程在临界区里待了多久，`lock.wait_ns` 说明其他线程等了多久。二者不是同一件事：临界区很短但竞争极高，等待时间仍可能很大；临界区很长但低频，吞吐可能仍可接受。若 `owner_preempted` 很多，说明持锁线程经常在临界区内被调度走，其他线程自旋只会浪费 CPU。若 `calls_unknown_code` 不为零，说明锁内执行了无法界定时长的逻辑，尾延迟没有可靠上界。

从竞争账本反推设计 拿有界队列举例，`mutex` 保护 `head/tail/size/closed` 的不变量；`not_empty` 和 `not_full` 保护等待谓词；`notify` 只是提示谓词可能变化。若报告显示锁竞争高，第一反应不能是把所有字段改成 `atomic`，而要先问四个问题。第一，临界区里是否做了拷贝、分配、日志、回调或析构。第二，队列是否太集中，是否需要每 `worker` 本地队列和全局注入队列。第三，是否可以批量 `push` 或批量 `pop`，把多次状态转换合并成一次。第四，等待者是否真正需要被全部唤醒，还是只需要唤醒一个能推进状态的线程。

这四个问题分别对应四类改造。缩短临界区解决持锁时间；分片队列减少锁字 `cache line` 迁移；批量操作降低获取次数；精确通知减少无效唤醒。只有在不变量已经收缩到单个状态字、等待协议也能由单个原子值表达时，才进入第 11 章的原子状态机设计。锁和原子不是谁替代谁，而是合同粒度不同：锁适合保护多字段一致性，原子适合承载小而明确的线性化点。

Listing 12.10 互斥锁保护队列状态转换

```

1 class QueueState {
2 public:
3     void push(std::int32_t value) {
4         std::lock_guard<std::mutex> lock(this->mutex_);

```



```

5     this->values_.push_back(value);
6 }
7
8 private:
9     std::mutex mutex_;
10    std::vector<std::int32_t> values_;
11 };

```

这个示例很小，但习惯要严格：成员访问使用 `this->`，锁对象用 RAII，锁的作用域就是临界区。后续加入 `head_`、`tail_`、`size_`、`closed_` 后，它们都应归入同一不变量边界，直到测量证明需要拆分。

12.4 futex 路径：用户态原子怎样变成内核等待

Linux 上许多高层同步原语的竞争慢路径会落到 futex 或类似机制。futex 可以理解为 fast userspace mutex 的底层接口：正常无竞争时，用户态原子操作完成同步；只有需要睡眠或唤醒时，才进入内核。这个设计来自一个简单事实：每次加锁都进内核太贵，但长期自旋也会浪费 CPU。futex 把“先检查用户态整数”和“必要时挂到等待队列”结合起来。

wait 必须带预期值 futex wait 的关键不是“睡眠”，而是“只有当用户态地址仍等于预期值时才睡眠”。这能避免经典竞态：线程检查到锁被占用，准备睡眠；就在进入内核前，持锁线程释放并唤醒；如果等待方无条件睡眠，它可能错过唤醒。带预期值的 wait 让内核重新读取用户态整数，若值已经变了，就不睡眠，直接返回让用户态重试。

Listing 12.11 futex wait 的竞态防护直觉

```

user space:
    if lock_word == locked:
        futex_wait(address=&lock_word, expected=locked)

kernel side:
    recheck *address
    if *address != expected:
        return immediately
    enqueue current thread on wait queue for address
    schedule away

```

这条设计说明条件变量为什么必须等待谓词。无论是 futex 还是 condition variable，通知都不是状态；状态在用户态整数或受 mutex 保护的变量里。等待者醒来后必须重新检查状态，因为醒来只说明“可能有变化”，不说明自己已经获得资源。

wake 不是交出锁 unlock 里的 wake 也常被误解。释放线程把锁字改成 unlocked，然后唤醒一个等待者。被唤醒线程只是变成 runnable；它还要等待调度器选择 CPU，还要重新执行获取锁的原子操作，还可能输给另一个正在运行的线程。wake 不保证公平，也不保证被唤醒者立刻进入临界区。

Listing 12.12 unlock 到 waiter 真正进入临界区之间的路径

```

owner thread:
    store lock_word = unlocked with release semantics
    if lock_word indicated waiters:
        futex_wake(address=&lock_word, count=1)

```

```
waiter thread:
  wakes from kernel wait queue
  becomes runnable
  scheduler runs it later
  retries compare-exchange on lock_word
  enters critical section only if CAS succeeds
```

这个路径解释了 tail latency。某个线程的 `wait_ns` 可能很长，不只是因为持锁时间长，也可能因为调度延迟、优先级、CPU 过度订阅、cgroup 限制或唤醒后重新竞争失败。锁报告必须区分 hold time、wait time 和 runnable delay。

futex wait/wake 的最小状态机 把 futex 看成“按用户态地址组织的条件睡眠”更准确。用户态先读一个整数，决定自己愿意在它仍保持某个值时睡眠；进入内核后，内核再读一次这个地址。若值已经变化，等待者不能睡，必须返回用户态重试；若值仍等于 `expected`，内核把当前线程挂到这个地址对应的等待队列，然后调度走。wake 只是在同一地址的等待队列上唤醒若干线程，不转移业务所有权。

Listing 12.13 futex wait/wake 的状态机

```
WaiterUserCheck:
  read *uaddr
  if value != expected:
    retry in user space
  call futex_wait(uaddr, expected, optional_timeout)

KernelRecheck:
  read *uaddr again
  if value != expected:
    return without sleeping
  enqueue current thread on wait queue keyed by uaddr

Sleeping:
  thread is not runnable
  wake, timeout, signal, or cancellation may end the sleep

RunnableAgain:
  futex_wait returns to user space
  user code must re-read state and retry protocol

Waker:
  change user-space state first
  call futex_wake(uaddr, count)
  selected waiters become runnable, not owners
```

这个状态机有三个重要边界。第一，`futex_wait` 返回不等于条件满足；它只表示线程不再睡在内核队列上，返回原因可能是 wake、超时、信号或值不匹配。第二，`futex_wake` 返回不等于业务完成；它只说明内核尝试唤醒了若干等待者。第三，futex key 是用户态地址和相关映射身份，不是 C++ 对象名；对象销毁、地址复用和跨进程共享都会让生命周期约束变得更硬。

futex 系统调用合同：返回码也是状态 只说“调用 futex 睡眠”还不够。futex 的基本合同是：用户态传入一个地址、一个操作和一个预期值；内核只在这个地址上的值仍等于预期值时把线程挂起。等待返回后，用户态不能根据返回值直接认为

资源可用，而要把返回值当作“为什么不再睡眠”的线索，再重新读取共享状态。

Listing 12.14 `futex wait/wake` 的系统调用层合同

```
wait side, simplified:
    rc = futex(uaddr, FUTEX_WAIT_PRIVATE, expected, timeout)
    if rc == 0:
        some wake path ran, but condition may still be false
    if rc == -EAGAIN:
        *uaddr did not match expected at kernel recheck
    if rc == -EINTR:
        signal interrupted the sleep
    if rc == -ETIMEDOUT:
        timeout expired
    always re-read user-space state before deciding next step

wake side, simplified:
    store new user-space state first
    n = futex(uaddr, FUTEX_WAKE_PRIVATE, wake_count)
    n tells how many waiters the kernel made runnable
    n does not tell how many acquired the mutex or consumed work
```

PRIVATE 版本表示等待地址只在同一进程内使用，内核可以走更便宜的路径；跨进程共享 mutex 或 process-shared condition variable 不能随便使用 private 语义。这个细节解释了为什么同步原语的“是否跨进程共享”不是装饰字段：它会改变 `futex key` 的解释，也会改变调试时看到的等待路径。

返回码还解释了虚假唤醒的工程来源。即使没有业务通知，等待也可能因为信号、超时或实现内部原因返回；即使收到了 `wake`，醒来的线程也可能重新竞争失败。因此正确循环永远是：

Listing 12.15 `futex` 用户态循环必须以状态为准

```
while true:
    state = load shared word
    if protocol says progress is possible:
        try progress with atomic operation
        if success:
            return
    futex_wait(shared word address, expected=state)
    // return reason is not the protocol; state is the protocol
```

实测证据：futex 返回码和 wake 数量 下面这组最小实验应在支持 `futex` 的 x86-64 Linux 环境中运行。报告只记录可公开复现的环境字段，例如内核版本、C 库版本、编译器版本和 PMU 权限状态；不要把私有机器名、账号或路径写入书稿。实验代码直接调用 `SYS_futex`，只验证用户可见的返回码、`errno` 和 `FUTEX_WAKE_PRIVATE` 返回数量；它不声称证明内核内部队列实现。

Listing 12.16 `futex` 合同实验命令

```
cmake -S books/cpu-volume-2 -B books/cpu-volume-2/build/compsys-debug \
    -DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-2/build/compsys-debug
books/cpu-volume-2/build/compsys-debug/labs/compute_systems/
```

compsys_futex_lab_probe

Listing 12.17 futex 合同实验输出摘要

```
futex_mutex,4,50,200,<contended>,<wait>,<wake>
futex_contract,1,1,1,1,1,1,1
```

第一行是一个故意简化的用户态锁实验：4 个 worker 各执行 50 次临界区递增，最终值必须是 200；后面的 `contended/wait/wake` 数字受调度影响，只要求能观测到竞争、进入等待慢路径并唤醒等待者。第二行把 futex 系统调用合同压成稳定字段：1 个等待者、`wake_count=1`、一次 `EAGAIN`、一次 `ETIMEDOUT`、一次 `EINTR`、一次 `ready` 状态观察，以及等待循环按 `ready` 状态正确退出。

这份输出把前面的合同钉住了。第一，用户态地址上的值已经不是 `expected` 时，等待不会真正睡下，返回 `EAGAIN`。第二，超时和信号中断都只是“离开等待”的原因，不表示业务条件成立。第三，`wake_count=1` 表示内核让一个等待者重新变为 `runnable`；等待者醒来后仍然要回到用户态协议重新判断。第四，锁实验里的 `wake` 数量不是吞吐指标，它只证明这次运行确实覆盖了 futex 慢路径；调度、CPU 过度订阅和临界区长度都会改变这个数字。

若实验环境不能采集 `cycles,instructions`，报告必须显式降级，例如：

Listing 12.18 perf 计数器边界

```
Error:
No supported events found.
The cycles:u event is not supported.
```

因此这组实验只能支持 futex 合同和返回码分析，不能支持 IPC、`cycles` 或调度开销结论。要分析锁竞争的性能，还需要在能访问 PMU 的环境里补 `perfstat`、`off-CPU profile` 或调度 `trace`。

futex 不替你建立内存序 futex 只负责睡眠和唤醒，不自动把业务数据发布给等待者。业务状态仍要用 `mutex`、`atomic acquire/release` 或其他同步关系建立可见性。典型顺序是：生产者先写业务数据，再用 `release` 语义发布状态，最后 `wake`；消费者醒来后用 `acquire` 语义重新读取状态，确认条件满足后再读业务数据。若状态发布和数据写入没有同步关系，futex `wake` 只能让线程醒来，不能保证它看到正确数据。

Listing 12.19 futex 和内存序的分工

```
producer:
    write payload fields
    store ready = 1 with release semantics
    futex_wake(&ready, 1)

consumer:
    while load ready with acquire semantics == 0:
        futex_wait(&ready, expected=0)
    read payload fields
```

这个例子故意很窄：单一 `ready` 标志可以用 `atomic wait` 或 futex 风格协议表达。若状态跨多个字段，例如 `size/closed/error/head/tail`，就不要把每个字段都硬塞进 futex word。更清楚的实现通常是 `mutex` 保护不变量，`condition`

variable 等待谓词，底层库再决定是否使用 futex 睡眠。

mutex 底层怎样从用户态快路径走到睡眠 高层 `std::mutex` 或 `pthread_mutex` 通常不是每次都进内核。无竞争时，它只是对锁字做原子转换：从 `unlocked` 改成 `locked`，成功后进入临界区。竞争时，库会把锁字推进到“有人等待”或类似状态，可能短暂自旋，然后才用 `futex` 或平台等待对象睡眠。释放时，持锁线程用 `release` 语义发布临界区写入，把锁字改回可获取状态；若锁字表明可能有等待者，再走 `wake` 路径。

Listing 12.20 mutex 的典型三态锁字模型

```
lock word states:
    0 = unlocked
    1 = locked, no known waiters
    2 = locked, waiters may exist

lock fast path:
    if compare_exchange(0 -> 1) succeeds:
        enter critical section

lock contended path:
    if state is 1:
        try mark state as 2
    while state != 0:
        futex_wait(&lock_word, expected=2)
        re-read lock_word
    acquire by changing 0 -> 2 or 0 -> 1

unlock:
    if exchange(lock_word, 0) was 1:
        no known waiter
    else:
        futex_wake(&lock_word, 1)
```

真实库的锁字编码会更复杂，可能包含自适应自旋、robust mutex、priority inheritance、owner 信息或平台差异，但教学上这三个状态足够说明核心原则：锁的正确性来自用户态锁字的状态转移，`futex` 只是把“等待锁字变化”从忙等变成睡眠。若锁字没有 waiter 信息，`unlock` 可能不知道是否需要 `wake`；若 waiter 信息维护错，可能出现无效唤醒或睡眠者没人唤醒。

直接用 futex 写 mutex 为什么危险 直接封装 `futex` 很诱人，因为接口看起来只有一个整数。但完整 `mutex` 还要处理所有权、递归策略、析构时是否仍有等待者、进程退出、优先级反转、超时、信号、取消和调试可见性。更关键的是，`futex wait` 的 `expected` 值必须和用户态状态机严格匹配；只要有一条状态边没有 `wake`，等待者就可能睡到超时或永远不醒。

Listing 12.21 简化 `futex mutex` 的审查清单

```
state word:
    what values exist?
    which thread may write each value?
    which value means waiters may exist?

acquire path:
```



```

where is the linearization point?
when does a thread decide to sleep?
what exact expected value does it pass to futex_wait?

```

```

release path:

```

```

does it publish protected writes before unlocking?
does every path that may satisfy waiters call wake?
can a woken thread race with a new fast-path acquirer?

```

```

lifetime:

```

```

who proves no thread is sleeping on this address before destroy?
can the address be reused while a late waiter still exists?

```

如果这些问题答不出来，使用标准库 `mutex` 或 `pthread mutex` 更可靠。系统书讲 `futex` 的目的不是鼓励读者重写锁，而是让读者能解释 `strace`、`perf`、`sched`、`off-CPU profile` 和线程 `dump` 里的等待现象。

condition variable 底层怎样避免丢失唤醒 条件变量比 `mutex` 更容易误解，因为它等待的不是一个锁字，而是“受 `mutex` 保护的谓词”。等待者调用 `wait(lock)` 时，库必须完成一个复合动作：把当前线程登记到条件变量等待集合，然后释放用户 `mutex`，然后睡眠。醒来后，它还必须重新获取 `mutex`，才把控制权还给用户代码。用户手写“先 `unlock`，再 `park`”会在登记等待者之前暴露丢失唤醒窗口。

Listing 12.22 condition variable wait 的库协议

```

cv_wait(mutex):
    user already holds mutex
    register current thread or sequence number in cv wait set
    unlock mutex with release semantics
    block on cv wait object, often through futex-like primitive
    wake because notify, timeout, signal, or spurious reason
    re-lock mutex with acquire semantics
    return to user code so predicate can be checked again

cv_notify_one:
    select one waiter or advance notify sequence
    wake corresponding futex/runtime wait object

```

这就是为什么条件变量必须配合 `while` 谓词。底层实现可以用序号、等待队列、内部锁和 `futex` 把“登记等待者 + 释放 `mutex` + 睡眠”做成一个库协议，但它不能替业务保存通知次数，也不能保证醒来的线程就是拿到资源的线程。被 `notify_one` 唤醒的线程还要重新竞争 `mutex`；在它拿到锁之前，别的线程可能已经消费了资源。因此条件变量语义不是“通知一次，消费一次”，而是“状态可能变化，请在锁内重新判断”。

condition variable 的序号模型 很多实现会把 `condition variable` 的等待集合抽象成一个或多个序号。等待者在持有用户 `mutex` 时观察当前序号和谓词；若谓词不满足，就把自己关联到这个序号，然后由库释放 `mutex` 并等待序号变化。通知者在改变业务状态后推进序号并唤醒等待者。序号不是业务状态，它只是帮助等待者判断“我睡觉期间可能发生过通知”。

Listing 12.23 用序号理解 condition variable

```

waiter:

```

```

lock user mutex
while predicate is false:
    seq = cv.sequence
    cv registers this waiter for seq
    atomically unlock user mutex and sleep while sequence == seq
    lock user mutex again
    // predicate must be checked again

notifier:
    lock user mutex
    modify business state
    unlock user mutex
    increment or publish cv.sequence
    wake one or many waiters

```

这个模型能解释两个常见现象。第一，`notify` 可以在不持用户 `mutex` 的情况下调用，但状态修改必须和等待谓词使用同一把 `mutex` 建立顺序；否则等待者可能看见旧状态或错过退出条件。第二，`notify_all` 可能造成惊群：很多等待者被唤醒，最终只有少数能使谓词成立，其余线程重新睡下。惊群是性能问题，不是正确性问题；正确性仍然来自谓词循环。

condvar、mutex 和 futex 地址为什么可能不一样 调试时常看到同一段条件变量代码在多个 `futex` 地址上等待或唤醒：有的地址属于 `mutex` 锁字，有的属于 `condition variable` 内部序号或等待字。等待者从 `condvar` 醒来后还要重新获取 `mutex`；如果 `mutex` 被别的线程持有，它可能马上又睡到 `mutex` 的 `futex` 地址上。于是一次 `notify_one` 后的慢请求，可能同时包含 `condvar wake`、调度延迟、`mutex` 重新竞争和业务谓词失败。

Listing 12.24 一次条件变量唤醒可能经过多段等待

```

consumer:
    sleep on condition-variable wait word
    wake after notify
    try to re-lock user mutex
    if mutex is owned:
        sleep on mutex lock word
    after mutex acquired:
        check predicate again
        if predicate false:
            sleep on condition-variable wait word again

```

这就是为什么只看一条 `futex` 地址很容易误判。报告要把应用等待点、`mutex` 名称、`condition variable` 名称、线程状态和业务队列状态一起记录。否则“`futex` 很多”只是现象，不是瓶颈解释。

mutex、condvar 和 futex 的职责分层 这三者的职责不要混在一起。`Mutex` 保护共享不变量；`condition variable` 定义哪些线程在等待哪些谓词；`futex` 或平台等待对象负责让线程在慢路径睡眠。把 `futex` 直接当业务事件用，会失去谓词、关闭和对象生命周期；把 `condition variable` 当消息队列用，会丢通知语义；把 `mutex` 当背压机制用，会把容量、关闭和错误路径塞进临界区。

层级	它负责什么	它不负责什么
mutex	保护多字段不变量和线性化点	记录业务消息、保存通知次数
condition variable	等待谓词可能变真	保证资源仍然可用、替代 while 检查
futex/wait object	让线程按地址或对象睡眠/唤醒	理解 C++ 对象生命周期和业务状态
业务状态机	close、cancel、drain、error、capacity	直接操作内核等待队列

用户态锁字和内核等待队列必须保持一致 futex 等待队列按用户态地址组织。若程序在等待者仍可能睡眠时销毁或复用锁对象，旧等待者可能挂在已经复用的地址语义上。高层库会通过对象生命周期规则避免这种错误；业务代码也必须保证 mutex、condition variable 和等待谓词的对象活得足够久。关闭队列时，先改状态，再通知所有等待者，再等待使用者退出，不能边销毁边唤醒。

Listing 12.25 销毁同步对象前必须清空等待协议

```
shutdown queue:
    lock mutex
    closed = true
    unlock mutex
    notify all waiters
    join worker threads or prove no waiter remains
    destroy mutex and condition variables
```

这不是形式主义。futex 的内核队列只知道地址，不知道 C++ 对象语义；条件变量也不会替业务证明生命周期。同步对象的生命周期是协议的一部分。

futex 观测字段 真实系统里，futex 问题常表现为 CPU 不高但延迟高，或者线程大量处于 sleep/runnable 之间。报告可以把应用字段和系统字段对齐：

Listing 12.26 futex 和锁慢路径的观测字段

```
mutex_name
lock_word_address_or_id
acquire_attempts
contended_attempts
futex_wait_count
futex_wake_count
spurious_or_timeout_wake_count
runnable_delay_ns
hold_ns_p99
wait_ns_p99
owner_thread_state_when_waiters_blocked
```

如果没有系统级 trace，也要在应用里记录等待开始、进入睡眠前、醒来后、获得锁、释放锁的时间点。这样至少能判断等待发生在临界区、内核睡眠、调度还是重新竞争。后续运行时章节会把这种时间线扩展到 worker、task 和 completion。

从 futex 回到设计选择 看到 futex wait 很多，不等于要删掉 mutex。可能的修复包括：缩短临界区、批量操作、分片队列、减少无效唤醒、把阻塞 I/O 移出 compute worker、提高任务粒度、或把复杂多字段状态保留在锁下。只有当状态机已经缩小到单个原子值、等待条件清楚、生命周期可证明时，才考虑 `atomic::wait` 或无锁结构。慢路径证据用来选择设计，不是用来制造对某个同步原语的偏见。

`futex`、`CV`、`atomic wait` 和 `coroutine await` 的同一张等待地图 到这里可以把几种“等待”放在一张地图里。`futex` 是线程睡眠的底层慢路径；`condition variable` 把“多字段谓词 + `mutex` + 等待队列”组织起来；`atomic::wait` 把等待条件收缩到一个原子值；`future` 的阻塞 `get` 会让当前线程 `park`；`coroutine` 的 `co_await` 则不 `park` 线程，而是把 `coroutine frame` 作为 `continuation` 注册到某个完成状态上。它们都在等状态变化，但保存等待上下文的位置不同。

机制	等待上下文放在哪里	唤醒或恢复条件	是否占住线程
<code>futex wait</code>	内核等待队列中的线程	用户态地址值变化后 <code>wake</code>	不占 CPU，但占 线程身份
<code>CV wait</code>	<code>CV</code> 等待队列中的线程	谓词可能变化后 <code>notify</code>	不占 CPU，但占 线程身份
<code>atomic::wait</code>	原子地址对应的等待队列	原子值不再等于 <code>old value</code>	不占 CPU，但占 线程身份
<code>future get</code>	当前线程栈和 <code>shared state waiter</code>	<code>shared state terminal</code>	不占 CPU，但占 线程身份
<code>co_await</code>	<code>coroutine frame</code> 和 <code>continuation list</code>	<code>awaitable ready</code> 后调 度 <code>handle</code>	不占线程，只占 <code>frame</code>
<code>I/O completion</code>	请求表和 <code>completion queue</code>	内核或 <code>I/O worker</code> 返 回结果	不占 <code>compute worker</code>

这张表解释为什么“把阻塞等待换成 `coroutine`”并不是简单替换 API。阻塞等待保存的是线程调用栈，调度器看到的是一个 `sleeping thread`；协程等待保存的是 `coroutine frame`，调度器看到的是一个尚未 `ready` 的 `continuation`。若 `awaitable` 最终仍在 `compute worker` 上调用阻塞 `read` 或 `future.get`，它只是把阻塞包进协程，执行资源仍会被耗尽。真正的非阻塞等待必须让完成事件来自 `I/O completion`、`timer`、`FutureState completion` 或任务图 `ready` 边，而不是让 `worker` 在 `awaiter` 里睡眠。

Listing 12.27 等待机制选择的底层事件链

```
blocking mutex/CV/future:
  user code keeps a native thread as the waiting context
  slow path may enter futex or kernel wait object
  wake makes the thread runnable
  scheduler later runs it
  user code resumes on the same thread stack

coroutine await:
  user code stores live locals in coroutine frame
  await_suspend registers coroutine_handle as continuation
  current worker returns to scheduler immediately
  completion enqueues the handle
  some executor later resumes the frame

async I/O await:
  submit request and transfer buffer ownership
  await_suspend records coroutine_handle in request state
  I/O completion records bytes/status
  completion loop enqueues coroutine_handle to executor
  await_resume returns bytes or throws error
```

从这个角度看，`futex` 和 `coroutine` 不是竞争关系。`futex` 是“线程需要睡眠时”常见的内核路径；`coroutine` 是“不要把等待上下文绑在线程上”的语言机制。线

程池、I/O runtime 和 coroutine scheduler 可以同时使用 futex：例如 executor 队列为空时，worker 线程可能用 CV/futex 睡眠；但某个业务 coroutine 等 I/O 时，不应占住这个 worker。能否区分这两层，是异步运行时设计的关键。

什么时候保留线程等待，什么时候改成 awaiter 不是所有等待都要改成 coroutine。外部控制线程等一次初始化结果，用 `future.get` 或 CV 很直接；低频 shutdown 等待也可以阻塞。需要改成 awaiter 的，是那些发生在有限 compute worker 内、可能等待设备、网络、定时器、远端 RPC 或同一运行时任务的路径。判断标准不是“API 新不新”，而是等待期间这个线程是否本来应该继续推进 ready work。

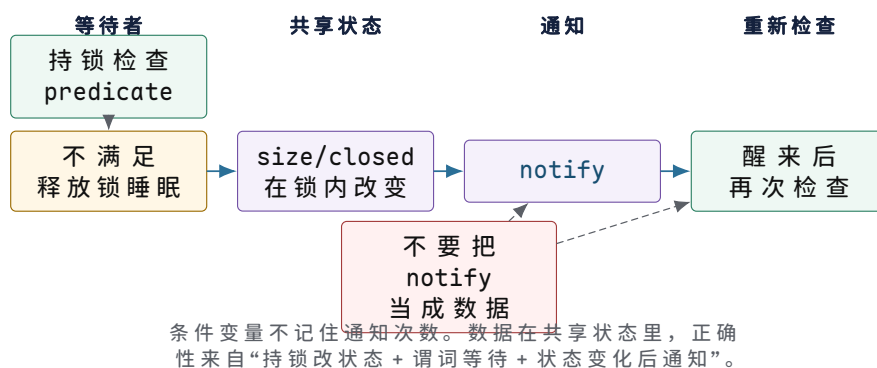
场景	合适形态	原因
主线程等初始化完成	阻塞 future 或 latch	不占 compute worker，语义简单
worker 等同池子任务	continuation 或 coroutine await	阻塞会耗尽执行槽
worker 等磁盘或网络	async I/O await 或 blocking pool	设备等待不应占 compute worker
executor 队列为空	worker 线程 CV/futex sleep	等的是调度器内部队列，不是业务 I/O
单原子版本发布	<code>atomic::wait</code>	状态足够窄，避免 mutex/CV 复杂度
多字段队列关闭	mutex + CV	谓词跨 size、closed、error、unfinished

因此，报告里不能只写“这里用了 coroutine”。要写：等待对象是什么，completion 从哪里来，awaiter 是否会阻塞线程，恢复到哪个 executor，取消时谁把状态推到终态，底层 worker 什么时候才需要 futex sleep。否则 coroutine 只会把阻塞和生命周期错误藏进更深的状态机。

12.5 条件变量等待的是谓词

条件变量常被误解为事件通知系统。正确模型是：线程持锁检查共享状态；若谓词不满足，`wait` 原子地释放锁并睡眠；被通知或虚假唤醒后，重新持锁检查谓词。通知本身不保存状态；如果通知发生时没有等待者，它不会留在条件变量里等后来者；被唤醒也不代表条件已经成立，因为别的线程可能先拿到锁并消费资源。

条件变量：通知只是让等待者重新检查谓词



图示：本图要证明的命题是：等待者醒来只说明状态可能变了，不说明谓词一定为真。

谓词必须覆盖退出条件 有界队列中，生产者等待的谓词不是“收到 `not_full` 通知”，而是“队列未满，或者队列已关闭”。消费者等待的谓词不是“收到 `not_empty`

通知”，而是“队列非空，或者队列已关闭”。关闭状态必须进入所有相关谓词，否则关闭时即使 `notify_all`，线程醒来后发现队列仍满或仍空，又会睡回去。

Listing 12.28 条件变量等待的是谓词，不是通知次数

```

1 std::optional<std::int32_t> pop_wait() {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_empty_.wait(lock, [this]() {
4         return this->closed_ || this->size_ > 0;
5     });
6
7     if (this->size_ == 0) {
8         return std::nullopt;
9     }
10
11     const std::int32_t value =
12         this->buffer_[static_cast<std::size_t>(this->head_)];
13     this->head_ = (this->head_ + 1) %
14         static_cast<std::int32_t>(this->buffer_.size());
15     --this->size_;
16     this->not_full_.notify_one();
17     return value;
18 }

```

这段代码的线性化点是 `head/size` 更新。消费者取出元素并释放一个槽位后，通知一个生产者重新检查 `not_full` 谓词。若此时有多个消费者被唤醒，只有持锁后看到 `size>0` 的消费者能继续；其他消费者必须继续等待或在关闭状态下退出。

notify 的时机 常见写法是先在锁内修改状态，再通知。通知可以发生在仍持锁时，也可以在释放锁后发生，两者有性能和竞争细节差异。初始阶段不要为了微优化打乱状态逻辑：先保证所有等待者的谓词正确，所有能改变谓词结果的状态修改都在锁保护下，关闭时通知所有相关等待者。

丢失唤醒不是“少 notify 一次” 丢失唤醒的根因通常不是通知次数少，而是“检查谓词”和“进入等待”没有作为一个受锁保护的動作。错误写法常先在锁外检查 `size==0`，再进入 `wait`；如果生产者在两个动作之间完成 `push` 和 `notify`，消费者随后睡下去，就可能永远等不到下一次通知。标准条件变量的 `wait(lock, predicate)` 把这个模式固定下来：先持锁检查，谓词不满足时原子释放锁并睡眠，醒来后重新持锁检查。

Listing 12.29 错误模式：if 等待无法抵抗虚假唤醒和关闭竞态

```

1 std::optional<std::int32_t> broken_pop() {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     if (this->size_ == 0) {
4         this->not_empty_.wait(lock);
5     }
6     if (this->size_ == 0) {
7         return std::nullopt;
8     }
9     return this->pop_locked();
10 }

```

这段代码有两个问题。第一，`wait` 允许虚假唤醒，醒来后不能假设队列非空。第

二，关闭语义缺失：若队列空且已关闭，消费者应退出，而不是继续等待。正确写法不是在 `wait` 后补更多 `if`，而是把完整谓词写进去：`closed_||size_>0`。同步代码的可读性来自谓词完整，而不是通知语句多。

notify_one 和 notify_all 的边界 `notify_one` 适合一次状态变化最多只能让一个等待者继续，例如 `push` 一个元素后唤醒一个消费者，或 `pop` 一个元素后释放一个槽位唤醒一个生产者。`notify_all` 适合全局状态变化，例如 `close`、配置切换、取消、阶段结束。关闭时只 `notify_one` 很危险，因为可能有多个生产者和消费者分别睡在不同条件变量上；只醒一个等待者，其他等待者可能没有下一次状态变化来唤醒。

状态变化	推荐通知	原因
插入一个元素	<code>not_empty.notify_one()</code>	最多释放一个消费者的工作
弹出一个元素	<code>not_full.notify_one()</code>	最多释放一个生产者的槽位
队列关闭	两个条件变量都 <code>notify_all()</code>	所有等待者都要重新判断退出
取消全部任务	相关等待点 <code>notify_all()</code>	等待目标从“资源可用”变成“停止等待”
阶段屏障完成	<code>notify_all()</code> 或 <code>barrier</code>	多个等待者共享同一阶段结果

12.6 等待协议的三件事：状态、谓词、通知

每个等待点都要写成三件事。第一，状态在哪里，由哪把锁或哪个原子保护。第二，等待谓词是什么，退出条件是否包含关闭、取消、错误和超时。第三，哪些状态变化会让谓词从 `false` 变成 `true`，并由谁负责通知。只写“在条件变量上等待”没有任何同步含义；条件变量本身不保存业务事实，事实必须在共享状态中。

状态变化必须能解释通知 生产者 `push` 成功后，`size_` 从 0 变成 1，消费者的 `size_>0||closed_` 谓词可能变真，所以通知 `not_empty_`。消费者 `pop` 成功后，`size_` 从 `capacity` 变成 `capacity-1`，生产者的 `size_<capacity||closed_` 谓词可能变真，所以通知 `not_full_`。关闭时，`closed_` 从 `false` 变成 `true`，所有生产者和消费者的退出谓词都可能变真，所以两个条件变量都广播。

这套推导比背通知规则可靠。若以后增加 `cancelled_`、`error_`、`draining_`，必须重新检查每个等待谓词和通知点。同步代码的 `bug` 常来自新增状态后没有把它加入所有等待谓词。

Listing 12.30 等待点审查表

```
waiter: producer
  predicate: closed || size < capacity
  state changes that may satisfy it:
    pop decreases size
    close sets closed
  notifications:
    not_full.notify_one after pop
    not_full.notify_all after close

waiter: consumer
  predicate: closed || size > 0
  state changes that may satisfy it:
    push increases size
```

```

close sets closed
notifications:
    not_empty.notify_one after push
    not_empty.notify_all after close

```

丢失唤醒的真实时间线 丢失唤醒常被误解成“少调用了一次 `notify`”。更准确地说，它发生在等待者检查谓词和进入睡眠之间存在未受保护窗口，而通知者在这个窗口里改变状态并发出通知。通知本身不保存业务事实；若等待者睡下时没有重新检查状态，也没有被登记到等待集合，就可能永远睡眠。

Listing 12.31 错误等待：检查和睡眠之间有窗口

```

consumer locks mutex
consumer sees size == 0
consumer unlocks mutex

producer locks mutex
producer pushes one element
producer unlocks mutex
producer notifies not_empty

consumer parks after notify has already happened
consumer sleeps even though size == 1

```

条件变量的 `wait(lock, predicate)` 之所以重要，是因为它把三件事组合成一个库协议：持锁检查谓词；谓词不满足时，把线程加入等待路径并释放 `mutex`；被唤醒后重新获得 `mutex`，再次检查谓词。用户代码不能把这三件事拆开。若拆开，通知和睡眠之间就会出现无主窗口。

Listing 12.32 正确等待：谓词和 `wait` 绑定

```

consumer locks mutex
while size == 0 and not closed:
    condition_variable_wait(mutex)
    ; wait returns only after re-locking mutex
consumer rechecks predicate under the same mutex

```

这条时间线也解释了虚假唤醒。线程醒来不一定表示资源已经可用，可能因为实现、信号、广播、竞争者先拿走资源或关闭路径触发。正确代码不依赖“为什么醒”，只依赖“醒来后在锁内重新检查谓词”。因此 `if` 等待是错误形状，`while` 或带谓词重载才是正确形状。

capacity-one 队列能放大所有错误 容量为一的队列是同步协议的放大镜。只有一个槽位，任何遗漏都会迅速变成可复现事故：生产者和消费者交替时能暴露丢失唤醒；两个生产者同时 `push` 时能暴露谓词错误；`close` 与 `push` 竞争时能暴露提交点不清；消费者被唤醒后另一个消费者先拿走元素时能暴露 `if` 等待；异常发生在写槽位之后、递增 `size` 之前时能暴露状态不一致。

Listing 12.33 容量一队列的故障注入序列

```

case lost_wakeup:
    consumer reaches wait boundary
    producer pushes element and notifies
    scheduler parks consumer at the wrong time

```

```

expected: consumer must not sleep forever

case close_full_queue:
    producer fills the only slot
    another producer waits on not_full
    close is called
    expected: waiting producer returns Closed, consumer may drain one element

case spurious_wakeup:
    consumer wakes while size == 0 and closed == false
    expected: consumer waits again without changing state

```

容量小不是为了模拟真实吞吐，而是为了把状态边界压缩到最少变量。若容量一队列都不能解释每个等待者为什么醒、为什么继续睡、为什么退出，容量变大只会把错误藏进概率。

超时等待不是简单返回 false 超时会改变协议。生产者等待空间超时后，是返回失败、重试、丢弃、记录背压，还是关闭上游？消费者等待数据超时后，是继续轮询、上报 idle、检查取消，还是触发心跳？超时不是条件变量的附加参数，而是状态机的一条边。尤其不能把超时当作“队列永久空”或“下游永久满”的证明，它只说明在这段时间内谓词没有满足。

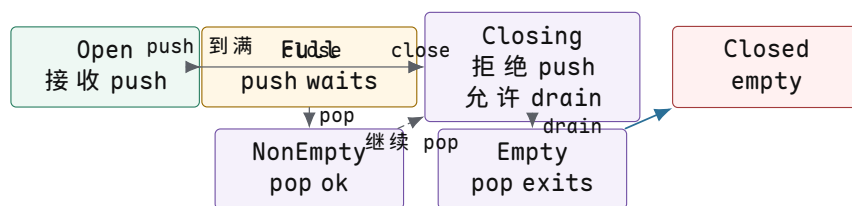
取消和关闭不是同一个状态 关闭通常表示不再接受新输入，但已接受元素可以 drain；取消表示当前工作不再需要，等待者应尽快退出，已入队元素可能被丢弃或标记失败。二者的等待谓词相似，都要唤醒等待者，但语义不同。I/O 队列可能 close 后继续写完已接收 block；用户按下取消后则应停止提交新写并尽快释放资源。把二者都塞进一个 `closed_` 布尔值，后续恢复和错误报告会变模糊。

状态	语义	等待者行为
Open	接收新元素，允许 pop	按容量和非空谓词等待
Closing	拒绝新 push，保留已入队元素	producer 退出，consumer drain
Cancelled	不再要求完成旧元素	所有等待者退出，未处理元素进入取消摘要
Closed	队列空且生命周期结束	后续操作失败或无效

12.7 有界队列：同步语义的最小系统

Compute Systems Engine 的线程池、I/O 写出队列、MessageBus、分布式模拟请求队列，都需要一个共同概念：容量有限、能背压、能关闭、能 drain 的队列。这个队列不是普通容器，而是生产者和消费者之间的同步合同。

有界队列状态机：Open、Closing、Closed



close 不是析构细节，而是状态转换。Closing 拒绝新输入，但允许已接收元素被取完；Closed 表示队列空且等待者都应退出。

图示：本图要证明的命题是：关闭协议必须区分“拒绝新输入”和“排空旧输入”。

接口先表达合同 队列接口应分层：非阻塞尝试、阻塞等待、关闭和可选 drain/timeout。非阻塞 `try_push/try_pop` 适合事件循环或自定义调度；阻塞 `push/pop` 适合普通生产者消费者；`close` 负责唤醒等待者并改变生命周期状态。哨兵值不是好方案，因为泛型元素未必有安全哨兵，且哨兵无法表达多个等待者和关闭策略。

Listing 12.34 有关闭语义的有界队列接口草图

```

1 class BoundedQueue {
2 public:
3     explicit BoundedQueue(std::int32_t capacity);
4
5     bool try_push(std::int32_t value);
6     bool push(std::int32_t value);
7     std::optional<std::int32_t> pop();
8     void close();
9
10 private:
11     std::vector<std::int32_t> buffer_;
12     std::int32_t head_ = 0;
13     std::int32_t tail_ = 0;
14     std::int32_t size_ = 0;
15     bool closed_ = false;
16     std::mutex mutex_;
17     std::condition_variable not_empty_;
18     std::condition_variable not_full_;
19 };
  
```

`push` 返回 `false`，表示关闭后不再接受新元素；`pop` 返回 `std::nullopt`，表示队列已关闭且没有剩余元素。这比在内部偷偷抛异常或让线程永久阻塞更适合作为运行时基础。业务层可以选择把这种状态转换成错误码、异常或取消，但底层合同必须明确。

push 和 pop 的状态转换 生产者进入 `push` 后，先等待“有空间或关闭”。若关闭，返回失败；若有空间，写入槽位，更新 `tail/size`，通知消费者。消费者进入 `pop` 后，先等待“有元素或关闭”。若没有元素且关闭，返回空；若有元素，取出槽位，更新 `head/size`，通知生产者。

Listing 12.35 `push` 的谓词包含关闭状态


```

1 bool BoundedQueue::push(std::int32_t value) {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_full_.wait(lock, [this]() {
4         return this->closed_ ||
5             this->size_ < static_cast<std::int32_t>(this->buffer_.size());
6     });
7     if (this->closed_) {
8         return false;
9     }
10
11     this->buffer_[static_cast<std::size_t>(this->tail_)] = value;
12     this->tail_ = (this->tail_ + 1) %
13         static_cast<std::int32_t>(this->buffer_.size());
14     ++this->size_;
15     this->not_empty_.notify_one();
16     return true;
17 }

```

Listing 12.36 close 必须唤醒所有可能等待的谓词

```

1 void BoundedQueue::close() {
2     {
3         std::lock_guard<std::mutex> lock(this->mutex_);
4         this->closed_ = true;
5     }
6     this->not_empty_.notify_all();
7     this->not_full_.notify_all();
8 }

```

close 的线性化点和幂等性 close 的线性化点是 `closed_=true`。从这一刻开始，新的 `push` 必须失败；已经进入队列的元素仍可被 `pop` 排空；空队列上的 `pop` 返回 `std::nullopt`。重复 `close` 应该是安全的，因为析构、错误路径和取消路径可能从不同层次同时尝试关闭同一个队列。重复通知通常只是性能浪费，不应破坏语义；若需要减少通知风暴，可以只在第一次关闭时通知。

Listing 12.37 只在第一次关闭时广播，语义仍保持幂等

```

1 bool BoundedQueue::mark_closed() {
2     std::lock_guard<std::mutex> lock(this->mutex_);
3     if (this->closed_) {
4         return false;
5     }
6     this->closed_ = true;
7     return true;
8 }
9
10 void BoundedQueue::close_once() {
11     const bool changed = this->mark_closed();
12     if (!changed) {
13         return;
14     }

```

```

15  this->not_empty_.notify_all();
16  this->not_full_.notify_all();
17  }

```

drain、cancel、shutdown 和析构不是同义词 很多工业事故来自把这些词混成一个布尔量。Drain 表示拒绝新输入，但处理已接收元素；cancel 表示未开始元素应尽快退出或标记取消；shutdown 表示系统进入停止流程，可能包含 close、cancel、等待 running 任务结束和释放资源；析构要求没有线程仍可能访问对象。一个队列可以支持 close + drain，却不支持取消已经弹出的任务；线程池可以 cancel 未开始任务，但运行中任务只能协作检查停止标志。

术语	状态承诺	常见错误
close	拒绝新输入，唤醒等待者	只设置 flag，不通知所有条件变量
drain	已接收元素仍会被消费	close 后直接丢弃队列内容却不记录
cancel	未开始工作被标记取消或移除	把运行中线程强行停止，破坏不变量
shutdown	停止接收、排空或取消、等待线程退出	只关队列，不等待 running 任务
destructor	没有并发访问者，资源可释放	析构里调用未知回调或在持锁时 join

异常安全和提交点 示例代码用 `std::int32_t`，元素赋值不会抛出复杂异常。泛型队列则必须考虑移动构造、拷贝、分配和析构。状态提交点要清楚：元素成功进入槽位并更新 `tail/size` 后，`push` 才对消费者可见；元素成功移出并更新 `head/size` 后，槽位才重新可用。若元素操作可能抛异常，应在不改变不变量的阶段完成可能失败的工作，或者要求元素满足 `noexcept move`，或者使用更复杂的槽位生命周期管理。

背压是正常控制流 队列满了，生产者等待或失败，这不是程序坏了，而是背压在工作。背压防止下游慢时上游无限制制造任务和占用内存。容量不是越大越好：容量过大能隐藏短期拥塞，但会放大排队延迟和内存峰值；容量过小会频繁阻塞，降低吞吐。项目报告应记录队列满次数、生产者等待时间、最大队列长度、拒绝数和关闭耗时。

从队列到线程池：unfinished count 是另一个不变量 线程池不能只看队列是否为空。Worker 从队列弹出任务后，任务已经不在队列里，但系统仍不 idle。线程池至少需要一个未完成计数或等价状态：提交任务时增加，任务函数执行结束后减少，异常路径也必须减少。等待空闲的谓词应是“队列为空且未完成计数为零”，关闭等待的谓词还要包含“所有 worker 已退出”。这就是为什么同步设计要从对象不变量出发，而不是从 API 名字出发。

Listing 12.38 任务异常也必须提交 unfinished 状态转换

```

1 void ThreadPoolState::finish_task() {
2     std::lock_guard<std::mutex> lock(this->mutex_);
3     --this->unfinished_count_;
4     if (this->unfinished_count_ == 0 && this->ready_queue_empty_) {
5         this->idle_.notify_all();
6     }
7 }

```

真实代码还要保证 `finish_task` 被 `RAII guard` 或 `try/catch` 包住，用户任务抛异常时也执行。若异常跳过未完成计数递减，`wait_idle` 会永久阻塞。若在持锁时执行用户任务，则用户任务可能递归提交任务、等待 `future`、记录日志或抛异常，把线程池状态机拖进未知代码。

12.8 信号量：资源额度，不是对象不变量

信号量表达“还有多少许可”。它适合连接池、I/O in-flight 限额、同时写文件数量、令牌桶、空槽/元素数量等资源计数。Mutex 保护复杂状态关系，semaphore 控制有多少个并发使用权。拿到许可后，线程仍可能需要 mutex 修改共享队列；释放许可时，还要保证失败和取消路径归还资源。

先分清四种 semaphore 工程里说“信号量”时，至少可能指四件不同的东西：抽象计数信号量、二值信号量、POSIX `sem_t` 或 C++20 `std::counting_semaphore`。它们共享“许可计数”这个核心模型，但生命周期、进程边界、API 错误处理、取消/信号语义和调试方式不同。把这四者混在一起，会让读者误以为“会 `acquire/release` 就掌握了 semaphore”。

类型	核心状态	典型接口	适合场景
计数信号量	<code>permits >= 0</code>	<code>acquire/release(n)</code>	连接池、I/O in-flight、容量预算
二值信号量	<code>permits</code> 为 0 或 1	<code>acquire/release</code>	一次事件许可、单个后台阶段完成
POSIX semaphore	<code>sem_t</code> 或 <code>named semaphore</code>	<code>sem_wait/sem_post</code>	C/POSIX 代码、跨进程共享许可
C++20 semaphore	模板化标准库对象	<code>acquire/try_acquire/release</code>	C++ 线程内资源预算和 RAII 封装

这张表不是 API 速查，而是状态边界。计数信号量和二值信号量是语义模型；POSIX 和 C++20 是具体接口族。具体接口不能改变底层事实：semaphore 只保存许可，不保存业务对象、不保存关闭原因、不保存 payload，也不证明等待者的生命周期。

计数信号量的形式化状态 计数信号量的核心状态可以抽象成一个非负整数 `permits` 和一组等待者。构造时给定初始许可数；`acquire` 尝试消耗一个许可；`release` 增加一个或多个许可，并使等待者有机会继续。这个整数不是业务对象数量，而是预算。连接池里许可数可以等于“最多允许借出多少连接”，但真实连接对象还在 `free list`、`closing list` 或 `broken list` 中；I/O 系统里许可数可以等于“最多允许多少个请求同时提交”，但请求 `buffer`、`completion record` 和错误状态仍在其他结构里。

Listing 12.39 counting semaphore 的最小状态机

```
state:
    permits >= 0
    waiters = threads blocked because permits == 0

acquire:
    if permits > 0:
        permits -= 1
        return acquired
    else:
        block until a release may make permits > 0
        retry acquisition
```

```

try_acquire:
    if permits > 0:
        permits -= 1
        return true
    else:
        return false

release(n):
    permits += n
    wake up to n waiters, or make permits visible to future acquirers

```

这份状态机刻意没有 `closed`、`cancelled`、`owner`、`payload`、`freelist`。这些事不属于信号量本身。Semaphore 的强项正是窄：它只回答“预算还有没有”。它的弱点也正是窄：只靠它不能证明对象生命周期、关闭语义和业务错误路径。

acquire 和 release 的线性化点 线性化点是并发操作在抽象历史中“真正生效”的瞬间。对计数信号量来说，成功 `acquire` 的线性化点是许可计数被消耗；`release` 的线性化点是许可被归还并对其他线程可见。等待线程从睡眠中醒来不是线性化点，因为醒来后它还可能输给另一个线程，或发现关闭状态要求退出。正确代码必须把“拿到许可”和“拿到业务对象”分成两个提交点。

Listing 12.40 许可提交点和对象提交点不能混在一起

```

borrow connection:
    acquire permit                # permit commit point
    lock pool mutex
    if pool is closing:
        unlock
        release permit            # undo the budget claim
        return closed
    remove object from free list   # object ownership commit point
    unlock
    return connection handle

```

这段账本能解释两个常见 bug。第一，拿到许可后发现系统关闭，必须释放许可；否则许可泄漏，所有后来者最终睡眠。第二，从 free list 取出对象失败时，不能假装许可就是对象；许可只是预算，真正的对象状态仍要在 mutex 下检查。很多 semaphore 事故不是 `acquire/release` API 用错，而是没有分清两个线性化点。

计数信号量：资源预算的正确用法 计数信号量最自然的用法是把外部资源转成一个明确预算。例如最多 64 个请求进入压缩阶段，最多 8 个文件同时执行 `fsync`，最多 16 个连接被借出。预算耗尽时，调用方要么等待，要么超时，要么拒绝请求。这个选择由业务 SLO 决定，不由 semaphore 自动决定。

Listing 12.41 计数信号量的资源预算账本

```

resource budget:
    capacity = 64
    permits = 64 - in_flight

request enters expensive stage:
    if try_acquire succeeds:
        in_flight += 1 logically
        run stage

```

```

    release exactly once
else:
    wait, timeout, reject, or shed load according to SLO

```

计数信号量的核心审查问题是“许可守恒”。如果系统没有关闭、取消和超时，长期运行后应满足：

Listing 12.42 许可守恒检查

```

initial_permits + release_count - acquire_success_count == current_permits
logical_in_flight == acquire_success_count - release_count
0 <= logical_in_flight <= capacity

```

真实实现不一定能直接读取准确的 `current_permits`，但可以在业务层维护调试计数。若 `logical_in_flight` 长期低于容量而请求仍卡在 `acquire`，优先怀疑许可泄漏、等待者被旧对象挂住、或异常路径没有归还许可。

二值信号量：一次许可，不是 mutex `std::binary_semaphore` 可以看成最大计数为 1 的计数信号量。它适合表达“一个事件许可”或“最多一个后台阶段运行”。例如主线程启动后台 `warmup`，后台线程完成后 `release`，主线程 `acquire` 等待一次完成。这种场景中，`release` 的线程和 `acquire` 的线程本来就不是同一个线程，二值信号量的无所有者语义正好合适。

Listing 12.43 `binary_semaphore` 适合一次完成信号

```

initial:
    ready = binary_semaphore(0)

background:
    build immutable table
    publish table pointer
    ready.release()

foreground:
    ready.acquire()
    read published table

```

反过来，用二值信号量保护 `vector`、`map`、队列或线程池状态，通常是坏设计。Mutex 的关键语义是所有者和临界区：同一线程 `lock`，完成受保护状态转换，再 `unlock`。二值信号量没有 `owner`，不阻止另一个线程 `release`；工具也更难判断谁持有临界区。若代码需要保护多字段不变量，默认用 `mutex`；若代码只需要一次事件许可，二值信号量才合适。

Listing 12.44 `binary_semaphore` 和 `mutex` 的边界

```

binary semaphore:
    no owner
    release may happen on a different thread
    good for event permit or single in-flight budget

mutex:
    owner enters and leaves critical section
    protects object invariant while fields temporarily disagree
    better for containers, queues, pools, and runtime state

```


POSIX semaphore: C 接口、进程边界和信号中断 POSIX semaphore 是 C/POSIX 世界里的具体接口族，常见对象是 `sem_t`。未命名 semaphore 通常由 `sem_init` 初始化，由 `sem_destroy` 销毁；命名 semaphore 通过 `sem_open` 获取，通过 `sem_close` 关闭句柄，通过 `sem_unlink` 删除名字。它比 C++20 semaphore 更靠近操作系统边界，也因此有更多系统级错误条件。

Listing 12.45 POSIX unnamed semaphore 的基本形态

```
sem_t sem;

sem_init(&sem, pshared, initial_value);

producer or releaser:
    sem_post(&sem);

consumer or acquirer:
    while sem_wait(&sem) == -1:
        if errno == EINTR:
            continue or handle cancellation policy
        else:
            report fatal synchronization error

shutdown:
    prove no waiter can still use sem
    sem_destroy(&sem);
```

`pshared` 为 0 时，semaphore 用于同一进程内线程；非 0 时，semaphore 必须放在可被多个进程共享的内存中，用于进程间同步。这个边界很重要：把进程共享 semaphore 放在普通进程私有堆内存里，不能让另一个进程看到同一个对象。命名 semaphore 则通过系统名字定位，适合不方便共享内存中放置 `sem_t` 的场景，但也引入名字清理、权限和崩溃后残留的问题。

POSIX `sem_wait` 可能因为 signal 返回 -1 并设置 `errno=EINTR`。这与 C++20 `acquire` 不同；C++20 接口不会把 EINTR 暴露给业务。POSIX 代码必须显式决定：收到 signal 后继续等待、返回取消、还是进入关闭路径。另一个常见错误是用 `sem_getvalue` 做并发决策。它最多适合诊断，不适合作为“如果值大于 0 就一定能继续”的正确性依据，因为读取后状态可能立刻被其他线程改变。

Listing 12.46 POSIX semaphore 的工程边界

```
must handle:
    sem_wait may return EINTR
    sem_trywait may return EAGAIN when no permit exists
    sem_timedwait needs a timeout policy
    sem_destroy requires no thread can still wait or post
    process-shared sem_t must live in shared memory
    named semaphore needs close/unlink lifecycle
```

C++20 counting_semaphore: 标准库接口和限制 `std::counting_semaphore<LeastMaxValue>` 是 C++20 对计数信号量的标准化封装。`LeastMaxValue` 是实现必须至少支持的最大计数下界，不应被误读成“这个对象运行时一定只允许这么多业务资源”。构造函数给出初始计数；`release(update)` 增加许可；`acquire` 阻

塞直到成功消耗许可；`try_acquire` 尝试获取；`try_acquire_for` 和 `try_acquire_until` 提供超时等待。`std::binary_semaphore` 是 `std::counting_semaphore<1>` 的别名。

Listing 12.47 C++20 `counting_semaphore` 的最小接口形状

```

1 class InFlightLimit {
2 public:
3     explicit InFlightLimit(std::ptrdiff_t initial)
4         : permits_(initial) {}
5
6     bool try_enter() {
7         return this->permits_.try_acquire();
8     }
9
10    void leave() {
11        this->permits_.release();
12    }
13
14 private:
15     std::counting_semaphore<1024> permits_;
16 };

```

C++20 semaphore 对象不可复制，通常也不应被随意移动进容器；它的地址和生命周期应稳定。标准没有给它业务关闭语义，也不承诺严格 FIFO 公平。若要支持取消、关闭、超时错误码和 drain，必须在外层状态机实现。还要注意 `try_acquire` 允许失败，业务代码不能把一次失败解释成永久无资源；带超时接口也必须记录 timeout 是正常背压、用户取消还是系统关闭。

Listing 12.48 C++20 semaphore 审查清单

```

C++20 counting_semaphore checklist:
    initial count matches resource budget
    every successful acquire has exactly one release
    timeout path records whether work was accepted
    close/cancel state is stored outside the semaphore
    object outlives all waiting and releasing threads
    code does not depend on FIFO fairness
    try_acquire failure is not treated as a correctness proof

```

从内存可见性看，semaphore 也不能替代对象协议。一个线程在 `release` 前写入某个对象，另一个线程 `acquire` 后读取该对象，通常可以建立生产者到消费者的同步边界；但对象本身的所有权、生命周期和别名仍要由外层合同证明。不要因为用了 semaphore，就在没有互斥或单所有权协议的情况下并发修改同一个容器。

C++20 中的共同边界：没有 close，也没有 ownership `std::counting_semaphore` 和 `std::binary_semaphore` 都不绑定线程所有者：一个线程可以 `acquire`，另一个线程可以 `release`。这与 mutex 完全不同。Mutex 表达的是“这个线程进入临界区并负责退出”；semaphore 表达的是“系统预算增加或减少一份”。因此用 binary semaphore 当 mutex，通常会丢掉所有权、调试和不变量语义。

能力	semaphore 提供吗	工程后果
计数许可	提供	适合容量、in-flight 和令牌桶
线程所有权	不提供	不能替代 mutex 的 owner 语义
复杂谓词	不提供	close/drain/cancel 仍要额外状态
业务 payload	不提供	数据必须在队列、pool 或 record 中
公平性保证	标准不承诺强公平	不能把它当严格 FIFO 调度器
对象生命周期证明	不提供	等待者退出前不能销毁关联状态

公平性尤其容易被误解。实现可能让等待者大体按某种队列醒来，也可能允许正在运行的线程在释放后很快再次获取许可。标准层面不应依赖“最早等待者一定最早获得许可”。若业务需要严格顺序，应在受 mutex 保护的队列或调度器里表达顺序，而不是把公平性寄托给 semaphore。

许可守卫：异常路径必须自动归还 凡是 `acquire` 后有多条返回路径，就应把许可归还写进 RAII。否则异常、取消、早返回和错误分支会慢慢吞掉许可。许可泄漏不像崩溃那样明显：系统会先变慢，然后并发度越来越低，最后所有线程阻塞在 `acquire`，看起来像下游服务挂住。

Listing 12.49 许可守卫把 `release` 绑定到作用域

```

1 class PermitGuard {
2 public:
3     explicit PermitGuard(std::counting_semaphore<1024>& permits)
4         : permits_(&permits), owns_(true) {
5         this->permits_->acquire();
6     }
7
8     PermitGuard(const PermitGuard&) = delete;
9     PermitGuard& operator=(const PermitGuard&) = delete;
10
11     PermitGuard(PermitGuard&& other) noexcept
12         : permits_(std::exchange(other.permits_, nullptr)),
13         owns_(std::exchange(other.owns_, false)) {}
14
15     ~PermitGuard() {
16         if (this->owns_) {
17             this->permits_->release();
18         }
19     }
20
21     void dismiss() noexcept {
22         this->owns_ = false;
23     }
24
25 private:
26     std::counting_semaphore<1024>* permits_;
27     bool owns_;
28 };

```

`dismiss` 只能在许可所有权已经转移到另一个对象时调用，例如连接 handle 接管归还责任。不要为了“避免析构 `release`”随手调用 `dismiss`。这类 guard 的代码审查问题很明确：每个 `acquire` 是否有唯一归还路径；所有权转移后谁负责

release；失败路径是否归还；对象析构是否可能在等待者仍存在时发生。

Listing 12.50 许可泄漏的证据形状

```
symptom:
    request throughput gradually decreases
    active in-flight count is below configured limit
    many threads blocked in semaphore acquire
    no downstream saturation is visible

likely cause:
    acquired permits - released permits keeps growing
    early return, exception, timeout, or cancellation skipped release
```

双信号量 bounded buffer 的严格账本 用两个信号量写有界队列时，`empty_slots` 代表可写槽预算，`filled_slots` 代表可读元素预算。它的好处是容量和元素数量的等待对象很清楚；代价是关闭语义更难，因为标准 semaphore 不知道 `closed`。先看不含关闭的正常路径：

Listing 12.51 双信号量 bounded buffer 的正常路径

```
initial:
    empty_slots = capacity
    filled_slots = 0
    head, tail, buffer protected by mutex

push:
    empty_slots.acquire()
    lock mutex
    write buffer[tail]
    tail = next(tail)
    unlock mutex
    filled_slots.release()

pop:
    filled_slots.acquire()
    lock mutex
    read buffer[head]
    head = next(head)
    unlock mutex
    empty_slots.release()
```

这条账本里有三个必须保持的关系。第一，`empty_slots+filled_slots==capacity`，除非某个线程已经拿到许可但尚未完成锁内提交；这段短窗口必须被代码审查标出来。第二，`filled_slots` 的 `release` 必须发生在元素写入并提交之后，否则消费者可能拿到“可读许可”却读到未初始化槽。第三，`empty_slots` 的 `release` 必须发生在元素读出并槽位重新可写之后，否则生产者可能覆盖尚未被安全移出的元素。

关闭语义：semaphore 不能广播一个不存在的许可 条件变量可以在 `closed=true` 后 `notify_all`，等待者醒来检查 `size>0||closed`。Semaphore 没有“状态变化但不增加资源”的通知语义。若消费者睡在 `filled_slots.acquire()`，队列关闭且没有元素时，应该让它退出；但调用 `filled_slots.release()` 会制造一个看起

来真实的元素许可。生产者睡在 `empty_slots.acquire()` 时也有同样问题：关闭后唤醒它们，不等于真的多了空槽。

Listing 12.52 semaphore 队列关闭时最容易写错的地方

```
bad close:
    closed = true
    filled_slots.release(number_of_consumers)

why bad:
    fake filled permits look like real elements
    consumers must add another check after acquire
    permit accounting becomes tied to waiter count
    a late consumer may consume a fake permit after close logic changes
```

这不是说 semaphore 不能用于队列，而是说关闭协议要显式设计。常见选择有三种。第一，队列核心仍用 `mutex+condition_variable`，把 semaphore 只用于外层 in-flight 限额。第二，使用支持停止令牌或超时的获取路径，等待者周期性检查 `closed`，但要接受超时轮询成本和复杂性。第三，向队列插入结束哨兵，但这要求元素类型和消费者数量能承载该协议，且必须证明哨兵不会与业务数据混淆。对通用线程池队列，默认选择仍应是 CV，因为 `close/drain/cancel` 是业务谓词，不是资源额度。

二值信号量、mutex 和 latch 不要混用 二值信号量、mutex 和 latch 都可能让线程“等一下”，但等待对象不同。二值信号量等一个可重复的一份许可；mutex 等对象临界区；latch 等一次性倒计时归零。若要等 N 个 worker 初始化完成，用 latch；若要限制最多一个后台 flush in-flight，用 binary semaphore 或普通 mutex，取决于是否需要 owner 和对象不变量；若要保护 `head/tail/size/closed`，用 mutex。

令牌桶和背压：semaphore 是容量控制，不是排队策略 在服务端，semaphore 常用于 admission control：同时最多允许 N 个请求进入某个昂贵阶段。拿不到许可时，可以等待、拒绝、降级或排队。选择哪一种不是 semaphore 决定的，而是 SLO 决定的。等待会保护下游但增加队列延迟；直接拒绝能保护尾延迟但牺牲成功率；按优先级排队需要额外调度结构。

Listing 12.53 用 semaphore 表达 in-flight 预算

```
request arrives:
    if permits.try_acquire():
        enter expensive stage
    else if latency_slo_is_strict:
        reject or shed load
    else:
        wait with timeout and record queue_wait_ns

request finishes:
    release permit exactly once
```

因此 semaphore 相关指标至少要包括 `permit_acquire_count`、`permit_wait_ns`、`permit_timeout_count`、`permit_reject_count`、`permit_leak_check` 和 `in_flight_high_watermark`。若只记录吞吐，无法区分“下游慢导致等待增加”和“许可泄漏导致并发度下降”。

令牌和队列容量不是一回事 令牌可以限制并发 I/O 数量，但不等于队列容

量。一个系统可能允许队列里积压 100 个待写 block，但同时只允许 4 个 fsync in-flight；也可能允许 32 个 RPC in-flight，但请求队列容量按内存字节控制。把所有限制都写成一个队列长度，会混淆内存预算、设备并发度和业务优先级。第 15 章的异步 I/O 会复用这个区分。

12.9 一次性完成、阶段同步和一次性结果

Mutex/CV 和 semaphore 解决的是对象状态与资源额度；另一类常见等待对象是“某个阶段到了没有”。这里要分三种：latch 表达一次性归零，barrier 表达多轮固定参与者阶段，future/promise 表达一次性值或异常。三者都常被误写成“等一下”，但状态模型完全不同。

latch：一次性倒计时 `std::latch` 表达的是一次性完成门闩。它内部有一个计数；参与者完成某个一次性阶段后调用 `count_down`；等待者调用 `wait`，直到计数归零后通过。归零之后，latch 永远保持打开，不能重置到下一轮。这一点决定了它适合“等待所有 worker 初始化完成”“等待 N 个分片加载完 metadata”“等待测试线程同时准备好后开始压测”这类一次性边界。

Listing 12.54 latch 的状态机

```
state:
    count = N
    opened = false

count_down:
    if count > 0:
        count -= 1
    if count == 0:
        opened = true
        wake all waiters

wait:
    if opened:
        return immediately
    else:
        block until opened
```

标准接口要按状态机读。`count_down(n)` 表示有 `n` 个一次性参与份额完成；`try_wait()` 只观察是否已经打开；`wait()` 等待打开；`arrive_and_wait(n)` 等价于先贡献 `n` 个倒计时再等待打开。所有这些操作都围绕同一个不可重置计数工作。构造时给的计数就是本次门闩的全部债务，后续不能补参与者，也不能进入第二轮。

接口	语义	审查问题
<code>count_down(n)</code>	提交 <code>n</code> 个一次性完成份额	所有异常和早退路径是否都会执行
<code>try_wait()</code>	非阻塞观察计数是否归零	不能用来轮询可重复事件
<code>wait()</code>	等待计数归零	等待者是否有失败状态可读
<code>arrive_and_wait(n)</code>	倒计时后等待同一个 latch 打开	调用线程是否本身也是参与者

Latch 的关键不是互斥，也不是资源许可，而是“一次性归零”。它不保护对象不变量；它不保存每个参与者的结果；它不处理某个参与者失败后如何取消其他参与者。若 worker 初始化可能失败，外层仍要有错误状态，例如 `init_error`、错误队

列或异常聚合。Latch 只保证所有成功或失败路径都调用了倒计时时，等待者不会早于计数归零继续。

Listing 12.55 latch 等待所有 worker 完成初始化

```
1 std::latch initialized(worker_count);
2 std::atomic<bool> failed{false};
3
4 worker:
5     try {
6         initialize_local_state();
7     } catch (...) {
8         failed.store(true, std::memory_order_release);
9     }
10    initialized.count_down();
11
12 controller:
13    initialized.wait();
14    if (failed.load(std::memory_order_acquire)) {
15        enter_startup_failure_path();
16    }
```

常见错误是把 latch 当 condition variable 使用。队列从空到非空会发生很多次，线程池 idle 也会在 busy 和 idle 之间反复切换；latch 一旦打开就不会再次关闭，无法表达这些可重复谓词。另一个错误是漏掉失败路径的 `count_down`。如果某个 worker 初始化抛异常后直接退出，控制线程会永久等待。可靠代码应使用 scope guard 或统一的 worker 入口，保证所有路径都会倒计时。

Listing 12.56 latch 最常见的异常路径事故

```
bad_worker:
    initialize_part_a()
    if part_b_failed:
        return          # count_down is skipped
    initialized.count_down()

robust_worker:
    on_scope_exit initialized.count_down()
    try:
        initialize_part_a()
        initialize_part_b()
    catch:
        publish init_error
```

Latch 的错误用法通常有三个信号。第一，代码想把 latch “关回去”，说明等待对象不是一次性完成，而是可重复阶段或谓词。第二，等待者需要知道每个 worker 的结果，却只看到 latch 打开，说明结果通道缺失。第三，参与者数量来自动态队列长度，运行过程中还会变化，说明应该建任务计数、future 聚合或 barrier drop 协议，而不是在 latch 上猜测。

barrier：多轮阶段同步 `std::barrier` 表达的是多轮阶段边界。它有参与者数量和当前阶段。每个参与者完成本阶段工作后调用 `arrive_and_wait`；最后一个到达者触发阶段完成函数，然后所有参与者进入下一阶段。与 latch 不同，barrier

会重复使用，每轮结束后进入下一代 phase。

Listing 12.57 barrier 的多轮状态机

```
state:
    expected = participant_count
    arrived = 0
    phase = 0

arrive_and_wait:
    local_phase = phase
    arrived += 1
    if arrived == expected:
        run completion step for local_phase
        arrived = 0
        phase += 1
        wake all waiters for local_phase
    else:
        wait until phase != local_phase
```

Barrier 的接口比 `arrive_and_wait` 更细。`arrive()` 表示“我对当前 phase 的贡献已经完成”，并返回一个 arrival token；调用者可以在不再读写本 phase 共享边界的前提下做一点独立工作，随后把 token 交给 `wait`。这个 token 不是资源许可，而是当前 phase 的等待凭证。调用 `arrive()` 之后还继续修改本 phase 会制造 phase skew：其他线程可能已经触发完成函数并进入下一轮。

接口	语义	审查问题
<code>arrive()</code>	到达当前 phase，返回等待 token	到达后是否还会触碰本 phase 数据
<code>wait(token)</code>	等待 token 对应 phase 完成	token 是否只被等待一次
<code>arrive_and_wait()</code>	到达并等待 phase 推进	每个继续参与者每轮是否恰好一次
<code>arrive_and_drop()</code>	到达本轮并永久退出后续 phase	退出是生命周期事件还是异常漏调用
completion function	最后到达者执行阶段完成动作	是否短、确定、不阻塞、不抛异常

Barrier 适合 stencil、仿真、多阶段流水和分片批处理：所有分片先完成本轮计算，再统一交换边界或提交下一轮。它不适合动态任务队列，因为参与者数量和每轮到达次数必须被协议定义。若某个参与者提前退出，其他参与者可能永远卡在当前 phase；若一个线程在同一 phase 到达两次，状态机也会被破坏。C++20 `barrier` 支持 `arrive_and_drop`，允许参与者永久退出后减少后续 phase 的 expected count，但这必须是显式生命周期事件，不是异常路径随手漏掉一次等待。

Listing 12.58 barrier 表达多轮分片阶段

```
1 std::barrier step_done(worker_count, []) {
2     publish_phase_summary();
3 };
4
5 worker:
6     for (std::size_t phase = 0; phase != phase_count; ++phase) {
7         compute_local_partition(phase);
```

```
8     step_done.arrive_and_wait();
9     consume_published_summary(phase);
10 }
```

完成函数不是任意回调仓库。它运行在阶段切换的关键路径上，应该短、确定、不能阻塞 I/O，不能提交新一轮需要同一个 barrier 的工作，也不能抛出让参与者状态不一致的异常。若完成函数需要聚合大量数据，应让每个 worker 在到达前写入本地结果，完成函数只做轻量发布或版本推进。

Listing 12.59 barrier 阶段同步的可审查账本

```
phase p, worker i:
    read input snapshot for phase p
    compute local partition
    write local boundary and local metrics for phase p
    arrive at barrier exactly once

completion for phase p:
    publish summary for phase p
    advance global phase to p + 1

after release:
    workers may read summary for phase p
    workers start phase p + 1
```

阶段同步最危险的不是 API 调错，而是 phase 语义没有写成合同。若 completion function 里等待网络、写日志、提交任务或拿另一个可能被 worker 持有的锁，所有参与者都会卡在阶段边界。若一个 worker 因过滤条件本轮没有工作却直接跳过 barrier，其他 worker 会永久等待。若参与者数量随任务队列变化，就要用任务图、计数器或 continuation 表达动态依赖；barrier 适合固定参与者的重复阶段，不适合“谁有任务谁来一下”的队列。

错误用法	失败原因	正确方向
用 barrier 管动态任务队列	每轮 expected count 不稳定	任务队列加 unfinished 计数或 task graph
worker 无工作就跳过本轮	其他参与者等不到它	无工作也到达，或显式 arrive_and_drop
arrive() 后继续写本轮数据	completion 可能已经发布 summary	到达前完成本 phase 所有写入
completion function 阻塞 I/O	所有参与者卡在阶段边界	completion 只做轻量状态推进
异常路径绕过到达	phase 永久无法完成	统一 worker 入口或显式 drop 协议

future/promise: 一次性结果和异常通道 std::future 和 std::promise 不是队列、锁或 barrier，而是一份一次性 shared state 的两个端口。promise 是生产端，负责把一个值或异常写入 shared state；future 是消费端，负责等待并取出结果。这个原语最适合“后台加载完成后把结果交给控制线程”“一次异步请求返回一个结果”“任务异常要跨线程传播”这类一对一结果通道。

Listing 12.60 future/promise 的一次性共享状态机

```
state:
    Empty(producer alive)
```

```

ValueReady(value)
ExceptionReady(exception)
BrokenPromise
Consumed                # std::future::get has taken the result

promise.set_value(v):
    Empty -> ValueReady(v), wake waiters

promise.set_exception(e):
    Empty -> ExceptionReady(e), wake waiters

promise destroyed while Empty:
    Empty -> BrokenPromise, wake waiters

future.get():
    wait until ready
    if ValueReady: return value and consume state
    if ExceptionReady: rethrow and consume state
    if BrokenPromise: throw future_error

```

这里的“ready”只表示结果槽位已被填充，不表示某个队列关闭、不表示所有任务完成、不表示运行时 idle。`std::future::get` 对普通 `future` 只能调用一次，因为它消费结果；多个观察者需要 `std::shared_future`。生产端也只能满足一次：重复 `set_value` 或先 `set_value` 后 `set_exception` 都是协议错误，标准库会以 `future_error` 报告。

shared state 不是 future 对象本身 `std::future` 和 `std::promise` 都只是访问同一份 shared state 的句柄。Shared state 才保存 ready 标志、结果存储、异常对象、等待队列、引用关系和同步边界。`promise.get_future()` 只是在这份状态上创建消费端；`future` 移动之后，旧 `future` 变成 invalid，shared state 仍然存在；`promise` 移动之后，生产端责任也随之移动。把 `future` 对象本身当成结果槽，会导致生命周期判断错误。

Listing 12.61 future shared state 的概念字段

```

FutureSharedState<T>:
    producer_present: bool
    consumer_present: bool
    ready: bool
    result_storage: optional<T>
    exception_storage: exception_ptr
    waiters: condition_variable/futex waiters
    continuations: callbacks or coroutine handles
    mutex_or_atomic_protocol: publishes ready and result

std::promise<T>:
    owns producer endpoint of the shared state

std::future<T>:
    owns single-consumer endpoint of the shared state

std::shared_future<T>:
    shares read-only access after ready

```


这解释了几个标准库现象。`future.valid()` 只说明这个句柄是否还关联 shared state, 不说明结果是否 ready。`wait()` 只等待 ready, 不消费结果;`get()` 等待 ready 后消费普通 future 的结果;`shared_future::get()` 可以被多次调用, 因为它不把共享状态从消费者侧移走。Shared state 的生命周期通常持续到最后一个相关句柄释放, 而不是到任务函数返回那一刻。

端点生命周期: `promise` 和 `future` 不是两个对象互相等 把 shared state 看成中心对象之后, `promise` 和 `future` 就只是两个端点。生产端负责把 Pending 推到某个终态; 消费端负责观察终态并取出值或异常。端点可以移动, shared state 不会因为句柄移动而移动。端点可以提前析构, shared state 也不一定立刻销毁; 它必须活到所有可能观察它的句柄和等待者都结束。

Listing 12.62 future shared state 的端点生命周期

```
create promise:
    allocate shared state
    producer endpoint is present

promise.get_future:
    create exactly one ordinary future endpoint
    repeated get_future is a protocol error

move promise:
    producer endpoint moves to the new promise object
    old promise no longer owns producer responsibility

move future:
    consumer endpoint moves to the new future object
    old future.valid() becomes false

destroy promise while Pending:
    Pending -> BrokenPromise
    wake waiters

destroy future before ready:
    consumer endpoint disappears
    producer may still run unless runtime has a cancellation contract

destroy last endpoint after terminal state:
    shared state storage may be reclaimed
```

这几个转移决定了很多工程边界。第一, `promise.get_future()` 通常只能调用一次, 因为普通 `future` 是单消费端; 要广播结果, 应先转成 `shared_future` 或用发布快照。第二, consumer 提前析构不等于 producer 被取消; 标准 future 没有取消语义。第三, producer 析构且还没满足结果, 不是静默丢弃, 而是 broken promise。运行时若把 rejected task、canceled task、abandoned future 都混成一个布尔值, 就无法给调用者正确错误。

标准操作表：每个 API 都是状态转移 下面这张表不是背 API, 而是给代码审查用的状态账本。任何线程池封装 `std::promise`、`std::packaged_task` 或自研 future-like 类型, 都应逐项回答这些操作在 Pending、Ready、Canceled、Broken、Consumed 下如何表现。

操作	状态影响	工程边界
<code>promise::get_future</code>	创建普通消费端	只能一次；重复调用是协议错误
<code>promise::set_value</code>	Pending -> ValueReady	必须发布结果后再唤醒等待者
<code>promise::set_exception</code>	Pending -> ExceptionReady	异常对象进入 shared state，不停在 worker 栈
<code>promise</code> 析构	Pending -> BrokenPromise	等待者必须醒来并得到可诊断错误
<code>future::wait</code>	只等待 ready	不消费结果，不执行任务，不取消 producer
<code>future::wait_for</code>	在期限内尝试观察 ready	timeout 不表示任务结束或取消
<code>future::get</code> <code>future::valid</code>	观察终态并消费普通 future 查询句柄是否有关联状态	第二次 <code>get</code> 不是合法读取 不等于 ready，不等于任务仍在运行
<code>shared_future::get</code>	多观察者读取 ready 状态	适合广播结果，不适合事件流

最容易错的是把 `wait_for(timeout)` 的 `timeout` 当成取消成功。Timeout 只是调用者本轮停止等待。producer 可能还在执行，结果稍后仍然会进入 shared state。若 timeout 后上层释放了 producer 需要写入的 buffer，后续 completion 就会写悬垂内存。正确做法是把 timeout、stop request、producer acknowledgement 和 shared state 终态分成四个事实，不要让一个返回值承担所有语义。

Listing 12.63 timeout 不等于取消完成

```

caller:
    if future.wait_for(10 ms) == timeout:
        request_stop()
        keep shared state and captured storage alive
        decide whether to wait, detach, or observe cancellation later

producer:
    if stop requested:
        publish Canceled or ExceptionReady
    else:
        publish ValueReady or ExceptionReady

```

异常传播是 shared state 的终态 跨线程异常不能靠 worker 日志完成。生产端必须把异常捕获成 `std::exception_ptr` 写入 shared state；消费端 `get()` 看到异常终态时在调用线程重新抛出。这样调用者能用普通 `try/catch` 处理后台任务失败，错误也不会被 worker 线程吞掉。

Listing 12.64 异常必须进入 promise，而不是停在 worker 日志

```

1 std::promise<LoadReport> promise;
2 std::future<LoadReport> future = promise.get_future();
3
4 worker:
5     try {
6         promise.set_value(load_metadata());
7     } catch (...) {

```

```

8     promise.set_exception(std::current_exception());
9 }
10
11 caller:
12 try {
13     LoadReport report = future.get();
14     use(report);
15 } catch (const std::exception& e) {
16     handle_background_failure(e);
17 }

```

若 producer 在 shared state 仍为 Empty 时析构，consumer 不应永远等待；标准库会让 consumer 观察到 broken promise，并在 `get()` 时抛出 `std::future_error`。这不是小角落，而是生命周期协议：每个被接纳的异步任务都必须以 value、exception、canceled 或 broken promise 之一结束。若运行时取消任务，也要把取消转成 shared state 中可观察的终态，而不是只从 ready queue 删除任务。

阻塞等待的语义边界 `wait()`、`wait_for()`、`wait_until()` 和 `get()` 都围绕同一件事：等待 shared state 进入 ready 终态。它们不保证哪个线程执行生产者，不保证队列被排空，不保证运行时仍有 worker 可用，也不保证超时后任务已经取消。超时只说明本次等待期限内没有观察到 ready；任务可能稍后成功、失败或取消。

Listing 12.65 future 等待不能被误读成调度保证

```

future.wait:
    blocks current thread until shared_state.ready may be true
    does not execute queued tasks
    does not create worker threads
    does not cancel the producer on timeout

future.get:
    waits until ready
    returns value, rethrows exception, or reports broken promise
    consumes ordinary future result

wait_for timeout:
    means caller stopped waiting for now
    producer may still be running
    shared state must remain valid

```

因此，持有 mutex 时调用 `get()` 是危险形状：producer 可能需要同一把锁才能写结果或推进状态，consumer 却在锁内睡眠。在线程池 worker 内调用同池 future 的 `get()` 也危险：等待者占住了执行被等待任务的 worker。外部控制线程阻塞等待一次后台结果通常可以接受；compute worker 内长期阻塞等待必须被运行时显式管理。

阻塞等待的真实时间线 阻塞等待不是一个魔法暂停点。普通 mutex/CV 实现通常可以按下面的时间线理解；futex 或平台事件对象只是把 sleep/wake 的慢路径换成内核设施，语义仍然是“状态先改变，通知后观察”。

Listing 12.66 future.get 的 park/unpark 时间线

```

consumer future.get:

```

```

lock shared_state
if state is terminal:
    consume or rethrow
    unlock
    return
register waiter
while state is Pending:
    atomically release lock and park current thread
    wake up and reacquire lock
remove waiter
consume or rethrow terminal state
unlock

producer set_value/set_exception:
    lock shared_state
    write result or exception storage
    state = ValueReady or ExceptionReady
    detach waiters and continuations
    unlock
    notify waiters
    schedule continuations

```

这条线有三个审查点。第一，consumer 必须在检查 Pending 和进入等待队列之间保持原子性；否则 producer 可能已经通知，而 consumer 还没登记，形成丢失唤醒。第二，producer 必须先写结果、再改 ready、再通知；否则 consumer 可能醒来读到未构造完成的结果。第三，通知和调度 continuation 应在释放 shared state 锁之后发生，避免在内部锁下执行未知代码。

错误时间线	表现	应检查的证据
先通知后写 ready	偶发等待者继续睡眠或读旧状态	状态写入与 notify 的顺序
锁内执行 continuation	长临界区、锁顺序反转、重入运行时	completion trace 和锁等待时间
timeout 后 释 放 captured storage	producer 稍后写悬垂对象	捕获对象生命周期和 stop ack
worker 内 park 等同池 future	ready 队列有任务但无 worker 执行	worker state trace 与 ready depth

线程池集成：submit 返回 future 不是自动安全 线程池常见接口是 `submit(callable)->future<R>`。这个接口只在语义上承诺“一次被接纳的 callable 会满足 shared state”。它没有自动解决容量、关闭、取消、同池等待和异常传播。正确集成至少要保证四件事：提交被拒绝时不给调用者一个永远不 ready 的 future；任务函数正常返回时写 value；任务函数抛异常时写 exception；运行时取消已接纳任务时写取消终态或异常。

Listing 12.67 submit 返回 future 的最低合同

```

submit:
    if runtime is not Open:
        return Rejected, no pending future
    create shared state
    enqueue task wrapper that owns producer endpoint
    return future endpoint to caller

```

```

task wrapper:
    try:
        result = callable()
        shared_state.set_value(result)
    catch:
        shared_state.set_exception(current_exception)

cancel accepted task before running:
    shared_state.set_exception(task_canceled)
    decrement unfinished count
    wake waiters

```

`std::packaged_task` 可以把 callable 和 future shared state 包在一起，适合写最小线程池。但它仍然只是“callable 执行一次后满足结果”的工具，不是运行时。任务队列容量、worker 是否能阻塞等待、取消如何传播、shutdown drain 是否执行已接纳任务，都必须由线程池自己的状态机定义。第 14 章会把这条线推进到 task node、continuation 和 coroutine。

coroutine：把等待端变成 continuation 注册 C++ coroutine 不是另一种线程，也不是自动并行。它把函数的局部状态放进 coroutine frame，在 `co_await` 处可以暂停当前函数，把恢复动作注册到某个 awaitable 上；生产者完成时再恢复这个 coroutine。换句话说，coroutine 把“线程阻塞在 `get()`”改成“shared state ready 后调度 continuation”。这正好对应 future 的 shared state 模型。

Listing 12.68 `co_await` future-like 状态的语义

```

await_ready:
    if shared_state.ready:
        continue without suspension

await_suspend(coroutine_handle h):
    register h as continuation in shared_state
    return to scheduler without occupying worker thread

producer completion:
    store value or exception
    mark shared_state ready
    schedule registered coroutine handles

await_resume:
    return value or rethrow exception

```

这里的关键边界是调度位置。恢复 coroutine 不应在持有 shared state 锁时直接执行用户代码；更稳的做法是把 coroutine handle 作为 continuation 放回 executor 或 runtime 队列。否则一个完成路径可能在锁内重入用户代码，制造锁顺序反转、长临界区和难以追踪的递归调用。Coroutine 购买的是非阻塞等待表达能力，不是替代运行时调度。

coroutine lowering：顺序写法背后是 frame 和状态机 C++ coroutine 的难点不在 `co_await` 这个关键字，而在编译器把函数拆成了可暂停状态机。一个 coroutine 调用不会简单创建线程；它通常分配或嵌入一个 coroutine frame，frame 里保存参数、局部变量、当前暂停点、返回对象的 `promise_type` 和恢复入口。每次 `co_await` 都是在问 awaitable 三个问题：现在能不能继续，若不能继续

把我挂到哪里，恢复时给我什么结果或异常。

Listing 12.69 coroutine frame 的概念字段

```
CoroutineFrame:
  promise_object
  parameters and live locals
  current_suspend_point
  continuation_handle
  exception_slot
  result_slot or reference to FutureState
  allocator and destroy protocol

promise_type:
  get_return_object
  initial_suspend
  final_suspend
  return_value or return_void
  unhandled_exception
```

`initial_suspend` 决定 coroutine 创建后是立刻运行，还是先把 handle 交给调用者或 executor。`final_suspend` 决定 coroutine 结束时如何通知等待者或恢复 continuation。`return_value` 把普通返回值写入 promise 或 FutureState。`unhandled_exception` 把逃逸异常保存起来，让 `await_resume` 或最终 `get` 能在正确线程重新抛出。若这些函数没有定义清楚，coroutine 代码看起来像同步函数，错误传播却比回调更难诊断。

Listing 12.70 `co_await` 的三段合同

```
co_await expression:
  awaiter = get awaiter from expression
  if awaiter.await_ready():
    result = awaiter.await_resume()
    continue in the same call path
  else:
    suspend current coroutine frame
    decision = awaiter.await_suspend(current_handle)
    if decision means suspended:
      return to caller or scheduler
    later, executor resumes current_handle
    result = awaiter.await_resume()
```

`await_suspend` 是最危险的点。它运行在 coroutine 已经暂停、但还没有完全交出控制权的边界上。若 awaitable 在这里把 handle 注册到 shared state 后，producer 同时完成，就会出现竞态：completion 可能试图恢复一个刚注册的 handle。健壮实现需要在同一把锁或同一个原子状态机下处理“注册 continuation”和“发现已经 ready”两条路径。

Listing 12.71 `await_suspend` 的竞态处理

```
await_suspend(handle h):
  lock shared_state
  if state is terminal:
```

```

        unlock
        enqueue h to executor
        return suspended_or_transfer
    append h to continuation list
    unlock
    return suspended

completion:
    lock shared_state
    state = terminal
    continuations = move continuation list
    unlock
    enqueue continuations to executor

```

final suspend：结束点不能把 frame 过早销毁 很多 coroutine 事故发生在结束路径。用户代码执行完并不等于 frame 可以立刻销毁，因为还有等待者可能需要读取结果、异常或恢复 continuation。可靠的 `final_suspend` 往往返回一个 `awaiter`：它在 coroutine 终态发布后，把 continuation 放回 executor，然后让外部 owner 在安全时机销毁 frame。

Listing 12.72 coroutine 结束路径的状态账本

```

body returns value:
    promise_type.return_value stores result
    enter final_suspend
    publish terminal state
    schedule continuation
    frame remains until owner destroys it

body throws:
    promise_type.unhandled_exception stores exception_ptr
    enter final_suspend
    publish ExceptionReady
    schedule continuation
    await_resume later rethrows

consumer destroys task object before ready:
    according to API:
        request cancellation, detach, or destroy only if coroutine is not running

```

因此，coroutine 类型的析构合同必须非常明确。若 task 对象析构就直接 `destroy()` frame，而 producer 线程或 executor 还可能恢复它，就是 use-after-free。若 task 析构只 detach，却没有任何 owner 最终销毁 frame，就是泄漏。若取消只是丢掉 continuation，不把 FutureState 推到终态，等待者会永久 pending。协程不是免生命周期的语法糖，它把生命周期从调用栈搬到了 heap 或运行时管理的 frame。

Listing 12.73 promise 必须在成功和失败路径都满足共享状态

```

1 std::promise<LoadReport> promise;
2 std::future<LoadReport> result = promise.get_future();
3

```

```

4 loader_thread:
5   try {
6     LoadReport report = load_metadata();
7     promise.set_value(std::move(report));
8   } catch (...) {
9     promise.set_exception(std::current_exception());
10  }
11
12 controller:
13   try {
14     LoadReport report = result.get();
15     publish_loaded_manifest(report);
16   } catch (const std::exception& e) {
17     enter_startup_failure_path(e);
18   }

```

`std::async` 也返回 `future`，但它不是生产级 task runtime。未显式指定策略时，实现可以选择异步执行，也可以选择 deferred：任务直到 `get` 或 `wait` 才在等待线程上运行。指定 `std::launch::async` 可以要求异步执行，但仍然缺少队列容量、任务分类、worker 可观测性、取消、背压和调度策略。`std::packaged_task` 可以把 callable 包成 future 生产端；它仍然只解决“一次 callable 产生一次结果”，不解决运行时调度。

future/coroutine 的最低验收矩阵 如果一段运行时宣称支持 future 或 coroutine，不能只跑成功返回值。最低验收应覆盖下面这些边界：

用例	必须观察到的事实	常见失败
成功 <code>set_value</code>	<code>wait</code> 醒来， <code>get</code> 返回值，只能消费一次	结果发布顺序错误
生产者抛异常 promise 未满足析构 <code>wait_for</code> 超时	<code>get</code> 在 caller 线程重新抛出 consumer 观察 broken promise timeout 后 producer 仍可完成	worker 吞异常或崩溃 永远 pending 把 timeout 当 cancel
<code>shared_future</code> 多读	多个 reader 都能读 ready 结果	误用普通 future 广播
提交被拒绝	返回 Rejected，不返回 pending future	调用者拿到无人满足状态
已接纳任务取消 worker 同池等待	FutureState 进入取消或异常终态 trace 显示执行资源耗尽或接口拒绝	从队列删除但不唤醒 偶发饥饿
coroutine ready-fast path	<code>await_ready</code> 为真时不注册 continuation	重复恢复
coroutine late completion	注册和 completion 并发仍只恢复一次	丢 continuation 或 双 resume
final suspend	continuation 入 executor, frame 不被过早销毁	锁内 resume 或 use-after-free

错误用法	失败原因	正确方向
worker 内 等 待 同 池 future	所有 worker 可能阻塞，子任务无人执行	用 continuation、helping 或 分 离 blocking pool
持有 mutex 时调用 get()	producer 可能需要同一把锁发布结果	释放锁后等待，或把结果写入受控状态机
把 future 当广播	普通 future 只能消费一次	用 shared_future、CV 或发布快照
把 future 当可重复事件	shared state 一次 ready 后不重置	用队列、CV、barrier 或版本号
promise 析构前未满足	consumer 得到 broken promise	成功、失败、取消路径都显式满足状态
依 赖 默 认 std::async 并 行	可能 deferred 到等待线程执行	显式策略或使用项目 runtime

第 14 章会把 future 等待放进任务运行时讨论：worker 在同一个 compute pool 里同步等待自己提交的子任务，不是 mutex 死锁，却会造成同池饥饿。这里先固定边界：future/promise 是一次性结果和异常通道，不是调度器。只要等待发生在 worker 内，就必须问“等待期间谁来执行使 future ready 的工作”。

latch、barrier、future、semaphore、CV 的选择边界 这些原语都能让线程等待，但等待对象完全不同。Latch 等一次性归零；barrier 等每一轮所有参与者到达；future 等一次性结果或异常；semaphore 等资源许可；condition variable 等受 mutex 保护的术语谓词。把它们互换，代码通常还能编译，状态机却会错。

等待对象	合适原语	反例
N 个初始化任务都完成	std::latch	用 semaphore 计数后忘记失败路径 release
每 轮 所 有 worker 到达阶段边界	std::barrier	用 latch，第一轮后无法重新关闭
一次后台加载返回结果或异常	std::future/std::promise	用 CV 只通知完成却丢掉异常和结果所有权
最多 K 个请求进入昂贵阶段	counting_semaphore	用 barrier 等固定参与者，动态请求会卡死
队列非空、未关闭、未取消	condition_variable	用 semaphore 后 close 必须制造假许可
单个版本号变化	atomic::wait	用 CV 也能写，但状态已窄到单原子

这张表应该在代码审查中反复使用。问题不是哪个 API 更现代，而是等待事实是否被它完整表达。若事实是阶段，选 barrier；若事实是一次性完成，选 latch；若事实是一次性结果或异常，选 future/promise；若事实是额度，选 semaphore；若事实是对象谓词，选 CV；若事实是单个原子值变化，选 atomic wait。

C++20 和标准库同步原语的适用边界 std::latch 适合一次性等待 N 个参与者完成，例如主线程等待所有 worker 初始化；std::barrier 适合多轮阶段同步，例如每轮模拟都要等所有分片完成再进入下一轮；std::future/std::promise 适合一次性结果和异常传播；std::jthread 和 std::stop_token 适合把协作停止变成显式参数；std::atomic::wait 适合围绕单个原子状态等待。它们都不能替代队列合同：队列仍要定义容量、所有权、关闭、drain 和异常路径。

原语	适合表达	不适合替代
<code>std::latch</code>	一次性倒计时完成	可重复队列等待和关闭
<code>std::barrier</code>	多参与者多阶段推进	生产者消费者背压
<code>std::future/std::promise</code>	一次性结果或异常共享状态	队列、广播、任务调度和重复事件
<code>std::counting_semaphore</code>	资源额度、in-flight 限制	多字段不变量和生命周期
<code>std::stop_token</code>	协作停止请求	强制杀死线程或回滚已提交状态
<code>std::atomic::wait</code>	单原子状态等待	复杂谓词和多条件唤醒

12.10 同步原语选择族谱：把坐标系统一回工程报告

前面已经分别展开了复杂对象谓词、资源许可、一次性完成、多轮阶段和一次性结果。现在把它们收束成工程报告格式。只会说“mutex 慢，atomic 快”不是并发能力；真正要能说清的是：等待事实是什么，状态在哪里，哪个转移会让等待完成，失败和关闭如何退出，运行时证据如何证明这个选择是对的。

按统一模型填写报告 一份同步设计报告不应该只写“使用 mutex”或“使用 semaphore”。它应该把等待事实压成固定字段。下面是同一套字段在不同场景里的填写方式：

场景	等待对象	状态位置	选择理由
bounded queue pop 连接池借用	<code>size>0 closed</code> 一个借出许可	mutex 下 <code>size/closed</code> semaphore 计数 + pool mutex	的多字段谓词，选 CV 许可和对象列表分层
worker 初始化	N 个一次性完成份额	latch 计数 + 错误 状态	只开一次，不能重置
stencil phase	固定参与者到达 phase	barrier phase	多轮、参与者固定
后台加载结果	value 或 exception ready	future shared state	一次性结果通道
配置版本发布 epoll 唤醒	version 变化 fd readable	atomic version eventfd counter	单原子值足够表达 只唤醒，payload 在队列

这张表比 API 速查更重要。它把“为什么选这个原语”落到等待对象上，也把“为什么不选别的原语”变成可审查结论。比如 bounded queue 不选 semaphore，不是因为 semaphore 慢，而是因为 close/drain/cancel 是多字段业务谓词；连接池可以用 semaphore，但只能限制借出预算，不能替代 pool mutex；future 可以返回一次后台加载结果，但不能表达一条长期任务流。

反向排除比正向选择更能暴露错误 很多同步事故不是选不出原语，而是排除理由没写。报告中必须有 `rejected_alternatives`。如果写不出为什么不用另一个原语，通常说明等待对象还没定义清楚。

Listing 12.74 反向排除理由示例

```
bounded_queue:
  chosen: mutex + condition_variable
  rejected:
    counting_semaphore:
```



```
permits cannot broadcast close without fake slots
atomic_wait:
    queue state spans size, head, tail, closed, cancelled
future:
    queue produces a stream, not one value
barrier:
    producers and consumers are dynamic, not fixed participants

phase_stencil:
    chosen: barrier
    rejected:
        latch:
            phase repeats; latch cannot close again
        condition_variable:
            possible, but barrier directly encodes fixed participants and phase
    semaphore:
        permits do not encode all participants reached the same generation
```

从报告反推测试 报告字段必须能导出故障注入。等待对象是谓词，就测试 close 期间等待者是否退出；等待对象是许可，就测试异常路径是否泄漏许可；等待对象是阶段，就测试参与者跳过一轮是否卡住；等待对象是 future shared state，就测试 value、exception、broken promise 和同池等待饥饿。没有这种从模型到测试的映射，章节仍然只是概念说明。

错误选择案例库 下面这些错误比 API 拼写错误危险，因为它们经常通过小测试：

错误选择	为什么错	正确方向
用 semaphore 当 mutex	没有所有者、递归和对象不变量语义	mutex 保护对象，semaphore 限制许可
CV wait 没有谓词	notify 丢失或虚假唤醒后状态不成立	在锁下循环检查完整谓词
atomic wait 等多字段状态	一个原子值不能代表队列和 running 状态	先用 mutex 写状态机，再窄化热点
future 等同池任务	worker 等待期间无人执行使 future ready 的任务	continuation、helping 或分离 blocking pool
future 当广播事件	普通 future 结果只能被 get 一次	shared_future、CV 或发布快照
eventfd 存业务数据	eventfd 只有计数，不保存 payload	eventfd 唤醒，队列保存 completion
futex 地址被提前复用	内核等待队列按地址组织，不懂 C++ 生命周期	等待者退出后再销毁或复用对象
spin 等待长 I/O	浪费 CPU，过度订阅下拖垮系统	阻塞等待或事件驱动

SyncPrimitiveDecision 报告 每个非平凡同步对象都应留下选择报告。报告不是文档负担，而是防止半年后团队只记得“这里用了 semaphore”，却忘了它表达的是资源许可还是状态谓词。

Listing 12.75 SyncPrimitiveDecision

```
SyncPrimitiveDecision:
    protected_state:
        fields
        invariant
    waited_predicate
```

```

permits_or_capacity
result_or_exception_state
phase_or_generation
waiters:
    producer
    consumer
    controller
    event_loop
wake_source:
    push
    pop
    close
    cancel
    completion
timeout_or_cancellation
shutdown_path
lifecycle_owner
metrics:
    wait_count
    wait_ns
    wake_count
    timeout_count
    shutdown_ns
    result_ready_ns
chosen_primitive
rejected_alternatives

```

写出 `rejected_alternatives` 很有用。例如“没有选 `semaphore`，因为关闭时需要同时唤醒空等待和满等待，并保留 `drain` 语义”；“没有选 `future`，因为这里是一条长期任务流，不是一次性结果”；“没有选 `atomic wait`，因为等待谓词跨 `queue size` 和 `unfinished count`”；“使用 `eventfd` 只是为了唤醒 `epoll`，`completion` 数据仍在队列中”。这些句子就是工程判断。

统一判定矩阵：从等待事实到原语合同 同步原语最容易被误用，是因为代码只看到“我要等一下”，却没有把等待事实、状态位置、完成条件和证据字段放在同一张表里。下面这张矩阵是本章后半部分的审查口径：每一行都必须能回答“它能表达什么、不能表达什么、用什么证据证明”。

等待事实	状态位置与完成条件	合适原语	不能替你表达的事
多字段临时不一致	对象字段；状态转换提交后不变量恢复	<code>mutex</code>	长等待、资源额度、结果传播
谓词可能变真	<code>mutex</code> 下字段；醒来后重查谓词	<code>condition_variable</code>	业务事实、通知历史、许可计数
资源预算少一份	原语计数；成功获取许可	<code>counting_semaphore</code>	对象所有权、 <code>close</code> 、 <code>payload</code>
单版本号变化	一个 <code>atomic</code> 值；值不等于旧值	<code>atomic::wait</code>	跨字段谓词、队列生命周期
一次性完成归零	<code>latch</code> 计数；计数归零后打开	<code>std::latch</code>	可重复阶段、每个参与者结果
固定参与者阶段	<code>barrier</code> generation；所有参与者到达	<code>std::barrier</code>	动态任务队列、资源预算
一次结果或异常	<code>future shared state</code> ； <code>ready</code>	<code>future/promise</code>	队列、广播、运行时调度
事件循环被唤醒	<code>fd readiness</code> ； <code>counter</code> 可读	<code>eventfd/pipe</code>	<code>completion payload</code> 、顺序和错误码
睡眠慢路径	用户态地址； <code>expected</code> 仍匹配才睡眠	<code>futex</code>	C++ 对象语义和业务状态机

这些类别不是互斥的工程模块，而是不同层次。一个连接池可能同时使用 `semaphore` 限制借出许可，用 `mutex` 保护空闲连接链表，用 `condition variable` 管理关闭时的 `drain`，用 `future` 把一次后台健康检查结果交给控制线程，用 `eventfd` 唤醒事件循环。错误不在于“用了多种原语”，而在于没有写清每种原语表达哪一层事实。后面几段不再重新讲 API，而是按这张矩阵检查每一类原语的边界。

`semaphore` 只表达许可，不表达所有权和生命周期 信号量的计数是抽象许可，不是对象本身。拿到许可说明容量预算允许继续，不说明你已经拥有某个 C++ 对象，也不说明系统没有关闭。比如连接池：

Listing 12.76 `semaphore` 许可和连接对象是两份状态

```

borrow:
    permits.acquire()           # budget says one borrower may proceed
    lock pool mutex            # object list still needs protection
    if closing:
        unlock
        permits.release()
        return Closed
    take one connection from free list
    unlock

return:
    lock pool mutex
    put connection back, or mark it broken
    unlock
    permits.release()

```

这个例子暴露两个常见事故。第一，`borrow` 拿到许可后发现池已关闭，必须归还许可；否则系统会泄漏许可，最终所有线程都睡在 `acquire`。第二，连接坏了不能简单 `release` 后塞回池里；许可计数和对象健康状态是两层。信号量适合表达“最多 N 个 in-flight”，不适合单独表达“这些对象的列表、状态和生命周期都正确”。

`condition variable` 只保存等待队列，不保存业务事实 条件变量不记住业务事件。队列里有没有元素、线程池是否 `idle`、运行时是否关闭，都在受 `mutex` 保护的字段里。条件变量只让等待者在谓词暂时不满足时睡眠，并在状态可能变化后

回来重查。

Listing 12.77 CV 谓词必须写完整

```
bad:
    wait until queue_size > 0

better:
    wait until queue_size > 0 or closed or cancelled or error

why:
    empty + closed should exit
    empty + cancelled should exit
    empty + error should report failure
    wakeup without data should continue waiting
```

因此，新增一个状态字段就必须重新审查所有 CV 谓词。给队列加 `cancelled`，却只在 `push` 路径检查取消，消费者仍可能睡在 `not_empty`；给线程池加 `failed`，却不唤醒 `wait_idle`，控制线程会永远等待。CV 的正确性不来自 `notify` 次数，而来自完整谓词和所有状态变化的通知覆盖。

`atomic::wait` 适合版本，不适合复杂谓词 `atomic::wait` 的强项是单个原子值。它等待的是“这个值不再等于我看到的旧值”。这非常适合版本号、单状态枚举、一次性发布标志和窄 `ready bit`。它不适合跨多个字段的队列状态。

Listing 12.78 `atomic wait` 的合适形态

```
reader:
    old = version.load(acquire)
    version.wait(old)
    new = version.load(acquire)
    reload immutable snapshot

writer:
    publish immutable snapshot
    version.fetch_add(1, acq_rel)
    version.notify_all()
```

反例是把线程池 `idle` 压成一个原子布尔：

Listing 12.79 错误：`idle` 不是单个布尔事实

```
wrong_idle_flag:
    ready_queue_empty = true
    running_count may still be nonzero
    submitter may be between increment and enqueue
    a completion callback may enqueue more work
    shutdown may be draining accepted tasks
```

这里等待谓词跨 `ready queue`、`running count`、`submission in progress`、`shutdown state` 和 `failure state`。把它压成一个 `atomic bool` 会制造提前返回。正确路径是先用 `mutex` 写清多字段谓词；若后续发现某个窄版本发布路径是热点，再把那个路径单独改成 `atomic wait`。

`eventfd` 唤醒事件循环，不承载 `payload` `eventfd` 是 Linux `fd` 形式的计数

器，适合把“有 completion 了”通知给 epoll/io_uring 所在的线程。它不保存业务数据，不表达队列容量，也不替代 mutex。正确形态通常是：业务数据写入 completion queue；再写 eventfd 唤醒事件循环；事件循环读 eventfd 后 drain completion queue。

Listing 12.80 eventfd 的正确分层

```
worker thread:
    lock completion queue
    push completion record
    unlock
    write eventfd counter by 1

event loop:
    epoll_wait returns eventfd readable
    read eventfd counter
    lock completion queue
    drain completion records
    unlock
```

错误形态是把 eventfd 当业务消息队列。eventfd 只有一个 64-bit counter；多次写入可以合并成一个可读计数，它不会保存每个请求的 id、错误码、buffer 所有权或完成顺序。payload 必须在另一个受保护的数据结构中，eventfd 只负责把事件循环从睡眠中叫醒。

futex 是等待慢路径，不是业务抽象 futex 的核心是“用户态地址上的整数状态 + 内核等待队列”。它解决的是高层同步原语的睡眠慢路径：用户态先检查值，若值仍等于 expected，再让内核把线程挂到该地址对应的等待队列上。它不懂 C++ 对象生命周期、不懂队列 close、不懂元素所有权。

Listing 12.81 futex 语义边界

```
what futex knows:
    user address
    expected integer value
    wait queue keyed by address
    wake count

what futex does not know:
    C++ object lifetime
    queue closed/drainning/cancelled state
    whether a task was accepted
    whether a permit leaked
    whether a wake is business-complete
```

因此，直接拿 futex 写业务队列时，最难的不是系统调用语法，而是证明地址生命周期、等待谓词、关闭唤醒和对象复用都正确。高层代码通常应先用标准库 mutex/CV/semaphore 建立合同，再通过性能证据决定是否需要更底层的等待协议。

六个错误案例：为什么“能跑”不等于选对原语 下面这些案例适合写进实验报告：

Listing 12.82 同步原语错误选择案例

```
case semaphore_as_mutex:
```



```

use a binary semaphore to protect a vector
forget owner/lifetime rules
destructor runs while a waiter still holds or waits
result: race, permit leak, or use-after-free

case cv_without_close_predicate:
  consumer waits for size > 0
  controller sets closed = true and notifies
  consumer wakes, sees size == 0, waits again
  result: shutdown hangs

case atomic_wait_for_thread_pool_idle:
  worker pops last ready task and sets queue_empty = true
  controller observes idle flag
  running task has not finished
  result: wait_idle returns before work is complete

case barrier_for_dynamic_task_queue:
  worker with no local tasks skips phase p
  other workers call arrive_and_wait for phase p
  expected count is never reached
  result: phase p never completes

case future_wait_inside_same_pool:
  each worker submits a child task to the same pool
  each worker then calls future.get()
  child tasks stay ready but no worker is runnable
  result: starvation without a mutex deadlock

case eventfd_as_payload_queue:
  worker writes eventfd for each completion
  event loop reads only aggregate counter
  no completion record exists elsewhere
  result: request identity and error state are lost

```

这些案例的共同点是：原语表面上都能“等”，但等待对象错了。同步设计要先写业务事实，再选原语；不能先选一个看起来高级的 API，再倒推业务语义。

同步原语 evidence pack 一份合格同步报告至少保存这些字段：

Listing 12.83 SyncPrimitiveEvidencePack

```

sync_primitive_evidence_pack:
  state_model:
    protected_fields:
    invariant:
    lifecycle_owner:

  wait_model:
    wait_object: invariant / predicate / permit / version / phase / result / ↔
    ↔ event_fd
    predicate_or_expected_value:
    phase_or_generation:

```

```

    result_or_exception_state:
    close_cancel_error_timeout:

primitive_choice:
    chosen:
    rejected:
        mutex:
        condition_variable:
        semaphore:
        atomic_wait:
        latch:
        barrier:
        future_promise:
        eventfd:
        futex_direct:

wake_model:
    who_changes_state:
    who_notifies:
    notify_one_or_all:
    completion_function:
    spurious_wakeup_policy:
    lost_wakeup_defense:

failure_injection:
    close_while_empty:
    close_while_full:
    cancel_while_waiting:
    timeout:
    exception_or_early_return:
    missing_latch_count_down:
    barrier_phase_skip:
    broken_promise:
    same_pool_future_wait:
    object_destruction_with_waiters:

metrics:
    wait_count:
    wake_count:
    timeout_count:
    phase_wait_ns:
    result_ready_ns:
    max_wait_ns:
    shutdown_ns:

```

这份报告的价值在于边界清楚。看到 `semaphore`，读者知道它只负责 `permit`；看到 `CV`，读者知道谓词在哪里；看到 `latch`，读者知道计数是否一次性归零；看到 `barrier`，读者知道 `phase` 和参与者数在哪里；看到 `future`，读者知道 `value`、`exception` 和 `broken promise` 如何传播；看到 `atomic wait`，读者知道只等哪个原子值；看到 `eventfd`，读者知道 `payload` 在哪里；看到 `futex`，读者知道这是底层等待路径，不是业务语义本身。

12.11 锁顺序、性能路径和工业现实

单个锁保护单个队列容易证明；多个锁组合后，死锁和尾延迟变得真实。死锁的必要条件包括互斥、持有并等待、不可抢占和循环等待。工程上最常见的破坏方式是规定锁顺序，或者把组合操作改成消息传递/两阶段提交，避免同时持有多把锁。

锁顺序表 项目应维护锁顺序表。例如：全局配置锁先于任务图锁，任务图锁先于队列锁，队列锁先于指标锁。任何反向获取都应被重构或严格证明不会并发。锁顺序表还要写明哪些锁持有时禁止调用用户回调、禁止 I/O、禁止等待 future。持锁进入未知代码，比持有两把锁更危险，因为未知代码可能再进入运行时并获取另一把锁。

优先级反转和 convoy 优先级反转发生在低优先级线程持有锁，高优先级线程等待它，而中等优先级线程持续占用 CPU，使低优先级线程迟迟不能释放锁。普通服务端也有类似现象：前台请求等待后台 checkpoint 持有的锁，尾延迟被放大。Lock convoy 则是大量线程排队等待同一锁，释放后唤醒、抢占和再次竞争降低吞吐。

公平锁可以减少饥饿，但可能降低吞吐；非公平锁吞吐可能更高，但某些线程等待时间更长。同步原语的选择要服务业务目标：批处理更关注总吞吐，交互服务更关注尾延迟和饥饿边界。

mutex 的 fast path 和 futex 慢路径 Linux 上常见 mutex 在无竞争时尽量只做用户态原子状态改变；竞争严重时，线程通过 futex 进入内核睡眠，释放方再唤醒等待者。条件变量等待也会释放 mutex 并进入等待，唤醒后重新竞争 mutex。理解这点后，“锁慢”就不再是单一结论：低竞争短临界区很便宜，高竞争睡眠唤醒会涉及内核和调度，过度订阅下自旋会浪费 CPU。

同步性能要按等待原因归因 只记录吞吐不够。同步性能问题要拆成：锁等待时间、临界区执行时间、条件变量睡眠时间、被唤醒后重新竞争锁的时间、队列满导致的背压时间、队列空导致的 idle 时间、关闭广播后的退出时间。若没有这些指标，团队很容易把“下游 I/O 慢”误判成“mutex 慢”，或者把“用户任务太长”误判成“condition variable 唤醒慢”。

指标	回答的问题	典型解释
<code>lock_wait_ns</code>	拿锁前等了多久	临界区过长或锁粒度过粗
<code>critical_ns</code>	持锁执行多久	锁内做了 I/O、日志或用户代码
<code>not_empty_wait_ns</code>	消费者睡眠多久	上游产出不足或关闭未通知
<code>not_full_wait_ns</code>	生产者睡眠多久	下游慢，背压正在生效
<code>notify_to_run_ns</code>	唤醒到真正运行多久	过度订阅、调度延迟或 convoy
<code>close_join_ns</code>	close 到所有线程退出多久	运行中任务太长或退出谓词错误

12.12 完整执行账本：从 wait 到 futex 再到关闭

条件变量最容易被写成“通知机制”，但底层路径更接近“围绕共享谓词的睡眠协议”。下面把消费者等待空队列、生产者插入任务、关闭队列、线程池等待 idle 四条路径写成账本。账本的目标不是模拟某个标准库实现，而是固定每个状态变化的语义边界：谁持有锁，哪个谓词被检查，线程是否还在 run queue，通知之后为什么还要重新检查。

空队列等待的事件链 消费者进入 pop，持有 mutex 后看到 `size==0` 且 `closed==false`。此时继续持锁睡眠会死锁，因为生产者无法进入临界区插入元素；先释放锁再睡眠又可能丢通知。因此条件变量的 `wait` 必须把“把自己加入等待集合、释放 mutex、进入睡眠”做成一个不会被普通用户代码拆开的协议。

Listing 12.84 消费者等待空队列的语义事件链

```

consumer locks queue mutex
consumer observes size == 0 and closed == false
consumer enters condition-variable wait
wait operation releases mutex and parks the thread atomically
kernel/runtime removes the thread from runnable execution
producer can now acquire mutex and change queue state

```

这里的“atomically”不是说 CPU 执行一条神奇指令，而是说标准库和内核配合，避免在“已经决定睡眠但尚未真正等待”这个窗口里错过通知。Linux futex 路径的常见思想是：用户态先检查某个内存状态，只有仍然不满足时才让内核挂起线程；唤醒方改变内存状态后唤醒等待者。状态仍是权威，futex 只是等待队列。

生产者插入后的唤醒链 生产者持锁写入槽位，更新 `tail` 和 `size`，这时谓词 `size>0||closed` 从 `false` 变成 `true`。通知可以在锁内或锁外发生，但必须发生在状态变化之后。被唤醒的消费者不是立刻运行：它先从睡眠变成 `runnable`，等待调度器分配 CPU，随后重新竞争 `mutex`，最后再次检查谓词。

Listing 12.85 `notify` 到真正执行之间仍有调度路径

```

producer locks queue mutex
producer writes element into tail slot
producer increments size
producer unlocks mutex
producer notifies not_empty
waiting consumer becomes runnable
scheduler eventually runs consumer
consumer re-locks queue mutex
consumer rechecks size > 0 or closed

```

这条链解释了两个现象。第一，`notify_one` 之后消费者的延迟不只由条件变量决定，还包含调度延迟和重新拿锁的时间。第二，醒来的消费者可能发现队列又空了，因为另一个消费者先拿到锁并取走元素。谓词循环不是保守写法，而是条件变量正确性的中心。

关闭广播的事件链 关闭是全局状态变化，不能只唤醒一个等待者。生产者可能睡在 `not_full`，消费者可能睡在 `not_empty`，等待 `idle` 的线程可能睡在 `idle`。关闭必须改变共享状态，再广播所有受影响的等待点，让每个等待者回到锁内重新判断退出路径。

Listing 12.86 队列关闭的语义事件链

```

close locks queue mutex
close sets closed = true
close unlocks queue mutex
close notifies not_empty waiters
close notifies not_full waiters
each waiter wakes, re-locks, rechecks predicate
push waiters return Closed
pop waiters drain remaining elements or return empty

```

关闭时如果只通知 `not_full`，空队列上的消费者不会醒；如果只通知 `not_empty`，满队列上的生产者不会醒；如果谓词不包含 `closed`，醒来的线程可能重新

睡回去。可靠关闭是“状态 + 所有等待点 + 完整谓词”的组合。

wait idle 的不变量 线程池的 `wait_idle` 不是等待队列为空。队列为空只能说明没有尚未领取的 `ready task`；`worker` 已经领取并正在执行的 `task` 不在队列里。正确谓词通常是：`ready` 队列为空，`unfinished count` 为零，并且没有正在提交的任务跨过计数边界。若任务执行中抛异常，仍必须递减 `unfinished count`；若 `completion` 回调递归提交任务，必须先增加 `unfinished count` 再把任务放入队列，防止 `wait_idle` 看到瞬时空状态提前返回。

Listing 12.87 `wait idle` 的谓词账本

```
submit task:
    lock runtime
    increment unfinished_count
    push task to ready queue
    notify worker

worker finish task:
    run task outside runtime lock
    lock runtime
    decrement unfinished_count
    if unfinished_count == 0 and ready queue is empty:
        notify idle waiters

wait idle:
    lock runtime
    wait until unfinished_count == 0 and ready queue is empty
```

这条账本比“任务队列为空”更强，也更接近运行时真实语义。第 14 章的任务图会把 `ready`、`running`、`finished`、`failed` 和 `canceled` 状态写得更细；这里先固定同步底座：等待者必须等待系统状态，而不是等待某次通知。

惊群和 `convoy notify_all` 能保证等待者有机会重新检查状态，但也可能制造惊群：很多线程同时醒来，只有少数线程能取得资源，其余线程重新睡眠。关闭、取消和阶段完成必须广播；普通 `push` 一个元素时通常只需要 `notify_one`。如果队列容量很小、生产者频繁 `notify_all`，系统会在唤醒、调度、抢锁和再睡眠之间浪费大量时间，表现为吞吐下降和 `p99` 上升。

Lock convoy 则常出现在高竞争单锁路径：一个线程释放锁后，多个等待者被唤醒或排队，锁在核心间迁移，下一位持锁者又执行较长临界区。队列、指标、日志、内存分配若都塞在同一锁下，`convoy` 会把短操作拖成尾延迟。拆锁前要先写不变量边界，不能把一个完整状态机拆成数据竞争。

工程案例

Linux `futex`。 `futex` 的核心工程思想是：同步状态放在用户态地址，内核只在需要睡眠和唤醒时介入。它购买的能力是低争用 `fast path` 和高争用 `blocking path` 的分离；代价是用户态必须正确维护等待条件，避免丢失唤醒。`Compute Systems Engine` 不实现 `futex`，但要吸收“状态检查和睡眠必须成对”这条原则。

pthread `mutex/condition variable`。 `pthread` 把互斥、等待、唤醒和调度交给成熟库和内核配合处理。它购买的是可移植、可靠和经过大量边界打磨的同步语义；代价是抽象层隐藏了一些性能细节。示例代码应使用标准库 `RAII` 包装，而不是手写锁原语。

Linux `wait queue` 和 `workqueue`。内核 `wait queue` 强调等待条件和唤

醒来源；workqueue 强调工作项与执行线程分离。两者共同提醒我们：等待者要围绕状态睡眠，工作项不等于线程。Compute Systems Engine 的 `QueueContract` 和 worker 退出协议应吸收这个原则。

生产级 executor。 oneTBB、Folly executor、Java executor 和 Go scheduler 都把“提交工作”“执行工作”“等待完成”“关闭运行时”拆成不同协议。成熟设计通常不会让队列析构隐式杀死 worker，也不会把任务函数放在队列锁内执行。它们购买的是清楚生命周期和可观测性；代价是状态更多，测试矩阵更大。Compute Systems Engine 应先实现清楚的锁版 executor，再把热点迁移到更细的队列或 work stealing。

数据库后台线程。 RocksDB compaction、PostgreSQL background writer、Kafka log cleaner 这类后台任务通常有队列、条件变量、停止标志、in-flight 工作和错误传播。它们最重视的不是“锁最少”，而是关闭时不丢已提交事实、故障路径可恢复、等待状态可诊断。可迁移原则是：后台线程必须能解释自己在等什么，停止请求必须能唤醒所有等待点。

RCU。 RCU 通过“读取方读旧版本，写者发布新版本，延迟回收旧对象”优化读多写少路径。它购买的是极轻读路径；代价是对象生命周期和回收协议复杂。Compute Systems Engine 可在配置快照或路由表章节借鉴“不可变版本发布”，但不要再用 RCU 替代需要复杂关闭语义的队列。

12.13 读写锁、RCU 和原子的边界

读写锁允许多个读取方并发、写者独占，适合读临界区相对较长、写入低频、读路径不修改共享状态的对象，例如配置表或路由快照。它不适合每次请求都要更新统计、LRU、引用计数或访问时间的“伪读操作”。读临界区极短时，读写锁维护读取方计数和公平策略的成本可能超过普通 mutex。

RCU 或 copy-on-write 更适合不可变快照：读取方读取稳定版本，写者复制并发布新版本，旧版本延迟回收。它让读路径很轻，却把复杂度转移到写路径和内存回收。第 12 章会深入生命周期和回收问题。

Atomic 适合更窄的协议：统计计数、一次性发布、小状态机、槽位 ready 标志、单生产者单消费者 ring 的索引。Atomic 不是锁的替代品。若一个状态需要维护多字段不变量、复杂关闭和等待队列，先用 mutex/reference 写清语义，再考虑是否有足够窄的原子协议。

12.14 QueueContract：项目中的同步合同

`QueueContract` 不是一个必须存在的 C++ 类，而是每个队列必须填写的工程合同。线程池队列、I/O 队列、MessageBus 队列、completion 队列可以使用不同实现，但必须回答同一组问题：容量按任务数还是字节数计算；满时阻塞、失败还是丢弃；空时等待还是返回；关闭后是否允许 drain；是否支持 timeout；元素所有权何时转移；等待谓词是什么；关闭时通知哪些等待者。

合同字段	要回答的问题	Compute Systems Engine 默认
容量	按元素、字节还是 in-flight 许可	线程池按任务数, I/O 按字节和 in-flight
满队列	阻塞、返回失败、丢弃还是背压上游	计算任务阻塞或背压, 不静默丢弃
空队列	阻塞等待、返回空还是轮询	worker 阻塞等待谓词
关闭	拒绝新输入, 是否 drain 已接收元素	默认 drain accepted
取消	未开始元素如何处理, 运行中如何协作退出	状态记录为 canceled
所有权	push 成功后元素归谁, pop 后谁负责处理	队列只转移所有权, 不执行任务
通知	哪些状态变化会唤醒哪些等待者	push/pop/close/task finish 明确列出
线性化点	操作何时对其他线程可见	push 更新 <code>size/tail</code> , close 设置 <code>closed</code>
错误传播	任务异常或写出失败如何报告	不吞异常, 转成 <code>result/error</code> 状态
观测指标	等待、拒绝、关闭、尾延迟如何记录	每个等待点都有计数和耗时

从队列迁移到线程池 线程池比队列多两个状态：正在运行的任务和未完成计数。队列为空不代表线程池 idle, 因为 worker 可能已经取出任务正在执行; 队列关闭不代表任务完成, 因为已接收任务可能还在 running。线程池的 `wait_idle` 谓词应围绕 `unfinished count`, 而不是队列长度。任务抛异常时, 也必须减少 `unfinished count` 并通知等待者。

从队列迁移到 I/O 背压 I/O 队列还要考虑字节容量和 in-flight 许可。一个 `ShuffleBlock` 可能大小差异很大, 只按任务数限制会导致内存峰值失控。可靠写出还会引入 `temp file`、`checksum`、`fsync`、`rename`、`manifest commit`, 队列只负责交付 block 描述, 不能把长 I/O 放进队列锁。第 15 章会把这里的背压合同扩展成完整 I/O 管线。

可关闭有界队列是本章最小综合路径。实现一个 `BoundedQueue<int32_t>` 或等价示例队列, 覆盖容量为 1、容量为 2、多个 producer/consumer、空队列 close、满队列 close、重复 close、运行中 close、消费者慢、生产者慢。每个元素包含 `producer id` 和 `sequence`, 最终检查不丢、不重、每个 producer 内部顺序符合接口声明。证据保留 `push` 等待次数、`pop` 等待次数、最大队列长度、close 到所有线程退出的时间。

测试要攻击等待路径 并发测试应故意制造极端条件：队列容量为一, worker 数为一或二, 随机 `yield`, 随机短 `sleep`, 固定 `seed` 重放失败, 任务抛异常, close 与 `push/pop` 并发。大容量和大线程数有时只会掩盖状态机错误。ThreadSanitizer 可以发现数据竞争, 但会改变性能, 不能用于性能结论。

反例驱动的验收 实验不应只提交一个“正确版本”。报告要包含至少三个反例：等待谓词不含 `closed_` 导致空队列 close 卡住; 任务异常跳过 `unfinished_count_` 递减导致 `wait_idle` 卡住; 持队列锁执行任务导致任务递归提交时死锁。每个反例都要有可复现 `seed`、线程数量、容量和事件日志。反例能够被复现和修复, 才说明同步原理进入了工程判断。

源码阅读任务

源码阅读按“等待为何能正确醒来”这条线组织。

1. Linux `futex`: 阅读 `kernel/futex/core.c` 相关等待和唤醒路径，重点看用户态地址、等待队列和 `wake` 的关系，不需要追完整优先级继承实现。
2. Linux `mutex/wait queue`: 阅读 `kernel/locking/mutex.c` 和 `include/linux/wait.h`，说明 `fast path`、阻塞路径和唤醒条件如何分层。
3. `pthread` 或 `libc++` 同步原语: 观察 `std::condition_variable` 如何映射到底层 `pthread`，理解标准库为什么要求配合 `std::unique_lock`。
4. 工业队列或 `executor`: 找一个有 `shutdown/drain` 语义的队列，分析它怎样定义关闭、取消和等待者唤醒。

报告必须从一个具体失败开始，例如“`close` 后消费者不醒”或“任务异常后 `wait` 永久阻塞”，然后把源码机制和项目 `QueueContract` 对上。

12.15 完整案例：容量为一的队列如何暴露所有同步错误

容量为一的有界队列是最好的同步显微镜。容量大时，许多错误会被缓冲掩盖；容量为一时，生产者和消费者几乎每一步都要交接状态，满、空、关闭、异常和通知都会被频繁触发。若一个队列在容量为一、多生产者、多消费者、随机调度和并发关闭下仍能解释所有状态，扩大容量只是工程问题；若容量为一就说不清，复杂系统只会把错误埋得更深。

状态变量必须能解释每一种等待 队列至少有四类状态：缓冲区内容、`size` 或 `head/tail`、`closed` 标志、等待者关心的容量和空非空谓词。生产者等待的是“有空间或关闭”；消费者等待的是“有元素或关闭”；关闭者改变的是“未来不再接纳新元素，并唤醒所有可能等待的人”。这些谓词不是注释，而是条件变量等待的核心。

Listing 12.88 容量为一队列的谓词

```
producer may push when:
    size < capacity and not closed

producer must stop when:
    closed

consumer may pop when:
    size > 0

consumer may finish when:
    size == 0 and closed
```

若 `pop` 只等待 `size>0`，空队列 `close` 时消费者会永久睡眠；若 `push` 只等待 `size<capacity`，满队列 `close` 时生产者也可能永久睡眠。关闭必须进入所有等待谓词。

线性化点要和通知配套 `push` 的线性化点是元素进入队列并且 `size` 增加的状态变化。通知消费者应发生在这个状态变化之后。若先通知再写元素，消费者醒来后可能仍看不到元素；若写元素后没有通知，消费者可能一直睡到下一次偶然通知。由于等待者会在锁下重新检查谓词，通知在锁内还是锁外不是最核心问题；核心是状态变化和通知不能脱节。

Listing 12.89 push 的状态和通知顺序

```

push:
    lock
    wait until size < capacity or closed
    if closed:
        return Closed
    write element into slot
    size += 1
    unlock
    notify not_empty

```

pop 对称：取走元素，`size` 减少，通知 `not_full`。若元素类型移动构造可能抛异常，必须决定异常发生在线性化点之前还是之后。更稳的工程做法是把可能抛异常的构造尽量放在进入临界区之前，临界区内只做不会失败的移动或状态交换；若无法保证，就要写异常安全状态表。

Close 不是销毁 `close` 表示不再接收新元素，并唤醒等待者。它不是析构，也不是清空队列。Drain 语义下，`close` 后消费者仍可取走已经入队的元素；队列为空且 `closed` 时，消费者返回结束。Cancel 语义则可以丢弃未处理元素，但必须把丢弃写成可观察事实。两者不能混用。

Listing 12.90 drain close 的事件链

```

initial:
    size = 1, closed = false

producer calls close:
    lock
    closed = true
    unlock
    notify all not_empty and not_full

consumer pop:
    sees size = 1, pops last element
    later pop sees size = 0 and closed = true
    returns Closed

```

重复 `close` 应幂等。第一个 `close` 改变状态并通知；后续 `close` 可以返回 `already closed`，但不能破坏队列。析构则要求更强：不能在仍有生产者或消费者访问队列时销毁内部 `mutex` 和 `condition variable`。对象生命周期必须由外层 `join` 或 `shared ownership` 管理。

容量为一的失败日志 一次典型 bug 是生产者在满队列等待，另一个线程 `close`，但 `close` 只通知 `not_empty`，没有通知 `not_full`。消费者已经退出，生产者永远睡在 `not_full` 上。容量为一时很容易复现：

Listing 12.91 关闭没有唤醒生产者的失败日志

```

T0 consumer pops first item and exits early
T1 producer pushes item 7, queue becomes full
T2 producer tries to push item 8, waits on not_full
T3 controller closes queue, notify not_empty only
T2 remains asleep although closed == true

```

修复不是“多 notify 几次”这么粗。正确修复是：所有等待谓词包含 `closed`；`close` 状态变化后广播所有相关条件变量；醒来的生产者看到 `closed`，返回 `Closed`；醒来的消费者要么 `drain` 剩余元素，要么看到空且 `closed` 后退出。测试应验证 `close` 后所有等待线程能在有限时间内退出。

从同步状态推出报告字段 队列报告至少记录 `push` 成功、`push closed`、`push` 等待次数、`pop` 成功、`pop closed`、`pop` 等待次数、`close` 次数、最大 `size`、等待总时间和最长等待。这样当系统卡住时，可以知道谁在等空间、谁在等元素、`close` 是否发生、等待者是否被唤醒。同步原语本身很底层，但工程排障需要这些业务化指标。

Listing 12.92 QueueReport 的语义字段

```
push_success
push_closed
push_wait_count
push_wait_ns
pop_success
pop_closed
pop_wait_count
pop_wait_ns
close_count
max_size
```

这些字段不能都在同一热锁内高频更新，否则报告本身会成为竞争源。可以在每个 `worker` 本地累积统计，周期性汇总；但控制面状态，例如 `closed` 和队列 `size`，仍必须由队列锁或严格原子协议保护。

12.16 接口延伸

锁、条件变量、信号量、`latch`、`barrier` 和 `future` 不是一组可以随便替换的“等待 API”，而是不同状态模型的工具。`Mutex` 保护不变量；`condition variable` 等待谓词；`semaphore` 表达资源额度；`latch` 表达一次性归零；`barrier` 表达多轮阶段边界；`future/promise` 表达一次性结果或异常。同步设计的质量取决于状态机是否完整、等待者是否有退出路径、关闭和异常是否覆盖、锁内是否只做可控状态转换、测试是否故意攻击边界。

这里留下的项目对象是 `QueueContract`。第 9 章的 `ThreadPoolPlan` 和 `WorkerSnapshot` 需要队列合同才能可靠 `shutdown`；第 14 章的任务运行时会在队列之上增加 `ready count` 和 `unfinished count`；第 15 章的 I/O 管线会把容量从任务数扩展到字节和 `in-flight`；第 16 章以后的 `MessageBus` 会复用同样的 `close/drain/backpressure` 语义。

12.17 习题

习题与作业

1. 写出 `BoundedQueue<T>` 的不变量表，并标出每个字段由哪把锁保护。解释为什么只把 `size` 改成 `atomic` 不能让队列正确。
2. 给出一个错误版本：`pop` 的等待谓词只检查 `size>0`。构造空队列 `close` 的测试，让消费者永久睡眠，再修复它。
3. 实现容量为一的有界队列压力测试。多个 `producer` 产生唯一 `id`，多个 `consumer` 记录消费日志，最终检查不丢、不重、关闭后所有线程退出。

4. 设计 `QueueContract` 文档，分别填写 `compute task queue`、`I/O byte queue`、`MessageBus queue` 三种队列的容量、满队列、空队列、`close`、`drain` 和取消语义。
5. 分析一个锁顺序死锁：线程 A 持有队列锁后等待指标锁，线程 B 持有指标锁后等待队列锁。给出锁顺序表和重构方案。
6. 阅读 `futex` 或 `pthread condition variable` 的资料，解释为什么“修改状态”和“进入等待”必须配合，不能只靠一次 `notify`。
7. 为连接池、`bounded queue`、配置快照发布、`epoll completion loop` 各写一份 `SyncPrimitiveDecision`，明确什么时候用 `semaphore`、`condition variable`、`atomic wait` 和 `eventfd`。
8. 构造一个错误版本：用 `semaphore` 替代队列 `mutex`，并在 `close/drain` 路径中复现生命周期或许可泄漏问题。说明 `semaphore` 只能表达许可，不能保护多字段不变量。
9. 写一个 `std::latch` 初始化实验：其中一个 worker 在异常路径漏掉 `count_down`，复现 `controller` 永久等待；再用 `scope guard` 或统一 worker 入口修复。
10. 写一个 `std::barrier` 阶段实验：固定 4 个 worker 跑 3 个 phase，其中一个 worker 因“无工作”跳过 phase 1，复现卡住；再改成无工作也到达或显式 `arrive_and_drop`。
11. 写一个 `std::promise/std::future` 实验：生产者成功、抛异常、`promise` 析构未满足三条路径都要覆盖；说明 `future::get` 为什么只能消费一次。
12. 构造同池 `future` 饥饿：4 个 worker 各自提交一个子任务后同步 `get`，解释为什么这不是锁死锁，却会让 `ready queue` 无人执行。给出 `continuation` 或 `helping` 的修复方向。

工程案例

知识点小案例：条件变量等待的不是通知，而是不变量。特例是一个有界队列：consumer 在空队列上等待，producer push 后 `notify`。错误版本只在 `pop` 中等待 `size>0`，没有把 `closed` 纳入谓词；当队列关闭且没有元素时，consumer 可能永久睡眠。由此推出规律：`condition variable` 不是消息队列，`notify` 只提醒重新检查状态，真正的语义在被锁保护的不变量和等待谓词里。工程上每个同步原语都要写清保护字段、等待条件、`close/drain/cancel` 语义和虚假唤醒后的重查规则。

原子操作与内存序

先看一个在强内存模型机器上很容易被掩盖的 bug。生产者构造 `RuntimeConfig`，写入 `worker_count`、`queue_capacity` 和 `version`，然后把 `ready` 标志设为 `true`。消费者轮询 `ready`，看到 `true` 后读取配置字段。开发机上测试长期通过，于是有人认为“`ready` 已经是 `atomic`，肯定没数据竞争”。问题在于：这个 `atomic` 标志只是保证自己不被撕裂，并不自动把旁边的普通字段发布给另一个线程。若 `ready` 使用 `memory_order_relaxed`，程序缺少可移植的跨线程发布关系。

这个错误不能靠背一组枚举解决。要先回到机器：每个核心有自己的流水线、store buffer、私有 L1、共享的下级缓存和一致性协议。普通写入先进入本核心的写路径，随后才让其他核心有机会观察到；普通读取可能从本核心已经持有的 cache line、乱序执行窗口或编译器保留的寄存器值中得到结果。缓存一致性保证同一条 cache line 的写入最终形成可解释的修改顺序，但它不等于“所有普通字段按源代码顺序立刻出现在所有核心面前”。C++ 还要允许编译器重排、合并、缓存普通访问；没有同步关系的并发普通读写甚至会落入未定义行为。于是，原子操作的真正问题不是“这条指令是否神奇”，而是怎样在语言、编译器和硬件之间写出一条可验证的同步合同。

一个原子变量必须有用途：它只是统计计数，还是发布普通数据；它是状态机的线性化点，还是指针生命周期的一部分；它是否会被等待，是否会参与关闭。用途不同，内存序才不同。`relaxed` 适合只关心自身数值的指标；`release/acquire` 适合把普通字段发布给另一个线程；`CAS` 适合定义单点责任转移；`seq-cst` 适合需要全局顺序或初始收敛的协议。原子代码贵在少而准，最危险的不是不会写 `std::atomic`，而是把 `atomic` 当成“无锁万能胶”贴到复杂状态上。

13.1 从锁版 Reference 到原子协议

第 10 章用 `mutex` 和 `condition variable` 保护有界队列的不变量。锁的优点是把多个字段的状态转换放进同一个临界区，容易证明正确。原子操作适合更窄的同步协议：单个计数器、一次发布、一个槽位状态、一个 `CAS` 线性化点、一个引用计数变化。若状态本身是多字段不变量，直接把字段改成 `std::atomic` 通常只会把错误藏得更深。

从总线锁到缓存行所有权 早期处理器可以用锁总线的方式让某个读改写操作独占内存总线。现代多核机器不能把每次计数器递增都变成全系统停顿，于是原子读改写通常围绕 cache line 的独占所有权实现：核心必须先让包含目标原子对象的 cache line 进入可写独占状态，再执行读、修改、写这一组不可分割动作。其他核心若也要修改同一条线，就要通过一致性协议抢所有权。由此可以推出两个重要结论。第一，原子性有真实硬件代价，热点 `atomic` 会制造 cache line 所有权迁移。第二，原子性只覆盖目标对象所在的那次操作，不会自动把附近的普通对象变成同一个事务。

C++ 原子和汇编不是简单一一对应 第一册读汇编时，可以把 `mov`、`cmp`、`lock xadd` 看作机器动作。本章还要加一层：C++ 内存序首先约束编译器和抽象机，然后再由编译器为目标 ISA 选择合适指令或屏障。x86-64 的普通 aligned load/store 已经有较强的硬件顺序，因此某些 `acquire load` 和 `release store` 在汇编上可能看起来只是普通 `mov`；ARM、POWER 等更弱内存模型机器可能需要带 `acquire/release` 语义的 load/store 或显式 `barrier`。不能因为某次 x86 反汇编没有看到 `fence`，就认为 `acquire/release` 没有意义；它仍然约束编译器，并提供可移植 C++ 同步语义。

C++ 操作	x86-64 上常见形状	审查重点
relaxed load	<code>movreg,[addr]</code>	只读 <code>atomic</code> 自身，不发布其他数据
acquire load	常也可能是 <code>movreg,[addr]</code>	是否读到对应 <code>release</code> 发布值
release store	常也可能是 <code>mov[addr],reg</code>	发布前普通写是否完整，读取方是否 <code>acquire</code>
relaxed fetch_add	<code>lockadd/xadd[addr],reg</code>	热点 <code>line</code> 迁移，是否需要分片
compare_exchange	<code>lockcmpxchg[addr],reg</code>	成功/失败内存序、失败路径和责任转移
seq-cst fence 或 强序需求	可能出现 <code>fence</code> 或 <code>locked op</code>	是否真的需要单一全局顺序

这张表不能当作编译器保证。它的作用是建立审查顺序：先用 C++ 语义判断同步边是否正确，再用反汇编确认编译器生成了预期形状，最后用性能证据判断硬件成本。把顺序反过来很危险：只因为看到 `mov` 就删掉 `acquire`，代码可能在别的架构上坏；只因为看到 `lock xadd` 就说原子一定慢，也可能忽略它在低频路径上完全可接受。

CAS 汇编暴露了失败路径 CAS 的典型机器形状也能帮助审查协议：

Listing 13.1 CAS 状态转换的典型汇编形状

```
; eax holds expected old state
; edx holds desired new state
lock cmpxchg dword ptr [rdi], edx
jne .Lfailed
; success path owns the transition
```

`cmpxchg` 失败时，机器会把观察到的旧内存值写回 `eax`。这和 C++ `compare_exchange` 失败后更新 `expected` 的语义对应。失败不是无信息的“没抢到”，而是“看到了别的状态”。若失败后要读取赢家发布的数据，失败内存序不能随意写成 `relaxed`；若失败后只重试，才可能更弱。汇编里的 `jne .Lfailed` 提醒我们：失败路径是协议的一半，不能只写成功路径。

缓存一致性不是语言同步 缓存一致性协议通常以 `cache line` 为单位维护状态。某条线可以被多个核心共享读取；一旦某个核心要写，它必须让其他副本失效或通过目录协议取得独占权限。这个机制解决的是“同一地址最终不能同时存在两个互相矛盾的写事实”。它没有解决三件事：不同地址之间是否存在全局顺序，普通字段是否按源代码顺序对另一个线程可见，读取方拿到一个指针后对象是否仍然存活。也就是说，一致性给 `atomic` 的单对象修改提供硬件基础，但 C++ 仍需要用 `happens-before` 描述跨对象的同步关系。

把发布配置的例子放回这个模型中，`ready` 和 `RuntimeConfig` 字段很可能落在不同 `cache line`。生产者写配置字段时取得的是那些字段所在 `cache line` 的写权限；随后写 `ready` 时取得的是 `ready` 所在线的写权限。消费者看到 `ready` 的新值，只说明它观察到了 `ready` 所在线的某次修改；若没有 `release/acquire`，语言层面没有要求“配置字段的普通写入必须随 `ready` 一起成为可依赖事实”。这就是 `atomic flag` 和发布协议之间的差别。

Store buffer 为什么会改变直觉 核心执行 `store` 时，通常不必等写入立刻对所有核心可见。写入先进入 `store buffer`，后续指令可以继续执行；本核心读取同一地址时还可能从 `store buffer` 转发。这个设计隐藏了写延迟，却让“我已经执行了 `store`，所以别人立刻能看到”变成错误直觉。`release` 操作不是把世界冻结，而是给编译器和硬件加上顺序约束：发布点之前的写入不能在协议意义上跑到发布之后；与之匹配的 `acquire` 读到这个发布值时，获取方后续读取才能依赖发布方此前写入的事实。

编译器也是并发参与者 即使硬件恰好按直觉执行，编译器仍可能根据单线程语义优化普通访问。它可以把循环中的普通标志读取提升出循环，可以把相邻写入合并，可以把寄存器里的值复用到后续表达式。数据竞争被定义为未定义行为，正是为了给这些优化留下空间。Atomic 访问告诉编译器：这个对象可能被其他线程以受控方式观察，不能按普通变量那样任意消除或重排。内存序则进一步告诉编译器和硬件，这次 atomic 是否还要约束周围普通访问。

Atomic 解决什么，不解决什么 `std::atomic<T>` 首先保证这个 atomic 对象本身的并发访问没有数据竞争，读写或读改写不可撕裂。它可以参与同步，建立发布和获取关系，也可以作为 CAS 状态机的线性化点。但它不自动保护多个变量之间的不变量，不自动让普通字段可见，不自动管理指针指向对象的生命周期，也不自动让等待者睡眠和醒来。

因此，原子化之前先问四个问题。第一，这个 atomic 的线性化含义是什么：计数、状态、指针、索引、版本，还是引用计数。第二，它是否发布其他普通数据。第三，读取者拿到值后，关联资源如何保持存活。第四，它是否会被当成等待条件，如果会，长时间等待靠忙等、`atomic::wait`、条件变量还是运行时调度。

三种最常见的正确小协议 原子使用先限制在三类小协议中。指标协议：多个线程递增计数，读取者只看数值，不借计数同步其他数据。发布协议：生产者先写普通数据，再 release 发布标志或指针；消费者 acquire 看到该发布后读取普通数据。CAS 状态机：多个线程竞争一个状态转换，CAS 成功的线程获得责任，失败线程根据新状态重新决策。三类协议的边界不同，不应因为都用了 `std::atomic` 就混在一起。

原子用途	典型内存序	必须写清	不适合做什么
指标计数	relaxed	只关心自身数值	发布普通数据
对象发布	release/acquire	发布了哪些字段，谁负责回收	热更新回收无协议
CAS 状态机	acq_rel / acquire / relaxed	成功点、失败路径、责任转移	循环内做外部副作用
索引/槽位	release/acquire 或 relaxed 组合	单写者是谁，槽位何时可读写	多生产者随意共享
引用计数	分阶段选择	最后释放同步和对象存活条件	保护对象内部字段

把原子协议落成三张账本 审查一段原子代码，至少要同时写三张账本。第一张是语言账本：哪些普通访问被哪个同步边保护，是否存在数据竞争，happens-before 从哪里到哪里。第二张是硬件账本：哪个 cache line 被哪些核心读写，是否有读改写写，store buffer 和一致性所有权是否可能主导成本。第三张是生命周期账本：读取到的状态、指针、索引或句柄，在使用期间是否仍然有效，谁负责释放，失败路径是否也释放。

Listing 13.2 原子审查的三张账本

```
language ledger:
  ordinary writes before release
  acquire load observes the released value
  ordinary reads after acquire
  no unsynchronized ordinary read/write pair

hardware ledger:
  atomic object address and cache line
  writers and readers by core or worker role
  read-modify-write frequency
```


expected contention and false sharing risk

lifetime ledger:

- value represents counter, state, pointer, or index
- owner after successful transition
- valid actions after failed transition
- reclamation or reuse rule

三张账本缺一张，代码都可能出问题。只有语言账本，可能同步正确但热点 `atomic` 把吞吐打穿；只有硬件账本，可能把普通变量并发读写写成未定义行为；只有生命周期账本，可能对象活着但发布顺序不成立。原子代码之所以难，不是 API 多，而是它把语言、硬件和对象生命周期压在一两行表达式里。

完整原子账本：从一条 `fetch_add` 到协议变量 原子变量最容易被滥用在计数器上，因为 `fetch_add` 看起来像普通的 `++`。先把一条 `fetch_add` 放到机器路径里。x86-64 上，热点递增常见形状是带 `lock` 前缀的 `xadd` 或 `add`；核心要取得目标 cache line 的可修改所有权，把旧值读出，计算新值，再把新值写回。若八个 worker 都在更新同一个计数器，这条 line 会在核心之间来回迁移。操作仍然是正确的，但吞吐可能被互连和一致性协议限制，而不是被加法本身限制。

Listing 13.3 原子递增的典型机器证据

```
; rdi = address of atomic counter
mov     eax, 1
lock xadd dword ptr [rdi], eax
; eax now contains the old value
```

`fetch_add(relaxed)` 适合做 telemetry：例如完成了多少个 chunk、丢弃了多少条坏记录、某个调试计数器当前是多少。它不适合做发布许可。下面这个反例很典型：

Listing 13.4 relaxed 完成计数不是读取结果的许可

```
1 struct ChunkResult {
2     std::uint64_t checksum = 0;
3     std::uint64_t records = 0;
4 };
5
6 std::vector<ChunkResult> results;
7 std::atomic<std::uint32_t> completed_count{0};
8
9 void worker(std::uint32_t chunk_id) {
10     results[chunk_id].checksum = compute_checksum(chunk_id);
11     results[chunk_id].records = count_records(chunk_id);
12     completed_count.fetch_add(1, std::memory_order_relaxed);
13 }
14
15 void coordinator(std::uint32_t chunk_count) {
16     while (completed_count.load(std::memory_order_relaxed) != chunk_count) {
17     }
18     consume(results); // 这里没有可移植同步保证。
19 }
```


`completed_count` 的数值能告诉协调者“某些 `atomic` 递增发生过”，却没有建立“所有 `results[i]` 的普通写入都对当前线程可见”的发布关系。修复方式取决于真正的合同。若只需要等待所有 `worker` 结束，`join` 可以提供同步。若 `worker` 在线程池中不能 `join`，每个 `chunk` 可以有 `release` 状态位，协调者 `acquire` 读取每个状态位；也可以把结果写入受 `mutex` 保护的队列；还可以用任务运行时的完成状态发布结果。不能用一个 `relaxed` 计数器替代所有发布边。

Listing 13.5 每个 `chunk` 用状态位发布自己的结果

```
1 enum class ChunkState : std::int32_t {
2     Empty,
3     Ready,
4     Failed,
5 };
6
7 struct PublishedChunk {
8     ChunkResult result;
9     std::atomic<ChunkState> state{ChunkState::Empty};
10 };
11
12 void publish_chunk(PublishedChunk& chunk, ChunkResult result) {
13     chunk.result = result;
14     chunk.state.store(ChunkState::Ready, std::memory_order_release);
15 }
16
17 std::optional<ChunkResult> try_read_chunk(const PublishedChunk& chunk) {
18     if (chunk.state.load(std::memory_order_acquire) != ChunkState::Ready) {
19         return std::nullopt;
20     }
21     return chunk.result;
22 }
```

这里的线性化点不是 `result` 赋值，而是 `release store` 把状态改成 `Ready`。读取方只有 `acquire` 读到 `Ready`，才有权读取此前发布的普通字段。状态位和结果对象的生命周期也要一起审查：`try_read_chunk` 返回时，`chunk` 仍必须存活，不能被生产者复用为下一轮结果。若需要复用槽位，就必须引入 `epoch` 或 `generation`，防止读取方把上一轮 `Ready` 当成下一轮 `Ready`。

协议变量必须写角色 审查原子代码时，不要只看变量类型，要先给每个原子变量写角色。指标计数、发布标志、状态机、指针、引用计数和等待条件的责任不同。一个变量承担多个角色时，要把每个角色的内存序和生命周期都写清，否则迟早出现“这个 `relaxed` 到底能不能读结果”的争论。

Listing 13.6 `AtomicProtocolNote` 字段

```
atomic.name
role = metric | publication | state | pointer | refcount | wait
linearization_point
published_fields
acquire_readers
lifetime_owner
reuse_or_epoch_rule
contention_shape
success_order
```

```
failure_order
wait_strategy
diagnostic_counters
```

`role=metric` 时, `published_fields` 应为空, 因为指标不发布普通数据。`role=publication` 时, 必须列出哪些字段在 `release` 前写好、哪些读取者通过 `acquire` 获取。`role=state` 时, 必须说明状态转移图和 CAS 成功者获得什么责任。`role=pointer` 时, 生命周期比内存序更危险: 读取方 `acquire` 得到指针后, 对象如何防止被释放或复用。若这张 `note` 写不出来, 代码通常还没到该写 `lock-free` 的阶段。

一条 CAS 同时包含成功合同和失败合同 CAS 常被写成“抢到就干活, 没抢到就重试”。这只描述了控制流, 没有描述同步合同。成功 CAS 可能同时读取旧状态、发布新状态并接管资源; 失败 CAS 可能只是发现别人改了状态, 也可能需要读取赢家发布的数据。C++ 因此要求 CAS 写成功内存序和失败内存序。失败内存序不能比成功内存序更强, 也不能是 `release`, 因为失败路径没有写入目标对象。

Listing 13.7 CAS 状态机必须说明失败路径要不要读取赢家数据

```
1 enum class CompletionState : std::int32_t {
2     Pending,
3     Succeeded,
4     Failed,
5 };
6
7 struct Completion {
8     Result result;
9     Error error;
10    std::atomic<CompletionState> state{CompletionState::Pending};
11 };
12
13 bool try_publish_success(Completion& completion, Result result) {
14     CompletionState expected = CompletionState::Pending;
15     completion.result = result;
16     return completion.state.compare_exchange_strong(
17         expected,
18         CompletionState::Succeeded,
19         std::memory_order_release,
20         std::memory_order_acquire);
21 }
```

这段代码中, 成功路径用 `release` 发布 `result`。失败路径用 `acquire` 的理由是: 若失败时 `expected` 被更新成 `Succeeded`, 调用者可能接着读取已经由赢家发布的 `result`; 若失败后只是丢弃并重试, 失败内存序可以更弱。内存序选择必须来自失败路径是否消费赢家事实, 而不是来自“CAS 看起来重要”。

原子等待: 状态值和睡眠队列仍然分开 C++20 的 `atomic::wait` 让原子变量也能进入等待协议。它的底层可以使用 `futex` 或运行库等待结构, 避免长时间忙等。但它没有改变核心原则: 状态在原子值里, 通知只是让等待者重新读取状态。写线程必须先改变原子值, 再调用 `notify_one` 或 `notify_all`; 等待线程必须在循环中检查目标值, 不能把 `notify` 当成消息。

Listing 13.8 `atomic::wait` 仍然等待状态变化

```
1 void wait_until_ready(std::atomic<ChunkState>& state) {
```

```
2  ChunkState observed = state.load(std::memory_order_acquire);
3  while (observed == ChunkState::Empty) {
4      state.wait(observed, std::memory_order_relaxed);
5      observed = state.load(std::memory_order_acquire);
6  }
7 }
8
9 void publish_ready(std::atomic<ChunkState>& state) {
10     state.store(ChunkState::Ready, std::memory_order_release);
11     state.notify_all();
12 }
```

`state.wait(observed)` 的含义是“当 `state` 仍等于 `observed` 时可以睡眠”。醒来后必须重新 `acquire` 读取状态，因为唤醒可能来自真正发布、虚假唤醒、其他状态变化或竞争窗口。若状态发布普通数据，最终读取到 `Ready` 的 `load` 必须有 `acquire` 语义；`wait` 本身不能替代这条获取边。

Memory order 是约束，不是动作名称 `memory_order_release` 这个名字容易让人误以为它“释放 `cache`”或“刷新内存”。更准确的理解是：它给编译器和硬件施加一个发布约束，让发布点之前的写入在与 `acquire` 配对时成为可依赖事实。`memory_order_acquire` 不是“获取锁”，而是在读取到对应发布值后约束后续访问。`memory_order_relaxed` 不是“可能乱写”，它仍保证 `atomic` 对象自身的原子性和修改顺序，只是不建立跨对象同步边。

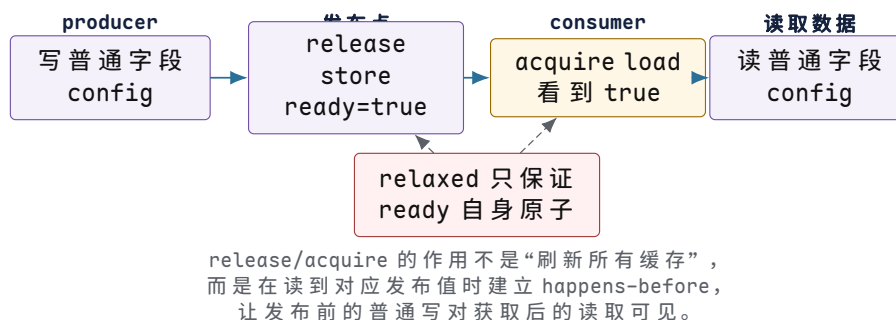
内存序	约束的对象	不能误解成什么
relaxed	该 <code>atomic</code> 自身的原子读写和修改顺序	不能发布普通字段，不能保护多字段不变量
release	发布点之前的写不能越过发布语义	不是强制把所有 <code>cache line</code> 写回内存
acquire	获取点之后的读写不能越过获取语义	不是读取任意旧 <code>release</code> 都能同步，必须读到对应值
acq_rel	一次读改写写同时获取旧状态并发布新状态	仍不管理对象回收和等待唤醒
seq_cst	seq-cst 原子参与单一全局顺序	不是自动修复数据竞争或 ABA

这张表的目的是防止错误类比。内存序约束的是可观察顺序和同步关系；`cache line` 所有权决定的是某次读写怎样在硬件上传播；对象生命周期决定的是值背后的资源是否仍然有效。三者相交但不等价。

13.2 内存模型的四个边界

原子操作必须把四件事分开。原子性回答“这个变量自身的读写是否不可分割、是否有数据竞争”。可见性回答“一个线程写的普通字段，另一个线程什么时候能看到”。顺序性回答“编译器和硬件能否重排，其他线程能观察到什么顺序”。生命周期回答“读取方拿到指针、引用或状态后，对象是否还活着”。很多错误来自用原子性去替代后三个问题。

发布协议：ready 标志必须同步普通字段



图示：本图要证明的命题是：atomic flag 本身没有数据竞争，不代表它同步了旁边的普通字段。

Happens-before 不是物理时间 生产者在物理时间上先写 data，消费者在物理时间上后读 data，不代表 C++ 程序有同步关系。并发正确性看的是 happens-before。Mutex 的 unlock 与后续 lock 可以建立同步；release store 与读到它的 acquire load 可以建立同步；线程 join 也有同步语义。没有这些关系，编译器和硬件有自由度，测试顺序不能替代语言保证。

Acquire 和 Release 有方向 release 用在发布者，把之前的普通写放在发布点之前；acquire 用在消费者，读到发布值后，让之后的读写不能越过获取点。把消费者的 store 写成 release，不能让它看到生产者的数据；把生产者的 load 写成 acquire，也不能发布数据。内存序必须放在建立同步关系的那次读写上。

13.3 用反例推导内存序

内存序最容易学错的方式，是先背枚举，再给每个枚举找例子。更可靠的路线是先构造反例：如果没有这条同步边，会出现什么坏结果。每个内存序选择都应回答一个反例。若回答不出来，要么这个 atomic 只是指标，可以 relaxed；要么协议还没有写清，不应急着用原子。

消息传递反例 最小发布协议是 message passing。生产者先写普通数据，再发布 ready；消费者看到 ready 后读取普通数据。错误版本把 ready 写成 relaxed，测试在 x86 上可能长期通过，但 C++ 语义没有保证消费者看到 data 的新值。

Listing 13.9 错误反例：relaxed ready 没有发布普通数据

```

1 struct Message {
2     std::int32_t value = 0;
3 };
4
5 Message message;
6 std::atomic<bool> ready{false};
7
8 void producer() {
9     message.value = 42;
10    ready.store(true, std::memory_order_relaxed);
11 }
12
13 void consumer() {
14     while (!ready.load(std::memory_order_relaxed)) {
15     }
16 }

```

```

16 // C++ 语义上不能保证这里看到 message.value == 42。
17 use(message.value);
18 }

```

修复不是把所有操作都改成 `seq_cst`，而是在真正传递事实的那条边上建立同步：生产者 `release` 存储 `ready=true`，消费者 `acquire` 读取到 `true`。这样发布前的 `message.value=42` happens-before 获取后的读取。这个例子也说明：同步关系绑定在“读到那次发布的值”上；消费者 `acquire` 读到 `false`，不代表它已经获取了消息。

Store buffering 反例 另一个经典反例是两个线程分别写自己的变量，再读对方变量。若都使用 `relaxed`，两个线程都可能读到旧值 0。这个结果不违反 `relaxed`，因为 `relaxed` 只保证每个 `atomic` 自身有修改顺序，不保证跨变量全局顺序。

Listing 13.10 Store buffering: 两个 `relaxed` 读都看到旧值并不矛盾

```

1 std::atomic<std::int32_t> x{0};
2 std::atomic<std::int32_t> y{0};
3
4 // Thread A
5 x.store(1, std::memory_order_relaxed);
6 const std::int32_t observed_y = y.load(std::memory_order_relaxed);
7
8 // Thread B
9 y.store(1, std::memory_order_relaxed);
10 const std::int32_t observed_x = x.load(std::memory_order_relaxed);

```

若协议要求不允许 `observed_x==0&&observed_y==0`，需要更强的同步设计。可能是 `seq_cst`，也可能是引入一个锁、一个阶段 `barrier`、一个单独发布点或一个消息队列。内存序不是局部优化选项，而是协议的一部分；只把其中一个 `load` 改成 `acquire` 并不能凭空建立同步，因为没有对应 `release` 被它读到。

Load buffering 反例 `Load buffering` 把顺序反过来：两个线程先读对方变量，再写自己的变量。若都使用 `relaxed`，两个线程都读到 0 更容易理解，因为两个 `load` 都可能在两个 `store` 之前发生。这个反例的价值不在于结果奇怪，而在于它提醒一件事：不是每组原子读写都构成通信。两个线程各自“看一眼再宣布自己完成”，并没有建立任何跨线程承诺。

Listing 13.11 Load buffering: 先观察再发布不能互相证明

```

1 std::atomic<std::int32_t> x{0};
2 std::atomic<std::int32_t> y{0};
3
4 // Thread A
5 const std::int32_t observed_y = y.load(std::memory_order_relaxed);
6 x.store(1, std::memory_order_relaxed);
7
8 // Thread B
9 const std::int32_t observed_x = x.load(std::memory_order_relaxed);
10 y.store(1, std::memory_order_relaxed);

```

如果业务语义是“至少有一个线程必须看到另一个线程已经到达”，这段协议就不够。修复可能是 `barrier`、`mutex`、`condition variable`、`latch`，或一个明确的阶段状态机。`Acquire/release` 也不是随便加上就好：`acquire` 必须读取到某次 `release`

写出的值，才能建立同步边。读到初始 0 的 acquire load 没有获取到对方后续 store 的任何事实。

IRIW：多个观察线程也要写进协议 IRIW 可以理解为 independent reads of independent writes。两个写者分别写 `x` 和 `y`，两个观察线程以相反顺序读取它们。在弱内存系统上，如果没有足够同步，两个观察线程可能对两个写入的相对顺序形成不同观察。这个反例说明：即使每个 `atomic` 自身有修改顺序，也不代表多个 `atomic` 之间自动有所有线程一致认同的全局顺序。

Listing 13.12 IRIW：不同观察者可能形成不同跨变量顺序

```

1 std::atomic<std::int32_t> x{0};
2 std::atomic<std::int32_t> y{0};
3
4 // Writer A
5 x.store(1, std::memory_order_relaxed);
6
7 // Writer B
8 y.store(1, std::memory_order_relaxed);
9
10 // Reader C
11 const auto c1 = x.load(std::memory_order_relaxed);
12 const auto c2 = y.load(std::memory_order_relaxed);
13
14 // Reader D
15 const auto d1 = y.load(std::memory_order_relaxed);
16 const auto d2 = x.load(std::memory_order_relaxed);

```

若系统报告只把 `x` 和 `y` 当独立指标，观察者看到不同快照也许可以接受；若它们共同描述一个状态机，例如 `route_epoch` 和 `manifest_generation`，就不能用两个 `relaxed` 指标拼出提交事实。要么用一个锁保护快照，要么用 `seqlock/versioned snapshot`，要么把两个字段合并进单一发布对象。跨变量一致快照是协议目标，不是 `relaxed atomic` 自动提供的附赠能力。

发布链和中间状态 Release/acquire 可以通过一条清楚的发布链传递普通数据，但链条不能断。生产者写数据后 `release` 存 `ready=1`；中间线程 `acquire` 读到 `ready=1` 后，经过验证再 `release` 存 `validated=1`；消费者 `acquire` 读到 `validated=1` 后，可以看到生产者发布前的数据以及中间线程发布前的验证结果。这是同步传递，不是缓存刷新。

Listing 13.13 发布链的语义账本

```

producer:
    write payload
    ready.store(1, release)

validator:
    if ready.load(acquire) == 1:
        verify payload
        validated.store(1, release)

consumer:
    if validated.load(acquire) == 1:
        read payload and validation result

```

链条的每一段都要读到上一段 release 写出的值。若 validator 用 relaxed 读取 `ready`，它没有获取 `payload`；若 consumer acquire 读取的是 `validated=0`，它也没有获取验证结果；若 `payload` 在发布后又被无同步修改，则发布链不再保护后续修改。内存序选择必须绑定到状态机上的每个责任转移点。

从反例到审查句 审查原子代码时，可以强制写一句反例句：

1. 若这个 load 读到旧值，会不会破坏业务语义？
2. 若这两个指标来自不同时间点，报告是否仍然可接受？
3. 若 CAS 失败，失败路径是否需要看到赢家发布的数据？
4. 若读取方拿到指针后写者释放对象，生命周期由谁保护？
5. 若等待者睡眠，谁负责唤醒，状态变化是否可观察？

能用反例句解释的内存序，才是可维护的内存序。否则，项目应退回锁版 reference 或把状态机拆窄。

13.4 证明工具：happens-before 图和线性化点

原子代码不能靠“看起来没问题”通过审查。最小证明要有两张图：一张 happens-before 图，说明普通数据怎样通过同步边被看见；一张线性化点图，说明并发操作在什么瞬间对外生效。前者防止读到未发布数据，后者防止两个线程都以为自己完成了同一件事。若代码无法画出这两张图，通常说明协议还没写清。

Happens-before 图从普通字段开始 画图时先标普通字段，而不是先标 atomic。假设 producer 构造 `payload`，然后 release 存 `ready=1`；consumer acquire 读到 `ready=1` 后读取 `payload`。图中有三类边：线程内顺序边、同步边、由同步推出的 happens-before 边。

Listing 13.14 message passing 的 happens-before 图

```
producer thread:
  A: payload.value = 42
  B: ready.store(1, release)

consumer thread:
  C: r = ready.load(acquire)  // reads value written by B
  D: use(payload.value)

edges:
  A sequenced-before B
  B synchronizes-with C
  C sequenced-before D

conclusion:
  A happens-before D
```

这张图同时说明两个边界。若 C 没有读到 B 写的值，就没有这条同步边；若 `payload` 在 B 之后又被无同步修改，A 到 D 的结论不能保护后续修改。Happens-before 图必须绑定具体值和具体状态转换，不能只写“这里用了 acquire”。

手算一次 message passing 把上面的图变成逐步审查。初始状态：

Listing 13.15 message passing 初始状态

```
payload.value = 0
ready = false
```

生产者执行：

```
A: payload.value = 42
B: ready.store(true, release)
```

消费者执行：

```
C: while (!ready.load(acquire)) {}
D: read payload.value
```

若 C 的某次 acquire load 读到 `false`，它只说明还没观察到发布值，不能读取 `payload`。若 C 读到 B 写入的 `true`，C 与 B 建立 `synchronizes-with`；再加上 A 在线程内先于 B，C 在线程内先于 D，于是 A happens-before D。结论是：D 读取 `payload` 时必须看到 A 或 A 之后在同一同步保护下的写入。

如果 B 改成 `relaxed`，C 即使用 `acquire load` 读到 `true`，也没有对应 `release` 可同步；如果 C 改成 `relaxed`，B 的 `release` 没有被 `acquire` 获取；如果 A 被移到 B 之后，`release` 只发布 B 之前的写，D 仍然没有理由看到 A。内存序不是变量装饰，而是语句位置和读取值共同决定的边。

错误改动	缺失的边	可能后果
B 用 <code>relaxed</code>	<code>release</code> 不存在	<code>ready</code> 为 <code>true</code> 但 <code>payload</code> 未发布
C 用 <code>relaxed</code>	<code>acquire</code> 不存在	读到 <code>ready</code> 也不能依赖 <code>payload</code>
A 在 B 之后	发布点太早	发布的是旧 <code>payload</code> 状态
C 读到 <code>false</code>	没读到发布值	不能建立同步边

SPSC 槽位状态的最小协议 单生产者单消费者队列比 `message passing` 多一个问题：槽位会复用。一个槽位不能只有 `ready`，还要有 `generation` 或者严格的 `head/tail` 规则，避免消费者把上一轮 `ready` 当成下一轮 `ready`。

Listing 13.16 SPSC 槽位的教学协议

```
1 enum class SlotState {
2     Empty,
3     Ready
4 };
5
6 template <class T>
7 struct SpscSlot {
8     T value;
9     std::atomic<SlotState> state{SlotState::Empty};
10 };
11
12 template <class T>
13 void produce(SpscSlot<T>& slot, const T& value)
14 {
15     while (slot.state.load(std::memory_order_acquire) != SlotState::Empty) {}
16 }
```

```

17 slot.value = value;
18 slot.state.store(SlotState::Ready, std::memory_order_release);
19 }
20
21 template <class T>
22 T consume(SpscSlot<T>& slot)
23 {
24     while (slot.state.load(std::memory_order_acquire) != SlotState::Ready) {
25     }
26     T value = slot.value;
27     slot.state.store(SlotState::Empty, std::memory_order_release);
28     return value;
29 }

```

生产者写 `value` 后 `release` 发布 `Ready`；消费者 `acquire` 读到 `Ready` 后才能读 `value`。消费者读完 `value` 后 `release` 发布 `Empty`；生产者 `acquire` 读到 `Empty` 后才能覆盖 `value`。这是一条双向协议，不是一个 `relaxed flag`。若扩展到环形队列，还要把每个槽位的 `generation`、`head/tail` 单写者、`wrap-around` 和析构责任写进协议。

double checked locking 的真正边界 双重检查锁定常被写错：

Listing 13.17 危险形状：指针可见不代表对象构造已发布

```

if (ptr.load(relaxed) == nullptr) {
    lock(m);
    if (ptr.load(relaxed) == nullptr) {
        ptr.store(new Object(...), relaxed);
    }
    unlock(m);
}
use(*ptr.load(relaxed));

```

问题不是“用了两次 `if`”，而是指针发布和对象构造之间缺少可移植同步。正确设计通常有三个选择。第一，直接用 `std::call_once`，把复杂性交给库；第二，用 `mutex` 覆盖读取和构造，牺牲一点读路径成本；第三，用 `release store` 发布完整构造后的指针，读取方 `acquire load` 获取指针，并且对象生命周期必须保证不被提前释放。

Listing 13.18 原子指针发布至少要写清的合同

```

writer:
    construct object completely
    published.store(ptr, release)

reader:
    ptr = published.load(acquire)
    if ptr != null:
        object fields published before release are readable

lifetime:
    object is never deleted until all readers are stopped
    or protected by shared_ptr, epoch, hazard pointer, or RCU

```

Acquire/release 只能发布构造完成事实，不能解决释放时机。很多 DCL bug 不是读到半构造对象，而是热更新后旧对象被释放，读者仍持有裸指针。

ABA 是生命周期问题，不只是 CAS 问题 Treiber stack 之类结构常见 ABA：线程 A 读取 head 为节点 X，准备 CAS；线程 B pop X，再 push X 或复用同一地址；线程 A 的 CAS 看到 head 仍是 X，于是误以为状态没变。CAS 比较的是位模式，不知道对象是否经历过出栈、释放和复用。

Listing 13.19 ABA 的时间线

```
T1: old = head.load()           // old == X
T1: next = old->next

T2: pop X
T2: free or reuse X
T2: push X at same address

T1: CAS(head, old, next) succeeds by address equality
    but logical stack changed
```

解决思路包括 tagged pointer、hazard pointer、epoch based reclamation、引用计数或退回锁。选择哪一种，要看对象复用频率、读者数量、延迟边界和实现复杂度。单纯把 CAS 内存序改成 seq_cst 不能解决 ABA，因为问题不在排序，而在对象身份和生命周期。

MemoryModelReport 模板 任何非平凡原子协议都应附一份报告：

Listing 13.20 MemoryModelReport 最小字段

```
shared_objects
ordinary_fields_protected
atomic_protocol_variables
for_each_atomic:
    role: metric | publication | state | pointer | slot
    writers
    readers
    success_order
    failure_order_if_CAS
    published_fields
    lifetime_rule
happens_before_edges
linearization_points
allowed_stale_observations
forbidden_observations
fallback_lock_reference
tests:
    litmus
    stress
    sanitizer
    linearization_log
```

这份报告不是形式主义。它能直接暴露问题：若 published_fields 为空，这个 atomic 也许只是 relaxed 指标；若 lifetime_rule 写不出来，不能发布裸指针；若 forbidden_observations 不明确，就无法判断是否需要 acquire/release

或 seq-cst。

线性化点回答操作何时生效 线性化点是一个并发操作在逻辑上生效的瞬间。对 mutex 保护的队列, push 的线性化点通常在锁内状态更新完成; 对 CAS 状态机, 成功 CAS 往往就是责任转移点; 对 completion, manifest append 成功才是业务提交点。线性化点不是性能术语, 而是正确性合同。

Listing 13.21 TaskCompletion 的线性化点账本

```
state Pending:
    no result is visible

complete(result):
    prepare result object
    CAS state Pending -> Completing
    linearization point for winning producer
    publish result pointer
    store state Completed with release
    wake waiters

wait():
    acquire state Completed
    read result pointer
```

这份账本能抓住常见 bug。若 CAS 成功前就把 result 暴露给其他线程, 失败线程可能看到半成品; 若 CAS 成功后构造 result 可能抛异常, 状态就进入无人能修复的中间态; 若 wake 发生在发布前, 等待者可能醒来却读不到结果。线性化点要求状态转换和副作用顺序都能解释。

失败路径也要画出来 CAS 循环的失败路径不是空白。失败 load 读到了赢家写出的状态, 它可能需要 acquire 才能看到赢家发布的数据; 也可能只需要知道“我输了”, 不读任何普通字段, 此时 relaxed 或 weaker failure order 可能足够。失败路径是否获取数据, 要写成协议。

Listing 13.22 CAS 失败路径的两种语义

```
case 1: failure only retries
    compare_exchange_weak(..., success=acq_rel, failure=relaxed)
    failed thread reads no data published by winner

case 2: failure returns winner result
    compare_exchange_strong(..., success=release, failure=acquire)
    failed thread reads state/result written by winner
```

错误经常出在第二种场景: 失败线程看到状态不是 Pending, 就直接读取 result; 但 failure order 使用 relaxed, 没有获取赢家发布的 result。审查时要强制回答: “CAS 失败后会不会读赢家发布的数据?” 答案决定失败内存序。

Litmus test 是小模型, 不是压力测试 压力测试能发现一些 bug, 但不能证明没有 bug。Litmus test 用极小程序枚举允许结果, 帮助判断协议是否依赖偶然硬件强度。Store buffering、message passing、IRIW、Dekker 这类小模型应该保留在项目测试或文档里。它们的价值是把“这台机器没复现”转换成“这个结果在模型上是否允许”。

Listing 13.23 并发协议审查的最小证明包

```
AtomicProtocolNote:
  state variables
  ordinary data protected or published
  linearization point for each operation
  happens-before graph for publication
  CAS failure semantics
  object lifetime owner
  wait/wake condition
  litmus tests that would fail if order is weakened
```

这份证明包比“所有原子都用 seq-cst”更有用。Seq-cst 可以减少某些跨变量顺序风险，但不能自动保护生命周期、ABA、等待谓词或对象回收。证明包迫使代码说明每个普通字段由谁发布、每个状态由谁推进、每个等待者靠什么醒来。

TSAN 和模型证明的边界 ThreadSanitizer 能发现许多普通数据竞争，但它不会证明协议正确。一个全由 atomic 组成的错误状态机可能没有数据竞争，却仍然会丢任务、重复提交或读到已回收对象。反过来，某些低层无锁结构可能使用 TSAN 难以理解的同步模式，需要额外注解或专门测试。工具是证据，不是证明本身。

成熟流程可以分层：锁版 reference 用普通单元测试保证语义；原子版用 TSAN 压力测试找显性数据竞争；litmus test 和小状态枚举检查内存序；线性化点审查检查责任转移；生命周期测试检查对象回收。缺一层，报告就要写明未验证边界。

13.5 从锁到原子的迁移步骤

原子协议最危险的写法，是直接把锁版代码里的字段逐个替换成 `std::atomic`。锁版临界区通常保护的是多字段不变量，而原子操作一次只线性化一个对象或一个窄状态转换。迁移必须先把状态机拆成可以独立证明的小协议，再决定每个协议的线性化点和内存序。

第一步：保留锁版 reference 锁版实现通常更容易检查：所有状态转换在 mutex 下完成，condition variable 表达等待谓词，关闭和析构路径可测试。迁移前保留锁版 reference，用相同测试、压力输入和故障注入比较。若原子版输出不同，先修语义，不解释性能。原子版不是替代正确性证明的工具，它只是把某些同步边界变窄。

第二步：找单点责任转移 适合 CAS 的地方，通常有一个明确的责任从“无人拥有”转移到“某个线程拥有”。例如 task 状态从 Ready 到 Running，只有 CAS 成功者能执行 task；future 状态从 Pending 到 Completed，只有成功者能发布结果；队列槽位从 Empty 到 Reserved，只有成功生产者能写数据。若一个状态转换需要同时修改多个容器、多个计数和多个等待队列，先不要用单个 CAS 硬压。

Listing 13.24 CAS 协议要先写责任转移

```
state Pending:
  no result is visible
  many threads may attempt completion

CAS Pending -> Completing:
  exactly one thread wins
  winner owns result construction and publication

state Completed:
  result and error summary are visible
  waiters may acquire and read
```

第三步：把失败路径也写成协议 CAS 失败不是“再试一下”这么简单。失败线程读到了另一个线程写入的新状态，它可能需要根据新状态决定等待、帮助完成、返回已经完成的结果、释放临时资源或报告重复 attempt。失败 load 的内存序也要匹配语义：若失败后需要读取赢家发布的数据，失败内存序至少要 acquire；若失败后只重试，不读取关联数据，可以更弱。失败路径不写清，CAS 循环很容易在正确性、性能和资源释放上出错。

Listing 13.25 CAS 失败路径的语义草图

```
1 State expected = State::Pending;
2 if (state.compare_exchange_strong(expected,
3                                   State::Completing,
4                                   std::memory_order_acq_rel,
5                                   std::memory_order_acquire)) {
6     publish_result();
7     state.store(State::Completed, std::memory_order_release);
8 } else {
9     // expected now contains the observed state.
10    // If it is Completed, acquire failure ordering lets this path read result.
11    observe_or_wait_for(expected);
12 }
```

这个例子不是模板，而是提醒：成功序和失败序承担不同语义。成功者获得责任并发布结果；失败者不能继续执行副作用，必须根据观察到的状态走另一路。

第四步：等待不能靠无界自旋 短时间自旋可以避免睡眠唤醒成本，但无界自旋会占用 CPU、抢占其他 worker、增加互连流量。C++20 的 `atomic::wait` 可以在某些场景把“等待某个原子值变化”表达成阻塞等待；条件变量适合多字段谓词；运行时 continuation 适合 task 完成通知。选择等待机制前，先问等待时间分布、是否在 compute worker 内、是否会阻塞同池任务、是否需要超时和取消。

等待方式	适合场景	风险
短自旋	预计很快完成的低延迟状态变化	消耗 CPU，热点 line 反复读取
<code>atomic::wait</code>	单个原子状态变化，等待者多但谓词简单	只能等原子值，复杂谓词仍需锁或状态机
条件变量	多字段谓词、关闭和取消较复杂	需要 mutex 和正确通知协议
continuation	task 依赖和异步完成	运行时复杂，错误传播要清楚

第五步：生命周期独立证明 发布一个指针只解决“消费者能看到对象字段”，不解决“对象何时释放”。若对象发布后永不释放，协议简单；若会替换配置，需要引用计数、epoch、RCU、shared ownership 或停止世界切换；若节点从 lock-free stack 弹出后可能被其他线程仍然读取，需要 hazard pointer 或 epoch reclamation。生命周期协议往往比内存序更难，不能用 acquire load 一笔带过。

13.6 Relaxed: 只关心自身数值

`memory_order_relaxed` 保证原子对象自身不会数据竞争，不保证其他变量可见，不建立跨变量顺序。它适合统计计数、采样次数、CAS 失败次数、低风险指标和某些引用计数阶段。使用 relaxed 时，必须能说出一句话：读取这个数值的人不借它观察其他普通数据。

Listing 13.26 Relaxed 适合不发布其他数据的统计计数

```
1 class RequestStats {
2 public:
3     void record_success() {
4         this->success_count_.fetch_add(1, std::memory_order_relaxed);
5     }
6
7     std::uint64_t success_count() const {
8         return this->success_count_.load(std::memory_order_relaxed);
9     }
10
11 private:
12     std::atomic<std::uint64_t> success_count_{0};
13 };
```

这个类没有承诺读取 `success_count` 时能看到某个请求对象、错误对象或队列状态。它只承诺计数器自身不会丢更新。若后来有人把“计数增加”当作“对应结果对象已经发布”的信号，就发生了语义漂移。指标变量被升级成协议变量时，必须重新设计同步。

多指标快照可能不一致 监控线程分别 `relaxed` 读取 `accepted`、`completed` 和 `failed`，这些数值可能来自不同时间点。短时间内 `completed+failed` 小于或大于某个直觉关系，不一定是 bug，而是近似指标语义。如果业务不能接受这种不一致，需要锁保护快照、sequence lock、双缓冲、周期性汇总或单线程采样器。把每个计数都改成 `seq_cst`，也不会自动生成业务一致快照。

热点原子计数器的成本 `fetch_add` 不是免费。多个核心反复修改同一个 `atomic`，会争夺同一 `cache line` 的独占所有权，产生一致性流量。线程越多，热点越严重。高性能计数常用 `per-worker` 分片，写路径只写本地槽，读取时再汇总。它牺牲实时全局精确读取，换取写入热路径扩展性。第 9 章的 `WorkerSnapshot` 就应优先采用这种思路。

分片计数器的语义 分片计数器的本质是改变读取语义。写路径每个 `worker` 更新自己的分片，读路径汇总所有分片。这个设计适合吞吐指标、周期性报告和调试统计，不适合做“是否所有任务完成”的协议判断。若 `unfinished_count` 决定 `wait_idle` 是否返回，它必须有严格状态机；若 `processed_records` 只是监控曲线，它可以分片并允许近似。

计数用途	推荐设计	不能承诺什么
请求吞吐统计	per-worker relaxed 分片	读取瞬间的全局一致快照
CAS 失败次数	relaxed 累加或采样	与某次操作一一对应
未完成任务数	mutex 状态机或严格 atomic 协议	近似值不能驱动 <code>wait_idle</code>
内存预算字节	有界队列/信号量/锁保护账本	不能用滞后指标做容量判断
错误发布	result 状态 + release/acquire	单独错误计数不能发布错误对象

13.7 Release/Acquire：发布普通数据

最适合学习 `release/acquire` 的场景，是一次性发布不可变对象。生产者独占构造对象，写完所有普通字段，用 `release store` 发布指针或状态。消费者 `acquire load` 读到发布值后，读取对象字段。若对象会热更新或释放，还要额外生命周期协议；`release/acquire` 只解决“字段写入可见”，不解决“对象何时活着”。

Listing 13.27 一次性发布不可变配置

```

1 struct RuntimeConfig {
2     std::uint32_t worker_count = 0;
3     std::uint32_t queue_capacity = 0;
4     std::uint64_t version = 0;
5 };
6
7 class OneShotConfig {
8 public:
9     void publish(const RuntimeConfig* config) {
10         this->current_.store(config, std::memory_order_release);
11     }
12
13     const RuntimeConfig* wait_current() const {
14         const RuntimeConfig* config = nullptr;
15         while (config == nullptr) {
16             config = this->current_.load(std::memory_order_acquire);
17         }
18         return config;
19     }
20
21 private:
22     std::atomic<const RuntimeConfig*> current_{nullptr};
23 };

```

这个示例刻意把生命周期简化为“配置活到进程结束”。若 `RuntimeConfig` 发布后还会被修改，代码不够；若旧配置会被释放，代码也不够。发布协议要写清边界，否则很容易误以为 `acquire` 读到指针后，所有后续生命周期问题都自动消失。

槽位状态发布数据 队列槽位、SPSC ring 和任务完成对象都常用“先写数据，再发布状态”的协议。生产者写入槽位内容，然后 `release store` 状态为 `Ready`；消费者 `acquire load` 看到 `Ready` 后读取槽位内容。反过来，消费者处理完槽位后 `release` 发布 `Empty`，生产者 `acquire` 看到 `Empty` 后才能复用槽位。

Listing 13.28 槽位状态发布数据

```

1 inline constexpr std::size_t BUFFER_SLOT_CAPACITY = 16;
2
3 enum class BufferState : std::int32_t {
4     Empty,
5     Ready,
6 };
7
8 struct BufferSlot {
9     std::array<std::int32_t, BUFFER_SLOT_CAPACITY> values{};
10    std::atomic<BufferState> state{BufferState::Empty};
11 };
12
13 void publish_slot(BufferSlot& slot, std::span<const std::int32_t> input) {
14     const std::size_t limit = std::min(input.size(), slot.values.size());
15     for (std::size_t index = 0; index < limit; ++index) {
16         slot.values[index] = input[index];
17     }

```



```

18 slot.state.store(BufferState::Ready, std::memory_order_release);
19 }

```

这个例子只展示生产者发布。消费者必须用 `acquire` 读取 `state`，看到 `Ready` 后再读 `values`。如果先发布 `Ready` 再写 `values`，消费者可能读到未完成数据。如果使用 `relaxed` 读取 `Ready`，语言模型上没有建立普通字段的可见性关系。

版本化发布 发布配置时，版本号通常应放进不可变配置对象中，随指针一起发布。消费者拿到 `snapshot` 后记录 `version`，日志和 `trace` 才能说明某次请求使用哪一版。不要把指针和版本分成两个独立 `atomic` 后随便读取，因为读取方可能读到新指针旧版本，或旧指针新版本。若需要多字段一致快照，把字段放进同一个不可变对象，通过一个发布点发布。

原子等待：等待单一状态，不等待复杂谓词 C++20 提供 `atomic::wait` 和 `atomic::notify_one/all`。它适合等待单个 `atomic` 从某个值变成另一个值，例如任务完成状态从 `Pending` 进入终态。它不适合替代第 10 章的条件变量队列，因为队列等待谓词通常同时依赖 `size`、`closed`、容量、取消状态和未完成计数。

Listing 13.29 `atomic wait` 适合单原子状态等待

```

1 enum class CompletionState : std::int32_t {
2     Pending,
3     Succeeded,
4     Failed,
5 };
6
7 class CompletionFlag {
8 public:
9     void publish_success() {
10         this->state_.store(CompletionState::Succeeded,
11                           std::memory_order_release);
12         this->state_.notify_all();
13     }
14
15     CompletionState wait() const {
16         CompletionState state =
17             this->state_.load(std::memory_order_acquire);
18         while (state == CompletionState::Pending) {
19             this->state_.wait(state, std::memory_order_relaxed);
20             state = this->state_.load(std::memory_order_acquire);
21         }
22         return state;
23     }
24
25 private:
26     std::atomic<CompletionState> state_{CompletionState::Pending};
27 };

```

`wait` 的参数是“只要值仍等于这个旧值就睡眠”，醒来后仍要重新 `load`。发布者必须先写结果对象，再 `release` 存储终态，最后通知等待者。若等待者 `acquire` 读到 `Succeeded`，才能读取结果。这个协议比忙等省 CPU，但它仍然只围绕一个 `atomic` 状态；复杂生命周期和取消传播仍要单独设计。

Seqlock：一致快照的另一种取舍 多字段指标若需要一致快照，但写者很少、读

线程很多，可以学习 seqlock 思想：写者在更新前把版本变成奇数，写完后变成偶数；读线程先读版本，再读字段，再读版本，若版本变化或为奇数就重试。Seqlock 的代价是读线程可能重试，且读线程读取的数据类型和访问方式必须不会产生未定义行为；在 C++ 中不能随使用普通非原子字段跨线程读写。

Listing 13.30 示例版 seqlock 形状：真实 C++ 实现还需处理字段数据竞争

```

1 struct MetricsSnapshot {
2     std::uint64_t accepted = 0;
3     std::uint64_t completed = 0;
4     std::uint64_t failed = 0;
5 };
6
7 class VersionedMetrics {
8 public:
9     bool try_read(MetricsSnapshot& output) const {
10         const std::uint64_t begin =
11             this->version_.load(std::memory_order_acquire);
12         if ((begin & 1U) != 0U) {
13             return false;
14         }
15         output = this->snapshot_;
16         const std::uint64_t end =
17             this->version_.load(std::memory_order_acquire);
18         return begin == end;
19     }
20
21 private:
22     std::atomic<std::uint64_t> version_{0};
23     MetricsSnapshot snapshot_{};
24 };

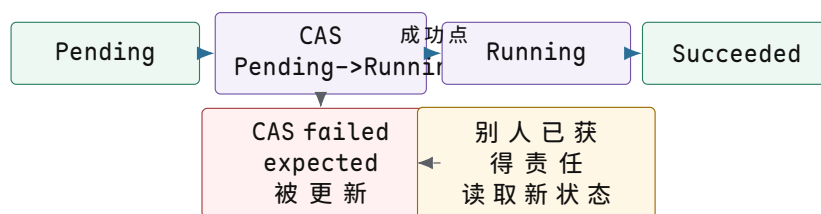
```

这段代码是语义形状，不是可直接复制的生产实现。它要说明一致快照要么用锁，要么用全原子字段和版本协议，要么用双缓冲不可变快照。重点不是“seqlock 很快”，而是“多字段一致性必须有协议”。

13.8 CAS：线性化点和责任转移

Compare-and-swap 读取当前值，若等于期望值就写入新值。CAS 成功的那个瞬间常常是并发对象的线性化点：从外部看，操作在这里生效。CAS 失败不是错误，而是其他线程已经改变状态；失败时 `expected` 会被写成实际观察值，调用者应根据新状态重新判断。

CAS 状态机：成功者获得责任，失败者读取事实



CAS 循环里只能准备候选状态。写文件、释放对象、提交消息、记录不可重复日志，都应放在成功之后或设计成幂等。

图示：本图要证明的命题是：CAS 成功不仅改值，还把后续责任交给成功线程；失败路径同样需要语义。

Listing 13.31 CAS 循环只在成功后提交外部副作用

```

1 bool try_set_max(std::atomic<std::int64_t>& target,
2                 std::int64_t candidate) {
3     std::int64_t observed = target.load(std::memory_order_relaxed);
4     while (observed < candidate) {
5         if (target.compare_exchange_weak(
6             observed,
7             candidate,
8             std::memory_order_release,
9             std::memory_order_relaxed)) {
10            return true;
11        }
12    }
13    return false;
14 }

```

失败时，`observed` 被更新为当前实际值，下一轮循环基于新值判断。函数没有在循环内部写外部日志、释放节点或提交文件，因此重试不会产生重复副作用。若调用者要记录“最大值被更新”，应在函数返回 `true` 后记录。

成功和失败内存序 `compare_exchange` 有成功内存序和失败内存序。成功时操作既读旧值又写新值，可以发布或获取状态；失败时只读取当前值，不能使用 `release`，因为没有发布新值。失败内存序也不能比成功内存序更强。常见写法是成功使用 `acq_rel` 或 `release`，失败使用 `acquire` 或 `relaxed`，取决于失败路径是否需要基于读取到的状态观察其他数据。

Weak 和 strong `compare_exchange_weak` 允许伪失败，通常放在循环里；`compare_exchange_strong` 不允许伪失败，更适合单次尝试或状态转换诊断。不要机械背“weak 快、strong 慢”。先看协议：是否本来就在循环里，失败后是否重算候选值，失败状态是否要报告给调用者。

ABA 和生命周期 CAS 比较的是位模式，不理解对象历史。一个指针从 A 变成 B 又变回 A，CAS 可能认为“没有变化”。这就是 ABA。解决方法包括标签指针、hazard pointer、epoch 回收、RCU 或避免地址复用。ABA 不是单纯内存序问题，而是状态历史和对象回收问题。第 12 章会把它放进无锁数据结构中详细展开。

CAS 状态机要有完整转移表 工业代码中，CAS 状态机不能只写成功路径。每个状态都要说明谁能进入、谁能离开、失败时返回什么、是否发布数据、是否通知等待者。没有转移表，CAS 循环很容易在取消、超时和重复完成时制造双提交。

当前状态	允许转换	成功者责任	失败者动作
Pending	Pending -> Running	获得执行权，开始任务	读取新状态，不能执行任务
Running	Running -> Succeeded	release 发布结果，通知等待者	不应由其他线程改写结果
Running	Running -> Failed	release 发布错误，通知等待者	记录重复完成或取消竞态
Pending	Pending -> Canceled	拒绝未来执行，通知等待者	若已 Running，只能请求协作取消
Succeeded/Failed	终态不可改	只允许读取	重复 completion 必须被识别

这个表能直接暴露设计缺口：如果 Running 状态下收到 cancel，是强制失败、协作取消、还是只设置 stop request？如果 I/O completion 迟到，终态已经存在，迟到事件清理资源还是覆盖结果？CAS 成功点只是责任转移，完整系统还要定义迟到、重复和失败路径。

CAS 循环的副作用边界 CAS 循环通常会执行多次。循环体内部只能做可重复、可丢弃的准备工作，例如读取当前状态、计算候选值、构造尚未发布的局部对象。不可重复副作用必须放在 CAS 成功之后，或者被设计成幂等并带 attempt 身份。写文件、提交 manifest、释放节点、发送网络消息、递增业务计数、唤醒等待者，都不能随意放在可能失败的 CAS 尝试里。

Listing 13.32 CAS 循环中的动作分层

```

inside retry loop:
    load current state
    validate transition
    compute desired state
    allocate local candidate that can be discarded

only after CAS success:
    publish ownership transfer
    perform non-repeatable external action
    notify waiters
    retire old resource according to reclamation protocol

after CAS failure:
    read updated observed state
    release local candidate or reuse it
    decide retry, return, wait, or report race

```

这条边界来自线性化点。CAS 成功前，当前线程还没有赢得责任；失败意味着另一个线程已经改变事实。若失败尝试已经向外部系统提交了不可回滚动作，就会出现“双提交但状态机只记录一次”的矛盾。第十五章的迟到 completion、第十六章的重试 attempt，都遵守同一原则：外部副作用必须有身份，状态机必须能拒绝重复或迟到结果。

CAS 失败路径也要有内存语义 失败路径不是“什么都没发生”。失败的 CAS 读取到了当前实际值，并把它写回 `expected`。如果失败线程要根据这个值读取其他普通数据，就可能需要 acquire 失败内存序；如果失败线程只决定重试或返回，relaxed 可能足够。把失败内存序随手写成 relaxed，常见于指标类 CAS；把状态机失败路径写成 relaxed，则可能让失败线程看到新状态，却无法依赖该状态发布

的关联数据。

例如，一个线程尝试把任务从 `Running` 改成 `Succeeded`，失败后看到状态已经是 `Failed`。如果它接下来要读取失败原因，而失败原因由另一个线程在 `release` 状态转换前写入，那么失败读取也需要 `acquire` 语义才能安全观察失败原因。成功路径和失败路径都要按“是否读取关联普通数据”判断，而不是按“成功才重要”判断。

13.9 生命周期：可见不等于活着

发布一个指针，有两个独立问题：消费者能否看到对象字段，消费者使用期间对象是否还活着。`release/acquire` 解决前者；引用计数、共享所有权、`hazard pointer`、`epoch` 或 `RCU` 解决后者。把这两个问题混成“`atomic pointer` 很安全”，是无锁代码中最危险的误解之一。

共享所有权的示例方案 热更新配置的简化实现可以使用不可变对象加 `std::shared_ptr`。发布者构造新对象，原子发布 `shared pointer`；读取方加载后持有一份 `shared pointer`，本次请求期间对象保持存活。这个方案不一定是最快的，但它把发布可见性和生命周期都交给成熟库，适合项目早期。

Listing 13.33 热更新配置使用不可变对象和共享所有权

```

1 class ConfigRegistry {
2 public:
3     void publish(std::shared_ptr<const RuntimeConfig> config) {
4         this->current_.store(std::move(config), std::memory_order_release);
5     }
6
7     std::shared_ptr<const RuntimeConfig> snapshot() const {
8         return this->current_.load(std::memory_order_acquire);
9     }
10
11 private:
12     std::atomic<std::shared_ptr<const RuntimeConfig>> current_;
13 };

```

这个设计仍要求配置对象不可变。`Shared pointer` 保证对象活着，不保证对象内部字段被多个线程并发修改时安全。若配置需要更新，构造新对象并发布新版本；不要在旧对象上原地修改。

生命周期谱系 生命周期方案是一条谱系。永不释放旧对象最简单，适合少量配置和示例，代价是内存增长。`Shared pointer` 把生命周期交给引用计数，易用但热路径有原子计数成本。`Hazard pointer` 要求线程公开自己正在保护哪个指针，回收精确但实现复杂。`Epoch` 回收按批次延迟释放，读路径轻但回收不精确。`RCU` 把读侧临界区变成系统级合同，适合读多写少，但需要成熟运行时或内核支持。

方案	能力	代价	项目取舍
永不释放	最容易证明，读取方无悬垂	内存只增不减	可用于一次性配置
shared pointer	生命周期自动，接口清楚	引用计数热点成本	推荐默认热更新
hazard pointer	精确保护单个指针	读取方登记和扫描复杂	第 12 章 深入
epoch	读侧低成本，批量回收	释放延迟，长时间读取方阻塞回收	第 12 章 深入
RCU	极轻读路径，成熟工业实践	系统级合同和回收复杂	只借鉴思想

引用计数不是对象内部同步 引用计数只回答“对象什么时候可以释放”，不回答“对象内部字段能否并发修改”。`std::shared_ptr<constRuntimeConfig>` 是好接口，因为 `const` 把热更新变成发布新对象；`std::shared_ptr<RuntimeConfig>` 若多个线程拿到后原地修改字段，仍然需要 `mutex` 或内部原子协议。很多线上 bug 不是对象提前释放，而是对象活着但内部状态被并发写坏。

最后一次 release 的语义 引用计数还有一个微妙点：最后一个引用释放时，析构线程要看到对象初始化和使用期间需要看到的事实。成熟 `shared pointer` 实现已经封装了这些细节；自研引用计数必须写清 `increment`、`decrement`、最后释放的 `acquire/release` 关系。`Compute Systems Engine` 不应自研通用引用计数，应把注意力放在“共享所有权能解决生命周期，但不能解决内部并发语义”。

13.10 Seq-cst、Fence 和硬件差异

`memory_order_seq_cst` 提供一个所有 `seq-cst` 操作参与的单一全局顺序，让推理更接近直觉。它适合入门、初始实现和确实需要全局顺序的协议。但它不是让并发程序自动正确的开关：对象回收、关闭协议、多字段不变量和等待者唤醒仍要单独设计。

`Fence` 是更底层的工具，把顺序约束和具体原子对象分离。除非实现底层库、移植成熟算法或有明确性能证据，普通项目应优先使用带内存序的原子读写、`mutex`、`condition variable` 或成熟并发容器。`Fence` 代码更难审查，因为必须追踪 `fence` 前后的普通访问和相关原子操作。

x86、ARM 和编译器 `x86-64` 内存模型相对强，很多 `acquire/release` `load/store` 在硬件指令上看起来很轻；`ARM` 和 `POWER` 更弱，可能需要显式屏障或带 `acquire/release` 语义的指令。可移植 `C++` 不能因为代码在 `x86` 上测试通过，就省略语言层面的同步。编译器也会根据内存序决定能否重排、缓存或消除访问。内存模型是编译器和硬件共同遵守的契约。

x86 TSO 的强处和盲区 `x86-64` 常被概括为 `TSO`。直觉上，每个核心的 `store` 先进入 `store buffer`；本核心后续 `load` 可以继续执行，并可能从自己的 `store buffer` 转发；其他核心稍后才看到这些 `store`。这个模型比许多弱内存架构强：普通 `load/load`、`load/store`、`store/store` 的可见顺序限制更多，`release/acquire` 常常不需要额外屏障指令。但 `TSO` 仍允许 `store buffering` 反例中两个线程都读到 0，因为每个核心的 `store` 可能还停在自己的 `store buffer` 中，另一个核心的 `load` 先读到旧值。

Listing 13.34 TSO 直觉下的 store buffering 账本

```
core A:
  x.store(1) enters A's store buffer
  y.load() reads old y from cache before B's store becomes visible
```

```

core B:
  y.store(1) enters B's store buffer
  x.load() reads old x from cache before A's store becomes visible

possible observation:
  A sees y == 0
  B sees x == 0

```

因此，“x86 比较强”不能推出“relaxed 可以当同步用”。它只能说明某些 release/acquire 编译后指令看起来轻，或某些弱内存反例在 x86 上较难出现。C++ 源码仍要写出语言层面的同步，否则编译器就有自由度，跨平台也没有合同。

ARM 弱内存让错误更早暴露 ARM、POWER 这类弱内存系统允许更多重排和更弱的跨地址观察，通常需要带 acquire/release 语义的 load/store 或显式屏障来建立边界。它们不是“不可靠”，而是把顺序成本留给需要顺序的程序。若协议已经用 C++ release/acquire 写清，编译器会选择目标架构上合适的指令或屏障；若协议靠 x86 偶然行为通过测试，迁移到弱内存平台时可能暴露旧 bug。

工程报告可以把架构差异写成三层：C++ 源码承诺什么；目标 ISA 提供什么默认顺序；编译器为指定 `memory_order` 生成什么指令。只有第一层是可移植语义，第二层和第三层是性能与验证证据。把第二层当第一层，是并发代码跨平台失效的常见根因。

Fence 为什么难审查 Fence 把顺序约束从具体原子对象上拆下来，审查者必须在代码中寻找“哪个 release fence 与哪个 acquire fence 通过哪个原子值建立同步”。相比之下，`flag.store(value, release)` 和 `flag.load(acquire)` 把发布点和观察点绑定在同一个对象上，更容易读。Fence 适合底层库、移植经典算法、或需要表达语言 API 无法直接表达的硬件序列；普通业务状态机通常应避免。

Listing 13.35 fence 审查必须写出的三件事

```

release side:
  ordinary writes before release fence
  atomic store or RMW that carries the synchronization value

acquire side:
  atomic load or RMW that reads that synchronization value
  acquire fence before ordinary reads that rely on publication

review question:
  which exact value connects the two sides?

```

如果回答不了“哪个值把 fence 两侧连起来”，这段 fence 代码大概率只是制造错觉。更强的 fence 也不能保护对象生命周期，不能修复 ABA，不能让复杂多字段状态机自动原子化。它只约束某些访问的顺序。

测试不能替代证明 内存序 bug 可能只在特定架构、优化级别、核心数和时序下出现。ThreadSanitizer 擅长发现数据竞争，但不能证明 CAS 线性化、ABA 回收或最弱内存序正确。压力测试能增加信心，不能穷尽交错。可靠的原子代码必须有协议说明、锁版 reference、反例测试和架构说明。

最弱内存序不是目标 很多文章喜欢追求“最弱正确内存序”。工业代码的目标不是炫耀最弱，而是让协议可审查、性能足够好、跨平台稳定。若 `acquire/release` 与 `relaxed` 的性能差异不可测，选择更清楚的 acquire/release 往往更合适。若某段代码处在每秒数十亿次热路径，才值得用 benchmark 和模型证明进一步放松。

每次放松都应留下原因、反例和回退方案。

选择	适合情况	审查要求
mutex	多字段不变量、复杂等待、生命周期复杂	不变量、谓词、锁边界清楚
release/acquire	单一发布点、槽位状态、任务终态	写明发布字段和读取者
relaxed	纯指标、单对象计数、不会发布数据	写明不参与协议判断
seq-cst	初始实现、全局顺序要求、调试收敛	说明是否可放松
fence	底层库、成熟算法移植、硬件适配	需要强审查和专门测试

13.11 工业设计参照

工程案例

Linux atomic、refcount 和 RCU。 Linux 区分普通 atomic 操作、引用计数安全封装和 RCU 读侧临界区。它购买的能力是低成本状态变化和极轻读路径；代价是严格的生命周期合同、架构相关屏障和大量代码审查规则。Compute Systems Engine 应采用这种分类思想：计数、状态、发布和回收不能混为一谈。

C++ 标准库。 `std::atomic` 给出跨平台语言模型，`std::shared_ptr` 的原子操作给热更新提供较稳妥的生命周期方案，`atomic::wait` 为单原子状态等待提供直接工具。代价是底层细节隐藏，性能需要测量。项目早期应优先使用标准库表达正确语义，再决定是否需要更底层方案。

Folly、Abseil 和高性能基础库。 这些库通常把原子封装在更高层类型中，例如原子 shared pointer、RCU 域、分片计数器、hazard pointer 或 executor 状态。它们购买的是可复用协议和集中审查；代价是引入库依赖和学习成本。Compute Systems Engine 不应照搬全部实现，但应照搬“把原子协议封装并写明合同”的做法。

数据库和流处理系统中的版本发布。 DuckDB、ClickHouse、RocksDB、Flink 等系统大量使用不可变快照、版本号、manifest、epoch 或 checkpoint 来避免读取方看到半更新状态。单机原子发布和分布式 manifest 发布不是同一个机制，但思想相通：先准备完整新事实，再用一个清楚提交点发布。

浏览器、游戏引擎和交易系统的热配置。 这些系统常把配置、路由表、策略表或资源索引做成不可变版本。写者构造新版本，验证完整后一次发布；读取方拿 snapshot，使用期间不被修改。它购买的是读路径稳定和故障回滚简单；代价是内存瞬时增加、版本回收和写路径复制成本。Compute Systems Engine 的 topology、runtime config 和 partition map 都应优先使用这种模式。

日志系统和 telemetry。 高性能日志计数通常不要求每次读取都是一致快照，而是用 per-thread buffer、分片计数、批量 flush 和后台汇总。它购买的是写入热路径低成本；代价是读取滞后和近似。项目报告要区分 telemetry 指标和控制面状态，不能用近似指标驱动关闭、提交或资源释放。

13.12 实现边界：AtomicProtocolNote

Compute Systems Engine 使用 AtomicProtocolNote 约束每个原子变量。它可以是设计文档、代码旁注释或审查表，但每个 atomic 都要能填写。若填写不出来，

默认改回 mutex 或把状态机重构得更窄。

字段	要回答的问题	示例
变量用途	指标、发布、状态、索引、引用、等待	<code>completed_tasks</code> 是指标
写者/读取方	谁写，谁读，是否单写者	<code>worker</code> 写， <code>driver</code> 汇总
发布数据	是否同步普通字段，字段有哪些	<code>state=Ready</code> 发布 <code>result</code>
内存序	每个 <code>load/store/CAS</code> 的理由	<code>relaxed</code> 不发布数据
线性化点	操作何时对外生效	<code>CAS Pending->Running</code> 成功
失败路径	<code>CAS</code> 失败、关闭、取消如何处理	<code>expected</code> 更新后分类返回
生命周期	指针或结果对象如何保持存活	<code>shared pointer snapshot</code>
等待策略	忙等、 <code>atomic wait</code> 、条件变量还是任务图	长等待不用忙等

三条最小对照路径 第一，指标路径：比较 mutex counter、atomic relaxed counter 和 sharded counter，关注 cache 热点、汇总成本和读取滞后。它说明 relaxed 只适合不发布数据的计数事实。第二，发布路径：实现一次性配置发布和 shared pointer 热更新，写清 release 写入哪个协议变量、acquire 读取后允许访问哪些普通字段，以及旧版本什么时候能释放。第三，CAS 路径：实现 `try_set_max` 或 `TaskCompletion` 状态机，记录 CAS 失败次数和失败分支，证明循环内没有不可重复副作用。

反例必须保留 错误版本比成功版本更能说明内存序边界。至少保留 relaxed ready 发布配置、CAS 循环内写不可重复日志、多 atomic 读取伪一致快照、裸原子指针热更新后释放旧对象四个反例。判读不能只写“修复后通过”，而要写清：错误版本允许什么坏观察，为什么测试可能不稳定复现，修复版本用哪条同步边或生命周期协议消除了坏观察。

并发验证要分层 并发验证不能只靠压力测试。压力测试能提高交错覆盖率，却不能证明所有交错；ThreadSanitizer 能发现很多数据竞争，却不能证明内存序足够强，也不能证明线性化点正确；AddressSanitizer 能抓 use-after-free，却未必能稳定触发 ABA；模型检查或手写小状态枚举能穷举小规模状态，却不能直接覆盖生产实现的内存分配、调度和 I/O。成熟的报告要把这些方法分层使用。

方法	能发现什么	不能替代什么
锁 版 reference 对照	输出语义、边界行为、关闭路径差异	原子协议证明和生命周期证明
ThreadSanitizer	普通数据竞争、部分同步误用	最弱内存序正确性、ABA、线性化点
AddressSanitizer	use-after-free、越界、双重释放	没触发的回收竞态、内存序错误
压力和随机 yield	调度窗口、等待路径、CAS 竞争	穷尽交错和跨架构弱内存行为
线性化日志	操作是否能排成合法顺序	内存回收安全性和性能结论
litmus 小程序	某类内存序反例是否可观察	业务状态机完整性

线性化日志尤其重要。每个并发操作记录开始时间、结束时间、线程 id、观察到的状态、CAS 成败、返回值和逻辑对象身份；测试结束后检查是否存在一个顺序历史，既尊重非重叠操作的先后，又能解释所有返回值。队列测试不能只看最终元素数量，因为丢一个再重复一个可能抵消；应验证每个 logical item 恰好出现一次，关闭后没有新提交，取消后没有伪成功。

Listing 13.36 并发操作日志的最小字段

```

ConcurrentEvent:
    thread_id
    operation_id
    operation_kind
    begin_seq
    end_seq
    observed_state
    cas_success
    logical_item_id
    return_status

```

这个日志会增加测试成本，但它把“偶尔失败”变成可复盘事实。若某次运行出现重复 item，日志能显示两个 push 是否都认为自己占有同一槽位；若 wait 永久不返回，日志能显示最后一次状态变化是否通知了对应等待点；若 CAS 失败率暴涨，日志能区分真实竞争和错误重试。并发代码应保留这种事件级证据，而不是只贴吞吐曲线。

从锁版迁移到原子版 迁移路线要保守。先用 mutex 写 reference，测试正确性和边界；再把纯指标降为 relaxed；再实现边界清楚的 SPSC 槽位状态；最后才考虑 MPSC、MPMC 或无锁结构。每次迁移都要写 [AtomicProtocolNote](#)。不要从“这里有锁”直接跳到“这里用 CAS”，因为锁可能保护的是一组尚未列出的不变量。

最小综合案例是 [TaskCompletion](#)。状态为 Pending、Running、Succeeded、Failed、Canceled；worker 用 CAS 从 Pending 转 Running；完成时用 release 发布结果或错误；等待者 acquire 读取终态后读取结果。这个案例要写状态转换表、内存序理由、异常路径、取消语义和等待策略，并保留 mutex/condition variable 版本作为语义 reference。它覆盖了本章最关键的链路：状态机、发布边、CAS 失败、等待唤醒和结果生命周期。

源码阅读任务

源码阅读按原子变量用途分类。

1. Linux：阅读 atomic/refcount/RCU 相关文档或代码，区分普通计数、引用计数和 RCU 生命周期合同。
2. C++ 标准库或 libc++：观察 `std::shared_ptr`、`atomic<shared_ptr>`、`condition_variable` 与底层同步的边界。
3. Folly 或 Abseil：找一个原子封装类型，说明它怎样把裸 atomic 协议变成可审查 API。
4. 一个数据库或流处理系统：找版本化配置、manifest、checkpoint 或快照发布路径，说明它如何避免读取方看到半更新状态。

报告必须从一个具体协议变量开始，而不是泛泛介绍“项目用了 atomic”。

13.13 完整案例：TaskCompletion 的三次重构

原子操作最适合通过一个小状态机学习。这里用 [TaskCompletion](#) 表示任务完成对象：worker 执行任务，发布成功结果或错误；等待者等待终态并读取结果；取消者可能在任务尚未运行时取消。这个对象小到可以画完整状态表，又足以覆盖指标、发布、CAS、等待和生命周期。

第一版：锁保护多字段不变量 锁版 reference 把状态、结果、错误和等待者放在同一个临界区里。它不一定最快，但最容易写清语义。状态从 Pending 到 Running，由获得执行权的 worker 改变；Running 到 Succeeded 或 Failed，由执行中的 worker 改变；Pending 到 Canceled，由取消者改变；终态不可再改。等待

者用条件变量等待终态。

Listing 13.37 锁版 completion 的状态机

```
Pending:
    worker may move to Running
    canceler may move to Canceled

Running:
    running worker may move to Succeeded or Failed
    canceler may request cooperative cancellation only

Succeeded/Failed/Canceled:
    terminal, notify all waiters
```

这个版本的价值是把多字段不变量写清：终态出现时，结果或错误对象已经写好；等待者醒来后能读取；重复 completion 被拒绝；异常路径也进入 Failed。原子版必须保持这些语义，而不是只把 `state` 字段替换成 `std::atomic`。

第二版：CAS 只负责责任转移 把 Pending 到 Running 改成 CAS，可以让多个 worker 竞争同一个任务执行权。CAS 成功者获得责任；失败者读取新状态并退出。此时 CAS 的线性化点是“谁拥有执行权”，不是“任务已经完成”。成功后 worker 在普通字段中写结果，最后用 `release store` 发布终态。

Listing 13.38 CAS 获得执行权的语义伪代码

```
try_start:
    expected = Pending
    if CAS(state, expected, Running, acquire-release):
        return StartedByThisThread
    else:
        return AlreadyOwnedOrTerminal(expected)

finish_success:
    write result fields
    state.store(Succeeded, release)
    state.notify_all()
```

这里有两个不同同步点。CAS 成功让 worker 获得从 Pending 到 Running 的责任；`release store` 到 Succeeded 发布结果字段给等待者。若等待者 `acquire` 读到 Succeeded，才能读取 `result`。若等待者 `acquire` 读到 Running，只说明任务已经有人执行，不说明结果存在。

第三版：等待用单原子状态，但谓词仍要重读 `atomic::wait` 可以让等待者在 `state` 仍等于某个旧值时睡眠。它不是条件变量的万能替代；它只等待单个 `atomic` 状态。等待者必须循环 `load`，因为 `notify` 只是醒来机会，状态可能没有进入终态，也可能被另一个线程先观察。

Listing 13.39 等待终态的语义伪代码

```
wait_terminal:
    state = load(acquire)
    while state is Pending or Running:
        state.wait(state)
        state = load(acquire)
```

```

if state == Succeeded:
    read result fields
if state == Failed:
    read error fields

```

如果 `finish_success` 先 store Succeeded 再写 result，等待者可能 acquire 读到 Succeeded 后读到未完成结果。正确顺序是先写普通结果字段，再 release 发布终态。这个例子把 release/acquire 的方向性固定下来：发布者把普通写放在 release 前，获取者在 acquire 后读取普通字段。

取消和迟到 completion 取消让状态机更接近真实系统。若状态仍是 Pending，取消者可以 CAS 到 Canceled，通知等待者；若状态是 Running，取消者不能直接覆盖 Running，因为任务可能已经持有资源或即将发布结果。它只能设置协作取消请求，或者记录 cancel requested，由 worker 在安全点处理。若 I/O completion 迟到，终态已经是 Canceled 或 Failed，迟到事件不能覆盖终态，只能清理资源并记录 late completion。

当前状态	事件	正确处理
Pending	cancel	CAS 到 Canceled, release 发布取消原因, notify all
Pending	worker start	CAS 到 Running, 成功者执行任务
Running	cancel	设置协作取消请求, 不覆盖 Running
Running	success	写 result, release store Succeeded, notify all
Succeeded	late failure	拒绝覆盖, 记录重复 completion
Canceled	late success	拒绝提交结果, 清理 attempt 资源

这张表说明 CAS 状态机必须覆盖失败路径。原子状态只负责“哪个终态生效”，不负责自动清理迟到事件，也不负责取消运行中的外部 I/O。

指标原子不能驱动协议 `completed_count.fetch_add(relaxed)` 可以记录有多少任务完成，但等待者不能靠它判断某个结果对象是否可读。任务完成协议由 `state` 的 release/acquire 负责；指标只是 telemetry。若把 relaxed 计数当作发布信号，等待者可能看到计数增加，却没有同步到对应结果字段。这正是“指标变量升级成协议变量”的危险。

Listing 13.40 completion 中两类 atomic 的边界

```

protocol atomic:
    state: Pending/Running/Succeeded/Failed/Canceled
    publishes result and error fields

telemetry atomic:
    completed_count
    cas_failures
    late_completion_count
    does not publish result objects

```

AtomicProtocolNote 示例 这个对象的 AtomicProtocolNote 应写清：`state` 是协议变量，发布 result/error；CAS Pending→Running 是执行权线性化点；release store 终态是结果发布点；等待者 acquire 读终态；`cas_failures` 和

`completed_count` 是 `relaxed` 指标；结果对象生命周期由 `completion` 对象持有，等待者返回前 `completion` 不能销毁。这样原子代码才有审查入口。

x86 TSO 和 ARM 弱内存序：一次运行通过不等于协议成立 C++ 内存序是语言层合同，不是某一种处理器的说明书。同一段原子代码在 x86-64 和 ARM 上可能生成不同形状的指令。x86-64 常被概括为 TSO, total store order。它保证本核心的 `store` 对其他核心以较强顺序出现，但 `store buffer` 仍然允许本线程后续 `load` 先从别处取值；普通 `load` 的 `acquire` 和普通 `store` 的 `release` 在很多情况下不需要额外 `fence`，锁前缀 RMW 则会提供更强的原子性和排序效果。ARM 和 RISC-V 等体系通常更弱，`release/acquire` 往往要通过专门的 `load-acquire`、`store-release` 或 `fence` 指令表达。

下面的消息传递协议在 C++ 层必须写 `release/acquire`，不能因为 x86 反汇编看起来只是普通 `mov` 就改成 `relaxed`：

Listing 13.41 发布数据的语言层合同

```
producer:
    payload.x = 1
    payload.y = 2
    ready.store(true, release)

consumer:
    if ready.load(acquire):
        read payload.x and payload.y
```

在 x86-64 上，编译器可能把 `release store` 和 `acquire load` 编成普通 `store/load`，因为硬件模型已经给出足够约束；在 ARM 上，发布和获取可能对应 `stlr` 和 `ldar`，或者在更复杂路径中出现 `dmb`。这不是 ARM “更慢”这么简单，而是两种 ISA 把排序责任放在不同层面。语言层写 `release/acquire`，是为了让编译器在目标 ISA 上生成足够的约束。

C++ 操作	x86-64 常见形状	ARM 常见形状
relaxed load/store	普通 load/store	普通 load/store
release store	多数为普通 store	store-release 或 fence + store
acquire load	多数为普通 load	load-acquire 或 load + fence
seq-cst store/RMW	可能使用锁前缀或 fence 边界	需要更强排序指令或序列

因此，原子协议的正确性不能靠“我在当前机器上反汇编没有 `fence`”来证明。正确审查顺序是：先写出共享普通数据由哪个 `release` 发布、由哪个 `acquire` 获取；再写出 CAS 成功和失败各自携带什么语义；最后才读目标平台反汇编，判断编译器是否生成了预期成本。若把顺序反过来，很容易在 x86 上偶然可用，在 ARM 上失效，或者在编译器优化后失效。

Fence 不是补救所有并发问题的胶水 `atomic_thread_fence` 只建立排序约束，不会替一个普通变量制造原子性，也不会给对象生命周期兜底。典型错误是在两个普通字段之间插入 `fence`，然后以为另一个线程就能安全读取它们。如果没有共同参与同步的原子对象，两个线程之间仍然缺少可命名的同步边。Fence 适合少数底层场景，例如用一个 `atomic flag` 发布一批非原子数据，或者把设备/共享内存协议映射到语言模型；普通业务状态机更应该优先用 `release/acquire` 的协议变量表达。

Listing 13.42 fence 不能替代协议变量

```

bad shape:
  writer stores ordinary fields
  writer executes a fence
  reader executes a fence
  reader reads ordinary fields

missing:
  no atomic object connects writer and reader
  no acquire observes a release sequence
  no object lifetime transfer is defined

```

看到并发 bug 时，先问“哪个原子值被观察到后，允许读取哪些普通字段”，不要先问“要不要加一个 fence”。Fence 只能收紧已有协议的顺序，不能替代协议本身。

何时退回 mutex 如果 `TaskCompletion` 继续增加多个结果槽、多个等待条件、父子任务传播、超时回调、资源清理和复杂取消，单个 atomic state 可能不再是清楚设计。退回 mutex 不是失败。Mutex 能把多字段不变量重新放进一个临界区，使语义先稳定。等 profile 证明某条状态转换是瓶颈，再把它拆成窄原子协议。原子代码的目标是缩小同步边界，不是把复杂状态硬塞进枚举。

Listing 13.43 退回锁版的信号

```

one state transition updates many ordinary fields
wait predicate depends on more than one atomic
cancel path must coordinate external resources
object lifetime cannot be expressed by ownership snapshot
test failures require reasoning about many interleavings

```

这些信号出现时，先用锁版 reference 收敛状态机，再考虑优化。最危险的原子代码通常不是语法错误，而是状态已经复杂到没人能写出完整反例。

13.14 接口延伸

原子操作不是“更快的锁”，而是一组更窄、更需要协议说明的同步工具。Relaxed 适合只看自身数值的指标；release/acquire 适合发布普通数据；CAS 适合定义单点责任转移；seq-cst 适合初始推理或确实需要全局顺序的场景；原子指针必须配生命周期协议。多个 atomic 不会自动组成一致快照，atomic flag 不会自动发布对象，CAS 成功也不会自动处理 ABA 和回收。

`AtomicProtocolNote` 是后续并发实现的边界文件。第 12 章的无锁数据结构会把这些小协议组合成 SPSC ring、Treiber stack、ABA 防护和内存回收；第 14 章的任务运行时会把原子状态机用于 ready count 和 completion；第 18 章的单机提交状态会迁移到分布式 manifest 和 attempt commit。

13.15 习题

习题与作业

1. 给出一个 relaxed ready flag 发布配置的错误版本。说明为什么它没有数据竞争但仍缺少发布关系，并改成 release/acquire。
2. 为 `WorkerSnapshot` 中的计数字段设计 relaxed 或分片计数方案。说明哪些读数可以近似，哪些不能参与协议判断。
3. 实现 `try_set_max`，记录 CAS 失败次数。把日志写在 CAS 循环内

和成功后各实现一次，解释语义差异。

4. 设计一个 `TaskCompletion` 状态机，用 `atomic enum` 表达 `Pending`、`Running`、`Succeeded`、`Failed`、`Canceled`。写出每个转换的线性化点和内存序。
5. 比较裸原子指针、`shared_ptr` 热更新、`epoch` 和 `RCU` 四种配置发布生命周期方案，说明各自的成本和适用场景。
6. 阅读一个工业原子封装或 `RCU` 实现，填写 `AtomicProtocolNote`：变量用途、写者/读取方、发布数据、内存序、线性化点、失败路径和生命周期。

工程案例

知识点小案例：`atomic flag` 不会自动发布普通字段。特例是生产者先写 `config.worker_count`，再执行 `ready.store(true, relaxed)`；消费者看到 `ready.load(relaxed)` 为 `true` 后读取 `config`。这个程序的 `atomic flag` 自身没有数据竞争，但它没有建立“配置字段先于读取方可见”的发布关系。改法是生产者用 `release store`，消费者用 `acquire load`，并且 `config` 在发布后不再被无保护修改。由此推出规律：内存序要从用途推导，指标计数可以 `relaxed`，发布普通数据需要 `release/acquire`。工程上每个 `atomic` 都要写变量用途、发布数据、读取方动作和生命周期规则。

无锁数据结构

先看一个 Treiber stack 事故。线程 T1 读取 `head` 指向节点 A，还没来得及 CAS；线程 T2 弹出 A，释放它，又从对象池分配了一个新节点，地址恰好仍是 A；T1 恢复后看到 `head` 的位模式仍等于 A，于是 CAS 成功，把一个生命周期已经变化的节点当成旧节点使用。这里没有 mutex 阻塞，也没有普通数据竞争，atomic 和 CAS 都用上了，但程序仍然错了。CAS 比较的是值，不理解对象的历史。

无锁不是更高级的写法，而是一组更窄、更难偷懒的工程合同。锁把失败竞争交给内核或运行时：线程拿不到锁时可以睡眠，持锁线程释放后再唤醒。无锁结构把失败竞争留在用户态：线程读一个状态，尝试 CAS，失败后重新读取并重算。这样避开了持锁线程暂停导致的全局阻塞，却把问题转移成 cache line 所有权迁移、CAS 重试、状态历史、节点回收、关闭等待和外层背压。少了一把 mutex，不代表少了同步；同步只是从临界区合同变成了每个原子字段的合同。

因此，无锁结构必须同时回答三个平面的问题。第一是进展平面：竞争、满、空、取消和线程暂停时，系统是否仍有操作能前进。第二是语义平面：push、pop、steal、close 的线性化点在哪里，失败返回何时对外生效，内存序发布了哪些普通数据。第三是生命周期平面：数组槽位何时可复用，链表节点何时可释放，地址复用是否会制造 ABA。任何一个平面缺失，结构都可能在 benchmark 中很快、在真实系统中很脆。

14.1 先问是否需要无锁

Blocking 结构可能因为一个线程持锁后暂停而阻塞其他线程。Lock-free 算法保证系统整体持续前进，但单个线程可能反复失败。Wait-free 算法保证每个线程在有限步内完成，通常实现代价更高。Obstruction-free 只在无竞争时保证进展。这些术语不是装饰，它们定义的是竞争和失败时的承诺。

从锁等待到缓存行争用 Mutex 的慢路径通常会进入内核或运行时等待队列，线程让出 CPU，等待持锁者释放并唤醒。CAS 循环没有这层默认睡眠机制：失败线程继续加载同一个原子对象，下一次 CAS 又要争同一条 cache line 的独占所有权。低竞争时，这种用户态重试很快；高竞争时，所有核心都在把同一条 cache line 拉来拉去，既消耗互连带宽，也抬高尾延迟。无锁的第一个判断标准不是“有没有锁”，而是这条状态线是否真的适合被多个核心反复争夺。

线性化点先于代码优化 并发结构对外必须像每个操作在某个瞬间生效。这个瞬间叫线性化点。SPSC ring 的 push 通常在线程 release 存储新 `tail` 时对消费者生效；Treiber stack 的 push/pop 通常在 CAS 成功时生效；队列满或空的失败返回，也要说明读取到哪个状态后决定失败。没有线性化点，性能优化没有参照物，因为无法判断某次重排、预留或批量提交是否改变了对外语义。

无锁不是默认优化按钮 如果临界区短、竞争不高、状态复杂或需要阻塞等待，mutex 加 condition variable 往往更清楚、更省 CPU，也更容易维护。无锁结构适合边界窄、热点明确、生产者消费者模型稳定、容量和生命周期可证明的路径。高竞争 CAS 循环可能烧满 CPU，产生 cache line 迁移和尾延迟；mutex 在严重竞争时让线程睡眠，反而更节能。

工程决策要从 profile 和语义出发。队列操作只占端到端 2

进展保证要写具体 “没有 mutex”不等于 lock-free。若算法等待某个槽位从 Writing 变成 Ready，而写该槽位的线程崩溃后无人修复，其他线程可能一直等。若 push 每次都调用可能阻塞的通用 allocator，整体也不能简单宣称 lock-free。进

展保证要写清依赖条件：是否允许动态分配，是否允许线程取消，是否会在队列满时返回失败，是否把阻塞等待放在结构外层。

进展保证和尾延迟是两件事 Lock-free 保证的是系统整体有线程能前进，不保证某个请求低延迟。一个线程可能在 CAS 循环里失败很多次，业务请求尾延迟仍然很差。Wait-free 保证每个线程有限步完成，但常常需要更多元数据、更多内存和更复杂的证明。Blocking mutex 在竞争严重时让线程睡眠，尾延迟未必总比 CAS 自旋差。工程报告不能只写“lock-free 所以快”，必须记录 CAS 失败次数、重试时间、CPU 占用、满/空返回次数和 p99/p999 延迟。

进展术语	保证	工程解释
blocking	持锁线程停住可能阻塞别人	容易证明，适合复杂状态和等待别人
obstruction-free	无竞争时能完成	需要外部退避或仲裁，单独价值有限
lock-free	总有某个线程能前进	单个线程可能饥饿，尾延迟仍需测量
wait-free	每个线程有限步完成	证明和实现成本最高，接口通常更窄

先定义失败语义 无锁结构常把阻塞变成失败返回。队列满时 `try_push` 返回 `false`，队列空时 `try_pop` 返回空，这不是异常，而是接口语义。调用者必须决定：重试、退避、切换队列、阻塞在外层 Channel、还是向上游施加背压。若上层没有策略，所谓 non-blocking 只会变成忙等烧 CPU。

Listing 14.1 `try` 接口必须配套外层策略

```

1 bool submit_with_backoff(SpscIntQueue& queue, std::int32_t value) {
2     for (std::int32_t attempt = 0; attempt < 8; ++attempt) {
3         if (queue.try_push(value)) {
4             return true;
5         }
6         std::this_thread::yield();
7     }
8     return false;
9 }

```

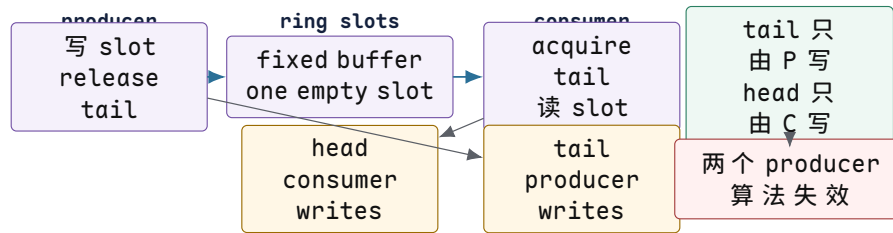
这段代码不是推荐最终实现，而是提醒这一点：`try_push` 的 `false` 是系统设计的一部分。生产系统可能用 semaphore 睡眠、事件循环注册可写、或者把失败转成背压指标。不能在热循环里无限 `while(!try_push()){}`，否则下游慢会变成上游空转。

14.2 SPSC Ring：最合适的起点

单生产者单消费者队列是学习无锁的最佳起点，因为所有权非常清楚：只有生产者写 `tail` 和待发布槽位，只有消费者写 `head` 和待释放槽位。生产者读取 `head` 判断空间，消费者读取 `tail` 判断数据。每个索引只有一个写者，这比多个线程 CAS 同一个端点简单得多。

这个简单性来自硬件事实。生产者频繁写 `tail`，消费者频繁写 `head`；若每个字段只有一个写者，写入方不需要和其他写者争同一条 cache line 的独占所有权。读取对方索引仍会产生一致性流量，但它不会造成两个写者同时修改同一状态。也因此，SPSC ring 不是“简化版 MPMC”，而是另一个问题：它用角色约束换取更简单的内存序、更少的 CAS 和更容易证明的槽位生命周期。

SPSC ring: 单写者约束让协议变简单



SPSC 的性能来自角色限制，不来自算法能抵抗任意误用。类型名和接口合同必须写清 Single Producer / Single Consumer。

图示：本图要证明的命题是：SPSC 简单的根本原因是单写者所有权。

不变量和线性化点 常见 SPSC ring 保留一个空槽区分满和空。若 `head==tail`，队列空；若 `next(tail)==head`，队列满。生产者写入元素后 `release` 发布新 `tail`，这是 `try_push` 对消费者可见的线性化点。消费者 `acquire` 读取 `tail` 后才能读槽位，读取元素后 `release` 发布新 `head`，让生产者知道槽位可复用。

Listing 14.2 SPSC ring buffer 的核心实现

```

1 class SpscIntQueue {
2 public:
3     explicit SpscIntQueue(std::uint64_t capacity)
4         : buffer_(static_cast<std::size_t>(capacity + 1), 0) {}
5
6     bool try_push(std::int32_t value) {
7         const std::uint64_t tail =
8             this->tail_.load(std::memory_order_relaxed);
9         const std::uint64_t next_tail = this->next(tail);
10        const std::uint64_t head =
11            this->head_.load(std::memory_order_acquire);
12        if (next_tail == head) {
13            return false;
14        }
15        this->buffer_[static_cast<std::size_t>(tail)] = value;
16        this->tail_.store(next_tail, std::memory_order_release);
17        return true;
18    }
19
20    std::optional<std::int32_t> try_pop() {
21        const std::uint64_t head =
22            this->head_.load(std::memory_order_relaxed);
23        const std::uint64_t tail =
24            this->tail_.load(std::memory_order_acquire);
25        if (head == tail) {
26            return std::nullopt;
27        }
28        const std::int32_t value =
29            this->buffer_[static_cast<std::size_t>(head)];
30        this->head_.store(this->next(head), std::memory_order_release);
31        return value;

```

```

32 }
33
34 private:
35     std::uint64_t next(std::uint64_t index) const {
36         const std::uint64_t size =
37             static_cast<std::uint64_t>(this->buffer_.size());
38         return (index + 1) % size;
39     }
40
41     std::vector<std::int32_t> buffer_;
42     alignas(64) std::atomic<std::uint64_t> head_{0};
43     alignas(64) std::atomic<std::uint64_t> tail_{0};
44 };

```

这段代码只适用于一个生产者和一个消费者。两个生产者同时调用 `try_push` 会同时写 `tail` 和同一槽位；两个消费者同时调用 `try_pop` 会同时写 `head`。无锁结构的安全来自约束被遵守，不能靠模糊命名让调用者猜。

关闭放在外层 Channel 底层 SPSC ring 可以提供非阻塞 `try_push/try_pop`。若系统需要阻塞等待、关闭、`drain`、取消和超时，更好的分层是外层 Channel 负责生命周期，底层 ring 负责快速交换。关闭不是一个可以随便塞进 ring 的 `bool`，它要唤醒等待者、拒绝新写、排空旧数据，并与线程池 `shutdown` 协同。第 10 章的 `QueueContract` 仍然适用。

测试 SPSC SPSC 测试要覆盖容量 1、2、非二次幂、环绕多轮、生产者快消费者慢、消费者快生产者慢、生产者提前结束、消费者 `drain`。生产者生成单调序号，消费者检查严格递增和不丢不重。还要故意让两个生产者误用，确认文档和 debug 检查能暴露错误。性能报告记录吞吐、空/满返回次数、CPU 占用和 `cache line` 对齐影响。

固定数组为什么避开节点回收 SPSC ring 使用固定数组时，槽位内存从队列创建到销毁一直存在。消费者弹出元素后，只是把槽位所有权交还给生产者，并不会释放槽位地址。这样 ABA 的主要来源被拿掉了：同一个数组位置确实会被反复复用，但复用轮次由 `head/tail` 的顺序协议约束，地址本身不被释放给通用 `allocator`。链式无锁栈完全不同。`pop` 成功后节点脱离结构，若立即 `delete`，其他线程可能仍持有旧指针；若 `allocator` 很快复用同一地址，就会产生 ABA。因此，固定数组结构优先讲槽位状态和环绕轮次，链式结构必须额外讲节点回收。

容量、取模和缓存形状 SPSC ring 可以使用任意容量配合取模，也可以使用二次幂容量配合 `bit mask`。二次幂版本更快，但接口要明确实际可用容量和内部数组大小；保留一个空槽时，内部大小为 `capacity + 1`，二次幂 `mask` 设计则常把 `sequence` 或可用容量重新定义。示例版优先使用清楚的 `next(index)`，性能版才在测量后改成 `mask`。优化前要先证明不丢、不重、环绕正确。

false sharing 和索引缓存行 SPSC 的 `head` 和 `tail` 会被两个核心频繁读写。若它们落在同一 `cache line`，生产者写 `tail` 会让消费者所在核心的同一条线失效，消费者写 `head` 也会反向影响生产者。把两个索引分开对齐能减少 `false sharing`，但会增加对象大小。报告要测量对齐前后吞吐和 CPU 占用，不要把 `alignas(64)` 当成无条件正确。

Listing 14.3 索引对齐表达的是缓存所有权边界

```

1 struct alignas(64) ProducerIndex {
2     std::atomic<std::uint64_t> tail{0};
3 };
4

```

```

5 struct alignas(64) ConsumerIndex {
6     std::atomic<std::uint64_t> head{0};
7 };

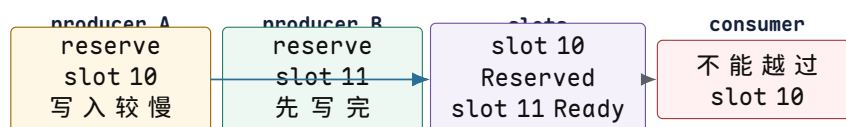
```

14.3 SPSC、MPSC、MPMC 不能混讲

队列难度由写者数量决定。SPSC 每端单写者。MPSC 有多个生产者争用 `tail`，消费者独占 `head`。SPMC 生产者独占 `tail`，多个消费者争用 `head`。MPMC 两端都有多个写者，通常需要每槽位序号、CAS 预留、提交状态和复杂失败路径。不要把 SPSC 加一个 CAS 就宣传成 MPMC。

MPSC 的预留和提交 MPSC 中，多个生产者可以用 CAS 或 `fetch_add` 预留槽位，但预留不等于数据已可读。生产者 A 可能预留了较早槽位却还没写完，生产者 B 预留了较后槽位并写完。消费者不能越过 A 读取 B，除非算法允许乱序并写清语义。因此 MPSC 常需要槽位状态：Reserved、Ready、Consumed，或类似序号。`tail` 表示预留进度，`slot state` 表示数据发布进度。

MPSC：预留顺序不等于提交顺序



`tail` 只说明槽位已被预留。消费者能否读取，要看每个槽位自己的提交状态或 `sequence`。

图示：本图要证明的命题是：多生产者队列必须区分预留进度和可读提交进度。

MPMC 的槽位序号模型 经典有界 MPMC 队列常让每个槽位维护 `sequence`。生产者根据 `tail` 找槽位，看到 `sequence` 表示空闲后 CAS 预留，写数据，再 `release` 更新 `sequence` 表示可读。消费者根据 `head` 找槽位，看到 `sequence` 表示可读后 CAS 预留读取，读数据，再更新 `sequence` 表示下一轮空闲。`Sequence` 用于区分同一数组位置的不同轮次，避免旧 Empty/Ready 被误认。

这个模型很强，但 Compute Systems Engine 不应轻率自研生产级 MPMC。关闭、阻塞等待、动态分配、异常、元素析构、CAS 失败风暴和性能退避都会进来。若确有 MPMC 需求，优先评估成熟库；若只是线程池全局队列，`mutex bounded queue` 或分片队列加 `work stealing` 往往更稳。

有界 MPMC 的槽位状态账本 MPMC 队列至少有三层状态：全局 `tail` 用于生产者抢预留，全局 `head` 用于消费者抢读取，每个槽位的 `sequence` 用于说明该轮次是空、可读还是已消费。全局索引只解决“谁拿到一个位置”，槽位 `sequence` 才解决“这个位置当前是哪一轮事实”。缺少 `sequence`，数组位置环绕后，旧的 Ready/Empty 状态会被误认为新一轮状态。

状态字段	解决的问题	典型错误
<code>tail</code>	多生产者预留哪个槽位	预留后未写完就让消费者读
<code>head</code> <code>slot.sequence</code>	多消费者竞争哪个槽位 区分环绕轮次和可读性	两个消费者读同一元素 旧 Ready 被当成新 Ready
<code>slot.storage</code>	存放元素生命周期	构造失败后槽位状态无法恢复
外层 <code>close</code>	等待、取消、 <code>drain</code> 、阻塞策略	把关闭塞进每个槽位导致协议爆炸

成熟库也需要外层合同

成熟库能提供经过验证的核心算法，但不能替代项目自己的外层合同。容量、关闭、取消、背压、元素析构、`allocator`、线程绑定、监控指标和错误传播仍然是使用者责任。把一个高质量 MPMC 队列塞进系统，不会自动解决“队列满时怎么办”“消费者退出时生产者如何醒来”“请求取消后已经入队任务如何回收”这些问题。

14.4 线性化、回收和进展：无锁结构的三份证明

无锁数据结构的实现说明至少要有三份证明。第一份是线性化证明：每个成功或失败操作在哪个瞬间对外生效。第二份是内存回收证明：任何线程持有的节点不会被提前释放或复用成另一个逻辑对象。第三份是进展证明：在竞争、满、空、线程暂停和取消条件下，系统承诺什么级别的前进。

线性化证明的写法 以 Treiber stack 为例，`push` 的线性化点通常是把新节点 CAS 到 `head` 成功的瞬间；`pop` 的线性化点通常是把 `head` 从旧节点 CAS 到 `next` 成功的瞬间；`pop` 失败返回空的线性化点是读取到 `head==nullptr` 并确认失败的瞬间。若 CAS 失败，操作没有生效，必须重新读取状态。

操作	线性化点	失败路径
SPSC <code>push</code>	<code>release</code> 存储新 <code>tail</code>	满时读取 <code>head</code> 后返回 <code>false</code>
SPSC <code>pop</code>	<code>release</code> 存储新 <code>head</code>	空时 <code>acquire</code> 读取 <code>tail</code> 后返回空
Treiber <code>push</code>	CAS <code>head</code> 成功	CAS 失败重读 <code>head</code>
Treiber <code>pop</code>	CAS <code>head</code> 成功	空栈读取到 <code>null</code> 后返回空
MPMC <code>push</code>	<code>slot sequence</code> 发布 Ready	预留失败或满状态返回 <code>false</code>

线性化点必须和内存序配套。生产者在发布 `tail` 或 `slot sequence` 之前写入数据；消费者 `acquire` 观察发布状态之后读取数据。若把 `release/acquire` 去掉，线性化点可能仍然“看起来存在”，但普通数据未必被正确发布。

内存回收证明的写法 CAS 比较的是地址值，不比较对象历史。一个线程读取旧 `head` 后，即使还没有 CAS，也可能持有指向节点的裸指针。其他线程不能在它放弃或保护该指针前释放节点。解决路线有三类：`hazard pointer` 让线程公开“我正在看这个节点”；`epoch-based reclamation` 等所有活跃线程离开旧 `epoch` 后再释放；引用计数或 `tagged pointer` 用额外状态降低 ABA 风险。

Listing 14.4 节点回收证明要回答的问题

```
when a thread reads a node pointer:
```

how is the node protected from deletion?

when pop removes a node:
where is the node retired?

before delete/reuse:
how do we know no thread can still hold the old pointer?

before CAS:
how do we distinguish same address, different lifetime?

固定数组 ring 避开了很多节点释放问题，因为槽位地址不归还给通用 allocator；链表、栈和 freelist 则必须写回收协议。没有回收证明的 lock-free 链表，在小 benchmark 中可能跑得很快，在真实压力下会变成 use-after-free 或 ABA。

进展证明要避开隐藏阻塞 一个算法核心使用 CAS，不代表整体 lock-free。若每次 push 都可能调用阻塞 allocator，若满队列时无限自旋等待消费者，若关闭路径等待某个已取消线程清理槽位，进展保证就变了。报告应列出所有可能阻塞点：内存分配、等待事件、系统调用、日志、回调、析构、外部锁。只要热路径依赖这些点，就不能简单宣称 lock-free。

常见误区

无锁结构的正确性不是“用了 atomic 就安全”。必须同时证明线性化点、发布/观察内存序、节点生命周期和进展条件。缺一项，都可能只是把锁里的复杂性搬到了更难调试的位置。

ABA 的三种处理取舍 Tagged pointer 在指针旁边加版本号，CAS 比较地址和版本；它简单直接，但受指针位数、原子宽度和溢出周期限制。Hazard pointer 不阻止 ABA 本身，而是阻止节点在仍可能被观察时释放复用；它适合节点结构，但每次读指针要发布 hazard，读路径有额外成本。Epoch 回收批量延迟释放，读路径轻，但若某线程长期停在旧 epoch，会积压退休节点。工程选择取决于读写频率、节点数量、线程生命周期和延迟要求。

方案	优点	代价
tagged pointer	CAS 直接识别版本变化	需要足够原子宽度，版本可能回绕
hazard pointer	精确保护线程正在看的节点	读路径发布 hazard，扫描成本
epoch 回收	读路径轻，批量回收	慢线程会阻塞回收，内存峰值上升
固定数组槽位	不释放节点，证明简单	容量固定，需 sequence 区分轮次

成熟无锁队列或并发容器可以减少内部协议错误，但不能替外层系统决定语义。库通常回答“push/pop 在线程安全下如何工作”，不回答“关闭时是否 drain”“失败后谁重试”“buffer 何时可复用”“迟到 completion 是否仍能提交”“队列满是否阻塞还是返回失败”。Compute Systems Engine 即使使用成熟库，也要在外层写清 Channel、TaskQueue 或 MessageQueue 的生命周期合同。

14.5 从 CAS 到对象生命周期：无锁结构真正难在哪里

CAS 只改变一个机器字大小或有限宽度的原子位置。一个工程队列或栈却要维护对象构造、发布、读取、析构、回收、关闭、失败和等待。很多无锁示例短，是因

为它们把元素类型限制为整数，把节点永不释放，或者把关闭语义放在外面。真实系统不能这样逃避：元素可能有析构函数，构造可能抛异常，节点要回收，消费者可能取消，生产者可能在关闭后迟到，线程可能暂停很久。无锁结构的难点不是写出一个 CAS 循环，而是证明每个对象在任何并发交错下只被构造一次、读取一次、析构一次，并且释放时没有线程仍持有旧观察。

原子状态和对象状态必须分开 一个槽位或节点通常同时有两类状态。原子状态用于线程间协调，例如 Empty、Reserved、Ready、Consumed；对象状态描述存储里是否已经构造了一个有效 T，是否已经被移动走，是否需要析构。把二者混在一个布尔值里，会在构造失败、移动失败或关闭竞态中出错。即使元素类型在项目里暂时是简单整数，接口也应说明是否支持非平凡对象；如果不支持，就把限制写进合同。

Listing 14.5 槽位协议要同时维护两张账本

```
atomic slot state:
    Empty -> ReservedByProducer -> Ready -> ReservedByConsumer -> Empty

object lifetime:
    uninitialized storage
    construct T in place
    publish Ready only after construction succeeds
    consumer reads or moves T
    destroy T exactly once
    mark storage reusable only after destruction
```

这两张账本的顺序不能颠倒。生产者必须先构造对象，再 release 发布 Ready；消费者 acquire 看到 Ready 后才能读取对象；消费者完成读取和析构后，才能把槽位发布为空闲。若生产者先发布 Ready 再构造，消费者可能读取未初始化存储；若消费者先发布 Empty 再析构，生产者可能在同一存储上构造新对象，和旧对象析构交叠。

动态分配会改变进展保证 许多无锁算法的进展保证只覆盖内部原子协议，不覆盖内存分配器。一次 newNode 可能拿锁、触发系统调用、等待其他线程释放 arena，甚至失败。若 push 路径依赖动态分配，它就很难保持 wait-free；若 pop 后立即 delete，又会遇到 hazard/epoch 回收和析构成本。固定容量 ring 的优势不仅是数组访问快，更是把内存生命周期提前到队列创建时，避免在热路径里引入分配器协议。

设计	购买的能力	付出的成本
固定容量 SPSC ring	无节点回收，字段单写者，生命周期清楚	容量固定，角色限制强
动态链式队列	容量弹性，适合未知负载	分配器、节点回收、ABA、析构路径复杂
对象池 + generation 成熟 MPMC 库	降低系统分配，地址复用可追踪 内部协议经过更多验证	池耗尽、generation 回绕、旧句柄测试复杂 外层关闭、背压、错误语义仍需自定义

关闭语义会穿透无锁协议 队列关闭不是在 push 里加一个 if 那么简单。关闭要回答：关闭后新 push 是返回 false、抛异常还是阻塞；已经 Ready 的元素是否 drain；等待中的消费者如何醒来；生产者预留了槽位但尚未发布时，关闭如何等待或取消；失败元素是否析构；调用者如何知道所有元素已经处理。无锁内部可以只提供非阻塞 try_push/try_pop，但外层 Channel 必须用状态机、semaphore、condition variable 或 park/wake 协议补齐等待和关闭。

Listing 14.6 Channel 外层关闭协议示意

```

Open:
  try_push may publish elements
  try_pop may consume elements

ClosingDrain:
  new push is rejected
  existing Ready elements remain consumable
  waiters are woken to observe state

Closed:
  no Ready elements remain
  storage is safe to destroy
  late producers receive a stable rejection result

```

这也是为什么无锁不等于没有等待。内部数据结构可以无锁，外层仍然需要背压和等待协议；否则生产者在队列满时只能忙等，消费者在队列空时只能空转。高吞吐结构如果用满 CPU 换吞吐，可能不适合 Compute Systems Engine 的 I/O completion 或后台任务。

测试要覆盖生命周期，不只覆盖线性化 线性化测试验证并发历史能否解释成某个顺序历史；生命周期测试验证对象是否被构造、移动、析构、释放和复用得正确。二者缺一不可。一个队列可能线性化正确，却在异常构造后泄漏槽位；也可能普通对象正确，却在析构有副作用的类型上双析构。测试类型应故意记录构造、移动、析构和活动对象数；容量应故意设得很小，制造满、空、环绕、关闭和重试。

Listing 14.7 无锁生命周期测试矩阵

```

element types:
  trivial integer
  move-only object
  object with destructor counter
  object whose construction can fail in a controlled test

queue states:
  empty pop
  full push
  wrap-around
  producer closes while consumer drains
  consumer stops while producer sees full

checks:
  linearized sequence
  constructed == destroyed
  no slot reused before destruction
  no accepted push after close
  no busy-spin without backoff budget

```

这些测试会让“短小无锁代码”的真实复杂度显形。若一个使用场景只需要单生产者、单消费者、固定容量、平凡元素和外层关闭，SPSC ring 很合适；若需求包含多生产者、多消费者、异常、动态容量和复杂关闭，成熟库加清晰外层合同通常比自研更可靠。

14.6 ABA 和内存回收：CAS 看不见历史

Treiber stack 事故的根源是 CAS 只比较当前值。若 head 先是地址 A，中间变成 B，又变回地址 A，CAS 看到的位模式仍然是 A。可对象历史已经变了：旧 A 可能被 pop、释放、复用成新节点。无锁链式结构必须处理这种历史丢失，否则测试很难稳定复现，线上却可能在对象池、高并发和线程暂停中出现。

地址相等不等于对象相同 在普通顺序程序里，指针值常被自然理解成对象身份。无锁结构中，这个理解会失效。分配器可以把刚释放的地址重新分配给新对象；对象池更会主动复用节点地址，以减少系统分配成本。于是同一个地址 A 在时间上可以代表旧节点 A0，也可以代表新节点 A1。CAS 比较的是位模式 A，不比较“这是第几代 A”。这就是 ABA 的根本：地址相等只说明当前位置位模式相同，不说明中间历史没有变化。

层次	看到的事实	丢失的事实
CAS	head 当前位模式是否等于 A	A 中间是否离开过 head，A 是否换代
指针	某个虚拟地址值	对象是否仍存活，地址是否被复用
对象池	空闲节点可再次返回	外部线程是否还持有旧观察
回收协议	哪些线程可能仍在读	需要登记、扫描或 epoch 证明

这张表说明，ABA 不是“CAS 太弱”这么简单。CAS 的工作本来就是比较当前位模式。若算法需要比较历史，就必须把历史编码进被比较的值，或者禁止地址在可能被旧观察使用期间复用。前者是 tagged pointer 的方向，后者是 hazard pointer、epoch、RCU 或固定槽位的方向。

ABA 的最小事件链 先写事件链，而不是直接写解决方案：

Listing 14.8 Treiber stack ABA 的事件链

```
T1 reads head = A
T1 reads A.next = B
T1 is paused

T2 pops A
T2 pops B
T2 frees A and B to the pool
T2 allocates a new node, pool returns address A
T2 pushes new A

T1 resumes
T1 CAS head from A to B succeeds by address comparison
T1 installs a pointer to old B, whose lifetime is no longer valid
```

这条链说了两个事实。第一，ABA 不要求硬件错误，所有 CAS 都可能按规范成功。第二，问题不只是 head 值，而是节点生命周期和地址复用。只把 head 改成更强内存序不能修复历史问题，因为 acquire/release 解决的是可见性，不是“这个地址是否仍代表同一个对象”。

ABA 需要暂停窗口 ABA 往往依赖一个线程在关键窗口中暂停：读到 head=A，还没有成功 CAS，中间被抢占、等待 cache miss、发生 page fault、被调度器换出，或者被调试器、信号、GC、安全点打断。另一个线程利用这段时间 pop、push、释放和复用节点。无锁算法不能假设“这段窗口很短所以不会发生”，因为系统调度、I/O 中断和高负载都可能把短窗口拉长。正确性证明必须允许任意暂停。

这也是测试 ABA 困难的原因。正常压力测试可能很久不触发精确交错；一旦在线上出现线程暂停和对象池复用，错误又可能变成低概率内存破坏。测试需要人为

插入延迟、强制对象池快速复用同一地址、暂停特定线程、开启 sanitizers，并记录 head 版本或节点 generation。没有故障注入，只靠跑吞吐，很难覆盖 ABA。

Listing 14.9 ABA 测试需要放大的条件

```
force allocator or pool to reuse recently freed nodes
pause a thread after it reads head and before CAS
let another thread pop, free, reuse, and push nodes
resume the paused thread
check whether CAS can install a stale next pointer
```

带版本指针 一种修复方向是让被比较的值包含地址和版本。每次 head 改变时增加版本，即使地址回到 A，版本也不同。这样 CAS 比较的是 (pointer, tag)，而不是裸指针。代价是需要能原子比较足够宽的值，或者把指针和 tag 压进一个机器字；还要处理 tag 溢出、对齐位和平台可移植性。

Listing 14.10 带版本 head 的抽象形态

```
TaggedPointer:
  pointer = address of node
  tag = incremented on every head update

CAS expected(pointer=A, tag=17)
  desired(pointer=B, tag=18)
```

带版本指针降低 ABA 风险，但不自动解决节点释放。T1 读取 A 后，如果 T2 释放了 A，T1 在读取 A.next 时仍可能访问已释放内存。也就是说，tag 保护 head 历史，不保护所有已读节点的生命周期。

Tagged pointer 解决的是哪一段历史 Tagged pointer 常被过度宣传。它能防止 head 从 A 变到别处又回到 A 时被误认为没有变化，因为 tag 会改变；但它不能阻止线程在发布保护之前解引用已释放节点，也不能让节点内部字段自动可见，还不能解决 tag 溢出后的重复历史。若 tag 位数很少，在极高频更新下理论上可能绕回；实际风险取决于位数、更新频率和暂停窗口。工程报告必须写清 tag 宽度、平台假设、溢出处理和仍需的回收协议。

一个安全的链式无锁结构通常同时需要两层保护：比较值里有足够身份，防止状态历史被当前位模式掩盖；节点释放被延迟到没有线程可能持有旧观察之后，防止悬空解引用。只做其中一半，常常只是把崩溃概率降低，而不是把协议证明完整。

Hazard pointer 的直觉 Hazard pointer 的思路是：线程在解引用某个共享节点前，先把“我可能正在使用这个节点”的指针发布到一个全局可见的 hazard slot。删除节点的线程不能立刻释放节点，而是把它放进 retired list；当扫描所有 hazard slot 后确认没有线程还持有该节点，才真正释放。它用更多元数据和扫描成本换取安全回收。

Listing 14.11 Hazard pointer pop 的抽象流程

```
repeat:
  p = head.load(acquire)
  publish_hazard(current_thread, p)
  if head.load(acquire) != p:
    clear_hazard(current_thread)
    continue
  next = p.next
```

```

if CAS(head, p, next):
    clear_hazard(current_thread)
    retire_later(p)
    return p.value

```

重新读取 head 是必要的：线程发布 hazard 之前，p 可能已经被别人 pop 并准备回收。发布后再次确认 head 仍是 p，才能安全读取 `p->next`。这类协议很细，说明无锁内存回收不适合靠印象手写。

Hazard pointer 的责任转移 Hazard pointer 的核心动作不是“保存一个指针”，而是把责任写成协议。读取线程负责在解引用前发布 hazard，并在离开读侧临界区后清除；删除线程负责不立即释放节点，而是退休节点并定期扫描所有 hazard；内存管理器负责只有在没有 hazard 指向节点时才回收。任何一方偷懒都会破坏安全性。

Listing 14.12 Hazard pointer 的责任分工

```

reader:
    load candidate pointer
    publish hazard pointer with sufficient ordering
    re-check that shared pointer still equals candidate
    safely read candidate fields
    clear hazard before leaving the operation

remover:
    unlink node with a successful CAS
    move node to retired list
    scan hazard slots before freeing retired nodes

runtime:
    register and unregister thread hazard slots
    define behavior when a thread exits or is canceled
    bound retired memory or report stalled reclamation

```

这份分工会带来指标需求：hazard slot 数量、retired list 长度、扫描次数、单次扫描成本、最长未清除 hazard 时间。无锁结构如果只报 push/pop 吞吐，不报回收债务，就可能把内存增长藏到后面。

Epoch 回收的直觉 Epoch 回收把时间分成阶段。线程进入无锁操作时标记自己处于当前 epoch；删除节点时不立即释放，而是标记在某个 retired epoch；只有当所有线程都离开早期 epoch 后，才释放那些 retired 节点。它适合短临界无锁操作和线程集合可控的运行。若某个线程长时间停在 epoch 内，回收会停滞，内存上涨。若线程崩溃或被取消，epoch 系统必须有登记和注销协议。

Epoch 的失败模式 Epoch 的优势是读路径轻，失败模式是回收被慢线程拖住。一个 worker 进入 epoch 后执行了长时间阻塞 I/O，或者被调度器换出很久，或者在读侧临界区里调用用户回调，都会阻止旧 epoch 退休节点释放。系统表面上仍然 lock-free，内存却持续上涨，最后触发 page fault、回收压力甚至 OOM。Epoch 方案必须规定读侧临界区里禁止阻塞、禁止调用未知用户代码、禁止持有跨 task 的 epoch guard。

Listing 14.13 epoch guard 的使用约束

```

allowed inside epoch:
    load shared pointers

```

```
read node fields
perform bounded local computation

not allowed inside epoch:
    blocking I/O
    waiting on condition variables or futures
    invoking arbitrary callbacks
    allocating unbounded memory
    keeping the guard across task boundaries
```

这条约束把无锁回收和运行时调度连接起来。一个线程池如果允许 task 在 epoch 内等待同池 future，回收可能被永久卡住；如果取消或关闭没有注销线程 epoch，retired 内存也可能无法释放。无锁不是只写数据结构，还要把线程生命周期纳入协议。

回收方案	购买的能力	主要代价
不释放节点	避免 ABA 和悬垂访问	内存随操作增长，只适合短生命周期测试
Tagged pointer	区分 head 历史变化	不保护已读节点生命周期，平台限制多
Hazard pointer	精确保护当前被读节点	每线程 hazard slot、retire 扫描、实现复杂
Epoch 回收	批量回收，热路径较轻	慢线程阻塞回收，线程注册和退出复杂
固定 ring slots	避免节点释放，生命周期简单	容量固定，适用模型窄

项目中的取舍 Compute Systems Engine 的主路径应优先使用固定容量 SPSC ring、mutex bounded queue、分片队列或成熟并发库。Treiber stack 和 MPMC 队列适合作为实验和源码阅读对象，而不是默认基础设施。若真的需要无锁链式结构，代码实践卷必须实现或引入可审计的内存回收，并配套压力测试、延迟注入、对象池复用和 sanitizers。原理部分要把风险讲清，避免把 CAS 当成锁的简单替代。

14.7 退避、让出和背压

CAS 失败时立即重试，在低竞争下很快；在高竞争下，会让所有核心反复争同一条 line。退避策略让失败线程暂停一点、让出 CPU 或转入外层等待，降低互连风暴。退避不是语义修复，它只改变竞争成本；队列满、下游慢和写入阻塞仍然需要背压合同。

失败次数是第一类指标 无锁结构的报告必须记录 CAS 成功次数、失败次数、平均重试、最大重试、满/空返回、退避次数、yield 次数和 CPU 占用。吞吐高但 CAS 失败极多，说明结构可能靠烧 CPU 换平均值；p99 高且失败集中，说明某些线程饥饿。没有这些指标，就无法区分“无锁结构很快”和“基准输入碰巧竞争低”。

Listing 14.14 无锁结构的运行指标

```
1 struct NonBlockingStats {
2     std::uint64_t push_success = 0;
3     std::uint64_t push_fail_full = 0;
4     std::uint64_t pop_success = 0;
5     std::uint64_t pop_fail_empty = 0;
6     std::uint64_t cas_success = 0;
```

```

7  std::uint64_t cas_failure = 0;
8  std::uint64_t backoff_count = 0;
9  std::uint64_t max_retry_observed = 0;
10 };

```

这些统计本身不能变成共享写热点。高频路径可以使用 per-worker 统计，阶段结束后汇总。用一个全局 atomic 记录每次 CAS 失败，会把观测系统变成新的瓶颈。

退避和调度的边界 短退避可以用 pause 指令或空转几轮，适合预计马上成功的竞争；稍长退避可以 yield，让调度器运行其他 runnable thread；更长等待应进入条件变量、semaphore、atomic::wait 或运行时事件。队列满若代表下游长期慢，就应背压上游，而不是让生产者无限自旋。队列空若代表没有任务，就应让 worker 睡眠或窃取，而不是占满 CPU。

等待长度	合理动作	错误动作
极短竞争	pause、有限重试	立刻系统调用导致过重开销
短期失衡	yield、尝试本地其他任务	无限重试同一个热点 line
下游慢	有界队列、semaphore、背压	无界排队或忙等满队列
系统关闭	广播退出、drain/cancel	等待一个永远不会变化的状态

无锁和背压不是对立面 底层 ring 可以 non-blocking，外层 Channel 仍然可以 blocking 或 backpressured。比如生产者先尝试 try_push；失败后登记等待、释放 CPU，等消费者释放空间后再醒来。这样热路径低延迟，慢路径不烧 CPU。把 non-blocking API 暴露给上层后，如果上层只会 while 重试，系统整体仍然不健康。无锁结构要嵌入运行时背压设计，而不是替代背压。成熟 MPMC 队列通常只承诺队列操作，不承诺业务关闭语义。库可能支持 non-blocking try_enqueue，但不知道任务是否可取消；可能支持 blocking pop，但不知道 shutdown 时是否 drain；可能要求元素 noexcept move，或者在内部长期持有内存块。引入库前要把库合同翻译成第 10 章的 QueueContract，否则 benchmark 再好也可能不适合项目。

14.8 Treiber 栈：短代码背后的生命周期

Treiber stack 的 push/pop 看起来短，正好适合作为反面案例：如果只展示 CAS，不讲回收，就是不完整。push 分配节点，把 node->next 指向旧 head，CAS head 为新节点。pop 读取旧 head 和 next，CAS head 为 next。线性化点通常是 CAS 成功，但 pop 的节点何时能释放，仍然没有答案。

Listing 14.15 Treiber 栈的示例骨架，不包含回收协议

```

1 struct StackNode {
2     std::int32_t value = 0;
3     StackNode* next = nullptr;
4 };
5
6 class LockFreeStack {
7 public:
8     void push(StackNode* node) {
9         StackNode* observed = this->head_.load(std::memory_order_relaxed);
10        do {
11            node->next = observed;

```

```

12     } while (!this->head_.compare_exchange_weak(
13         observed,
14         node,
15         std::memory_order_release,
16         std::memory_order_relaxed));
17 }
18
19 private:
20     std::atomic<StackNode*> head_{nullptr};
21 };

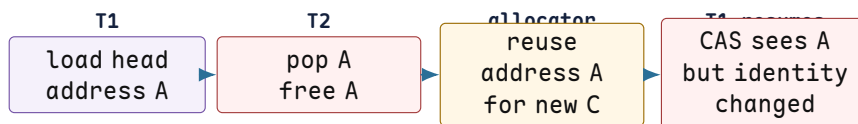
```

这段代码不能作为完整栈。它没有 pop，更没有说明节点所有权。节点由调用者拥有、栈拥有、对象池拥有，还是回收系统拥有？pop 返回节点后，其他线程是否还可能持有旧指针？这些问题决定结构能否用于生产。

ABA 时间线 ABA 可以这样发生：T1 读取 `head=A`，并读取 `A->next=B`，随后暂停。T2 pop A，又 pop B，释放 A；对象池把地址 A 分给新节点 C，并 push 到栈顶。T1 恢复后看到 `head` 的地址仍是 A，CAS 成功，把 `head` 改成旧的 B。地址相同，生命周期已经不同，结构被破坏。

这个故事里有两条独立线索。第一条是状态历史：`head` 经历了 `A -> B -> A`，单纯比较当前位模式看不出中间变化。第二条是对象生命周期：第一次的 A 和第二次的 A 可能是同一地址上的两代对象，旧的 `A->next` 已经不再描述新对象。Tagged pointer 主要处理状态历史；hazard pointer、epoch、RCU 主要处理释放时机。真实结构常常两类问题同时存在，不能只靠更强内存序修复。

ABA：地址相同不代表对象身份相同



CAS 只比较位模式。若旧指针仍可能被线程持有，释放和复用地址就会制造 ABA。

图示：本图要证明的命题是：无锁节点从结构中摘除后，不能立即等同于无人访问。

Tagged pointer 和 generation 带标签指针把指针或索引与 generation 一起比较。地址 A 回到 `head` 时，generation 已变化，CAS 不会误认为状态没变。这里更推荐用数组索引加 generation，而不是偷指针低位，因为后者依赖对齐、地址空间和平台 ABI。Generation 思想也适用于对象池、句柄系统和分布式 epoch：位置相同，不代表身份相同。

Listing 14.16 标签句柄的语义形状

```

1 struct TaggedIndex {
2     std::uint32_t index = 0;
3     std::uint32_t generation = 0;
4 };

```

对象池句柄比裸指针更适合入门 Compute Systems Engine 可以用固定数组对象池和 `TaggedIndex` 句柄模拟节点身份。数组 `index` 说明位置，`generation` 说明该位置当前是哪一代对象。释放槽位时 `generation` 增加，旧句柄自然失效。这个

模型比裸指针更容易测试 ABA，因为可以打印 index/generation，也能故意把对象池缩小到 2 个槽位快速复用位置。

Listing 14.17 对象池句柄检查把身份问题显式化

```

1 struct PoolHandle {
2     std::uint32_t index = 0;
3     std::uint32_t generation = 0;
4 };
5
6 struct PoolSlot {
7     std::uint32_t generation = 0;
8     StackNode node{};
9 };
10
11 bool is_live(const PoolSlot& slot, PoolHandle handle) {
12     return slot.generation == handle.generation;
13 }

```

真正生产级无锁栈仍需完整回收协议；tagged handle 只是让“同一位置不同身份”变得可见。若 generation 位数太小，长时间运行后可能溢出；若旧句柄能跨很久保存，溢出后仍可能 ABA。因此报告要写 generation 位数、最大复用速率和是否接受溢出风险。

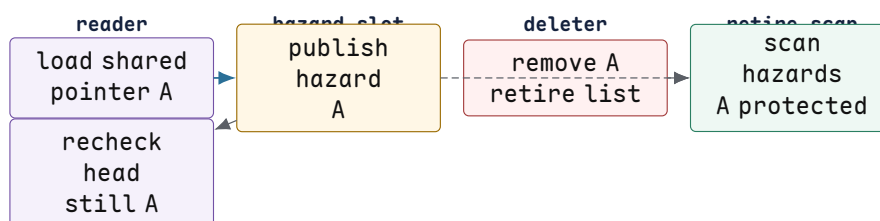
14.9 回收协议：Hazard、Epoch、RCU

无锁链式结构必须把“从结构中摘除”和“释放内存”分开。摘除说明新的入口路径到不了该节点，不说明旧操作没有持有指针。C++ 没有自动垃圾回收，必须显式证明没有线程仍可能解引用旧节点。

Hazard pointer：先声明，再解引用 Hazard pointer 的流程是：线程读取共享指针；将该指针发布到自己的 hazard 槽位；重新读取共享指针确认仍然相同；确认后才能解引用。删除者不能直接释放节点，而是放入 retired 列表；当列表足够大时，扫描所有 hazard 槽位，释放没有被任何线程声明保护的节点。

它的优点是精确，某个节点没人保护就能释放；代价是读路径要发布 hazard，释放路径要扫描所有线程槽位。线程退出时必须清空槽位，否则 retired 节点可能永远不能释放。Compute Systems Engine 可以固定 worker 数简化注册，但文档必须说明假设。

Hazard pointer：保护声明必须早于解引用



删除者只把节点移入 retired 列表。只有扫描不到任何 hazard 槽位保护该节点时，才能真正释放。

图示：本图要证明的命题是：无锁读取方必须先公开自己可能解引用的节点，删除者才能安全延迟释放。

Hazard 的失败模式 Hazard pointer 最危险的 bug 通常不是 CAS 写错，而是

保护窗口写错。读取方如果先解引用节点，再发布 hazard，删除者可能已经扫描并释放该节点；读取方如果发布 hazard 后不重新读取 head，可能保护的是已经离开结构的旧地址；线程退出时如果不清空 hazard 槽位，删除者会永久保留某些 retired 节点；扫描时如果没有足够同步，删除者可能看不到刚发布的保护声明。

Listing 14.18 错误 hazard 顺序：先解引用后声明

```
reader:
    observed = head.load()
    next = observed->next          ; use-after-free window
    hazard_slot.store(observed)

deleter:
    remove observed from stack
    scan hazard slots and sees no observed
    free observed
```

正确顺序的关键不是“多写一个槽位”，而是责任转移：读取方在解引用前把风险公开；删除者在释放前观察所有公开风险。两侧必须用内存序和注册协议连接起来。工程实现还要处理线程注册、槽位复用、异常退出、线程池 worker 复用、retired list 阈值和扫描成本。若这些边界没有写进接口，hazard 代码会比锁版代码更难维护。

Hazard 扫描成本怎样进入设计 扫描所有 hazard 槽位是 $O(\text{thread_count} \times \text{hazard_slots_per_thread})$ 的工作。若每弹出一个节点就扫描一次，释放路径会被扫描成本支配；若 retired list 阈值太大，内存峰值会上升。常见做法是批量退休：每个线程维护本地 retired 列表，列表达到阈值后复制所有 hazard 指针到临时集合，再释放不在集合中的节点。阈值要和线程数、节点大小、延迟目标一起选择。

Listing 14.19 Hazard retired 扫描的成本账本

```
per thread:
    retired_nodes
    hazard_slots

when retired_nodes.size >= threshold:
    collect all non-null hazard slots
    for each retired node:
        if node not in collected hazards:
            free node
        else:
            keep it retired
```

报告不能只说“使用 hazard pointer”。它要写最大 retired 数、扫描次数、扫描耗时、最大保留年龄、线程退出清理方式和槽位上限。否则性能下降或内存上涨时，无法判断是数据结构争用、回收阈值、线程停顿还是节点生命周期 bug。

Epoch：批量延迟回收 Epoch 回收把时间分成阶段。线程进入无锁操作时记录当前 epoch，退出时离开。删除节点时记录退休 epoch。只有当所有线程都离开旧 epoch 后，旧 epoch 中退休的节点才能释放。它把单节点精确保护换成批量延迟回收。

Epoch 的读路径通常更轻，但害怕暂停线程。若某个线程进入 epoch 后被抢占、阻塞 I/O、执行未知回调或长时间不退出，旧节点无法释放，retired 列表和内存

峰值会增长。Epoch 临界区必须短，不应包含阻塞等待和用户回调。这个约束要写进报告。

Epoch 停顿的具体事故 Epoch 的失败常常很安静。线程 T1 进入 epoch E，读取队列 head 后被操作系统抢占，或者在临界区里调用了可能阻塞的日志函数。其他线程继续 pop，成千上万个节点进入 retired list，但全局最老 active epoch 仍然是 E。回收器不能证明 T1 没有旧指针，只能继续保留这些节点。吞吐看起来正常，内存却持续增长。

Listing 14.20 Epoch 被暂停线程卡住的事件链

```
T1 enters epoch 17
T1 reads a shared pointer
T1 is paused before leaving epoch

T2..T8 remove many nodes
retire nodes with epoch 17, 18, 19

reclaimer checks active threads:
  T1 is still active in epoch 17
  cannot free nodes retired after epoch 17 boundary

memory grows until T1 leaves or is declared inactive by policy
```

这条链说明 epoch 临界区必须小，不能跨 I/O、锁等待、用户回调、条件变量等待或长时间计算。线程池可以给每个 worker 注册 epoch guard，但 guard 的作用域必须包住“读取共享无锁指针到完成 CAS 或放弃”的短窗口，而不是包住整个 task。若 worker 可能被外部暂停，报告要写最大 epoch 停顿时间和内存上限策略。

Epoch 和 Hazard 的取舍不是快慢二选一 Hazard 更精确，读取方要发布指针，回收方扫描槽位；epoch 读取方更轻，回收延迟更粗，害怕停顿线程。链式栈、队列、跳表、无锁哈希表和只读快照各自适合不同方案。小型项目中，若节点数量有限、运行时间短、目标是理解线性化点，可以先用“不释放直到容器销毁”作 reference；若要长时间运行，再引入 hazard 或 epoch，并把回收指标作为功能的一部分。

方案	保护对象	主要失败	观测指标
Hazard	单个正在解引用的节点	发布太晚、槽位泄漏、扫描成本高	hazard slots、scans、retired age
Epoch	一段短临界区内可能访问的节点	暂停线程卡住回收、临界区过大	oldest epoch、retired bytes、blocked thread
RCU	旧版本快照和读侧临界区	grace period 过长、写路径复制成本	version age、grace wait、old bytes
统一释放	整个容器生命周期	长进程内存不可控	peak nodes、destroy time

回收协议也要有指标 回收失败不会立刻表现为错误结果，常常表现为内存慢慢上涨。实验必须记录 retired 节点数量、最大退休年龄、epoch 推进次数、阻塞

回收的线程 id、hazard 扫描耗时和实际释放数量。没有这些指标，epoch 泄漏会被误判为“数据结构正常但内存有点高”。

14.10 完整执行账本：Treiber pop、ABA 和回收

Treiber stack 的危险在于代码短，容易让人以为 CAS 成功就等于结构正确。下面把一次 pop 从读 head 到安全释放节点写成账本。这个账本会刻意区分三类事实：共享栈头的位模式、节点对象的生命周期、线程是否仍可能持有旧指针。

没有回收协议的 pop 账本 最朴素的 pop 会读取 head，读取 head->next，然后 CAS 把 head 改为 next。CAS 成功是线性化点：从对外语义看，节点在这个瞬间离开栈。但是，节点离开栈不代表没有线程还拿着旧指针。

Listing 14.21 不完整 pop 的语义事件链

```
T1 loads head = A
T1 loads next = A.next
T1 is paused before CAS

T2 pops A successfully
T2 returns A to allocator or pool
allocator later reuses address A for another node

T1 resumes
T1 compares head with address A
CAS may succeed although object identity changed
```

这条账本说明 CAS 的比较范围太窄。它只比较当前 head 的位模式是否等于期望值；它不知道地址 A 是否被弹出过、释放过、重新构造过，也不知道 T1 读取的 A.next 是否仍属于当前对象。内存序再强也不能补上对象历史。Acquire/release 能约束可见性，不能让旧指针自动安全。

Tagged index 解决哪一半 Tagged index 把位置和 generation 一起放入 CAS 比较。T2 释放 A 并重新分配同一 index 时，generation 增加；T1 恢复后，期望值是 (index=A, generation=7)，当前 head 是 (index=A, generation=8)，CAS 失败。这样可以发现“位置回来了但历史变了”。

Listing 14.22 Tagged head 的 ABA 拒绝路径

```
T1 observes head = (slot 3, generation 7)
T2 pops slot 3 generation 7
T2 retires and reuses slot 3 as generation 8
current head = (slot 3, generation 8)
T1 CAS expected (3, 7) -> fails
```

Tagged index 不能单独解决所有释放问题。T1 在发布保护之前，如果已经解引用了旧节点，而 T2 同时释放该节点，仍可能 use-after-free。Tagged head 保护的是栈头状态历史；hazard/epoch 保护的是读取者解引用期间的节点生命周期。两者经常要配合。

Hazard pop 的完整顺序 Hazard pointer 版本的 pop 多了一个“声明可能解引用的节点”的步骤。线程先读取 head，把它写入自己的 hazard 槽位，再重新读取 head 确认没有变化。只有确认后，才能读 head->next。如果确认失败，说明 head 在发布 hazard 期间变了，要重来。

Listing 14.23 Hazard pointer pop 的语义伪代码

```

repeat:
    observed = atomic_load(head)
    if observed is null:
        clear hazard slot
        return empty

    publish hazard slot = observed
    if atomic_load(head) != observed:
        continue

    next = observed->next
    if CAS(head, observed, next) succeeds:
        clear hazard slot
        retire observed
        return observed.value

```

发布 hazard 后重新读取 head 是关键。如果少了这一步，线程可能保护了一个已经不再是 head 的节点，甚至保护的是刚被释放并复用的地址。删除者释放 retired 节点前，要扫描所有线程的 hazard 槽位；只要某个槽位仍等于该节点，就不能释放。

Hazard 的内存序直觉 Hazard 槽位发布必须对删除者可见，删除者扫描 hazard 时必须能看到足够新的保护声明。具体实现会使用 acquire/release 或更强顺序来约束发布和扫描。这里的底层直觉是：读取者在解引用节点前，先把“我可能访问这个节点”这个事实发布出去；删除者在释放节点前，必须观察所有这种事实。没有这条发布-观察关系，就不能证明释放安全。

Epoch pop 的完整顺序 Epoch 方案不保护单个指针，而是保护一段时间。线程进入无锁操作时进入当前 epoch；只要它还在 epoch 内，回收系统就假设它可能持有旧指针。pop 成功后，节点进入 retired list，标记退休 epoch。只有所有线程都离开旧 epoch，旧 epoch 退休的节点才释放。

Listing 14.24 Epoch 回收的语义伪代码

```

thread enters epoch E
perform pop with CAS
if node removed:
    add node to retired list with retired_epoch = E
thread leaves epoch

reclaimer:
    if all active threads are in epoch > E or inactive:
        free nodes retired in epoch E

```

Epoch 版本的危险在于“进入 epoch 后做太多事”。如果线程在 epoch 临界区里等待 I/O、执行用户回调、阻塞在条件变量或被长期抢占，旧节点都不能释放。Epoch 的读路径轻，是用回收延迟和暂停敏感换来的。报告必须记录最老 active epoch 和阻塞线程。

对象池小模型 为了把 ABA 讲清，实验可以使用 2 个槽位的小对象池。每次分配都会复用槽位，generation 每次递增。这样很容易制造“slot 0 被 pop、free、reuse，再回到 head”的时间线。测试日志不要只打印地址，要打印 slot, generation, next。

Listing 14.25 ABA 可视化日志示例

```
T1 load head=(slot=0, gen=5), next=(slot=1, gen=2)
T2 pop  head=(slot=0, gen=5)
pool retire slot=0 gen=5
pool allocate slot=0 gen=6
push head=(slot=0, gen=6)
T1 CAS expected=(slot=0, gen=5), actual=(slot=0, gen=6) -> fail
```

这类日志比抽象图更能形成工程判断：地址相同不代表身份相同，节点不在结构中不代表无人访问，CAS 成功必须绑定线性化点和生命周期。生产实现可以更复杂，底层事实不变。

RCU：读多写少的系统合同 RCU 可以看成读多写少的发布和回收模型。读取方进入 read-side 临界区，读取当前版本；写者复制新版本，修改后发布新指针，然后等待所有旧读取方离开 grace period，再释放旧版本。它适合路由表、配置快照、只读索引、分片元数据。代价是写路径复制、旧版本延迟回收和系统级读侧合同。

Compute Systems Engine 不需要自研完整 RCU，但应借鉴思想：driver 发布新 route table 或 config snapshot 时，worker 读取不可变版本；旧版本在没有读取方后释放。若更新频繁、对象很大或读取方可能阻塞很久，RCU 思想就不一定合适。

回收方案	适合场景	主要成本	项目建议
不释放/统一释放	一次性对象、短进程、reference	内存增长	可作 reference
Tagged handle	对象池、槽位复用、ABA 检测	版本溢出、句柄检查	推荐理解身份
Hazard pointer	指针访问复杂，回收要较精确	hazard 发布和扫描	第 12 章实验选做
Epoch	操作短、线程固定、读侧要轻	暂停线程阻塞回收	可做小模型
RCU	读多写少不可变快照	写路径复制和 grace period	借鉴设计

14.11 无锁和等待、关闭、异常

很多系统需要无锁 fast path 和可阻塞 slow path。SPSC ring 的 try_push 满时返回 false；上层可以用 semaphore、condition variable、eventfd 或运行时任务图等待空间。底层无锁结构负责快速状态转换，上层 Channel 负责 close、drain、cancel、timeout 和唤醒。把所有语义塞进一个“万能无锁队列”，通常会把证明复杂度推高到不可维护。

异常和元素类型 无锁热路径最好不抛异常。若生产者 CAS 预留槽位后，元素构造抛异常，槽位状态如何回滚？消费者如何知道槽位不可读？因此示例代码使用 std::int32_t、指针或预构造对象，不是偷懒，而是隔离同步机制。泛型无锁容器必须写明元素要求，例如 noexcept move、外部构造好对象、析构无副作用，或使用复杂槽位生命周期管理。

动态分配会改变进展保证 无锁算法内部如果调用可能阻塞的通用 allocator，就不能简单宣称整个操作 lock-free。分配器可能加锁、系统调用或在内存压力下阻塞。固定数组、有界对象池、预分配节点和回收批处理，是无锁结构常见配套。性能报告必须包含分配和回收成本，否则只测了 CAS 的一半。

退避策略是协议的一部分 CAS 失败后立刻重试，在低竞争下很快；在高竞争下可能让所有线程持续打同一条 cache line。退避策略包括短暂停顿、yield、指数

退避、切换本地队列、批量提交或回到 blocking slow path。退避不是补丁，而是负载策略。报告要记录退避次数和平均重试轮数，说明它降低了 CAS 风暴还是只是掩盖吞吐问题。

Listing 14.26 CAS 失败计数和退避让竞争可观察

```

1 bool try_transition(std::atomic<std::int32_t>& state,
2                     std::int32_t from,
3                     std::int32_t to,
4                     std::atomic<std::uint64_t>& cas_failures) {
5     std::int32_t expected = from;
6     if (state.compare_exchange_strong(expected,
7                                       to,
8                                       std::memory_order_acq_rel,
9                                       std::memory_order_acquire)) {
10         return true;
11     }
12     cas_failures.fetch_add(1, std::memory_order_relaxed);
13     return false;
14 }

```

14.12 工业设计参照

工程案例

Linux RCU 和 ring buffer。 Linux RCU 把读侧临界区、发布新版本和 grace period 回收分开，购买极轻读路径；代价是系统级约束和复杂调试。内核 ring buffer 也围绕 per-cpu、提交点和可观测性设计。可迁移原则是：读写角色、提交点和回收时机必须分开写清。

Chase-Lev work stealing deque。 Owner 从 bottom push/pop, stealer 从 top steal；大多数路径单写者，只有最后一个元素需要 CAS 仲裁。它购买的是本地性和低竞争窃取；代价是 top/bottom、扩容、内存序和边界竞争复杂。第 14 章应先用 mutex deque 验证策略，再决定是否需要无锁 deque。

Moodycamel、Folly、Boost.Lockfree 等用户态库。 成熟库通常明确支持 SPSC、MPSC、MPMC、有界/无界、阻塞/非阻塞、元素要求和内存回收策略。引入前要读接口合同，不要只看 benchmark。若库没有项目需要的 close/drain 语义，就要外层封装 `QueueContract`。

LMAX Disruptor 和序号环。 Disruptor 系列设计把序号、槽位、发布和消费者进度拆开，适合低延迟流水线。它购买的是预分配、顺序访问和清楚的发布序号；代价是模型更专用，消费者拓扑和等待策略要显式设计。可迁移原则是：高性能 ring 的核心不是“数组很快”，而是每个序号的所有权、发布和消费进度都有账本。

io_uring completion queue。 `io_uring` 的提交队列和完成队列把生产者消费者边界做成共享 ring，但业务仍要处理 request id、buffer 生命周期、取消和迟到 completion。它说明无锁或低锁 ring 只能交付事件，不能替业务层证明提交语义。第 15 章的 I/O 管线会复用这个边界。

JVM/Go 等有 GC 的运行时。 垃圾回收能缓解节点回收和 ABA 的一部分风险，因为对象不会在仍可达时释放；但 CAS 线性化点、状态机、竞争失败和队列语义仍然存在。C++ 语境中必须把回收显式讲出，因为运行时不会自动兜底。

14.13 实现边界：LockFreeLifecycle

Compute Systems Engine 不应把所有队列改成无锁。更合理的目标是建立 LockFreeLifecycle 边界：对每个候选无锁结构，写明使用场景、生产者消费者数量、容量、线性化点、进展保证、内存序、ABA 防护、回收策略、关闭外置方式、验证矩阵、性能对照和回退方案。

字段	要回答的问题	示例
模型	SPSC、MPSC、SPMC、MPMC、stack、deque	reader -> parser 是 SPSC
容量	有界还是无界，满时如何处理	满 时 返 回 false，Channel 背压
线性化点	成功和失败操作何时生效	push 发布 tail
进展保证	blocking、lock-free、wait-free，依赖什么	SPSC try 操作无阻塞分配
内存序	哪 些 release/acquire 发布数据	tail 发布槽位内容
回收	固 定 数 组、统 一 释 放、hazard、epoch、RCU	SPSC 固定数组无需节点回收
关闭	close/drain/cancel 在 哪 一层实现	外层 QueueContract
指标	CAS 失 败、满 空 次 数、retired 长度、尾延迟	per-worker 汇总
回退	锁版 reference 是否保留	mutex bounded queue

推荐迁移路线 第一步，保留 mutex bounded queue 作为 reference 和默认实现。第二步，在一对一管线中实现 SPSC ring，用严格验证证明不丢不重。第三步，若出现多生产者热点，优先尝试分片 SPSC 或每 worker 队列；只有 profile 证明需要时，再评估成熟 MPSC/MPMC。第四步，无锁栈只作为 ABA 和回收示例模型，不进入核心路径。第五步，所有无锁路径都必须输出 CAS 失败、满空次数、内存峰值和关闭时间。

选型矩阵 项目不应该问“要不要无锁”，而应问“哪个边界需要哪种并发结构”。单个生产者到单个消费者的高频数据流，SPSC ring 是合理候选；多个 worker 提交到一个低频控制队列，mutex queue 更稳；工作窃取 deque 适合 owner 本地 push/pop 很多、steal 较少的任务运行时；分布式消息模拟通常更需要可追踪状态和故障注入，而不是极限无锁。

场景	首选结构	理由
reader -> parser 一 对 一 流	SPSC ring + 外 层 Channel	角色固定，热路径清楚
全局任务提交队列	mutex bounded queue	关闭、异常、等待语义更重要
每 worker 本地任务	deque，先锁版再优化	本地性比全局无锁更关键
多生产者日志缓冲	分片 buffer 或 成熟 MPSC	避免所有线程争一个 tail
配置/路由快照	atomic shared pointer 或 RCU 思想	读多写少，不可变版本更清楚
ABA 示例模型	小 对 象 池 + tagged handle	容易复现和解释身份变化

两条必要对照 第一条对照是 SPSC ring 与 mutex queue。实现 `SpSCIntQueue` 和锁版 `BoundedQueue<int32_t>`，用同一 producer/consumer 输入生成 0 到 N-1 的序号，验证不丢、不重、严格递增。容量要覆盖 1、2、3、64 和非二次幂，证据保留吞吐、满/空返回次数、CPU 占用和尾延迟。再故意用两个 producer 调用 SPSC，说明接口约束被破坏后为什么不能保证正确。

第二条对照是 ABA 与回收可视化。实现一个只有 2 到 4 个槽位的小对象池，每个槽位带 generation。构造 Treiber stack 的示例模型，让一个线程暂停在读取 head 后，另一个线程 pop、free、reuse 同一 index，再让第一个线程恢复。判读分别比较裸 index、tagged index、统一释放、hazard pointer 小模型和 epoch 小模型的行为差异。重点不是吞吐，而是能用日志解释每个节点的身份变化和释放时机。

测试矩阵 无锁测试必须比锁版更严。测试容量一、容量二、非二次幂、wrap around、多轮循环、随机 `yield`、生产者快、消费者快、长时间压力。Treiber/节点结构测试要使用小对象池快速复用地地址，制造 ABA 条件；开启 AddressSanitizer/ThreadSanitizer 时要理解工具限制；记录 retired 列表长度，防止回收静默失控。

线性化测试和生命周期测试要分开 无锁结构至少要通过两类测试。线性化测试检查操作是否能排列成一个合法的顺序历史：push 成功后元素可被 pop，pop 不能凭空返回不存在的元素，empty 返回只能发生在某个合法瞬间。生命周期测试检查节点是否仍然活着：读取方拿到指针后，删除方不能提前释放；retired 节点最终能释放；线程退出不会留下永久 hazard；epoch 不会被阻塞线程无限卡住。两类测试互相独立，缺一不可。

Listing 14.27 无锁结构测试分层

```
linearizability:
  operation history
  begin/end interval
  success/failure result
  item identity
  legal sequential model

lifetime:
  allocation id
  generation
  retire time
  protection state
  actual free time
  sanitizer status
```

Treiber 栈可能在线性化上看似正确，却因为缺少回收协议出现 use-after-free；hazard pointer 版本可能生命周期安全，却因为 CAS 失败路径没有重读状态导致返回值错误。测试报告要把这两类结果分开写，避免一个“stress passed”掩盖另一个边界。

误用测试也要保留 无锁结构通常有强前提。SPSC ring 只允许一个生产者和一个消费者；MPSC 允许多个生产者但消费方唯一；某些结构要求元素类型移动不抛异常；某些结构不支持阻塞等待或关闭。误用测试不是为了让错误用法通过，而是为了让接口边界清楚。若两个 producer 同时调用 SPSC，测试可以断言 debug 版本触发检查，或文档明确标为未定义使用；不能悄悄表现得“偶尔也能跑”。

误用场景	应暴露的边界	推荐处理
两个 producer 使用 SPSC	单写者不变量被破坏	debug 断言或测试标红
close 直接写进 ring 槽位	数据协议和生命周期混杂	外层 Channel 管理关闭
epoch 临界区内阻塞 I/O	回收可能被无限卡住	禁止并记录最大临界区时间
hazard 槽位线程退出未清理	retired 节点泄漏	注册/注销测试和泄漏检查
CAS 循环内写日志或分配	失败重试产生重复副作用	副作用移到 CAS 成功后

这类测试会让接口更窄，但窄接口比模糊接口安全。无锁结构真正难的地方不在几行 CAS，而在把可用范围压到能证明的边界内。

源码阅读任务

源码阅读以“线性化点和回收”为主线。

- 1. Linux RCU: 阅读 RCU 文档或 `include/linux/rcupdate.h`, 说明读侧临界区、`grace period` 和回收如何分工。
- 2. Linux ring buffer: 阅读 `kernel/trace/ring_buffer.c` 的高层设计, 关注 `per-cpu`、提交点和 `reader/writer` 边界。
- 3. Chase-Lev deque 或 oneTBB work stealing: 找 `owner` 路径和 `thief` 路径, 说明最后一个元素竞争如何线性化。
- 4. 成熟用户态队列库: 填写支持模型、有界/无界、关闭语义、元素要求、回收策略和测试状态。

阅读报告要说明哪些思想适合 Compute Systems Engine, 哪些复杂度暂时拒绝。

14.14 完整案例：SPSC ring 为什么比 Treiber 栈更适合作为起点

学习无锁结构时，最常见的错误是先看 Treiber 栈，因为它代码短。代码短不代表概念少。Treiber 栈立刻引入多线程 CAS、ABA、节点生命周期和内存回收；而 SPSC ring 只有一个生产者写 `tail`，一个消费者写 `head`，槽位固定不回收，线性化点和发布关系都更窄。若目标是把无锁思想落到项目里，SPSC ring 是更好的起点。

角色固定让字段单写者成立 SPSC 的关键不是数组，而是单写者。生产者独占写 `tail` 和槽位数据；消费者独占写 `head`；双方可以读取对方的索引来判断满或空。因为每个索引只有一个写者，很多竞争被设计消掉了。多生产者误用 SPSC 时，`tail` 立刻变成多写者字段，两个生产者可能写同一槽位或覆盖彼此的 `tail`。

Listing 14.28 SPSC 字段所有权

```
producer owns:
  write buffer[tail]
  write tail
  read head to check full

consumer owns:
  read buffer[head]
  write head
  read tail to check empty
```


这张所有权表比代码更重要。它说明哪些 relaxed 读取只是观察对方进度，哪些 release/acquire 发布槽位内容，哪些字段不能被第二个生产者或消费者写。

槽位固定避免节点回收 固定 ring 的槽位在队列生命周期内一直存在。push 和 pop 只是在槽位上转移元素语义，不释放槽位本身。因此它避开了 Treiber 栈最难的问题：消费者读到某个槽位地址后，生产者不会把这个槽位释放给分配器再复用成完全不同对象。槽位内容的生命周期仍要处理，例如元素移动构造和析构，但槽位地址本身稳定。

Listing 14.29 SPSC push/pop 的线性化直觉

```
try_push:
    write element into buffer[tail]
    release store new tail
    linearization: tail publication

try_pop:
    acquire load tail
    if head != tail:
        read element from buffer[head]
        release store new head
    linearization: head publication
```

生产者 release 发布 tail，是为了让消费者 acquire 看到新 tail 后能读取槽位内容。消费者 release 发布 head，是为了让生产者看到空间可复用前，知道消费者已完成读取和销毁。具体内存序可根据实现细化，但发布方向必须与槽位生命周期一致。

满空判断是容量合同 Ring 通常用一个空槽区分满和空，或者使用递增序号区分 wrap around。容量为 N 的数组若保留一个空槽，最多存 N-1 个元素；若用序号，则要处理整数回绕和序号宽度。这个选择影响 QueueContract：用户看到的容量到底是物理槽位数还是可存元素数，满时返回 false 还是阻塞，close 在外层 Channel 中如何实现。

Listing 14.30 保留一个空槽的满空判断

```
empty:
    head == tail

full:
    next(tail) == head

usable_capacity:
    physical_capacity - 1
```

若文档写容量为 1024，实际只能存 1023 个元素，而测试按 1024 验收，就会出现边界争议。无锁结构的接口必须把这种细节写成合同。

为什么 Treiber 栈更难 Treiber 栈的 head 是多线程共同 CAS 的指针。一个线程读 head 后，还要读 head->next；在它 CAS 之前，另一个线程可能 pop head、释放节点、再把同一地址作为新节点 push 回来。第一个线程看到 head 地址没变，CAS 可能成功，但对象历史已经变了。这就是 ABA。即使用 tagged pointer 检测历史变化，读到节点后它是否仍然活着仍要靠 hazard pointer、epoch 或统一释放保护。

Listing 14.31 Treiber pop 的两个独立问题

```

ABA problem:
    head address changed away and back
    CAS compares the same address and cannot see history

reclamation problem:
    thread reads head pointer
    another thread frees that node
    first thread dereferences a freed object
    
```

这两个问题不同。Tagged pointer 主要增加历史身份；hazard/epoch 主要保护释放时机。把它们混在一起，会误以为“加个 counter 就解决无锁栈”。实际上，counter 不能阻止另一个线程释放节点；hazard 也不一定防止所有 ABA 历史误判，取决于地址复用和算法形状。

项目选型结论 Compute Systems Engine 的一对一字节流、parser 到 aggregator 的单向管线，可以考虑 SPSC ring，因为角色固定、槽位固定、关闭可由外层 Channel 处理。全局任务队列、分布式 MessageBus、可靠 I/O completion 则更看重关闭、取消、迟到事件和故障注入，锁版队列或成熟库更合适。Treiber 栈应保留为理解 ABA 和回收的模型，不作为默认工程结构。

问题	SPSC ring	Treiber stack
写者数量	head/tail 单写者	head 多写者 CAS
节点地址	固定槽位，随队列生命周期存在	动态节点，必须回收
线性化点	发布 head/tail	CAS head 成功
关闭语义	外层 Channel 处理较清楚	需要额外状态和回收协同
主要风险	误用多生产者、多消费者，容量边界	ABA、悬垂指针、回收延迟

验收必须包含误用测试 SPSC ring 的报告不仅要证明正确用法快，也要证明接口约束清楚。测试应故意用两个 producer 调用同一个 SPSC ring，说明这不是受支持模型；debug 版本可以用 owner thread id 或构造期绑定断言发现误用。生产实现不一定保留这些检查，但文档和测试必须说明：SPSC 的正确性依赖角色固定，而不是依赖运气。

无锁结构的性能报告不能只报吞吐 无锁队列常用吞吐图说服人，但工程选型还要看失败次数、空轮询、满返回、CAS 失败、退避时间、尾延迟、CPU 占用、内存峰值、retired 节点数量和关闭时间。一个队列平均吞吐高，却让消费者忙等占满核心，或让 epoch 回收长期不能释放内存，仍然可能不适合项目。Lock-free 保证的是系统级进展，不是低资源消耗。

Listing 14.32 LockFreeLifecycleReport 的指标扩展

```

throughput
p95_latency
p99_latency
cas_failures
empty_polls
full_returns
backoff_ns
retired_nodes_max
    
```

```
reclamation_lag
close_drain_time
```

这些指标把“无锁是否值得”变成可复查问题。若 mutex bounded queue 已经满足吞吐且关闭语义清楚，而无锁版本只在极端 microbenchmark 中更快、真实路径 p99 更差，就应保留锁版。

回退方案必须随无锁结构一起提交 任何无锁结构都要有回退方案：锁版 reference、成熟库替代、或分片降低竞争的设计。回退不是失败预案，而是上线条件。若线上发现某个 ARM 平台内存序 bug、某个输入导致 retired 节点堆积、某个关闭路径卡住，系统必须能切回已验证的锁版队列，同时保留同一 `QueueContract`。接口稳定，内部实现才有优化空间。

14.15 展开：linearizability、progress 和 reclamation 的证明模板

无锁结构要同时证明三件事。第一，线性化正确：并发操作看起来像按某个顺序瞬间发生。第二，进展保证成立：系统不会因为某个线程失败而整体停住。第三，内存回收安全：没有线程还可能访问的对象不会被释放。少证明任何一件，都不是生产级无锁结构。

linearizability：给每个成功操作找一个瞬间

线性化点是操作在并发历史中“生效”的瞬间。对 SPSC ring, push 的线性化点可以是发布 tail; pop 的线性化点可以是发布 head。对 Treiber stack, push/pop 的线性化点通常是 CAS head 成功。失败操作也要定义线性化点，例如 pop 看到 empty 的瞬间。

结构/操作	成功线性化点	失败线性化点
SPSC push	release store tail	读取 head 后判断 full
SPSC pop	release store head	acquire load tail 后判断 empty
Treiber push	CAS head 成功	通常重试，不暴露失败
Treiber pop	CAS head 成功	读取 head 为 null
MPMC slot push	slot sequence 发布 ready	发现 slot 不可预留

证明时要拿一条并发历史逐步标注：每个操作开始时间、结束时间、观察到的共享状态、线性化点、返回值。若找不到一个顺序能解释所有返回值，结构就不是 linearizable。压力测试能发现 bug，但不能替代这个证明。

progress：lock-free 不等于每个线程都不饿

进展保证有层级。Obstruction-free 表示一个线程单独运行足够久能完成；lock-free 表示系统整体总有某个线程能完成；wait-free 表示每个线程在有限步内完成。很多工程队列只是 lock-free，不是 wait-free。若一个线程不断 CAS 失败，它可能饿死，但只要其他线程在成功，系统仍满足 lock-free。

保证	含义	常见代价
blocking	某线程持锁停住可阻塞其他线程	简单，尾延迟受持锁者影响
obstruction-free	无竞争时可完成	需要冲突管理
lock-free	总有线程取得进展	个体可能饥饿，CAS 风暴
wait-free	每个线程有限步完成	实现复杂，常有更高常数

报告不能只写“无锁”。应写清是哪一级保证、哪些路径违反保证。例如 CAS 循环中分配内存、写日志、等待条件变量、扫描无限 retired list，都可能把进展保证降级。退避策略也要纳入合同：指数退避降低总线争用，但可能增加单请求尾延迟。

memory reclamation: 保护的是对象历史，不只是地址

Treiber pop 的危险窗口是：线程 A 读到 head，准备读 head->next；线程 B pop 并释放这个节点；分配器把同一地址复用；线程 A 再解引用旧地址。这里同时有两个问题：地址历史可能 ABA，节点释放时机可能过早。

Listing 14.33 Treiber pop 的危险窗口

```
Thread A:
  p = head.load()
  // paused before reading p->next

Thread B:
  pop p
  delete p
  allocate q at same address
  push q

Thread A:
  reads p->next through stale object identity
```

Hazard pointer 的证明思路是：线程 A 在解引用前先发布“我可能访问 p”，释放线程扫描所有 hazard slot，只有没有任何线程保护 p 时才回收。Epoch 的证明思路是：释放不立即发生，节点进入 retired list；等所有线程都离开可能持有旧指针的 epoch 后，再批量释放。RCU 的证明思路类似：读侧临界区很轻，写侧等待 grace period。

三种回收协议的状态机

协议	状态机核心	主要失败模式
Hazard pointer	publish hazard -> validate pointer -> dereference -> clear hazard -> retire scan	忘 clear 泄漏, publish 后不复查
Epoch	enter epoch -> operate -> leave epoch; retire node with epoch -> wait all advance	暂停线程阻塞回收, retired 堆积
RCU	read lock/unlock; writer publish new version -> wait grace period -> reclaim old	读侧不能阻塞太久, 更新延迟

每个协议都要测试线程退出。线程退出时 hazard slot 是否清理？epoch 是否离开？retired list 归谁处理？如果测试只覆盖正常 push/pop，不覆盖线程中途停止、异常退出、关闭和析构，就没有验证生命周期。

ABA 和回收不是同一个修复

Tagged pointer 给 head 加版本号，可以发现“地址离开又回来”的历史变化。但如果线程在读 p->next 前，p 已经被释放，版本号也救不了解引用。Hazard

pointer 保护解引用时对象不被释放，但如果算法语义还依赖“head 没有离开又回来”，仍可能需要版本或其他设计。两者解决的问题不同。

技术	主要解决	不自动解决
Tagged pointer	CAS 比较看见部分历史	旧节点是否可安全解引用
Hazard pointer	被保护节点暂不释放	所有 ABA 语义历史
Epoch	延迟释放旧节点	暂停线程导致内存增长
Object pool	index+generation 防旧句柄	多线程发布和回收协议
handle		

这张表是审查无锁代码的底线。看到“我们用了 CAS，所以无锁安全”不够；看到“我们加了版本号，所以解决 ABA”也不够；看到“我们用了 epoch，所以性能好”仍然不够。要问它分别证明了线性化、进展和回收哪一部分。

工业取舍：成熟库也要写外层合同

直接使用成熟队列库可以减少算法错误，但不能免除外层合同。你仍要写清生产者/消费者模型、阻塞/非阻塞语义、关闭协议、元素移动/析构要求、内存分配策略、backpressure、统计字段和 fallback。成熟库只解决内部并发协议；业务系统还要解决生命周期和故障语义。

Listing 14.34 无锁结构引入前的审查单

```
ConcurrencyContract:
  supported model: SPSC | MPSC | MPMC
  bounded or unbounded
  progress guarantee
  allocation behavior
  reclamation strategy
  close/cancel semantics
  element exception policy
  fallback implementation
  metrics exposed
  misuse behavior in debug build
```

如果这份合同写不出来，说明还没有资格把无锁结构放进生产路径。锁版实现也许慢一点，但它的失败模式更容易理解；无锁版本必须用证明和指标买回复杂度。

14.16 状态机：SPSC、MPMC 和关闭协议

无锁结构真正难的地方，是同一个对象同时有“槽位状态”“元素生命周期”“队列关闭状态”和“等待者状态”。下面先用 SPSC ring 固定最小状态机。

槽位状态	所有者	合法转移
Empty	producer	producer 构造元素后发布 Ready
Ready	consumer	consumer 读取或移动元素后发布 Empty
Closing	外层 channel	阻止新 push，允许 drain 或 cancel
Poisoned	debug/test harness	检测误用后拒绝继续运行

SPSC 的简洁来自单写者：producer 写 tail 和 Empty->Ready；consumer 写 head 和 Ready->Empty。若第二个 producer 加入，tail 和槽位构造就变成多写者，

原状态机失效。这个反例比“文档说不支持”更有训练价值：读者要指出哪个状态从单写者变成多写者。

MPMC 的状态机更像预留和提交协议

有界 MPMC 队列不能只靠 head/tail。producer 先抢一个序号，得到的只是预留权；元素构造完成后，还要发布 slot ready。consumer 也类似，抢到读取位置后，必须确认该 slot 属于当前轮次。

Listing 14.35 MPMC slot 状态机

```
Empty(seq = k):
    producer CAS/reserve position k

Reserved:
    producer owns construction
    element not visible to consumer

Ready(seq = k):
    release publish slot
    consumer may acquire and read

Consumed:
    consumer destroys or moves element
    slot sequence advances to next round
```

这个状态机解释了为什么 MPSC/MPMC 中“tail 增加”不等于“数据可读”。若 producer A 预留早槽位后暂停，producer B 预留晚槽位并提交，consumer 是否能越过 A，取决于队列语义。FIFO 队列不能随便越过；乱序队列可以，但必须在接口里写清。

关闭协议的工程事故

很多无锁 demo 没有关闭状态。生产系统一定有：服务退出、上游取消、超时、错误恢复、消费者提前结束。没有关闭协议时，常见事故是 producer 还在 `try_push`，consumer 已销毁队列；或者关闭后 Ready 元素没 drain，资源泄漏；或者等待线程永远等一个不会变化的状态。

Listing 14.36 Channel 关闭状态机

```
Open:
    push and pop allowed

ClosingDrain:
    new push rejected
    existing Ready items are drained

ClosingCancel:
    new push rejected
    Reserved items must complete or roll back
    Ready items may be discarded with destructor

Closed:
    all waiters woken
    storage can be destroyed
```

底层 ring 可以只提供非阻塞 `try_push/try_pop`；外层 Channel 必须补齐 `wait`、`close`、`drain`、`cancel` 和 `wake`。把关闭语义塞进几个布尔 flag，往往会产生不可线性化的历史。

数学模型：CAS 风暴的代价

假设 P 个 producer 争同一个 tail，每个失败 CAS 都要读写同一条 cache line。若成功率为 $1/P$ ，每个成功 push 平均伴随约 $P-1$ 次失败尝试：

$$attempts_per_success \approx P$$

这不是严格预测，但足以说明为什么高竞争 MPMC 可能吞吐差、尾延迟高、功耗高。退避能减少无效尝试，但会增加单操作等待；分片队列能降低竞争，但引入全局顺序和负载均衡问题。工业取舍不是“无锁一定快”，而是选择哪条状态线允许被多少线程争。

实验：证明和压力测试要分工

习题与作业

实验一：为 SPSC ring 写并发历史表。至少包含一个 push 成功、一个 pop 成功、一个 empty 失败、一个 full 失败。标出每个操作的线性化点。

实验二：故意用两个 producer 调用 SPSC ring。加入 debug owner 检查，证明错误不是随机，而是破坏了 tail 单写者状态。

实验三：实现小对象池加 tagged index，构造 ABA trace。日志必须打印 index、generation、next 和线程步骤，不能只打印地址。

实验四：给一个队列库填写 `ConcurrencyContract`。若无法回答关闭、回收、元素异常和 progress guarantee，就判定它只能作为实验依赖，不能进入主路径。

14.17 接口延伸

无锁结构的本质不是“没有锁”，而是用更细的原子协议维护并发对象。SPSC ring 依靠单写者所有权和 `release/acquire` 发布；MPSC/MPMC 需要预留、提交和槽位序号；Treiber 栈展示 CAS 线性化点，也暴露 ABA 和回收难题；hazard pointer、epoch 和 RCU 都是在回答“旧节点何时安全释放”。Lock-free 保证系统整体进展，不保证每个线程低延迟；无锁代码必须可证明、可测试、可回退。

并发同步到这里形成闭环：第 9 章把线程和 worker 状态变得可见；第 10 章用锁和条件变量保护不变量与等待谓词；第 11 章把窄状态迁移成可审查的原子协议；无锁结构进一步固定一条边界：若继续走向无锁，就必须把线性化点和生命周期一起带上。第 14 章的任务运行时与工作窃取，会复用这些边界。

14.18 习题

习题与作业

1. 为 `SpscIntQueue` 写不变量表：容量、head、tail、满/空判断、发布顺序和单写者约束。指出 `try_push` 和 `try_pop` 的线性化点。
2. 用两个 producer 误用 SPSC ring，说明它为什么不再正确。不要只说“文档禁止”，要指出哪个字段变成多写者。
3. 画出 Treiber stack 的 ABA 时间线，并分别说明 tagged pointer、hazard pointer 和 epoch 如何缓解不同部分的问题。
4. 设计一个对象池句柄 `TaggedIndex`，包含 index 和 generation。说明 generation 何时增加，旧句柄如何失效。

5. 比较 hazard pointer 和 epoch: 读路径成本、释放延迟、暂停线程影响、线程退出处理、适用场景。
6. 填写一个 `LockFreeLifecycleReport`, 决定 Compute Systems Engine 的某条一对一管线是否值得从 mutex queue 替换成 SPSC ring。

工程案例

知识点小案例: SPSC ring 一换成两个 producer 就不成立。 SPSC 队列的特例很清楚: 只有 producer 写 `tail`, 只有 consumer 写 `head`, 槽位由 `release/acquire` 在两端发布。若第二个 producer 也来写 `tail`, 两个写者可能同时认为自己占有同一个槽位, 或者覆盖对方尚未提交的数据。由此推出规律: 无锁结构的正确性常常依赖“谁是唯一写者”这种所有权事实, 而不是依赖某个 `atomic` 类型本身。工程上接口名、文档、测试和断言都要暴露生产者/消费者数量; 违反模型时应选择 MPSC/MPMC 结构, 而不是偷偷复用 SPSC。

第 15 章

无锁正确性证明、回收与实现边界

15.1 为什么原理之后还要证明

第 12 章已经说明了 SPSC、MPSC、MPMC、ABA、hazard pointer 和 epoch 的核心问题。但无锁结构只讲直觉仍然不够。真正能进入工程的无锁实现，必须给出三个证明面：线性化证明、进展证明和内存回收证明。缺少任意一面，结构都可能在小测试中正确，在压力、取消、线程暂停或对象池复用失败。

Listing 15.1 无锁实现的三份证明

```
correctness:
  linearizability
  memory_order_happens_before
  reclamation_safety

progress:
  blocking / obstruction-free / lock-free / wait-free
  allocator_dependency
  backoff_and_starvation

engineering:
  stress_test
  sanitizer
  model_or_litmus_test
  metrics
  failure_replay
```

15.2 Linearizability: 每个操作在哪一瞬间生效

线性化证明的任务，是为每个公开操作找一个瞬间，使并发历史等价于某个串行历史。对栈而言，push 成功 CAS 通常是线性化点；pop 成功 CAS 通常是线性化点；pop 发现空的线性化点是读取到空 head 的那个观察点，但前提是没有并发 push 已经在线性化。失败返回同样需要证明。

操作	候选线性化点	证明要点
SPSC push 成功	release 写 tail	slot 数据先于 tail 可见
SPSC pop 成功	release 写 head 或 acquire 观察 tail	数据读取属于已发布区间
Treiber push 成功	CAS head 成功	新节点 next 指向旧 head
Treiber pop 成功	CAS head 成功	被弹节点从结构中唯一移除
MPMC enqueue	slot sequence 发布 Ready	预留和可读提交要分开
MPMC dequeue	消费者 CAS 或 sequence 更新	元素只被一个消费者取得

证明不能停在“CAS 成功”。CAS 成功只是修改了某个原子对象。你还要证明这个修改与普通数据写入之间有 release/acquire 关系，证明没有另一个线程能同时取得同一元素，证明失败路径没有把未发布数据当成可见事实。

15.3 内存序证明：发布的是普通数据，不是心情

无锁结构通常用 relaxed 读写索引，用 release/acquire 发布数据。关键问题是：哪个普通写必须先于哪个普通读可见？SPSC push 中，生产者先写槽位，再 release 存储 tail；消费者 acquire 读取 tail 后读槽位。这样 slot 写入 happens-before 消费者读取。

Listing 15.2 SPSC push/pop 的 happens-before 证明

```
producer:
    buffer[tail] = value
    tail.store(next, release)

consumer:
    observed_tail = tail.load(acquire)
    value = buffer[head]

proof:
    release tail synchronizes-with acquire tail
    buffer write is sequenced-before release
    buffer read is sequenced-after acquire
    therefore consumer sees published value
```

若把 tail 存储放到写槽位之前，或者把消费者 acquire 改成没有同步关系的普通读，就可能读到未初始化或旧值。内存序不是装饰参数，而是把普通内存和原子状态连接起来的合同。

15.4 ABA：地址相同不代表对象相同

ABA 的本质是历史丢失。线程 T1 观察到 head 是地址 A；T2 把 A 弹出并释放；allocator 又把同一地址给了新节点；T1 再看 head 仍是 A，于是 CAS 成功。CAS 只比较位模式，不比较对象生命周期。解决 ABA 可以用 tagged pointer、hazard pointer、epoch reclamation、引用计数或固定数组 sequence。每种方案都转移成本。

方案	解决方式	代价
tagged pointer	指针加版本号	位宽、溢出、平台约束
hazard pointer	读者声明正在保护的地址	每线程槽位、扫描成本、保护顺序严格
epoch reclamation	旧 epoch 无活动读者后回收	暂停线程会阻塞回收
固定数组 sequence	地址不释放，用轮次区分状态	有界容量、槽位状态更复杂
引用计数	对象存活由计数控制	原子成本和循环引用问题

15.5 Hazard pointer：保护窗口必须重新读取

Hazard pointer 的关键顺序是：先读取候选指针，发布到 hazard 槽位，再重新读取共享指针确认没有变化。少了重新读取，就可能保护了已经不再是结构头的对象；发布 hazard 太晚，就可能在发布前对象已经被回收。

Listing 15.3 hazard pointer 的最小保护顺序

```
retry:
    p = head.load(acquire)
    hazard_slot.store(p, release)
    if (head.load(acquire) != p):
```



```

    goto retry
// p is protected while hazard_slot == p
next = p->next
if (CAS(head, p, next)):
    hazard_slot.store(nullptr, release)
    retire(p)

```

回收线程扫描所有 hazard 槽位。retired list 中不被任何 hazard 保护的节点才能真正 delete。这个算法不是免费：线程越多，扫描越贵；retired list 太小会频繁扫描，太大又占内存；线程退出时必须清理 hazard 槽位；异常和取消路径必须释放保护。

15.6 Epoch reclamation: 快，但怕暂停线程

Epoch based reclamation 把时间分成 epoch。线程进入无锁读临界区时声明自己处于当前 epoch；节点 retired 时记录 retire epoch；当所有活动线程都离开旧 epoch 后，旧节点可以回收。它的 fast path 通常比 hazard pointer 轻，但一个线程如果长时间停在临界区或被暂停，可能阻止旧 epoch 回收，导致 retired list 增长。

Listing 15.4 epoch 回收的状态机

```

ThreadOutside
-> enter critical section, publish local epoch
ThreadActive(epoch)
-> read lock-free structure
-> leave, clear active flag
ThreadOutside

NodeLinked
-> removed by successful CAS
Retired(epoch)
-> all active threads are newer or inactive
Reclaimed

```

Epoch 适合短读临界区和受控线程集合。若线程可能在临界区里做 I/O、等待 future、阻塞系统调用或长期暂停，epoch 回收会失去边界。工程报告要记录最大 retired list、最长 active epoch、线程退出清理和压力下内存峰值。

15.7 RCU: 读多写少的另一种选择

RCU 的核心是让读者几乎不阻塞，写者发布新版本，并等待 grace period 后回收旧版本。它适合读多写少、读路径极短、更新可以复制或延迟发布的场景。RCU 不是通用队列替代品。若更新频繁、对象很大或读者可能长期停留，写者延迟和内存占用会增加。

教学项目中，RCU 可用于配置表、路由表或模型版本指针：请求读取当前版本，更新线程构造新版本后原子发布，旧版本等无读者后释放。它不适合把每个 token 的 KV cache 更新都做成 RCU；那会把高频写路径变成版本垃圾制造器。

15.8 进展证明要包含 allocator 和等待策略

很多论文算法假设内存分配不会阻塞，节点不会耗尽，线程不会在临界区暂停。工程实现不能忽略这些条件。若 push 在循环中调用 new，而 allocator 内部拿锁或 mmap，那么整体进展性质已经依赖 blocking allocator。若队列满时自旋等待，

系统在下游停顿时会烧 CPU。若关闭路径等待所有线程退出，取消和析构也属于进展证明的一部分。

Listing 15.5 进展证明字段

```
progress_proof:
  public_operations
  shared_atomic_fields
  possible_retry_loops
  allocation_in_fast_path
  full_or_empty_behavior
  blocking_calls_inside_operation
  cancellation_and_close_path
  starvation_risk
  backoff_strategy
```

15.9 测试矩阵：无锁需要压力和反例

无锁测试不能只跑一次吞吐。至少要覆盖：

- 单线程退化路径：空、满、容量边界、环绕。
- 多线程压力：不同生产者/消费者比例、不同线程绑定、不同输入速率。
- 生命周期压力：对象池复用、快速 allocate/free、retired list 增长。
- 取消和关闭：生产者退出、消费者退出、drain、stop。
- sanitizer：TSAN 能发现部分数据竞争，但不能证明线性化。
- model/litmus：小容量、小步数穷举或随机调度重放。
- 指标：CAS 失败次数、平均重试、p99 重试、retired list 峰值、CPU 占用。

15.10 验收标准

一个无锁结构进入默认路径前，必须提交：

1. 公开接口语义和失败语义。
2. 每个操作的线性化点。
3. 每个普通数据发布的 release/acquire 证明。
4. ABA 和内存回收策略。
5. 进展保证和不满足保证的条件。
6. 压力测试、sanitizer、模型测试或故障重放。
7. 与 mutex/condition variable 版本的吞吐、CPU 占用和尾延迟对比。

核心思想

无锁结构的门槛不在 CAS 语法，而在证明。不能写出线性化点、回收安全和进展边界的无锁代码，不应进入生产路径。

工程案例

知识点小案例：CAS 成功不代表旧节点可以马上释放。Treiber stack 的特例是线程 A 读到 top 指向节点 X，准备 CAS；线程 B pop 出 X 并释放，又把同一地址复用成新节点；线程 A 的 CAS 可能看到“地址没变”而误以为状态没变。ABA 和回收问题说明：线性化点只解决状态更新顺序，不解决对象生命周期。由此推出规律：无锁证明必须同时写线性化点、ABA 防护和回收策略。工程上没有 hazard pointer、epoch、RCU 或 tagged pointer 这类方案时，不能把含指针回收的无锁结构放进默认路径。

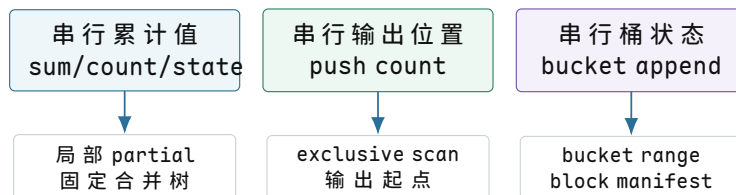
归约、扫描与分区

先看一个并行 filter 的常见事故。多个线程扫描输入，遇到满足条件的元素就把它 `push_back` 到同一个输出 vector。加锁后结果正确但很慢；改成原子递增下标后，输出位置唯一了，顺序却变成线程调度顺序；改成每个线程一个本地 vector 后，最后拼接又要说明按什么顺序拼接，失败后哪些元素有效也说不清。问题不在于没有找到“更快的容器”，而在于并程序没有先回答：每个输出元素到底写到哪里。

从 CPU 的视角看，串行 `push_back` 并不是一个抽象动作，而是一串具体内存事件：读取 `size`，检查 `capacity`，必要时分配并搬移存储，向 `data[size]` 写入元素，再更新 `size`。其中 `size`、`capacity` 和 `data` 指针通常落在少数几条 cache line 上。多个核心同时修改这些字段时，cache coherence 必须不断把这些 cache line 的独占所有权从一个核心转给另一个核心。锁能把结构保护起来，却把所有命中元素都排成串行队列；原子 `fetch_add` 能给每个元素唯一编号，却仍然让所有核心争同一个计数器 cache line；每线程本地 vector 消除了热计数器，却把全局顺序、拼接边界和失败恢复留到后面补账。

因此三个基础模式不是从术语表里来的，而是从内存写入约束里被推出来的。归约（reduce）把一个全局累计状态拆成多个局部 partial，等高频更新结束后再合并；扫描（scan）把“前面已经输出多少”变成可计算的前缀位置，让每个 chunk 先拿到独占写区间；分区（partition）把多个输出目的地展开成 `chunkxbucket` 的二维地址合同。它们共同解决三个问题：共享状态如何消失，输出位置如何提前确定，写出的数据如何留下可审计、可重试的身份。数据库执行器、日志计算、图处理、排序、shuffle、checkpoint 和分布式恢复都会反复使用这套骨架。

从串行隐含状态到并行显式结构



并行算法的关键不是线程 API，而是把串行循环里的隐含状态显式化为 partial、offset 和 manifest。

图示：本图要证明的命题是：reduce、scan、partition 都是在把串行状态改造成可合并、可定位、可恢复的结构。

16.1 从串行语义推导并行结构

并行算法不能从“开几个线程”开始。第一步永远是串行语义：输入中哪些元素参与计算，输出顺序是否重要，错误如何报告，浮点误差如何比较，输出是否可重试。串行 reference 可以慢，但它定义了问题。并行版本的每一次重排、切分和合并，都必须能回到 reference 解释。

Reference 和合同 以 `count_if` 为例，串行 reference 只做一件事：从左到右检查每个元素，满足谓词就加一。

Listing 16.1 串行 count-if reference

```

1 std::int64_t count_at_least(std::span<const std::int32_t> values,
2                             std::int32_t threshold) {
3     std::int64_t count = 0;
4     for (const std::int32_t value : values) {
5         if (value >= threshold) {
6             ++count;
7         }
8     }
9     return count;
10 }

```

这个 reference 暗含四个事实：每个输入元素被检查一次且只检查一次；谓词没有外部副作用；输出只是一个整数；整数加法是合并规则。并行版本可以把输入切成块，是因为这些事实允许局部计数和最后求和。若问题改成“输出所有满足谓词的元素并保持顺序”，算法结构立刻改变，因为输出位置不再是单个计数器。

半开区间和覆盖证明 数组切分应使用半开区间 `[begin,end)`。好的切分满足三个条件：所有块覆盖整个输入；相邻块不重叠；线程数大于元素数时仍然正确。一个常用公式如下：

Listing 16.2 按块编号计算半开区间

```

1 struct Range {
2     std::size_t begin = 0;
3     std::size_t end = 0;
4 };
5
6 Range block_range(std::size_t total,
7                  std::size_t block_index,
8                  std::size_t block_count) {
9     const std::size_t begin = (total * block_index) / block_count;
10    const std::size_t end = (total * (block_index + 1)) / block_count;
11    return Range{begin, end};
12 }

```

工程实现还要处理 `block_count==0` 和极大输入乘法溢出。这个公式先暴露覆盖关系：第 `k` 块的 `end` 等于第 `k+1` 块的 `begin`，第一块从 `0` 开始，最后一块到 `total` 结束。并行正确性的很多证明，都从这个切分证明开始。

ParallelSemanticContract 项目中可以引入一个合同对象，记录并行化前必须回答的问题。

Listing 16.3 ParallelSemanticContract 的示例形态

```

1 enum class OutputOrder {
2     Stable,
3     Unordered,
4     DeterministicByKey
5 };
6
7 struct ParallelSemanticContract {
8     bool has_identity = true;
9     bool associative = true;

```

```

10 bool commutative = true;
11 bool allows_floating_error = false;
12 OutputOrder output_order = OutputOrder::Stable;
13 bool retryable_by_chunk = false;
14 };

```

这个结构不是要把所有算法抽象成一个框架，而是把思考顺序固定下来。Reduce 关注 identity 和合并函数；scan 关注输出位置；partition 关注目标桶和 manifest；恢复关注 chunk 是否可重做。若合同写不清，就不应进入线程实现。

16.2 Reduce：把共享状态变成 partial

Reduce 的第一原则是避免高频共享写。多个线程对一个全局原子计数器 `fetch_add`，结果可能正确，但所有 worker 都在争同一条 cache line。线程本地 partial 把高频更新留在本地，阶段末尾再合并。这个思想在 Linux per-cpu counter、数据库 map-side combine、分布式 reduce 和高性能 telemetry 中反复出现。

Identity、结合律和确定性 Reduce 需要 identity 和合并函数。整数求和的 identity 是 0，最大值的 identity 可以是类型最小值，集合并的 identity 是空集合。若 identity 选错，空输入和空块就会错。结合律决定能否改变合并顺序；交换律决定能否重排 partial；确定性决定输出是否每次一致。

浮点求和是最好的反例。数学实数加法满足结合律，但机器浮点加法会舍入。串行左折叠、四累加器、树形合并、按 NUMA 节点分层合并，可能得到不同低位结果。若业务需要逐位可复现，就要固定合并树；若业务允许误差，就要写明绝对误差、相对误差或 ULP 边界。并行化之前必须选择语义。

Partial 的所有权 线程本地 partial 应避免 false sharing。每个 worker 写自己的槽位，最后由合并阶段读取。若 partial 很小，可以用对齐槽；若 partial 是大型 histogram 或 hash map，内存预算和合并策略就成为核心问题。

Listing 16.4 线程本地 partial 的基本形态

```

1 constexpr std::size_t CACHE_LINE_BYTES = 64;
2
3 struct alignas(CACHE_LINE_BYTES) CountPartial {
4     std::int64_t count = 0;
5 };
6
7 void count_worker(std::span<const std::int32_t> values,
8                 std::int32_t threshold,
9                 CountPartial& partial) {
10     std::int64_t local = 0;
11     for (const std::int32_t value : values) {
12         if (value >= threshold) {
13             ++local;
14         }
15     }
16     partial.count = local;
17 }

```

这个 worker 只写自己的 partial。正确性来自区间切分和整数加法；性能收益来自消除每个命中元素上的全局同步。代价也要写清：partial 占用额外空间，读取全局结果有延迟，最后合并可能成为新瓶颈。

线性合并和树形合并 线性合并最简单：一个线程从头到尾扫描 partial。worker 数少、partial 小时足够。树形合并把 partial 两两合并，关键路径从 $O(p)$ 降到 $O(\log p)$ ，其中 p 是 partial 数。树形合并的代价是更多阶段、更多临时对象和更复杂调度。

合并方式	适用场景	主要风险
线性合并	partial 很小, worker 数少, 简单优先	单线程尾部, 合并阶段不可并行
固定树形合并	需要可复现, partial 较多	调度自由度降低, 临时对象更多
动态树形合并	吞吐优先, partial 大小不均	浮点或顺序敏感结果可能不稳定
分层合并	NUMA 或分布式场景	层级设计复杂, 需要记录阶段边界

合并树是并行算法的提交顺序 Reduce 常被写成“局部算完再合并”，但合并不是附属步骤。它决定哪些 partial 被接受、按什么顺序组合、是否能并行、失败后从哪里恢复。对整数计数，线性和树形通常给出同样数值；对浮点、top-k、字符串拼接、诊断摘要和近似 sketch，合并顺序可能改变结果。即使操作满足结合律，工程系统仍要固定合并树，因为报告、checksum、重放和性能分析都依赖稳定结构。

Listing 16.5 8 个 partial 的固定树形合并

```
level 0:
  p0 p1 p2 p3 p4 p5 p6 p7

level 1:
  m01 = combine(p0, p1)
  m23 = combine(p2, p3)
  m45 = combine(p4, p5)
  m67 = combine(p6, p7)

level 2:
  m03 = combine(m01, m23)
  m47 = combine(m45, m67)

level 3:
  result = combine(m03, m47)
```

这棵树购买了并行合并和较短关键路径，也带来临时对象、任务调度、cache 访问和错误处理。若 partial 很小，树形合并的任务开销可能超过收益；若 partial 很大，树形合并可以利用多个核心，但要关注内存带宽和 NUMA 放置；若 partial 是稀疏 map，合并顺序还会影响分配器、rehash 和峰值内存。合并树要进入报告，而不是藏在实现里。

合并也有 cache 和 TLB 账本 把 64 个 worker 的 dense partial 表合并成一个全局表，表面上只是加法。硬件上可能是多次扫描大型数组。若每个 partial 表有 1 MiB，线性合并要反复读 partial、写 result；树形合并会产生中间表，读写总字节可能更多，但并行度更高；按 NUMA 分层合并可以先在节点内读取本地 partial，再跨节点合并较少结果。合并策略要用字节账本判断，而不是只用算法复杂度。

Listing 16.6 partial 合并报告字段

```
partial_count
partial_bytes_each
merge_tree_shape
merge_levels
temporary_merge_bytes
merge_order_checksum
numa_local_merge_bytes
cross_numa_merge_bytes
result_checksum
```

若 merge 时间超过局部计算时间，说明 partial 设计可能过大或合并树不合适。解决方向可能是压缩 partial、使用稀疏表示、提前 map-side combine、按 bucket 分区合并，或改变 key 编码。Reduce 的深度不在公式，而在这些字节和所有权边界。

工业参照：per-cpu counter 和 map-side combine Linux per-cpu counter 的核心思想，是高频写本 CPU 的局部状态，低频读取时再汇总。它牺牲了读取的即时一致性，换取写路径低争用。分布式计算中的 map-side combine 也是同一个思想：map worker 先本地聚合，把网络传输从“每条记录”降低到“每个本地 key 的 partial”。关键不在名字，而在取舍：写快了，读或合并更复杂；状态分散了，观测和恢复要更清楚。

浮点归约的数值合同 整数计数让 reduce 看起来简单，但很多工程计算是浮点。浮点归约不能只写“加法满足结合律”。有限精度下，合并顺序改变会改变舍入路径；FMA、fast-math、SIMD 水平合并和多线程树形合并都会影响结果。正确顺序是在性能之前先选择数值合同。

合同	适用场景	代价
逐位一致	审计、回归测试、可复现构建	固定合并树，调度自由度下降
误差范围	科学计算、统计、机器学习指标	需要误差测试和边界输入
最快近似	临时监控、趋势估计	结果可能随线程数和机器变化
补偿求和	高精度求和、抵消严重输入	计算更多，代码复杂

实验应构造四类输入：全正均匀数、极大极小混合、正负抵消、包含 NaN 或 Inf 的边界。比较串行左折叠、固定树形、动态树形、Kahan 或 pairwise sum。报告要同时给误差、耗时、重复运行一致性和编译选项。若只给“并行版快了”，没有说明误差，结论不完整。

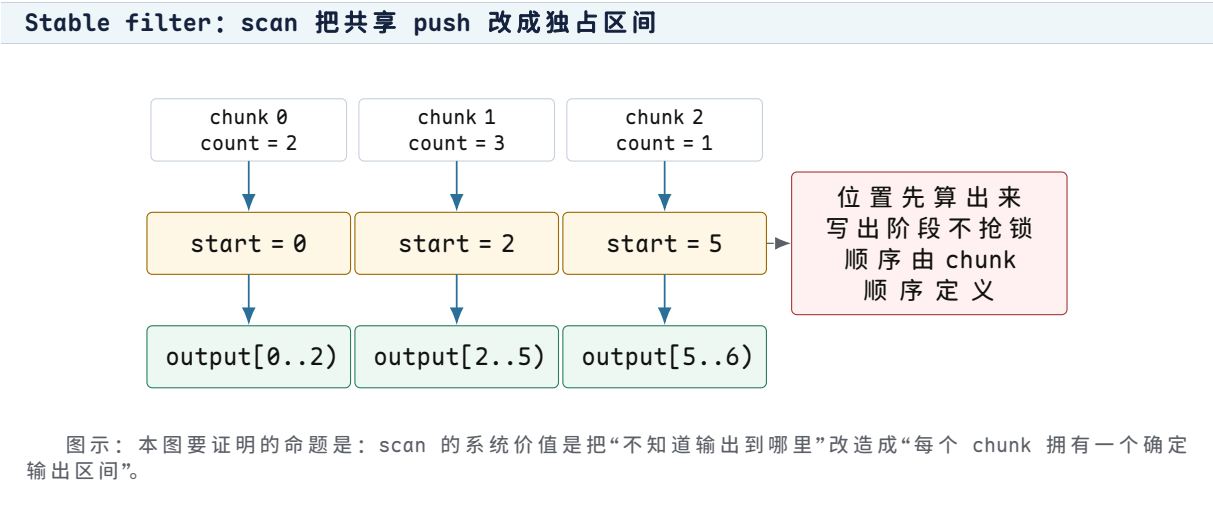
Top-k 也是 reduce Top-k 合并可以看作 reduce：每个块产生局部候选集合，合并函数把两个候选集合合成新的前 k。它要求稳定 tie-break 和候选覆盖。若局部阶段提前丢掉全局仍需要的 key，后续 reduce 无法恢复。这个例子说明：reduce 的合并函数不一定是加法，也可能是集合、堆、错误列表、统计摘要或 sketch。每一种都要写明 identity、合并顺序和信息丢弃边界。

16.3 Scan：把未知输出数量变成确定位置

Scan 也叫 prefix sum。算法题里它常被写成“计算前缀和”，但系统工程里更重要的作用是分配输出位置。稳定 filter、stream compaction、radix partition、稀疏矩阵构建、shuffle 文件 offset，都在回答同一个问题：每个并行任务要写到哪里。

从 `push_back` 推导 `scan` 串行 `filter` 中，`push_back` 隐含维护了一个状态：前面已经输出了多少元素。并行后，这个状态不能由所有线程抢。稳定 `filter` 的三阶段推导如下：

- 1. Count：每个 chunk 只统计自己会输出多少元素，不写全局输出。
- 2. Exclusive scan：对 chunk counts 做前缀和，得到每个 chunk 的输出起点。
- 3. Write：每个 chunk 重新扫描输入，把保留元素写入自己的连续区间。



Inclusive 和 exclusive Inclusive scan 的第 `i` 个输出包含第 `i` 个输入；exclusive scan 的第 `i` 个输出只包含它之前的输入。分配输出起点通常使用 exclusive scan。API 名称必须写清楚，不要只叫 `scan`。

输入	输出	语义
<code>[3,1,4]</code> inclusive	<code>[3,4,8]</code>	每个位置包含自身
<code>[3,1,4]</code> exclusive	<code>[0,3,4]</code>	每个位置是之前元素总量

Scan 的成本 并行 scan 通常比串行 scan 做更多工作。局部扫描读输入写输出，扫描 block sums，回填 offset 还可能再读写输出。小输入上串行更快；大输入上并行才可能胜出。报告必须记录输入规模、block 数、临时数组大小和阶段耗时。Scan 是强工具，但不是免费工具。

为什么连续写比共享追加更适合 CPU `count + scan + write` 看起来多扫了一遍输入，却常常比共享 `push_back` 更稳。原因在硬件路径上。共享追加把每个命中元素都压到同一组元数据：`size` 的 cache line 在核心之间转移，allocator 元数据可能被锁保护，vector 扩容会把旧数据搬到新地址，输出写入位置还取决于谁先拿到锁或原子下标。CPU 预取器很难从这种控制流里提前判断下一个写地址，store buffer 也会不断等待所有权和提交顺序。

三阶段 filter 把这些事件拆开。Count 阶段顺序读输入，只写每个 worker 的少量 partial；scan 阶段处理很小的 counts 数组；write 阶段每个 worker 写自己的连续输出区间。连续写有几个直接收益：地址递增，硬件预取器更容易跟上；写入集中在少量页面，TLB 行为更可预测；store buffer 可以按连续 cache line 排队排出；大部分 cache line 只由一个核心写。即使相邻 chunk 的输出边界落在同一条 cache line 上，争用也只发生在边界附近，不再发生在每一个命中元素上。

这个判断也解释了为什么 scan 不能只用“大 0”评价。两遍顺序扫描的渐进复杂度可能比一遍共享追加更高，但它把随机同步、扩容和所有权迁移换成了顺序读写和小数组 scan。现代 CPU 上，后者通常更容易被流水线、cache、预取和内存控制器消化。若输入很小，三阶段固定成本会输；若输入很大且选择率稳定，连续写路径通常胜出。实验要同时记录阶段耗时和内存带宽，否则无法解释这类反直觉结果。

共享追加在汇编层是什么形状 串行 `push_back` 看起来只是一个函数调用。进入并行热路径后，它至少包含三个共享事实：当前大小、容量边界和目标地址。简化到不扩容的情况，追加一个 8 字节元素可以抽象成：

Listing 16.7 不扩容追加的抽象机器动作

```
load size from vector metadata
compute target = data + size * sizeof(T)
store value to target
store size + 1 back to vector metadata
```

单线程里这很自然。多线程要让它正确，就必须保护 `size`。锁版会在每次追加前后修改锁字，锁字所在 `cache line` 在核心间转移；原子下标版可能接近：

Listing 16.8 原子抢输出位置的抽象机器动作

```
old = atomic_fetch_add(output_size, 1)
target = output_data + old * sizeof(T)
store value to target
```

在 x86-64 上，`atomic_fetch_add` 常落成带 `lock` 前缀的 read-modify-write 形态，例如 `lockxadd` 或等价循环。即使使用 `relaxed` 内存序，硬件仍要对保存 `output_size` 的整条 `cache line` 取得独占所有权，并把旧值和新值作为一个不可分割的修改发布给其他核心。所有命中元素都争同一个 `line`，扩展性自然会被压住。

Scan 的三阶段方案本质上把这条 `lockxadd` 从每个元素上移走。Count 阶段只写 `worker-local count`；exclusive scan 在小数组上计算每个 `chunk` 的起点；write 阶段用普通 `store` 写独占区间。热循环的抽象机器动作变成：

Listing 16.9 scan 后写出阶段的抽象机器动作

```
target = output_data + (chunk_begin + local_rank) * sizeof(T)
store value to target
```

这里没有共享计数器，地址随输出区间递增，store buffer 和预取器都更容易工作。若需要稳定顺序，`chunk_begin` 由按输入顺序扫描出来；若允许非稳定顺序，可以改变 `chunk` 排列，但必须在接口里声明。这样，scan 的概念不是凭空出现，而是由“不能让每个输出元素都抢同一个元数据 `line`”这个硬件事实推出来的。

两遍扫描为什么仍可能更快 Count + write 会读输入两次，直觉上似乎浪费。硬件账本显示它购买了三件东西。第一，第一次读只做谓词和局部计数，不产生随机共享写。第二，scan 数组很小，可以留在 `cache` 中。第三，第二次读虽然重复，但写出路径是连续独占区间。相比之下，共享追加的一次扫描每个命中元素都要经历锁、原子、容量检查或分配器元数据，且输出顺序和失败边界不清。

这也是为什么选择率会改变策略。选择率极低时，selection vector 或原子按块抢区间可能可接受；选择率很高时，直接 copy 或保留原 batch 可能更好；选择率中等且需要稳定输出时，count/scan/write 往往最稳。第 5 章的输入矩阵应覆盖 0%、1%、10%、50%、90%、100% 选择率，因为 scan 的固定成本和共享追加的争用成本会随选择率呈现完全不同的曲线。

错误边界 Count 阶段和 write 阶段必须使用同一谓词版本。如果谓词依赖外部状态，count 得到的数量和 write 实际写出的数量可能不一致。工程实现应在 plan 或 manifest 中记录 predicate 版本、输入范围、expected count 和 checksum。数学上的 scan 正确，不代表系统里的 scan 一定正确。

块内位置和块间位置 前面讲的是 chunk 级输出起点。若要让每个元素都知道自己的精确输出位置，还需要块内位置。常见做法是把谓词结果变成 0/1 mask，然后对块内 mask 做 exclusive scan。元素 *i* 的全局输出位置等于块起点加块内 rank。这个结构同时解释了 SIMD filter、GPU compaction 和数据库 selection vector 的位置生成。

Listing 16.10 用 mask rank 表达元素输出位置

```

1 struct SelectedPosition {
2     std::size_t input_index = 0;
3     std::size_t output_index = 0;
4 };
5
6 SelectedPosition make_selected_position(std::size_t input_index,
7                                         std::size_t block_output_begin,
8                                         std::size_t rank_in_block) {
9     return SelectedPosition{input_index, block_output_begin + rank_in_block};
10 }

```

这个示例故意只表达位置计算，不把谓词、scan 和写出混在一起。Compute Systems Engine 可以把 mask、rank、output index 分别输出到报告，帮助定位错误：是谓词错、块内 scan 错、块间 offset 错，还是写出错。

Selection vector 和材料化输出 Filter 后不一定立刻复制元素。若记录很大，可以输出 selection vector，只保存通过元素的下标；后续算子根据下标访问原始列。选择率低时，这能减少数据移动；选择率高时，下标本身也占带宽，后续间接访问可能破坏局部性。若后续要顺序扫描大量字段，材料化输出可能更好。工业执行器经常在 selection vector、bitmap 和 materialized batch 之间做取舍。

因此 scan 的结果可以服务多种输出形态：直接写紧凑数组、生成 selection vector、生成 bitmap rank、生成 block manifest。可靠实现不会把 scan 只绑定到一种容器，而是把“位置分配”作为可复用机制。

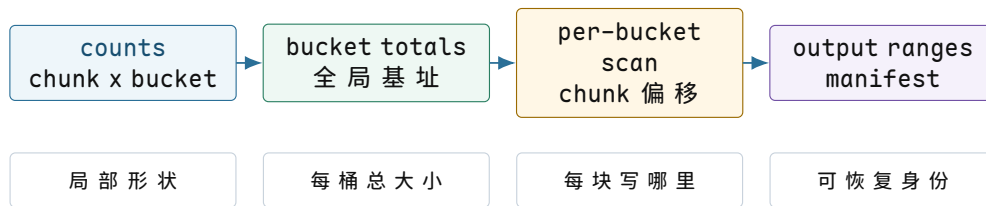
16.4 Partition: 多目的的 scan

Partition 把元素按 key、bucket 或目标 worker 分到多个输出区域。Filter 可以看成二路 partition：保留和不保留；多路 partition 则是每个 bucket 都要分配输出区间。它是单机并行算法和分布式 shuffle 之间最重要的桥。

二维计数表 多路 partition 的第一步是局部直方图：每个 chunk 统计每个 bucket 有多少元素，形成 `counts[chunk][bucket]`。第二步对每个 bucket 沿 chunk 维度做 prefix sum，得到每个 chunk 在该 bucket 中的局部起点；同时计算每个 bucket 在全局输出中的基址。两者相加，才是最终写入位置。

这里的二维表不是纸面公式，它会真实占用 cache 和 TLB。若按 `counts[chunk][bucket]` 连续存放，某个 chunk 在 count 阶段更新自己的所有 bucket 时局部性较好；但 scan 某个 bucket 时要跨 chunk 读取，访问步长等于 bucket 数。若按 `counts[bucket][chunk]` 存放，per-bucket scan 连续，count 阶段更新多个 bucket 则可能跳得更远。工程实现常用两份布局、分块转置或 bucket blocking，在 count 阶段和 scan 阶段之间交换成本。布局选择不是美观问题，而是决定 cache line 被谁连续访问、TLB 是否频繁换页、以及 NUMA 上哪颗核心会读远端内存。

多路 partition 的二维前缀



图示：本图要证明的命题是：partition 是对每个 bucket 分别做 scan，manifest 是这些输出区间的工程化身份。

Manifest 是算法结果的一部分 单机 partition 可以只返回一个重排后的数组，但工程系统还需要知道每个 bucket 的范围、记录数、字节数、checksum 和生成 attempt。这些信息就是 shuffle manifest 的雏形。

Listing 16.11 PartitionManifestEntry 的示例形态

```

1 struct PartitionManifestEntry {
2     std::int32_t partition_id = 0;
3     std::uint64_t offset = 0;
4     std::uint64_t records = 0;
5     std::uint64_t bytes = 0;
6     std::uint64_t checksum = 0;
7 };
  
```

Manifest 让失败恢复和审计成为可能。某个 chunk 写出失败时，系统知道哪些输出区间无效；重复执行同一个 chunk 时，系统知道它应该写回同一段；reduce 读取时，系统知道哪些 block 已提交。没有 manifest，partition 只是一次内存重排；有了 manifest，它就能成为分布式 shuffle 的可靠边界。

写出边界和 cache line 边界 每个 chunk 或 bucket 拥有独占区间，并不等于完全没有硬件边界成本。如果两个 worker 同时写相邻区间，而区间分界落在同一条 cache line 内，这条边界 cache line 可能在两个核心之间来回转移。好消息是，这种争用只覆盖边界处的一两条线；坏消息是，小记录、极小 chunk 或大量 bucket 会放大边界数量。处理方式有三种：增大 chunk，让边界成本摊薄；在内部临时 buffer 中按 cache line 对齐，最后串行或批量提交；对吞吐优先且允许空洞的场景，在区间之间加 padding。稳定紧凑输出通常不能随意加空洞，因此更常见的做法是让任务粒度足够大，并把边界成本写进实验解释。

写出边界还关系到恢复。若一个 chunk 的输出范围是 `[begin,end)`，commit 前失败时可以丢弃整段；若多个 chunk 交错写同一个 bucket 的同一段 cache line，恢复逻辑必须知道哪些元素有效、哪些元素只是部分写入。Manifest 把区间身份和提交状态分开：物理内存里可能已经有字节，只有 manifest commit 后才成为后续 stage 可见的事实。

稳定和非稳定 partition 稳定 partition 保持同一 bucket 内元素的原始相对顺序，适合审计、测试、可复现输出和需要稳定错误报告的场景。非稳定 partition 可以交换元素或按完成顺序写出，可能更快，但后续 stage 不能假设顺序。稳定性必须写进接口，不应由实现偶然决定。

热点 bucket Partition 解决写入位置，不自动解决负载均衡。若 70% 记录落到同一个 bucket，下游 reduce 仍会长尾。工业系统会用局部 combine、热点 key 拆分、两级 hash、范围分区或采样来缓解。报告必须记录每个 bucket 大小和最大 bucket 占比，否则无法解释后续慢在哪里。

Radix partition: histogram、scan、scatter 的组合 Radix partition 按 key 的若干 bit 分桶，常用于整数 key、哈希 join、radix sort 和数据库执行器。它的核心步骤正好是三个基础模式的组合：先做局部 histogram，得到每个桶数量；再 scan 得到桶偏移；最后 scatter 到目标区间。每轮处理多少 bit 是重要参数：bit 多，桶多，counts 矩阵大，cache 压力高；bit 少，轮数多，读写次数增加。

这说明工业实现里的“参数调优”不是玄学。Radix bits 要根据 cache、TLB、输入规模、key 宽度和线程数测量。项目可以从 8 bit 一轮和 4 bit 一轮开始比较，记录 counts 矩阵大小、临时输出字节、总读写轮数和吞吐。实验会显示，算法复杂度相同，常数和内存布局仍能决定性能。

Partition 规则要版本化 分区函数一旦改变，旧输出和新输出不能混用。分布式系统中，如果一部分 worker 使用旧 hash seed 或旧 bucket 数，另一部分使用新规则，同一个 key 会被送到不同 reducer。单机项目也应提前建立这个习惯：manifest 记录 partition version、bucket count、hash seed 或 range boundaries。Driver 发现版本不一致时拒绝合并。

Listing 16.12 PartitionPlan 记录分区规则身份

```
1 struct PartitionPlan {
2     std::uint32_t version = 0;
3     std::uint32_t bucket_count = 0;
4     std::uint64_t hash_seed = 0;
5 };
```

这个结构看起来简单，却能防止很多恢复事故。失败重试、增量执行、缓存复用和分布式 rolling upgrade 都需要知道输出是按哪套规则生成的。没有版本，manifest 只能说明“这里有一段数据”，不能说明“这段数据属于哪套分区语义”。

内存预算 多路 partition 的额外内存来自 counts、offsets、临时输出和 manifest。若 chunk_count 很大、bucket_count 很大，二维 counts 可能很可观。报告应写出实际字节，而不是只写 $O(chunks * buckets)$ 。例如 counts 使用 std::uint64_t，空间就是 chunk_count*bucket_count*8 字节；selection vector 若使用 std::uint32_t，最坏是 input_count*4 字节。

内存预算会改变算法选择。桶数太多时，可以分批处理 bucket；热点明显时，可以先采样识别热点；输出太大时，可以按 batch 写临时 block；稳定性要求低时，可以使用局部 buffer 后合并。可靠工程不是只追求吞吐，而是在时间、空间、稳定性和恢复之间选择。

16.5 工业界设计参照

工业系统中的 reduce、scan 和 partition 很少直接露出算法名称，但它们的原理到处都是。可靠设计的共性，是把局部性、输出身份、失败恢复和确定性放在一起考虑。

Linux per-cpu counter per-cpu counter 把高频写放到每个 CPU 的局部槽位，读取全局值时再汇总。它的原理就是线程本地 partial。它牺牲了全局读的即时精确性，换取写路径低争用。迁移到 Compute Systems Engine，就是 worker 本地事件计数、任务完成数和 I/O 字节数；但若计数用于协议决策，例如“所有任务是否完成”，就不能使用近似汇总。

数据库 vectorized execution DuckDB、ClickHouse、Velox 等执行器常用 batch 和 selection vector。Filter 先生成 selection vector，而不是立刻复制整行；aggregate 先本地聚合，再合并；hash partition 先统计和分配区间，再把数据送到后续 stage。关键不在“用了向量化”四个字，而是让数据布局、输出身份和 pipeline breaker 都有明确边界。

Spark shuffle 和 MapReduce combine Map-side combine 是 reduce 的网络尺度版本：先本地聚合，再跨网络发送 partial。Shuffle manifest 是 partition 输出身份的工程版本：每个 block 有分区、大小、位置和校验。失败重试时，driver 通过 manifest 决定哪些 block 有效，哪些 attempt 应被丢弃。第 13 章的 partition manifest，正是这个工业设计的单机缩小版。

并行 STL 和 oneTBB 并行算法库把范围切分、任务粒度、异常传播和合并策略封装起来。阅读它们时，不要只看是否开线程，而要看四件事：如何切 chunk，如何保存 partial，如何决定 cutoff，如何处理异常和取消。成熟库通常不会对所有输入强行并行，小输入会走串行或轻量路径。这是工业设计中的重要现实：并行也有固定成本。

源码阅读任务

工业案例阅读任务：选择 Linux per-cpu counter、DuckDB/ClickHouse aggregate、Spark shuffle、oneTBB parallel reduce 或 C++ 并行 STL 中一个实现。报告必须回答：它怎样切分输入；局部状态由谁拥有；合并顺序是否确定；输出身份如何记录；失败或异常如何处理；它能和哪个实验对照。

16.6 确定性、审计和恢复

并行算法常被默认允许“不确定顺序”，但工程项目不能这么粗糙。某些输出只关心集合内容，顺序无所谓；某些输出需要 bitwise reproducible，方便测试、缓存、审计和恢复；某些输出允许浮点误差，但要求误差范围稳定。确定性不是装饰，而是语义合同的一部分。

测试 oracle 由输出语义决定 稳定 filter 可以逐项比较；非稳定输出要排序后比较或按多重集合比较；浮点 reduce 要按误差合同比较；partition 要比较每个 bucket 的内容和范围；manifest 要比较条目顺序、checksum 和版本。很多并行 bug 不是计算错，而是测试 oracle 和语义不匹配。

恢复边界 Scan 产生的 offsets、partition 产生的 ranges、reduce 产生的 partial，都是恢复材料。若 write 阶段失败，可以根据 chunk 的输出区间重写；若 partition block 校验失败，可以只重做对应 chunk 或 bucket；若 dynamic reduce 结果不稳定，则恢复后可能无法 bitwise 对齐。算法结构直接影响恢复成本。

失败注入实验 实验要求实现一个故障注入：count 和 scan 完成后，让某个 chunk 在 write 阶段失败。恢复程序应能知道该 chunk 的输出区间无效，其他 chunk 是否可保留由 manifest 决定。若输出由共享 push_back 产生，恢复程序很难定位边界。这个实验会暴露 scan 的系统价值，而不仅是性能价值。

16.7 深度案例：把共享 push 改造成可恢复 stage

现在把 reduce、scan 和 partition 放进一个完整案例。输入是一批日志记录，目标是保留所有错误记录，并按错误码分区，供后续 reduce 统计错误类型。串行程序可以边扫边 push_back 到不同错误码 vector；并行后如果照搬这个结构，就会遇到锁、顺序、扩容和恢复问题。可靠设计要把它拆成四个 stage：count、scan、write、commit。

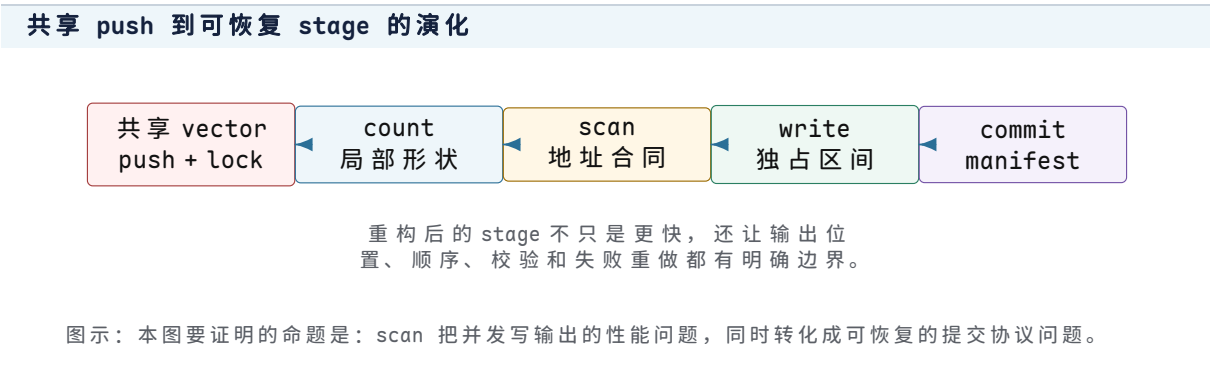
Stage 1: count 只收集形状 每个 chunk 扫描输入，统计每个错误码会输出多少条记录。它不写最终输出，只产生 counts[chunk][error_code]。这个阶段容易重做，也容易校验。若 count 阶段已经看到非法格式，可以把错误写入 chunk-local error buffer，而不是立刻提交全局错误报告。

Stage 2: scan 生成地址合同 对每个 error code 的 chunk counts 做 exclusive scan，得到每个 chunk 在该 error code 输出区域里的起点。这个结果就是地址合同：chunk 只能写自己的范围，不能越界，也不需要锁。若 write

阶段写出的数量和 count 不一致，说明谓词版本、输入或错误处理发生了变化，不能提交。

Stage 3: write 写入独占范围 每个 chunk 重新扫描输入，把错误记录写到分配给自己的区间。为了保持稳定顺序，chunk 按输入范围顺序分配区间，chunk 内按原始顺序写。若业务不要求稳定顺序，可以选择更快的非稳定版本，但 manifest 必须标注输出顺序语义。

Stage 4: commit 提交 manifest write 完成后，系统为每个 error code 输出一个或多个 manifest 条目：错误码、offset、records、bytes、checksum、chunk id、attempt。只有 commit 成功的条目才被后续 stage 读取。若 chunk 失败，未提交条目被丢弃或标记失败；重试 chunk 时写回同一语义范围或新的 attempt 临时范围，再由 commit 选择有效版本。



为什么这是工程化思路 这个案例的四阶段结构和数据库执行器、shuffle 写出、日志处理管线非常接近。工业系统常把正常路径和恢复路径一起设计：正常路径要少锁、顺序写、批量处理；恢复路径要知道哪个输入产生哪个输出，重复执行是否幂等，哪些输出已经提交。共享 push_back 的最大问题不是慢，而是它没有留下这些边界。

16.8 从账本到伪代码：完整执行路径

概念只有落到一条可检查的执行路径上才可靠。Reduce、scan 和 partition 的难点不在公式，而在程序运行时每个数组是谁写的、每个位置为什么唯一、失败时哪些字节可以相信、哪些状态只是中间产物。下面用同一批输入把 stable filter 和 multiway partition 走到底。

Stable filter 的小输入账本 设输入有 12 条记录，按原始下标排列，切成 3 个 chunk。谓词是“记录是否需要保留”。串行 reference 会从下标 0 扫到 11，把通过记录按原顺序追加到输出。并行版本不能用共享追加来偷懒，它先生成下面的账本：

Listing 16.13 stable filter 的 count 和 offset 账本

```
chunk    input range    selected input indexes    count    output begin
0        [0, 4)         1, 3                      2        0
1        [4, 8)         4, 7                      2        2
2        [8, 12)        8, 11                     2        4

counts   = [2, 2, 2]
offsets  = [0, 2, 4]      // exclusive scan of counts
total    = 6
output   = [input[1], input[3], input[4], input[7], input[8], input[11]]
```


这个账本有两个关键点。第一，`offsets[c]` 不是线程抢来的下标，而是由所有 chunk 的 `count` 推导出来的地址合同。第二，稳定顺序由两个事实共同保证：chunk 按输入范围排序，chunk 内按原始下标递增写出。若 chunk 1 比 chunk 0 先运行完，也只能写 `output[2]` 和 `output[3]`，不能把记录提前到 chunk 0 的区间。

数据结构先固定身份 实际实现不要先写 worker lambda，而要先把 stage 之间传递的状态固定下来。输入快照要有版本，谓词要有版本，chunk 要有半开区间，输出范围要有期望数量。这里的版本不是分布式系统专属概念；它只是一个可以比较的编号，用来说明 count 阶段和 write 阶段看见的是同一套输入和同一个谓词。

Listing 16.14 stable filter 的状态对象草图

```

1 struct ChunkRange {
2     std::uint32_t chunk_id = 0;
3     std::uint64_t begin = 0;
4     std::uint64_t end = 0;
5 };
6
7 struct FilterPlan {
8     std::uint64_t input_epoch = 0;
9     std::uint64_t predicate_version = 0;
10    std::uint64_t output_capacity = 0;
11    std::vector<ChunkRange> chunks;
12 };
13
14 struct ChunkCount {
15     std::uint32_t chunk_id = 0;
16     std::uint64_t selected = 0;
17 };
18
19 struct ChunkOutputRange {
20     std::uint32_t chunk_id = 0;
21     std::uint64_t begin = 0;
22     std::uint64_t end = 0;
23 };

```

`input_epoch` 表示输入快照身份。若输入数组会被增量加载、压缩、重排或替换，count 阶段记录的数量只对同一个 epoch 有效。`predicate_version` 表示谓词身份。若过滤条件从“错误级别大于等于 3”改成“大于等于 2”，旧 counts 不能复用。并行算法一旦跨阶段，就必须把这些身份写入 plan 或 manifest，否则错误会伪装成偶发越界、数量不一致或 checksum 不匹配。

Count 阶段：只读输入，只写局部结果 Count worker 的职责很窄：扫描自己的半开区间，计算通过数量，把结果写到自己的 `ChunkCount` 槽位。它不分配最终输出，不写 manifest，不更新全局 vector 大小。

Listing 16.15 stable filter count 阶段伪代码

```

1 ChunkCount count_filter_chunk(const FilterPlan& plan,
2                               std::span<const Record> input,
3                               const ChunkRange range,
4                               const Predicate& predicate) {
5     std::uint64_t selected = 0;
6     for (std::uint64_t i = range.begin; i != range.end; ++i) {

```



```

7     if (predicate.keep(input[i])) {
8         ++selected;
9     }
10 }
11 return ChunkCount{range.chunk_id, selected};
12 }

```

从 CPU 的角度看，这个阶段是顺序读输入、少量写本地计数。若 `ChunkCount` 数组按 chunk 排列，并且每个 worker 只写自己的元素，共享写压力很低。谓词若只访问记录中的连续字段，硬件预取器能够提前拉入后续 cache line。若谓词访问指针链、哈希表或压缩字典，count 阶段会变成随机读，后续 scan 再精巧也救不了这部分代价。账本要记录谓词成本，而不是只记录线程数。

Scan 阶段：把数量变成地址合同 Scan 阶段读取 `counts`，生成每个 chunk 的输出区间。这个阶段可以串行，也可以并行；在 chunk 数远小于输入记录数时，串行 scan 往往足够。关键不是炫耀并行 scan，而是保证加法不溢出、输出容量足够、offset 单调。

Listing 16.16 生成 chunk 输出区间的伪代码

```

1  enum class FilterPlanError {
2      None,
3      CountOverflow,
4      OutputTooSmall,
5      MismatchedChunkId,
6  };
7
8  struct FilterLayout {
9      std::vector<ChunkOutputRange> ranges;
10     std::uint64_t total_selected = 0;
11 };
12
13 Expected<FilterLayout, FilterPlanError>
14 build_filter_layout(const FilterPlan& plan,
15                    std::span<const ChunkCount> counts) {
16     FilterLayout layout;
17     layout.ranges.resize(counts.size());
18
19     std::uint64_t cursor = 0;
20     for (std::size_t c = 0; c != counts.size(); ++c) {
21         if (counts[c].chunk_id != plan.chunks[c].chunk_id) {
22             return FilterPlanError::MismatchedChunkId;
23         }
24         if (counts[c].selected > UINT64_MAX - cursor) {
25             return FilterPlanError::CountOverflow;
26         }
27
28         const std::uint64_t begin = cursor;
29         const std::uint64_t end = cursor + counts[c].selected;
30         if (end > plan.output_capacity) {
31             return FilterPlanError::OutputTooSmall;
32         }
33     }

```

```

34     layout.ranges[c] = ChunkOutputRange{counts[c].chunk_id, begin, end};
35     cursor = end;
36 }
37
38 layout.total_selected = cursor;
39 return layout;
40 }

```

这个伪代码比一句“做 exclusive scan”更长，因为工程错误也更具体。计数溢出会让后续范围回绕；输出容量不足会让 write 阶段越界；chunk id 错位会把某个 chunk 的 count 套到另一个 chunk 上。Scan 阶段一旦产生 layout，后续 worker 就不应该再问“我可以写哪里”，只能按 layout 执行。

Write 阶段：独占区间内的局部 cursor Write worker 重新扫描输入区间。每命中一个元素，就写到自己范围里的下一个位置。局部 cursor 从 `range.begin` 开始，结束时必须等于 `range.end`。这个检查非常重要，它把谓词版本不一致、输入变化、count bug 和 write bug 都变成可观测错误。

Listing 16.17 stable filter write 阶段伪代码

```

1  enum class WriteChunkError {
2      None,
3      PredicateVersionChanged,
4      InputEpochChanged,
5      ProducedTooMany,
6      ProducedTooFew,
7  };
8
9  WriteChunkError write_filter_chunk(const FilterPlan& plan,
10                                     const RuntimeSnapshot& snapshot,
11                                     std::span<const Record> input,
12                                     std::span<Record> output,
13                                     const ChunkRange input_range,
14                                     const ChunkOutputRange output_range,
15                                     const Predicate& predicate) {
16      if (snapshot.input_epoch != plan.input_epoch) {
17          return WriteChunkError::InputEpochChanged;
18      }
19      if (predicate.version() != plan.predicate_version) {
20          return WriteChunkError::PredicateVersionChanged;
21      }
22
23      std::uint64_t cursor = output_range.begin;
24      for (std::uint64_t i = input_range.begin; i != input_range.end; ++i) {
25          if (!predicate.keep(input[i])) {
26              continue;
27          }
28          if (cursor == output_range.end) {
29              return WriteChunkError::ProducedTooMany;
30          }
31          output[cursor] = input[i];
32          ++cursor;
33      }

```

```
34
35 if (cursor != output_range.end) {
36     return WriteChunkError::ProducedTooFew;
37 }
38 return WriteChunkError::None;
39 }
```

这里没有锁，也没有全局原子下标。每个 worker 的写入范围由 layout 保证不重叠。若两个相邻范围落在同一条 cache line，边界处仍可能有 false sharing，但它只发生在边界线附近；共享 push_back 则会让所有命中元素都争同一个 size 或尾指针。三阶段设计把高频共享写变成了低频边界成本。

稳定性证明 稳定 filter 的正确性可以直接从四个不变量推出。第一，chunk ranges 覆盖输入且不重叠，所以每个输入下标只由一个 worker 检查。第二，counts[c] 等于 chunk c 内满足谓词的元素个数。第三，exclusive scan 生成的输出区间长度正好等于 counts[c]，并且区间按 chunk 顺序连续排列。第四，write 阶段在 chunk 内按输入下标递增写入。

因此任意两个被保留元素 x 和 y，若 x 的输入下标小于 y，分两种情况。若二者在同一 chunk，write 循环先遇到 x，输出位置小于 y。若二者在不同 chunk，x 所在 chunk 编号更小，exclusive scan 给它的整个输出区间都排在 y 所在 chunk 区间之前。两种情况都保持原始相对顺序。

硬件账本：为什么多扫一遍可能更快 stable filter 多扫输入至少两遍，看起来违反直觉。真正比较的不是“几遍循环”，而是每遍循环在硬件上制造什么事件。共享 push_back 的每个命中元素都可能触发锁竞争、原子尾指针更新、vector 容量检查、allocator 元数据访问和非连续写入。Count + scan + write 则把路径拆成可预取的顺序读、很小数组上的 scan、独占区间内的顺序写。

阶段	主要内存事件	需要关注的风险
count	顺序读输入，写本 chunk 计数	谓词随机访问、分支误判、计数槽 false sharing
scan	读写小型 counts 和 ranges 数组	溢出、容量不足、chunk id 错位
write	顺序读输入，顺序写独占输出区间	写分配、边界 false sharing、TLB miss、谓词版本变化
commit	写 manifest 和 checksum	提交前崩溃、重复 attempt、部分输出可见性

写入一个尚未在 cache 中的输出 cache line 时，常见 CPU 会先通过 read-for-ownership 或等价机制取得该 cache line 的独占所有权，这叫写分配。若输出地址连续，硬件可以批量处理这些 cache line；若所有 worker 通过共享追加写到调度顺序决定的位置，地址流就难以预测。输出跨很多页面时，TLB 还要保存虚拟页到物理页的翻译；连续写可以让页访问更规则，随机写会增加 TLB 压力。算法报告若只写 $O(n)$ ，就漏掉了这些决定真实吞吐的事件。

Multiway partition 的小输入账本 现在把 stable filter 推广到多路 partition。输入仍然切成 3 个 chunk，目标 bucket 有 3 个。每个 chunk 先统计本地 histogram：

Listing 16.18 multiway partition 的 counts 矩阵

	bucket 0	bucket 1	bucket 2
chunk 0 [0, 4)	1	1	2

chunk 1 [4, 8)	0	3	1
chunk 2 [8, 12)	2	0	2
bucket totals	3	4	5
bucket bases	0	3	7

`bucketbases` 决定每个 bucket 在输出数组中的全局区域：bucket 0 是 [0, 3)，bucket 1 是 [3, 7)，bucket 2 是 [7, 12)。但是每个 chunk 在 bucket 内还要有自己的局部偏移，需要沿 chunk 维度做 exclusive scan：

Listing 16.19 每个 bucket 沿 chunk 维度的 exclusive scan

	bucket 0	bucket 1	bucket 2
chunk 0 offset	0	0	0
chunk 1 offset	1	1	2
chunk 2 offset	1	4	3

最终写入范围由公式给出：

```
begin = bucket_base[bucket] + chunk_offset[chunk][bucket]
end   = begin + counts[chunk][bucket]
```

以 chunk 1 的 bucket 2 为例，`bucket_base[2]=7`，`chunk_offset[1][2]=2`，`counts[1][2]=1`，所以它写 [9, 10)。这个位置不是任何线程抢来的，而是二维 scan 推导出的地址合同。

Partition 伪代码：histogram、layout、scatter 多路 partition 的实现可以拆成三个函数。第一阶段只统计 bucket 数量；第二阶段生成每个 `chunkxbucket` 的输出范围；第三阶段按范围写出。下面的伪代码故意保留二维索引，让数据流可检查。

Listing 16.20 multiway partition 的三阶段伪代码

```
1 void count_buckets(const PartitionPlan& plan,
2                   std::span<const Record> input,
3                   const ChunkRange range,
4                   Matrix<std::uint64_t>& counts) {
5     for (std::uint64_t i = range.begin; i != range.end; ++i) {
6         const std::uint32_t bucket = plan.bucket_of(input[i]);
7         ++counts(range.chunk_id, bucket);
8     }
9 }
10
11 PartitionLayout build_partition_layout(const PartitionPlan& plan,
12                                       const Matrix<std::uint64_t>& counts) {
13     PartitionLayout layout;
14     std::uint64_t global_base = 0;
15
16     for (std::uint32_t bucket = 0; bucket != plan.bucket_count; ++bucket) {
17         std::uint64_t bucket_total = 0;
18         for (std::uint32_t chunk = 0; chunk != plan.chunk_count; ++chunk) {
19             bucket_total += counts(chunk, bucket);
20         }
```

```

21
22     std::uint64_t cursor_in_bucket = 0;
23     for (std::uint32_t chunk = 0; chunk != plan.chunk_count; ++chunk) {
24         const std::uint64_t begin = global_base + cursor_in_bucket;
25         const std::uint64_t end = begin + counts(chunk, bucket);
26         layout.range(chunk, bucket) = OutputRange{begin, end};
27         cursor_in_bucket += counts(chunk, bucket);
28     }
29
30     layout.bucket_range(bucket) =
31         OutputRange{global_base, global_base + bucket_total};
32     global_base += bucket_total;
33 }
34 return layout;
35 }
36
37 void scatter_partition_chunk(const PartitionPlan& plan,
38                             std::span<const Record> input,
39                             std::span<Record> output,
40                             const ChunkRange range,
41                             const PartitionLayout& layout) {
42     std::vector<std::uint64_t> cursor(plan.bucket_count);
43     for (std::uint32_t b = 0; b != plan.bucket_count; ++b) {
44         cursor[b] = layout.range(range.chunk_id, b).begin;
45     }
46
47     for (std::uint64_t i = range.begin; i != range.end; ++i) {
48         const std::uint32_t bucket = plan.bucket_of(input[i]);
49         const std::uint64_t pos = cursor[bucket]++;
50         output[pos] = input[i];
51     }
52 }

```

这段伪代码还缺少错误处理、checksum、attempt 和容量检查，但骨架已经能说明关键性质：count 阶段没有最终输出写，layout 阶段没有输入记录访问，scatter 阶段每个 bucket 使用自己的局部 cursor。若需要稳定 partition，scatter_partition_chunk 必须按输入下标递增处理；若不要求稳定，可以用更激进的局部 buffer 或交换策略，但接口必须标明。

二维布局的 cache 和 TLB 代价 counts(chunk, bucket) 是一个小矩阵，但小矩阵也会变成瓶颈。若 bucket 数少，比如 8 或 16，worker 在 count 阶段更新本 chunk 的一整行，行式布局很舒服。若 bucket 数是 4096，每个 chunk 的 counts 行就可能跨很多 cache line，甚至跨页面；worker 的随机 bucket 更新会让 TLB 和 cache 都承压。若改成列式布局，per-bucket scan 很连续，但 count 阶段写入可能变成更大的跨步访问。

常见折中是 bucket blocking：一次只处理一组 bucket，或者先在 worker 本地小数组里计数，再批量写回全局 counts。另一个折中是采样：先扫描小样本估计热点 bucket，热点 bucket 用特殊路径，普通 bucket 用常规矩阵。参数选择不能靠口号，报告要写出 chunk_count*bucket_count*sizeof(counter) 的真实字节数、counts 矩阵跨多少页面、每轮处理多少 bucket、以及最大 bucket 占比。

Manifest commit: 字节写出不等于结果可见 write 或 scatter 完成后，输出数组里已经有字节，但这些字节还不是下游可读取的事实。事实由 manifest commit

建立。manifest 至少要记录 partition version、bucket id、chunk id、attempt、range、records、bytes 和 checksum。commit 可以是内存中的原子状态转换，也可以是文件系统里的临时文件 rename；无论哪种形式，原则都是先写数据和校验，再发布 manifest。

Listing 16.21 partition commit 条目的扩展示例

```

1 enum class CommitState : std::int32_t {
2     Pending,
3     Committed,
4     Failed,
5     Superseded,
6 };
7
8 struct PartitionCommitEntry {
9     std::uint32_t partition_version = 0;
10    std::uint32_t bucket_id = 0;
11    std::uint32_t chunk_id = 0;
12    std::uint32_t attempt = 0;
13    std::uint64_t begin = 0;
14    std::uint64_t end = 0;
15    std::uint64_t records = 0;
16    std::uint64_t checksum = 0;
17    CommitState state = CommitState::Pending;
18 };

```

如果 chunk 2 的 scatter 在写到一半时失败，它的 Pending 条目不能被下游读取。重试可以生成 attempt 1；当 attempt 1 commit 成功后，attempt 0 应标记为 Failed 或 Superseded。这样恢复程序不需要猜内存里哪些位置有效，只看 manifest。这个边界会在 I/O、shuffle 和分布式重试中继续放大。

16.9 阶段账本：让 reduce、scan、partition 可恢复

并行算法进入系统后，目标不再只是“算完得到一个数组”。它还必须回答：哪个阶段开始了，哪个阶段完成了，失败发生在 count 之前还是 write 之后，哪些输出字节可以相信，哪些只是旧 attempt 的残留。Reduce、scan 和 partition 都可以写成阶段状态机。状态机不是额外文档，而是恢复程序、测试和运行时调度共同依赖的事实表。

Listing 16.22 stable filter 或 partition 的阶段状态机

```

StageCreated
CountStarted
CountDone
ScanStarted
ScanDone
WriteStarted
WriteDone
Verified
ManifestCandidate
Committed
Failed
Superseded

```

CountDone 表示每个 chunk 的数量或 partial 已经写入局部账本，但还没有生成全局输出位置。**ScanDone** 表示 offsets 或 layout 已经产生，后续写入范围可以由账本推导。**WriteDone** 表示 worker 声称写完自己的独占范围，但还没有经过校验。**Verified** 表示记录数、checksum、范围边界和版本都通过检查。**ManifestCandidate** 表示提交条目已经准备好但尚未成为下游可见事实。**Committed** 才表示后续 stage 可以依赖这些结果。把这些状态压成一个布尔 **done**，会让恢复路径失去信息。

每个 block 必须有身份 阶段状态机还不够，必须把每个 block 的输入、输出和 attempt 写成记录。Block 是一个可恢复单位：可能是输入 chunk、partition 的 **chunkxbucket** 单元、reduce 树上的一个 partial，或 scan 的一个 tile。身份字段要能回答“这条记录属于哪次输入、哪次配置、哪次重试”。

Listing 16.23 可恢复并行 stage 的 block 记录

```

1 enum class BlockState : std::int32_t {
2     Empty,
3     CountDone,
4     LayoutAssigned,
5     WriteStarted,
6     WriteDone,
7     Verified,
8     Committed,
9     Failed,
10    Superseded,
11 };
12
13 struct StageBlockRecord {
14     std::uint64_t stage_id = 0;
15     std::uint64_t input_epoch = 0;
16     std::uint64_t predicate_version = 0;
17     std::uint32_t chunk_id = 0;
18     std::uint32_t bucket_id = 0;
19     std::uint32_t attempt = 0;
20     std::uint32_t worker_id = 0;
21     std::uint64_t input_begin = 0;
22     std::uint64_t input_end = 0;
23     std::uint64_t output_begin = 0;
24     std::uint64_t output_end = 0;
25     std::uint64_t records_expected = 0;
26     std::uint64_t records_written = 0;
27     std::uint64_t local_counts_checksum = 0;
28     std::uint64_t output_checksum = 0;
29     BlockState state = BlockState::Empty;
30 };

```

input_begin/input_end 说明这条记录覆盖哪个输入半开区间。**output_begin/output_end** 说明该 block 独占哪个输出半开区间。**records_expected** 来自 count 和 scan，**records_written** 来自 write。两个数字不一致时，write 阶段不能进入 **Verified**。**local_counts_checksum** 用来确认 count 阶段的本地直方图没有被错配；**output_checksum** 用来在崩溃恢复后确认输出字节仍然对应这条记录。字段看似多，但每个字段都对应一个事故。

失败点一：count 完成，scan 未完成 若程序在 **CountDone** 之后崩溃，恢复程

序可以读取每个 chunk 的 count 或 histogram，重新执行 scan。此时不应该信任任何输出文件或输出数组，因为地址合同还没有形成。恢复动作是检查 count 记录数量是否覆盖所有 chunk、`input_epoch` 和 `predicate_version` 是否一致、count checksum 是否通过，然后重建 layout。若缺少某个 chunk 的 count，只重跑缺失 chunk，不需要重跑整个输入。

Listing 16.24 CountDone 后恢复 scan 的判定

```
if all block records for stage have CountDone:
    verify same input_epoch and predicate_version
    verify chunk coverage and no duplicate chunk_id
    rebuild offsets or partition layout
    write LayoutAssigned records
else:
    rerun missing or failed count blocks
```

这里不能用文件修改时间或 worker 日志推断完成状态。完成事实必须来自 block 记录。日志可以帮助诊断，但日志行不是提交协议：它可能先于数据刷出，也可能在崩溃前丢失。

失败点二：scan 完成，write 只写了一半 ScanDone 后，输出范围已经确定。若某个 worker 在 WriteStarted 后崩溃，该 worker 负责的输出范围可能被写了一部分。恢复程序不能把这段输出直接交给下游，也不能简单继续从中间写，因为部分写的位置和校验可能无法可靠确定。最稳妥的策略是把旧 attempt 标记为 Failed，为同一输入范围生成新 attempt，从该 block 的 `output_begin` 重新写完整范围。由于输出区间由 layout 固定，新 attempt 可以覆盖旧 attempt 的字节；只有通过 Verified 后才能进入 manifest。

Listing 16.25 WriteStarted 崩溃后的重试策略

```
old attempt:
    state = WriteStarted
    output range may contain partial bytes
    never publish to downstream readers

retry attempt:
    same stage_id, input_epoch, predicate_version, chunk_id, bucket_id
    attempt = old attempt + 1
    rewrite the entire output range
    verify records_written, checksum, and boundaries
    publish only the verified retry attempt
```

这个策略要求 write 阶段是幂等的：同一输入、同一谓词、同一 layout 重写同一输出范围，得到同一字节序列和 checksum。若 write 阶段有外部副作用，例如向共享日志追加、发送网络请求或更新全局指标，就必须把副作用移到 commit 之后，或者为副作用本身设计去重键。

失败点三：重复写和重复提交 重试会带来另一个问题：两个 attempt 可能并发存在。也许旧 worker 没死，只是暂停很久；driver 认为它超时，于是启动 attempt 1。后来 attempt 0 恢复并试图提交。没有 attempt 字段，系统无法判断谁是合法结果。正确做法是用 manifest 的 CAS 或事务化 rename 选择唯一赢家。

Listing 16.26 manifest 选择唯一赢家

```
candidate key:
```

```
stage_id, chunk_id, bucket_id

candidate value:
  attempt, output range, records, checksum

commit rule:
  if no committed candidate exists for the key:
    publish this verified attempt
  else:
    mark this attempt as Superseded
```

唯一赢家不一定是 attempt 编号最大的那个。若 attempt 0 已经完成验证并提交，attempt 1 即使后来也写出了相同结果，也只能标记为 **Superseded**。这样下游读取 manifest 时不会看到两个都声称覆盖同一区间的 block。若策略要求“较新 attempt 覆盖较旧 attempt”，也必须让覆盖本身成为 manifest 中的明确状态转换，不能让两个版本同时可见。

失败点四：manifest 缺失但文件存在 分布式和异步 I/O 中常见一种尴尬状态：输出文件或内存段已经存在，但 manifest 里没有对应条目。可能是 write 完成后、commit 前崩溃；也可能是旧 attempt 的残留。恢复程序应默认“不在 manifest 中的输出不可见”。它可以扫描孤儿文件，把能通过 checksum 且匹配某个 **Verified** 记录的文件转成 **ManifestCandidate**；如果没有可靠记录，就删除或隔离。不能因为文件名看起来对，就把它加入结果集。

观察到的状态	不能做什么	正确动作
有 count 记录，无 layout	不能推断输出位置	重建 scan 或重跑缺失 count
有 layout，verified write	无	重写 block 并校验
有 verified attempt，manifest	不能自动视为提交	通过 commit 规则发布或隔离
有文件，无 block 记录	不能按文件名恢复事实	删除、隔离或人工审计
两个 attempt 都 verified	不能让二者都可见	manifest 选择唯一赢家

把阶段账本交给运行时 这些状态不是只给离线恢复看。第 14 章的任务运行时也要读取它们：**CountDone** 让 scan task 的 ready count 递减；**ScanDone** 让 write tasks 入队；**Verified** 让 commit task 入队；**Committed** 让下游 reduce 或 I/O stage 入队。若 worker 在任务内部阻塞等待“所有 count 都完成”，运行时就失去了调度权；正确做法是把阶段状态变成任务图边，让完成事件发布后继，而不是让线程占着 CPU 等待。

常见故障路径 下面这些错误都不是边角料，而是并行算法进入工程系统后最常见的失效点。

故障	触发方式	检测或恢复方式
谓词版本变化	count 和 write 使用不同配置	plan 记录 <code>predicate_version</code> , write 前比较
输入快照变化	输入被重排、压缩或替换	plan 记录 <code>input_epoch</code> , stage 入口比较
计数溢出	极大输入或错误计数类型	scan 阶段使用溢出检查, 拒绝 layout
输出容量不足	预分配容量小于 total	scan 阶段返回 <code>OutputTooSmall</code>
写出数量不一致	write 命中数与 count 不同	chunk cursor 结束时必须等于 range end
部分写后失败	worker 崩溃或 I/O 错误	commit 前不发布 manifest, 重试新 attempt
分区规则变化	hash seed 或 bucket 数不同	manifest 记录 partition version, driver 拒绝混合

这些检查看似增加代码, 但它们把“不知道哪里错了”的并发事故变成可定位的 stage 错误。可靠并行算法不是只在成功路径上快, 而是在失败路径上也能说明哪些状态可信、哪些状态必须丢弃。

16.10 实现路径

Compute Systems Engine 的并行算法部分不应表现为一组离散实验, 而应形成一条从串行循环到可恢复阶段图的实现路径。核心不是“把任务丢给多个线程”, 而是把串行循环里的隐含状态拆成 partial、offset、bucket range 和 commit state, 让每个阶段都能说明自己读了什么、写了什么、是否保持顺序、失败后哪些事实可信。

模块	最小能力	关键判读
parallel count-if stable filter	串行 reference、global atomic 反例、partial 版本 count、exclusive scan、 write 三阶段	正确性、线程扩展、共享写 解释 输出顺序、chunk counts、 offsets、失败恢复
multiway partition deterministic reduce	二维 counts、per-bucket scan、manifest 固定合并树、误差合同	每桶大小、最大倾斜、 checksum、稳定性 浮点误差、重复运行一致性、 性能代价

风险输入 输入不能只有随机均匀数组。必须覆盖空输入、一个元素、刚好一个 chunk、最后 chunk 不满、全通过、全不通过、低选择率、高选择率、单热点 bucket、Zipf 倾斜、浮点极大极小混合和失败 chunk。风险输入要攻击合同边界, 而不是只扩大规模。

Cutoff 并行算法不是输入越小越应该并行。任务提交、队列调度、scan、partial、合并都有固定成本。项目应测量串行和并行交叉点, 形成 cutoff。小输入走串行, 大输入走并行; 中等输入可能单线程 SIMD 更合适。Cutoff 不是永恒常数, 而是随机器、编译器和输入分布变化的配置。

判读顺序 并行算法的判读顺序应从语义开始。第一, 写清串行语义: 顺序、稳定性、误差、异常和输出身份。第二, 写清并行合同: 切分方式、局部状态所有权、合并顺序和共享写边界。第三, 列出风险输入: 选择率、热点、chunk 边界、浮点极

值和失败点。第四，进入阶段账本：input records、output records、chunk count、temporary bytes、partial bytes、max bucket、checksum 和 elapsed time。性能数字只能在这四层之后解释。

并行形状	风险输入	必须解释的现象
parallel count-if	空输入、全命中、全不命中、随机命中、小输入、大输入	全局 atomic 为什么慢，partial 何时值得，cutoff 在哪里
浮点 reduce	极大极小混合、正负抵消、NaN/Inf、重复运行	误差合同、固定树形代价、fast 模式边界
stable filter	0%、1%、50%、100% 选择率，最后 chunk 不满	scan 如何分配位置，稳定顺序成本，失败恢复边界
multiway partition	均匀 key、单热点、Zipf 倾斜、bucket 数变化	counts 矩阵大小、最大 bucket、manifest 是否足够恢复
radix partition	4 bit、8 bit、不同 key 宽度和输入规模	桶数、轮数、cache 压力和吞吐之间的取舍

阶段账本 每个并行阶段都要输出阶段账本，而不是只输出总耗时。建议字段如下：

Listing 16.27 并行算法阶段账本字段

```
1 stage,input_records,output_records,chunks,buckets,temporary_bytes,
2 partial_bytes,max_bucket,checksum,elapsed_ns,mode
```

这些字段帮助解释现象。若并行 filter 在 1% 选择率下不快，可能是 count 和 scan 固定成本主导；若 partition 在单热点输入下慢，max_bucket 会暴露下游长尾；若 radix partition 的 8 bit 版本变慢，buckets 和 temporary_bytes 能说明 cache 压力。没有阶段账本，性能解释很容易变成猜测。

最小对照保留在 stable filter 上即可：串行 reference 说明语义；加锁 push_back 暴露共享追加和顺序问题；每线程本地 buffer 暴露合并和材料化成本；count + scan + write 说明输出位置如何由共享状态变成确定区间。再加入一个 write 阶段失败点，就能把性能、顺序和恢复边界串在同一条流程里。

16.11 并行算法实验门禁

一、Reduce 是否写清合并语义 检查 identity、结合性、交换性、确定性、浮点误差和 partial 所有权。若这些没有写清，就不能称为可靠并行 reduce。

二、Scan 是否解释输出位置 检查是否能从共享 push_back 推导出 count、exclusive scan、write 三阶段。若只会背 prefix sum 公式，说明还没学到系统意义。

三、Partition 是否留下 manifest 检查每个 bucket 的范围、记录数、字节数和 checksum 是否可见。没有 manifest，后续 shuffle 和恢复会断链。

四、是否区分稳定和非稳定输出 稳定性影响测试、审计、缓存和幂等。接口和报告必须说明是否保持顺序，以及为此付出了什么成本。

五、是否记录资源预算 Partial、counts、offsets、本地 buffer、manifest 都占内存。报告要写实际字节，不只写复杂度。

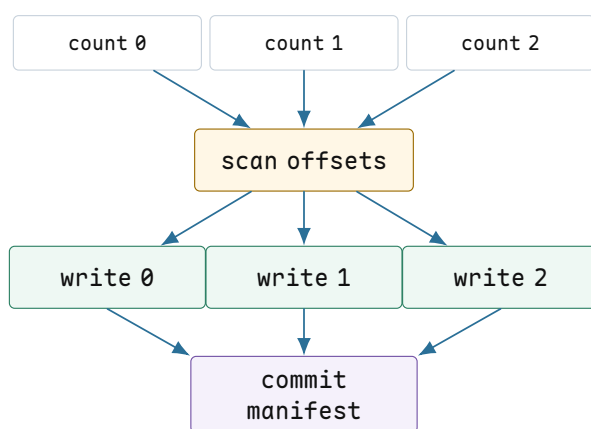
六、是否能变成运行时依赖 Reduce、scan 和 partition 都形成任务依赖。局部任务完成后才能 scan，scan 完成后才能 write，所有 partition block 提交后才能 reduce。任务运行时要把这些依赖显式调度，而不是让 worker 阻塞等待。

16.12 任务运行时接口

Reduce、scan 和 partition 一旦写成 stage, 就自然产生任务图。Count tasks 可以并行; scan offsets 必须等所有 count 完成; write tasks 必须等 offsets 完成; reduce tasks 必须等对应 partition block 提交。若用普通线程池, 最容易写成 worker 内部阻塞等待下一阶段, 造成线程池耗尽或低效 barrier。运行时要解决的, 正是如何把这些依赖显式化, 让 worker 只执行 ready task, 而不是占着线程等待。

从算法阶段到任务图节点 一个 stable filter 可以拆成三类节点: count node、scan node、write node。Count nodes 只依赖输入 chunk; scan node 依赖所有 count nodes; write nodes 依赖 scan node 和对应 input chunk。Partition 多一个 bucket 维度; reduce 多一个合并树维度。这样算法结构直接变成运行时调度结构。

Stable filter 的任务依赖



图示: 本图要证明的命题是: 并行算法的阶段依赖应该交给任务运行时调度, 而不是让 worker 线程阻塞等待。

运行时要保存哪些指标 为了分析任务运行时, 实验应该输出每个 task 的 submit、ready、start、finish 时间, 以及输入记录数、输出记录数和错误数。这样才能解释 worker 空闲、队列长尾、barrier 等待和任务粒度。若并行算法只输出总耗时, 运行时就没有诊断材料。

从 barrier 到依赖调度 最简单实现是在每个阶段之间放 barrier: 所有 count 完成后主线程做 scan, 再启动所有 write。这个版本容易理解, 适合入门起点。更进一步, 运行时可以把依赖显式化: 当最后一个 count 完成时, scan node 变 ready; scan 完成后, 所有 write nodes 变 ready; write 完成后, commit node 变 ready。这种结构减少不必要等待, 也为 work stealing、故障重试和背压打基础。

16.13 习题

习题与作业

习题一: 为 parallel count-if 写完整正确性证明。证明每个输入元素被统计一次且只统计一次, partial 合并结果等于串行 reference。

习题二: 设计 inclusive scan 和 exclusive scan 的 API。给出输入 [3,1,4] 的两个输出, 并说明哪个更适合分配输出偏移。

习题三: 为 stable filter 写三阶段伪代码。要求说明每阶段读写哪些

数组、是否并行、是否有共享写、需要哪些临时空间。

习题四：设计一个倾斜 key 的 partition 测试。要求解释为什么写入区间没有冲突，但下游 reduce 仍可能负载不均。

习题五：比较线性合并和树形合并。给出 partial 数为 8 时的合并步骤，并说明什么时候树形合并不值得。

习题六：阅读一个工业案例，例如 per-cpu counter、vectorized aggregate 或 Spark shuffle manifest。写出它的真实约束、核心原理、取舍，以及它和实验的对应关系。

工程案例

知识点小案例：stable filter 为什么要 count、scan、write 三步。特例是输入 `[a,b,c,d]`，保留 `b,d`。串行 `push_back` 很自然；并行时若多个线程同时 push，就会共享写输出尾指针，还可能打乱顺序。三阶段做法先让每个 block 统计保留数量，再 exclusive scan 得到每个 block 的输出起点，最后各 block 写入自己独占区间。由此推出规律：scan 的作用不是数学炫技，而是把不确定的共享追加变成确定的写区间。工程上任何并行压缩、分区和稳定输出都要先定义输出位置，再谈并行写。

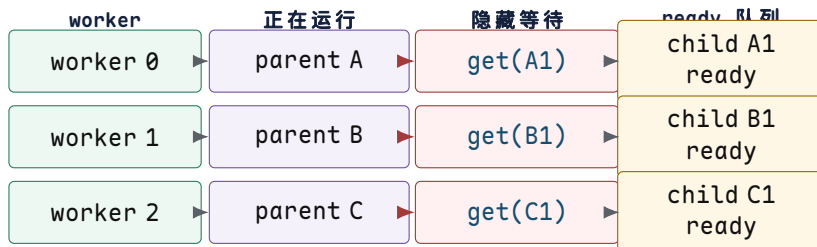
任务运行时与工作窃取

先看一个很小、但足够刺痛工程师的事故：线程池有四个 worker，四个父任务都已经开始执行。每个父任务提交一个子任务后立刻调用 `future.get()`。子任务已经排进同一个线程池的队列，队列里有可运行工作；可是四个 worker 都被父任务占住，正在等待这些还没有机会运行的子任务。程序没有数据竞争，锁也没有形成循环等待，CPU 甚至可能还在忙等，但作业不再前进。

操作系统调度器看不见“父任务”和“子任务”。它只调度内核线程。线程池里的 task 只是用户态内存中的一段函数对象和状态记录，除非某个 worker 线程取出它并开始运行，否则它不会自动获得 CPU 时间。父任务在 worker 内调用 `future.get()` 时，可能进入条件变量或 futex 等待，也可能忙等；无论是哪一种，结果都是一个原本可以执行 ready task 的 worker 执行槽被等待占住。队列里有 ready task，不等于系统有可用执行资源。

这个事故说明：线程池只知道“有函数就找线程执行”，并不自动理解任务之间的依赖。真正的任务运行时要把依赖、就绪、等待、取消、异常和关闭写成显式状态，让 worker 只执行已经 ready 的工作，而不是把宝贵的执行资源拿去等待同一个资源池里的未来工作。工作窃取也不是“更高级的线程池”这个标签，而是在任务已经 ready 之后，解决本地性、负载均衡和长尾的调度策略。

Future 饥饿：ready 任务无人执行



队列里有 ready 子任务，但所有 worker 都被父任务的同步等待占住。修复点不是盲目加线程，而是让等待关系进入运行时状态。

图示：本图要证明的命题是：future 饥饿不是锁环，而是执行资源被隐藏等待耗尽。

17.1 从线程池事故到运行时语义

线程池的基本工作很简单：创建一组 worker 线程，外部提交函数对象，worker 从队列中取出并执行。这个模型适合互相独立的任务，例如并行压缩多个文件、处理多个互不依赖的请求、把输入数组切块后分别计算。它的危险在于接口太容易让人误以为“能提交函数”就等于“能表达并行计算”。

任务运行时比线程池多回答几类问题：某个任务依赖哪些前驱，什么时候 ready，结果由谁拥有，worker 能否阻塞等待，失败怎样传播，取消怎样收束，关闭时已入队和未入队任务分别如何处理。只要这些问题没有写清，后续加入 work stealing、priority、future、I/O completion 或分布式 attempt 都会变成难以复现的隐式控制流。

Thread、worker、task 和 runtime Thread 是操作系统调度的执行实体；worker 是运行时长长期持有、用于消费任务的线程；task 是可执行工作单位；runtime 是管

理 task 状态、ready 队列、worker、结果、取消和关闭协议的系统。把这些词混用，会导致错误设计。

一个 task 不应等同于一个 thread。创建一百万个 task 可以合理，创建一百万个 OS thread 通常不合理。一个 future 也不应等同于一个运行中的线程，它只是“将来会有结果或错误”的共享状态。运行时的核心价值，就是让大量 task 在有限 worker 上推进，并且让等待不必占住 worker。

这一区分还解释了为什么“多开几个线程”不是根本修复。线程变多会增加内核调度对象、栈内存、上下文切换和 cache/TLB 扰动；若任务依赖仍然藏在 worker 栈上的同步等待里，线程数只是把饥饿推迟到更大规模。运行时要做的是把“谁等谁”从调用栈里拿出来，变成 task node、ready count、continuation 和队列状态。这样 OS 仍然只调度 worker 线程，但用户态运行时能决定哪个 task 应该占用这些线程。

最小线程池也要有状态机 很多入门线程池只有一个 `closed` 布尔值，这会把“拒绝新任务”“排空旧任务”“取消未开始任务”“worker 已经退出”混在一起。更清楚的模型至少有三个状态：

Listing 17.1 运行时生命周期的最小枚举

```
1 enum class RuntimeState : std::int32_t {
2     Open,
3     Draining,
4     Stopped,
5 };
6
7 enum class SubmitResult : std::int32_t {
8     Accepted,
9     RejectedDraining,
10    RejectedStopped,
11    QueueFull,
12 };
```

`Open` 允许提交新任务。`Draining` 拒绝新任务，但继续执行已经接纳的任务。`Stopped` 表示 worker 已退出，所有等待者都必须被唤醒。若支持立即取消，还要把 `shutdownrain` 和 `shutdowncancel` 分成两条路径。状态机不是形式主义，它直接决定条件变量谓词、析构安全、future 等待者能看到什么结果。

队列谓词和唤醒不是装饰 线程池最小循环通常是：worker 拿锁，检查队列是否非空，空则等待条件变量，被唤醒后再检查。这里的“再检查”不是保守写法，而是正确性要求。条件变量允许虚假唤醒；多个 worker 被唤醒时，先拿到锁的 worker 可能已经取走任务；shutdown 也可能在等待期间发生。可靠等待谓词至少要表达“队列非空或运行时状态不再允许继续等待”。若谓词只写成“被通知了”，就可能出现空队列取任务、shutdown 后睡死或遗漏取消。

futex 的思想也类似：用户态先检查内存中的条件，不满足时才让内核把线程挂到等待队列；唤醒方先改变内存状态，再通知等待者。条件变量、futex 和无锁队列的细节不同，但原则一样：状态是事实，通知只是加速观察事实。运行时设计应先定义状态转移，再定义谁负责通知，而不是把 `notify_one()` 当成语义本身。

Future 的本质是共享结果状态 Future 的重要部分不是 `get()` 这个阻塞调用，而是 shared state。任务函数运行在 worker 栈上，提交者可能在另一个线程等待结果；值、异常、取消状态、完成通知和 continuation 必须放在一个拥有明确生命周期的对象里。若异常只打印到 worker 日志，调用者无法知道任务失败；若取消只设置标志但不唤醒等待者，系统会悬挂；若结果保存在已销毁的 lambda 捕获里，就会出现悬垂引用。

Listing 17.2 Future 共享状态需要表达的语义

```

1 enum class CompletionState : std::int32_t {
2     Pending,
3     Succeeded,
4     Failed,
5     Canceled,
6 };
7
8 struct CompletionRecord {
9     std::uint64_t task_id = 0;
10    CompletionState state = CompletionState::Pending;
11    std::uint32_t error_code = 0;
12    std::uint64_t result_bytes = 0;
13 };

```

这段代码不是完整 future 实现，而是在固定语义。一个可靠运行时必须能区分成功、失败和取消；也必须能让等待者在所有终态下醒来。生产实现还要保存异常对象、结果存储、等待队列、引用计数和内存同步关系。

运行时 FutureState：结果、异常、等待者和 continuation 线程池如果提供 `submit->future`，就必须有一份比 `CompletionRecord` 更具体的结果状态。它不一定要复刻标准库实现，但要表达同一组事实：生产端是否还存在，结果是否 ready，结果或异常存在哪里，谁在等待，哪些 continuation 要在 ready 后调度，以及这份状态何时可以释放。

Listing 17.3 运行时 FutureState 的语义字段

```

1 enum class FutureStateKind : std::int32_t {
2     Pending,
3     ValueReady,
4     ExceptionReady,
5     Canceled,
6     BrokenProducer,
7     Consumed,
8 };
9
10 struct FutureStateHeader {
11     std::uint64_t task_id = 0;
12     FutureStateKind state = FutureStateKind::Pending;
13     std::uint32_t waiter_count = 0;
14     std::uint32_t continuation_count = 0;
15     bool producer_alive = true;
16     bool consumer_alive = true;
17 };

```

真实的结果存储可以是类型擦除、模板化 storage、`std::variant` 或标准库 `std::promise` 包装；本章先关心状态边界。任务被接纳后，FutureState 必须活到 producer 进入终态，并且活到 consumer 取走结果或放弃句柄。若任务完成后立即释放任务对象，而 future 还没 `get()`，调用者会读到悬垂状态。若 consumer 放弃 future，任务仍可能继续运行；运行时可以选择继续计算、协作取消，或在背压策略里丢弃后继，但必须写成合同。

FutureState 的完整状态机 最小运行时不能只保存一个 `ready` 布尔值。它至

少要区分“还没有结果”“有值”“有异常”“被取消”“生产者消失”“普通 future 已消费”。这些状态不是为了日志好看，而是为了让 `get`、`wait_idle`、取消传播和 `coroutine` 恢复都能做出确定动作。

Listing 17.4 FutureState 状态转移表

```

Pending:
  complete_value      -> ValueReady
  complete_exception  -> ExceptionReady
  cancel_before_run   -> Canceled
  producer_destroyed  -> BrokenProducer
  consumer_destroyed  -> Pending or producer policy decides cancel

ValueReady:
  ordinary_get        -> Consumed
  shared_get          -> ValueReady
  destroy_last_handle -> reclaim storage

ExceptionReady:
  ordinary_get        -> Consumed after rethrow path is prepared
  shared_get          -> ExceptionReady
  destroy_last_handle -> reclaim storage

Canceled:
  get/await_resume    -> throw task_canceled or return canceled result
  destroy_last_handle -> reclaim storage

BrokenProducer:
  get/await_resume    -> throw broken_task or future_error
  destroy_last_handle -> reclaim storage

Consumed:
  second ordinary_get -> diagnostic or future_error
  destroy_last_handle -> reclaim storage

```

这张表必须和接口一致。如果运行时返回普通 single-consumer future, `ValueReady->Consumed` 是合法消费；如果返回 shared future-like handle, ready 状态不能被第一个 reader 移走。如果 consumer 提前丢弃句柄，运行时也不能默认杀掉任务，除非 API 写明“future destructor requests cancellation”。标准库 `std::future` 没有取消语义；项目运行时可以有，但必须显式。

FutureState 对象边界：热字段、冷字段和等待者 FutureState 同时被 worker、外部 caller、continuation 和 coroutine completion 访问。为了让后续优化有边界，先把字段按用途拆开：

Listing 17.5 FutureState 的语义布局

```

1 struct FutureStateHot {
2     std::atomic<FutureStateKind> state{FutureStateKind::Pending};
3     std::uint64_t task_id = 0;
4     std::uint32_t flags = 0;
5 };
6
7 struct FutureStateCold {

```

```

8  std::uint64_t create_ns = 0;
9  std::uint64_t ready_ns = 0;
10 std::uint64_t first_wait_ns = 0;
11 std::uint32_t waiter_count = 0;
12 std::uint32_t continuation_count = 0;
13 FutureErrorKind error_kind = FutureErrorKind::None;
14 };
15
16 struct FutureStateSync {
17     std::mutex mutex;
18     std::condition_variable cv;
19     ContinuationList continuations;
20 };

```

热字段服务快速检查和 trace 关联；冷字段服务诊断；同步字段服务等待和 continuation 注册。真实实现可以合并或重排，但不能丢掉语义。特别是 continuation list 不能只是一组裸回调：每个 continuation 还要知道恢复到哪个 executor、是否携带取消令牌、是否只能恢复一次。

完成路径的原子提交 FutureState completion 必须是单点提交。不要让 worker loop 写一部分，future 写一部分，task graph 又写一部分。下面的账本展示了一个可靠完成函数要做的事：

Listing 17.6 complete_future 的提交边界

```

complete_future(task_id, terminal_result):
    lock future_state
    if state is not Pending:
        record duplicate completion bug
        unlock
        return
    store value, exception, canceled marker, or broken marker
    state = terminal state
    ready_ns = now
    waiters = waiter_count
    continuations = move continuation list
    unlock

    notify waiters
    enqueue continuations to their executors

    complete_task_state(task_id, terminal_result)
    decrement unfinished_tasks exactly once
    notify wait_idle if unfinished_tasks == 0

```

这里故意把“发布 FutureState 终态”和“运行 continuation”分开。终态要在锁内一次提交；唤醒和入队要在锁外做。若在锁内调用 continuation，用户代码可能再次调用 runtime、提交任务、等待另一个 future 或拿用户锁，最终把完成路径变成不可控递归。若 FutureState ready 以后没有同步更新 TaskState，调用者能拿到结果，但任务图还以为节点未完成；若只更新 TaskState，不发布 FutureState，外部 get 会卡住。

等待路径：外部线程、worker 和 coroutine 是三种不同入口 同一个 FutureState 可以被三种入口观察，但它们的合同不同。

Listing 17.7 三种等待入口的伪代码

```
external_get:
    lock FutureState
    while state is Pending:
        register external waiter
        cv.wait(lock)
    consume or rethrow
    unlock

worker_get_same_runtime:
    if runtime policy forbids blocking:
        return diagnostic: BlockingWaitInWorker
    if policy allows helping:
        leave user locks and runtime locks
        execute allowed ready work while target is Pending
    else if policy allows blocking pool:
        mark worker blocked and compensate

future_await:
    lock FutureState
    if state is terminal:
        unlock
        enqueue coroutine handle or continue fast path
    else:
        append continuation with resume executor
        unlock
        suspend without occupying worker
```

把这三条入口混成一个 `wait()` 函数，是很多运行时事故的根源。外部线程阻塞只是调用方延迟；worker 阻塞会减少执行槽；coroutine 等待应该登记 continuation 而不是 park 线程。接口层应能区分调用者是否位于 runtime worker 内。不能区分时，宁可默认禁止 worker 同池阻塞等待。

FutureState 和 TaskState 的一致性矩阵 FutureState 面向结果消费者；TaskState 面向调度器。二者不完全相同，但终态必须一致。

FutureState	TaskState	含义
ValueReady	Succeeded	用户函数成功，后继可按成功边推进
ExceptionReady	Failed	用户函数或 runtime wrapper 捕获异常，后继按失败策略处理
Canceled	Canceled	任务已被接纳但未执行或协作停止，等待者必须醒来
BrokenProducer	Failed	producer 端丢失，说明运行时合同破坏或 wrapper 未完成
Consumed	Succeeded/Failed/Canceled	结果已被普通 future 取走，不改变调度终态
Pending	Waiting/Ready/Running	任务还未进入可观察结果终态

不允许出现的组合也要写进断言。例如 TaskState 已经 Succeeded 而 FutureState 仍 Pending，说明完成函数漏写结果；TaskState 已经 Canceled 而 FutureState 仍 Pending，说明取消没有唤醒等待者；FutureState ExceptionReady 但 TaskState 仍 Running，说明 worker 异常路径没有收束。运行时 trace 应把这些组合当成 invariant violation，而不是普通错误日志。

submit 到 future ready 的完整链路 `submit` 返回 `future` 时，接纳、入队、执行、完成和读取必须连成一条线。下面的账本比“把 `callable` 放进队列”更严格：

Listing 17.8 `submit -> FutureState -> worker -> get` 的账本

```
submit(callable):
    lock runtime
    if state != Open:
        return RejectedDraining or RejectedStopped
    allocate FutureState and TaskNode
    producer endpoint is owned by the task wrapper
    consumer endpoint is returned to caller
    unfinished_tasks += 1
    enqueue ReadyItem
    unlock runtime
    notify worker

worker runs task wrapper:
    try:
        value = callable()
        complete_future_with_value(future_state, value)
    catch:
        complete_future_with_exception(future_state, current_exception)
    complete_task and decrement unfinished_tasks exactly once

future.get from caller:
    wait until FutureState is terminal
    ValueReady -> move result and mark Consumed
    ExceptionReady -> rethrow in caller thread
    Canceled/BrokenProducer -> report a defined error
```

这条链有三个容易错的提交点。第一，`submit` 只有在任务和 `FutureState` 都被接纳后才能返回成功；否则调用者会拿到一个无人满足的 `future`。第二，`worker` 必须捕获所有逃逸异常并写入 `FutureState`；否则异常停在 `worker` 线程，调用者只会永远等待或得到静默失败。第三，未完成计数和 `FutureState` 终态要一起提交；不能 `future` 已 `ready` 但 `wait_idle` 还永远等，也不能 `wait_idle` 返回但 `future` 仍 `Pending`。

提交失败也必须是可观察语义 线程池 `submit` 最危险的返回值不是 `Rejected`，而是“返回了一个永远 `Pending` 的 `future`”。拒绝提交有两种合理形态：要么返回 `SubmitResult::Rejected` 且不创建消费端；要么返回一个已经 `ready` 的 `future`，里面保存明确的 `rejection` 异常。两者都可以审查；不能审查的是“分配了 `FutureState`，入队失败，然后忘了满足它”。

Listing 17.9 `submit` 失败的两种合法合同

```
contract A:
    submit returns expected<Future<T>, SubmitError>
    if QueueFull or Draining:
        return unexpected(error)
    no FutureState is exposed to caller

contract B:
    submit returns Future<T>
```



```
if QueueFull or Draining:
    FutureState = ExceptionReady(submit_rejected)
    return future already ready

invalid:
    create FutureState
    fail to enqueue task
    return Future<T> whose state remains Pending forever
```

合同 A 更适合系统库，因为调用者必须处理拒绝。合同 B 更像异步 API，把拒绝也当成异步结果。无论选择哪种，trace 都应记录 reject reason、queue depth、runtime state 和是否创建 FutureState。没有这些字段，线上看到等待超时，很难区分任务还在排队、任务被拒绝、worker 死掉，还是 future 合同本身破坏。

阻塞等待、helping 等待和非阻塞 continuation 等待 FutureState 有三种工程形态。外部线程可以阻塞等待，因为它不是运行时的执行资源；compute worker 阻塞等待同一 runtime 的 future 会吃掉 worker；coroutine 或 continuation 则把等待者保存为后续任务，不占线程。它们不是性能微调，而是三种不同合同。

等待形态	语义	主要风险
外部 <code>get()</code>	调用线程睡眠，直到 future ready	调用方延迟增加，但不耗 compute worker
worker <code>get()</code>	worker 睡眠或忙等同池结果	ready 任务无人执行，形成饥饿
worker helping	等待时执行允许范围内 ready 任务	重入、锁顺序、栈增长、范围选择复杂
continuation	future ready 后把后继放入队列	控制流拆分，异常和取消要沿边传播
coroutine <code>co_await</code>	suspend frame, ready 后调度恢复	coroutine frame 生命周期和恢复 executor 要清楚

一个严格的最小运行时可以先禁止 worker 内部阻塞 `get()`，只允许外部 `get()` 和 continuation。这样错误更早暴露：若用户在 worker 里等待同池 future，接口直接失败或诊断，而不是偶发卡死。等任务图、trace 和执行域稳定后，再决定是否购买 helping 或 compensation worker 的复杂度。

异常传播不能绕开任务图 FutureState 保存异常只是第一步。任务图还要决定后继如何处理异常。若节点 A 失败，依赖 A 的 B 不能永远保持 Waiting；它必须变成 Canceled、Failed、RetryPending，或由上层策略改接到补偿节点。FutureState 面向调用者传播一次结果；TaskState 面向运行时传播调度事实。两者要一致。

Listing 17.10 异常从 callable 传播到 future 和后继

```
callable throws:
    FutureState: Pending -> ExceptionReady(exception_ptr)
    TaskState: Running -> Failed
    successors:
        if policy == fail_fast:
            Waiting/Ready successors -> Canceled
        if policy == retry:
            create retry attempt or RetryPending
    waiters:
        external future.get rethrows
        wait_idle wakes when all affected nodes are terminal
```

若只设置 `FutureState` 异常，不更新 `TaskState`，任务图可能还在等后继 `ready count`；若只把 `TaskState` 标为 `Failed`，不写 `FutureState`，调用者会拿不到异常。运行时完成路径必须同时处理两边，而且只能处理一次。

worker 内等待为什么危险 外部线程等待 `future` 通常只是让调用者停住；`worker` 等待同一个运行时里的 `future` 则可能消耗执行资源。若被等待的任务还需要 `worker` 执行，等待者就挡住了被等待者。这不是“`future` 有问题”，而是“等待发生在错误的位置”。

可选修复有三类。第一，禁止 `worker` 内部同步等待同池任务，把后续逻辑注册成 `continuation`。第二，允许 `work helping`：`worker` 等待某个 `task group` 时，主动执行同一组的 `ready` 任务。第三，把可能阻塞的任务隔离到专用执行域，例如 `I/O blocking pool`，不占用 `compute worker`。三者都可以正确，但语义和复杂度不同，必须写进接口。

从底层看，等待有两种常见形态。阻塞等待会让线程睡到内核对象上，Linux 上常见路径最终会落到 `futex` 或类似机制；线程不烧 CPU，但 `worker` 数量减少了。忙等会不断读某个状态位，可能还用 `pause` 降低流水线压力；线程仍占用 CPU，状态位所在 `cache line` 还会被反复读取。二者都不适合长期发生在 `compute worker` 内。`compute worker` 的职责是推进 `ready work`，而不是保存等待上下文。

Continuation 的直觉 `Continuation` 把“等 B 完成后继续做 A 的后半段”变成一个新的 `task`。B 完成时，运行时更新 `shared state`，递减后继的依赖计数，计数归零后把 `continuation` 放入 `ready` 队列。这样等待不占线程，`worker` 可以去执行其他 `ready task`。

`Continuation` 也有代价。控制流不如同步代码直观，异常和取消需要沿依赖边传播，深层 `continuation` 可能造成调试困难。可靠运行时不是盲目禁止同步，而是明确区分：外部同步等待可以简化调用者；`worker` 内同步等待必须被运行时理解，或者被接口禁止。

Coroutine：把 future 等待编译成 continuation C++ `coroutine` 可以把“等待结果后继续执行”的代码写成顺序形态，但运行时看到的仍然是 `continuation`。`Coroutine frame` 保存局部变量、暂停点和返回对象；`co_await` 某个 `future-like` 对象时，`awaiter` 决定是否立即继续、是否挂起、挂起后把 `coroutine handle` 注册到哪里、恢复时如何返回值或重抛异常。

Listing 17.11 `future-like awaiter` 的运行时语义

```
await_ready:
    if FutureState is terminal:
        return true
    return false

await_suspend(coroutine_handle h):
    lock FutureState
    if state became terminal:
        enqueue h to current executor
    else:
        append h to continuations
    unlock
    return to worker without blocking the worker thread

future completion:
    store value, exception, or cancellation
    detach continuation handles
    enqueue handles onto the selected runtime queue
```



```
Suspended:
    awaited FutureState completes -> Runnable
    cancellation requested         -> CancelRequested

FinalSuspended:
    schedule continuation
    owner observes result
    destroy frame -> Destroyed
```

这张状态机比 `co_await` 语法更重要。若 `Running` 状态下直接销毁 `frame`，就是正在执行的栈被释放。若 `Suspended` 状态没有记录它挂在哪个 `FutureState` 上，取消和诊断都找不到等待边。若 `FinalSuspended` 直接恢复 `continuation` 而不是入 `executor`，`completion` 线程可能执行大量用户代码。若 `Destroyed` 之后 `FutureState completion` 仍握着 `handle`，下一次 `resume` 就是 `use-after-free`。

协程恢复执行域 协程等待完成后由谁恢复，是运行时合同的一部分。恢复到 `producer` 线程，延迟最低但容易把 `I/O completion` 或锁内路径变成用户代码执行路径；恢复到原 `compute executor`，语义稳定但需要一次入队；恢复到 `caller` 线程通常不可行，因为 `caller` 可能早已返回；恢复到 `dedicated continuation executor` 可以隔离长回调，但增加调度成本。

恢复位置	购买能力	主要风险
producer completion 线程	少一次入队，延迟低	锁内或 I/O 线程执行用户代码
原 compute executor blocking/I/O executor	执行域清楚，trace 连续 隔离慢 continuation	多一次调度，可能排队 需要跨域 backpressure
inline fast path	ready 时避免 suspend	双 resume 和递归深度要审查

本书建议最小运行时只允许两条路径：`ready fast path` 直接继续；`pending path` 注册 `continuation`，并在 `completion` 后把 `handle` 入原 `executor`。这样 `trace` 能把每次恢复都记成 `ReadyItem`，锁顺序也更容易审查。后续若要 `inline resume`，必须先证明不会在 `FutureState` 锁内、`runtime` 锁内或用户不可重入区域内执行。

协程和取消：取消请求不是终态 取消协程有三层事实：有人请求取消，协程观察到取消，`FutureState` 进入可观察终态。只设置 `stop flag` 不会唤醒等待者；只丢弃 `handle` 会泄漏或悬挂；只把 `FutureState` 标 `Canceled` 但 `frame` 还会被 `executor` 恢复，也会形成重复终态。

Listing 17.14 coroutine cancellation 的审查账本

```
request_cancel(coroutine_task):
    mark cancel_requested
    if task is queued:
        remove or let worker observe canceled before running
        complete FutureState as Canceled
    if task is suspended on FutureState X:
        unregister continuation from X if possible
        complete outer FutureState as Canceled
    if task is running:
        set cooperative stop token
        running code must reach a cancellation point
```

```
await_resume after cancellation:
    throw task_canceled or return canceled result
    never silently continue with default value
```

这里没有免费的强制杀线程。C++ coroutine 取消通常是协作式的：运行中的用户代码必须在某些点观察 stop token 或被等待对象返回取消结果。运行时能做的是不再调度尚未运行的 continuation，唤醒等待者，并确保所有状态进入终态。

Awaiter 合同：它是调度器边界，不是语法装饰。一个 awaiter 同时面对三方：当前 coroutine frame、被等待的状态对象、恢复它的 executor。只写 `await_ready/await_suspend/await_resume` 三个函数名还不够，运行时必须写清它们分别持有什么锁、是否允许 inline resume、completion 发生在哪个线程、恢复后异常如何传播。下面是 Compute Systems Engine 可以采用的最小 awaiter 合同。

Listing 17.15 运行时 awaiter 的最低合同

```
RuntimeAwaiter:
    observed_state:
        FutureState, IoRequestState, TimerState, or TaskDependency
    await_ready:
        read terminal state with acquire semantics
        return true only if await_resume can complete immediately
    await_suspend:
        register coroutine_handle exactly once
        record resume_executor
        never block the current worker
        never call user code under observed_state lock
    completion path:
        publish terminal state with release semantics
        detach continuation handles
        enqueue handles to resume_executor
    await_resume:
        read terminal result
        return value, return cancellation result, or rethrow exception
```

这个合同把 awaiter 和普通回调区分开。回调常常由 completion 线程直接调用，容易在不可控线程和锁内执行用户代码；awaiter 应该把恢复动作交给 executor。这样运行时 trace 能看到“哪个请求完成，哪个 coroutine 被入队，哪个 worker 恢复了它”。没有这条 trace，`co_await` 后面的延迟很难归因：可能是 I/O 慢、completion loop 慢、executor 队列堵，也可能是 coroutine 自己执行慢。

Awaitable 的四种来源 运行时会遇到多种 awaitable，但它们都应被降低成“状态对象 + continuation 注册 + executor 恢复”。

awaitable 来源	状态对象	completion 由谁产生
任务 future	FutureState	worker 完成 task wrapper
任务依赖	TaskState/ready count	最后一个前驱完成线程
timer/sleep	TimerState	timer wheel 或 event loop
异步 I/O	IoRequestState 或 CompletionRecord	I/O completion loop 或 blocking I/O domain
远端 RPC	RpcCallState	network event loop 或 RPC runtime

表里每一行都能用同一个调度动作结束：把 coroutine handle 包成

ReadyItem, 进入某个 executor 队列。不同的只是状态对象和 completion 来源。这个统一模型非常重要, 因为它让 future、timer、I/O、RPC 都走同一套取消、trace、背压和 shutdown 规则。

Scheduler 看到的是 ReadyItem, 不是 C++ 语法 协程恢复时, scheduler 不需要知道源代码写的是 `co_awaitread()` 还是 `co_awaitsleep()`。它看到的应是一条 ReadyItem: 恢复哪个 coroutine、属于哪个 request/task、恢复到哪个执行域、是否带取消或异常摘要。

Listing 17.16 协程恢复 ReadyItem 的语义字段

```
1 enum class ReadyItemKind : std::int32_t {
2     TaskFunction,
3     CoroutineResume,
4     IoCompletionContinuation,
5     TimerContinuation,
6 };
7
8 struct CoroutineResumeItem {
9     std::uint64_t coroutine_id = 0;
10    std::uint64_t source_state_id = 0;
11    ReadyItemKind kind = ReadyItemKind::CoroutineResume;
12    ExecutorId resume_executor;
13    FutureErrorKind error_kind = FutureErrorKind::None;
14 };
```

这段结构不是为了限制实现, 而是为了固定观测字段。若恢复项没有 `source_state_id`, trace 只能看到 coroutine 突然运行, 无法知道它等的是哪个 I/O 或 future。若没有 `resume_executor`, 跨执行域恢复就会变成隐式回调。若没有 `error_kind`, 异常路径只能靠用户日志追。

调度器和操作系统调度器的边界 项目里的 scheduler 调度的是用户态工作项; Linux scheduler 调度的是内核线程。Coroutine suspend 只让用户态工作项离开 ready 队列, 不会让操作系统知道“这个业务请求在等 I/O”。只有当某个 worker 队列为空、线程进入 CV/futex sleep, 或者执行阻塞系统调用时, 内核才看到线程睡眠。

Listing 17.17 两层调度器的边界

```
user-level scheduler:
    owns ready task queue
    owns coroutine ready handles
    decides which worker executes which ReadyItem
    records task/coroutine/request trace

OS scheduler:
    owns native threads
    runs or parks worker threads
    observes futex sleep, blocking syscall, runnable delay
    does not know task dependency graph
```

这解释了为什么“CPU 线程数很多”不等于“任务运行时有执行资源”。所有 worker 都在 futex 上睡, 说明 ready 队列可能为空; 所有 worker 都在阻塞 I/O, 说明 runtime 把设备等待放错执行域; 所有 worker 都在 `future.get` 等同池任

务，说明用户态调度器还有 ready work，但执行槽被隐藏等待占住。报告必须同时记录 OS 线程状态和用户态 worker/task 状态。

Awaiter 的失败模式清单 协程相关 bug 经常不是语法错误，而是运行时合同破坏。最小失败模式清单如下：

失败模式	具体错误	证据
双恢复	ready fast path 和 completion path 都 resume	coroutine id resume count 大于 1
丢恢复	completion 发生在注册窗口，handle 未入队	FutureState terminal 但 coroutine Suspended
锁内恢复	completion 持锁直接执行用户代码	lock hold ns 包含用户函数
错执行域恢复	I/O loop 直接跑 CPU continuation	worker trace 显示 event loop 被占住
取消不终态	stop flag 设置但 FutureState Pending	waiters 不醒，shutdown 卡住
frame 过早销毁	task 析构 destroy 仍可能被 resume 的 frame	use-after-free 或 handle stale

这些失败模式应该进入测试矩阵：并发注册和 completion、ready fast path、late completion、取消、shutdown、跨 executor 恢复、锁内恢复检测。只测试 `co_await` 成功返回值，没有覆盖协程运行时真正危险的边界。

17.2 任务图：把等待变成状态

第 13 章的 reduce、scan、partition 已经把算法拆成阶段。count 必须先完成，scan 才能计算 offsets；scan 完成后，write 才能写入独占区间；所有 write 完成后，manifest 才能提交。这种结构天然就是任务图。若运行时只提供 `submit`，程序很容易写成 barrier 或 worker 内等待；若运行时支持任务图，依赖就能成为调度事实。

ready count 任务图的节点表示可执行工作，边表示依赖。每个节点记录还有多少前驱未完成。初始计数为零的节点进入 ready 队列；某个节点完成后，遍历后继并递减它们的计数；减到零的那个线程负责把后继入队。这个计数通常叫 ready count 或 remaining predecessors。

ready count 的本质是把“等待所有前驱”变成一个原子状态变化。假设节点 X 有三个前驱，计数初值为 3。每个前驱完成时执行一次递减，只有观察到递减前值为 1 的那个完成线程，才拥有把 X 发布到 ready 队列的权利。这个点就是 X 从 Waiting 到 Ready 的线性化点。若两个线程都能把 X 入队，就会重复执行；若没有线程负责入队，X 会永远等待。任务图运行时的正确性，往往就藏在这种看似小的原子边界里。

Listing 17.18 任务图节点的核心调度字段

```
1 enum class TaskState : std::int32_t {
2     Waiting,
3     Ready,
4     Running,
5     Succeeded,
6     Failed,
7     Canceled,
8 };
9
10 struct TaskNode {
```

```

11  std::uint64_t task_id = 0;
12  std::uint32_t first_successor = 0;
13  std::uint32_t successor_count = 0;
14  std::atomic<std::uint32_t> remaining_predecessors{0};
15  std::atomic<TaskState> state{TaskState::Waiting};
16  };

```

这里直接把状态写成枚举，是为了让示例代码先表达清楚生命周期。大型运行时可能把状态压缩成整数编码，或者把热调度字段和冷诊断字段分开存放；但无论编码方式怎样，状态都必须是程序可判断的事实，而不是日志字符串。

任务状态不是日志字符串 状态要能被程序判断，而不是只写在日志里。一个任务从 `Waiting` 进入 `Ready`，必须有唯一线性化点；从 `Running` 进入 `Succeeded` 或 `Failed`，必须释放结果并唤醒后继；从 `Ready` 进入 `Canceled`，worker 取出时必须能跳过它并完成计数。若状态只是“打印了一行 `ready`”，运行时无法防止重复入队、重复完成和取消后仍执行。

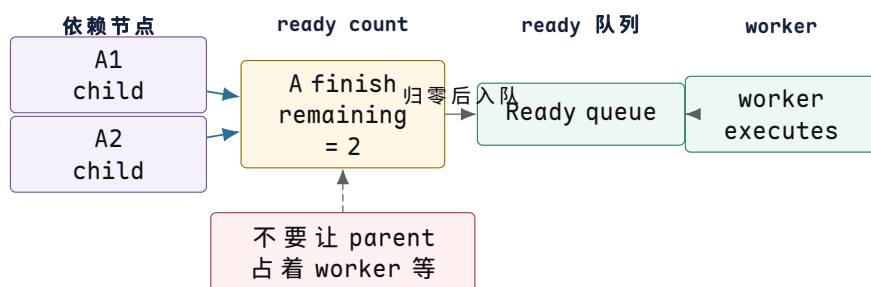
状态	进入条件	允许的后续状态
Waiting	依赖尚未满足	Ready、Canceled、Failed
Ready	ready count 归零，任务已入队	Running、Canceled
Running	worker 成功取得任务并开始执行	Succeeded、Failed、Canceled
Succeeded	用户函数正常完成，结果可见	终态
Failed	用户函数或运行时错误不可忽略	终态，触发后继策略
Canceled	上游失败、用户取消或 shutdown cancel	终态

完成路径是运行时最关键的代码 任务完成时，运行时要保存结果或异常、转换任务状态、更新 `trace`、递减后继 `ready count`、把变 `ready` 的任务入队、通知等待者，并在失败时触发取消传播。这些动作应集中在一个完成函数中，不能让 `worker loop`、`future`、取消逻辑各自修改状态。状态转换分散，是任务运行时最常见的错误来源。

完成路径还要避免在锁内执行未知用户代码。若一个 `continuation` 是用户函数，完成线程不应在持有 `shared state` 锁时直接调用它；应先收集需要唤醒或入队的任务，释放锁，再发布到 `ready` 队列。这个原则会在 `I/O completion` 和分布式回调中反复出现。

完成路径还承担内存发布。用户函数写入结果对象后，运行时把状态改成 `Succeeded`；等待者或后继看到 `Succeeded` 后必须能看到结果内容。`mutex` 版本可以依赖锁的 `release/acquire`；无锁版本需要显式 `release` 存储和 `acquire` 读取。`ready` 队列也是同一个问题：生产者必须先完整构造 `task node`，再发布队列指针或索引；worker 取出 `task` 后才能读到正确字段。若发布顺序错了，bug 可能表现为偶发空函数、旧参数、丢失 `successor` 或错误 `trace`。

任务图把等待移出 worker 调用栈



父任务的后半段成为 A finish 节点。A1 和 A2 完成时只改变状态，不让父任务阻塞在 worker 栈上。

图示：本图要证明的命题是：任务图不是画法，而是把等待关系变成 ready count 和后继入队规则。

结果所有权和任务所有权要分开 任务对象完成了，不表示结果对象可以释放。future 可能还没被调用者读取，trace 可能还要汇总，后继任务可能只需要错误摘要而不是完整任务对象。高性能运行时通常把热调度节点、用户结果、诊断记录和队列节点分开管理。示例实现可以简单一些，但必须说明为什么“任务执行完就 delete”在 future 和 continuation 存在时非常危险。

关键路径决定任务图上限 任务图性能由总工作量和关键路径共同决定。总工作量是所有任务耗时之和；关键路径是依赖图中必须串行经过的最长路径。线程数再多，也不能突破关键路径。若实际时间远高于关键路径，说明调度、同步、I/O 或负载倾斜有问题；若实际时间接近关键路径，继续增加 worker 收益有限，应优化依赖结构或关键任务本身。

实验必须输出 trace，而不是只输出总耗时。Trace 至少要能估计每个任务的 ready 等待时间、运行时间和前驱关系。没有这些数据，无法判断慢在关键路径、队列等待、窃取失败还是任务粒度。

17.3 从 worker loop 到任务图调度：完整执行路径

运行时的最小内核可以拆成六条路径：提交任务、发布 ready 任务、worker 取任务、任务完成、等待 idle、关闭运行时。每条路径都要有线性化点。线性化点是一次并发操作在语义上“生效”的瞬间：submit 何时算被接纳，任务何时算 ready，完成何时算对后继可见，关闭何时开始拒绝新任务。没有这些边界，偶发重复执行、丢任务、等待不醒和关闭卡住都很难定位。

运行时表：把热调度字段和冷诊断字段分开 一个小型实现可以把任务存入 task table。调度热路径只需要状态、ready count、函数入口和 successor 范围；trace、错误摘要、输入范围和调试字符串可以放在冷表。这样 worker loop 不会为了每个任务搬运大量诊断字段。

Listing 17.19 任务运行时表的最小形态

```
1 struct ReadyItem {
2     std::uint64_t task_id = 0;
3     std::uint32_t attempt = 0;
4 };
5
6 struct TaskHotFields {
7     std::atomic<TaskState> state{TaskState::Waiting};
8     std::atomic<std::uint32_t> remaining_predecessors{0};
9     std::uint32_t first_successor = 0;
```

```

10  std::uint32_t successor_count = 0;
11  TaskFunction function;
12  };
13
14  struct TaskColdFields {
15      std::uint64_t submit_ns = 0;
16      std::uint64_t ready_ns = 0;
17      std::uint64_t start_ns = 0;
18      std::uint64_t finish_ns = 0;
19      TaskErrorSummary error;
20  };
21
22  struct RuntimeCore {
23      std::mutex mutex;
24      std::condition_variable cv;
25      RuntimeState state = RuntimeState::Open;
26      std::deque<ReadyItem> global_ready;
27      std::uint64_t unfinished_tasks = 0;
28      std::vector<TaskHotFields> hot;
29      std::vector<TaskColdFields> cold;
30  };

```

这个版本故意使用 `mutex` 和全局 `ready` 队列。它不是最终高性能结构，却适合作为语义 `reference`。全局队列让关闭、异常、未完成计数和 `trace` 更容易验证；等这些语义稳定后，再把队列换成本地 `deque` 和 `stealing`。若一开始就写无锁 `deque`，调度策略错误和内存同步错误会混在一起。

submit：接纳和拒绝必须有返回值 `submit` 的线性化点应在持锁状态下完成：检查 `runtime` 状态、分配 `task id`、写入 `task` 表、增加 `unfinished` 计数、如果无依赖则入 `ready` 队列。只有这些动作完成后，`submit` 才能返回 `Accepted`。若 `runtime` 已经进入 `Draining` 或 `Stopped`，它必须返回明确拒绝，而不是静默丢弃函数对象。

Listing 17.20 `submit` 的语义伪代码

```

1  SubmitResult submit(RuntimeCore& rt, TaskSpec spec) {
2      std::unique_lock lock(rt.mutex);
3      if (rt.state == RuntimeState::Draining) {
4          return SubmitResult::RejectedDraining;
5      }
6      if (rt.state == RuntimeState::Stopped) {
7          return SubmitResult::RejectedStopped;
8      }
9
10     const std::uint64_t task_id = allocate_task_id(rt);
11     rt.hot[task_id] = build_hot_fields(spec);
12     rt.cold[task_id].submit_ns = now_ns();
13     ++rt.unfinished_tasks;
14
15     if (spec.predecessor_count == 0) {
16         mark_ready_locked(rt, task_id);
17     }
18

```



```

19 lock.unlock();
20 rt.cv.notify_one();
21 return SubmitResult::Accepted;
22 }

```

这里有两个容易忽略的点。第一，`unfinished_tasks` 只在接纳成功后增加；被拒绝的任务不能计入，否则 `wait_idle` 会永远等不到归零。第二，`ready` 入队必须发生在通知之前；通知只是让 `worker` 更快观察队列非空，不能替代队列状态本身。

mark ready: 只能发布一次 任务从 `Waiting` 到 `Ready` 必须只能发生一次。对于无依赖任务，`submit` 负责发布；对于有依赖任务，最后一个前驱的完成线程负责发布。发布前要写好任务字段和 `trace` 时间，发布后 `worker` 才可能取出执行。

Listing 17.21 发布 `ready` 任务的伪代码

```

1 void mark_ready_locked(RuntimeCore& rt, std::uint64_t task_id) {
2     TaskState expected = TaskState::Waiting;
3     const bool changed = rt.hot[task_id].state.compare_exchange_strong(
4         expected,
5         TaskState::Ready,
6         std::memory_order_release,
7         std::memory_order_relaxed);
8     if (!changed) {
9         return;
10    }
11
12    rt.cold[task_id].ready_ns = now_ns();
13    rt.global_ready.push_back(ReadyItem{task_id, 0});
14 }

```

`mutex` 版本通常不需要显式写这么细的 `memory order`，因为锁已经建立 `happens-before`。伪代码仍然写出 `release`，是为了强调发布语义：生产者必须先构造完整任务，再让 `worker` 通过队列看到它。后续换成无锁 `ready queue` 时，这个边界不能丢。

worker loop: 只长期执行 ready work `worker loop` 的职责很窄：等待 `ready item`，取出任务，尝试把状态从 `Ready` 改成 `Running`，执行用户函数，把结果交给 `complete_task`。它不应在持有 `runtime` 锁时执行用户函数，也不应在用户函数内部同步等待同一 `runtime` 的 `future`。

Listing 17.22 全局队列版 `worker loop` 伪代码

```

1 void worker_loop(RuntimeCore& rt, std::uint32_t worker_id) {
2     for (;;) {
3         ReadyItem item;
4         {
5             std::unique_lock lock(rt.mutex);
6             rt.cv.wait(lock, [&] {
7                 return !rt.global_ready.empty() ||
8                     rt.state == RuntimeState::Stopped ||
9                     (rt.state == RuntimeState::Draining &&
10                      rt.unfinished_tasks == 0);
11             });
12

```

```

13     if (rt.global_ready.empty()) {
14         if (rt.state != RuntimeState::Open) {
15             return;
16         }
17         continue;
18     }
19
20     item = rt.global_ready.front();
21     rt.global_ready.pop_front();
22 }
23
24 TaskHotFields& task = rt.hot[item.task_id];
25 TaskState expected = TaskState::Ready;
26 if (!task.state.compare_exchange_strong(expected, TaskState::Running)) {
27     continue;
28 }
29
30 rt.cold[item.task_id].start_ns = now_ns();
31 TaskErrorSummary error = run_user_function(task.function);
32 complete_task(rt, item.task_id, error);
33 }
34 }

```

这个循环展示了三个原则。第一，等待谓词检查队列和生命周期状态，而不是检查“是否收到通知”。第二，取任务后释放锁，再执行用户函数。第三，从 **Ready** 到 **Running** 用状态转换保护，即使取消或关闭把 ready 任务改成终态，worker 取到旧 item 也能跳过。

complete task：运行时的关键提交点 任务完成时，要把用户函数结果发布到 shared state，记录 trace，转换终态，递减后继依赖，减少 unfinished 计数，并唤醒等待者。这个路径应集中在一个函数里，避免 worker loop、future、取消传播各自修改计数。

Listing 17.23 complete task 的语义伪代码

```

1 void complete_task(RuntimeCore& rt,
2                     std::uint64_t task_id,
3                     TaskErrorSummary error) {
4     std::vector<std::uint64_t> ready_successors;
5     bool notify_all = false;
6
7     {
8         std::unique_lock lock(rt.mutex);
9         TaskState terminal =
10             error.kind == TaskErrorKind::None ? TaskState::Succeeded
11             : TaskState::Failed;
12
13         rt.cold[task_id].error = error;
14         rt.cold[task_id].finish_ns = now_ns();
15         rt.hot[task_id].state.store(terminal, std::memory_order_release);
16
17         for_each_successor(rt, task_id, [&](std::uint64_t succ) {
18             if (error.kind != TaskErrorKind::None) {

```

```

19     cancel_successor_locked(rt, succ, error);
20     return;
21 }
22 if (try_make_ready_locked(rt, succ)) {
23     ready_successors.push_back(succ);
24 }
25 });
26
27 --rt.unfinished_tasks;
28 if (rt.unfinished_tasks == 0) {
29     notify_all = true;
30 }
31 }
32
33 if (!ready_successors.empty()) {
34     rt.cv.notify_all();
35 } else if (notify_all) {
36     rt.cv.notify_all();
37 }
38 }

```

这里 `ready_successors` 只是说明完成路径会产生新 ready 任务；实际实现可以在锁内入队后统一通知。失败路径不能遗漏 `unfinished` 计数，也不能让后继永远停在 `Waiting`。若策略是“前驱失败则后继取消”，取消本身也要进入终态并减少 `unfinished` 计数；若策略是“前驱失败后重试”，则 driver 要创建新的 `attempt`，并让旧 `attempt` 的后继等待新的结果。

try make ready: ready count 的线性化点 后继有多个前驱时，只有最后一个完成的前驱能把它入队。这个线性化点通常是 `fetch_sub` 观察到旧值为 1。旧值不是 1 的完成线程，只是把计数减一，没有发布权。

Listing 17.24 后继进入 ready 的原子边界

```

1 bool try_make_ready_locked(RuntimeCore& rt, std::uint64_t task_id) {
2     const std::uint32_t old =
3         rt.hot[task_id].remaining_predecessors.fetch_sub(
4             1, std::memory_order_acq_rel);
5     if (old != 1) {
6         return false;
7     }
8     mark_ready_locked(rt, task_id);
9     return true;
10 }

```

若 `remaining_predecessors` 初值错了，运行时会出现两类相反事故：初值过大，后继永远不 ready；初值过小，后继提前执行，可能读到未产生的 `counts`、`offsets` 或 `manifest`。任务图构建阶段必须检查边数、重复边和自环；调度阶段不能靠运气弥补图构建错误。

wait idle: 不能只看队列为空 `wait_idle` 的含义是所有已接纳任务都进入终态。它不能只看 ready 队列为空，因为 worker 可能已经取出任务正在运行；也不能只看 worker 空闲，因为某个提交者可能已经接纳任务但尚未入队。`unfinished` 计数把这些状态统一起来：接纳成功加一，进入终态减一。

Listing 17.25 wait idle 的语义伪代码

```

1 void wait_idle(RuntimeCore& rt) {
2     std::unique_lock lock(rt.mutex);
3     rt.cv.wait(lock, [&] {
4         return rt.unfinished_tasks == 0 ||
5             rt.state == RuntimeState::Stopped;
6     });
7 }

```

异常、取消和关闭路径都要审查 unfinished 是否平衡。最常见的 bug 是用户函数抛异常后 worker 提前退出，没有减少计数；或者 shutdown cancel 标记 ready 任务为 **Canceled**，却没有为每个被取消任务减少计数。这样的运行时在成功路径上看似正常，一到失败路径就卡死。

shutdown: drain 和 cancel 是两种协议 shutdownrain 停止接收新任务，继续执行已接纳任务，直到 unfinished 归零后 worker 退出。shutdowncancel 停止接收新任务，把未开始任务标记为 **Canceled**，运行中任务通过 cancel token 协作结束。二者返回给调用方的语义不同，不能混成一个含糊的 shutdown()。

Listing 17.26 shutdown drain 的语义伪代码

```

1 void shutdown_drain(RuntimeCore& rt) {
2     {
3         std::unique_lock lock(rt.mutex);
4         if (rt.state == RuntimeState::Stopped) {
5             return;
6         }
7         rt.state = RuntimeState::Draining;
8     }
9     rt.cv.notify_all();
10
11     wait_idle(rt);
12
13     {
14         std::unique_lock lock(rt.mutex);
15         rt.state = RuntimeState::Stopped;
16     }
17     rt.cv.notify_all();
18     join_all_workers();
19 }

```

与 submit 并发时，shutdown_drain 的状态转换是拒绝新任务的线性化点。若 submit 先在锁内观察到 Open 并完成接纳，它就必须被 drain 执行完；若 drain 先把状态改成 Draining，后续 submit 必须返回拒绝。不能出现 submit 返回成功但任务既不执行也不取消。

三条事故账本 把完整路径写清后，经典事故会变成可解释的状态账本。

Listing 17.27 future 饥饿的事件序列

```

t0: worker 0..3 start parent tasks
t1: each parent submits one child task
t2: child tasks are Ready in the same runtime
t3: each parent calls future.get inside worker

```

```
t4: all workers block or spin waiting for children
t5: ready queue has children, but no worker can execute them
fix: parent continuation becomes a task; worker never waits on same pool
```

Listing 17.28 关闭竞态的事件序列

```
t0: submit A observes Open under runtime mutex
t1: submit A increments unfinished and enqueues A
t2: shutdown_drain changes state to Draining
t3: submit B observes Draining and returns RejectedDraining
t4: workers finish A, unfinished becomes zero
t5: runtime becomes Stopped and workers join
```

Listing 17.29 前驱失败传播的事件序列

```
t0: count 2 starts Running
t1: count 2 detects corrupted input and returns UserException
t2: complete_task marks count 2 Failed
t3: scan node is marked Canceled, not left Waiting
t4: write nodes and commit are canceled or skipped by policy
t5: wait_idle observes all affected nodes reached terminal states
```

这三条账本分别约束 worker 内等待、关闭线性化点和失败传播。运行时实现完成后，测试应能复现这些序列，并在 trace 中看到相同状态转移。若 trace 无法表达这些事件，说明观测字段还不够。

17.4 运行时证据链：ready、running、blocked 和 helping

任务运行时的错误常藏在“线程还活着，但系统不推进”里。全局队列不为空，却没有 worker 执行；worker 数量足够，却全部卡在 future、条件变量或 I/O；任务图理论上有并行度，trace 里却出现长时间空洞。要定位这些问题，运行时不能只记录 task start 和 finish，还要记录 worker 状态。Worker 是执行资源，任务是工作单位，二者的关系随时间变化。只有把关系写进 trace，才能判断慢在队列、运行、阻塞、窃取还是关闭。

Listing 17.30 worker trace 状态

```
Idle
DequeueLocal
DequeueGlobal
StealAttempt
Running
PublishingCompletion
WaitingExternal
Helping
Parked
Waking
Shutdown
```

Idle 表示 worker 暂时没有工作；StealAttempt 表示它正在寻找别的 deque；Running 表示它正在执行用户任务；PublishingCompletion 表示用户函数已经结束，运行时正在发布结果、递减后继 ready count 和唤醒等待者；WaitingExternal

表示 worker 进入了运行时无法直接推进的阻塞，例如磁盘 I/O、网络等待或外部 mutex；**Helping** 表示 worker 在等待某个结果时没有睡死，而是执行运行时中其他 ready work；**Parked** 表示没有可执行任务时进入睡眠。

隐藏阻塞：线程池最容易被自己耗尽 第 13 章的 stable filter 会产生 count、scan、write、commit 任务。如果 count 任务在 worker 内部等待“所有 count 完成后再继续 scan”，就可能让 worker 全部占在父任务栈上，ready 队列里的后继无人执行。这个事故不需要死锁循环，只要 worker 数等于阻塞父任务数就够了。更隐蔽的版本是任务调用 `future.get()`、等待条件变量、同步读文件、请求网络服务、或在锁内执行会阻塞的析构。运行时看到 worker 还没退出，但执行资源已经从 ready 队列中消失。

Listing 17.31 worker 被隐藏阻塞耗尽的 trace

```
t0 worker 0 Running parent_count_group
t1 worker 1 Running parent_count_group
t2 worker 2 Running parent_count_group
t3 worker 3 Running parent_count_group
t4 each parent submits child scan or waits for sibling count
t5 workers enter WaitingExternal or future wait
t6 ready queue has runnable children
t7 no worker returns to DequeueLocal or DequeueGlobal
```

这类问题不能靠增加 `notify_all` 修复，因为等待的不是条件变量漏通知，而是执行资源被错误占用。修复必须改变等待协议：把后续工作写成 continuation、允许 worker helping、创建补偿 worker、把阻塞 I/O 放进单独的 blocking pool，或干脆禁止 worker 等待同一运行时的 future。

四种 managed blocking 策略 Managed blocking 的核心是：运行时知道某个 worker 即将长期不可用，于是采取动作保持执行资源。策略不同，语义和成本也不同。

策略	适用场景	风险
禁止 worker wait	可把依赖全部改成 continuation	API 约束强，调用方需要改结构
continuation work helping	任务完成后发布后半段等待者执行其他 ready task	状态拆分更细，错误传播要清楚可能重入运行时，栈和锁顺序要审查
compensation worker	worker 阻塞时临时补一个线程	可能过度创建线程，CPU 过度订阅
blocking pool	把 I/O 或长等待放到专门线程池	跨池调度复杂，背压必须明确

禁止 worker wait 是最干净的语义：任务函数不能调用同一运行时 future 的阻塞等待，只能返回、发布 continuation 或把后继节点交给任务图。Continuation 把“等待后继续执行”改成“前驱完成后新任务 ready”，正好对应第 13 章的 stage 状态机。Helping 允许 worker 在等待目标 future 时执行其他 ready task，它能避免线程池耗尽，但必须禁止在持有用户锁或运行时内部锁时进入 helping，否则会把锁顺序变成难以分析的递归图。Compensation worker 像急救措施：当 worker 声明自己将阻塞，运行时临时增加执行线程；它能保护吞吐，但会增加调度成本和内存栈成本。Blocking pool 则承认 I/O 等待和 CPU 任务不是一种资源，用单独线程承接阻塞。

helping 的边界 Helping 不是“随便跑点别的任务”。等待者必须声明自己等待的目标、允许执行的任务范围、是否允许执行依赖当前栈帧的后继、以及如何避免重入锁。一个简化协议可以这样写：

Listing 17.32 work helping 的语义边界

```

before helping:
    worker must not hold runtime mutex
    worker must not hold user locks declared non-reentrant
    waited future belongs to the same runtime
    target task is not already terminal

helping loop:
    if target becomes terminal, return its result
    run one ready task that is safe for this worker
    publish completion and trace
    periodically check cancellation and shutdown

exit:
    do not consume the waited result twice
    restore worker state from Helping to Running or Idle

```

若 helping 可以执行任意任务，可能出现重入用户代码、递归提交、栈增长和锁顺序反转。若 helping 范围太窄，又可能找不到可执行任务，退化成忙等。实践中常给任务标注执行域：CPU task、blocking task、scheduler internal task、continuation task。等待 CPU future 的 worker 可以帮助 CPU task，但不能在持有某个用户对象锁时执行会回调同一对象的任务。

park 和 wake 也要有合同 没有 ready work 时，worker 不能无限自旋；它应该尝试窃取、检查全局队列和关闭状态，然后 park。Park 的本质和条件变量相同：线程睡眠等待谓词变化。谓词通常是“本地队列非空、全局队列非空、某个 victim 可偷、运行时进入关闭状态”。发布 ready 任务时必须先把任务放进队列，再 wake 或 notify。若先 wake 后入队，worker 可能醒来看到空队列并再次睡眠，ready 任务就会滞留到下一次唤醒。

Listing 17.33 ready 发布和 worker park 的顺序

```

publish ready:
    write task fields
    push ReadyItem into a visible queue
    update counters used by park predicate
    wake one or more parked workers

worker park:
    check local queue
    check global queue
    attempt bounded stealing
    if no work and runtime open:
        sleep on condition variable, semaphore, or platform park primitive
    after wake:
        recheck queues and shutdown state

```

这正是第 10 章条件变量原则在运行时中的放大版本：通知不是任务，任务在队列和状态表里；醒来不是拥有任务，worker 仍要重新检查。Park/wake 的 bug 往往表现为低概率 hang，原因是丢失唤醒窗口、ready 计数和队列状态不一致，或者关闭状态没有进入等待谓词。

运行时报告字段 运行时报告至少要能把“没有任务可跑”和“有任务但没人跑”

区分开。前者可能是关键路径限制，后者是调度事故。

Listing 17.34 任务运行时的 trace 和汇总字段

```
runtime.ready_wait_ns.p50
runtime.ready_wait_ns.p99
runtime.run_ns.p50
runtime.run_ns.p99
runtime.blocked_worker_count
runtime.helped_tasks
runtime.steal_attempts
runtime.steal_success
runtime.critical_path_ns
runtime.unfinished_tasks
runtime.ready_queue_depth.max
runtime.park_wake_count
runtime.compensation_workers_created
runtime.external_wait_ns
```

`ready_wait_ns` 是任务进入 ready 到开始 running 的时间。它高，说明 ready work 等不到 worker，可能是队列竞争、worker 阻塞、窃取失败或线程数不足。`run_ns` 是用户函数执行时间，能暴露任务粒度和长尾。`blocked_worker_count` 和 `external_wait_ns` 能说明 CPU worker 是否被 I/O 或 future 等待吞掉。`critical_path_ns` 给出任务图理论下限；若总耗时远大于关键路径，要看调度和阻塞；若总耗时接近关键路径，要改依赖结构而不是继续加线程。

把第 13 章的 `stage` 接入运行时 Stable filter 接入运行时，不应该让一个父任务顺序调用 count、等待 count、再调用 scan。更好的形状是：每个 count chunk 是独立任务；count 完成时递减 scan 的 `remaining_predecessors`；最后一个 count 发布 scan；scan 生成 layout 后发布 write tasks；所有 write verified 后发布 commit。整个过程没有 worker 需要阻塞等待同一 stage。

Listing 17.35 stable filter 的非阻塞任务图账本

```
count[i]:
  reads input chunk i
  writes CountDone block record
  decrements scan.remaining_predecessors

scan:
  runs when every count[i] is terminal success
  builds layout and assigns output ranges
  publishes write[i] tasks

write[i]:
  rewrites its entire output range for its attempt
  verifies records and checksum
  decrements commit.remaining_predecessors

commit:
  chooses unique verified attempts
  publishes manifest
  marks stage Committed or Failed
```

这条链把算法合同、恢复合同和运行时合同对齐了。Count 失败时，scan 不会被错误发布；write 失败时，commit 不会读取未验证输出；commit 完成时，下游任务只依赖 manifest，而不依赖 worker 的局部变量。运行时需要做的不是理解 filter 公式，而是执行 ready count、状态发布和失败传播。

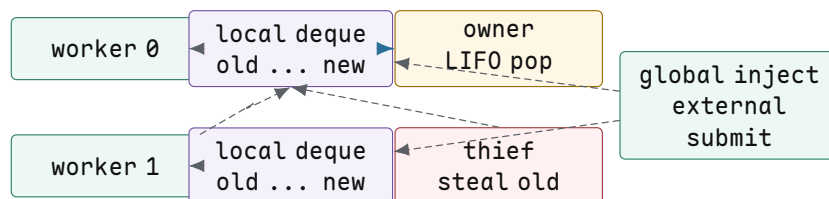
阻塞 I/O 为什么要离开 CPU worker 第 15 章会进入 I/O。这里先建立边界：CPU worker 适合执行短而可抢占的计算任务，不适合在同步文件读、网络请求或外部服务调用上长期停住。若任务必须读文件，常见做法是提交异步 I/O，I/O 完成事件再发布 continuation；或者把同步 I/O 放到 blocking pool，完成后把 CPU 后继任务放回 CPU runtime。把同步 I/O 直接放在 work stealing worker 里，会让 steal 成功率、ready wait 和 park/wake 数据全部失真，因为 worker 不是真的没工作，而是被外部等待扣住了。

17.5 Work Stealing：本地性和负载均衡的折中

全局队列是最好的第一版 reference。所有任务进入同一队列，worker 从同一队列取任务。它容易证明关闭语义，也容易测试异常和未完成计数。但全局队列有两个明显问题：所有 worker 竞争同一把锁或同一组原子位置；本地生成的子任务可能被任意 worker 拿走，破坏缓存局部性。

Work stealing 的思路是每个 worker 拥有本地 deque。本地 worker 高频 push/pop 自己的一端；空闲 worker 低频从其他 worker 的另一端 steal。这样把常见路径留在本地，把少见路径用于负载均衡。这个不对称所有权，是工作窃取的核心。

本地 deque 和窃取方向



本地 LIFO 保留刚生成任务的 cache 热度；空闲 worker 从另一端偷较老任务，通常能拿到更粗的工作块。

图示：本图要证明的命题是：work stealing 的性能来自所有权不对称，而不是来自“多几个队列”这个表象。

owner LIFO, thief FIFO Owner 从本地端按 LIFO 执行新任务，常能保持递归分治或局部数据的 cache 热度。Thief 从另一端偷较老任务，倾向拿到较大的工作块，减少偷取频率。这个策略并不是绝对真理，但它解释了很多经典运行时的设计：常见路径低开销、本地性好；不均衡时再通过窃取把远处工作拉过来。

cache 热度来自时间和地址两个维度。一个 worker 刚处理完父任务，父任务访问的输入块、局部 partial、栈上的小对象和刚生成的子任务元数据，可能还在这个核心的 L1/L2 或共享 LLC 中。owner LIFO 让它立刻执行最新子任务，增加复用概率。thief FIFO 偷较老任务，是为了减少和 owner 争最新 cache 热数据，也更可能拿到粒度较大的上层任务。若任务几乎不共享数据，或者每个任务都只做 I/O 等待，这个策略的优势就会下降。

Chase-Lev deque 的直觉边界 Chase-Lev deque 是经典工作窃取队列。它利用单 owner 的事实简化本地 push/pop，让多个 thief 通过另一个端点竞争 steal。最难的地方通常不是“数组怎么放任务”，而是边界状态：deque 只剩最后一个元素时，owner pop 和 thief steal 可能竞争同一个任务；扩容和节点回收还要保证被偷走的任务不会被 owner 同时释放。

Compute Systems Engine 不应一开始就实现无锁 Chase-Lev。更稳的路线是先实现全局队列 reference，再用 mutex 保护每个 worker deque 验证调度策略，最后在 trace 能证明 deque 同步成为瓶颈时再研究无锁队列。这样能把“调度策略是否有效”和“无锁协议是否正确”分开。

deque 竞争点从 head 和 tail 推出来 工作窃取 deque 的本地路径之所以能快，是因为 owner 是唯一频繁修改 tail 的线程。owner push 把任务写入数组槽位，再发布新的 tail；owner pop 从 tail 端取最近任务。Thief 只从另一端偷，竞争 head。简化状态可以写成：

Listing 17.36 工作窃取 deque 的关键状态

```
array slots:
    task storage, usually a power-of-two circular buffer

head:
    old end, thieves compete here

tail:
    new end, owner mostly owns this cursor
```

当 deque 中有多个任务时，owner pop 和 thief steal 操作不同端点，冲突概率低。真正危险的是只剩一个任务时：owner 从 tail 端 pop，thief 从 head 端 steal，二者可能指向同一个槽位。此时必须用一个线性化点决定任务归谁。常见做法是在最后一个元素竞争时对 head 做 CAS：谁成功推进 head，谁获得任务；失败者必须把状态修正并返回空。

Listing 17.37 最后一个任务竞争的语义伪代码

```
owner pop:
    tail = tail - 1
    if head < tail:
        return slot[tail]
    if head == tail:
        if compare_exchange(head, head, head + 1):
            return slot[tail]
        else:
            restore empty state and return empty

thief steal:
    read head and tail
    if head >= tail:
        return empty
    task = slot[head]
    if compare_exchange(head, head, head + 1):
        return task
    return retry or empty
```

这段伪代码省略了很多细节，却保留了关键事实：无锁 deque 的正确性不是来自“数组加两个下标”，而是来自最后一个任务的 CAS 竞争、发布顺序、槽位生命周期和扩容回收协议。C++ 层面的 `compare_exchange` 在 x86-64 上常落成 `lockcmpxchg` 形态；它会让保存 head 的 cache line 成为 thief 竞争点。正常情况下 steal 频率远低于 owner 本地 pop，所以这条共享线还能接受；若任务太小

导致大量偷取，head 的 CAS 争用会迅速暴露。

发布顺序决定 thief 能否看见任务内容 owner push 不能先发布 tail，再写 slot。否则 thief 可能看到 tail 已经前进，偷到尚未构造完成的任务。正确顺序是先写任务内容，再用 release 语义发布 tail；thief 看到可偷区间后，用 acquire 语义读取并确认任务内容。使用 mutex 的第一版由锁隐含建立 happens-before；无锁版本必须把这条关系写出来。

Listing 17.38 push 发布顺序的抽象

```
slot[index] = fully constructed task descriptor
publish tail with release ordering

thief observes tail/head with acquire ordering
read slot[index] after successful claim
```

任务描述符本身也要有生命周期。若 owner 扩容或回收旧数组，而 thief 正在读取旧 slot，就会出现悬空访问。成熟实现会用受控的数组替换、epoch/hazard 机制、延迟释放或运行时全局 quiescent point。项目第一版使用 mutex deque，并不是逃避底层，而是先避免把调度语义、内存回收和原子协议混在一起。

无锁 deque 的性能报告要看偷取率 如果本地 pop 占绝大多数，deque 的热路径主要是 owner 私有 tail 和本地数组，性能很好；如果任务粒度太小、负载分布很不均或 worker 频繁空转，steal 尝试会增加，head CAS、随机远端访问和失败重试会进入成本。报告不能只写吞吐，还要写 local pops、steal attempts、steal successes、failed CAS、empty steals、平均任务耗时和 worker idle 时间。没有这些字段，就无法判断瓶颈是任务太小、负载不均、deque 协议争用，还是任务本身慢。

窃取成功不一定是好消息 Steal success 高通常说明负载不均被修复，但它也可能说明任务粒度太小、本地队列长期放不住工作、父任务不断制造跨 worker 迁移，或者 NUMA 本地性被牺牲。一个任务被偷走后，它携带的不只是函数指针，还携带输入 split、partial buffer、字典页、错误摘要和后续提交身份。若这些数据仍在 owner 所在 CPU 或 NUMA 节点附近，thief 执行会把第 3、4 章的地址路径拉远。

观测形状	可能解释	下一步实验
idle 高，steal success 低	任务分布不可偷、victim 选择差或 park 太早	增加任务切分、改变 victim 策略、检查 ready 发布
idle 低，steal success 高，p99 升高	窃取减少空闲但破坏本地性或粒度过小	记录 stolen task 数据节点，增大粒度或 NUMA 分组
steal attempts 高，CAS 失败高	head 热点竞争，任务太细或 thief 太多	限制偷取频率、批量偷取、调整 park/backoff
local pops 高，makespan 仍高	关键路径或长任务主导	拆长任务、改 DAG 依赖，不继续调 deque
global inject 队列长期高	外部提交超过本地吸收能力	分片 inject queue、背压或直接投递到目标 worker

因此，work stealing 报告要同时写调度和地址事实：任务从哪个 worker 生成，被谁执行，输入 split 在哪个 NUMA 节点，partial buffer 谁写，执行前 ready 等了多久，执行后是否触发远端合并。只有调度事实没有地址事实，会把远端访存误判成“任务本身慢”；只有地址事实没有调度事实，又会看不见 worker 空闲和窃取失败。

数组扩容和悬空引用边界 固定容量 deque 可以把任务槽位的生命周期绑定到 runtime；可扩容 deque 则要处理数组替换。Owner 可能发现本地 deque 满了，分

配新数组，复制旧槽位，发布新数组指针；同时某个 thief 可能已经读取旧数组指针并准备 steal。若旧数组立即释放，thief 会读悬空内存。这个问题和第 12 章的节点回收同构，只是节点变成了任务数组。

第一版运行时可以主动避开这个复杂度：每个 worker 使用固定容量 deque，满时把任务推入全局 inject 队列，或者向上游返回背压。这样做牺牲一部分峰值性能，却让测试集中在任务图、ready count、失败传播和关闭。等 trace 显示固定容量溢出频繁且成为瓶颈，再评估扩容、分段数组、延迟释放或成熟 deque 实现。

Listing 17.39 第一版 work stealing 的容量合同

```
local deque:
    fixed capacity per worker
    owner push may fail when local deque is full

overflow policy:
    push to global inject queue
    or reject submit with backpressure
    or split less aggressively in the parent task

reclamation:
    task slot storage lives until runtime shutdown
    no old deque array is freed while workers are running
```

批量偷取和公平性 一次偷一个任务的开销在任务很小的时候会很高。批量偷取可以把多个旧任务搬到 thief 本地 deque，减少 head CAS 和 victim 选择成本，也能让 thief 后续执行本地 LIFO。代价是 owner 可能被偷走太多剩余工作，局部性和公平性受影响；若被偷的是同一 NUMA 节点的大量数据，跨节点流量会放大。批量大小应由任务粒度、steal 成功率、idle 时间和 NUMA 证据决定，不应写死成“越大越好”。

Listing 17.40 批量偷取的判读字段

```
steal_batch_size
stolen_tasks_per_batch
stolen_task_run_ns_median
victim_remaining_depth_after_steal
cross_numa_steal_count
owner_idle_after_being_stolen
thief_local_pops_after_batch
```

若批量偷取后 thief 很快消化本地任务、owner 没有长期空闲、cross NUMA steal 不高，说明批量有价值；若 owner 被偷空而 thief 又处理远端数据变慢，批量策略应收窄或按 NUMA 分组。调度策略需要证据驱动，不能靠单个吞吐数字决定。

任务粒度决定调度是否值得 任务太小，提交、入队、出队、唤醒、trace 和 ready count 成本会吃掉计算；任务太大，最后少数长任务拖住整个 stage，work stealing 也救不了已经不可拆的长尾。一个实用经验是：任务数量要明显多于 worker 数，每个任务又要有足够计算量，使调度固定成本只占小比例。

任务粒度还要和 cache、NUMA 和取消检查点一起选择。按 cache 友好的块切分，有利于顺序访问；按过小粒度切分，有利于负载均衡但增加调度成本；长任务如果没有检查点，取消响应会很慢。项目报告应扫描 block size，而不是凭感觉写一个常数。

粒度选择可以从一个账本估算。若一次 task 调度固定成本约为几百纳秒到数

微秒，而任务体只做几十条指令，调度成本会主导；若任务体扫描数 MB 数据，调度成本可以忽略，但长尾和 NUMA 位置会变成主要问题。合理实验应同时输出 task count、平均运行时间、p95/p99 运行时间、ready 等待时间、steal 次数和 worker idle 比例。只看平均耗时，会把“小任务过多”和“大任务长尾”混在一起。

work helping 和 work stealing 不同 Work stealing 发生在 worker 空闲时，它去别人的队列里找 ready 任务。Work helping 发生在 worker 即将等待某个 task group 时，它不睡眠，而是帮助执行能推进该等待条件的 ready 任务。前者解决负载不均，后者缓解 worker 内等待导致的资源浪费。二者可以组合，但语义不同。

Work helping 的风险是重入。等待路径执行了另一个任务，另一个任务可能再等待更多任务，异常传播和栈增长都会复杂。Compute Systems Engine 可以先禁止 worker 内等待，再把 helping 作为扩展。若实现 helping，至少要限制在同一 task group 内，并用 trace 标记帮助执行事件。

work-first 和 help-first 递归分治调度 常讨论 work-first 和 help-first。Work-first 倾向让当前 worker 继续沿着串行执行路径推进，把需要别人帮忙的 continuation 暴露给 thief；help-first 倾向先把子任务交给别人，当前 worker 继续后续逻辑。二者影响关键路径开销、cache 局部性、任务数量和窃取频率。

Compute Systems Engine 不要求复刻 Cilk 理论模型，但必须说明：spawn 后谁继续执行、谁进入 deque，不是无关细节。它决定本地性、关键路径和调度开销。报告中如果只写“使用 work stealing”，没有说明任务生成策略，结论不完整。

NUMA、本地性和公平 在多 NUMA 节点机器上，随机偷取可能把访问某个内存节点的数据搬到远端执行，降低吞吐。更稳的策略是先在同 NUMA 节点内偷取，空闲时间超过阈值后再跨节点偷取。对于小计算任务，跨节点代价可能不明显；对于大数组块、hash table 或 page cache 相关任务，远端访问会成为主要成本。

公平也不是免费。严格公平可能破坏本地性；强优先级可能让低优先级任务长期饥饿。控制面任务，例如取消传播、manifest commit、错误汇总，通常很短但影响关键路径，可能需要高优先级或独立队列。运行时设计是在吞吐、尾延迟、本地性、公平和实现复杂度之间取舍。

17.6 取消、异常、关闭和可见性

可靠运行时的难点往往不在成功路径，而在任务失败、用户取消、关闭过程和结果可见性。一个只在正常输入下跑得快的线程池，不能作为合格实现的终点。真正可用的运行时必须让每个任务进入终态，让每个等待者醒来，让每个错误有归属，让每个输出有提交边界。

取消是协作式状态转换 取消不是强杀线程。强行终止 C++ 线程会破坏 RAII、锁状态、对象不变量和外部资源。运行时通常设置 cancel token，任务在安全点检查，例如处理完一个 chunk、完成一次 I/O、写出一个 block、进入下一轮循环之前。Waiting 或 Ready 任务可以直接标记 Canceled；Running 任务只能请求停止；Succeeded 或 Failed 任务已经是终态，不能再取消。

取消传播要沿任务图发生。若前驱失败，依赖它的后继可能永远不会 ready。运行时必须根据策略把后继取消、跳过、等待重试或转入补偿任务。这个策略不能由每个 worker 临时决定，应由 task graph、stage 或 driver 统一定义。

异常不能停在 worker 线程 用户任务抛异常后，worker 不能静默退出，更不能让未完成任务计数泄漏。示例版至少要捕获逃逸异常、记录 failed、减少未完成计数、唤醒等待者。future 版要把异常保存在 shared state 中，让外部 get() 观察。任务图版还要决定下游如何处理：取消、重试、继续使用部分结果，还是让整个 job 失败。

```

1 enum class TaskErrorKind : std::int32_t {
2     None,
3     UserException,
4     Canceled,
5     RejectedByShutdown,
6     ResourceLimit,
7     InternalRuntimeError,
8 };
9
10 struct TaskErrorSummary {
11     std::uint64_t task_id = 0;
12     std::uint32_t attempt = 0;
13     TaskErrorKind kind = TaskErrorKind::None;
14     std::uint32_t error_code = 0;
15 };

```

错误分类影响上层决策。用户异常可能让 stage 失败；取消是预期控制流；资源不足可能触发背压；关闭拒绝说明提交时机错误；内部运行时错误可能要求停止整个 runtime。把它们都写成 `failed`，driver 就无法做正确恢复。

Drain shutdown 和 cancel shutdown `shutdownrain` 表示停止接收新任务，但已经接纳的任务继续完成。它适合作业正常结束。`shutdowncancel` 表示未开始任务进入取消，运行中任务协作检查 token，等待者收到取消或失败结果。两者不能混成一个 `shutdown()`，否则调用者不知道结果是否应该完成。

关闭期间的 `submit` 要有线性化点。若 `submit` 先观察到 `Open` 并成功入队，任务必须被执行或取消并通知；若 `shutdown` 先把状态改成 `Draining`，后续提交必须失败。偶发“submit 返回成功但任务丢失”的运行时，是关闭协议没有定义清楚。

未完成计数的语义 `wait_idle` 不能只看队列是否为空，因为 worker 可能已经取出任务正在执行。它也不能只看 worker 是否空闲，因为提交者可能正准备入队。更可靠的做法是维护未终态任务计数：提交成功加一，任务进入 `Succeeded`、`Failed` 或 `Canceled` 终态时减一，计数归零时唤醒等待者。异常路径、取消路径和拒绝路径都必须审查计数是否平衡。

内存可见性 使用 mutex 和 condition variable 的第一版，发布关系通常由锁建立：提交者先写完任务对象，再在锁内入队并通知；worker 在锁内取出任务后能看到字段。无锁 ready 队列和工作窃取 deque 会把这个问题暴露出来：生产者必须先构造任务，再用 `release` 语义发布；消费者必须用 `acquire` 语义获取，才能看到任务字段。

结果发布也是同样道理。worker 写入结果后把状态设为完成，等待者读取完成状态后必须能看到结果。标准库 `future/promise` 封装了这些同步；自研 `shared state` 就要自己建立 `happens-before`。运行时不是只管调度，它还管结果何时对其他线程可见。

低开销观测 `Trace` 和 `counters` 是运行时的眼睛，但观测本身不能成为瓶颈。每个任务完成时更新同一个全局原子计数器，在小任务高频场景下会形成共享写热点。更好的做法是 worker 写自己的 `WorkerSnapshot` 或 per-worker buffer，周期性汇总。协议状态必须精确，例如未完成任务计数；实验指标可以延迟汇总，例如平均任务耗时和空闲时间。

观测数据也要分冷热。调度热路径只应写少量固定字段，例如 task id、worker id、start/finish 时间戳和终态；字符串格式化、路径名、错误栈和详细上下文应进入冷路径，或者只在失败时采集。per-worker trace buffer 能把写入限制在本 worker 常用 cache line 上，降低共享原子争用；汇总线程再离线计算关键路径、等待分布和 steal 成功率。若 trace 让所有 worker 抢同一个日志锁，测到的就

是日志系统，而不是运行时。

Listing 17.42 运行时 trace 和 worker 指标的示例形态

```

1 struct TaskStateRecord {
2     std::uint64_t task_id = 0;
3     std::uint32_t attempt = 0;
4     std::uint32_t worker_id = 0;
5     std::uint32_t remaining_dependencies = 0;
6     std::uint64_t submit_ns = 0;
7     std::uint64_t ready_ns = 0;
8     std::uint64_t start_ns = 0;
9     std::uint64_t finish_ns = 0;
10    TaskErrorKind error_kind = TaskErrorKind::None;
11 };
12
13 struct WorkerSnapshot {
14     std::uint32_t worker_id = 0;
15     std::uint64_t executed_tasks = 0;
16     std::uint64_t local_pops = 0;
17     std::uint64_t steal_attempts = 0;
18     std::uint64_t steal_successes = 0;
19     std::uint64_t idle_ns = 0;
20     std::uint64_t running_ns = 0;
21 };

```

这些字段会在分布式章节继续扩展成 task attempt、executor、request id、epoch 和 shuffle block。单机运行时先把身份和状态打稳，后续扩展才不会换一套概念。

17.7 工业界设计案例

工业系统的价值不在于名字，而在于它们在约束下做出的选择。读源码或论文时，报告应回答五个问题：它面对什么负载和失败约束；核心机制是什么；机制购买了什么能力；付出了什么成本；Compute Systems Engine 应该采用、简化还是拒绝哪些部分。

工程案例

Cilk 和 work-first 调度 Cilk 系列运行时把 fork-join 并行的常见路径做得很轻，窃取发生在空闲 worker 上。它的核心原则不是“有一个 deque”，而是把大部分调度开销放到被偷取的少数路径上，让普通串行路径接近原程序执行。这个思想适合递归分治和细粒度并行，但要求任务函数遵守比较清晰的 fork-join 结构。

Compute Systems Engine 可以借鉴它的两个原则：一是本地执行路径要便宜，二是窃取应该暴露较粗工作而不是极小任务。不必一开始实现 Cilk 的完整 continuation 模型，也不必把所有算法都改成递归 spawn。对 stable filter、partition 和 reduce 来说，显式 task table 更利于 trace、取消和恢复。

工程案例

oneTBB 和任务粒度 oneTBB 的工程价值在于把任务划分、worker 调度、work stealing 和可组合并行算法组织在一起。它不会神奇消除任务粒度

问题：`parallel_for` 或 flow graph 的性能仍取决于 range 切分、body 成本、数据局部性和阻塞行为。成熟设计提供机制和默认策略，但无法替业务选择语义。

Compute Systems Engine 借鉴 oneTBB 的方式是：先写 range、blocked range、grain size 和 trace，再决定是否 work stealing。报告要说明任务体是否足够重、是否访问连续内存、是否会阻塞 I/O。若任务只是几十条指令，换成成熟运行时也不会自动变快。

工程案例

OpenMP task 和依赖表达 OpenMP task 让程序员声明任务及其依赖，运行时在依赖满足后调度。这说明工业界也不会把所有并行性都塞进线程数量。依赖表达可以提高可读性和调度自由度，但也会引入运行时开销、调试复杂度和数据依赖声明错误的风险。

Compute Systems Engine 不需要复制 OpenMP 语法，但要学习它的抽象：task 不是立即执行的函数调用，而是带依赖和生命周期的工作记录。对 count、scan、write、commit 来说，依赖边比 worker 内等待更清楚。

工程案例

Folly executor 和执行域 Folly 等工程库强调 executor 抽象：不同任务应进入合适的执行域。CPU-bound 任务、I/O completion、定时器、后台维护、优先级队列，资源特征不同，不能全部塞进一个 FIFO 池。这个思想会在 I/O completion 和背压里继续展开。

Compute Systems Engine 可以先定义 compute domain 和 blocking I/O domain。Compute worker 只执行 ready 的 CPU 任务；I/O 任务或慢等待进入独立执行域；完成事件再把后继任务放回任务图。这样慢磁盘、慢网络或模拟 I/O 不会占满 compute worker。

工程案例

Linux workqueue Linux workqueue 把“稍后执行的工作项”和“执行它的 worker pool”分开，并处理并发限制、阻塞、救援线程和 CPU 亲和性等工程问题。用户态不应照抄内核实现，但可以学习它的状态分离：工作项有身份和状态，worker pool 有资源和策略，阻塞不能让整个系统无法前进。

源码阅读时不要试图一次读完整个 workqueue。围绕一个问题进入：工作如何入队，worker 如何被唤醒，work item 如何避免重复排队，阻塞路径为什么需要救援机制。把这些问题和 task state、ready queue、shutdown、trace 对照，收获会比背函数名更大。

数据库执行器和分布式调度的同构性 DuckDB、ClickHouse、Spark 这类系统表面差异很大，但都在处理类似对象：可执行 fragment、依赖、worker、队列、partition、manifest、失败和重试。数据库 vectorized execution 关注批和局部性；Spark DAG scheduler 关注 stage 依赖和 task attempt；本地运行时关注 worker 和 deque。它们的共同原则是：先把工作身份和依赖写清，再谈调度策略。

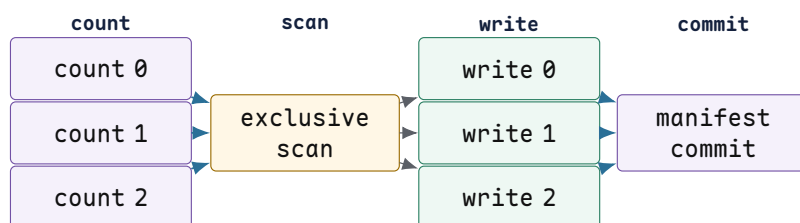
17.8 深度案例：stable filter 如何进入任务运行时

第 13 章已经给出 stable filter 的算法结构：count、scan、write、commit。现在把它放入任务运行时。朴素实现会在阶段之间使用 barrier：提交所有 count，主线程等待；做 scan；再提交 write；最后等待 commit。这个版本简单、可靠，是第一版 reference。但它也暴露问题：barrier 处可能让 worker 空闲；阶段等待

由主线程控制，trace 不足；失败和取消只能粗粒度处理。

任务图版本把每个阶段写成节点和边。所有 count block 初始 ready；最后一个 count 完成后，scan node ready；scan 完成后，所有 write block ready；write 完成后递减 commit 的计数；commit 发布 manifest。worker 不需要阻塞等待下一阶段，运行时能看到每个节点处于 Waiting、Ready、Running 还是终态。

Stable filter 的运行时 DAG



DAG 让运行时知道谁在等谁。失败、取消、重试和 trace 都沿节点身份传播，而不是散落在 barrier 和 worker 调用栈里。

图示：本图要证明的命题是：第 13 章的算法阶段一旦进入工程系统，就应该变成任务图节点和提交边界。

barrier reference 仍然要保留 任务图不是为了否定 barrier。Barrier 版本简单，适合作为 reference 和基线起点。它帮助验证输出、manifest、错误处理和基础性能。任务图版本必须和 barrier reference 在语义上对齐：同样输入、同样谓词、同样稳定顺序、同样 manifest。没有 reference，任务图变快或变慢都不好解释。

失败路径 若某个 count 失败，scan 不应永远 waiting。运行时应把 scan 和后续 write、commit 标记为 canceled 或让 driver 发起重试。若某个 write 失败，commit 不能发布 manifest。若 write 已经产生临时输出，commit 前崩溃恢复时必须能清理未提交文件。这正是任务运行时和第 15 章 I/O 提交协议的连接点。

长尾和 work stealing 如果一个 write block 因输入倾斜变得很慢，work stealing 只能帮助尚未开始或可拆分的工作。已经运行且不可拆的长任务，偷不走。解决方式可能是更细粒度切分、热点 block 特殊处理、动态拆分剩余范围，或在下一轮根据历史统计调整块大小。Work stealing 不是长尾万能药，它需要算法暴露可偷取的剩余工作。

关键路径判读 Stable filter DAG 的关键路径大致是最长 count、scan、最长 write、commit 之和。若 trace 显示端到端时间远高于这个估计，检查 ready 队列等待、worker 空闲、窃取失败和 I/O 等待。若接近关键路径，则优化方向是加速 scan、拆分长 write、批量 commit 或改变算法依赖，而不是继续加 worker。

17.9 实现路径

Compute Systems Engine 的任务运行时不要一次写成完整工业框架。正确路线是小步推进，每一步都有 reference、验证边界和运行证据。

第一步：全局队列 reference 实现固定 worker 数、全局 FIFO 队列、submit、wait_idle、shutdownrain、异常捕获和未完成计数。测试包括：提交 N 个任务全部执行一次；任务抛异常不会杀死 worker；wait_idle 在任务终态后返回；shutdown 后 submit 失败；析构不会遗留 joinable thread；多生产者提交不破坏队列。

第二步：任务图和 trace 加入 task id、ready count、successor 列表、TaskStateRecord 和错误摘要。先用全局 ready 队列执行任务图。用第 13 章 stable filter 的 count、scan、write、commit 作为 workload。测试并发前驱同

时完成时，后继只入队一次；前驱失败时，后继不会永远 waiting；空图和单节点图都能正常终止。

第三步：本地 deque 和工作窃取 每个 worker 增加 mutex deque。外部提交进入 global inject queue；worker 产生的本地后继可以进入自己的 deque；空闲 worker 先取全局，再随机或轮询 steal。记录 local pops、steal attempts、steal successes、empty steals、worker idle time 和 stolen task duration。先验证策略收益，再考虑无锁 deque。

第四步：执行域和背压接口 区分 compute domain 和 blocking 或 I/O domain。CPU 任务不要阻塞等磁盘或网络；I/O 完成事件通过 CompletionRecord 重新进入任务图。队列容量要有返回值，不能无限堆积。这个步骤和第 15 章衔接。

Listing 17.43 项目中 TaskRuntimePlan 的接口草图

```
1 enum class QueuePolicy : std::int32_t {
2     GlobalOnly,
3     LocalDeque,
4     WorkStealing,
5 };
6
7 struct TaskRuntimePlan {
8     std::uint32_t worker_count = 0;
9     QueuePolicy queue_policy = QueuePolicy::GlobalOnly;
10    bool enable_task_graph = false;
11    bool enable_trace = true;
12    bool enable_cancellation = true;
13 };
```

这个结构不是为了把运行时配置化成复杂产品，而是为了让机制能逐项打开。全局队列、任务图、work stealing、trace、cancellation 每个复杂度都要有购买理由。若某机制说不出修复哪个反例，就不应该进入最小实现。

运行时判读矩阵

运行路径	输入形状	要解释的现象
future 饥饿复现	worker 数等于父任务数，父任务等待同池子任务	ready 队列有工作但 worker 被 wait 占住
barrier 对比 DAG	stable filter 的 count、scan、write、commit	worker 空闲、关键路径、失败传播
任务粒度扫描	从很小 chunk 到很大 chunk	调度开销、长尾、cache 局部性
work stealing	倾斜分治任务、热点 block、均匀任务	steal 是否减少空闲，是否破坏本地性
取消传播	上游 count 或 write 注入失败	后继是否进入终态，等待者是否醒来
关闭协议	submit 与 shutdown 并发	submit 返回值、未完成计数、worker join

Trace 摘要命令 项目应提供类似 `enginetracesummarize` 的输出。最低字段包括任务总数、终态分布、最长 ready 等待、最长 running、估计关键路径、worker idle 比例、steal 成功率、取消传播时间和首个错误任务。摘要可以是 key-value 文本，不要求 GUI。关键是能从证据解释运行时，而不是凭感觉判断。

Listing 17.44 任务运行时 trace 摘要示例

```

tasks.total=1536
tasks.succeeded=1536
critical_path_ns=18400000
makespan_ns=23700000
worker.idle_ratio.p95=0.18
steal.attempts=421
steal.successes=73
task.ready_wait_ns.max=2600000
task.running_ns.max=9100000

```

若 makespan 远大于 critical path, 继续查 ready wait、idle ratio、steal 失败和阻塞事件。若 steal attempts 很多但 successes 很少, victim 选择或任务分布有问题。若 success 很多但吞吐不升, 任务可能太小或跨 NUMA 破坏了局部性。

最小综合路径先实现全局队列线程池 reference, 再实现任务图版 stable filter。worker 内禁止调用阻塞 `future.get()` 等待同池任务。运行证据必须能解释 future 饥饿反例、DAG trace、关键路径估计、取消传播和 shutdown 行为。这样一条路径已经覆盖运行时最关键的边界, 不需要额外堆很多分散场景。

源码阅读任务

源码阅读任选一条窄路径, 不要泛读。

1. 阅读 oneTBB 或其他成熟任务运行时的 worker loop、task enqueue、work stealing 触发条件和 shutdown。报告要区分源码直接事实、实验测得事实和推断解释。
2. 阅读 OpenMP runtime 的 task dependency 或 task team 相关路径。报告重点是依赖如何进入 ready 状态, 而不是罗列 API 名称。
3. 阅读 Linux `kernel/workqueue.c` 和 `include/linux/workqueue.h` 中 work item 入队、worker 唤醒和救援机制相关路径。报告要回答“阻塞或排队过多时系统如何避免停住”。
4. 阅读 Folly executor 或类似库的 executor、priority、blocking 和 keep-alive 设计。报告要说明执行域、生命周期和关闭协议。

17.10 运行时实验门禁

一、是否从事故推出机制 所有机制都应能回到 future 饥饿或任务图等待事故。若某段话只是在介绍术语, 不能解释哪个失败现象, 就应该改成具体反例。

二、是否区分等待和 ready Waiting 表示依赖未满足, Ready 表示可以执行。worker 只应长期执行 Ready 任务。若代码让 worker 睡在 Waiting 条件上, 必须有 helping、隔离执行域或明确禁止规则。

三、是否保留 reference 全局队列和 barrier 版本不是落后版本, 而是语义 reference。work stealing 和 DAG 优化必须能和 reference 对齐输出、错误和 manifest。

四、是否有失败路径 只测试成功路径的运行时不合格。至少要测试用户任务异常、上游失败导致后继取消、shutdown 并发 submit、future 饥饿反例和 worker join。

五、是否解释复杂度购买理由 本地 deque 购买局部性和低争用; stealing 购买负载均衡; task graph 购买显式依赖; continuation 购买非阻塞等待; trace 购买诊断能力; cancellation 购买资源收束。若购买理由不成立, 就不应增加机制。

六、是否连接 I/O completion CPU 任务的等待和完成已经被写成状态机。I/O 会把设备等待也写成提交和 completion。TaskStateRecord、CompletionRecord、背压和 manifest 提交边界应自然衔接。

17.11 习题

习题与作业

习题一：构造一个两个 worker 的 future 饥饿复现。写出事件序列、ready 队列状态和 worker 状态，再把它改成 continuation 版本。

习题二：为 RuntimeState 写状态转移表。说明 submit、wait_idle、shutdownrain、shutdowncancel 在每个状态下的行为。

习题三：给 stable filter 的 count、scan、write、commit 画任务图，标出每个节点的 ready count。说明某个 count 失败时后继如何进入终态。

习题四：设计一个 work stealing 实验。先用 mutex deque，不使用无锁 deque。比较全局队列、本地 deque 无窃取、本地 deque 加窃取三种版本。

习题五：解释 owner LIFO、thief FIFO 的本地性理由，并构造一个它表现不好的反例。

习题六：设计 TaskStateRecord 和 WorkerSnapshot 的低开销采集方案。说明哪些字段必须精确，哪些字段可以采样或延迟汇总。

习题七：阅读一个工业运行时的源码片段，按“约束、核心机制、购买能力、成本、迁移到本项目的方式”写一页报告。

工程案例

知识点小案例：future 等子任务会饿死同一个线程池。特例是 4 个 worker 各自执行一个父任务，父任务提交一个子任务后同步 future.get()。此时所有 worker 都在等，ready queue 里有 4 个子任务却没人执行。这不是锁顺序死锁，而是运行时调度协议错误。由此推出规律：任务运行时不能让 worker 在持有执行资源时盲等同池子任务；应使用 continuation、helping 或结构化 join。工程上要输出 task state、ready count、worker state 和 wait reason，否则“线程池卡住”无法区分死锁、饥饿和背压。

第 18 章

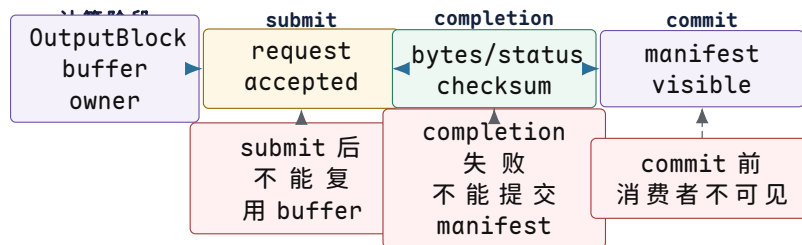
I/O、异步与背压

先看一个写出队列事故。解析和计算阶段很快，reduce worker 很快生成大量 `OutputBlock`；磁盘偶尔变慢，写出线程跟不上。第一版用无界队列，短时间 benchmark 很漂亮，因为计算线程从不等待；几分钟后内存持续上涨，尾延迟越来越长，最终进程被系统杀掉。第二版改成异步写，`submit_write` 返回成功后，上游立刻复用 buffer；几毫秒后 I/O 线程才真正写文件，落盘内容偶尔变成下一批数据。第三版又修了 buffer 生命周期，却把 `submit` 成功当作业务成功，manifest 过早发布，短写和失败 completion 被日志吞掉。

这三个事故逼出 I/O 章节的主线：I/O 不是“换一个系统调用”这么简单，而是必须分清 **提交** (submit)、**完成** (completion)、**提交业务结果** (commit)。提交只表示请求被接纳；完成才说明底层读写有结果；业务提交才说明外部消费者可以相信这个输出。异步把等待从调用线程上移走，同时也把所有权、短读短写、错误、取消、背压、可靠写入和崩溃恢复全部暴露出来。

后面的结构都围绕同一条因果链展开：阻塞调用为什么占住 worker；非阻塞 ready 为什么不等于消息完整；异步 completion 为什么要有请求身份；buffer 为什么要有唯一所有者；队列为什么要按 item 和 byte 双重限流；可靠写入为什么要把临时文件、checksum、fsync、rename 和 manifest 分层处理；这些对象之后会扩展成分布式 reply、shuffle block、checkpoint 和流控窗口。

提交、完成和业务提交不是同一件事



异步 I/O 的正确性边界不是函数返回，而是请求状态机：
accepted、in-flight、completed、committed 必须分开记录。

图示：本图要证明的命题是：异步化后，提交路径、完成路径和业务提交路径必须由同一个请求身份串起来。

18.1 从阻塞 I/O 到完成事件

阻塞 I/O 的语义最接近普通函数调用：调用线程发起 `read` 或 `write`，线程等待，返回值告诉本次调用处理了多少字节或出了什么错误。它适合配置读取、低并发工具、简单批处理和语义优先的 reference。它的问题是等待设备时占住线程。若计算 worker 在 `fsync`、网络发送或慢磁盘上等待，任务运行时再精巧也无法让这些 worker 执行 ready 的 CPU 任务。

异步 I/O 的目标不是让设备变快，而是把等待从执行线程上移走。提交请求后，CPU 可以继续处理其他工作；完成事件稍后返回，告诉系统这次请求成功、失败、短写、取消还是超时。这个分离提升并发度，也引入新的状态机。任何异步设计都要先回答：请求提交后谁拥有 buffer，completion 由谁消费，失败如何传播，取消后迟到 completion 怎么处理。

文件 I/O 的第一层对象：path、fd、open file description、inode 文件

I/O 不能只记住 `read(fd,...)` 和 `write(fd,...)`。应用传入的是 `path`，内核打开后返回 `file descriptor`，`fd` 指向一个 `open file description`，`open file description` 再引用文件对象、当前 `offset`、打开标志和访问模式，底层文件系统对象通常可理解为 `inode` 或类似结构。多个 `fd` 可能指向同一个 `open file description`，也可能分别打开同一个 `path` 得到不同 `description`。

Listing 18.1 文件 I/O 对象关系

```
path string
-> pathname lookup
-> inode or filesystem object
-> open file description: flags, current offset, status
-> file descriptor number in this process
```

这条链解释了很多并发 I/O bug。复制 `fd` 的 `dup`、`fork` 继承或把同一个 `fd` 传给多个线程，可能共享同一个 `current offset`；两个线程同时用 `read` 读取同一个 `fd`，读到的位置由共享 `offset` 推进顺序决定。若每个线程单独 `open` 同一个文件，通常得到各自的 `open file description`，`offset` 不共享。若要并行读取固定区间，应优先使用带显式 `offset` 的 `pread`，避免把文件位置当成隐藏共享状态。

操作	offset 行为	并发后果
<code>read(fd)</code>	使用并推进 <code>open file description</code> 的 <code>current offset</code>	多线程共享 <code>fd</code> 时顺序变成隐藏状态
<code>pread(fd, off)</code>	使用显式 <code>offset</code> ，不改变 <code>current offset</code>	更适合分片并行读取
<code>dup(fd)</code>	新 <code>fd</code> 指向同一 <code>open file description</code>	<code>offset</code> 和状态仍共享
<code>open(path)</code> 两次	通常得到两个 <code>description</code>	<code>offset</code> 通常分离，底层文件相同
<code>O_APPEND</code> 写	每次写追加到文件末尾	避免手动 <code>seek</code> 竞争，但仍要处理短写和错误

报告里如果只写“多线程读同一个文件”，信息不够。要写清：是共享一个 `fd`，还是每线程独立 `open`；是 `read`，还是 `pread`；是否使用 `O_APPEND`；是否有 `dup/fork`；是否把 `fd` 交给了库函数。否则很多“随机错位”“重复读取”“漏读”的 bug 会被误判成 `parser` 问题。

基础闭环：fd、VFS、page cache 和业务字节 这一章后面会讲异步、`completion` 和背压，但在那之前，必须先把同步文件 I/O 的基础闭环钉住。应用代码里的 `intfd` 只是一个进程局部编号；编号通过进程 `fd table` 找到 `open file description`；`description` 再指向 `VFS` 文件对象；`VFS` 根据底层文件系统、设备和打开标志决定读写路径。业务看到的是“读写字节”，内核维护的是 `offset`、权限、引用计数、`page cache`、`dirty page`、`writeback` 和错误延迟。

Listing 18.2 一次普通文件 I/O 的最小对象链

```
process-local fd table
fd 3
-> open file description
  flags, current offset, file status, refcount
-> VFS file object
  file operations: read_iter, write_iter, fsync, mmap
-> filesystem object: inode, address_space, metadata
  -> page cache pages or direct I/O mapping
  -> block layer and device
```

这条链把几个基础问题连在一起。第一，fd 的生命周期和底层文件对象不同：close(fd) 只是释放当前进程里的一个引用，别的 fd、子进程或库对象仍可能持有同一个 open file description。第二，read/write 的 current offset 属于 open file description，不属于 fd 号码本身。第三，普通文件的 buffered I/O 会经过 page cache，write 成功经常只说明字节进入内核缓存并变脏，不说明设备完成。第四，direct I/O 试图绕过 page cache 的数据路径，但不会绕过权限检查、VFS、文件系统、错误处理和持久化协议。

对象或状态	它保存什么	常见误判
fd number	当前进程 fd table 的索引	以为它就是文件本身
open file description	offset、status flags、引用计数	以为 dup 后 offset 分离
inode/address space	文件身份、metadata、page cache 映射	以为路径字符串决定所有状态
page cache page	文件页的内存副本和 dirty 状态	以为读写一定碰设备
block request	设备队列里的真实 I/O 请求	以为 write 返回就已经有 request 完成

最小闭环可以按状态机记住：

Listing 18.3 fd 和普通文件数据路径状态机

```
Closed:
  no process-local handle exists

Opened:
  fd table entry references an open file description

BufferedReader:
  read checks page cache; cache miss may allocate page and wait for device

BufferedWriter:
  write copies user bytes into page cache and marks pages dirty

WritebackPending:
  dirty pages wait for background writeback or fsync

DurabilityWait:
  fsync/fdatasync waits for filesystem/device durability boundary

ClosedOrReleased:
  close drops this reference; delayed write error may be reported
```

读路径和写路径都要按这个状态机报告。读路径要写清是 cold page cache 还是 warm page cache，是 read 复制到用户 buffer，还是 mmap 后由 page fault 填页。写路径要写清是否 buffered，是否调用 fsync/fdatasync，是否把 close 返回值纳入错误处理，是否有 manifest 或 rename 作为业务提交边界。没有这些字段，I/O benchmark 的数字很容易把内存 copy、缺页、设备吞吐和持久化等待混在一起。

open、read、write、close 的最小合同 open 的返回值不是文件内容，而是进程 fd 表中的一个小整数句柄。它可能失败，失败原因包括路径不存在、权限不足、

fd 数量耗尽、目录不存在、文件系统只读、路径太长等。成功后，fd 必须在所有路径上关闭；但 `close` 本身也可能返回错误，尤其是写出路径中，某些错误可能延迟到 `close` 或 `fsync` 才暴露。写可靠输出时，不能只检查 `write` 返回值而忽略 `fsync/close`。

Listing 18.4 同步文件读取的基础合同，不是高性能版本

```
1 int fd = ::open(path, O_RDONLY | O_CLOEXEC);
2 if (fd < 0) {
3     return error_from_errno(errno);
4 }
5
6 std::uint64_t done = 0;
7 while (done < size) {
8     ssize_t n = ::read(fd, buffer + done, size - done);
9     if (n > 0) {
10         done += static_cast<std::uint64_t>(n);
11         continue;
12     }
13     if (n == 0) {
14         break; // EOF
15     }
16     if (errno == EINTR) {
17         continue;
18     }
19     return error_from_errno(errno);
20 }
21
22 if (::close(fd) != 0) {
23     return error_from_errno(errno);
24 }
```

这段代码故意保留循环，因为短读是正常路径。它也区分 `n==0` 的 EOF、EINTR 的可重试中断、和真实错误。写路径同理：`write` 返回正数但小于请求长度时必须继续写；返回 `-1` 且 `errno==EINTR` 可重试；非阻塞 fd 的 `EAGAIN` 表示当前不能推进，应该进入等待机制，而不是当成 fatal error。

stdio 缓冲和系统调用不是同一层 `FILE*`、`fread`、`fprintf`、`std::ofstream` 这类接口通常在用户态有缓冲。用户调用 `fprintf` 成功，可能只是写入了 libc 或 C++ `iostream` 缓冲；真正的 `write` 系统调用稍后才发生。`fflush` 把用户态缓冲推给内核，不等于数据持久化；`fsync` 针对 fd 的内核侧脏数据和 metadata 才讨论持久边界。混用 `FILE*` 缓冲和底层 fd 系统调用时，如果不 `fflush` 或不理解 offset，同一文件的读写顺序很容易错乱。

层次	成功说明	不说明什么
<code>fprintf/ofstream</code>	用户态库接受了数据	不说明已进入内核 page cache
<code>fflush</code>	用户态缓冲尝试写给内核	不说明已经落盘
<code>write</code>	内核接受了部分或全部字节	不说明 durable
<code>fsync</code>	推进文件持久化边界	不说明业务 manifest 已提交
<code>close</code>	释放 fd，可能报告延迟错误	不等同于完整恢复协议

工程建议很直接：热路径和可靠写路径尽量不要混用多层 I/O 抽象。同一个 fd 上如果既用 `FILE*` 又用 `read/write/lseek`，必须明确缓冲刷新、offset 和错误传

播。性能报告也要写清使用的是 raw syscall、stdio、iostream、mmap 还是库封装；否则“文件 I/O 慢”这个结论不可比较。

文件 offset 是隐藏共享状态 线程安全不等于语义正确。两个线程同时调用 `read(fd)`，内核可以让每次读取按某种顺序推进共享 offset，数据竞争未必发生在 C++ 对象上，但业务分片可能已经错了。更隐蔽的是一个线程 `lseek(fd, off)`，另一个线程随后 `read(fd)`；如果中间有第三个线程改变 offset，读取位置就不是调用者以为的位置。并行扫描文件时，显式 offset 比共享 offset 更可靠。

Listing 18.5 共享 offset 导致分片语义不稳定

```
thread 0: lseek(fd, 0)
thread 1: lseek(fd, 4096)
thread 0: read(fd, buf0, 4096) // may read from 4096, not 0
thread 1: read(fd, buf1, 4096) // offset already advanced
```

正确的并行读设计通常让分片计划先产生 `(offset, length)`，worker 使用 `pread` 或独立 fd 读取自己的范围。若文件是日志或变长记录，还要处理边界：每个 split 从某个 offset 附近开始，但第一条完整记录可能要向前或向后找换行。I/O 基础和第 13 章 partition 的联系就在这里：分片不仅是计算问题，也是文件位置和记录边界问题。

阻塞、非阻塞和异步 阻塞 I/O 调用可能让当前线程睡眠，直到有结果。非阻塞 I/O 调用立即返回，若当前不能读写，返回 `EAGAIN` 或类似状态；应用需要等待 ready 事件后再试。异步 I/O 提交请求，完成队列稍后返回结果。三者不是“低级到高级”的线性关系，而是控制流和资源模型不同。

epoll 这类 ready 机制只说明 fd 可能可读或可写，不说明业务消息已经完整。Socket 可读可能只有半个 header；可写也不保证整个响应一次写完。`io_uring` 更接近提交队列和完成队列模型，但仍然要处理 buffer 生命周期、短读短写、错误码、取消和完成顺序。必须避免把具体 API 神化，核心是状态机。

eventfd：把用户态状态变化变成 fd 可读 `eventfd` 常被用来唤醒 `epoll` 线程。它的语义不是传输业务消息，而是维护一个内核计数器：写入一个 64 位值会增加计数；读取会取走计数并让它回到零，或者在 semaphore 模式下一次读走一个单位。于是 `eventfd` 适合表达“有事情需要 event loop 回来检查”，不适合把每个任务 payload 塞进 fd。

Listing 18.6 eventfd 唤醒的正确分层

```
producer thread:
  push request into user-space queue
  if queue was previously empty:
    write(eventfd, 1) // wake event loop

event loop:
  epoll_wait returns eventfd readable
  read(eventfd) // drain wake counter
  pop all available requests from user-space queue
  process bounded amount of control work
```

这条链里，业务事实在用户态队列中；`eventfd` 只是把“队列状态可能变了”转成 fd readiness。若生产者每 push 一个请求都无条件写 `eventfd`，event loop 可能收到大量重复唤醒；若 event loop 只读 `eventfd` 却不 drain 用户态队列，计数器清零后仍有工作积压；若把 request id 当作 `eventfd` 计数写进去，多个写会被合并，身份信息会丢失。

对象	保存什么状态	常见误用
用户态队列	request、buffer、attempt、deadline	不加容量，无法背压
eventfd 计数器	至少一次唤醒事实或计数	当消息队列或 payload 通道
epoll interest list	关注哪些 fd 的 ready 事件	当 completion queue
业务状态表	request 是否完成、取消、提交	只靠 fd ready 推导业务完成

epoll: ready 事件不是 completion epoll 管的是 fd readiness。它告诉事件循环“这个 fd 现在做某类 I/O 操作可能推进”，不是“某个业务请求完成”。TCP socket readable 可能只够读半个 header；writable 可能只够写响应的一部分；eventfd readable 只说明计数器非零；管道 readable 只说明至少有字节可读。应用必须在 fd ready 后推进自己的状态机，直到遇到 **EAGAIN**、完成一个业务消息、达到本轮预算或进入错误路径。

Listing 18.7 epoll reactor 的基本状态循环

```

event loop:
  events = epoll_wait(epoll_fd)
  for each event:
    if event.fd is listen socket:
      accept until EAGAIN
    if event.fd is data socket and readable:
      read into connection buffer until EAGAIN or budget exhausted
      parse zero or more complete messages
      enqueue CPU work to compute executor
    if event.fd is data socket and writable:
      flush output buffer until EAGAIN or buffer empty
    if event.fd is eventfd:
      read eventfd counter
      drain user-space control queue

```

这个循环里有两个预算。I/O 预算防止单个热 fd 独占 event loop；CPU 预算防止事件回调直接变成业务执行线程。解析出完整消息后，重 CPU 工作应该作为 task 进入 compute executor；event loop 只保留连接状态、buffer offset、订阅的读写兴趣和必要的轻量协议推进。

水平触发、边沿触发和 drain 规则 水平触发下，只要 fd 仍 ready，**epoll_wait** 之后还可能继续报告它。边沿触发下，内核只在 ready 状态变化时提示；应用若没有一直读写到 **EAGAIN**，可能把剩余数据留在 fd 里，却再也等不到下一次边沿通知。边沿触发常见于高性能 reactor，但它购买的是更严格的 drain 责任，不是自动更快。

Listing 18.8 边沿触发必须 drain 到 EAGAIN

```

on EPOLLIN for edge-triggered socket:
  loop:
    n = read(fd, buffer)
    if n > 0:
      append to connection input buffer
      parse complete messages
      continue
    if n == -1 and errno == EAGAIN:
      stop reading; wait for next edge

```

```
if n == 0:
    peer closed
else:
    handle error
```

这条规则同样适用于 eventfd: event loop 看到 eventfd readable 后应读取计数器，并检查对应用户态队列。若多个 producer 写入 eventfd，读到的计数可能是合并值；它只能告诉你“至少发生过这些唤醒写”，不能替代队列长度。真正的 drain 条件是用户态队列空、连接 buffer 没有完整消息、输出 buffer 暂时写不动，或者本轮处理预算耗尽。

eventfd/epoll 和条件变量的相似点 eventfd/epoll 和 condition variable 看起来一个是 fd，一个是线程同步原语，但它们共享同一条原则：通知不是状态。条件变量的通知要求等待者回到锁内检查谓词；epoll 的 ready 要求 event loop 回到 fd 和用户态连接状态检查进度；eventfd 的可读要求 event loop 回到用户态队列检查真实工作。把通知当事实，是两类系统都会犯的错误。

机制	通知含义	必须重新检查的状态
condition variable	谓词可能变化	mutex 保护的队列/关闭/error
eventfd	有线程写过唤醒计数	用户态请求队列和 shutdown flag
epoll readable	fd 可能可读	connection buffer 和 message framing
epoll writable	fd 可能可写	output buffer、short write offset、backpressure
io completion	请求返回 bytes/status	request record、buffer owner、commit state

event loop 唤醒报告字段 如果一个 I/O runtime 只记录 `epoll_wait` 次数，很难解释延迟。报告应把 eventfd、epoll、用户态队列和业务 completion 对齐。最小字段如下：

Listing 18.9 eventfd/epoll runtime 的观测字段

```
epoll_wait_count
epoll_wait_timeout_count
epoll_events_returned
eventfd_write_count
eventfd_read_count
eventfd_counter_sum
control_queue_depth_max
control_queue_drain_items
socket_read_ready_count
socket_write_ready_count
read_until_eagain_count
write_until_eagain_count
messages_parsed_count
cpu_tasks_enqueued_from_event_loop
event_loop_callback_ns_p99
```

这些字段能区分几种不同瓶颈: eventfd 写很多但队列很浅，说明唤醒合并策略差；epoll 返回很多 writable 但写不完，说明下游慢或输出 buffer 过大；read ready 很多但 messages parsed 少，说明消息分片或 framing 成本高；callback p99 很高，说明 event loop 执行了不该由它执行的 CPU 工作。

eventfd 的读写合同 eventfd 的基础合同很小，但细节足以制造 bug。创建时通常选择 `EFD_NONBLOCK` 和 `EFD_CLOEXEC`：前者避免 event loop 或生产者在 fd 操作上意外阻塞，后者避免子进程继承控制 fd。普通模式下，读一次返回当前计数并把计数清零；semaphore 模式下，读一次返回 1 并把计数减 1。写入会增加计数，但计数有上限；非阻塞写在无法增加时返回 `EAGAIN`。

Listing 18.10 eventfd read/write 的基本语义

```
create:
    fd = eventfd(0, EFD_NONBLOCK | EFD_CLOEXEC)

write side:
    write uint64 value > 0
    counter += value
    fd becomes readable
    nonblocking write may fail with EAGAIN if counter would overflow

read side, normal mode:
    if counter > 0:
        read returns counter
        counter = 0
    else if nonblocking:
        read fails with EAGAIN

read side, semaphore mode:
    if counter > 0:
        read returns 1
        counter -= 1
```

因此 eventfd wake 的正确用法通常是“合并唤醒”。用户态队列保存真实工作；eventfd 只负责把 event loop 从 `epoll_wait` 里拉出来。若需要每个请求都有身份，就在队列元素里保存 request id；若需要每个完成都有状态，就用 CompletionRecord；不要把 eventfd counter 当 request id、字节数或完成次数的权威来源。

实测证据：eventfd 合并计数和 semaphore 模式 下面的最小实验应在支持 eventfd 和 epoll 的 x86-64 Linux 环境中运行。报告只记录可公开复现的环境字段，例如内核版本、C 库版本、编译器版本和是否能使用 PMU；不要把某台私有机的名字、路径或账号写进书稿。这个实验只验证用户可见的 eventfd 读写合同和 epoll readiness 行为，不声称分析内核内部实现。

Listing 18.11 eventfd/epoll 合同实验命令

```
cmake -S books/cpu-volume-2 -B books/cpu-volume-2/build/compsys-debug \
    -DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-2/build/compsys-debug
books/cpu-volume-2/build/compsys-debug/labs/compute_systems/\
    compsys_wait_channels_probe
```

Listing 18.12 eventfd 读写实验输出摘要

```
contract,2,1,1,2,2,3,3,1,1
```

这行输出按顺序记录：两次 `eventfd` 写入、普通模式两次 `counter` 读取、两次 `epoll ready` 观察、两次空轮询观察、三条用户态 `completion` 被 `drain`、`semaphore` 模式读出三次 `1`，以及 `FIFO` 和空读合同都成立。普通模式证明 `eventfd counter` 可以合并唤醒；`semaphore` 模式证明每次 `read` 只消费一个计数单位。这就是为什么 `eventfd` 适合作为“有工作了”的唤醒 `fd`，而不适合作为工作身份或完成记录本身。

`eventfd` 的常见竞态：队列先还是写 `fd` 先 生产者必须先发布用户态队列中的工作，再写 `eventfd`。若顺序反过来，`event loop` 可能醒来、读掉 `eventfd` 计数、检查队列为空，然后回到 `epoll_wait`；生产者随后才把工作放入队列，这个工作就可能一直等到下一次唤醒。和 `condition variable` 一样，通知必须跟在状态变化之后。

Listing 18.13 `eventfd` 唤醒顺序的对错

```
wrong:
    write(eventfd, 1)
    push request into queue

right:
    push request into queue with release semantics or under queue mutex
    write(eventfd, 1)

event loop:
    read(eventfd)
    drain queue with acquire semantics or under queue mutex
```

如果队列本身用 `mutex` 保护，`push` 和 `drain` 通过锁建立可见性；如果用无锁 `MPSC` 队列，发布节点和消费节点必须有 `acquire/release` 关系。`eventfd` 不替队列建立内存序，它只让等待线程变成 `runnable` 或让 `epoll` 返回。

`epoll` 的 `interest`、`ready` 和 `user data` `epoll` 有三层状态。第一层是 `interest list`：应用通过 `epoll_ctl` 注册 `fd` 和关注的事件掩码，例如 `readable`、`writable`、`hangup`、`edge-triggered`、`one-shot`。第二层是内核观察到的 `ready` 状态。第三层是应用自己的 `connection/request` 状态。事件返回时携带的 `data` 字段应该能定位稳定的连接对象或 `id`，而不是指向可能已经释放的栈对象。

Listing 18.14 `epoll fd` 注册要能回到稳定状态对象

```
connection table:
    conn_id -> ConnectionState {
        fd
        input_buffer
        output_buffer
        parse_state
        subscribed_events
        generation
    }

epoll_ctl add:
    event.events = EPOLLIN | EPOLLET
    event.data.u64 = pack(conn_id, generation)

event loop:
```

```

unpack conn_id and generation
find ConnectionState
reject stale event if generation changed
advance fd and connection state

```

generation 很重要。fd number 会被内核复用；连接关闭后，同一个整数 fd 可能很快属于另一条连接。若 event loop 中还有旧事件或业务队列里还有旧任务，裸 fd 可能把旧事件投给新连接。稳定的 id/generation 句柄能把迟到事件识别为 stale。

EPOLLIN、EPOLLOUT、HUP 和 ERR 的边界 readable 和 writable 不是唯一事件。对端关闭、错误、半关闭、连接失败都可能通过 hangup 或 error 事件进入循环。健壮的 reactor 不应只在 EPOLLIN 时读、只在 EPOLLOUT 时写；它要在每次事件后检查错误状态，并让 ConnectionState 进入 Closing、Draining 或 Failed。

Listing 18.15 epoll 事件处理的边界检查

```

for event in epoll_wait results:
    if event has EPOLLERR:
        read socket error with getsockopt(SO_ERROR)
        mark connection Failed
    if event has EPOLLHUP or EPOLLRDHUP:
        mark peer close or half-close
        drain remaining readable bytes if protocol allows
    if event has EPOLLIN:
        read until EAGAIN or budget
    if event has EPOLLOUT:
        write until EAGAIN or output empty
    if output empty:
        remove EPOLLOUT interest to avoid writable wake storms

```

可写事件尤其容易误用。大多数 socket 大部分时间都 writable；若一直订阅 EPOLLOUT，event loop 会被“可以写但没有东西写”的事件淹没。常见策略是：只有 output buffer 非空且上次写到 EAGAIN 后，才订阅 writable；buffer 清空后取消 writable interest。

实测证据：epoll readiness、one-shot 和半关闭 同一个实验还验证了 epoll 的几条用户可见边界：

Listing 18.16 epoll 合同实验输出摘要

```

case=epoll_ctl_eventfd_add rc=0
case=epoll_eventfd_ready epoll_n=1 errno=OK events=0x1 data=0xfeed0011
case=epoll_eventfd_drain read_n=8 errno=OK value=1
case=epoll_eventfd_after_drain epoll_n=0 errno=OK events=0x0 data=0x0
case=epoll_ctl_pipe_oneshot_add rc=0
case=oneshot_first_ready epoll_n=1 errno=OK events=0x1 data=0xabc00001
case=oneshot_without_rearm epoll_n=0 errno=OK events=0x0 data=0x0
case=epoll_ctl_pipe_oneshot_rearm rc=0
case=oneshot_after_rearm epoll_n=1 errno=OK events=0x1 data=0xabc00002
case=oneshot_drain read_n=2 errno=OK
case=epoll_ctl_socket_add rc=0
case=socket_peer_half_close epoll_n=1 errno=OK events=0x2001 data=0xbeef0001

```


第一段证明 `eventfd` 进入 `readable` 后会令 `epoll_wait` 返回 `EPOLLIN`，读掉 `counter` 后再次等待返回 `0`。第二段证明 `EPOLLONESHOT` 触发一次后会暂时禁用该 `fd`：即使管道又写入了字节，不 `rearm` 的 `epoll_wait` 仍返回 `0`；执行 `EPOLL_CTL_MOD` 重新 `armed` 后，同一个 `fd` 再次返回 `ready`。第三段用 `socketpair` 和 `shutdown(SHUT_WR)` 观察半关闭，事件位 `0x2001` 表示 `EPOLLRDHUP|EPOLLIN`。这说明 `reactor` 不能只处理读写，还要把 `half-close`、`hangup` 和 `error` 统一纳入连接状态机。

若实验环境不能采集 `cycles, instructions`，报告必须显式降级，例如：

Listing 18.17 `perf` 计数器边界

```
Error:
No supported events found.
The cycles:u event is not supported.
```

因此这类结果只支持 I/O 合同和 `readiness` 状态分析，不能支持 `event loop` 开销、IPC 或 `cache` 行为结论。若要讨论性能，需要另补可用 `PMU`、调度 `trace`、`callback p99` 和队列水位。

EPOLLONESHOT 和跨线程处理 如果多个线程同时消费同一个 `epoll fd`，或者 `event loop` 把 `fd` 事件交给 `worker` 处理，就要防止同一个 `fd` 的事件被并发处理。一个常见模式是 `EPOLLONESHOT`：事件触发后，内核暂时禁用这个 `fd` 的后续通知，应用处理完并更新 `ConnectionState` 后，再用 `epoll_ctlMOD` 重新 `armed`。它购买的是“同一连接同一时间只有一个处理者”的边界，代价是每次处理后都要正确 `rearm`。

Listing 18.18 `one-shot reactor` 的 `rearm` 协议

```
on event:
  mark connection Handling
  process bounded read/write work
  compute next interest mask from ConnectionState
  if connection still open:
    epoll_ctl MOD fd with EPOLLONESHOT | next mask
    mark connection Armed
  else:
    epoll_ctl DEL fd
    close fd after no owner remains
```

若忘记 `rearm`，连接会永久沉默；若在处理尚未结束时 `rearm`，另一个线程可能并发处理同一 `ConnectionState`；若 `close` 后没有删除或识别 `generation`，迟到事件可能打到新连接。`epoll` 编程的难点不在 API 名字，而在 `fd` 生命周期、事件订阅和用户态状态三者必须一致。

从 `epoll` 到任务运行时的边界 `event loop` 不应该直接执行重 CPU work。更稳的结构是：`event loop` 只推进 `fd` 状态和消息 `framing`；完整消息变成 `MessageEnvelope` 或 `TaskInput`；`compute executor` 处理 CPU 逻辑；结果再以 `OutputBuffer` 或 `WriteRequest` 回到 I/O 层。这样可观测性也清楚：`ready` 延迟、`parse` 延迟、`compute` 排队、`write backpressure` 是不同阶段。

Listing 18.19 `reactor` 和 `compute executor` 的分层

```
event loop:
  fd readiness -> ConnectionState progress
  complete message -> enqueue TaskInput
```

```

task output available -> enqueue OutputBuffer and arm EPOLLOUT

compute executor:
  TaskInput -> CPU work -> TaskOutput
  never blocks in epoll_wait
  never owns fd readiness state

I/O state:
  owns fd, input buffer, output buffer, parse offset
  owns event subscription mask
  owns connection close/drain/fail state

```

这条边界和第 14 章 scheduler/awaiter 边界一致：ready 事件只是把某个状态推进了一步，不能直接变成任意用户代码。否则 event loop 的 tail latency 会污染所有连接，系统看起来像 I/O 慢，实际是事件线程被 CPU continuation 占住。

异步 I/O 和 coroutine: completion 才能恢复 awaiter 把 I/O 接到 coroutine 时，关键不是把 read 包在 co_await 后面，而是让 awaiter 等真正的 I/O completion。若 await_suspend 里调用阻塞 read，这个 coroutine 只是换了一种写法，compute worker 仍被设备等待占住。正确模型是：提交请求，保存 buffer 所有权和 coroutine handle，worker 返回调度器；completion 到达后，I/O 层更新请求状态，再把 handle 入恢复 executor。

Listing 18.20 异步 I/O awaiter 的事件链

```

co_await async_read(request):
  await_ready:
    return request state is already completed

  await_suspend(handle h):
    allocate or find IoRequestState
    record buffer_id, offset, remaining, attempt
    record h and resume_executor
    submit request to I/O backend
    return to scheduler without blocking worker

I/O completion loop:
  receive CompletionRecord
  update bytes_done, status, error_code
  if request needs more bytes:
    resubmit next slice or enqueue retry
  else:
    mark request terminal
    detach coroutine handle
    enqueue handle to resume_executor

await_resume:
  return completed byte count
  or throw/read error/canceled according to request state

```

这条链把第 14 章的 awaiter 合同和本章的 CompletionRecord 接起来。awaiter 的 observed state 是 IoRequestState；completion 来源是 I/O backend；恢复动作是向 executor 投递 CoroutineResumeItem。其中任何一步

直接执行用户代码，都会把 I/O completion loop 变成业务执行线程，背压和尾延迟都会失真。

ready、completion 和 await resume 的区别 非阻塞 fd 的 ready、异步 I/O 的 completion、coroutine 的 resume 是三件事。Ready 表示“现在重试系统调用可能推进”；completion 表示“某个已经提交的请求有结果”；resume 表示“用户态 coroutine 可以继续执行”。把三者混起来，会导致半包、短写、重复提交和错执行域恢复。

事件	它证明什么	不证明什么
epoll readable	fd 可能读到一些字节	不证明完整消息或请求完成
epoll writable	fd 可能接受一些字节	不证明整个 buffer 已发送
io completion	某个 request 有 bytes/status	不证明业务 manifest 可见
awaiter ready	await_resume 可立即完成	不证明一定 inline 恢复安全
coroutine resume	executor 开始执行 frame	不证明 I/O 层还有 buffer 所有权
manifest commit	业务结果可见	不证明底层临时文件都已清理

因此，I/O coroutine 的 trace 至少要记录三组时间：request submit 到 completion 的 I/O 时间，completion 到 coroutine resume 的调度时间，resume 到 manifest commit 的业务时间。只看总耗时，无法判断慢在设备、completion loop、executor 队列还是业务提交。

IoRequestState: 把 awaiter 和 CompletionRecord 串起来 CompletionRecord 是一次完成事件，IoRequestState 是请求跨多次 completion 的状态。短读短写、重试、取消和迟到 completion 都要求这个状态对象稳定存在。Coroutine awaiter 不应直接等一个裸 fd，也不应只等一个布尔 ready，而应等这条请求状态进入终态。

Listing 18.21 I/O awaitable 的请求状态字段

```

1 enum class IoRequestStateKind : std::int32_t {
2     Created,
3     Submitted,
4     WaitingCompletion,
5     Partial,
6     RetryScheduled,
7     Completed,
8     Failed,
9     Canceled,
10    Late,
11 };
12
13 struct IoRequestState {
14     std::uint64_t request_id = 0;
15     std::uint64_t buffer_id = 0;
16     std::uint64_t bytes_total = 0;
17     std::uint64_t bytes_done = 0;
18     IoRequestStateKind state = IoRequestStateKind::Created;
19     ExecutorId resume_executor;
20     CoroutineHandleId awaiting_coroutine = 0;
21 };

```

真实 C++ 实现会用平台的 `std::coroutine_handle<>`、类型擦除 handle 或自定义 handle id；这里使用 `CoroutineHandleId` 是为了突出状态边界。关键

点是: completion handler 更新 `bytes_done/state`; awaiter 注册 `awaiting_coroutine/resume_executor`; scheduler 只恢复 terminal request 对应的 coroutine。若 `bytes_done < bytes_total`, awaiter 不能恢复为成功, 只能继续提交剩余字节或返回 partial 语义。

Blocking pool、event loop 和 compute executor 的分工 不是所有平台都直接用 `io_uring`。有些实现会用 dedicated blocking I/O thread pool 包装同步系统调用; 有些用 epoll event loop; 有些用真正的异步提交/完成队列。无论后端怎样, compute executor 都不应该执行长期阻塞 I/O。

执行域	负责什么	不应该做什么
compute executor	CPU task、短 continuation、coroutine resume	阻塞磁盘、网络、长 fsync
blocking I/O pool	包装可能阻塞的 syscall	执行大量 CPU continuation
event loop	epoll ready、timer、轻量状态推进	在回调里跑重业务代码
completion loop	消费 I/O completion, 更新请求表	持锁恢复 coroutine 或提交 manifest 重活
commit executor	checksum、manifest gate、批量提交	无限积压 completion buffer

这张表对应第 14 章的 resume executor 选择。I/O completion loop 可以把 coroutine 恢复项投到 compute executor, 也可以投到专用 continuation executor; 但它不应在 completion 回调里直接运行用户 coroutine。否则一个慢 continuation 会阻塞后续 completion, buffer 归还变慢, 背压链失真。

异步 I/O awaiter 的关闭和取消 关闭时最危险的是“请求已经提交, coroutine 等着, buffer 也被 I/O 层持有”。Cancel request 可能成功, 也可能请求已经完成, 还可能稍后返回 late completion。awaiter 必须把这些路径都推到终态。

Listing 18.22 I/O awaiter 取消和关闭路径

```

shutdown drain:
    reject new IoRequestState
    keep submitted requests alive
    process completions until request table empty
    resume awaiting coroutines with value or error

shutdown cancel:
    reject new IoRequestState
    request cancellation for submitted entries
    mark not-yet-submitted entries Canceled
    keep buffers and states until completion or reap
    resume awaiting coroutines with canceled or final result

late completion:
    find request_id
    if request already terminal:
        classify as Late
        return or retire buffer exactly once
        do not resume coroutine twice

```

这和 future 的取消语义一致: 取消请求不是终态, 终态必须能被等待者观察。若 cancel 后直接销毁 `IoRequestState`, 迟到 completion 会找不到归属; 若直接

销毁 coroutine frame, completion 可能恢复悬垂 handle; 若不恢复 coroutine, 调用者会永久挂起。I/O awaiter 的正确性就是请求表、buffer 所有权、completion 和 coroutine frame 四者同时闭合。

短读短写是正常路径 read 和 write 返回的字节数可能小于请求字节数。网络、管道、非阻塞 fd、信号中断、设备队列和某些文件系统路径都可能出现 partial completion。正确代码必须维护 offset 和 remaining, 直到完成、出错、超时或进入等待。把一次返回当作完整消息, 是 I/O 程序最常见的边界错误。

Listing 18.23 同步可靠写循环的示例形态

```
1 enum class IoStepResult : std::int32_t {
2     Complete,
3     RetryLater,
4     Failed,
5 };
6
7 struct WriteProgress {
8     std::uint64_t bytes_total = 0;
9     std::uint64_t bytes_done = 0;
10 };
11
12 bool is_write_complete(const WriteProgress& progress) {
13     return progress.bytes_done == progress.bytes_total;
14 }
```

这个片段没有包装系统调用, 而是先固定语义: 写出路径必须有进度对象。同步版本在循环里推进进度; 异步版本在 completion handler 里推进进度。异步化不改变短写语义, 只是把“下一步继续写”从调用栈搬到请求状态表。

CompletionRecord Compute Systems Engine 使用 CompletionRecord 保存 I/O completion 的结构化事实, 作用类似第 14 章的 TaskStateRecord。业务层只能根据 completion 改变提交状态, 不能在 submit 后提前宣布成功。

Listing 18.24 CompletionRecord 的示例形态

```
1 enum class IoStatus : std::int32_t {
2     Succeeded,
3     Partial,
4     RetryableError,
5     FatalError,
6     Canceled,
7     TimedOut,
8 };
9
10 struct CompletionRecord {
11     std::uint64_t request_id = 0;
12     std::uint64_t block_id = 0;
13     std::uint32_t attempt = 0;
14     IoStatus status = IoStatus::Succeeded;
15     std::uint64_t bytes_requested = 0;
16     std::uint64_t bytes_transferred = 0;
17     std::uint64_t checksum = 0;
18     std::uint64_t submit_ns = 0;
19     std::uint64_t complete_ns = 0;
```



```

20 std::uint32_t error_code = 0;
21 };

```

`request_id` 区分一次 I/O 请求，`block_id` 区分业务输出，`attempt` 区分重试或迟到完成。若 `completion` 不带这些身份，完成顺序一乱，`driver` 就无法判断哪个结果仍然有效。第 16 章的 RPC reply 会复用同样思想，只是 `completion` 从单节点设备事件变成网络响应。

迟到 completion 请求超时后，底层操作可能仍然完成。上层“不等了”不等于设备“不会再回来”。Completion 到达时，状态机要先检查 `request` 是否仍然有效。若新 `attempt` 已经提交，旧 `completion` 只能释放资源、记录 `late completion` 或清理临时文件，不能重复提交业务结果。

这就是为什么写出路径不能直接写最终文件。旧 `attempt` 即使迟到成功，也应只写自己的临时文件；最终 `manifest` 由 `driver` 决定哪个 `attempt` 生效。没有临时文件和 `manifest`，迟到 `completion` 就可能覆盖新结果。

18.2 从系统调用到设备完成：底层路径账本

理解 I/O 不能只看 C++ 函数返回值。一次文件写入可能穿过用户态 `buffer`、系统调用入口、VFS、文件系统、`page cache`、`writeback`、块层队列、设备缓存和完成中断。不同路径暴露给应用的“成功”含义不同：`write` 返回成功，可能只说明字节进入 `page cache`；`fsync` 返回成功，才把承诺推进到更强的持久化边界；`manifest` 提交成功，才说明业务上这份输出可见。

普通文件写入的事件链 最常见的 `write(fd,buf,n)` 并不是直接把 `buf` 写到盘片或闪存。典型路径可以粗略写成：

Listing 18.25 普通文件写入的底层事件链

```

user buffer contains OutputBlock bytes
enter kernel through write syscall
copy bytes from user memory into page cache pages
mark pages dirty and update inode/file metadata in memory
return bytes_written to user space
writeback thread later submits dirty pages to block layer
storage device completes writeback requests
fsync waits for selected dirty data and metadata to reach durable boundary
manifest commit records which block is externally visible

```

这条链解释了两个常见误判。第一，`write` 很快不代表设备很快，它可能只是内存 `copy` 快。第二，进程崩溃和机器掉电不是同一类故障：进程崩溃后 `page cache` 仍在内核中，掉电后易失缓存可能消失。可靠性承诺必须说明恢复边界：只承诺进程崩溃恢复，还是承诺掉电后也能恢复；只承认 `fsync` 后的数据，还是允许未同步中间结果重算。

案例：一次 `write` 穿过 VFS 和 `page cache` 现在把普通文件写入固定成一个可分析案例。Compute Systems Engine 要写出一个 64 MiB 的 `ShuffleBlock` 到本地文件系统。业务代码看到的是 `write_all(fd,buffer)`；内核看到的是 `fd` 对应的 `open file description`、VFS 文件对象、文件系统方法、`page cache` 页、脏页队列、`writeback` 和块层请求。这个案例的目标不是背 Linux 源码函数名，而是训练读者分清：哪一步证明用户态请求被接受，哪一步证明数据进入内核缓存，哪一步证明文件系统推进持久边界，哪一步才是业务提交。

Listing 18.26 `write VFS probe`：固定一个写入案例

```

1 int fd = ::open("shuffle-block.tmp",
2                 O_CREAT | O_TRUNC | O_WRONLY | O_CLOEXEC,
3                 0644);
4 if (fd < 0) {
5     return error_from_errno(errno);
6 }
7
8 std::vector<std::byte> block(64ull * 1024 * 1024);
9 fill_shuffle_block(block);
10
11 std::uint64_t off = 0;
12 while (off < block.size()) {
13     ssize_t n = ::write(fd, block.data() + off, block.size() - off);
14     if (n > 0) {
15         off += static_cast<std::uint64_t>(n);
16         continue;
17     }
18     if (n < 0 && errno == EINTR) {
19         continue;
20     }
21     return error_from_errno(errno);
22 }
23
24 if (require_durable) {
25     if (::fsync(fd) != 0) {
26         return error_from_errno(errno);
27     }
28 }
29 if (::close(fd) != 0) {
30     return error_from_errno(errno);
31 }

```

这段代码故意使用普通 `write`，不使用 `direct I/O`、`io_uring` 或 `mmap`。原因是普通路径最容易误判：`write` 返回很快时，读者容易以为“磁盘很快”；实际可能只是用户态字节被复制到 `page cache` 并标成 `dirty`。若不调用 `fsync`，短时间 `benchmark` 很可能测的是内存带宽、页分配、VFS 锁和 `dirty throttling` 前的提交速度，而不是设备持久化吞吐。

Listing 18.27 本案例的 VFS/page cache 逻辑路径

```

fd number in process
-> open file description: current offset and status flags
-> VFS file object and file operations
-> filesystem write path
-> address_space page cache pages
-> dirty page accounting
-> background writeback
-> block layer request queue
-> device completion
-> fsync/fdatasync waits for the promised durability boundary

```

VFS 的作用是把“对 `fd` 写字节”转成“对某个文件系统对象执行写方法”。应用不

应假设 ext4、xfs、tmpfs、FUSE、NFS 和 overlayfs 的细节相同。VFS 层提供统一入口，具体文件系统决定 page cache、metadata、日志、权限、copy、direct I/O 和同步语义如何落地。因此报告必须写明文件系统、挂载选项、是否 tmpfs、是否容器 overlay、是否网络文件系统。没有这些字段，`write` 案例就不能迁移。

层级	观察到的事实	不能推出的结论
<code>write</code> 返回正数	内核接受了这些字节进入写路径	设备已写完，掉电可恢复
文件大小变大	inode size 或目录可见状态更新	数据页已经 durable
dirty bytes 增长	page cache 中有未回写债务	业务 block 已提交
writeback 活跃	内核正在把脏页推向设备	本次请求已经完成 durable
<code>fsync</code> 返回 0	文件系统承诺的同步边界完成	manifest 已经接受该 attempt

案例证据包：命令要证明哪一层 这个案例至少要收集四类证据。第一类证明应用确实调用了 `open/write/fsync/close`。第二类证明写入不是一次魔法设备操作，而是经过 page cache 和 dirty accounting。第三类观察 writeback 或同步等待。第四类记录业务层是否已经发布 manifest。每类证据都有边界，不能互相替代。

Listing 18.28 write/VFS 案例的观测命令

```

1 # 系统调用边界：证明用户态请求形状。
2 strace -tt -e trace=openat,write,fsync,fdatasync,close ./write_vfs_probe
3
4 # 文件系统和挂载事实：证明案例跑在哪个语义环境。
5 findmnt -T ./shuffle-block.tmp
6 stat -f .
7
8 # page cache / dirty 债务：观察内核是否积累脏页。
9 grep -E 'Dirty|Writeback|Cached' /proc/meminfo
10 cat /proc/vmstat | grep -E 'nr_dirty|nr_writeback|pgpgout'
11
12 # 进程级 I/O 账本：观察写入字节和系统调用数量。
13 cat /proc/$PID/io
14
15 # 调度和阻塞：观察 fsync/write 是否让线程睡眠。
16 perf trace -e sys_enter_write,sys_exit_write,sys_enter_fsync,sys_exit_fsync ./↵
    ↵ write_vfs_probe

```

`strace` 能证明 `write` 调用次数、返回值和 `fsync` 是否发生，但它不能证明数据已经到设备。`/proc/meminfo` 和 `/proc/vmstat` 能看到系统级 dirty/writeback 变化，却不能把每个脏页精确归因到某个业务 block。`/proc/<pid>/io` 能说明进程发起的 I/O 账本，但字段含义受内核路径和缓存影响。若有权限，可以再用 `ftrace`、`eBPF` 或文件系统 `tracepoint` 细化路径；没有权限时，报告必须写明证据边界。

两个对照：无 `fsync` 和有 `fsync` 同一段写入代码要至少跑两个模式。无 `fsync` 模式回答“应用把字节交给内核有多快”；有 `fsync` 模式回答“文件系统同步承诺有多贵”。二者不是同一个指标。

Listing 18.29 write/VFS 对照报告字段

```

write_vfs_report:
  filesystem
  mount_options
  file_size_bytes

```

```
block_size_bytes
write_call_count
write_returned_bytes
write_elapsed_ms
fsync_enabled
fsync_elapsed_ms
close_elapsed_ms
dirty_before_bytes
dirty_after_write_bytes
dirty_after_fsync_bytes
proc_io_write_bytes
manifest_committed
evidence_limitations
```

若无 `fsync` 模式吞吐很高，而有 `fsync` 模式 p99 很差，结论不是“`fsync` 可删”。正确结论是：同步承诺成本高，需要决定哪些输出必须 durable、哪些中间结果可重算，是否使用 group commit，是否把 `fsync` 从 compute worker 隔离到 I/O worker。若两个模式都很慢，瓶颈可能是内存 copy、文件系统锁、dirty throttling、设备队列或容器存储层。只有把对照字段写全，才知道要优化哪一层。

VFS 案例的错误结论清单 这个案例最重要的训练价值，是防止把不同层级证据混用。看到 `write` 返回 64 MiB，不得写“64 MiB 已经落盘”；看到文件大小变大，不得写“崩溃后一定可恢复”；看到 `fsync` 慢，不得写“线程池调度器慢”；看到 dirty bytes 增长，不得写“进程内存泄漏”；看到 `write` 偶尔变慢，不得只怀疑磁盘，也要考虑 dirty page 阈值、回写限速、文件系统日志和同步写策略。

错误结论	更准确的解释	需要补的证据
<code>write</code> 快，所以磁盘快	可能只是 page cache 接受快	<code>fsync</code> 、dirty/writeback、设备指标
<code>write</code> 慢，所以 CPU 算法慢	线程可能睡在 VFS、回写或 dirty throttling	syscall latency、worker state
RSS 没涨，所以没有 I/O 债务	dirty page 属于系统缓存，不一定计入进程 RSS	<code>/proc/meminfo</code> 、vmstat
<code>fsync</code> 慢，所以删掉它	删除会改变持久化承诺	crash matrix、manifest 策略
tmpfs 上很快，所以实现很好	tmpfs 主要是内存路径	真实文件系统和设备对照

这个 `write/VFS` 案例应成为后续所有 I/O 优化的 reference。换成 `io_uring`、direct I/O、registered buffer 或异步 writer 后，仍要能回答同样的问题：请求被哪一层接受，字节在哪一层，谁拥有 buffer，哪些错误可重试，哪一步推进 durable，哪一步推进 business commit。

读路径和 page fault 读取也不一定真的触碰设备。热 page cache 中的 `read` 可以只做内存 copy；冷 page cache 的 `read` 会触发设备 I/O；`mmap` 顺序访问表面上是普通 load，底层可能在 page fault 中等待磁盘。若 compute worker 直接访问冷 `mmap` 区域，线程可能在缺页路径上睡眠，表现为 CPU 空闲但 ready task 堆积。

路径	应用看到的动作	隐藏等待点
<code>read</code> 热缓存	系统调用返回数据	内存 copy、页表访问、锁竞争
<code>read</code> 冷缓存	调用线程等待返回	设备读、page cache 填充、调度等待
<code>mmap</code> 热页	普通 load 命中内存	TLB/cache 成本
<code>mmap</code> 冷页	load 触发 page fault	缺页 I/O、内核锁、回收压力
<code>write</code> 普通文件	返回写入字节数	后台 writeback、dirty page 限制
<code>fsync</code>	等待同步完成	文件系统、块层、设备缓存和 metadata

因此 I/O 实验要说明 cache 状态。热缓存读性能不能代表设备吞吐；无 `fsync` 的写性能不能代表可靠提交；`mmap` 版本慢不一定是解析慢，可能是 page fault 混入 compute worker。报告至少要把 cold cache、warm cache、tmpfs、真实文件系统、是否 `fsync`、文件大小是否超过内存写清。

实测证据：mmap 首次触页和 socket 分帧 下面是一份很小的证据包形态，目的不是声称某台机器的磁盘或网络性能，而是训练 I/O 报告如何分层。实验环境只记录可公开复现的字段：内核版本、架构、C 库版本、文件系统类型、是否 tmpfs、是否容器或虚拟化环境。探针写入 16 MiB 临时文件并调用 `fsync`，随后用 `mmap` 每页触碰一次，再触碰第二次；另一个探针用 `socketpair` 发送三条带长度头的帧，但故意把发送 chunk 和接收 chunk 切得不一樣。

Listing 18.30 storage/socket 探针输出形态

```
env_kernel=Linux <version>
env_arch=x86_64
env_libc=<libc and version>
storage_probe
file_bytes=16777216
write_ms=3.692
fsync_ms=0.002
mmap_first_ms=0.527 minflt=257 majflt=0 checksum=94208
mmap_second_ms=0.154 minflt=0 majflt=0 checksum=94208
socket_probe
sent_chunk_sizes=3,2,4,5,7,11
recv_chunk_sizes=4,4,4,4,4,4,4
parsed_frames=alpha|beta-gamma|delta
leftover_bytes=0
```

这份输出能支持三条有限结论。第一，第一次 `mmap` 触页产生了 minor fault，第二次没有，说明“同样的 load 循环”可能包含不同的页表和 page cache 状态；因为 major fault 为 0，不能把这次触页解释成真实设备读。第二，`fsync_ms` 很小，只能说明这个 16 MiB 临时文件在该环境和该文件系统路径上的同步成本很低，不能推广到真实 NVMe、网络盘、掉电保护或更大文件。第三，socket 接收 chunk 全是 4 字节，而发送 chunk 是 3、2、4、5、7、11 字节；帧仍然被解析成 `alpha|beta-gamma|delta`，说明 socket stream 不保留应用消息边界，应用必须靠长度头、状态机和缓冲区重组。

Listing 18.31 socket stream parser 的必要状态


```
FrameParser:
  buffer
  need_header_bytes
  current_payload_bytes
  parsed_frames
  error_state
```

这个证据包也给出边界。它没有 PMU counter，所以不能声称 IPC、cache miss 或 syscall cycles 改善；它使用本地 socketpair，所以不能代表真实 TCP 拥塞、网卡队列、Nagle、TLS 或跨机器 RTT；它没有 drop cache 权限，也没有证明 cold storage 路径。合格报告要把这些限制写出来。证据越诚实，结论越可靠。

Ready 事件和 completion 事件的根本差别 Ready 事件说明“现在尝试可能不会立刻阻塞”。它是机会通知，不是完成事实。Socket 可读可能只有半个消息头，管道可写可能只能写一部分，边沿触发还要求应用一直读到 EAGAIN。Completion 事件说明“某个已提交请求已经得到结果”。它带 request id、字节数和错误码，可以推进请求状态机。

这两种模型会导出不同代码结构。Ready 模型需要应用自己维护每个连接的读写进度：还有多少 header 没读、body 还差多少、send buffer 还剩多少。Completion 模型需要应用维护请求表：哪个 request 已提交，buffer 归谁，completion 回来时是否仍有效。二者都不能省掉业务状态机，只是把等待位置放在不同地方。

Direct I/O 和 registered buffer 的边界 Direct I/O、registered buffer、零拷贝发送看起来能减少 copy，但它们会收紧所有权和对齐要求。Buffer 可能要求按页对齐，大小按块对齐，生命周期必须覆盖内核或设备访问窗口。减少一次 copy 的代价，是业务层更难提前复用内存，也更难处理取消和迟到 completion。

Direct I/O 的基础合同必须讲得更窄：它不是“更快的 write”，而是“尽量不通过 page cache 保存文件数据页”的 I/O 形态。Linux 上常见入口是 O_DIRECT，但具体支持、对齐要求、和 page cache 的一致性细节受文件系统、块设备、内核版本和挂载选项影响。通常要同时满足用户 buffer 地址对齐、文件 offset 对齐、I/O size 对齐；不满足时可能失败，或者在某些路径上退回到不同实现。它仍然要经过 VFS、权限检查、文件系统方法、块层队列和 completion；它也不自动提供异步语义，更不自动提供掉电持久化。

路径	优点	代价和边界
buffered I/O	简单，利用 page cache、readahead、writeback 合并	可能把设备等待隐藏成 dirty debt，热缓存测不到设备吞吐
direct I/O	减少 page cache copy 和缓存污染，更接近设备队列	对齐和生命周期更硬，随机小 I/O 可能更差，仍需处理 completion 和 fsync 语义
mmap	load/store 形式访问文件，适合部分随机读	page fault 混入计算路径，SIGBUS 和映射生命周期要处理
registered buffer	减少重复 pin/register 成本	buffer pool 必须和请求状态机绑定，取消和迟到 completion 更复杂

因此选择 buffered 还是 direct，必须先写 workload。顺序读大文件时，buffered I/O 的 readahead 可能已经很好；大量一次性读写大 block、且不希望污染 page cache 时，direct I/O 才可能有意义；可靠写出时，不管选择哪条路径，都要说明 durable 边界和业务 commit 边界。若报告只写“用了 O_DIRECT 所以绕过内核缓存”，仍然不合格：它还要列出对齐、文件系统、设备、是否混用 buffered I/O、以及如何证明实际路径。

因此 Compute Systems Engine 的第一版不应追求零拷贝。先用普通 buffer 和 I/O worker 把状态机做对,再用报告证明 copy 成为瓶颈。若 copy 只占总时间很小,零拷贝会增加复杂度却没有明显收益。若 block 很大、CPU copy 确实占主导,再把 buffer pool、对齐、pinning、registered buffer 和 completion ownership 加入设计。

写入路径的五个成功层级 “写成功”必须拆成层级,否则可靠性讨论会混乱。

层级	含义	仍未解决的问题
Accepted	请求进入用户态或内核队列	可能尚未写任何字节
Copied	字节进入内核 page cache 或发送缓冲	设备可能未完成,掉电可能丢失
Completed	底层请求返回成功或失败	业务是否接受该 attempt 尚未确定
Durable	数据和必要 metadata 到达承诺边界	是否被 manifest 引用尚未确定
Committed	manifest 接受该 block	后续消费者可以读取

异步 I/O 编程的核心就是不把这些层级混成一个布尔值。Submit 返回只能说明 Accepted; completion 只能说明 Completed; fsync 推进 Durable; manifest 推进 Committed。不同层级的失败处理不同: Accepted 前失败可以直接把 buffer 还给 producer; Completed 失败要记录错误和释放 in-flight; Durable 前崩溃可能重写; Committed 后崩溃必须承认提交事实。

Dirty page 是隐藏队列 普通文件写入返回后,脏页可能留在 page cache 中等待 writeback。这个等待队列不在用户态线程池里,却消耗内存、内核回写带宽、块层队列和设备吞吐。若写入速度长期高于设备完成速度,dirty pages 会积累;达到系统阈值后,普通 write 可能突然变慢,甚至调用线程被迫参与回写或等待脏页下降。应用如果只看自己的提交队列长度,会误以为没有积压;实际上债务已经转移到内核。

Listing 18.32 写入债务从用户态转移到内核的过程

```
producer generates output blocks faster than storage can persist
writer calls write and receives success quickly
page cache dirty bytes grow
background writeback submits requests to storage
storage queue latency grows
kernel throttles dirtying tasks or fsync waits longer
application sees sudden write or fsync latency spikes
```

这就是背压必须看多层指标的原因。用户态队列空,不代表系统没有债务;write 平均延迟低,不代表可靠提交快;吞吐看起来高,可能只是把数据堆进易失缓存。稳态判断要同时观察 output 生成速率、write 返回速率、fsync 或 durable 速率、dirty bytes、completion 延迟和 manifest commit 速率。任何一层长期低于上游,都需要把背压向上游传播。

fsync 不是单纯慢函数 fsync 慢,是因为它把之前隐藏的持久化债务拉到当前线程面前。它可能等待文件数据、metadata、日志提交、块层请求和设备缓存 flush;具体语义受文件系统、挂载选项、设备和内核影响。不能因为 fsync 慢就删掉它,也不能因为 benchmark 中 fsync 很快就认为可靠写没有成本。正确做法是说明承诺边界:需要掉电恢复时,必须把必要数据和 metadata 推到文件系统承诺的持久边界;只需进程崩溃恢复时,可以把部分中间结果作为可重算缓存处理。

承诺	需要等待的事实	可接受的恢复动作
进程内完成	request 状态转为 completed	进程崩溃后可丢弃并重算
进程崩溃恢复	内核仍可见的文件或临时状态	重启扫描并校验 checksum
掉电恢复	fsync 后的数据和必要 metadata	依赖文件系统承诺，缺失则重算
业务提交	manifest 接受某个 attempt	后续消费者以 manifest 为准

这张表把可靠性和性能拆开。系统可以选择某些中间 block 不做强持久化，只要 manifest 不把它们宣布为已提交；也可以选择 group commit，把多个 block 的持久化成本合并。无论选择哪种策略，都要写入恢复协议，而不是让恢复逻辑从文件名猜状态。

rename 只是名字切换，不是完整提交协议 可靠写常用模式是：写临时文件，校验，fsync 文件，rename 到目标名，再更新 manifest。这里每一步都有不同含义。rename 在许多本地文件系统上能提供同目录下名字切换的原子性：恢复时通常看到旧名字或新名字，不会看到半个名字。但名字原子性不等于数据已经持久，也不等于目录项一定经受住掉电。若承诺掉电恢复，通常还要考虑对文件和目录的同步边界；若只承诺进程崩溃恢复，page cache 中的目录和文件状态仍可能足够，但恢复协议要明确。

Listing 18.33 可靠 block 写入的提交层次

```

write temp file:
    bytes enter page cache or direct I/O path

fsync temp file:
    block bytes and required file metadata reach durable boundary

rename temp -> final:
    directory name now points to the final block name

fsync parent directory:
    directory entry update reaches durable boundary when required

append or rewrite manifest:
    business layer accepts this attempt as visible output

```

这个顺序不表示所有系统都必须对每个中间 shuffle block 做最强同步。批处理系统可以把中间 block 视为可重算缓存，只对最终 manifest 做更强承诺；也可以用 group commit 把多个文件和 manifest 的同步成本合并。关键是不能把 rename、fsync 和 manifest commit 混成一个“保存成功”。rename 解决名字切换，fsync 解决持久边界，manifest 解决业务可见性和 attempt 选择。

Manifest 为什么比最终文件名更强 只靠最终文件名判断成功，会在重试和迟到 completion 中出错。假设 split 7 的 attempt 0 写出了 part-7.tmp.0，超时后 attempt 1 写出了 part-7.tmp.1 并提交 manifest。之后 attempt 0 迟到完成并被 rename 成 part-7.final。如果下游只按文件名读取，它可能读到旧 attempt；如果下游只读 manifest，它会知道 accepted attempt 是 1，attempt 0 即使文件完整也不能生效。

Listing 18.34 manifest 条目表达的业务事实

```

logical_partition = 7
accepted_attempt = 1
file_path = part-7-attempt-1.block

```

```
records = 842193
bytes = 67128320
checksum = 0x8f21...
partition_plan_version = 3
```

Manifest 的作用不是重复文件系统目录，而是把业务身份、attempt、校验、分区规则和可见性绑在一起。文件存在是物理事实；manifest 接受是业务事实。I/O 层必须把这两个事实分开，否则恢复程序无法判断哪些文件是 orphan，哪些是 late attempt，哪些才是后续 stage 的输入。

系统调用边界和 C++ 内存边界不同 一个常见混淆是把线程间可见性和持久化可见性放在一起。C++ 的 release/acquire 或 mutex 可以保证另一个线程看到 buffer 中已经写好的字节；系统调用可以把这些字节交给内核；fsync 可以把文件推进到文件系统承诺的持久边界；manifest commit 可以让业务层承认输出。它们是不同的层级的顺序关系。

边界	解决的问题	不能替代的层级
C++ release/acquire	线程看到已构造请求和 buffer 字段	不能保证内核接受或磁盘持久
write 返回	内核接受部分或全部字节	不能保证 durable 或业务提交
fsync 返回 rename 返回	文件系统承诺的持久边界推进 目录命名状态改变	不能决定哪个 attempt 生效 不能单独表达业务校验和分区版本
manifest commit	业务可见输出被接受	依赖底层文件和恢复协议的承诺

这张表能解释为什么 I/O 代码看起来“步骤很多”。每一步解决一个不同故障模型。删掉其中一步，系统未必立刻变错，但承诺边界已经改变。报告必须写明这种改变，而不是只写“优化掉 fsync 后变快”。

Completion 线程不能成为新瓶颈 异步 I/O 常把等待从提交线程移到 completion loop。Completion loop 的职责应尽量短：取回结果、更新 request 状态、释放或转交 buffer、记录错误、推进 manifest 或通知上层。若在 completion 回调里做压缩、解析、复杂合并、同步日志格式化或长时间持锁，完成队列会被阻塞，设备已经完成的请求无法及时回收 buffer，上游看到的背压会被放大。

Completion 线程本质上也是运行时资源。它需要自己的队列、优先级、关闭协议和异常边界。若 completion loop 和 compute worker 共享同一个线程池，I/O 完成可能被数据任务淹没；若 completion loop 过多，又会争抢 CPU 和 cache。通常更稳的结构是：completion loop 只做轻量状态推进，重 CPU 后处理重新投递到计算队列，可靠提交和 manifest 更新有独立、小而可审计的路径。

Listing 18.35 completion loop 的短路径原则

```
on completion:
  locate request by request_id
  validate attempt and current state
  record bytes, error, and latency
  release or transfer buffer ownership
  enqueue follow-up CPU work if needed
  perform only bounded manifest/commit work
```

这条原则不是追求代码整洁，而是防止完成事件本身形成新的隐藏队列。异步系统真正的吞吐上限取决于 submit、device、completion、commit 和 recovery 这些阶段的共同稳态。

18.3 完整案例：从阻塞 write 改造成有背压的完成流水线

现在把 I/O 路径写成一次完整改造。初始版本由 compute worker 直接调用 `write` 写 shuffle block。小输入时它很简单；输入变大后，worker 有时卡在 `write`、`fsync` 或冷 `mmap` 缺页里，线程池看起来还有线程，实际可运行计算任务越来越少。第 9 章把这种现象归为 blocked worker；本章要把它改成可观测的 I/O 状态机。

第一版：阻塞写把等待混进计算预算 朴素代码通常长这样：

Listing 18.36 阻塞写版本把设备等待放进 worker

```
for each batch assigned to worker:
    partial = aggregate(batch)
    block = serialize(partial)
    write_all(output_fd, block.bytes)
    fsync_if_required(output_fd)
    append_manifest(block.metadata)
```

这个版本的问题不是“同步 I/O 一定错”，而是等待边界不清。Worker 既做 CPU 聚合，又做序列化，又等待存储，又提交 manifest。Profile 可能显示 worker 在系统调用或内核路径里，任务运行时却只看到 worker 忙。上游继续提交任务，dirty page 和输出队列继续增长，背压来得太晚。

第二版：把请求、buffer 和 attempt 分开 改造第一步是定义 `WriteRequest`。它携带逻辑身份、attempt、buffer、目标路径、可靠性策略和状态。提交请求后，buffer 所有权转移给 I/O 子系统；compute worker 只拿到一个 request id 或 future。

Listing 18.37 WriteRequest 表达 I/O 状态机输入

```
1 struct WriteRequest {
2     std::uint64_t logical_partition = 0;
3     std::uint32_t attempt = 0;
4     std::uint64_t request_id = 0;
5     BufferHandle buffer;
6     std::string temp_path;
7     bool requires_fsync = false;
8     std::uint64_t checksum = 0;
9 };
```

`logical_partition` 和 `attempt` 防止迟到 completion 越权提交；`request_id` 定位 in-flight 表；`BufferHandle` 保护生命周期；`requires_fsync` 明确持久化承诺；`checksum` 让 completion 和 manifest 能验证数据身份。没有这些字段，异步只是把错误延迟到以后发生。

第三版：completion 只推进状态 I/O worker 或 completion loop 负责处理 `WriteRequest`。它可以内部使用阻塞写线程、`epoll`、`io_uring` 或模拟完成队列；原理卷不绑定某个接口。关键是 completion 回来时只做有界工作：校验 request 当前仍有效，记录结果，释放或转交 buffer，必要时把 manifest 候选投递给提交线程。

Listing 18.38 完成流水线的状态转移

```
compute worker:
    build block in owned buffer
```



```

submit WriteRequest
continue CPU work or respect backpressure

io subsystem:
    write temp bytes, handle short writes
    fsync if requested by policy
    emit CompletionRecord(request_id, bytes, error, durable_state)

completion loop:
    validate request still current
    recycle or transfer buffer
    enqueue ManifestCandidate only if completion is acceptable

```

这条流水线把等待从 compute worker 移走，但没有让等待消失。设备仍然慢，fsync 仍然慢，buffer 仍然有限。区别是等待变成了可测队列，而不是混在任意 worker 的调用栈里。

第四版：背压从最低吞吐阶段向上游传播 异步系统最危险的错觉是“提交很快，所以系统很快”。如果 submit 速率长期高于 durable 或 commit 速率，in-flight 请求、buffer、dirty pages 或临时文件会堆积。背压要从最慢的稳定阶段向上游传播。可以设多个水位线：free buffer 数量、in-flight bytes、dirty estimate、completion latency、manifest queue 长度。

Listing 18.39 多层背压水位线

```

if free_buffers < low_watermark:
    stop accepting new serialization work

if inflight_write_bytes > write_budget:
    slow down map-side combine output

if fsync_latency_p99 exceeds budget:
    increase group commit window or reduce output concurrency

if manifest_queue grows:
    stop admitting new attempts for the same stage

```

背压不是错误路径，而是正常控制流。它的目标不是让队列永远为空，而是让每层债务有上限：buffer 不被复用过早，dirty pages 不无限增长，completion 不被 CPU 后处理堵住，manifest 不被重复 attempt 淹没。

第五版：关闭和取消 关闭路径要比正常路径更严格。停止接收新请求，不代表已有请求不存在；取消 attempt，不代表底层 I/O 不会完成；释放 buffer，不代表设备不再访问。关闭状态机应按阶段推进：

Listing 18.40 I/O 流水线关闭顺序

```

stop admission:
    reject new WriteRequest or mark stage closing

drain or cancel:
    choose whether in-flight requests may finish
    mark canceled attempts so late completions cannot commit

```

```

reap completions:
    process every completion until ownership is resolved
    recycle buffers only after completion or proven cancellation boundary

commit or discard:
    append manifest for accepted attempts
    remove orphan temp files according to recovery policy

```

这个顺序和线程池关闭、条件变量谓词、原子状态机是一条主线：状态先改变，等待者或 completion 再观察状态，资源最后释放。若顺序反过来，就会出现 use-after-free、重复提交、孤儿文件或丢失唤醒。

第六版：状态观测点 状态观测点要能解释系统是 CPU 慢、设备慢、completion 慢还是 commit 慢：

Listing 18.41 异步写流水线观测点

```

io.submit.count
io.submit.rejected_by_backpressure
io.inflight.requests
io.inflight.bytes
io.short_write.count
io.completion.latency_p50
io.completion.latency_p99
io.fsync.latency_p99
io.buffer.free_count_min
io.dirty_bytes_if_available
manifest.queue_depth_max
manifest.commit.latency_p99
late_completion.count
orphan_temp_files_reclaimed

```

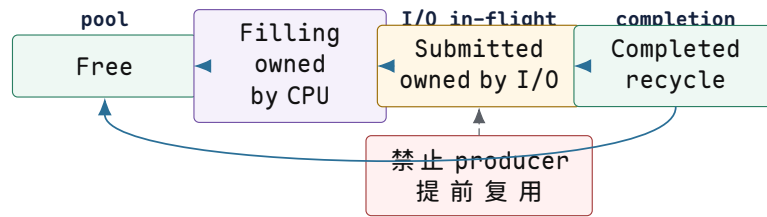
有了这些观测点，调优才不会盲目。若 free buffer 见底而设备延迟高，瓶颈在持久化；若 completion 队列增长但设备延迟不高，completion loop 太重；若 manifest 队列增长，提交路径串行化；若 late completion 多，attempt timeout 或取消策略需要复查。

18.4 所有权、Buffer 池和取消

异步 I/O 最常见的正确性 bug 不是系统调用用错，而是 buffer 生命周期用错。同步调用返回后，调用者通常可以复用 buffer；异步提交后，设备、内核、I/O 线程或发送队列可能仍在读取这个 buffer。若上游提前复用，磁盘或网络看到的的就是后续数据。高性能系统常用 buffer pool 减少分配，但 buffer pool 只有配套状态机才安全。

所有权转移 Producer 从 pool 取出 free buffer，填充数据后提交给 writer。提交成功后，buffer 所有权转移给 I/O 子系统；producer 不能再写。Completion 处理完以后，buffer 才能进入 recycling 或 free。取消、失败、短写都必须走 completion 或 reap 路径，不能绕开状态机。

Buffer 生命周期保护异步写



buffer pool 本身不是优化魔法。只有每个状态都有唯一所有者，pool 才不会产生悬垂、提前复用和双重归还。

图示：本图要证明的命题是：异步请求提交后，buffer 生命周期必须由 completion 决定，而不能由 submit 返回值决定。

Buffer 池也是背压 Buffer 池大小决定最多 in-flight 字节。若每个 buffer 1 MiB，允许 512 个 in-flight，就已经占用 512 MiB。池耗尽时，上游必须等待、降低读取速率、减小 batch 或返回 backpressure。这比只限制队列条目数更直接，因为少量超大 block 也会击穿内存预算。

项目应同时记录 item count 和 byte count。队列里 100 个 4 KiB 请求和 100 个 64 MiB 请求不是同一种压力。第 18 章 shuffle 的 in-flight bytes 限制，正是从这里来的。

取消是尽力而为 取消请求后，底层操作可能已经完成、正在内核或设备中、还在队列里、或者尚未提交。取消只能改变上层接受结果的策略，不能保证物理写入没有发生。状态机至少要区分 **Canceled** 和 **Reaped**：前者表示用户不再接受业务结果，后者表示底层完成事件已经被消费、资源可以释放。

若取消后直接销毁上下文，迟到 completion 就可能访问已释放对象。正确方式是让 request context 活到 reaped；completion 到达时检查状态，若已取消，则释放 buffer、清理临时文件、记录 late completion，不提交 manifest。

关闭时 drain 还是 cancel I/O 子系统关闭有两种常见语义。Drain 表示拒绝新请求，但继续处理已经接受的请求，直到它们完成或失败；cancel 表示未开始请求取消，in-flight 请求尽力取消或等待 completion 后清理。日志调试数据可能允许 cancel，checkpoint、manifest、必须完成的 shuffle block 通常需要 drain 或明确失败。

关闭不是析构时设置一个布尔值。它要停止接收新请求、唤醒等待者、处理队列中请求、消费 completion queue、归还 buffer、报告未完成请求并 join 线程。若 completion queue 没有 drain，底层稍后完成可能访问已经销毁的上下文。

WriteRequest 和 completion loop 异步 I/O 的中心不是 submit，而是 completion loop。Submit 只是把请求交出去；completion loop 接收完成事件，更新请求状态，推进短写 offset，归还 buffer，触发后续 task，汇总错误。若 completion loop 设计不清，异步代码就会散成回调、锁和共享变量，最后很难证明同一个请求只完成一次。

Listing 18.42 WriteRequest 的示例状态

```

1 enum class WriteRequestState : std::int32_t {
2     Created,
3     Queued,
4     Submitted,
5     PartialWritten,
6     Completed,

```

```

7   Failed,
8   Canceled,
9   Reaped,
10 };
11
12 struct WriteRequest {
13     std::uint64_t request_id = 0;
14     std::uint64_t block_id = 0;
15     std::uint32_t attempt = 0;
16     std::uint64_t buffer_id = 0;
17     std::uint64_t bytes_total = 0;
18     std::uint64_t bytes_done = 0;
19     WriteRequestState state = WriteRequestState::Created;
20 };

```

Completion loop 收到事件后，第一步不是执行业务回调，而是查找 `request_id`，验证 `attempt`，检查当前状态。若状态已经是 `Canceled`，completion 只能清理底层资源并进入 `Reaped`；若是 `Submitted` 且只写了一部分，就更新 `bytes_done` 并提交剩余部分；若全部完成且 checksum 通过，才触发 commit task。这个顺序能防止重复回调、迟到完成污染结果和失败路径泄漏 buffer。

完成回调不要做重 CPU 工作 Completion loop 线程不应执行重 CPU 计算，例如解析大文件、压缩、排序或大规模 checksum 聚合。它可以做必要的状态验证和轻量 checksum 检查，然后把后续 CPU work 放回任务运行时。否则底层 I/O 已经完成，completion queue 却因为回调线程忙于计算而积压，表现为“设备不慢，但用户态迟迟看不到完成”。

这个边界和第 14 章一致：worker 只执行 ready 的 CPU 任务，I/O completion 只把等待完成的事实带回状态表。若 completion handler 直接提交 manifest、执行 reduce 或递归触发大量回调，系统会把控制面和数据面混在一起，调试难度急剧上升。

18.5 背压：容量合同和传播链

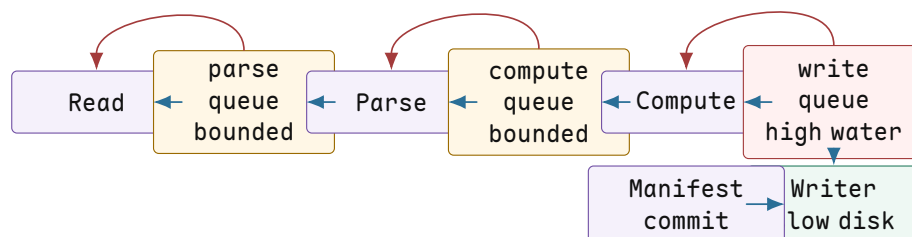
背压是系统告诉上游“下游处理不过来”的机制。无界队列把这个事实藏进内存，直到 OOM、尾延迟爆炸或恢复窗口无法清理。有界队列把容量变成合同：队列满时，生产者必须等待、失败、降级、丢弃低优先级数据或把压力继续传回源头。背压不是性能失败，而是系统保护自己。

队列长度不是唯一指标 队列长度、队列等待时间、入队速率、出队速率、in-flight bytes、completion 延迟和错误率要一起看。长度高但短暂，可能只是 burst；长度持续增长，说明生产速度长期超过消费速度；长度不高但等待时间长，可能是每个请求很大；completion 延迟高，可能是设备慢；completed 到 committed 时间高，可能是提交路径慢。

I/O 报告应拆分延迟：queued 到 submitted，submitted 到 completed，completed 到 committed。只看端到端总时间，无法判断背压该加在 reader、compute、writer 还是 manifest。

高低水位线 单一容量会让系统在“满”和“不满”之间频繁抖动。高低水位线更稳定：超过高水位时上游暂停或减速；低于低水位时恢复。高水位不能太接近容量，否则反馈太晚；低水位不能太接近高水位，否则频繁开关。水位线要用实验调，不是随手写一个数字。

背压必须穿过整条流水线



写盘慢时，压力应从 write queue 传回 compute、parse、read。
若中间任何一段是无界队列，背压链断开，内存就在那里堆积。

图示：本图要证明的命题是：背压是整条数据流的协议，不是某一个队列的局部属性。

策略有业务语义 队列满时怎么处理，取决于数据价值。准确计算的 shuffle block 不能静默丢弃；调试日志可以在过载时丢弃并增加 drop counter；交互请求可能返回 busy 或 deadline exceeded；离线批处理可以等待更久。一个 bool 返回值不够，接口应区分 accepted、would block、timed out、closed、canceled 和 failed。

Listing 18.43 背压提交结果的示例枚举

```
1 enum class BackpressureResult : std::int32_t {
2     Accepted,
3     WouldBlock,
4     TimedOut,
5     Closed,
6     DroppedByPolicy,
7 };
8
9 struct QueueCapacity {
10     std::uint64_t max_items = 0;
11     std::uint64_t max_bytes = 0;
12     std::uint64_t high_water_bytes = 0;
13     std::uint64_t low_water_bytes = 0;
14 };
```

策略要可观测。若丢弃低优先级日志，必须记录丢弃数量、优先级和原因；若阻塞生产者，必须记录阻塞时间；若返回 busy，必须记录上游是否尊重了 busy。静默背压和静默丢弃都会让系统不可诊断。

deadline 和 timeout Timeout 是某个操作愿意等待多久；deadline 是整个请求最晚可以完成的时间点。流水线更适合传 deadline，因为请求经过 read、parse、compute、write、commit 多个阶段，每个阶段都应知道剩余预算。若每段都重新获得完整 timeout，端到端延迟可能无限膨胀。

Deadline 也约束重试。剩余 5 ms 的请求，不应启动预期 50 ms 的 fsync 或 RPC 重试。分布式章节还会遇到时钟和跨节点传播问题；这里先用单调时钟建立“剩余预算”这个思维。

延迟分解比总耗时更有解释力 I/O pipeline 的端到端时间应拆成阶段：read service、parse queue wait、parse service、compute queue wait、compute service、write queue wait、write service、commit wait。若总耗时上升但

write service 没变，可能是队列等待增加；若 write service 上升，可能是设备、fsync 或文件系统路径变慢；若 commit wait 上升，可能是 group commit 等批或 manifest 锁。

观察	可能原因	下一步检查
queue wait 高， service 低	下游并发不足或背压过早	worker 数、队列容量、 水位线
service 高，queue wait 低	设备或同步策略慢	fsync、batch、page cache、模拟器延迟
in-flight bytes 高	大 block 或 completion 慢	block size、buffer pool、completion loop
late completion 多	deadline 太紧或重试过快	deadline 预 算、 attempt 去 重、清 理路径
orphan temp files 多	commit 前失败或恢复清理 弱	崩溃点矩阵、manifest 扫描规则

这张表的价值在于建立判读口径。背压生效时，吞吐可能下降，但内存稳定、错误不增加、队列等待有上界；背压失败时，短期吞吐可能更好，但 in-flight bytes 和队列等待持续上涨。可靠报告必须解释这些曲线，而不是只宣布“异步版更快”。

18.6 可靠写入：从临时文件到 Manifest

写文件成功不等于结果可靠。普通 write 可能只是把数据放进 page cache；rename 可能只更新目录项；manifest 可能已写但未持久化；进程可能在任意步骤崩溃。可靠写入的目标不是让所有中间状态都消失，而是让每个中间状态都能被恢复程序解释。

先写临时文件 输出不应直接覆盖最终文件。Writer 先写临时路径，临时名包含 stage、partition、task、attempt 和随机后缀。数据完整、checksum 通过、必要同步完成后，再通过 rename 或 manifest commit 让它成为可见结果。未提交临时文件不是有效输出，reduce 不应扫描临时目录取数据。

崩溃点矩阵 可靠 I/O 不是一句“使用 fsync”，而是一张崩溃点矩阵。每一步都要说明崩溃后恢复怎么处理。

崩溃位置	恢复时看到什么	恢复动作
创建临时文件前	没有候选输出	重新执行 task 或保持 未完成
写临时文件中	可能有残缺临时文件	校验失败，删除或保留诊 断
写完但未校验	文件存在但身份未确认	重新校验，失败则重做
校验通过但未提交 manifest	候选文件存在，外部不可见	按策略补提交或清理孤 儿
manifest 已提交	权威记录引用该 block	接受为有效输出
清理旧文件前	旧临时文件仍在	根据 manifest 再次清 理

项目可以先承诺进程崩溃恢复，再把掉电持久化作为扩展。若要求掉电后恢复，需要考虑文件 fsync、目录 fsync、manifest fsync、文件系统和设备缓存语义。不同平台的细节不可能完全一致，但可靠性承诺必须分层说明。

fsync、rename 和 manifest 的边界 常见本地文件模式是：写临时文件，确保内容完整，必要时 fsync 文件，rename 到最终名，必要时 fsync 父目录，最后发

布 manifest 或把 manifest 作为唯一权威入口。不同文件系统、对象存储、NFS 和移动平台语义会变化，所以项目报告必须写清文件系统、测试方式和未验证边界。

Manifest 的意义是把“哪些输出有效”集中成权威事实。外部消费者只信 manifest，不扫描目录。这样崩溃后，孤儿临时文件可清理，重复 attempt 可去重，迟到 completion 不会直接污染最终结果。第 18 章的 shuffle manifest 会直接复用这个模型。

批处理和 group commit 每个 block 都 fsync，可靠但慢；多个 block 共享一次持久化，吞吐高但每个请求等待更久，崩溃后可能重做更多未提交工作。数量阈值和时间阈值要一起设置：达到 N 个 block 或等待 T 时间就提交。只按数量低流量时延迟高；只按时间高流量时批太小。

策略	适用场景	风险
每 block 同步	审计强、数据少、语义优先	吞吐低，尾延迟高
按数量批量提交	高吞吐离线写出	低流量等待时间长
按时间窗口提交	延迟预算明确	高流量批次可能偏小
数量或时间触发	常见工程折中	需要调参和监控

可靠写入也是性能取舍 可靠写入会增加系统调用、flush、metadata 和恢复扫描成本。它不是无条件越强越好。离线中间结果可能允许重算，最终 checkpoint 需要更强提交；调试日志可以丢，manifest 不可以丢。可靠性设计要把可靠性承诺、重做成本和性能成本放在同一个表里比较。

Page cache 会误导 benchmark 文件 I/O benchmark 很容易被 page cache 误导。第一次读大文件可能来自设备，第二次读同一文件可能来自内存；写文件返回快，可能只是进入 page cache，真正 writeback 稍后发生。若报告没有说明冷 cache、热 cache、是否 fsync、文件大小是否超过内存、是否使用 tmpfs，结果就无法解释。

实验应区分三类目标。研究背压时，可以使用模拟 I/O 稳定控制延迟和错误；研究 page cache 时，要明确预热、文件大小和系统限制；研究可靠提交时，要关注 fsync、rename、manifest 和崩溃点。把三类目标混在同一张性能图里，缓存效果很容易被误判为 I/O 设计效果。

Page cache 还会影响内存账本。大量未落盘 dirty page 可能不计入进程 RSS，却消耗系统内存并触发回写压力。报告可以记录进程内存、系统可用内存、输出目录大小和 completion 延迟。即使指标粗略，也比完全忽略系统缓存更诚实。

18.7 工业界设计案例

工业 I/O 设计的共同点，是把等待、完成、容量和提交边界写成可审查状态。读源码时不要只列 API 名称，要说明它购买了什么能力、付出什么成本、适合什么负载。

工程案例

Linux page cache 和 writeback 普通文件写入常先进入 page cache，dirty page 之后由 writeback 写到设备。于是 write 返回快，不代表掉电后可恢复。fsync 的价值是推动数据和相关元数据到更强持久化边界。这个设计购买了吞吐和合并写入，也带来 benchmark 误判：只测 write 返回时间，可能完全测的是内存路径。

项目报告应区分应用层背压实验、page cache 实验和可靠提交实验。研究背压时可以用模拟 I/O；研究可靠性时要关注 fsync 和崩溃点；研究真实设备吞吐时要记录 cache 热冷、文件大小、同步策略和系统权限。

工程案例

epoll 的 ready 语义 `epoll` 通知 fd ready，不通知业务消息完整。它购买的是少量线程等待大量 fd 的能力，不替应用处理 framing、短读、短写和业务队列。边沿触发减少重复通知，但要求应用读写到 `EAGAIN`，否则容易遗漏机会。

Compute Systems Engine 不必实现复杂网络 reactor，但要吸收 ready 语义的边界：事件循环线程不能做重 CPU 工作；业务消息要有状态机；完成后要把 CPU work 交给任务运行时；背压要通过停止读取、减少发送或应用层 busy 传出去。

工程案例

io_uring 的 SQ/CQ 模型 `io_uring` 把提交队列和完成队列显式化。用户态提交请求，内核推进，完成队列返回结果。它购买了更低系统调用开销和更丰富的异步操作，但没有取消设备延迟、错误、容量和生命周期问题。Buffer 注册和零拷贝还会让 buffer 生命周期更严格。

项目可以先用 I/O worker 模拟异步，引入 CompletionRecord、buffer ownership、backpressure 和 reliable commit；之后再把底层替换成 `io_uring`。这样 API 变化不会改变主线。

SQE、CQE 和业务状态不是同一层 `io_uring` 的提交队列项 SQE 描述“希望内核执行什么操作”，完成队列项 CQE 描述“内核观察到这个操作怎样结束”。业务层的 block 状态还要再往上走一层：buffer 是否仍被内核持有，短写是否已经补齐，临时文件是否完整，manifest 是否接受，迟到 completion 是否需要清理。把 CQE 直接当作业务提交，是异步 I/O 中最常见的层级错误。

Listing 18.44 从 SQE 到业务提交的状态层次

```
application:
    allocate buffer and build WriteRequest

submit:
    fill SQE with fd, offset, buffer, length, user_data
    publish request as InFlight

complete:
    read CQE result
    if result is short write:
        submit next SQE for remaining range
    if result is error:
        classify retryable or terminal
    if full byte range is written:
        verify temp block state
        request manifest commit

commit:
    manifest accepts exactly one attempt
    buffer may return to pool after no late user owns it
```

这里的 `user_data` 很关键。它不能只是一个裸指针，更不能指向可能已经释放的栈对象。它应该能定位稳定的 request record，例如 request id、attempt id 或对象池中的 generation handle。否则取消、超时和迟到 completion 会把已经

复用的 buffer 或 request 错认成当前请求。底层 API 越异步，身份和生命周期越不能靠调用栈保存。

注册 buffer 购买少拷贝，也购买更严格的所有权 Registered buffer 可以减少页固定、映射和检查开销，也更接近零拷贝路径。但注册 buffer 在 I/O 完成前不能被上层复用，取消也不保证设备或内核立刻停止访问。Buffer pool 因此必须有状态：Free、Filling、Submitted、Completed、Committing、Reclaimable。若只有一个 `std::vector<char>` 被多个请求轮流写，异步完成顺序稍一变化，就会把 block 内容、checksum 和 manifest 身份混在一起。

状态	谁拥有 buffer	允许动作
Free	buffer pool	分配给新的写请求
Filling	compute task	写入 block 内容，尚未提交 I/O
Submitted	kernel/device path	上层不能复用，等待 CQE 或取消结果
Completed	completion loop	检查短写、错误和字节范围
Committing	manifest task	等待业务提交或拒绝
Reclaimable	buffer pool	清理后回收，generation 前进

这张状态表比“使用 `io_uring` 会更快”重要得多。没有它，性能优化会把生命周期边界压扁；有了它，底层从 worker thread、`epoll`、`io_uring` 到直接 I/O 的替换，都只是完成来源改变，业务语义不变。

工程案例

Kafka、RocksDB 和 group commit Kafka、RocksDB 等系统都大量使用批处理、顺序写、后台 flush、日志或 manifest 记录提交事实。它们的设计说明：高吞吐存储系统不是每条记录都立即同步到最终结构，而是把数据面和控制面分开，用日志、manifest、sequence number、flush 和 compaction 管理可见性。

Compute Systems Engine 借鉴的是原则：大数据块可以提前流式写入，真正需要强一致提交的是小的 manifest 或 commit record；后台清理和恢复扫描必须以 commit record 为权威，不以目录扫描为权威。

工程案例

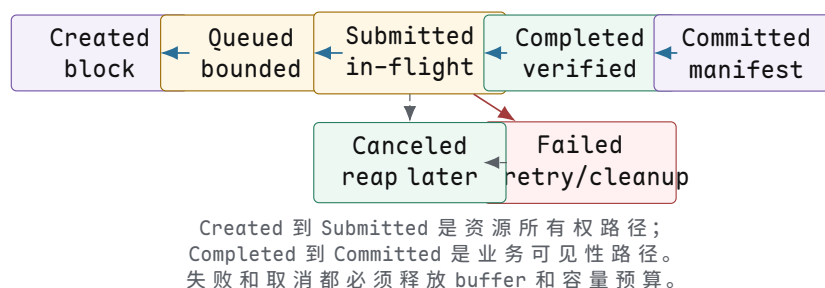
TCP 背压和应用层背压 TCP 接收端慢会让接收窗口缩小，本端发送最终阻塞或返回 `EAGAIN`。这是字节流层面的背压。应用层仍要有自己的背压，因为 TCP 不知道业务优先级、deadline、幂等、内存预算和队列语义。Reduce 端忙时，应用可以返回 busy 或降低 shuffle 拉取窗口；只依赖 TCP buffer，反馈太晚且语义太粗。

第 18 章分布式 shuffle 会把这里的单节点 write queue 扩展成远端 in-flight bytes、per-worker window 和 busy reply。概念相同，尺度变大。

18.8 深度案例：异步写出 Shuffle Block

把 partition manifest、任务运行时和 I/O 放到一起，得到一个可恢复写出管线。Compute task 生成 `OutputBlock`，I/O stage 写临时文件，completion 成功后触发 commit task，commit task 更新 manifest。Reduce 只读取 manifest 中已提交的 block。这个结构看起来比“直接写文件”复杂，但它能解释短写、失败、迟到 completion、重复 attempt、崩溃恢复和背压。

异步写出管线的状态机



图示：本图要证明的命题是：异步写出必须同时管理资源状态和业务提交状态，不能用一个成功布尔值替代。

第一版：同步 reliable write reference 先写同步 reference：创建临时文件，可靠写循环推进 offset，完成后计算 checksum，执行项目承诺的同步策略，然后返回候选 block。这个版本可能慢，但它固定短写、错误和文件状态语义。异步版本必须和它输出同样的 manifest 结果。

第二版：I/O worker 和 completion queue 把 reliable write 放到 I/O worker，计算线程提交 WriteRequest 后继续处理 CPU work。Completion queue 返回 CompletionRecord。Completion loop 只更新状态、归还 buffer、触发后续 task，不在 completion 线程里做重 CPU 计算。若 completion loop 自己被 CPU 工作拖慢，已经完成的 I/O 也会迟迟不被业务层看到。

第三版：背压闭环 为 write queue 和 buffer pool 设置 item、byte、高低水位。高水位时，reduce 暂停生成新 block 或降低 batch；reduce 队列满时，partition 减速；partition 减速后，map 输出缓冲减少；最终 reader 停止读取更多输入。报告要画出这条背压传播链，不能只说“队列满了会等”。

第四版：迟到 completion 和 attempt 请求超时后，driver 可以创建新 attempt。旧 attempt 后来完成，只能写自己的临时文件并进入 late/reaped 状态。Commit task 检查当前 task attempt 和 manifest，只有获胜 attempt 能提交。这个结构和第 16 章“成功响应丢失后重试”的问题完全同构。

18.9 实现流程：把 I/O 代码拆成可分析状态

I/O 章节不需要堆很多外部实验。真正关键的是把一段看似普通的写文件代码，拆成内核路径、缓冲区所有权、完成事件和业务提交四层。第一版实现可以很小：同步可靠写作为 reference，一个有界队列，一个 writer 线程，一个 completion 处理函数，一个 commit gate。只要这四层清楚，后续把 writer 线程替换成 io_uring 或直接 I/O，都不会改变语义。

从同步写循环推导异步状态 同步版本的核心是 offset 推进。它把系统调用返回值转成请求状态，而不是把正返回值直接当成功。

Listing 18.45 可靠写循环的最小语义

```

1 WriteResult write_all(int fd, std::span<const std::byte> bytes) {
2     std::uint64_t offset = 0;
3     while (offset < bytes.size()) {
4         const ssize_t n = ::write(fd, bytes.data() + offset,
5                                 bytes.size() - offset);
6         if (n > 0) {
7             offset += static_cast<std::uint64_t>(n);
8             continue;
9         }
10    }
11 }
```



```

9     }
10    if (n < 0 && errno == EINTR) {
11        continue;
12    }
13    return WriteResult::failed(offset, errno);
14 }
15 return WriteResult::completed(offset);
16 }

```

这段代码里已经包含异步版本需要的状态：`offset`、剩余字节、可重试错误、终端错误和完成字节数。异步化不是把这段循环扔给线程池，而是把调用栈里的局部变量搬到 `WriteRequestRecord` 中。每次 `completion` 回来，都相当于同步循环执行了一次系统调用，然后根据返回值决定重新提交、失败还是进入校验。

从代码走到内核路径 `write` 进入内核后，普通文件常先走 `page cache`。用户态字节被复制或引用到内核维护的页中，页被标记为 `dirty`，真正设备写回可能稍后发生。若使用 `direct I/O` 或注册 `buffer`，复制路径和页缓存路径会改变，但“提交请求、等待完成、确认业务生效”这三层仍然存在。代码分析时要按顺序问四个问题：系统调用是否接受请求，接受了多少字节；这些字节现在归谁所有；`completion` 或同步返回证明了哪一层完成；业务是否已经允许 `reduce` 读取这份输出。

只保留三个验证点 验证不需要铺成大矩阵，保留三个点即可。第一，短写：每次只写一部分字节，确认 `offset` 推进和最终 `checksum`。第二，迟到 `completion`：旧 `attempt` 完成后不能覆盖已经提交的新 `attempt`。第三，背压：`writer` 停住时，上游不能无限分配 `buffer`。这三个点分别覆盖字节状态、业务状态和资源状态，足够支撑本章主线。

18.10 完整案例：一次 `ShuffleBlock` 写出怎样变成提交事实

把 I/O 讲清楚，不能只讲 `write`、`epoll` 或 `io_uring` 的 API。必须沿着一块 `ShuffleBlock` 的生命周期走完：计算线程生成字节，`buffer pool` 转移所有权，I/O 层把请求提交给内核，内核可能只接受一部分字节，`page cache` 或设备稍后完成，`completion` 回到用户态，校验通过后才生成 `manifest` 候选，最后提交逻辑决定这个 `attempt` 是否生效。每一步都可能成功、失败、迟到或被取消。

T0：计算完成不等于可以释放 `buffer` `Reduce` 或 `map-side combine` 生成 `block` 后，`buffer` 里有业务字节，但所有权仍在计算阶段。提交写请求时，`buffer` 所有权转给 I/O 管线；从这个时刻开始，计算线程不能复用或修改 `buffer`，直到 `completion` 明确归还。若计算线程为了省内存提前复用 `buffer`，底层写出可能把后续内容写进当前 `block`，`checksum` 会不稳定，故障通常很难复现。

Listing 18.46 `buffer` 所有权转移账本

```

compute owns buffer:
    fill bytes
    compute logical checksum

submit write:
    request owns buffer
    compute must not mutate bytes

completion:
    success -> commit candidate owns metadata, pool may reclaim buffer
    failure -> retry owns buffer or retire buffer with error
    cancel -> ownership remains pending until completion or explicit reap

```

所有权账本是异步 I/O 的第一条线。没有它，后面的短写、取消和迟到 completion 都无法解释。

T1：系统调用返回值只是提交边界的一层 普通文件 `write` 返回正数时，只说明内核接受了这么多字节进入某条写路径；它可能少于请求字节数，也可能只是进入 page cache，并不等于设备已经持久化。异步接口更明显：`submit` 成功只表示请求进入提交队列或内核队列；`completion` 才表示内核对这次请求给出结果；业务 `commit` 仍要等待 `checksum`、`attempt` 和 `manifest` 判断。

Listing 18.47 写出成功的层级

```
submit accepted:
    kernel or I/O worker accepted request

bytes copied or referenced:
    some bytes entered kernel/page-cache/device path

completion success:
    request finished according to API result

durability boundary:
    fsync/fdatasync or project-defined weaker policy completed

manifest commit:
    logical task attempt is accepted as business fact
```

这五层经常被混成一个 `ok=true`。混淆后，系统会在响应丢失、崩溃恢复、短写和重复 `attempt` 中产生重复输出或坏 block。

T2：短写把循环拆成状态机 同步 `reference` 可以在一个循环里处理短写：写了多少推进多少 `offset`，剩余继续写。异步版本不能把循环藏在调用栈里，因为每次 `completion` 才知道推进了多少。于是 `WriteRequest` 必须保存 `offset`、`remaining`、`attempt id`、`buffer id` 和错误状态。一次 `partial completion` 后，请求重新进入 `Submitted` 或 `Queued`；`fatal error` 后进入 `Failed`；完全写完后进入 `Completed`。

Listing 18.48 异步短写状态推进

```
completion(result = 4096 bytes):
    request.offset += 4096
    request.remaining -= 4096
    if request.remaining > 0:
        resubmit request
    else:
        verify checksum and move to Completed
```

如果 `completion` 只返回“成功”而不记录写入字节数，短写会被误提交。测试必须故意让每次只写少量字节，并断言最终 `block checksum` 与 `reference` 一致。

T3：Page cache 会把快假象放大 写入普通文件时，内核可能先把数据放入 page cache，标记 `dirty`，稍后 `writeback`。短 benchmark 只测到用户态到 page cache 的速度，看起来非常快；长时间运行或内存压力上来后，`dirty page` 达到阈值，写回开始限速，`write`、`fsync` 或内核后台线程都可能成为等待点。若中间队列无界，前半段吞吐会掩盖后半段压力，最终表现为内存暴涨和长尾。

因此写出报告必须分开：用户态排队时间、提交系统调用时间、`completion` 时

间、fsync 或 commit 时间、队列水位、dirty/writeback 现象的可观察指标。没有这些字段时，“磁盘慢”只是猜测。

T4: Manifest 才能拒绝迟到成功 假设 attempt 0 超时，driver 重试 attempt 1。attempt 1 先完成并提交 manifest；attempt 0 后来 completion 成功。此时 attempt 0 的字节可能完整，checksum 也正确，但它已经不能成为业务事实。Commit 逻辑必须按 logical task 查询 manifest：若已有 accepted attempt，迟到 attempt 只能进入 LateAttempt，临时文件可稍后清理。

Listing 18.49 迟到 completion 的提交判定

```
completion for attempt 0:
  block checksum is valid
  logical task = split 7
  manifest already accepts attempt 1 for split 7
  decision = LateAttempt
  do not append manifest
  release or mark temp block for cleanup
```

这个例子说明 completion success 不等于 commit success。底层 I/O 只知道请求完成；业务层必须知道 logical task、attempt、epoch 和 manifest 状态。

T5: 背压从 commit 失败向上游传播 如果 manifest commit 变慢，completion queue 会堆积；completion queue 堆积后 buffer 归还变慢；buffer pool 变少后 write queue 不能接受新请求；write queue 满后 reduce 暂停生成新 block；reduce 暂停后 partition 和 map 也应减速。背压不是某个队列的局部行为，而是一条从下游提交点传到输入源头的控制链。

Listing 18.50 背压传播链

```
manifest commit slow
-> completion loop cannot retire requests fast enough
-> buffer pool free bytes drop below low watermark
-> write queue rejects or blocks new ShuffleBlock
-> reduce tasks wait on output capacity
-> upstream partition stops producing more bytes
-> reader eventually slows down
```

若其中某一段使用无界队列，背压链就断了。系统短时间看似吞吐更高，实际上只是把压力存成内存、临时文件或未提交 completion。

T6: 关闭时选择 drain 还是 cancel I/O 管线关闭不能只设置一个 bool。Drain 表示不再接收新 block，但已经接收的请求继续写完、完成校验和提交；Cancel 表示尽力取消尚未提交的请求，正在内核或设备中的请求可能仍会返回 completion。两种模式都必须处理迟到 completion 和 buffer 归还。若析构直接销毁队列和 buffer pool，未完成请求回来时就可能访问已释放内存。

Listing 18.51 I/O 关闭模式

```
drain:
  reject new writes
  wait queued/submitted requests reach terminal state
  commit or fail each request
  release all buffers
```

```
cancel:
    reject new writes
    request cancellation for queued/submitted requests
    accept late completions as cleanup events
    release buffers only after ownership is known
```

关闭报告应记录 close mode、queued at close、submitted at close、completed after close、canceled、late completion 和 unreacted requests。这样 shutdown 卡住或丢数据时，能知道是等设备、等 manifest、等 buffer，还是有请求状态丢失。

T7: 可靠写入的最小恢复扫描 进程重启后，恢复程序会看到临时目录、manifest 和可能的 checkpoint。正确顺序仍然是先读 manifest，得到已提交 block；再扫描临时目录，把文件分为 accepted、candidate、orphan、corrupt。Candidate 不能直接进入结果，必须回到 attempt/commit 表验证。Corrupt 文件可以删除或隔离；orphan 文件要等审计策略允许再清理。

Listing 18.52 I/O 恢复扫描分类

```
for each temp file:
    if referenced by manifest and checksum matches:
        classify AcceptedData
    elif has valid block metadata but no accepted attempt:
        classify CandidateOrLateAttempt
    elif checksum mismatch:
        classify Corrupt
    else:
        classify Orphan
```

这条恢复路径把第 15 章和第 18 章连接起来。I/O 层负责不把坏字节伪装成完成；分布式提交层负责决定哪个 attempt 生效。两层都不能用“文件存在”替代 manifest。

T8: I/O 到 manifest 的状态表必须能重放 前面的生命周期沿一个 ShuffleBlock 讲清了 submit、completion 和 commit 的边界，但实现时还缺一张能落到代码里的状态表。异步 I/O 的危险点恰好在这里：事件从多个方向进入系统，计算线程提交请求，writer 线程发起系统调用，completion loop 收到结果，commit 线程更新 manifest，关闭线程要求 cancel 或 drain，恢复程序又从磁盘扫描出旧文件。若每条路径各自改几个布尔字段，状态很快就无法重放。

最小实现应把 WriteRequest 写成一条可持久化或可审计的记录。它不一定每次状态变化都同步落盘，但日志、trace 和恢复表必须能重建同一条路径。请求身份至少包含 request_id、logical_task、attempt、block_id 和 buffer_id；进度包含 bytes_done、remaining、checksum 和最后一次错误；业务身份包含 stage、partition、route epoch、manifest generation 候选和 deadline。这样 completion 回来时，系统先查请求表，而不是根据文件名猜它属于谁。

Listing 18.53 可重放写请求记录的示例形态

```
1 enum class WriteRequestState : std::uint8_t {
2     Created,
3     Queued,
4     Submitted,
5     PartiallyCompleted,
6     Completed,
```

```

7   Verified,
8   Durable,
9   Candidate,
10  Committed,
11  Failed,
12  Canceled,
13  Reaped,
14 };
15
16 struct WriteRequestRecord {
17     std::uint64_t request_id = 0;
18     std::uint64_t logical_task = 0;
19     std::uint32_t attempt = 0;
20     std::uint64_t block_id = 0;
21     std::uint64_t buffer_id = 0;
22     std::uint32_t stage_id = 0;
23     std::uint32_t partition_id = 0;
24     std::uint64_t route_epoch = 0;
25     std::uint64_t manifest_generation = 0;
26     std::uint64_t bytes_total = 0;
27     std::uint64_t bytes_done = 0;
28     std::uint64_t checksum = 0;
29     std::uint64_t deadline_ns = 0;
30     WriteRequestState state = WriteRequestState::Created;
31 };

```

状态转移要写成有限集合，而不是任意赋值。`Created` 只能进入 `Queued` 或 `Failed`；`Queued` 可进入 `Submitted`、`Canceled` 或 `Failed`；`Submitted` 收到短写进入 `PartiallyCompleted` 后重新提交，写完进入 `Completed`，错误进入 `Failed`，取消请求则进入 `Canceled` 但等待 `reaped`；`Completed` 通过 checksum 后进入 `Verified`；满足项目承诺的持久化边界后进入 `Durable`；只有 commit 线程检查 `attempt`、`epoch` 和 `manifest` 后，才从 `Candidate` 进入 `Committed`。所有未被接受的完成结果最后都要进入 `Reaped`，释放 `buffer`、容量预算和临时文件引用。

Listing 18.54 状态转移不是布尔成功

```

Created -> Queued -> Submitted
Submitted -> PartiallyCompleted -> Submitted
Submitted -> Completed -> Verified -> Durable -> Candidate
Candidate -> Committed
Candidate -> Reaped           # late attempt or stale route
Submitted -> Failed           # fatal I/O error
Submitted -> Canceled -> Reaped

```

这张表的价值在崩溃恢复时才完全显现。恢复不能从临时目录开始，因为目录只告诉系统“某些字节存在”。恢复顺序应先读 `manifest`，得到业务上已经接受的 `block`；再读 `completion log`、`request table` 或 `checkpoint`，恢复每个 `request` 的最后已知状态；最后扫描临时目录和最终目录，把文件分类为 `accepted`、`candidate`、`late`、`orphan`、`corrupt`。分类规则必须能解释所有中间状态：`Committed` 且 `checksum` 匹配的是 `accepted`；`Durable` 或 `Candidate` 但 `manifest` 未接受的是候选，需重新走 `commit` 判定；旧 `attempt` 完整但已有新 `attempt` `accepted`，是 `late`；无请求记录或元数据不完整是 `orphan`；`checksum` 不匹配是 `corrupt`。

Listing 18.55 恢复重放的顺序

```

recover:
  load manifest and accepted attempt table
  load request table or completion log
  scan temp and final directories
  for each discovered file:
    join by block_id, attempt, checksum
    classify accepted, candidate, late, orphan, or corrupt
  replay candidate commit only through manifest gate
  reap stale and orphan files after cleanup policy allows

```

注意最后一行：候选文件不能直接变成结果。恢复程序和在线 commit 线程必须调用同一个 manifest gate。若在线路径检查 attempt 和 route epoch，恢复路径却按“文件完整且较新”接受，系统仍会在重启后分叉。项目中可以把这个 gate 做成小函数，例如 `try_commit_manifest_candidate(record, ownership, manifest)`，让在线 completion、重试和恢复扫描都走同一条判断。

背压也应使用同一张状态表计算债务。粗略的队列长度只能看到 `Queued`；真正的债务还包括 `Submitted` 的 in-flight bytes、`Completed` 到 `Candidate` 之间的未提交 bytes、`late/orphan` 临时文件和最老未提交时间。推荐维护一张 `IoDebtLedger`：`queued_bytes` 限制生产速度，`inflight_bytes` 限制 buffer pool 和取消成本，`completed_uncommitted_bytes` 暴露 manifest 瓶颈，`orphan_temp_bytes` 暴露清理和恢复窗口，`oldest_uncommitted_age` 暴露尾部债务。

Listing 18.56 I/O 债务账本的示例形态

```

1 struct IoDebtLedger {
2   std::uint64_t queued_bytes = 0;
3   std::uint64_t inflight_bytes = 0;
4   std::uint64_t completed_uncommitted_bytes = 0;
5   std::uint64_t orphan_temp_bytes = 0;
6   std::uint64_t retry_backlog = 0;
7   std::uint64_t oldest_uncommitted_age_ns = 0;
8 };

```

这几个字段直接决定背压打到哪里。`queued_bytes` 超过高水位，compute 暂停生成新 block；`inflight_bytes` 长期不降，writer 或设备路径需要降速、拆小 block 或增加 completion 能力；`completed_uncommitted_bytes` 增长，说明 manifest gate 或 group commit 是瓶颈，继续增加 writer 只会制造更多候选；`orphan_temp_bytes` 增长，说明恢复窗口正在扩大，清理任务要获得预算；`oldest_uncommitted_age` 超过 deadline，driver 应该停止制造新 attempt，先处理提交或失败传播。

至此，异步 I/O 的核心不再是某个 API，而是一张能被在线路径、关闭路径和恢复路径共同解释的账本。只要账本能重放，短写、迟到 completion、崩溃扫描和背压都能落到同一个模型；账本不能重放，系统迟早会退化成“日志里说成功、目录里也有文件、但 manifest 不知道该信谁”的状态。

I/O 指标要同时看吞吐和债务 异步 I/O 最容易制造“吞吐看起来高，债务不断增加”的假象。报告除了 bytes/s，还要记录 queued bytes、dirty bytes 近似指标、in-flight requests、oldest request age、oldest uncommitted block age、buffer pool free bytes、retry backlog 和 orphan temp bytes。吞吐表示当前流量，债务表示未来必须偿还的工作。若吞吐高但 oldest uncommitted block age 持续增长，系统实际上已经失去稳态。

Listing 18.57 I/O 债务指标

```

write_queue.bytes_current
write_queue.bytes_high_water
inflight.requests
oldest_request_age_ms
oldest_uncommitted_block_age_ms
buffer_pool.free_bytes
retry.backlog
temp.orphan_bytes

```

这些字段能解释背压是否真的闭环。没有债务指标时，无界队列和无限重试会伪装成高吞吐。

背压策略要写业务语义 当下游慢时，上游可以阻塞、降级、丢弃、合并、采样、失败或扩容。批处理 shuffle 通常选择阻塞和限速，因为丢弃会破坏结果；监控 telemetry 可能选择采样或丢弃；在线请求可能选择失败并返回 overloaded。背压不是单一算法，而是业务语义在资源不足时的表达。Compute Systems Engine 的默认策略是：计算结果不丢弃，任务可等待，重试有预算，错误要结构化报告。

这条默认策略会降低某些压力场景下的瞬时吞吐，但它换来结果不丢、内存有界和恢复可解释。若未来某条 telemetry 支路允许丢弃，也必须单独声明，不能把可丢弃策略混进 shuffle 输出路径。

背压还要进入用户可见报告。一次运行如果因为写出慢而主动限速，应明确写出限速次数、限速持续时间、最高水位、最低可用 buffer 和最终恢复时间。这样吞吐下降时，结论不是“程序变慢”，而是“系统按合同保护了内存和提交事实”，并能继续定位下游真正的慢点。没有这些字段，限速会被误判成随机抖动，甚至被后续优化错误地删除。

关键观测点本身就是 I/O 合同的一部分：没有被记录的压力，迟早会被当作不存在。

18.11 稳态判断：吞吐、债务和恢复窗口

异步 I/O 最容易制造一种假稳态：每秒处理的输入看起来很高，但未提交 block、dirty page、in-flight request、orphan candidate 和 retry backlog 都在增长。真正的稳态不是“当前吞吐高”，而是“在给定输入速率下，队列、水位、债务和恢复窗口不会持续增长”。如果系统只能靠不断增加内存、临时文件和未完成请求维持吞吐，它已经失去稳态。

吞吐和债务要同时画 吞吐曲线告诉当前处理速度，债务曲线告诉未来还欠多少工作。写出系统至少有四类债务：已生成但未提交的 block，已提交给 I/O 但未 completion 的 request，已 completion 但未 manifest 的候选，已过期或迟到但未清理的临时文件。任何一类债务持续上升，最终都会变成内存、磁盘、恢复时间或尾延迟问题。

债务类型	含义	失控后果
queued bytes	上游已生产，下游尚未接收	内存上涨，背压延迟变长
in-flight requests	已提交 I/O，尚未 completion	buffer pool 枯竭，取消复杂
uncommitted blocks	字节完成，业务未提交	恢复扫描变慢，重复 attempt 增多
orphan temp bytes	未被 manifest 引用的候选文件	磁盘占用，恢复审计变重
retry backlog	等待重试或退避的请求	重试风暴，deadline 过期

恢复窗口也是资源 崩溃恢复要扫描 manifest、checkpoint、临时目录和候选 block。未提交候选越多，恢复窗口越长；orphan 文件越多，恢复时越难区分 late attempt、corrupt block 和清理失败。一个系统即使运行时内存稳定，如果每小时留下数百万个 orphan temp file，恢复时间也会不可接受。RunReport 应记录 recovery scan count、orphan bytes、candidate bytes 和 cleanup lag。恢复窗口和内存一样，是资源预算。

Listing 18.58 恢复窗口指标

```
recovery.manifest_entries
recovery.temp_files_scanned
recovery.accepted_blocks
recovery.candidate_blocks
recovery.orphan_blocks
recovery.corrupt_blocks
recovery.scan_elapsed_ms
cleanup.oldest_orphan_age_ms
```

稳态判读 稳态分析不必变成复杂实验表。只要沿着状态表看债务是否回落：writer 变慢时 queued bytes 上升，buffer pool 下降，上游开始等待；writer 恢复后 queued bytes 和 completed-uncommitted bytes 应下降，oldest uncommitted age 不应无限增长。若吞吐曲线好看但债务不回落，系统只是把压力藏到了内存、临时文件或恢复窗口里。

Listing 18.59 稳态判读的最小链条

```
writer slow
-> queued bytes rises
-> buffer pool free bytes drops
-> upstream waits instead of allocating more buffers

writer recovers
-> queued bytes drains
-> completed-uncommitted bytes drains
-> oldest uncommitted age returns below budget
```

债务无法回落时的修复方向 如果 queued bytes 不回落，说明下游平均服务能力低于输入速率，需要降低输入、增加 writer、减少输出字节或改变提交策略。若 in-flight request 不回落，可能是 completion loop 慢、设备慢或请求太大。若

uncommitted blocks 不回落, manifest commit 是瓶颈。若 orphan bytes 不回落, 清理和恢复审计路径不足。不同债务对应不同修复, 不能都归因于“磁盘慢”。

稳态报告要写保护动作 背压真正生效时, 吞吐曲线可能主动下降。报告必须把保护动作写出来: 暂停读取多少次, reduce 等待 output capacity 多久, buffer pool 低水位持续多久, manifest group commit 批了多少条。否则后续优化可能把这些保护动作误删, 以为自己提高了吞吐。系统的目标不是始终满速, 而是在预算内持续正确地前进。

18.12 I/O 可靠性门禁

一、是否分清 submit、complete、commit Submit 成功只表示请求被接受; completion 成功只表示底层 I/O 有结果; commit 才表示业务结果可见。任何把三者混成一个布尔值的设计都不合格。

二、buffer 生命周期是否有唯一所有者 Filling、submitted、completed、free 每个状态都要有唯一所有者。失败、取消、迟到 completion 都要归还或 retire buffer, 不能泄漏或双重归还。

三、背压是否传到源头 write queue 高水位必须影响 compute、parse 或 read。若中间某段仍无限生产, 背压链断了。

四、可靠写入是否有崩溃点矩阵 只写“使用临时文件和 fsync”不够。每个崩溃位置都要有恢复动作, manifest 必须是权威入口。

五、错误是否结构化分类 Retryable、fatal、canceled、timed out、corrupt、backpressure 不应都变成字符串日志。driver 要能根据错误类型做不同决策。

六、是否连接分布式边界 CompletionRecord 对应 RPC reply; attempt 对应远端 task attempt; manifest 对应 checkpoint 和 shuffle 元数据; in-flight bytes 对应分布式流控窗口。若这些对象不能复用, 说明边界还不稳。

18.13 习题

习题与作业

习题一: 设计一个异步写请求状态机, 至少包含 Created、Queued、Submitted、PartialWritten、Completed、Failed、Canceled、Committed、Reaped。说明每个状态允许哪些转移。

习题二: 给一个使用 buffer pool 的异步写出模块画所有权图。说明 submit 成功、submit 失败、completion 成功、completion 失败、取消和关闭时 buffer 如何归还。

习题三: 写一个短写处理流程。要求维护 offset、remaining、可重试错误和 fatal 错误, 并说明异步版本如何把循环拆成 completion 事件。

习题四: 设计高低水位线实验。比较无界队列、有界队列、有界队列加水位线三种方案的内存峰值、吞吐和 p99 延迟。

习题五: 为可靠写入写崩溃点矩阵。状态包括写临时文件前、写中、校验后、rename 后、manifest 前、manifest 后。每个状态写恢复动作。

习题六: 解释 epoll ready 和 io_uring completion 的差别。为什么 ready 不等于业务消息完整, completion 也不等于业务提交?

习题七: 阅读一个工业 I/O 或存储系统片段, 按“约束、核心机制、购买能力、成本、迁移到本项目的方式”写报告。

工程案例

知识点小案例: 异步提交成功不等于业务提交成功。特例是 把一个 block 写到临时文件。提交异步写请求成功, 只说明内核接受了请求; completion 成功, 只说明某次写入完成; 业务提交还需要 checksum、rename 或

manifest 记录。若把 buffer 在 completion 前归还池子，设备可能还在使用旧内容；若没有高低水位线，in-flight bytes 会把内存打爆。由此推出规律：异步 I/O 必须拆开 submit、completion、validation、commit 和 reclaim。工程上每个请求都要有状态机、owner、offset、remaining、bytes in flight 和取消/关闭规则。

第零部分

分布式计算与工程边界

第 19 章

分布式计算模型

分布式系统不应该从术语表开始。把 RPC、CAP、租约、逻辑时钟、幂等、背压按字典顺序列出来，很快会背几个名字，却仍然不知道程序为什么会错。本章从一个会在小样例里通过、在故障注入里立刻出错的代码开始：Driver 把 task 7 发给 worker，worker 已经把结果写到临时 block，成功响应在返回途中丢失。Driver 等不到响应，按本地 timeout 重新派发同一个 task。几秒后，旧响应和新响应先后到达，两个物理执行都声称自己完成了 task 7。

这不是“网络不可靠”这种空话，而是第 15 章 completion 模型在网络边界上的放大。提交请求不等于远端执行，远端执行不等于响应到达，响应到达不等于业务提交。调用一旦跨过线程、进程或机器边界，原来由调用栈、共享内存和同步返回值提供的保证，就必须由消息身份、attempt、deadline、epoch、状态表、提交表、trace 和 manifest 重新写出来。

Listing 19.1 反例：把远端调用当成本地函数

```
1 enum class RemoteStatus : std::uint16_t {
2     Ok,
3     Timeout,
4     Fatal,
5 };
6
7 struct RemoteReply {
8     RemoteStatus status = RemoteStatus::Fatal;
9     std::uint32_t task_id = 0;
10    std::uint64_t block_id = 0;
11    std::uint64_t checksum = 0;
12 };
13
14 bool run_task_once(std::uint32_t task_id) {
15     const RemoteReply reply = run_remote(task_id);
16     if (reply.status == RemoteStatus::Timeout) {
17         mark_failed(task_id);
18         return false;
19     }
20     if (reply.status != RemoteStatus::Ok) {
21         mark_failed(task_id);
22         return false;
23     }
24     append_manifest(reply.task_id, reply.block_id, reply.checksum);
25     return true;
26 }
```

这段代码至少有四个漏洞。第一，timeout 是本地等待结束，不是远端失败事实。第二，重试不是撤销第一次执行后再试一次，而是制造新的物理 attempt。第三，Ok 只说明某个远端执行声称成功，不说明它属于当前 attempt、当前 Driver epoch、当前 deadline，也不说明它有权进入 manifest。第四，manifest 写入前后崩溃的恢复语义完全不同，内存状态不能成为权威事实。

本章沿几条边界展开：远端 timeout 只能说明 unknown；logical task、physical attempt、request、reply、epoch、deadline 和 commit 必须分开；响应进入状态机前要带足消息身份；状态表、提交表和事件日志要处理重试、迟到响应、重复响应、Driver 重启和旧 epoch。先用可控 MessageBus 复现分布式错误，再迁移到多进程和真实网络。

19.1 从响应丢失开始

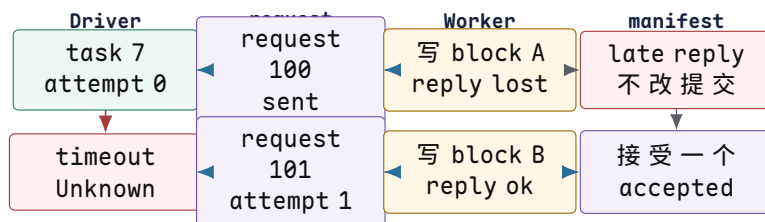
正常路径很诱人。Driver 取出 split 7，调用 `run_remote(split)`；worker 读取输入、统计 key、写出 block，再返回路径和 checksum；Driver 追加 manifest，reduce 后续读取这个 block。小样例能跑，日志也干净。很多坏设计都在正常路径里显得合理，因为还没有事件打破本地调用直觉。

现在只改变一件事：worker 业务代码全部成功，成功响应在返回途中丢失。Driver 看到 timeout，于是把 task 标记为 failed。这里错误已经发生。请求可能根本没有到达；也可能到达 worker 队列但尚未执行；可能执行完成但响应丢失；可能正在发送响应；也可能 worker 崩溃。timeout 把这些物理情况都压成一个本地事实：“我没有在规定时间内看到结果”。它不能证明远端失败。

再让 Driver 重试。第一次执行可能已经写出了 block A，第二次执行又写出 block B。若程序让 reduce 扫描输出目录，就会把同一个 split 统计两次；若 `append_manifest` 无条件追加，也会制造重复提交。提交表不是凭空出现的概念，它是从“执行可以重复，但外部结果只能生效一次”这个缺口里长出来的。

最后让旧响应迟到。响应里只有 task id、block id 和 checksum，Driver 无法判断它属于第几次物理执行，无法判断它是否来自重启前的 Driver，无法判断它是否已经过了 deadline，也无法判断它是否重复。于是 request id、attempt、epoch、deadline 和 trace 不是装饰字段，而是响应进入状态机前必须携带的证据。缺一个字段，就会有一个故障位置只能靠猜。

响应丢失后，执行可以重复，提交只能唯一



Driver 可以创建新的 attempt，但 manifest 只能接受一个 attempt。旧响应迟到只能进入 trace 或重复指标，不能重开终态。

图示：本图要证明的命题是：分布式 exactly-once 的可落地边界不是“只执行一次”，而是“外部可见提交只生效一次”。

本地调用直觉为什么失效 本地函数的失败边界比较窄：没调用、抛异常、返回错误，通常能在同一进程里用栈和断点观察。远端调用被拆到多个位置：客户端序列化、发送队列、网络、服务端接收队列、业务线程、输出写入、响应队列、客户端 completion。每一段都可能排队，也都可能在“对方已经做了事”以后才失败。

RPC 库可以隐藏连接池、编码、线程和重试样板，但不能消除不确定性。调用者必须明确写出：请求身份是什么，timeout 后进入什么状态，是否允许重试，重复请求如何处理，迟到响应如何处理，远端输出如何提交，取消是否影响远端提交。没有这些答案，程序只是把分布式错误藏在库调用后面。

本地栈消失后，状态表接管 本地函数调用有几个隐含保证：调用者知道自己调

用了哪个函数；返回值天然属于这次调用；栈帧保存局部状态；异常沿调用链传播；函数返回后局部对象销毁。远端调用把这些保证拆碎了。请求和响应不再共享栈帧，异常不能直接跨网络传播，响应可能迟到或重复，远端对象生命周期与本地变量无关。于是系统必须把原本藏在调用栈里的事实外化成状态表。

本地调用隐含事实	远端边界上的缺口	替代结构
返回值属于当前调用	响应可能迟到、重复、乱序	request id、attempt、状态表去重
异常沿栈传播	远端失败只能编码成消息	RpcStatus、error code、trace id
局部变量随栈存活	远端执行跨线程和进程	request table、timeout table、buffer 所有权
函数返回表示结束	timeout 后远端可能仍在执行	Unknown 状态、deadline、取消协议
调用者独占控制流	多个 Driver 或 epoch 可能并存	epoch、fencing token、manifest commit

这张表给分布式系统一个底层入口。消息身份、状态表和 manifest 不是企业级样板，而是调用栈不再存在后的替代物。每个远端交互都应能回答：这条消息对应哪个状态表 entry，它能把状态从哪里转到哪里，失败时状态是否仍可恢复。

状态表要能重放，而不是只服务在线逻辑 分布式故障最难的地方，是事故发生时在线内存已经不可信：Driver 可能重启，worker 可能丢失日志，消息可能重复。状态表若只在内存里服务调度，恢复时仍然靠猜。可靠做法是把关键转移写成可重放事件，例如 `AttemptStarted`、`ReplyReceived`、`BlockVerified`、`ManifestCommitted`、`LateReplyRejected`。重启后按事件重建状态，得到与崩溃前等价的提交事实。

Listing 19.2 远端 attempt 的可重放事件

```
AttemptStarted(task=T2, attempt=A0, worker=W0, request=R10)
ReplyTimeout(task=T2, attempt=A0, request=R10)
AttemptStarted(task=T2, attempt=A1, worker=W1, request=R11)
BlockVerified(task=T2, attempt=A1, block=B2a1, checksum=ok)
ManifestCommitted(task=T2, accepted_attempt=A1, block=B2a1)
LateReplyRejected(task=T2, attempt=A0, reason=AlreadyCommitted)
```

事件日志不一定要像数据库 WAL 那样复杂。小型项目可以先把事件写成追加文本或二进制记录，再用 checksum 和 generation 保护。关键是：恢复程序不应从目录和零散日志推理业务事实，而应从可重放状态转移得到 manifest 和 task table。

Timeout 是 Unknown 把 timeout 写成 failure，是分布式代码的第一个常见错误。Timeout 后可以重试、查询、降级或放弃，但不能凭本地 timer 推断远端事实。更稳的状态名应该是 `TimedOut` 或 `UnknownRemoteState`，而不是 `Failed`。Failed 应表示系统已经根据协议决定这个逻辑任务不能再自动成功。

Deadline 和 timeout 也要区分。Timeout 是某一层本地等待多久；deadline 是这次逻辑请求最晚还能被接受到什么时候。请求迟到但还在 deadline 内，可能仍可接收；请求已经过 deadline，即使远端稍后成功，也不能直接提交业务结果。第 15 章 I/O 迟到 completion 的状态机，在这里变成远端 reply 的状态机。

19.2 消息身份和协议边界

本地返回值靠调用栈归属，远端响应靠字段归属。消息必须携带足够证据，让接收方能判断它属于哪个逻辑任务、哪个物理 attempt、哪个 request、哪个 Driver epoch、哪个 deadline 和哪个 trace。字段看起来多，但每个字段都对应一个故障问题。

消息信封 消息信封不是把业务字段包一层好看外壳，而是协议边界的证据集合。示例版可以这样写：

Listing 19.3 消息信封的示例形态

```

1 constexpr std::uint16_t DISTRIBUTED_PROTOCOL_VERSION = 1;
2
3 enum class MessageKind : std::uint16_t {
4     TaskRequest,
5     TaskReply,
6     Busy,
7     Cancel,
8 };
9
10 enum class RpcStatus : std::uint16_t {
11     Ok,
12     Retryable,
13     Busy,
14     StaleEpoch,
15     DeadlineExpired,
16     InvalidRequest,
17     Fatal,
18 };
19
20 struct MessageEnvelope {
21     std::uint16_t protocol_version = DISTRIBUTED_PROTOCOL_VERSION;
22     MessageKind kind = MessageKind::TaskRequest;
23     RpcStatus status = RpcStatus::Ok;
24     std::uint32_t job_id = 0;
25     std::uint32_t stage_id = 0;
26     std::uint32_t task_id = 0;
27     std::uint32_t attempt = 0;
28     std::uint64_t request_id = 0;
29     std::uint64_t trace_id = 0;
30     std::uint32_t driver_epoch = 0;
31     std::uint64_t deadline_ns = 0;
32     std::uint32_t payload_bytes = 0;
33     std::uint64_t payload_checksum = 0;
34 };

```

这个结构不是 wire format。真实网络协议要明确大小端、变长字段、版本兼容、长度限制、校验位置和错误处理。Compute Systems Engine 可以在单进程 MessageBus 中复制这个结构；一旦进入 socket、文件或跨版本存储，就不能把 C++ 对象内存直接发送出去。

字段对应的失败问题 没有 `task_id`，响应不知道属于哪个逻辑工作单元；没有 `attempt`，旧 attempt 和当前 attempt 无法区分；没有 `request_id`，重复消息和迟到 completion 无法去重；没有 `driver_epoch`，Driver 重启或控制面切换后的旧响应可能污染新状态；没有 `deadline_ns`，过期请求仍可能被错误提交；没有 `trace_id`，跨组件日志无法串联。

字段	解决的问题	缺失后果
task id	逻辑工作单元身份	结果无法归属
attempt	物理执行身份	旧执行覆盖新执行
request id	一次消息交互身份	重复响应无法去重
epoch	控制面新旧身份	重启前事件污染新状态
deadline	结果是否还可接受	过期结果被提交
trace id	跨组件因果链	事故无法复盘

控制消息和数据消息分开 控制消息携带 task、attempt、deadline、状态和提交决定；数据消息携带 payload 或 block 引用。把大数据直接塞进控制消息，会让重试、日志、trace 和持久化都变重。高吞吐系统通常让数据面走批处理、分片和背压，控制面只传身份、校验和提交事实。Compute Systems Engine 中，worker 可以写临时 shuffle block，reply 只返回 block id、bytes 和 checksum；manifest 再决定是否接受。

部分读写和消息边界 进入多进程后，send 和 recv 不会按消息对象自然对齐。一次 recv 可能只有半个 header，一次 send 可能只写出部分字节。第 15 章短读短写在这里再次出现：协议要有 framing，例如固定 header 加 payload length；读取方先收完整 header，校验长度，再收完整 payload。若项目从 C++ 结构体复制直接跳到 socket，很快会在部分读写、大小端、对齐和版本上出错。

单进程 MessageBus 可以把一个 Message 对象直接放进队列，接收方 pop 出来的仍然是同一个结构化对象。迁移到 socket 后，这个假设消失。进程之间没有共享地址空间，指针没有意义；TCP 提供的是有序字节流，不提供“这一段正好是一条业务消息”的边界；操作系统的 send buffer 和 receive buffer 会按可用空间拆分和合并字节；连接关闭也不等于某个业务请求失败。于是协议必须自己定义 frame，并把 frame parser 当作核心状态机。

Listing 19.4 TCP 字节流可能呈现的接收形状

```
sender writes:
[message A frame][message B frame]

receiver may read:
[first 7 bytes of A header]
[rest of A header + part of A payload]
[rest of A payload + complete B frame]

or:
[A and B together in one recv]
```

这就是为什么 recv 返回一次成功不等于收到一条完整消息，send 返回成功也不等于对端业务已经处理。应用层需要保存解析状态：当前是否正在读 header，header 是否通过版本和长度检查，payload 还差多少字节，checksum 是否匹配，deadline 是否已经过期，连接关闭时当前半包如何分类。缺少这层状态，网络错误会直接污染业务状态机。

Frame header 是最小控制面 Frame header 至少要包含 magic、protocol version、message type、header length、payload length、request id、flags 和 checksum 或校验字段。Magic 用于快速拒绝明显错误流；version 用于兼容判断；type 决定 payload 的解释方式；length 让 parser 知道还要读多少字节；request id 把响应放回状态表；checksum 帮助发现截断、编码错误或内存破坏。字段越少，状态机越短，但也越难诊断；字段越多，兼容和校验成本越高。最小协议要在这两边平衡。

Listing 19.5 FrameHeader 的抽象字段

```

FrameHeader:
    magic
    protocol_version
    header_bytes
    payload_bytes
    message_type
    request_id
    attempt_id
    deadline_epoch_ms
    flags
    header_checksum
    payload_checksum

```

这些字段不能被业务 payload 随意覆盖。Parser 必须先验证 header 长度、payload 长度和版本，再分配 buffer 或进入 payload 读取。否则一个错误长度就能触发巨大分配、内存耗尽或状态机越界。网络边界上的第一条安全规则是：先验证控制字段，再相信 payload。

Partial write 也会改变背压 发送端同样不能假设一次 `write` 把完整 frame 放入内核。非阻塞 socket 可能只接受一部分字节，剩余部分要留在发送队列；阻塞 socket 可能在内核 buffer 满时停住当前线程；TLS 或压缩层还可能改变写入粒度。若 CPU worker 直接执行阻塞 write，它会变成第 9、14、15 章反复警告的 hidden blocking。正确做法是把待发送 frame 变成拥有偏移量的输出状态机，由 I/O loop 在 socket 可写时继续推进，并对队列水位施加背压。

Listing 19.6 发送侧状态机要保存已写偏移

```

OutboundFrame:
    connection_id
    request_id
    bytes
    bytes_written
    deadline
    retry_class
    completion_target

on socket writable:
    n = write(bytes + bytes_written)
    bytes_written += n
    if bytes_written == bytes.size:
        mark frame flushed to kernel buffer

```

`flushed to kernel buffer` 仍然不是业务成功。它只说明本进程把字节交给内核；对端是否收到、解析、执行、写出结果、提交 manifest，都需要后续响应和提交表证明。把这些事实分层，才能避免把 transport success 误当 application success。

连接事件不是业务事件 连接建立、断开、半关闭、TLS 握手失败、对端重置，都属于传输层事件。它们会影响 in-flight 请求的状态，但不能直接替代业务判断。连接断开时，已经发送但未收到响应的请求进入 Unknown；正在解析的半包进入 TruncatedFrame 或 ConnectionClosed；已经提交 manifest 的请求保持 Committed；迟到响应若在新连接上出现，仍要经过 request id、attempt、epoch 和 deadline 检查。

Listing 19.7 连接关闭时的分类顺序

```
connection closed:
  parser has partial frame:
    classify TruncatedFrame
  outbound frames not fully written:
    classify TransportIncomplete
  requests written but no reply:
    classify Unknown
  requests already committed:
    keep Committed as authoritative
```

这个分类让网络事件进入本章前面建立的 attempt 状态机。网络失败不再是一个吞掉上下文的异常，而是一组可重放、可统计、可恢复的事件。

19.3 协议版本、兼容和滚动升级

协议版本不是为了在 header 里放一个好看的数字。它回答一个很具体的问题：当新旧 Driver、新旧 Worker、新旧 manifest 同时存在时，接收方如何知道这条消息能不能安全解释。没有版本规则，升级就会退化成“全系统同时停机换代码”。小型实验可以暂时忍受这种切换，长期运行系统通常不能接受。

兼容性是语义承诺，不是字段数量 新增字段不一定破坏兼容，前提是旧接收方忽略它以后仍然安全，新接收方给缺失字段一个保守默认值。例如新版本给 `reply` 增加 `worker_load_score`，旧 Driver 不看它，最多失去调度优化；新 Driver 收到旧 Worker 的 `reply`，把 `load score` 当成 `unknown`，也不会重复提交。相反，改变旧字段含义通常破坏兼容。若 `deadline_ns` 过去表示绝对时间，新版本改成相对时长，即使字段名和类型不变，也必须视为协议破坏。

未知 enum 值也不能随便 cast 后继续执行。收到未知 `RpcStatus` 时，安全策略通常是拒绝提交、记录 trace、按 `retryable` 或 `invalid` 分类进入状态机，而不是把它当 `Ok`。分布式协议里的默认值必须保守：宁可拒绝一个需要人工观察的结果，也不能把不理解的消息提交为业务事实。

Listing 19.8 协议兼容判断的示例形态

[illegible]

```

20 if (header.protocol_version < DISTRIBUTED_MIN_PROTOCOL_VERSION) {
21     return ProtocolDecision::RejectTooOld;
22 }
23 if (header.protocol_version > DISTRIBUTED_MAX_PROTOCOL_VERSION) {
24     return ProtocolDecision::RejectTooNew;
25 }
26 if ((header.feature_bits & ~supported_features) != 0) {
27     return ProtocolDecision::RejectUnsupportedFeature;
28 }
29 return ProtocolDecision::Accept;
30 }

```

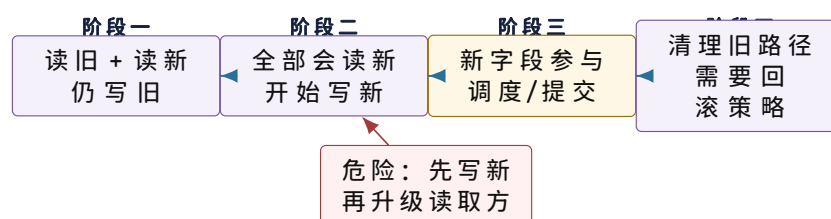
这段代码仍然是示例形态。真实系统还要限制 header 长度、payload 长度、字段重复、压缩标记、认证信息和校验范围。核心不在代码长度，而在纪律：接收方先判断自己是否理解协议，再让消息进入业务状态机。

消息兼容和持久化兼容不同 短生命周期消息可以用更宽松的兼容策略：未知优化字段可以忽略，未知状态可以拒绝并重试。但 manifest、checkpoint、commit log 是恢复入口，兼容规则必须更严格。一个新版本写出的 checkpoint，旧版本如果读不懂，不能假装读懂；一个旧版本 manifest 缺少新版本需要的字段，新版本必须有明确迁移逻辑。否则系统在升级当天能跑，第一次崩溃恢复时才暴露数据语义不一致。

变化类型	一般是否可滚动升级	必要条件
新增可选观测字段	通常可以	旧端忽略仍安全，新端有保守默认值
新增可选调度字段	通常可以	不影响提交、恢复和幂等判断
新增必需身份字段	通常不可以直接升级	先部署能读旧格式的副本，再切写新格式
改变字段含义	不可以当兼容变化	必须新版本号、新解析路径和迁移策略
删除持久化字段	高风险	证明恢复不再依赖，且旧版本不会回滚读取
扩展 enum	取决于默认策略	未知值必须拒绝或保守降级，不能当成功

滚动升级要分阶段 工业系统常用的升级纪律是：第一阶段让所有节点先学会读新旧两种格式，但仍写旧格式；第二阶段确认集群里没有旧读取方，再开始写新字段；第三阶段让新功能真正参与调度或提交；最后才考虑清理旧兼容路径。这里的关键不是流程繁琐，而是避免“一个节点写了别的节点读不懂的事实”。

滚动升级先扩展读取能力，再改变写入语义



滚动升级的核心顺序是先扩大解释能力，再改变外部事实的写入语义。否则故障恢复会读到自己不理解的 manifest 或 checkpoint。

图示：本图要证明的命题是：协议版本的价值在于约束升级顺序，而不是记录构建号。

版本边界 Compute Systems Engine 可以采用一个简化规则：每条 MessageBus 消息携带 `protocol_version` 和 `feature_bits`；manifest 文件携带 `manifest_version`；状态恢复只接受自己明确支持的 manifest 版本；未知消息 feature 进入 rejected trace，不得提交。这样项目不会被复杂兼容系统拖慢，但项目已经保留真实系统最重要的升级纪律。

19.4 Deadline、重试预算和重试风暴

Timeout 是一次等待的本地结果；deadline 是业务请求还能被接受的最晚边界；retry budget 是系统允许为同一个逻辑请求付出的额外工作上限。三者如果混在一起，系统会在下游变慢时制造重试风暴：原请求还在执行，客户端超时后不断创建新 attempt，worker 被更多重复工作压垮，响应更慢，超时更多，最终把一个慢点放大成全局故障。

为什么简单重试会放大故障 假设 worker 正常处理一个 task 需要 900 ms，Driver timeout 设置为 500 ms。每个 attempt 都会超时，Driver 每 500 ms 创建一个新 attempt，而旧 attempt 仍可能继续占 CPU、I/O 和临时文件。几秒后，同一个 logical task 可能有多个 in-flight attempt，worker 的真实负载被重复工作放大，最终所有请求都超时。问题不是 retry 本身，而是 retry 没有预算、没有退避、没有取消语义、没有 in-flight 限制。

Listing 19.9 无预算重试风暴的事件链

```

T0 attempt 0 starts, expected service time 900ms
Driver waits 500ms, times out, marks unknown
Driver immediately starts attempt 1
attempt 0 continues consuming worker and I/O
attempt 1 also exceeds 500ms under higher load
Driver starts attempt 2
load increases, latency increases, timeout rate increases
  
```

Retry budget 是控制面资源 Retry budget 可以按 job、stage、partition、worker 或 task 维度设置。它不是一个简单计数器，而是资源保护：同一个 logical task 最多允许多少 attempt，整个 stage 同时允许多少 retry，某个 worker 返回 Busy 后多久再试，超出预算后进入 Failed 或等待人工处理。预算用完不代表业务正确失败，只表示自动恢复策略不能再安全推进。

Listing 19.10 RetryBudget 的示例形态


```

1 struct RetryBudget {
2     std::uint32_t max_attempts_per_task = 3;
3     std::uint32_t max_inflight_attempts_per_task = 1;
4     std::uint32_t max_stage_retries_per_minute = 1000;
5     std::uint64_t base_backoff_ns = 10'000'000;
6     std::uint64_t max_backoff_ns = 2'000'000'000;
7 };

```

`max_inflight_attempts_per_task=1` 是最保守策略：新 attempt 启动前，旧 attempt 已经确定不可接受或已经进入可清理状态。Speculative execution 会放宽这个限制，但必须先有幂等提交和资源预算，否则它只是主动制造重复工作。

退避要携带原因 Retryable error、Busy、deadline near、stale epoch、protocol mismatch 不能用同一种 sleep 处理。Busy 表示对方有压力，应退避并减少 in-flight；stale epoch 表示路由过期，应刷新控制面；deadline near 表示可能不应再重试；protocol mismatch 表示继续重试同样消息没有意义。错误分类越粗，重试越容易变成风暴。

响应类型	含义	合理动作
Timeout	本地未知远端状态	标记 Unknown，按预算和 deadline 决定是否重试
Busy	远端主动背压	指数退避，降低 in-flight，换 worker 或排队
Retryable	临时失败	保留 attempt 记录，退避后重试
StaleEpoch	控制面过期	刷新 route/epoch，不直接重试旧请求
DeadlineExpired	结果已不应接受	停止新 attempt，清理旧 completion
ProtocolRejected	双方不兼容	停止重试，进入 audit

Deadline 要向下游传播 如果 Driver 的 deadline 是 5 秒，worker 不能在剩余 100 ms 时启动一个预计 1 秒的可靠写或远端请求。请求进入每一层时，应计算剩余预算：排队、执行、写出、reply 都共享同一个 deadline。下游如果发现剩余预算不足，应返回 DeadlineExpired 或 Busy，而不是继续制造迟到结果。迟到结果仍可能完成，但提交层要根据 deadline 和 attempt 状态决定是否接受。

Listing 19.11 deadline 传播的判定形态

```

before starting expensive operation:
    remaining = deadline_ns - now_ns
    if remaining < estimated_minimum_service_ns:
        return DeadlineExpired or Busy
    else:
        continue with the same deadline

```

估算可以粗糙，但不能完全没有。没有 deadline 传播的系统会在用户已经放弃结果后继续消耗资源，导致下一个请求更容易超时。

幂等键和提交键 幂等不是“重复执行没有副作用”这么理想。更常见的边界是：重复执行可以发生，但外部提交根据幂等键只接受一次。幂等键至少包含 logical

task id、stage id、attempt 或 request 关系、driver epoch 和 output kind。服务端或 Driver 通过 commit table 判断是否已接受同一 logical contribution。不同业务的幂等键不同，不能只用 request id；request id 每次重试都会变，无法表达“同一 logical task 的重复 attempt”。

Listing 19.12 幂等提交键的形态

```
IdempotencyKey:
  job_id
  stage_id
  logical_task_id
  output_partition
  manifest_generation_or_epoch

Commit value:
  accepted_attempt
  block_id
  checksum
```

这个键保证一个 logical contribution 最多提交一次。Attempt 和 request 仍然要记录，但它们是物理执行和消息身份，不是最终提交身份。若使用 request id 做提交键，超时重试会生成新 request id，重复输出就会被当成新事实。

重试报告 分布式报告必须把 retry amplification 写出来：attempts / logical tasks。一个 stage 有 1000 个 task、产生 2000 个 attempt，说明资源消耗翻倍。还要记录 timeout 后旧 attempt 完成数量、late accepted rejected 数量、Busy 次数、budget exhausted 数量和 average backoff。没有这些字段，系统可能在平均吞吐上看起来正常，却把集群资源花在重复工作上。

Listing 19.13 重试和 deadline 指标

```
retry.attempts_total
retry.logical_tasks
retry.amplification
retry.budget_exhausted
reply.timeout
reply.busy
reply.late_after_retry
deadline.expired_before_start
deadline.expired_after_completion
backoff.total_sleep_ms
```

这些指标还能指导修复。Timeout 高但 Busy 低，可能是 deadline 过短或网络/worker 慢；Busy 高且 backoff 生效，说明背压正在保护下游；late_after_retry 高，说明 timeout 太激进或旧 attempt 没有取消/清理；budget_exhausted 高，说明系统需要人工介入、扩大资源或降低输入速率。

19.5 状态表和响应处理

调用栈消失以后，状态表就是新的事实中心。Driver 日志说 timeout，worker 日志说 done，MessageBus 日志说 drop reply，manifest 日志说 commit attempt 1。若没有统一状态表，这些局部事实互相矛盾。状态表把逻辑 task、物理 attempt、request、reply、deadline、epoch 和 commit 连接起来。

逻辑任务和物理 attempt Task 7 是逻辑工作单元，attempt 0 和 attempt 1

是两次物理执行，request 100 和 request 101 是两次消息交互，trace 9001 是跨组件观测链。它们不能混成一个 id。混在一起会产生两类错误：重试时换 request id 导致服务端无法去重；迟到响应只按 task id 匹配导致旧 attempt 覆盖新 attempt。

Listing 19.14 Driver 侧任务状态记录

```

1 enum class DistributedTaskState : std::int32_t {
2     Pending,
3     Sent,
4     Running,
5     TimedOut,
6     Retrying,
7     Committed,
8     Failed,
9     Abandoned,
10 };
11
12 struct DistributedTaskRecord {
13     std::uint32_t job_id = 0;
14     std::uint32_t stage_id = 0;
15     std::uint32_t task_id = 0;
16     std::uint32_t current_attempt = 0;
17     std::uint64_t active_request_id = 0;
18     std::uint32_t driver_epoch = 0;
19     std::uint64_t deadline_ns = 0;
20     DistributedTaskState state = DistributedTaskState::Pending;
21     std::uint32_t retry_count = 0;
22     std::uint32_t late_reply_count = 0;
23     std::uint32_t stale_reply_count = 0;
24 };

```

终态要明确。Committed 表示外部结果已经生效；Failed 表示系统决定不再接受自动成功；Abandoned 表示上游不再等待但远端事件仍可能迟到。终态以后，旧响应不能重新打开状态，除非进入人工恢复流程且带新的 epoch。很多分布式 bug 都来自“失败以后又成功”“取消以后又回调”“重启以后旧 leader 又写入”这类终态不清。

响应处理顺序 收到响应后，Driver 不应直接写 manifest。它要先验证协议版本和 checksum，防止损坏消息进入业务状态；再验证 epoch，拒绝旧 Driver 或旧控制面产生的响应；然后验证 task 和 request；接着比较 attempt；最后查询提交表，判断结果是否仍能提交。

Listing 19.15 响应进入状态机前的分类

```

1 enum class ReplyDecision : std::int32_t {
2     AcceptForCommit,
3     DuplicateOfCommitted,
4     LateAttempt,
5     StaleEpoch,
6     DeadlineExpired,
7     CorruptMessage,
8     UnknownRequest,
9 };

```

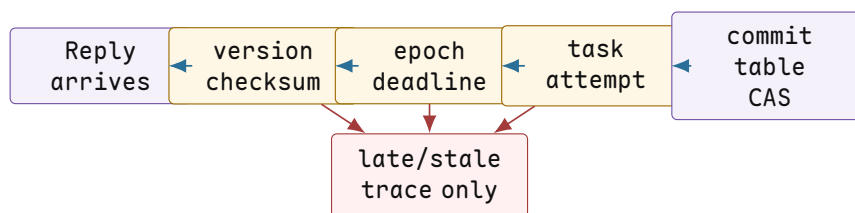
```

10
11 ReplyDecision classify_reply(const MessageEnvelope& reply,
12                             const DistributedTaskRecord& task,
13                             std::uint64_t now_ns) {
14     if (reply.payload_checksum == 0) {
15         return ReplyDecision::CorruptMessage;
16     }
17     if (reply.driver_epoch != task.driver_epoch) {
18         return ReplyDecision::StaleEpoch;
19     }
20     if (reply.task_id != task.task_id ||
21         reply.request_id != task.active_request_id) {
22         return ReplyDecision::UnknownRequest;
23     }
24     if (now_ns > task.deadline_ns) {
25         return ReplyDecision::DeadlineExpired;
26     }
27     if (reply.attempt != task.current_attempt) {
28         return ReplyDecision::LateAttempt;
29     }
30     if (task.state == DistributedTaskState::Committed) {
31         return ReplyDecision::DuplicateOfCommitted;
32     }
33     return ReplyDecision::AcceptForCommit;
34 }

```

真实代码还要查询持久化提交表，不能只看内存里的 `DistributedTaskRecord`。但这个流程展示核心纪律：成功响应只是一个事件。事件进入状态机以后，状态机根据当前事实决定它是否还有权改变结果。

响应处理先验证身份，再尝试提交



响应不能越过状态表直接写 manifest。任何身份不匹配都只能成为可观测事件，不能改变外部结果。

图示：本图要证明的命题是：分布式成功路径也必须先做身份验证，最后才尝试提交。

Trace 替代跨进程调用栈 本地函数用调用栈复盘因果，分布式系统用 trace 复盘因果。每个事件至少写 trace id、request id、attempt、component、state、time 和 reason。响应丢失事故的 trace 应能看到 send、deliver、execute、write temp block、drop reply、timeout、retry、commit、late reply。

Trace 不是为了日志好看。只有 Driver 日志，无法知道 worker 是否写过输出；只有 worker 日志，无法知道 Driver 是否已经放弃；只有平均延迟，无法知道慢在网络、队列、执行还是提交。Trace 把局部事实按身份连接起来，变成可审查事件序列。

19.6 重放日志和确定性调试

Trace 告诉我们发生过什么，replay log 则让我们把状态机输入重新喂一遍。这两个东西容易混淆。Trace 可以有采样、聚合和面向人的文字；replay log 必须是机器可读、顺序明确、足以重建状态机决策的输入序列。响应丢失、迟到回复、乱序消息、Driver 重启这些错误，不应该靠“多跑几次也许复现”，而应该变成可重放测试。

记录边界事件，不记录内部猜测 重放日志应该记录状态机边界输入：消息发送、消息投递、消息丢弃、timer 触发、worker 完成、commit 尝试、Driver crash、recovery start。不要记录“我觉得 worker 可能失败了”这种推断，也不要每个循环变量都写进去。重放的目标是：同一份输入事件序列，在同一份初始 checkpoint 上，必须产生同样的 commit 表、manifest 和拒绝计数。

Listing 19.16 可重放事件的示例形态

```
1 enum class DistributedEventKind : std::int32_t {
2     MessageSent,
3     MessageDelivered,
4     MessageDropped,
5     TimerFired,
6     WorkerCompleted,
7     CommitAttempted,
8     DriverCrashed,
9     RecoveryStarted,
10 };
11
12 struct DistributedReplayEvent {
13     std::uint64_t event_id = 0;
14     DistributedEventKind kind = DistributedEventKind::MessageSent;
15     std::uint64_t trace_id = 0;
16     std::uint64_t request_id = 0;
17     std::uint32_t task_id = 0;
18     std::uint32_t attempt = 0;
19     std::uint32_t driver_epoch = 0;
20     std::uint64_t logical_time_ns = 0;
21 };
```

这里用 `logical_time_ns`，不是为了伪装成真实时钟，而是为了给 timer 和 deadline 一个可重放的时序。测试时钟由 harness 推进，业务代码不能直接读系统墙钟。否则同一份日志今天能复现，明天因为调度抖动又不能复现。

重放要比较外部事实 重放完成后，不应该只比较日志行数。真正的断言是外部事实：每个 logical task 是否最多一个 commit，manifest checksum 是否一致，late reply 计数是否一致，orphan block 清理是否一致，Busy 后 in-flight 是否下降。若重放只验证“程序没有崩溃”，它仍然可能默默重复提交。

事故	Replay 输入	必须比较的结果
成功响应丢失	deliver request、worker done、drop reply、timer fire	retry 创建新 attempt, commit 仍唯一
旧响应迟到	attempt 1 commit 后再 deliver attempt 0 reply	late reply 增加, manifest 不变
Driver 重启	commit 前后分别 crash, 再 recovery	commit 前不承认临时文件, commit 后必须承认
协议不兼容	deliver unsupported feature reply	rejected trace 增加, 不能提交
Busy 风暴	连续 busy 和 timer fire	in-flight 降低, retry 受预算约束

Snapshot 缩短重放，不改变语义 长时间运行的系统不能每次从事件零开始重放。可以周期性写 snapshot：状态表、提交表、manifest offset、worker backpressure 摘要和最后 event id。恢复测试从 snapshot 开始读后续事件。Snapshot 只是加速，不是新的事实来源；若 snapshot 和 manifest 冲突，恢复仍要按本章前面的权威记录规则处理。

确定性调试不是掩盖并发问题 重放日志能把协议状态机变成确定性测试，但不能自动证明数据竞争不存在。MessageBus、状态表和提交表内部仍要遵守线程安全规则。Compute Systems Engine 最简单的做法是让状态机单线程消费事件，worker 线程只产生 completion 事件；等协议正确后，再讨论多线程执行器如何提高吞吐。先把语义做确定，再优化并发度，比一开始把所有东西放进共享队列更容易排障。

19.7 幂等提交和 Manifest

Exactly once 不能解释成“远端函数只执行一次”。在真实故障下，执行可能发生零次、一次或多次。工程上更可实现的目标是：外部可见提交只生效一次。Worker 可以重复算，临时文件可以重复写，响应可以重复到达，但最终 manifest 只能接受一个结果。

提交表保存结果，而不只是保存见过的 ID 很多错误实现只保存一个 set：见过某个 request id 就拒绝重复。这不够。若第一次请求已经完成但响应丢失，客户端重试时需要得到同一语义结果；只知道“见过”不能告诉客户端上次结果是什么。提交表应保存 accepted attempt、block id、checksum、record count、bytes、commit state 和错误分类。

Listing 19.17 提交表记录外部可见事实

```

1 enum class CommitState : std::int32_t {
2     Empty,
3     Preparing,
4     Committed,
5     RejectedDuplicate,
6     Failed,
7 };
8
9 struct CommitRecord {
10     std::uint32_t job_id = 0;
11     std::uint32_t stage_id = 0;
12     std::uint32_t task_id = 0;
13     std::uint32_t accepted_attempt = 0;

```

```

14  std::uint64_t block_id = 0;
15  std::uint64_t checksum = 0;
16  std::uint64_t record_count = 0;
17  CommitState state = CommitState::Empty;
18  };

```

提交表要和 manifest 发布在同一边界。若先记录 committed，再写 manifest 失败，恢复时会看到状态冲突；若先写 manifest，再状态表没记录，重试可能重复提交。Compute Systems Engine 可以把二者简化成同一个 manifest 文件或一个事务性 append；真实系统会用日志、数据库事务、WAL 或共识日志保证这个边界。

重复请求和重复响应不同 重复请求发生在客户端重试，服务端可能要返回旧结果或拒绝旧 attempt。重复响应发生在网络重传、客户端重启或旧完成迟到，Driver 要根据状态表决定是否接受。两者都叫 duplicate，但处理位置不同。服务端侧幂等表保护执行和副作用，Driver 侧提交表保护外部可见结果。

临时输出不是提交 Worker 写出临时 block，不表示 reduce 可以读取。Reduce 只读 manifest 中 accepted 的 block。临时目录可能有多个 attempt 输出、半写文件、过期文件和诊断文件。恢复时以 manifest 为权威，扫描临时目录只是清理或补偿。这个模型直接承接第 15 章可靠写入。

19.8 Deadline、重试预算和背压

不受控制的重试会放大故障。一个 worker 慢，Driver 超时重试；重试增加队列压力，worker 更慢；更多请求超时，更多重试进入系统，最后形成重试风暴。分布式系统必须把 deadline、重试预算和 backpressure 写进协议，而不是藏在 sleep 和日志里。

重试预算按逻辑请求计算 预算属于 logical task，而不是某一次 socket 调用。task 7 attempt 0 超时后，attempt 1 不能重新获得完整无限预算。预算至少包括最大 attempt 数、总 deadline、退避策略和是否允许 speculation。若剩余时间不足以完成一次合理执行，就应失败或降级，而不是启动注定迟到的 attempt。

Busy 是控制信号，不是普通失败 Worker 队列满、磁盘慢、内存高水位或 shuffle 写出阻塞时，应该返回 Busy 或 backpressure，而不是伪装成 Fatal。Busy 表示“我现在不能接更多工作，稍后或换节点”。Driver 对 Busy 的处理应是退避、减少该 worker in-flight、调整调度，而不是立即把 task 标成失败或疯狂重试。

Listing 19.18 节点背压的协议摘要

```

1  struct WorkerBackpressure {
2      std::uint32_t worker_id = 0;
3      std::uint32_t route_epoch = 0;
4      std::uint64_t inflight_tasks = 0;
5      std::uint64_t inflight_bytes = 0;
6      std::uint64_t retry_after_ns = 0;
7      bool accepts_new_tasks = true;
8  };

```

这跟第 15 章 write queue 的高水位线同构。单机里写盘慢导致 write queue high water；集群里某个 worker 慢导致 per-worker in-flight high water。尺度变了，控制原则没变。

慢节点不是失败节点 慢节点要分类型处理。CPU 慢、磁盘慢、网络慢、队列积压、GC 或系统暂停，对应不同动作。盲目把慢节点标为 failed，会触发不必要重

试和数据迁移；完全不处理慢节点，会拖长尾。Speculation 可以缓解长尾，但必须和幂等提交绑定：多个 attempt 可以同时执行，manifest 只能接受一个。

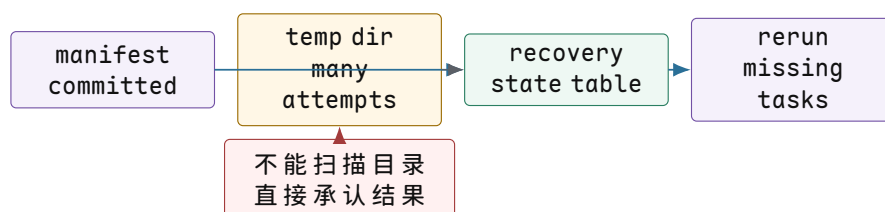
19.9 Epoch、恢复和旧事件隔离

Driver 重启以后，内存里的 Running、TimedOut、Retrying 都消失了，但远端 worker、临时 block、迟到响应和旧连接可能仍然存在。Epoch 用来隔离新旧控制面。新的 Driver epoch 启动后，旧 epoch 的响应、提交请求和心跳不能直接改变新状态。

Epoch 不是时间戳 Epoch 是控制面版本，不是墙钟时间。Driver 每次重启、leader 切换或恢复状态时获得新的 epoch。消息携带 epoch，接收方比较自己的当前 epoch。旧 epoch 消息可以记录为 stale，但不能提交结果。若系统没有 epoch，重启前的迟到响应可能在重启后污染新的任务表。

恢复时谁是权威事实 恢复不能从临时目录猜结果。权威事实应该是 manifest、提交表、checkpoint 或日志。恢复流程先读取权威记录，确认哪些 task 已提交；再扫描临时输出，清理不在 manifest 的孤儿文件；最后为未提交 task 重新创建 attempt。若先扫描目录再补 manifest，就会把半写文件或旧 attempt 当成结果。

恢复时 manifest 是权威入口



恢复先读权威提交记录，再用临时目录做清理和补偿。临时文件存在不代表业务结果已提交。

图示：本图要证明的命题是：崩溃恢复必须以提交事实为入口，而不是以残留物理文件为入口。

Driver commit 前后崩溃 commit 前崩溃：worker 输出可能存在，但没有权威记录，恢复时不能自动接纳。commit 后崩溃：manifest 已经接受结果，恢复时必须承认，即使内存状态丢失。这个边界和第 15 章写 manifest 前后崩溃完全一致，只是现在旧响应和远端 worker 也会迟到。

19.10 单进程 MessageBus：可控故障先于真实网络

真实网络故障很多，直接上多机器会同时引入 socket、序列化、线程、日志和协议 bug。更稳的训练入口是先实现单进程 MessageBus：显式消息队列、可配置延迟、丢包、乱序、重复、Busy、队列容量和 timer。它不是真实网络，但能稳定复现分布式协议错误。

故障规则是核心功能 MessageBus 不是一个简单队列，而是故障注入器。每条消息可以按规则 drop、delay、duplicate、reorder。Timer 驱动状态表，而不是业务线程 sleep。Inbox 有界，队列满时返回 Busy 或 backpressure。只要这些规则足够明确，故障就能复现响应丢失、迟到 reply、重复 reply 和 Driver 重启。

Listing 19.19 MessageBus 故障规则的示例形态

```

1 enum class FaultAction : std::int32_t {
2     Deliver,
3     Drop,

```

```

4   Delay,
5   Duplicate,
6   Reorder,
7 };
8
9 struct MessageFaultRule {
10  std::uint64_t rule_id = 0;
11  FaultAction action = FaultAction::Deliver;
12  std::uint64_t delay_ns = 0;
13  std::uint32_t match_task_id = 0;
14 };

```

多进程会暴露新的边界 单进程 MessageBus 仍然会掩盖一部分问题：Driver 和 Worker 可能不小心共享同一个全局变量，配置更新看起来同时生效，C++ 指针在日志里似乎能代表对象身份，队列里的消息也像完整对象一样一次到达。因此模拟阶段要主动约束自己：消息只通过 envelope 和 payload 传递，状态只通过显式事件改变，测试禁止 Worker 直接读 Driver 的状态表。这样迁移到多进程时，协议语义不会重写。

Linux 观察只服务协议问题 ss 可以看连接状态，但不能证明业务请求安全；tcpdump 可以看包，但 TCP 字节流不等于应用消息边界；strace 可以看系统调用，但不能替代状态表；tcnetem 可以制造延迟和丢包，但测试通过仍要看 manifest 和提交表。工具是证据来源，不是协议设计本身。

19.11 从单进程 MessageBus 到多进程

多进程迁移不是“把队列换成 socket”这么简单。它会把单进程模拟阶段被隐藏的边界一次性暴露出来：地址空间隔离、进程生命周期、字节流 framing、配置一致性、部分读写、连接关闭、权限和临时文件清理。若前面的协议身份、状态表和 manifest 做得扎实，多进程只是更真实的传输层；若前面靠共享内存和对象引用偷懒，多进程会迫使设计重写。

地址空间隔离会消灭隐式依赖 单进程版本里，一个 TaskRequest 也许偷偷带着指向输入 buffer 的指针；一个 worker 也许能调用全局单例读取最新配置；一个测试也许直接检查 Driver 内存对象。多进程以后这些路径全断了。跨进程只能传值、文件描述符、路径、block id 或网络地址，不能传对象身份。这反而是好事：它迫使协议说清楚什么是事实，什么只是进程内部实现细节。

单进程阶段容易偷懒的地方	多进程暴露的问题	正确设计动作
共享全局配置	节点看到的配置版本不同	配置 epoch/version, 带消息记录使用版本
传递对象指针	地址空间不同, 指针无意义	传递稳定 id、offset、path 或 serialized value
队列保证完整对象	socket 是字节流	定义 frame header、长度、校验和解析状态机
线程退出即清理	进程 crash 留下临时文件	以 manifest 恢复, 清理 orphan block
连接关闭当失败	关闭只说明传输断开	task 状态进入 unknown, 再由 deadline/查询/重试处理

Framing 是协议的一部分 TCP 提供有序字节流, 不提供应用消息边界。读取方必须先读 frame header, 再根据长度读取 payload; 若长度超过上限, 立即拒绝; 若 checksum 不匹配, 进入 corrupt message; 若连接中途关闭, 当前 frame 是 incomplete transport event, 不是业务 Fatal。这个状态机和第 15 章短读短写完全一致, 只是现在读写对象从文件变成网络连接。

Listing 19.20 网络 frame header 的示例形态

```

1 constexpr std::uint32_t DISTRIBUTED_FRAME_MAGIC = 0x44535431U;
2 constexpr std::uint32_t DISTRIBUTED_MAX_FRAME_BYTES = 16777216U;
3
4 struct NetworkFrameHeader {
5     std::uint32_t magic = DISTRIBUTED_FRAME_MAGIC;
6     std::uint16_t protocol_version = DISTRIBUTED_PROTOCOL_VERSION;
7     std::uint16_t header_bytes = 0;
8     std::uint32_t payload_bytes = 0;
9     std::uint64_t payload_checksum = 0;
10 };

```

这仍然不是直接 `send(&header, sizeof(header))`。真实 wire format 要逐字节编码, 固定大小端, 明确 padding 不进入网络。这里的结构只是把 frame 需要的概念列出来: magic 识别协议, version 选择解析规则, header bytes 支持扩展, payload bytes 定义边界, checksum 防止损坏 payload 进入业务状态。

连接事件不是业务事件 连接建立、连接关闭、读超时、写失败, 都属于 transport event。它们会影响消息是否可达, 却不直接证明 task 成功或失败。连接关闭时, Driver 最多知道“这条连接上还有未完成 request 的状态未知”。正确动作是把相关 request 标成 transport unknown, 等待 deadline、查询提交表、重试或进入恢复流程。把 connection reset 直接映射成 task failed, 会重新犯 timeout 当 failure 的错误。

最小多进程判读矩阵 不要求做完整 RPC 框架, 但至少覆盖四类迁移边界。第一, partial frame: 发送方分多次写 header 和 payload, 接收方必须正确拼帧。第二, worker crash after temp write: worker 写完临时 block 后崩溃, Driver 不得扫描目录承认结果。第三, driver restart with old reply: 旧进程退出后, 旧响应或旧连接事件不得污染新 epoch。第四, version mismatch: 新 Worker 带

未知 feature，旧 Driver 必须拒绝提交并留下 trace。

迁移边界	故障输入	判读重点
partial frame	header/payload 分片到达	frame parser 不越界，不误提交
connection close	reply 发送前连接关闭	request 进入 unknown，不直接 failed
worker crash	temp block 已写，reply 未发	恢复不扫描临时目录承认结果
old epoch reply	Driver 重启后旧 reply 到达	stale epoch 计数增加，manifest 不变
version mismatch	未知 feature 或过高版本	协议拒绝，trace 可解释原因

19.12 工业设计参照

读工业系统要读约束，不只读名字 工业设计参照不是搬运系统名字。案例必须回答五个问题：它面对的故障和规模约束是什么；它用什么身份、日志或状态机把不确定性压住；它购买了什么能力；它付出了什么复杂度；Compute Systems Engine 应该采用、简化还是暂时拒绝哪一部分。下面几个案例都按这个方式阅读。

工程案例

gRPC deadline 和 status gRPC 等 RPC 系统把 deadline、status、metadata 和 cancellation 放进调用模型，解决的是跨服务调用的传输边界：调用不能无限等，错误要有分类，取消要能传播，metadata 要能携带认证、trace 或路由信息。它的核心机制是把 deadline 随请求向下游传播，并把状态码作为协议的一部分返回，而不是让每个调用点随手写一个 timeout。

代价是：框架只能说明调用层发生了什么，不能替业务决定远端副作用是否已经发生。Deadline exceeded 不等于远端没有执行，cancelled 也不一定撤销远端已经完成的写入。Compute Systems Engine 应借鉴 deadline/status/metadata 的显式化，但不能借用“RPC 返回失败所以 task failed”的错误直觉。提交表、attempt 和 manifest 仍由业务协议负责。

工程案例

Spark task attempt 和 speculative execution Spark 这类计算引擎必须处理慢节点、失败节点、Executor 丢失和长尾任务。它允许同一个 logical task 出现多个 physical attempt，甚至用 speculative execution 让另一个节点并行跑同一份工作。核心机制是把 task id、stage attempt、task attempt 和输出提交分开，使调度器能多次执行，但下游只承认被提交协议接受的结果。

代价是额外计算、更多临时输出、attempt 清理和更复杂的指标解释。Speculation 不是免费加速器，它会占用集群资源，也会放大非幂等输出的风险。Compute Systems Engine 可以采用 task/attempt/manifest 的身份模型，暂时简化掉复杂的资源调度策略；只有当 late reply 和 duplicate commit 已经被测试覆盖后，才适合加入 speculation。

工程案例

MapReduce output commit MapReduce 的 output commit 设计面对一个现实约束：task 可能重试，worker 可能崩溃，输出文件系统可能只看到物理文件，不能理解哪个 attempt 才是语义结果。核心机制是让 worker 写 attempt 私有的临时位置，再由 committer 决定哪些输出进入最终可见目录或 manifest。这样重复执行不会自动变成重复业务结果。代价是提交协议依赖底层存储能力。某些文件系统的 rename 接近原子操作，某些对象存储的目录语义和一致性模型更弱，committer 就要更谨慎。Compute Systems Engine 不应假设“文件出现在目录里就是提交”，而要让 manifest 成为唯一读取入口；临时目录只用于清理、诊断和恢复补偿。

工程案例

Kafka producer id 和 sequence number Kafka 的幂等生产路径面对的约束是高吞吐追加日志和不可避免的网络重试。Producer 发送记录后可能没收到 ack，于是重试；Broker 必须分辨这是新记录还是重复记录。核心机制是 producer id、producer epoch 和 per-partition sequence number，Broker 用这些身份信息识别重复、乱序和旧 producer 事件。代价是协议状态更重，Broker 要维护生产者状态，异常恢复时要处理 epoch 变化和序列约束。它给 Compute Systems Engine 的启发不是照搬 Kafka 事务，而是理解“幂等必须落在身份和日志上”。request id、attempt、driver epoch 和 commit table，是示例版的同类机制。

工程案例

Kubernetes controller 的 reconcile 思路 Kubernetes 控制器处理的是另一类分布式问题：观察到的事件可能丢失、重复、乱序，控制器进程也可能重启。它的核心思路不是相信每个 watch 事件都完整可靠，而是不断读取当前期望状态和实际状态，执行幂等 reconcile。对象的 uid、resource version、generation 和 status，把“我看到了什么事件”和“系统现在应该变成什么状态”分开。这个案例对分布式计算模型很有价值：事件是提示，不是最终事实；状态机要能重复运行而不制造重复副作用。代价是收敛式控制通常不是最低延迟路径，也需要处理冲突和版本检查。Compute Systems Engine 可以借鉴 reconcile 的幂等思想，用 manifest/commit table 作为当前事实，不要让单条 reply 直接支配最终状态。

19.13 实现路径

Compute Systems Engine 在这一章不需要立刻写真实集群。目标是把单机 completion 模型扩展成单节点分布式模拟：Driver、Worker、MessageBus、CommitTable、Manifest 和 Trace 全部显式化。重点是状态语义，而不是网络框架体积。

实现边界 第一版，把远程调用改成显式消息：Driver 发送 `TaskRequest`，Worker 返回 `TaskReply`。第二版，加入协议版本和 feature 检查，但只支持一个版本。第三版，加入状态表和提交表，但先不重试。第四版，加入故障注入，只开一个故障：成功响应丢失。第五版，加入 retry、attempt、deadline 和 late reply 处理。第六版，加入 Busy 和 per-worker backpressure。第七版，把状态表和 manifest 写成可恢复文件。第八版，加入 replay log。第九版，把 MessageBus 的一个路径迁移到多进程 frame parser。

消息判读矩阵

路径	故障输入	判读重点
响应丢失	Worker 成功, reply drop	timeout 是 unknown, retry 创建新 attempt
旧响应迟到	attempt 0 晚于 attempt 1 到达	manifest 只接受一个 attempt
Driver 重启	commit 前后分别崩溃	恢复以 manifest 为权威
Busy 响应	worker inbox 高水位	Driver 退避并减少 in-flight
重复请求	同一 request 重放	服务端不重复提交副作用
乱序消息	reply 顺序打乱	状态表按身份处理, 不按到达顺序处理
协议不兼容	未知 feature 或过高版本	拒绝提交并留下可解释 trace
事件重放	固定 replay log 输入	commit 表、manifest 和计数完全一致
多进程分帧	header/payload 部分到达	frame parser 正确拼帧并保护长度上限

运行证据 分布式模拟至少保留这些运行证据: task 总数、attempt 总数、request 总数、timeout 数、retry 数、late reply 数、stale epoch 数、protocol rejected 数、corrupt frame 数、busy 数、committed 数、duplicate rejected 数、orphan block 清理数、replay event 数、每个 worker in-flight 峰值、每个队列高水位次数。没有这些指标, 分布式协议只能靠猜。

Listing 19.21 分布式模拟运行证据示例

```
tasks.total=128
attempts.total=137
reply.late=5
reply.stale_epoch=2
protocol.rejected=1
frame.corrupt=0
commit.accepted=128
commit.duplicate_rejected=9
worker.busy=14
replay.events=402
orphan_blocks.cleaned=9
```

源码阅读任务

源码阅读任选一条窄路径:

1. 阅读一个 RPC 框架的 deadline、status 和 cancellation 文档或源码路径, 说明框架解决了什么、没有替业务解决什么。
2. 阅读 Spark、MapReduce 或类似系统的 task attempt / output commit 设计, 重点看重复 attempt 如何避免重复输出。
3. 阅读 Kafka 或类似消息系统的 producer epoch、sequence number 或幂等生产路径, 说明它如何处理重试和重复。
4. 阅读 Kubernetes controller 或类似控制系统的 reconcile 路

径，说明它如何处理重复、乱序和进程重启。

5. 阅读 Linux 网络观察工具的输出，说明 `ss`、`tcpdump`、`strace` 能证明什么，不能证明什么。

19.14 完整案例：一次成功响应丢失后的正确恢复

分布式系统的第一条硬边界，是“没有收到响应”不能推出“对方没有执行”。下面只跟踪一个 `split`。Driver 把 `Split-8` 发给 Worker A，请求号为 `request=7001`，`attempt` 为 0，`deadline` 为 `T+5s`。Worker A 正常执行，写出临时 `block`，并把 `reply` 发回。MessageBus 故障规则丢弃 `reply`。Driver 到 `deadline` 后只知道结果未知，不能把 `attempt 0` 标记为失败后直接删除事实，也不能把 `split` 当作从未执行。

第一步：timeout 进入 Unknown，而不是 Failed Timeout 表示观察者等不到响应。它可能对应三种真实情况：Worker 没收到请求；Worker 收到但没执行完；Worker 已经执行成功，`reply` 丢了。只有第三种会留下临时 `block` 和潜在提交候选。若 `timeout` 直接进入 `Failed`，重试后系统可能同时接受旧结果和新结果；若 `timeout` 直接重用同一个 `attempt id`，旧 `reply` 迟到时无法分辨是哪次执行。正确状态是 `Unknown`，并创建新 `attempt`。

Listing 19.22 timeout 的状态变化

```
Driver sends request 7001 for split 8 attempt 0
Worker A executes and writes block B80
reply is dropped
deadline expires at Driver
Driver marks attempt 0 as Unknown
Driver schedules attempt 1 with request 7002
```

`Unknown` 不是软弱，而是诚实。它承认系统缺少事实，并把后续重试放进 `attempt` 状态机，而不是用本地猜测填补网络边界。

第二步：新 attempt 不能继承旧身份 Attempt 1 必须有新的 `request id`、`attempt id` 和输出 `block id`。它可以处理同一个 `logical split`，但物理执行身份不同。Worker B 成功执行后返回 `reply`，Driver 先检查身份：`split id` 是否匹配，`attempt` 是否当前可接受，`driver epoch` 是否未过期，`block checksum` 是否匹配，`deadline` 是否仍有效。只有通过这些检查，`reply` 才能进入提交判定。

Listing 19.23 reply 验证顺序

```
reply:
  split_id = 8
  attempt = 1
  request_id = 7002
  driver_epoch = 12
  block_id = B81
  block_checksum = 0x8b81...

validate identity
validate epoch and deadline
validate checksum and output metadata
ask commit table whether split 8 has accepted attempt
```

这条顺序避免了一个常见错误：看到 `Ok` 就写 `manifest`。分布式 `reply` 的成功

状态只说明远端认为自己成功；本地还必须把它放回当前控制面事实中验证。

第三步：迟到 reply 只能被分类，不能被忽略 Attempt 1 提交后，attempt 0 的 reply 迟到。系统不能静默忽略，因为旧 block 需要清理，统计也要说明发生过 late reply；也不能接受它，因为 split 8 已经有 accepted attempt。正确动作是分类为 LateAttempt，记录 trace，拒绝 manifest append，并安排临时 block 回收。

Listing 19.24 迟到 reply 的分类

```
late reply:
  split_id = 8
  attempt = 0
  block_id = B80

commit table:
  split 8 accepted attempt = 1

decision:
  LateAttempt
  do not append manifest
  mark B80 as orphan candidate
  increment reply.late
```

“分类”是工程上非常重要的词。迟到、重复、旧 epoch、未知 request、checksum mismatch、protocol mismatch，后续动作都不同。把它们都写成一条错误日志，Driver 就无法做正确恢复。

第四步：replay 必须得到同一外部事实 把上述事件写入 replay log: send 7001、execute A、drop reply、timeout、send 7002、reply 7002、commit attempt 1、deliver late reply 7001。重放时，即使事件在实现中经过不同线程调度，最终 manifest 仍只能有一条 split 8 entry，accepted attempt 为 1，late reply count 为 1，B80 进入 orphan 清理。Replay 验收比较的是外部事实，不是内存地址或日志打印顺序。

Listing 19.25 replay 验收断言

```
manifest.entries_for(split 8).size == 1
manifest.accepted_attempt(split 8) == 1
reply.late == 1
commit.duplicate_rejected == 1
orphan_blocks contains B80
final checksum equals reference checksum
```

这个案例把第 16 章的所有核心概念连在一起：timeout 是 unknown，attempt 区分物理执行，request id 识别消息，epoch 保护控制面，manifest 定义提交事实，replay 固化事故。第 17 章会把同样原则扩展到 partition owner 和 leader term。

复查必须覆盖五项事实：timeout、retry、late reply、duplicate reply、Driver 重启都不会重复提交；消息身份、attempt、request、epoch、deadline、status 字段完整且语义清楚；版本不兼容会被拒绝，manifest 和提交表是权威事实，commit 前后崩溃行为明确；Busy、deadline、retry budget、replay log、frame parser 有指标和测试；从 RPC、Spark、MapReduce、Kafka、Kubernetes 等系统借鉴了什么、哪些复杂度暂不引入，必须写成明确判断。

19.15 消息故障门禁

一、Timeout 是否仍被当成失败 若 timeout 分支直接 `mark_failed`，没有 `unknown`、`retry budget` 和 `attempt` 状态，本章核心没有学到。

二、响应是否先验证身份 成功响应必须先检查 `task`、`attempt`、`request`、`epoch`、`deadline` 和 `checksum`，再尝试提交。不能直接写 `manifest`。

三、协议版本是否真的生效 未知 `feature`、过高版本、过低版本、未知成功状态都不能静默进入提交路径。持久化 `manifest` 的兼容规则必须比临时消息更严格。

四、提交是否唯一 多个 `attempt` 可以执行，`manifest` 只能接受一个。重复响应和迟到响应必须可观测，但不能改变终态。

五、恢复是否以权威记录为入口 恢复先读 `manifest` 或提交表，再清理临时输出。不能扫描目录直接承认结果。

六、背压是否写进协议 Worker 忙时应返回 `Busy` 或 `backpressure`，Driver 应退避并调整 `in-flight`。不能把 `Busy` 当 `Fatal`，也不能立即疯狂重试。

七、重放是否比较外部事实 Replay 测试要比较 `commit table`、`manifest`、计数和 `orphan` 清理结果。只比较日志行数或程序退出码不够。

八、多进程是否只替换传输层 迁移到 `socket` 后，状态机语义不应重写。连接关闭、`partial frame` 和进程崩溃都必须进入已有 `unknown`、`deadline`、`recovery` 和 `manifest` 模型。

九、是否为第 17 章留下边界 第 16 章解决单个 `task attempt` 的消息、重试和提交。第 17 章会把问题扩展到分片归属、复制日志和共识。不要在第 16 章里提前把所有控制面问题塞进一个“大 leader”。

19.16 习题

习题与作业

习题一：为响应丢失事故写事件表。事件至少包括 `send`、`execute`、`write temp block`、`reply drop`、`timeout`、`retry`、`commit`、`late reply`。标出每个事件能证明什么，不能证明什么。

习题二：设计 `MessageEnvelope`。解释每个字段对应哪个故障问题。再说明为什么这个结构不能直接 `memcpy` 到 `socket`。

习题三：设计协议版本规则。给出新增可选字段、新增必需字段、改变字段含义、扩展 `enum`、删除持久化字段五种变化的兼容判断。

习题四：写一个 `classify_reply` 的测试矩阵，覆盖 `stale epoch`、`late attempt`、`unknown request`、`deadline expired`、`duplicate committed` 和 `accept for commit`。

习题五：设计提交表。要求同一个 `logical task` 的多个 `attempt` 只能有一个进入 `Committed`。说明 `commit` 前崩溃和 `commit` 后崩溃的恢复动作。

习题六：设计 `MessageBus` 故障注入规则。至少支持 `drop`、`delay`、`duplicate`、`reorder` 和 `Busy`。说明如何保证实验可复现。

习题七：写一份 `replay log`，让它稳定复现“`attempt 0` 成功但 `reply` 丢失，`attempt 1` 提交，`attempt 0 reply` 迟到”的事故。给出重放后的断言。

习题八：比较 `timeout`、`deadline` 和 `retry budget`。构造一个没有预算的重试风暴，再给出修复方案。

习题九：实现一个最小 `frame parser` 测试。要求 `header` 和 `payload` 可以分片到达，超过最大长度必须拒绝，`checksum` 错误不能进入业务状态机。

习题十：阅读一个工业系统的 `task attempt`、幂等消息或 `reconcile` 设计，按“约束、核心机制、购买能力、成本、迁移到本项目的方式”写报告。

工程案例

知识点小案例：响应丢失会把“执行一次”变成协议问题。特例是 worker 已经写出 block，但 reply 在网络中丢失；controller 超时后启动 attempt 1，attempt 1 也写出 block 并提交。稍后 attempt 0 的迟到 reply 到达。若消息里没有 task id、attempt id、epoch、deadline 和输出身份，controller 无法判断该接受还是拒绝。由此推出规律：分布式系统不承诺“调用一次执行一次”，它承诺在身份、重试和提交表约束下做出可恢复决定。工程上每个 reply 只能进入 classify，再由 commit 规则决定是否写 manifest。

分片、复制与共识

第 16 章只跟踪一个 task attempt：请求可能丢、响应可能迟到、提交必须唯一。本章把问题扩大到一段状态的归属。日志统计系统按 key 做 reduce，`event=login` 突然成为热点，partition 12 的队列持续上涨。Driver 决定把 partition 12 从 worker A 迁到 worker B，并把这个热 key 拆成两个子分片。迁移过程中，旧客户端仍按旧路由把写发给 A，新客户端已经按新路由把写发给 B；A 网络短暂隔离后恢复，还带着旧 owner 身份继续接受写；B 只追到一半增量就开始服务读。最后，两个 worker 都认为自己有权提交同一段 key range 的结果。

这个事故比术语表更能说明本章。为什么需要分片？因为单个 owner 扛不住所有 key，也不能让热点 key 把整个 reduce 阶段拖成长尾。为什么需要路由版本？因为客户端、worker 和 Driver 在迁移期间会同时看到新旧归属事实。为什么需要复制？因为只有一个 owner 时，owner 慢或死就让这段状态不可用。为什么需要共识或等价的强协调？因为“partition 12 现在归谁、哪个 manifest generation 已提交”这类控制面事实不能靠两个节点各自宣布。为什么需要 fencing？因为旧 owner 即使还活着，也必须失去写入权。

核心问题是状态归属。分片先回答“状态放在哪里、请求送给谁”；复制回答“owner 故障后谁有足够状态接管”；共识回答“多个控制面可能同时行动时，哪条顺序才算真”。三者不是并列名词，而是同一个工程问题逐步升级后的三个回答：容量不够时拆开，单点不可靠时复制，多个副本或多个控制面可能争权时建立唯一提交顺序。

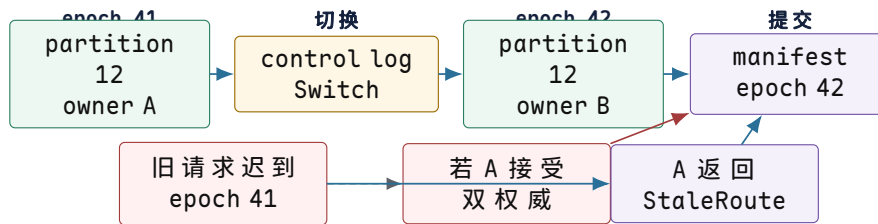
分析沿八条线展开：从旧 owner 迟到写入事故解释分片、路由 epoch、fencing 和提交权；比较 hash 分片、range 分片、虚拟分片和热点 key 拆分的适用条件；设计可恢复的再分片状态机，而不是把再分片写成一次后台拷贝；说明复制提交点如何影响写成功、读新鲜度、故障恢复和尾延迟；用 quorum 的交集解释为什么多数确认能保留已提交事实，也能说出它不能自动解决的问题；区分 stale read、monotonic read、linearizable read，并把读语义写进接口；解释 leader term、log index、commit index 和 snapshot 的工程意义；在 Compute Systems Engine 中落地 partition assignment、route epoch、replica progress、control log 和运行证据。

20.1 旧 Owner 事故：状态归属先于算法

先不讲 Raft，也不讲 CAP。只看一条迟到写。Epoch 41 时，partition 12 归 worker A；Driver 发布 epoch 42 后，partition 12 迁到 worker B；一个 epoch 41 的写请求因为网络延迟，在 epoch 42 生效后才到达 A。若 A 只按 partition id 判断“我曾经负责过它”，它会接受旧写；B 同时接受新写，系统就出现两个权威。

这就是分布式状态的第一条纪律：同一个名字不代表同一个权利。Partition id 相同，不代表 owner 仍然有效；worker 还活着，不代表它还有提交权；文件存在，不代表它已经进入 manifest；leader 能响应，不代表它仍处于当前 term。每个能改变外部状态的请求，都必须携带 ownership epoch 或等价的 fencing token。

旧 owner 迟到写入：没有 epoch 就会出现两个权威



Epoch 的作用不是刷新缓存，而是把“曾经拥有”和“现在有权”分开。旧请求可以被记录、转发或拒绝，但不能悄悄提交。

图示：本图要证明的命题是：分片正确性的第一层不是哈希函数，而是带版本的所有权。

状态归属包含五个问题 一个 partition 的归属至少要回答五件事：谁是当前 owner，哪个 epoch 证明它是当前 owner，当前处于 serving 还是 migrating，哪些副本拥有足够新状态，哪个控制日志位置已经提交。只知道 key 经过 hash 落到 partition 12，不足以处理故障、迁移和恢复。

旧消息是正常输入 旧 epoch 请求、旧 owner 响应、旧 leader 心跳、旧 manifest 写入，都不是“理论上不会发生”的异常。网络延迟、进程暂停、重试、恢复和连接复用都会制造旧事件。成熟系统不是祈祷旧事件不出现，而是让每条旧事件进入状态机后只能产生受控结果：拒绝、转发、记录、清理或人工介入。

Fencing token 比“我认为自己是 owner”更强 Fencing token 是由当前控制面授予的一段单调权利，例如 route epoch、leader term 或 log index。外部资源在接受写入时必须检查 token 是否仍然新鲜。旧 owner 即使没有崩溃、仍能访问磁盘或网络，也会因为 token 过期而失去写权。没有 fencing，暂停很久的旧进程恢复后，可能用旧身份继续写外部系统，造成双写。

Listing 20.1 Fencing 的判定形态

```

request carries:
  partition_id = 12
  fencing_token = route_epoch 42

storage or commit layer checks:
  current_token(partition_id=12) == 42

if current token is 43:
  reject as stale owner
  do not write manifest or final output
  
```

Fencing 的关键是检查点不能只在客户端。若旧 owner 自己检查一次后就写最终文件，控制面切换期间仍可能竞态。提交层、manifest 层或外部存储层都要带 token 检查。单机项目中，这个检查可以在 Driver 的 commit 函数里；真实分布式系统中，外部存储或 coordinator 也常需要参与。

Lease 依赖时间，必须写明假设 有些系统用 lease 表示“在某段时间内 owner 有权服务”。Lease 可以降低每次读写都访问强控制面的成本，但它引入时间假设：时钟偏差、暂停、GC、调度延迟、网络延迟和续租失败都会影响安全。若旧 owner 的本地时钟慢，可能以为 lease 仍有效；若新 owner 过早接管，就可能与旧 owner 重叠。

因此 lease 适合读缓存、低风险观测或有严格时钟基础设施的系统；控制面关键提交仍应有 term/epoch/log index 这类不依赖本地时间直觉的 fencing。若项目使用 lease 模拟，报告必须写 lease duration、clock source、最大暂停假设和过期检查位置。不能把“超时了”当作“对方绝对不会再写”的证明。

权利机制	购买的能力	主要风险
Epoch / term	单调权利，旧 owner 可拒绝	需要强控制面提交和检查点
Fencing token	外部提交层能拒绝旧写	所有写入口都必须检查 token
Lease	减少控制面访问，低延迟读写	依赖时间假设和续租协议
裸 owner id	实现简单	旧 owner 恢复后可能继续写

Listing 20.2 分片请求必须携带所有权证据

```
1 enum class PartitionRequestKind : std::uint16_t {
2     Read,
3     Write,
4     CommitOutput,
5 };
6
7 struct PartitionRequest {
8     std::uint64_t request_id = 0;
9     PartitionRequestKind kind = PartitionRequestKind::Write;
10    std::uint32_t partition_id = 0;
11    std::uint64_t route_epoch = 0;
12    std::uint64_t leader_term = 0;
13    std::uint32_t owner_worker = 0;
14    std::uint32_t attempt = 0;
15    std::uint64_t input_block_id = 0;
16    std::uint64_t input_checksum = 0;
17 };
```

这个结构仍然是示例形态。真实系统还要携带认证、payload、长度、校验和版本兼容字段。本章关心的是控制身份：partition、epoch、term、owner 和 request id 必须进入提交路径。若接口只提供 `commit(partition_id,block_id)`，调用者很容易绕过所有权检查。

20.2 分片：集群尺度的数据布局

分片可以看成单机数据布局在集群尺度上的版本。单机上，数组怎样落到 cache line、页和 NUMA 节点决定访问成本；集群里，key、任务、shuffle block 和副本怎样落到 worker 决定网络成本、负载均衡、故障半径和扩容方式。分片不是取模公式，而是访问模式、负载分布和恢复边界的共同设计。

Hash、Range 和虚拟分片 Hash 分片把 key 打散，适合点查、均匀 key 和简单扩容，但范围扫描要访问多个分片，也不能自动拆开单个热点 key。Range 分片保留顺序，适合范围查询和排序，但递增 id、时间戳或热门区间会把写入压到某个范围。虚拟分片把逻辑分片数量做得比物理 worker 多，便于迁移、限流和负载均衡；它类似单机运行时把任务块数量做得多于线程数，让调度器有调整空间。

策略	适合场景	主要风险	项目取舍
Hash 分片	均匀 key、点查、普通 shuffle	热点 key 仍集中，范围扫描差	默认实现
Range 分片	排序、范围查询、时间窗口扫描	递增写入和热门区间倾斜	作为对照路径
虚拟分片	动态扩缩容、迁移、限流	元数据和调度更复杂	推荐加入
热点 key 拆分	单个 key 压垮 reduce	需要二级合并和版本记录	必须覆盖

热点 key 不能只靠加节点解决 若 `event=login` 占大多数记录，普通 hash 会把它稳定地发到同一个 reduce partition。增加 worker 数量只会让冷 key 更分散，热点仍然长尾。常见处理包括 map-side combine、热 key 拆成多个子 key、二级 reduce、缓存、限流和业务异步化。每种方法都要回答两个问题：最终输出是否仍是原 key 的精确结果，监控是否还能把子 key 聚回业务 key。

Listing 20.3 热点 key 拆分的执行身份和业务身份要分开

```
normal key:
    reduce_partition = hash(key) % reduce_count

hot key:
    local_shard = hash(input_split, local_counter) % hot_shard_count
    reduce_partition = hot_route[key][local_shard]
    final output key = key
```

子 key 是执行层身份，不是业务层 key。Manifest 要记录 hot route version、子分片范围和最终合并规则。否则作业中途改变热点策略，会让同一个业务 key 的 partial 按两套规则落盘，后续 reduce 无法解释。

分片函数必须有版本 分片函数、key 规范化、hash seed、热点列表和二级合并规则都属于协议。作业进行中改变这些规则，不带版本就会让相同 key 前后落到不同位置。Shuffle block、manifest entry 和 trace 都应记录 partition version。Reduce 端要么只合并同一版本的数据，要么执行明确的迁移转换，不能把不同规则下的 block 混在一起。

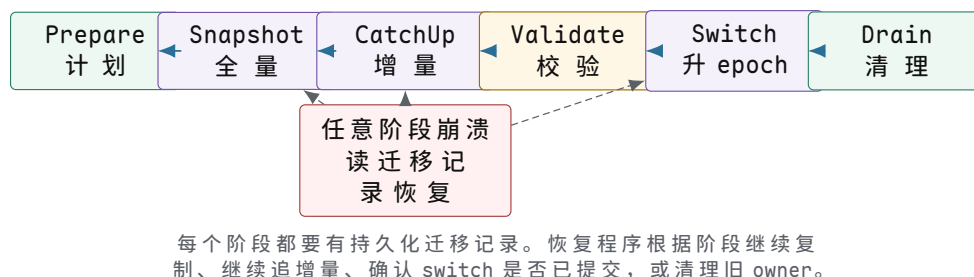
20.3 路由 Epoch 和再分片状态机

再分片最容易被误讲成“把数据从 A 拷到 B”。真正危险的是迁移期间的新写、旧写、迟到响应和崩溃恢复。历史数据复制只是第一步；复制期间旧 owner 仍可能接收写，新 owner 可能还没有追平，客户端路由表可能过期，Driver 可能重启。一个可靠再分片必须写成状态机。

冻结、追赶和双写 冻结写入最容易证明：旧 owner 停止写，复制完成后切 epoch，新 owner 服务。代价是不可用窗口。日志追赶减少停顿：旧 owner 生成快照后继续写 WAL，新 owner 加载快照并追增量，追到切换点再发布新 epoch。代价是需要可靠日志、增量重放和追赶速度控制。双写看起来可用性最高：旧 owner 和新 owner 都写成功才返回。代价是一边成功一边失败、重试重复写、读路径选择和恢复依据都变复杂。

Compute Systems Engine 应先实现冻结或日志追赶，再把双写作为反例分析。若无法回答 request id 如何去重、部分成功如何补偿、旧响应如何处理、恢复时以哪边日志为准，就不要把双写当成“平滑迁移”的简单方案。

再分片是可恢复状态机，不是一次拷贝



图示：本图要证明的命题是：再分片的正确性来自阶段和恢复记录，而不是后台 copy 的成功日志。

Listing 20.4 分区归属记录的示例形态

```

1 enum class OwnershipState : std::uint8_t {
2     Serving,
3     PreparingMove,
4     CopyingSnapshot,
5     CatchingUp,
6     Switching,
7     Draining,
8     Retired,
9 };
10
11 struct OwnershipRecord {
12     std::uint32_t partition_id = 0;
13     std::uint64_t route_epoch = 0;
14     std::uint64_t leader_term = 0;
15     std::uint32_t owner_worker = 0;
16     std::uint32_t target_worker = 0;
17     std::uint64_t snapshot_version = 0;
18     std::uint64_t catchup_log_index = 0;
19     OwnershipState state = OwnershipState::Serving;
20 };

```

提交接口必须检查当前权利 让非法提交难以写出来，是 C++ 接口设计的一部分。提交函数应要求传入 request 和当前 ownership record，并在函数内部检查 partition、epoch、term、owner 和状态，而不是把这些检查散落在调用方。

Listing 20.5 提交前验证所有权和任期

```

1 enum class PartitionCommitDecision : std::int32_t {
2     Accept,
3     StaleRoute,
4     StaleTerm,
5     WrongOwner,
6     NotServing,
7 };
8
9 PartitionCommitDecision can_commit_partition_output(

```

```
10     const PartitionRequest& request,
11     const OwnershipRecord& owner,
12     std::uint64_t current_term) {
13     if (request.partition_id != owner.partition_id ||
14         request.route_epoch != owner.route_epoch) {
15         return PartitionCommitDecision::StaleRoute;
16     }
17     if (request.leader_term != current_term ||
18         owner.leader_term != current_term) {
19         return PartitionCommitDecision::StaleTerm;
20     }
21     if (request.owner_worker != owner.owner_worker) {
22         return PartitionCommitDecision::WrongOwner;
23     }
24     if (owner.state != OwnershipState::Serving) {
25         return PartitionCommitDecision::NotServing;
26     }
27     return PartitionCommitDecision::Accept;
28 }
```

这个函数不是完整分布式存储，只展示边界：提交不是“结果文件写完了”，而是“当前权利允许这个结果进入外部事实”。未来若把控制面换成真实 Raft，`leader_term` 的来源会改变，但提交前验证权利的接口形状仍然成立。

20.4 复制：写成功到底证明什么

只有一个 owner 时，owner 死了就无法继续服务。复制的动机很朴素：让其他副本拥有足够状态，在 owner 故障后可以接管。但复制不是“多写几份文件”。要问客户端看到成功时，哪些副本已经保存了这个事实；读请求从哪里读，允许多旧；故障恢复时哪个副本有资格成为新 owner；落后副本怎样补齐；删除怎样传播。

提交点决定恢复语义 同一条写，在不同提交点下含义完全不同。Leader 内存返回，延迟最低，但 leader 崩溃可能丢已承诺结果；本地 WAL 返回，单节点崩溃恢复更好，但 leader 永久丢失时其他副本可能不知道；多数副本持久化后返回，故障恢复更强，但写延迟和尾部成本上升；所有副本确认最保守，但最慢副本决定可用性。

提交点	购买的能力	主要成本	适合对象
Leader 内存 Leader WAL	极低延迟 单机恢复清楚	崩溃易丢已承诺结果 leader 永久丢失仍危险	临时指标 可重试任务 状态
多数副本 WAL	已提交事实更可恢复	跨节点 RTT 和尾延迟	manifest、 路由元数据
所有副本确认	最强副本一致	可用性受最慢副本限制	少数关键审 计状态

WAL 是复制和 I/O 的连接点 Write-ahead log 的核心不是文件格式，而是状态改变先进入可恢复记录，再对外承诺。第 15 章讲过 write、completion、commit 不能混淆；这里同样成立。Leader 追加日志不等于 follower 持久化，多数确认不等于状态机已经应用，状态机应用不等于 snapshot 已经安全截断。要把 append、persist、commit、apply、snapshot 分开。

Listing 20.6 副本进度摘要

```

1 struct ReplicaProgress {
2     std::uint32_t worker_id = 0;
3     std::uint32_t failure_domain = 0;
4     std::uint64_t last_appended_index = 0;
5     std::uint64_t last_persisted_index = 0;
6     std::uint64_t last_committed_index = 0;
7     std::uint64_t last_applied_index = 0;
8     bool voting_member = true;
9 };

```

这些指标必须按 partition 或 replica group 维度记录。全局平均复制滞后小，不代表热点 partition 健康。一个关键 partition 的 follower 落后，故障时会直接影响可接管性。

复制和备份不同 复制解决在线可用性：某个副本不可用时，系统还能服务。备份解决历史恢复：误删、程序 bug、坏版本写入或运维误操作后，系统能回到过去某个正确版本。复制会快速传播错误删除，不能替代备份。备份要有快照时间点、保留策略、隔离权限、校验和恢复演练。Compute Systems Engine 的 checkpoint 也应如此：只写文件不算完成，必须能恢复并验证 input offset、manifest、去重表和输出 checksum。

20.5 Quorum 和读一致性

Quorum 模型常用 N 表示副本数， W 表示写入需要确认的副本数， R 表示读取需要查询的副本数。若 $R + W > N$ ，读集合和写集合至少有交集。这个交集能帮助读请求发现已提交版本，也能帮助新 leader 找到共同事实。但公式本身不是数据库。系统仍要处理版本比较、并发写、删除 tombstone、慢副本、读修复、反熵和冲突解决。

用恢复案例理解多数派 三副本 A、B、C，写版本 10。若 $W=2$ ，A 和 B 确认后返回成功，A 随后崩溃，B 和 C 中至少 B 知道版本 10。新 leader 从多数派中选出时，有机会看到已提交版本。若只有 A 写完就返回成功，A 崩溃后 B 和 C 都停在版本 9，系统没有证据证明版本 10 曾被承诺。多数派的意义不是公式漂亮，而是故障后保留共同事实。

多数派交集的故障账本 多数派的核心是交集。 $N=3$ 时，任意两个大小为 2 的集合必有一个共同副本； $N=5$ 时，任意两个大小为 3 的集合也必有交集。这个交集让“旧提交事实”和“新选举事实”有机会相遇。若写入在一个多数派上被承诺，新 leader 也必须从一个多数派中产生，那么新 leader 的选举或日志同步路径至少会接触到一个知道旧事实的副本。协议还要规定如何比较 term、log index 和版本，交集本身只是保留证据的最低结构。

配置	已提交写集合	新 leader 可见集合	是否必有交集
$N=3, W=2$	A, B	B, C	是，B 保留版本证据
$N=3, W=1$	A	B, C	否，A 崩溃后证据消失
$N=5, W=3$	A, B, C	C, D, E	是，C 保留版本证据
$N=5, W=2$	A, B	C, D, E	否，两个集合可完全分离

这张表不能单独证明系统安全。副本 C 知道旧事实，还需要协议要求候选 leader 收集足够日志信息、拒绝落后候选、提交前满足当前 term 的规则。否则交集副本即使存在，信息也可能被忽略。多数派不是“投票数量够了就安全”，而是“提交、选举、日志比较和恢复都围绕同一组交集证据工作”。

R、W、N 公式的边界 $R+W>N$ 只说明读集合和写集合至少相交，不说明相交副本一定返回最新值，也不说明冲突如何解决。若副本返回多个版本，读路径要比较 version、term、timestamp 或 vector clock；若删除用 tombstone 表示，读路径要理解 tombstone 是否仍在保留期；若读的是 follower，副本可能落后但仍参与 R。公式解决的是集合交集问题，不解决版本语义。

Listing 20.7 一次 quorum read 还要处理版本比较

```
read key K from replicas A, B
A returns value=v10, version=10, tombstone=false
B returns value=v9, version=9, tombstone=false

reader:
  choose version 10 according to version rule
  optionally repair B to version 10
  return response with observed_version=10
```

如果版本规则靠物理时钟，时钟假设要写清；如果版本规则靠 leader log index，读路径要知道 leader 或多数派确认；如果允许多版本冲突，接口要返回冲突而不是静默选择。Compute Systems Engine 的 manifest 应尽量避免复杂冲突：同一 logical task 只允许一个 accepted attempt，重复 attempt 走幂等或拒绝路径。

读路径也有合同 写路径认真复制，读路径随便读 follower，用户仍可能看到旧状态。读语义要写进接口，而不是藏在实现里。

读语义	能保证什么	成本和适用场景
Best effort stale read	可能返回落后副本状态	延迟低，适合监控预览、搜索、缓存
Monotonic read	同一客户端不读到低于已见版本	需要客户端携带版本，适合用户会话
Read-your-writes	写后自己能读到新值	需要会话版本或 leader 路径
Linearizable read	读开始前已提交写必须可见	延迟高，适合路由、权限、manifest

版本号让旧读可解释 响应中带上 route epoch、leader term、log index、data version 或 replica lag，能让调用方知道自己读到哪个历史点。没有版本号，读到旧值只能猜。控制面读通常需要强读或等价保证；观测型读可以陈旧，但要诚实标注滞后边界。若调度器用旧 route 读决定 owner，会把请求发错；若监控看板读旧统计，只要标注版本，业务风险低得多。

读修复、反熵和 Merkle tree 副本不一致需要修复。读修复在读请求发现旧副本时顺手修复，热数据收敛快，但会增加读路径成本。反熵在后台比较副本并修复冷数据，前台稳定但收敛慢。大规模 key-value 系统常用 Merkle tree 之类摘要结构先比较范围哈希，再下钻到差异 key。Compute Systems Engine 不必实现完整 Merkle tree，但可以用 manifest checksum 学同一个原则：先比摘要，再决定是否重读或修复。

20.6 副本放置、成员变更和删除

复制数量不等于可靠性。三个副本如果在同一台机器、同一机架、同一电源域或同一个对象存储故障域里，某个物理故障可能同时影响它们。工业系统讨论副本时一定会讨论 failure domain：磁盘、机器、机架、可用区、网络交换机、维护批次和软件版本。副本放置不是运维细节，而是复制协议购买可靠性的前提。

故障域进入放置策略 单节点项目也可以模拟故障域。给 worker 加 `failure_domain`，要求同一个 replica group 的 voting members 尽量分布在不同 domain。然后注入一个 domain 故障，看是否仍有 quorum。这个实验会暴露：N=3 只是副本数，真正的问题是这三个副本是否会被同一个故障同时打掉。

成员变更不能随手改副本列表 复制组扩容、缩容、替换机器或迁移副本时，quorum 的定义会变化。成员变更比普通写入更危险，因为旧配置和新配置如果没有安全交叠，可能出现两个多数派，各自认为自己能提交。成熟共识协议会把成员变更写入日志，用联合共识或分阶段切换保证交叠。Compute Systems Engine 可以简化为两阶段：先把新副本加入为 non-voting member 并追平日志，再在控制日志中提交配置切换，最后移除旧副本。不能直接把 vector 里的 worker id 改掉。

删除需要 tombstone 和安全点 复制系统中的删除不能简单等同于 `erase`。若某个副本离线时错过删除，恢复后可能把旧值重新传播回来。Tombstone 用删除标记表示“这个 key 已被删除”，让落后副本也能学习删除事实。Tombstone 需要保留到系统确信所有相关副本、日志和 snapshot 都越过删除点；保留太短会旧值复活，保留太长会增加空间和查询成本。Compute Systems Engine 里的旧 shuffle block 清理也同构：只有当 manifest generation、checkpoint 和恢复策略都不再需要某个 block 时，物理删除才安全。

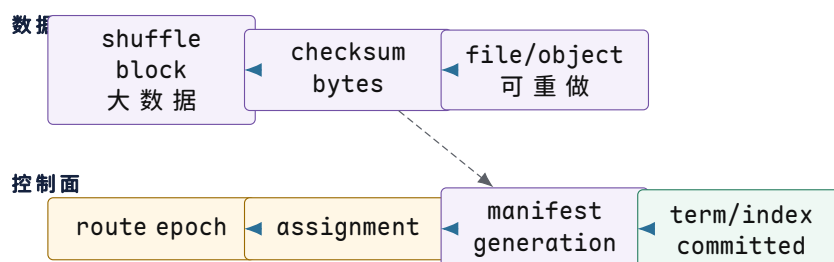
再平衡要受背压约束 副本迁移和再平衡会消耗网络、磁盘和 CPU。后台迁移如果不受限制，会和前台请求争资源，导致更多 timeout 和重试。工业系统通常限制同时迁移的 partition 数、每个 worker 的迁移带宽、每个 replica group 的落后上限，并在高水位时暂停迁移。这里又回到第 15 章和第 16 章：背压不是局部队列技巧，而是跨 I/O、运行时和分布式控制面的共同纪律。

20.7 Leader、Term 和共识边界

复制还没有解决控制面分裂。网络分区时，旧 Driver A 认为自己仍是 leader，新 Driver B 在另一侧也被选出。A 发布 epoch 42a，B 发布 epoch 42b，两个 manifest 都声称自己有效。Worker 如果只要听到 Driver 命令就执行，最终 reduce 会看到两个互斥真相。这就是共识边界：系统需要让少量关键控制面事实进入一条唯一顺序。

共识解决什么 Leader 决定谁能提出控制面写入；term 区分新旧领导权；log index 给控制操作排序；commit index 表示哪些操作已经被足够副本接受并可对外生效；apply 表示状态机已经执行已提交日志。共识的价值不是让操作更快，而是在故障、延迟和旧消息中保持一条可恢复历史。

共识日志只承载控制面事实，数据面走高吞吐路径



大数据块不必全部进入共识日志；决定哪些块有效的元数据必须进入强一致控制面。控制面太大，吞吐会垮；控制面太弱，结果会分叉。

图示：本图要证明的命题是：共识应用在“决定哪些数据有效”的小状态上，而不是替代数据面吞吐设计。

共识不解决什么 共识不自动解决业务幂等、热点 key、数据布局、磁盘损坏、查询优化、外部 I/O 重放和读路径选择。请求去重表仍要进入状态机，输出文件仍要校验，重试预算仍要限制，热点 key 仍要拆，snapshot 仍要验证。把所有问题推给共识，是分布式内容常见的偷懒。

从 manifest commit 推导日志复制 用最终项目的 manifest 提交来理解共识日志。某个 Worker 写出 block 后，请求 Driver 接受 `task=17, attempt=2, block=B17a2`。如果只有当前 Driver 的内存 map 记录这个接受事实，Driver 崩溃后系统无法证明该 attempt 是否已经提交。若把接受事实写入复制日志，流程会变成：leader 在当前 term 创建日志条目；复制给 followers；多数派持久接受后推进 commit index；状态机应用该条目，manifest generation 增加；Driver 才对外返回 committed。

Listing 20.8 manifest commit 进入控制日志的事件链

```

client request:
  CommitTask(task=17, attempt=2, block=B17a2)

leader:
  append log[index=881, term=12, op=CommitManifestEntry]
  replicate entry to followers

followers:
  persist entry if previous log check passes
  reply accepted(index=881, term=12)

leader:
  after quorum accepted, commit_index = 881
  apply entry to state machine
  manifest_generation++
  reply Committed
  
```

这里的提交点不是 worker 写完 block，也不是 leader append 到本地日志，而是日志条目被当前配置的多数派承认并被状态机应用。Worker reply 丢失后重试，同一 `task, attempt, block` 会命中幂等表；不同 attempt 会被拒绝或进入候选审计。这样，重试和恢复都围绕同一条控制日志，而不是围绕目录扫描。

prevLogIndex 和 prevLogTerm 的直觉 日志复制不能只说“把新条目发给

follower”。Follower 可能曾经跟随旧 leader，尾部日志和新 leader 不一致。AppendEntries 一类协议会携带前一条日志的位置和 term: `prevLogIndex`、`prevLogTerm`。Follower 只有在本地对应位置匹配时才接受新条目；不匹配则拒绝，leader 回退查找共同前缀。这个机制保证日志不是随意拼接，而是在共同历史后继续追加。

Listing 20.9 日志尾部分叉时的拒绝路径

```
leader log:
  index 10 term 4
  index 11 term 5
  index 12 term 5

follower log:
  index 10 term 4
  index 11 term 4 ; old leader wrote a conflicting tail

append request:
  prevLogIndex=11
  prevLogTerm=5
  new entry index=12 term=5

follower decision:
  local index 11 term is 4, not 5
  reject append and require leader to find earlier common prefix
```

这条路径和 manifest 恢复同构：不能因为一个文件或条目存在就接受它，要先证明它接在当前权威历史之后。共同前缀是分布式版本的“不从孤儿文件恢复提交事实”。

commit index 和 apply index 要分开 日志条目被多数派接受后可以成为 committed，但状态机未必已经应用到该位置。Leader 和 follower 都需要区分 `commit_index` 与 `last_applied_index`。前者是协议承诺，后者是本地执行进度。读请求如果要线性化，必须读到至少覆盖目标 index 的已应用状态；checkpoint 也必须说明包含到哪个 log index。

Listing 20.10 commit 与 apply 的差异

```
log contains entries 1..900
commit_index = 880
last_applied_index = 872

state machine:
  entries 873..880 are committed but not yet visible in local state

linearizable read:
  wait until last_applied_index >= required index
```

若把 committed 和 applied 混在一起，读路径可能返回旧状态，snapshot 可能漏掉已承诺条目，恢复可能重复外部副作用。控制面状态机通常很小，仍要保留这两个字段，因为它们是读一致性和恢复边界的证据。

线性读为什么不能随便读 follower 读请求看似不修改状态，实际上也要选择一致性合同。若直接读任意 follower，它可能落后于 leader 的 commit index，也

可能还没有 apply 最新 committed 条目；网络分区时，旧 leader 或旧 follower 还可能停在旧 term。这样的读适合明确声明为 stale read 或 cached read，不能伪装成线性读。线性读要求读结果能放进全局顺序里，并且排在读开始前已经完成的写之后。

有三类常见方案。第一，把读也作为日志操作提交，简单但成本高。第二，leader 在当前 term 确认自己仍有 quorum，再等状态机 apply 到当前 commit index，然后服务读。第三，在强时间假设下使用 lease read，但 lease 必须考虑时钟漂移、暂停和网络延迟。无论哪种，读路径都要写明它依赖的权威事实，而不是因为“读不写”就绕过控制面。

Listing 20.11 线性读的最小检查顺序

```
linear read on leader:
  confirm this node is still leader in current term
  confirm leadership with quorum or valid lease assumption
  choose required_index = current commit_index
  wait until last_applied_index >= required_index
  read from state machine snapshot at or after required_index
```

这条顺序和本地 manifest 读取同构。读取 manifest 时，不能从临时 block 目录直接推断结果；读取分布式控制面时，也不能从某个副本当前内存直接推断权威状态。必须先证明当前节点有读取权利，再证明本地状态至少应用到需要的日志位置。

Follower read 的安全边界 Follower 不是不能服务读，但必须声明语义。Stale read 可以从 follower 返回，适合监控、近似负载信息和非关键查询；bounded stale read 需要写最大落后界限；linearizable follower read 需要向 leader 或 quorum 验证 read index，再等本地 apply 到该 index。若一个接口既想要 follower 低延迟，又想要线性一致，又不做验证，那只是把一致性成本藏起来。

读类型	可接受事实	不能承诺
Stale read	本副本已应用状态，可能落后	不能保证看到最近提交
Bounded stale read	落后不超过声明界限或版本	不能超过边界仍继续返回成功
Leader linear read	当前 leader 权利和 apply index 已验证	需要 quorum 或 lease 假设
Follower linear read	通过 read index 或 quorum 验证后读取	不能只读本地内存

Compute Systems Engine 的控制面读可以先实现 leader linear read 和 explicit stale read 两类。前者用于提交、路由变更和 manifest generation；后者用于 dashboard、调试和不影响外部事实的观察。接口名字和报告字段要把语义写清，避免把“快但可能旧”的结果传给需要强一致的提交路径。

Listing 20.12 控制日志条目的示例形态

```
1 enum class ControlOperationKind : std::uint16_t {
2     UpdateRouteEpoch,
3     ChangePartitionOwner,
4     CommitManifestEntry,
5     StartMigration,
6     FinishMigration,
7 };
8
```

```

9 struct ControlLogEntry {
10     std::uint64_t log_index = 0;
11     std::uint64_t leader_term = 0;
12     ControlOperationKind kind = ControlOperationKind::UpdateRouteEpoch;
13     std::uint64_t request_id = 0;
14     std::uint32_t partition_id = 0;
15     std::uint64_t route_epoch = 0;
16     std::uint64_t payload_checksum = 0;
17 };

```

Snapshot 和日志截断 控制日志无限增长不可接受。Snapshot 保存某个 log index 的状态机快照，恢复时先加载 snapshot，再回放之后的日志。Snapshot 必须记录 last included index 和 term，不能只写一个状态文件。截断旧日志前，要确认需要恢复的节点已经拥有 snapshot 或不再需要旧日志。计算作业也有同构问题：stage checkpoint 成功后，哪些 shuffle block 可以删除，哪些还要保留给回滚或重试。

Snapshot 是状态机前缀，不是文件副本 Snapshot 的语义是“状态机执行到某个日志前缀后的结果”。它不是随手复制当前内存 map，也不是把 manifest 文件压缩一下。一个有效 snapshot 至少要写入 last_included_index、last_included_term、状态机版本、成员配置、route epoch、manifest generation、去重表摘要和 checksum。恢复时，系统先加载 snapshot，确认 checksum 和版本，再回放 last_included_index 之后的日志。

Listing 20.13 控制面 snapshot 的最小身份

```

Snapshot:
  snapshot_id
  last_included_index
  last_included_term
  config_generation
  route_epoch_table_checksum
  manifest_generation
  dedup_table_checksum
  state_machine_version
  payload_checksum

```

若 snapshot 没有 last included term，新 leader 无法判断日志截断边界是否和 follower 的历史匹配；若没有状态机版本，升级后可能用旧解释读取新状态；若没有去重表摘要，恢复后可能重新接受已经提交过的 request。Snapshot 购买的是恢复速度和日志空间，不是降低提交协议要求。

状态机必须确定 共识只决定日志顺序，状态机应用日志必须确定。若应用日志时读取本地时间、随机数、目录扫描顺序或不稳定的哈希遍历，不同副本会得到不同状态。C++ 实现中，生成 manifest 或 snapshot 时不要依赖 std::unordered_map 的遍历顺序；要按 key、partition id 或 log index 排序。外部 I/O 也不能在日志重放时无条件重复执行，必须通过提交表和幂等协议保护。

20.8 控制日志模拟器：把共识边界落到项目

前面已经说明 leader、term、log index、commit index、apply index 和 snapshot 的意义。还差一步：这些字段怎样变成 Compute Systems Engine 里能运行、能注入故障、能写报告的最小控制日志。这里不追求实现完整 Raft；目标是让 route epoch、partition owner、manifest generation 和 dedup table 都只

能从一条已提交的控制日志中产生。这样第 18 章的项目代码不会把“当前内存里看见的 owner”误当成权威。

控制日志只承载小而关键的事实 数据面仍然走高吞吐路径:shuffle block 写临时文件,checksum 和字节数进入候选元数据,真正的大字节不进控制日志。控制日志承载的是决定外部事实的小对象:哪个 route epoch 生效,哪个 owner 有提交权,哪个 manifest entry 被接受,哪个 request id 已去重,哪个 snapshot 覆盖到哪个 log index。若把所有数据字节塞进日志,吞吐会垮;若连 manifest 接受事实都不进日志,旧 leader 和恢复程序会制造分叉。

Listing 20.14 控制日志模拟器的核心记录

```
1 enum class ControlDecision : std::uint8_t {
2     Accepted,
3     RejectedStaleTerm,
4     RejectedLogGap,
5     RejectedStaleRoute,
6     RejectedDuplicate,
7 };
8
9 struct ControlLogState {
10     std::uint64_t current_term = 0;
11     std::uint64_t last_log_index = 0;
12     std::uint64_t commit_index = 0;
13     std::uint64_t apply_index = 0;
14     std::uint64_t manifest_generation = 0;
15     std::uint64_t route_epoch_table_checksum = 0;
16     std::uint64_t dedup_table_checksum = 0;
17 };
```

这个状态不是数据库完整 schema,只是共识边界的最小账本。`current_term` 拒绝旧 leader; `last_log_index` 和 `commit_index` 区分本地追加与已承诺事实; `apply_index` 区分已承诺与本地已执行; `manifest_generation` 让下游知道读到哪个 manifest 历史;两个 checksum 让 snapshot 和恢复能证明 route 表、去重表不是任意内存状态。

追加、持久化、多数确认、提交、应用 一次控制写入至少经过五个动作。Leader 先检查 term 和输入身份,生成下一个 log index;本地持久化 entry;发送给 followers;获得当前配置多数派持久接受后推进 commit index;最后按 log 顺序应用到状态机。只有 apply 完成后,route epoch 或 manifest generation 才能对业务可见。

Listing 20.15 控制日志提交链

```
append(entry):
    require entry.term == current_term
    assign entry.log_index = last_log_index + 1
    persist entry locally
    replicate to followers with prevLogIndex and prevLogTerm
    if quorum persisted:
        commit_index = entry.log_index
    while apply_index < commit_index:
        apply log[apply_index + 1] to deterministic state machine
        apply_index++
```

这条链把几个常见误区拆开。本地 append 成功，不代表提交；多数派持久化成功，不代表本节点状态机已经可读；状态机 apply 成功，才说明 manifest 或 route 表对外可见。若 worker 在 append 后就收到成功响应，leader 崩溃会丢承诺；若 read path 只看 commit index 不等 apply index，就可能读旧 route；若 snapshot 不记录 apply index，恢复后不知道哪些日志已经影响过 manifest。

状态机不允许绕过提交入口 控制状态机可以小，但入口必须唯一。更新 route epoch、接受 manifest entry、推进 migration state、写 tombstone、记录 dedup request，都应通过 `apply_control_entry`。恢复重放、在线 commit 和测试注入不能各写一份逻辑。这样旧 leader 请求、重复 attempt、迟到 reply 和目录扫描都只能得到同一类判定。

Listing 20.16 控制状态机应用入口

```
apply_control_entry(entry):
    if entry.kind == UpdateRouteEpoch:
        require entry.route_epoch == current_route_epoch(partition) + 1
        update owner and route epoch

    if entry.kind == CommitManifestEntry:
        require request_id not in dedup table
        require entry.route_epoch == current route epoch
        require entry.term == current term
        append manifest record in sorted deterministic order
        manifest_generation++
        remember request_id in dedup table

    recompute deterministic checksums for snapshot/report
```

这段伪代码故意把 manifest 接受放在控制状态机里，而不是放在 worker 文件写成功之后。Worker 写完 block 只产生候选；控制日志决定候选是否成为业务事实。重复 attempt 可以拥有完整字节和正确 checksum，但若 request id 已被 dedup 表接受，或者 route epoch 已过期，它只能进入 late 或 stale 路径。

四条不变量 控制日志模拟器最重要的不是代码行数，而是不变量。第一，route epoch 只能通过 committed 且 applied 的日志前进，不能由本地内存修改。第二，manifest generation 只能在应用 `CommitManifestEntry` 后前进，不能由目录扫描或 completion 成功前进。第三，任何提交请求的 term、route epoch 和 owner 必须与当前已应用状态匹配；不匹配时返回结构化拒绝，例如 `RejectedStaleTerm` 或 `RejectedStaleRoute`。第四，snapshot 必须覆盖 `last_included_index/term`、route epoch 表 checksum、manifest generation、dedup table checksum 和成员配置，否则恢复后无法证明它代表哪段日志前缀。

Listing 20.17 控制日志不变量

```
invariant route_epoch:
    visible route epoch == applied state, not local cache

invariant manifest_generation:
    generation advances only when CommitManifestEntry is applied

invariant fencing:
    request.term == applied current_term
    request.route_epoch == applied route_epoch(partition)
```

```

request.owner == applied owner(partition)

invariant snapshot:
  snapshot has last_included_index, last_included_term,
  route_epoch_table_checksum, manifest_generation,
  dedup_table_checksum, config_generation

```

这些不变量应进入测试断言。只靠日志打印“reject stale request”不够；测试要在旧请求、旧 leader、恢复扫描和 snapshot 加载后读取状态机，确认 generation 没分叉、route epoch 没倒退、dedup 表没有丢失。

确定性场景：旧 term 7 与新 term 8 下面的场景适合作为项目里的第一组控制日志测试。三副本 A、B、C，term 7 的 leader A 曾追加 index 51：接受 partition 12、epoch 41 的 manifest 候选，但只复制到 A 自己，没有达到多数派。随后网络分区，B 和 C 在 term 8 选出新 leader B。B 的日志停在 index 50，于是它以 index 50 term 7 作为共同前缀，追加 index 51 term 8：把 partition 12 route epoch 切到 42，owner 改为 B；多数派 B/C 持久化后提交并应用。

Listing 20.18 旧 leader 尾部被新 leader 修正

```

A log:
  index 50 term 7  committed previous state
  index 51 term 7  manifest candidate from old owner

B log:
  index 50 term 7  committed previous state

C log:
  index 50 term 7  committed previous state

new leader B term 8:
  append index 51 term 8 with prevLogIndex=50 prevLogTerm=7
  followers delete conflicting uncommitted tail after index 50
  quorum accepts index 51 term 8
  apply route_epoch(partition 12) = 42, owner = B

```

此时旧 leader A 恢复网络，尝试把 term 7、epoch 41 的 manifest candidate 提交。正确结果不是“文件存在所以补提交”，也不是“旧 A 还活着所以让它完成”。控制面先比较 term：请求 term 7 小于 current term 8，返回 `RejectedStaleTerm`；即使忽略 term 再看 route epoch，也会发现 epoch 41 小于当前 epoch 42，返回 `RejectedStaleRoute`。这两个拒绝都应进入报告计数，且 manifest generation 不变。

Listing 20.19 旧 leader 提交被结构化拒绝

```

old request:
  leader_term=7
  route_epoch=41
  partition=12
  owner=A
  block=BA

applied state:

```

```

current_term=8
route_epoch(partition 12)=42
owner(partition 12)=B

decision:
  RejectedStaleTerm
  do not append manifest
  count stale_term_rejected++
  classify BA as stale candidate

```

这个测试把 quorum、日志共同前缀、term fencing、route epoch fencing 和 manifest gate 合在一起。它比单独问“多数派为什么安全”更有工程价值，因为每一步都有可观测字段：哪个 index 被提交，哪个 index 被应用，旧 tail 怎样被替换，旧请求为什么被拒绝，manifest generation 是否保持单调。

线性读、check-index 和 lease read 的边界 控制日志写清之后，读路径也要选择语义。最保守的是 leader read：请求到 leader，leader 确认自己仍处于当前 term，并等待 `last_applied_index >= required_index` 后返回。更常见的优化是 check-index：leader 先通过一次轻量多数确认或等价机制证明自己仍是 leader，再把当前 commit index 作为 read index，本地 apply 到该 index 后读。Lease read 更快，但依赖时钟和 lease 假设，必须记录最大时钟偏差、续租方式和暂停边界。

Listing 20.20 控制面读的三种边界

```

leader read:
  contact leader
  require current term
  wait apply_index >= commit_index

check-index read:
  leader proves authority through quorum heartbeat
  read_index = commit_index observed under that authority
  wait apply_index >= read_index

lease read:
  rely on unexpired leader lease
  require explicit clock and pause assumptions

```

Route 查询、manifest generation 查询、提交前 owner 检查应默认使用 leader read 或 check-index。监控统计可以使用 stale read，但响应必须带 `observed_term`、`observed_commit_index`、`observed_apply_index` 和 `replica_lag`。若接口返回一个裸 owner id，调用方无法判断它读到的是当前事实、旧事实还是未应用事实。

控制日志可观察状态 控制日志模拟器要能还原上述场景。输出不需要一开始就很漂亮，但必须能回答“这个 manifest entry 是在哪个 term、哪个 index、哪个 apply 点变成事实的”。最低状态可以先固定成下面这一组，后续再扩展延迟分布、每副本 lag 和迁移耗时。

Listing 20.21 控制日志报告字段

```

control_log.appended
control_log.persisted_local

```

```
control_log.quorum_accepted
control_log.commit_index
control_log.apply_index
control_log.conflict_truncated
route.epoch
manifest.generation
stale_term_rejected
stale_route_rejected
duplicate_rejected
snapshot.last_included_index
snapshot.last_included_term
```

Listing 20.22 控制日志报告摘要示例

```
control_log.appended=91
control_log.quorum_accepted=88
control_log.commit_index=88
control_log.apply_index=88
control_log.conflict_truncated=1
route.epoch.partition12=42
manifest.generation=37
stale_term_rejected=1
stale_route_rejected=4
duplicate_rejected=9
snapshot.last_included_index=64
snapshot.last_included_term=8
```

20.9 工业设计参照

工业案例的阅读口径 读工业系统不是为了背系统名，而是读它面对的约束、核心机制、购买的能力、付出的成本，以及 Compute Systems Engine 应采用、简化或拒绝哪一部分。

工程案例

Kafka partition、leader 和 ISR Kafka 把 topic 拆成 partition，每个 partition 有 leader，followers 复制日志。它面对的是高吞吐追加、broker 故障、生产者重试和消费者按 offset 读取。核心机制是 partition leader 定义写入口，日志 offset 定义顺序，ISR 表示足够跟上的副本集合，producer id 和 sequence number 处理重试重复。代价是 partition 数量、leader 分布、ISR 抖动、跨机复制和消费者 rebalancing 都会影响尾延迟和可用性。Compute Systems Engine 可以借鉴 partition leader、offset、producer request id 和 manifest offset 的思想，但不应把每条 shuffle 记录都塞进强控制日志。数据面批量写 block，控制面提交 block 引用即可。

工程案例

Dynamo/Cassandra 风格的一致性哈希和 quorum 这类系统面对节点频繁增减、低延迟读写和跨副本不一致。核心机制包括一致性哈希或 token range、N/R/W quorum、hinted handoff、read repair、反熵和版本冲突

处理。它们说明 quorum 不是单独公式，而是一整套补副本、修冲突和处理删除的机制。

代价是读写可能看到多个版本，删除需要 tombstone，后台修复消耗 I/O，最后写入胜出可能因为时钟问题丢语义。Compute Systems Engine 可以借鉴虚拟分片、读写 quorum 的实验和后台校验；暂时拒绝复杂冲突模型，因为计算引擎的 manifest 和 route 元数据应尽量保持单 leader 强提交。

工程案例

etcd/Raft 控制面 etcd 用 Raft 维护小而关键的 key-value 控制状态，常被 Kubernetes 等系统用于配置、租约和集群元数据。它面对的是控制面必须一致、节点可能失败、旧 leader 可能恢复、客户端 watch 可能断开。核心机制是 leader、term、log index、commit index、snapshot 和 watch revision。

代价是写入要走 leader 和多数派，吞吐不应和数据面混在一起。它给 Compute Systems Engine 的启发非常直接：route epoch、partition assignment、manifest generation 属于控制面，可以用简化日志模拟；shuffle block 和大输出属于数据面，不要进入共识热路径。

工程案例

HDFS NameNode 和 block replication HDFS 把文件切成 block，DataNode 存储副本，NameNode 维护命名空间和 block 元数据。它面对的是大文件吞吐、DataNode 故障和副本放置。核心机制是控制面集中管理 block location 和副本健康，数据面负责大块传输；副本放置考虑机架等故障域。

代价是 NameNode 元数据成为关键控制面，需要 checkpoint、journal 和高可用机制。Compute Systems Engine 可以借鉴“控制面记录 block 引用和 checksum，数据面传输大 block”的分层；同时要记住复制数量不等于备份，误删和坏写会被复制。

工程案例

Spanner 的强一致读写和时间假设 Spanner 这类系统把复制、事务和时间边界结合起来，面对跨数据中心一致性和高可用读写。它的核心思想之一是把提交时间、复制日志和读时间戳变成协议对象，让读写能在明确时间边界下解释。

代价是系统复杂度、时钟基础设施和跨地域延迟都很高。Compute Systems Engine 不应照搬 TrueTime 或跨地域事务，但应学习一个原则：读语义要明确，时间假设要写出来，租约和时间戳不能被当作无限可靠的魔法。

20.10 贯穿审查：只跟踪一个 Key 的一生

分片、复制和共识一旦讲得太大，就容易变成名词列表。一个更有效的审查方法，是只跟踪一个 key: `event=login`。它最初属于 partition 12, owner 是 worker A, route epoch 是 41。随着流量增加，它被识别为热点，被拆成两个执行子分片，迁移到 worker B 和 worker C。期间 A 仍在处理旧请求，B 正在加载快照，C 正在追增量，Driver 又经历一次 leader term 切换。只要把这个 key 的所有事件写清，本章的大部分概念都会自然出现。

进入分片函数 记录被解析后，系统根据 key 规范化规则和 partition function version 把 `login` 映射到 partition 12。这里至少有三个事实：key 的字节表示、hash 或 range 规则、partition version。若两个 worker 对大小

写、编码或 hash seed 理解不同，同一个业务 key 会进入不同 partition。分片函数不是数学背景，而是协议的一部分。

成为热点 全局吞吐正常时，partition 12 仍可能长尾，因为大多数 login 都打到它。报告不能只看平均每个 partition 多少记录，要看最大 partition、top-k key 占比、reduce p99、队列水位和二级合并成本。热点 key 逼迫系统把执行身份和业务身份分开：子 key 可以用于并行，最终输出仍必须聚回 login。

迁移和复制 Epoch 41 时 A 有权提交 partition 12；Epoch 42 发布后，B 和 C 接管子分片。A 可以清理旧状态，可以转发旧请求，可以返回 StaleRoute，但不能在 epoch 42 下提交新结果。复制时还要问：A 的版本 100 是否已被多数确认，B 或 C 是否追到 100，哪个副本能在 A 崩溃后接管。跟踪一个 key 的版本，比背 quorum 公式更能说明哪个历史被承诺过。

读取和删除 迁移后，监控看板可以读带版本的 stale 统计，调度器不能用 stale route 决定 owner，manifest commit 必须读强一致控制面。旧子分片和旧 block 也不能立刻物理删除：旧 attempt 响应可能迟到，落后副本可能还要补日志，恢复程序可能需要解释旧文件。清理要等 manifest generation、checkpoint 和 tombstone 安全点都越过旧状态。

20.11 完整执行账本：热点 key 从发现到提交

下面把 event=login 的生命周期写成可手算账本。初始状态下有 64 个 partition，login 通过 hash 落到 partition 12，owner 是 worker A，route epoch 为 41，leader term 为 9。输入开始倾斜后，partition 12 的队列水位上升，reduce p99 超过阈值，Driver 决定把 login 作为热点 key 拆成 4 个执行子分片，并迁移到 worker B 和 worker C。

初始控制面 控制面最初只有普通分片规则，没有热点 route。

Listing 20.23 初始 assignment 表

```
partition  owner  route_epoch  leader_term  state
12          A      41             9           Serving

hot_route:
  empty

partition_function:
  version=3
  rule=hash(event_type) % 64
```

这个状态下，所有 login 记录都进入 partition 12。若只增加 worker 数，login 仍然落到同一个 partition，长尾不会消失。热点检测必须看 key 级别和 partition 级别两个水位：partition 12 很热，且其中 login 占比很高，才说明需要 key 拆分；如果 partition 12 中许多 key 都均匀偏热，扩 partition 或迁移 owner 可能更合适。

热点检测账本 热点检测不能只看某一秒的峰值。它应记录窗口、阈值、持续时间和冷却规则，避免短暂 burst 触发频繁迁移。

Listing 20.24 热点检测摘要

```
window=30s
partition=12
partition_share=0.29
top_key=login
```

```

top_key_share_in_partition=0.82
reduce_p99_ms=740
queue_high_water=1048576
decision=SplitHotKey(login, shards=4)
route_plan_version=17

```

这里的 `route_plan_version` 很关键。它把“什么时候决定拆分、拆成几份、分到哪些 worker、如何二级合并”固定下来。若同一个作业内先用 2 个子分片，后用 4 个子分片，manifest 必须能解释每个 block 属于哪个 route plan。否则 final reduce 会把不同规则下的 partial 混在一起。

准备迁移 Driver 先写控制日志：准备把 `login` 从普通 partition route 升级为 hot route。此时旧 owner A 仍服务旧 route，新 owner B/C 还没有提交权。准备阶段只创建计划，不改变外部可见路由。

Listing 20.25 Prepare 阶段控制日志

```

log_index=201
term=9
operation=StartHotKeyMigration
key=login
from_partition=12
from_owner=A
from_epoch=41
target_shards=4
target_workers=[B,C,B,C]
target_epoch=42
state=PreparingMove

```

如果 Driver 在这里崩溃，恢复时看到 `StartHotKeyMigration` 但没有 `SwitchHotRoute`，外部路由仍是 epoch 41。恢复程序可以继续迁移，也可以撤销计划；不能让 B/C 接受 epoch 42 提交，因为 Switch 尚未发生。

快照和追增量 Worker A 给 `login` 的当前 partial 状态生成快照，记下 `snapshot_version=100`。之后 A 继续接收 epoch 41 的请求，并把增量写入 WAL。B/C 加载快照后，从日志追到切换点。追赶速度必须超过新写入速度，否则迁移永远追不上，需要限流或冻结。

Listing 20.26 Snapshot 和 CatchUp 账本

```

Snapshot:
  key=login
  from_owner=A
  snapshot_version=100
  checksum=0x8f10

CatchUp:
  B shard 0 catches log 101..128
  C shard 1 catches log 101..128
  B shard 2 catches log 101..128
  C shard 3 catches log 101..128

Validate:
  merged_shard_checksum == source_checksum_at_128

```

Validate 不是可选步骤。若 B/C 的四个子分片合起来的 checksum 与 A 在切换点的 checksum 不同，说明快照、增量或拆分函数有错。此时不能发布 epoch 42。错误应停在控制面，而不是让 reduce 后面才发现计数不一致。

Switch: 提交新路由 只有当快照和追增量通过校验，Driver 才追加 `SwitchHotRoute`。这条日志一旦 committed，epoch 42 成为外部事实：新请求按 hot route 进入 4 个子分片，A 失去对 `login` 的提交权。

Listing 20.27 Switch 后的路由表

```
route_epoch=42
hot_route(login):
  shard 0 -> worker B, partition 12.0
  shard 1 -> worker C, partition 12.1
  shard 2 -> worker B, partition 12.2
  shard 3 -> worker C, partition 12.3
final_merge:
  login = sum(shard 0, shard 1, shard 2, shard 3)

old_owner:
  A may forward or reject epoch 41 requests
  A cannot commit login output for epoch 42
```

Switch 的提交点要进入强控制面。若两个 Driver 分别发布不同的 epoch 42，系统就分叉。若 Switch 只写在内存里，Driver 崩溃后 B/C 可能认为自己有权提交，恢复程序却看不到证据。Route epoch 是提交权，不是缓存字段。

旧写到达 A Switch 后，一条旧 epoch 41 写请求迟到到达 A。正确处理不是“看 A 曾经负责过 partition 12”，而是比较 request route epoch 与当前 ownership record。A 应返回 `StaleRoute` 或把请求转发到新路由；无论哪种策略，它都不能写 manifest。

Listing 20.28 旧路由写的拒绝路径

```
request:
  key=login
  route_epoch=41
  owner=A
  request_id=9007

current:
  route_epoch=42
  hot_route(login)=enabled

decision:
  StaleRoute
  increment stale_route_rejected
  do not append manifest
  optionally return new route epoch 42
```

这条拒绝路径要有测试。没有测试时，旧 owner 通常会在“只是清理临时数据”或“兼容旧客户端”的名义下重新获得写权。Fencing 的含义就是旧 owner 还活着也不能提交。

二级 reduce 拆成 4 个子分片后，业务输出仍然只有 `login` 一个 key。二级

reduce 读取 manifest 中 route plan 17 的四个子分片 block，按 `final_merge` 规则求和。它不能把子分片名当业务 key 输出，也不能把 route plan 16 和 route plan 17 的 block 无条件混合。

Listing 20.29 二级 reduce 的语义伪代码

```
for each hot key route plan:
    entries = manifest.entries_for(key, route_plan_version)
    require all required shards are present or have empty marker
    total = 0
    for entry in entries ordered by shard_id:
        total += read_partial(entry.block)
    emit business_key, total
```

空子分片也要有 marker。否则 reduce 无法区分“这个 shard 没有数据”和“这个 shard 丢了”。有些系统允许稀疏 manifest，只记录非空 block，但必须有 route plan 和 expected shard count，让 reduce 能判断缺失是否合理。

清理安全点 Switch 完成后，A 的旧状态和临时 block 不能马上删除。安全删除需要满足几个条件：manifest generation 已经越过所有可能读取旧 route 的阶段；checkpoint 记录了 epoch 42；迟到 epoch 41 request 的最大 deadline 已过；B/C 的 route plan 17 输出已提交；恢复程序不再需要 A 的快照诊断。满足条件后，A 可以把旧状态写 tombstone，再清理物理文件。

Listing 20.30 旧 route 清理条件

```
safe_to_delete_old_route(login, epoch 41):
    manifest_generation >= generation_after_hot_merge
    checkpoint_epoch >= 42
    max_deadline_of_epoch_41_requests_expired
    recovery_audit_written
    tombstone_committed
```

清理是状态机的一部分，不是后台脚本的随意行为。删早了，恢复和迟到响应无法解释；删晚了，占用磁盘和内存，但语义仍安全。工程取舍通常先保证删晚可控，再优化清理速度。

审计材料 最终运行证据应能按 key 查询：`login` 在 epoch 41 属于 partition 12，热点检测何时触发，拆分规则版本是多少，快照 checksum 是多少，增量日志范围是什么，Switch 的 log index 是多少，epoch 42 发布后旧请求拒绝了多少，最终 manifest 引用哪些子分片。这样的审计材料不是装饰，而是出现错账、重复提交或长尾时证明系统可信的证据。

20.12 实现路径

Compute Systems Engine 不要求写完整生产级共识。目标是实现一个最小但语义清楚的分片复制模拟：Driver 维护 partition assignment、route epoch、migration state、replica progress、control log 和 manifest generation；Worker 处理数据面 block；MessageBus 注入旧路由请求、旧 leader commit、慢副本和崩溃恢复。

实现边界 第一版，给 shuffle partition 增加 `partition_id`、`partition_version` 和 `route_epoch`。第二版，实现 partition assignment 表，所有提交都走 `can_commit_partition_output`。第三版，加入热点 key 生成器，比较普通 hash、map-side combine、热 key 拆分和二级 reduce。第四版，实现冻结式再分片状态

机。第五版，加入三副本元数据日志的简化多数提交。第六版，加入读模式：stale read、monotonic read、leader read。第七版，加入 snapshot 和恢复演练。

控制面判读矩阵

路径	故障或压力输入	判读重点
热点 key	一个 key 占大多数记录	普通 hash 长尾，拆分后二级合并正确
旧路由写	epoch 41 写 在 epoch 42 后到达	旧 owner 返回 StaleRoute, manifest 不变
迁移崩溃	Prepare/CatchUp/Switch 各阶段崩溃	恢复按迁移记录继续或回滚
落后副本接管	版本只在单副本上存在	未达提交点的版本不能被承诺
多数提交	三副本中一个慢、一个故障	多数可用时提交，少于多数时拒绝
旧 leader 写	新 term 后旧 Driver 尝试提交	StaleTerm 增加，manifest generation 不分叉
读一致性	follower 落后两个版本	stale/monotonic/leader read 行为可解释
snapshot 恢复	日志截断后重启	snapshot 加后续日志恢复同一状态

运行证据 至少保留这些运行证据：partition 总数、route epoch、迁移次数、stale route 数、stale term 数、hot key top-k、最大 partition 负载、二级 reduce 成本、replica lag 最大值、quorum commit 数、quorum reject 数、manifest generation、snapshot index、orphan block 清理数。没有这些指标，分片复制问题只能靠猜。

Listing 20.31 分片复制运行证据示例

```
partitions.total=64
route.epoch=42
migration.completed=1
route.stale_rejected=17
term.stale_rejected=2
hot_key.max_share=0.61
quorum.commit=128
quorum.reject=3
replica.max_lag=2
manifest.generation=42
snapshot.last_included_index=350
```

最小综合路径只跟踪一个 key 的一生：让 `event=login` 从普通 key 变成热点，触发拆分和迁移；在迁移切换后投递一条旧 epoch 写；让一个副本落后；再让旧 Driver 尝试提交旧 term manifest。运行证据要能按 trace id 列出 route epoch、owner、replica version、commit index 和最终 manifest。这条路径覆盖热点、迁移、旧请求、副本落后和 term 变化，足够支撑本章的控制面语义。

源码阅读任务

源码阅读任选一条窄路径：

1. 阅读 Kafka partition leader、replica lag、producer sequence 或 consumer offset 的一条路径，说明 offset 和 leader 如何定义顺序。
2. 阅读 etcd/Raft 的 log entry、commit index、snapshot 或 watch revision 路径，说明控制面状态如何被排序和恢复。
3. 阅读 Cassandra/Dynamo 类系统的 token range、quorum、read repair 或 tombstone 设计，说明 quorum 之外还需要哪些修复机制。
4. 阅读 HDFS 或类似系统的 block location、replication、checkpoint 或 journal 路径，说明控制面和数据面如何分离。

报告必须按“约束、核心机制、购买能力、成本、迁移到本项目的方式”组织，不能只罗列名词。

20.13 完整案例：旧 leader 为什么不能再写 manifest

分片复制中最危险的错误之一，是旧 leader 还活着，并且还能访问旧状态。网络分区或调度延迟之后，新 leader 已经在更高 term 中接管 partition，旧 leader 的线程、文件句柄和临时 block 可能仍然存在。如果提交接口只检查 block checksum，不检查 leader term、route epoch 和 owner，旧 leader 就可能把过期输出写进 manifest，造成控制面分叉。

初始事实：权利来自 term 和 epoch 假设 partition 12 在 route epoch 41 中由 Worker A 管理，leader term 为 7。控制面随后选出新的 leader term 8，并把 partition 12 迁移到 Worker B，route epoch 42。A 没有立刻退出，它仍在处理 epoch 41 的旧请求，并写出了 block BA。B 在 term 8 中写出了 block BB。两者的数据面 block 都可能 checksum 正确，但只有 term 8、epoch 42、owner B 有权提交。

Listing 20.32 旧 leader 事故的事实表

```
old state:
  partition=12
  owner=A
  route_epoch=41
  leader_term=7

new state:
  partition=12
  owner=B
  route_epoch=42
  leader_term=8

late old block:
  writer=A
  route_epoch=41
  leader_term=7
  block=BA
```

这张表说明提交检查必须读取当前控制面，而不是相信 block 自带的声明。Block metadata 可以作为候选身份，不能作为权威。

提交判定先看权利，再看数据 正确提交顺序是：检查 partition id 是否存在；检查 request 的 route epoch 是否等于当前 epoch；检查 leader term 是否等于当前 term；检查 owner worker 是否等于当前 owner；再检查 block checksum、record count 和 manifest generation。若权利检查失败，即使 block 数据完整，也不能进入 manifest。

Listing 20.33 旧 leader 提交判定

```
request from A:
  partition=12
  route_epoch=41
  leader_term=7
  owner_worker=A
  block=BA

current ownership:
  partition=12
  route_epoch=42
  leader_term=8
  owner_worker=B

decision:
  StaleRoute or StaleTerm
  do not append manifest
  record stale rejection
```

注意“先看权利”。如果系统先花很久校验旧 block、压缩旧 block 或上传旧 block，再发现 term 过期，就浪费了资源；更糟的是某些副作用可能已经发生。控制面检查要尽早，外部副作用要尽量放在权利确认之后，或设计成可回收候选。

Fencing 的工程含义 Fencing 不是把旧进程杀死这么简单。旧进程可能因为网络延迟没收到新 term，可能正在执行系统调用，可能之后才恢复连接。Fencing 的核心是：任何会改变外部事实的接口都必须携带并验证 fencing token，例如 leader term、route epoch、lease version 或 ownership generation。旧持有者即使还活着，也没有提交权。

Listing 20.34 fencing token 的接口边界

```
CommitPartitionOutput(request):
  require request.partition_id
  require request.route_epoch
  require request.leader_term
  require request.owner_worker
  require request.attempt_id
  validate against current OwnershipRecord
  only then append manifest candidate
```

如果某条后台清理、二级 reduce 或诊断导出路径能绕过这个接口，它就可能成为旧 leader 写入的旁路。控制面安全要求所有提交都走同一个权利检查。

恢复时如何处理旧 block 恢复程序启动后扫描临时目录，发现 BA 和 BB。它不能因为 BA 时间更早或文件名更小就接受 BA。恢复仍然先读控制日志和 manifest，重建当前 OwnershipRecord 和 accepted manifest generation；然后把 BA 分类为 stale candidate，把 BB 分类为 accepted 或 candidate。旧 block 可以保留

到审计期结束，再在 tombstone 安全点后清理。

Listing 20.35 恢复分类

```
load control log:
  current term=8
  route epoch for partition 12=42
  owner=B

scan temp blocks:
  BA has term=7 epoch=41 owner=A -> stale candidate
  BB has term=8 epoch=42 owner=B -> accepted or pending commit

cleanup:
  stale candidates are deleted only after audit safe point
```

这条路径再次说明目录扫描不是权威。控制日志和 manifest 决定事实，临时文件只是候选材料。

测试要保留旧 owner 活着 很多测试只模拟旧 owner 崩溃，因此看不到 fencing 问题。更强的测试是让旧 owner 继续运行：它先获得旧 epoch 请求，网络分区后新 owner 接管，旧 owner 晚到提交。测试断言旧 owner 收到 StaleTerm 或 StaleRoute，manifest generation 不变，stale counter 增加，旧 block 被标记为 orphan 或 stale。这样才能证明系统不是靠“旧进程碰巧死了”保持正确。

读路径也需要 fencing 结果 写路径拒绝旧 owner 只是第一步。读路径也要说明自己读到哪个 epoch。监控读可以接受 stale，但报告要带 `observed_route_epoch`；调度读通常需要 monotonic，不能从 epoch 42 倒退到 epoch 41；提交前检查必须读取当前强控制面，不能使用缓存过期的 owner。若读接口不带版本，调用方无法分辨“读到旧事实”和“读到当前事实”。

Listing 20.36 读响应携带版本

```
PartitionReadResponse:
  partition_id
  owner_worker
  route_epoch
  leader_term
  read_mode
  replica_lag
```

版本字段让旧读不再伪装成当前读。系统可以允许某些路径读旧信息，但不能让旧信息进入提交权判断。

控制面变更要写入日志而不是只改内存 Owner、route epoch、leader term、migration state、tombstone 都是控制面事实。它们不能只存在于某个 Driver 的内存 map 中，否则 Driver 崩溃后无法证明旧 owner 是否还有权提交。最小实现也应把控制面变更写成顺序日志，恢复时按日志重放得到 OwnershipRecord，再读取 manifest。数据面 block 可以重算或清理，控制面日志决定谁有权承认结果，也决定哪些旧文件只能作为审计材料和恢复线索，而不能回头影响已经提交的事实。

复查必须覆盖五项事实：partition、owner、route epoch、leader term、state 是否都进入提交路径，旧 owner 是否不能提交；迁移状态机是否可恢复，旧路由、切换崩溃、重复请求和清理是否都有测试；提交点、replica lag、quorum、stale/monotonic/leader read 行为是否清楚；倾斜输入、普通 hash、combine、热 key 拆分和二级合并是否形成对照；Kafka、Dynamo/Cassandra、etcd/Raft、HDFS、

Spanner 等设计中采用、简化和拒绝的部分是否写成具体判断。

20.14 分片复制门禁

一、是否先问状态归属 遇到 key、partition、manifest、checkpoint 或 replica，先问当前 owner 是谁，epoch 是多少，谁有权提交。若只看到 hash 或文件路径，还没有进入本章核心。

二、旧事件是否有明确处理 旧 route 请求、旧 leader commit、旧副本响应、旧 snapshot 都要进入状态机。不能静默接受，也不能静默丢弃。

三、再分片是否可恢复 Prepare、Snapshot、CatchUp、Validate、Switch、Drain 每个阶段崩溃后，都要知道恢复动作。若恢复只能“重新跑脚本”，就不可靠。

四、复制是否定义提交点 客户端看到写成功时，哪些副本已经内存保存、WAL 持久化、多数确认、状态机应用，要写清楚。没有提交点，就无法判断故障后哪个版本可信。

五、读接口是否声明一致性 控制面读、调度读、监控读不应混用一个含糊 API。读结果要带版本或滞后，调用方要选择 stale、monotonic 或 strong。

六、共识边界是否足够小 Route epoch、assignment、manifest generation、commit log 属于强控制面；shuffle block、临时 partial、可重算数据属于数据面。把边界画错，系统要么慢，要么不可信。

七、是否为第 18 章留下接口 第 18 章会把 map、shuffle、reduce、runtime、I/O 和分布式控制面合成完整计算引擎。本章留下的 OwnershipRecord、PartitionRequest、ControlLogEntry 和 PartitionEvidence 应能被直接复用。

20.15 习题

习题与作业

习题一：为一个日志统计系统选择分片策略。分别讨论点查、范围查询、递增时间戳写入和热点 key 下 hash、range、虚拟分片的表现。

习题二：构造旧 owner 迟到写入时间线。要求写出 route epoch、owner、请求到达顺序、服务端决策和 manifest 变化。

习题三：设计再分片状态机。至少包含 Prepare、Snapshot、CatchUp、Validate、Switch、Drain。说明每个状态崩溃后的恢复动作。

习题四：比较冻结、日志追赶、双写三种迁移策略。必须说明部分成功、旧请求、读路径和恢复成本。

习题五：设计热点 key 拆分方案。说明如何检测热点、如何生成子 key、如何二级合并、如何保持 manifest 幂等。

习题六：比较三种写提交点：leader 内存返回、本地 WAL 返回、多数副本 WAL 返回。分别说明延迟、数据丢失风险和读路径影响。

习题七：用三副本 A、B、C 手算 quorum 恢复。版本 10 分别只在 A、在 A/B、在 B/C 三种情况下，判断 A 崩溃后能否承诺版本 10。

习题八：设计读一致性接口。至少支持 stale read、monotonic read 和 leader read，并说明响应中应返回哪些版本字段。

习题九：解释 leader term、log index、commit index、apply index、snapshot index 的区别。每个概念都要对应一个故障或恢复问题。

习题十：阅读一个工业系统的分片、复制或控制面日志设计，按“约束、核心机制、购买能力、成本、迁移到本项目的方式”写报告。

工程案例

知识点小案例：旧 owner 的迟到写入为什么需要 route epoch。特例是 shard S 从节点 A 迁移到节点 B。客户端还拿着旧路由，把写请求发给

A；同时新请求已经到达 B。如果请求和存储记录没有 route epoch，A 的迟到写入可能覆盖新 owner 的事实。加入 epoch 后，A 只能接受自己仍是 owner 的 epoch；旧 epoch 请求必须拒绝或转发。由此推出规律：分片迁移不是移动数据那么简单，而是控制“谁有资格写入某个逻辑事实”。工程上路由表、请求、manifest 和恢复日志都要带 epoch，并写清旧请求、迟到响应和切换点。

分布式计算工程实践

收束项目不再新增一个孤立主题，而是把硬件、布局、并发、运行时、I/O 和分布式机制组合成一个可运行、可测量、可故障注入、可恢复的贯穿系统：Compute Systems Engine。它处理一批日志记录，按 key 聚合计数，经过 map、combine、shuffle、reduce 和 manifest commit，最后输出确定结果。目标不是替代 Spark、Flink 或数据库执行器，而是在可控规模内把现代计算系统的关键边界写清：数据怎样进入热路径，任务怎样并行，I/O 怎样可靠完成，远端消息怎样提交，失败后谁是权威事实。

先看最终项目要能复现的一次完整事故。输入有三个 split，其中一个 split 含非法字段，一个 key 是热点；worker 1 写出 task 0 后，Driver 在 manifest 提交后崩溃；worker 2 的 task 1 第一次写出遇到短写；task 2 的成功响应被 MessageBus 丢弃，Driver 超时重试，旧响应随后迟到。正常路径里这只是一个日志统计作业，事故路径里它同时考验解析错误、I/O 可靠写、任务状态、重试、幂等提交、热点 key、trace 和恢复。

判断标准很简单：程序不能只跑通，还要能解释程序为什么可信。解释必须落到具体对象：InputSplit、TaskId、AttemptId、ShuffleBlock、ManifestEntry、MessageEnvelope、Checkpoint 和 RunReport。如果一个结果出错，报告应能把它缩小到解析、分区、写出、提交或恢复中的某个边界，而不是让人翻日志猜。

21.1 Reference 先行

任何分布式计算项目都要先有可验证 reference。Reference 可以慢，但必须定义语义：输入格式是什么，坏行如何处理，key 如何规范化，计数类型是否溢出，输出顺序是否稳定，错误摘要如何记录。没有 reference，后面的线程池、shuffle、重试、checkpoint 和热点优化都没有正确性锚点。

日志记录和输出语义 Compute Systems Engine 可以从固定字段日志开始：timestamp、user id、event type 和 value。Reference 单线程读取全部输入，跳过或记录坏行，按 event type 累加 value，最后按 event type 排序输出。优化版本可以改变布局、并行方式、shuffle 方式和写出方式，但不能改变这份语义。

Listing 21.1 Reference 使用的稳定数据合同

```
1 struct LogRecord {
2     std::uint64_t timestamp = 0;
3     std::uint64_t user_id = 0;
4     std::uint32_t event_type = 0;
5     std::int64_t value = 0;
6 };
7
8 struct EventCount {
9     std::uint32_t event_type = 0;
10    std::int64_t total = 0;
11 };
```

这段结构体是内存语义，不等于文件格式。文件格式可以是文本、二进制或列式块，但解析后必须进入稳定数据合同。若未来加入 schema 版本、字符串 key 或变长字段，也要先扩展 reference，再扩展优化路径。

输入身份 输入 split 不能只是“第几个任务”。它要能定位字节范围和恢复边界。文本文件按字节切分时，非零 begin 要跳到下一行起点；end 可以读到当前行结束；错误记录要能回到 file id、offset 和 record id。若每次运行都无法证明处理的是同一批字节，benchmark 和恢复报告都不可信。

Listing 21.2 输入分片元数据

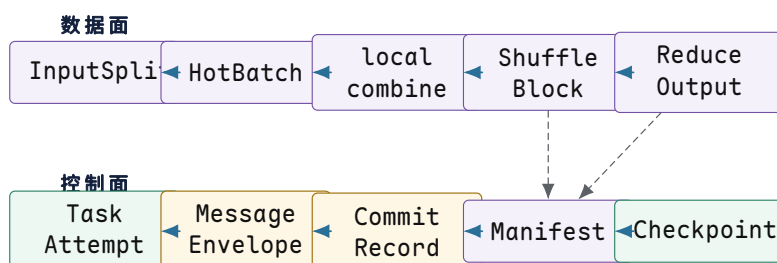
```
1 struct InputSplit {
2     std::uint64_t split_id = 0;
3     std::uint64_t file_id = 0;
4     std::uint64_t begin_offset = 0;
5     std::uint64_t end_offset = 0;
6     std::uint64_t input_checksum = 0;
7 };
```

split_id 是逻辑 task 的基础。Attempt 重试时 split_id 不变，物理执行变的是 attempt。这个区分贯穿全章：逻辑身份决定语义，物理 attempt 只是实现路径。

21.2 端到端架构：控制面和数据面分开

Compute Systems Engine 的架构可以分成两条路径。数据面搬运和处理大块数据：读取 split、解析 batch、本地 combine、写 shuffle block、reduce 合并。控制面维护小而关键的事实：task 状态、attempt、manifest、checkpoint、route epoch、retry budget 和 recovery audit。控制面决定哪些数据有效，数据面负责高吞吐处理。

最终引擎：数据面搬运大块，控制面定义有效性



数据面可以重做、重发、重读；控制面决定哪个 attempt 的哪个 block 对下游可见。最终系统的恢复能力来自这条边界。

图示：本图要证明的命题是：最终计算引擎不是一个大函数，而是数据面和控制面共同维护的状态机。

模块边界 Parser 接收 InputSplit，输出 batch 和错误摘要；它不提交结果。Kernel 接收热字段 span，输出本地 partial；它不理解重试。Partitioner 接收 partial，输出按 partition 分组的 block；它不写 manifest。I/O 层负责临时文件、buffer 生命周期、短写和 checksum；它不决定业务生效。Driver 维护 task table、retry、commit 和 checkpoint；它不解析每行日志。MessageBus 传递消息和故障事件；它不决定业务成功。

边界清楚不是为了目录好看，而是为了事故时能定位责任。若 Parser 直接写 manifest，解析错误会污染提交语义；若 I/O completion handler 直接提交业务结果，短写和迟到 completion 会绕过状态表；若 Runtime 理解业务 key，任务运行时就无法复用到后续推理引擎。

目录建议 项目可以按责任组织：`engine/model` 放稳定 ID 和消息结构；`engine/reference` 放串行 reference；`engine/input` 放 split 和解析；`engine/kernel` 放 combine、filter、partition；`engine/runtime` 放队列、线程池和任务图；`engine/io` 放可靠写和 block 校验；`engine/driver` 放 task 状态、manifest 和 checkpoint；`engine/fault` 放故障注入；`engine/report` 放 RunReport。

21.3 Map、Combine、Shuffle 和 Reduce

朴素 shuffle 会把每条记录都发给 reduce。若一亿条日志只有一万个 event type，这样会浪费大量网络、序列化和写盘成本。Map-side combine 先在每个 worker 本地按 key 聚合，再把 partial 发给 reduce。发送量从记录数下降到本地出现过的 key 数。这是第 13 章 reduce 原则在网络尺度上的应用。

本地聚合策略 如果 event type 是小范围整数，可以用连续数组，内存访问规则、容易向量化；如果 key 稀疏或是字符串，可以用哈希表，但要承担随机访问、分配和 rehash 成本；如果输入已经按 key 排序，可以顺序合并。项目报告不应只写“用了哈希表”，而要记录 key 基数、最大 bucket、combine ratio、内存峰值和输出 block 数量。

Listing 21.3 小整数 key 的本地 combine

```
1 std::vector<std::int64_t> combine_events(
2     std::span<const LogRecord> records,
3     std::uint32_t event_type_count) {
4     std::vector<std::int64_t> partial(event_type_count, 0);
5     for (const LogRecord& record : records) {
6         if (record.event_type < event_type_count) {
7             partial[record.event_type] += record.value;
8         }
9     }
10    return partial;
11 }
```

真实项目不能静默忽略越界 event type。示例片段只突出 combine 结构；生产版应把非法 key 进入错误摘要，并让 reference 和优化路径共享同一错误合同。

ShuffleBlock 和 Manifest Shuffle 可以先用单进程内存队列模拟，也可以写本地文件再由 reduce 拉取。无论哪种方式，都要有 manifest 描述哪些 block 有效。Reduce 不应扫描临时目录，也不应接受 MessageBus 里随机到达的所有 partial。它只读取 manifest 接受的 block。

Listing 21.4 Shuffle block 和 Manifest entry

```
1 struct ShuffleBlock {
2     std::uint64_t block_id = 0;
3     std::uint64_t map_task_id = 0;
4     std::uint32_t attempt = 0;
5     std::uint32_t reduce_partition = 0;
6     std::uint64_t byte_size = 0;
7     std::uint64_t checksum = 0;
8 };
9
10 struct ManifestEntry {
11     std::uint64_t generation = 0;
12     std::uint64_t logical_task_id = 0;
```

```

13  std::uint32_t accepted_attempt = 0;
14  ShuffleBlock block;
15 };

```

Manifest entry 是提交事实。临时文件存在只是存储事实；worker 返回 Ok 只是执行事实；只有 manifest 接受某个 logical task 的某个 attempt，下游才允许读取对应 block。这个规则是最终项目可靠性的中心。

Reduce 端去重 Reduce 端最危险的错误是重复合并同一个 logical map task。Attempt 0 写出后响应丢失，Attempt 1 重试并提交；如果 Reduce 扫描目录，它会把两个 block 都读进去，计数翻倍。正确策略是 reduce 根据 manifest 拉取已接受 attempt，或按 logical task 去重。按 attempt 去重不够，因为不同 attempt 仍可能属于同一个 logical task。

热点 key 如果某个 key 占输入 90%，普通 hash partition 会让一个 reduce partition 长尾。第一层优化是 map-side combine，减少重复传输；第二层是热 key 拆成多个执行子 key，分给多个 reduce；第三层再做二级 reduce，把子 key 合回业务 key。子 key 是执行身份，不是业务身份。Manifest 和 RunReport 要记录 hot route version、subkey count、final merge checksum 和二级合并成本。

21.4 网络、序列化和 shuffle 现实成本

分布式计算不能只把单机函数调用替换成远程调用。一次 shuffle block 传输会穿过序列化、buffer 分配、压缩、系统调用、内核 socket buffer、TCP 拥塞控制、网卡队列、对端内核、反序列化和 reduce 输入队列。每一层都可能排队，每一层也可能改变背压位置。若报告只写“发送给 reduce”，就把最重的系统路径藏掉了。

RPC 不是函数调用 本地函数调用的失败边界很窄：要么返回，要么抛异常，要么进程崩溃。RPC 的边界更复杂：请求可能没有发出，可能发出但服务端没收到，可能服务端执行成功但响应丢失，可能响应迟到，可能客户端重试后两次都执行。于是 RPC 参数必须带逻辑身份、attempt、epoch、deadline 和幂等键。

Listing 21.5 Shuffle RPC 的最小身份字段

```

ShufflePushRequest:
  stage_id
  logical_map_task
  attempt
  reduce_partition
  block_id
  block_checksum
  route_epoch
  deadline
  idempotency_key

```

这些字段不是“分布式样板”。它们分别回答不同故障：**attempt** 区分物理执行；**route_epoch** 防止旧路由写到新 owner；**idempotency_key** 让服务端识别重复请求；**deadline** 防止无限等待；checksum 防止传输或存储错误越过提交边界。

序列化是数据布局的第二次选择 内存里的 BatchView 适合 CPU 热循环，不一定适合网络。网络 block 需要稳定字节序、版本、字段长度、checksum 和可跳过的扩展字段。若直接把内存结构体按字节发送，会把对齐空洞、指针、平台字节序和进程内地址泄漏到协议里。正确做法是定义 wire format：字段顺序、整数宽度、编码方式、压缩策略和校验范围。

Listing 21.6 ShuffleBlock wire format 的账本

```

header:
  magic
  format_version
  partition_plan_version
  record_count
  payload_bytes
  payload_checksum

payload:
  encoded event_id/value pairs
  optional compressed columns
  no raw process pointers

```

序列化格式也会反馈到 CPU 路径。行式格式便于逐条生成，列式格式便于压缩和向量化 reduce；变长编码节省网络字节，却增加解析分支；压缩减少传输，但增加 CPU 和内存峰值。格式选择不是 I/O 细节，而是端到端成本模型。

零拷贝的边界 零拷贝经常被过早追求。真正的问题是哪些 copy 在热点路径上，copy 的字节量相对网络和磁盘是否主导，buffer 生命周期是否允许内核或网卡继续引用。若为了减少一次 copy 引入 pinned buffer、复杂引用计数、跨线程生命周期和迟到 completion，可能把正确性成本推高。

策略	购买的能力	需要支付的成本
普通序列化到 buffer	简单、容易校验和重试	多一次内存 copy
writetv/scatter-gather	减少拼接 copy	buffer 生命周期更复杂
sendfile/零拷贝	降低用户态 copy	文件、socket、设备和错误处理受限
registered buffer	降低提交成本	对齐、pinning、回收和内存压力

第一版系统应先把普通 buffer 的报告做完整。只有当 `serialize_copy_ns` 或 `memory_bandwidth` 成为主瓶颈，再引入更复杂路径。否则“零拷贝”容易把一个可测的 copy 成本换成不可测的生命周期 bug。

shuffle 背压从 reduce 端开始 Reduce 端慢时，map 端继续推 block 会制造网络和磁盘债务。背压应沿着 reduce queue、network in-flight、map output buffer、task admission 逐层向上游传播。只在 map 端看本地队列是不够的，reduce partition 的队列深度和处理延迟才是下游能力。

Listing 21.7 shuffle 背压字段

```

shuffle.map_output_bytes
shuffle.inflight_rpc_bytes
shuffle.reduce_queue_depth
shuffle.reduce_partition_lag
shuffle.retry_count
shuffle.late_reply_count
shuffle.deserialize_ns
shuffle.network_send_ns
shuffle.network_receive_ns
shuffle.spill_bytes

```

这些字段把“网络慢”拆开。若 `deserialize_ns` 高，问题在格式或 CPU；若 `reduce_queue_depth` 高，问题在下游计算；若 `retry_count` 高，问题在网络或 deadline；若 `spill_bytes` 高，问题在内存预算和 partition 形状。分布式性能必须能定位到层级。

观测性：metrics、logs、traces 三件事不要混 分布式 shuffle 的调试不能只靠日志。Metrics 给趋势，logs 给离散事实，traces 给一次请求跨进程的因果链。三者缺一项，复盘都会失真：只有 metrics 看不到哪次 attempt 出错；只有 logs 很难看 p95；只有 trace 没有聚合指标，无法判断是否全局退化。

观测对象	适合回答	不适合回答
Metrics	队列深度、吞吐、p95、错误率趋势	单个 attempt 的完整因果链
Logs	某次状态转移、错误码、恢复动作	大规模分位数和跨服务耗时分解
Traces	request/span 的父子关系和时间线	长窗口容量规划和低基数告警

一条 shuffle trace 至少要能把 map 端序列化、发送、reduce 端接收、反序列化、入队和提交连起来。字段可以参考 OpenTelemetry 风格的 span 模型，但重点不是依赖某个库，而是固定跨进程身份和时间边界。

Listing 21.8 ShuffleSpan 最小字段

```
trace_id
span_id
parent_span_id
stage_id
logical_task
attempt
reduce_partition
route_epoch
block_id
bytes
record_count
queue_wait_ns
serialize_ns
send_ns
network_wait_ns
recv_ns
deserialize_ns
reduce_queue_depth
retry_reason
status
```

这份 trace 能回答“慢在哪里”。若 `serialize_ns` 高，优先查 wire format、压缩和内存分配；若 `network_wait_ns` 高，查 RTT、拥塞、重试和 route；若 `reduce_queue_depth` 高，查下游算子或热点 key；若 `retry_reason` 集中在 deadline，查 SLO 和 backpressure。没有这些字段，工程师只能把所有问题归咎为“网络不稳定”。

RDMA 不是跳过状态机 RDMA 和高速互联会出现在 AI Infra、参数同步、shuffle 和多机推理中，但它不是分布式正确性的捷径。RDMA verbs 的基本对象包括 protection domain、memory region、queue pair、send/receive queue、completion queue 和 completion entry。它能让网卡直接 DMA 到注册内存，减少内核参与和 copy，但它也把 buffer 生命周期、注册成本、对齐、权限和 completion 处理变得更硬。

RDMA 对象	购买的能力	新增约束
Memory Region	网卡可访问用户内存	pin/register 成本、权限、生命周期
Queue Pair Completion Queue	连接和发送/接收队列 获取完成事件	状态转移、重连、队列深度 CQ overflow、轮询成本、错误处理
RDMA Write/Read Send/Recv	绕过远端 CPU 的数据搬运 消息式语义	远端内存布局和可见性协议 receive buffer 预投递和背压

最容易犯的错误，是以为 RDMA write 成功就等于业务提交。它只说明某个工作请求产生了 completion，或者本端观察到对应完成；远端应用是否已经把这段内存解释成一个 block、是否校验 checksum、是否接受 attempt、是否写入 manifest，仍然需要协议。RDMA 减少了数据路径成本，没有消除 logical task、attempt、route epoch、checksum、backpressure 和 commit point。

Listing 21.9 RDMA shuffle 仍然需要的业务状态

```
RdmaBlockTransfer:
  logical_map_task
  attempt
  route_epoch
  remote_buffer_id
  remote_offset
  bytes
  checksum
  qp_id
  work_request_id
  completion_status
  manifest_commit_status
```

第一版分布式计算项目不应该先上 RDMA。更稳的路线是先用普通 TCP/RPC 把身份、重试、背压、manifest、观测性和恢复协议做对；当报告证明 copy、syscall 或 TCP stack 成为主瓶颈，再引入 registered buffer、zero-copy 或 RDMA。否则你会把一个还没写清的状态机搬到更难调试的网卡 completion 路径里。

Exactly-once 是提交协议，不是传输属性 网络可能重复发送，RPC 可能重试，worker 可能迟到，driver 可能崩溃。所谓 exactly-once 不能靠“只发送一次”实现，而要靠幂等身份、去重表、manifest 提交点和恢复扫描。每个逻辑输出只能有一个 accepted attempt；重复到达的同一 attempt 可以确认已处理；不同 attempt 只能由 manifest 决定哪一个生效。

Listing 21.10 服务端处理重复 shuffle push 的判定顺序

```
on ShufflePushRequest(req):
  if req.route_epoch is stale:
    reject StaleRoute
  if manifest already accepted another attempt for req.logical_map_task:
    reject SupersededAttempt
  if idempotency_key was already stored:
    return previous result
  validate checksum and write temp block
  record idempotency_key before replying success
```

这段伪代码说明：传输层保证不了业务 exactly-once；业务层必须让重复和迟到成为正常输入。这个原则和第 15 章 completion、第 17 章 fencing、manifest commit 是同一条线。

21.5 端到端执行账本：三 split、两 worker、一个 Driver

把抽象对象放到一次具体运行里。输入被切成三个 split: S0、S1、S2。Driver 维护 task table；两个 worker 执行 map/combine/write；MessageBus 模拟远端消息；manifest 是唯一提交入口。故障规则固定为三条：S1 的第一次写出发生短写；S2 的第一次成功 reply 被丢弃；Driver 在接受 S0 manifest 后崩溃并重启。

初始状态 运行开始时，只有逻辑任务，没有 attempt，也没有 block。Task table 是控制面事实，输入文件是数据面事实。

Listing 21.11 初始 task table

task	split	state	current_attempt	accepted_attempt
T0	S0	Pending	-	-
T1	S1	Pending	-	-
T2	S2	Pending	-	-

manifest.generation = 0
checkpoint.generation = 0

调度器把 T0 分给 worker 0，把 T1 分给 worker 1。创建 attempt 时，逻辑任务不变，物理执行身份增加：

Listing 21.12 创建 attempt 后的 task table

task	split	state	current_attempt	accepted_attempt
T0	S0	Running	A0	-
T1	S1	Running	A0	-
T2	S2	Pending	-	-

T0 正常提交后 Driver 崩溃 Worker 0 解析 S0，本地 combine，写出 shuffle block B0a0.tmp，checksum 通过，发送 TaskReply(T0,A0,B0a0)。Driver 在当前 epoch 下检查：任务仍然 Running，attempt 等于 current attempt，block checksum 正确，于是把 manifest 从 generation 0 推进到 1。

Listing 21.13 T0 提交后的 manifest

generation	logical_task	accepted_attempt	block	checksum
1	T0	A0	B0a0	ok

随后 Driver 崩溃。崩溃不会撤销已经写入 manifest 的事实。重启后，恢复程序先读 manifest，再读 checkpoint 或 task table 快照。若它只读旧 checkpoint，可能以为 T0 未完成；但 manifest generation 1 已经说明 T0 对外生效。恢复顺序必须以 manifest 为权威。

T1 短写：存储事实尚未完成 Worker 1 执行 T1/A0 时，I/O 层第一次 write 只写了部分字节。此时不能发送成功 reply，也不能提交 manifest。WriteRequest 保存 bytes_done 和 remaining，继续写剩余部分；若后续失败，block 只能成为 failed candidate。

Listing 21.14 T1 短写时的 block table

block	logical_task	attempt	bytes_done	bytes_total	state
B1a0	T1	A0	4096	16384	PartialWritten

短写不是异常边角，而是正常 I/O 语义。只有当 `bytes_done==bytes_total`，checksum 通过，并且 Driver 接受该 attempt，B1a0 才能进入 manifest。否则 reduce 不得扫描这个文件。

T2 reply 丢失：执行成功不等于提交成功 Driver 重启后调度 T2/A0。Worker 完成并写出 B2a0，但 MessageBus 丢弃成功 reply。Worker 的执行事实和存储事实都可能为真：attempt 做过，block 存在，checksum 正确。Driver 没收到 reply，manifest 没有接受它，所以提交事实不存在。超时后 Driver 创建 T2/A1。

Listing 21.15 T2 超时重试时的三类事实

```

execution facts:
  T2/A0 may have finished on worker 0
  T2/A1 is now running on worker 1

storage facts:
  B2a0 may exist and checksum may be ok
  B2a1 may be produced later

commit facts:
  manifest has no entry for T2 yet

```

这就是 attempt 存在的原因。若 reduce 扫描目录，B2a0 和 B2a1 可能都被读入；若 manifest 按 logical task 接受唯一 attempt，旧 block 只会成为 orphan 或 late candidate。

旧 reply 迟到：epoch 和 attempt 检查 假设 T2/A1 先完成并提交，manifest 接受 B2a1。随后丢失的 T2/A0 reply 迟到。Driver 必须按 logical task 检查：T2 已经有 accepted attempt A1，所以 A0 进入 LateAttempt 或 DuplicateAttempt，不得改写 manifest。

Listing 21.16 T2 最终 manifest 和迟到响应

```

manifest:
  generation  logical_task  accepted_attempt  block
  1            T0           A0             B0a0
  2            T1           A0             B1a0
  3            T2           A1             B2a1

late reply:
  TaskReply(T2, A0, B2a0) -> rejected as LateAttempt

```

若系统支持 route epoch 或 driver epoch，还要检查 reply 属于当前 epoch。旧 epoch 的成功响应即使 attempt 编号看起来合理，也不能拥有提交权。这个检查防止 Driver 重启、任务迁移或 ownership 切换后，旧执行者继续宣布结果有效。

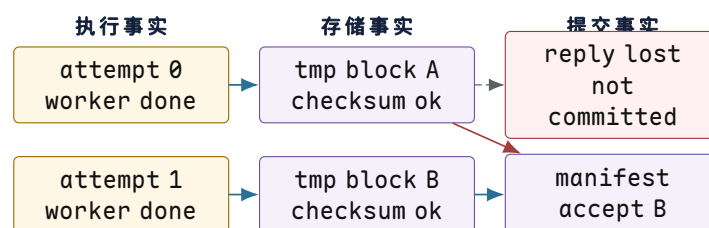
最终 reduce：只读提交事实 Reduce 阶段读取 manifest 中 T0/A0、T1/A0、T2/A1 对应 block。它不读取 B2a0，即使该文件真实存在且 checksum 正确。最终 checksum 和 reference 比较；如果一致，RunReport 仍要记录 late attempt、short write、driver recovery 和 orphan cleanup。正确运行不是没有事故，而

是事故被状态机吸收后没有污染结果。

21.6 Attempt、提交事实和恢复权威

最终项目必须区分三种事实。执行事实：某个 worker 是否做过某次 attempt。存储事实：某个 block 是否存在且 checksum 正确。提交事实：下游是否允许把这份结果计入最终输出。系统可以有多个执行事实和多个存储事实，但每个 logical task 只能有一个提交事实。

执行事实、存储事实和提交事实不能混淆



Attempt 0 可以真实执行过，block A 也可以真实存在，但它没有进入 manifest。恢复和 reduce 只能相信提交事实。

图示：本图要证明的命题是：最终结果由 manifest 决定，不由目录扫描或 worker 成功响应决定。

状态表 最小状态表可以分四类。Task table 保存逻辑任务：task id、stage id、current attempt、deadline、retry budget、state。Attempt table 保存物理执行：attempt、worker id、start、finish、reply state、candidate block。Block table 保存存储事实：block id、path、bytes、checksum、write state、orphan reason。Manifest table 保存提交事实：logical task、accepted attempt、accepted block、generation。

Listing 21.17 Task 和 attempt 的示例形态

```

1 enum class TaskState : std::int32_t {
2     Pending,
3     Running,
4     Retrying,
5     Committed,
6     Failed,
7 };
8
9 struct TaskRecord {
10     std::uint64_t task_id = 0;
11     std::uint32_t current_attempt = 0;
12     std::uint64_t deadline_ns = 0;
13     std::uint32_t retry_count = 0;
14     TaskState state = TaskState::Pending;
15 };
16
17 struct AttemptRecord {
18     std::uint64_t task_id = 0;
19     std::uint32_t attempt = 0;
20     std::uint32_t worker_id = 0;
  
```

```

21  std::uint64_t candidate_block_id = 0;
22  bool reply_received = false;
23  };

```

恢复顺序 恢复不是把旧进程内存复活。正确顺序是先读 manifest，确定哪些 logical task 已经外部生效；再读 checkpoint，恢复 job、stage、retry budget 和未完成任务；再扫描临时目录，把未被 manifest 引用的 block 标成 orphan candidate；最后重新调度 pending 或 uncertain task。若恢复程序先扫描目录并承认文件，就会把未提交 attempt 当成结果；若只读 checkpoint 而忽略 manifest，就会丢掉 commit 后崩溃的事实。

恢复算法伪代码 恢复程序要明确读什么、信什么、丢什么。下面的伪代码表达的是顺序，不是文件格式。

Listing 21.18 从 manifest 和 checkpoint 恢复控制面的伪代码

```

load manifest entries
mark all accepted logical tasks as Committed

load latest checkpoint if present
restore job configuration, stage graph, retry budgets, route epochs

scan temporary block directory
for each block:
    if block is referenced by manifest:
        verify checksum and keep as committed data
    else:
        mark as orphan candidate

for each logical task:
    if task is committed by manifest:
        do not reschedule
    else if retry budget remains:
        schedule new attempt under current epoch
    else:
        mark task failed

write RecoveryAudit into RunReport

```

注意第一行和第三行的顺序。Manifest 先于目录扫描，是为了避免“文件存在即有效”的错误；checkpoint 不能覆盖 manifest，是为了避免“提交后崩溃又重做”的错误。RecoveryAudit 应记录 committed tasks、rescheduled tasks、orphan blocks、stale replies 和 checksum failures。

21.7 Driver 状态机：消息怎样变成提交事实

最终项目的核心不是 worker 写文件，而是 Driver 如何把不可靠消息、重试、迟到响应和存储候选转成唯一提交事实。Worker 可以执行多次，MessageBus 可以丢包或重复投递，临时目录可以留下孤儿文件；Driver 只能让每个 logical task 有一个 accepted attempt。这个约束应写成状态机和伪代码，而不是散在回调里。

消息必须带完整身份 一个成功响应至少要带 job id、stage id、logical task id、attempt、driver epoch、worker id、request id、block id、bytes、checksum 和状态。缺少任何一个关键字段，Driver 都可能把旧消息误认为新消息，或者把不

同逻辑任务的 block 混在一起。消息不是函数返回值，它是跨时间、跨线程、跨进程边界的事实载体。

Listing 21.19 TaskReply 的示例形态

```

1 enum class TaskReplyStatus : std::int32_t {
2     Succeeded,
3     FailedRetryable,
4     FailedFatal,
5     Busy,
6     Canceled,
7 };
8
9 struct TaskReply {
10     std::uint64_t job_id = 0;
11     std::uint64_t stage_id = 0;
12     std::uint64_t task_id = 0;
13     std::uint32_t attempt = 0;
14     std::uint64_t driver_epoch = 0;
15     std::uint32_t worker_id = 0;
16     std::uint64_t request_id = 0;
17     std::uint64_t block_id = 0;
18     std::uint64_t bytes = 0;
19     std::uint64_t checksum = 0;
20     TaskReplyStatus status = TaskReplyStatus::Succeeded;
21 };

```

`request_id` 区分一次消息交互，`attempt` 区分物理执行，`driver_epoch` 区分控制权时代，`task_id` 区分逻辑贡献。四者不能互相替代。超时重试后，旧 request 可能回来；Driver 重启后，旧 epoch 的 worker 可能继续发送；同一 task 的不同 attempt 可能都写出 block；这些情况都要被显式识别。

Driver 接收 reply 的判定顺序 收到 reply 后，Driver 不能先写 manifest。它要按从外到内的顺序做检查：job/stage 是否匹配，epoch 是否当前，task 是否存在，task 是否已经 committed，attempt 是否仍然是当前 attempt，block 是否校验通过，status 是否允许提交，最后才尝试 manifest append。顺序错误会放大事故。例如先校验 block 再看 task 状态，可能把已提交 task 的旧 block 误加入候选；先接受 attempt 再看 epoch，可能让旧 Driver 时代的消息污染新控制面。

Listing 21.20 Driver 处理 TaskReply 的语义伪代码

```

handle_reply(reply):
    if reply.job_id != current_job_id:
        record rejected_wrong_job
        return

    if reply.driver_epoch != current_driver_epoch:
        record stale_epoch_reply
        reap_candidate_block(reply.block_id)
        return

    task = task_table.find(reply.task_id)
    if task is missing:
        record unknown_task_reply

```

```

        reap_candidate_block(reply.block_id)
        return

    if task.state == Committed:
        record_late_or_duplicate_reply
        reap_candidate_block(reply.block_id)
        return

    if reply.attempt != task.current_attempt:
        record_stale_attempt_reply
        reap_candidate_block(reply.block_id)
        return

    if reply.status == Busy:
        reschedule_with_backoff(task)
        return

    if reply.status is retryable_failure:
        retry_or_fail(task)
        return

    if reply.status is fatal_failure:
        mark_task_failed(task)
        return

    if not verify_block(reply.block_id, reply.bytes, reply.checksum):
        retry_or_fail(task)
        return

    append_manifest_if_absent(task.task_id, reply.attempt, reply.block_id)
    mark task Committed

```

这段伪代码刻意把 `reap_candidate_block` 放在拒绝路径里。旧 block 可以存在一段时间，但必须有分类：late、duplicate、stale epoch、unknown task、checksum failed。清理失败不是业务失败，接受失败才是业务语义。RunReport 应把这些分类写清，便于判断是网络抖动、Driver 重启还是状态机错误。

manifest append 必须幂等 `append_manifest_if_absent` 是提交路径的核心。它的语义不是“追加一行文本”，而是“若 logical task 尚未提交，则接受该 attempt；若已经提交同一 attempt，则返回 already committed；若已经提交不同 attempt，则拒绝”。文件格式可以是 append-only log、SQLite、简单二进制记录或内存模拟，但语义必须一致。

Listing 21.21 幂等 manifest 提交的语义伪代码

```

append_manifest_if_absent(task_id, attempt, block_id):
    lock manifest_index

    existing = manifest_index.find(task_id)
    if existing exists:
        if existing.attempt == attempt and existing.block_id == block_id:
            return AlreadyCommitted
        return DuplicateRejected

```

```

entry = build_manifest_entry(task_id, attempt, block_id)
durable_append(entry)
manifest_index.insert(task_id, entry)
return Committed

```

这个函数有两个关键点。第一，内存索引和持久记录要保持同一语义；崩溃后通过重放 manifest log 重建索引。第二，重复提交同一条记录可以幂等返回成功，重复提交不同 attempt 必须拒绝。否则 retry、迟到响应和恢复重放都会把提交路径变成随机结果。

checkpoint 不能替代 manifest Checkpoint 是恢复加速器，不是提交权威。它可以保存 task table、retry budget、route epoch 和 manifest generation，但不能让 reduce 读取 checkpoint 中的候选 block。原因很简单：checkpoint 可能比 manifest 旧，也可能在 manifest append 之前生成。恢复时若 checkpoint 说 task Running，而 manifest 已经接受该 task，应以 manifest 为准；若 checkpoint 说 task Committed，但 manifest 没有记录，应重新审计，不能直接相信 checkpoint。

恢复时看到的组合	含义	动作
manifest 有， checkpoint 无	提交后崩溃或 checkpoint 落后	承认提交，更新 task table
manifest 无， checkpoint Running	未提交或提交前崩溃	重试或等待恢复策略
manifest 无， checkpoint Committed	checkpoint 不可信或写序错误	进入 audit，重新校验，不直接提交
manifest 有两个同 task	manifest 损坏或提交协议错误	停止并报告，不自动选择

恢复扫描要先分类，不先接受 恢复程序启动后会看到许多材料：manifest log、checkpoint、临时 block、worker reply 日志、故障注入记录、未清理目录。错误做法是扫描目录，看到 block 文件就加入 reduce；正确做法是先重建权威状态，再把每个材料分类。Manifest 中的 accepted entry 是提交事实；与 manifest 匹配的 block 是已接受数据；没有 manifest 的完整 block 只是 candidate；旧 epoch、旧 attempt、未知 task 和 checksum 错误都是拒绝材料；格式损坏或身份不完整的材料进入 corrupt 类。

分类	判定依据	恢复动作
accepted	manifest 有 task/attempt/block, checksum 匹配	同 承认提交，供 reduce 使用
candidate	block 完整但 manifest 无记录，task 仍可重试	保留审计或重新调度，不直接提交
orphan	task 不存在、stage 已结束或 job generation 不匹配	记录并等待清理安全点
stale attempt	task 已提交其他 attempt，或 attempt 不是当前 attempt	拒绝，计入 duplicate/stale 指标
stale epoch	driver epoch、route epoch 或 owner 已过期	拒绝，禁止补写 manifest
corrupt	bytes/checksum/metadata 不一致或文件不完整	隔离，触发 retry 或失败策略

这张表要进入恢复代码，而不是只出现在说明里。恢复扫描的输出应是 **RecoveryReport**：每个目录发现多少 accepted、candidate、orphan、stale、corrupt，

分别占用多少字节，哪些被清理，哪些等待审计。只有分类稳定，恢复才不会被文件名、修改时间和目录遍历顺序支配。

端到端 stale attempt 场景 下面把一个旧 attempt 从产生到被拒绝写成完整链路。Task 7 的 attempt 0 被派给 Worker A。A 写 block 时网络变慢，Driver 超时后派 attempt 1 给 Worker B。B 成功写出 block，reply 到达，manifest 接受 T7,A1,B7a1。稍后 A 的 attempt 0 才完成并发送成功 reply。若 Driver 只看 checksum，A0 可能覆盖 A1；若 Driver 按身份链判定，A0 会进入 stale attempt。

Listing 21.22 迟到 attempt 的端到端事件链

```
T0:
  Driver assigns task=7 attempt=0 to worker A

T1:
  worker A writes block B7a0 slowly
  Driver timeout expires

T2:
  Driver assigns task=7 attempt=1 to worker B
  worker B writes block B7a1
  Driver receives reply for attempt=1
  manifest accepts task=7 attempt=1 block=B7a1

T3:
  worker A finally sends success reply for attempt=0
  Driver sees task=7 already committed with attempt=1
  reply is stale_attempt
  block B7a0 is not visible to reduce
```

这里的关键不是“谁先完成”，而是“哪个 attempt 被 manifest 接受”。真实系统中，旧 attempt 可能在 Driver 重启后才到达；可能只有 block 文件留下而 reply 丢失；也可能旧 worker 处在旧 driver epoch。每种情况都应落到同一套分类：没有 accepted manifest 就不是提交事实，已经提交其他 attempt 就不能再改变结果。

Checkpoint 提前和落后的双向事故 Checkpoint 和 manifest 的相对顺序会制造两种事故。第一，manifest 已提交，checkpoint 尚未更新；恢复必须承认 manifest，不能因为 checkpoint 落后就重做并产生重复 attempt。第二，checkpoint 声称 task committed，但 manifest 没有对应 entry；这可能是 checkpoint 写序错误、崩溃点落在 manifest 前，或 checkpoint 文件损坏。恢复不能直接相信 checkpoint，应进入 audit 或按未提交重试。

Listing 21.23 checkpoint 与 manifest 的恢复优先级

```
case checkpoint_lag:
  manifest: task 7 -> attempt 1
  checkpoint: task 7 Running
  decision: task 7 is Committed, update recovered task table

case checkpoint_ahead:
  manifest: no entry for task 8
  checkpoint: task 8 Committed
  decision: audit checkpoint, do not expose block to reduce
```

这条规则让系统避免“快照覆盖日志”。在控制面里，日志或 manifest 是提交权威，checkpoint 是缓存；缓存可以落后，也可能损坏，不能反向创造提交事实。

故障注入规则也要结构化 故障注入不是在代码里随手 `ifrandom()`。规则应能匹配对象身份，并且可重放。一个规则可以表达“第一次发送 `TaskReply(T2,A0)` 时丢弃”，另一个规则表达“`block_id=B1a0` 第一次 write 只写 4096 字节”。规则应有触发计数，避免每次都触发导致系统永远无法前进。

Listing 21.24 故障注入规则的语义形态

```
FaultRule:
  id: drop_reply_T2_A0_once
  match:
    event_kind: SendTaskReply
    task_id: T2
    attempt: A0
  trigger:
    max_times: 1
  action:
    DropMessage
```

结构化故障的收益是回归稳定。修复 manifest 去重后，`drop_reply_T2_A0_once` 必须长期保留；若未来重构再次让 reduce 扫描目录，这条规则会重新失败。随机压力测试可以发现新问题，但确定性规则负责防止旧事故复发。

MessageBus 的最小语义 单机模拟的 MessageBus 不应假装网络可靠。它至少支持延迟、丢弃、重复、乱序、Busy 和旧 epoch。每条消息进入 bus 时带 envelope，bus 决定何时投递或是否投递。Driver 和 worker 都不能依赖调用栈返回值来判断对方状态。

Listing 21.25 MessageEnvelope 的语义字段

```
MessageEnvelope:
  message_id
  source_node
  target_node
  driver_epoch
  request_id
  logical_task_id
  attempt
  deadline_ns
  payload_kind
  payload_checksum
```

这个 envelope 未来可以落到进程间 socket、管道、共享内存队列或真实网络。早期用内存队列模拟时，也必须保留序列化和反序列化边界；否则两进程版本会突然暴露字段缺失、指针不可传、对象生命周期错误和共享状态假设。

端到端不变量 最终状态机至少要守住六条不变量：

1. 同一个 logical task 在 manifest 中最多有一个 accepted attempt。
2. Reduce 只读取 manifest 引用的 block。
3. Worker 成功、block 存在、reply 成功都不能单独推出 committed。
4. 旧 epoch、旧 attempt、未知 task 的 reply 只能进入拒绝和清理路径。
5. Checkpoint 不能覆盖 manifest，恢复必须先承认提交事实。

6. 每个被拒绝的候选 block 都要有分类和清理策略。

这些不变量应写进测试名，而不只写在说明里。例如 `late_reply_does_not_replace_committed_attempt`、`reduce_reads_manifest_not_directory`、`checkpoint_cannot_commit_without_manifest`。测试名本身就是系统边界的索引。

21.8 可靠 I/O、Checkpoint 和 Backpressure

最终项目的 I/O 层承接第 15 章：submit、completion 和 commit 必须分开。Worker 写出 block 时，先写临时文件，可靠写循环处理短写，完成后计算 checksum，再向 Driver 提交。Driver 接受后，manifest 才让 block 对 reduce 可见。短写、延迟 completion、取消后 completion、rename 前后崩溃，都应进入测试。

Checkpoint 内容 Checkpoint 保存控制面，不复制所有大数据。至少包含 job id、stage 状态、task table、manifest generation、retry budget、route epoch、输入身份和 idempotency 摘要。Shuffle block 本身可以留在数据面，只要有稳定路径、bytes、checksum 和 manifest 引用。Checkpoint 小而强，block 大而可校验，这就是控制面和数据面分离在最终项目里的落地。

Backpressure 贯穿流水线 读取、解析、map combine、shuffle 写出、reduce 合并和 manifest commit 都可能成为瓶颈。每个阶段都要有有界队列或资源窗口：input queue、in-flight shuffle bytes、buffer pool、reduce partition queue、checkpoint writer。队列满时，上游应等待、降速、合并或返回 Busy，而不是无限分配内存。RunReport 要记录 high watermark、busy count、wait time 和 dropped/late events。

资源预算 最终项目不是某个函数最快，而是在资源预算内稳定完成工作。预算包括线程数、队列容量、buffer pool bytes、临时文件数、in-flight requests、checkpoint frequency 和 retry count。若吞吐提升伴随内存无限增长，不能算成功；若平均延迟下降但 p99 和 recovery cost 爆炸，也要写入报告。

21.9 失败路径判读：不要让 happy path 定义系统

分布式计算路径不能只读成功分支。成功分支通常很短：worker 执行、写出 block、reply 到达、manifest 接受、reduce 读取。真正决定系统质量的是失败分支是否和成功分支共享同一套状态机。故障规则可以很少，但必须命中关键边界：短写、reply 丢失、manifest 前后崩溃、旧 epoch 响应。

故障矩阵

故障	注入方式	预期行为
坏行	split 中插入非法字段	reference 和优化路径记录同样错误摘要
短写	第一次 write 只写部分字节	继续写剩余字节，未完整不提交 manifest
响应丢失	drop 某次 TaskReply	timeout 进入 unknown, retry 新 attempt, manifest 唯一
重复响应	同一 reply 投递两次	第二次进入 duplicate 或 late 指标
Driver 崩溃	manifest 前后分别终止	前者重做，后者承认已提交事实
Reduce busy	partition queue 高水位	Map 端 背 压, retry budget 不被放大
热点 key	一个 key 占大多数输入	普通 hash 长尾，拆分后正确二级合并
旧 epoch	route 切换后旧请求到达	StaleRoute, 不 改 变 manifest

崩溃点矩阵要贴着提交路径 Driver 崩溃不能只测“运行中 kill 一次”。提交路径由多个边界组成：worker 写临时 block，写满字节，计算 checksum，发送 reply，Driver 校验 block，append manifest，更新内存 task table，写 checkpoint，reduce 读取 manifest。每个边界前后崩溃，恢复动作都不同。矩阵要贴着这些边界，而不是随机挑一个时间点。

崩溃点	恢复时可能看到什么	正确动作
临时 block 写到一半	文件存在但 bytes/checksum 不匹配	corrupt，删除或隔离，重试 task
block 完整但 reply 未发送	完整 candidate，无 manifest	不提交，按 task 状态重试或审计
reply 到达但 manifest 前	candidate block，checkpoint 可能 Running	重试或重新校验，不对 reduce 可见
manifest append 后、内存表前	manifest 有 accepted，checkpoint 落后	承认提交，重建 task table
内存表后、checkpoint 前	manifest 有 accepted，checkpoint 落后	以 manifest 为准，更新 checkpoint
checkpoint 声称 committed 但 manifest 无	checkpoint ahead 或损坏	audit，不直接提交
reduce 过程中崩溃	manifest 不变，partial output 可能残留	清理候选输出，重新 reduce committed set

这个矩阵把提交权威固定在 manifest。Checkpoint、目录和内存表都可能落后或领先，不能反过来创造提交事实。故障注入规则应该能精确命中这些边界，例如 `crash_after_manifest_append_before_task_table_update`，而不是只写 `crash_randomly`。

恢复报告要解释每个文件的命运 恢复扫描后，RunReport 应能回答三个问题：哪些文件进入 accepted 集合，哪些文件被拒绝，哪些文件暂时保留等待审计。每个文件都要有身份链和分类理由。否则，恢复程序虽然最终 checksum 正确，也无法证明没有把孤儿文件碰巧排除，或把旧 attempt 碰巧覆盖。

```

recovery:
  manifest_generation_before
  manifest_generation_after
  checkpoint_generation_loaded
  accepted_blocks
  candidate_blocks
  orphan_blocks
  stale_attempt_blocks
  stale_epoch_blocks
  corrupt_blocks
  bytes_reclaimed
  audit_required

block_decision:
  block_id
  task_id
  attempt
  driver_epoch
  bytes
  checksum_status
  manifest_status
  decision
  reason

```

这份报告会让恢复从“目录里剩了什么”变成“每个材料被哪条规则处理”。当后续把单机 MessageBus 换成多进程或真实网络，恢复报告仍然有效，因为它依赖的是身份、attempt、epoch、manifest 和 checksum，不依赖传输方式。

故障规则要覆盖迟到和重复的组合 真实事故经常不是单个故障，而是组合：reply 丢失导致 retry，旧 attempt 稍后成功，Driver 又在 manifest 后崩溃，重启时目录里有两个完整 block。若测试只覆盖单一 drop 或单一 crash，无法证明组合安全。确定性故障规则要支持顺序触发、次数限制和组合命名。

Listing 21.27 组合故障规则示例

```

FaultScenario late_attempt_after_commit_and_restart:
  1. drop TaskReply(task=7, attempt=0) once
  2. allow timeout and assign attempt=1
  3. allow attempt=1 manifest commit
  4. crash driver after manifest append before checkpoint
  5. deliver delayed reply for attempt=0 after restart

expected:
  manifest keeps attempt=1
  attempt=0 is stale_attempt
  recovery accepts B7a1 only
  RunReport records one stale reply and one checkpoint lag

```

组合规则的价值在于长期回归。后续重构 writer、MessageBus、manifest 或 checkpoint 时，这条场景都应继续通过；一旦失败，报告能指出是幂等提交、恢复优先级还是旧 reply 分类被破坏。

状态复盘是最终系统的用户界面 状态复盘不是日志汇总，而是可审查证据。它至少要说明输入摘要、错误行数、stage 耗时、队列高水位、task 数、attempt

数、late reply 数、duplicate reject 数、manifest generation、checkpoint generation、故障规则、恢复动作和未验证边界。字段不必多，但必须能回答“这个结果为什么生效，那个候选为什么被拒绝”。

Listing 21.28 状态复盘摘要示例

```
input.records=3000000
input.bad_records=12
reference.checksum=0x51b76290
engine.checksum=0x51b76290
tasks.total=96
attempts.total=101
reply.late=2
commit.duplicate_rejected=3
shuffle.bytes=1843200
queue.reduce.high_watermark=65536
manifest.generation=17
checkpoint.generation=4
fault.rule=drop_task_reply_once
boundary.unverified=real_network_power_loss
```

复盘要机器可读，也要人能读。机器可读便于回归比较，人能读便于排查事故。若输出只能说“success”，它不能支持工程判断；若输出能解释每个故障的状态变化，它就是系统的一部分。

21.10 最终验收：不是跑完，而是可证明

最终项目的验收必须分层。能在无故障输入上跑完，只通过了最浅的一层。一个现代计算系统至少要证明五类事实：语义正确，资源受控，失败可恢复，性能可解释，边界可审计。任何一类缺失，系统都可能在演示中成功、在真实负载中失控。

语义正确 语义正确的最低证据是 reference checksum 与 engine checksum 一致，错误摘要一致，输出顺序稳定，manifest 引用的 block 集合与 logical task 集合一致。输入包含坏行、空 split、重复 key、热点 key、短写、迟到响应和 Driver 崩溃时，仍要保持这些事实。若优化路径在坏行密集输入下少报 diagnostics，不能用吞吐解释；若 reduce 扫描目录导致重复 attempt 被计入，不能用“文件确实存在”解释。

资源受控 资源受控要求线程数、队列长度、in-flight bytes、buffer pool、临时文件数量、retry 数和 checkpoint 大小都有上界。无界队列跑得快不是优点；它只是把压力推迟到内存。报告应能给出最大水位和限速次数。若写出慢，reader 和 parser 最终必须减速；若 manifest commit 慢，completion、buffer pool 和上游计算都应感受到背压。资源控制不是性能附属项，而是正确运行的一部分。

失败可恢复 恢复证明至少包含三类崩溃点：manifest 前、manifest 后、checkpoint 前后。Manifest 前崩溃，未提交 block 只能成为候选或孤儿；manifest 后崩溃，恢复必须承认提交事实；checkpoint 落后时不能覆盖 manifest；checkpoint 提前声称 committed 但 manifest 没有记录时，必须进入 audit。恢复程序不能靠目录扫描决定结果，也不能让旧 epoch 或旧 attempt 的响应补写 manifest。

性能可解释 性能可解释不是要求所有指标完美，而是要求瓶颈能被定位。RunReport 至少给出阶段耗时、队列等待、service time、worker idle、retry amplification、combine ratio、shuffle bytes、commit wait 和故障注入开销。若端到端变慢，报告能说明是 parse、combine、shuffle write、manifest commit、retry 还是背压；若吞吐提升，报告能说明是否牺牲了错误摘要、可靠写或资源上

界。

边界可审计 边界可审计要求每个对外事实都有身份链。输入记录能追到 file id 和 offset; partial 能追到 split 和 attempt; block 能追到 logical task、attempt、checksum; manifest 能追到 generation; message 能追到 request id 和 driver epoch; 恢复动作能追到故障规则和审计记录。没有身份链, debug 只能靠人工猜测; 有身份链, 系统可以把事故缩小到某个边界。

验收层	必须提供的证据	常见假通过
语义	reference、checksum、diagnostics、稳定输出	只比较成功输入的最终计数
资源	水位线、in-flight bytes、线程数、临时文件数	无界队列短跑吞吐高
恢复	崩溃点矩阵、manifest 优先、orphan 分类	重启后扫描目录接受所有文件
性能	阶段时间、队列等待、背压和 retry 指标	只给总耗时和平均吞吐
审计	input/split/attempt/block/manifest 日志里只有字符串消息身份链	

小规模也要完整 最终项目可以只处理几百万行日志, 不必追求工业规模。但状态机必须完整。小规模隐藏不了语义漏洞: 迟到响应仍能出现, 短写仍能模拟, Driver 崩溃仍能注入, manifest 仍要唯一, checkpoint 仍可能落后。规模小只是让测试可复现, 不是降低边界要求。

失败样本不应从性能统计中删除 如果某次运行 checksum 不一致、diagnostics 不一致、manifest 重复、资源越界或恢复 audit 失败, 该样本不能作为性能样本参与平均值。它应进入 failure report。性能报告只统计 Passed 样本, 并记录 failed/skipped 数量。否则系统会奖励那些在压力下跳过工作的版本。

Listing 21.29 最终验收摘要的机器可读形态

```
validation:
  semantic_status: Passed
  resource_status: Passed
  recovery_status: Passed
  performance_status: Explained
  audit_status: Passed

samples:
  measured: 20
  passed: 20
  failed: 0
  skipped: 0

boundaries:
  real_network: Unverified
  power_loss_after_fsync: Unverified
  multi_socket_numa: Unverified
```

把未验证边界写出来, 不会削弱项目, 反而说明结论没有越界。单进程 MessageBus 验证了消息乱序、丢弃和迟到的控制逻辑; 它没有验证真实网络拥塞、内核 socket buffer 和跨主机时钟。普通文件模拟验证了短写和 manifest; 它不一定验证掉电持久化。边界越清楚, 后续扩展越稳。

21.11 里程碑：每一步都能失败、解释和回退

最终项目不应一次性实现完整框架。每个里程碑只引入一个新边界，同时保留上一版可运行结果和失败测试。这样复杂度有来源，出错时能回退。

一、串行 reference 和输入身份 实现输入解析、错误摘要、排序输出和 reference report。测试空文件、单行、坏行、越界 event type、超长行和固定随机输入。这个阶段回答“到底在算什么”。

二、HotBatch 和本地 combine 把热字段从原始记录中分离，保留 record id 回溯。实现本地 combine，并记录 key 数、combine ratio、内存峰值。这个阶段回答“如何少搬热路径数据，同时不丢证据”。

三、并行 partition 和任务运行时 用 count-scan-write 替代共享 push，建立 bucket range 和 partition manifest；再用线程池或任务图执行 chunk 任务，记录 ready、running、waiting、done、cancelled。这个阶段回答“多个 worker 如何写同一个逻辑输出而不互相踩”。

四、可靠写和 manifest commit 实现临时 block、短写处理、checksum、manifest append 和恢复扫描。测试写中崩溃、写完未提交崩溃、提交后崩溃。这个阶段回答“文件存在为什么不等于结果有效”。

五、MessageBus、retry 和幂等提交 把本地调用改成消息事件。加入 request id、attempt、deadline、retry budget、Busy 和 late reply。测试响应丢失后只提交一个 attempt。这个阶段回答“远端执行成功为什么不是外部生效”。

六、Checkpoint、热点 key 和两进程边界 加入 checkpoint generation、恢复 audit、热点 key 拆分和可选两进程部署。两进程部署不追求性能，而是检查序列化、framing、process crash 和共享状态假设。这个阶段回答“系统重启后谁说了算，以及单进程模拟有没有偷共享内存的懒”。

21.12 一次失败排查日记

最终系统覆盖的不只是设计能力，还包括排查能力。假设小输入只有一百条记录，reference 输出 key A 为三十七，优化版输出三十九。差异很小，最容易被误判成并行顺序、随机噪声或“测试不稳定”。正确做法不是先改代码，而是沿状态链一层一层收窄。

一、先排除输入和解析 先比较 input checksum、record count、bad record count 和 reference parser 的错误摘要。若这些不同，问题还在输入身份或 parser 合同；不要继续查 shuffle。若这些一致，再比较 HotBatch checksum 和 record id 回溯。若 HotBatch 与 reference 的 key/value 序列一致，说明冷热布局没有丢字段。排查要先把上游语义排除，而不是一看到并行版本就怀疑线程。

二、再排除输出位置 查看 count、scan、write 三阶段的 bucket range。若 count 阶段显示 chunk 4 对 bucket 2 有七条，write 阶段写了九条，就说明 predicate 在两次执行中不一致，或者 chunk 4 被执行了两次。若 range 正确但最终多了两条，就继续查 manifest。Scan 和 partition 的中间账本，是把“结果不一致”缩小到“输出位置或重复提交”的关键证据。

三、最后查提交点 Task table 显示 chunk 4 有两个 attempt。Attempt 0 写出成功，但 TaskReply 被故障规则丢弃；Attempt 1 后来写出并提交。Manifest 却包含 attempt 0 和 attempt 1 两条 block。根因不是 reduce 算错，而是 manifest 按 block 文件名接受提交，没有按 logical task 去重。修复应落在 CommitRecord：同一个 logical task 只能接受一个 attempt，旧 attempt 只能进入 LateAttempt 或 Duplicate 分类。

四、把事故固化成回归测试 新增回归测试：drop_reply_then_retry_keeps_one_manifest_entry。输入固定，故障规则固定丢 attempt 0 的 reply。测试断言 parallel checksum 等于 reference，manifest 中 chunk 4 只有一条 accepted entry，late reply count 为一，临时目录可以存在旧 block。注意最后一条：旧

block 存在不是错误，错误是把它当提交事实。这个测试能防止后续重构再次把目录扫描引回 reduce。

四、把事故固化成判定规则 新增规则：同一个 logical task 只能有一个 accepted attempt；旧 block 可以存在，但不能被 reduce 读取；迟到 reply 必须进入 LateAttempt 或 Duplicate 分类。规则不是越多越好，而是每次事故后补上能缩短排查路径的判定点。成熟系统会把排查经验沉淀成状态机和回归用例，而不是只沉淀在某个维护者的记忆里。

从事故回到代码改动 这次事故对应的代码改动很窄：CommitRecord 按 logical task 保存 accepted attempt；manifest.try_accept 拒绝不同 attempt；reduce 只从 manifest 枚举 block；恢复扫描只产生 candidate，不直接提交。这样修复落在提交边界，而不是在 reduce 里补特殊判断。

21.13 工业设计参照

工程案例

Spark: stage、shuffle 和 output commit Spark 的核心经验不是“分布式计算可以并行”，而是把宽依赖切成 stage，把 map output 和 shuffle metadata 管起来，把 task attempt 和输出提交分开。它面对 executor 故障、长尾 task、shuffle 文件丢失和 speculative execution。核心机制是 stage boundary、task attempt、map output tracker、shuffle block 和 output commit protocol。代价是 driver 元数据和 shuffle 服务成为关键路径，speculation 也会放大资源消耗。Compute Systems Engine 应借鉴 stage、attempt、shuffle manifest 和 speculative attempt 的幂等提交边界，但不需要实现 Spark 的完整调度、资源管理和外部 shuffle 服务。

工程案例

Flink: checkpoint barrier 和状态一致性 Flink 这类流式系统面对的是持续输入、算子状态和外部输出的一致恢复。核心机制是 checkpoint barrier、operator state、input offset 和 sink commit 的绑定。它说明 checkpoint 不能只保存“程序运行到哪里”，而要把输入位置、状态和输出提交点放在同一一致边界里。代价是 checkpoint 频率、状态大小和外部 sink 协议会直接影响延迟和吞吐。Compute Systems Engine 是批处理模拟，但应借鉴这个原则：checkpoint 保存控制面和提交边界，不只保存内存 dump。若只保存 task table，不保存 manifest generation，恢复后仍可能重复输出。

工程案例

DuckDB/ClickHouse: 向量化执行和 pipeline 单机分析型执行器的经验同样适用最终项目。DuckDB、ClickHouse 这类系统强调列式或向量化 batch、pipeline、selection vector、压缩和 operator profiling。它们面对的是 CPU cache、SIMD、分支、内存带宽和复杂查询管线。核心机制是批量处理热字段，让算子间传递结构化 batch，并输出可解释 profile。Compute Systems Engine 应借鉴 batch、selection vector、operator profile 和 pipeline breaker 的思想。Map-side combine 和 reduce merge 不应只当分布式概念，也要继承单机执行器的热路径纪律。

工程案例

Ray/actor runtime: 任务、对象和故障边界 Ray 等系统把分布式执行抽象成 task、actor 和 object store, 面对的是任务依赖、对象传输、worker 故障和调度可观察性。它提醒我们: 数据对象和任务状态要有稳定身份, 运行时不能只靠调用栈理解依赖, 失败后要知道哪些对象可重建、哪些需要重新提交。

Compute Systems Engine 不需要复制 Ray 的 API, 但应借鉴对象身份、任务 trace 和故障重建的思想。ShuffleBlock、ManifestEntry 和 TaskRecord 就是示例版的对象和任务账本。

21.14 端到端反例：一个小优化怎样破坏整条链

到这里, 最终系统已经有很多对象和边界。如果继续用抽象词堆叠, 很容易看起来完整, 实际却没有经受事故检验。下面只看一个具体优化: 为了减少磁盘写入, 开发者把 map 端输出从“每个 split 一个 shuffle block”改成“多个 split 共享一个大 block”, 并把 manifest entry 写成共享 block 的文件名和总 checksum。单次正常运行可能更快, 目录里文件更少, reduce 也能读出结果; 但一旦某个 split 重试, 问题就暴露出来。

共享 block 把逻辑身份压扁 原设计中, logical split、attempt、block 和 manifest entry 一一对应或有明确映射。Attempt 0 迟到时, manifest 可以拒绝它; split 1 重试时, 只替换 split 1 的 block。共享 block 把 split 0、split 1、split 2 的输出混在同一个文件里, manifest 只知道文件存在, 不知道每段字节属于哪个 split、哪个 attempt、哪个 route epoch。于是一个局部重试变成全文件重写; 一个迟到 reply 可能让旧 attempt 的字节混进新 block; 一个短写也无法判断影响哪个 logical split。

Listing 21.30 错误优化后的模糊身份

```
bad merged block:
  file = shuffle-0007.tmp
  contains split 0 attempt 0
  contains split 1 attempt 1
  contains split 2 attempt 0
  manifest only records file name and total checksum

missing:
  byte ranges per logical split
  accepted attempt per split
  route epoch per contribution
  per-range verification
```

这个反例不是说不能合并小文件。正确做法是把合并后的物理文件继续切成可审计的逻辑 extent: 每段记录 split id、attempt id、range、record count、checksum 和 route epoch。Reduce 可以按物理文件顺序读取以获得顺序 I/O, 提交语义仍按逻辑 extent 判定。性能优化不能删除身份, 只能改变身份的物理布局。

优化前先问它删除了哪个判定点 一个系统级优化如果只说明“减少文件数”“减少锁”“减少 copy”, 还不够。它必须回答删除了哪个边界、合并了哪些状态、失败后能否恢复原来的判定粒度。共享 block 优化删除的是 block 与 logical split 的直接映射; 全局队列改 work stealing 改变的是 task 执行位置; 异步写改变的是调用返回和业务提交之间的距离; 缓存 route 查询改变的是读到控制面事实的时刻。每个优化都要保留原判定点, 或者明确用新的判定点替代。

优化	容易删除的判定点	保留语义的办法
合并 shuffle 文件	每个 split 的 attempt 身份	物理文件内记录 logical extent
批量 manifest commit 缓存 route	单个 entry 的接受/拒绝原因 当前 owner 判定	批内每项独立状态和错误码 响应携带 epoch, 提交前强检查
投机执行 异步 buffer 复用	唯一 accepted attempt buffer 唯一所有者	commit gate 拒绝迟到成功 generation handle 和终态回收

21.15 面向后续推理系统的接口

没有在这里直接展开 CPU 推理引擎, 是为了先把计算系统地基打稳。后续推理系统会处理模型权重、tensor buffer、算子内核、batch 调度、KV cache、请求 deadline 和流式输出。这些不是全新问题, 而是当前接口的延伸。

模型文件对应输入和 manifest 模型文件有 tensor 名称、shape、dtype、量化参数、checksum 和版本。它比日志输入更严格, 但同样需要输入身份、可靠读取、manifest 和版本边界。加载权重时, CompletionRecord、checksum 和 manifest 仍然适用。

算子对应 kernel 和 task 矩阵乘、归一化、softmax、采样和量化都需要 scalar reference、误差合同、HotKernelReport、MemoryShape 和 KernelVariant。线程池和任务图会调度 layer、tile、token 或 batch。背压从 shuffle queue 变成 request queue 和 token generation queue。

KV cache 对应生命周期和所有权 KV cache 是跨 token 持续存在的状态, 既要高效访问, 又要按请求隔离。它会重新考验 BatchView、buffer ownership、ProgressAndReclaim、队列关闭和取消语义。若当前项目已经把所有权和状态机写清, 进入 KV cache 时就不会从零开始。

21.16 端到端交付门禁

一、Reference 是否仍然是权威 所有优化版本、故障版本和分布式模拟都要能回到 reference。若 reference 未定义错误行、输出顺序或溢出语义, 先补 reference, 不要继续优化。

二、Manifest 是否是唯一提交入口 Reduce 和恢复都只能读取 manifest 接受的 block。临时目录、worker 0k 响应和 checkpoint 内存状态都不能越过 manifest。

三、故障是否可复现 响应丢失、短写、Driver 崩溃、旧 epoch、Reduce busy 和热点 key 都要用固定规则复现。随机压力可以补充, 不能替代确定性故障。

四、报告是否能解释失败 RunReport 应能沿输入、batch、partition、block、manifest、checkpoint 查到差异位置。若失败只能靠翻源码, 报告还不合格。

五、性能结论是否有边界 性能报告必须带输入、环境、线程数、阶段耗时、资源峰值和未验证条件。移动端或受限 Linux 环境的数字不能被写成通用硬件结论。

六、后续接口是否克制 可以预留 ModelArtifact、TensorBuffer、KernelTask 和 InferenceRequest 的接口方向, 但不要在这里展开 AI 算子正文。当前任务是收束计算系统, 不是提前分散主题。

源码阅读任务

源码和工业阅读任选一条:

1. 阅读 Spark 的 task attempt、shuffle block 或 output commit 相关路径, 说明它如何避免重复输出。
2. 阅读 Flink checkpoint 和 sink commit 的一条路径, 说明

input offset、operator state 和外部输出如何绑定。

3. 阅读 DuckDB、ClickHouse 或类似执行器的 vectorized pipeline/profile 路径，说明 batch 和 pipeline breaker 如何影响性能证据。
4. 阅读 Linux 的 futex、page cache、epoll 或进程退出路径之一，必须从项目现象进入，不允许只列函数名。

报告按“项目现象、源码入口、能解释的事实、不能证明的边界、回到项目的设计结论”组织。

21.17 从 C++ 提交代码走到协议状态

最终章的关键不是再列一遍运行清单，而是把代码路径读透。下面这段 C++ 伪代码看起来只是“收到 worker 回复后提交 block”，实际跨过了消息身份、I/O 完成、幂等提交和恢复权威四个边界：

Listing 21.31 Driver 提交路径的最小形状

```

1 CommitDecision on_task_reply(const TaskReply& reply) {
2     TaskRecord& task = task_table.lookup(reply.logical_task);
3
4     if (reply.driver_epoch != current_driver_epoch) {
5         return reap(reply, RejectReason::StaleDriverEpoch);
6     }
7     if (task.state == TaskState::Committed) {
8         return reap(reply, RejectReason::AlreadyCommitted);
9     }
10    if (reply.attempt != task.current_attempt) {
11        return reap(reply, RejectReason::LateAttempt);
12    }
13    if (!verify_block(reply.block_id, reply.bytes, reply.checksum)) {
14        task.state = TaskState::RetryableFailed;
15        return CommitDecision::Retry;
16    }
17    return manifest.try_accept(reply.logical_task,
18                               reply.attempt,
19                               reply.block_id,
20                               reply.checksum);
21 }

```

第一层是普通分支和内存访问。编译成汇编后，`task_table.lookup` 会变成哈希、数组索引或树查找，真正成本取决于地址局部性；连续 task id 可以让表访问接近数组，随机 id 会把路径变成多次 cache miss。第二层是状态判断顺序。先检查 epoch，是为了让旧 Driver 时代的消息没有机会影响当前 task；先检查 committed，是为了让迟到成功不能覆盖已经接受的 attempt；最后才校验 block，是为了避免给已经无权提交的旧 block 做昂贵 I/O。

第三层是提交线性化点。`manifest.try_accept` 才是 logical task 从“有候选输出”变成“对 reduce 可见”的位置。它内部必须按 logical task 查重，而不是按文件名查重。若这里用普通 append 而没有索引保护，两条并发 reply 可能都写入 manifest；若用锁保护，锁释放前 manifest index 和持久记录必须语义一致；若用单线程 commit loop，所有回复都必须排队到这个 loop，不能绕开。

Listing 21.32 manifest gate 的语义，不是文件追加技巧

```

try_accept(task, attempt, block, checksum):
    existing = accepted_by_task.find(task)
    if existing is empty:
        append durable entry
        accepted_by_task[task] = attempt
        return Accepted
    if existing.attempt == attempt:
        return AlreadyAccepted
    return RejectedLateAttempt

```

第四层是恢复。在线路径和恢复路径必须走同一个 gate。重启后扫描到一个完整 block，只说明存储事实存在；它仍要带着 logical task、attempt、epoch 和 checksum 回到 `try_accept`。这样代码、协议和恢复才是同一套逻辑，而不是正常运行一套、重启修复另一套。

21.18 模块边界：代码可以换，语义不能换

最终系统可以持续替换内部实现，但不能替换语义边界。Parser 可以从对象数组改成列式 batch；runtime 可以从全局队列改成本地 deque；I/O 可以从同步 write 改成 `io_uring`；MessageBus 可以从单进程模拟改成 TCP。每次替换都要问同一个问题：哪条底层路径变短，哪条状态边界变复杂，原来的线性化点在哪里。

模块边界的最终不变量 最终系统可以分成输入、热批次、运行时、I/O、提交、恢复六个模块。每个模块都要交出一个不变量，而不是只交出一组函数。输入模块保证 RecordId 稳定；热批次模块保证同一 row index 的字段来自同一记录；运行时保证 accepted task 最终进入终态；I/O 模块保证 completion 不等于 commit 且 buffer 有唯一所有者；提交模块保证一个 logical split 只接受一个 attempt；恢复模块保证 manifest 是权威入口。只要这些不变量守住，内部实现可以替换。

模块	不变量	主要证据
输入	RecordId 能定位唯一输入事实	input checksum、offset、line number
热批次	row 对齐，冷热回查不断链	BatchView checksum、cold lookup
运行时	接纳任务必有终态	task table、 timeline
I/O	buffer 唯一所有，completion 分层	CompletionRecord、 buffer report
提交	logical split 只接受一个 attempt	manifest、commit table
恢复	manifest 优先于 checkpoint 和目录	recovery audit

这张表是全书后半部分的最终压缩。它让重构有边界：可以换 SIMD backend，但不能破坏 BatchView；可以换线程池，但不能丢 task 终态；可以换 I/O backend，但不能让 completion 直接越过 manifest；可以多进程部署，但不能改变 attempt 和 commit 语义。

从对象到接口：最小可实现切片 最终系统的第一版不需要追求大而全，但每个模块都要有能编译、能被代码路径解释的接口。接口的目标不是把所有实现细节提前固定，而是让不变量有落点。输入模块交出 `InputSplit` 和 `RecordId`；热批次模块交出 `BatchView`；运行时交出 `TaskRecord`；I/O 模块交出 `CompletionRecord`；提

交模块交出 `ManifestEntry`。这些对象一旦存在，后续优化就必须围绕它们说明自己没有破坏语义。

Listing 21.33 最终系统第一版的核心接口

```

1 struct InputSplit {
2     std::uint64_t input_generation = 0;
3     std::uint64_t byte_begin = 0;
4     std::uint64_t byte_end = 0;
5     std::uint64_t first_record = 0;
6     std::uint64_t record_count = 0;
7     std::uint64_t checksum = 0;
8 };
9
10 struct BatchView {
11     std::span<const std::uint32_t> record_ids;
12     std::span<const std::uint32_t> event_types;
13     std::span<const std::int64_t> values;
14 };
15
16 struct TaskRecord {
17     std::uint64_t logical_task = 0;
18     std::uint32_t attempt = 0;
19     std::uint32_t route_epoch = 0;
20     std::string block_id;
21 };
22
23 struct ManifestEntry {
24     std::uint64_t logical_task = 0;
25     std::uint32_t accepted_attempt = 0;
26     std::string block_id;
27     std::uint64_t block_checksum = 0;
28     std::uint64_t accepted_records = 0;
29     std::uint64_t rejected_records = 0;
30 };

```

这段接口故意很小。它不绑定某个线程池、不绑定某个文件格式、不要求一开始支持网络，也不提前引入复杂模板。它只锁住六件不能变的事实：输入有稳定代号，batch 行之间能对齐，task 和 attempt 分开，block 有 checksum，manifest 是提交入口，记录数量和错误数量能被审计。后续把 `BatchView` 换成 AoSoA，把本地线程池换成多进程，把文件写入换成对象存储，都不能越过这些事实。

第一版只实现一条窄路径 第一版只需要三份 split、两个 worker、一个 reducer 和一个 manifest gate。输入中放一条坏行和一个热点 key；故障只保留两个：一次短写和一次 reply 丢失。短写证明 completion 不能直接等于 commit；reply 丢失证明 worker 成功不等于 Driver 接受。路径窄，分析才清楚。后续扩大输入、换内核、加 SIMD、加 NUMA 策略，都不能改变这条提交路径。

接口替换要说明底层成本迁移 最终系统会不断替换内部实现。比如把胖对象 parser 替换成热列 batch，成本从对象跳转迁移到列构造和冷表回查；把全局队列替换成本地队列加窃取，成本从全局锁迁移到任务迁移和 deque 边界；把普通写替换成异步写，成本从调用栈等待迁移到请求状态机和 buffer 生命周期。每次替换都要说明哪条底层路径变短，哪条路径变复杂，不能只写“更快”。

替换方向	变短的底层路径	新增的复杂边界
对象记录到热列 batch	热循环少追指针, cache line 利用率更高	构造 batch、冷表回查、row 对齐
全局计数到 worker partial	共享写减少, cache line 迁移减少	合并顺序、partial 生命周期
全局队列到本地 deque	常见 push/pop 本地化	steal 边界、任务迁移、NUMA 本地性
同步写到异步写	CPU worker 不被 I/O 等待占用	buffer owner、completion、迟到结果
单进程消息到多进程消息	共享内存假设被移除	framing、序列化、旧 epoch 消息

保留小模式是为了读代码 大输入模式用于吞吐, 小模式用于读代码和复盘状态。小模式和大模式共用同一代码路径, 只是 split 少、故障固定、状态更容易人工核对。它的价值不是多一个实验, 而是让提交路径、恢复路径和错误路径能被逐行追踪。

21.19 习题

习题与作业

习题一: 为日志统计系统写端到端数据流。要求包含 InputSplit、HotBatch、map-side combine、ShuffleBlock、ManifestEntry、ReduceOutput 和 Checkpoint。

习题二: 设计一个 reference 合同。说明输入坏行、event type 越界、value 溢出、输出排序和错误摘要如何处理。

习题三: 构造响应丢失事故。Attempt 0 写出 block 后 reply 被 drop, Attempt 1 提交, Attempt 0 reply 迟到。写出 task table、block table、manifest table 的状态变化。

习题四: 设计 shuffle manifest。说明它记录哪些字段, 为什么 reduce 不能扫描临时目录。

习题五: 设计 checkpoint 内容清单。要求说明哪些属于控制面, 哪些属于可校验数据面, 恢复时按什么顺序读取。

习题六: 分析热点 key 的执行路径。说明普通 hash 为什么形成单 reducer 长尾, map-side combine 怎样减少传输, 热点拆分为什么还需要二级 reduce。

习题七: 沿 `on_task_reply` 伪代码分析一个迟到响应。写出每个 if 分支为什么必须在 manifest append 之前执行。

习题八: 把普通同步写替换成异步写。说明 offset、buffer owner、completion、checksum 和 manifest commit 分别落在哪个状态。

习题九: 把当前接口迁移到 CPU 推理引擎。说明 ModelArtifact、TensorBuffer、KernelTask、KV cache、InferenceRequest 分别复用哪些系统能力。

习题十: 写一份最终项目评审报告。每个模块用“输入输出合同、不变量、失败路径、指标、未验证边界”五项说明。

工程案例

知识点小案例: reduce 不能靠扫描临时目录发现 shuffle。特例是 map attempt 0 写出 `block.tmp` 后 reply 丢失, attempt 1 重试并成功提交。目录里可能同时存在两个物理 block, 其中一个未提交或迟到产物。若 reduce 直接扫描目录, 它无法知道哪个 block 属于被接受的 attempt。manifest 记录 logical partition、attempt、checksum、

size、commit time 和状态，reduce 只读取 committed entry。由此推出规律：数据面文件存在不等于控制面事实成立。工程上所有 shuffle、checkpoint 和推理模型 artifact 都应通过 manifest 建立可恢复提交边界。

附录 A

实验路线

第二册的实验围绕 Compute Systems Engine 展开。它不是一组互不相干的 microbenchmark，而是一条逐步扩展的工程主线：先建立 reference、测量纪律和阶段报告，再加入数据布局、SIMD、同步、并行算法、运行时、I/O 背压和单节点分布式模拟。每个阶段都要保留可运行代码、正确性测试、性能报告和失败输入。

核心思想

本册实验不追求一次跑出最高吞吐，而是要求系统被拆成可验证的合同、可复现的测量和可解释的性能瓶颈。

A.1 统一交付格式

每个实验都必须提交五类材料。第一，环境记录：CPU 型号、逻辑 CPU、物理核心、SMT、NUMA、内存、编译器版本、编译选项、操作系统、是否容器或虚拟化环境。第二，输入记录：规模、分布、随机种子、错误样本、边界样本。第三，正确性记录：reference、checksum、输出规模、不变量和失败样例。第四，性能记录：运行命令、重复次数、原始数据、统计摘要和异常值处理。第五，结论边界：哪些机器、输入和参数下结论成立，哪些尚未验证。

验收与评分标准

实验报告不得只给一张耗时表。完整报告至少包含环境、输入族、命令、reference 对照、原始数据摘要、阶段报告、图表或表格、结论边界和下一步实验。若平台不支持 perf、NUMA 或亲和性，报告要写清限制，并用用户态 trace 或简化指标替代。

A.2 工程目录和交付闭环

实验若只散落成若干小程序，很快会失去可复查性。Compute Systems Engine 应从第一天就按职责分目录，让每一章新增的机制都有固定落点。目录不需要一开始很复杂，但必须能区分语义、内核、运行时、I/O、分布式模拟、测试和报告。这样后续从顺序 reference 走到故障注入时，不会把所有代码堆进一个 main.cpp。

Listing A.1 Compute Systems Engine 的建议目录形态

```
1 cse/
2   include/cse/
3     input/           # InputSplit, RecordId, generators
4     report/          # RunReport, StageReport, HotKernelReport
5     memory/          # MemoryShape, AccessTrace, BatchView
6     kernels/         # filter, histogram, reduce, topk, join
7     runtime/         # ThreadPoolPlan, WorkerSnapshot, QueueContract
8     io/              # ReliableWriter, ManifestEntry, Checkpoint
9     dist/            # MessageEnvelope, OwnershipRecord, MessageBus
10  src/
11    reference/        # simple sequential implementations
12    kernels/
13    runtime/
14    io/
```

```

15     dist/
16 tests/
17     correctness/
18     concurrency/
19     recovery/
20     performance_smoke/
21 bench/
22     cases/
23     runners/
24 tools/
25     generate_input
26     run_case
27     summarize_report
28 results/
29     yyyy-mm-dd-case-name/
30         env.txt
31         commands.txt
32         input.json
33         samples.csv
34         stages.csv
35         counters.txt
36         trace.jsonl
37         notes.md

```

这棵目录表达了一个边界：原理中的对象不是装饰名词，而是代码里的接口和报告字段。比如 `RecordId` 不能只出现在错误日志字符串里，它要被 `parser`、`diagnostics`、`manifest` 和 `recovery` 共同使用；`HotKernelReport` 不能只在测量章出现，它要被 `SIMD`、`layout`、`histogram` 和 `thread-local partial` 实验复用；`ManifestEntry` 不能只在分布式章节出现，它从可靠写阶段就要定义外部可见事实。

每次实验的最小文件包 每次实验都要能被后来重新打开。最小文件包包含六类文件。第一，`input.json` 写输入规模、seed、分布、坏行比例、冷热状态和 checksum。第二，`commands.txt` 保存构建命令、运行命令、环境变量和可选绑定策略。第三，`samples.csv` 保存每轮耗时、状态、checksum 和备注。第四，`stages.csv` 保存阶段时间、records in/out、bytes in/out、temporary bytes、state bytes。第五，`trace.jsonl` 保存任务、队列、I/O、message 和 recovery 的关键事件。第六，`notes.md` 只写解释、边界和下一步，不重复原始数据。

Listing A.2 一次实验结果必须能回答的问题

```

Can the input be regenerated?
Can the reference result be reproduced?
Can failed samples be separated from passed samples?
Can the timing boundary be located?
Can a later commit compare the same stage fields?
Can unavailable counters be distinguished from zero events?
Can a crash/retry case replay the same attempt sequence?

```

如果这些问题答不上来，实验即使跑出了数字，也不能支撑章节结论。可复现性不是附加要求，而是系统工程的一部分：没有输入身份，就没有正确性比较；没有阶段字段，就没有瓶颈迁移；没有 `trace`，就无法解释等待和重试；没有 `manifest`，就无法判断输出何时生效。

模块边界随章节逐步收紧 早期可以用简单实现，但接口边界要提前留好。第

一阶段的 parser 可以很朴素，但输出必须包含 `RecordId` 和 `checksum`；第二阶段的 batch 可以只用 `std::vector`，但聚合核只能接收 `BatchView`；第五阶段的队列可以先用 `mutex`，但必须暴露 `QueueContract`；第七阶段的可靠写可以先用本地文件，但必须通过 `ManifestEntry` 发布提交事实；第八阶段的消息总线可以在同进程内模拟，但必须带 `MessageEnvelope`、`deadline`、`attempt` 和 `delivery fault`。

阶段	可以先简单处理	不能省略的边界
顺序 reference	单线程、朴素容器、稳定排序	输入合同、错误摘要、checksum、RecordId
数据布局	vector 存热列，冷表朴素 map	BatchView、MemoryShape、冷路径回查
热内核	标量循环先行	HotKernelReport、反汇编摘要、输入族
线程同步	mutex 队列先行	QueueContract、关闭、drain、等待谓词
可靠写	本地临时文件	ManifestEntry、checksum、短写处理
分布式模拟	单进程消息队列	MessageEnvelope、AttemptId、重复和迟到消息

这张表说明“先做简单版本”和“偷掉边界”不是一回事。简单版本可以慢，可以减少功能，但不能让后续恢复、测量和并发语义无处接入。

A.3 统一实验协议

实验路线如果只写“做哪些实验”，仍然不够。真正能支撑系统判断的是协议：输入怎样生成，reference 怎样产生，计时边界在哪里，反汇编怎样保存，trace 怎样对齐，故障怎样重放，回归怎样比较。每次实验都应能被另一个时间、另一台机器、另一个提交重新打开，并判断结论是否仍成立。这里给出统一协议，后续阶段只是在它上面增加字段。

一、输入协议 输入不是一个文件名，而是一组可复现事实。每份输入至少要写入 `input.json`：生成器版本、随机种子、记录数、原始字节数、key 分布、坏行分布、行长分布、热点 key 比例、checksum 和 schema version。若输入来自真实文件，也要保存采样摘要和文件 checksum；不能只写“使用线上日志”。真实输入无法公开时，仍要用生成器构造同分布的可公开样本。

Listing A.3 input.json 的最小字段

```
generator_version
schema_version
seed
record_count
raw_bytes
key_distribution
line_length_distribution
malformed_record_policy
malformed_record_count
hot_key_ratio
input_checksum
expected_reference_checksum
```

输入族要覆盖三个层次。第一是语义边界：空输入、一条记录、缺字段、字段过多、整数溢出、超长行、最后一行无换行。第二是硬件边界：小工作集、跨 L1/L2/LLC、跨大量页、随机 key、长字符串、分支不稳定。第三是系统边界：热点 key、下游

变慢、短写、响应丢失、旧 epoch、driver 崩溃。一个实验只需要选择相关输入族，但报告必须说明为什么这些输入能攻击本章假设。

二、oracle 协议 Reference 生成期望结果，oracle 负责比较。不同实验的 oracle 不一定是字节完全相等。整数计数要求完全一致；浮点 reduce 要求误差合同；稳定 filter 要求顺序一致；partition 可以要求每个 bucket 内容和 manifest 一致；运行时实验可能要求所有 accepted task 进入终态；I/O 实验要求未提交 block 不被读取。Oracle 必须写在代码和报告里，不能只靠人工肉眼看输出。

实验类型	oracle 比较对象	不能只比较什么
整数聚合	counts、diagnostics、输出顺序或 checksum	不能只比较总记录数
浮点归约	误差界、NaN/Inf 策略、固定树形可重复性	不能只比较一次近似值
stable filter	输出记录 id 序列、records written、range checksum	不能只比较输出数量
任务运行时	task 终态、ready/run/block trace、unfinished 归零	不能只比较最终耗时
可靠写	manifest、block checksum、orphan 分类、恢复动作	不能只看文件存在
分布式模拟	accepted attempt、late/duplicate/stale 分类、final checksum	不能只看 worker 返回 Ok

Oracle 的作用是把错误缩小到某个边界。若优化版和 reference 的 counts 不同，先比较 parser 输出；parser 一致再比较 batch；batch 一致再比较 partial；partial 一致再查 manifest。每个阶段都要有 checksum 或可抽样比较字段，避免最终错误只能靠调试器从头跟。

三、测量协议 计时必须说明边界。冷启动计时包含 page fault、文件读取、缓存预热和可能的 I/O；热循环计时只测已经准备好的内存数据；端到端计时包含 parse、aggregate、write、commit 和 report。三者都可以有价值，但不能混用。每个 `samples.csv` 至少保存样本编号、模式、输入 checksum、开始时间、结束时间、状态、结果 checksum 和失败原因。异常样本不能随意删除，必须用状态字段标记。

Listing A.4 samples.csv 字段

```
sample_id,mode,input_checksum,threads,batch_size,queue_capacity,
start_ns,end_ns,status,result_checksum,error_kind,notes
```

统计摘要要写重复次数、预热策略、是否固定 CPU、是否关闭动态频率不可控因素、是否受容器或权限限制。若平台无法使用 PMU 计数器，报告不能把 cache miss 写成已测事实，只能写“由输入对照和阶段时间推断，仍需计数器验证”。严谨的弱结论比漂亮的假结论更有价值。

四、反汇编和源码证据协议 热循环优化必须保留机器证据。最小材料包括编译器、版本、优化选项、目标架构、符号名、关键循环反汇编摘要和源码提交号。报告不必贴满长反汇编，但要说明是否存在函数调用、边界检查、向量指令、lock 前缀、分支密度、load/store 形状。若编译器没有生成预期形态，性能解释要先回到编译结果。

Listing A.5 disasm.txt 摘要字段

```
compiler
compiler_flags
target_arch
```



```

source_commit
kernel_symbol
loop_instruction_summary
calls_in_hot_loop
branches_in_hot_loop
vector_width_if_any
locked_operations_if_any

```

这份证据把“我以为代码会这样执行”变成“机器确实大致这样执行”。例如一个数组归约如果热循环里仍有虚函数调用，不能直接讨论 SIMD；一个原子计数实验如果没有看到锁定读改写或等价同步形态，要先确认编译目标和代码路径；一个 parser 如果错误路径被内联进热循环，要先修取指形态再讨论分支预测。

五、trace 协议 并发、运行时、I/O 和分布式实验必须输出 trace。Trace 不是普通日志，它是可排序、可聚合的事件流。每条事件至少包含 `time_ns`、`thread_id`、`worker_id`、`task_id`、`attempt_id`、`event_kind`、`state_before`、`state_after` 和可选 `correlation_id`。事件名要稳定，不能把人类句子当作唯一结构。

Listing A.6 trace.jsonl 事件字段

```

time_ns
thread_id
worker_id
task_id
attempt_id
event_kind
state_before
state_after
queue_depth
correlation_id
payload_checksum

```

Trace 至少要能回答：任务何时 ready，何时开始执行，执行多久，是否阻塞，阻塞原因是什么，完成时发布了哪些后继，是否有 late reply，是否有 duplicate 被拒绝。若 trace 无法回答这些问题，就不能支撑“运行时调度合理”或“故障恢复正确”的结论。

六、故障注入协议 故障实验必须可重复。每个故障用规则描述，而不是靠随机崩溃。规则应包含触发点、目标对象、触发次数、动作和期望外部事实。例如：`write_once_short(block=S1,A0,bytes=128);drop_reply(task=S2,attempt=0,count=1);crash_driver(after=manifest_append,entry=S0)`。故障脚本保存到 `faults.json`，RunReport 记录实际触发次数。

Listing A.7 faults.json 的示例

```

[
  {
    "kind": "write_once_short",
    "target": "split:1 attempt:0",
    "bytes": 128,
    "expected": "block not committed before checksum"
  },
  {
    "kind": "drop_reply",

```

```
    "target": "split:2 attempt:0",
    "count": 1,
    "expected": "retry and accept exactly one attempt"
  }
]
```

随机压力测试可以补充，但不能替代确定性故障。确定性故障证明某条状态机边界正确，随机压力只帮助发现尚未想到的组合。发现随机失败后，应把它缩成新的确定性故障脚本，再进入回归集。

七、回归比较协议 每次优化都要和上一个稳定版本比较，不只比较耗时，还比较语义、资源和报告字段。回归比较至少包括：reference checksum 是否不变；diagnostics 是否不变；manifest 是否一致；阶段时间变化；峰值内存变化；队列高水位变化；新增或消失的 trace 事件；不可用指标是否仍被正确标记。性能变快但 diagnostics 变了，是错误；性能变慢但修复了重复提交，可能是正确交换。

Listing A.8 regression summary 字段

```
base_commit
candidate_commit
input_checksum
reference_checksum_equal
diagnostics_checksum_equal
manifest_equal
stage_time_delta
peak_memory_delta
queue_high_watermark_delta
new_faults_passed
known_unverified_boundaries
```

这份协议让实验结论能跨提交生存。没有回归比较，项目很容易在某次重构中悄悄丢掉错误摘要、提交去重或关闭语义，直到后面的大实验才发现。

A.4 章节到实验的证据矩阵

每章都要交出可复查证据。证据可以是手工账本、伪代码状态机、反汇编摘要、用户态 trace、阶段报告、故障脚本或运行目录，但不能只停留在概念解释。下面的矩阵把正文主题和工程实验连起来。它的目的不是增加形式负担，而是保证每个机制都有输入、oracle、测量和失败样例。

章节	实验切片	必留证据	攻击的失败
1	五行日志到 split/partial/manifest	输入合同、reference、RecordId、manifest 样例	结果定义含糊，重复 attempt 被合并
2	parser 热循环和分支输入族	反汇编摘要、短行/长行/坏行对照、stage time	把控制流波动误判成算法问题
3	AoS/SoA、stride、TLB 覆盖	MemoryShape、页覆盖估算、规模阶梯	胖对象搬运冷字段，随机页访问失控
4	线程放置、NUMA 首次触页、共享 line	affinity 记录、跨节点对照、coherence 现象	把放置问题误判成线程池问题

章节	实验切片	必留证据	攻击的失败
5	benchmark harness 和反例复盘	commands、samples、counter 可用性、bad benchmark	被优化掉或测错边界
6	HotBatch 和冷热回查	hot/cold bytes、BatchView checksum、错误回查	优化后 diagnostics 断链
7	scalar、auto-vectorized、mask 版本对照	编译报告、尾部输入、checksum、指令摘要	只测整向量长度，漏掉边界输入
8	histogram/top-k/join、reference、内核	reference、输入族、阶段报告、资源峰值	只在均匀输入上成立
9	线程创建、affinity、blocking 污染	WorkerSnapshot、ThreadTimeline、上下文切换或替代指标	ready work 有而 worker 不可用
10	有界队列、关闭协议和 futex 慢路径	QueueContract、wait trace、futex 返回码、drain/cancel 测试	等待谓词漏关闭，析构卡死
11	发布标志、CAS 状态、atomic wait	AtomicProtocolNote、成功/失败路径、发布字段	relaxed 计数被当作结果许可
12	SPSC/MPMC 和回收策略	线性化点、ABA 反例、lifecycle report	节点复用后读取旧对象
13	stable filter 和 multiway partition	count/scan/write 账本、range checksum、故障重试	输出位置靠抢，部分写被当有效
14	任务图、same-pool future 和 continuation	ready/run/block/help trace、关键路径估算、unfinished task 账本	worker 阻塞等待后继，线程池耗尽
15	可靠写、watermark、背压	short write 脚本、manifest、queue 水位、completion	文件存在被误当提交，队列无界增长
16	MessageBus、deadline、retry	request id、attempt、late reply、timeout 分类	超时后重复执行污染结果
17	partition、replication、epoch	OwnershipRecord、旧 owner 测试、manifest generation	route 变化后旧响应仍提交
18	端到端三 split 故障演练	RunReport、faults.json、recovery audit、最终 checksum	模块都能单独跑，组合后不可恢复

A.5 已落地的同步原语实验

当前配套代码已经把第 10 章的一部分同步原语合同落进 [books/cpu-volume-2/labs/compute_systems](https://books.google.com/books/cpu-volume-2/labs/compute_systems)。它不是完整运行时项目，但已经超过“只在正文出现 API 名称”的阶段。构建并运行：

Listing A.9 第二册同步实验的 CTest 入口

```
1 cmake -S books/cpu-volume-2 -B books/cpu-volume-2/build/compsys-debug \  
2   -DCMAKE_BUILD_TYPE=Debug  
3 cmake --build books/cpu-volume-2/build/compsys-debug  
4 ctest --test-dir books/cpu-volume-2/build/compsys-debug --output-on-failure
```

实验对象	代码入口	验证的同步事实
可关闭 bounded queue	BoundedMpmcQueue	阻塞 push/pop、close 唤醒、drain 不丢不重、等待计数
许可限制器	run_permit_limiter	counting_semaphore 限制 in-flight，不表达对象所有权
多阶段 barrier	run_phase_barrier	固定参与者每个 phase 都到达，completion 每轮只发生一次
future/promise	run_future_result_paths	value、exception、broken promise 三条路径分别可观测
atomic wait	run_atomic_wait_version_counter	等待单个版本号变化，不替代多字段谓词
eventfd/epoll	run_eventfd_epoll_contract	fd 只负责唤醒事件循环，payload 留在用户态队列
futex	run_futex_contract_probe	wait 返回原因、wake 数量和 ready 状态必须分开解释

这批实验的边界也要写清。它没有证明 futex 内核队列实现，没有给出锁竞争性能结论，也没有覆盖 coroutine scheduler、work stealing deque 或 io_uring completion 生命周期。它的作用是把“什么时候用 semaphore、什么时候用 condition variable、什么时候用 future、什么时候用 atomic wait、什么时候只把 eventfd/futex 当底层等待通道”变成可运行语义测试。新加入的 `compsys_futex_lab_probe` 补上了“用户态锁字、`FUTEX_WAIT_PRIVATE` 返回码、`FUTEX_WAKE_PRIVATE` 返回数量和 ready 状态重读”的最小合同。下一步实践卷应继续补 `threadpool_lab`、`lockfree_lab`、`io_lab` 和 `raft_lab`，并把 sanitizer、TSan、故障注入和性能回归接入同一套报告目录。

这张矩阵还提供排查顺序。若最终 checksum 错，先回到第 1 章的 reference 和 RecordId；若输入一致但 stage 慢，回到第 2 到第 8 章的机器证据；若并行后变慢或卡住，回到第 9 到第 14 章的等待和状态；若崩溃恢复错误，回到第 15 到第 18 章的 manifest、attempt、epoch 和 checkpoint。章节之间不是并列清单，而是同一条证据链的不同位置。

A.6 统一对象账本

从第一阶段开始，实验就使用同一套对象语言。这样第 18 章最终项目不是突然出现的新框架，而是前面实验逐步长出来的系统。

对象	含义	最早出现的实验
InputSplit	输入文件和字节范围的稳定身份	顺序 reference、输入生成器
HotKernelReport	单核热循环的输入族、反汇编、计数器和结论边界	核心流水线、分支预测、SIMD 前置实验
HotBatch	热字段列式批次和 record id 回溯	数据布局、SIMD、解析内核
TaskId/AttemptId	逻辑任务和物理执行尝试	线程池、任务图、故障重试
ThreadPoolPlan	worker 数、任务类别、队列容量和关闭策略	线程调度、阻塞隔离、线程池实验略
WorkerSnapshot/ThreadTimeline	worker 状态、任务时间线和等待点证据	线程调度、任务运行时、性能复盘
QueueContract	容量、谓词、关闭、drain、cancel 和背压语义	锁、条件变量、有界队列、I/O 队列
AtomicProtocolNote	atomic 变量用途、内存序、线性化点和生命周期说明	原子操作、发布协议、任务完成状态
LockFreeLifecycleReport	无锁结构的模型、线性化点、进展保证和回收策略	SPSC ring、ABA、hazard、epoch、RCU
ShuffleBlock	map 输出的可校验数据块	partition、I/O 写出、shuffle
ManifestEntry	外部可见提交事实	I/O commit、reduce 去重、恢复
MessageEnvelope	跨 worker 消息身份和状态	MessageBus、重试、迟到响应
OwnershipRecord	partition owner、epoch 和迁移状态	分片、热点 key、旧 owner 拒绝
Checkpoint	恢复时读取的控制面快照	I/O、分布式恢复、最终项目
RunReport	正确性、性能、故障和边界证据	每个阶段报告、毕业项目

A.7 阶段一：测量基础、热循环报告和顺序 reference

目标是给项目建立可信地基。实现顺序事件计数、顺序归约、输入生成器、checksum、阶段报告、benchmark harness 和 [HotKernelReport](#)。报告必须解释为什么 reference 可以慢，但必须简单、确定、可复查；也必须解释为什么一个热循环报告要同时保存输入族、编译选项、反汇编、计数器或替代证据，以及结论边界。

实验	能力目标	必交证据
环境记录	建立测量上下文	lscpu 、编译器版本、运行命令
顺序归约	固定语义和 checksum	reference 输出、边界输入、重复运行一致性
热循环报告	把单核瓶颈写成证据包	HotKernelReport 、反汇编摘要、计数器可用性
阶段报告	记录 records in/out 和状态大小	CSV 或文本报告、字段定义、样例输出
坏 benchmark 复盘	识别被优化掉、测错路径和噪声	错误代码、修复代码、反汇编或计时证据

A.8 阶段二：硬件成本和数据布局

目标是把数据放置、访问顺序和缓存/TLB 成本变成可测事实。实验从 AoS/SoA、顺序访问、随机访问、stride、TLB 覆盖和 false sharing 开始，再连接到事件流中的热字段列式布局。

实现 [HotEventBatch](#) 和 [ColdEventPayload](#) 两种布局。热路径只扫描 event_type、session_id 和时间戳列；冷字段保留在单独区域。报告比较对象数组、列式热字段和混合布局的 records/s、cache miss 或可替代指标、临时字节和代码复杂度。

A.9 阶段三：单机高性能内核

目标是把 map、filter、scan、partition、histogram、top-k、join、sort、stencil 和 graph frontier 做成 kernel suite。每个内核都要有 reference、优化版、输入族和报告。

内核	最小实现	压力输入
Stable filter	count、scan、write 三阶段	全通过、全不通过、低选择率、最后块不满
Thread-local histogram	每 worker partial, 最后合并	均匀 key、单热点、Zipf、稀疏 key
Top-k	精确聚合后选择, 稳定 tie-break	分散强 key、全 tie、候选不足
Join	小表 build, 大表 probe 或 sort-merge	重复 key、空匹配、输出爆炸
Graph frontier	CSR、vector frontier、bitmap frontier	规则图、随机图、幂律图

A.10 阶段四：SIMD、ILP 和 Roofline

目标是把单线程内核优化从经验变成模型。先写 scalar reference, 再尝试自动向量化, 阅读优化报告和汇编, 最后才决定是否手写 intrinsics 或保留 portable 版本。

必做实验包括多累加器归约、数组 map 融合、阈值过滤、ASCII 字节分类、简单 transpose 和 dot product。每个实验都要回答: 这个内核是 compute-bound 还是 memory-bound; SIMD 是否改变字节流量; 尾部和错误输入怎样处理; 数值重排是否允许。第 2 章的 [HotKernelReport](#) 在这里升级为 SIMD 版本对照: 同一输入族下必须同时保留 scalar reference、自动向量化版本、可选手写版本、反汇编摘要和误差或 checksum。

常见误区

SIMD 实验不能只测长度刚好是向量宽度倍数的输入。必须覆盖长度为 0、1、向量宽度减一、向量宽度、向量宽度加一、非对齐起点和大输入。

A.11 阶段五：线程、同步和内存模型

目标是把共享状态正确性放在性能之前。实验从线程池阻塞污染、mutex counter、atomic counter、condition variable queue 和 semaphore 开始, 再进入 memory order、false sharing、SPSC ring buffer、MPMC queue、ABA 和内存回收。本阶段的交付物不是“换成更高级同步原语”, 而是写出 [ThreadPoolPlan](#)、[QueueContract](#)、[AtomicProtocolNote](#) 和 [LockFreeLifecycleReport](#), 让每个等待点、发布点和回收点都可审查。

实验	必须证明	常见误区
Mutex counter	互斥保护不变量	把锁成本归因成线程数不足
Atomic counter	原子性不等于扩展性	以为 relaxed 会消除 cache line 争用
Condition queue	等待谓词和关闭语义正确	用 if 代替 while 等待
Memory order	release/acquire 建立发布关系	把 x86 现象写成 C++ 语义
Lock-free queue	线性化点和回收方案清楚	只写 CAS, 不处理 ABA 和生命周期

A.12 阶段六：并行算法和任务运行时

目标是从“开线程”进入“组织任务”。实现 reduce、scan、partition、thread-local histogram、任务粒度扫描、线程池、任务图、work stealing 指

标和 worker snapshot。

实验必须记录 task submit、enqueue、dequeue、start、finish、block begin、block end 和 worker state。没有内核 trace 权限时，用户态 trace 是最低要求。报告要能解释队列等待、worker 空闲、长尾任务、阻塞任务和上下文切换之间的关系。第 9 章的 [WorkerSnapshot/ThreadTimeline](#) 在这里升级为任务图 trace：线程池里看到的 blocked worker，要能映射到第 14 章的 ready count、continuation 和 work stealing 决策。

构造一个混合 CPU 任务和模拟阻塞 I/O 的线程池实验。比较单池、分池、增加 worker 和异步模拟四种方案。报告 p95 排队延迟、worker 空闲比例、队列最大长度、上下文切换和输出一致性。

A.13 阶段七：I/O、异步和背压

目标是把计算管线接到有限速率的上下游。实现有界队列、批处理、watermark、限流、背压传播、错误队列和 shutdown protocol。报告必须解释队列满时应该阻塞、丢弃、降级、重试还是反压上游。

事件流项目在本阶段加入 [event_time](#) 和 [ingest_time](#) 的区分。实验要构造乱序、迟到、突发、下游变慢和 checkpoint 失败输入。报告要记录 watermark 推进、迟到数据数量、窗口修正、重试次数和最终输出一致性。

A.14 阶段八：单节点分布式计算模拟

目标是在单机多进程或多线程模拟中引入网络、消息、故障和恢复。实现 [MessageBus](#)、[MessageEnvelope](#)、partition、[ShuffleBlock](#)、[ManifestEntry](#)、幂等提交、超时重试、[OwnershipRecord](#)、checkpoint、热点 key 和故障注入。这个阶段不追求完整分布式框架，而是把共享内存原则怎样扩展到消息系统讲清楚。

主题	实验要求	证据
Shuffle	分区、 ShuffleBlock 、checksum、manifest	block 数量、大小分布、热点 bucket
重试和幂等	重复请求不能重复提交	request id、attempt、commit log、重复消息测试
Checkpoint	失败后从一致边界恢复	checkpoint generation、manifest generation、恢复前后 checksum
旧 owner	epoch 切换后旧请求到达	OwnershipRecord 、StaleRoute、manifest 不变
热点 key	单个 reducer 被拖慢	key 分布、长尾时间、二级 reduce 对照

A.15 毕业项目

第二册毕业项目是一个可运行的 Compute Systems Engine。最低形态包括：输入生成器、顺序 reference、[InputSplit](#)、[HotKernelReport](#)、[HotBatch](#)、阶段报告、stable filter、thread-local histogram、partition manifest、top-k、[ThreadPoolPlan](#)、[WorkerSnapshot](#)、[ThreadTimeline](#)、[QueueContract](#)、用户态 trace、可靠写、[ManifestEntry](#)、背压策略和单节点 shuffle 模拟。高级形态再加入 SIMD 热内核、NUMA 初始化策略、[AtomicProtocolNote](#) 审查表、SPSC/MPMC 队列、[LockFreeLifecycleReport](#)、work stealing、[Checkpoint](#)、[MessageBus](#)、[OwnershipRecord](#)、热点 key 拆分、两进程部署和故障注入。

毕业故障	注入方式	验收重点
坏行	输入 split 中插入非法字段	reference 和优化路径错误摘要一致
短写	写 ShuffleBlock 时只完成部分字节	可靠写补齐，未完整不提交 manifest
响应丢失	drop 某个 attempt 的成功响应	retry 后 manifest 只接受一个 attempt
Driver 崩溃	manifest 前后分别终止	前者重做，后者承认已提交事实
旧 owner	route epoch 切换后旧请求到达	返回 StaleRoute, manifest generation 不变
热点 key	一个 key 占大多数输入	普通 hash 长尾，拆分后二级合并正确

验收与评分标准

毕业项目按 100 分评分。语义完整性 25 分，要求 reference、输入身份、错误合同、checksum 和失败输入完整；性能分析 20 分，要求实验矩阵、原始数据、阶段报告和瓶颈解释；系统设计 20 分，要求合同清楚、状态可观察、关闭和错误路径可靠；恢复和故障注入 20 分，要求短写、响应丢失、重复 attempt、Driver 崩溃、旧 epoch 和热点 key 都有确定性测试；源码阅读与工程质量 15 分，要求至少两组真实源码对照、C++ 接口清楚、位宽明确、测试可重复、报告可复现。

附录 B

源码阅读索引

本附录不是开源项目目录，而是第二册的源码阅读任务表。源码阅读必须从一个可验证问题进入：线程为什么迁移，futex 为什么进入内核，TLB miss 为什么上升，top-k 为什么漏报，shuffle 为什么出现热点。没有问题边界的源码阅读，很容易变成复制函数名和模块名。

核心思想

源码阅读报告要区分三类事实：代码中直接看到的控制流或数据结构，实验中直接测得的现象，以及从二者推导出的解释。不要把推测写成事实。

B.1 阅读方法

每次源码阅读只选择一个窄入口，最多沿两到三条调用路径展开。报告固定回答六个问题：入口文件和函数是什么；它对应本书哪个机制；关键状态对象是什么；正常路径和慢路径如何分开；错误、阻塞或回收边界在哪里；它如何解释本书实验中的一个现象。

验收与评分标准

源码报告评分重点不是读了多少文件，而是问题是否明确、路径是否收敛、图是否能说明状态变化、实验是否能验证解释。大段粘贴源码、泛泛复述项目架构或只列函数名，不计为高质量阅读。

B.2 工业案例报告模板

工业案例必须写成可迁移的设计分析，而不是项目介绍。每份报告固定包含七段：第一段写真实约束，例如吞吐、延迟、内存预算、故障恢复、确定性输出或多租户隔离。第二段写核心原理，例如列式 batch、work stealing、futex slow path、LSM compaction、shuffle manifest、hazard pointer 或 backpressure。第三段写关键数据结构和状态转换。第四段写 fast path 和 slow path 的分界。第五段写设计取舍：它牺牲什么，换来什么。第六段写和本书实验的对应关系。第七段写一个最小复现实验或伪代码接口。

常见误区

工业案例不是权威背书。不能因为 DuckDB、ClickHouse、Spark、Kafka、Redis、RocksDB、oneTBB 或 Folly 使用了某种设计，就直接说它适合所有场景。必须说明该设计的输入假设、硬件环境、并发模型、失败模型和不可迁移之处。

B.3 Linux 内核

主题	建议入口	阅读任务
调度与迁移	kernel/sched/core.c 、 kernel/sched/fair.c	解释 runnable 、 running 、 wakeup 、 migration 和 context switch 怎样对应用户态线程池指标。
亲和性和拓扑	kernel/sched/ 、 /sys/devices/system/cpu/ 相关文档路径	解释 CPU affinity 成功或失败时，用户态实验为什么会变化。
缺页和页表	mm/memory.c 、 mm/mmap.c	把 page fault 、 mmap 、 VMA 和 TLB/page walk 实验连接起来。
透明大页和 NUMA	mm/huge_memory.c 、 mm/mempolicy.c	解释 huge page 、 first touch 、 NUMA placement 对吞吐曲线的影响。
perf 事件	kernel/events/	解释 perfstat 事件从用户态请求到内核计数器配置的大致路径。
PMU Top-Down	tools/perf/ 、架构事件文档	把 retiring 、 frontend bound 、 backend bound 和 bad speculation 映射到 HotKernelReport 的证据边界。
futex	kernel/futex/ 或对应版本路径	解释 mutex/condition variable 从用户态 fast path 进入内核等待的 slow path 。
epoll 和 I/O 等待	fs/eventpoll.c	解释 readiness 、等待队列和异步 I/O 实验的状态变化。
排序和集合辅助	lib/sort.c 、 bitmap/hashtable 相关入口	对照第 8 章的 sort 、 bitmap frontier 和集合表示实验。

B.4 C/C++ 运行时和标准库实现

[glibc](#) 和 [musl](#) 适合阅读 [pthread](#)、[mutex](#)、[condition variable](#)、[malloc](#) 和系统调用封装。阅读重点是 [fast path](#) 和 [slow path](#) 的分界：无竞争时是否只走用户态原子操作，竞争时何时进入 [futex](#)，等待谓词和唤醒怎样组织，[malloc](#) 怎样处理线程本地缓存和全局 [arena](#)。

[libstdc++](#)、[libc++](#) 和 [MSVC STL](#) 的容器与算法实现适合对照标准库语义。阅读 [std::sort](#)、[std::stable_sort](#)、[std::vector](#) 扩容、并行算法实现时，要把接口合同、异常安全、迭代器失效和临时空间写进报告。不要只看最终用了哪种算法。

B.5 编译器和运行时

[LLVM](#) 适合阅读自动向量化、循环优化、[OpenMP runtime](#) 和 [task runtime](#)。建议入口包括 [LoopVectorize](#)、[SLP vectorizer](#)、[OpenMP runtime](#) 的 [task](#) 和 [thread](#) 管理路径。阅读目标是解释为什么某个循环不能向量化，为什么某个任务粒度导致 [runtime overhead](#)，或者为什么 [reduction](#) 需要特殊语义。第 2 章的阅读报告应从 [HotKernelReport](#) 进入：先给出循环源码、优化报告和反汇编，再说明拒绝向量化或内联失败的直接理由。

[oneTBB](#)、[LLVM OpenMP runtime](#) 或其他成熟任务运行时适合阅读线程池、任务队列、工作窃取和关闭协议。报告重点是任务状态、队列结构、[work stealing](#) 触发

条件、阻塞任务处理、异常传播和 shutdown。只画一个 worker 循环不够，要说明任务生命周期。

本书对象	建议阅读方向	报告必须回答
HotKernelReport	Linux perf、PMU Top-Down、LLVM 优化报告、数据库执行器 profile	输入族、二进制形状、计数器可用性和瓶颈分类如何形成一条可复查证据链
ThreadPoolPlan	oneTBB arena、OpenMP runtime、Folly executor、Java ForkJoinPool	worker 数、任务类别、阻塞任务和 shutdown 如何进入接口合同
WorkerSnapshot/ ThreadTimeline	Linux sched_switch trace、executor metrics、Spark/Flink task metrics	running、blocked、idle、steal、queue wait 怎样成为可解释指标
QueueContract	pthread condition variable、Linux futex、blocking queue、Kafka/Redis 队列边界	等待谓词、close、drain、cancel 和背压怎样定义

B.6 并发库和无锁结构

Folly、Abseil、Boost.Lockfree、Concurrency Kit 或类似项目适合阅读并发容器、原子封装、hazard pointer、epoch reclamation、RCU 风格回收和高性能队列。阅读时先写清进度保证：wait-free、lock-free、obstruction-free 还是 blocking。然后再找线性化点、ABA 防护、内存序和内存回收边界。

常见误区

无锁源码阅读最常见的错误，是只盯着 CAS 循环。真正难点通常在线性化点、对象生命周期、内存回收和失败重试成本。报告若没有说明被删除节点何时可以释放，就没有读懂无锁结构。

本书对象	建议阅读方向	报告必须回答
AtomicProtocolNote	C++ std::atomic 示例、Folly/Abseil 原子封装、Linux refcount	变量用途、发布数据、内存序、线性化点、失败路径和生命周期是什么
LockFree LifecycleReport	SPSC/MPMC 队列、Treiber stack、hazard pointer、epoch、RCU	生产者消费者模型、进展保证、ABA 防护、回收策略、关闭外置方式和测试矩阵是什么

B.7 HPC 和 AI 计算库

BLIS、OpenBLAS 和 oneDNN 适合阅读 blocking、packing、microkernel、ISA dispatch、primitive cache 和 shape specialization。第二册只要求读懂高性能内核的工程组织，不要求提前进入第三册的完整 AI 推理引擎。报告应回答：reference 在哪里，packing 怎样改变内存访问，microkernel 的 MR/NR 或 tile 形状怎样服务寄存器和 cache，fallback 怎样选择。若用作第 2 章或第 7 章案例，还要把阅读结果写成 HotKernelReport 的字段：输入形状、热循环摘要、指令级并行来源和不可迁移的 ISA 假设。

XNNPACK、llama.cpp CPU backend 或类似项目可以作为第三册前置阅读。第二册阶段只关注线程池、任务切分、数据布局 and 调度，不提前深挖量化算子细节。这样可以把 CPU 系统能力和 AI kernel 能力分开建立。

B.8 数据库、存储和分布式系统

Redis、RocksDB、Kafka、Spark、Flink、etcd/Raft、DuckDB 或 ClickHouse 适合阅读事件循环、后台线程、LSM compaction、分区、复制、shuffle、背压、vectorized execution、checkpoint 和故障恢复。选择项目时要跟章节问题匹配：第 2 章读向量化执行器或解析器 hot path，重点是 batch 如何减少每行分派并改善前端供给；第 8 章读 hash aggregate、sort aggregate 或 top-k；第 15 章读 I/O 和背压；第 16 章读 deadline、request id、幂等和 retry；第 17 章

读 partition owner、replica progress、leader term 或 quorum；第 18 章读 shuffle manifest、checkpoint、output commit 或 RunReport 形态。

章节主题	可选源码方向	最小报告产出
典型内核	DuckDB/ClickHouse	一张数据流图、一组合同、一组输入反例
异步与背压	aggregate、top-k、sort Redis event loop、Kafka producer/consumer、 RocksDB compaction	等待点、队列边界、背压传播图
消息和幂等	gRPC deadline/status、 Kafka producer id、Spark task attempt	request、attempt、deadline、重复请求处理
分区和复制	Kafka partition、 etcd/Raft、 Cassandra/RocksDB 相关路径	owner、epoch、replica progress、commit index
分布式 shuffle	Spark shuffle、Flink checkpoint、RocksDB checkpoint	manifest、幂等、checkpoint、失败恢复路径
任务运行时	oneTBB、OpenMP runtime、 Folly executor	任务状态机、worker 指标、关闭协议

B.9 第十六至十八章的专项阅读

最后三章要求源码阅读直接服务 Compute Systems Engine 的最终对象。报告不应写“某系统支持容错”这种泛泛描述，而要把对象映射清楚：本书的 `MessageEnvelope` 对应哪个 request metadata，本书的 `ManifestEntry` 对应哪个 output commit 事实，本书的 `OwnershipRecord` 对应哪个 partition owner 或 leader term，本书的 `Checkpoint` 对应哪个恢复边界。

本书对象	建议阅读路径	报告必须回答
<code>MessageEnvelope</code>	gRPC deadline/status、Kafka producer metadata、Spark RPC/task message	timeout 证明什么，request id/attempt 如何进入状态机
<code>ManifestEntry</code>	Spark output commit、MapReduce committer、Flink sink commit	worker 输出何时从候选变成外部可见事实
<code>OwnershipRecord</code>	Kafka partition leader、etcd term/index、Cassandra token range	旧 owner 或旧 leader 为什么不能继续提交
<code>Checkpoint</code>	Flink checkpoint、RocksDB checkpoint、etcd snapshot	snapshot 覆盖到哪个 index/offset，恢复时谁是权威
<code>RunReport</code>	DuckDB/ClickHouse profile、Spark UI/event log、Flink metrics	报告如何帮助定位慢 stage、重复 attempt 或恢复动作
<code>WorkerSnapshot</code>	oneTBB/OpenMP runtime metrics、Spark executor metrics、Linux scheduler trace	worker 空闲、阻塞、运行、窃取和队列等待如何解释吞吐或尾延迟
<code>AtomicProtocolNote</code>	Linux refcount/RCU、Folly atomics、C++ atomic shared pointer	单机状态发布、引用计数和对象回收如何迁移到分布式提交状态

常见误区

第十六至十八章的源码阅读必须从一个事故进入，例如“响应丢失后两个 attempt 都提交”“旧 owner 在新 epoch 后继续写”“Driver 在 manifest 后崩溃”。如果报告不能把源码路径映射回这个事故，它就没有完成本册要求。

B.10 版本和可复现性

源码阅读必须记录项目、版本、commit、编译选项和阅读日期。现代开源项目变化很快，不能把某个 commit 的实现细节写成永久事实。若本地无法构建项目，可以做静态阅读，但报告必须标注“未运行验证”。若能运行最小实验，应把输入、命令、输出和观察结果附到报告中。

实践卷：并发、运行时和系统合同

第二册实践卷分两层。第一层是本册已有实验：

```
1 books/cpu-volume-2/labs/compute_systems
```

第二层是配套代码实践卷：

```
1 books/compute-systems-engine-code/labs/reference_pipeline
```

本附录先把第一层并发实验拆成阶段。并发代码不能只看最终数字；它必须证明没有丢任务、重复执行、等待泄漏、关闭遗漏或把失败样本误当正常吞吐。

C.1 零、贯穿主线工程

第二册主线工程是 `compute_systems` 运行时实验，再向 `reference_pipeline` 代码实践卷扩展。第一层先把每章模型、单机并发和运行时合同做实：逐章 `chapter models`、串行 `reference`、并行归约、计数器、有界队列、`wait channel`、`TaskRuntime`、`futex probe`。第二层再把输入身份、`manifest`、`split`、`merge` 和恢复报告接上。不要把这些看成散装 demo；它们共同回答一个问题：一个计算系统怎样证明任务没有丢、没有重、没有卡住，并且能发布可复查报告。

阶段	主线代码	学到的工程能力
逐章模型	<code>chapter models</code>	每章抽象概念都先落到可运行报告结构
Reference 与 切片	并行归约	串行结果是并行优化的可信尺子
同步基础	同步模块	<code>mutex</code> 、 <code>atomic</code> 、队列和 <code>close</code> 都有可观察合同
等待通道	<code>wait channel</code>	把唤醒事件和数据 <code>drain</code> 分开验证
任务运行时	<code>TaskRuntime</code>	<code>submitted</code> 、 <code>completed</code> 、 <code>failed</code> 、 <code>rejected</code> 和 CSV 报告可复查
内核等待边界	<code>futex probe</code>	用户态快路径、内核慢路径和错误返回都要被测试
恢复扩展	代码实践卷	进入输入身份、 <code>manifest</code> 、 <code>split</code> 、 <code>merge</code> 和恢复报告

C.2 一、工程入口

本册实验的主要文件如下：

```
1 include/compsys/parallel_reduce.hpp
2 include/compsys/chapter_models.hpp
3 include/compsys/sync_primitives.hpp
4 include/compsys/task_runtime.hpp
5 include/compsys/wait_channels.hpp
6 include/compsys/futex_lab.hpp
```

```

7 tests/compsys_tests.cpp
8 src/chapter_models_probe.cpp
9 src/wait_channels_probe.cpp
10 src/task_runtime_probe.cpp
11 src/futex_lab_probe.cpp

```

构建和测试命令如下：

Listing C.1 第二册 compute systems 实验构建和测试

```

1 cmake -S books/cpu-volume-2 -B books/cpu-volume-2/build/compsys-debug -↵
   ↪ DCMMAKE_BUILD_TYPE=Debug
2 cmake --build books/cpu-volume-2/build/compsys-debug
3 ctest --test-dir books/cpu-volume-2/build/compsys-debug --output-on-failure

```

`compsys_chapter_models_probe` 是每章模型的人工入口。它不是替代测试，而是把 21 个章节/深入章节模型的报告字段按稳定顺序打印出来：

Listing C.2 运行第二册逐章模型探针

```

1 books/cpu-volume-2/build/compsys-debug/labs/compute_systems/↵
   ↪ compsys_chapter_models_probe

```

输出第一行是 `chapter_models,21`。后续行分别对应第 1 章到第 18 章，以及第 5b、9b、12b 三个深入章。读者修改任一模型时，至少要同时更新三处：模型函数、`compsys_tests.cpp` 中的断言、这个 probe 的输出字段。

C.3 二、阶段 0：并行归约先对照串行 reference

`sequential_sum` 是 reference, `parallel_sum` 是优化版。reference 是可信但不追求快的版本；并行版必须和它对同一份输入返回同一结果。

最小推导如下：把数组切成若干连续块，每个 worker 求一段局部和，最后主线程把局部和相加。若每个元素恰好属于一个块，且最终汇总不漏块、不重复块，那么并行结果等于串行结果。

失败样本也要写进报告：worker 数大于元素数时，不能生成空范围越界；worker 数为 1 时，应等价于串行；包含负数时，不能只测试自然数递增数组；空输入的语义必须明确。测试 `test_parallel_sum` 用含负数的固定数组对照 `sequential_sum` 和 `parallel_sum`。

C.4 三、阶段 1：计数器和同步原语

`MutexCounter` 和 `AtomicCounter` 都实现 `add/value`，但它们的保护方式不同。`mutex` 是互斥锁，一次只允许一个线程进入临界区；`atomic` 是原子变量，单个读改写操作不能被其他线程切开。

实践时不要只说“线程安全”。要写一个多线程样本：4 个线程，每个线程加 250 次，期望最终值是 1000。若少于 1000，说明更新丢失；若 `value` 读取没有同步，可能读到不一致状态。测试 `test_counters` 正是这条验收。

C.5 四、阶段 2：有限队列和 backpressure

`BoundedMpmcQueue` 是有容量上限的多生产者多消费者队列。MPMC 表示 multiple producers, multiple consumers。backpressure 是下游处理慢时，上游被容量限制住；它不是“队列满了”一句话，而是一个可观察合同：生产者等待次数、等待时间、关闭后返回值都要能报告。

队列实践按四步写：

1. 单线程 `push/try_pop`：验证 FIFO 顺序、`size` 和 `max_size`。
2. 消费者空队列阻塞后 `close`：验证 `pop` 返回空、`pop_closed` 增加、等待时间被记录。
3. 生产者满队列阻塞后 `close`：验证第二次 `push` 返回 `false`、`push_closed` 增加。
4. 多生产者多消费者 `drain`：容量为 1 时仍然不能丢值、重值或越界。

这些测试对应 `test_queue`、`test_queue_close_wakes_consumer`、`test_queue_close_wakes_blocked_producer` 和 `test_queue_multi_producer_consumer_drain`。如果报告只写“push/pop 正常”，没有证明 `close` 唤醒，队列合同是不完整的。

C.6 五、阶段 3：permit、barrier、future 和 atomic wait

`run_permit_limiter` 训练“许可证”概念：最多允许 `permit_count` 个任务同时进入关键区域。测试要检查 `max_in_flight ≤ permit_count`，否则限流只是装饰。

`run_phase_barrier` 训练 `barrier`。`barrier` 是阶段边界：所有 `worker` 到达同一阶段后，才能一起进入下一阶段。报告中的 `arrival_count` 应等于 `worker_count * phase_count`，否则有人没到或被重复统计。

`run_future_result_paths` 训练 `future` 的三种结果：正常值、异常、`broken promise`。`broken promise` 指承诺方消失或没有设置结果，等待方不能永远挂住。

`run_atomic_wait_version_counter` 训练版本号等待。`version` 是单调增长的状态编号；等待者看到版本变化才继续。这里的 `wake_count` 和 `observed_sum` 证明等待路径真的被走到，而不是只靠忙等碰巧结束。

C.7 六、阶段 4：wait channel 和 eventfd/epoll

`wait channel` 是“等待者、唤醒条件和唤醒通道”的组合。只说“线程睡眠”不够，因为系统程序真正关心的是谁能唤醒、唤醒后读到什么、队列是否被完整 `drain`。

`run_eventfd_epoll_contract` 检查两类事实：`eventfd` 计数被读到几次，队列和信号量语义是否被 `drain`。报告字段 `epoll_ready_observations`、`epoll_empty_observations`、`queue_fifo_ok` 和 `semaphore_empty_after_reads` 都是合同的一部分。若只检查 `epoll` 返回“可读”，就还没证明数据结构本身正确。

C.8 七、阶段 5：任务运行时和同池等待事故

`TaskRuntime` 的报告字段包括 `submitted`、`completed`、`failed`、`rejected`、`unfinished`、`max queue depth`、`active workers` 和 `accepting/stopping` 状态。实践时要覆盖三类场景：正常完成、任务抛异常、`shutdown` 后提交被拒绝。

同池 `future wait` 是一个典型事故：`worker` 执行父任务，父任务又提交子任务并在同一个 `worker` 池中等待子任务；如果所有 `worker` 都被父任务占住，子任务没有 `worker` 可运行，就会饥饿。函数 `run_same_pool_future_wait_probe` 用报告字段描述这个停滞点，`run_continuation_runtime_probe` 则展示用 `continuation` 拆开等待边界后的路径。

这类实验的结论不能写成“`future` 不好”或“线程池不好”。正确结论是：同一执行池里阻塞等待自己提交的后续任务，会把 `worker` 资源耗尽；解决办法是改变调度结构、拆 `continuation`，或使用独立等待资源。

C.9 八、阶段 6：futex 快路径和慢路径

`futex` 的核心边界是用户态快路径和内核等待路径。无竞争时，锁应主要在用户态完成；有竞争时，等待者进入 `futex wait`，释放者 `futex wake`。报告中的 `contended_count`、`wait_count` 和 `wake_count` 用来证明慢路径真的发生过。

`run_futex_contract_probe` 还要覆盖三个容易被忽略的路径：值不匹配返回 `EAGAIN`，超时返回 `timeout`，被信号打断返回 `EINTR`。如果只测“最后计数正确”，就没有证明 `futex` 合同完整。

C.10 九、和代码实践卷衔接

当实验开始需要输入身份、诊断、`manifest`、`checkpoint` 或恢复报告时，应转入 `books/compute-systems-engine-code/labs/reference_pipeline`。本册原理解释机制，代码实践卷负责把机制做成可复查工程。连接规则很简单：原理卷中的对象如果已经有 C++ 类型，就引用类型名；如果还没有类型，就不能把它当作已实现能力。

阶段	代码切片	必须回答的问题
0	<code>sequential_sum/parallel_sum</code>	每个元素是否恰好参与一次归约
1	<code>MutexCounter/AtomicCounter</code>	多线程更新是否丢失
2	<code>BoundedMpmcQueue</code>	满队列、空队列和 <code>close</code> 是否都能唤醒
3	<code>permit</code> 等同步对象	失败路径是否都被测试
4	<code>eventfd</code> 与 <code>epoll</code>	唤醒后数据是否按合同 <code>drain</code>
5	<code>TaskRuntime</code> probes	是否有任务丢失、失败、拒绝或饥饿
6	<code>TaskRuntime</code> CSV	报告字段是否能稳定导出和复查
7	<code>futex</code> probes	快路径、慢路径和错误返回是否都可观察

大作业阶段 6 已经有入口：`write_task_runtime_report_csv` 能把 `TaskRuntime::Report` 导出为稳定 CSV。报告不能只测 happy path；至少要用正常完成、任务抛异常、关停后提交被拒绝、同池等待饥饿四类情况解释字段如何变化。

C.11 十、每章代码任务矩阵

第二册章节多、跨度大，但代码主线必须连续：先证明单机计算和同步合同，再进入运行时、异步 I/O 和分布式边界。每章读完都要把一个字段、一个状态或一个失败路径写成代码。

章节	已给出的完整模型代码	验收证据
计算系统地图	<code>model_ch01_input_pipeline</code>	输入总数、 <code>accepted</code> 、 <code>rejected</code> 和 <code>checksum</code> 都由记录值推导出来
核心、流水线、分支	<code>model_ch02_branch_predictor</code>	交替输入下 <code>actual taken</code> 和 <code>misprediction</code> 不再只停留在概念
Cache、TLB、虚拟内存	<code>model_ch03_cache_tlb</code>	同一组 <code>byte offset</code> 同时推出 <code>cache line</code> 、 <code>page</code> 和 <code>TLB miss</code>
NUMA 与一致性	<code>model_ch04_numa_coherence</code>	<code>local</code> 、 <code>remote</code> 和 <code>coherence transfer</code> 分开计数
测量、Roofline、计数器	<code>model_ch05_roofline</code>	<code>arithmetic intensity</code> 、 <code>memory roof</code> 和 <code>attainable GFLOPS</code> 可断言

PMU 与 SIMD 证据	model_ch05b_pmu_simd_evidence	counter 可用性、retired instruction 和 vector fraction 同时进入报告
布局、局部性、prefetch	model_ch06_layout_locality	AoS/SoA 触碰字节数和 saved bytes 由字段数推导
SIMD 与 ILP	model_ch07_simd_ilp	vector iterations、scalar tail 和 lane utilization 可复查
计算 kernel 模式	model_ch08_map_filter_reduce	filter 后的 mapped sum 与 reduced sum 一致
线程、调度、亲和性	model_ch09_thread_scheduling	worker 负载分配能暴露 imbalance
Linux 调度与内存管理	model_ch09b_linux_wait_wake	wait loop 区分 spurious wakeup 和真正完成
锁、条件变量、信号量	model_ch10_sync_backpressure	生产、消费、容量和 backpressure event 都可观察
原子与内存序	model_ch11_atomic_publication	stale read 和 monotonic observation 分开
无锁结构	model_ch12_spsc_ring	push、pop、full rejection 和 empty pop 都有字段
无锁证明与回收	model_ch12b_epoch_reclamation	protected node 延迟回收，未保护 node 才释放
reduce、scan、partition	model_ch13_reduce_scan_partition	reduce sum、last prefix 和 partition true count 来自同一输入
任务运行时	model_ch14_work_stealing	初始队列不均衡时 stolen task 和 completed task 可见
异步 I/O 与 backpressure	model_ch15_async_io_backpressure	submitted、completed、max in-flight 和背压事件同时验收
分布式模型	model_ch16_distributed_attempts	logical task、duplicate attempt 和 stale generation 分开
分区、复制、共识	model_ch17_quorum_commit	quorum size 和 committed 状态由 ack 集合推出
分布式工程边界	model_ch18_checkpoint_recovery	checkpoint 数、replay 起点和 replay 数量明确

这些函数都在 `include/compsys/chapter_models.hpp` 声明，在 `src/chapter_models.cpp` 实现，在 `tests/compsys_tests.cpp` 断言。完整源码见下一附录。这里的关键不是函数规模多大，而是每章至少有一个从输入到报告的闭合模型：输入是什么，状态怎样变化，哪些字段证明结论，错误输入怎样拒绝，都必须能被 C++ 代码说明。

C.12 十一、递进大作业：Compute Systems Runtime

阶段	交付物	通过标准
0	逐章模型	21 个 <code>model_ch*</code> 函数都能构建、测试并由 probe 打印
1	串行 reference	并行优化必须先和它对照

2	并行归约	数据切片不丢、不重、不越界
3	同步基础	mutex、atomic、condition、semaphore 各有失败路径
4	有界队列	backpressure、close、drain 和报告字段可复查
5	wait channel	eventfd/epoll 的唤醒和数据 drain 分开验证
6	TaskRuntime	正常、异常、拒绝、饥饿都能改变报告字段
7	CSV 发布	<code>write_task_runtime_report_csv</code> 固定表头和字段顺序
8	分布式衔接	需要 manifest 或 checkpoint 时 接入代码实践卷

附录 D

完整源码清单：Compute Systems 并发实验

本附录完整收录 [books/cpu-volume-2/labs/compute_systems](https://books.cpu-volume-2.com/labs/compute_systems)。第二册主线工程先用 chapter models 给每章建立可运行模型，再从串行 reference、并行归约、同步原语、有界队列、wait channel、任务运行时一路推进到 futex 合同。阅读时不要跳过测试：系统代码最重要的证据往往在失败路径测试里。

D.1 一、CMakeLists.txt

CMake 文件把库、命令行 probes 和测试目标接在一起。读者应看清每个源文件进入哪个 target，以及 Linux 平台相关代码如何被组织。

Listing D.1 compute_systems/CMakeLists.txt

```
1 add_library(compsys STATIC
2     src/chapter_models.cpp
3     src/parallel_reduce.cpp
4     src/futex_lab.cpp
5     src/sync_primitives.cpp
6     src/task_runtime.cpp
7     src/wait_channels.cpp
8 )
9
10 target_include_directories(compsys
11     PUBLIC
12     ${CMAKE_CURRENT_SOURCE_DIR}/include
13 )
14
15 target_compile_features(compsys PUBLIC cxx_std_20)
16
17 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
18     target_compile_options(compsys PRIVATE -Wall -Wextra -Wpedantic)
19 endif()
20
21 add_executable(compsys_demo
22     src/main.cpp
23 )
24
25 target_link_libraries(compsys_demo PRIVATE compsys)
26
27 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
28     target_compile_options(compsys_demo PRIVATE -Wall -Wextra -Wpedantic)
29 endif()
30
31 add_executable(compsys_tests
32     tests/compsys_tests.cpp
33 )
```

```

34
35 target_link_libraries(compsys_tests PRIVATE compsys)
36
37 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
38     target_compile_options(compsys_tests PRIVATE -Wall -Wextra -Wpedantic)
39 endif()
40
41 add_test(NAME compsys_tests COMMAND ${TARGET_FILE:compsys_tests})
42
43 add_executable(compsys_chapter_models_probe
44     src/chapter_models_probe.cpp
45 )
46
47 target_link_libraries(compsys_chapter_models_probe PRIVATE compsys)
48
49 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
50     target_compile_options(compsys_chapter_models_probe PRIVATE -Wall -Wextra -↵
↵ Wpedantic)
51 endif()
52
53 add_test(NAME compsys_chapter_models_probe COMMAND ${TARGET_FILE:↵
↵ compsys_chapter_models_probe})
54 set_tests_properties(compsys_chapter_models_probe PROPERTIES
55     PASS_REGULAR_EXPRESSION "chapter_models,21"
56 )
57
58 add_executable(compsys_wait_channels_probe
59     src/wait_channels_probe.cpp
60 )
61
62 target_link_libraries(compsys_wait_channels_probe PRIVATE compsys)
63
64 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
65     target_compile_options(compsys_wait_channels_probe PRIVATE -Wall -Wextra -↵
↵ Wpedantic)
66 endif()
67
68 add_test(NAME compsys_wait_channels_probe COMMAND ${TARGET_FILE:↵
↵ compsys_wait_channels_probe})
69 set_tests_properties(compsys_wait_channels_probe PROPERTIES
70     PASS_REGULAR_EXPRESSION "contract,2,1,1,2,2,3,3,1,1"
71 )
72
73 add_executable(compsys_task_runtime_probe
74     src/task_runtime_probe.cpp
75 )
76
77 target_link_libraries(compsys_task_runtime_probe PRIVATE compsys)
78
79 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
80     target_compile_options(compsys_task_runtime_probe PRIVATE -Wall -Wextra -↵
↵ Wpedantic)

```



```

81 endif()
82
83 add_test(NAME compsys_task_runtime_probe COMMAND $<TARGET_FILE:↵
    ↵ compsys_task_runtime_probe>)
84 set_tests_properties(compsys_task_runtime_probe PROPERTIES
85     PASS_REGULAR_EXPRESSION "task_runtime,3,3,3,0,1,5,5,15,"
86 )
87
88 add_executable(compsys_futex_lab_probe
89     src/futex_lab_probe.cpp
90 )
91
92 target_link_libraries(compsys_futex_lab_probe PRIVATE compsys)
93
94 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
95     target_compile_options(compsys_futex_lab_probe PRIVATE -Wall -Wextra -↵
    ↵ Wpedantic)
96 endif()
97
98 add_test(NAME compsys_futex_lab_probe COMMAND $<TARGET_FILE:↵
    ↵ compsys_futex_lab_probe>)
99 set_tests_properties(compsys_futex_lab_probe PROPERTIES
100     PASS_REGULAR_EXPRESSION "futex_mutex,"
101 )

```

D.2 二、README

README 给出实验范围和运行命令。它说明本目录是第二册原理卷的本地实验层，完整工程化恢复和 manifest 逻辑会继续衔接代码实践卷。

Listing D.2 compute_systems/README.md

```

1 # Compute Systems Engine 早期实验素材
2
3 这是第二册早期计算系统实验素材，保留顺序归约、并行归约、计数器和有界队列等基础↵
    ↵ 代码。第二册正式正文现在作为原理卷维护；完整 C++20 工程实现、测试、↵
    ↵ benchmark、故障注入和报告会进入配套代码实践卷：
4
5 ```text
6 books/compute-systems-engine-code/
7 ```
8
9 当前实验覆盖：
10
11 - 逐章可运行模型：`chapter_models.hpp/.cpp` 为第二册每个主章和深入章给出最小完↵
    ↵ 整模型函数
12 - 顺序归约和并行归约
13 - 线程切分和局部累加
14 - mutex 计数器和 atomic 计数器
15 - 可关闭 bounded MPMC 队列：阻塞 `push`、阻塞 `pop`、`close`、drain 和等待报告
16 - 同步原语语义切片：`counting_semaphore` 许可限制、`barrier` 多阶段同步、↵
    ↵ `future/promise` 值/异常/broken-promise、`atomic::wait` 版本发布

```

```

17 - `eventfd`/`epoll` 唤醒合同：用户态队列、单次唤醒、drain 和 semaphore 模式
18 - futex 合同探针：用户态锁字、`FUTEX_WAIT_PRIVATE` 返回码、`FUTEX_WAKE_PRIVATE`↵
    ↪ 唤醒数量和 ready 状态重读
19 - 最小 task runtime: same-pool future 等待风险、ready 子任务饥饿和 continuation↵
    ↪ 修复路径
20 - 章节模型探针：`compsys_chapter_models_probe` 输出每章模型的稳定摘要
21 - CTest 正确性测试
22
23 逐章模型不是性能 benchmark，而是把每章的抽象名词落到一个能编译、能断言、能打印↵
    ↪ 报告的最小 C++20 函数。当前映射如下：
24
25 | 章节 | 模型函数 | 固定下来的问题 |
26 | --- | --- | --- |
27 | 第 1 章 | `model_ch01_input_pipeline` | 输入记录怎样被接受、拒绝并进入 ↵
    ↪ checksum |
28 | 第 2 章 | `model_ch02_branch_predictor` | 分支历史预测为什么会在交替输入上失↵
    ↪ 误 |
29 | 第 3 章 | `model_ch03_cache_tlb` | 字节地址怎样映射到 cache line、page 和 TLB↵
    ↪ miss |
30 | 第 4 章 | `model_ch04_numa_coherence` | 本地访问、远端访问和写者迁移怎样分开↵
    ↪ 计数 |
31 | 第 5 章 | `model_ch05_roofline` | FLOP、字节数、带宽和峰值算力怎样推出 ↵
    ↪ attainable GFLOPS |
32 | 第 5b 章 | `model_ch05b_pmu_simd_evidence` | counter 可用性和 SIMD 指令比例怎↵
    ↪ 样进入证据 |
33 | 第 6 章 | `model_ch06_layout_locality` | AoS 与 SoA 在只读热字段时触碰多少字↵
    ↪ 节 |
34 | 第 7 章 | `model_ch07_simd_ilp` | 向量 lane、完整向量轮次和 scalar tail 怎样↵
    ↪ 计算 |
35 | 第 8 章 | `model_ch08_map_filter_reduce` | map/filter/reduce 的输入、过滤数量↵
    ↪ 和归约值怎样对齐 |
36 | 第 9 章 | `model_ch09_thread_scheduling` | 任务成本如何分配到 worker 并暴露负↵
    ↪ 载不均 |
37 | 第 9b 章 | `model_ch09b_linux_wait_wake` | 等待循环为什么要区分真实完成和伪唤↵
    ↪ 醒 |
38 | 第 10 章 | `model_ch10_sync_backpressure` | 容量、生产速率和消费速率怎样产生↵
    ↪ 背压事件 |
39 | 第 11 章 | `model_ch11_atomic_publication` | 版本发布怎样区分 stale read 和乱↵
    ↪ 序观察 |
40 | 第 12 章 | `model_ch12_spsc_ring` | ring buffer 怎样区分成功 push、满拒绝、成↵
    ↪ 功 pop 和空 pop |
41 | 第 12b 章 | `model_ch12b_epoch_reclamation` | 被保护节点为什么必须延迟回收 |
42 | 第 13 章 | `model_ch13_reduce_scan_partition` | reduce、scan 和 partition 三↵
    ↪ 种结果如何来自同一输入 |
43 | 第 14 章 | `model_ch14_work_stealing` | 本地队列不均衡时哪些任务需要被窃取 |
44 | 第 15 章 | `model_ch15_async_io_backpressure` | in-flight 深度如何限制提交并↵
    ↪ 产生背压 |
45 | 第 16 章 | `model_ch16_distributed_attempts` | attempt、duplicate 和 stale ↵
    ↪ generation 怎样分开 |
46 | 第 17 章 | `model_ch17_quorum_commit` | 多数派 quorum 怎样决定 commit |
47 | 第 18 章 | `model_ch18_checkpoint_recovery` | checkpoint 后失败时从哪里 ↵
    ↪ replay |

```

```

48
49 队列实验对应正文第 10 章的同步原语合同。它故意使用容量为 1 的压力用例，覆盖：
50
51 - 空队列关闭唤醒消费者
52 - 满队列关闭唤醒生产者
53 - 多 producer/multiple consumer drain，不丢、不重
54 - `QueueReport` 中的 `push_success`、`push_closed`、`push_wait_count`、`↵
    ↵ push_wait_ns`、`pop_success`、`pop_closed`、`pop_wait_count`、`↵
    ↵ pop_wait_ns`、`max_size`
55
56 `sync_primitives` 实验对应正文第 10 章的同步原语族谱，目标不是测哪个 API 更快，↵
    ↵ 而是固定每类原语表达的状态：
57
58 - semaphore 只限制 permit，不保护多字段对象不变量
59 - barrier 表达固定参与者的重复 phase，所有 worker 每轮都必须到达
60 - future/promise 表达一次结果或异常，broken promise 是独立失败路径
61 - atomic wait 等待单个原子版本变化，不适合跨字段队列谓词
62 - eventfd/epoll 表达唤醒和 ready，不表达业务 payload 或 completion
63 - futex 表达底层睡眠/唤醒路径，不表达 C++ 对象生命周期或业务谓词
64 - task runtime 实验把 worker、ready queue、unfinished task、same-pool 等待和 ↵
    ↵ continuation 分开，避免把 `future.get()` 当作安全的调度原语
65
66 构建：
67
68 ```bash
69 cmake -S books/cpu-volume-2 -B books/cpu-volume-2/build/compsys-debug -↵
    ↵ DCMMAKE_BUILD_TYPE=Debug
70 cmake --build books/cpu-volume-2/build/compsys-debug
71 ctest --test-dir books/cpu-volume-2/build/compsys-debug --output-on-failure
72 ```
73
74 运行：
75
76 ```bash
77 books/cpu-volume-2/build/compsys-debug/labs/compute_systems/compsys_demo
78 ```
79
80 eventfd/epoll 合同探针：
81
82 ```bash
83 books/cpu-volume-2/build/compsys-debug/labs/compute_systems/↵
    ↵ compsys_wait_channels_probe
84 ```
85
86 futex 合同探针：
87
88 ```bash
89 books/cpu-volume-2/build/compsys-debug/labs/compute_systems/↵
    ↵ compsys_futex_lab_probe
90 ```
91
92 task runtime 合同探针：

```

```

93
94 ```bash
95 books/cpu-volume-2/build/compsys-debug/labs/compute_systems/↵
    ↵ compsys_task_runtime_probe
96 ```
97
98 逐章模型探针：
99
100 ```bash
101 books/cpu-volume-2/build/compsys-debug/labs/compute_systems/↵
    ↵ compsys_chapter_models_probe
102 ```

```

D.3 三、逐章模型头文件

`chapter_models.hpp` 为第二册每个主章和深入章定义一个报告结构和一个模型函数。它的目标是让“分支预测、TLB、NUMA、Roofline、背压、epoch、quorum、checkpoint”等词都有能编译的输入和输出。

Listing D.3 `include/compsys/chapter_models.hpp`

```

1 #pragma once
2
3 #include <cstdint>
4 #include <span>
5
6 namespace compsys {
7
8 struct Chapter01PipelineReport {
9     std::int32_t records_total = 0;
10    std::int32_t accepted = 0;
11    std::int32_t rejected = 0;
12    std::int64_t checksum = 0;
13 };
14
15 [[nodiscard]] Chapter01PipelineReport model_ch01_input_pipeline(
16     std::span<const std::int32_t> records);
17
18 struct Chapter02BranchReport {
19     std::int32_t iterations = 0;
20     std::int32_t predicted_taken = 0;
21     std::int32_t actual_taken = 0;
22     std::int32_t mispredictions = 0;
23 };
24
25 [[nodiscard]] Chapter02BranchReport model_ch02_branch_predictor(
26     std::span<const std::int32_t> values,
27     std::int32_t taken_threshold);
28
29 struct Chapter03MemoryAccessReport {
30     std::int32_t access_count = 0;
31     std::int32_t unique_cache_lines = 0;

```

```

32     std::int32_t unique_pages = 0;
33     std::int32_t tlb_misses = 0;
34 };
35
36 [[nodiscard]] Chapter03MemoryAccessReport model_ch03_cache_tlb(
37     std::span<const std::uint64_t> byte_offsets,
38     std::int32_t cache_line_bytes,
39     std::int32_t page_bytes,
40     std::int32_t tlb_entries);
41
42 struct Chapter04NumaReport {
43     std::int32_t access_count = 0;
44     std::int32_t local_accesses = 0;
45     std::int32_t remote_accesses = 0;
46     std::int32_t coherence_transfers = 0;
47 };
48
49 [[nodiscard]] Chapter04NumaReport model_ch04_numa_coherence(
50     std::span<const std::int32_t> worker_nodes,
51     std::span<const std::int32_t> memory_nodes);
52
53 struct Chapter05RooflineReport {
54     double arithmetic_intensity = 0.0;
55     double compute_bound_gflops = 0.0;
56     double memory_bound_gflops = 0.0;
57     double attainable_gflops = 0.0;
58     bool memory_bound = false;
59 };
60
61 [[nodiscard]] Chapter05RooflineReport model_ch05_roofline(
62     double flops,
63     double bytes_read_written,
64     double peak_gflops,
65     double bandwidth_gbytes_per_second);
66
67 struct Chapter05bPmuSimdReport {
68     std::int64_t scalar_instructions = 0;
69     std::int64_t vector_instructions = 0;
70     std::int64_t retired_instructions = 0;
71     double vector_fraction = 0.0;
72     bool counter_available = false;
73     bool evidence_complete = false;
74 };
75
76 [[nodiscard]] Chapter05bPmuSimdReport model_ch05b_pmu_simd_evidence(
77     std::int64_t scalar_instructions,
78     std::int64_t vector_instructions,
79     bool counter_available);
80
81 struct Chapter06LayoutReport {
82     std::int64_t records = 0;
83     std::int64_t aos_bytes_touched = 0;

```



```

84     std::int64_t soa_bytes_touched = 0;
85     std::int64_t saved_bytes = 0;
86 };
87
88 [[nodiscard]] Chapter06LayoutReport model_ch06_layout_locality(
89     std::int64_t records,
90     std::int32_t field_count,
91     std::int32_t hot_field_count,
92     std::int32_t field_bytes);
93
94 struct Chapter07SimdReport {
95     std::int32_t elements = 0;
96     std::int32_t lane_width = 0;
97     std::int32_t vector_iterations = 0;
98     std::int32_t scalar_tail = 0;
99     double lane_utilization = 0.0;
100 };
101
102 [[nodiscard]] Chapter07SimdReport model_ch07_simd_ilp(
103     std::int32_t elements,
104     std::int32_t lane_width);
105
106 struct Chapter08KernelReport {
107     std::int32_t input_count = 0;
108     std::int32_t filtered_count = 0;
109     std::int64_t mapped_sum = 0;
110     std::int64_t reduced_sum = 0;
111 };
112
113 [[nodiscard]] Chapter08KernelReport model_ch08_map_filter_reduce(
114     std::span<const std::int32_t> values);
115
116 struct Chapter09SchedulingReport {
117     std::int32_t worker_count = 0;
118     std::int32_t task_count = 0;
119     std::int64_t min_load = 0;
120     std::int64_t max_load = 0;
121     std::int64_t imbalance = 0;
122 };
123
124 [[nodiscard]] Chapter09SchedulingReport model_ch09_thread_scheduling(
125     std::span<const std::int32_t> task_costs,
126     std::int32_t worker_count);
127
128 struct Chapter09bLinuxWaitReport {
129     std::int32_t poll_count = 0;
130     std::int32_t wakeups = 0;
131     std::int32_t spurious_wakeups = 0;
132     bool completed = false;
133 };
134
135 [[nodiscard]] Chapter09bLinuxWaitReport model_ch09b_linux_wait_wake(

```

```

136     std::span<const bool> ready_observations);
137
138 struct Chapter10BackpressureReport {
139     std::int32_t ticks = 0;
140     std::int32_t capacity = 0;
141     std::int32_t produced = 0;
142     std::int32_t consumed = 0;
143     std::int32_t backpressure_events = 0;
144     std::int32_t max_depth = 0;
145 };
146
147 [[nodiscard]] Chapter10BackpressureReport model_ch10_sync_backpressure(
148     std::int32_t ticks,
149     std::int32_t produce_per_tick,
150     std::int32_t consume_per_tick,
151     std::int32_t capacity);
152
153 struct Chapter11AtomicPublicationReport {
154     std::int32_t published_versions = 0;
155     std::int32_t observed_versions = 0;
156     std::int32_t stale_reads = 0;
157     std::int32_t final_version = 0;
158     bool monotonic_observations = true;
159 };
160
161 [[nodiscard]] Chapter11AtomicPublicationReport model_ch11_atomic_publication(
162     std::span<const std::int32_t> observed_versions,
163     std::int32_t final_version);
164
165 struct Chapter12LockFreeQueueReport {
166     std::int32_t capacity = 0;
167     std::int32_t pushed = 0;
168     std::int32_t popped = 0;
169     std::int32_t rejected_full = 0;
170     std::int32_t empty_pops = 0;
171 };
172
173 [[nodiscard]] Chapter12LockFreeQueueReport model_ch12_spsc_ring(
174     std::span<const std::int32_t> operations,
175     std::int32_t capacity);
176
177 struct Chapter12bReclamationReport {
178     std::int32_t retired_nodes = 0;
179     std::int32_t protected_nodes = 0;
180     std::int32_t reclaimed_nodes = 0;
181     std::int32_t deferred_nodes = 0;
182 };
183
184 [[nodiscard]] Chapter12bReclamationReport model_ch12b_epoch_reclamation(
185     std::span<const std::int32_t> retired_node_ids,
186     std::span<const std::int32_t> protected_node_ids);
187

```

```

188 struct Chapter13ReduceScanPartitionReport {
189     std::int32_t input_count = 0;
190     std::int64_t reduce_sum = 0;
191     std::int64_t last_prefix = 0;
192     std::int32_t partition_true_count = 0;
193     std::int32_t partition_false_count = 0;
194 };
195
196 [[nodiscard]] Chapter13ReduceScanPartitionReport ↵
    ↵ model_ch13_reduce_scan_partition(
197     std::span<const std::int32_t> values,
198     std::int32_t partition_threshold);
199
200 struct Chapter14WorkStealingReport {
201     std::int32_t worker_count = 0;
202     std::int32_t initial_tasks = 0;
203     std::int32_t stolen_tasks = 0;
204     std::int32_t completed_tasks = 0;
205     std::int32_t max_queue_depth = 0;
206 };
207
208 [[nodiscard]] Chapter14WorkStealingReport model_ch14_work_stealing(
209     std::span<const std::int32_t> initial_worker_queues);
210
211 struct Chapter15AsyncIoReport {
212     std::int32_t requested = 0;
213     std::int32_t submitted = 0;
214     std::int32_t completed = 0;
215     std::int32_t backpressure_events = 0;
216     std::int32_t max_in_flight = 0;
217 };
218
219 [[nodiscard]] Chapter15AsyncIoReport model_ch15_async_io_backpressure(
220     std::int32_t request_count,
221     std::int32_t queue_depth,
222     std::int32_t completions_per_tick);
223
224 struct Chapter16DistributedModelReport {
225     std::int32_t logical_tasks = 0;
226     std::int32_t attempts = 0;
227     std::int32_t duplicate_attempts = 0;
228     std::int32_t stale_generation_attempts = 0;
229 };
230
231 [[nodiscard]] Chapter16DistributedModelReport model_ch16_distributed_attempts(
232     std::span<const std::int32_t> task_ids,
233     std::span<const std::int32_t> generations,
234     std::int32_t current_generation);
235
236 struct Chapter17ReplicationReport {
237     std::int32_t replica_count = 0;
238     std::int32_t ack_count = 0;

```

```

239     std::int32_t quorum_size = 0;
240     bool committed = false;
241 };
242
243 [[nodiscard]] Chapter17ReplicationReport model_ch17_quorum_commit(
244     std::int32_t replica_count,
245     std::span<const bool> acknowledgements);
246
247 struct Chapter18RecoveryReport {
248     std::int32_t input_records = 0;
249     std::int32_t checkpoint_interval = 0;
250     std::int32_t checkpoints = 0;
251     std::int32_t replay_from_record = 0;
252     std::int32_t replay_records = 0;
253 };
254
255 [[nodiscard]] Chapter18RecoveryReport model_ch18_checkpoint_recovery(
256     std::int32_t input_records,
257     std::int32_t checkpoint_interval,
258     std::int32_t failed_after_record);
259
260 } // namespace compsys

```

D.4 四、并行归约头文件

`parallel_reduce.hpp` 定义串行 `reference`、并行归约和报告结构。读者应从这里确认每个元素必须恰好参与一次归约，`worker` 数只是调度参数，不改变语义。

Listing D.4 `include/compsys/parallel_reduce.hpp`

```

1 #pragma once
2
3 #include <atomic>
4 #include <condition_variable>
5 #include <cstdint>
6 #include <mutex>
7 #include <optional>
8 #include <span>
9 #include <vector>
10
11 namespace compsys {
12
13     std::int64_t sequential_sum(std::span<const std::int32_t> values);
14
15     std::int64_t parallel_sum(std::span<const std::int32_t> values,
16                             std::int32_t worker_count);
17
18     class MutexCounter {
19     public:
20         void add(std::int64_t delta);
21         [[nodiscard]] std::int64_t value() const;
22

```

```

23 private:
24     mutable std::mutex mutex_;
25     std::int64_t value_ = 0;
26 };
27
28 class AtomicCounter {
29 public:
30     void add(std::int64_t delta);
31     [[nodiscard]] std::int64_t value() const;
32
33 private:
34     std::atomic<std::int64_t> value_{0};
35 };
36
37 class BoundedMpmcQueue {
38 public:
39     explicit BoundedMpmcQueue(std::int32_t capacity);
40
41     [[nodiscard]] bool push(std::int32_t value);
42     [[nodiscard]] std::optional<std::int32_t> pop();
43     [[nodiscard]] std::optional<std::int32_t> try_pop();
44     void close();
45     [[nodiscard]] bool closed() const;
46     [[nodiscard]] std::int32_t size() const;
47
48     struct Report {
49         std::int64_t push_success = 0;
50         std::int64_t push_closed = 0;
51         std::int64_t push_wait_count = 0;
52         std::int64_t push_wait_ns = 0;
53         std::int64_t pop_success = 0;
54         std::int64_t pop_closed = 0;
55         std::int64_t pop_wait_count = 0;
56         std::int64_t pop_wait_ns = 0;
57         std::int64_t close_count = 0;
58         std::int32_t max_size = 0;
59     };
60
61     [[nodiscard]] Report report() const;
62
63 private:
64     std::vector<std::int32_t> buffer_;
65     std::int32_t head_ = 0;
66     std::int32_t tail_ = 0;
67     std::int32_t size_ = 0;
68     bool closed_ = false;
69     Report report_;
70     mutable std::mutex mutex_;
71     std::condition_variable not_full_;
72     std::condition_variable not_empty_;
73 };
74

```



```
75 } // namespace compsys
```

D.5 五、同步原语头文件

这个头文件暴露 mutex counter、atomic counter 和 bounded queue。它把“线程安全”拆成可测试接口：add、value、push、pop、close 和统计报告。

Listing D.5 include/compsys/sync_primitives.hpp

```
1 #pragma once
2
3 #include <cstdint>
4 #include <string>
5
6 namespace compsys {
7
8 struct PermitLimiterReport {
9     std::int32_t task_count = 0;
10    std::int32_t permit_count = 0;
11    std::int32_t entered_count = 0;
12    std::int32_t max_in_flight = 0;
13 };
14
15 [[nodiscard]] PermitLimiterReport run_permit_limiter(std::int32_t task_count,
16                                                     std::int32_t permit_count)↵
17     ↵ ;
18
19 struct PhaseBarrierReport {
20     std::int32_t worker_count = 0;
21     std::int32_t phase_count = 0;
22     std::int32_t completed_phases = 0;
23     std::int32_t arrival_count = 0;
24 };
25
26 [[nodiscard]] PhaseBarrierReport run_phase_barrier(std::int32_t worker_count,
27                                                     std::int32_t phase_count);
28
29 struct FutureResultReport {
30     bool value_ready = false;
31     bool exception_ready = false;
32     bool broken_promise_seen = false;
33     std::int32_t value = 0;
34     std::string exception_message;
35 };
36
37 [[nodiscard]] FutureResultReport run_future_result_paths();
38
39 struct AtomicWaitReport {
40     std::int32_t final_version = 0;
41     std::int32_t wake_count = 0;
42     std::int32_t observed_sum = 0;
43 };
44 }
```

```

43
44 [[nodiscard]] AtomicWaitReport run_atomic_wait_version_counter(
45     std::int32_t publish_count);
46
47 } // namespace compsys

```

D.6 六、等待通道头文件

wait channel 头文件描述 eventfd/epoll 合同报告。读者要区分“被唤醒”和“数据已经 drain 完成”，这两个事实不能混写。

Listing D.6 include/compsys/wait_channels.hpp

```

1 #pragma once
2
3 #include <cstdint>
4
5 namespace compsys {
6
7 struct EventFdEpollReport {
8     std::int32_t wake_write_count = 0;
9     std::uint64_t first_counter_read = 0;
10    std::uint64_t second_counter_read = 0;
11    std::int32_t epoll_ready_observations = 0;
12    std::int32_t epoll_empty_observations = 0;
13    std::int32_t queue_drain_count = 0;
14    std::int32_t semaphore_read_count = 0;
15    bool queue_fifo_ok = false;
16    bool semaphore_empty_after_reads = false;
17 };
18
19 [[nodiscard]] EventFdEpollReport run_eventfd_epoll_contract();
20
21 } // namespace compsys

```

D.7 七、任务运行时头文件

task_runtime.hpp 是第二册主线工程的运行时接口。它把提交、完成、失败、拒绝、队列深度和 CSV 报告都变成结构化字段。

Listing D.7 include/compsys/task_runtime.hpp

```

1 #pragma once
2
3 #include <condition_variable>
4 #include <cstdint>
5 #include <deque>
6 #include <functional>
7 #include <mutex>
8 #include <ostream>
9 #include <thread>
10 #include <vector>

```

```

11
12 namespace compsys {
13
14 class TaskRuntime {
15 public:
16     struct Report {
17         std::int32_t worker_count = 0;
18         std::int32_t active_workers = 0;
19         std::int32_t queued_tasks = 0;
20         std::int32_t max_queue_depth = 0;
21         std::int64_t unfinished_count = 0;
22         std::int64_t submitted_count = 0;
23         std::int64_t completed_count = 0;
24         std::int64_t failed_count = 0;
25         std::int64_t rejected_count = 0;
26         bool accepting = false;
27         bool stopping = false;
28     };
29
30     explicit TaskRuntime(std::int32_t worker_count);
31     ~TaskRuntime();
32
33     TaskRuntime(const TaskRuntime&) = delete;
34     TaskRuntime& operator=(const TaskRuntime&) = delete;
35     TaskRuntime(TaskRuntime&&) = delete;
36     TaskRuntime& operator=(TaskRuntime&&) = delete;
37
38     [[nodiscard]] bool submit(std::function<void()> task);
39     void wait_idle();
40     void shutdown();
41
42     [[nodiscard]] Report report() const;
43     [[nodiscard]] std::int32_t queued_task_count() const;
44
45 private:
46     void worker_loop();
47
48     std::deque<std::function<void()>> tasks_;
49     std::vector<std::thread> workers_;
50     mutable std::mutex mutex_;
51     std::condition_variable ready_;
52     std::condition_variable idle_;
53     bool accepting_ = true;
54     bool stopping_ = false;
55     std::int32_t active_workers_ = 0;
56     Report report_;
57 };
58
59 struct SamePoolFutureWaitReport {
60     std::int32_t worker_count = 0;
61     std::int32_t parent_count = 0;
62     std::int32_t blocked_parent_count = 0;

```

```

63     std::int32_t child_submitted_count_at_stall = 0;
64     std::int32_t queued_child_count_at_stall = 0;
65     std::int32_t child_started_before_release = 0;
66     std::int32_t child_completed_count = 0;
67     bool reached_stall = false;
68     bool starved_without_extra_worker = false;
69 };
70
71 [[nodiscard]] SamePoolFutureWaitReport run_same_pool_future_wait_probe(
72     std::int32_t worker_count);
73
74 struct ContinuationRuntimeReport {
75     std::int32_t worker_count = 0;
76     std::int32_t root_task_count = 0;
77     std::int32_t completed_roots = 0;
78     std::int32_t completed_continuations = 0;
79     std::int32_t rejected_continuations = 0;
80     std::int32_t max_queue_depth = 0;
81     std::int64_t final_sum = 0;
82     bool wait_idle_completed = false;
83 };
84
85 [[nodiscard]] ContinuationRuntimeReport run_continuation_runtime_probe(
86     std::int32_t worker_count,
87     std::int32_t root_task_count);
88
89 void write_task_runtime_report_csv(std::ostream& output,
90     const TaskRuntime::Report& report);
91
92 } // namespace compsys

```

D.8 八、Futex 头文件

futex 头文件只暴露合同 probe 和报告字段。读者应注意：futex 不是普通 mutex 的替代模板，它要分别证明快路径、慢路径和错误返回。

Listing D.8 include/compsys/futex_lab.hpp

```

1 #pragma once
2
3 #include <cstdint>
4
5 namespace compsys {
6
7 struct FutexMutexProbeReport {
8     std::int32_t worker_count = 0;
9     std::int32_t increments_per_worker = 0;
10    std::int32_t final_value = 0;
11    std::int32_t contended_count = 0;
12    std::int32_t wait_count = 0;
13    std::int32_t wake_count = 0;
14 };

```

```

15
16 [[nodiscard]] FutexMutexProbeReport run_futex_mutex_probe(
17     std::int32_t worker_count,
18     std::int32_t increments_per_worker);
19
20 struct FutexContractProbeReport {
21     std::int32_t waiter_count = 0;
22     std::int32_t wake_count = 0;
23     std::int32_t eagain_count = 0;
24     std::int32_t timeout_count = 0;
25     std::int32_t eintr_count = 0;
26     std::int32_t observed_ready_count = 0;
27     bool wait_released_on_ready = false;
28 };
29
30 [[nodiscard]] FutexContractProbeReport run_futex_contract_probe();
31
32 } // namespace compsys

```

D.9 九、逐章模型实现

`chapter_models.cpp` 是对“每章完整模型代码”的直接落实。每个函数都从小输入出发，计算报告字段；测试文件会逐章断言这些字段，`probe` 会把报告打印出来。

Listing D.9 `src/chapter_models.cpp`

[illegible]


```

26     require_positive(denominator, "denominator must be positive");
27     if (numerator <= 0) {
28         return 0;
29     }
30     return (numerator + denominator - 1) / denominator;
31 }
32
33 [[nodiscard]] bool contains_value(std::span<const std::int32_t> values,
34                                   std::int32_t needle) {
35     return std::find(values.begin(), values.end(), needle) != values.end();
36 }
37
38 } // namespace
39
40 Chapter01PipelineReport model_ch01_input_pipeline(
41     std::span<const std::int32_t> records) {
42     Chapter01PipelineReport report;
43     report.records_total = static_cast<std::int32_t>(records.size());
44     for (const std::int32_t record : records) {
45         if (record >= COMPSYS_MODEL_RECORD_ACCEPT_THRESHOLD) {
46             ++report.accepted;
47             report.checksum = (report.checksum * 131) + record + 17;
48         } else {
49             ++report.rejected;
50             report.checksum = (report.checksum * 131) - record + 19;
51         }
52     }
53     return report;
54 }
55
56 Chapter02BranchReport model_ch02_branch_predictor(
57     std::span<const std::int32_t> values,
58     std::int32_t taken_threshold) {
59     Chapter02BranchReport report;
60     report.iterations = static_cast<std::int32_t>(values.size());
61     bool predicted_taken = false;
62     for (const std::int32_t value : values) {
63         const bool actual_taken = value >= taken_threshold;
64         if (predicted_taken) {
65             ++report.predicted_taken;
66         }
67         if (actual_taken) {
68             ++report.actual_taken;
69         }
70         if (predicted_taken != actual_taken) {
71             ++report.mispredictions;
72         }
73         predicted_taken = actual_taken;
74     }
75     return report;
76 }
77

```

```

78 Chapter03MemoryAccessReport model_ch03_cache_tlb(
79     std::span<const std::uint64_t> byte_offsets,
80     std::int32_t cache_line_bytes,
81     std::int32_t page_bytes,
82     std::int32_t tlb_entries) {
83     require_positive(cache_line_bytes, "cache line bytes must be positive");
84     require_positive(page_bytes, "page bytes must be positive");
85     require_positive(tlb_entries, "TLB entries must be positive");
86
87     std::set<std::uint64_t> cache_lines;
88     std::vector<std::uint64_t> tlb_lru;
89     std::set<std::uint64_t> unique_pages;
90     std::int32_t tlb_misses = 0;
91
92     for (const std::uint64_t offset : byte_offsets) {
93         cache_lines.insert(offset / static_cast<std::uint64_t>(cache_line_bytes↵
↵ ));
94         const std::uint64_t page = offset / static_cast<std::uint64_t>(↵
↵ page_bytes);
95         unique_pages.insert(page);
96
97         const auto it = std::find(tlb_lru.begin(), tlb_lru.end(), page);
98         if (it == tlb_lru.end()) {
99             ++tlb_misses;
100             if (static_cast<std::int32_t>(tlb_lru.size()) == tlb_entries) {
101                 tlb_lru.erase(tlb_lru.begin());
102             }
103             } else {
104                 tlb_lru.erase(it);
105             }
106             tlb_lru.push_back(page);
107     }
108
109     Chapter03MemoryAccessReport report;
110     report.access_count = static_cast<std::int32_t>(byte_offsets.size());
111     report.unique_cache_lines = static_cast<std::int32_t>(cache_lines.size());
112     report.unique_pages = static_cast<std::int32_t>(unique_pages.size());
113     report.tlb_misses = tlb_misses;
114     return report;
115 }
116
117 Chapter04NumaReport model_ch04_numa_coherence(
118     std::span<const std::int32_t> worker_nodes,
119     std::span<const std::int32_t> memory_nodes) {
120     if (worker_nodes.size() != memory_nodes.size()) {
121         throw std::runtime_error("worker and memory node spans must have equal ↵
↵ size");
122     }
123
124     Chapter04NumaReport report;
125     report.access_count = static_cast<std::int32_t>(worker_nodes.size());
126     std::int32_t previous_writer_node = -1;

```

```

127     for (std::size_t i = 0; i < worker_nodes.size(); ++i) {
128         const std::int32_t worker_node = worker_nodes[i];
129         const std::int32_t memory_node = memory_nodes[i];
130         if (worker_node < 0 || memory_node < 0) {
131             throw std::runtime_error("NUMA node ids must not be negative");
132         }
133         if (worker_node == memory_node) {
134             ++report.local_accesses;
135         } else {
136             ++report.remote_accesses;
137         }
138         if (previous_writer_node != -1 && previous_writer_node != worker_node) ↵
↵ {
139             ++report.coherence_transfers;
140         }
141         previous_writer_node = worker_node;
142     }
143     return report;
144 }
145
146 Chapter05RooflineReport model_ch05_roofline(
147     double flops,
148     double bytes_read_written,
149     double peak_gflops,
150     double bandwidth_gbytes_per_second) {
151     if (flops <= 0.0 || bytes_read_written <= 0.0 || peak_gflops <= 0.0 ||
152         bandwidth_gbytes_per_second <= 0.0) {
153         throw std::runtime_error("roofline inputs must be positive");
154     }
155
156     Chapter05RooflineReport report;
157     report.arithmetic_intensity = flops / bytes_read_written;
158     report.compute_bound_gflops = peak_gflops;
159     report.memory_bound_gflops =
160         report.arithmetic_intensity * bandwidth_gbytes_per_second;
161     report.attainable_gflops =
162         std::min(report.compute_bound_gflops, report.memory_bound_gflops);
163     report.memory_bound = report.memory_bound_gflops < report.↵
↵ compute_bound_gflops;
164     return report;
165 }
166
167 Chapter05bPmuSimdReport model_ch05b_pmu_simd_evidence(
168     std::int64_t scalar_instructions,
169     std::int64_t vector_instructions,
170     bool counter_available) {
171     if (scalar_instructions < 0 || vector_instructions < 0) {
172         throw std::runtime_error("instruction counts must not be negative");
173     }
174     Chapter05bPmuSimdReport report;
175     report.scalar_instructions = scalar_instructions;
176     report.vector_instructions = vector_instructions;

```

```

177     report.retired_instructions = scalar_instructions + vector_instructions;
178     report.counter_available = counter_available;
179     if (report.retired_instructions > 0) {
180         report.vector_fraction =
181             static_cast<double>(vector_instructions) /
182             static_cast<double>(report.retired_instructions);
183     }
184     report.evidence_complete = counter_available && report.retired_instructions <=
185     <= > 0;
186     return report;
187 }
188 Chapter06LayoutReport model_ch06_layout_locality(
189     std::int64_t records,
190     std::int32_t field_count,
191     std::int32_t hot_field_count,
192     std::int32_t field_bytes) {
193     require_positive(records, "records must be positive");
194     require_positive(field_count, "field count must be positive");
195     require_positive(hot_field_count, "hot field count must be positive");
196     require_positive(field_bytes, "field bytes must be positive");
197     if (hot_field_count > field_count) {
198         throw std::runtime_error("hot field count must not exceed field count");
199     }
200
201     Chapter06LayoutReport report;
202     report.records = records;
203     report.aos_bytes_touched =
204         records * static_cast<std::int64_t>(field_count) * field_bytes;
205     report.soa_bytes_touched =
206         records * static_cast<std::int64_t>(hot_field_count) * field_bytes;
207     report.saved_bytes = report.aos_bytes_touched - report.soa_bytes_touched;
208     return report;
209 }
210
211 Chapter07SimdReport model_ch07_simd_ilp(std::int32_t elements,
212                                         std::int32_t lane_width) {
213     require_positive(elements, "elements must be positive");
214     require_positive(lane_width, "lane width must be positive");
215     Chapter07SimdReport report;
216     report.elements = elements;
217     report.lane_width = lane_width;
218     report.vector_iterations = elements / lane_width;
219     report.scalar_tail = elements % lane_width;
220     const std::int32_t vector_slots =
221         ceil_div(elements, lane_width) * lane_width;
222     report.lane_utilization =
223         static_cast<double>(elements) / static_cast<double>(vector_slots);
224     return report;
225 }
226

```

```

227 Chapter08KernelReport model_ch08_map_filter_reduce(
228     std::span<const std::int32_t> values) {
229     Chapter08KernelReport report;
230     report.input_count = static_cast<std::int32_t>(values.size());
231     for (const std::int32_t value : values) {
232         if ((value % 2) == 0) {
233             ++report.filtered_count;
234             const std::int32_t mapped = value * value;
235             report.mapped_sum += mapped;
236             report.reduced_sum += mapped;
237         }
238     }
239     return report;
240 }
241
242 Chapter09SchedulingReport model_ch09_thread_scheduling(
243     std::span<const std::int32_t> task_costs,
244     std::int32_t worker_count) {
245     require_positive(worker_count, "worker count must be positive");
246     std::vector<std::int64_t> loads(static_cast<std::size_t>(worker_count), 0);
247     for (const std::int32_t cost : task_costs) {
248         if (cost < 0) {
249             throw std::runtime_error("task cost must not be negative");
250         }
251         auto it = std::min_element(loads.begin(), loads.end());
252         *it += cost;
253     }
254
255     Chapter09SchedulingReport report;
256     report.worker_count = worker_count;
257     report.task_count = static_cast<std::int32_t>(task_costs.size());
258     report.min_load = *std::min_element(loads.begin(), loads.end());
259     report.max_load = *std::max_element(loads.begin(), loads.end());
260     report.imbalance = report.max_load - report.min_load;
261     return report;
262 }
263
264 Chapter09bLinuxWaitReport model_ch09b_linux_wait_wake(
265     std::span<const bool> ready_observations) {
266     Chapter09bLinuxWaitReport report;
267     report.poll_count = static_cast<std::int32_t>(ready_observations.size());
268     for (const bool ready : ready_observations) {
269         ++report.wakeups;
270         if (ready) {
271             report.completed = true;
272             return report;
273         }
274         ++report.spurious_wakeups;
275     }
276     return report;
277 }
278

```



```

279 Chapter10BackpressureReport model_ch10_sync_backpressure(
280     std::int32_t ticks,
281     std::int32_t produce_per_tick,
282     std::int32_t consume_per_tick,
283     std::int32_t capacity) {
284     require_positive(ticks, "ticks must be positive");
285     require_positive(produce_per_tick, "produce rate must be positive");
286     require_positive(consume_per_tick, "consume rate must be positive");
287     require_positive(capacity, "capacity must be positive");
288
289     Chapter10BackpressureReport report;
290     report.ticks = ticks;
291     report.capacity = capacity;
292     std::int32_t depth = 0;
293     for (std::int32_t tick = 0; tick < ticks; ++tick) {
294         const std::int32_t available_space = capacity - depth;
295         const std::int32_t accepted = std::min(produce_per_tick, ←
↪ available_space);
296         const std::int32_t rejected = produce_per_tick - accepted;
297         report.produced += accepted;
298         if (rejected > 0) {
299             ++report.backpressure_events;
300         }
301         depth += accepted;
302         const std::int32_t consumed = std::min(depth, consume_per_tick);
303         depth -= consumed;
304         report.consumed += consumed;
305         report.max_depth = std::max(report.max_depth, depth);
306     }
307     return report;
308 }
309
310 Chapter11AtomicPublicationReport model_ch11_atomic_publication(
311     std::span<const std::int32_t> observed_versions,
312     std::int32_t final_version) {
313     require_positive(final_version, "final version must be positive");
314     Chapter11AtomicPublicationReport report;
315     report.published_versions = final_version;
316     report.observed_versions = static_cast<std::int32_t>(observed_versions.size()↪
↪ ());
317     report.final_version = final_version;
318     std::int32_t previous = COMPSYS_MODEL_HISTORY_INITIAL_PREDICTION;
319     for (const std::int32_t observed : observed_versions) {
320         if (observed < 0) {
321             throw std::runtime_error("observed version must not be negative");
322         }
323         if (observed < previous) {
324             report.monotonic_observations = false;
325         }
326         if (observed < final_version) {
327             ++report.stale_reads;
328         }

```

```

329     previous = observed;
330 }
331 return report;
332 }
333
334 Chapter12LockFreeQueueReport model_ch12_spsc_ring(
335     std::span<const std::int32_t> operations,
336     std::int32_t capacity) {
337     require_positive(capacity, "capacity must be positive");
338     Chapter12LockFreeQueueReport report;
339     report.capacity = capacity;
340     std::int32_t depth = 0;
341     for (const std::int32_t operation : operations) {
342         if (operation == COMPSYS_MODEL_PUSH_OPERATION) {
343             if (depth == capacity) {
344                 ++report.rejected_full;
345             } else {
346                 ++depth;
347                 ++report.pushed;
348             }
349         } else if (operation == COMPSYS_MODEL_POP_OPERATION) {
350             if (depth == 0) {
351                 ++report.empty_pops;
352             } else {
353                 --depth;
354                 ++report.popped;
355             }
356         } else {
357             throw std::runtime_error("queue operation must be 1 for push or -1 ←
↪ for pop");
358         }
359     }
360     return report;
361 }
362
363 Chapter12bReclamationReport model_ch12b_epoch_reclamation(
364     std::span<const std::int32_t> retired_node_ids,
365     std::span<const std::int32_t> protected_node_ids) {
366     Chapter12bReclamationReport report;
367     report.retired_nodes = static_cast<std::int32_t>(retired_node_ids.size());
368     report.protected_nodes = static_cast<std::int32_t>(protected_node_ids.size()↪
↪ ());
369     for (const std::int32_t node : retired_node_ids) {
370         if (contains_value(protected_node_ids, node)) {
371             ++report.deferred_nodes;
372         } else {
373             ++report.reclaimed_nodes;
374         }
375     }
376     return report;
377 }
378

```

```

379 Chapter13ReduceScanPartitionReport model_ch13_reduce_scan_partition(
380     std::span<const std::int32_t> values,
381     std::int32_t partition_threshold) {
382     Chapter13ReduceScanPartitionReport report;
383     report.input_count = static_cast<std::int32_t>(values.size());
384     std::int64_t prefix = 0;
385     for (const std::int32_t value : values) {
386         report.reduce_sum += value;
387         prefix += value;
388         report.last_prefix = prefix;
389         if (value >= partition_threshold) {
390             ++report.partition_true_count;
391         } else {
392             ++report.partition_false_count;
393         }
394     }
395     return report;
396 }
397
398 Chapter14WorkStealingReport model_ch14_work_stealing(
399     std::span<const std::int32_t> initial_worker_queues) {
400     if (initial_worker_queues.empty()) {
401         throw std::runtime_error("at least one worker queue is required");
402     }
403     Chapter14WorkStealingReport report;
404     report.worker_count = static_cast<std::int32_t>(initial_worker_queues.size()
↵    ());
405     for (const std::int32_t queue_depth : initial_worker_queues) {
406         if (queue_depth < 0) {
407             throw std::runtime_error("queue depth must not be negative");
408         }
409         report.initial_tasks += queue_depth;
410         report.max_queue_depth = std::max(report.max_queue_depth, queue_depth);
411     }
412     const std::int32_t target_load =
413         ceil_div(report.initial_tasks, report.worker_count);
414     for (const std::int32_t queue_depth : initial_worker_queues) {
415         if (queue_depth > target_load) {
416             report.stolen_tasks += queue_depth - target_load;
417         }
418     }
419     report.completed_tasks = report.initial_tasks;
420     return report;
421 }
422
423 Chapter15AsyncIoReport model_ch15_async_io_backpressure(
424     std::int32_t request_count,
425     std::int32_t queue_depth,
426     std::int32_t completions_per_tick) {
427     require_positive(request_count, "request count must be positive");
428     require_positive(queue_depth, "queue depth must be positive");
429     require_positive(completions_per_tick, "completions per tick must be ↵

```

```

↪ positive");
430
431 Chapter15AsyncIoReport report;
432 report.requested = request_count;
433 std::int32_t remaining = request_count;
434 std::int32_t in_flight = 0;
435 while (remaining > 0 || in_flight > 0) {
436     while (remaining > 0 && in_flight < queue_depth) {
437         --remaining;
438         ++in_flight;
439         ++report.submitted;
440         report.max_in_flight = std::max(report.max_in_flight, in_flight);
441     }
442     if (remaining > 0 && in_flight == queue_depth) {
443         ++report.backpressure_events;
444     }
445     const std::int32_t completed = std::min(in_flight, completions_per_tick↪
↪ );
446     in_flight -= completed;
447     report.completed += completed;
448 }
449 return report;
450 }
451
452 Chapter16DistributedModelReport model_ch16_distributed_attempts(
453     std::span<const std::int32_t> task_ids,
454     std::span<const std::int32_t> generations,
455     std::int32_t current_generation) {
456     if (task_ids.size() != generations.size()) {
457         throw std::runtime_error("task id and generation spans must have equal ↪
↪ size");
458     }
459     Chapter16DistributedModelReport report;
460     report.attempts = static_cast<std::int32_t>(task_ids.size());
461     std::map<std::int32_t, std::int32_t> current_attempt_counts;
462     for (std::size_t i = 0; i < task_ids.size(); ++i) {
463         if (task_ids[i] < 0 || generations[i] < 0) {
464             throw std::runtime_error("task ids and generations must not be ↪
↪ negative");
465         }
466         if (generations[i] != current_generation) {
467             ++report.stale_generation_attempts;
468             continue;
469         }
470         ++current_attempt_counts[task_ids[i]];
471     }
472     report.logical_tasks = static_cast<std::int32_t>(current_attempt_counts.↪
↪ size());
473     for (const auto& [_, count] : current_attempt_counts) {
474         if (count > 1) {
475             report.duplicate_attempts += count - 1;
476         }

```

```

477     }
478     return report;
479 }
480
481 Chapter17ReplicationReport model_ch17_quorum_commit(
482     std::int32_t replica_count,
483     std::span<const bool> acknowledgements) {
484     require_positive(replica_count, "replica count must be positive");
485     if (static_cast<std::int32_t>(acknowledgements.size()) != replica_count) {
486         throw std::runtime_error("acknowledgement count must equal replica ↵
↵ count");
487     }
488     Chapter17ReplicationReport report;
489     report.replica_count = replica_count;
490     report.quorum_size = (replica_count / 2) + 1;
491     for (const bool acknowledged : acknowledgements) {
492         if (acknowledged) {
493             ++report.ack_count;
494         }
495     }
496     report.committed = report.ack_count >= report.quorum_size;
497     return report;
498 }
499
500 Chapter18RecoveryReport model_ch18_checkpoint_recovery(
501     std::int32_t input_records,
502     std::int32_t checkpoint_interval,
503     std::int32_t failed_after_record) {
504     require_positive(input_records, "input records must be positive");
505     require_positive(checkpoint_interval, "checkpoint interval must be positive↵
↵ ");
506     if (failed_after_record < 0 || failed_after_record > input_records) {
507         throw std::runtime_error("failed record must be inside input range");
508     }
509
510     Chapter18RecoveryReport report;
511     report.input_records = input_records;
512     report.checkpoint_interval = checkpoint_interval;
513     report.checkpoints = input_records / checkpoint_interval;
514     report.replay_from_record =
515         (failed_after_record / checkpoint_interval) * checkpoint_interval;
516     report.replay_records = input_records - report.replay_from_record;
517     return report;
518 }
519
520 } // namespace compsys

```

D.10 十、并行归约实现

实现文件展示从串行 reference 到多 worker 切片的完整过程。阅读重点是切片边界、空输入、worker 数过大、局部和汇总和报告字段。

Listing D.10 src/parallel_reduce.cpp

```

1  #include <compsys/parallel_reduce.hpp>
2
3  #include <algorithm>
4  #include <chrono>
5  #include <stdexcept>
6  #include <thread>
7
8  namespace compsys {
9      namespace {
10
11         constexpr std::int32_t COMPSYS_MIN_WORKER_COUNT = 1;
12
13         using Clock = std::chrono::steady_clock;
14
15         std::int64_t sum_range(std::span<const std::int32_t> values,
16                               std::size_t begin,
17                               std::size_t end) {
18             std::int64_t sum = 0;
19             for (std::size_t i = begin; i < end; ++i) {
20                 sum += values[i];
21             }
22             return sum;
23         }
24
25         std::int64_t sum_partials(std::span<const std::int64_t> partials) {
26             std::int64_t sum = 0;
27             for (const std::int64_t value : partials) {
28                 sum += value;
29             }
30             return sum;
31         }
32
33         std::size_t checked_queue_capacity(std::int32_t capacity) {
34             if (capacity <= 0) {
35                 throw std::runtime_error("queue capacity must be positive");
36             }
37             return static_cast<std::size_t>(capacity);
38         }
39
40     } // namespace
41
42     std::int64_t sequential_sum(std::span<const std::int32_t> values) {
43         return sum_range(values, 0, values.size());
44     }
45
46     std::int64_t parallel_sum(std::span<const std::int32_t> values,
47                              std::int32_t worker_count) {
48         if (worker_count < COMPSYS_MIN_WORKER_COUNT) {
49             throw std::runtime_error("worker_count must be positive");
50         }
51         if (values.empty()) {

```

```

52     return 0;
53 }
54
55 const std::int32_t actual_workers = std::min<std::int32_t>(
56     worker_count,
57     static_cast<std::int32_t>(values.size()));
58 std::vector<std::thread> threads;
59 std::vector<std::int64_t> partials(static_cast<std::size_t>(actual_workers)↵
↵ , 0);
60 threads.reserve(static_cast<std::size_t>(actual_workers));
61
62 for (std::int32_t worker = 0; worker < actual_workers; ++worker) {
63     const std::size_t begin =
64         values.size() * static_cast<std::size_t>(worker) /
65         static_cast<std::size_t>(actual_workers);
66     const std::size_t end =
67         values.size() * static_cast<std::size_t>(worker + 1) /
68         static_cast<std::size_t>(actual_workers);
69     threads.emplace_back([&values, &partials, worker, begin, end]() {
70         partials[static_cast<std::size_t>(worker)] = sum_range(values, ↵
↵ begin, end);
71     });
72 }
73
74 for (std::thread& thread : threads) {
75     thread.join();
76 }
77
78 return sum_partials(partials);
79 }
80
81 void MutexCounter::add(std::int64_t delta) {
82     std::lock_guard<std::mutex> lock(this->mutex_);
83     this->value_ += delta;
84 }
85
86 std::int64_t MutexCounter::value() const {
87     std::lock_guard<std::mutex> lock(this->mutex_);
88     return this->value_;
89 }
90
91 void AtomicCounter::add(std::int64_t delta) {
92     this->value_.fetch_add(delta, std::memory_order_relaxed);
93 }
94
95 std::int64_t AtomicCounter::value() const {
96     return this->value_.load(std::memory_order_relaxed);
97 }
98
99 BoundedMpmcQueue::BoundedMpmcQueue(std::int32_t capacity)
100     : buffer_(checked_queue_capacity(capacity), 0) {}
101

```

```

102 bool BoundedMpmcQueue::push(std::int32_t value) {
103     std::unique_lock<std::mutex> lock(this->mutex_);
104     if (this->closed_) {
105         ++this->report_.push_closed;
106         return false;
107     }
108     Clock::time_point wait_begin;
109     bool waited = false;
110     if (this->size_ == static_cast<std::int32_t>(this->buffer_.size())) {
111         ++this->report_.push_wait_count;
112         wait_begin = Clock::now();
113         waited = true;
114     }
115     this->not_full_.wait(lock, [this]() {
116         return this->closed_ ||
117             this->size_ < static_cast<std::int32_t>(this->buffer_.size());
118     });
119     if (waited) {
120         this->report_.push_wait_ns +=
121             std::chrono::duration_cast<std::chrono::nanoseconds>(
122                 Clock::now() - wait_begin)
123                 .count();
124     }
125     if (this->closed_) {
126         ++this->report_.push_closed;
127         return false;
128     }
129
130     this->buffer_[static_cast<std::size_t>(this->tail_)] = value;
131     this->tail_ = (this->tail_ + 1) % static_cast<std::int32_t>(this->buffer_.↵
↵ size());
132     ++this->size_;
133     ++this->report_.push_success;
134     this->report_.max_size = std::max(this->report_.max_size, this->size_);
135     lock.unlock();
136     this->not_empty_.notify_one();
137     return true;
138 }
139
140 std::optional<std::int32_t> BoundedMpmcQueue::pop() {
141     std::unique_lock<std::mutex> lock(this->mutex_);
142     Clock::time_point wait_begin;
143     bool waited = false;
144     if (this->size_ == 0 && !this->closed_) {
145         ++this->report_.pop_wait_count;
146         wait_begin = Clock::now();
147         waited = true;
148     }
149     this->not_empty_.wait(lock, [this]() {
150         return this->closed_ || this->size_ > 0;
151     });
152     if (waited) {

```

```

153         this->report_.pop_wait_ns +=
154             std::chrono::duration_cast<std::chrono::nanoseconds>(
155                 Clock::now() - wait_begin)
156                 .count();
157     }
158     if (this->size_ == 0) {
159         ++this->report_.pop_closed;
160         return std::nullopt;
161     }
162
163     const std::int32_t value = this->buffer_[static_cast<std::size_t>(this->head_)]
164     this->head_ = (this->head_ + 1) % static_cast<std::int32_t>(this->buffer_.size());
165     --this->size_;
166     ++this->report_.pop_success;
167     lock.unlock();
168     this->not_full_.notify_one();
169     return value;
170 }
171
172 std::optional<std::int32_t> BoundedMpmcQueue::try_pop() {
173     std::lock_guard<std::mutex> lock(this->mutex_);
174     if (this->size_ == 0) {
175         return std::nullopt;
176     }
177     const std::int32_t value = this->buffer_[static_cast<std::size_t>(this->head_)]
178     this->head_ = (this->head_ + 1) % static_cast<std::int32_t>(this->buffer_.size());
179     --this->size_;
180     ++this->report_.pop_success;
181     this->not_full_.notify_one();
182     return value;
183 }
184
185 void BoundedMpmcQueue::close() {
186     std::lock_guard<std::mutex> lock(this->mutex_);
187     if (!this->closed_) {
188         this->closed_ = true;
189         ++this->report_.close_count;
190     }
191     this->not_empty_.notify_all();
192     this->not_full_.notify_all();
193 }
194
195 bool BoundedMpmcQueue::closed() const {
196     std::lock_guard<std::mutex> lock(this->mutex_);
197     return this->closed_;
198 }
199
200 std::int32_t BoundedMpmcQueue::size() const {

```

```

201     std::lock_guard<std::mutex> lock(this->mutex_);
202     return this->size_;
203 }
204
205 BoundedMpmcQueue::Report BoundedMpmcQueue::report() const {
206     std::lock_guard<std::mutex> lock(this->mutex_);
207     return this->report_;
208 }
209
210 } // namespace compsys

```

D.11 十一、同步原语实现

同步实现覆盖 mutex、atomic 和有界队列。读者应重点看 close 如何唤醒等待中的生产者和消费者，以及报告字段如何记录等待、关闭和队列水位。

Listing D.11 src/sync_primitives.cpp

```

1  #include <compsys/sync_primitives.hpp>
2
3  #include <algorithm>
4  #include <atomic>
5  #include <barrier>
6  #include <future>
7  #include <latch>
8  #include <limits>
9  #include <semaphore>
10 #include <stdexcept>
11 #include <string>
12 #include <thread>
13 #include <vector>
14
15 namespace compsys {
16 namespace {
17
18 constexpr std::int32_t COMPSYS_MIN_SYNC_COUNT = 1;
19 constexpr std::int32_t COMPSYS_READY_VALUE = 42;
20
21 void require_positive(std::int32_t value, const char* name) {
22     if (value < COMPSYS_MIN_SYNC_COUNT) {
23         throw std::runtime_error(std::string(name) + " must be positive");
24     }
25 }
26
27 void update_max(std::atomic<std::int32_t>& max_value, std::int32_t candidate) {
28     std::int32_t current = max_value.load(std::memory_order_relaxed);
29     while (candidate > current &&
30           !max_value.compare_exchange_weak(current,
31                                             candidate,
32                                             std::memory_order_relaxed,
33                                             std::memory_order_relaxed)) {
34     }

```



```

35 }
36
37 } // namespace
38
39 PermitLimiterReport run_permit_limiter(std::int32_t task_count,
40                                       std::int32_t permit_count) {
41     require_positive(task_count, "task_count");
42     require_positive(permit_count, "permit_count");
43
44     const std::int32_t expected_peak = std::min(task_count, permit_count);
45     std::counting_semaphore<std::numeric_limits<std::int32_t>::max()> permits(
46         permit_count);
47     std::atomic<std::int32_t> in_flight{0};
48     std::atomic<std::int32_t> entered_count{0};
49     std::atomic<std::int32_t> max_in_flight{0};
50     std::latch start_gate(1);
51     std::latch release_gate(1);
52     std::vector<std::thread> workers;
53     workers.reserve(static_cast<std::size_t>(task_count));
54
55     for (std::int32_t task = 0; task < task_count; ++task) {
56         workers.emplace_back([&permits,
57                               &in_flight,
58                               &entered_count,
59                               &max_in_flight,
60                               &start_gate,
61                               &release_gate]() {
62             start_gate.wait();
63             permits.acquire();
64             const std::int32_t now =
65                 in_flight.fetch_add(1, std::memory_order_acq_rel) + 1;
66             entered_count.fetch_add(1, std::memory_order_relaxed);
67             update_max(max_in_flight, now);
68             release_gate.wait();
69             in_flight.fetch_sub(1, std::memory_order_acq_rel);
70             permits.release();
71         });
72     }
73
74     start_gate.count_down();
75     while (entered_count.load(std::memory_order_acquire) < expected_peak) {
76         std::this_thread::yield();
77     }
78     release_gate.count_down();
79
80     for (std::thread& worker : workers) {
81         worker.join();
82     }
83
84     PermitLimiterReport report;
85     report.task_count = task_count;
86     report.permit_count = permit_count;

```

```

87     report.entered_count = entered_count.load(std::memory_order_relaxed);
88     report.max_in_flight = max_in_flight.load(std::memory_order_relaxed);
89     return report;
90 }
91
92 PhaseBarrierReport run_phase_barrier(std::int32_t worker_count,
93                                     std::int32_t phase_count) {
94     require_positive(worker_count, "worker_count");
95     require_positive(phase_count, "phase_count");
96
97     std::atomic<std::int32_t> completed_phases{0};
98     std::atomic<std::int32_t> arrival_count{0};
99     std::barrier phase(worker_count, [&completed_phases]() noexcept {
100         completed_phases.fetch_add(1, std::memory_order_relaxed);
101     });
102     std::vector<std::thread> workers;
103     workers.reserve(static_cast<std::size_t>(worker_count));
104
105     for (std::int32_t worker = 0; worker < worker_count; ++worker) {
106         workers.emplace_back([phase_count, &arrival_count, &phase]() {
107             for (std::int32_t phase_index = 0; phase_index < phase_count; ++phase_index) {
108                 arrival_count.fetch_add(1, std::memory_order_relaxed);
109                 phase.arrive_and_wait();
110             }
111         });
112     }
113
114     for (std::thread& worker : workers) {
115         worker.join();
116     }
117
118     PhaseBarrierReport report;
119     report.worker_count = worker_count;
120     report.phase_count = phase_count;
121     report.completed_phases = completed_phases.load(std::memory_order_relaxed);
122     report.arrival_count = arrival_count.load(std::memory_order_relaxed);
123     return report;
124 }
125
126 FutureResultReport run_future_result_paths() {
127     FutureResultReport report;
128
129     std::promise<std::int32_t> value_promise;
130     std::future<std::int32_t> value_future = value_promise.get_future();
131     value_promise.set_value(COMPSYS_READY_VALUE);
132     report.value = value_future.get();
133     report.value_ready = report.value == COMPSYS_READY_VALUE;
134
135     std::promise<std::int32_t> exception_promise;
136     std::future<std::int32_t> exception_future = exception_promise.get_future();
137     ;

```

```

137     exception_promise.set_exception(std::make_exception_ptr(
138         std::runtime_error("future path failed")));
139     try {
140         static_cast<void>(exception_future.get());
141     } catch (const std::runtime_error& error) {
142         report.exception_ready = true;
143         report.exception_message = error.what();
144     }
145
146     std::future<std::int32_t> broken_future;
147     {
148         std::promise<std::int32_t> broken_promise;
149         broken_future = broken_promise.get_future();
150     }
151     try {
152         static_cast<void>(broken_future.get());
153     } catch (const std::future_error& error) {
154         report.broken_promise_seen =
155             error.code() == std::make_error_code(std::future_errc::←
156             ↪ broken_promise);
157     }
158     return report;
159 }
160
161 AtomicWaitReport run_atomic_wait_version_counter(std::int32_t publish_count) {
162     require_positive(publish_count, "publish_count");
163
164     std::atomic<std::int32_t> version{0};
165     std::atomic<std::int32_t> observed_sum{0};
166     std::atomic<std::int32_t> wake_count{0};
167     std::latch waiter_started(1);
168
169     std::thread waiter([publish_count, &version, &observed_sum, &wake_count, &←
170     ↪ waiter_started]() {
171         std::int32_t observed = version.load(std::memory_order_acquire);
172         waiter_started.count_down();
173         while (observed < publish_count) {
174             version.wait(observed, std::memory_order_acquire);
175             const std::int32_t next = version.load(std::memory_order_acquire);
176             if (next != observed) {
177                 wake_count.fetch_add(1, std::memory_order_relaxed);
178                 observed_sum.fetch_add(next, std::memory_order_relaxed);
179                 observed = next;
180             }
181         }
182     });
183
184     waiter_started.wait();
185     for (std::int32_t value = 1; value <= publish_count; ++value) {
186         version.store(value, std::memory_order_release);
187         version.notify_all();

```

```

187     }
188
189     waiter.join();
190
191     AtomicWaitReport report;
192     report.final_version = version.load(std::memory_order_acquire);
193     report.wake_count = wake_count.load(std::memory_order_relaxed);
194     report.observed_sum = observed_sum.load(std::memory_order_relaxed);
195     return report;
196 }
197
198 } // namespace compsys

```

D.12 十二、等待通道实现

wait channel 实现把 eventfd、epoll、队列和信号量语义放在同一个 probe 里。它的价值是把内核可读事件和用户态数据结构 drain 分开验证。

Listing D.12 src/wait_channels.cpp

```

1  #include <compsys/wait_channels.hpp>
2
3  #include <array>
4  #include <cstdint>
5  #include <deque>
6  #include <stdexcept>
7  #include <vector>
8
9  #include <cerrno>
10 #include <sys/epoll.h>
11 #include <sys/eventfd.h>
12 #include <unistd.h>
13
14 namespace compsys {
15 namespace {
16
17 constexpr std::int32_t COMPSYS_EVENTFD_QUEUE_FIRST = 11;
18 constexpr std::int32_t COMPSYS_EVENTFD_QUEUE_SECOND = 22;
19 constexpr std::int32_t COMPSYS_EVENTFD_QUEUE_THIRD = 33;
20 constexpr std::uint64_t COMPSYS_EVENTFD_WAKE_VALUE = 1;
21 constexpr std::uint64_t COMPSYS_EVENTFD_SEMAPHORE_PULSES = 3;
22 constexpr std::size_t COMPSYS_EVENTFD_EXPECTED_DRAIN_COUNT = 3;
23
24 class UniqueFd {
25 public:
26     UniqueFd() = default;
27     explicit UniqueFd(int fd) noexcept : fd_(fd) {}
28     ~UniqueFd() { this->reset(); }
29
30     UniqueFd(const UniqueFd&) = delete;
31     UniqueFd& operator=(const UniqueFd&) = delete;
32

```

```

33     UniqueFd(UniqueFd&& other) noexcept : fd_(other.release()) {}
34
35     UniqueFd& operator=(UniqueFd&& other) noexcept {
36         if (this != &other) {
37             this->reset(other.release());
38         }
39         return *this;
40     }
41
42     [[nodiscard]] int get() const noexcept {
43         return this->fd_;
44     }
45
46     [[nodiscard]] bool valid() const noexcept {
47         return this->fd_ >= 0;
48     }
49
50     int release() noexcept {
51         const int fd = this->fd_;
52         this->fd_ = -1;
53         return fd;
54     }
55
56     void reset(int fd = -1) noexcept {
57         if (this->fd_ >= 0) {
58             ::close(this->fd_);
59         }
60         this->fd_ = fd;
61     }
62
63 private:
64     int fd_ = -1;
65 };
66
67 [[nodiscard]] UniqueFd make_eventfd(unsigned int flags) {
68     const int fd = ::eventfd(0, flags);
69     if (fd < 0) {
70         throw std::runtime_error("failed to create eventfd");
71     }
72     return UniqueFd(fd);
73 }
74
75 [[nodiscard]] UniqueFd make_epollfd() {
76     const int fd = ::epoll_create1(EPOOL_CLOEXEC);
77     if (fd < 0) {
78         throw std::runtime_error("failed to create epoll instance");
79     }
80     return UniqueFd(fd);
81 }
82
83 void write_eventfd_value(int fd, std::uint64_t value) {
84     while (true) {

```



```

85     const ssize_t written = ::write(fd, &value, sizeof(value));
86     if (written == static_cast<ssize_t>(sizeof(value))) {
87         return;
88     }
89     if (written < 0 && errno == EINTR) {
90         continue;
91     }
92     throw std::runtime_error("failed to write eventfd value");
93 }
94 }
95
96 [[nodiscard]] bool read_eventfd_value(int fd, std::uint64_t& value) {
97     while (true) {
98         const ssize_t read_bytes = ::read(fd, &value, sizeof(value));
99         if (read_bytes == static_cast<ssize_t>(sizeof(value))) {
100             return true;
101         }
102         if (read_bytes < 0 && errno == EINTR) {
103             continue;
104         }
105         if (read_bytes < 0 && errno == EAGAIN) {
106             return false;
107         }
108         throw std::runtime_error("failed to read eventfd value");
109     }
110 }
111
112 [[nodiscard]] bool poll_epoll_ready(int epoll_fd, int expected_fd) {
113     std::array<epoll_event, 1> events{};
114     const int ready = ::epoll_wait(epoll_fd, events.data(), 1, 0);
115     if (ready < 0) {
116         throw std::runtime_error("epoll_wait failed");
117     }
118     if (ready == 0) {
119         return false;
120     }
121     if (ready != 1) {
122         throw std::runtime_error("epoll_wait returned unexpected count");
123     }
124     if (events[0].data.fd != expected_fd) {
125         throw std::runtime_error("epoll returned unexpected fd");
126     }
127     if ((events[0].events & EPOLLIN) == 0) {
128         throw std::runtime_error("epoll returned unexpected event mask");
129     }
130     return true;
131 }
132
133 } // namespace
134
135 EventFdEpollReport run_eventfd_epoll_contract() {
136     EventFdEpollReport report;

```

```

137
138 UniqueFd queue_wake_fd = make_eventfd(EFD_CLOEXEC | EFD_NONBLOCK);
139 UniqueFd semaphore_fd = make_eventfd(EFD_CLOEXEC | EFD_NONBLOCK | ↵
↵ EFD_SEMAPHORE);
140 UniqueFd epoll_fd = make_epollfd();
141
142 epoll_event event{};
143 event.events = EPOLLIN;
144 event.data.fd = queue_wake_fd.get();
145 if (::epoll_ctl(epoll_fd.get(), EPOLL_CTL_ADD, queue_wake_fd.get(), &event)↵
↵ != 0) {
146     throw std::runtime_error("failed to register eventfd with epoll");
147 }
148
149 std::deque<std::int32_t> queue;
150 std::vector<std::int32_t> drained_values;
151 drained_values.reserve(COMPSYS_EVENTFD_EXPECTED_DRAIN_COUNT);
152
153 auto enqueue_request = [&](std::int32_t value) {
154     const bool was_empty = queue.empty();
155     queue.push_back(value);
156     if (was_empty) {
157         write_eventfd_value(queue_wake_fd.get(), COMPSYS_EVENTFD_WAKE_VALUE↵
↵ );
158         ++report.wake_write_count;
159     }
160 };
161
162 auto drain_queue = [&]() {
163     while (!queue.empty()) {
164         drained_values.push_back(queue.front());
165         queue.pop_front();
166         ++report.queue_drain_count;
167     }
168 };
169
170 enqueue_request(COMPSYS_EVENTFD_QUEUE_FIRST);
171 enqueue_request(COMPSYS_EVENTFD_QUEUE_SECOND);
172 if (!poll_epoll_ready(epoll_fd.get(), queue_wake_fd.get())) {
173     throw std::runtime_error("eventfd should become readable after queue ↵
↵ wake");
174 }
175 ++report.epoll_ready_observations;
176
177 std::uint64_t value = 0;
178 if (!read_eventfd_value(queue_wake_fd.get(), value)) {
179     throw std::runtime_error("queue wake fd should be readable");
180 }
181 report.first_counter_read = value;
182
183 drain_queue();
184 if (poll_epoll_ready(epoll_fd.get(), queue_wake_fd.get())) {

```

```

185         throw std::runtime_error("eventfd should be empty after queue drain");
186     }
187     ++report.epoll_empty_observations;
188
189     enqueue_request(COMPSYS_EVENTFD_QUEUE_THIRD);
190     if (!poll_epoll_ready(epoll_fd.get(), queue_wake_fd.get())) {
191         throw std::runtime_error("eventfd should become readable after second ↵
↵ wake");
192     }
193     ++report.epoll_ready_observations;
194
195     value = 0;
196     if (!read_eventfd_value(queue_wake_fd.get(), value)) {
197         throw std::runtime_error("queue wake fd should be readable after second ↵
↵ wake");
198     }
199     report.second_counter_read = value;
200
201     drain_queue();
202     if (poll_epoll_ready(epoll_fd.get(), queue_wake_fd.get())) {
203         throw std::runtime_error("eventfd should be empty after second drain");
204     }
205     ++report.epoll_empty_observations;
206
207     report.queue_fifo_ok =
208         drained_values.size() == COMPSYS_EVENTFD_EXPECTED_DRAIN_COUNT &&
209         drained_values[0] == COMPSYS_EVENTFD_QUEUE_FIRST &&
210         drained_values[1] == COMPSYS_EVENTFD_QUEUE_SECOND &&
211         drained_values[2] == COMPSYS_EVENTFD_QUEUE_THIRD;
212
213     if (::write(semaphore_fd.get(), &COMPSYS_EVENTFD_SEMAPHORE_PULSES,
214             sizeof(COMPSYS_EVENTFD_SEMAPHORE_PULSES)) !=
215         static_cast<ssize_t>(sizeof(COMPSYS_EVENTFD_SEMAPHORE_PULSES))) {
216         throw std::runtime_error("failed to prime eventfd semaphore mode");
217     }
218
219     for (std::int32_t i = 0; i < static_cast<std::int32_t>(↵
↵ COMPSYS_EVENTFD_SEMAPHORE_PULSES);
220         ++i) {
221         std::uint64_t one = 0;
222         if (!read_eventfd_value(semaphore_fd.get(), one)) {
223             throw std::runtime_error("semaphore eventfd should be readable");
224         }
225         if (one != 1U) {
226             throw std::runtime_error("semaphore eventfd should return one wake ↵
↵ per read");
227         }
228         ++report.semaphore_read_count;
229     }
230
231     std::uint64_t empty = 0;
232     report.semaphore_empty_after_reads = !read_eventfd_value(semaphore_fd.get(), ↵

```

```

    ↪ , empty);
233     return report;
234 }
235
236 } // namespace compsys

```

D.13 十三、任务运行时实现

任务运行时实现负责 worker 生命周期、任务队列、异常捕获、shutdown、同池等待 probe、continuation probe 和 CSV 导出。读者应逐段检查每个报告字段在什么路径上变化。

Listing D.13 src/task_runtime.cpp

```

1  #include <compsys/task_runtime.hpp>
2
3  #include <algorithm>
4  #include <atomic>
5  #include <chrono>
6  #include <future>
7  #include <latch>
8  #include <memory>
9  #include <ostream>
10 #include <stdexcept>
11 #include <string>
12 #include <utility>
13
14 namespace compsys {
15 namespace {
16
17 constexpr std::int32_t COMPSYS_TASK_RUNTIME_MIN_WORKERS = 1;
18 constexpr std::int32_t COMPSYS_TASK_RUNTIME_CHILD_VALUE = 1;
19 constexpr std::int32_t COMPSYS_TASK_RUNTIME_STALL_TIMEOUT_MS = 2000;
20
21 void require_positive(std::int32_t value, const char* name) {
22     if (value < COMPSYS_TASK_RUNTIME_MIN_WORKERS) {
23         throw std::runtime_error(std::string(name) + " must be positive");
24     }
25 }
26
27 } // namespace
28
29 TaskRuntime::TaskRuntime(std::int32_t worker_count) {
30     require_positive(worker_count, "worker_count");
31     this->report_.worker_count = worker_count;
32     this->workers_.reserve(static_cast<std::size_t>(worker_count));
33     for (std::int32_t worker = 0; worker < worker_count; ++worker) {
34         this->workers_.emplace_back([this]() {
35             this->worker_loop();
36         });
37     }
38 }

```

```

39
40 TaskRuntime::~TaskRuntime() {
41     this->shutdown();
42 }
43
44 bool TaskRuntime::submit(std::function<void()> task) {
45     if (!task) {
46         throw std::runtime_error("cannot submit an empty task");
47     }
48
49     {
50         std::lock_guard<std::mutex> lock(this->mutex_);
51         if (!this->accepting_ || this->stopping_) {
52             ++this->report_.rejected_count;
53             return false;
54         }
55         this->tasks_.push_back(std::move(task));
56         ++this->report_.submitted_count;
57         ++this->report_.unfinished_count;
58         this->report_.max_queue_depth = std::max(
59             this->report_.max_queue_depth,
60             static_cast<std::int32_t>(this->tasks_.size()));
61     }
62     this->ready_.notify_one();
63     return true;
64 }
65
66 void TaskRuntime::wait_idle() {
67     std::unique_lock<std::mutex> lock(this->mutex_);
68     this->idle_.wait(lock, [this]() {
69         return this->report_.unfinished_count == 0;
70     });
71 }
72
73 void TaskRuntime::shutdown() {
74     {
75         std::lock_guard<std::mutex> lock(this->mutex_);
76         this->accepting_ = false;
77         this->stopping_ = true;
78     }
79     this->ready_.notify_all();
80
81     for (std::thread& worker : this->workers_) {
82         if (worker.joinable()) {
83             worker.join();
84         }
85     }
86 }
87
88 TaskRuntime::Report TaskRuntime::report() const {
89     std::lock_guard<std::mutex> lock(this->mutex_);
90     Report snapshot = this->report_;

```



```

91     snapshot.active_workers = this->active_workers_;
92     snapshot.queued_tasks = static_cast<std::int32_t>(this->tasks_.size());
93     snapshot.accepting = this->accepting_;
94     snapshot.stopping = this->stopping_;
95     return snapshot;
96 }
97
98 std::int32_t TaskRuntime::queued_task_count() const {
99     std::lock_guard<std::mutex> lock(this->mutex_);
100     return static_cast<std::int32_t>(this->tasks_.size());
101 }
102
103 void TaskRuntime::worker_loop() {
104     while (true) {
105         std::function<void()> task;
106         {
107             std::unique_lock<std::mutex> lock(this->mutex_);
108             this->ready_.wait(lock, [this]() {
109                 return this->stopping_ || !this->tasks_.empty();
110             });
111             if (this->tasks_.empty()) {
112                 if (this->stopping_) {
113                     return;
114                 }
115                 continue;
116             }
117             task = std::move(this->tasks_.front());
118             this->tasks_.pop_front();
119             ++this->active_workers_;
120         }
121
122         bool failed = false;
123         try {
124             task();
125         } catch (...) {
126             failed = true;
127         }
128
129         {
130             std::lock_guard<std::mutex> lock(this->mutex_);
131             --this->active_workers_;
132             ++this->report_.completed_count;
133             if (failed) {
134                 ++this->report_.failed_count;
135             }
136             --this->report_.unfinished_count;
137             if (this->report_.unfinished_count == 0) {
138                 this->idle_.notify_all();
139             }
140         }
141     }
142 }

```

```

143
144 SamePoolFutureWaitReport run_same_pool_future_wait_probe(std::int32_t ↵
    ↵ worker_count) {
145     require_positive(worker_count, "worker_count");
146
147     TaskRuntime runtime(worker_count);
148     std::latch start_gate(1);
149     std::mutex progress_mutex;
150     std::condition_variable progress_cv;
151     std::condition_variable release_cv;
152     bool release_parents = false;
153     std::int32_t blocked_parent_count = 0;
154     std::int32_t child_submitted_count = 0;
155     std::int32_t child_started_count = 0;
156     std::int32_t child_completed_count = 0;
157
158     for (std::int32_t parent = 0; parent < worker_count; ++parent) {
159         const bool accepted = runtime.submit([&runtime,
160                                             &start_gate,
161                                             &progress_mutex,
162                                             &progress_cv,
163                                             &release_cv,
164                                             &release_parents,
165                                             &blocked_parent_count,
166                                             &child_submitted_count,
167                                             &child_started_count,
168                                             &child_completed_count]() {
169             start_gate.wait();
170             auto promise = std::make_shared<std::promise<std::int32_t>>();
171             std::future<std::int32_t> future = promise->get_future();
172             const bool child_accepted = runtime.submit([promise,
173                                                         &progress_mutex,
174                                                         &progress_cv,
175                                                         &child_started_count,
176                                                         &child_completed_count↵
    ↵ ]() {
177                 {
178                     std::lock_guard<std::mutex> lock(progress_mutex);
179                     ++child_started_count;
180                     progress_cv.notify_all();
181                 }
182                 promise->set_value(COMPSYS_TASK_RUNTIME_CHILD_VALUE);
183                 {
184                     std::lock_guard<std::mutex> lock(progress_mutex);
185                     ++child_completed_count;
186                     progress_cv.notify_all();
187                 }
188             });
189             if (!child_accepted) {
190                 throw std::runtime_error("child task should be accepted");
191             }
192

```

```

193         if (future.wait_for(std::chrono::milliseconds(0)) !=
194             std::future_status::ready) {
195             std::unique_lock<std::mutex> lock(progress_mutex);
196             ++child_submitted_count;
197             ++blocked_parent_count;
198             progress_cv.notify_all();
199             release_cv.wait(lock, [&release_parents]() {
200                 return release_parents;
201             });
202         }
203     });
204     if (!accepted) {
205         throw std::runtime_error("parent task should be accepted");
206     }
207 }
208 start_gate.count_down();
209
210 bool reached_stall = false;
211 std::int32_t child_submitted_count_at_stall = 0;
212 std::int32_t queued_child_count_at_stall = 0;
213 std::int32_t child_started_before_release = 0;
214 {
215     std::unique_lock<std::mutex> lock(progress_mutex);
216     reached_stall = progress_cv.wait_for(
217         lock,
218         std::chrono::milliseconds(COMPSYS_TASK_RUNTIME_STALL_TIMEOUT_MS),
219         [&blocked_parent_count, worker_count]() {
220             return blocked_parent_count == worker_count;
221         });
222     child_submitted_count_at_stall = child_submitted_count;
223     queued_child_count_at_stall = runtime.queued_task_count();
224     child_started_before_release = child_started_count;
225     release_parents = true;
226 }
227 release_cv.notify_all();
228
229 runtime.wait_idle();
230 const std::int32_t final_child_completed = [&]() {
231     std::lock_guard<std::mutex> lock(progress_mutex);
232     return child_completed_count;
233 }();
234 runtime.shutdown();
235
236 SamePoolFutureWaitReport report;
237 report.worker_count = worker_count;
238 report.parent_count = worker_count;
239 report.blocked_parent_count = blocked_parent_count;
240 report.child_submitted_count_at_stall = child_submitted_count_at_stall;
241 report.queued_child_count_at_stall = queued_child_count_at_stall;
242 report.child_started_before_release = child_started_before_release;
243 report.child_completed_count = final_child_completed;
244 report.reached_stall = reached_stall;

```

```

245     report.starved_without_extra_worker =
246         reached_stall &&
247         blocked_parent_count == worker_count &&
248         child_submitted_count_at_stall == worker_count &&
249         child_started_before_release == 0;
250     return report;
251 }
252
253 ContinuationRuntimeReport run_continuation_runtime_probe(std::int32_t ↵
    ↵ worker_count,
254                                                         std::int32_t ↵
    ↵ root_task_count) {
255     require_positive(worker_count, "worker_count");
256     require_positive(root_task_count, "root_task_count");
257
258     TaskRuntime runtime(worker_count);
259     std::atomic<std::int32_t> completed_roots{0};
260     std::atomic<std::int32_t> completed_continuations{0};
261     std::atomic<std::int32_t> rejected_continuations{0};
262     std::atomic<std::int64_t> final_sum{0};
263
264     for (std::int32_t task_id = 0; task_id < root_task_count; ++task_id) {
265         const std::int32_t value = task_id + 1;
266         const bool accepted = runtime.submit([&runtime,
267                                             &completed_roots,
268                                             &completed_continuations,
269                                             &rejected_continuations,
270                                             &final_sum,
271                                             value]() {
272             completed_roots.fetch_add(1, std::memory_order_relaxed);
273             const bool continuation_accepted = runtime.submit([&
    ↵ completed_continuations,
274                                                         &final_sum,
275                                                         value]() {
276                 completed_continuations.fetch_add(1, std::memory_order_relaxed)↵
    ↵ ;
277                 final_sum.fetch_add(value, std::memory_order_relaxed);
278             });
279             if (!continuation_accepted) {
280                 rejected_continuations.fetch_add(1, std::memory_order_relaxed);
281             }
282         });
283         if (!accepted) {
284             throw std::runtime_error("root task should be accepted");
285         }
286     }
287
288     runtime.wait_idle();
289     const TaskRuntime::Report runtime_report = runtime.report();
290     runtime.shutdown();
291
292     ContinuationRuntimeReport report;

```

```

293     report.worker_count = worker_count;
294     report.root_task_count = root_task_count;
295     report.completed_roots = completed_roots.load(std::memory_order_relaxed);
296     report.completed_continuations =
297         completed_continuations.load(std::memory_order_relaxed);
298     report.rejected_continuations =
299         rejected_continuations.load(std::memory_order_relaxed);
300     report.max_queue_depth = runtime_report.max_queue_depth;
301     report.final_sum = final_sum.load(std::memory_order_relaxed);
302     report.wait_idle_completed = runtime_report.unfinished_count == 0;
303     return report;
304 }
305
306 void write_task_runtime_report_csv(std::ostream& output,
307                                   const TaskRuntime::Report& report)
308 {
309     output << "worker_count,active_workers,queued_tasks,max_queue_depth,"
310             "unfinished_count,submitted_count,completed_count,failed_count,"
311             "rejected_count,accepting,stopping\n";
312     output << report.worker_count << ','
313             << report.active_workers << ','
314             << report.queued_tasks << ','
315             << report.max_queue_depth << ','
316             << report.unfinished_count << ','
317             << report.submitted_count << ','
318             << report.completed_count << ','
319             << report.failed_count << ','
320             << report.rejected_count << ','
321             << (report.accepting ? 1 : 0) << ','
322             << (report.stopping ? 1 : 0) << '\n';
323 }
324
325 } // namespace compsys

```

D.14 十四、Futex 实现

futex 实现覆盖用户态快路径、内核 wait/wake、EAGAIN、timeout 和 EINTR。读者应把这些分支和正文中“快路径不是全部合同”的说明对照。

Listing D.14 src/futex_lab.cpp

```

1 #include <compsys/futex_lab.hpp>
2
3 #include <atomic>
4 #include <cerrno>
5 #include <chrono>
6 #include <cstdint>
7 #include <ctime>
8 #include <latch>
9 #include <stdexcept>
10 #include <string>
11 #include <thread>

```



```

12 #include <vector>
13
14 #include <linux/futex.h>
15 #include <pthread.h>
16 #include <signal.h>
17 #include <sys/syscall.h>
18 #include <unistd.h>
19
20 namespace compsys {
21 namespace {
22
23 constexpr std::int32_t COMPSYS_FUTEX_MIN_WORKERS = 1;
24 constexpr std::int32_t COMPSYS_FUTEX_LOCKED = 1;
25 constexpr std::int32_t COMPSYS_FUTEX_CONTENDED = 2;
26 constexpr std::int32_t COMPSYS_FUTEX_READY_VALUE = 1;
27 constexpr std::int32_t COMPSYS_FUTEX_CONTRACT_TIMEOUT_MS = 100;
28 constexpr std::int32_t COMPSYS_FUTEX_SIGNAL_WAIT_MS = 20;
29 constexpr std::int32_t COMPSYS_FUTEX_MUTEX_SLEEP_MS = 10;
30 constexpr std::int32_t COMPSYS_FUTEX_MS_PER_SECOND = 1000;
31 constexpr long COMPSYS_FUTEX_NS_PER_MS = 1000000L;
32 constexpr std::int32_t COMPSYS_FUTEX_WAKE_SPIN_LIMIT = 100000;
33 constexpr std::int32_t COMPSYS_FUTEX_SIGNAL_RETRY_LIMIT = 100;
34 constexpr std::int32_t COMPSYS_FUTEX_SIGNAL_RETRY_SLEEP_MS = 1;
35 constexpr std::int32_t COMPSYS_FUTEX_STAGE_SIGNAL_WAITING = 1;
36 constexpr std::int32_t COMPSYS_FUTEX_STAGE_SIGNAL_INTERRUPTED = 2;
37 constexpr std::int32_t COMPSYS_FUTEX_STAGE_WAKE_WAITING = 3;
38 constexpr std::int32_t COMPSYS_FUTEX_STAGE_WAKE_DONE = 4;
39 constexpr std::int32_t COMPSYS_FUTEX_STAGE_READY_WAITING = 5;
40 constexpr int COMPSYS_FUTEX_SIGNAL = SIGUSR1;
41
42 void require_positive(std::int32_t value, const char* name) {
43     if (value < COMPSYS_FUTEX_MIN_WORKERS) {
44         throw std::runtime_error(std::string(name) + " must be positive");
45     }
46 }
47
48 void futex_signal_handler(int) {}
49
50 class FutexSignalGuard {
51 public:
52     FutexSignalGuard() {
53         struct sigaction action {};
54         action.sa_handler = futex_signal_handler;
55         sigemptyset(&action.sa_mask);
56         action.sa_flags = 0;
57         if (::sigaction(COMPSYS_FUTEX_SIGNAL, &action, &this->old_action_) != ↵
↵ 0) {
58             throw std::runtime_error("failed to install futex signal handler");
59         }
60     }
61
62     FutexSignalGuard(const FutexSignalGuard&) = delete;

```

```

63     FutexSignalGuard& operator=(const FutexSignalGuard&) = delete;
64
65     ~FutexSignalGuard() {
66         static_cast<void*> (::sigaction(COMPSYS_FUTEX_SIGNAL,
67                                         &this->old_action_,
68                                         nullptr));
69     }
70
71 private:
72     struct sigaction old_action_ {};
73 };
74
75 [[nodiscard]] int futex_wait_word(std::int32_t* word, std::int32_t expected) {
76     return static_cast<int*> (::syscall(SYS_futex,
77                                       word,
78                                       FUTEX_WAIT_PRIVATE,
79                                       expected,
80                                       nullptr,
81                                       nullptr,
82                                       0));
83 }
84
85 [[nodiscard]] int futex_wait_word_timeout(std::int32_t* word,
86                                           std::int32_t expected,
87                                           std::int32_t timeout_ms) {
88     timespec timeout {};
89     timeout.tv_sec = timeout_ms / COMPSYS_FUTEX_MS_PER_SECOND;
90     timeout.tv_nsec =
91         static_cast<long*> (timeout_ms % COMPSYS_FUTEX_MS_PER_SECOND) *
92         COMPSYS_FUTEX_NS_PER_MS;
93     return static_cast<int*> (::syscall(SYS_futex,
94                                       word,
95                                       FUTEX_WAIT_PRIVATE,
96                                       expected,
97                                       &timeout,
98                                       nullptr,
99                                       0));
100 }
101
102 [[nodiscard]] int futex_wake_word(std::int32_t* word, std::int32_t wake_count) ↵
    ↵ {
103     return static_cast<int*> (::syscall(SYS_futex,
104                                       word,
105                                       FUTEX_WAKE_PRIVATE,
106                                       wake_count,
107                                       nullptr,
108                                       nullptr,
109                                       0));
110 }
111
112 class FutexMutex {
113 public:

```

```

114 void lock() {
115     std::atomic_ref<std::int32_t> state(this->state_word_);
116     std::int32_t expected = 0;
117     if (state.compare_exchange_strong(expected,
118                                     COMPSYS_FUTEX_LOCKED,
119                                     std::memory_order_acquire,
120                                     std::memory_order_relaxed)) {
121         return;
122     }
123
124     this->contended_count_.fetch_add(1, std::memory_order_relaxed);
125     while (state.exchange(COMPSYS_FUTEX_CONTENTED,
126                           std::memory_order_acquire) != 0) {
127         this->wait_count_.fetch_add(1, std::memory_order_relaxed);
128         const int rc =
129             futex_wait_word(&this->state_word_, COMPSYS_FUTEX_CONTENTED);
130         if (rc == 0 || errno == EAGAIN || errno == EINTR) {
131             continue;
132         }
133         throw std::runtime_error("futex mutex wait failed");
134     }
135
136 }
137
138 void unlock() {
139     std::atomic_ref<std::int32_t> state(this->state_word_);
140     const std::int32_t previous =
141         state.exchange(0, std::memory_order_release);
142     if (previous == COMPSYS_FUTEX_CONTENTED) {
143         const int woke = futex_wake_word(&this->state_word_, 1);
144         if (woke < 0) {
145             throw std::runtime_error("futex mutex wake failed");
146         }
147         this->wake_count_.fetch_add(woke, std::memory_order_relaxed);
148     }
149 }
150
151 [[nodiscard]] std::int32_t contended_count() const {
152     return this->contended_count_.load(std::memory_order_relaxed);
153 }
154
155 [[nodiscard]] std::int32_t wait_count() const {
156     return this->wait_count_.load(std::memory_order_relaxed);
157 }
158
159 [[nodiscard]] std::int32_t wake_count() const {
160     return this->wake_count_.load(std::memory_order_relaxed);
161 }
162
163 private:
164     alignas(std::int32_t) std::int32_t state_word_ = 0;
165     std::atomic<std::int32_t> contended_count_{0};

```

```

166     std::atomic<std::int32_t> wait_count_{0};
167     std::atomic<std::int32_t> wake_count_{0};
168 };
169
170 [[nodiscard]] bool wait_for_futex_mutex_waiter(const FutexMutex& mutex) {
171     for (std::int32_t attempt = 0;
172          attempt < COMPSYS_FUTEX_WAKE_SPIN_LIMIT;
173          ++attempt) {
174         if (mutex.wait_count() > 0) {
175             std::this_thread::sleep_for(
176                 std::chrono::milliseconds(COMPSYS_FUTEX_MUTEX_SLEEP_MS));
177             return true;
178         }
179         std::this_thread::yield();
180     }
181     return false;
182 }
183
184 } // namespace
185
186 FutexMutexProbeReport run_futex_mutex_probe(std::int32_t worker_count,
187                                             std::int32_t increments_per_worker)↵
188     ↵ {
189     require_positive(worker_count, "worker_count");
190     require_positive(increments_per_worker, "increments_per_worker");
191
192     FutexMutex mutex;
193     std::int32_t shared_value = 0;
194     mutex.lock();
195
196     std::latch ready_gate(worker_count);
197     std::latch start_gate(1);
198     std::vector<std::thread> workers;
199     workers.reserve(static_cast<std::size_t>(worker_count));
200
201     for (std::int32_t worker = 0; worker < worker_count; ++worker) {
202         workers.emplace_back([worker,
203                               increments_per_worker,
204                               &mutex,
205                               &shared_value,
206                               &ready_gate,
207                               &start_gate]() {
208             ready_gate.count_down();
209             start_gate.wait();
210             for (std::int32_t step = 0; step < increments_per_worker; ++step) {
211                 mutex.lock();
212                 ++shared_value;
213                 mutex.unlock();
214             }
215         });
216     }

```

```

217     ready_gate.wait();
218     start_gate.count_down();
219
220     const bool saw_waiter = wait_for_futex_mutex_waiter(mutex);
221     mutex.unlock();
222
223     for (std::thread& worker : workers) {
224         worker.join();
225     }
226
227     if (!saw_waiter) {
228         throw std::runtime_error("futex mutex did not observe a waiting thread"↵
↵ );
229     }
230
231     FutexMutexProbeReport report;
232     report.worker_count = worker_count;
233     report.increments_per_worker = increments_per_worker;
234     report.final_value = shared_value;
235     report.contended_count = mutex.contended_count();
236     report.wait_count = mutex.wait_count();
237     report.wake_count = mutex.wake_count();
238     return report;
239 }
240
241 FutexContractProbeReport run_futex_contract_probe() {
242     FutexSignalGuard signal_guard;
243
244     std::int32_t ready_word = 0;
245     std::int32_t park_word = 0;
246     std::atomic<std::int32_t> stage{0};
247     std::atomic<std::int32_t> eagain_count{0};
248     std::atomic<std::int32_t> timeout_count{0};
249     std::atomic<std::int32_t> eintr_count{0};
250     std::atomic<std::int32_t> observed_ready_count{0};
251     std::atomic<std::int32_t> wake_count{0};
252     std::atomic_ref<std::int32_t> ready_ref(ready_word);
253
254     std::thread waiter([&ready_word,
255                      &park_word,
256                      &stage,
257                      &eagain_count,
258                      &timeout_count,
259                      &eintr_count,
260                      &observed_ready_count,
261                      &ready_ref]() {
262         const int eagain_rc = futex_wait_word(&ready_word, 1);
263         if (eagain_rc < 0 && errno == EAGAIN) {
264             eagain_count.fetch_add(1, std::memory_order_relaxed);
265         }
266
267         const int timeout_rc =

```



```

268         futex_wait_word_timeout(&ready_word,
269                                 0,
270                                 COMPSYS_FUTEX_CONTRACT_TIMEOUT_MS);
271     if (timeout_rc < 0 && errno == ETIMEDOUT) {
272         timeout_count.fetch_add(1, std::memory_order_relaxed);
273     }
274
275     while (eintr_count.load(std::memory_order_relaxed) == 0) {
276         stage.store(COMPSYS_FUTEX_STAGE_SIGNAL_WAITING,
277                   std::memory_order_release);
278         const int signal_rc =
279             futex_wait_word_timeout(&ready_word,
280                                    0,
281                                    COMPSYS_FUTEX_SIGNAL_WAIT_MS);
282         if (signal_rc == 0) {
283             continue;
284         }
285         if (errno == EINTR) {
286             eintr_count.fetch_add(1, std::memory_order_relaxed);
287             stage.store(COMPSYS_FUTEX_STAGE_SIGNAL_INTERRUPTED,
288                       std::memory_order_release);
289             break;
290         }
291         if (errno == ETIMEDOUT || errno == EAGAIN) {
292             continue;
293         }
294         throw std::runtime_error("futex signal wait failed");
295     }
296
297     stage.store(COMPSYS_FUTEX_STAGE_WAKE_WAITING,
298               std::memory_order_release);
299     const int wake_rc = futex_wait_word(&spark_word, 0);
300     if (wake_rc < 0 && errno != EAGAIN && errno != EINTR) {
301         throw std::runtime_error("futex wake-count wait failed");
302     }
303     stage.store(COMPSYS_FUTEX_STAGE_WAKE_DONE,
304               std::memory_order_release);
305
306     while (true) {
307         if (ready_ref.load(std::memory_order_acquire) != 0) {
308             observed_ready_count.fetch_add(1, std::memory_order_relaxed);
309             break;
310         }
311
312         stage.store(COMPSYS_FUTEX_STAGE_READY_WAITING,
313                   std::memory_order_release);
314         const int rc = futex_wait_word(&ready_word, 0);
315         if (rc == 0 || errno == EAGAIN) {
316             continue;
317         }
318         if (errno == EINTR) {
319             continue;

```

```

320     }
321     throw std::runtime_error("futex contract wait failed");
322 }
323 });
324
325 while (stage.load(std::memory_order_acquire) <
326        COMPSYS_FUTEX_STAGE_SIGNAL_WAITING) {
327     std::this_thread::yield();
328 }
329
330 for (std::int32_t attempt = 0;
331      attempt < COMPSYS_FUTEX_SIGNAL_RETRY_LIMIT &&
332      stage.load(std::memory_order_acquire) <
333      COMPSYS_FUTEX_STAGE_SIGNAL_INTERRUPTED;
334      ++attempt) {
335     const int signal_rc =
336         ::pthread_kill(waiter.native_handle(), COMPSYS_FUTEX_SIGNAL);
337     if (signal_rc != 0) {
338         throw std::runtime_error("failed to signal futex waiter");
339     }
340     std::this_thread::sleep_for(
341         std::chrono::milliseconds(COMPSYS_FUTEX_SIGNAL_RETRY_SLEEP_MS));
342 }
343
344 if (stage.load(std::memory_order_acquire) <
345     COMPSYS_FUTEX_STAGE_SIGNAL_INTERRUPTED) {
346     throw std::runtime_error("futex signal wait was not interrupted");
347 }
348
349 std::int32_t actual_wake_count = 0;
350 for (std::int32_t attempt = 0;
351      attempt < COMPSYS_FUTEX_WAKE_SPIN_LIMIT && actual_wake_count == 0;
352      ++attempt) {
353     if (stage.load(std::memory_order_acquire) >=
354         COMPSYS_FUTEX_STAGE_WAKE_WAITING) {
355         const int woke = futex_wake_word(&spark_word, 1);
356         if (woke < 0) {
357             throw std::runtime_error("futex wake failed");
358         }
359         actual_wake_count += woke;
360     }
361     std::this_thread::yield();
362 }
363 if (actual_wake_count == 0) {
364     throw std::runtime_error("futex wake did not observe a waiting thread")↵
↵ ;
365 }
366
367 while (stage.load(std::memory_order_acquire) <
368        COMPSYS_FUTEX_STAGE_READY_WAITING) {
369     std::this_thread::yield();
370 }

```

```

371
372 ready_ref.store(COMPSYS_FUTEX_READY_VALUE, std::memory_order_release);
373 for (std::int32_t attempt = 0;
374      attempt < COMPSYS_FUTEX_WAKE_SPIN_LIMIT &&
375      observed_ready_count.load(std::memory_order_relaxed) == 0;
376      ++attempt) {
377     const int woke = futex_wake_word(&ready_word, 1);
378     if (woke < 0) {
379         throw std::runtime_error("futex wake failed");
380     }
381     std::this_thread::yield();
382 }
383 if (observed_ready_count.load(std::memory_order_relaxed) == 0) {
384     throw std::runtime_error("futex waiter did not observe ready state");
385 }
386 wake_count.store(actual_wake_count, std::memory_order_relaxed);
387
388 waiter.join();
389
390 FutexContractProbeReport report;
391 report.waiter_count = 1;
392 report.wake_count = wake_count.load(std::memory_order_relaxed);
393 report.eagain_count = eagain_count.load(std::memory_order_relaxed);
394 report.timeout_count = timeout_count.load(std::memory_order_relaxed);
395 report.eintr_count = eintr_count.load(std::memory_order_relaxed);
396 report.observed_ready_count =
397     observed_ready_count.load(std::memory_order_relaxed);
398 report.wait_released_on_ready =
399     report.observed_ready_count == 1;
400 return report;
401 }
402
403 } // namespace compsys

```

D.15 十五、主程序

`src/main.cpp` 是最小命令行入口，用来确认库可以被普通可执行程序调用。它不承担完整测试责任，也不应该复制库内部逻辑。

Listing D.15 `src/main.cpp`

```

1 #include <compsys/parallel_reduce.hpp>
2
3 #include <cstdint>
4 #include <cstdlib>
5 #include <iostream>
6 #include <vector>
7
8 namespace {
9
10 constexpr std::int32_t COMPSYS_DEMO_SIZE = 1024;
11 constexpr std::int32_t COMPSYS_DEMO_WORKERS = 4;

```

```

12
13 std::vector<std::int32_t> make_demo_values() {
14     std::vector<std::int32_t> values(static_cast<std::size_t>(COMPSYS_DEMO_SIZE↵
↵ ));
15     for (std::int32_t i = 0; i < COMPSYS_DEMO_SIZE; ++i) {
16         values[static_cast<std::size_t>(i)] = i % 17;
17     }
18     return values;
19 }
20
21 } // namespace
22
23 int main() {
24     const std::vector<std::int32_t> values = make_demo_values();
25     const std::int64_t sequential = compsys::sequential_sum(values);
26     const std::int64_t parallel = compsys::parallel_sum(values, ↵
↵ COMPSYS_DEMO_WORKERS);
27
28     compsys::AtomicCounter counter;
29     counter.add(sequential);
30     counter.add(parallel);
31
32     std::cout << "sequential_sum " << sequential << "\n";
33     std::cout << "parallel_sum " << parallel << "\n";
34     std::cout << "counter " << counter.value() << "\n";
35     return EXIT_SUCCESS;
36 }

```

D.16 十六、逐章模型 probe

`chapter_models_probe.cpp` 运行 21 个逐章模型并打印稳定摘要。它让读者不需要先读完整测试，就能看到每章模型到底输出哪些字段。

Listing D.16 `src/chapter_models_probe.cpp`

```

1 #include <compsys/chapter_models.hpp>
2
3 #include <array>
4 #include <cstdint>
5 #include <cstdlib>
6 #include <exception>
7 #include <iostream>
8
9 namespace {
10
11 constexpr std::int32_t COMPSYS_CHAPTER_MODEL_COUNT = 21;
12 constexpr std::int32_t COMPSYS_BRANCH_THRESHOLD = 5;
13 constexpr std::int32_t COMPSYS_CACHE_LINE_BYTES = 64;
14 constexpr std::int32_t COMPSYS_PAGE_BYTES = 4096;
15 constexpr std::int32_t COMPSYS_TLB_ENTRIES = 2;
16 constexpr double COMPSYS_ROOFLINE_FLOPS = 200.0;
17 constexpr double COMPSYS_ROOFLINE_BYTES = 100.0;

```

```

18 constexpr double COMPSYS_ROOFLINE_PEAK_GFLOPS = 10.0;
19 constexpr double COMPSYS_ROOFLINE_BANDWIDTH_GBPS = 20.0;
20 constexpr std::int32_t COMPSYS_LAYOUT_FIELDS = 4;
21 constexpr std::int32_t COMPSYS_LAYOUT_HOT_FIELDS = 1;
22 constexpr std::int32_t COMPSYS_FIELD_BYTES = 8;
23 constexpr std::int32_t COMPSYS_SIMD_ELEMENTS = 10;
24 constexpr std::int32_t COMPSYS_SIMD_LANES = 4;
25 constexpr std::int32_t COMPSYS_SCHEDULING_WORKERS = 2;
26 constexpr std::int32_t COMPSYS_BACKPRESSURE_TICKS = 3;
27 constexpr std::int32_t COMPSYS_BACKPRESSURE_PRODUCE_RATE = 4;
28 constexpr std::int32_t COMPSYS_BACKPRESSURE_CONSUME_RATE = 1;
29 constexpr std::int32_t COMPSYS_BACKPRESSURE_CAPACITY = 5;
30 constexpr std::int32_t COMPSYS_FINAL_VERSION = 3;
31 constexpr std::int32_t COMPSYS_RING_CAPACITY = 2;
32 constexpr std::int32_t COMPSYS_PARTITION_THRESHOLD = 2;
33 constexpr std::int32_t COMPSYS_IO_REQUESTS = 5;
34 constexpr std::int32_t COMPSYS_IO_QUEUE_DEPTH = 2;
35 constexpr std::int32_t COMPSYS_IO_COMPLETIONS_PER_TICK = 1;
36 constexpr std::int32_t COMPSYS_CURRENT_GENERATION = 2;
37 constexpr std::int32_t COMPSYS_REPLICA_COUNT = 3;
38 constexpr std::int32_t COMPSYS_INPUT_RECORDS = 100;
39 constexpr std::int32_t COMPSYS_CHECKPOINT_INTERVAL = 25;
40 constexpr std::int32_t COMPSYS_FAILED_AFTER_RECORD = 63;
41
42 } // namespace
43
44 int main() {
45     try {
46         const std::array<std::int32_t, 3> records{4, -1, 7};
47         const std::array<std::int32_t, 4> branch_values{1, 9, 2, 10};
48         const std::array<std::uint64_t, 5> offsets{0, 64, 4096, 8192, 0};
49         const std::array<std::int32_t, 3> worker_nodes{0, 1, 1};
50         const std::array<std::int32_t, 3> memory_nodes{0, 0, 1};
51         const std::array<std::int32_t, 4> kernel_values{1, 2, 3, 4};
52         const std::array<std::int32_t, 4> task_costs{5, 1, 4, 2};
53         const std::array<bool, 3> ready_observations{false, false, true};
54         const std::array<std::int32_t, 4> observed_versions{1, 2, 2, 3};
55         const std::array<std::int32_t, 6> ring_operations{1, 1, 1, -1, -1, -1};
56         const std::array<std::int32_t, 3> retired_nodes{1, 2, 3};
57         const std::array<std::int32_t, 1> protected_nodes{2};
58         const std::array<std::int32_t, 3> scan_values{1, 2, 3};
59         const std::array<std::int32_t, 3> worker_queues{6, 0, 0};
60         const std::array<std::int32_t, 4> task_ids{1, 1, 2, 3};
61         const std::array<std::int32_t, 4> generations{2, 2, 1, 2};
62         const std::array<bool, 3> acknowledgements{true, true, false};
63
64         const compsys::Chapter01PipelineReport ch01 =
65             compsys::model_ch01_input_pipeline(records);
66         const compsys::Chapter02BranchReport ch02 =
67             compsys::model_ch02_branch_predictor(branch_values,
68                                                     COMPSYS_BRANCH_THRESHOLD);
69         const compsys::Chapter03MemoryAccessReport ch03 =

```



```

70         compsys::model_ch03_cache_tlb(offsets,
71                                     COMPSYS_CACHE_LINE_BYTES,
72                                     COMPSYS_PAGE_BYTES,
73                                     COMPSYS_TLB_ENTRIES);
74     const compsys::Chapter04NumaReport ch04 =
75         compsys::model_ch04_numa_coherence(worker_nodes, memory_nodes);
76     const compsys::Chapter05RooflineReport ch05 =
77         compsys::model_ch05_roofline(COMPSYS_ROOFLINE_FLOPS,
78                                     COMPSYS_ROOFLINE_BYTES,
79                                     COMPSYS_ROOFLINE_PEAK_GFLOPS,
80                                     COMPSYS_ROOFLINE_BANDWIDTH_GBPS);
81     const compsys::Chapter05bPmuSimdReport ch05b =
82         compsys::model_ch05b_pmu_simd_evidence(80, 20, true);
83     const compsys::Chapter06LayoutReport ch06 =
84         compsys::model_ch06_layout_locality(COMPSYS_SIMD_ELEMENTS,
85                                             COMPSYS_LAYOUT_FIELDS,
86                                             COMPSYS_LAYOUT_HOT_FIELDS,
87                                             COMPSYS_FIELD_BYTES);
88     const compsys::Chapter07SimdReport ch07 =
89         compsys::model_ch07_simd_ilp(COMPSYS_SIMD_ELEMENTS,
90                                     COMPSYS_SIMD_LANES);
91     const compsys::Chapter08KernelReport ch08 =
92         compsys::model_ch08_map_filter_reduce(kernel_values);
93     const compsys::Chapter09SchedulingReport ch09 =
94         compsys::model_ch09_thread_scheduling(task_costs,
95                                             COMPSYS_SCHEDULING_WORKERS);
96     const compsys::Chapter09bLinuxWaitReport ch09b =
97         compsys::model_ch09b_linux_wait_wake(ready_observations);
98     const compsys::Chapter10BackpressureReport ch10 =
99         compsys::model_ch10_sync_backpressure(
100             COMPSYS_BACKPRESSURE_TICKS,
101             COMPSYS_BACKPRESSURE_PRODUCE_RATE,
102             COMPSYS_BACKPRESSURE_CONSUME_RATE,
103             COMPSYS_BACKPRESSURE_CAPACITY);
104     const compsys::Chapter11AtomicPublicationReport ch11 =
105         compsys::model_ch11_atomic_publication(observed_versions,
106                                             COMPSYS_FINAL_VERSION);
107     const compsys::Chapter12LockFreeQueueReport ch12 =
108         compsys::model_ch12_spisc_ring(ring_operations, ↵
↵ COMPSYS_RING_CAPACITY);
109     const compsys::Chapter12bReclamationReport ch12b =
110         compsys::model_ch12b_epoch_reclamation(retired_nodes,
111                                             protected_nodes);
112     const compsys::Chapter13ReduceScanPartitionReport ch13 =
113         compsys::model_ch13_reduce_scan_partition(scan_values,
114                                             ↵
↵ COMPSYS_PARTITION_THRESHOLD);
115     const compsys::Chapter14WorkStealingReport ch14 =
116         compsys::model_ch14_work_stealing(worker_queues);
117     const compsys::Chapter15AsyncIoReport ch15 =
118         compsys::model_ch15_async_io_backpressure(
119             COMPSYS_IO_REQUESTS,

```

```

120         COMPSYS_IO_QUEUE_DEPTH,
121         COMPSYS_IO_COMPLETIONS_PER_TICK);
122     const compsys::Chapter16DistributedModelReport ch16 =
123         compsys::model_ch16_distributed_attempts(task_ids,
124                                                     generations,
125                                                     COMPSYS_CURRENT_GENERATION↵
↵ );
126     const compsys::Chapter17ReplicationReport ch17 =
127         compsys::model_ch17_quorum_commit(COMPSYS_REPLICA_COUNT,
128                                             acknowledgements);
129     const compsys::Chapter18RecoveryReport ch18 =
130         compsys::model_ch18_checkpoint_recovery(
131             COMPSYS_INPUT_RECORDS,
132             COMPSYS_CHECKPOINT_INTERVAL,
133             COMPSYS_FAILED_AFTER_RECORD);
134
135     std::cout << "chapter_models," << COMPSYS_CHAPTER_MODEL_COUNT << '\n';
136     std::cout << "ch01_pipeline," << ch01.records_total << ','
137                 << ch01.accepted << ',' << ch01.rejected << '\n';
138     std::cout << "ch02_branch," << ch02.actual_taken << ','
139                 << ch02.mispredictions << '\n';
140     std::cout << "ch03_cache_tlb," << ch03.unique_cache_lines << ','
141                 << ch03.unique_pages << ',' << ch03.tlb_misses << '\n';
142     std::cout << "ch04_numa," << ch04.local_accesses << ','
143                 << ch04.remote_accesses << ',' << ch04.coherence_transfers <<↵
↵ '\n';
144     std::cout << "ch05_roofline," << ch05.arithmetic_intensity << ','
145                 << ch05.attainable_gflops << ',' << (ch05.memory_bound ? 1 : ↵
↵ 0)
146                 << '\n';
147     std::cout << "ch05b_pmu_simd," << ch05b.retired_instructions << ','
148                 << ch05b.vector_fraction << ','
149                 << (ch05b.evidence_complete ? 1 : 0) << '\n';
150     std::cout << "ch06_layout," << ch06.aos_bytes_touched << ','
151                 << ch06.soa_bytes_touched << ',' << ch06.saved_bytes << '\n';
152     std::cout << "ch07_simd," << ch07.vector_iterations << ','
153                 << ch07.scalar_tail << '\n';
154     std::cout << "ch08_kernel," << ch08.filtered_count << ','
155                 << ch08.reduced_sum << '\n';
156     std::cout << "ch09_scheduling," << ch09.task_count << ','
157                 << ch09.imbalance << '\n';
158     std::cout << "ch09b_wait," << ch09b.spurious_wakeups << ','
159                 << (ch09b.completed ? 1 : 0) << '\n';
160     std::cout << "ch10_backpressure," << ch10.produced << ','
161                 << ch10.backpressure_events << ',' << ch10.max_depth << '\n';
162     std::cout << "ch11_atomic," << ch11.stale_reads << ','
163                 << (ch11.monotonic_observations ? 1 : 0) << '\n';
164     std::cout << "ch12_ring," << ch12.pushed << ',' << ch12.popped << ','
165                 << ch12.rejected_full << ',' << ch12.empty_pops << '\n';
166     std::cout << "ch12b_reclamation," << ch12b.reclaimed_nodes << ','
167                 << ch12b.deferred_nodes << '\n';
168     std::cout << "ch13_reduce_scan_partition," << ch13.reduce_sum << ','

```

```

169         << ch13.last_prefix << ',' << ch13.partition_true_count << '\n'
    ↪ n';
170     std::cout << "ch14_work_stealing," << ch14.stolen_tasks << ','
171         << ch14.completed_tasks << '\n';
172     std::cout << "ch15_async_io," << ch15.submitted << ','
173         << ch15.completed << ',' << ch15.backpressure_events << '\n';
174     std::cout << "ch16_distributed," << ch16.logical_tasks << ','
175         << ch16.duplicate_attempts << ','
176         << ch16.stale_generation_attempts << '\n';
177     std::cout << "ch17_quorum," << ch17.quorum_size << ','
178         << (ch17.committed ? 1 : 0) << '\n';
179     std::cout << "ch18_recovery," << ch18.checkpoints << ','
180         << ch18.replay_from_record << ',' << ch18.replay_records << '\n'
    ↪ n';
181     return EXIT_SUCCESS;
182 } catch (const std::exception& error) {
183     std::cerr << "compsys chapter models probe failure: " << error.what()
    ↪ << '\n';
184     return EXIT_FAILURE;
185 }
186 }

```

D.17 十七、wait channel probe

`wait_channels_probe.cpp` 把 `eventfd/epoll` 合同报告打印出来。读者应重点看输出字段是否能区分 `ready`、`empty`、`FIFO` 和 `semaphore drain`，而不是只看程序是否退出。

Listing D.17 `src/wait_channels_probe.cpp`

```

1 #include <compsys/wait_channels.hpp>
2
3 #include <cstdlib>
4 #include <iomanip>
5 #include <iostream>
6
7 int main() {
8     try {
9         const compsys::EventFdEpollReport report = compsys::
    ↪ run_eventfd_epoll_contract();
10         std::cout << std::fixed << std::setprecision(0);
11         std::cout << "contract,"
12             << report.wake_write_count << ','
13             << report.first_counter_read << ','
14             << report.second_counter_read << ','
15             << report.epoll_ready_observations << ','
16             << report.epoll_empty_observations << ','
17             << report.queue_drain_count << ','
18             << report.semaphore_read_count << ','
19             << (report.queue_fifo_ok ? 1 : 0) << ','
20             << (report.semaphore_empty_after_reads ? 1 : 0) << '\n';
21         return EXIT_SUCCESS;

```

```

22     } catch (const std::exception& error) {
23         std::cerr << "compsys wait channel probe failure: " << error.what() << ↵
        ↵ '\n';
24         return EXIT_FAILURE;
25     }
26 }

```

D.18 十八、TaskRuntime probe

`task_runtime_probe.cpp` 运行任务运行时样本并输出报告。它用于人工观察 submitted、completed、failed、rejected 和 queue depth, 不替代 `compsys_tests.cpp` 的断言。

Listing D.18 src/task_runtime_probe.cpp

```

1  #include <compsys/task_runtime.hpp>
2
3  #include <cstdlib>
4  #include <exception>
5  #include <iostream>
6
7  namespace {
8
9  constexpr std::int32_t COMPSYS_TASK_PROBE_WORKERS = 3;
10 constexpr std::int32_t COMPSYS_TASK_PROBE_ROOTS = 5;
11
12 } // namespace
13
14 int main() {
15     try {
16         const compsys::SamePoolFutureWaitReport same_pool =
17             compsys::run_same_pool_future_wait_probe(COMPSYS_TASK_PROBE_WORKERS, ↵
        ↵ );
18         const compsys::ContinuationRuntimeReport continuation =
19             compsys::run_continuation_runtime_probe(COMPSYS_TASK_PROBE_WORKERS,
20                                                     COMPSYS_TASK_PROBE_ROOTS);
21
22         std::cout << "task_runtime,"
23                   << same_pool.worker_count << ','
24                   << same_pool.blocked_parent_count << ','
25                   << same_pool.child_submitted_count_at_stall << ','
26                   << same_pool.child_started_before_release << ','
27                   << (same_pool.starved_without_extra_worker ? 1 : 0) << ','
28                   << continuation.completed_roots << ','
29                   << continuation.completed_continuations << ','
30                   << continuation.final_sum << ','
31                   << continuation.max_queue_depth << ','
32                   << (continuation.wait_idle_completed ? 1 : 0) << '\n';
33         return EXIT_SUCCESS;
34     } catch (const std::exception& error) {
35         std::cerr << "compsys task runtime probe failure: " << error.what() << ↵
        ↵ '\n';

```

```

36         return EXIT_FAILURE;
37     }
38 }

```

D.19 十九、futex probe

`futex_lab_probe.cpp` 打印 futex 合同样本。读者要把快路径计数、慢路径等待、唤醒和错误返回分开看，不能只用最终计数判断正确。

Listing D.19 src/futex_lab_probe.cpp

```

1  #include <compsys/futex_lab.hpp>
2
3  #include <cstdlib>
4  #include <exception>
5  #include <iostream>
6  #include <stdexcept>
7
8  namespace {
9
10 constexpr std::int32_t COMPSYS_FUTEX_MUTEX_WORKERS = 4;
11 constexpr std::int32_t COMPSYS_FUTEX_MUTEX_INCREMENTS = 50;
12
13 void require(bool condition, const char* message) {
14     if (!condition) {
15         throw std::runtime_error(message);
16     }
17 }
18
19 } // namespace
20
21 int main() {
22     try {
23         const compsys::FutexMutexProbeReport mutex_report =
24             compsys::run_futex_mutex_probe(COMPSYS_FUTEX_MUTEX_WORKERS,
25                                             COMPSYS_FUTEX_MUTEX_INCREMENTS);
26         const compsys::FutexContractProbeReport contract_report =
27             compsys::run_futex_contract_probe();
28         const std::int32_t expected_final_value =
29             COMPSYS_FUTEX_MUTEX_WORKERS * COMPSYS_FUTEX_MUTEX_INCREMENTS;
30
31         require(mutex_report.final_value == expected_final_value,
32                 "futex mutex final value mismatch");
33         require(mutex_report.contended_count > 0,
34                 "futex mutex did not observe contention");
35         require(mutex_report.wait_count > 0,
36                 "futex mutex did not enter wait path");
37         require(mutex_report.wake_count > 0,
38                 "futex mutex did not wake a waiter");
39         require(contract_report.waiter_count == 1,
40                 "futex contract waiter count mismatch");
41         require(contract_report.wake_count > 0,

```



```

42         "futex contract wake count mismatch");
43     require(contract_report.eagain_count == 1,
44         "futex contract EAGAIN count mismatch");
45     require(contract_report.timeout_count == 1,
46         "futex contract timeout count mismatch");
47     require(contract_report.eintr_count == 1,
48         "futex contract EINTR count mismatch");
49     require(contract_report.observed_ready_count == 1,
50         "futex contract ready observation mismatch");
51     require(contract_report.wait_released_on_ready,
52         "futex contract ready release mismatch");
53
54     std::cout << "futex_mutex,"
55         << mutex_report.worker_count << ','
56         << mutex_report.increments_per_worker << ','
57         << mutex_report.final_value << ','
58         << mutex_report.contended_count << ','
59         << mutex_report.wait_count << ','
60         << mutex_report.wake_count << '\n';
61     std::cout << "futex_contract,"
62         << contract_report.waiter_count << ','
63         << contract_report.wake_count << ','
64         << contract_report.eagain_count << ','
65         << contract_report.timeout_count << ','
66         << contract_report.eintr_count << ','
67         << contract_report.observed_ready_count << ','
68         << (contract_report.wait_released_on_ready ? 1 : 0) << '\n';
69     return EXIT_SUCCESS;
70 } catch (const std::exception& error) {
71     std::cerr << "compsys futex lab probe failure: " << error.what() << '\n';
72     return EXIT_FAILURE;
73 }
74 }

```

D.20 二十、完整测试

测试文件是实验的核心证据。它不仅检查最终计数，还逐章断言 chapter models，并检查 close 唤醒、队列 drain、同池等待、CSV 表头、futex 错误路径等容易被漏掉的边界。

Listing D.20 tests/compsys_tests.cpp

```

1 #include <compsys/chapter_models.hpp>
2 #include <compsys/futex_lab.hpp>
3 #include <compsys/parallel_reduce.hpp>
4 #include <compsys/sync_primitives.hpp>
5 #include <compsys/task_runtime.hpp>
6 #include <compsys/wait_channels.hpp>
7
8 #include <array>
9 #include <cstdint>

```

```

10 #include <cstdlib>
11 #include <cmath>
12 #include <iostream>
13 #include <mutex>
14 #include <optional>
15 #include <sstream>
16 #include <stdexcept>
17 #include <thread>
18 #include <vector>
19
20 namespace {
21
22 constexpr std::int32_t COMPSYS_TEST_SIZE = 1000;
23 constexpr std::int32_t COMPSYS_TEST_WORKERS = 8;
24 constexpr std::int32_t COMPSYS_TEST_COUNTER_THREADS = 4;
25 constexpr std::int32_t COMPSYS_TEST_COUNTER_STEPS = 250;
26 constexpr std::int32_t COMPSYS_TEST_QUEUE_CAPACITY = 3;
27 constexpr std::int32_t COMPSYS_TEST_WAIT_SPIN_LIMIT = 100000;
28 constexpr std::int32_t COMPSYS_TEST_FUTEX_MUTEX_WORKERS = 4;
29 constexpr std::int32_t COMPSYS_TEST_FUTEX_MUTEX_INCREMENTS = 50;
30 constexpr std::int32_t COMPSYS_TEST_RUNTIME_WORKERS = 3;
31 constexpr std::int32_t COMPSYS_TEST_CONTINUATION_ROOTS = 5;
32 constexpr std::int64_t COMPSYS_TEST_CONTINUATION_SUM = 15;
33 constexpr double COMPSYS_TEST_DOUBLE_TOLERANCE = 0.000001;
34
35 void require_near(double actual, double expected, const char* message) {
36     if (std::fabs(actual - expected) > COMPSYS_TEST_DOUBLE_TOLERANCE) {
37         throw std::runtime_error(message);
38     }
39 }
40
41 void require(bool condition, const char* message) {
42     if (!condition) {
43         throw std::runtime_error(message);
44     }
45 }
46
47 template <typename Predicate>
48 bool wait_until(Predicate predicate) {
49     for (std::int32_t i = 0; i < COMPSYS_TEST_WAIT_SPIN_LIMIT; ++i) {
50         if (predicate()) {
51             return true;
52         }
53         std::this_thread::yield();
54     }
55     return false;
56 }
57
58 std::vector<std::int32_t> make_values() {
59     std::vector<std::int32_t> values(static_cast<std::size_t>(COMPSYS_TEST_SIZE↵
↵ ));
60     for (std::int32_t i = 0; i < COMPSYS_TEST_SIZE; ++i) {

```

```

61     values[static_cast<std::size_t>(i)] = (i % 11) - 5;
62 }
63 return values;
64 }
65
66 void test_parallel_sum() {
67     const std::vector<std::int32_t> values = make_values();
68     const std::int64_t sequential = compsys::sequential_sum(values);
69     const std::int64_t parallel = compsys::parallel_sum(values, ←
    ↪ COMPSYS_TEST_WORKERS);
70     require(sequential == parallel, "parallel sum mismatch");
71 }
72
73 void test_counters() {
74     compsys::MutexCounter mutex_counter;
75     compsys::AtomicCounter atomic_counter;
76     std::vector<std::thread> threads;
77     threads.reserve(static_cast<std::size_t>(COMPSYS_TEST_COUNTER_THREADS));
78
79     for (std::int32_t worker = 0; worker < COMPSYS_TEST_COUNTER_THREADS; ++↵
    ↪ worker) {
80         threads.emplace_back([&mutex_counter, &atomic_counter]() {
81             for (std::int32_t i = 0; i < COMPSYS_TEST_COUNTER_STEPS; ++i) {
82                 mutex_counter.add(1);
83                 atomic_counter.add(1);
84             }
85         });
86     }
87
88     for (std::thread& thread : threads) {
89         thread.join();
90     }
91
92     const std::int64_t expected =
93         static_cast<std::int64_t>(COMPSYS_TEST_COUNTER_THREADS) *
94         static_cast<std::int64_t>(COMPSYS_TEST_COUNTER_STEPS);
95     require(mutex_counter.value() == expected, "mutex counter mismatch");
96     require(atomic_counter.value() == expected, "atomic counter mismatch");
97 }
98
99 void test_queue() {
100     compsys::BoundedMpmcQueue queue(COMPSYS_TEST_QUEUE_CAPACITY);
101     require(queue.push(7), "first push should succeed");
102     require(queue.push(9), "second push should succeed");
103     require(queue.size() == 2, "queue size mismatch after push");
104
105     const std::optional<std::int32_t> first = queue.try_pop();
106     const std::optional<std::int32_t> second = queue.try_pop();
107     const std::optional<std::int32_t> third = queue.try_pop();
108
109     require(first.has_value() && first.value() == 7, "first queue value ←
    ↪ mismatch");

```

```

110     require(second.has_value() && second.value() == 9, "second queue value ←
    ↪ mismatch");
111     require(!third.has_value(), "empty queue should return nullopt");
112
113     const compsys::BoundedMpmcQueue::Report report = queue.report();
114     require(report.push_success == 2, "queue push report mismatch");
115     require(report.pop_success == 2, "queue pop report mismatch");
116     require(report.max_size == 2, "queue max size report mismatch");
117 }
118
119 void test_queue_close_wakes_consumer() {
120     compsys::BoundedMpmcQueue queue(1);
121     std::optional<std::int32_t> result;
122
123     std::thread consumer([&queue, &result]() {
124         result = queue.pop();
125     });
126
127     const bool saw_wait = wait_until([&queue]() {
128         return queue.report().pop_wait_count == 1;
129     });
130     queue.close();
131     consumer.join();
132
133     require(saw_wait, "consumer should enter pop wait before close");
134     require(!result.has_value(), "closed empty queue should end pop");
135     const compsys::BoundedMpmcQueue::Report report = queue.report();
136     require(report.close_count == 1, "close count mismatch");
137     require(report.pop_closed == 1, "closed pop report mismatch");
138     require(report.pop_wait_count == 1, "closed pop wait count mismatch");
139     require(report.pop_wait_ns > 0, "closed pop wait ns should be recorded");
140     require(queue.closed(), "queue should be closed");
141 }
142
143 void test_queue_close_wakes_blocked_producer() {
144     compsys::BoundedMpmcQueue queue(1);
145     require(queue.push(11), "initial push should succeed");
146
147     bool second_push_result = true;
148     std::thread producer([&queue, &second_push_result]() {
149         second_push_result = queue.push(22);
150     });
151
152     const bool saw_wait = wait_until([&queue]() {
153         return queue.report().push_wait_count == 1;
154     });
155     queue.close();
156     producer.join();
157
158     require(saw_wait, "producer should enter push wait before close");
159     require(!second_push_result, "push waiting on full queue should stop after ←
    ↪ close");

```

```

160     const compsys::BoundedMpmcQueue::Report report = queue.report();
161     require(report.push_success == 1, "push success report mismatch after close↵
↵ ");
162     require(report.push_closed == 1, "push closed report mismatch");
163     require(report.push_wait_count == 1, "push wait count mismatch");
164     require(report.push_wait_ns > 0, "push wait ns should be recorded");
165     require(report.close_count == 1, "close report mismatch for blocked ↵
↵ producer");
166 }
167
168 void test_queue_multi_producer_consumer_drain() {
169     constexpr std::int32_t PRODUCERS = 3;
170     constexpr std::int32_t VALUES_PER_PRODUCER = 20;
171     constexpr std::int32_t CONSUMERS = 2;
172     constexpr std::int32_t EXPECTED_TOTAL = PRODUCERS * VALUES_PER_PRODUCER;
173
174     compsys::BoundedMpmcQueue queue(1);
175     std::vector<std::thread> producers;
176     std::vector<std::thread> consumers;
177     std::vector<std::int32_t> consumed;
178     std::mutex consumed_mutex;
179
180     producers.reserve(static_cast<std::size_t>(PRODUCERS));
181     consumers.reserve(static_cast<std::size_t>(CONSUMERS));
182
183     for (std::int32_t producer = 0; producer < PRODUCERS; ++producer) {
184         producers.emplace_back([producer, &queue]() {
185             for (std::int32_t i = 0; i < VALUES_PER_PRODUCER; ++i) {
186                 const std::int32_t value = producer * VALUES_PER_PRODUCER + i;
187                 if (!queue.push(value)) {
188                     return;
189                 }
190             }
191         });
192     }
193
194     for (std::int32_t consumer = 0; consumer < CONSUMERS; ++consumer) {
195         consumers.emplace_back([&queue, &consumed, &consumed_mutex]() {
196             while (true) {
197                 std::optional<std::int32_t> value = queue.pop();
198                 if (!value.has_value()) {
199                     return;
200                 }
201                 std::lock_guard<std::mutex> lock(consumed_mutex);
202                 consumed.push_back(value.value());
203             }
204         });
205     }
206
207     for (std::thread& producer : producers) {
208         producer.join();
209     }

```



```

210     queue.close();
211     for (std::thread& consumer : consumers) {
212         consumer.join();
213     }
214
215     require(static_cast<std::int32_t>(consumed.size()) == EXPECTED_TOTAL,
216             "drain queue consumed count mismatch");
217
218     std::vector<bool> seen(static_cast<std::size_t>(EXPECTED_TOTAL), false);
219     for (const std::int32_t value : consumed) {
220         require(value >= 0 && value < EXPECTED_TOTAL, "drain queue value out of ↵
↵ range");
221         const std::size_t index = static_cast<std::size_t>(value);
222         require(!seen[index], "drain queue duplicate value");
223         seen[index] = true;
224     }
225
226     const compsys::BoundedMpmcQueue::Report report = queue.report();
227     require(report.push_success == EXPECTED_TOTAL, "drain queue push report ↵
↵ mismatch");
228     require(report.pop_success == EXPECTED_TOTAL, "drain queue pop report ↵
↵ mismatch");
229     require(report.max_size == 1, "capacity one queue should report max size ↵
↵ one");
230 }
231
232 void test_invalid_queue_capacity() {
233     bool threw = false;
234     try {
235         compsys::BoundedMpmcQueue queue(0);
236     } catch (const std::runtime_error&) {
237         threw = true;
238     }
239     require(threw, "invalid queue capacity should throw");
240 }
241
242 void test_permit_limiter() {
243     constexpr std::int32_t TASK_COUNT = 16;
244     constexpr std::int32_t PERMIT_COUNT = 3;
245
246     const compsys::PermitLimiterReport report =
247         compsys::run_permit_limiter(TASK_COUNT, PERMIT_COUNT);
248
249     require(report.task_count == TASK_COUNT, "permit limiter task count ↵
↵ mismatch");
250     require(report.permit_count == PERMIT_COUNT, "permit limiter permit count ↵
↵ mismatch");
251     require(report.entered_count == TASK_COUNT, "permit limiter entered count ↵
↵ mismatch");
252     require(report.max_in_flight <= PERMIT_COUNT,
253             "permit limiter exceeded permit count");
254     require(report.max_in_flight == PERMIT_COUNT,

```

```

255         "permit limiter should observe the full permit window");
256     }
257
258     void test_phase_barrier() {
259         constexpr std::int32_t WORKER_COUNT = 4;
260         constexpr std::int32_t PHASE_COUNT = 5;
261
262         const compsys::PhaseBarrierReport report =
263             compsys::run_phase_barrier(WORKER_COUNT, PHASE_COUNT);
264
265         require(report.worker_count == WORKER_COUNT, "barrier worker count mismatch" ←
        ↪ ");
266         require(report.phase_count == PHASE_COUNT, "barrier phase count mismatch");
267         require(report.completed_phases == PHASE_COUNT, "barrier completion count ←
        ↪ mismatch");
268         require(report.arrival_count == WORKER_COUNT * PHASE_COUNT,
269             "barrier arrival count mismatch");
270     }
271
272     void test_future_result_paths() {
273         const compsys::FutureResultReport report = compsys::run_future_result_paths ←
        ↪ ();
274
275         require(report.value_ready, "future value path failed");
276         require(report.value == 42, "future value mismatch");
277         require(report.exception_ready, "future exception path failed");
278         require(report.exception_message == "future path failed",
279             "future exception message mismatch");
280         require(report.broken_promise_seen, "broken promise path failed");
281     }
282
283     void test_atomic_wait_version_counter() {
284         constexpr std::int32_t PUBLISH_COUNT = 6;
285         const compsys::AtomicWaitReport report =
286             compsys::run_atomic_wait_version_counter(PUBLISH_COUNT);
287
288         require(report.final_version == PUBLISH_COUNT, "atomic wait final version ←
        ↪ mismatch");
289         require(report.wake_count >= 1, "atomic wait did not observe any wake");
290         require(report.observed_sum >= PUBLISH_COUNT,
291             "atomic wait observed sum should include final version");
292     }
293
294     void test_eventfd_epoll_contract() {
295         const compsys::EventFdEpollReport report = compsys:: ←
        ↪ run_eventfd_epoll_contract();
296
297         require(report.wake_write_count == 2, "eventfd wake write count mismatch");
298         require(report.first_counter_read == 1, "eventfd first counter read ←
        ↪ mismatch");
299         require(report.second_counter_read == 1, "eventfd second counter read ←
        ↪ mismatch");

```

```

300     require(report.epoll_ready_observations == 2,
301             "eventfd epoll ready observation mismatch");
302     require(report.epoll_empty_observations == 2,
303             "eventfd epoll empty observation mismatch");
304     require(report.queue_drain_count == 3, "eventfd queue drain count mismatch" ←
    ↪ );
305     require(report.semaphore_read_count == 3, "eventfd semaphore read count ←
    ↪ mismatch");
306     require(report.queue_fifo_ok, "eventfd queue FIFO contract failed");
307     require(report.semaphore_empty_after_reads, "eventfd semaphore empty ←
    ↪ contract failed");
308 }
309
310 void test_futex_lab_probe() {
311     const compsys::FutexMutexProbeReport mutex_report =
312         compsys::run_futex_mutex_probe(COMPSYS_TEST_FUTEX_MUTEX_WORKERS,
313                                         COMPSYS_TEST_FUTEX_MUTEX_INCREMENTS);
314     const std::int32_t expected_final_value =
315         COMPSYS_TEST_FUTEX_MUTEX_WORKERS * COMPSYS_TEST_FUTEX_MUTEX_INCREMENTS;
316
317     require(mutex_report.worker_count == COMPSYS_TEST_FUTEX_MUTEX_WORKERS,
318             "futex mutex worker count mismatch");
319     require(mutex_report.increments_per_worker ==
320             COMPSYS_TEST_FUTEX_MUTEX_INCREMENTS,
321             "futex mutex increment count mismatch");
322     require(mutex_report.final_value == expected_final_value,
323             "futex mutex final value mismatch");
324     require(mutex_report.contended_count > 0,
325             "futex mutex should observe contention");
326     require(mutex_report.wait_count > 0,
327             "futex mutex should enter the wait slow path");
328     require(mutex_report.wake_count > 0,
329             "futex mutex should wake a waiting thread");
330
331     const compsys::FutexContractProbeReport contract_report =
332         compsys::run_futex_contract_probe();
333
334     require(contract_report.waiter_count == 1,
335             "futex contract waiter count mismatch");
336     require(contract_report.wake_count > 0,
337             "futex contract should wake a waiter");
338     require(contract_report.eagain_count == 1,
339             "futex contract EAGAIN path mismatch");
340     require(contract_report.timeout_count == 1,
341             "futex contract timeout path mismatch");
342     require(contract_report.eintr_count == 1,
343             "futex contract signal interruption path mismatch");
344     require(contract_report.observed_ready_count == 1,
345             "futex contract ready observation mismatch");
346     require(contract_report.wait_released_on_ready,
347             "futex contract ready release mismatch");
348 }

```

```

349
350 void test_same_pool_future_wait_probe() {
351     const compsys::SamePoolFutureWaitReport report =
352         compsys::run_same_pool_future_wait_probe(COMPSYS_TEST_RUNTIME_WORKERS);
353
354     require(report.worker_count == COMPSYS_TEST_RUNTIME_WORKERS,
355             "same-pool probe worker count mismatch");
356     require(report.parent_count == COMPSYS_TEST_RUNTIME_WORKERS,
357             "same-pool probe parent count mismatch");
358     require(report.blocked_parent_count == COMPSYS_TEST_RUNTIME_WORKERS,
359             "same-pool probe should block every parent");
360     require(report.child_submitted_count_at_stall == ↵
361         ↵ COMPSYS_TEST_RUNTIME_WORKERS,
362             "same-pool probe should submit every child at stall");
363     require(report.queued_child_count_at_stall >= 0,
364             "same-pool queued child count should be observable");
365     require(report.child_started_before_release == 0,
366             "same-pool child should not run while every worker is blocked");
367     require(report.child_completed_count == COMPSYS_TEST_RUNTIME_WORKERS,
368             "same-pool child completion mismatch after release");
369     require(report.reached_stall, "same-pool probe did not reach stall");
370     require(report.starved_without_extra_worker,
371             "same-pool probe should expose starvation risk");
372 }
373
374 void test_continuation_runtime_probe() {
375     const compsys::ContinuationRuntimeReport report =
376         compsys::run_continuation_runtime_probe(COMPSYS_TEST_RUNTIME_WORKERS,
377         COMPSYS_TEST_CONTINUATION_ROOTS↵
378         ↵ );
379
380     require(report.worker_count == COMPSYS_TEST_RUNTIME_WORKERS,
381             "continuation probe worker count mismatch");
382     require(report.root_task_count == COMPSYS_TEST_CONTINUATION_ROOTS,
383             "continuation probe root count mismatch");
384     require(report.completed_roots == COMPSYS_TEST_CONTINUATION_ROOTS,
385             "continuation probe root completion mismatch");
386     require(report.completed_continuations == COMPSYS_TEST_CONTINUATION_ROOTS,
387             "continuation probe continuation completion mismatch");
388     require(report.rejected_continuations == 0,
389             "continuation probe should not reject continuations");
390     require(report.final_sum == COMPSYS_TEST_CONTINUATION_SUM,
391             "continuation probe final sum mismatch");
392     require(report.max_queue_depth > 0,
393             "continuation probe should observe queued work");
394     require(report.wait_idle_completed,
395             "continuation probe wait_idle should finish with no unfinished work↵
396         ↵ ");
397 }
398
399 void test_task_runtime_report_csv() {
400     compsys::TaskRuntime runtime(COMPSYS_TEST_RUNTIME_WORKERS);

```

```

398     const bool accepted = runtime.submit([]() {});
399     require(accepted, "task runtime should accept first task");
400     runtime.wait_idle();
401     runtime.shutdown();
402
403     const compsys::TaskRuntime::Report report = runtime.report();
404     std::ostringstream output;
405     compsys::write_task_runtime_report_csv(output, report);
406
407     const std::string csv = output.str();
408     require(csv.find("worker_count,active_workers,queued_tasks,max_queue_depth"↵
↵ ) == 0,
409         "task runtime CSV header mismatch");
410     require(csv.find(",1,") != std::string::npos,
411         "task runtime CSV should include a completed count");
412 }
413
414 void test_chapter_models_pipeline_branch_and_memory() {
415     const std::vector<std::int32_t> records{4, -1, 7};
416     const compsys::Chapter01PipelineReport pipeline =
417         compsys::model_ch01_input_pipeline(records);
418     require(pipeline.records_total == 3, "chapter 1 total mismatch");
419     require(pipeline.accepted == 2, "chapter 1 accepted mismatch");
420     require(pipeline.rejected == 1, "chapter 1 rejected mismatch");
421
422     const std::vector<std::int32_t> branch_values{1, 9, 2, 10};
423     const compsys::Chapter02BranchReport branch =
424         compsys::model_ch02_branch_predictor(branch_values, 5);
425     require(branch.iterations == 4, "chapter 2 iteration mismatch");
426     require(branch.actual_taken == 2, "chapter 2 actual taken mismatch");
427     require(branch.mispredictions > 0, "chapter 2 should expose branch misses"↵
↵ ;
428
429     const std::vector<std::uint64_t> offsets{0, 64, 4096, 8192, 0};
430     const compsys::Chapter03MemoryAccessReport memory =
431         compsys::model_ch03_cache_tlb(offsets, 64, 4096, 2);
432     require(memory.access_count == 5, "chapter 3 access count mismatch");
433     require(memory.unique_cache_lines == 4, "chapter 3 cache line mismatch");
434     require(memory.unique_pages == 3, "chapter 3 unique page mismatch");
435     require(memory.tlb_misses == 4, "chapter 3 TLB miss mismatch");
436 }
437
438 void test_chapter_models_numa_roofline_and_layout() {
439     const std::vector<std::int32_t> worker_nodes{0, 1, 1};
440     const std::vector<std::int32_t> memory_nodes{0, 0, 1};
441     const compsys::Chapter04NumaReport numa =
442         compsys::model_ch04_numa_coherence(worker_nodes, memory_nodes);
443     require(numa.local_accesses == 2, "chapter 4 local access mismatch");
444     require(numa.remote_accesses == 1, "chapter 4 remote access mismatch");
445     require(numa.coherence_transfers == 1, "chapter 4 transfer mismatch");
446
447     const compsys::Chapter05RooflineReport roofline =

```



```

448     compsys::model_ch05_roofline(200.0, 100.0, 10.0, 20.0);
449     require_near(roofline.arithmetic_intensity, 2.0,
450                 "chapter 5 arithmetic intensity mismatch");
451     require_near(roofline.memory_bound_gflops, 40.0,
452                 "chapter 5 memory roof mismatch");
453     require_near(roofline.attainable_gflops, 10.0,
454                 "chapter 5 attainable mismatch");
455     require(!roofline.memory_bound, "chapter 5 should be compute-bound");
456
457     const compsys::Chapter05bPmuSimdReport simd_evidence =
458         compsys::model_ch05b_pmu_simd_evidence(80, 20, true);
459     require(simd_evidence.retired_instructions == 100,
460             "chapter 5b retired instruction mismatch");
461     require_near(simd_evidence.vector_fraction, 0.2,
462                 "chapter 5b vector fraction mismatch");
463     require(simd_evidence.evidence_complete,
464             "chapter 5b evidence should be complete");
465
466     const compsys::Chapter06LayoutReport layout =
467         compsys::model_ch06_layout_locality(10, 4, 1, 8);
468     require(layout.aos_bytes_touched == 320, "chapter 6 AoS bytes mismatch");
469     require(layout.soa_bytes_touched == 80, "chapter 6 SoA bytes mismatch");
470     require(layout.saved_bytes == 240, "chapter 6 saved bytes mismatch");
471 }
472
473 void test_chapter_models_kernels_and_scheduling() {
474     const compsys::Chapter07SimdReport simd =
475         compsys::model_ch07_simd_ilp(10, 4);
476     require(simd.vector_iterations == 2, "chapter 7 vector iteration mismatch");
477     require(simd.scalar_tail == 2, "chapter 7 scalar tail mismatch");
478
479     const std::vector<std::int32_t> kernel_values{1, 2, 3, 4};
480     const compsys::Chapter08KernelReport kernel =
481         compsys::model_ch08_map_filter_reduce(kernel_values);
482     require(kernel.filtered_count == 2, "chapter 8 filtered count mismatch");
483     require(kernel.mapped_sum == 20, "chapter 8 mapped sum mismatch");
484     require(kernel.reduced_sum == 20, "chapter 8 reduced sum mismatch");
485
486     const std::vector<std::int32_t> task_costs{5, 1, 4, 2};
487     const compsys::Chapter09SchedulingReport scheduling =
488         compsys::model_ch09_thread_scheduling(task_costs, 2);
489     require(scheduling.task_count == 4, "chapter 9 task count mismatch");
490     require(scheduling.imbalance > 0, "chapter 9 should expose imbalance");
491
492     const std::array<bool, 3> ready_observations{false, false, true};
493     const compsys::Chapter09bLinuxWaitReport wait =
494         compsys::model_ch09b_linux_wait_wake(ready_observations);
495     require(wait.spurious_wakeups == 2, "chapter 9b spurious wake mismatch");
496     require(wait.completed, "chapter 9b should complete");
497 }
498

```

```

499 void test_chapter_models_sync_atomic_and_reclamation() {
500     const compsys::Chapter10BackpressureReport backpressure =
501         compsys::model_ch10_sync_backpressure(3, 4, 1, 5);
502     require(backpressure.backpressure_events > 0,
503             "chapter 10 should observe backpressure");
504     require(backpressure.max_depth <= backpressure.capacity,
505             "chapter 10 max depth exceeded capacity");
506
507     const std::vector<std::int32_t> observed_versions{1, 2, 2, 3};
508     const compsys::Chapter11AtomicPublicationReport atomic =
509         compsys::model_ch11_atomic_publication(observed_versions, 3);
510     require(atomic.stale_reads == 3, "chapter 11 stale read mismatch");
511     require(atomic.monotonic_observations,
512             "chapter 11 observations should be monotonic");
513
514     const std::vector<std::int32_t> ring_operations{1, 1, 1, -1, -1, -1};
515     const compsys::Chapter12LockFreeQueueReport ring =
516         compsys::model_ch12_spsc_ring(ring_operations, 2);
517     require(ring.pushed == 2, "chapter 12 pushed mismatch");
518     require(ring.rejected_full == 1, "chapter 12 rejected mismatch");
519     require(ring.popped == 2, "chapter 12 popped mismatch");
520     require(ring.empty_pops == 1, "chapter 12 empty pop mismatch");
521
522     const std::vector<std::int32_t> retired_nodes{1, 2, 3};
523     const std::vector<std::int32_t> protected_nodes{2};
524     const compsys::Chapter12bReclamationReport reclamation =
525         compsys::model_ch12b_epoch_reclamation(retired_nodes, protected_nodes);
526     require(reclamation.reclaimed_nodes == 2, "chapter 12b reclaimed mismatch")↵
↵ ;
527     require(reclamation.deferred_nodes == 1, "chapter 12b deferred mismatch");
528 }
529
530 void test_chapter_models_runtime_io_and_distributed() {
531     const std::vector<std::int32_t> scan_values{1, 2, 3};
532     const compsys::Chapter13ReduceScanPartitionReport scan =
533         compsys::model_ch13_reduce_scan_partition(scan_values, 2);
534     require(scan.reduce_sum == 6, "chapter 13 reduce sum mismatch");
535     require(scan.last_prefix == 6, "chapter 13 last prefix mismatch");
536     require(scan.partition_true_count == 2, "chapter 13 partition true mismatch")↵
↵ );
537
538     const std::vector<std::int32_t> worker_queues{6, 0, 0};
539     const compsys::Chapter14WorkStealingReport stealing =
540         compsys::model_ch14_work_stealing(worker_queues);
541     require(stealing.stolen_tasks > 0, "chapter 14 should steal tasks");
542     require(stealing.completed_tasks == 6, "chapter 14 completed mismatch");
543
544     const compsys::Chapter15AsyncIoReport io =
545         compsys::model_ch15_async_io_backpressure(5, 2, 1);
546     require(io.submitted == 5, "chapter 15 submitted mismatch");
547     require(io.completed == 5, "chapter 15 completed mismatch");
548     require(io.max_in_flight == 2, "chapter 15 max in-flight mismatch");

```

```

549     require(io.backpressure_events > 0,
550            "chapter 15 should observe backpressure");
551
552     const std::vector<std::int32_t> task_ids{1, 1, 2, 3};
553     const std::vector<std::int32_t> generations{2, 2, 1, 2};
554     const compsys::Chapter16DistributedModelReport distributed =
555         compsys::model_ch16_distributed_attempts(task_ids, generations, 2);
556     require(distributed.logical_tasks == 2, "chapter 16 logical task mismatch")↵
↵ ;
557     require(distributed.duplicate_attempts == 1,
558            "chapter 16 duplicate attempt mismatch");
559     require(distributed.stale_generation_attempts == 1,
560            "chapter 16 stale generation mismatch");
561
562     const std::array<bool, 3> acknowledgements{true, true, false};
563     const compsys::Chapter17ReplicationReport replication =
564         compsys::model_ch17_quorum_commit(3, acknowledgements);
565     require(replication.quorum_size == 2, "chapter 17 quorum mismatch");
566     require(replication.committed, "chapter 17 should commit");
567
568     const compsys::Chapter18RecoveryReport recovery =
569         compsys::model_ch18_checkpoint_recovery(100, 25, 63);
570     require(recovery.checkpoints == 4, "chapter 18 checkpoint mismatch");
571     require(recovery.replay_from_record == 50,
572            "chapter 18 replay start mismatch");
573     require(recovery.replay_records == 50, "chapter 18 replay count mismatch");
574 }
575
576 void test_chapter_model_error_paths() {
577     bool saw_invalid_ring = false;
578     try {
579         const std::vector<std::int32_t> invalid_operations{1, 0};
580         static_cast<void>(compsys::model_ch12_spisc_ring(invalid_operations, 2))↵
↵ ;
581     } catch (const std::runtime_error&) {
582         saw_invalid_ring = true;
583     }
584     require(saw_invalid_ring, "chapter 12 should reject unknown operations");
585
586     bool saw_mismatched_spans = false;
587     try {
588         const std::vector<std::int32_t> task_ids{1};
589         const std::vector<std::int32_t> generations{1, 1};
590         static_cast<void>(
591             compsys::model_ch16_distributed_attempts(task_ids, generations, 1))↵
↵ ;
592     } catch (const std::runtime_error&) {
593         saw_mismatched_spans = true;
594     }
595     require(saw_mismatched_spans, "chapter 16 should reject mismatched spans");
596 }
597

```

```
598 } // namespace
599
600 int main() {
601     try {
602         test_parallel_sum();
603         test_counters();
604         test_queue();
605         test_queue_close_wakes_consumer();
606         test_queue_close_wakes_blocked_producer();
607         test_queue_multi_producer_consumer_drain();
608         test_invalid_queue_capacity();
609         test_permit_limiter();
610         test_phase_barrier();
611         test_future_result_paths();
612         test_atomic_wait_version_counter();
613         test_eventfd_epoll_contract();
614         test_futex_lab_probe();
615         test_same_pool_future_wait_probe();
616         test_continuation_runtime_probe();
617         test_task_runtime_report_csv();
618         test_chapter_models_pipeline_branch_and_memory();
619         test_chapter_models_numa_roofline_and_layout();
620         test_chapter_models_kernels_and_scheduling();
621         test_chapter_models_sync_atomic_and_reclamation();
622         test_chapter_models_runtime_io_and_distributed();
623         test_chapter_model_error_paths();
624         std::cout << "compsys tests passed\n";
625         return EXIT_SUCCESS;
626     } catch (const std::exception& error) {
627         std::cerr << "compsys test failure: " << error.what() << "\n";
628         return EXIT_FAILURE;
629     }
630 }
```

第二册能力清单

第二册的出口不是“知道很多系统名词”，而是能把性能、并发、运行时、I/O 和分布式边界写成可验证的工程报告。

能实现

- 实现顺序 `reference`、并行 `reduce/scan/partition`、线程切分和局部 `partial` 合并。
- 实现可关闭 `bounded queue`，覆盖阻塞 `push`、阻塞 `pop`、`close`、`drain`、等待报告和容量为一压力测试。
- 实现一个最小 `thread pool` 或 `task runtime`，能解释 `ready`、`running`、`blocked`、`finished`、`failed`、`canceled` 和 `wait_idle`。
- 实现 I/O 管线的最小状态机：`buffer ownership`、`short write`、`completion`、`manifest`、`cancel` 和 `backpressure`。

能诊断

- 用 `FLOPs`、`bytes`、`arithmetic intensity`、工作集、页数和线程扩展曲线判断 `kernel` 可能受什么限制。
- 用 `PMU/Top-Down` 或替代证据区分 `frontend bound`、`bad speculation`、`backend bound`、`memory bound`、`cache/TLB miss` 和 `branch miss`。
- 诊断 `false sharing`、`lock convoy`、`futex` 等待、`same-pool future` 饥饿、`barrier phase skip` 和 `broken promise`。
- 区分 `eventfd/epoll readiness`、`io_uring completion`、`page cache`、`direct I/O`、`fsync` 和可靠提交。

能解释

- 解释 `cache line`、`TLB`、`page size`、`huge page`、`NUMA first-touch`、`coherence`、`HITM` 和 `remote access` 的证据边界。
- 解释 `AVX2/AVX-512/AMX` 类后端优化为什么要看 `lane`、`tail`、`horizontal reduction`、`gather/scatter`、`port pressure` 和 `downclock`。
- 解释 C++ `memory model` 中 `data race`、`happens-before`、`release/acquire`、`relaxed`、`seq_cst` 和 `litmus test`。
- 解释 `lock-free` 的 `linearization point`、`ABA`、`hazard pointer`、`epoch reclamation`、`RCU` 和 `progress guarantee`。

不能夸大的部分

第二册主要训练现代单机计算系统和分布式边界的基本方法。它不替代第三册的 `tokenizer`、真实模型加载、`KV cache`、`quantization`、`AI compiler`、`serving` 和 `GPU kernel` 训练。

术语表

算术强度 操作数与数据移动字节数的比值，用来判断一个内核更可能受计算吞吐还是内存带宽限制。

Roofline 用计算峰值、内存带宽和算术强度估算性能上界的模型。

CPU Central Processing Unit，中央处理器。

GPU Graphics Processing Unit，图形处理器或通用并行加速器。

OS Operating System，操作系统，负责调度线程、管理内存、文件、网络和设备。

AMD Advanced Micro Devices，常见 CPU/GPU 厂商名。性能报告要记录具体平台，不能只写 x86。

C++20 C++ 标准版本名。本册并发、同步和运行时代码示例默认使用的语言能力边界。

MSVC Microsoft Visual C++，微软 C++ 工具链。源码阅读标准库实现时，要和 libstdc++、libc++ 区分。

ISA Instruction Set Architecture，指令集架构，规定软件可见的指令、寄存器和状态。

API Application Programming Interface，源码层调用合同。

ABI Application Binary Interface，二进制层调用、对象布局 and 链接合同。

CFG Control-Flow Graph，控制流图，把基本块和跳转边连起来。

STL Standard Template Library，C++ 标准库容器、迭代器和算法的常见统称。

HPC High-Performance Computing，高性能计算，关注吞吐、并行、带宽和可复现实验。

SIMD 一条指令处理多个数据 lane 的执行方式，是现代 CPU 单核吞吐的重要来源。

ILP Instruction-Level Parallelism，指令级并行。互不依赖的指令可以重叠执行，用来隐藏部分延迟。

ASCII 早期单字节字符集合，适合用 SIMD 做简单字节分类。

UTF-8 Unicode 的常见变长字节编码。解析器要处理跨块 continuation byte 和非法序列。

AVX/AVX2 x86 向量指令扩展，常用于 256 位向量数据并行。

AVX-512 x86 512 位向量扩展，提供更宽寄存器和 mask 机制，但要验证频率、平台覆盖和短输入成本。

ARM 一类常见处理器架构生态。跨 ARM 和 x86 比较性能时，要重新验证 ISA、cache、内存序和工具事件。

POWER 一类处理器架构生态，常作为弱内存模型讨论样本。可移植 C++ 同步不能依赖 x86 的偶然强顺序。

RISC Reduced Instruction Set Computer，精简指令集计算机风格的架构类别。

RISC-V 开放指令集架构，常作为和 x86、ARM 对照的 ISA 样本。

SSE Streaming SIMD Extensions，较早的 x86 SIMD 扩展族。

LLVM 编译器基础设施项目，第二册常用其向量化诊断、IR 和工具链证据。

GCC GNU 编译器集合，常与 LLVM/Clang 对照观察优化和向量化。

SLP Superword-Level Parallelism，把相邻相似标量语句合成向量操作的一类向量化方式。

perf Linux 性能分析工具，可用于采样、读取 PMU 事件、观察调度和调用栈。它提供证据，但需要绑定具体命令、事件和环境。

objdump 二进制反汇编和目标文件查看工具，常用于确认编译器是否生成了预期指

令。

strace 系统调用跟踪工具, 用于确认程序是否频繁进入内核、是否遇到 EINTR/EAGAIN 等状态。

numactl NUMA 控制和观测工具, 可用于绑定 CPU、内存节点或查看 NUMA 拓扑。

VNNI Vector Neural Network Instructions, 面向低精度 dot-product 的向量指令能力。

AMX Advanced Matrix Extensions, x86 tile 级矩阵扩展, 适合足够大的矩阵块计算。

FMA Fused Multiply-Add, 一条指令完成乘加, 常用于向量浮点内核。

LTO Link-Time Optimization, 链接期优化, 允许跨翻译单元做全局分析和内联。

PGO Profile-Guided Optimization, 基于运行画像反馈优化分支、布局和内联决策。

JIT Just-In-Time compilation, 运行时即时编译或生成代码。

PC Program Counter, 程序计数器, 保存下一条取指地址。前端、分支预测、I-cache 和 ITLB 都围绕 PC 流工作。

ALU Arithmetic Logic Unit, 算术逻辑单元, 执行整数算术、逻辑和部分比较操作。

AGU Address Generation Unit, 地址生成单元, 为 load/store 计算有效地址。

SIB Scale-Index-Base, x86 地址编码字段, 用 base、index、scale 组合有效地址。

LEA Load Effective Address, x86 地址计算指令, 常被编译器用于整数地址/乘加计算。

ROB Reorder Buffer, 重排序缓冲区, 保存乱序执行但尚未按程序顺序提交的微操作。

乱序执行 CPU 在保持单线程可观察结果不变的前提下, 提前执行已经准备好的指令以隐藏延迟。

分支预测 CPU 在分支结果确定前预测执行路径, 预测失败会清空错误路径上的工作。

BTB Branch Target Buffer, 分支目标缓存, 用来预测跳转目标地址。

RAS Return Address Stack, 返回地址栈, 用来预测函数返回会回到哪里。

cache line 缓存一致性和缓存填充的基本粒度, 常见大小为 64 字节。

MMU Memory Management Unit, 内存管理单元, 负责地址翻译和权限检查。

TLB 地址翻译缓存, 用于缓存虚拟页到物理页的映射。

ITLB/DTLB ITLB 缓存取指地址翻译, DTLB 缓存数据访问地址翻译。二者 miss 的症状和优化手段不同。

PTE Page Table Entry, 页表项, 记录虚拟页到物理页的映射和权限。

L1/L2/L3/L1D CPU 缓存层级。L1D 是一级数据缓存; L3 常是多个核心共享的最后一级缓存。

LLC Last-Level Cache, 最后一级缓存, 通常是多个核心共享的较大缓存。

MESI Modified、Exclusive、Shared、Invalid, 常见缓存一致性状态模型, 用来解释共享缓存行如何在核心间转移。

MESIF/MOESI MESI 的一致性协议变体。F 常表示 forward, O 常表示 owned; 状态名不同, 但共享写要协调所有权这一事实不变。

HITM Hit Modified, 访问的数据命中在其他核心修改过的缓存行, 常提示共享写热点或 false sharing。

LL/SC Load-Linked/Store-Conditional, 一类原子更新机制。先建立链接读取, 再在链接仍有效时条件写入。

SRAM Static Random-Access Memory, 静态随机访问存储器, 常用于 CPU cache。

RFO Read For Ownership, 写入共享缓存行前获取独占所有权的事务。

THP Transparent Huge Pages, 透明大页, 系统自动尝试用更大的页减少 TLB 压力。

ULP Unit in the Last Place, 浮点末位单位, 用于描述浮点误差尺度。

DRAM Dynamic Random-Access Memory, 主存常见介质, 容量大但远慢于缓存。

VM Virtual Memory, 虚拟内存。多线程共享同一进程 VM, 所以指针、mmap、堆对象和 data race 都落在同一地址空间。

VMA Virtual Memory Area, 进程地址空间中的连续虚拟内存区域。

VFS Virtual File System, Linux 中统一文件系统接口的抽象层。

ASLR Address Space Layout Randomization, 地址空间布局随机化, 会影响代码、栈、堆和共享库地址。

PMU Performance Monitoring Unit, 性能监控单元, 用硬件计数器观察周期、cache miss、分支失败等事件。

TMA Top-down Microarchitecture Analysis, 自顶向下微架构分析方法, 把瓶颈粗分为前端受限、后端受限、错误推测和有效退休等类别。

IPC Instructions Per Cycle, 每周期执行指令数。它必须和内存、分支、前后端瓶颈一起解释。

CPI Cycles Per Instruction, 每条指令平均周期数。它和 IPC 互为倒数视角, 但解释瓶颈仍要结合事件和代码。

CSV Comma-Separated Values, 逗号分隔文本表格, 常用于保存 benchmark 和计数器结果。

GFLOP/s 每秒十亿次浮点运算, 必须说明 FLOP 计数口径和输入形状。

NUMA 非统一内存访问结构, 不同核心访问不同内存节点的成本不同。

SMT Simultaneous Multithreading, 同时多线程, 同一物理核心运行多个硬件线程并共享部分资源。

COW Copy-On-Write, 写时复制, 先共享对象或页面, 写入时再复制。

RMW Read-Modify-Write, 读改写, 把读旧值、计算和写回合成一个原子更新序列。

TSO Total Store Order, x86 常见内存模型名。它比 C++ relaxed/acquire/release 更具体, 不能和语言内存序混为一谈。

TSAN ThreadSanitizer, 线程 sanitizer, 用来检测数据竞争和部分同步错误。

ASan AddressSanitizer, 发现越界、use-after-free 等内存错误。

UBSan UndefinedBehaviorSanitizer, 发现有符号溢出、非法移位、错误对齐等未定义行为。

sanitizer 编译器插桩诊断工具族。它们适合正确性检查, 不适合直接代表发布性能。

RAII Resource Acquisition Is Initialization, 用 C++ 对象生命周期管理锁、文件、线程句柄和缓冲区等资源。

CV Condition Variable, 条件变量, 用于线程等待某个条件变为真。

CFS Completely Fair Scheduler, Linux 默认调度器族中的公平调度核心思想。

IRIW Independent Reads of Independent Writes, 内存模型 litmus test, 用来观察多处理器可见性顺序。

ISR Interrupt Service Routine, 中断服务例程, 用来响应硬件或内核事件。

false sharing 多个线程写不同变量, 但变量落在同一 cache line 上, 导致一致性流量和扩展性下降。

内存序 原子操作对可见性和重排施加的约束, 例如 relaxed/acquire/release 和 `memory_order_seq_cst`。

CAS Compare-And-Swap, 比较并交换, 是很多无锁数据结构的基础原子操作。

ABA 无锁算法中的经典问题: 值从 A 变到 B 又变回 A, 比较值相同但对象历史已经改变。

lock-free 一类进度保证, 表示系统整体能持续前进, 但不保证每个线程有限步完成。

SPSC/MPSC SPSC 是单生产者单消费者队列，MPSC 是多生产者单消费者队列。

SPMC/MPMC SPMC 是单生产者多消费者队列，MPMC 是多生产者多消费者队列。

EAGAIN 常见系统调用错误码，表示资源暂时不可用，调用者应根据阻塞模式和状态机决定是否重试。

EINTR 系统调用被信号中断。可靠 I/O 循环要明确重试、取消和超时策略。

ETIMEDOUT 等待超时错误码，常见于 futex、条件变量或带超时的等待路径。

EOF End Of File，输入结束状态。它不是一个普通字节，读取循环要和短读、错误分开。

POSIX Portable Operating System Interface，Unix-like 系统常见接口规范。

SIGBUS Bus error signal，总线错误信号。mmap 文件被截断、映射访问越界或某些对齐/设备访问错误可能触发它。

epoll Linux I/O 事件通知机制，用文件描述符集合和事件标志驱动非阻塞 I/O。

eventfd/EFD Linux 事件计数文件描述符及其 flag 前缀，常用于唤醒 event loop 或跨线程通知。

EPOLLIN/EPOLLOUT epoll 可读和可写事件，分别推动读状态机和写状态机。

EPOLLONESHOT epoll 一次性触发模式，事件后需要重新 armed。

EPOLLRDHUP epoll 对端半关闭事件，连接状态机必须把它和普通可读、挂起、错误分开处理。

EPOLLET epoll 边沿触发模式，只在状态变化时通知。使用它时通常必须循环读/写到 EAGAIN。

EPOLLERR/EPOLLHUP epoll 错误和挂起事件。可靠连接状态机要显式读取错误、关闭资源并记录诊断。

SO_ERROR socket 选项，用于读取异步连接或 I/O 错误原因。

io_uring Linux 异步 I/O 接口，用提交队列和完成队列减少系统调用和拷贝成本。

SQ/CQ io_uring 的提交队列和完成队列。SQ 表示用户态提交请求的队列，CQ 表示内核写回完成结果的队列。

SQE/CQE io_uring 的提交队列项和完成队列项。SQE 表示请求已提交，CQE 表示底层完成有结果。

FIFO First In First Out，先进先出顺序，常用于队列和管道。

LIFO Last In First Out，后进先出顺序，典型结构是栈。

PID Process ID，进程编号，用于定位运行实例和系统资源。

TID Thread ID，线程编号，用于定位线程调度、亲和性和诊断信息。

RCU Read-Copy-Update，读路径尽量无锁、更新路径复制新版本并延迟回收旧版本的同步机制。

SLAB/SLUB Linux 内核小对象分配器族，用对象缓存降低频繁分配释放成本。SLUB 是现代 Linux 常见实现。

TLS Thread-Local Storage，线程局部存储，每个线程有独立变量实例。

背压 下游处理不过来时向上游施加的流量控制机制。

DMA/RDMA DMA 是设备直接访问内存；RDMA 是远端机器通过网卡直接读写内存区域。

UPI Ultra Path Interconnect，Intel 多 socket 系统中连接 CPU socket 的互连之一，会影响跨 socket 通信和远端内存访问成本。

RPC Remote Procedure Call，远程过程调用，跨网络和失败边界执行看似本地的调用。

RTT Round-Trip Time，往返时间，请求从发出到收到响应的端到端网络时间。

TCP Transmission Control Protocol，可靠字节流协议。它保证有序字节流，不自动提供业务消息边界。

WAL Write-Ahead Log，预写日志。先持久化意图或状态转移，再发布结果，用于崩溃恢复。

NFS Network File System, 网络文件系统。可靠写、rename 和缓存语义需要按具体环境验证。

DAG Directed Acyclic Graph, 有向无环图, 常用于表达任务依赖。

BFS Breadth-First Search, 广度优先搜索。第二册图 kernel 中常用 frontier 表达当前层。

CSR Compressed Sparse Row, 压缩稀疏行格式, 常用于图和稀疏矩阵的连续存储。

GEMM General Matrix-Matrix Multiplication, 矩阵乘矩阵, 通常具有较强数据复用。

GEMV General Matrix-Vector Multiplication, 矩阵乘向量, 常更受权重读流和内存带宽影响。

SLO Service Level Objective, 服务等级目标, 例如 p95 延迟或错误率阈值。

P95/P99 95/99 分位数指标, 用来描述尾延迟或尾部资源占用。它比平均值更能暴露排队和抖动。

p50 50 分位数, 也就是中位数。它适合描述典型请求, 但不能覆盖尾延迟风险。

RSS Resident Set Size, 常驻内存大小, 表示进程当前驻留在物理内存中的页面量。

GC Garbage Collection, 垃圾回收。即便 C++ 主项目不用 GC, 分布式系统对比中也会遇到 GC 暂停和内存回收成本。

HDFS Hadoop Distributed File System, Hadoop 生态的分布式文件系统。

LSM Log-Structured Merge Tree, 日志结构合并树, 存储引擎常用结构, 写入先进入内存和日志, 再分层合并到磁盘文件。

BLIS 一套高性能 BLAS 风格线性代数库, 常用于阅读 blocking、packing、microkernel 和 ISA dispatch 的工程组织。

MR/NR GEMM microkernel 常见 tile 维度名, 通常表示一个内核一次处理的 M/N 方向寄存器块大小。

XNNPACK 面向神经网络推理的高性能 CPU kernel 库, 常用于阅读线程池、数据布局 and 平台特化 kernel。

LMAX 常指 LMAX Disruptor 设计案例, 用序号环和发布协议组织低延迟队列。

oneTBB Intel oneAPI Threading Building Blocks, C++ 并行运行时/算法库, 常用于对照任务调度、work stealing 和线程池工程边界。

EEVDF Earliest Eligible Virtual Deadline First, Linux 调度器中的虚拟截止期选择思想, 用于解释任务选择和延迟公平性。

futex Fast userspace mutex, Linux 用户态同步原语和内核等待队列之间的桥。锁快路径留在用户态, 竞争时进入内核等待或唤醒。

FUTEX_WAIT/FUTEX_WAKE futex 的等待和唤醒操作。WAIT 检查内存值并睡眠, WAKE 唤醒等待者。

实验常量 实验或数据结构参数名, 例如 `SUM_LANE_COUNT`、`CACHE_LINE_BYTES`、`EVENT_BUCKET_COUNT`。它们用名字固定实验口径, 不是新算法。

OOM Out Of Memory, 内存不足。系统设计中要明确谁回收资源、失败是否可恢复。

CSV/JSON CSV 适合保存表格实验数据; JSON 适合保存结构化配置、manifest 或报告。

DWORD double word, x86/Intel 风格汇编文本中常表示 32 位内存操作数。

KV Key/Value, 键值状态或键值存储的简写。AI 章节中的 KV cache 另有更具体含义。

CAP Consistency、Availability、Partition tolerance 的分布式系统权衡框架。

Raft 一类易理解的共识协议, 用 leader、log、term 和 commit index 组织复制日志。

Paxos 共识协议族, 核心是多数派接受和提案编号, 常作为理解共识边界的理论入

□。

quorum 法定票数集合，常见实现是多数派，用于避免两个互斥事实同时提交。

leader term 领导者任期编号，用于区分新旧 leader，防止旧 leader 继续提交。

fencing 用 term、epoch、lease version 等 token 拒绝旧持有者写外部事实。

protocol version 协议版本号，用于让新旧节点判断消息格式和语义是否兼容。

magic 协议帧中的固定识别字节，用来快速发现误读、串流错位或错误文件。

max frame 协议允许的最大单帧大小，用于限制内存占用和恶意输入风险。

trace 代号 教学日志或表格里的短代号，例如 **T2/A1**，通常表示 task 2 的第 1 次 attempt。

共识 分布式系统中多个节点在故障下对日志顺序或状态达成一致的协议问题。

索引

AoS, 174
AoSoA, 175
attempt, 18

Compute Systems Engine, iii, 2
CSR, 180

D-cache, 7

false sharing, 78

I-cache, 7
instruction
 add, 116, 349
 add [mem], imm, 75
 add eax, ..., 57
 add eax, r8d, 57
 add qword ptr [rdi], 1, 104
 call, 34
 cmp, 34, 35, 41, 42, 46, 57, 65, 346
 cmpxchg, 287, 347
 dmb, 377
 imul, 57
 imul r8d, dword ptr [rsi + rcx*4], 57
 ja, 41
 jcc, 41, 46, 65
 je, 35
 jne, 35, 42, 64, 287
 jne .Lfailed, 347
 jne .Lloop, 35, 41
 ldar, 377
 lea, 57
 lock, 104, 287, 349, 614
 lock add, 64, 104, 118
 lock cmpxchg, 46, 287
 lock xadd, 46, 118, 346, 347
 mov, 46, 57, 64, 73, 75, 116, 346, 347, 377
 mov [mem], reg, 75
 mov eax, 0, 57
 mov rdi, qword ptr [rdi + 8], 56
 movzx, 35
 movzx edx, byte ptr [rdi + rcx], 35
 ret, 34, 221
 setcc, 65
 stlr, 377
 test, 34
 xadd, 349
 xor eax, eax, 57

manifest, 18, 91, 231

oracle, 4, 132, 229

partial, 5
pipeline breaker, 230

reference, 3, 131, 227, 229
register
 eax, 57, 73, 75, 287, 347
 rax, 14, 46, 53
 rcx, 35, 46, 56, 57, 73, 75, 142
 rdi, 35, 56, 73, 75, 142
 rip, 34
Roofline, 137

selection vector, 177
SoA, 174
split, 4

TLB, 7

内存级并行度, 57
内核, 227
冷路径, 5
分区, 417
合同, 228
唤醒来源, 283
完成, 480
工作集, 79
归约, 417

扫描, 417
提交, 480
提交业务结果, 480
材料化, 230
流水线, 230

热路径, 5
状态位置, 283
状态转移, 283
等待对象, 283

算术强度, 137

退休指令, 142

背压, 17